



PUBLIC

SAP Crystal Reports 2025

Document Version: 2025 – 2025-03-12

SAP Crystal Reports 2025 User Guide

Content

1	Document History.	46
2	Introduction to SAP Crystal Reports 2025.	47
2.1	About Crystal Reports.	47
2.2	About the Crystal Reports documentation.	48
	Locate information quickly.	48
	Sample Reports.	48
3	Quick Start.	49
3.1	Learning how to use Crystal Reports.	49
	Sample data - Xtreme.mdb.	49
3.2	Report Creation Wizards.	49
	Standard.	50
	Cross-Tab.	50
	Mailing Label.	50
	OLAP.	50
3.3	Quick start for new users.	51
	Before you begin.	51
	Creating the report.	52
	Record Selection.	65
	Grouping and sorting.	67
	Completing the report.	70
3.4	Quick start for advanced users.	72
	Choosing a report type and data source.	72
	Working with report elements on the Design tab.	73
	Using other reporting features.	74
4	Report Design Concepts.	77
4.1	Basic report design.	77
4.2	Deciding on the content of the report.	77
	Stating the purpose.	77
	Determining the layout of the report.	78
	Finding the data.	79
	Manipulating the data.	80
	Determining printing area characteristics.	81
4.3	Developing a prototype on paper.	83
5	Introduction to Reporting.	85

5.1	Report creation options.	85
5.2	Choosing data sources and database fields.	86
	The Database Expert.	86
	The Field Explorer.	88
5.3	About the report design environment.	90
	Design tab.	90
	Preview tab.	93
	HTML Preview tab.	97
5.4	Creating a new report.	98
	Selecting the data source.	98
	Adding tables.	99
	Linking multiple tables.	99
	Placing data on the report.	101
	BLOB (Binary Large Object) fields.	108
	Formatting data.	109
	Record selection.	109
	Grouping, sorting, and summarizing data.	109
	Using the drill-down option on summarized data.	110
	Using the zoom feature.	111
	Inserting page headers and footers.	111
	Adding a title page to the report.	112
	Adding summary information to the report.	113
5.5	Exploring reports and working with multiple reports.	113
	The Report Explorer.	114
	The Workbench.	115
	The Dependency Checker.	116
	Opening and docking explorers.	117
5.6	Beyond basic reports.	118
6	SAP BusinessObjects Enterprise Repository.	120
6.1	What is the SAP BusinessObjects Enterprise Repository?.	120
	Work flow.	121
6.2	Accessing the SAP BusinessObjects Enterprise Repository.	121
	To open the SAP BusinessObjects Enterprise Repository.	122
6.3	Repository Explorer toolbar.	123
6.4	Adding subfolders and subcategories to the repository.	123
	To add a new subfolder or subcategory.	124
6.5	Adding items to the repository.	124
	Adding a text object or bitmap image.	124
	Adding a text object or bitmap image - another method.	125
	Adding a custom function.	126
	Adding a command.	126

6.6	Using repository objects in reports	127
	Adding a text object or a bitmap image to a report.	127
	Adding a custom function to a report.	128
	Adding a command to a report.	128
	Adding a list of values to a parameter.	129
6.7	Modifying objects in the repository.	129
	To modify a repository object.	130
6.8	Updating connected repository objects in reports.	130
	To set the update option.	131
6.9	Deleting items from the repository.	131
	To delete an object from the repository.	132
6.10	Using Undo in the repository.	132
7	Designing Optimized Web Reports.	133
7.1	Overview.	133
	Key strategies for optimizing web reports.	134
7.2	Scale with SAP BusinessObjects Business Intelligence platform	134
	Evaluation times for date functions in SAP BusinessObjects Business Intelligence platform	135
7.3	Making the right design choices.	136
	Using faster report formats.	136
	Choosing between live and saved data.	136
	Designing summary reports.	138
	Using subreports carefully.	139
	Using other design elements effectively.	140
	Designing reports to maximize data sharing.	140
7.4	Streamlining your reporting environment.	141
	Selecting the fastest database and connection.	141
	Using table indexes.	141
	Improving table-linking choices.	142
	Using thread-safe database drivers.	142
	Using stored procedures for faster processing.	143
7.5	Using enhanced record selection formulas	144
	Pushing down record selection—an example.	144
	Record selection performance tips.	145
	Strategies for writing efficient record selection formulas.	146
	Incorporating parameter fields into record selection formulas.	149
	Using SQL expressions where appropriate.	150
7.6	Improving grouping, sorting, and totaling.	150
	Performing grouping on server.	151
	Benefits of grouping on server - an example.	151
	Using SQL Expressions for groups, sorts, and totals.	152

	Using SQL Expressions for Case Logic.	153
	Inserting summary and running total fields where possible.	153
7.7	Formatting reports for viewing on different platforms.	154
8	Record Selection.	155
8.1	Selecting records.	155
	Options for selecting records.	155
	Determining which field(s) to use.	156
	Using the Select Expert.	156
	Using formulas.	158
	Interaction of the Select Expert and the Formula Editor.	159
	Saved data selection formulas.	160
8.2	Using formula templates.	160
	Record selection formula templates.	160
8.3	Pushing down record selection to the database server.	163
8.4	Troubleshooting record selection formulas.	163
	To troubleshoot record selection formulas.	164
	Correcting selections that do not generate data.	165
9	Sorting, Grouping, and Totaling.	167
9.1	Sorting data.	167
	Understanding sort options.	167
	Sorting single and multiple fields.	169
	Sort controls.	170
9.2	Grouping data.	173
	Grouping data.	173
	Creating custom groups.	174
	Sorting groups conditionally.	175
	Sorting records within groups.	177
	Group selection.	178
	Grouping data in intervals.	181
	Grouping by the first letter of a company name.	184
	Grouping data hierarchically.	185
	Editing groups.	191
9.3	Summarizing grouped data.	192
	To summarize grouped data.	192
	Ordering groups by summarized values.	193
	Selecting top or bottom N groups or percentages.	194
	Selecting top or bottom groups or percentages conditionally.	196
9.4	Subtotaling.	197
	Subtotaling data.	198
	Extending prices and subtotaling the extensions.	198

9.5	Percentages.	200
	Calculating a percentage.	200
9.6	Group headers.	202
	Creating group headers.	202
	Suppressing group headers.	206
	Drilling-down on group headers.	206
10	Running Totals.	208
10.1	Understanding running totals.	208
	How running totals work.	208
10.2	Creating running totals.	209
	Creating running totals in a list.	209
	Creating running totals for a group.	210
	Creating conditional running totals.	212
	Creating running totals in a one-to-many linking relationship.	213
10.3	Creating running totals using a formula.	215
	To create running totals using a formula.	215
11	Multiple Section Reports.	217
11.1	About sections.	217
11.2	Working with sections.	218
	Inserting a section.	218
	Deleting a section.	218
	Moving a section.	219
	Merging two related sections.	220
11.3	Splitting and resizing sections.	221
	Splitting a section.	221
	Resizing a section.	222
11.4	Using multiple sections in reports	222
	Keeping variable length objects from overwriting each other.	223
	Eliminating blank lines when fields are empty.	224
	Adding blank lines conditionally.	225
11.5	Form letters.	225
	Working with text objects.	226
	Creating a form letter using a text object.	227
	Printing conditional messages in form letters.	233
12	Formatting.	235
12.1	Formatting concepts.	235
12.2	Using a template.	236
	Applying a template.	236
	Removing an applied template.	237

	Reapplying the last template selected.	238
	Using Template Field Objects.	238
	Template considerations.	239
12.3	Using the Report Design Environment.	241
	Section characteristics.	241
	Making an object underlay a following section.	242
	Pre-printed forms.	244
	Multiple columns.	244
	Hiding report sections.	245
	Hiding report objects.	245
	Placing text-based objects.	248
	Placing multi-line, text-based objects.	252
	Importing text-based objects from a file.	253
	Spacing between text-based objects.	253
	Deleting a field	260
	Balancing field spacing.	261
	To allow for overflow field representation.	262
	Selecting multiple objects.	264
	Vertical placement.	264
	Inserting character and line spacing.	265
	Setting fractional font sizes.	266
	Setting page size and page orientation.	266
	Setting page margins.	267
	TrueType fonts.	268
	Printer drivers.	269
	Formatting a report for web viewing.	270
12.4	Formatting properties.	270
12.5	Working with absolute formatting.	271
	Adding borders, color, and shading to a field.	271
	Making a report, section, area, or object read-only.	271
	Locking an object's size and position.	273
	Changing your default field formats.	273
	Adding and editing lines.	276
	Adding and editing boxes.	277
	Expected behavior of line and box formatting.	278
	Adding shapes to a report.	280
	Scaling, cropping, and sizing objects.	280
	Using conventional accounting formats.	282
	Repeating report objects on horizontal pages.	283
	Using white space between rows.	284
12.6	Working with conditional formatting.	287

	Conditional on or off properties.	288
	Conditional attribute properties.	288
	Changing margins conditionally.	291
	Changing X position conditionally.	292
	Creating footers after the first page.	293
	Using the Highlighting Expert.	294
	Undo/Redo activities.	297
12.7	Using the Format Painter.	298
	To copy and apply formatting.	299
12.8	Working with barcodes.	300
	Adding a barcode.	300
	Changing the appearance of a barcode.	301
	Removing a barcode.	302
13	Charting.	303
13.1	Charting concepts.	303
	Charting overview.	303
	Chart layouts.	304
	Chart types.	305
	Where to place a chart.	308
	Drill-down with charts.	309
	Drill-down with legends.	309
13.2	Creating charts.	309
	Charting on details or formula fields (Advanced layout).	309
	Charting on summary or subtotal fields (Group layout).	310
	Charting on Cross-Tab summaries (Cross-Tab layout).	311
	Charting on an OLAP cube (OLAP layout).	312
13.3	Working with charts.	314
	Editing charts using the Chart Expert.	314
	Editing charts using the Chart Options menu items.	314
	Editing charts using other menu items.	315
	Using the zooming features with bar and line charts.	316
	Auto-arranging charts.	317
	Formatting charts.	317
	Using the underlay feature with charts.	319
14	Mapping.	320
14.1	Mapping concepts.	320
	Mapping overview.	320
	Map layouts.	321
	Map types.	322
	Where to place a map.	323

	Drill-down with maps.	324
14.2	Creating maps.	324
	Mapping on group fields (Group layout).	325
	Mapping on Cross-Tab summaries (Cross-Tab layout).	326
	Mapping on an OLAP cube (OLAP layout).	327
14.3	Working with maps.	328
	Editing maps using the Map Expert.	328
	Changing the map title.	329
	Changing the map type.	329
	Changing map layers.	329
	Resolving data mismatches.	331
	Changing the geographic map.	331
	Zooming in and out on a map.	332
	Panning a map.	333
	Centering a map.	333
15	OLE.	335
15.1	OLE overview.	335
	OLE terminology.	335
	Types of OLE objects.	336
	General OLE considerations.	336
15.2	Inserting OLE objects into reports.	337
	To copy and paste OLE objects.	337
15.3	How OLE objects are represented in a report.	338
15.4	Editing OLE objects in reports.	338
	In-place editing.	338
	Dynamic OLE menu commands.	339
	OLE and the Insert Picture command.	339
15.5	Working with static OLE objects.	339
	Inserting a static OLE object.	340
	Making a static OLE object dynamic.	340
	Converting a static OLE object to a bitmap image object.	342
15.6	Working with embedded vs. linked objects.	343
	Embedded objects.	343
	Linked objects.	344
16	Cross-Tab Objects.	346
16.1	What is a Cross-Tab object?.	346
16.2	Cross-Tab example.	347
	Report of order data - no sorting/grouping.	348
	Report of order data - grouped by region.	348
	Report of order data - grouped by product.	349

	Report of order data - grouped by region and product.	350
	Order data in a Cross-Tab object.	351
16.3	Creating a Cross-Tab report.	352
	To create a new cross-tab report.	353
	To add a Cross-Tab to an existing report.	353
	Specifying the data source.	354
	Modifying the links.	355
	Adding a chart.	355
	Selecting records.	356
	Defining the structure of the Cross-Tab.	356
	Applying a predefined style and finishing the report.	357
	Adding a Cross-Tab to an existing report using the Cross-Tab Expert.	358
16.4	Working with Cross-Tabs.	360
	Showing values as percentages.	360
	Abbreviating large summarized fields.	361
	Customizing row/column labels.	362
	Using running totals in Cross-Tabs.	363
	Printing Cross-Tabs that span multiple pages.	364
16.5	Formatting Cross-Tabs.	365
	Changing width, height, and alignment of Cross-Tab cells.	365
	Formatting background color of entire rows/columns.	366
	Formatting fields individually.	366
	Formatting several fields at one time.	366
	Suppressing Cross-Tab data.	367
	Displaying summarized fields horizontally.	368
16.6	Advanced Cross-Tab features.	369
	Calculated Members.	369
	Embedded Summaries.	373
17	Reporting on Universes and managed connections.	376
17.1	Prerequisite: Install Microsoft WebView2.	376
17.2	Connecting to a universe.	376
	To connect to a universe.	377
17.3	Connecting to a managed OLAP connection.	377
17.4	Defining the data selection for a query.	378
	To build a simple query.	378
	To sort objects.	379
	To create a combined query.	379
	Quick reference to objects.	380
17.5	Editing an existing query.	381
	To edit an existing query.	381
17.6	Viewing the SQL behind a query.	382

	To view the SQL when you create a query.	382
	To view the SQL after you have created a query.	382
17.7	Query filters and prompts.	382
	Creating query filters.	383
	Building prompts.	384
	Combining query filters and prompts.	385
	Using And or Or to combine query filters.	385
	Quick reference to query filter operators.	386
	Editing and removing query filters.	387
17.8	Filtering data using subqueries and database ranking.	388
	What is a subquery?.	388
	Building a subquery.	389
	What is a database ranking?.	391
	Creating a database ranking.	391
18	Creating and Updating OLAP Reports.	394
18.1	OLAP reporting with Crystal Reports.	394
	OLAP grid objects.	395
18.2	Creating an OLAP report.	395
	To create an OLAP report.	395
	Specifying the data source.	396
	Defining the structure of the grid.	396
	Setting sliced dimensions and specifying the number of grids.	398
	Applying a predefined style.	400
	Inserting a chart.	401
18.3	Updating an OLAP report.	401
	To update a cube location in an OLAP grid object.	402
18.4	Formatting data in an OLAP grid.	403
	Changing the background color of a dimension.	403
	Creating an alias for a dimension.	404
	Formatting grid lines.	405
	Labeling dimensions.	405
18.5	Changing the view of OLAP data.	406
	To show or hide dimension members	406
	To create asymmetry in an OLAP grid.	406
	To add totals to an OLAP grid.	407
	To change the display format for member names.	407
	To alter the data displayed in the OLAP grid.	408
	To define the order of fields in the OLAP grid	408
18.6	Sorting and filtering OLAP grid data.	408
	Sorting data in an OLAP grid.	408
	Filtering data in an OLAP grid.	410

18.7	Adding calculations to OLAP grids.	411
19	Using the OLAP Worksheet.	412
19.1	Sample Reports.	412
19.2	Changing the view of your data.	412
	Swapping dimensions.	412
	Stacking dimensions.	413
	Selecting row and column members.	415
	Changing the slice of data.	424
	Reordering members.	425
	Changing member captions.	426
	Drilling down.	427
	Drilling through.	429
	Using asymmetry in your view.	430
19.3	Changing the formatting.	434
	Formatting data in the Worksheet.	434
	Resizing columns.	436
	Resizing rows.	436
	Viewing cell and member properties.	437
19.4	Highlighting exceptions.	437
	Overview.	438
	Applying global highlighting.	438
	Highlighting high or low absolute values.	439
	Highlighting exceptions compared with other members.	440
	Highlighting exceptions based on a calculation result.	441
	Editing highlighting.	441
	Removing highlighting.	442
	Using other formats to highlight exceptions.	442
19.5	Filtering and sorting data.	443
	Filtering data.	444
	Sorting data.	448
19.6	Adding calculated members.	450
	Adding Automatic Totals to rows and columns.	450
	Adding calculated members to the Worksheet.	451
	Adding data analysis calculations.	456
	Using calculation definitions.	457
19.7	OLAP Syntax.	461
	Introducing the OLAP Syntax.	461
	Dimension functions.	463
	Level functions.	465
	Logical functions and operators.	468
	Member functions.	488

	Numeric functions.	504
	Statistical functions.	536
	Set functions.	550
	String functions.	606
	Tuple functions.	609
	Operators.	612
	Sample Dimension.	614
20	Printing, Exporting, and Viewing Reports.	615
20.1	Distributing reports.	615
	Printing a report.	615
	Faxing a report.	616
	Exporting a report.	617
	Working with Enterprise folders.	628
20.2	Viewing reports.	631
	Report Parts and other Report objects.	632
	Creating an Enterprise Hyperlink.	643
	Using Smart View to view a report in the CMC.	644
20.3	Using smart tags.	645
	To use smart tags with a Crystal Reports object.	645
21	Publishing.	647
21.1	Publishing overview.	647
21.2	Role of the report designer in publishing.	647
21.3	Dynamic recipient sources (DRSs).	648
	Design considerations.	649
	Creating a dynamic recipient source.	652
	Active Directory recipients.	652
21.4	Delivery rules.	653
	Recipient delivery rules.	654
	Global delivery rules.	655
22	Report Alerts.	657
22.1	About Report Alerts.	657
22.2	Working with Report Alerts.	657
	Creating Report Alerts.	657
	Editing Report Alerts.	660
	Deleting Report Alerts.	660
	Viewing Report Alerts.	660
	Referring to Report Alerts in formulas.	662
23	XML.	663
23.1	XML overview	663

	XML elements, tags, and structure.	664
	XML Schema.	664
	Document Type Definition.	664
	Style sheets.	665
23.2	XML in Crystal Reports.	665
23.3	Accessing XML data.	665
	Native XML driver.	665
23.4	Exporting to XML format.	675
	To export a report to XML.	675
	To export a report to XML (Legacy).	676
23.5	Managing XML exporting formats.	676
	To Import an XML Transform.	677
23.6	Customizing XML report definitions.	677
	Why create your own XML format?.	678
	Customizing XML tags.	678
23.7	Legacy XML output specifications.	683
	Crystal ML Schema.	684
	Custom Format.	685
24	Using Formulas.	687
24.1	Formulas overview.	687
	Typical uses for formulas.	687
24.2	Formula components and syntax.	689
	Formula components.	689
	Formula syntax.	690
24.3	User Function Libraries in formulas.	691
24.4	Specifying formulas.	692
	Working with the Formula Workshop.	693
	Working with the Formula Editor.	697
24.5	Creating and modifying formulas.	700
	To create a formula and insert it into a report.	700
	Creating a formula in the Formula Expert.	701
	Editing formulas.	703
	Searching for formula text.	703
	Copying formulas.	704
24.6	Deleting formulas.	706
	Removing the working formula from your report.	707
	Deleting the formula specification.	707
24.7	Debugging formulas.	708
	Debugging evaluation time errors.	708
	Debugging tutorial.	708
24.8	Error Messages and Formula Compiler Warnings.	713

25	Using Custom Functions.	731
25.1	Online support, samples, and technical briefs.	731
25.2	Sample Reports.	731
25.3	What are custom functions?	731
	Advantages.	732
	Sample work flow.	732
25.4	Working with custom functions.	733
	Creating custom functions.	733
	Viewing and modifying custom functions.	735
	Defining default argument values for custom functions.	736
	Optional Values.	737
	Calling custom functions.	737
	Adding custom functions to the SAP BusinessObjects Enterprise Repository.	739
	Adding custom functions from the SAP BusinessObjects Enterprise Repository.	740
	Refreshing a report's repository custom functions.	740
25.5	Custom function syntax.	741
	Basic syntax for custom functions.	741
	Crystal syntax for custom functions.	743
	Custom function syntax rules.	744
	Evaluation time.	745
25.6	Custom function examples.	745
	Example of a custom function that returns the sum of two numbers.	746
	Example that capitalizes a string using an optional argument.	746
	Example of a function that returns an array.	747
	Example of a function that has only optional arguments.	748
26	Creating Formulas with Basic Syntax.	750
26.1	Basic syntax overview	750
	Tips on learning Basic syntax.	750
26.2	Basic syntax fundamentals.	752
	The result of a formula.	752
26.3	Comments (Basic syntax).	755
26.4	Fields (Basic syntax)	755
	How fields appear in formulas.	756
	Sample formulas using fields.	756
26.5	Statements (Basic syntax).	756
26.6	Assignment (Basic syntax)	757
26.7	Simple data types (Basic syntax)	757
	Number (Basic syntax).	757
	Currency (Basic syntax).	758
	String (Basic syntax).	758
	Boolean (Basic syntax).	758

	Date, Time, and DateTime (Basic syntax)	759
26.8	Range data types (Basic syntax)	760
	Using ranges in formulas (Basic syntax)	760
26.9	Array data types (Basic syntax)	761
26.10	Variables (Basic syntax)	762
	Variable overview	762
	Variable declarations using Dim (Basic syntax)	763
	Variable Scope (Basic syntax)	764
	Declaring array variables (Basic syntax)	767
	Default values for the simple types (Basic syntax)	769
	Automatic type conversions (Basic syntax)	770
26.11	Functions (Basic syntax)	771
	Functions similar to Visual Basic functions of the same name (Basic syntax)	771
	Summary functions (Basic syntax)	772
	Date ranges (Basic syntax)	772
	Array functions (Basic syntax)	773
	Evaluation Time functions (Basic syntax)	773
	Print State functions (Basic syntax)	774
	Document Properties functions (Basic syntax)	774
	Additional functions (Basic syntax)	775
	Conditional formatting functions (Basic syntax)	775
	General purpose conditional formatting functions (Basic syntax)	776
26.12	Operators (Basic syntax)	777
	Arithmetic operators	777
	Comparison operators (Basic syntax)	778
	Boolean operators (Basic syntax)	778
	Null fields and how to use IsNull (Basic syntax)	779
26.13	Control structures (Basic syntax)	780
	If statements (Basic syntax)	780
	Select statements (Basic syntax)	783
	For/Next loops (Basic syntax)	784
	Do Loops (Basic syntax)	787
	While loops (Basic syntax)	788
	Safety mechanism for loops (Basic syntax)	788
	Option Loop (Basic Syntax)	789
26.14	Limitations (Basic syntax)	790
27	Creating Formulas with Crystal Syntax	791
27.1	Crystal syntax overview	791
27.2	Crystal syntax fundamentals	791
	The result of a formula	791
27.3	Comments (Crystal syntax)	792

27.4	Fields (Crystal syntax).	793
	How fields appear in formulas.	793
	Sample formulas using fields (Crystal syntax).	794
27.5	Expressions (Crystal syntax).	794
	Multiple expressions (Crystal syntax).	794
	How earlier expressions affect later expressions (Crystal syntax).	795
	Using the If expression (Crystal syntax).	796
27.6	Assignment (Crystal syntax).	797
27.7	Simple data types (Crystal syntax).	797
	Number.	797
	Currency (Crystal syntax).	798
	String (Crystal syntax).	798
	Boolean (Crystal syntax).	799
	Date, Time, and DateTime (Crystal syntax).	799
27.8	Range data types (Crystal syntax).	800
	Using ranges in formulas (Crystal syntax).	801
27.9	Array data types (Crystal syntax).	801
27.10	Variables (Crystal syntax).	802
	Variables overview.	802
	Variable declarations (Crystal syntax).	803
	Variable Scope (Crystal syntax).	804
	Declaring array variables (Crystal syntax).	807
	Default values for the simple types (Crystal syntax).	808
	Automatic type conversions (Crystal syntax).	809
27.11	Functions (Crystal syntax).	810
	Non reporting-specific functions (Crystal syntax).	811
	Summary functions (Crystal syntax).	811
	Date Ranges (Crystal syntax).	812
	Array functions (Crystal syntax).	812
	Evaluation Time functions (Crystal syntax).	813
	Print State functions (Crystal syntax).	813
	Document Properties functions.	813
	Additional functions (Crystal syntax).	814
	Conditional formatting functions (Crystal syntax).	814
	General purpose conditional formatting functions (Crystal syntax).	815
27.12	Operators (Crystal syntax).	816
	Arithmetic operators.	816
	Comparison operators (Crystal syntax).	817
	Boolean operators (Crystal syntax).	818
	Null fields and how to use IsNull (Crystal syntax).	818
27.13	Control structures (Crystal syntax).	819

	If expressions (Crystal syntax).	819
	Select expressions (Crystal syntax).	824
	For loops (Crystal syntax).	825
	While Loops (Crystal syntax).	827
	Safety mechanism for loops (Crystal syntax).	829
	Option Loop (Crystal Syntax).	829
27.14	Limitations (Crystal syntax).	830
28	Parameter Fields and Prompts.	831
28.1	Parameter and prompt overview.	831
	Parameter field considerations.	831
	Prompt considerations.	832
	Data and non-data parameters.	833
	Optional parameters.	833
28.2	Understanding dynamic prompts.	834
28.3	Understanding lists of values.	836
	List-of-values types.	837
	Determining which list-of-values type to use.	838
	Lists of values and prompt groups contrasted.	839
28.4	Creating a parameter with a static prompt.	839
	To create a parameter with a static prompt	840
	To incorporate the parameter into the record selection filter.	842
	To incorporate the parameter into a saved data selection formula.	843
28.5	Creating a parameter with a dynamic prompt.	844
	To create a parameter with a dynamic prompt.	844
28.6	Creating a parameter with a cascading list of values.	846
	To create a parameter with a cascading list of values.	846
28.7	The Parameter Panel.	848
28.8	Working with lists of values.	849
	Sharing common lists of values within a report.	850
	Using separate value and description fields.	851
	Using command objects as list-of-values data sources.	852
	Null handling.	852
	Long lists of values.	852
28.9	Best practices for prompting.	853
	Unmanaged reports.	854
	Managed reports.	854
	Converting unmanaged reports to managed reports.	855
	Deploying managed reports with dynamic prompts.	855
28.10	Deleting parameter fields.	855
	To delete a parameter that is not used in a formula.	856
	To delete a parameter used with the Select Expert.	856

	To delete a parameter that is used in a formula.	856
28.11	Responding to parameter field prompts.	857
	Previewing a report for the first time.	857
	Refreshing report data.	857
28.12	Advanced parameter features.	859
	Creating a parameter with multiple prompting values.	859
	Applying conditional formatting using parameter fields.	859
	Creating a report title using parameter fields.	860
	Specifying single or ranges of values.	861
	Incorporating a parameter into a formula.	862
	Defining sort order using parameter fields.	863
	Defining entry type and format using the Edit Mask.	864
	Creating a saved-data record filter using parameter fields.	865
	To create a parameter appearing on the Parameter Panel.	866
	To incorporate the parameter into a saved data selection formula.	867
	Adding dynamic grouping using parameter fields.	868
29	Subreports.	870
29.1	What are subreports?.	870
	Unlinked vs. linked subreports.	871
	How subreport linking works.	872
29.2	Inserting subreports.	873
	Previewing subreports.	874
	Saving a subreport as a primary report.	875
	Updating subreports.	875
29.3	Linking a subreport to the data in the primary report.	877
	To link a subreport to the data in the primary report.	877
29.4	Linking a subreport to the main report without modifying the selection formula.	878
	To link a subreport to a main report without modifying the selection formula.	878
29.5	Combining unrelated reports by using subreports.	879
	To combine two unrelated reports.	879
	To combine three or more unrelated reports.	879
29.6	Using subreports with unlinkable data.	880
	Linking to/from a formula field.	880
	Linking unindexed tables.	881
29.7	Creating an on-demand subreport.	882
	To create an on-demand subreport.	882
29.8	Adding captions to on-demand subreports.	882
	To add a caption.	883
29.9	Showing different views of the same data in a report.	883
30	Understanding Databases.	885

30.1	Databases overview.	885
	Relational database basics.	885
	Indexed tables.	887
30.2	Using SQL and SQL databases.	888
	What is SQL?.	889
	SQL DBMS.	891
	How does Crystal Reports use SQL?.	893
	The SQL language.	894
	Defining an SQL Command.	896
30.3	Changing data sources.	899
	To change the data source.	900
30.4	Working with aliases.	901
30.5	Linking tables.	902
	Link from and link to.	902
	Link relationships.	902
	Performance considerations in one-to-many links.	903
	Data file considerations.	905
	SQL database considerations.	906
	Performance considerations for all reports.	907
	The Database Expert Links tab.	909
	Linking indexed tables.	909
	Link processing order.	910
	Linking options.	911
30.6	Server-side processing.	921
	How server-side grouping affects the SQL query.	923
30.7	Mapping database fields.	924
	About the Map Fields dialog box.	924
	Remapping processes.	925
	Remapping altered database fields.	926
30.8	Saved Data Indexes.	927
	How Report Indexing works.	928
	Considerations for using Saved Data Indexes.	928
	Indexing the right field(s).	929
30.9	Unicode support in Crystal Reports.	929
30.10	Working with databases.	930
	Using Access queries.	930
	Using ODBC data sources.	933
	Using OData data sources.	935
30.11	Advanced database features.	938
	One-to-many links	938
	Reporting on an NT Event Log.	939

30.12	For additional information.	940
31	Integrating Business Applications with Crystal Reports.	941
31.1	Integrating with SAP.	941
	Introduction.	941
	Reporting with SAP.	947
31.2	Integrating with PeopleSoft.	1012
	Overview.	1012
	To access data in SAP Crystal Reports.	1013
32	Creating Accessible Reports.	1015
32.1	About accessibility.	1015
	Benefits of accessible reports.	1015
	About the accessibility guidelines.	1016
	Accessibility and Business Objects products.	1017
32.2	Improving report accessibility.	1017
	Placing objects in reports.	1018
	Text.	1019
	Color.	1023
	Navigation.	1024
	Parameter fields.	1025
32.3	Designing for flexibility.	1025
	To create an accessibility parameter field.	1026
	Accessibility and conditional formatting.	1026
	Accessibility and suppressing sections.	1027
	Accessibility and subreports.	1028
32.4	Improving data table accessibility.	1028
	Text objects and data table values.	1029
	Other data table design considerations.	1033
32.5	Accessibility and BusinessObjects Business Intelligence platform.	1034
	Setting accessible preferences for SAP BusinessObjects Business Intelligence platform.	1034
32.6	Accessibility and customization.	1035
32.7	Resources.	1036
33	Accessing Data Sources.	1038
33.1	Introduction.	1038
	Four types of data.	1038
33.2	Direct access database files.	1039
	Advantages.	1039
	Disadvantages.	1039
	Three layers.	1039
	Common database formats.	1040

33.3	ODBC data sources.	1045
	Advantages.	1045
	Disadvantages.	1046
	Five layers.	1046
	Installing database clients.	1048
	Common ODBC database formats.	1049
33.4	JDBC data sources	1051
	JNDI.	1052
	Five layers.	1052
	Supported JDBC database formats.	1053
	Configuring the Crystal Reports JDBC driver.	1054
33.5	Business Objects universes.	1054
33.6	Managed OLAP connections.	1055
33.7	Business Views.	1055
34	Report Processing Model.	1057
34.1	Overview.	1057
	What is a "pass"?	1057
	Pre-pass 1.	1057
	Pass 1.	1057
	Pre-pass 2.	1058
	Pass 2.	1058
	Pass 3.	1059
35	Crystal Reports Error Messages.	1060
35.1	Drive:\filename.extension.	1060
35.2	Drive:\test.rpt.	1060
35.3	Failed to load database connector.	1061
35.4	Failed to open the connection. Details: [Database Vendor Code: <vendor code number>].	1061
35.5	Failed to retrieve data from the database. Details: [Database Vendor Code: <vendor code number>].	1062
36	CRConfig.xml Tag Reference.	1064
36.1	Parameter reference.	1064
37	Command Reference.	1071
37.1	File menu.	1071
	New submenu.	1071
	Open command.	1072
	Close command.	1073
	Save command.	1073
	Save As command.	1073
	Save Data with Report command.	1073

	Save Subreport As command	1074
	Log On or Off Server command.	1074
	Print command.	1074
	Page Setup command.	1075
	Export submenu.	1075
	Send To submenu.	1076
	Options command.	1077
	Report Options command.	1077
	Summary Info command.	1077
	Recently Opened Reports command.	1078
	Exit command.	1078
37.2	SAP menu.	1078
37.3	Edit menu.	1079
	Undo command.	1079
	Redo command.	1080
	Cut command.	1080
	Copy command.	1080
	Paste command.	1080
	Paste Special command.	1081
	Delete command.	1081
	Select All command.	1081
	Find command.	1081
	Go To Page command.	1082
	Edit Report Object command	1082
	Subreport Links command.	1082
	Object command.	1083
	Links command.	1085
37.4	View menu.	1086
	Design command.	1086
	Preview command.	1086
	Other Views submenu.	1086
	Print Preview command.	1086
	Preview Sample command.	1087
	HTML Preview command.	1087
	Close Current View command.	1087
	Field Explorer command	1087
	Report Explorer command.	1088
	Repository Explorer command.	1088
	Dependency Checker command.	1088
	Workbench command.	1089
	Toolbars command.	1089

	Status Bar command.	1090
	Preview Panel command.	1090
	Zoom command.	1090
	Rulers submenu.	1090
	Guidelines submenu.	1091
	Grid submenu.	1092
	Tooltips submenu.	1093
	Product Locale command.	1093
	Preferred Viewing Locale command.	1093
37.5	Insert menu.	1094
	Text Object command.	1094
	Summary command.	1094
	Field Heading command.	1095
	Group command.	1095
	OLAP Grid command.	1096
	Cross-Tab command.	1096
	Subreport command.	1096
	Line command.	1097
	Box command.	1098
	Picture command.	1098
	Chart command.	1099
	Map command.	1099
	Flash command.	1100
	OLE Object command.	1100
	Template Field Object command.	1100
37.6	Format menu.	1101
	Format command.	1101
	Format Painter command.	1102
	Hyperlink command.	1102
	Use Expert command.	1103
	Highlighting Expert command.	1103
	Text Formatting command.	1103
	Move Submenu.	1104
	Align submenu.	1105
	Make Same Size submenu.	1107
	Size and Position command.	1107
	Pivot OLAP Grid command.	1107
	Pivot Cross-Tab command.	1108
37.7	Database menu.	1108
	Database Expert command.	1108
	Set Datasource Location command.	1109

	Log On or Off Server command.	1109
	Browse Data command.	1109
	Set OLAP Cube Location command.	1110
	Verify Database command.	1110
	Show SQL Query command.	1111
	Perform Grouping On Server command.	1111
	Select Distinct Records command.	1111
	Query Panel command.	1112
37.8	Report menu.	1112
	Select Expert command.	1112
	Selection Formulas submenu.	1114
	Formula Workshop command.	1115
	Alerts submenu.	1115
	Report Bursting Indexes command.	1116
	Section Expert command.	1116
	Group Expert command.	1117
	Group Sort Expert command.	1117
	Record Sort Expert command.	1118
	Template Expert command.	1118
	OLAP Design Wizard command.	1119
	Hierarchical Grouping Options command.	1119
	Show Current Parameter Values command.	1119
	Check Dependencies command.	1119
	Refresh Report Data command.	1120
	Set Print Date and Time command.	1121
	Performance Information command.	1121
37.9	Chart menu.	1121
	Format Background command (Chart menu).	1121
	Chart Options command (Chart menu).	1121
	Axis Options command (Chart menu).	1121
	Series Options command (Chart menu).	1122
	3D Viewing Angle command (Chart menu).	1122
	Edit Axis Label command (Chart menu).	1122
	Trendlines command (Chart menu).	1122
	Auto-Arrange Chart command (Chart menu).	1123
	Apply Changes to All Charts command (Chart menu).	1123
	Discard Custom Changes command (Chart menu).	1123
	Load Template command (Chart menu).	1123
	Save as Template command (Chart menu).	1123
	Select Mode command (Chart menu).	1124
	Zoom In command (Chart menu).	1124

	Zoom Out command (Chart menu).	1124
	Pan command (Chart menu).	1124
37.10	Map menu.	1124
	Select Mode command (Map menu).	1125
	Zoom In command (Map menu).	1125
	Zoom Out command (Map menu).	1125
	Pan command (Map menu).	1125
	Center Map command (Map menu).	1125
	Title command (Map menu).	1126
	Type command (Map menu).	1126
	Layers command (Map menu).	1126
	Resolve Mismatch command (Map menu).	1126
	Map Navigator command (Map menu).	1127
37.11	Window menu.	1127
	Tile Vertically command.	1127
	Tile Horizontally command.	1127
	Cascade command.	1128
	Arrange Icons command.	1128
	Close All command.	1128
	Active Report command.	1128
37.12	Help menu.	1128
	Crystal Reports Help command.	1128
	Context Help command.	1129
	Show Start Page command.	1129
	Check for Updates on Start Up command.	1129
	Check for Updates command.	1129
	Register or Change Address command.	1130
	License Manager command.	1130
	About Crystal Reports command.	1130
37.13	Shortcut menus.	1130
	Add Calculated Member command.	1130
	Add Current Report command.	1131
	Add Filter command.	1131
	Add First Sort submenu.	1131
	Add to Repository command.	1132
	Arrange Lines command.	1132
	Bind Sort Control command.	1132
	Break Hierarchies command.	1133
	Calculated Member command.	1133
	Cancel Menu command.	1133
	Change Caption command.	1133

Change Direction of Sort command.	1133
Change Group command.	1134
Change to Barcode command.	1134
Collapse Member command.	1134
Column Grand Totals submenu.	1134
Column Options submenu.	1134
Copy Smart Tag command.	1135
Delete Group command.	1135
Delete Last Line command.	1136
Delete Section command.	1136
Disconnect from Repository command.	1136
Drill-down command.	1136
Duplicate command.	1136
Edit Filter command.	1137
Embedded Summary command.	1137
Expand Member command.	1137
Expert command.	1137
Find in Field Explorer command.	1137
Find in Formulas command.	1137
Find in Grouping condition command.	1138
Find in Sorting condition command.	1138
Find in Subreport links command.	1138
Fit Section command.	1138
Formatting command.	1138
Go to Hyperlink command.	1138
Grid Options submenu.	1139
Hide command.	1139
Hide (Drill-Down OK) command.	1139
Insert command.	1140
Insert Line command.	1140
Insert Section Below command.	1140
Merge Section Below command.	1140
Remove All Horizontal Guidelines command.	1140
Remove All Vertical Guidelines command.	1140
Remove Filter command.	1140
Remove Sort command.	1141
Reorder Displayed Members command.	1141
Row Grand Totals submenu.	1141
Row Options submenu.	1141
Select All Section Objects command.	1142
Set Prompt Order command.	1142

	Show Hidden Sections in Design command.	1143
	Show Section Names command.	1143
	Snap to Grid command.	1143
	Sort Fields Alphabetically command.	1143
	Sort Tables Alphabetically command.	1143
	Standard command.	1143
	Summarized Field Labels submenu.	1144
	Suppress (No Drill-Down) command.	1144
	View Cube command.	1145
37.14	Standard toolbar.	1145
37.15	Formatting toolbar.	1147
37.16	Insert Tools toolbar.	1148
37.17	Expert Tools toolbar	1149
37.18	Navigation Tools toolbar.	1150
37.19	SAP toolbar.	1151
37.20	External Command toolbar.	1151
38	Dialog Box Reference.	1153
38.1	Add a New XML Format dialog box.	1153
38.2	Add Command to Report dialog box.	1154
38.3	Add Custom Function to Repository dialog box.	1155
38.4	Add Item dialog box.	1155
38.5	Add or Update Object dialog box.	1156
38.6	Browse and Paste Data dialog box.	1156
38.7	Browse Data dialog box.	1157
38.8	Calculated Members dialog box.	1158
	Calculation Expert tab (Calculated Members dialog box).	1158
	Data Analysis Expert tab (Calculated Members dialog box).	1159
	Calculation tab (Calculated Members dialog box).	1161
38.9	Change Caption dialog box.	1162
38.10	Change Group Options dialog box.	1162
38.11	Change Icon dialog box.	1162
38.12	Change In Record Selection Formula dialog box.	1164
38.13	Change Map Title dialog box.	1164
38.14	Character-Separated Values dialog box.	1164
38.15	Chart Sort Order dialog box.	1166
38.16	Choose Export File dialog box.	1166
38.17	Choose Field dialog box.	1167
38.18	Choose Profile dialog box.	1168
38.19	Color dialog box.	1168
38.20	Command Parameter dialog box.	1168
38.21	Connection Properties dialog box.	1169

38.22	Convert dialog box.	1170
38.23	Create Alerts dialog box.	1171
38.24	Create Alert dialog box.	1172
38.25	Create New Parameter dialog box.	1173
	Parameter name area.	1173
	List of values area.	1174
	Options area.	1179
38.26	Create Parameter Field dialog box.	1184
	Options.	1185
38.27	Create Running Total Field dialog box.	1186
38.28	Cross-Tab Group Options dialog box.	1189
38.29	Current Parameter Values dialog box.	1189
38.30	Custom Function Name dialog box.	1189
38.31	Custom Function Properties dialog box.	1190
38.32	Custom Style dialog box.	1192
	Currency Symbol tab (Custom Style dialog box).	1192
	Number tab (Custom Style dialog box).	1194
	Date and Time tab (Custom Style dialog box)	1197
	Date tab (Custom Style dialog box)	1197
	Time tab (Custom Style dialog box)	1202
38.33	Customize Map dialog box.	1204
38.34	Data Explorer dialog box.	1205
38.35	Data source connectivity dialog boxes.	1206
	Access/Excel (DAO) dialog box.	1206
	ADO.NET (XML).	1207
	COM Connectivity dialog box.	1208
	Crystal Data Object dialog box.	1208
	DB2 Unicode dialog box.	1209
	Field Definitions Only dialog box.	1209
	File System Data dialog box.	1209
	Java Beans Connectivity dialog box.	1210
	JDBC (JNDI) dialog box.	1211
	ODBC (RDO) dialog box.	1212
	OLE DB (ADO) dialog box.	1214
	Oracle 8 Server dialog box.	1215
	XML dialog box.	1216
38.36	Default Values for dialog box.	1220
38.37	Define Description dialog box.	1221
38.38	Define Filter dialog box.	1221
38.39	Define Named Group dialog box.	1222
38.40	Dependency Checker dialog box.	1223

38.41	Display Properties dialog box.	1223
38.42	Document Properties dialog box.	1224
	Summary tab (Document Properties dialog box).	1224
	Statistics tab (Document Properties dialog box).	1225
38.43	Edit Parameter dialog box.	1227
38.44	Edit Parameter Field dialog box.	1227
38.45	Edit Property Value dialog box.	1227
38.46	Edit Running Total Field dialog box.	1227
38.47	Edit Summary dialog box.	1228
38.48	Embedded Summaries dialog box.	1228
38.49	Enter ODBC Table Name dialog box.	1229
38.50	Enter Parameter Values dialog box.	1230
38.51	Enter Search Name dialog box.	1231
38.52	Enter Value for dialog box.	1231
38.53	Excel Format Options dialog box.	1232
	Microsoft Excel.	1232
	Microsoft Excel - Data only.	1234
38.54	Export dialog box.	1236
38.55	Export Options dialog box.	1237
38.56	Export to Directory dialog box.	1238
38.57	Export to Text dialog box.	1238
38.58	Extract Custom Function from Formula dialog box.	1239
38.59	Field Explorer dialog box.	1241
	Database Fields (Field Explorer dialog box).	1241
	Formula Fields (Field Explorer dialog box).	1242
	SQL Expression Fields (Field Explorer dialog box).	1243
	Parameter Fields (Field Explorer dialog box).	1244
	Running Total Fields (Field Explorer dialog box).	1245
	Group Name Fields (Field Explorer dialog box).	1247
	Special Fields (Field Explorer dialog box).	1247
38.60	Find dialog box.	1252
38.61	Find dialog box (Formula Editor).	1253
38.62	Font dialog box.	1254
38.63	Format dialog box (OLAP Worksheet).	1255
38.64	Format Editor dialog box.	1256
	Boolean tab (Format Editor dialog box).	1256
	Border tab (Format Editor dialog box).	1257
	Common tab (Format Editor dialog box).	1258
	Number tab (Format Editor dialog box).	1262
	Date and Time tab (Format Editor dialog box).	1264
	Box tab (Format Editor dialog box).	1264

Line tab (Format Editor dialog box).	1266
Picture tab (Format Editor dialog box).	1268
Hyperlink tab (Format Editor dialog box).	1270
Rounding tab (Format Editor dialog box).	1272
Subreport tab (Format Editor dialog box).	1272
Font tab (various dialog boxes).	1274
Paragraph tab (various dialog boxes).	1275
Flash tab (Format Editor dialog box).	1277
38.65 Format Grid Lines dialog box.	1278
38.66 Format Other Dimensions dialog box.	1279
38.67 Formula Editor dialog box.	1280
38.68 Formula Name dialog box.	1282
38.69 Formula Workshop dialog box.	1282
38.70 Function Library dialog box.	1283
38.71 Go To dialog box.	1283
38.72 Group Selection Formula Editor dialog box.	1284
38.73 Help Text dialog box.	1285
38.74 Hierarchical Group Options dialog box.	1285
38.75 Highlight Exceptions dialog box.	1286
38.76 Index Legend dialog box.	1287
38.77 Insert Group dialog box.	1287
Common tab (various dialog boxes).	1288
Options tab (various dialog boxes).	1289
Specified Order tab (various dialog boxes).	1291
Others tab (various dialog boxes).	1292
38.78 Insert Object dialog box.	1293
38.79 Insert Object dialog box (Create from File version).	1294
38.80 Insert Subreport dialog box.	1295
Subreport tab (Insert Subreport dialog box).	1295
Link tab (Insert Subreport dialog box).	1297
38.81 Insert Summary dialog box.	1298
38.82 Label Aliasing dialog box.	1299
38.83 Layer Control dialog box.	1300
38.84 Link Options dialog box.	1301
38.85 Links dialog box.	1302
38.86 Log On to SAP BusinessObjects Enterprise dialog box.	1304
38.87 Manage XML Exporting Formats dialog box.	1305
38.88 Map Fields dialog box.	1306
38.89 Member Selector dialog box.	1307
38.90 New Formatting Formula dialog box.	1309
38.91 Number and Date Format dialog box.	1309

38.92	Object Information dialog box	1310
38.93	Object Size and Position dialog box.	1311
38.94	ODBC Formats dialog box.	1312
38.95	OLAP Connection Browser dialog box.	1312
	Shortcut menu commands.	1313
38.96	Open dialog box.	1314
38.97	Open Hyperlink dialog box.	1314
38.98	Options dialog box.	1314
	Types of setting options.	1315
	Layout tab (Options dialog box).	1315
	Database tab (Options dialog box).	1319
	Formula Editor tab (Options dialog box).	1323
	Reporting tab (Options dialog box).	1326
	Fields tab (Options dialog box).	1329
	Fonts tab (Options dialog box).	1331
	Smart Tag & HTML Preview tab (Options dialog box).	1332
	Dependency Checker tab (Options dialog box).	1334
38.99	Order Links dialog box.	1337
38.100	Page Setup dialog box.	1337
38.101	Paste Special dialog box.	1338
38.102	Performance Information dialog box.	1339
38.103	Preview Sample dialog box.	1340
38.104	Properties dialog box.	1340
38.105	Publish Object dialog box.	1341
38.106	Record Selection Formula Editor dialog box.	1342
38.107	Refresh Report Data dialog box.	1343
38.108	Reorder Displayed Members dialog box.	1343
38.109	Report Alerts dialog box.	1344
38.110	Report Explorer dialog box.	1344
38.111	Report Options dialog box.	1345
38.112	Report Part Map Fields dialog box.	1352
38.113	Repository Explorer dialog box.	1353
38.114	Resolve Map Mismatch dialog box.	1354
	Change Map tab (Resolve Map Mismatch dialog box).	1354
	Resolve Mismatch tab (Resolve Map Mismatch dialog box).	1355
38.115	SAP BusinessObjects Enterprise Explorer dialog box.	1356
38.116	Save As dialog box.	1356
38.117	Saved Data Indexes dialog box.	1356
38.118	Search dialog box (OLAP Worksheet).	1357
38.119	Select Barcode Type dialog box.	1358
38.120	Select Export File dialog box.	1358

38.121	Set Datasource Location dialog box.	1359
38.122	Set Default Export Options dialog box.	1360
38.123	Set Default Values dialog box.	1361
38.124	Set Line Height dialog box.	1362
38.125	Set OLAP Cube Location dialog box.	1363
38.126	Set Print Date and Time dialog box.	1363
38.127	Set Prompt Order dialog box.	1364
38.128	Show SQL Query dialog box.	1364
38.129	Sort Control dialog box.	1365
38.130	SQL Expression Editor dialog box.	1366
38.131	SQL Expression Name dialog box.	1367
38.132	Start Page.	1367
	Getting Started.	1367
	Online Resources.	1369
38.133	Subreport Links dialog box.	1369
38.134	Symbol Style dialog box.	1371
38.135	Text Format dialog box.	1373
	Tabs tab (Text Format dialog box).	1373
38.136	Toolbars dialog box.	1374
38.137	Trendlines dialog box.	1375
38.138	Workbench dialog box.	1377
38.139	XML Attribute dialog box.	1377
38.140	XML Export Schema dialog box.	1379
38.141	Zoom dialog box.	1379
39	Wizard and Expert Reference.	1380
39.1	Overview.	1380
39.2	Report Creation Wizards.	1380
	Standard Report Creation Wizard.	1380
	Cross-Tab Report Creation Wizard.	1381
	Mailing Labels Report Creation Wizard.	1381
	OLAP Report Creation Wizard.	1382
	Screen reference.	1382
39.3	SAP BusinessObjects Registration Wizard.	1399
39.4	Calculation Expert.	1399
39.5	Chart Expert.	1400
	Type tab (Chart Expert).	1400
	Data tab (Chart Expert).	1401
	Color Highlight tab (Chart Expert).	1405
	Axes tab (Chart Expert).	1406
	Options tab (Chart Expert).	1408
	Text tab (Chart Expert).	1410

39.6	Cross-Tab Expert.	1412
	Cross-Tab tab (Cross-Tab Expert).	1413
	Style tab (various experts).	1415
	Customize Style tab (Cross-Tab Expert).	1415
39.7	Cross-Tab Calculated Members Expert.	1419
39.8	Data Analysis Expert.	1420
39.9	Database Expert.	1420
	Data tab (Database Expert).	1421
	Links tab (Database Expert).	1422
39.10	Flash Data Binding Expert.	1424
	Xcelsius Data tab (Flash Data Binding Expert).	1424
	Flash Vars tab (Flash Data Binding Expert).	1425
39.11	Formula Expert.	1426
39.12	Group Expert.	1428
39.13	Group Sort Expert.	1429
39.14	Highlighting Expert.	1431
39.15	Map Expert.	1433
	Data tab (Map Expert).	1433
	Type tab (Map Expert).	1438
	Text tab (Map Expert).	1441
39.16	OLAP Expert.	1442
	Customize Style tab (OLAP Expert).	1443
	Labels tab (OLAP Expert).	1445
39.17	Record Sort Expert.	1446
39.18	Search Expert.	1447
39.19	Section Expert.	1449
	Common tab (Section Expert).	1451
	Paging tab (Section Expert).	1454
	Color tab (Section Expert).	1455
	Layout tab (Section Expert).	1456
39.20	Select Expert.	1457
39.21	Template Expert.	1459
39.22	XML Expert.	1460
	Format tab (XML Expert).	1460
	Options tab (XML Expert).	1462
40	Operators.	1464
40.1	Arithmetic operators.	1464
	Add operator.	1464
	Subtract operator.	1465
	Multiply operator.	1466
	Divide operator.	1466

	Percent operator.	1467
	Integer divide ($x \setminus y$).	1468
	Modulus ($x \text{ Mod } y$).	1469
	Negate operator.	1469
	Exponentiate ($x \wedge y$).	1470
40.2	Conversion operators.	1471
	To currency operator.	1471
40.3	Comparison operators.	1472
	Equal operator	1472
	Not Equal operator.	1474
	Less Than operator	1475
	Greater Than operator.	1476
	Less or equal operator.	1477
	Greater or Equal operator	1477
40.4	String operators.	1478
	Concatenate operator ($x + y$).	1478
	Concatenate ($x \& y$).	1479
	Subscript for arrays and strings.	1481
	In String operator.	1483
	Insert Empty String	1484
40.5	Range operators.	1485
	In Range operator	1485
	Make Range operator ($x \text{ To } y$).	1486
	Left end point excluded range ($x _ \text{To } y$).	1487
	Right end point excluded range ($x \text{ To } _ y$).	1488
	Both end points excluded range ($x _ \text{To } _ y$).	1488
	Is $> x$	1489
	Is $< x$	1490
	Is $\geq x$	1491
	Is $\leq x$	1491
	Up from (upFrom x)	1492
	Up from but not including (upFrom_ x)	1492
	Up to (upTo x)	1493
	Up to but not including (upTo_ x)	1494
40.6	Boolean operators.	1494
	Not operator	1494
	And operator	1495
	Or operator	1496
	Xor (Logical exclusion).	1497
	Eqv (Logical equivalence).	1498
	Imp (Logical implication).	1499

40.7	Array operators.	1500
	Make Array	1500
	Subscript (x[y]).	1500
	In Array	1501
	Redim.	1501
	Redim Preserve.	1503
40.8	Pattern operators.	1504
	Starts with operator	1504
	Like pattern operator.	1505
40.9	Control structure operators.	1506
40.10	Other operators.	1507
	Parentheses operators.	1507
	Assignment operators.	1508
	Comments.	1510
	Date-time literal (#...#)	1513
40.11	Variable declaration operators.	1514
	Variable declarators for Basic syntax	1514
	Variable declarators for Crystal syntax.	1515
40.12	Scope operators	1516
	Variable scopes for Basic syntax.	1516
	Variable scopes for Crystal syntax.	1517
40.13	Expression separator.	1518
40.14	Constants	1519
	Date constants.	1519
	Formatting constants.	1521
41	Functions.	1524
41.1	About number formatting in the examples.	1524
41.2	Math.	1524
	Abs (x)	1524
	Sgn (number)	1526
	Int (number)	1527
	Round.	1528
	RoundUp.	1531
	Truncate.	1533
	MRound.	1535
	Ceiling.	1536
	Floor.	1538
	Fix	1539
	Remainder (num, denom).	1541
	Sin (number)	1542
	Cos (number)	1543

	Tan (number)	1544
	Atn (number)	1545
	Pi	1546
	Sqr (number)	1546
	Exp (number)	1547
	Log (number)	1548
	Rnd	1549
41.3	Miscellaneous	1550
	GetValueDescriptions (x)	1551
41.4	Summary	1551
	Conditions for summary functions	1552
	Sum	1552
	Average	1554
	StdDev	1555
	PopulationStdDev	1557
	Variance	1559
	PopulationVariance	1561
	Maximum	1563
	Minimum	1565
	Count	1567
	DistinctCount	1568
	Correlation	1570
	Covariance	1572
	WeightedAverage	1573
	Median	1575
	PthPercentile	1576
	NthLargest	1577
	NthSmallest	1579
	Mode	1580
	NthMostFrequent	1582
	PercentOfSum	1583
	PercentOfAverage	1585
	PercentOfMaximum	1587
	PercentOfMinimum	1588
	PercentOfCount	1590
	PercentOfDistinctCount	1592
41.5	Financial	1593
	ACCRINT	1595
	ACCRINTM	1596
	AmorDEGRC	1598
	AmorLINC	1600

CoupDayBS	1602
CoupDays	1603
CoupDaysNC	1605
CoupNCD (settlementDate, maturityDate, frequency)	1607
CoupNum (settlementDate, maturityDate, frequency)	1608
CoupPCD (settlementDate, maturityDate, frequency)	1610
CumIPMT (rate, nPeriods, presentValue, startPeriod, endPeriod, type)	1611
CumPrinc (rate, nPeriods, presentValue, startPeriod, endPeriod, type)	1612
DB	1614
DDB	1615
Days360	1617
DISC	1618
DollarDE (fractionNumber, fractionBase)	1620
DollarFR (decimalNumber, fractionBase)	1621
Duration	1622
Effect (rate, nCompoundingPeriods)	1624
FV	1625
FVSchedule (value, rates)	1626
IntRate	1627
IPmt	1629
IRR	1631
ISPMT (rate, period, nPeriods, presentValue)	1632
MDuration	1633
MIRR (values, financeRate, reinvestRate)	1635
Nominal (rate, nCompoundingPeriods)	1636
NPer	1637
NPV (rate, values)	1638
OddFPrice	1640
OddFYield	1641
OddLPrice	1643
OddLYield	1645
Pmt	1647
PPmt	1649
Price	1651
PriceDisc	1652
PriceMat	1654
PV	1656
Rate	1657
Received	1658
SLN (cost, salvage, life)	1660
SYD (cost, salvage, life, period)	1661

	TBillEq (settlementDate, maturityDate, discountRate)	1662
	TBillPrice (settlementDate, maturityDate, discountRate)	1663
	TBillYield (settlementDate, maturityDate, price)	1665
	VDB	1666
	XIRR	1668
	XNPV (rate, values, dates)	1670
	YearFrac	1671
	Yield	1672
	YieldDisc	1674
	YieldMat	1676
41.6	Grid Value.	1677
	CurrentColumnIndex.	1677
	CurrentRowIndex.	1678
	CurrentSummaryIndex.	1679
	GetColumnGroupIndexOf (columnIndex).	1680
	GetColumnPathIndexOf (columnPath).	1681
	GetIndexOf (groupAlias, searchValue, (optional) startingIndex).	1682
	GetNumColumnGroups.	1683
	GetNumColumns.	1684
	GetNumRowGroups.	1685
	GetNumRows.	1685
	GetNumSummaries.	1686
	GetRowGroupIndexOf (rowIndex).	1687
	GetRowPathIndexOf (rowPath).	1688
	GetTotalValueFor (groupAlias, (optional) summaryIndex).	1688
	GridLabelAt (groupAlias, row/columnIndex).	1689
	GridRowColumnValue (groupAlias, (optional) row/columnIndex).	1691
	GridValueAt (rowIndex, columnIndex, summaryIndex).	1691
41.7	Strings.	1692
	Len (str)	1693
	Length (str)	1695
	Trim (str)	1697
	LTrim (str)	1698
	TrimLeft (str)	1699
	RTrim (str)	1701
	TrimRight (str)	1702
	UCase (str)	1703
	UpperCase (str)	1705
	LCase (str)	1706
	LowerCase (str)	1707
	ProperCase(str)	1709

StrReverse (str)	1710
IsNumeric (str)	1711
NumericText (str)	1712
ToNumber	1714
ToText	1716
ToWords	1725
ReplicateString (str, #copies)	1727
Space (integer)	1729
InStr	1730
InStrRev	1732
StrCmp	1733
Mid	1735
Left (str, length)	1736
Right (str, length)	1737
Val (str)	1738
ChrW (x)	1739
AscW (str)	1741
Filter	1742
Replace	1744
Join	1745
Split	1747
Roman	1749
URLEncode	1750
URLDecode	1753
41.8 Date and Time	1754
CurrentDate	1755
CurrentTime	1756
CurrentDateTime	1757
Date	1758
Time	1760
DateTime	1762
DateValue	1764
TimeValue	1766
DateTimeValue	1768
DateSerial (year, month, day)	1771
TimeSerial (hour, minute, second)	1773
IsTime	1774
IsDate	1775
IsDateTime	1777
Year (x)	1779
Month (x)	1780

	Day (x)	1781
	WeekDay	1782
	DayOfWeek	1784
	Hour (x)	1785
	Minute (x)	1786
	Second (x)	1787
	MonthName	1788
	WeekdayName	1789
	Timer	1791
	DateAdd (intervalType, nIntervals, startDateTime)	1791
	DateDiff	1793
	DatePart	1797
	First Day of Week/Year constants	1799
	ShiftDateTime (inputDateTime, inputTimeZone, newTimeZone).	1799
41.9	Date Ranges.	1801
	WeekToDateFromSun	1801
	MonthToDate	1802
	YearToDate	1803
	Last7Days	1804
	Last4WeeksToSun	1805
	LastFullWeek	1806
	LastFullMonth	1807
	AllDatesToToday	1808
	AllDatesToYesterday	1809
	AllDatesFromToday	1810
	AllDatesFromTomorrow	1811
	Aged0To30Days, Aged31To60Days, Aged61To90Days	1812
	Over90Days	1813
	Next30Days, Next31To60Days, Next61To90Days, Next91To365Days	1814
	Calendar1stQtr, Calendar2ndQtr, Calendar3rdQtr, Calendar4thQtr.	1816
	Calendar1stHalf, Calendar2ndHalf	1817
	LastYearMTD	1818
	LastYearYTD	1819
41.10	Arrays.	1820
	Array (x,...)	1820
	MakeArray (x, ...)	1821
	UBound (array)	1822
	Array summary functions	1824
41.11	Ranges.	1824
	GetLowerBound (x).	1824
	GetUpperBound (x).	1825

	HasLowerBound (x)	1826
	HasUpperBound (x)	1827
	IncludesLowerBound (x)	1828
	IncludesUpperBound (x)	1829
41.12	Type Conversion	1830
	CBool (number or currency)	1830
	CCur	1831
	CDBl	1832
	CStr	1834
	CDate	1838
	CTime	1840
	CDateTime	1842
41.13	Programming Shortcuts	1844
	Choose (index, choice1, choice2, ...)	1845
	IIF (expression, truePart, falsePart)	1846
	Switch (expr1, value1, expr2, value2, ...)	1848
41.14	Evaluation Time.	1851
	BeforeReadingRecords	1851
	WhileReadingRecords	1852
	WhilePrintingRecords	1853
	EvaluateAfter (x)	1854
41.15	Print State.	1855
	CountHierarchicalChildren(groupLevel)	1855
	CurrentPageOrientation.	1856
	DrillDownGroupLevel	1857
	GroupNumber	1857
	GroupSelection	1858
	HasValue (fld)	1859
	HierarchyLevel (GroupingLevel)	1860
	InRepeatedGroupHeader	1861
	IsNull (fld)	1862
	Next (fld)	1863
	NextIsNull (fld)	1865
	OnFirstRecord	1866
	NextValue (fld)	1866
	OnLastRecord	1867
	PageNofM	1868
	PageNumber	1869
	Previous (fld)	1870
	PreviousIsNull (fld)	1871
	PreviousValue (fld)	1872

	RecordNumber	1873
	RecordSelection	1874
	TotalPageCount	1875
41.16	Document Properties.	1876
	PrintDate	1876
	PrintTime	1877
	PrintTimeZone.	1878
	ModificationDate	1879
	ModificationTime	1880
	DataDate	1881
	DateTime	1881
	DateTimeZone.	1882
	ReportTitle	1883
	ReportComments	1884
	Filename	1885
	FileAuthor	1885
	FileCreationDate	1886
	CurrentCEUserName	1887
	CurrentCEUserID	1888
	GroupingLevel (fld).	1888
	CurrentCEUserTimeZone.	1889
	ContentLocale.	1890
	SelectionLocale.	1891
41.17	Xcelsius.	1892
	ToXMLColumn(x).	1892
	ToXMLRow(x).	1893
	ToXMLData(x).	1894
41.18	Alerts.	1896
	IsAlertEnabled(alertName)	1896
	IsAlertTriggered(alertName)	1897
	AlertMessage(alertName)	1898
41.19	Additional Functions.	1899
	DateTo2000 (Date, Number).	1900
	DTSTo2000 (DateTimeString, Number).	1902
	DTSToDate (DateTimeString).	1904
	DTSToSeconds (DateTimeString).	1905
	DTSToTimeString (DateTimeString).	1906
	ExchGetId (address).	1907
	ExchGetOrganization (address).	1908
	ExchGetSite (address).	1909
	ExtractString (orig, startStr, endStr).	1910

EventNumber (eventNum).	1911
ExchGetPath (Path).	1912
ByteToText (byte).	1913
FRCurrentRatio (CurrentAssets, CurrentLiabilities).	1914
FRQuickRatio (CurrentAssets, Inventories, CurrentLiabilities).	1915
FRDebtEquityRatio (TotalLiabilities, TotalEquity).	1917
FREquityVsTotalAssets (TotalEquity, TotalAssets).	1918
FRNetProfitMargin (NetProfit, Sales).	1920
FRGrossProfitMargin (GrossProfit, Sales).	1921
FROperatingProfitMargin (OperatingProfit, Sales).	1923
FRInterestCoverage (CashFlow, InterestExpenses).	1924
FRCashFlowVsTotalDebt (CashFlow, TotalDebt).	1925
FRReturnOnEquity (NetProfit, TotalEquity).	1927
FRReturnOnNetFixedAssets (NetProfit, NetFixedAssets).	1928
FRReturnOnTotalAssets (NetProfit, TotalAssets).	1930
FRReturnOnInvestedCapital (NetProfit, TotalBankDebt, TotalEquity).	1931
FRReturnOnCommonEquity (NetProfit, PreferredDividend, CommonEquity).	1933
FREarningsPerCommonShare (NetProfit, PreferredDividend, NumOfCommonShare).	1934
FRAccRecTurnover (AccountReceivable, Sales, NumOfDays).	1936
FRInventoryTurnover (Inventory, Sales, NumOfDays).	1938
FRPriceEarningsRatio (MarketPrice, EarningsPerShare).	1939
FRDividendYield (Dividend, MarketPrice).	1940
Now.	1942
Picture (string, picture).	1943
LooksLike (string, mask).	1944
Soundex (string).	1946
DateTimeTo2000 (dateTime, number).	1947
DTSToDateTime (DateTimeString).	1949
DTSToTimeField (DateTimeString).	1950
DateTimeToDate (dateTime).	1951
DateTimeToTime (dateTime).	1951
DateTimeToSeconds (dateTime).	1952
41.20 Functions to duplicate group fields.	1953
41.21 Grand total functions.	1955
41.22 Conditional formatting functions.	1956
GridRowColumnValue (name).	1956
CurrentFieldValue.	1958
DefaultAttribute.	1959
RGB (red, green, blue).	1960
Color (red, green, blue).	1961
41.23 SQL Expression Builder additional functions.	1962

Attribute functions. 1962

CASE Expressions. 1963

Date/Time functions. 1963

Hash-related functions. 1963

Numeric functions. 1963

Logical Predicates. 1964

String functions. 1964

Built-in Functions. 1965

1 Document History

The following table provides an overview of the most important document changes:

Version	Date	Description
SAP BusinessObjects BI Platform 2025	March, 2025	Added information about the new SAP BW (DSL) driver in Reporting on Universes and managed connections [page 376]
SAP BusinessObjects BI Platform 4.3 SP4	December, 2023	Added information about the new Universe (Unx) in Reporting on Universes and managed connections [page 376]. Crystal Reports 2020 (CR2020) Fiori viewer can be used with Open Document now. For more details, see . One more thing, users can easily favorite or un-favorite a Crystal Reports 2020 (CR2020) report now, with a simple click in the CR2020 Fiori viewer toolbar.
SAP BusinessObjects BI Platform 4.3 SP3	December, 2022	Added information about the new dynamic picture options in To scale, crop, or size an object [page 280] .
SAP BusinessObjects BI Platform 4.3 SP2	December, 2021	Added information about the support of OData data sources in Using OData data sources [page 935] .
SAP BusinessObjects BI Platform 4.3	June, 2020	Added information about the new Microsoft Excel .xlsx export format support in Export format types [page 617]. Updated the Accessing Data Sources [page 1038] and Mapping [page 320] sections.

2 Introduction to SAP Crystal Reports 2025

CRYSTAL REPORTS

Help | Aide | Hilfe | Guida in linea | Ayuda | 帮助 | 説明 | 도움말 | ヘルプ

Access our product documentation online at: https://help.sap.com/viewer/p/SAP_ANALYTICS

2.1 About Crystal Reports

SAP Crystal Reports 2025 (Crystal Reports) is designed to work with your database to help you analyze and interpret important information. Crystal Reports makes it easy to create simple reports, and, it also has the comprehensive tools you need to produce complex or specialized reports.

Create any report you can imagine

Crystal Reports is designed to produce the report you want from virtually any data source. Built-in report experts guide you step by step through building reports and completing common reporting tasks. Formulas, cross-tabs, subreports, and conditional formatting help make sense of data and uncover important relationships that might otherwise be hidden. Geographic maps and graphs communicate information visually when words and numbers are simply not enough.

Extend reporting to the Web

The flexibility of Crystal Reports doesn't end with creating reports — your reports can be published in a variety of formats including Microsoft Word and Excel, E-mail and even over the Web. Advanced Web reporting lets other members of your workgroup view and update shared reports inside their web browser.

Incorporate reports into applications

Application and web developers can save time and meet their users needs by integrating the report processing power of Crystal Reports into their database applications. Support for most popular development languages makes it easy to add reporting to any application.

Whether it's the web master in IT, the promotion manager in marketing, the database administrator in finance or the CEO, Crystal Reports is a powerful tool designed to help everyone analyze and interpret the information that's important to them.

2.2 About the Crystal Reports documentation

The Crystal Reports documentation includes procedures for typical reporting tasks such as placing fields, formatting reports, and sorting records. It also contains information on more specific areas of interest such as advanced formula creation and accessing different types of data. Use the documentation as a reference for your basic reporting needs as well as an introduction to new concepts in report creation.

2.2.1 Locate information quickly

- Access online help from the Crystal Reports help menu.
Use the Contents tab to view all major sections in the online help and drill down to specific headings within each section, the Index tab to view topics in alphabetical order, and the Search tab to enter a keyword to view all the sections that relate to the keyword.
- Access a PDF version of the Crystal Reports documentation online at help.sap.com.
The PDF version contains a subset of the topics found in the online help, along with additional sections—such as a Quick Start—that can help you learn Crystal Reports.

2.2.2 Sample Reports

Many topics in the Crystal Reports documentation include a list of related sample reports. Use these reports to illustrate concepts the topic describes. The sample reports can be adapted to your own needs.

Samples reports can be downloaded from the [SAP Knowledge Base Article 1231843](#) .

3 Quick Start

3.1 Learning how to use Crystal Reports

You can teach yourself how to use Crystal Reports by choosing from the methods available in this section:

- You can study the sample reports and sample database included with Crystal Reports.
The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).
- You can use the detailed descriptions and instructions in the [Quick start for new users \[page 51\]](#).
- You can use the summaries and topic cross-references in the [Quick start for advanced users \[page 72\]](#)—especially useful if you're already familiar with reporting concepts.

Each method is a helpful way to learn and understand Crystal Reports and, although any one might be enough to get you up and running, you can always come back to this section and consult the other methods as you need them.

3.1.1 Sample data - Xtreme.mdb

A sample database, Xtreme.mdb, is available for you to use while you are learning Crystal Reports. Xtreme.mdb is a Microsoft Access database and all of the necessary drivers are included. You should be able to open the database directly and begin designing reports. Virtually all of the examples in this manual are based on Xtreme.mdb data.

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

Xtreme.mdb is a database that contains data for Xtreme Mountain Bikes, a fictitious manufacturer of mountain bikes and accessories.

Note

The sample data has been designed to illustrate various reporting concepts in a training environment, not to teach database design. While there are alternative ways of designing a database, this design was selected to keep the tutorials and examples focused on reporting, not on data manipulation.

3.2 Report Creation Wizards

The tutorials in this section show you how to build a report from scratch. As a complement or an alternative, however, you may want to use the Report Creation Wizards available on the Start Page. There are four Report Creation Wizards:

- [Standard \[page 50\]](#)

- [Cross-Tab \[page 50\]](#)
- [Mailing Label \[page 50\]](#)
- [OLAP \[page 50\]](#)

Each wizard guides you through the creation of a report by providing a series of screens. Many of the wizards have screens unique to a specific type of report. For example, the Mailing Labels Report Creation Wizard has a screen that allows you to specify the type of mailing label you want to use.

Note

After you've clicked the Finish button in a Report Creation Wizard, you can click the Stop button on the Navigation Tools toolbar if you don't want the program to gather all the data from your data source. This is useful if you want to make adjustments to the report layout that the wizard created.

3.2.1 Standard

The Standard Report Creation Wizard is the most generic of the wizards. It guides you through choosing a data source and linking database tables. It also helps you add fields and specify the grouping, summarization (totals), and sorting criteria you want to use. Finally, the Standard Report Creation Wizard leads you through chart creation and record selection.

The Templates screen contains predefined layouts for you to apply to your report to give it more impact.

3.2.2 Cross-Tab

The Cross-Tab Report Creation Wizard guides you through the creation of a report in which your data is displayed as a cross-tab object. Two special screens (Cross-Tab and Grid Style) help you create and format the cross-tab itself.

3.2.3 Mailing Label

The Mailing Labels Report Creation Wizard lets you create a report that is formatted to print on any size mailing label. You can use the Label screen to select a commercial label type, or you can define your own layout of rows and columns for any multi-column style report.

3.2.4 OLAP

The OLAP Report Creation Wizard lets you create a report in which your OLAP data is displayed as a grid object. Although similar to the Cross-Tab Report Creation Wizard in several ways, the OLAP Report Creation Wizard appears to be different due to the requirements of working with OLAP data sources. You first specify the location of your OLAP data, and then you choose the dimensions you want to include in the grid. Next you filter

the report data and choose the style of the grid object, which you can also customize. Finally, you can define labels for your grid and insert a chart, if you wish.

3.3 Quick start for new users

The following tutorial has been designed to give you confidence when creating your first report.

In this tutorial, you will get an introduction to the program as you create a Customer List report. The Customer List is one of the most basic business reports and typically has information such as Customer Name, City, Region, and Contact Name.

You begin by learning the basic concepts: selecting a database, placing some fields on the report, and then selecting specific records to be included. You will then learn how to:

- Insert and move database fields.
- Add and format a title.
- Display a report in the Preview tab so you can fine-tune your work.
- Use the Select Expert to ensure the report includes only the data you need.
- Move objects.
- Group and sort data.
- Insert pictures.
- Print a report.

3.3.1 Before you begin

This tutorial assumes you are familiar with Microsoft Windows and uses conventional terms and procedures common to the Windows environment. If you are not familiar with Windows, please refer to the documentation that came with Microsoft Windows for further explanation.

The default font for all report sections in the program is set to Arial, 10 point. If you have changed the default font, or if your printer does not support this font, the field size, field spacing, and screen shots will look different than those included in this tutorial.

This tutorial has been designed using Microsoft Windows 2000. Screen shots may vary slightly if you are using a different platform.

If you are not familiar with the Crystal Reports environment, review [Formatting \[page 235\]](#), which describes working with the grid, using guidelines, and formatting activities.

3.3.2 Creating the report

Procedure

1. In Windows, click [Start > Programs > SAP Business Intelligence](#), and then select [Crystal Reports 2025](#). The [Start Page](#) appears.
2. In the [Start a New Report](#) area, click [Blank Report](#).

The Database Expert dialog box appears.

ⓘ Note

You can create reports based on database files, SQL/ODBC data sources, Business Views, and a variety of other data sources.

3.3.2.1 Selecting a database to use

The next step in creating a report is to select a database. Use the Xtreme.mdb sample database for this tutorial.

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

Copy Xtreme.mdb to a location on your machine that you can access for the tutorial.

3.3.2.1.1 To select a database

Procedure

1. In the Database Expert dialog box, expand the [Create New Connection](#) folder and then expand the [Database Files](#) folder; then search for the [Xtreme.mdb](#) sample database.

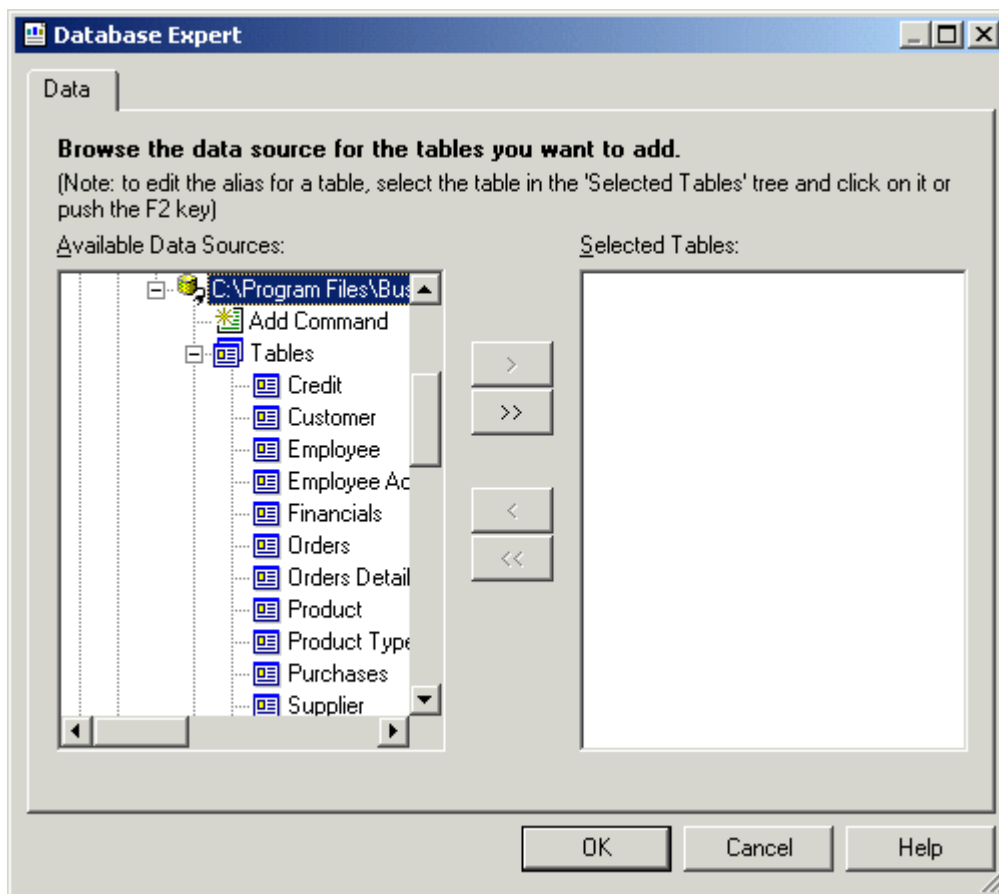
ⓘ Note

You have to use the Open dialog box to find the Xtreme database.

ⓘ Note

If you wish to see database and server properties after you have selected a database, right-click the database node in the Database Expert and select Properties from the shortcut menu.

2. Expand the [Tables](#) node of the [Xtreme.mdb](#) connection to see a list of tables.



Because you are dealing only with customers in this tutorial, you will select the Customer table.

3. Select Customer and click the > arrow to add it to the Selected Tables list, and then click **OK**.

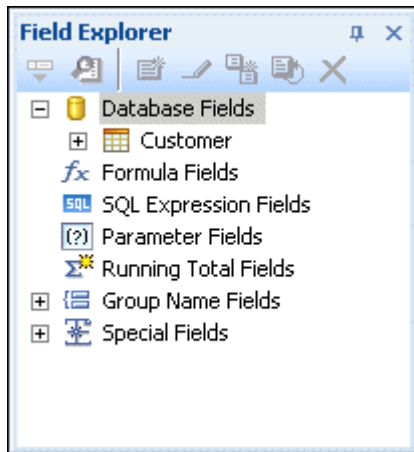
The Design tab of the Report Designer appears.

4.  Click *Field Explorer* on the Standard toolbar.

The Field Explorer dialog box appears.

Note

Depending on how it appeared when you last used Crystal Reports, the Field Explorer dialog box might be docked or in floating mode. For more information about docking explorers, see [Opening and docking explorers \[page 117\]](#).



3.3.2.2 Report sections

The Design tab is divided into five sections: Report Header (RH), Page Header (PH), Details (D), Report Footer (RF), and Page Footer (PF). If at any time you are unsure of the report section in which you are working, simply look at the shaded area to the left of the report which always displays either the section names or the initials that designate the names. See [Design tab \[page 90\]](#).

If the Short Section Names check box is selected in the Design View area of the Options dialog box, then the Report Header, Page Header, Details, Report Footer and Page Footer section names will appear as RH, PH, D, RF and PF respectively.

3.3.2.3 Inserting a field

You'll use the Field Explorer dialog box to insert database fields when you create a new report.

This dialog box is set to remain on-screen until you close it. All the tables available for use are listed in this box.

You will now start placing objects on the report by inserting the Customer Name field.

3.3.2.3.1 To insert a field

Procedure

1. Expand the *Database Fields* node in the Field Explorer dialog box and expand a table.
2. Highlight a field name by clicking the name once.

When you highlight a field name, you can review a subset of the values for that field as well as the field type and size by right-clicking the field and selecting Browse Data from the shortcut menu.



3. Click the *Customer Name* field and drag it into the *Details* section of the report.

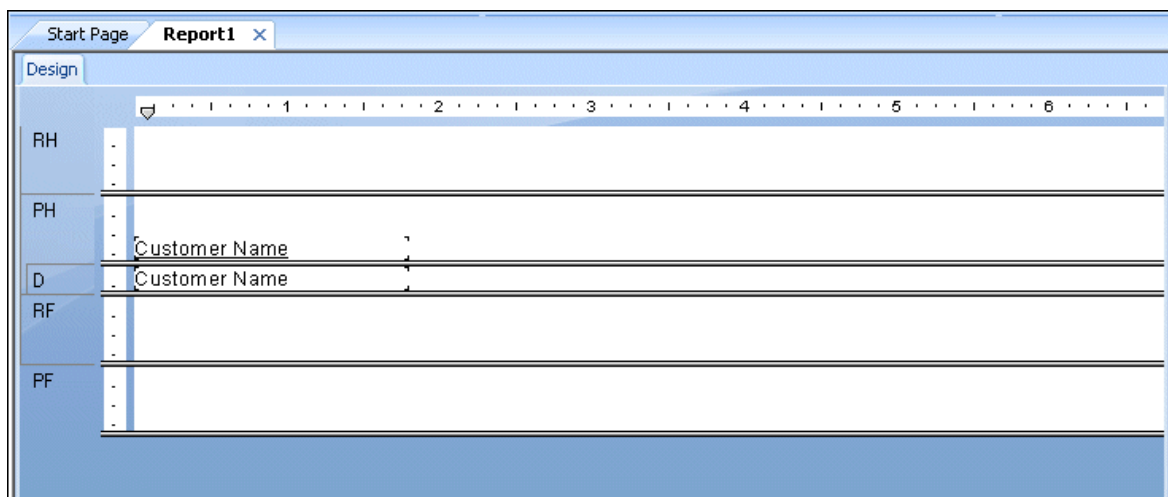
An object frame appears with the Arrow cursor as you drag the field onto the report:



- The object frame represents the object you have just selected for placement.
 - The size of the object frame approximates the size of the data in the field selected.
4. Move the object frame as far to the left as you can in the Details section.

If you move the field too far to the left, the Arrow cursor turns into a Stop cursor, indicating that you cannot drag the field that far. Keep in mind that you cannot place any objects outside the page margin.

The Design tab should look similar to this:



3.3.2.4 Understanding fields

Before going any further, take a look at the field you just placed in the Details section:

- First of all, the object frame indicates that when the report is printed, a field value will appear where the box is positioned.
- If you haven't selected the Show Field Names check box on the Layout tab of the Options dialog box, the object frame contains Xs to indicate that the database field contains a text string. Other data types have different character representations. For example, a currency data type is represented by \$55,555.56.
- If your report is not showing field names, the number of Xs in the object frame is the data width, the maximum number of characters in the field as defined by the database. Whether or not your report shows field names, the width of the object frame is the field width (the amount of space allocated to the field for printing). Initially it is set to the width needed to display the maximum number of characters in the field (using the font selected for the field). You can change this width by resizing the field.
- The size of the characters or Xs indicates the point size selected for the characters in the field.
- The font and style (Bold, Underline, and so on) used in displaying the characters or Xs indicate the font and style selected for the characters in the field. Later in this tutorial you will learn how to make changes to these properties.
- The line spacing is adjusted to the point size selected for the characters in the field.

3.3.2.5 Adding additional fields

Next, you will insert two additional fields in the report. This time, however, you will use the [Ctrl-click](#) combination to add them at the same time.

3.3.2.5.1 To add additional fields

Procedure

1. Highlight the [City](#) field in the Field Explorer dialog box, press the [Ctrl](#) key on your keyboard, and then highlight the [Country](#) field. Release the [Ctrl](#) key.

If you scroll through the field list, you will notice that both fields remain selected.

Note

Using the Ctrl-click combination allows you to select a non-continuous range of fields. The Shift-click combination can be used to select several fields from the list that are contiguous.

2. Drag the fields to place them.

As the cursor is moved over the report, an object frame appears along with the Arrow cursor.

3. Place the fields to the right of the [Customer Name](#) field.

Both fields appear in the Details section of the report in the same order in which they are listed in the Field Explorer dialog box.

3.3.2.6 Selecting fields

When a field is selected, the object frame appears with a handle (box) on its right, left, top, and bottom edge. These handles indicate that the field is selected, and therefore active. To do anything with a field (change the font, move it, and so on), you first have to select it:

- Position the cursor inside the object frame and click once. The handles appear, indicating the object is selected.
- Move the cursor away from the object frame and click in an empty part of the window. The handles disappear.

That's all it takes to select and deselect objects.

3.3.2.7 Resizing fields

Context

To resize the field, follow these steps:

Procedure

1. Click the *Customer Name* field in the Details section to select it.
2. Press the *Ctrl* key and click the field heading to select both objects.
3. Move the cursor over the resizing handle on the right edge of the fields until the cursor turns into a Resizing cursor.
4. Resize the fields to the right until they are approximately two inches in length.

3.3.2.8 Reviewing your work

Context

Now let's see how the report looks with three fields placed and positioned.

Procedure

1.  Click *Print Preview* on the Standard toolbar to activate the *Preview* tab.

The screen should look similar to this:



Note

The first time you preview a report, you must click Print Preview on the Standard toolbar to activate the Preview tab. The Preview tab appears to the right of the Design tab. You can then switch between designing and previewing the report by clicking the corresponding tab.

You have the beginnings of a customer list report, but you still have several fields to add.

- When you are finished reviewing the report, return to the *Design* tab by clicking it.

3.3.2.9 Displaying field names

Field pictures (object frames containing character representations) have been discussed, but there may be times when you want to see the field names in the Design tab.

3.3.2.9.1 To display field names

Procedure

- On the *File* menu, click *Options*.

The Options dialog box appears with the Layout tab active.

- Select the *Show Field Names* check box and click *OK*.

Now in the Design tab you will see the actual field names instead of character representations (X, \$, #, and so on).

3.3.2.10 Combining database fields in a text object

Instead of adding the Contact First Name and Contact Last Name fields as separate objects, you can insert both fields in a text object. This allows you to control the formatting of both fields by making changes to only one object. When you insert fields in a text object, the fields are automatically trimmed (they do not have any extra white space on either side). This is important because a field is a fixed size but the data in the fields can vary in size, leaving various amounts of unwanted white space.

3.3.2.10.1 To combine fields in a text object

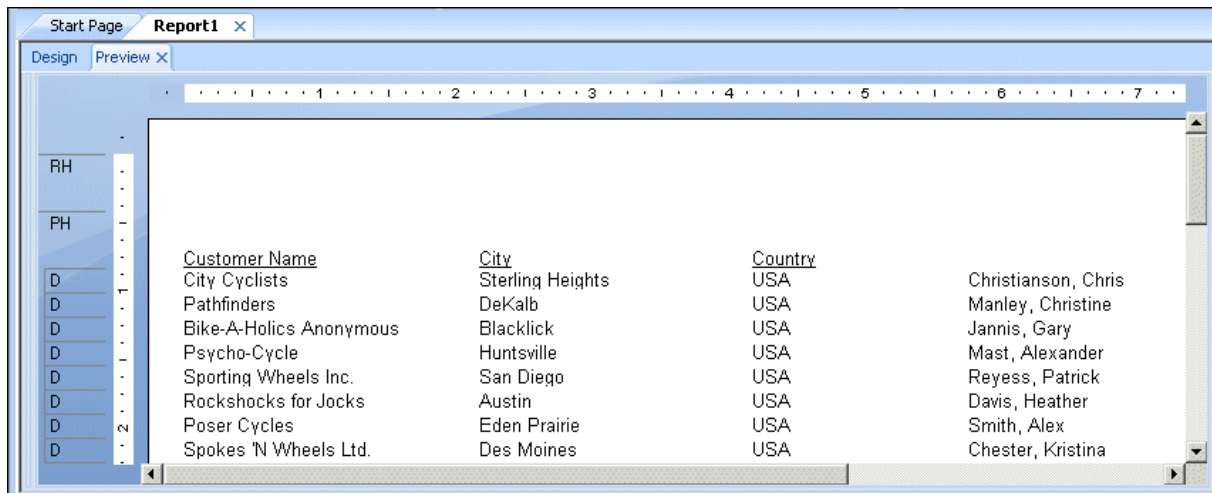
Procedure

1.  Click *Insert Text Object* on the Insert Tools toolbar. As you move the cursor over the report, an object frame appears next to it.
2. Insert the field to the right of the fields in the *Details* section.

As you drag the field, the Design tab automatically scrolls to the right, if necessary. When you click to place the object, a text object appears and the horizontal ruler of the Design tab changes to a ruler/tab selector that is used for editing the text object. If you click an empty area of the report or a field object, the standard Design tab ruler appears.
3. Click once on the border of the text object to select it for resizing. Handles appear on all sides of the object.
4. Move the cursor over the right sizing handle of the text object and increase the width by about 1 inch. You may need to scroll to the right and continue resizing.
5. Double-click inside the text object to select it for editing. Notice the insertion point is now flashing within the text object.
6. Select the *Contact Last Name* field in the Field Explorer dialog box. Remember, you can move the Field Explorer dialog box by dragging and dropping it by its title bar.
7. Drag the field to the text object.
8. Move the cursor over the text object until the cursor becomes a Drag and Drop cursor.
9. Release the mouse button to place the field in the text object. The cursor now appears after the *Contact Last Name* field, within the text object.
10. Type a comma and a space after *Contact Last Name*.
11. In the Field Explorer dialog box, highlight the *Contact First Name* field.
12. Drag the field to the text object.
13. Move the cursor over the text object until the cursor becomes a Drag and Drop cursor. Move the cursor to the right of the comma and space you just typed, and release the mouse button. The field will be inserted to the right of the comma and space.
14. Click the *Preview* tab to look at the fields you just placed.

Results

The report should now look similar to this:



The screenshot shows the SAP Crystal Reports interface. The 'Report1' tab is active, and the 'Preview' view is selected. The report displays a table with four columns: Customer Name, City, Country, and a fourth column containing individual names. The table has 8 rows of data. The interface includes a design tree on the left with fields like RH, PH, and D, and a horizontal ruler at the top.

<u>Customer Name</u>	<u>City</u>	<u>Country</u>	
City Cyclists	Sterling Heights	USA	Christianson, Chris
Pathfinders	DeKalb	USA	Manley, Christine
Bike-A-Holics Anonymous	Blacklick	USA	Jannis, Gary
Psycho-Cycle	Huntsville	USA	Mast, Alexander
Sporting Wheels Inc.	San Diego	USA	Reyess, Patrick
Rockshocks for Jocks	Austin	USA	Davis, Heather
Poser Cycles	Eden Prairie	USA	Smith, Alex
Spokes 'N Wheels Ltd.	Des Moines	USA	Chester, Kristina

3.3.2.11 Adding summary information

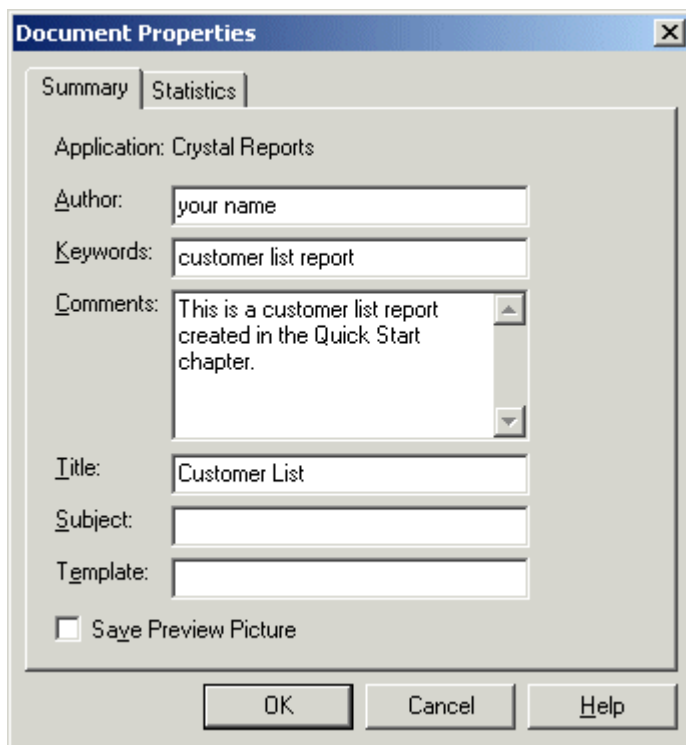
The next step is to add summary information to your report. Adding summary information allows you to specify the author, title, and subject of the report, as well as any keywords or comments related to the report. When you add summary information, users can find information related to the report quickly.

3.3.2.11.1 To add summary information

Procedure

1. On the *File* menu, click *Summary Info*.

The Document Properties dialog box appears with the Summary tab active.



2. Enter information about your report in the text boxes provided. Be sure to enter the title "Customer List" in the *Title* text box. This information will be used in the next section of the tutorial.
3. Click *OK* when finished.

3.3.2.12 Adding a title

As you can see, the report looks incomplete without a title. Although you can add a title using a text object, you can also tell the program to take the title information directly from the Title text box in the Document Properties dialog box.

3.3.2.12.1 To add a title

Procedure

1. Click the *Design* tab.
2. In the Field Explorer, scroll down to *Special Fields* and expand it.
3. Choose *Report Title*.
4. Drag the cursor over the report. An object frame appears.
5. Position the object frame in the upper left-hand corner of the Page Header (PH) section of the report and release the mouse button to place the object.

6. Click the [Preview](#) tab or [Refresh](#) to review your changes.

Results

The report title object now displays the title that you entered in the Title text box of the Document Properties dialog box.

3.3.2.13 Formatting objects

Now you can format the report title. This time, however, you will remain in the Preview tab to do the work. This will make it easier to see your work while you are formatting the title.

3.3.2.13.1 To format an object

Procedure

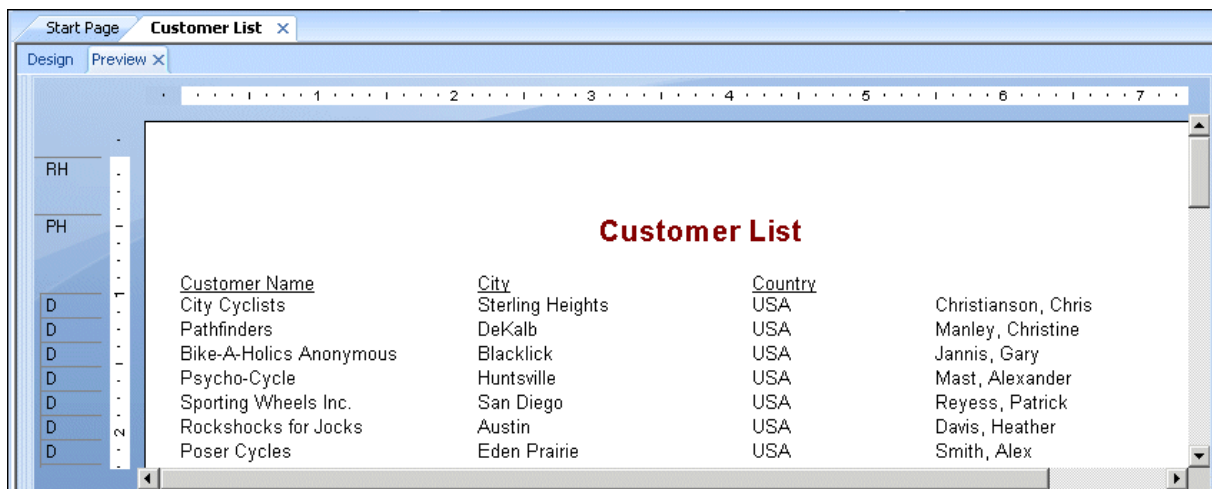
1. To center the title, you will first need to expand the [Title](#) field so that it's about the same width as the data in your report. To do this, select the object by clicking it.
2. Position the cursor on the right edge of the object until the cursor turns into a Resizing cursor. Drag the right edge of the field box until it is even with the right edge of the data in the [Contact Name](#) field object.

You have created a large field that extends from the left edge to the right edge of the report.

3.  With the report title object still selected, click [Align Center](#) on the Formatting toolbar. The title is centered within the object.
4. Right-click the object and choose [Format Field](#) from the shortcut menu.
5. When the Format Editor appears, click the [Font](#) tab.
6. Set the report title to a larger, bolder version of the active font by selecting [Bold](#) from the [Style](#) list and [16](#) (or a point size suitable to the font you are using) from the [Size](#) list.
7. Change the color of the text by selecting [Maroon](#) from the [Color](#) palette. Notice that the Sample box shows an example of how the text will look.
8. Click [OK](#) when finished.
9. Resize the report title object vertically to accommodate the increased size of the title.

Results

The title is now formatted to stand out on the report.



3.3.2.14 Adding a field heading

As you can see, the Contact Name field is the only field without a heading. In this section you will create a heading.

3.3.2.14.1 To add a field heading

Procedure

1.  Click *Insert Text Object* on the Insert Tools toolbar.
2. Insert the text object in the *Page Header* section directly above the Contact Name field.
3.  Click *Underline* on the Formatting toolbar, and then type *Contact Name* into the text object.
4. Click your cursor outside the text object.

Results

The Contact Name field now has a heading that looks just like the other field titles.

3.3.2.15 Saving the report

Procedure

1.  Click [Save](#) on the Standard toolbar to save your work.

Since this is the first time you are saving the report, the Save As dialog box appears displaying the default directory where the file will be saved. Notice that a default file name, based on the report title you defined earlier, is also displayed.

2. Type Custlist.rpt in the [File name](#) box and click [Save](#).

Your report is saved to the default directory or another directory you chose.

3. Click the [Preview](#) tab to view the report.

Results

The report should now look similar to the following:



The screenshot shows the SAP Crystal Reports interface. The 'Customer List' report is displayed in the 'Preview' tab. The report has a title 'Customer List' in red. Below the title is a table with four columns: 'Customer Name', 'City', 'Country', and 'Contact Name'. The table contains seven rows of data. The left sidebar shows the report design elements, including a list of data sources (D) and a list of fields (F).

<u>Customer Name</u>	<u>City</u>	<u>Country</u>	<u>Contact Name</u>
City Cyclists	Sterling Heights	USA	Christianson, Chris
Pathfinders	DeKalb	USA	Manley, Christine
Bike-A-Holics Anonymous	Blacklick	USA	Jannis, Gary
Psycho-Cycle	Huntsville	USA	Mast, Alexander
Sporting Wheels Inc.	San Diego	USA	Reyess, Patrick
Rockshocks for Jocks	Austin	USA	Davis, Heather
Poser Cycles	Eden Prairie	USA	Smith, Alex

Congratulations! You have just created a basic listing report. You will continue to refine this report throughout the rest of this tutorial.

Note

Locally stored documents are not protected by SAP Crystal Reports. Protection needs to be provided by the respective device management (e.g. access control, encryption, etc.)

3.3.3 Record Selection

Crystal Reports allows you to limit or restrict the records that are to be included in a report. In this section you will learn how to:

- Select the records you want included in the report.
- Save a report, including the selection criteria.

For example, it may be useful to have a customer list that only lists customers from the USA. The sample data contains records from the United States and International customers. It is easy to restrict lists like this using the Select Expert.

3.3.3.1 Entering the selection criteria

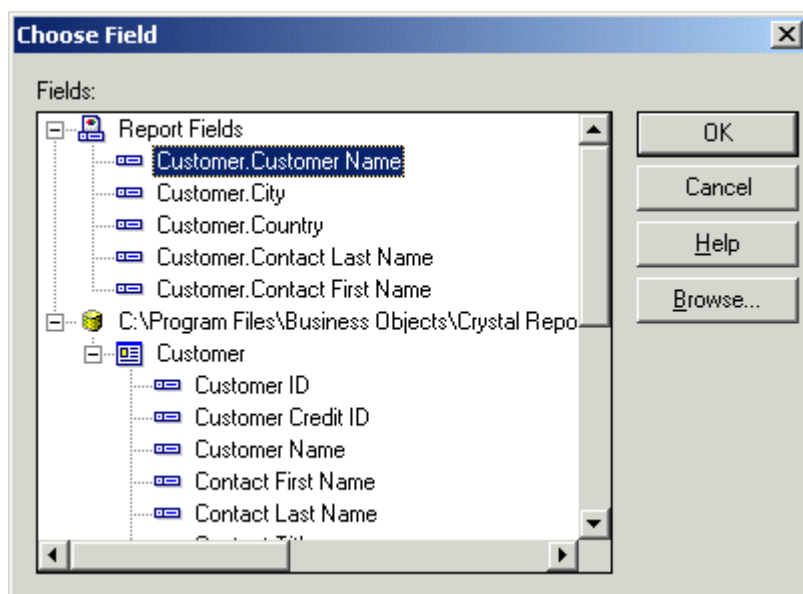
When you scroll through the report, you will see that it contains information for customers from many different countries. In this step, you will limit the number of countries displayed to the USA.

3.3.3.1.1 To enter selection criteria

Procedure

1. To begin, click the *Design* tab to return to design mode.
2. Click an empty area of the report to make sure all fields are deselected.
3.  Click *Select Expert* on the Expert Tools toolbar.

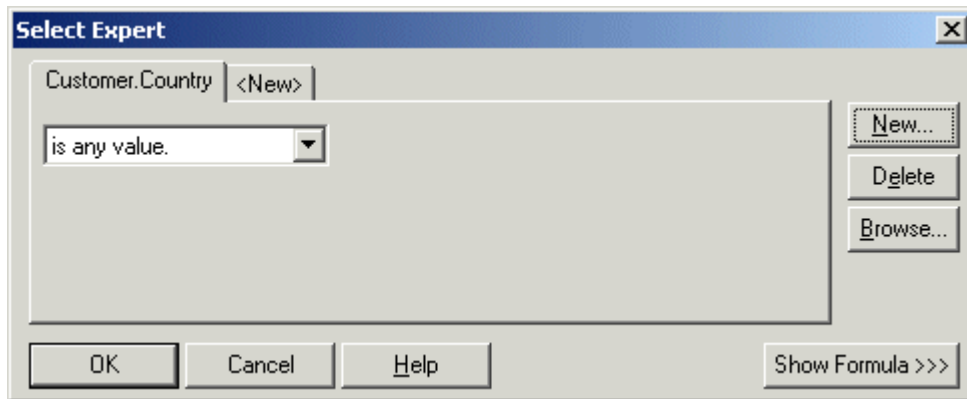
The Choose Field dialog box appears.



This dialog box lists all the fields currently in the report in the Report Fields section and then lists all fields that are available from each table in the database fields section.

4. Since you are going to base record selection on the country field, select Country in the [Fields](#) list and click [OK](#).

The Select Expert appears.



Imagine that you are completing the following sentence:

Select all records where a customer's country is

You complete the sentence with the condition you want the program to use when selecting records for your report. Right now the condition is any value, which implies there is no restriction on the record selection.

5. Click the arrow on the operators box to see what other options you have. Since you want only those records where the Country is USA, select the [is equal to](#) condition.

A new box appears on the right. The dialog box sentence now reads:

Select all records where a customer's country is equal to

All that you need to complete the sentence is the value USA.

6. Click the arrow on the empty list. A list of all the country values appears. Select [USA](#) from the list.

The sentence now reads:

Select all records where a customer's country is equal to USA

7. Click [OK](#) to return to the [Design](#) tab.
8. Click the [Preview](#) tab to review the results of your work.

The Change In Record Selection Formula dialog box appears.

9. Click [Refresh Data](#).
10. Save this version of the report without overwriting the original report by choosing [Save As](#) from the File menu and giving the new report the name USA.rpt.

Results

Congratulations! You have started formatting your report and have added selection criteria to it. More than that, you have learned how to manipulate your data. By now, you have a good idea of the powerful kinds of reports you can prepare.

3.3.4 Grouping and sorting

Data in reports can be grouped and sorted in a variety of ways. Sorting and grouping tools provide a great deal of flexibility for customizing reports.

3.3.4.1 Grouping the report

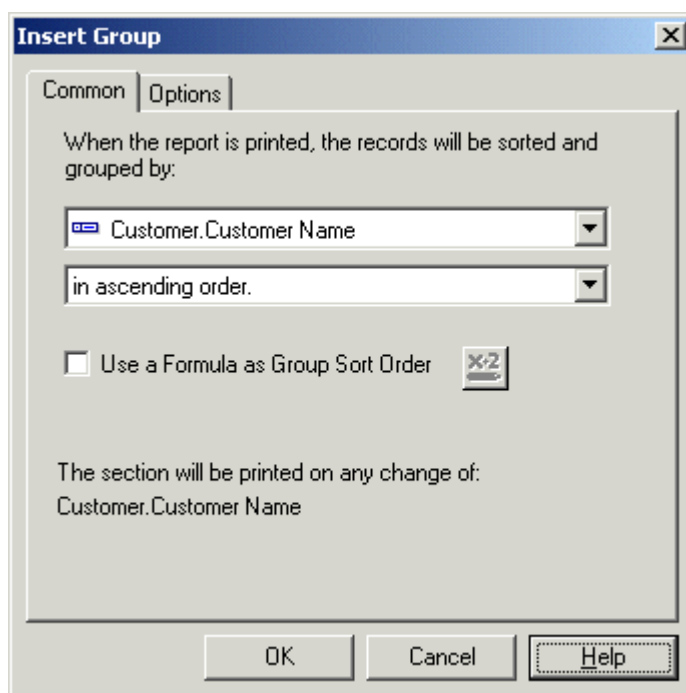
In many reports you need to break the data into groups in order to make it easier to read and to understand. Crystal Reports lets you do this easily. For this customer list, you will group the customers by region and then sort the customers alphabetically within each group.

3.3.4.1.1 To group a report

Procedure

1.  While on the *Design* tab, click *Insert Group* on the Insert Tools toolbar.

The Insert Group dialog box appears.



2. Select Region from the Customer table in the first drop-down list.

The program takes all records with the same value in the region field and places them together in a group on the report.

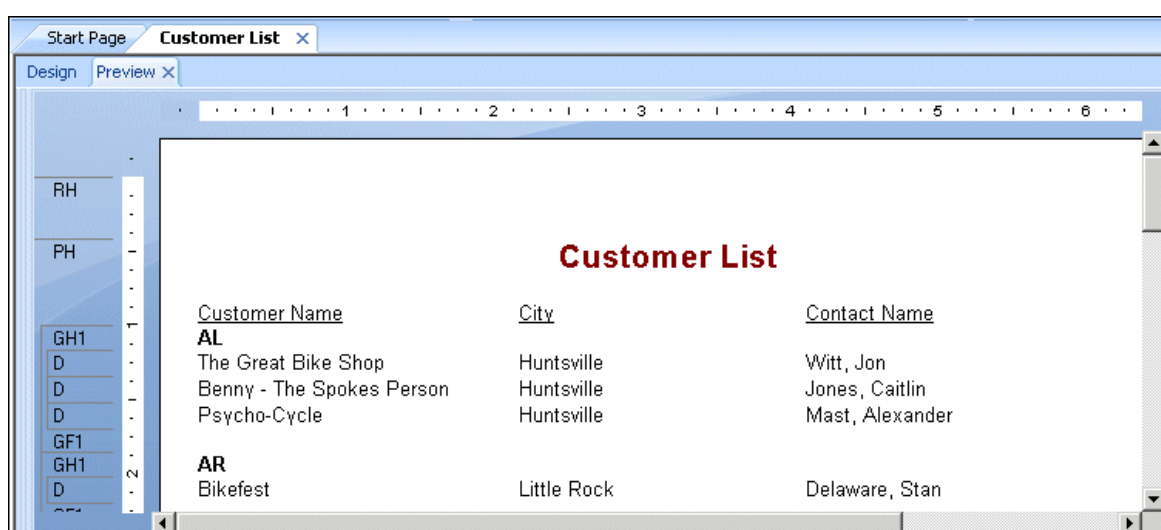
3. Select *in ascending order* from the second drop-down list.

The region grouping will be displayed on the report in alphabetic ascending order.

4. Click *OK*.

Notice that two new sections now appear in the Design tab: GH1 (Group Header #1) and GF1 (Group Footer #1). This is how the program shows that the report has been grouped.

5. Click the *Preview* tab to see what the report looks like.



6.  If the group tree is not visible, select *Toggle Group Tree* on the Standard toolbar to see the groups included in the report.

Results

You can view the group of interest by clicking on the group name in the Group Tree. For example, to see the Texas customer group, click TX in the Group Tree. The program jumps to the Texas group, displaying that group in the Preview tab. The Group Tree allows you to quickly jump to a specific group of interest instead of scrolling through the report looking for the group. For more information on the group tree, see [Group Tree \[page 95\]](#).

Note

For many reports, you will want to insert summaries, subtotals and grand totals. For example, when creating a sales report rather than a customer list, you would want to calculate the total sales amount for each region. See [Sorting, Grouping, and Totaling \[page 167\]](#).

3.3.4.2 Understanding "live" group headers

When a group is inserted, a group name field is automatically inserted in the Group Header section of the report. The group name field displays the current group's name. For example, if you group by region, and preview the report, the group header for the CA (California) group shows "CA."

The group field name is automatically formatted to stand out from the records in the group.

3.3.4.3 Sorting records

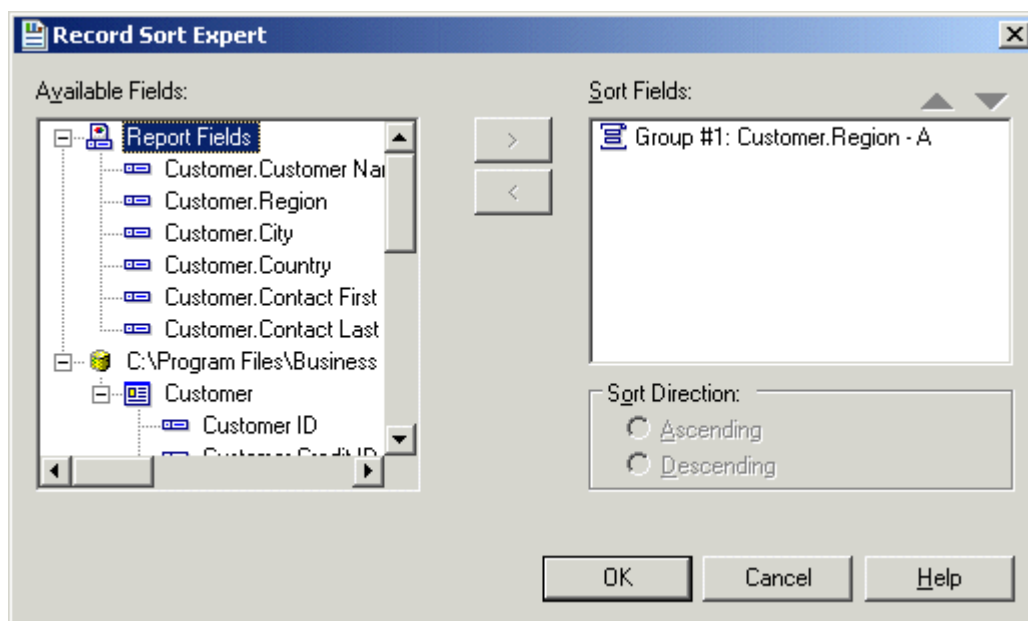
In a typical customer list report, customer names are listed alphabetically. In this example, you will sort the customer names alphabetically within each region.

3.3.4.3.1 To sort records

Procedure

1.  While on the *Preview* tab, click *Record Sort Expert* on the Expert Tools toolbar.

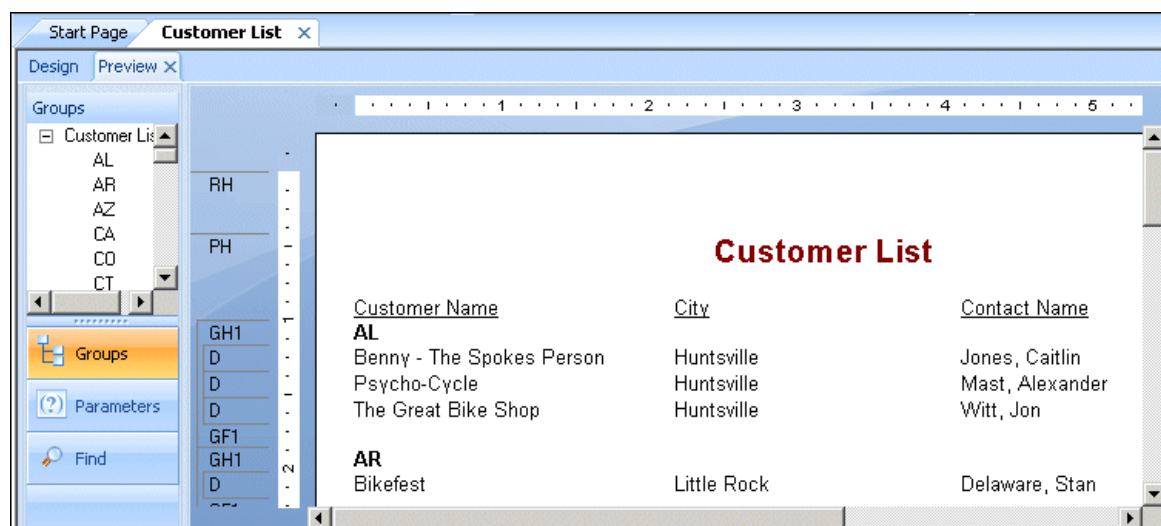
The Record Sort Expert appears.



The Available Fields list box displays all fields currently on your report and all the fields in the data source. You can choose to sort based on any of these fields.

The Sort Fields list box displays the fields that are already sorted in the report. Since the region field has already been sorted, the sorting you are about to do will be within each region, and not for the entire report.

2. Highlight the *Customer Name* field and click the > arrow button to add it to the Sort Fields list.
3. Select *Ascending* for the Sort Direction and click *OK*. The report should now look similar to the following:



Results

Notice that the records within each group are in alphabetic order.

3.3.5 Completing the report

You have just one step left to complete the report. A company logo needs to be added to the first page of the report.

3.3.5.1 Inserting a company logo

In this section, you will place a company logo at the top of the first page of the report.

3.3.5.1.1 To insert a company logo

Procedure

1.  While on the *Design* tab, click *Insert Picture* on the Insert Tools toolbar.
The Open dialog box appears.
2. Choose a bitmap logo (.bmp) file and click *Open*.
An object frame appears as you move the cursor over the report. The object frame represents the logo you will place.
3. Position the object frame in the upper left-hand corner of the Report Header (RH) section of the report and click once to place it.

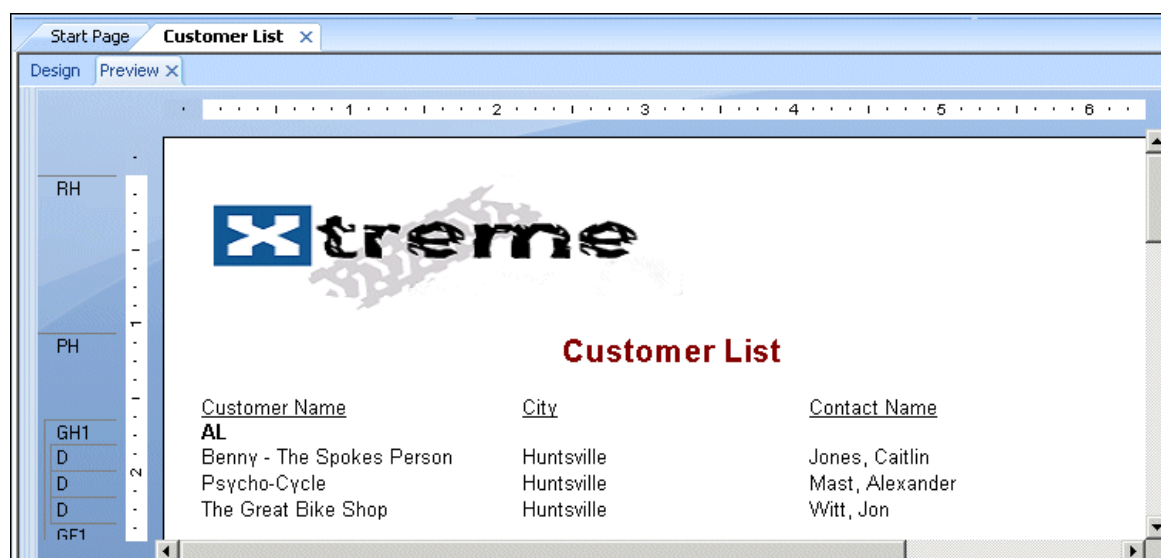
Placing the logo in the Report Header (RH) section ensures that the logo is printed only on the first page of the report.

Note

Although there does not appear to be enough room in the Report Header (RH) section when you place the graphic, the section will automatically expand to accommodate the picture.

4. Click the *Preview* tab to view the report.

The final report should look something like this (with the logo that you chose in step 2):



5. Save the report by clicking *Save* on the Standard toolbar.

Results

You have just completed your first report.

3.4 Quick start for advanced users

If you are an experienced Windows user who wants to get right into the program, follow these steps to set up a report for the first time.

If you're not an experienced user, check the [Quick start for new users \[page 51\]](#).

3.4.1 Choosing a report type and data source

Procedure

1. In Windows, click *Start > Programs > SAP Business Intelligence*, and then select *SAP Crystal Reports 2025*.
2. Click *Blank Report* to open the Database Expert dialog box.

As an alternative to creating a blank report, you can choose one of four Report Creation Wizards.

After choosing a wizard, you can build your report on a variety of data sources.

3. Browse through the contents of the Database Expert to find the data source you want to use for your report.
4. Select a table you want to use in your report and click the > arrow to add it to the Selected Tables list.
5. After you have added all the tables you want to work with, click *OK* on the Database Expert.

Note

If you select more than one table in the Database Expert dialog box, the Links tab appears. For more information on linking, see [Linking multiple tables \[page 99\]](#).

The Design tab appears with Report Header, Page Header, Details, Page Footer, and Report Footer areas. A report is created by inserting and formatting items in each of these areas.

Results

Note



To use additional database tables for a report and match them up on a record-by-record basis, click Database Expert on the Expert Tools toolbar, select the table(s), and then set up the links on the Links tab when it appears.

3.4.2 Working with report elements on the Design tab

Procedure

-  Each of the default report areas contains a single section. To add additional sections, click [Section Expert](#) on the Expert Tools toolbar and use the Section Expert to add the desired sections.

Once you have added sections to an area, you can move, merge, and delete them in the Section Expert. See [Designing with guidelines \[page 255\]](#).
- To turn the grid on or off, choose the [Options](#) command from the [File](#) menu and select or clear the Grid check box in the Design View area of the Layout tab when the Options dialog box appears. See [Using the grid \[page 254\]](#).
- If you are working with the grid off and you want to use snap-to guidelines for positioning objects, click the top or left ruler wherever you want guidelines to appear.

Some further ways you can use guidelines:

- Drag a field to a guideline until it snaps to the guideline.
- Drag the guideline arrow to move the guideline (and any objects that are snapped to it).
- Drag the guideline arrow away from the ruler to remove the guideline. See [Designing with guidelines \[page 255\]](#).

-  If the Field Explorer dialog box is not visible, click [Field Explorer](#) on the Standard toolbar.

Expand the Database Fields node to display a list of fields. To speed the entry of multiple fields, this box will remain on-screen until you close it. This dialog box can be moved to a new location or resized, if you wish. See [Placing data on the report \[page 101\]](#).

- Select the field(s) you want to appear on the report.

You can select and place them one at a time, or use the Shift-click combination to select a number of contiguous fields, or the Ctrl-click combination to select fields from the list at random. Drag and drop is also active. Place the fields in the Details section where you want them to appear.

When you place multiple fields, they appear in the same order that they appear in the Field Explorer dialog box. The program marks the position of each field with a rectangular frame. The characters in the frame indicate whether the field is text (XXX...), number (555...), currency (\$555...), date (12/31/99), time (00:00:00), DateTime (12/31/99 00:00:00), or Boolean (T/F).

Note

The field names and field types can be viewed by selecting the Show Field Names check box on in the Options dialog box (Layout tab).

Note

The program automatically places field titles in the Page Header section unless the Insert Detail Field Headings check box is not selected in the Options dialog box (Layout tab).

Note

If additional Details sections are added to the report, field titles will only be placed in the Page Header section for fields in the Details A (the original) section of your report.

6. Once the objects are in place, you may want to adjust the report sections. To do this, right-click the shaded area to the left of the section ruler and use the shortcut menu that appears:
 - To expand a section to accommodate an additional line, choose the Insert Line command.
 - To have the program automatically align the objects in the section horizontally, choose the Arrange Lines command.
 - To reduce the size of a section to eliminate unnecessary white space above and below objects, choose the Fit Section command. See [Using white space between rows \[page 284\]](#).
7. To generate a report title, you must first enter the title in the Document Properties dialog box. On the **File** menu, click **Summary Info**. Enter a title in the **Title** text box of the Document Properties dialog box. Click **OK**.
8. Choose **Report Title** from the Special Fields list in the Field Explorer. A rectangular placement frame appears when you move the cursor over your report. Click once in the Report Header (RH) section to place the report title. The report title field contains the text that you typed in the **Title** text box of the Document Properties dialog box. See [Adding a title page to the report \[page 112\]](#).
9.  To see how the results will print, click **Print Preview** on the Standard toolbar.

To speed processing time while building a report, you can preview the report using only a small subset of the available data. To do this, go to the File menu, point to Print and then click Preview Sample. See [Preview tab \[page 93\]](#).

In either case, the program takes you to the Preview tab. You can fine-tune a report in the Preview tab while viewing the results as actual report data. You can also close the Preview tab and continue working on the report in the Design tab.

3.4.3 Using other reporting features

Procedure

1. If you want to format a field, change the placement or width of a field, or insert a summary, click the field to select it. Handles appear on the top, bottom, and sides of each selected field:
 - To change the placement of the field(s), use the mouse to drag the field placement frame to its new position.
 - To change the width of the field, use the mouse to drag the right or left handle.
 - Right-click the field to format or to summarize it. A shortcut menu appears listing commands for formatting and summarizing the field.

→ Tip

Many font and formatting options are available on the formatting toolbar.

Note



To apply formatting only under certain conditions, click **Conditional Formula** next to the formatting property in the Format Editor, and create a formula that defines those conditions. See [Working with conditional formatting \[page 287\]](#).

- To create a formula that makes data calculations or comparisons, select **Formula Fields** in the Field Explorer.

Note



You can also click **Formula Workshop** on the Expert Tools toolbar to work with formulas, custom functions, SQL expressions, and so on.

-  Click **New**. The Formula Name dialog box appears. Enter a name for the formula and click **OK**. The Formula Workshop appears with the Formula Editor active.
-  Enter the formula in the Formula Editor. Enter fields, operators, and functions by selecting them from their respective lists or by typing them in. You can check the formula syntax by clicking **Check**.
- When you are finished editing, click **Save and close** to return to the Field Explorer dialog box.

-  Click **Insert to Report** to place the formula just like you would a database field. See [Using Formulas \[page 687\]](#).

-  To insert a subreport (a report within a report), click **Insert Subreport** on the Insert Tools toolbar and choose an existing report to import as a subreport or use the Report Creation Wizard to create a new subreport. See [Inserting subreports \[page 873\]](#).

If you want the records in a subreport to match up with the records in a primary report, click the **Link** tab of the Insert Subreport dialog box and specify the link when the tab appears.

-  To insert a cross-tab object in a report, click **Insert Cross-Tab** on the Insert Tools toolbar and set up the cross-tab in the Cross-Tab Expert when it appears. See [Cross-Tab Objects \[page 346\]](#).

-  To create a parameter field (a field that prompts you for a value whenever you retrieve data for a report), click **Field Explorer** on the Standard toolbar, then select **Parameter Fields** in the Field Explorer dialog box when it appears. Click **New** to set up a parameter field. Once created, you can insert the parameter field in a report like a database field or select it from the **Fields** list in the Formula Workshop.

Parameter fields can be used in reports (as title or label prompts), in selection formulas (as selection criteria prompts), and in formulas (for a variety of purposes including specifying sort fields). See [Parameter Fields and Prompts \[page 831\]](#).

-  To add a chart, click **Insert Chart** on the Insert Tools toolbar. See [Charting \[page 303\]](#).

-  To add a map, click **Insert Map** on the Insert Tools toolbar. See [Mapping \[page 320\]](#).

12. To insert a spreadsheet, picture, or other OLE object that you can edit from within the Report Designer using the tools from the object's native application, choose *OLE Object* from the Insert menu. See [OLE \[page 335\]](#), and [Working with static OLE objects \[page 339\]](#).
13.  To change the record sort order, click *Record Sort Expert* on the Expert Tools toolbar. The Record Sort Expert appears. Highlight the field(s) you want to use for sorting the report data and the sort direction. See [Sorting single and multiple fields \[page 169\]](#).
14.  To limit the report to specific records (for example, the records of California customers who have year-to-date sales greater than \$10,000), click the first field on which you want your selection to be based and then click *Select Expert* on the Expert Tools toolbar. When the Select Expert appears, set up the record selection criteria.
15.  To print the report, click *Print* on the Standard toolbar.

Results

That's it! It is that easy to build a report.

4 Report Design Concepts

4.1 Basic report design

The purpose of this section is to suggest a structured approach to preparing a Crystal report. This approach includes the following elements:

- Deciding on the content of the report.
- Developing a prototype on paper.

This section has been designed to provide a conceptual understanding of the reporting process.

4.2 Deciding on the content of the report

Before you do anything else, you should outline the information you want the report to provide. The following sections provide a guide to making that outline.

Note

A report (.rpt) file might contain sensitive business data. You are advised to protect access to the file by using one of these methods:

- Manage the file with SAP BusinessObjects Business Intelligence platform server.
- Apply file security.
- Apply operating system-level encryption.

4.2.1 Stating the purpose

What is the overall purpose of the report?

Reports are management tools. Their purpose is to help you quickly grasp the essential elements and relationships found in raw data, to help you make effective decisions. For a report to be effective, it has to present the correct data in a logical way. If it presents the wrong data, or if it presents the right data in a haphazard manner, the report may slow the decision-making process or may even encourage incorrect decisions.

A good starting place in the development of a report is to write out the purpose of the report in a sentence or two. The purpose statement helps you focus on your primary needs, and it gives the report both a starting point and a goal.

Here are some examples of purpose statements.

- The purpose of this report is to show monthly and year-to-date sales by sales representatives, compare this year's numbers to last year's, and flag representatives whose sales figures do not meet company standards.
- The purpose of this report is to show sales activity for each item in inventory, and to suggest reorder quantities based on that activity.
- The purpose of this report is to calculate bowling averages and handicaps for each member of the bowling league.

Defining the purpose of the report before you start is a critical step in the overall process.

Who is going to read the report?

A single report is often used by many individuals. A detailed, company-wide sales report, for example, may be used by sales representatives, the regional sales manager, the national sales manager, and the Chief Operating Officer (COO).

These individuals will be interested in different aspects of the report:

- A sales representative will use the report to evaluate individual sales performance and compare this performance to that of other representatives in the region.
- The regional sales manager will use the report to evaluate regional representatives and compare the region's performance to that of other regions.
- The national sales manager will use the report to evaluate the performance of regional managers and compare overall sales to the current sales forecasts.
- The COO will use the report to evaluate the performance of the Vice-President of Marketing and the sales department as a whole, and to project such things as manufacturing needs and warehouse locations.

Since each user of the report has different interests, it is important to plan the report so it includes the information each user is looking for.

4.2.2 Determining the layout of the report

What is the report title going to be?

Write out a working title for the report. You may decide to change it later, but at least you will have a title to use when creating the prototype report.

What identifying information is needed in the header and footer?

You may wish to include the print date, information on who prepared the report, a block of text to describe the purpose of the report, the range of data covered, or something similar. If you are going to include such information, write it down so you can use it in preparing your prototype.

The information can come from a variety of sources, depending on the kind of information you plan to use.

- Information on who prepared the report might be drawn from individual data fields in the database table(s) used. If it is to be drawn from a database table, what table? Or, what combination of tables?
- A block of text can be created as a text object and placed anywhere on the report.
- Crystal Reports can generate information such as the print date or page numbers.

4.2.3 Finding the data

What data do you want to use in the report?

Do you know the type of database you are reporting from? Will you be reporting off a data file, SQL/ODBC, or another data source?

If you do not know, ask the database administrator in your organization for help in setting up the database type and location of the data. For more information, see *Accessing Data Sources* in the *Crystal Reports Online Help*.

Are you familiar enough with the data to find the necessary information? When looking for a Customer Contact name, can the field be found in a database table?

If not, your MIS professional, database administrator, or co-workers will have to help you become familiar with the data.

What specific data should appear in the body of the report?

The body should contain all the data needed to fulfill the statement of purpose you wrote for the report. It should also contain all of the data needed by the various users that you have identified.

This step requires you to look at the available database table(s). Crystal Reports allows you to combine data from different databases when you create reports, so you have a great deal of flexibility in your work.

- Much of the data in a typical report is taken directly from data fields. Which data fields will be used, and where are they located?
- Other data will be calculated based on data fields. Which data fields will be used in the calculations?
- Still other data will be placed directly into the report using text objects (headings, notes, labels, and so on).

Does the data exist or does it need to be calculated?

Some report information can be drawn directly from data fields (sales information, for example); other information will have to be calculated based on data field values (for example, sales commission, based on the relationship of sales to quota). In your planning, it can be helpful to segregate or flag data that needs to be calculated from that which can be used directly. See [Specifying formulas \[page 692\]](#).

What types of fields contain data?

You should take the time to get to know the data type for data fields that will be used in your calculations. Since formula functions and operators work with specific kinds of data, it is important to recognize the data type you are working with, before you start any calculations. For example, some functions require numeric data, while others work with only string fields. For more information on specific functions and operators, search for "Functions" and "Operators," or for the function or operator by name in the *Crystal Reports Online Help*.

4.2.4 Manipulating the data

There are several ways to manipulate data when you design a report. The following sections list some options you may want to consider.

Do you want the data organized into groups?

How? By customer? By date? By hierarchy? Or by other criteria? Crystal Reports provides several options for grouping data in a report. See [Grouping data \[page 173\]](#).

Do you want the data sorted based on record or group values?

Crystal Reports gives you both alternatives. See [Understanding sort options \[page 167\]](#).

Do you want the report to contain only specific records or groups?

Crystal Reports gives you the opportunity to base a report on all records in a given database, or on a limited set of records from the database. Crystal Reports can be used to select records based on simple date ranges or comparisons, or to create complex formulas to identify the records to be included. Take a few minutes to determine the records needed for the report and list the criteria to be used for selecting those records. See [Selecting records \[page 155\]](#).

Do you want to summarize the data?

Do you want to total, average, count, or determine the maximum or minimum value included in all the values in any column on the report?

Crystal Reports allows you to do this, and it also allows the grand total (or the grand total average, grand total count, and so on) to be placed at the bottom of the selected column. See [Calculating a percentage \[page 200\]](#).

What information should be flagged on the report?

You may want to call attention to some data by flagging it on the report. For example, non-moving inventory items are often flagged on inventory reports so they can be given special attention. You might want to flag each item that has shown no activity during the last month, during the last three months, or during some other defined period. To flag information, identify it and any conditions that will trigger the flagging.

How do you want information flagged?

You may want to flag items with an asterisk or some other symbol, or you may want a word to appear as a flag. In any case, you should write out flagging instructions so they are handy.

Crystal Reports gives you the opportunity to underline report elements, and change the font type, size, or color used for specific report items. It allows you to put borders around items and to draw lines and boxes (to break the report into sections), set off headings, and so on. All of these formatting tools can be used to highlight key data on a report. See [Formatting \[page 235\]](#).

4.2.5 Determining printing area characteristics

Each report area has its own printing characteristics. It is important to understand these characteristics because they affect when and how often different report objects get printed.

4.2.5.1 In what order will the areas print on the report?

Areas print in the order they appear on the Design tab (top to bottom). If there is more than one section in an area, the sections print in the order they appear. For example, if you have three Report Header sections, all three of those sections will print, in order, before the section(s) in the Page Header area begin to print.

4.2.5.2 How often do report objects print?

The way objects print will determine how you design your report. This will help you decide where to place charts, Cross-Tabs, and formulas to get specific results.

Report Header

Objects placed in the Report Header area print once, at the beginning of the report.

- Charts and Cross-Tabs placed in this area contain data for the entire report.

Note

Both charts and Cross-Tabs can filter report data by using a Group Sort; in such cases, the data shown in the chart or Cross-Tab is a subset of the data for the entire report.

- Formulas placed in this area are evaluated once, at the beginning of the report.

Page Header

Objects placed in the Page Header area print at the beginning of each new page.

- Charts or Cross-Tabs cannot be placed in this section.
- Formulas placed in this area are evaluated once per page, at the beginning of each new page.

Group Header

Objects placed in the Group Header area print at the beginning of each new group.

- Charts and Cross-Tabs placed in this area contain data just for the group.
- Formulas placed in this area are evaluated once for each group, at the beginning of the group.

Details area

Objects placed in the Details area print with each new record.

- Charts or Cross-Tabs cannot be placed in this area.
- Formulas placed in this area are evaluated once for each record.

Group Footer

Objects placed in the Group Footer area print at the end of each group.

- Charts and Cross-Tabs placed in this area contain data just for the group.
- Formulas placed in this area are evaluated once for each group, at the end of the group.

Report Footer

Objects placed in the Report Footer area print once at the end of the report.

- Charts and Cross-Tabs placed in this area contain data for the entire report.

Note

Both charts and Cross-Tabs can filter report data by using a Group Sort; in such cases, the data shown in the chart or Cross-Tab is a subset of the data for the entire report.

- Formulas placed in this area are evaluated once, at the end of the report.

Page Footer

Objects placed in the Page Footer area print at the bottom of each page.

- Charts and Cross-Tabs cannot be placed in this area.
- Formulas placed in this area are evaluated once per page, at the end of each new page.

4.3 Developing a prototype on paper

While a paper prototype is useful regardless of your level of expertise with Crystal Reports, it is particularly valuable when you are first learning the program. With the paper prototype in hand, you can put your full effort into learning and using the commands, rather than into trying to design and learn at the same time.

4.3.1 To design a paper prototype

Procedure

1. Get the same size paper you will be using for the finished report.
2. Position the title and other descriptive header information, using boxes or lines to represent report elements.
3. Position the footer information.
4. Review the page layout for balance.
5. Look at the information you intend to include in the body of the report:
 - Count the number of fields being used and estimate the appropriate spacing between fields.
 - Use rectangles to pencil in the fields within the estimated spacing.
 - Change the spacing if you need to.
 - Decide on a logical sequence for presenting the data in the body of the report.
 - Label the fields to indicate that sequence.
6. Use small boxes to indicate group values and totals.
7. Place random flags in the column where you want flags to appear.

8. Darken any elements you want highlighted to make them stand out from the rest of the prototype.
9. Review the finished product for layout and balance, and make changes as needed.

5 Introduction to Reporting

This section shows you the basics of creating a report in Crystal Reports. First you'll learn about the report design environment, then about how to select database tables, place objects on a report, and how to sort, group, and total your report data.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

5.1 Report creation options

Each time you create a new report, you have three options:

- Use a Report Creation Wizard.
- Use another report as a model.
- Create a report from scratch.

You will probably use each option at some time.

Report Creation Wizards

The Report Creation Wizards help create reports as quickly as possible and many new users and developers alike prefer to create the majority of their reports using them. All you have to do is choose the wizard that most closely matches your report type. The wizard walks you through the process of creating reports step-by-step.

Another report

To build a new report based on one that already exists, another report can be used as a model. Open the report you want to use in this way by selecting the Open File option and save it to a new file using Save As (found on the File menu). This method is useful to:

- Create a new report with a different grouping or different record selection than that of an existing report.
- Reconstruct a report based on an earlier time period using the same report structure used today.

- Create an entirely new report based on a set of databases that are linked in another report. You can create a report and delete the fields without disturbing the underlying links. Then, without relinking, you can build all your new reports based on this report.

Crystal Reports also lets you format a report by applying a template. See [Using a template \[page 236\]](#) for more information.

New report

The Blank Report option is used to create a report from scratch. This is useful when you want the full flexibility and control of building a report from the ground up, or when a report type is different from the many report types available in the wizards.

5.2 Choosing data sources and database fields

Crystal Reports makes it simple to select data sources and database fields by providing easy-to-use functionality in the Database Expert dialog box and the Field Explorer dialog box. Each of these dialog boxes uses the familiar Windows tree structure to allow you to navigate through the possible choices.

5.2.1 The Database Expert

The Database Expert provides an integrated tree view of all data sources you can use with Crystal Reports. In the Database Expert, you can select from the following as a data source for your report:

- A currently connected data source.
- An SQL command that has been saved to the BusinessObjects Enterprise Repository.
- A data source that has been added to your Favorites folder.
- A recently accessed data source (the Database Expert automatically maintains a list of such data sources for you).
- An existing data source (for example, a data file residing locally, or an ODBC data source that has already been set up).

In the Database Expert, you can also specify links between database tables when you have selected more than one table for your report.

The Database Expert is made up of two tabs: the Data tab and the Links tab.

5.2.1.1 Data tab

Tree View

The Data tab of the Database Expert shows a tree view of possible data sources you can select when creating a report. The tree—in the Available Data Sources list—is made up of folders for:

- My Connections
- Create New Connection

The Create New Connection folder contains subfolders for many popular data sources. Among these, you'll find:

- Database Files
- ODBC (RDO)
- OLAP
- OLE DB (ADO)

Note

The data source options available in the Create New Connection folder depend on the data access components selected during installation.

For a brief description of each of these folders and subfolders, see [Selecting the data source \[page 98\]](#).

Shortcut Menu

You can right-click any item in the Available Data Sources list of the Database Expert to see a shortcut menu with the following options:

- *Add to Report*
Use this option to add a table or stored procedure to your new report. This option is also available by clicking the > arrow on the Database Expert.
- *Add Command to Report*
Use this option to add a Table object that has been created using an SQL command.
- *Make New Connection*
Use this option to create a new connection.
- *Remove from Report*
Use this option to remove a table or stored procedure from your report. This option is also available by clicking the < arrow on the Database Expert.
- *Properties*
Use this option to obtain detailed information on the selected item.
- *Rename Connection*
Use this option to rename a data source connection.
- *Delete Connection*
Use this option to remove a data source connection.

Note

Database connections which have been connected to or referenced in an open report cannot be deleted.

- [Remove from repository](#)
Use this option to delete an existing SQL command from the SAP BusinessObjects Enterprise Repository.
- [Rename repository object](#)
Use this option to rename an existing SQL command in the SAP BusinessObjects Enterprise Repository.
- [Options](#)
Use this option to set the global options that appear on the Database tab of the Options dialog box.
- [Refresh](#)
Use this option to refresh the list of available data sources in the Database Expert.

5.2.1.2 Links tab

Database tables are linked so records from one database match related records from another. For example, if you activate a Suppliers table and a Product table, the databases are linked so that each product (from the Product table) can be matched up with the supplier (from the Supplier table) that made the product.

The majority of reports will probably require data from two or more tables, so linking will be necessary. The process of linking is made easy by using the Links tab of the Database Expert. See [Linking tables \[page 902\]](#).

5.2.2 The Field Explorer

Use the Field Explorer dialog box to insert, modify or delete fields on the Design and Preview tabs of Crystal Reports. To see the Field Explorer, select the Field Explorer command from the View menu.

5.2.2.1 Tree View

The Field Explorer shows a tree view of database fields and special fields that you can add to your report. It also shows formula fields, SQL expression fields, parameter fields, running total fields, and group name fields that you have defined for use in your report.

Fields that have already been added to the report, or fields that have been used by other fields (such as formula fields, groups, running total fields, summaries, and so on) have a green check mark next to them.

Find out more about the Field Explorer's [Toolbar and Shortcut Menu \[page 89\]](#) and about [Group Name Fields \[page 90\]](#).

5.2.2.2 Toolbar and Shortcut Menu

The Field Explorer's toolbar provides buttons with tool tips and hot-key combinations. You can right-click any item in the tree view to bring up a shortcut menu.

The toolbar offers these functions:

- *Insert to Report*
Use this option to add a field to the report. You can insert more than one field at a time by selecting multiple fields, right-clicking, and choosing Insert to Report.
Alternatively, to insert a field, you can drag and drop it in the Design or Preview tabs.
- *Browse*
Use this option to browse data for a database field, formula field or SQL expression field.

Note

On the shortcut menu, this option is called Browse Data.

- *New*
Use this option to create a formula field, SQL expression field, parameter field or running total field.
- *Edit*
Use this option to modify an existing formula field, SQL expression field, parameter field or running total field.
- *Duplicate*
Use this option to make a copy of the formula that you have selected in the Field Explorer. When the copy is created, you can rename it and edit it.
- *Rename*
Use this option to modify the name of an existing formula field, SQL expression field, parameter field or running total field.
- *Delete*
Use this option to remove a formula field, SQL expression field, parameter field or running total field. You can also select multiple fields, right-click, and choose Delete to remove them all at once.

The shortcut menus for the fields in the Field Explorer offer additional functions that are not on the toolbar, including the following options:

- *Find in Formulas*
Use this option to find occurrences of the selected item throughout all of the formulas in your report.
- *Set Prompt Order*
Use this option to open the Set Prompt Order dialog box, in which you can change the prompt order of your parameter fields.
- *Show Field Type*
Use this option to see the field type (string, number, and so on) when you're looking at a list of database fields. The length of string fields is included in brackets at the end of their names.
- *Sort Fields Alphabetically*
Use this option to display table fields in alphabetical order (instead of the order they appear in the table).
- *Sort Tables Alphabetically*
Use this option to display database tables in alphabetical order (instead of the order they appear in the database).
- *Refresh*
Use this option to refresh the list of available fields in the Field Explorer.

5.2.2.3 Group Name Fields

You can insert an existing Group Name field shown in the Field Explorer by right-clicking it and selecting Insert to Report. Unlike a formula field, parameter field or running total field, however, you cannot create a Group Name field through the Field Explorer. (A Group Name field is created when you insert a group.)

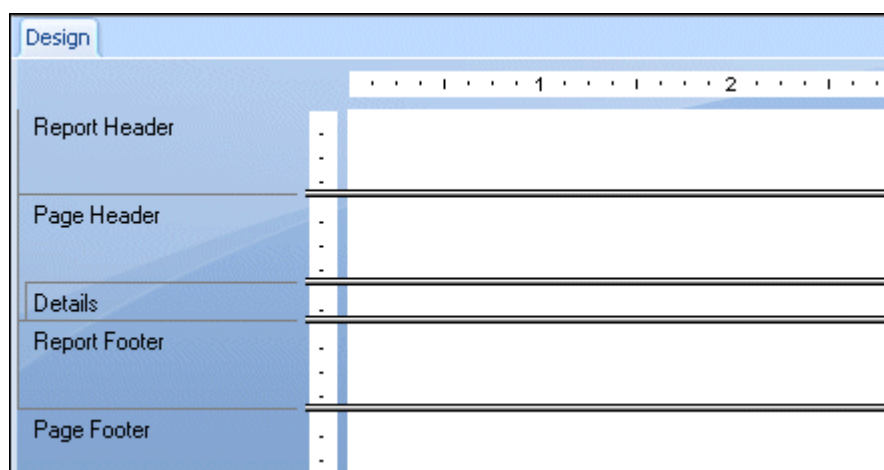
5.3 About the report design environment

Click a link to view one of the topics in this section:

- [Design tab \[page 90\]](#)
- [Preview tab \[page 93\]](#)
- [HTML Preview tab \[page 97\]](#)

5.3.1 Design tab

When working with Crystal Reports, you will probably use the Design tab more than any other part of the program.



The Design tab is the place you do most of the initial work when creating a report. It designates and labels the various sections of the report. You can do the initial formatting, place objects in the sections where you want them to appear, specify sorting, grouping, and totaling needs, and so forth.

The Design tab provides a very efficient environment for designing a report because you work in the tab with data representations, not with data itself. When a field is placed on the report, the program uses a frame to identify the field on the tab; it does not retrieve the data. Thus, you can add and delete fields and other objects, move them around, set up complex formulas, and more, without tying up the computer or network resources needed to gather the data.

The report created in the Design tab is a kind of virtual report; it has the structure and instructions for creating the final report, but it is not the report itself. To turn the Design tab report into a final report or into a report that you can fine-tune, you "just add data." You do this whenever you preview the report, print it, or output it in any other way. The actual data will now appear in the report.

5.3.1.1 Design tab areas

When you first begin creating a report, Crystal Reports automatically creates five areas in the Design tab.

- *Report Header*
This section is generally used for the report title and other information you want to appear at the beginning of the report. It can also be used for charts and cross-tabs that include data for the entire report.
- *Page Header*
This section is generally used for information that you want to appear at the top of each page. This can include such things as chapter names, the name of the document, and other similar information. This section can also be used to display field titles above the fields on a report.
- *Details*
This section is used for the body of the report, and is printed once per record. The bulk of the report data generally appears in this section.
- *Report Footer*
This section is used for information you want to appear only once at the end of the report (such as grand totals) and for charts and cross-tabs that include data for the entire report.
- *Page Footer*
This section usually contains the page number and any other information you want to appear on the bottom of each page.

If a group, summary, or subtotal is added to the report, the program creates two additional sections:

- *Group Header*
This section typically holds the group name field, and can be used to display charts or cross-tabs that include data specific to the group. It is printed once at the beginning of a group.
- *Group Footer*
This section generally holds the summary value, if any, and can be used to display charts or cross-tabs. It is printed once at the end of a group.

When a group, summary, or subtotal is added, the Group Header area appears directly above the Details area and the Group Footer area appears directly below the Details area.

If you set up additional groups, the program creates new group areas between the Details area and the existing Group Header and Group Footer area(s).

Like the original areas, each of these newly added areas can contain one or more sections. By default, they each contain a single section.

5.3.1.2 Identifying and working with areas and sections

By default, each area contains only a single section. The name for that section appears directly to the left of the section. If you have multiple sections in an area, the sections are designated as a, b, c, and so forth.

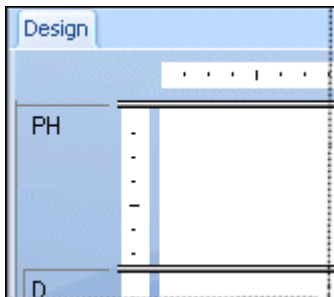
📌 Note

Initials, such as RH, PH, D, PF, RF, and so on, are used to identify each section if you have selected the Short Section Names check box in the Design View area of the Options dialog box.

Note

If you right-click the shaded area containing a section name, a shortcut menu appears with section-specific options.

If you have selected the Show Rulers options on the Layout tab of the Options dialog box, the program displays a section ruler immediately to the left of each section. The section ruler is used to add, remove, and move guidelines, and to provide a visual reference when you are placing objects. See [Designing with guidelines \[page 255\]](#).



Whenever a new section is added, the program creates a ruler for that section. See [Using multiple sections in reports \[page 222\]](#).

5.3.1.3 Other Design tab capabilities

There are several other capabilities built into the Design tab. With the Design tab, you can:

- Resize a section by dragging its boundary. See [Resizing a section \[page 222\]](#).
- Split a section (create two sections from one) by clicking its left boundary. See [Splitting a section \[page 221\]](#).
- Add horizontal and vertical guidelines by clicking the rulers. See [Designing with guidelines \[page 255\]](#).
- Zoom in and out on a report at any magnification from 25% to 400% of the original size. See [Using the zoom feature \[page 111\]](#).

5.3.2 Preview tab



To preview a report before printing it, click Print Preview on the Standard toolbar.

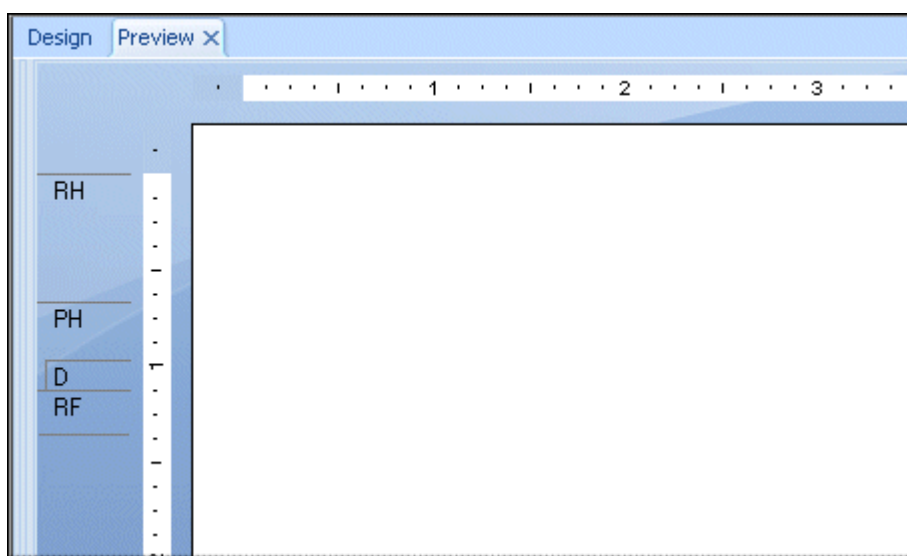
The program gathers the data, makes the necessary calculations, and displays the report in the Preview tab. With the data in place, you can review the spacing and formatting of your report and see the actual results of all your summaries, formula calculations, and record and group selections.

In true WYSIWYG (What You See Is What You Get) fashion, you can work directly on this live data, fine-tuning it until the report has the exact look you want.

The program works with data in the following manner:

- The first time the Preview tab is used, it retrieves data from your underlying data source(s) and saves it with the report (unless you have set up the program not to save data).
- From that point on, the program uses the saved data whenever you preview the report unless you specifically refresh it or add a field that requires the program to retrieve new data.

5.3.2.1 Standard view



In standard view, the report is displayed a page at a time. Using the navigation buttons in the Preview tab, you can move to the beginning or end of the report, or you can move backward and forward through the report one page at a time. For shorter reports or reports in which you're primarily interested in seeing the "bottom line" totals, the standard view provides all of the functionality you need.

The Data Age indicator

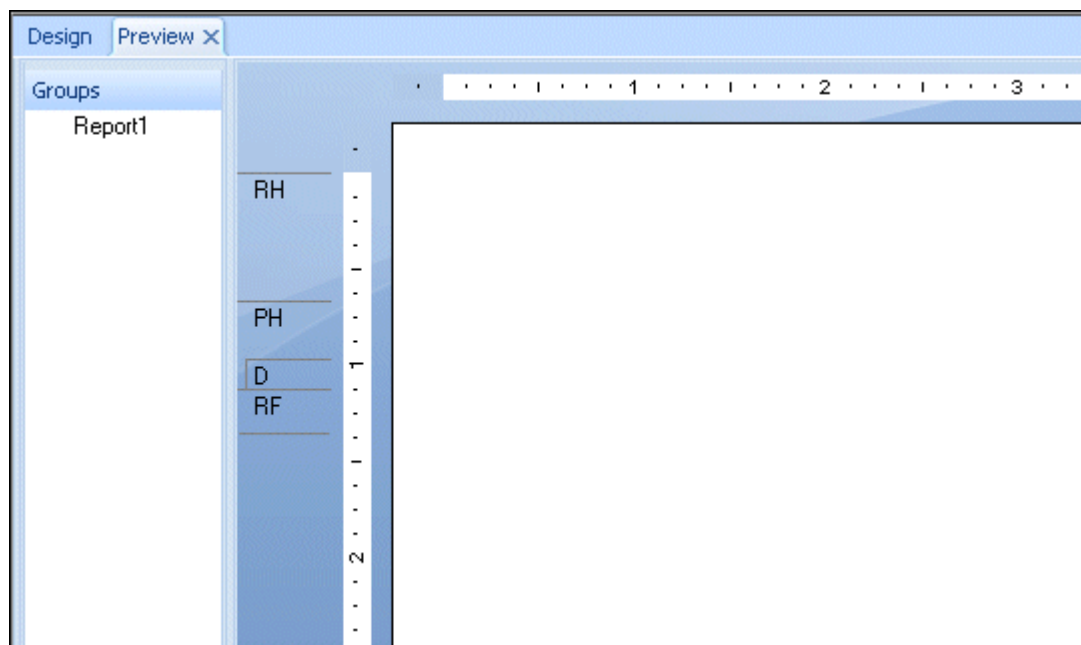
The Data Age indicator indicates the date the data was last refreshed or initially retrieved, whichever is the most recent. If the data was initially retrieved or refreshed today, it indicates the time it happened.

1/16/2002 11:32

5.3.2.2 Preview Panel

The *Preview Panel* houses the *Group Tree*, the *Parameter Panel*, and a *Find* tab.

5.3.2.2.1 Group Tree



The *Group Tree* can be shown or hidden using the *Toggle Preview Panel* button on the *Standard toolbar*.

The *Group Tree* presents a split screen:

- The right pane displays the report.
- The left pane displays a high level outline of the report, showing the hierarchy of groups and subgroups in a familiar tree format.

The *Group Tree* normally displays the names of the groups and subgroups you created in your report. You can, however, customize these names using the *Options* tab of the *Insert Group* or *Change Group Options* dialog box. For more information on customizing group names in the *Group Tree*, see [Grouping data \[page 173\]](#).

When you click the tree node for the group that interests you, the program jumps immediately to the part of the report that contains the information for that group. For longer reports or reports in which you wish to jump back and forth between different groups, the smart navigation features of the *Group Tree* make your work extremely efficient.

5.3.2.2.2 The Parameter Panel

The *Parameter Panel* lets users interactively format and filter report data by changing parameter values. It is located within the *Preview Panel*.

In the *Create New Parameter* and *Edit Parameter* dialog boxes, you can specify parameters to display on the Parameter Panel via the *Show on (Viewer) Panel* option. You can choose one of several settings:

- *Do not show*
The parameter is not visible on the panel. Users must refresh the report to change the parameter values.
- *Editable*
Users can view and change the parameter values on the panel.
- *Read only*
Users can view the parameter values on the panel, but cannot change them.

Note

Do not show is the default setting for a new parameter.

The *Parameter Panel* displays all parameters that have been set to *Editable* or *Read only*. Depending on the type of parameter, users can add, delete, or modify parameter values either by using the dropdown menus and applying their changes or by clicking the *Advanced Dialog* button.

The following are some of the more commonly used types of parameters that must be modified by clicking the *Advanced Dialog* button:

- Parameters with ranged values.
- Dynamic parameters that use cascading prompts.

On the panel, data parameters are differentiated from non-data parameters with a database icon. When a user modifies the value of a data parameter, the report refreshes and accesses the database to make the corresponding changes. When a user modifies the value of a non-data parameter, the report is formatted or filtered based on saved data within the report. It does not access the database.

5.3.2.2.3 Find tab

Use the *Find* tab to search your report for specific words or phrases.

Click the magnifying-glass button, or press *Enter*, to start the search. If you want to narrow your result set, click the down arrow to select *Match Case* or *Match Whole Word Only*.

Each result in the list displays the location in the Group Tree where it is found. Double-click a result to go to that spot in the report. You can click the *Stop* button while you are searching. Switching to a different report or tab while searching also stops the search.

5.3.2.3 Comparisons with the Design tab

You have the same formatting capabilities in the Preview tab as you do in the Design tab. Menus (both menu bar and shortcut menus) and toolbars remain active, providing essentially the same functionality you have when working with a report in the Design tab. However, when you are making numerous changes, it is quicker to make the changes in the Design tab. Some additional things to consider are:

- The Design tab and Preview tab are tied together internally. Any changes made in one are reflected in the other.
- The Preview tab has a single vertical ruler at the left of the tab rather than the individual section rulers seen in the Design tab. The functionality of the ruler is the same.

- The Preview tab identifies report sections in the shaded area to the left of the data. With a quick look you can tell which report section the data is printing from. While section names appear only once in the Design tab, they print each time a section prints in the Preview tab.
- The Record counter, the Data Age indicator (see [Standard view \[page 94\]](#)), and the Page Forward/Page Back controls (see [Preview tab \[page 93\]](#)), are all active in the Preview tab.
- The Preview tab highlights every value when you select a field; whereas, only the field frame is highlighted in the Design tab.

Working in the Preview tab has a different feel from working in the Design tab.

Each field in a database contains dozens, hundreds, or even thousands of values, depending on the number of records in the database. When you place a field in the Design tab, a single field frame represents all those values. When you highlight the field, sizing handles appear on the frame and the frame changes color.

In the Preview tab, however, you are working with the actual data. Instead of a field frame representing many field values, the values themselves appear. Some additional things to consider are:

- When you highlight a field or formula field value, you are actually selecting every value in the field:
 - The program places a sizing frame around the specific value you select.
 - It highlights every other value in the field.
- Likewise, when you select a summary value, you are actually selecting all the related summary values:
 - The program places a sizing frame around the specific value you select.
 - It highlights all the related summary values.

Aside from the differences in appearance, the process of building and modifying a report is the same in both the Design tab and the Preview tab. You should find it easy to work with your reports in both places.

5.3.3 HTML Preview tab

To see an HTML rendering of your Crystal report as it will appear when published to the web, click  ([HTML Preview](#)) on the Standard toolbar.

When you need to make sure that your report design is appropriate and correct over the web in a zero-client environment, the HTML Preview option lets you see an instant rendering of your report to HTML without leaving the Crystal Reports design environment. Unlike the Preview tab, which shows the true report format, the HTML Preview tab shows a converted format. By switching between the two tabs, you can make adjustments in your report design to yield the best results on the web.

In the case of reports that are published to SAP BusinessObjects Business Intelligence platform (managed reports), you don't have to do any special configuration to make the HTML Preview feature work.

Note

The guest account in SAP BusinessObjects Business Intelligence platform must be enabled before you can use the HTML Preview feature.

For stand-alone reports (unmanaged reports), the program must use the Report Application Server (RAS) to generate the preview. In this case, you need to configure the HTML Preview feature. Do this in the Smart Tag & HTML Preview tab of the Options dialog box in Crystal Reports.

For information about how to configure the HTML Preview Options, search the Crystal Reports online help for "Smart Tag & HTML Preview tab (Options dialog box)".

5.4 Creating a new report

This section guides you through several aspects of the report creation process. Use the Contents tab of this online help to navigate to a particular topic.

5.4.1 Selecting the data source

After deciding which option you want to use for creating your report (see [Report creation options \[page 85\]](#)), the next step is to select a data source to use.

Most data sources can be chosen through the Database Expert dialog box. The Database Expert appears when you create a report from scratch using Blank Report, or when you choose Database Expert from the Database menu.

Note

You also select a data source in the Report Creation Wizards. The Data screen in all of the Report Creation Wizards, except the OLAP Report Creation Wizard, is much like the Database Expert dialog box.

5.4.1.1 To select a data source

Procedure

1. Choose [Database Expert](#) from the [Database](#) menu.

The Database Expert dialog box appears.

2. Use the tree view in the Available Data Sources list of the Data screen to select your data source:

- [My Connections](#)
This folder shows a list of data sources you are currently connected to.
- [Create New Connection](#)
This folder shows subfolders for various data sources you can connect to.

Some popular choices in the Create New Connection folder are described here:

- [Database Files](#)
This option shows a list of standard PC databases that reside locally. You can use Find Database File to browse for a PC database using the Open dialog box.
- [ODBC \(RDO\)](#)
This option shows a list of ODBC data sources you have already configured for use.

- [OLAP](#)
This option opens the OLAP Connection Browser so you can choose an OLAP cube as a data source.
- [OLE DB \(ADO\)](#)
This option shows a list of OLE DB providers you have already configured for use. You can also specify a Microsoft Data Link file to use. For some of the OLE DB providers, you will have the option to connect using [Integrated Security](#). No user ID or password is required with this option, but it does require that Database Authentication is enabled on the server. Please consult your Database Administrator on how to enable and configure this function.
- [Repository](#)
This folder shows you the contents of your repository through the BusinessObjects Enterprise Explorer.

Note

The data source options available in the Create New Connections folder depend on the data access components selected during installation.

5.4.2 Adding tables

After selecting the data source, you can add one or more tables to base your report on.

5.4.2.1 To add a table

Procedure

1. Choose [Database Expert](#) from the [Database](#) menu.
The Database Expert dialog box appears.
2. On the [Data](#) tab, search for the database you want to use in your report. See [Selecting the data source \[page 98\]](#).
3. In the Available Data Sources list, select the table you want to add to your report and click the > arrow to add it to the Selected Tables list.

You can insert more than one table at a time by selecting multiple tables, right-clicking, and choosing Add to Report. You can also drag and drop tables to the Selected Tables list.

5.4.3 Linking multiple tables

If the report contains data from two or more database tables, they need to be linked at this point when creating reports.

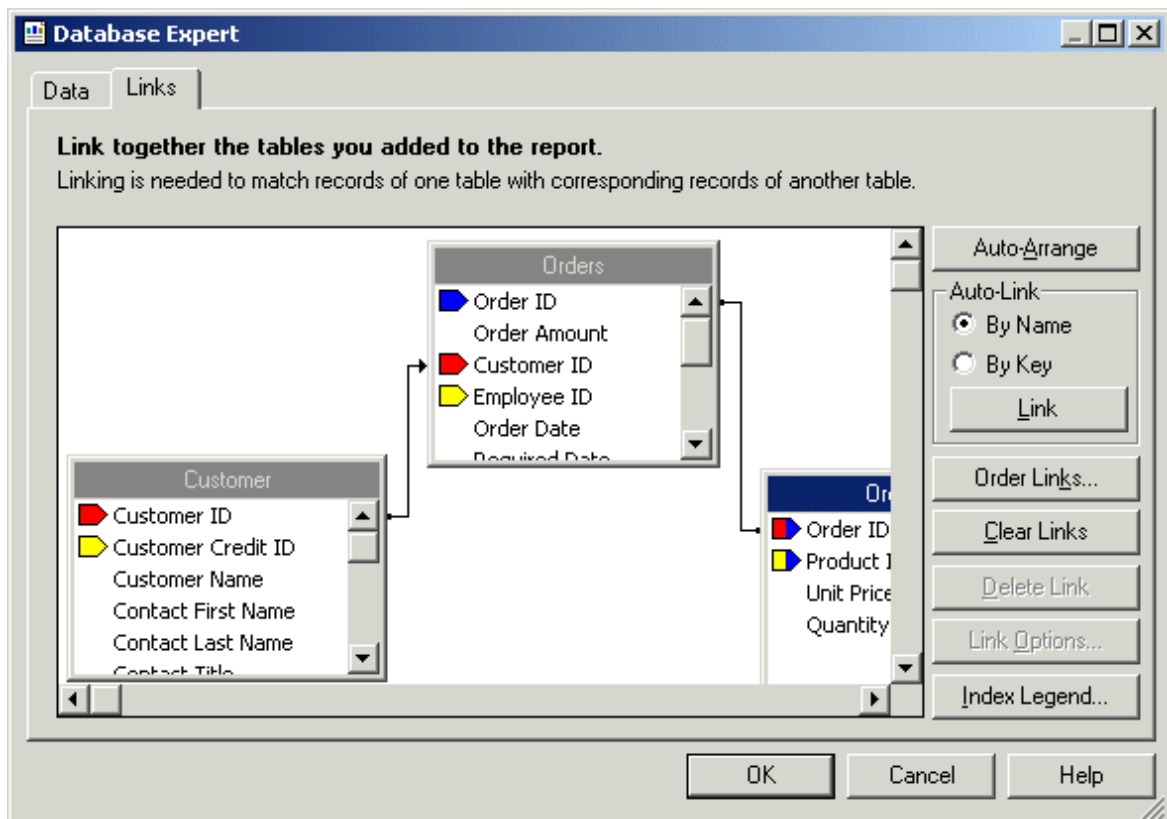
Note

It isn't necessary to link tables in reports created from a query or command because any links required by the data have already been processed.

5.4.3.1 To add and link multiple tables

Procedure

1. Choose *Database Expert* from the *Database* menu.
The Database Expert dialog box appears.
2. On the Data tab, select the tables you want to add to your report. See [Adding tables \[page 99\]](#).
The Links tab appears in the Database Expert.
3. Click the *Links* tab to display the databases currently available for linking.
Crystal Reports automatically links tables by name or key when possible.



4. To create links manually, drag a field from one table to a field in another table. If successful, a link line is created. If unsuccessful, a message is issued.

Note

You can link tables by table name or by foreign key information.

5. If you've deleted links and you want to recreate them automatically, click [Link](#).
6. Click [OK](#) when finished.

Note

When manually creating links, the field you are linking "to" must be of the same data type as the field you are linking from. When a native connection is used, the field you are linking "to" does not have to be indexed. For more information, see [Indexed tables \[page 887\]](#).

The Database Expert closes, and you are returned to your report. The linked databases are now available for use in your report. If you are not satisfied with the link, you can modify it using the Links tab of the Database Expert.

Related Information

[Understanding Databases \[page 885\]](#)

[Linking options \[page 911\]](#)

5.4.4 Placing data on the report

Placing data on a report is a very important task. You need to know what type of data should be placed on the report and where on the report it should be placed.

Note

A report (.rpt) file might contain sensitive business data. You are advised to protect access to the file by using one of these methods:

- Manage the file with SAP BusinessObjects Business Intelligence platform server.
- Apply file security.
- Apply operating system-level encryption.

5.4.4.1 Database fields

Much of the data placed on a report are database fields, displaying data as it is stored in the database. Normally, database fields will be placed in the Detail section, but under certain circumstances, they will be placed in other sections of the report.

5.4.4.1.1 To insert a database field

Procedure

1.  On the Standard toolbar, click *Field Explorer*.

The Field Explorer dialog box appears. To speed the report building process, this dialog box remains on-screen until you close it. You can move this dialog box wherever you wish.

2. Expand the *Database Fields* folder to see all the tables chosen from the database(s).
3. Expand the individual tables to see all the fields they contain.
4. Select the field you want to appear in the report.
5. Click *Browse* to review the values in the selected field.
6. Click *Insert to Report* to place it in the report.

5.4.4.2 Formula fields

To display data that is a calculated value, you need to create a formula field and place that formula field on the report. For example, if the database only stores the order dates and ship dates for orders but you need to display the number of days it takes to ship the order, you must create a formula field that will calculate the number of days between ordering and shipping. This is just one example of the use of formula fields. See [Using Formulas \[page 687\]](#), for an introduction to formulas.

5.4.4.3 SQL Expression fields

SQL expressions are like formulas, but they are written in Structured Query Language (SQL), not in the Crystal Reports formula language. An SQL expression can be used to query the database for specific sets of data. You can sort, group, and select based on SQL expression fields.

5.4.4.3.1 To create an SQL Expression field

Procedure

1.  On the Standard toolbar, click *Field Explorer*.

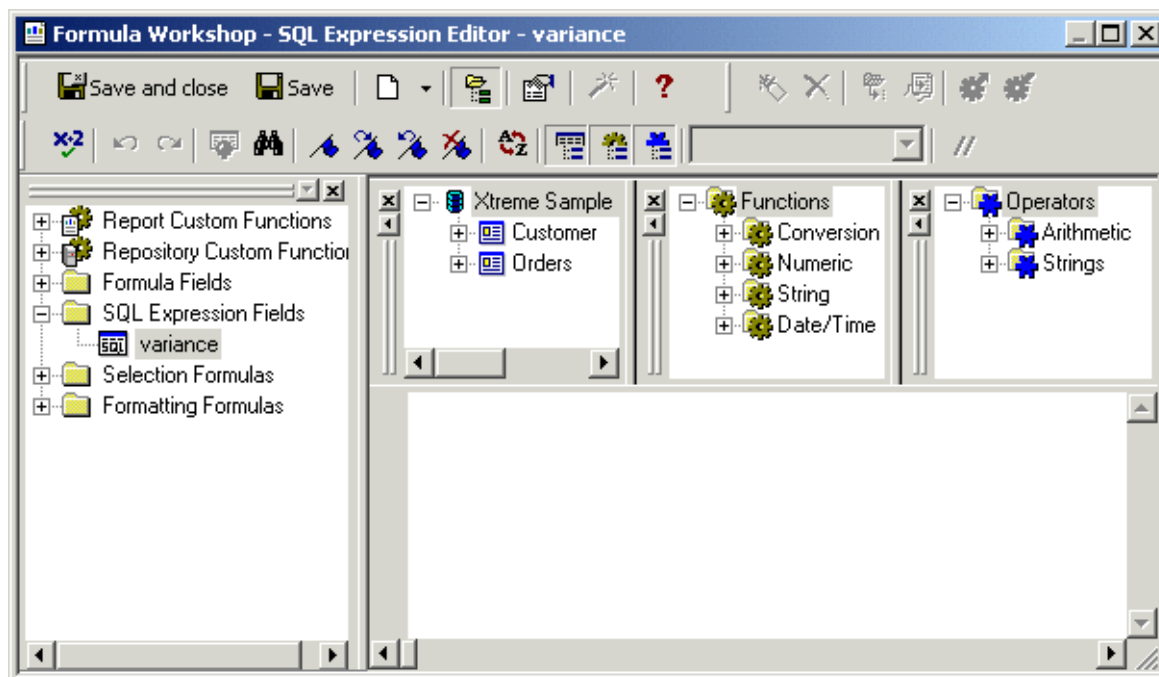
The Field Explorer dialog box appears.

2. Scroll down to *SQL Expression Fields* and highlight it. Click *New*.

The SQL Expression Name dialog box appears.

3. Enter a name in the [Name](#) box, and then click [OK](#).

The Formula Workshop appears with the SQL Expression Editor active.



4. Type the expression in the SQL Expression Editor.

Note

For an overview of the formula language, see [Formula components and syntax \[page 689\]](#).

5. Click [Save](#).

5.4.4.4 Parameter fields

To prompt the user of a report to enter information, create a parameter field. Think of a parameter as a question that the user needs to answer before the report is generated. The information users enter, or the way they respond, determines what appears in the report. For example, in a report used by salespeople, there might be a parameter that asks the user to choose a region. The report would return the results for the specific region, instead of returning the results for all of the regions. See [Parameter Fields and Prompts \[page 831\]](#) for an introduction to parameter fields.

5.4.4.5 Running total fields

To display a total that evaluates each record and provides a running sum of all the values in a field (or all the values in a certain set of values), a running total field needs to be created and placed in the report. If the first

three values in a field were 2, 4, and 6, a running total would print 2, and then 6 (the sum of 2 + 4), and then 12 (the sum of 2 + 4 + 6). See [Running Totals \[page 208\]](#), for an introduction to running totals.

5.4.4.6 Special fields

To display information such as Page Numbers, Print Date, and Report Comments use the commands in the Special Fields tree view of the Field Explorer dialog box.

5.4.4.6.1 To insert a special field

Context

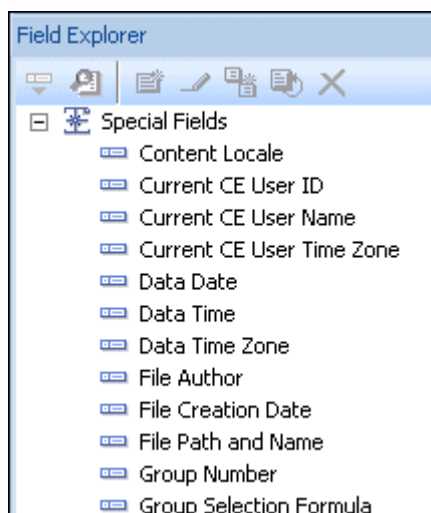
Crystal Reports allows you to easily insert Page Number, Record Number, Group Number, Print Date, and Total Page Count fields, among others, into your report.

Procedure

1.  On the Standard toolbar, click *Field Explorer*.

The Field Explorer dialog box appears.

2. Scroll down to *Special Fields* and expand it by clicking.



3. Choose a command from the Special Fields list to insert in the report.

Each special field is inserted into the report as an object. An object frame appears. You can now place it on the report.

Results

Note



To change the formatting of an inserted object, click the object to select it and click **Format** on the Expert Tools toolbar. The Format Editor appears where you can make the desired changes. See [Formatting \[page 235\]](#).

5.4.4.7 Text objects

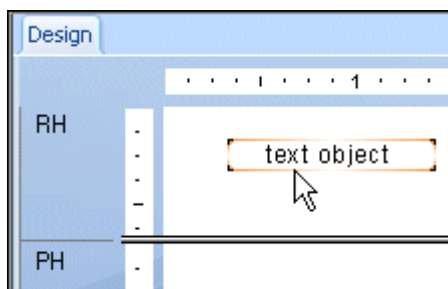
Text objects are used in reports for a multitude of purposes. They are a powerful way of inserting titles; labeling summaries and other data on your report; and of easily combining database fields.

5.4.4.7.1 To insert a text object

Procedure

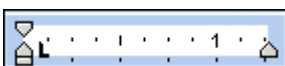


1. On the Insert Tools toolbar, click *Insert Text Object*. A cursor appears.
2. Position the cursor where you want the text object to appear in the report.



Click once on the border of the text object to select it for resizing and moving.

Double-click inside the text object to select it for editing. The Design tab ruler changes to a text object ruler sized to the length of the selected object. To the left, a tab indicator appears. By clicking the tab indicator, you can cycle through the four tab options available. Once you have chosen the desired tab, click the position on the ruler where you want to insert it.



The ruler allows you to add indents and align text within the text object.

Results

Note

When you first insert the text object into the report, the object is automatically selected for editing.

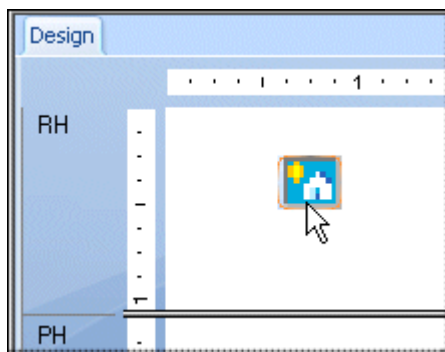
5.4.4.8 Picture fields

When designing reports, there will be times when you want to include a picture. For example, you may wish to put a company logo in the report header.

5.4.4.8.1 To insert a picture

Procedure

1.  On the Insert Tools toolbar, click *Insert Picture*.
The Open dialog box appears.
2. Select the desired picture file from the file list and click *Open* to return to the report.
An object frame appears with the picture inside, ready to be positioned.



3. Position the picture object where you want it to appear in the report and left-click once.

5.4.4.9 Hyperlink fields

You can select a report object on the Design or Preview tab and create a hyperlink to another location.

The hyperlink is saved with your report and is available to other users as a way of viewing additional information.

📘 Note

Crystal Reports lets you create hyperlinks with Relative URLs, so your web reports retain independence from their location on any particular server.

📘 Note

You can use the option [A Website on the Internet](#) to connect to SAP BusinessObjects, Web Intelligence, and Analysis workspace documents (this is also known as report linking). Click [Create enterprise hyperlink](#) to create these hyperlinks in Crystal Reports.

5.4.4.9.1 To create a hyperlink field

Procedure

1. Select a report object on the [Design](#) or [Preview](#) tab.
2. On the [Format](#) menu, click [Hyperlink](#).

The Format Editor dialog box appears with the Hyperlink tab active.

3. Click the type of hyperlink you want to create.

The available types in the upper area are:

- [No Hyperlink](#)
This is the default option. There is no hyperlink associated with the selected report object.
- [A Website on the Internet](#)
Select this option if you want your report object to be linked to a static web address. Use the formula button to create a URL based on a field value. For example, you might enter the following formula if your Customer Name field contained information that would create a series of meaningful URLs:
"http://www." + {Customer.Customer Name} + ".com"

📘 Note

You can also use this option to connect to SAP BusinessObjects, Web Intelligence, and Analysis workspace documents (this is also known as report linking). Click [Create enterprise hyperlink](#) to create these hyperlinks in Crystal Reports.

→ Tip

Alternatively, you can use the openDocument function in the [Formula Workshop](#) to create hyperlinks.

- [Current Website Field Value](#)
Select this option if you want the program to create a hyperlink out of the field you selected. The field must be stored as a proper hyperlink in your data source.
- [An E-mail Address](#)
Select this option if you want to create a "mailto" address from the field you selected. Use the formula button to create an address based on a field value.

- [A File](#)
Select this option to create a hyperlink to a file on a specific computer or networked computer. Use the formula button to create a file path based on a field value.
You can also use a hyperlink of this type to run an EXE with a command-line parameter. Enter the path and application name, and follow it by the parameter for the command.
- [Current E-mail Field Value](#)
Select this option if you want the program to create an email hyperlink out of the field you selected.
The field must be stored as a proper email address in your data source.

The available types in the DHTML Viewer Only area are:

- [Report Part Drilldown](#)
For information about how to use this option, see [The Report Part Drilldown option \[page 634\]](#).
- [Another Report Object](#)
For information about how to use this option, see [The Another Report Object option \[page 637\]](#).

Note

Not all hyperlink types are available at all times. The object you select and its location on the report determine which types are available.

4. After you have chosen a hyperlink type, enter the appropriate hyperlink information (the URL of a web site, for example).
5. Click [OK](#) when you are finished.

Results

The hyperlink is inserted as appropriate. Click it on the report to go to the web site, to send an e-mail, and so on. For information about how to use the hyperlink types in the DHTML Viewer Only area, see [Setting up navigation \[page 632\]](#).

5.4.5 BLOB (Binary Large Object) fields

A BLOB field is a database field whose data consists of Binary Large Objects—such as bitmap graphics, images, OLE objects, metafiles, and so on. Inserting a BLOB field into your report allows you to access these binary objects as you would other data types.

Note

Crystal Reports can also access BLOB objects by reference (that is, dynamically through a file path), so that you do not need to store the BLOB objects in your database. For more information, see [Making a static OLE object dynamic \[page 340\]](#).

In general, Crystal Reports allows you to access BLOB fields containing:

- Device-independent bitmaps (DIB).
- JPEG or PNG images.

In addition, if your data resides in a Microsoft Access database, then Crystal Reports enables you to report on BLOB fields containing OLE 1 and 2 objects and metafiles.

You insert BLOB fields as you would any other database field. For details, see [To insert a database field \[page 102\]](#).

A BLOB field object differs from other database field objects in that it offers options to control cropping, scaling, and sizing—just like an inserted picture or OLE object. To access these options, right-click the BLOB field object, select Format Graphic from the shortcut menu, and click the Picture tab.

Note

To retain the ability to activate and edit an OLE object with its server application, you should insert the object into your report independently, either by linking or by embedding, rather than inserting it in a BLOB field. For more information about inserting linked and embedded OLE objects, see [Inserting OLE objects into reports \[page 337\]](#).

5.4.6 Formatting data

At this point in creating a report, you may want to do some basic formatting. Perhaps you would like to change the font size and style of a text object used as a title. Or, if you have a number field, such as a sales figure, you might want to place a dollar sign before the number or change the number of decimal places displayed.

5.4.7 Record selection

Record selection, the task of paring down the data to include only the data required for your report, is a crucial step in report creation. You will rarely want a listing of all the information in a database. Most often you will be interested in only the sales in a given time period or for a certain product, and so on. For example, a sales report may be designed to only include sales from one product line for the last calendar month.

5.4.8 Grouping, sorting, and summarizing data

Once a basic report is created, you will want to organize the data by grouping related information, sorting individual records, summarizing, subtotalling, and grand totaling.

Grouping records

To organize the data, you may want to group related data together. For example, after grouping a customer list by region, you might divide the list into regional groups. That way, a sales manager for one region could quickly locate the appropriate region group and focus exclusively on the customers within that region.

Sorting records

Crystal Reports allows you to specify the order in which you want the records on your report displayed. For example, after grouping by region, you might sort the records within each region in alphabetic order by Customer Name. Many of your reports will use some type of sorting. Depending on the report, you will sort the records in a list or sort in conjunction with grouping.

Summaries, subtotals, and grand totals

Many reports use some sort of totaling. For example, in a North American sales report grouped by state, you might want to calculate the total dollar amount sold in each state. You do this by creating a subtotal on the sales field. Summaries are also used at the group level, allowing you to calculate averages, counts, and other group (aggregate) values. For example, in a sales report you may want to calculate an average of sales per state (average summary on the sales field) and calculate the number of products sold in the state (distinct count of the product name field).

5.4.9 Using the drill-down option on summarized data

You can drill down on your data, to show the data behind individual groups, using the Drill-down cursor. See [Sorting, Grouping, and Totaling \[page 167\]](#).

5.4.9.1 Drill-down cursor

Crystal Reports allows you to drill down on group or summary information in the Preview tab in both the Standard and the Group Tree. When you position the cursor over any summary value that you can drill down on, the program displays a Drill-down cursor.

→ Tip

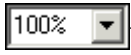
Group headers appear on the Drill-down tab just as they do in the main report itself.

If you then double-click, the program reveals the details behind that specific summary value. For example, if the Drill-down cursor becomes active over the city summary, you can double-click to see the details behind that summary.

- If you have only a single summary, you can look at the summary or at the data from the individual records that are summarized.
- If you have multiple summaries, you can look at the summaries behind summaries (the city summaries that make up the region summaries, for example), or at the data from the individual records that are summarized.

5.4.10 Using the zoom feature

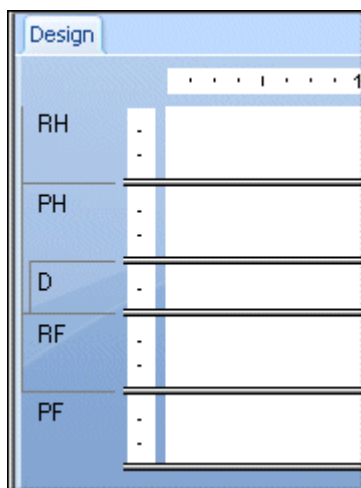
You can easily zoom in on a report. You can choose any magnification from 25% to 400%. This feature is active in both the Design and Preview tabs.

 To zoom the report in or out, set the zoom level in the Zoom Control box found on the Standard toolbar.

It is helpful to view reports at low magnifications in order to get an overall picture of the layout of your report. Views at higher magnifications focus attention on the details of the report.

5.4.11 Inserting page headers and footers

You can use page headers and footers by placing the information in the Page Header or Page Footer sections of the Design tab.



- Information to appear only on the first page of the report goes in the Report Header (RH).
- Information to appear only on the last page of the report goes in the Report Footer (RF).
- Information to appear at the top of every page goes in the Page Header (PH).
- Information to appear at the bottom of every page goes in the Page Footer (PF).

Text, fields, or formulas can be used in these sections just as in the Details section.

Related Information

[Creating footers after the first page \[page 293\]](#)

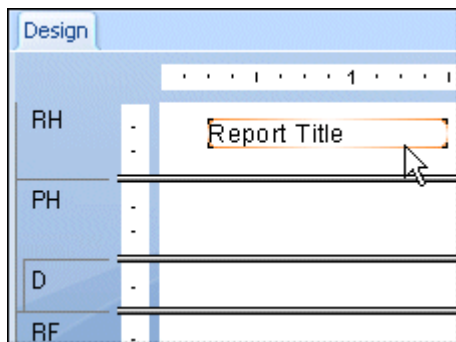
5.4.12 Adding a title page to the report

Crystal Reports provides a quick, easy way to add a title page to a report by selecting Report Title from the Special Fields in the Field Explorer dialog box. In order to use this field, you must have a title entered in the Summary tab of the Document Properties dialog box. See [Adding summary information to the report \[page 113\]](#).

5.4.12.1 To add a report title

Procedure

1.  On the Standard toolbar, click *Field Explorer*.
The Field Explorer dialog box appears.
2. Scroll down to *Special Fields* and expand it by clicking.
3. Select *Report Title*, and click *Insert to Report*. An object frame appears when the cursor is moved over the report.



4. Move the object frame to the Report Header section and click once to place the frame.
5.  With the report title selected, click *Section Expert* on the Expert Tools toolbar.
The Section Expert appears.
6. With the Report Header section highlighted, select the *New Page After* check box.

Results

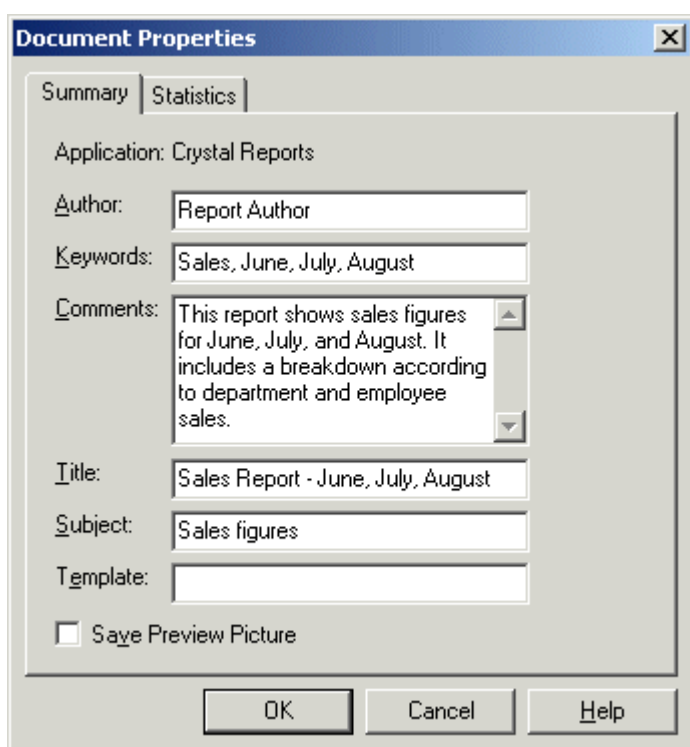
Now the title will appear on the first page and the report will begin on the second page.

5.4.13 Adding summary information to the report

There may be times when you want to include non-printing comments with a report (a personal note to the report recipient, a note to explain more thoroughly the data on which the report is based, a report title, a comment about some particular data on the report, and so on).

Summary Info on the File menu provides a facility for including anything from a short note to hundreds of lines of text with your report. The comments do not print with the report; they remain in the Summary tab of the Document Properties dialog box where they can be reviewed on demand.

When you choose Summary Info, the Document Properties dialog box appears with the Summary tab active.



The screenshot shows the 'Document Properties' dialog box with the 'Summary' tab selected. The 'Application' is 'Crystal Reports'. The 'Author' field contains 'Report Author'. The 'Keywords' field contains 'Sales, June, July, August'. The 'Comments' field contains a multi-line text entry: 'This report shows sales figures for June, July, and August. It includes a breakdown according to department and employee sales.' The 'Title' field contains 'Sales Report - June, July, August'. The 'Subject' field contains 'Sales figures'. The 'Template' field is empty. There is an unchecked checkbox for 'Save Preview Picture'. At the bottom are 'OK', 'Cancel', and 'Help' buttons.

Enter the desired information and click OK when finished to return to your report.

5.5 Exploring reports and working with multiple reports

Crystal Reports lets you work with your existing reports through the Report Explorer, the Workbench, and the Dependency Checker.

This section describes these tools and provides some general information about opening and docking each of the explorers that are available in Crystal Reports. For more information, see [Opening and docking explorers \[page 117\]](#).

5.5.1 The Report Explorer

The content of the Report Explorer represents the content of the report in a tree view. The root node is the report itself, while the first-level nodes represent the report's sections. Within each section, the report's fields and objects are listed. Any item you select in the tree view will be selected in the report (in either Design or Preview modes).

You can modify report fields and objects by selecting them in the Report Explorer. When you right-click the selected item, you see a menu that contains the actions you can carry out on the item. For example, if you right-click a text object, the menu will contain the options to edit the text, to format its font or color, and so on. Likewise, if you right-click a section node, the menu will contain the options to hide, suppress, format and so on.

Note

You cannot add additional fields or report objects when using the Report Explorer, but you can delete them.

You can select multiple fields for formatting by using *Shift-click* or *Ctrl-click*.

5.5.1.1 Toolbar

The Report Explorer's toolbar provides buttons with tool tips. These buttons let you expand or collapse your report's tree view and choose the report items to display by type:

-  *Expand*

Use this option to expand all child nodes of the tree view under the selected node. If you select this option while the root node is highlighted, all remaining nodes are expanded. If you select this option while a child node is highlighted, only the nodes that are part of the child are expanded.

Note

This option has no effect if selected while a report object is highlighted, or if the selected node has no children.

-  *Show/Hide Data Fields*

Use this option to show or hide data fields in your report. Data fields include formula fields, summary fields, and text objects.

-  *Show/Hide Graphical Objects*

Use this option to show or hide graphical objects in your report. Graphical objects include charts, maps, boxes, lines, BLOB fields, and OLE objects.

-  *Show/Hide Grids and Subreports*

Use this option to show or hide cross-tabs, OLAP grids, and subreports in your report.

Note



Items connected to an SAP BusinessObjects Enterprise Repository are shown with the repository icon beside them.

Note

Items with hyperlinks are shown as blue text with a solid underline. Items with Report Part hyperlinks are shown as green text with a dashed underline.

5.5.2 The Workbench

In the Workbench, you can create projects that contain one or more reports. Use the options on the toolbar to add, remove, or rename folders, reports, and object packages. You can reorganize the files in a folder or folders by dragging and dropping them where you want them to appear. You can also drag report files from Windows Explorer and drop them into the folder of your choice in the Workbench.

5.5.2.1 Toolbar

The toolbar in the Workbench provides the following options:

-  [Add Report](#)
Use this option to add a new object to the Workbench:
 - [Add Existing Report](#)
When you select this option, the Open dialog box appears so that you can search for a Crystal report that you have previously created.
 - [Add Current Report](#)
When you select this option, the program creates a root node that is called Untitled Project. It then adds the current active report to that project.
-  Note

This option is available only when there is an active report.
- [Add New Project](#)
When you select this option, the program adds a new folder under the root node that is called Untitled Project. You can add reports to this folder. Project folders offer a way to organize the reports in the Workbench.
 -  [Open](#)
Use this option to open the report or report package that you have selected in the project folders.
 -  [Check Dependencies](#)
Use this option to initiate the Dependency Checker. The option is available for individual reports or for folders of reports. Crystal Reports begins checking the report(s) immediately and displays the results in the Dependency Checker. For more information, see [The Dependency Checker \[page 116\]](#).

5.5.2.2 Shortcut Menus

You can right-click any item in the projects area of the Workbench to see a shortcut menu. Many of the options that are available on the shortcut menu are also available on the toolbar. This section describes only the additional options that are not available on the toolbar.

- [Add Object Package](#)
When you select this option, the program prompts you to log on to your SAP BusinessObjects Business Intelligence platform system. When you have connected to your system, you can choose an object package to add to the Workbench. For more information about object packages, see the *SAP BusinessObjects Business Intelligence Platform Administrator Guide*.
- [Publish to SAP BusinessObjects Business Intelligence platform](#)
Use this option to publish a folder of reports to SAP BusinessObjects Business Intelligence platform. You can publish your reports individually or as an object package. (Objects packages can be scheduled in SAP BusinessObjects Business Intelligence platform as a single entity.) When you choose this option, the [Publish Object](#) dialog box appears.
For information about using this dialog box, search the Crystal Reports online help for “Publish Object dialog box”.
- [Remove](#)
Use this option to remove a report, report package, or folder from the projects area.

Note

You cannot undo this option.

Note

When you remove a folder, you remove all of the items in it at the same time.

- [Rename](#)
Use this option to rename a folder in the projects area.

5.5.3 The Dependency Checker

The Dependency Checker appears when you check a report or project for errors in the Workbench, or when you select the command on the Report menu for a report that you have open in Crystal Reports. The Dependency Checker registers several types of errors:

- Report part hyperlink errors
- Repository object errors
- Formula compilation errors

Errors listed in the Dependency Checker show the following information:

- Error type.

Icon	Meaning
	Success. The Dependency Checker has verified that there are no errors in the file.
	Warning. The Dependency Checker has found something in the file that might be a problem. You can verify the warning, but you don't have to fix it to have the report to run correctly.
	Error. The Dependency Checker has found an error that you should fix so that the report will run correctly.

- Description of the error.
- Location of the file that contains the error.

Double-click an error to open the target report, and go to the report object so that you can fix the problem. If the report object cannot be found (because, for example, you've deleted part of the report since checking for errors), you receive a message that recommends you run the check again.

5.5.3.1 Shortcut Menu

When you right-click a message in the Dependency Checker, a shortcut menu appears:

- *Go To*
Use this option to open the report that contains the error and to go to the object that is causing the problem. You can also select a message and press Enter or double-click to open the report.
- *Sort By*
Use this option to sort the messages by type, number, description, or location.
- *Clear*
Use this option to remove the selected message.
- *Clear All*
Use this option to remove all of the messages.
- *Copy*
Use this option to copy the message to the clipboard so that you can paste it into another application.
- *Options*
Use this option to open the Dependency Checker tab of the Options dialog box. Use this tab to select the conditions that you want to check for when you run the Dependency Checker.
For more information about the options, search the Crystal Reports Online Help for the topic "Dependency Checker tab (Options dialog box)."

5.5.4 Opening and docking explorers

Crystal Reports includes these explorers and tools that you can use to see various reports and report objects:

- The Field Explorer
- The Report Explorer
- The Repository Explorer
- The Workbench
- The Dependency Checker

Open any of these explorers by using the commands on the View menu or the buttons on the Standard toolbar. If the explorer is closed, its command or button opens it; if the explorer is already open, its command or button sets focus to it.

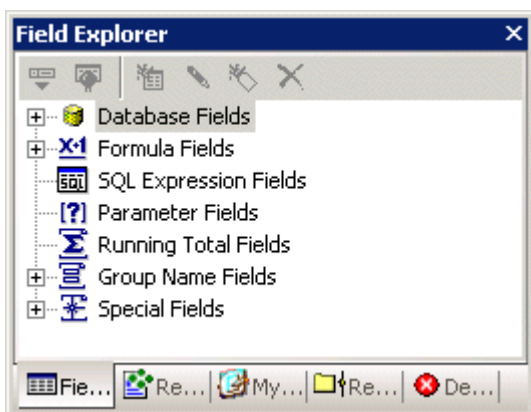
After installation, each explorer opens in a docked position in the Report Designer. You can dock explorers in any area of the designer by dragging them to a new location.

Double-click an explorer's title bar to change between docked and free-floating mode. In free-floating mode, the explorer can be dragged to any position. When you drag the explorer, a placement frame shows you where it will be placed. You can also resize an explorer by dragging any of its edges with the Resizing cursor.

Note

When you double-click the title bar of a free-floating explorer, the program docks it where it was when you last used Crystal Reports.

You can create a single, multi-tabbed explorer from the Field Explorer, the Report Explorer, the Repository Explorer, the Workbench, and the Dependency Checker by dragging the explorers on top of each other. This multi-tabbed explorer can be used in docked or free-floating mode.



Click the tabs in the amalgamated explorer to change focus from one explorer type to another; the name in the title bar changes accordingly to show you which explorer you're currently using.

5.6 Beyond basic reports

Once you are comfortable with the basics of reporting, you will be ready to investigate the more powerful reporting features of Crystal Reports, including:

- Charts (see [Charting \[page 303\]](#))
- OLE objects (see [OLE \[page 335\]](#))
- Form letters (see [Form letters \[page 225\]](#))
- Subreports (see [Subreports \[page 870\]](#))
- Cross-tabs (see [Cross-Tab Objects \[page 346\]](#))
- Multi-section reports (see [Multiple Section Reports \[page 217\]](#))

6 SAP BusinessObjects Enterprise Repository

This section introduces you to the SAP BusinessObjects Enterprise Repository. It shows you how to add items to the repository, how to update existing items, and how to use repository items in your Crystal reports.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

6.1 What is the SAP BusinessObjects Enterprise Repository?

The SAP BusinessObjects Enterprise Repository is a database in which you manage shared report objects. You can refresh a report's repository objects with the latest version from your SAP BusinessObjects Enterprise Repository when you publish reports to SAP BusinessObjects Business Intelligence platform.

The SAP BusinessObjects Enterprise Repository is hosted by the Central Management Server (CMS) system database in SAP BusinessObjects Business Intelligence platform.

The object types that the repository supports include the following items:

- Text objects
- Bitmaps
- Custom functions
- Commands (queries)

Note

Business Views and list-of-values objects are also stored in the SAP BusinessObjects Enterprise Repository. You can use Business Views in Crystal Reports, but you create them in the Business View Manager. You can create and use lists of values in Crystal Reports, but you add them to the repository through the Business View Manager.

By maintaining a shared repository of report objects, you can modify a particular object and update all reports containing that object as they are opened for use. A central location for report objects also helps with the task of managing your data—an important benefit in maximizing productivity and minimizing costs at your company.

6.1.1 Work flow

Although there are many ways to use the SAP BusinessObjects Enterprise Repository, this sample work flow will acquaint you with some of the things you might want to do:

- Set up subfolders and subcategories for your repository
The [Repository Explorer](#) displays the contents of your SAP BusinessObjects Enterprise Repository as a tree with folders, subfolders, categories, and subcategories. The following folders and categories are supplied through SAP BusinessObjects Business Intelligence platform:
 - Repository Items
 - Enterprise Items
 - Categories
 - Personal CategoriesYou can add subfolders and subcategories.
- Add report objects to your repository
There are different ways to do this depending on the type of object you want to add:
 - Select a text object or bitmap image and drag it to a folder or choose "Add to Repository" from the shortcut menu.
 - Select a Custom Function in the Formula Workshop and click the "Add to Repository" button.
 - Create or modify a command in the Database Expert and, once it appears in the Selected Tables list, right-click it and choose the "Add to Repository" option.
- Add repository objects to a report
There are different ways to do this, depending on the type of object:
 - Text objects and bitmap images can be dragged to the report from the [Repository Explorer](#).
 - Custom functions can be added through the [Formula Workshop](#).
 - Commands can be added through the [Database Expert](#).
 - Lists of values and prompt groups can be added to parameter fields by selecting them from the appropriate lists in the [Create New Parameter](#) or [Edit Parameter](#) dialog boxes.
- Update report objects in your repository
Again, there are different ways to do this, depending on the type of object:
 - Text objects and bitmap images can be dragged back to the original repository object.
 - Custom functions can be updated through its user interface.
 - Commands can be updated and added back to the repository through the [Database Expert](#).

6.2 Accessing the SAP BusinessObjects Enterprise Repository

The repository supplied by Crystal Reports is set up when you install. You don't have to do anything additional before you use the repository.

6.2.1 To open the SAP BusinessObjects Enterprise Repository

Procedure

1. Start *Crystal Reports* and open a report.

2.  Click the *Repository Explorer* button on the Standard toolbar.

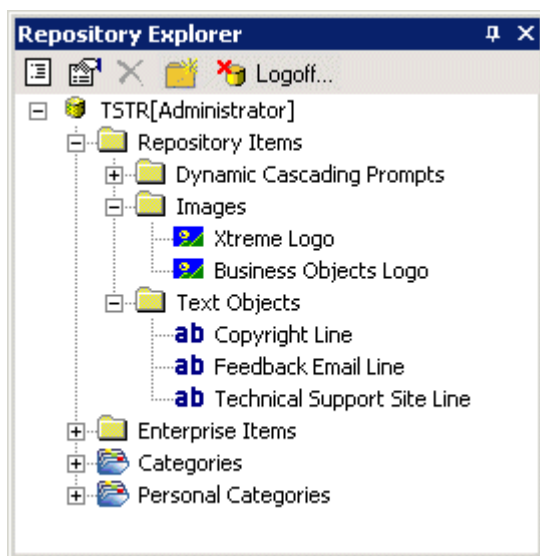
→ Tip

Another way to do this is to click the Repository Explorer option under the View menu.

3. Click Logon to connect to SAP BusinessObjects Business Intelligence platform.

For information about how to log on to SAP BusinessObjects Business Intelligence platform, see [Working with Enterprise folders \[page 628\]](#).

The Repository Explorer appears.



4. Expand nodes to see the repository contents.

Results

The *Repository Explorer* might appear in a docked position in the Report Designer depending on where it was when you last used Crystal Reports. For more information about docking explorers, see [Opening and docking explorers \[page 117\]](#).

Hover your mouse's cursor over any object in the *Repository Explorer* to see a tooltip. Tooltips include Author and Description information for text objects and bitmaps; for commands, tooltips include the complete text of the SQL statement.

6.3 Repository Explorer toolbar

The Repository Explorer's toolbar provides buttons with tool tips. These buttons let you add new folders or categories, search for items, and so on:

- [Change view settings](#)
Use this option to open the View Settings dialog box. Use this dialog box to limit the type of repository items displayed in the Repository Explorer. You can also select options to sort multiple items by name or by type.
- [Advanced filtering](#)
Use this option to display filtering options at the bottom of the Repository Explorer. Use these filters to find specific items by words from the Name or Author fields in the Add Item dialog box. For information about adding or changing an item's name or author, see [Adding items to the repository \[page 124\]](#) and [Modifying objects in the repository \[page 129\]](#).

Note

Text entered in the filtering fields of this option is not case sensitive.

- [Delete the item/folder](#)
Use this option to permanently remove the selected item or folder from the repository. When you delete a folder, you delete all the items it contains. For more information about deleting items from the repository, see [Deleting items from the repository \[page 131\]](#).
- [Insert a new folder](#)
Use this option to add a new subfolder to the repository. For information, see [Adding subfolders and subcategories to the repository \[page 123\]](#).

Note

This option is called "Insert a new category" when you have selected a category in the repository tree.

- [Logoff Server](#)
Use this option to log off the SAP BusinessObjects Business Intelligence platform server that you are connected to. When you are logged off, this option is called Logon; use it to reconnect to your SAP BusinessObjects Business Intelligence platform server or to connect to a different server.

6.4 Adding subfolders and subcategories to the repository

You can organize the contents of the repository by creating subfolders and subcategories in its tree view.

Folders are objects that you can use to separate content into logical groups. Like folders, categories are objects that you can use to organize documents. You can associate documents with multiple categories, and you can create subcategories within categories.

SAP BusinessObjects Business Intelligence platform provides the following types of categories:

- Corporate categories are created by the administrator, or other users who have been granted access to these categories. If you have the appropriate rights, you can create corporate categories.

- Personal categories can be created by each user to organize his or her own personal documents.

6.4.1 To add a new subfolder or subcategory

Procedure

1. In the Repository Explorer, select an existing node, and click *Insert a new folder* or *Insert a new category* on the Repository Explorer's toolbar.

→ Tip

The insert button changes automatically from folder to category depending on the type of object that you select.

A new subfolder or subcategory is added to the repository tree. (If your repository items are not sorted, the new subfolder or subcategory is added to the bottom of the selected node. If your repository items are sorted by type, the new subfolder or subcategory is added alphabetically with the default name New Folder.)

2. Name your new subfolder or subcategory and hit the *Enter* key.

6.5 Adding items to the repository

You can add text objects and bitmap images to the repository by dragging them from your report to the appropriate repository folder. A dialog box appears so you can add identifying information about the object.

Custom functions are added to the repository through the Formula Workshop and commands are added through the Database Expert.

Once report items are in the repository, they can be shared between many reports. Each person who uses Crystal Reports can connect to the repository and choose items to add to a report.

When you add an object to the repository, that object becomes "connected" to the repository. As long as an object remains connected, you can update any report that uses that object with the latest version in the repository.

6.5.1 Adding a text object or bitmap image

Procedure

1. Select a text object or bitmap image in your report and drag it to the appropriate folder in the Repository Explorer.

2. In the *Name* field of the Object Information dialog box, create a name for your object or image.

ⓘ Note

You cannot use these characters in your object's name: # " { } ; /

3. Enter an author and description if you want, and then click *OK*.

→ Tip

Author and description information appears in tooltips and can be searched for using the Advanced filtering option.

You'll see that your text object or bitmap image has been added to the appropriate folder.

6.5.2 Adding a text object or bitmap image - another method

Context

You could have added the text object or bitmap image without dragging and dropping it.

Procedure

1. Select a text object or bitmap image in your report and click *Add to Repository* from the shortcut menu.

The Add Item dialog box appears.

Notice that, because you did not drag the object to a specific repository folder, this dialog box contains an area to select a location from.

2. In the *Name* field, create a name for your object or image.

ⓘ Note

You cannot use these characters in your object's name: # " { } ; /

3. In the Location area, select a repository folder, and then click *OK*.

You'll see that your text object or bitmap image has been added to the appropriate folder.

Results

If you try to edit any of the objects you added to the repository, you'll see that you can't make changes; the objects are in read-only mode. This is true of any report object that is stored in the repository: as long as it is connected to the repository it can't be changed in the report. If you right-click the object in the report and choose "Disconnect from Repository" from the shortcut menu, the item is disconnected from the repository.

and becomes editable. If you want other reports to be updated with the edited report object, you must add it back to the repository.

6.5.3 Adding a custom function

Procedure

1. In the Formula Workshop, expand the *Report Custom Functions* node and select the custom function you want to add to the repository.

2.  Click the *Add to Repository* button.

You'll see that your custom function has been added to the Repository Custom Functions node.

→ Tip

You can also add a custom function to the repository by dragging it from the Report Custom Functions node—in the Workshop Tree—and dropping it on a Repository Custom Functions node.

→ Tip



The custom function icon in the Report Custom Functions node changes to indicate that the custom function exists in the repository.

Results

📘 Note

You must add all custom functions to the Repository Custom Functions node that is provided with Crystal Reports.

6.5.4 Adding a command

Procedure

1. In the Selected Tables area of the Database Expert, select the command you want to add to the repository.
2. Right-click the command and select *Add to Repository*.
3. In the Add Item dialog box, specify a name and repository location for the command.
4. In the Location area, select a repository folder, and then click *OK*.

You'll find the command in the SAP BusinessObjects Enterprise Explorer, the Repository Explorer, the Database Expert, and the Set Datasource Location dialog box.

6.6 Using repository objects in reports

Once you have an object or a collection of objects added to the repository, you can start using them in your Crystal reports. Each type of repository object is added through its own user interface. In the case of text objects and bitmap images, you simply drag them from the Repository Explorer to a report. Custom functions are selected when you work in the Formula Workshop, and commands are selected in the SAP BusinessObjects Enterprise Explorer, the Database Expert, or the Set Datasource Location dialog box. You can select list-of-values objects from within the Create New Parameter or Edit Parameter dialog boxes.

When you add a repository object to a report, it remains connected to the repository and is in read-only mode. To edit the object, you must disconnect it from the repository and unlock its formatting.

Note

To reconnect a report object that has been disconnected from its repository, re-add the object or update the repository copy. Objects that remain disconnected from the repository cannot be automatically updated when the report is next opened.

Note



You can see that a report object is connected to the repository by opening the Report Explorer and looking for it; if the object has an icon in front of it, it is connected to the repository.

This section shows you how to add a bitmap image, a custom function, or a command to a new report. It also shows you how to add a list of values to a parameter.

6.6.1 Adding a text object or a bitmap image to a report

Procedure

1.  Click the *Repository Explorer* button on the Standard toolbar.

→ Tip

Another way to do this is to click the Repository Explorer option under the View menu.

2. Log on to SAP BusinessObjects Business Intelligence platform if you have not done so already.
For information about how to log on to SAP BusinessObjects Business Intelligence platform, see [Working with Enterprise folders \[page 628\]](#).
3. Expand the appropriate folder in the Repository Explorer and drag a text object or bitmap image to your report.

Results

📘 Note

Folders in your SAP BusinessObjects Enterprise Repository do not have to be named to represent the objects they hold; you can use a name of your choice. See [Adding subfolders and subcategories to the repository \[page 123\]](#) for more information.

6.6.2 Adding a custom function to a report

Procedure

1.  Click the *Formula Workshop* button on the Expert Tools toolbar.

→ Tip

Another way to do this is to click the Formula Workshop option under the Report menu.

2. In the Formula Workshop, expand the *Repository Custom Functions* node until you find the custom function you want to add.
3. Right-click the custom function and click *Add to Report* from its shortcut menu.

If the custom function you are adding to your report requires other custom functions from the repository, they can be added at the same time.

Results

You'll see your custom function has been added to the Report Custom Functions node of the Formula Workshop.

📘 Note

You can also add custom functions to a report while creating formulas in the Formula Workshop. For more information about this method, see [Creating a formula in the Formula Expert \[page 701\]](#).

6.6.3 Adding a command to a report

Procedure

1.  Click the *Database Expert* button on the Expert Tools toolbar.

→ Tip

Another way to do this is to click the Database Expert option under the Database menu.

2. In the Database Expert, expand the *Repository* folder.

→ Tip

If the SAP BusinessObjects Enterprise Explorer doesn't open immediately, double-click Make New Connection.

3. In the SAP BusinessObjects Enterprise Explorer, expand folders until you find the command you want to add, and then click *Open*.

Results

You'll see your command in the Available Data Sources area of the Database Expert. When the command is in this area, you can add it to your report as you would add any other data source. For more information, see [Selecting the data source \[page 98\]](#).

6.6.4 Adding a list of values to a parameter

Procedure

1. In the Field Explorer, select *Parameter Fields* and click *New*.
2. In the List of Values area, click *Dynamic*.
3. Click *Existing* and choose a repository list of values from the tree view that appears.

ⓘ Note

If the list is empty (you cannot connect to a repository, or your report does not contain an existing list of values), this option is not available.

6.7 Modifying objects in the repository

You can modify repository objects by disconnecting them, changing them in a report, and adding them back to the repository. As well, you can rename objects and move them to different folders directly in the repository.

ⓘ Note

Be aware that when you modify an object and add it back to the repository, your changes affect all those who use that same repository: a renamed object is renamed for all users, and reports that contain the modified object may be updated upon opening them in the Report Designer.

6.7.1 To modify a repository object

Context

📘 Note

This procedure shows you how to modify and update a text object in the repository. Custom functions and commands are modified and updated in their respective user interfaces; in each case, you must first disconnect the custom function or command from the repository, make the modifications, and then add the custom function or command back to the repository.

Procedure

1. From the Repository Explorer, drag any text object into a report.
2. Right-click the text object in the report and select *Disconnect from Repository* from the shortcut menu.
3. Double-click the text object and modify its text.
4. Drag and drop the text object from the report to the folder containing the text object in the repository.

When you drop the text object, the Add or Update Object dialog box appears.

5. Click *Update*.
6. In the Modify Item dialog box, make any changes you require and click *OK*.
7. Right-click the text object in the repository and click *Properties* from its shortcut menu.

Notice that the date and time in the Modified area have changed to correspond to the time you added the text object back to the repository.

6.8 Updating connected repository objects in reports

Repository objects that are used in a report and are connected to that report's repository can be updated automatically when opening the report in Crystal Reports. This behavior is controlled by an Options setting and is global for all reports.

📘 Note

Objects that are used in a report but are disconnected from the repository are not automatically updated when the report is opened.

6.8.1 To set the update option

Procedure

1. On the *File* menu, click *Options*.
The Options dialog box appears.
2. Click the *Reporting* tab.
3. Select the *Update Connected Repository Objects on Open* check box.
4. Click *OK*.

Results

If you don't want to use this global option, you can also update a report's repository objects by selecting the Update Repository Objects option on the Open dialog box for an individual report.

Note

When you open a report that contains a command that is stored in the repository, and you have specified that you want to Update Connected Repository Objects on Open, only the definition of the command is automatically refreshed; the data that the command returns is not updated until you click the Refresh button in the report.

6.9 Deleting items from the repository

Any object you store in the repository can be deleted from that repository without removing it from the reports that use it. Once you remove an object from the repository, it is removed for all users.

Note

When you delete an object from the repository, it appears to remain connected in the reports that use it. When you try to update repository objects for those reports, a warning message is shown in the Missing Repository Items dialog box so you can quickly identify the object. Use the Report Explorer dialog box to find the object in the report; then you can disconnect it.

Note

If you have deleted an object from the repository, creating a new object with the same name and adding it back to the repository does not relink that object to your reports. Each repository object has a unique ID; linking is not done by name.

6.9.1 To delete an object from the repository

Procedure

1.  Click the *Repository Explorer* button on the Standard toolbar.

→ Tip

Another way to do this is to click the Repository Explorer option under the View menu.

2. In the appropriate folder, select the object you want to remove and press the *Delete* key.

You are asked to confirm the deletion.

3. Click the *Yes* button.

The object is removed from the repository.

Results

You can also delete a repository object by right-clicking it and selecting Delete from the shortcut menu.

6.10 Using Undo in the repository

You cannot undo any action that updates the repository. You can undo anything that affects only the report and not the repository. For example, you can undo the disconnect activity.

7 Designing Optimized Web Reports

Click a link to view one of the topics in this section:

- [Overview \[page 133\]](#)
- [Scale with SAP BusinessObjects Business Intelligence platform \[page 134\]](#)
- [Making the right design choices \[page 136\]](#)
- [Streamlining your reporting environment \[page 141\]](#)
- [Using enhanced record selection formulas \[page 144\]](#)
- [Improving grouping, sorting, and totaling \[page 150\]](#)
- [Formatting reports for viewing on different platforms \[page 154\]](#)

7.1 Overview

Whether you distribute your enterprise reports over a local network, a corporate Intranet, or the Web, you can use Crystal Reports' powerful, built-in performance features to deliver web reporting speed.

Crystal Reports automatically provides these major performance advantages—even if you don't apply any of the strategies set out in this section:

- **Page-on-demand technology**
Page-on-demand report access lets users download only the specific report pages they need to see, thus improving response times and reducing web traffic. Further, placeholders and partial page technology allow you to view report pages and data over the Web immediately, without having to wait for the processing of large objects, such as graphics and subreports.
- **Optimized, multithreaded Report Engine**
The Crystal Report Engine's multithreading capabilities and thread-safe database drivers allow you to continue working on your important tasks, while many other operations are processed simultaneously in the background. The Report Engine also minimizes the number of passes made through the data, speeds up processing with improved memory management, and handles subreports and parameters as efficiently as possible.

In addition to these built-in features, the [Key strategies for optimizing web reports \[page 134\]](#) discussed in the following sections bring additional performance benefits, which are often substantial. When you design new reports (or improve reports created in older versions of Crystal Reports) in accordance with these strategies, reports run faster and tie up fewer processing resources. Consequently, report users can easily access the data they need—faster than ever.

→ Tip

If you're new to web reporting, or to reporting in general, reading this section will prepare you for future reporting tasks—you'll gain important insight into designing faster, better reports.

Note

Understanding databases and how they work is often important when considering performance. See [Databases overview \[page 885\]](#) for background information.

7.1.1 Key strategies for optimizing web reports

- As your company grows, Crystal Reports and SAP BusinessObjects Business Intelligence platform can grow along with you. For details, see [Scale with SAP BusinessObjects Business Intelligence platform \[page 134\]](#).
- For tips and considerations on how to create fast, interactive reports, see [Making the right design choices \[page 136\]](#).
- For information about getting the most out of your existing database, see [Streamlining your reporting environment \[page 141\]](#).
- To minimize data transfer and enhance report performance, see [Using enhanced record selection formulas \[page 144\]](#).
- To decrease the processing and data transfer times of grouped, sorted, or totaled reports, see [Improving grouping, sorting, and totaling \[page 150\]](#).

In general, report performance issues are similar, no matter how you distribute your reports. By following these strategies, you'll notice significant improvements not only in multi-user web environments, but also in single-user situations.

7.2 Scale with SAP BusinessObjects Business Intelligence platform

SAP BusinessObjects Business Intelligence platform provides a flexible and efficient way to deliver your reports over the web or to integrate them into custom web applications.

The Report Application Server (RAS) provides a rich set of server-based reporting services that let you tightly integrate reporting into your custom web applications. RAS provides a base set of BusinessObjects Business Intelligence platform services, focused on report processing, application integration, and run-time report modification.

You can extend your application even further using the following advanced services available in other editions of SAP BusinessObjects Business Intelligence platform:

- **Security**
SAP BusinessObjects Business Intelligence platform provides granular group, user, and data-level security to help you protect sensitive reports and deliver a more personalized end-user experience.
- **Scheduling**
SAP BusinessObjects Business Intelligence platform incorporates a flexible time- and event-based scheduling system to allow you to process large reports during off hours and to avoid unnecessary database hits. (The Report Application Server provides on-demand reporting, so every viewing request results in a separate query on your database.)

- **Versioning**
Versioning works closely with scheduling to store "instances" of a report. Versioning not only reduces the number of database hits required to serve users, but also allows you to keep an archive of report instances for historical reference.
- **Clustering**
SAP BusinessObjects Business Intelligence platform incorporates proven clustering and load-balancing technology to help you deliver a highly available, reliable information delivery system.
- **Scalability**
SAP BusinessObjects Business Intelligence platform is built on a distributed, multi-server architecture, allowing you to scale up on a single machine (by adding processors) or scale out over multiple machines to handle heavier user loads.
- **Management**
SAP BusinessObjects Business Intelligence platform incorporates extensive administration and management controls that allow you to organize content, set up complex schedules and security, and tune the system for optimal performance.
- **External (third-party) authentication**
SAP BusinessObjects Business Intelligence platform allows you to leverage your existing security system to manage users and groups. All authentication can be delegated to a third-party NT or LDAP system.

7.2.1 Evaluation times for date functions in SAP BusinessObjects Business Intelligence platform

Different date and time functions available in Crystal Reports are evaluated at different stages during report processing. Some are evaluated when a scheduled report is processed, or when a report instance is refreshed: this evaluation time is known as "WhileReadingRecords." Others are evaluated each time a report page is formatted for display in a report viewer or a browser: this evaluation time is known as "WhilePrintingRecords."

If you use SAP BusinessObjects Business Intelligence platform to generate report instances and notice that date or time functions are not evaluating to the values that you expect, consider the following:

- The current date and time are always read from the clock that is locally accessible to the Report Engine—that is, from the clock on the computer that processes the report. In SAP BusinessObjects Business Intelligence platform, the Job Server processes scheduled reports against the database, and the Page Server processes on-demand reports against the database. The Page Server also formats individual pages of report instances when users view them.
- When a scheduled report is processed, all date and time formulas are evaluated by the Job Server in order to generate the report instance. When you simply view the resultant report instance, none of the formulas are re-evaluated.
- When you view a cached report page, none of the date functions are re-evaluated, because the records have already been read and the page has already been formatted.
- If you use date and/or time functions in a report's selection formula, the report's data is dependent upon the return values of those formulas. Consequently, when you view a report instance at a later date, the function may cause the report data to be updated from the database.
- To ensure that date and time functions return the values that you expect, you can force the evaluation time through the use of the WhileReadingRecords and WhilePrintingRecords functions.
For more information, see the "Functions" and "Report Processing Model" sections of the Crystal Reports Online Help.

7.3 Making the right design choices

This section offers design options and considerations that will help improve the performance of your reports. The topics covered range from basic suggestions, such as updating reports created in older versions of Crystal Reports to the latest file format, to more involved decisions, such as whether to use live or saved data, and how to use subreports efficiently.

When designing your reports, and especially when designing reports for the Web, you should allow report users to drive the data they see. In other words, display summarized information, so each user can navigate the report quickly and then drill down to access additional data. In this way, web traffic and response times are minimized, because only the data requested by the user is transferred from the database server.

These are only a few of the benefits of designing user-driven reports:

- Report users gain interactive control over the type and quantity of information they view over the Web.
- Data transfer and network traffic decrease, because only the information requested by users is returned from the database server.
- When users need real-time reporting of live data over the Web, user-oriented reports respond quickly and communicate efficiently with the database server.
- Reports become more useful, because each user customizes the report's contents, thereby creating a reporting solution specific to his or her particular decision-making problem.

7.3.1 Using faster report formats

The quickest way to help improve the performance of reports created in older versions of Crystal Reports is to save them in the latest Crystal Reports format. Crystal Reports has been enhanced to process reports faster than ever before: update your older reports to take advantage of these enhancements.

To update the format of an older report, just open it in Crystal Reports and select Save from the File menu. The older version of the report will be replaced with an updated version.

Note

If for some reason you need to keep a report in its original file format, use the Save As command (instead of Save) and enter a new name for the updated version of the report.

7.3.2 Choosing between live and saved data

When reporting over the Web, the choice to use live or saved data is one of the most important decisions you'll make. Whichever choice you make, however, Crystal Reports displays the first page as quickly as possible, so you can see your report while the rest of the data is being processed.

7.3.2.1 Live data

Live reporting gives users real-time access to live data, straight from the database server. Use live data to keep users up-to-date on constantly changing data, so they can access information that's accurate to the second. For instance, if the managers of a large distribution center need to keep track of inventory shipped on a continual basis, then live reporting is the way to give them the information they need.

Crystal Reports supports live reporting. However, you should first consider whether or not you want all of your users hitting the database server on a continual basis. If the data isn't rapidly or constantly changing, then all those requests to the database do little more than increase network traffic and consume server resources. In such cases, you may prefer to use reports with saved data.

To ensure the efficiency of real-time reporting, read all of the suggestions in this chapter. These topics, however, are of particular importance:

- [Taking advantage of on-demand subreports \[page 139\]](#)
- [Performing grouping on server \[page 151\]](#)
- [Incorporating parameter fields into record selection formulas \[page 149\]](#)

7.3.2.2 Saved data

Reports with saved data are useful for dealing with data that isn't continually updated. When users navigate through reports with saved data, and drill down for details on columns or charts, they don't access the database server directly; instead, they access the saved data. Consequently, reports with saved data not only minimize data transfer over the network, but also lighten the database server's workload.

You can schedule these reports within SAP BusinessObjects Business Intelligence platform, so they automatically refresh from the database on a predetermined basis. For example, if your sales database is only updated once a day, or once a week, then you can run the report on a similar schedule and save it with data. Sales representatives then always have access to current sales data, but they aren't hitting the database every time they open a report. Alternatively, you can refresh reports with saved data on an as-needed basis.

Saved data is discarded and refreshed when you perform any of these tasks in a report:

- Select the Refresh command.
- Change your database logon.
- Change any of the report's data parameters.
- Add a new field that doesn't exist in the saved data.
- Drill down in a report where [Perform Grouping On Server](#) is selected, and the details section is suppressed.
- Verify the database—if the database structure has changed drastically.
- Change the linking parameter on a subreport (the subreport is refreshed).
- Change the order of groups (only for reports in the Report Application Server).

If you do use reports with saved data, incorporate the other suggestions in this section to make sure your reports are designed for optimum performance.

To save a report with data, first make sure the Save Data with Report option is selected on the File menu; then Save your report.

7.3.3 Designing summary reports

Designing and distributing summary reports is a relatively easy way to ensure that users quickly find the data they need over the Web. A summary report can include as much data as any other report. However, by hiding a summary report's Details section, you avoid overwhelming users with data they may not immediately need.

When the Details section is hidden, users navigate with the Group tree first, to locate the desired data. Then, by drilling down on the report, they can request specific data, which is returned quickly without unnecessary records. This is especially important to improving navigation of long summary reports, which might consist of hundreds, thousands, or even tens of thousands of pages.

To facilitate navigation in this way, you first need to group the data and insert the summary fields you want to include in your report. For detailed information and instructions, see [Grouping data \[page 173\]](#) and [Summarizing grouped data \[page 192\]](#).

Once you've grouped and summarized your report data, hide the Details section (and any other large report sections) so that users can easily navigate to the data that's important to them.

7.3.3.1 To hide details in a summary report

Procedure

1. Open your report in Crystal Reports.
If you haven't yet created a grouped and summarized report, open Group.rpt from the Feature Examples samples folder.
2. On the *Report* menu, click *Section Expert* to open the Section Expert.
3. In the *Sections* list, click *Details*.
4. On the Section Expert's *Common* tab, select the *Hide (Drill-Down OK)* check box.
5. Click *OK*.

Results

You'll notice the details are now hidden. To view the details, navigate through the report using the Group tree, and then drill down on the appropriate area of the report.

Note

For details on minimizing data transfer with summary reports, see [Performing grouping on server \[page 151\]](#).

7.3.4 Using subreports carefully

A subreport is an excellent way to include additional data in a main report. Performance issues relating to subreports vary depending on the type of subreport used.

For general information on subreporting, see [Subreports \[page 870\]](#).

7.3.4.1 Taking advantage of on-demand subreports

If your report has a section that handles a large number of records, you can put that section into an on-demand subreport. An on-demand subreport appears as a hyperlink in the primary report. When you open the primary report, no data is retrieved for the on-demand subreport until you drill down on the hyperlink.

For example, when designing a report that shows each employee's quarterly sales for each product and each product type, you might also want to track each employee's progress by including weekly sales information. This additional data, however, may not be of interest to many users viewing the report. In such a case, extract the weekly sales portion of the report and attach it as an on-demand subreport. Detailed information about weekly sales is retrieved from the database only when a user drills down on the on-demand subreport.

Many report objects—such as large cross-tabs, OLAP grids, advanced charts, and maps—are ideal candidates to include in on-demand subreports, so that the object is not processed until you drill down.

To insert an on-demand subreport, see [Inserting subreports \[page 873\]](#) and [Creating an on-demand subreport \[page 882\]](#).

→ Tip

You could also place such report objects in a hidden Details section of a report that uses the Perform Grouping On Server option. When you do this, the database server performs the majority of the processing, and only a subset of the records is transferred from the server to the local machine (other records are retrieved when you drill down to a hidden section).

7.3.4.2 Using linked subreports

When a subreport is linked, Crystal Reports coordinates the data in the subreport with the matching records in the primary report. If you need to use regular linked subreports—that is, linked subreports that are not on-demand subreports—you should consider these guidelines:

- If the additional data provided by a regular linked subreport is useful to relatively few users, create a linked on-demand subreport instead. Users who need to see the extra data click a hyperlink to view the subreport; users who don't need to see the extra data won't have to download it from the database server.
- In some cases, placing regular linked subreports in the Details section of a main report may hinder performance—especially when your main report contains many records. (This is because you're creating a separate subreport for each record, and a separate query must be run for each database record in the main report.) As an alternative, consider using linked on-demand subreports in the Details section of your main report.

7.3.4.3 Linking tables instead of linking subreports

Whenever possible, coordinate your report data by linking database tables on the Links tab of the Database Expert, rather than by linking regular subreports—that is, subreports that aren't on-demand subreports—to the main report. Since each subreport is run as a separate report, linked tables often have a performance advantage.

7.3.5 Using other design elements effectively

Maps

Map rendering is a single-threaded operation that does not scale well. Although maps are supported, you need to be careful in considering the overall effect a map in your report will have on performance.

Report Templates

If you will be applying the same template to multiple reports, it is best to open the report once and cache it, because the template only needs to be read-only.

Including "Page N of M" or "Total Page Count"

If you include the special fields "Page N of M" or "Total Page Count" in your report, the report needs to finish processing before it can calculate this value. Avoid using these special fields in your report unless the report is very small or you absolutely require the value.

7.3.6 Designing reports to maximize data sharing

SAP BusinessObjects Business Intelligence platform has data-sharing functionality that improves performance by reducing the number of database calls that are made in a system with multiple users.

Data can be shared between users of a report object only if certain conditions are met. Try to maximize data sharing by designing reports such that the conditions for data sharing are met as often as possible, without compromising different user's need for report information.

When reports are processed by the Page Server, data is shared between users of a report when these conditions are met:

- Users view a report using the same database logon information.
- Users view a report using the same parameters.

- Users view a report using the same page layout options.
- Users view a report using the same locale settings.

Reports viewed using the DHTML viewer, the ActiveX viewer, or the Java viewer are processed using the Page Server. These viewers do not allow users to change the page layout or locale of a report. However, it is possible to develop custom viewer applications that provide this functionality.

When reports are processed by the Report Application Server (RAS), data is shared between users of a report when these conditions are met:

- Users view a report using the same database logon information.
- Users view a report using the same parameters.
- Users view a report using the same locale settings.
- Users do not modify the report.

Reports viewed using the Advanced DHTML viewer (or your own custom application that permits report viewing and modification) are processed using the Report Application Server.

7.4 Streamlining your reporting environment

Another step to ensuring that report users receive their information quickly is to assess your reporting environment. What kind of database do you use? How is data organized within the database? How are you connecting to the data you need to report off? How are you linking your database tables? By taking these important considerations into account, you can significantly minimize the amount of data that must physically travel across the network.

7.4.1 Selecting the fastest database and connection

To improve reporting performance, utilize your database to its full potential by having it do the majority of your report processing. Ideally, Structured Query Language (SQL) databases are the most efficient for carrying out this task.

For details on SQL databases, see [Using SQL and SQL databases \[page 888\]](#).

7.4.2 Using table indexes

You can also improve the way your data is organized within the database. For optimum processing speed, report off indexed fields on your SQL database. Using table indexes is an easy way to increase the speed of data access and to reduce the time it takes for Crystal Reports to evaluate data.

For a complete description of how table indexing works, see [Indexed tables \[page 887\]](#).

7.4.3 Improving table-linking choices

When you add multiple database tables to your report, you link the tables on a common field so that records from one table match related records from another. (Table linking works best if your database tables are indexed.) Linking your database tables in this way is usually much faster than incorporating linked subreports into your main report.

When you link two or more tables, you want your report to read as few records as possible, while at the same time finding all matching records. The best ways to adhere to this guideline are to assess your reporting needs and to plan your strategy before creating your report. When you know exactly what you need from your data source, Crystal Reports makes it easy to get that information.

There are many other specific issues to consider when linking tables. These additional considerations, however, are largely contingent upon your reporting environment. In other words, the steps to optimal table-linking performance depend upon your database type, the possibility of indexing tables, and the join type required between tables. You'll find complete descriptions for various reporting scenarios in the section entitled [Linking tables \[page 902\]](#).

In the majority of reporting scenarios, the following general procedures should ensure that your tables are linked for improved if not optimal performance.

7.4.3.1 To link tables effectively

Procedure

1. Ensure that each database table is indexed on the field you are going to use.
2. Add the database tables to your report, and then link from the primary table to the lookup table on a common field. (Do this instead of inserting a linked subreport and binding it to the data in your primary report.)
3. Use a record selection formula that sets range limits on the indexed field in the primary table. This minimizes the number of records in the primary table for which Crystal Reports must find matching records in the lookup table.

7.4.4 Using thread-safe database drivers

If you share and refresh reports over the web, open them using thread-safe database drivers whenever possible. The Crystal Report Engine supports multiple threads. Thus, when you make multiple data requests through thread-safe database drivers, the Report Engine can process all of the requests simultaneously. As the result, you'll be able to view your reports sooner.

The following generic database drivers are thread-safe:

- crdb_odbc.dll (ODBC)
- crdb_jdbc.dll (JDBC)
- crdb_ado.dll (OLE DB)

ODBC connections to the following database types assume that the ODBC database driver is also thread-safe and, therefore, operate in a multi-threaded manner:

- Microsoft SQL Server
- Oracle
- DB2
- Teradata
- Sybase

Note

It is also assumed that the database drivers used with JDBC and OLE DB are always thread-safe and, therefore, always operate in a multi-threaded manner.

The following native database drivers are thread-safe:

- crdb_ado_plus.dll (ADO.NET data provider)
- crdb_com.dll (COM data provider)
- crdb_db2cli.dll (DB2 Unicode)
- crdb_javabean (Java Bean data provider)
- crdb_xml (XML)
- crdb_olap.dll (OLAP)
- crdb_oracle.dll (Oracle)
- crdb_p2ssby10.dll (Sybase)

Note

Check the Release Notes in the Crystal Reports product distribution for updates to these lists; more database drivers may be added if they are found to be thread-safe during testing.

7.4.5 Using stored procedures for faster processing

A stored procedure is a compiled SQL program, consisting of one or more SQL statements, which resides and runs on your SQL server. Although stored procedures can take time to set up properly, they can be incredibly powerful.

The advantages of using stored procedures are most noticeable when reporting off large sets of data, or when running reports that demand long, complex calculations. In such cases, you should ideally use a predefined stored procedure that performs the complex work for you on the database server.

Because stored procedures reside on your SQL server, you may need to ask your Administrator about accessing or setting up a stored procedure.

- For more information on stored procedures, see [Stored procedures \[page 891\]](#).

7.5 Using enhanced record selection formulas

The most important thing you can do to speed up report processing is to limit the amount of data that is returned from the database. Your primary tool for doing this is the record selection formula.

Crystal Reports analyzes your record selection formula and generates an SQL query from it. This SQL query is then processed by the database, which sends the resulting records back to Crystal Reports. Crystal Reports then evaluates the record selection formula locally for each of the records retrieved from the database, thereby calculating the set of records used to generate the report.

Unnecessary records are eliminated at two stages: at the database with the SQL query and within Crystal Reports by the record selection formula. For speed, you want as many records as possible to be eliminated in the first stage. By designing your record selection formula effectively, you can off-load much of the processing to the database server, thus eliminating unnecessary records before returning them to Crystal Reports. This is commonly called "pushing record selection down to the database server."

This section offers several tips to ensure that your record selection formulas can be pushed down to the database server.

7.5.1 Pushing down record selection—an example

This example demonstrates the benefits of writing record selection formulas that can be pushed down to the database server.

In the Orders table of the Xtreme sample database, there are 2192 records, of which 181 have order dates prior to 2001. Suppose you want to report on only those records. On the one hand, you could use this record selection formula:

```
Year ({Orders.Order Date}) < 2001
```

The SQL query generated will send all 2192 records to Crystal Reports, and then the record selection formula will reduce this to 181. To see this, click Show SQL Query on the Database menu and notice that the SQL query has no WHERE clause. This is because Crystal Reports is not able to push down the Year () function in the WHERE clause.

On the other hand, this record selection formula generates the same report:

```
{Orders.Order Date} < #Jan 1, 2001#
```

This second formula, however, can be performed on the database server, so it is pushed down. The SQL query generated will send only 181 records to Crystal Reports. So, when the record selection formula is evaluated by Crystal Reports, no further records need to be eliminated. Click Show SQL Query on the Database menu and notice that the resulting SQL query has a WHERE clause.

As this example shows, your report's processing speed improves when you enhance your record selection formula. In this case, both formulas generate the same report, but the second takes advantages of the power and optimizations that the database server can use when handling its own data.

→ Tip

See the next section for additional information and limitations when setting up record selection requests.

Note

If you are new to record selection formulas, you may prefer to begin with the Select Expert or the sample record selection formula templates. For further details, along with an introduction to record selection, see [Selecting records \[page 155\]](#).

7.5.2 Record selection performance tips

Consider the following performance-related items when setting up record selection requests:

7.5.2.1 General

- To push down record selection, you must select "Use Indexes or Server for Speed" in the Report Options dialog box (available on the File menu).
- In record selection formulas, avoid data type conversions on fields that are not parameter fields. For example, avoid using `ToText( )` to convert a numeric database field to a string database field.
- You are able to push down some record selection formulas that use constant expressions.

7.5.2.2 PC Databases

- You can only push down record selection on indexed fields.
- You can only push down AND clauses (not OR).

7.5.2.3 SQL Databases

- You can push down record selection on indexed or non-indexed fields.
- Your SQL server will respond faster if you use indexed fields.
- You can push down AND and OR clauses.
- Record selection formulas containing some types of embedded formulas can be pushed down.
- You should incorporate SQL Expression fields to push down formula calculations necessary for the record selection.
- On the Database menu, click Show SQL Query to view the SQL that will be sent to the database server.

7.5.3 Strategies for writing efficient record selection formulas

Note

This section assumes that you are familiar with the Select Expert and that you are reporting off of a SQL database.

Consider the following points when creating a record selection formula:

Consideration 1

Any record selection formula that you generate completely with the Select Expert, without writing pieces of the formula yourself, can be pushed down.

This case actually follows from the points below. However, you can write more types of record selection formulas using the tips below than is possible with the Select Expert. To do this, you need to edit the record selection formula directly with the Formula Workshop or from within the text area that appears when you click Show Formula in the Select Expert.

To open the Formula Workshop to modify record selection, click the Report menu, point to Selection Formulas, and then select Record from the submenu.

Consideration 2

To open the Formula Workshop to modify record selection, click the Report menu, point to Selection Formulas, and then select Record from the submenu.

Any selection formula that is of the form: `DataBaseField SupportedOperator ConstantOrParameterExpression` can be pushed down.

Of course `DataBaseField` is just a database field. `SupportedOperator` is any of `=`, `<>`, `<`, `<=`, `>`, `>=`, `StartsWith`, `Like` or `In`.

`ConstantOrParameterExpression` is any expression that involves constant values, operators, functions, and parameter fields. It cannot involve variables, control structures, or fields other than parameter fields. By their very definition, constant and parameter expressions can be evaluated without accessing the database.

Note

A constant or parameter expression can evaluate to a simple value, a range value, an array value, or an array of range values. Here are some examples of such expressions:

```
{?number parameter} - 3
Year ({?run date})
CurrentDate + 5
DateDiff ("q", CurrentDate, CDate("Jan 1, 1996"))
Month (Maximum ({?date range parameter}) + 15)
```



```
["Canada", "Mexico", "USA", {?enter a country}]  
1000 To 5000  
[5000 To 10000, 20000 To 30000, 50000 To 60000]
```

A complete example:

```
{Orders.Order Date} >= CurrentDate - 3
```

The program can also push down an expression that just contains a Boolean field (without the operator and constant parts).

```
{Orders.Shipped}  
Not {Orders.Shipped}
```

Consideration 3

IsNull (**<DataBaseField>**) can be pushed down.

Consideration 4

<SqlExpression SupportedOperator ConstantOrParameterExpression> can be pushed down.

For example, the selection formula `{@ExtendedPrice} > 1000` cannot be pushed down if `{@ExtendedPrice} = (Quantity * Price)`. However, if the formula `@ExtendedPrice` is replaced with the equivalent SQL Expression, then your record selection formula will be pushed down.

Consideration 5

When using multiple expressions that follow the above considerations, separate them with AND and OR operators—you can also use NOT. You can have several of each, and you can use parentheses to give priority. For example:

```
{Orders.Order ID} < Minimum({?number range}) Or  
{Orders.Order Amount} >= 1000  
(IsNull({Customer.Region}) Or  
{Customer.Region} = "BC") And  
{Customer.Last Year's Sales} > 2000
```

Related Information

[Troubleshooting record selection formulas \[page 163\]](#)

7.5.3.1 To add a parameter field to your record selection formula

Procedure

1. Open the sample report *Group By Intervals.rpt*.

Take a moment to navigate the Group tree and see how the data is organized; notice also, in the bottom right of the Crystal Reports window, that 269 records were returned for this report.

Note

You'll see this number only if you have the Status Bar option selected on the View menu.

Now click the Design tab.

2. On the *View* menu, click *Field Explorer*.
3. In the Field Explorer, right-click *Parameter Fields* and select *New* from the shortcut menu.

The Create New Parameter dialog box appears.

4. For the *Name* of the parameter, type **SalesQuota**.
5. Click the *Type* list and select *Number*.
6. In the Options area, go to the *Prompt Text* option and type **What was last year's sales quota?**
7. In the Options area, make sure that the *Allow discrete values* option is *True*, and then click *OK*.

Note

You've now created the parameter field. The rest of these procedures describe how to add the parameter field to the record selection formula with the Select Expert.

8. On the *Report* menu, click *Select Expert*.

The Choose Field dialog box appears.

9. Select the *Customer.Last Year's Sales* field, and then click *OK*.

The Select Expert appears.

10. On the *Customer.Last Year's Sales* tab, click the drop-down list and select *is greater than*.

A new drop-down list appears.

11. Click this second list and select *{?SalesQuota}*.

Tip

Click the Show Formula button to view the new record selection formula, which appears as: `{Customer.Last Year's Sales} > {?SalesQuota}`. Instead of using the Select Expert, you could have created this formula yourself in the Record Selection Formula Editor. (To see the editor, open the Formula Workshop and select Record Selection from the Selection Formulas folder.)

12. Click *OK* in the Select Expert.

Results

You've now added your parameter field to the record selection formula. When you switch to Preview mode, or click the Refresh button, you will be prompted for new parameter values. You can then enter a numeric value representing last year's sales quota. The resulting report will display only those customers whose Last Year's Sales exceeds the numeric value you specify.

For instance, if you respond to the parameter by entering 40000, then the report will display only those customers whose sales exceeded \$40,000 last year. Notice also, in the bottom right of the Crystal Reports window, that only 58 records are returned for your report, instead of the 270 records that were returned before you put the parameter field into a record selection formula.

By enhancing this report, you've retrieved all the information you needed and, at the same time, you've ensured that the fewest number of records is transferred from the database server.

Related Information

[Advanced parameter features \[page 859\]](#)

[Parameter and prompt overview \[page 831\]](#)

[Record Selection \[page 155\]](#)

7.5.4 Incorporating parameter fields into record selection formulas

Instead of displaying all of a report's data every time the report is opened, you can create parameter fields that prompt users to specify the data they want to see. To decrease the amount of data transferred from the database server, incorporate these parameter fields right into your record selection formula.

In general, parameter fields provide interactivity for users, who respond to the parameter prompts in order to specify the data they want to see. However, by incorporating your parameter fields right into your record selection formula, you not only provide interactivity, but you also decrease data transfer and increase performance.

You can add a parameter field to your record selection formula by using the Select Expert or the Record Selection Formula Workshop. When using the Record Selection Formula Workshop, you treat the parameter field as you would any other field.

In the following example, you will create a parameter field and then add it to the record selection formula with the Select Expert.

7.5.5 Using SQL expressions where appropriate

SQL expressions are like formulas, but they are written in Structured Query Language (SQL). They are useful in optimizing report performance because the tasks they execute are normally performed on the database server (as opposed to a regular formula, which is sometimes executed on the local machine).

Don't use SQL expressions exclusively, because Crystal Reports includes its own formula language, which is much more powerful than standard SQL. Both Crystal Syntax and Basic Syntax allow you to enhance and hone your formulas in ways that are either difficult or impossible with SQL. In certain circumstances, however, creating and using SQL Expression fields can speed up the processing of your reports.

7.5.5.1 Key uses of SQL Expression fields

To maintain optimum report processing speeds, avoid using formulas (whether Crystal or Basic syntax) within record selection formulas. Instead, replace the original formula with an equivalent SQL Expression field, and then incorporate the SQL Expression field into your record selection formula. Doing so will greatly improve the chances of your record selection being pushed down to the server.

In addition, avoid sorting, grouping, or totaling on a formula field (whether Crystal or Basic syntax). Instead, replace the original formula field with an equivalent SQL Expression field, and then sort, group, or total on the SQL Expression Field. Again, this will greatly improve the chances of the processing being done on the server.

Finally, if your database supports Case Logic, and your report needs to summarize an If-Then-Else formula calculation, replace the formula with an SQL Expression field. In such cases, SQL Expression fields enable Crystal Reports to perform the report's grouping on the server. For more information, see [Using SQL Expressions for Case Logic \[page 153\]](#).

Related Information

[The SQL language \[page 894\]](#)

[SQL Expression fields \[page 102\]](#)

[Using enhanced record selection formulas \[page 144\]](#)

7.6 Improving grouping, sorting, and totaling

Click a link to view one of the topics in this section:

- [Performing grouping on server \[page 151\]](#)
- [Benefits of grouping on server - an example \[page 151\]](#)
- [Using SQL Expressions for groups, sorts, and totals \[page 152\]](#)
- [Using SQL Expressions for Case Logic \[page 153\]](#)
- [Inserting summary and running total fields where possible \[page 153\]](#)

7.6.1 Performing grouping on server

If you're reporting in real-time off live data over the Web, reduce the amount of data transferred from the database server by using the Perform Grouping on Server option. With this option, much data processing is off-loaded to the database server and only a subset of data is read initially. Detail data is returned from the database only when you drill down in a report.

Note

Server-side processing works only for sorted and grouped reports that are based on SQL data sources.

7.6.1.1 To enable server-side processing

Procedure

1. On the *File* menu, click *Report Options*.
2. Select *Perform Grouping on Server* on the Report Options dialog box.

This check box is inactive if Use Indexes or Server for Speed is not selected.

Tip

When Use Indexes or Server for Speed is selected, you can quickly enable Perform Grouping On Server from the Database menu.

3. Click *OK*.

7.6.2 Benefits of grouping on server - an example

The following example describes a typical reporting scenario wherein grouping on server can significantly reduce the amount of unnecessary data transferred from the database server.

7.6.2.1 To decrease data transfer by grouping on server

Procedure

1. Open the sample report *Group.rpt*.

Look in the bottom right corner of the Crystal Reports window and notice that 269 records are returned for this report.

Note

You'll see this number only if you have the Status Bar option selected on the View menu.

2. On the [Report](#) menu, click [Section Expert](#) to access the Section Expert.
3. In the [Sections](#) list, select [Details](#).
4. On the [Common](#) tab, select [Hide \(Drill-Down OK\)](#). This hides the report's Details section, so that only group header displays in the report. (In this case, the report is grouped by Country.)
5. Click [OK](#) in the Section Expert. The Details records will disappear from the report.
6. Press [F5](#) to refresh the report data (or click the [Refresh](#) button on the toolbar).

Notice that 269 records are still returned for the report, even though the Detail records are hidden from view.

7. On the [Database](#) menu, click [Perform Grouping on Server](#).

Now notice that only 71 records are returned for the report. The grouping (by Country) has been performed on the database server, and fewer records have been transferred to the report.

8. Drill down on a country by double-clicking the report.

Tip

When drill down is available, your cursor turns into a magnifying glass.

Crystal Reports retrieves the appropriate Detail records as you need them.

For instance, if you drill down on Australia, Crystal Reports quickly retrieves the seven records that make up the group.

Results

By hiding the Details section of this report, you've created a summary report that's easy for users to navigate. Each user can first locate the Country with which he or she is concerned, and can then drill down to retrieve valuable details.

Moreover, by enabling the Perform Grouping on Server option, you've ensured that the initial processing is completed on the database server. Consequently, only the necessary records are transferred to the report.

For more information on server-side processing, see [Server-side processing \[page 921\]](#).

7.6.3 Using SQL Expressions for groups, sorts, and totals

For reports using Perform Grouping on Server, avoid sorting, grouping, or totaling on a formula field (whether Crystal or Basic syntax). Instead, replace the original formula field with an equivalent SQL Expression Field, and then sort, group, or total on the SQL Expression field. This will greatly improve the chances of the processing being done on the server.

For details on when else to use SQL Expressions, see [Using SQL expressions where appropriate \[page 150\]](#).

7.6.4 Using SQL Expressions for Case Logic

If your database supports Case Logic, and your report needs to summarize an If-Then-Else formula calculation, replace the formula with an SQL Expression field. In such cases, SQL Expression fields enable Crystal Reports to perform the report's grouping on the server.

For instance, suppose that you're reporting off of an MS SQL Server database, which supports Case Logic. You need to include an If-Then-Else calculation in your report, and you need to summarize that calculation for each group in the report. By performing the calculation with an SQL Expression field of the following form, you use your database's ability to process Case Logic:

```
CASE <DatabaseTable><.> "<DatabaseField> "  
WHEN '<SpecifiedValue>' THEN <Calculation1>  
ELSE <Calculation2>  
END
```

If a record's **<DatabaseField>** value is equal to **<SpecifiedValue>**, then **<Calculation1>** is performed; for all other records, **<Calculation2>** is performed. By incorporating the SQL Expression field, you take advantage of your database server's ability to process Case Logic. Your report's grouping consequently takes place on the server, even when you summarize the SQL Expression field elsewhere in the report.

Note

The SQL syntax in this example is specific to MS SQL Server. You may need to refer to your database documentation or consult with your Administrator in order to determine the syntax appropriate to your database.

7.6.5 Inserting summary and running total fields where possible

Where possible, avoid creating formulas with global variables to calculate summaries or running totals.

Instead, create summaries by clicking the Insert menu and then selecting the appropriate command (Subtotal, Grand Total, or Summary). Create running total fields by opening the Field Explorer, right-clicking Running Total Fields, and then selecting New from the shortcut menu.

For general details on summarizing data, see [Sorting, Grouping, and Totaling \[page 167\]](#) and [Running Totals \[page 208\]](#).

7.7 Formatting reports for viewing on different platforms

Not all operating systems render characters or fonts the same way; a report that displays properly on a Microsoft Windows system may not display properly on a Linux system. You may see either truncated or overlapping lines for multi-line fields, or the text lines may not wrap as expected.

To ensure that the report displays on the web the way that you expect it to, use the [HTML Preview tab \[page 97\]](#) option to preview the output as you are designing your report.

8 Record Selection

This section shows you how to filter the records you want included in a report. For example, using the record selection tools, you can limit the records in your report to include only records for a specific group of customers, a specific range of account numbers, or a particular date range.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

8.1 Selecting records

When a field is selected to appear on a report, field values from every record in the active table(s) are printed by default. In many cases, you may not want to include all the values, but only a subset of those values. For example, you may want to include:

- Records only for a specific group of customers.
- Records for a specific range of account numbers out of the total number of records in the database.
- Values from only those records that fall within a particular date range.

8.1.1 Options for selecting records

Crystal Reports includes a very sophisticated formula language that you can use to specify virtually any type of record selection. However, you may not always need the flexibility in record selection that the formula language provides. The Select Expert is designed for such instances.

You can select records in one of two ways:

- [Using the Select Expert \[page 156\]](#)
- [Using formulas \[page 158\]](#)

Once you are familiar with the Select Expert and formulas, you can use your record selection techniques to improve the performance of your reports.

For additional performance tips and advanced record selection strategies, see [Using enhanced record selection formulas \[page 144\]](#).

For additional performance tips and advanced record selection strategies, see “Using enhanced record selection formulas” in the Designing Optimized Web Reports chapter of the *Crystal Reports User Guide*.

8.1.2 Determining which field(s) to use

When you select records, you are basing your report only on those records that meet some conditions that you have set. You base those conditions on the kind of information you want in the finished report.

Assume, for example, that you want a report that only shows data from California. The challenge is to find the best way to identify those records that come from California.

- If the table used in a report has a State or Region field, you can specify in your request that the program use only those records in which the value in the state field is equal to California (Region is equal to CA).
- If the table does not have a State field and you still want to report only on California data, there may be another way to identify that data.
 - If the table has a Postal Code field, you could base your record selection on the range of ZIP codes that apply to California (Postal Code is between n and N).
 - If the table has an Area Code field, you could base your record selection on California Area Codes (Area Code is one of x, y,...,z).

Note

If the Area Code is stored in the telephone number field, you will not be able to do this same record selection using the Select Expert based on the Area Code. You will have to create a record selection formula using the formula language to extract the Area Code part of the phone number and then do record selection based on that.

Note

As a general rule, if you can base your record selection on a number of fields (as in this example), you should select an indexed field instead of a field that is not indexed for better performance.

8.1.3 Using the Select Expert

The Select Expert makes it easy to specify the records you want included in your report. When you work with the Select Expert, you select the field to which you want to apply selection conditions and then you specify those conditions.

The Select Expert can be used to set up simple record selection requests. For example:

- Customers from Arizona.
- Orders in the first quarter.
- Sales over \$10,000.

The Select Expert can also be used to set up some sophisticated requests:

- Customers whose names start with "A", "M", or "S."
- Customers from California or Florida who ordered in July.

These are all range limit requests. One or more constants define the range. The program compares the field value in each record to the constant(s) and rejects records with values outside the range. The report is limited to values within the range. You can set up all of these types of record selection requests without any previous knowledge of the formula language.

Note

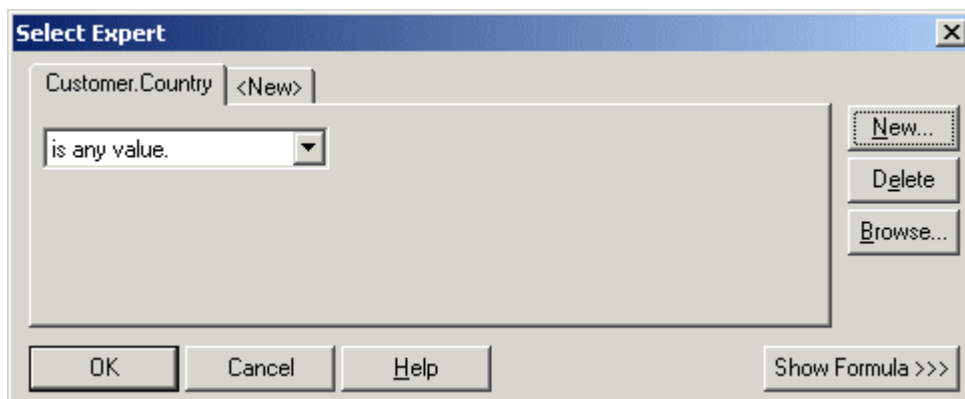
The Select Expert can be used to set up both record selection and group selection requests. When a group name or summary field is selected, the program knows that the selection criteria set up is intended for group selection. In all other cases, the program knows that you are setting up record selection.

8.1.3.1 To set up record selection using the Select Expert

Procedure

1.  On the Expert Tools toolbar, click *Select Expert*.

The Select Expert dialog box appears.



Note

If you click the Select Expert button without first highlighting a field in your report, the Choose Field dialog box appears. Highlight the field on which you want to base record selection and click OK. The Select Expert appears.

2. Use the drop-down lists to enter your selection criteria for the indicated field.
3. Click *OK* when finished.

→ Tip

To base record selection on more than one field, click the New tab. Select the next field from the Choose Field dialog box.

A selection formula is generated based on your specifications, limiting the report to the records you indicated.

Note

To view the selection formula, click the Show Formula button. The Select Expert expands to show the formula. You can modify the formula in the area that appears, or you can click the Formula Editor button to modify it in the Formula Editor.

8.1.4 Using formulas

Click a link to view one of the topics in this section:

- [To set up record selection using a formula \[page 158\]](#)
- [Further information about formulas \[page 158\]](#)

8.1.4.1 To set up record selection using a formula

Procedure

1. On the *Report* menu, point to *Selection Formulas* and click *Record*.
2. In the Record Selection Formula Editor, enter the formula by typing in the components or selecting them from the component trees.

Note

The resulting formula must be Boolean; that is, it must return either a True or False value.

3.  Click *Check* to identify any errors in the formula.
4. Fix any syntax errors the Formula Checker identifies.
5. When the formula has the correct syntax, click *Save*.

8.1.4.2 Further information about formulas

- For sample record and group selection templates, see [Using formula templates \[page 160\]](#).
- For complete instructions on creating formulas, see [Using Formulas \[page 687\]](#).
- For advanced record selection strategies and performance tips, see [Using enhanced record selection formulas \[page 144\]](#).
- For advanced record selection strategies and performance tips, see “Using enhanced record selection formulas” in the Designing Optimized Web Reports chapter of the *Crystal Reports User's Guide*.

8.1.5 Interaction of the Select Expert and the Formula Editor

The Select Expert and the Record/Group Selection Formula Editor are interactive. That is, record selection criteria you enter via the Select Expert automatically generates a record selection formula which can be reviewed and modified. Likewise, record selection formulas and modifications to existing record selection formulas automatically update the selection criteria in the Select Expert.

Because of this interactivity, you can use the two tools together as a tutorial for learning the formula language.

8.1.5.1 To view the Select Expert formula

Procedure

1. Right-click the field on which you want to view record selection.
2. Click [Select Expert](#).
3. Click the [Show Formula](#) button.

The Select Expert expands so you can review the formula the program generated based on your selection criteria.

The screenshot shows the 'Select Expert' dialog box. At the top, the title bar says 'Select Expert'. Below it, there's a text field containing 'Customer.Last Year's Sales' and a '<New>' button. Below this, there's a dropdown menu showing 'is less than' and a text field showing '\$1,000.00'. To the right of these fields are three buttons: 'New...', 'Delete', and 'Browse...'. Below these fields are three buttons: 'OK', 'Cancel', and 'Help'. To the right of these buttons is a button labeled 'Hide Formula <<<'. Below these buttons, there are two radio buttons: 'Record Selection' (which is selected) and 'Group Selection'. To the right of these radio buttons is a button labeled 'Formula Editor...'. At the bottom, there's a large text area containing the formula '{Customer.Last Year's Sales} < \$1000.00'.

4. Click the [Hide Formula](#) button when you are done with your review.
5. Use the Select Expert to change your selection formula.
6. Review the updated formula by clicking the [Show Formula](#) button again.

7. To make changes to the formula, click the [Formula Editor](#) button in the expanded Select Expert and use the tools in the Formula Workshop to make your formula changes.

Results

ⓘ Note

Selection formula components that do not fit any of the fixed criteria in the Select Expert will not be translated. For example, if part of your record selection formula extracts the last four characters in a customer number, the section of the formula code that performs that extraction will not be converted to Select Expert selection criteria.

8.1.6 Saved data selection formulas

Saved data selection formulas filter report data after the records have been stored in the report. Like record selection formulas, they can be created using both the [Select Expert](#) and formulas.

Unlike record selection formulas, any change to a saved data selection formula does not cause a refresh to the database. Instead, the report's saved data is used for all subsequent filtering. Saved data selection formulas are used with parameters to create interactive report filters that you can customize.

Optional parameters

You can combine saved data selection formulas and optional parameters. For more information on optional parameters, see [Optional parameters \[page 833\]](#).

8.2 Using formula templates

Click a link to view one of the topics in this section:

- [Record selection formula templates \[page 160\]](#)

8.2.1 Record selection formula templates

The following sample formulas can be used as templates to help you create your own selection formulas using the Formula Workshop. These examples illustrate different selections that you could do, not necessarily the best selections.

8.2.1.1 Selecting records using character strings

```
{file.FIELD} startswith "C"
```

Selects those records in which the value in the {file.FIELD} field begins with the character "C" (includes values like CyclePath, Corp. and Cyclist's Trail Co.; excludes values like Bob's Bikes Ltd. and Feel Great Bikes, Inc.).

```
not ({file.FIELD} startswith "C")
```

Selects those records in which the value in the {file.FIELD} field does not begin with the character "C" (includes values like Bob's Bikes Ltd. and Feel Great Bikes, Inc.; excludes values like CyclePath, Corp. and Cyclist's Trail Co.).

```
"999" in {file.FIELD}[3 to 5]
```

Selects those records in which the 3rd through 5th digits of the {file.FIELD} field is equal to "999" (includes values like 10999, 70999, and 00999; excludes values like 99901 and 19990).

```
"Cycle" in {file.FIELD}
```

Selects those records in which the value in the {file.FIELD} field contains the string "Cycle" (includes values such as CyclePath Corp. and CycleSporin, Inc.; excludes values like Cyclist's Trail Co. and Feel Great Bikes, Inc.).

8.2.1.2 Selecting records using numbers

Single values

```
{file.FIELD} > 99999
```

Selects those records in which the value in the {file.FIELD} field is greater than 99999.

```
{file.FIELD} < 99999
```

Selects those records in which the value in the {file.FIELD} field is less than 99999.

Range of values

```
{file.FIELD} > 11111 and {file.FIELD} < 99999
```

Selects those records in which the value in the {file.FIELD} field is greater than 11111 but less than 99999 (neither 11111 or 99999 is included in the range of values).

```
{file.FIELD} >= 11111 and {file.FIELD} <= 99999
```

Selects those records in which the value in the {file.FIELD} field is greater than 11111 but less than 99999 (both 11111 and 99999 are included in the range of values).

8.2.1.3 Selecting records using dates

The Month, Day, and Year functions can all be used in examples like the following:

```
Year ({file.DATE}) < 1999
```

Selects those records in which the year found in the {file.DATE} field is earlier than 1999.

```
Year ({file.DATE}) > 1992 and Year ({file.DATE}) < 1996
```

Selects those records in which the year found in the {file.DATE} field falls between 1992 and 1996 (1992 and 1996 not included).

```
Year({file.DATE}) >= 1992 and Year({file.DATE}) <= 1996
```

Selects those records in which the year found in the {file.DATE} field falls between 1992 and 1996 (1992 and 1996 are included).

```
Month({file.DATE}) in 1 to 4
```

Selects those records in which the month found in the {file.DATE} field is one of the first four months of the year (includes January, February, March, and April).

```
Month({file.DATE}) in [1,4]
```

Selects those records in which the month found in the {file.DATE} field is the first or fourth month of the year (includes January and April, excludes February and March).

8.2.1.4 Selecting records using preset date ranges

The preset date ranges can be used to create selection formulas similar to these:

```
{file.DATE} in LastFullMonth
```

Selects those records in which the date found in the {file.DATE} field falls within the last full month. (If the month is May, this selects all records with an April date.)

```
not({file.DATE} in LastFullMonth)
```

Selects all records except those in which the date found in the {file.DATE} field falls within the last full month (if the month is May, this selects all records except those with an April date).

8.2.1.5 Selecting records using date/number/character combinations

These formulas simply "mix and match" formulas from the categories above.

```
"C" in {file.FIELD}[1] and Month({file.DATE}) in [1,4]
```


Selects those records in which the value in the {file.FIELD} field begins with "C" and the month is either January or April. For example, if this kind of formula was used with an order database, you could be asking for a report showing all customers whose names begin with "C" and who placed orders in January or in April.

```
"AOK" in {file.HISTORY}[3 to 5] and {file.OPENCRED} >= 5000
```

Selects those records in which the {file.HISTORY} field shows the characters "AOK" as the 3, 4, and 5 characters and the {file.OPENCRED} field (the amount of available credit) is at least 5000.

These templates can be used as they are (with your own data), or they can be combined to create complex formulas.

8.3 Pushing down record selection to the database server

The drivers provided with Crystal Reports for SQL data sources allow "pushing down" record selection to the database server. When you specify a record selection formula in a report based on an SQL data source, Crystal Reports analyzes it, generates a SQL query from it and passes the SQL query to the database server. Record selection is then carried out in two stages:

- The first stage of record selection takes place when the database server processes the SQL query and returns a set of records to Crystal Reports.
- In the second stage, Crystal Reports further evaluates locally the record selection formula for the set of records returned from the database server.

Since database servers are usually faster machines than workstations, it is to your advantage to specify record selection formulas that can be processed by the server in the first stage. Doing so minimizes record selection on the local machine in the second stage. This process is commonly known as "pushing record selection down to the database server." The following kinds of record selections can be pushed down to the server:

- Selections with indexed and non-indexed fields (indexed fields provide faster response).
- SQL queries with AND and OR clauses.
- SQL expression fields that carry out formula calculations for record selection. (For the types of SQL expressions supported by your SQL server, consult the documentation provided with the server.)
- For complete details on pushing your record selection formulas down to the database server, see [Using enhanced record selection formulas \[page 144\]](#).
- For complete details on pushing your record selection formulas down to the database server, see "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the *Crystal Reports User Guide*.

8.4 Troubleshooting record selection formulas

To troubleshoot your selection formula, you should first begin by making sure that all of the fields referenced in the selection formula are placed on your report. Then delete the selection formula and test it as you rebuild it, step by step.

8.4.1 To troubleshoot record selection formulas

Procedure

1. Write down the record selection formula on paper. You will use this written copy to help you reconstruct the selection formula one step at a time.
2. Remove the record selection formula from your report by deleting the formula from the Record/Group Selection Formula Editor in the Formula Workshop.
3. Click [Close](#) when finished in the Formula Workshop.
4. Make certain that all fields referenced in the record selection formula (the selectors) are on the report physically and are not hidden.

For example, if one of the selectors is:

```
{customer.POSTAL CODE} > "80000"
```

but the {customer.POSTAL CODE} field is not used on your report (as in the case of a sales report that uses the postal code to define territories but does not include the postal code in the report data), then insert the {customer.POSTAL CODE} field into the report.

Or, if one of the fields referenced in the selection formula is on the report but is hidden, unhide it by deselecting the Suppress option in the Format Editor for that field.

5. Print the report and verify that the data in those fields which are referenced in the selection formula print satisfactorily. Make certain that all the data prints. For example, if there are x total records in the database you should have x records printing for each of the referenced fields. This establishes a baseline against which you can compare the results of printing with the selection formula.
6. When you are sure that you are getting satisfactory results without using the selection formula, you can enter the selection formula using only one of the selectors.

For example, to use this as the final selection formula:

```
{customer.POSTAL CODE} > "80000" and {customer.CONTACT LAST NAME}[1] = "C"  
and {customer.LAST YEAR'S SALES} >= 5000
```

this formula will select all of those records that show a Postal code greater than 80000, a value in the {customer.CONTACT LAST NAME} field beginning with "C", and a value in the {customer.LAST YEAR'S SALES} field greater than or equal to 5000.

You might start with this as the first test selection formula:

```
{customer.POSTAL CODE} > "80000"
```

Print the report and evaluate the data that prints when you have only one selector activated. Does the {customer.POSTAL CODE} field show only ZIP codes greater than 80000?

- If it does, then you know that this part of the selection formula is working.
 - If it does not, then troubleshoot this part of the selection formula.
7. Once the selection formula with one selector activated is working properly, add a second selector. For example, the new selection formula might look like this:

```
{customer.POSTAL CODE} > "80000" and {customer.CONTACT LAST NAME}[1] = "C"
```

8. Preview the report and evaluate the data that prints when you have two selectors activated. Evaluate the data in the {customer.CONTACT LAST NAME} field (since you already evaluated {customer.POSTAL CODE} in the last step).

Does the {customer.CONTACT LAST NAME} field show only text strings beginning with the letter "C"?

- If it does, then this part of the selection formula is working.
 - If it does not, then troubleshoot this part of the selection formula.
9. Once the selection formula with two selectors activated is working properly, add a third selector, then a fourth, and so on, until you have tested each selector in the selection formula.

8.4.2 Correcting selections that do not generate data

You may encounter a situation in which you create a record selection formula and, while header and footer information prints on your report, no detail information appears. The problem is the selection formula is rejecting all records. This usually occurs because of an error in the creation of the selection formula.

8.4.2.1 Correcting uppercase/lowercase inconsistency

Record selection formulas are case sensitive. That is, "Bob" matches only with "Bob". It does not match with "bob", "BOB", "BoB", "bOB", "boB", or "BOb". Thus, if your selection formula is set to include only those records with "BOB" in the {customer.CONTACT FIRST NAME} field, but all the entries in the {customer.CONTACT FIRST NAME} field are mixed case ("Bob", for example), the selection formula will find no matches and thus will not print any details for the report.

You can solve this problem by using the UpperCase (str) or LowerCase (str) functions in your selection formula to convert field data to a consistent case before the program begins its selection. For example, if you were using this formula:

```
{customer.CONTACT FIRST NAME} = "BOB"
```

you can change the formula to this:

```
UpperCase({customer.CONTACT FIRST NAME}) = "BOB"
```

This second formula first converts the value of the {customer.CONTACT FIRST NAME} field to upper case characters and then checks to see if the resulting value in that field is equal to "BOB". Using this formula, any instance of the three letters "b," "o," and "B" will be a match, regardless of case, because the case will be converted to uppercase.

You could use the LowerCase function in a similar manner to match with "bob."

Check your selection formula closely and make sure you have the correct case for any text you are trying to match. If in doubt, use the UpperCase (or LowerCase) function to assure consistency and proper matching.

Another formula which performs much the same functions is the following:

```
"BOB" in UpperCase({customer.CONTACT FIRST NAME})
```

8.4.2.2 Unwanted spaces appear in selection formula

Spaces are characters, and when you include spaces in the search key of a record selection formula, the formula looks for records with the exact match in the selected field, spaces and all. For example, the following formula:

```
"Mr . " in {customer.TITLE}
```

will not find any matches with the form of address "Mr." because there is an extra space in the search key between the letter "r" and the period. Likewise, "Ph. D" will not match "Ph.D".

Check your selection formula closely, and make sure that the spaces in the selection formula match the spaces in the fields you are trying to match.

8.4.2.3 Using If statements in selection formulas

When creating record selection formulas that include If statements, always include the Else keyword; otherwise, no records or unexpected records may be returned. For example, a record selection such as `If {parameter field} = "less than 100" then {field} < 100` evaluates as False and returns no records. To correct this problem, complete the formula with `Else True`.

9 Sorting, Grouping, and Totaling

Sorting, grouping, and totaling are the steps that turn disorganized data into useful information on a report. This section describes the types of sorting, grouping, and totaling you can do within a report.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

9.1 Sorting data

Sorting means placing data in some kind of order to help you find and evaluate it.

When you first insert a database field into your report, the data within the fields appears in the order in which it was originally entered into the database. Locating information in this kind of report is difficult. It is much easier to review or find information when you can see it sorted in a logical format. For example, you may want to have a customer list sorted alphabetically by name or by country.

9.1.1 Understanding sort options

When you sort, the program asks you to define two things:

- The field you want the sort to be based on (sort field).
- The sort direction.

9.1.1.1 Sorting data

Sorting means placing data in some kind of order to help you find and evaluate it.

When you first insert a database field into your report, the data within the fields appears in the order in which it was originally entered into the database. Locating information in this kind of report is difficult. It is much easier to review or find information when you can see it sorted in a logical format. For example, you may want to have a customer list sorted alphabetically by name or by country.

9.1.1.2 Sort field

A sort field is the field that determines the order in which data appears on your report. Almost any field can be used as a sort field, including formula fields. A field's data type determines the method in which the data from that field is sorted.

Note

You cannot sort on memo or BLOB fields.

Field Type	Sort Order
Single-character string fields	blanks
	punctuation
	numbers
	uppercase letter
	lowercase letters
Multiple character string field	two letters
	three letters
	four letters, and so on
	For example:
	• "123" comes before "124" • " " (blank) comes before "a" • "aa" comes before "aaa"
Currency fields	numeric order
Number fields	numeric order
Date fields	chronological order
DateTime fields	chronological order
	same-date values sorted by time
Time fields	chronological order
Boolean comparison fields	False values (0)
	True values (1)
Null values	null values
	non-null values

Note

If sorting and grouping are performed on the database server, sort order may vary when Unicode or UTF-8 data is used. The order applied depends on the rules in place for your data source. In certain cases, Unicode data is sorted by its binary value, but it can also be sorted according to a particular locale setting. Refer to the documentation for your data source for details on how Unicode data fields are sorted.

9.1.1.3 Sort Direction

Direction refers to the order in which the values are displayed, once sorted.

- **Ascending**
Ascending order means smallest to largest (1 to 9, A to Z, False to True). The program sorts the records in ascending order based on the values in the sort field you select.
- **Descending**
Descending order means largest to smallest (9 to 1, Z to A, True to False). The program sorts the records in descending order based on the values in the sort field you select.

9.1.2 Sorting single and multiple fields

In single field sorting, all the records used in the report are sorted based on the values in a single field. Sorting an inventory report by stock number or sorting a customer list by customer number are examples of single field sorts.

In multiple field sorts, the Report Designer first sorts the records based on the values in the first field selected, putting them in ascending or descending order as specified. When two or more records have the same field value in the first sort field, the program then sorts those records based on the value in the second sort field.

For example, if you choose to sort first by the {customer.COUNTRY} field and then by the {customer.REGION} field, both in ascending order, the report would appear with countries listed in alphabetic order, and regions within each country listed in alphabetic order. Any other fields, such as the postal codes within each region, would remain unsorted.

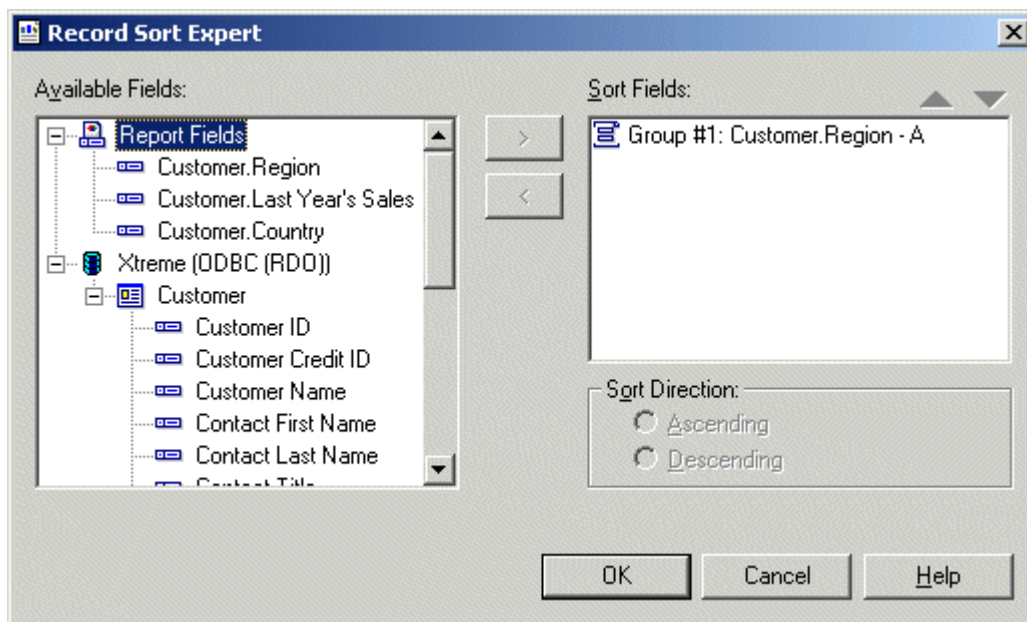
You create single or multiple field sorts using the same procedure.

9.1.2.1 To sort your data

Procedure

1.  On the *Report* menu, click *Record Sort Expert*.

The Record Sort Expert appears.



2. Highlight the field to be sorted from the Available Fields area.
3. Click the > arrow.

The selected field is added to the Sort Fields list.

4. Specify the sort direction.
5. If sorting by more than one field, highlight the second field you want the data to be sorted by and add it to the Sort Fields list.
6. If you want to change the order of fields in the Sort Fields list, highlight the field you want to move and click the arrow buttons to move it up or down.

→ Tip

The order of the fields listed in the Sort fields box is the order by which data will be sorted.

7. As you add each field to the Sort Fields list, specify the sort direction.
8. Click **OK** when finished.

Records are sorted based on the values in the Sort Fields list.

9.1.3 Sort controls

You may want to design your report so that users can modify a sort field or sort direction without refreshing information from the database. You can do this using a [Sort Control](#).

A [Sort Control](#) is particularly useful for several reasons:

- It allows users to sort report data for further analysis without leaving the canvas of the report.
- It eliminates processing demand on the database.
- It reduces the time users spend waiting for data to be sorted.

- It allows users to sort fields in the report even if they do not have a connection or rights to access the database at the time of viewing.

When planning to include a *Sort Control* in your report, it is important to consider the following points:

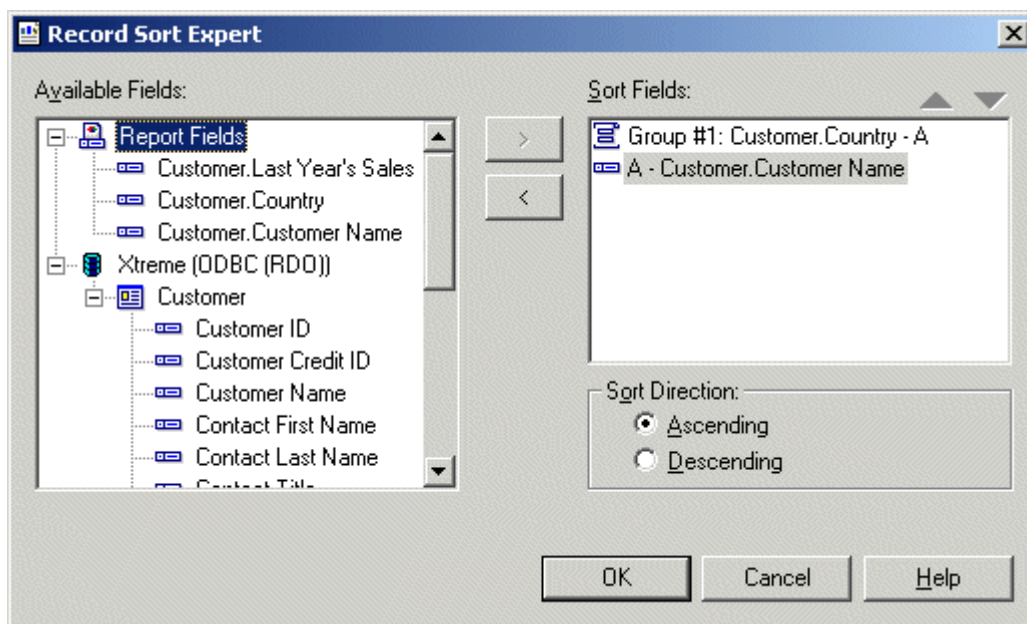
- Sort controls apply across an entire record set; you cannot sort only one group.
- Sort controls cannot be used in sub-reports.
- Sort controls cannot be created within a cross-tab or an OLAP grid.
- The use of sort controls causes any open drill-down tabs to close (a warning prompt alerts users).

9.1.3.1 To create a Sort Control

Procedure

1.  Once the data is grouped, on the *Report* menu, click *Record Sort Expert*.

The *Record Sort Expert* appears.



Note

Sort fields that begin with "Group" specify that the sort was done automatically when the data was grouped.

2. Highlight the fields that you want to sort by and click the > arrow to add them to the *Sort Fields* list. The order of the fields in the *Sort Fields* list is the initial order by which data is sorted.

Note

To enable a *Sort Control* on a field, that field must be included in the *Sort Fields* list.

3. Click [OK](#) when finished.
4. Create your [Sort Control](#):
 - If you want to use an existing text object:
 1. On your report, right-click the text object that you would like to use as a [Sort Control](#).
 2. Click [Bind Sort Control](#).
The [Sort Control](#) dialog box appears.
 3. Select a sort field and click [OK](#).
 - If you want to add a new text object:
 1. On the [Insert](#) menu, click [Sort Control](#).
 2. Select a sort field and click [OK](#).
 3. Click and drag your cursor to the report location where you would like to insert the [Sort Control](#).
 4. Enter a name for the new [Sort Control](#), and click outside the text object to complete the process.
5. Repeat Step 4 for any other sort controls you want to create.

Results

Users can sort the fields you have selected by clicking the sort arrows that appear beside the text object. The report promotes the selected field temporarily to the top of the sort order. When a second [Sort Control](#) is used, the first is returned to its original position in the sort order and the second is promoted to the top sort.

Note

Grouping hierarchy is not affected by [Sort Control](#).

9.1.3.2 To remove a sort control

Procedure

1. Right-click the text object that contains the existing [Sort Control](#).
2. Click [Bind Sort Control](#).
The [Sort Control](#) dialog box appears.
3. Click [< Not Interactive >](#).
4. Click [OK](#) to return to your report.

Results

The [Sort Control](#) is removed from the report.

9.2 Grouping data

Grouped data is data that is sorted and broken up into meaningful groups. In a customer list, for example, a group might consist of all those customers living in the same Zip Code, or in the same Region. In a sales report, a group might consist of all the orders placed by the same customer, or all of the orders generated by a particular sales representative.

9.2.1 Grouping data

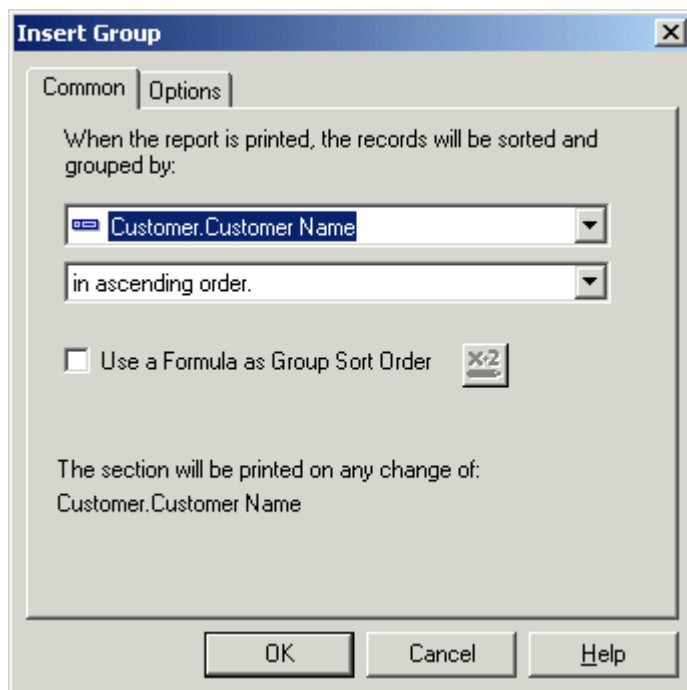
Grouped data is data that is sorted and broken up into meaningful groups. In a customer list, for example, a group might consist of all those customers living in the same Zip Code, or in the same Region. In a sales report, a group might consist of all the orders placed by the same customer, or all of the orders generated by a particular sales representative.

9.2.1.1 To group data

Procedure

1.  On the *Insert* menu, click *Group*.

The Insert Group dialog box appears.



2. Select the field you want the data grouped by from the top drop-down list.
3. Select the sort direction from the second drop-down list.

Note

If you want to use a formula for the sort order of your group, see [Sorting groups conditionally \[page 175\]](#).

4. If you want to show a different value in the group header, click the *Options* tab.

Note

By default, the group header of the report will display the value of the field you are grouping on.

5. Select the *Customize Group Name Field* check box and choose a new group name.

For example, if you grouped by {Customer.CustomerID}, at each change of a group you will see the corresponding customer ID. If you want to display a different value (customer ID as well as customer name), customize the group name field by choosing an alternate data field, or creating a formula.

Tip

To hide the group header name, right-click the group header, select Format Field, and click Suppress on the Common tab of the Format Editor.

6. Click *OK*.

If your records within each group are unsorted, you need to sort the records within each group. See [Sorting records within groups \[page 177\]](#).

Related Information

[Grouping data in intervals \[page 181\]](#)

[Creating group headers \[page 202\]](#)

9.2.2 Creating custom groups

Usually data is sorted based on the values from a field in the report. Sometimes, however, you may not want to group data based on the values found in one of the fields on your report. For example:

- The field you want to group on does not exist.
For example, your report may contain a City field and a State field, but no Country field, although you need to group by country.
- The field exists, but you do not want to group on the values in that field.
For example, you may have a Color field on your report that includes specific color names (Logan Green, Sky Blue, Emerald Green, Navy Blue, and so on) but you want all shades of each color to appear as a single group (Greens, Blues, Reds, and so on). In this case you can build custom groups and manually assign the records you want to be in each group.

- The field exists, but you want to select specific values or ranges of values for each group.
For example, you might want one group to contain records where gross sales are less than a certain value, a second group where gross sales are greater than a certain value, and a final group where gross sales fall between two values. In this case, you can build your groups using the same range of selection facilities that are available to you for building record selection queries.

Specified order grouping provides a solution to these custom sorting and grouping challenges. It enables you to create both the customized groups that will appear on a report and the records that each group will contain. The only limitation is that a record can be assigned to only one group.

To follow a tutorial where you create custom groups to rank customers by the amount of business they did in the previous year, see [Grouping data in intervals \[page 181\]](#).

9.2.2.1 To create a custom group

Procedure

1.  On the *Insert* menu, click *Group*.
The Insert Group dialog box appears.
2. Select the field you want the data grouped by from the top drop-down list.
3. Select *in specified order* as your sort option from the second drop-down list.
4. On the *Specified Order* tab, enter the name of the group in the *Named Group* field.
5. Click *New*.
6. In the Define Named Group dialog box, use the drop-down lists to select the data to be part of the group.
7. Click the *<New>* tab to add more selection criteria to your specified group, if necessary.
8. Click *OK*.
9. Click *New* to create more custom groups as necessary.
10. Click the *Others* tab to specify how you want to organize the data that is not part of the group(s) you defined.
11. Click *OK*.

9.2.3 Sorting groups conditionally

Although it is often sufficient to choose the sort order for a group in your report when you create the group, there may be times when you want your users to choose their own group sort order.

For example, if you group a sales report by invoice date, you might want your users to choose whether they see that data from earliest invoice date to most recent invoice date (ascending order) or most recent invoice date to earliest invoice date (descending order).

To create a conditional group sort order for this report, you might create a new parameter to prompt for the user's selection, and then pass this selection to a conditional group sorting formula.

9.2.3.1 To sort a group conditionally

Procedure

1. Open or create the report that you want to sort conditionally.

In the case of the example mentioned in this section, open or create a report that contains invoice information.

2.  In the Field Explorer, select *Parameter Fields*, and click *New*.

3. Create a string parameter that contains the sorting options that you want your users to see.

For this example, create a parameter called Sort Order that has two values: Ascending and Descending.

→ Tip

For information about how to create parameters, see [Creating a parameter with a static prompt \[page 839\]](#).

4. Click *OK* to save your parameter.

5.  On the *Insert* menu, click *Group*.

6. In the Insert Group dialog box, select the field that you want to group on.

For this example, select your invoice date field.

7.  Select *Use a Formula as Group Sort Order*, and click the Conditional Formula button next to it.

ⓘ Note

This option is not available if you choose "in specified order" as the sort order for your group.

8. In the Formula Workshop, enter your conditional formula text.

For this example, enter this text:

```
If {?Sort Order} = "Ascending" then crAscendingOrder else crDescendingOrder
```

9. Click *Save and close* to return to your report.
10. Click *OK* to save your group.
11. When you are prompted to select a Sort Order, select the option that you want, and click *OK*.

Results

Your report appears with groups for the field that you selected in the Insert Group dialog box and sorted in the order that you selected in your parameter prompt. To select a different sort order, click the Refresh button and choose "Prompt for new parameter values." The groups in your report are resorted to comply with the new order option that you selected.

9.2.4 Sorting records within groups

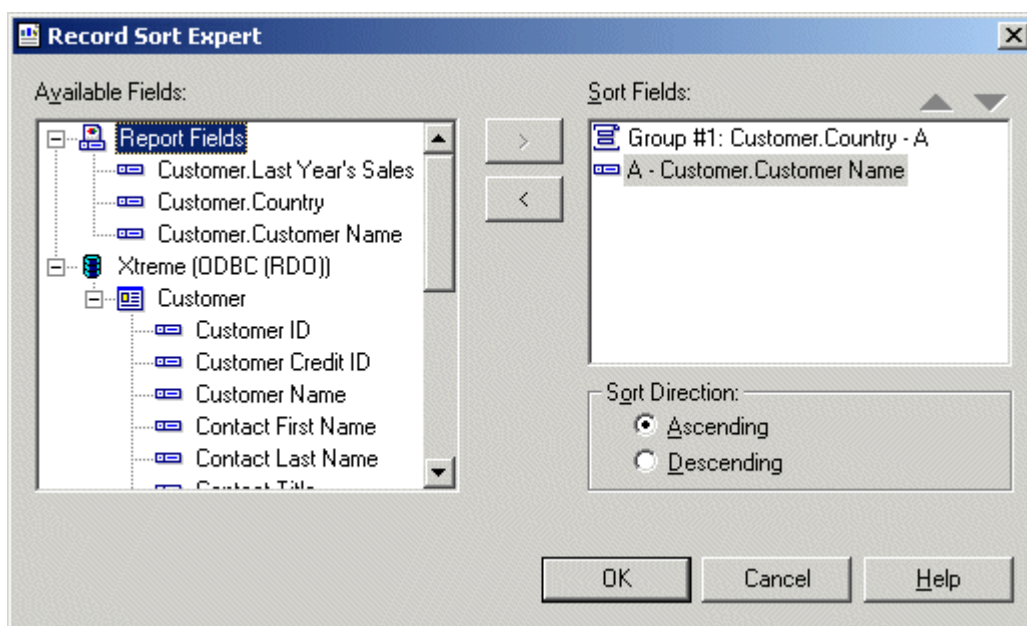
Once you have grouped your data, you can easily sort the records within the groups to further organize the information.

9.2.4.1 To sort records within groups

Procedure

1.  Once the data is grouped, on the *Report* menu, click *Record Sort Expert*.

The Record Sort Expert appears.



Note

Sort fields that begin with "Group" specify that the sort was done automatically when the data was grouped.

2. Highlight the field you want the records within the groups sorted by and click the > arrow to add it to the *Sort Fields* list.

Note

The order of the fields in the Sort Fields box is the order by which data will be sorted.

3. Specify the sort direction.

4. Click [OK](#) when finished.

9.2.5 Group selection

When you group or summarize data, all the groups in the report are included by default. There may be times, however, when you do not want to include all the groups. For example:

- You might want to see only those groups that have certain group names, or whose summarized values meet a certain condition.
- You might want to see only the groups with the highest summary values, or the lowest.

You can select the groups that appear in the report in two different ways:

- By using the Select Expert.
- By using selection formulas.

Note

For faster results, limit records through record selection before creating groups. See [Selecting records \[page 155\]](#).

9.2.5.1 Using the Select Expert

The Select Expert can be used to select groups of records in the same way that you select individual records.

When you are setting up group selection criteria, instead of basing the selection criteria on standard fields, as you do for record selection, you base the criteria on group name fields or summary fields.

- If you have grouped your data but have not summarized it, you can only set up group selection based on the group name field. For example, you may want to select only those groups whose Region is Massachusetts:

```
GroupName ({Customer.REGION}) = "MA"
```

- If you have summarized your data, you can set up group selection based on either the group name field or the summary field. For example:

```
Sum({Customer.LAST YEAR'S SALES}, {Customer.REGION}) > 10000
```

Note

The Select Expert can be used to set up record selection and group selection requests. When a group name or summary field is selected, the program knows that the selection criteria you set up is intended for group selection. In all other cases, the program knows that you are setting up record selection.

9.2.5.1.1 To set up group selection using the Select Expert

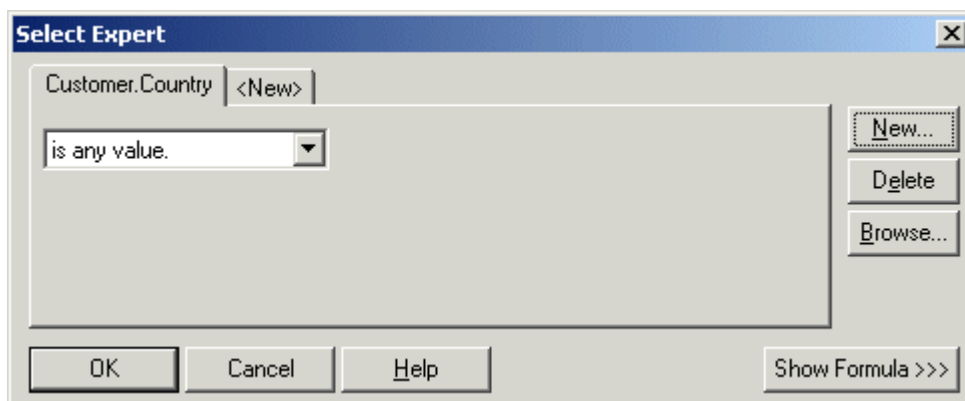
Procedure

1.  Right-click the summary field on which you want to base group selection and choose [Select Expert](#) from the shortcut menu.

The Select Expert dialog box appears.

Note

If you click the Select Expert button without first highlighting a summary field in your report, the Choose Field dialog box appears.



2. Use the drop-down list to enter your selection criteria for the indicated field.
3. To base the group selection on more than one field, click the [New](#) tab and choose the next field from the Choose Field dialog box.

Note



If you have not already previewed the report or refreshed the data, there will not be any data saved with the report. Without the data, the program cannot calculate group values; thus, no values appear when you click the arrow in the right drop-down list. In this situation, you will have to type in the values you want. If you want real values to work with, you will need to preview your report first. This will calculate the actual summary values available for you to work with.

4. When you are finished, click [OK](#) in the Select Expert to return to the report.

9.2.5.2 Using selection formulas

With the Group Selection Formula Editor, you can build your group selection request using group fields, group name fields, and other formulas. As with record selection formulas, your only restriction is that the formula you create must be Boolean; that is, it must return either a True or False value.

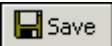
9.2.5.2.1 To create a record or group selection formula

Procedure

1. On the Report menu, point to [Selection Formulas](#).
2. Click [Record](#) to create a record selection formula.
- or -
Click Group to create a group selection formula.
The Formula Workshop appears.
3. Enter your selection formula in the Group Selection Formula Editor.

Note

The resulting formula must be Boolean; that is, it must return either a True or False value.

4.  Click [Check](#) to identify any errors in the formula.
5. Fix any syntax errors the Formula Checker identifies.
6.  When the formula has the correct syntax, click [Save](#).

When the program runs the report, it will include only those records or groups of records that you specified.

9.2.5.3 Troubleshooting group selection formulas

In some cases, no values will print when using a group selection formula, even though there are values that match the selection criteria. Typically, in these cases:

- The group selection formula references another formula.
- The referenced formula is one that calculates the value of each group as a percentage of the total value of all groups (in other words a subtotal as a percentage of a grand total).

9.2.5.3.1 To correct a group selection formula

Procedure

1. Use Xtreme.mdb to create a report that includes the following fields:

```
{customer.CUSTOMER NAME}  
{customer.REGION}  
{orders.ORDER ID}  
{orders.ORDER AMOUNT}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

For each order, the report shows the customer that placed the order, the region in which that customer is located, the order ID number, and the amount of the order.

2. Group the report by the {customer.REGION} field.
3. Insert a summary that subtotals the {orders.ORDER AMOUNT} field for each {customer.REGION} group.

The program calculates a subtotal in the {orders.ORDER AMOUNT} field every time the region changes. See [Subtotaling \[page 197\]](#).

4. Insert a grand total on the {orders.ORDER AMOUNT} field to see the total value of all orders placed.
5. Create a formula named Percent that calculates each subtotal as a percentage of the grand total to see the value of the orders for each region group as a percentage of all orders placed.

```
Sum({orders.ORDER AMOUNT}, {customer.REGION})  
% Sum({orders.ORDER AMOUNT})
```

6. Place the formula in the Group Footer section of the report.
7. Reference the formula (@Percent) in a group selection formula that selects only those groups for which the percentage (of subtotal to grand total) is less than 5% in order to find out which regions individually contributed less than 5% of total sales:

```
{@Percent} < 5
```



When you click Check, you will receive the following error message:

This formula cannot be used because it must be evaluated later.

8. Instead of using the formula name (in this case @Percent) in the group selection formula, enter the formula itself (the formula named @Percent). Thus, instead of using the group selection formula:

```
{@Percent} < 5
```

use the group selection formula:

```
Sum({orders.ORDER AMOUNT}, {customer.REGION})  
% Sum({orders.ORDER AMOUNT}) < 5
```

Now when you print, only the regions that contributed less than 5% will print.

9.2.6 Grouping data in intervals

You may want to group your data into intervals. Age groups, time periods, and sales categories are some of the interval groupings that can be created. In this example, you will rank customers by the amount of business they did in the previous year.

This example uses specified order grouping. This kind of grouping lets you specify the records that will be included in each group. You define the intervals you want and the program will do the rest.

9.2.6.1 To group data in intervals

Procedure

1. To get started, create a report using the sample data, [Xtreme.mdb](#). Link the Customer table and place the following fields from left to right in the Details section:

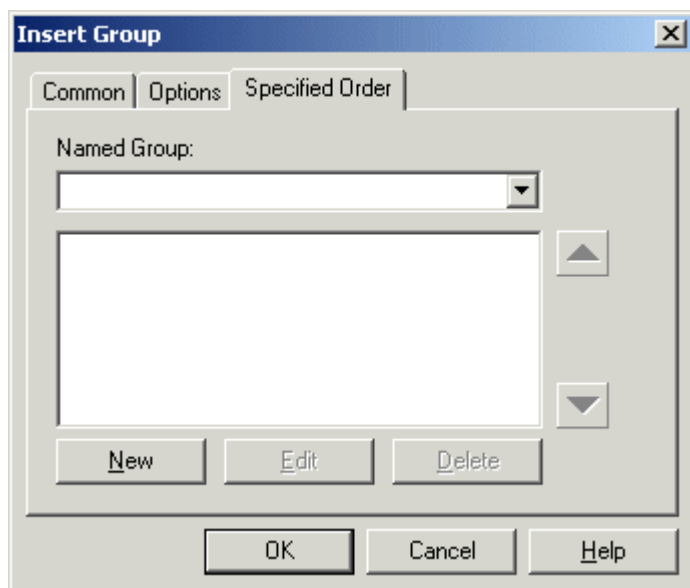
```
{customer.CUSTOMER NAME}  
{customer.REGION}  
{customer.POSTAL CODE}  
{customer.COUNTRY}  
{customer.LAST YEAR'S SALES}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

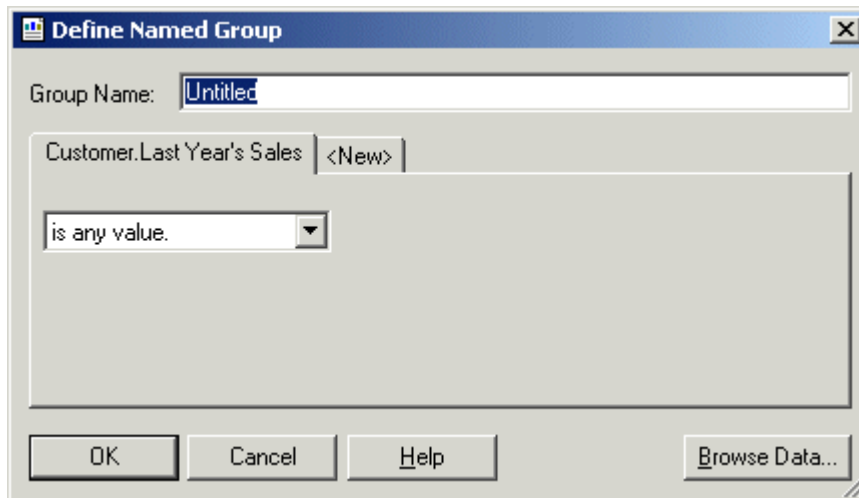
2. On the [Insert](#) menu, click [Group](#).
3. Because you want to set up intervals based on the previous year's sales, select Last Year's Sales as the sort-and-group-by field from the drop-down list on the Insert Group dialog box.
4. Select [in specified order](#) from the second drop-down list.

The Specified Order tab appears in the Insert Group dialog box.



5. Click the [New](#) button.

The Define Named Group dialog box appears.



6. Type "Less than \$10,000" in the *Group Name* field.

This is the name that will appear as the Group Name field value for the group.

7. Since the first group is to contain only those records that have a Last Year's Sales figure of less than \$10,000, set the fields so your condition reads:

is less than 10000

8. Click *OK* to return to the Specified Order Tab.
9. Click *New*.

The Define Named Group dialog box reappears.

10. Set up a second group that contains values between \$10,000 and \$25,000.

- Type "\$10,000 to \$25,000" in the Group Name field.
- Set the first field so the condition reads: *is between*.
- Specify a range of values:
 - Type "10000" in the top field.
 - Type "25000" in the bottom field.

11. Click *OK* to return to the Specified Order tab.
12. Click *New*.

The Define Named Group dialog box reappears.

13. Set up the final group that contains all values over \$25,000.

- Type "Over \$25,000" in the Group Name field.
- Set the first field so the condition reads: *is greater than*.
- Type "25000".

14. Click *OK* to return to the Specified Order Tab.
15. Click *OK*.

The report is grouped by interval in a specified order.

9.2.7 Grouping by the first letter of a company name

You might want to break your data into groups based on the first letter of the company name. For example, in a customer list you might want all the "A" customers in a group, then all the "B" customers, and so forth. To do this requires the use of a formula.

Do not worry if you are unfamiliar with formulas. This text will show you what formula you need here and how to enter it.

You can learn more about creating and editing formulas in [Formulas overview \[page 687\]](#).

You are going to create a formula that will extract the first letter of each customer's name. Then you will group the data using that formula field as the sort-and-group-by field. The program will sort the data based on the first letter of each customer name and start a new group whenever that letter changes.

9.2.7.1 To group data by the first letter of a company name

Procedure

1. To get started, create a report using the sample data, [Xtreme.mdb](#). Link the Customer table and place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}  
{customer.REGION}  
{customer.POSTAL CODE}  
{customer.COUNTRY}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2.  On the [View](#) menu, click [Field Explorer](#).
The Field Explorer dialog box appears.
3.  Select [Formula Fields](#) and click [New](#).
4. In the Formula Name dialog box, enter the name you want to identify the formula, for example "First Letter," and then click [OK](#).

The Formula Workshop appears with the Formula Editor active.

5. Type the following formula into the Formula text box:

```
{Customer.Customer Name}[1]
```

6. Click [Save and close](#) to return to your report.

7.  On the [Insert](#) menu, click [Group](#).

The Insert Group dialog box appears.

8. Select the formula field as the field you want your data grouped by from the first drop-down list.
9. Select the sort direction from the second drop-down list.
10. Click [OK](#).

You return to your report with the data grouped by the formula field as specified. The data is broken into groups based on the first letter in the customer's name. The formula provides a live header for every group.

For more information on live headers, see [Live headers \[page 203\]](#).

Related Information

[Formulas overview \[page 687\]](#)

[Grouping data \[page 173\]](#)

[Grouping data in intervals \[page 181\]](#)

[Creating group headers \[page 202\]](#)

9.2.8 Grouping data hierarchically

Characteristics of the data needed for hierarchical grouping

You can group data in a report to show hierarchical relationships. When you group data hierarchically, Crystal Reports sorts information based on the relationship between two fields. A hierarchical relationship must be inherent in the data that you use for the report:

- Parent and child fields must be of the same data type for the program to recognize a relationship between them.
- The data in the parent field must be a subset of the data in the child field.
- For the top level of a hierarchy to appear in a report, the value must appear in the child data, and the corresponding row in the parent data must be empty.
- There cannot be any circular logic in the data (that is, A cannot be related to B, while B is related to C, and C is related back to A).

For example, if you want to show the hierarchical relationship of the people who work in a department, you could group data by employee name (the child field) and specify the hierarchy by using the field that lists whom the employee reports to (the parent field). Your database tables might look like the following example.

Employee (child)	Supervisor (parent)
Jane	Thomas
Mina	
Gerard	Thomas

Employee (child)	Supervisor (parent)
Albert	Thomas
Thomas	Mina
Beth	Thomas
Teresa	Thomas
Gareth	Thomas
Valerie	Thomas
Gillian	Mina
Frances	Gillian
Ruth	Gillian
Margaret	Mina
Paul	Margaret
Charles	Margaret

The Employee and Supervisor fields contain overlapping data that implies a hierarchical relationship. There are 15 unique employee names, with four of these names also appearing as supervisors (Mina, Thomas, Gillian, and Margaret). Three employees report to Mina, seven employees report to Thomas, two employees report to Gillian, and two employees report to Margaret.

Note

Mina has no corresponding Supervisor. This data implies that Mina is a top-level supervisor who does not report to anyone else in this table.

If you group a Crystal report on the Employee field, you can further sort the data to show the hierarchical relationship between these employees and their supervisors.

Characteristics of a report that you have grouped hierarchically

In addition to a visual representation of the hierarchy inherent in your data, a Crystal report that you have grouped hierarchically has several other characteristics:

- When you drill down on a group in the hierarchy, the drill-down view also shows the records that are lower in the hierarchy.
- The report contains hierarchical group footers that include the records that are lower in the hierarchy of each group. You can summarize data across the hierarchies.
- Use the conditional-X-position feature to ensure that the indenting you set to show hierarchical relationships does not affect other fields in same section of your report.

- Hierarchical levels are supported in the formula language through the use of the GroupingLevel and HierarchyLevel functions.

Note

You cannot use hierarchical summaries in formulas.

9.2.8.1 To group data hierarchically

Procedure

1. Create or open a report that contains the data you want to group and sort hierarchically.

<u>Employee</u>	<u>Supervisor</u>
Jane	Thomas
Mina	
Gerard	Thomas
Albert	Thomas
Thomas	Mina
Beth	Thomas
Teresa	Thomas
Gareth	Thomas
Valerie	Thomas
Gillian	Mina
Frances	Gillian
Ruth	Gillian
Margaret	Mina
Paul	Margaret
Charles	Margaret

2. On the *Insert* menu, click *Group*.
3. In the Insert Group dialog box, select the field that you want to use as the basis of your hierarchy (the child field).

For example, if you want to see the hierarchical structure of a company's employees, select the employee field.

4. Select *in ascending order*.

By default, the group header of the report displays the value of the field you are grouping on.

5. If you want to show a different value in the group header, click the [Options](#) tab and select the [Customize Group Name Field](#) check box.

For example, if you grouped by the employee field, at each change of a group you will see the corresponding employee name. If you want to display a different value (employee ID instead of employee name), customize the group name field by choosing an alternate data field, or by creating a formula.

6. Click [OK](#).

The group you created is added to the report.

<u>Employee</u>	<u>Supervisor</u>
Albert	
Albert	Thomas
Beth	
Beth	Thomas
Charles	
Charles	Margaret
Frances	
Frances	Gillian
Gareth	
Gareth	Thomas
Gerard	
Gerard	Thomas
Gillian	
Gillian	Mina
Jane	
Jane	Thomas
Margaret	
Margaret	Mina
Mina	
Mina	
Paul	
Paul	Margaret
Ruth	
Ruth	Gillian
Teresa	
Teresa	Thomas
Thomas	
Thomas	Mina
Valerie	
Valerie	Thomas

7. On the [Report](#) menu, click [Hierarchical Grouping Options](#).
8. In the [Available Groups](#) list of the Hierarchical Options dialog box, select the group you want to organize hierarchically.
9. Select the [Sort Data Hierarchically](#) check box.
10. In the [Parent ID Field](#) list, select the field by which you want the Instance ID Field organized.

For example, for an employee hierarchical report, you might select the data field listing the supervisor to whom the employee reports.

Note

The Instance ID Field and Parent ID Field must be of the same data type. For example, if the Instance ID Field holds string data, then the Parent ID Field must also hold string data.

11. In the *Group Indent* field, enter the amount you want to indent for each subgroup.

The screenshot shows a dialog box titled "Hierarchical Group Options". On the left, under "Available Groups:", there is a list box containing "Employees.Employee". To the right of this list, there is a checked checkbox labeled "Sort Data Hierarchically". Below this checkbox, there are three fields: "Instance ID Field" with the text "Employees.Employee", "Parent ID Field" with a dropdown menu showing "Employees.Supervisor", and "Group Indent:" with a text box containing "0.25" and a unit label "in". At the bottom of the dialog are three buttons: "OK", "Cancel", and "Help".

The value that you enter in the Group Indent field affects all other objects that are in the same area as your hierarchical group. For example, if your report contains a salary field on the same line as the name of the employee, the salary field is also indented when you use the employee field to create a hierarchical group. To indent only the hierarchy records and not the other objects, leave this value as 0 (zero) and use the conditional-X-position feature.

12. Click **OK**.

The report data is now grouped hierarchically.

Note

In this example, the Details section in the report has been hidden to show the hierarchical sorting more clearly.

	<u>Employee</u>	<u>Supervisor</u>
Mina		
	Gillian	
	Frances	
	Ruth	
	Margaret	
	Charles	
	Paul	
	Thomas	
	Albert	
	Beth	
	Gareth	
	Gerard	
	Jane	
	Teresa	
	Valerie	

Results

The report is now grouped by employee name and further sorted to show the supervisor hierarchy. You can see that Mina, whose employee data had no supervisor data associate with it, has been sorted at the top of the list. Under Mina's name is each supervisor who reports to her; and under each of those supervisors is a list of employees who reports to him or her.

Note

The level at which a name appears in the hierarchy of this report is determined by the number of employees who report to him or her. Those employees with no one reporting to them are at the lowest level in the hierarchy.

If you want to, you can now calculate summary fields across your new hierarchical grouping. When inserting a subtotal, grand total, or summary in the usual manner, select the "Summarize across hierarchy" option. For more information, see [Summarizing grouped data \[page 192\]](#) and [Subtotaling \[page 197\]](#).

9.2.8.2 To indent a hierarchy without affecting other fields

Context

Note

For this procedure to work, you must ensure that the value in the Group Indent field of the Hierarchical Group Options dialog box is set to 0 (zero).

Procedure

1. Right-click the field that you grouped hierarchically, and select [Size and Position](#).
2.  Click the Conditional Formula button next to the X position value field.
3. In the Formula Workshop, enter your conditional X position formula text.

For example, enter formula text such as this:

```
numbervar hLevel := HierarchyLevel (1);  
numbervar deltaX := 0;  
if (hLevel > 1) then  
    deltaX := (hLevel - 1) * 0.4;  
deltaX := deltaX * 1440;
```

Note

There are several ways that you can create this formula; this code is an example of one way.

Note

Positions are measured in twips; there are 1440 twips in an inch.

Search the online help for "HierarchyLevel (GroupingLevel)" to learn more about the function used in this example.

4. Click [Save and close](#) to return to your report.
5. Click [OK](#) to save your position setting.

Results

Crystal Reports moves the values in your hierarchical data to new positions based on their level in the hierarchy, but leaves other objects that are on the same lines where you originally placed them.

9.2.9 Editing groups

Procedure

1. On the [Report](#) menu, click [Group Expert](#).
2. In the [Group By](#) list of the Group Expert dialog box, select the group you want to edit.
3. Click [Options](#).
4. In the Change Group Options dialog box, edit the group as necessary.
5. Click [OK](#) to close the Change Group Options dialog box and again to close the Group Expert dialog box.

The report reflects the changes you have made to the group.

9.3 Summarizing grouped data

One of the primary purposes for breaking data into groups is to run calculations on each group of records instead of on all the records in the report.

When the program summarizes data, it sorts the data, breaks it into groups, and then summarizes the values in each group. It does this all automatically.

The program includes a number of summarizing options. Depending on the data type of the field you plan to summarize, you can:

- Sum the values in each group.
- Count all the values or only those values that are distinct from one another.
- Determine the maximum, minimum, average, or Nth largest value.
- Calculate up to two kinds of standard deviations and variances.

For example:

- Customer list reports: determine the number of customers in each state. The summary would count the distinct customers in each state group.
- Order reports: determine the average order placed each month. The summary would calculate the size of the average order for each month group.
- Sales reports: determine the total sales per sales representative. The summary would sum or subtotal the order amounts for each sales representative group.

Note

You can also calculate summary fields across hierarchical groupings. To do so, select "Summarize across hierarchy" in the Insert Summary dialog box.

9.3.1 To summarize grouped data

Procedure

1.  On the *Insert* menu, click *Summary*.
The Insert Summary dialog box appears.
2. Select the desired field to summarize from the *Choose the field to summarize* list.
3. Select a summary operation from the *Calculate this summary* list.
4. Select a location in which to place the summary from the *Summary location* list.

Tip

You can create a new group for your report by clicking the Insert Group button.

→ Tip

You can add your summary to all group levels, or you can add it only once to the level you select as your location.

5. If you want to display your summary value as a percentage of a total, select *Show as a percentage of* from the Options area, and then select a total field from the list.

For more information about percentages, see [Percentages \[page 200\]](#).

6. If you want to summarize across a hierarchy, select *Summarize across hierarchy*.

For more information about hierarchies, see [Grouping data hierarchically \[page 185\]](#).

7. Click *OK* when finished.

Related Information

[Subtotaling \[page 197\]](#)

9.3.2 Ordering groups by summarized values

Groups can be organized in either ascending or descending order by summary value. For example, in an orders report, if you subtotal the order amount by state, you could order the groups:

- From lowest to highest order amount (ascending).
- From highest to lowest order amount (descending).

You order the groups in a report by summary values using the Group Sort Expert command on the Report menu.

9.3.2.1 To order groups by summary value

Procedure

1.  Click *Group Sort Expert* on the Expert Tools toolbar.

The Group Sort Expert dialog box appears with a tab for each of the groups in the report that has a summary.

2. Click the tab for the group you want to sort.
3. Select the *All* option from the drop-down list on the left.
4. Choose the summary that you want to base your selection on from the "based on" drop-down list on the right.

The "based on" drop-down list on the right is for those cases in which you have multiple summaries within a single group section. For example, in an orders report, you might sum and average the orders for each

customer and then display both the sum and the average in the same group section. In such a case, you would select the sum or the average from this drop-down list.

5. Specify the sort direction.
6. To select a second group sort, repeat Steps 2-5.

Results

When you run the report, the program will order the groups based on the summary value(s) specified.

9.3.3 Selecting top or bottom N groups or percentages

At times, you might want to show only the top or bottom groups or percentage values in a report: the fastest selling product lines, the countries that account for the top 25 percent of sales, the states that generate the most orders, and so on. Because this kind of selection is so popular, the program includes the Group Sort Expert for setting it up easily.

There is one other element to consider when setting up top N: what to do with all the records from other groups that do not fit the top N or bottom N criteria you have set. You need to decide whether to eliminate those records from your report entirely or to lump them all together into a single group. The program enables you to choose either option.

ⓘ Note

A report must contain a summary value in order to be able to perform a top N or bottom N selection. See [Summarizing grouped data \[page 192\]](#).

ⓘ Note

It is recommended that you do not use a group that is sorted hierarchically in a top or bottom N selection. The integrity of your hierarchical group sort might be affected by the top or bottom N selection.

9.3.3.1 To select the top or bottom N groups

Context

ⓘ Note

This procedure shows you how to select top or bottom N groups. Top or bottom percentages work the same way except that you define the percentage value instead of the number of groups.

Procedure

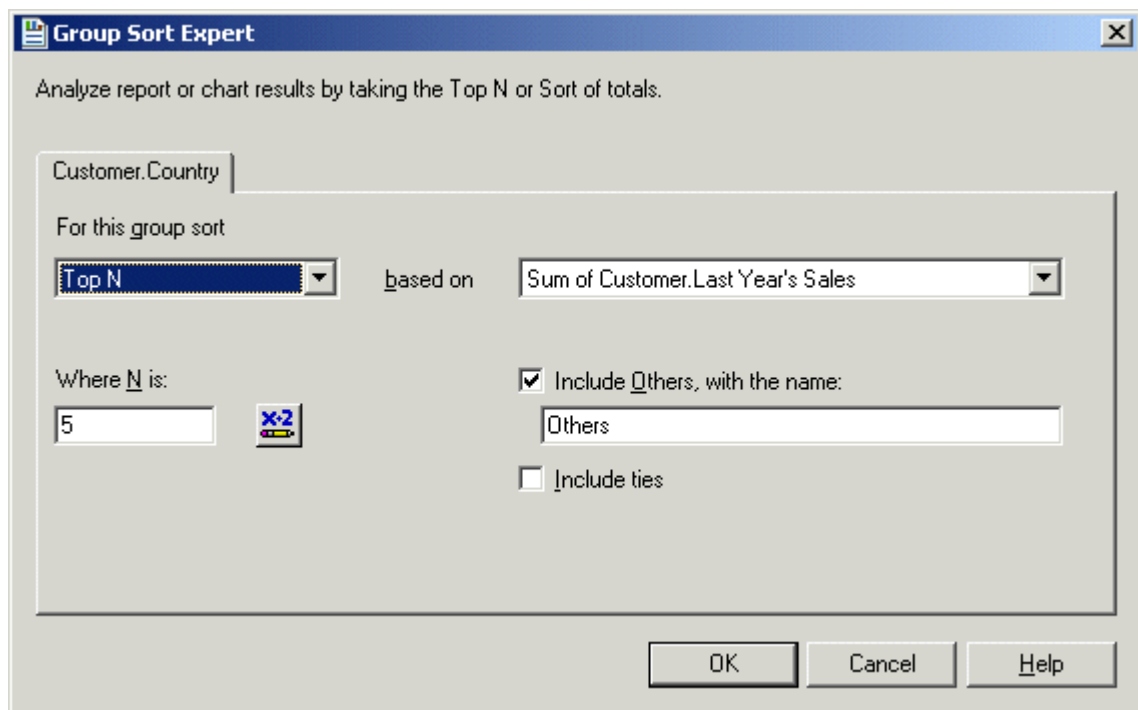
1. Create a report and summarize the data as desired. When you summarize the data, the program breaks the data into groups and summarizes each group.
 - With top N groups, you are instructing the program to display those groups that have the highest summary values (Top N).
 - With bottom N groups, you are instructing the program to display those groups that have the lowest summary values (Bottom N).

2.  Click the *Group Sort Expert* button on Expert Tools toolbar.

The Group Sort Expert appears with a tab for your group.

Note

If you have multiple groups, the program will display a tab for each of the groups.



3. Choose *Top N* or *Bottom N* from the first drop-down list.
4. Choose the summary that you want to base your selection on from the "based on" drop-down list on the right.

The "based on" drop-down list on the right is for those cases in which you have multiple summaries within a single group section. For example, in an orders report, you might sum and average the orders for each customer and then display both the sum and the average in the same group section. In such a case, you would select the sum or the average from this drop-down list.

5. In the *Where N is* text box, enter the number of groups you want to display. For example:

- To report on the three fastest selling product lines, set N to be equal to three.
 - To report on the five least productive sales regions, set N to be equal to five.
6. Click the *Include Others, with the name* check box and enter a name if you want to group all the other records into a single group.
 7. Select *Include ties* to accommodate groups whose summarized values are equal.

For example, suppose you had the following groups:

- Order 1 = 100
- Order 2 = 90
- Order 3 = 80
- Order 4 = 80

If you set your top N to be three, but you do not select "Include ties," your report will show Order 1, Order 2, and Order 3.

If, in the same scenario, you do select "Include ties," your report will show Order 1, Order 2, Order 3, and Order 4 even though N is set as three. In this way, the program accommodates the equal values of orders 3 and 4.

8. Click *OK* when finished.

When the program runs the report, it will include only those groups that you specified.

9.3.4 Selecting top or bottom groups or percentages conditionally

If you want your users to choose their own value for a Top N, a Bottom N, or a top or bottom percentage, create a parameter to prompt for the value that Crystal Reports can then pass to a conditional selection formula.

ⓘ Note

For groups, the top or bottom N value must be between 1 and 32,766.

ⓘ Note

For percentages, the top or bottom N value must be between 0 and 100.

9.3.4.1 To select a conditional number of groups or percentage value

Procedure

1. Open or create the report that you want to use for the conditional value or percentage selection.

Note

This report must contain a group and summary information as described in [Selecting top or bottom N groups or percentages \[page 194\]](#).

2.  In the Field Explorer, select *Parameter Fields*, and click *New*.
3. Create a number parameter.

Tip

For information about how to create parameters, see [Creating a parameter with a static prompt \[page 839\]](#).

4. Click *OK* to save your parameter.
5.  On the *Report* menu, click *Group Sort Expert*.
6. In the Group Sort Expert, select the type of group sort that you want.
Choose any option except No Sort or All.
7.  Click the Conditional Formula button next to the N or Percentage value field.
8. In the Formula Workshop, enter the parameter field that you created in step 3.
9. Click *Save and close* to return to your report.
10. Click *OK* to save your group sort.
11. When you are prompted to enter a number or percentage, enter the value that you want, and click *OK*.

Results

Your report appears with only the number of groups that match the value you entered in your parameter prompt. To enter a different value, click the Refresh button and choose "Prompt for new parameter values." The groups in your report are refreshed to show the new value that you entered.

9.4 Subtotaling

A subtotal is a summary that totals or sums numeric values in a group.

Note

If you are creating a subtotal using database tables that are grouped in a one-to-many linking relationship, you may need to use a running total instead of a subtotal. See [Creating running totals in a one-to-many linking relationship \[page 213\]](#).

9.4.1 Subtotaling data

In this example you will subtotal Last Year's Sales by Country.

9.4.1.1 To subtotal data

Procedure

1. To get started, create a report using the sample data, [Xtreme.mdb](#). Link the Customer table and place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}  
{customer.REGION}  
{customer.POSTAL CODE}  
{customer.COUNTRY}  
{customer.LAST YEAR'S SALES}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. Right-click the Last Year's Sales field, point to [Insert](#) and choose [Summary](#) from the shortcut menu.
The Insert Summary dialog box appears with the chosen field listed as the field to be summarized.
3. Click [Insert Group](#).
The Insert Group dialog box appears so you can specify the group you want to add to your report.
4. Choose the field you want the data grouped by, specify a sort direction and then click [OK](#) when finished.
5. On the Insert Summary dialog box, select the group you just created from the [Summary location](#) list and then click [OK](#).

The values in each group are now subtotaled.

9.4.2 Extending prices and subtotaling the extensions

In an orders report or invoice, you may need to extend the prices for individual line items and then subtotal the extensions. You do this by using a simple formula to extend the prices, and then you subtotal the formula field.

9.4.2.1 To extend the price and subtotal the extensions

Procedure

1. To get started, create a report using the sample data, [Xtreme.mdb](#). Link the Orders table and place the following fields from left to right in the Details section:

```
{Orders.CUSTOMER ID}  
{Orders_Detail.PRODUCT ID}  
{Orders_Detail.QUANTITY}  
{Orders_Detail.UNIT PRICE}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. To create the formula for extending the prices, go to the [View](#) menu and select [Field Explorer](#).

The Field Explorer dialog box appears.

3.  Select [Formula Fields](#) and click [New](#).

The Formula Name dialog box appears.

4. Enter the name you want to identify the formula, and then click [OK](#).

The Formula Workshop appears with the Formula Editor active.

5. Enter the following formula into the Formula text box:

```
{Orders_Detail.Quantity} * {Orders_Detail.Unit Price}
```

6. Click [Save and close](#) to return to the Field Explorer dialog box.

Your formula is listed in the Formula Fields node.

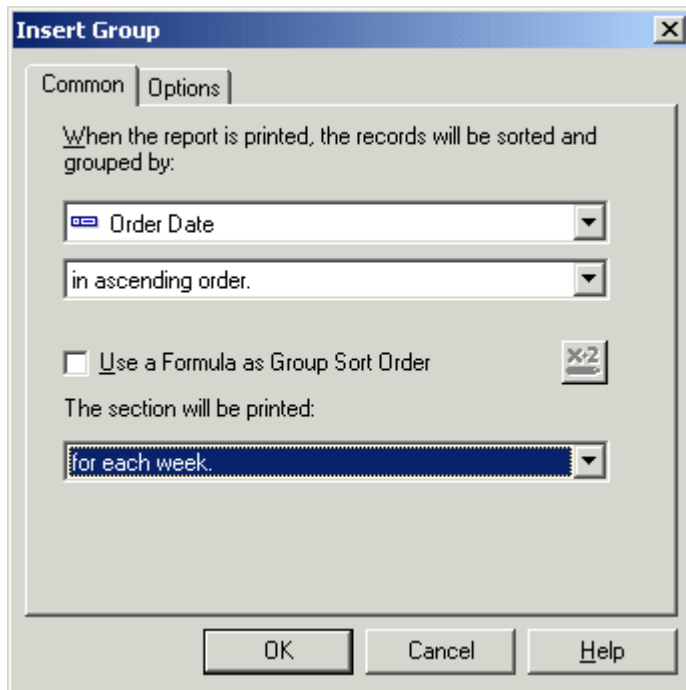
7. Drag your formula field to the right of the Unit Price field in the Details section of the report.
8. To subtotal your extension formula, right-click the formula field, point to [Insert](#) and choose [Summary](#) from the shortcut menu.

The Insert Summary dialog box appears.

9. Click [Insert Group](#) and create a group on the {orders.ORDER DATE} field.
10. Choose [for each week](#) as the group interval.

Note

"The section will be printed" drop-down box will not be activated until you choose the Order Date field.



11. Click [OK](#) to return to the Insert Summary dialog box.
12. Select the group you just created from the [Summary location](#) list and then click [OK](#).

Your data will be sorted by date and grouped in one week intervals.

9.5 Percentages

Click a link to view one of the topics in this section:

- [Calculating a percentage \[page 200\]](#)

9.5.1 Calculating a percentage

You can calculate the percentage of one group within a broader grouping. For example, you can show the percentage of sales in each city based on the total sales for each country. Or, you can see what percentage of the grand total of sales each country contributes.

9.5.1.1 To calculate a percentage

Procedure

1. On the *Insert* menu, click *Summary*.

The Insert Summary dialog box appears.

2. Select the field for which you want to calculate the sum.

For example, you may want to insert a field which calculates the sum of last year's sales.

3. Select *Sum* in the *Calculate this summary* list.
4. Select the summary location.

Note

The summary location cannot be Grand Total (Report Footer) when you are calculating a percentage.

5. Click the *Show as a percentage of* check box.
6. Select the group you want the percentage based on.

You can choose to show a percentage of a group within another group, or show a percentage of the grand total.

The screenshot shows the 'Insert Summary' dialog box. The 'Choose the field to summarize:' dropdown is set to 'Last Year's Sales'. The 'Calculate this summary:' dropdown is set to 'Sum'. The 'Summary location' dropdown is set to 'Group #1: Customer.Country - A'. In the 'Options' section, the 'Show as a percentage of' checkbox is checked, and its dropdown is set to 'Grand Total: Sum of Last Year's Sales'. The 'Summarize across hierarchy' checkbox is unchecked. The 'OK', 'Cancel', and 'Help' buttons are at the bottom.

7. Click *OK*. The summary percentage field is added to your report.

9.6 Group headers

Click a link to view one of the topics in this section:

- [Creating group headers \[page 202\]](#)
- [Suppressing group headers \[page 206\]](#)
- [Drilling-down on group headers \[page 206\]](#)

9.6.1 Creating group headers

Whenever you create a group, a subtotal, or a summary, the program creates both a Group Footer (GF) section (where it places any subtotal or summary value), and a Group Header (GH) section (where it automatically places the group name/header). Group Headers are useful, even necessary, if you want your report data to be clear and easily understood. Though the program creates a group header automatically, you may find that you would like to change or modify the header to suit your needs.

In this section, you will learn how to create the most common kinds of group headers.

9.6.1.1 Standard headers

A standard header is a block of text that is used to identify each group in a rather generic kind of way. "Customer," "State," and "Monthly Orders" are all examples of this kind of header.

GH1	Region Sales Figures	
D	Psycho-Cycle	AL
D	The Great Bike Shop	AL
D	Benny - The Spokes Person	AL
GF1		
GH1	Region Sales Figures	
D	Bikefest	AR
GF1		
GH1	Region Sales Figures	
D	Biking and Hiking	AZ
D	Bicycle Races	AZ
GF1		
GH1	Region Sales Figures	
D	Biking's It Industries	BC
D	Pedal Pusher Bikes Inc.	BC
D	Cycles and Sports	BC

While the header is somewhat descriptive ("Region Sales Figures" means it is a regional group), you never know what region is in the group without first looking at the details in the group.

9.6.1.1.1 To create a standard header

Procedure

1.  Click the *Insert Text Object* button on the Insert Tools toolbar.
2. When the object pointer appears, move the object frame into the Group Header section.
3. Enter the text you want to use for the header.
4. Click outside the frame when finished to complete the process. Now when you run the report, the same header will appear at the beginning of each group.

9.6.1.2 Live headers

A live header is a header that changes based on the content of the group. If the data is subtotaled by region, for example, a live header would typically identify the region detailed in each group. Thus, the Arizona group would have a header identifying the group as Arizona data, the California group would have a header identifying the group as California data, and so on.

Note

When creating a group, the program automatically inserts a group name field in the Group Header section unless you have toggled the option off using the Options command on the File menu. The information that follows details how to manually insert such a section (if you do not have the program insert one automatically) and how to create different kinds of live headers for different needs.

Group name only

The easiest live header to create is one based on identifying the value of the group field.

9.6.1.2.1 To create a live header by group name only

Procedure

1.  On the *View* menu, click *Field Explorer*.

2. In the Field Explorer dialog box, expand the *Group Name Fields* folder.
3. Select the Group Name field that matches the group you are working with and drag it into the Group Header section for that group.

When you run the report, the group field value identifier will appear as the group header for each region group.

9.6.1.2.2 Group name with text

A more complex type of live header combines a field value and text. A typical group header of this kind for data broken down by region would be, "Sales for California" or "Customers in Postal Code 60606". Creating these headers involves three steps:

- Insert a text object in the Group Header section.
- Type in the text you want to appear.
- Enter the Group Name field in the text field where you want it to appear in the Group Header.

For example, if you want your header to read "Sales for" and then the name of the region in the current group (Sales for AZ, Sales for CA, and so forth), follow these steps:

9.6.1.2.2.1 To create a live header with group name using text

Procedure

1.  Click the *Insert Text Object* button on the Insert Tools toolbar.
2. Place the object frame into the Group Header section for the group.
3. Type in the desired text with a space after it.
4.  Click the *Field Explorer* button on the Standard toolbar.
5. In the Field Explorer dialog box, expand the *Group Name Fields* folder.
6. Select the Group Name field that matches the group you are working with and drag it into the text object, immediately after the text and the space you entered.

→ Tip

Expand the size of the text box to fit both the text and the group field.

7. Format the text object as you want it to appear.

Results

When you run the report, the program will create a live header (with text) for each of your groups.

9.6.1.2.3 Live headers for groups based on a formula

When you create a group and use a formula field as a sort-and-group-by field, the program automatically creates a group name field based on the value returned by the formula.

For example, if you create this formula:

```
{customer.CUSTOMER_NAME}[1]
```

and then group on the formula, the program will group your data based on the first letter in the Customer Name field.

To create a live group header for a group based on a formula, simply insert the group name field in the Group Header section.

When you run the report, the "A" group will have the letter "A" as a header, the "B" group will have the letter "B", and so on. For more information, see [Grouping by the first letter of a company name \[page 184\]](#) and [Grouping data in intervals \[page 181\]](#).

9.6.1.3 Headers for custom groups

The final type of header is a header for the types of custom groups created when data is grouped in a specified order. When using specified order grouping, both the name for each group and the records that belong in it are specified. As in the other grouping situations, the program creates a group name field for each group based on the group names specified.

9.6.1.3.1 To create a header for custom groups

Procedure

1.  On the *View* menu, click *Field Explorer*.
2. In the Field Explorer dialog box, expand the *Group Name Fields* folder.
3. Select the Group Name field for the custom group and drag it into the Group Header section for that group.

Results

The program automatically applies each of the group names you assigned to the appropriate groups.

Note

Make certain that when you assign the names to the groups using the Define Named Group dialog box, the names you assign are the names you want to appear as group headers.

Design Preview X	
	1 2 3 4
GH1	\$10,000 or less
D	Biking and Hiking \$65.70
D	Bicycle Races \$659.70
D	Newpeople Biking Ltd. \$7,981.41
D	C&C Sports People \$2,546.12
D	AIC Childrens \$5,879.70
GF1	
GH1	Between \$10,001 and \$25,000
D	Off the Mountain Biking \$25,000.00
D	Tyred Out \$18,126.33
D	Galaxy Outdoors \$10,425.55
D	Cairo Bicycles, Inc. \$10,361.70
D	Globe Bike \$10,245.55
GF1	
GH1	\$25,001 or more
D	Sporting Wheels Inc. \$85,642.56
D	Rowdy Rims Company \$30,131.46
D	Changing Gears \$26,705.65

9.6.2 Suppressing group headers

You have the option to hide group headers in your report.

9.6.2.1 To suppress group headers

Procedure

1. Right-click the group header and select *Format Field*.
2. In the Format Editor, on the Common tab, click the *Suppress* check box.
3. Click *OK*.

→ Tip

To show the group header again, clear the Suppress check box.

9.6.3 Drilling-down on group headers

To make report viewing easier, you can hide the details of your report and only have the group headers visible. When necessary, you can click on the group header to view the report details.

9.6.3.1 To drill-down on group headers

Procedure

1. Right-click the gray Details section to the left of the report.
2. Select *Hide (Drill-Down OK)*.

3.  Click *Refresh*.

When you place the cursor over a group header, the cursor becomes a magnifying glass.

4. Double-click the group header to drill-down to the detail information.

A drill-down tab appears in the Report Designer. Click the Design or Preview tab to return to that view.

Related Information

[Using the drill-down option on summarized data \[page 110\]](#)

10 Running Totals

Running totals are a flexible and powerful way to create specialized summaries and continually incrementing totals.

This section shows you how to add a basic running total and a running total within a group to your report. You will also learn how to create conditional running totals and running totals using formulas.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com>

10.1 Understanding running totals

Running total fields are similar to summary fields but allow more control over how the total is calculated and when it is reset. Running total fields are specifically suited to perform the following totaling functions:

- Show values of a total accumulate as it is calculated record by record.
- Total a value independent of the report's grouping.
- Total a value conditionally.
- Total a value after a group selection formula has been applied.
- Total a value from the driving table in a one-to-many linking relationship.

10.1.1 How running totals work

A running total field is created with the Running Total Expert. The Running Total Expert creates a running total field by asking you to select a field to summarize, the summary operation to use, a condition upon which to base the evaluation, and a condition upon which to reset the evaluation.

Note

A running total field can be used on database fields and first-pass formulas, but cannot be used on second-pass formulas or formulas that reference other second-pass formulas.

10.1.1.1 Placement of running total fields

The calculation of a Running Total field is determined by the settings selected in the Running Total Expert. However, where you place the running total affects the value that appears on the report. For example, if a Running Total field that evaluates every record and never resets (a grand total) in the Report Header, only the value of the first record will appear. Placing the same Running Total field in the Report Footer returns the desired value. The Running Total field is properly calculated in both cases, but it is displayed too soon in the first case.

The following list summarizes the records that are included in the calculation when a running total is placed in various report sections. This list assumes the running total is not reset.

Report section	Records included in the running total calculation
Report Header	The first record in the report
Page Header	All records up to and including the first record on the current page
Group Header	All records up to and including the first record in the current group
Details	All records up to and including the current record
Group Footer	All records up to and including the last record in the current group
Page Footer	All records up to and including the last record on the current page
Report Footer	All records in the report

10.2 Creating running totals

Running totals are totals that can be displayed on a record by record basis. They total all records (in the report, in the group, and so on) up to and including the current record.

10.2.1 Creating running totals in a list

The most basic form of a running total is a running total maintained throughout a list. In this tutorial, you will create this kind of report by setting up a running total for a list of order amounts.

Note

Running total fields are prefixed by the # sign.

10.2.1.1 To create a running total in a list

Procedure

1. To get started, create a report using the sample database, [Xtreme.mdb](#). Link the Customer and Orders tables, and then place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}  
{orders.ORDER ID}  
{orders.ORDER AMOUNT}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

3.  Select [Running Total Fields](#) and click [New](#).

The Create Running Total Field dialog box appears.

4. Enter the name "TotalOrders" in the [Running Total Name](#) box.
5. Highlight Orders.Order Amount in the [Available Tables and Fields](#) box, and use the first arrow button to move it over to the [Field to summarize](#) box.
6. Select [sum](#) from the [Type of summary](#) list.
7. In the [Evaluate](#) section of the dialog box, click [On change of field](#), and select Orders.Order ID as the [On change of field](#).

The running total will execute each time this field changes.

8. In the [Reset](#) section of the dialog box, click [Never](#) (this gives you a running total that never resets; that is, the running total continues throughout the report).
9. Click [OK](#) to save the running total field.

The program returns you to the Field Explorer dialog box.

10. Insert the running total field in the Details section of the report, just to the right of Orders.Order Amount.

On your report, each row in the running total column displays the current record value added to the previous values. This total continues, unbroken, through the report.

10.2.2 Creating running totals for a group

Another common use for running totals is tallying items in a group. The running total starts with the first item in the group and ends with the last. Then it starts all over again for the next group, then the next, and so on.

In this tutorial, you will create a report that:

- Maintains a running total of customer orders.
- Groups customer orders and resets the running total for each group.

- Displays the subtotal for each customer group.

10.2.2.1 To create a running total for a group

Procedure

1. To get started, create a report using the sample data, [Xtreme.mdb](#).

Link the Customers and Orders tables and place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}
{orders.ORDER ID}
{orders.ORDER AMOUNT}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. On the [Insert](#) menu, click [Group](#) and group on the Customer.Customer Name field.
3. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

4.  Select [Running Total Fields](#) and click [New](#).

The Create Running Total Field dialog box appears.

5. Enter the name "GroupRunningTotal" in the [Running Total Name](#) box.
6. Highlight Orders.Order Amount in the [Available Tables and Fields](#) box, and use the first arrow button to move it over to the [Field to summarize](#) box.
7. Select [sum](#) from the [Type of summary](#) list.
8. In the [Evaluate](#) section of the dialog box, click [For each record](#).
9. In the [Reset](#) section of the dialog box, click [On change of group](#) and accept the default group name.
10. Click [OK](#) to save the running total field.

You return to the Field Explorer dialog box.

11. Place the running total field in the Details section of your report, just to the right of the Orders.Order Amount field.

Note

If you want to view a grand total of each group, place the running total field in the Group Footer section of your report.

10.2.3 Creating conditional running totals

There may be times when you have a list of values, and you only want to subtotal some of the values in the list. For example:

- You have a list that contains both Canadian and U.S. customers.
- You want to keep customer records sorted alphabetically based on customer name.
- You do not want to break the data into groups based on the country.
- You want a total of the values from just the Canadian records.
- You also want a total of the values from just the U.S. records.

To accomplish this, create two running totals, one to keep a running total of the U.S. records, and one to keep a running total of the Canadian records.

- *USTotal*
Maintains a running total of the U.S. records.
- *CanadaTotal*
Maintains a running total of the Canadian records.

10.2.3.1 To create a conditional running total

Procedure

1. To get started, create a report using the sample database, [Xtreme.mdb](#). Link the Customer table and then place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}  
{customer.COUNTRY}  
{customer.LAST YEAR'S SALES}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. On the [Report](#) menu, click [Record Sort Expert](#).

Tip



Another way to do this is to click the Record Sort Expert button on the Expert Tools toolbar.

3. Sort the records based on the Customer.Customer Name field.
4. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

5.  Select [Running Total Fields](#) and click [New](#).

The Create Running Total Field dialog box appears.

6. Enter the name "USTotal" in the *Running Total Name* box.
7. Highlight Customer.Last Year's Sales in the *Available Tables and Fields* box, and use the first arrow button to move it over to the *Field to summarize* box.
8. Select *sum* from the *Type of summary* list.

9.  In the *Evaluate* section of the dialog box, click *Use a formula* and then click the *Formula* button.

The Formula Workshop appears with Running Total Condition Formula active.

10. Enter the following formula in the Formula box:

```
{Customer.Country} = "USA"
```

This tells the program to evaluate the running total each time it comes to a record where Customer.Country is equal to "USA." The running total will ignore all other records (such as records for Canada).

11. When the formula has the correct syntax, click *Save and close*.

You return to the Create Running Total Field dialog box.

12. In the *Reset* section of the dialog box, click *Never*.
13. Click *OK* to save the running total field.

The program returns you to the Field Explorer dialog box.

14. Place the running total field in the Details section of your report.
15. Now create the "CanadaTotal" running total field using the process outlined in steps 5-13. The only difference is that this time you will set the evaluation formula to:

```
{Customer.Country} = "Canada"
```

16. When you are finished, place the *#CanadaTotal* field in the Details section of your report

Note

If you only want to see a grand total of the Canadian and American sales, place the two running total fields you created in the Report Footer section of your report.

10.2.4 Creating running totals in a one-to-many linking relationship

A one-to-many linking relationship occurs in linked tables when a single record in one table can be matched with many records in another table. For example, a one-to-many link might occur when you link a customer table to an orders table. It wouldn't be unusual in such a case for each customer in the primary table to have many orders in the second (lookup) table. In your report, the field values from the primary table are repeated for each new field value in the lookup table.

Creating a subtotal on a field from the primary table provides an incorrect result because the repeated values are included in the calculation. You can avoid this problem by creating a running total.

This concept can be demonstrated with the sample Xtreme database by using the Customer and Orders tables.

10.2.4.1 To create a running total in a one-to-many linking relationship

Procedure

1. From the Field Explorer dialog box, place the following fields from left to right in the Details section of your report:

```
{customer.CUSTOMER NAME}  
{customer.LAST YEAR'S SALES}  
{orders.ORDER ID}  
{orders.ORDER AMOUNT}
```

2.  On the Insert Tools toolbar, click *Insert Group* and create a group based on the Customer.Customer Name field.
3. Right-click the Customer.Last Year's Sales field and select *Summary* from the *Insert* submenu.
4. Choose *Group #1: Customer.Customer Name - A* as the location for your summary.

If you look at the subtotals for each group, you will notice they aren't accurate. This is because the Customer.Last Year's Sales field is duplicated for each order in the report. Follow the rest of this procedure to see how a running total produces an accurate result in the same situation.

5.  In the Field Explorer dialog box, select *Running Total Fields* and click *New*.

The Create Running Total Field dialog box appears.

6. Enter the name "LYSrunning" in the *Running Total Name* box.
7. Highlight Customer.Last Year's Sales in the *Available Tables and Fields* box, and use the first arrow button to move it over to the *Field to summarize* box.
8. Select *sum* from the *Type of Summary* list.
9. In the *Evaluate* section of the dialog box, click *On change of field* and add the Customer.Customer Name field from the *Available Tables and Fields* box.
10. In the *Reset* section of the dialog box, click *On change of group* and choose *Group #1: Customer.Customer Name - A*.
11. Click *OK* to save the Running Total field.
12. Place the running total in the *Group Footer* section.

Results

Compare the running total amount with the subtotal amount for each group. You will see the running total is accurate while the subtotal is not.

10.3 Creating running totals using a formula

If you have suppressed data, or your data is based on a formula that occurs WhilePrintingRecords, you should create a running total formula rather than using the Create Running Total Field dialog box.

When you create a running total manually, you need to create three formulas:

- A summary formula.
- A reset formula to set a variable to zero.
- A display formula to display the variable.

In the following procedure, you will create a report that performs the following functions:

- Maintains a running total of customer orders.
- Groups customer orders and resets the running total for each group.
- Displays the subtotal for each order (the last running total for that order).

10.3.1 To create running totals using a formula

Procedure

1. Create a report using the sample data, [Xtreme.mdb](#). Link the Customers and Orders tables and place the following fields from left to right in the Details section:

```
{customer.CUSTOMER NAME}  
{orders.ORDER ID}  
{orders.ORDER AMOUNT}
```

Note

The Xtreme sample database can be downloaded from the [SAP Knowledge Base Article 1231843](#).

2. On the **View** menu, click *Field Explorer*.
3.  Select *Formula Fields* in the Field Explorer dialog box and click *New*.
4. Name the formula "RunningTotal" and click *Use Editor*.

The Formula Workshop appears with Formula Editor active.

5. Enter the following into the Formula box:

```
WhilePrintingRecords;  
CurrencyVar Amount;  
Amount := Amount + {Orders.Order Amount};
```

6. Click the *Save and close* button on the Formula Workshop.
7. Place this formula in the Details section of your report, just to the right of the Orders.Order Amount field.

This formula prints the running total of the values in the Order Amount field.

8.  On the *Insert* menu, click *Group* and group the report on the Customer.Customer Name field.
9. In the Formula Workshop, create "AmountReset":

```
WhilePrintingRecords;
CurrencyVar Amount := 0;
```

This formula says:

Set the value in the Amount variable to 0.

10. Place this formula in the Group Header #1 section of your report.

Because the Group Header #1 section appears once for every group, @AmountReset will execute each time the group changes. Thus, the Amount variable is reset to 0 each time a new group begins.

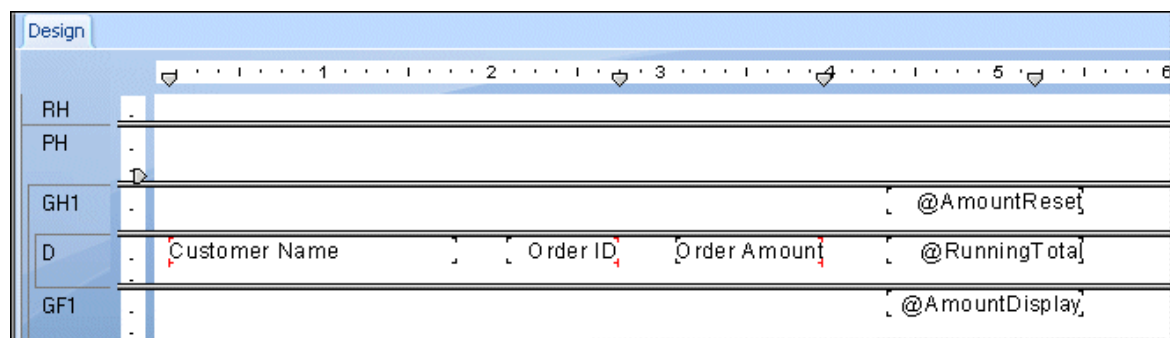
11. Select the @AmountReset formula on the report and use the Format Editor to suppress it so that it will not appear in the final print-out.
12. In the Formula Workshop, create "AmountDisplay":

```
WhilePrintingRecords;
CurrencyVar Amount;
```

This formula simply displays the current value of the Amount variable at any time.

13. Place this formula in the Group Footer #1 section of your report.

Because the Group Footer #1 section appears once for every group, @AmountDisplay will execute each time a group ends. Thus, the value stored in the Amount variable will be printed each time the group changes.



Section	Fields/Formulas
RH	
PH	
GH1	@AmountReset
D	Customer Name, Order ID, Order Amount, @RunningTotal
GF1	@AmountDisplay

Note

This formula prints the same value that @RunningTotal prints as the running total for the last record in each group. But since it is printing it in the Group Footer section, it acts as a group subtotal, not as a running total.

On your report, each row in the running total column displays the current record value added to the previous values. The running total starts fresh with each new group, and the final running total for each group becomes the subtotal for that group.

11 Multiple Section Reports

This section introduces the various types of sophisticated reports you can create using the multiple section reporting capabilities in Crystal Reports. These capabilities enable you to create reports that treat individual values differently based on sets of criteria you establish. These concepts are then applied to the creation of Form Letters.

Online support, samples, and technical briefs

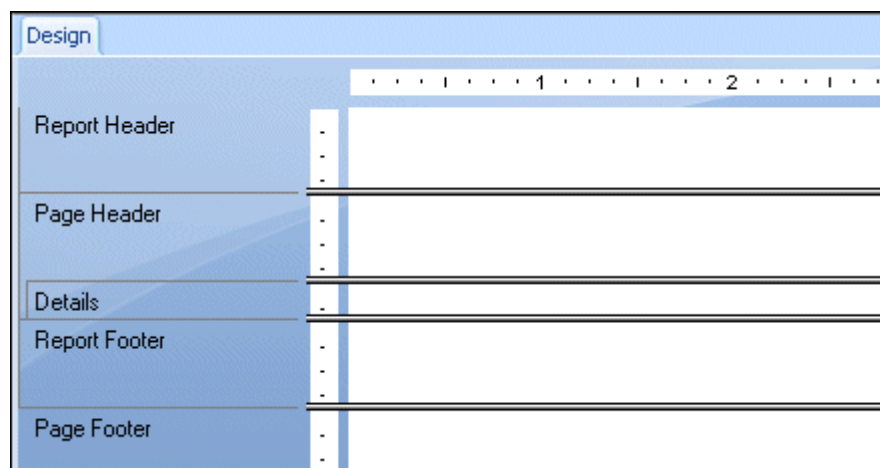
For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com>

11.1 About sections

Crystal Reports provides five design areas to use when building your report:

- Report Header
- Page Header
- Details
- Report Footer
- Page Footer



Each area contains a single section when you first create a new report. You cannot delete any of these original sections but you can hide them or add to them. Once you have added sections, you can delete them, move them in relation to other similar sections, or merge related sections together.

11.2 Working with sections

Many of the procedures in this section show you how to work with sections in the Section Expert. Sections can also be inserted, deleted, and so on by right-clicking the shaded area to the left of the section in the Design or Preview tabs and choosing the appropriate option from the shortcut menu.

11.2.1 Inserting a section

Procedure

1.  Click *Section Expert* on the Expert Tools toolbar.

The Section Expert appears with a list of all the sections in the report. When there are more than one of any kind of section, the sections are lettered A, B, C, and so on.

Note

The program enables only those options (free form, new page before, and so on) that apply to the highlighted section.

2. Highlight the section you want to insert a section after.

For example, to add another Details section, highlight the existing Details section.

3. Click *Insert*.

A new section is inserted immediately below the highlighted section.

11.2.2 Deleting a section

Procedure

1.  Click *Section Expert* on the Expert Tools toolbar.

The Section Expert appears with a list of all the sections in the report. When there are more than one of any kind of section, the sections are lettered A, B, C, and so on.

Note

The program enables only those options (free form, new page before, and so on) that apply to the highlighted section.

2. Highlight the section you want to delete.
3. Click *Delete*.

The program removes the highlighted section from the report.

11.2.3 Moving a section

Procedure

1.  Click *Section Expert* on the Expert Tools toolbar.

The Section Expert appears with a list of all the sections in the report. When there are more than one of any kind of section, the sections are lettered A, B, C, and so on.

Note

The program enables only those options (free form, new page before, and so on) that apply to the highlighted section.

2. Highlight the section you want to move.
3. Click the Up or Down arrow to move the section.

Results

Note

You can only move a section up or down within an area.

Note

The letters that identify the sections describe their relative (as opposed to original) position. Thus, if you move a "C" section up, it becomes a "B" section. It loses its original "C" designation.

Note

You can also move sections by dragging and dropping them in the Report Designer.

11.2.4 Merging two related sections

There may be times when you have placed objects in two sections (where they print sequentially) and you want to put them all in a single section (where they print simultaneously). You can merge the two sections and then rearrange the objects as needed in the new section.

11.2.4.1 To merge related sections

Procedure

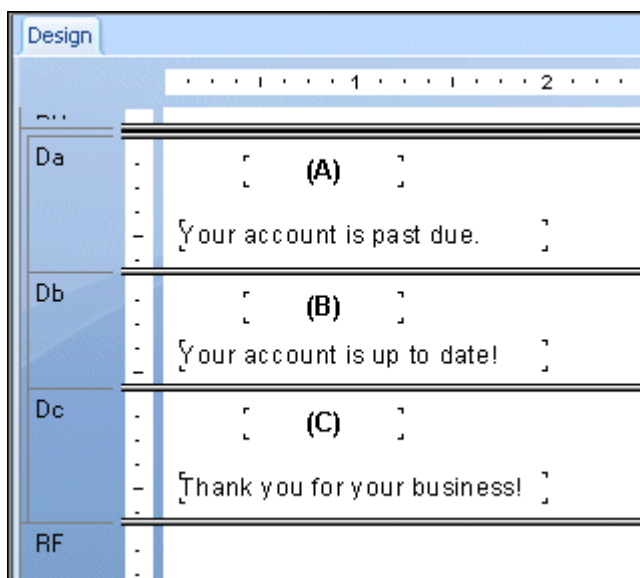
1.  Click [Section Expert](#) on the Expert Tools toolbar.

The Section Expert appears with a list of all the sections in the report. When there are more than one of any kind of section, the sections are lettered A, B, C, and so on.

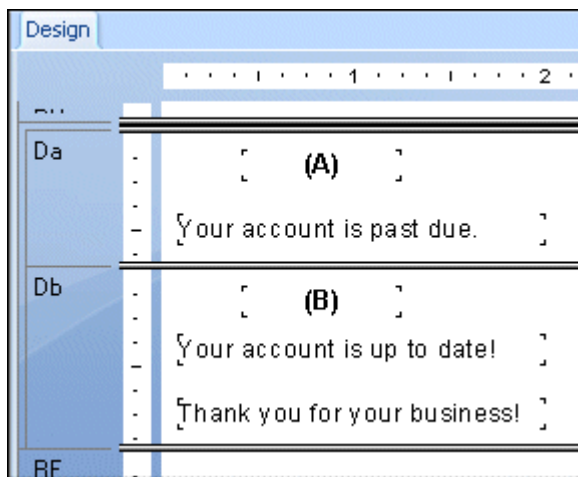
Note

The program enables only those options (free form, new page before, and so on) that apply to the highlighted section.

2. Move the sections so the two sections you want to merge follow each other in the list.
3. Highlight the top section.



4. With Section (B) highlighted, click [Merge](#) and Section (C) will be merged with Section (B) to form one section.



5. Rearrange the objects as needed.

11.3 Splitting and resizing sections

A section can be split into two or more sections and/or resized easily in the Design tab.

11.3.1 Splitting a section

Procedure

1. Move the pointer over the left boundary of the section you want to split.
2. When the pointer becomes a Section Splitting cursor, click the boundary and drag the pointer into the section.
3. When a horizontal line appears, drag it up or down to split the section the way you want it.





11.3.2 Resizing a section

Procedure

1. Move the pointer over the bottom boundary of the section you want to resize.
2. When the pointer becomes a Resizing cursor, drag the boundary to make the section bigger or smaller as you wish.

11.3.2.1 Resizing a section to remove white space

If you have one or more objects in a section and you want to resize the section to remove unnecessary white space, right-click in the shaded area to the left of the section in the Design and Preview tabs and choose Fit Section from the shortcut menu. The program automatically resizes the section, moving the bottom boundary of the section to the baseline of the bottom object in the section.

A section automatically expands vertically in two instances:

- When you place an object and the object is bigger (vertically) than the section you place it in.
- When you expand an object (vertically) so it becomes bigger than the section it resides in.

Note

You cannot resize a section smaller than the combined height of all the objects in the section.

Note

You can also remove white space at the bottom of a Page Footer by selecting Clamp Page Footer on the Common tab of the Section Expert.

11.4 Using multiple sections in reports

Certain reporting tasks are performed most efficiently by creating multiple sections within an area.

Once you understand the power of multiple sections, you will discover even more ways to produce the effects you want.

Related Information

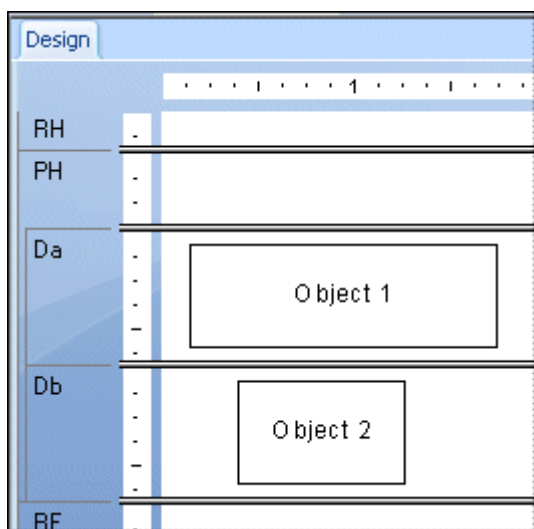
[Working with sections \[page 218\]](#)

11.4.1 Keeping variable length objects from overwriting each other

When subreports or other variable length objects are placed above other objects in one section of the report while the Can Grow option for the variable length object is toggled on in the Format Editor, that object may overprint objects positioned directly below it unless you have:

- Expanded the section to fit the maximum size of the object.
- Spaced the objects, allowing enough space for the first object to complete printing before the second one begins.

You can eliminate this overprinting problem by creating multiple sections in an area and placing objects below the variable length object in their own section(s).



Now, when the report runs, the section with the variable length object will finish printing before the section below it prints and you will get the results you want..

Note

Many report objects can use the Can Grow option and can, therefore, cause overprinting:

- Text fields
- Formula fields
- Memo fields
- BLOB fields
- Subreports
- Cross-Tabs
- OLAP grids

11.4.2 Eliminating blank lines when fields are empty

It is very common to have two address lines in a customer table, one for street address (Address 1), and one that can be used for a suite number or mail stop (Address 2). Address 1 usually contains a value, but Address 2 is often blank. If you create a customer list using this data and stack the fields on top of one another in mailing label format, those customer records with an empty Address 2 field will print with a blank line. You can eliminate this blank line either by using multiple sections, or by suppressing blank lines.

11.4.2.1 To eliminate blank lines by using multiple sections

Procedure

1.  Use the Section Expert to create two new Details sections so that you have a total of three. See [Working with sections \[page 218\]](#).
2. Place the [Address 2](#) field in the middle section and the other data in the sections above and below it as you want it to appear in the report.
3. In the Section expert, highlight the middle section.
4. On the [Common](#) tab, select the [Suppress Blank Section](#) check box.

Results

Now, when the report prints, if the Address 2 section is blank, the program will not print it and you will not get unwanted blank lines in the report.

Note

If the report section you want to suppress contains a blank subreport, use the Suppress Blank Subreport option, found on the Subreport tab of the Format Editor, as well as the Suppress Blank Section option.

11.4.3 Adding blank lines conditionally

Use multiple sections to print a blank line on your report under specific conditions. For example, you may want to insert a blank line after every fifth record in the report.

11.4.3.1 To add blank lines conditionally

Procedure

1. Use the Section Expert to create two Details sections. See [Working with sections \[page 218\]](#).
2. Place the report detail data in the top section.
3. Leave the second section empty.
4. In the Section expert, highlight the second section.
5. On the *Common* tab, select the *Suppress (No Drill-Down)* check box then click the conditional formatting button to its right.
6. Enter the following formula in the Format Formula Editor:

```
Remainder (RecordNumber,5) <> 0
```

This formula divides each record number by 5. If the division produces a remainder, it suppresses the blank section. But if the division produces no remainder, a zero (which it will for every fifth record printed), the program prints the second section, thus inserting a blank line.

Results

Note

To insert a blank line under different conditions, you can modify your formula appropriately. See [Working with conditional formatting \[page 287\]](#).

11.5 Form letters

While form letters themselves are not necessarily multi-section reports, they are often used in multi-section reports to generate custom mailings. The topic [Printing conditional messages in form letters \[page 233\]](#), explains how to use multiple form letters or multiple versions of the same form letter for custom mailings.

Form letters often use text objects to hold the content of the report. The following sections provide an introduction to text objects and demonstrate how to use them in form letters.

11.5.1 Working with text objects

You will use many of the capabilities of text objects when creating form letters. A brief discussion of text objects should make it easier for you to create the form letter in the next section. Some things to consider are:

- A text object can contain both text and fields; you will use both in this example.
- You can resize text objects; you will be resizing the text object so that it prints as a letter.

11.5.1.1 Move/Resize mode

When the object is in the move/resize mode, it appears as a solid frame with resizing handles.



In this mode, you can resize the object by dragging any of the resizing handles, or you can move it by placing the cursor inside the object and dragging it to a new location. You can also insert fields in this mode, but you cannot insert text. You put a text object into the move/resize mode by clicking it when it is inactive.

11.5.1.2 Edit mode

When the object is in the edit mode, it appears as a solid frame without sizing handles, and an in-place ruler appears at the top of the tab (if you have selected the Show Rulers options in the Options dialog box).



When you first place a text object, the program sets it in the edit mode. You can also put a text object into the edit mode by double-clicking it when it's inactive or in the move/resize mode. Finally, you can put a text object in edit mode by right-clicking it and selecting Edit Text from its shortcut menu.

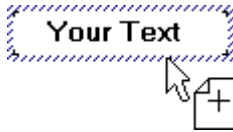
Each text object contains word processor capabilities, including the ability to change the fonts for individual characters and fields, and automatic word wrap. In the edit mode you can insert text and such non-text objects as database fields and formulas. Whenever the object is in edit mode, it contains an insertion point, a flashing vertical line that indicates the position at which typed text or inserted fields will begin.

The insertion point moves as you type, automatically staying to the right of the last character. It also moves when you insert a field, automatically staying to the right of the field. It moves one character position at a time when you press the space bar. It moves down one line, to the inside left edge of the text object, when you press Enter (this action inserts a carriage return). It moves to the cursor position when you click anywhere within the existing text.

As you work through the tutorials for multiple sections, it is always expected that you will type or place fields at the existing insertion point, unless otherwise indicated.

- To select text within a text object (to delete it, to change a font, and so on), position the cursor over the text and, when the I-beam cursor appears, drag the cursor to highlight the text you want to select.
- To select a field which is inside a text object, position the cursor over the field and, when the I-beam cursor appears, right-click.

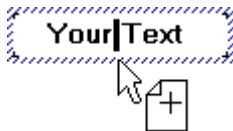
- To insert text, type in the desired text and it will appear at the insertion point.



Note

It is critical that you see the Drag and drop cursor before placing the field. If it does not appear, you might place the field so that it overlays the text object instead of being inserted into it. It may appear to be inside the text object, but if you move the object the field will not move with it.

- The insertion point is tied into the Drag and drop cursor. If you have existing text or fields in the text object, the insertion point moves as you move the Drag and drop cursor, enabling you to pick the exact position where you want to place the field. The program always places the field at the insertion point.



- You can leave edit mode by clicking outside of the text frame. You can also click [Ctrl+Enter](#).

11.5.2 Creating a form letter using a text object

The following section shows you how to create a form letter.

You are going to use a text object to create a form letter. The form letter you create will be tied to a database table so that each letter will be customized with company information from a different record.

If you have difficulty performing any of the steps, please see [Working with text objects \[page 226\]](#).

11.5.2.1 To create a form letter

Context

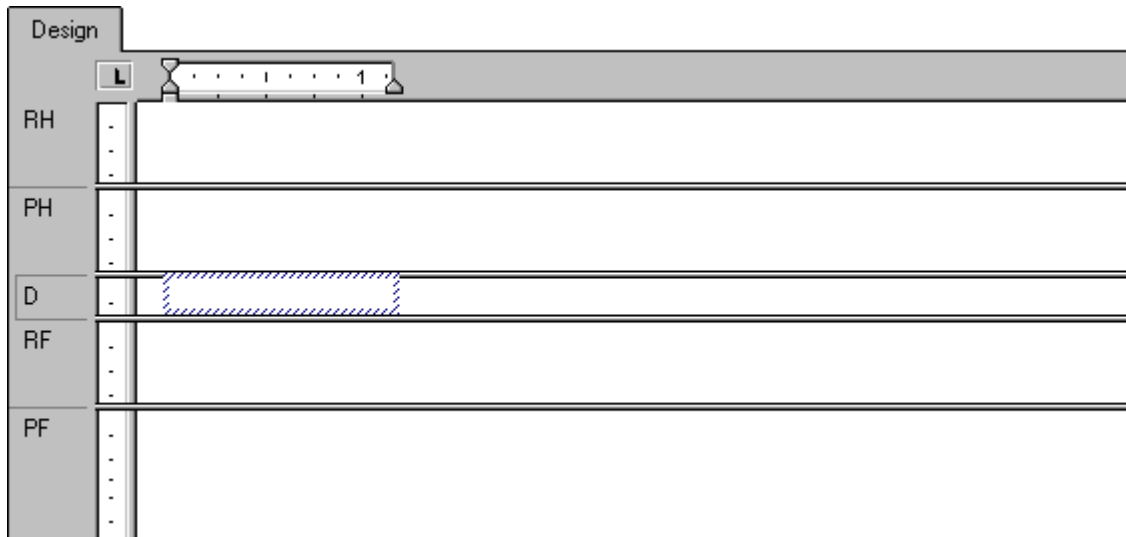
The letter will consist of a date, an inside address, a salutation, a one paragraph letter body, and a closing section.

Procedure

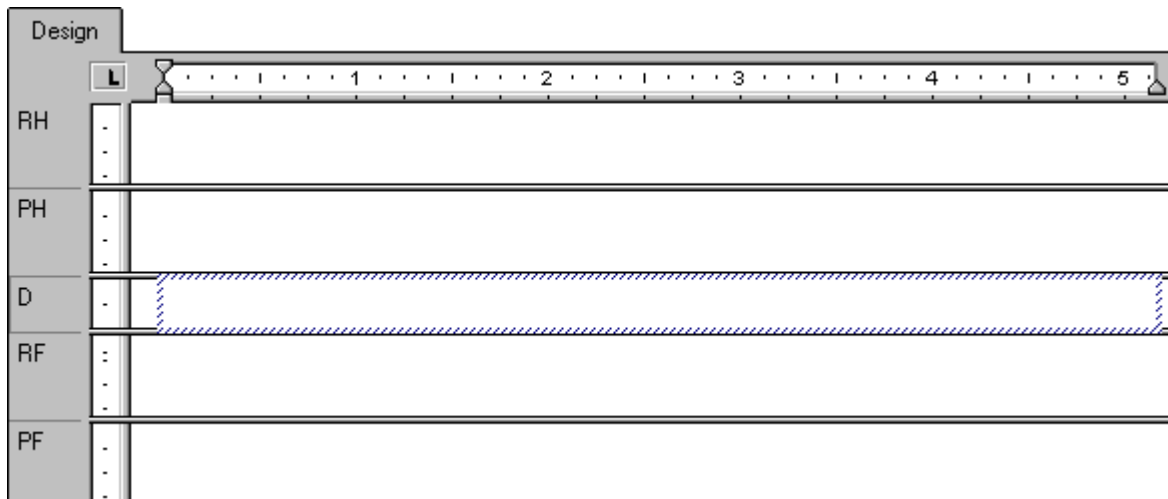
- Create a blank report. Use the [Customer](#) table of [Xtreme.mdb](#).
The Design tab appears.
- Since you do not want field titles to appear above the fields that you insert into the letter, clear the [Insert Detail Field Headings](#) check box on the [Layout](#) tab of the Options dialog box.



3. Insert a text object into the *Details* section of the report.
4. Click the text object frame to put the object in move/resize mode.



5. Drag the resizing handle on the right side of the object to the right edge of the *Design* tab. This will make the object about eight inches wide, the approximate width of a page. You may have to stop resizing, scroll the window, and resize some more to accomplish this.



6. Double-click inside the text object to place it in edit mode. It is now ready for you to begin your work. The insertion point appears at the extreme left, inside the object.

11.5.2.2 Inserting a date

Procedure

1. To insert a date into the letter, expand *Special Fields* in the Field Explorer dialog box and scroll until you find *Print Date*.
2. Click *Print Date* and drag the placement frame into the text object and place it at the insertion point.

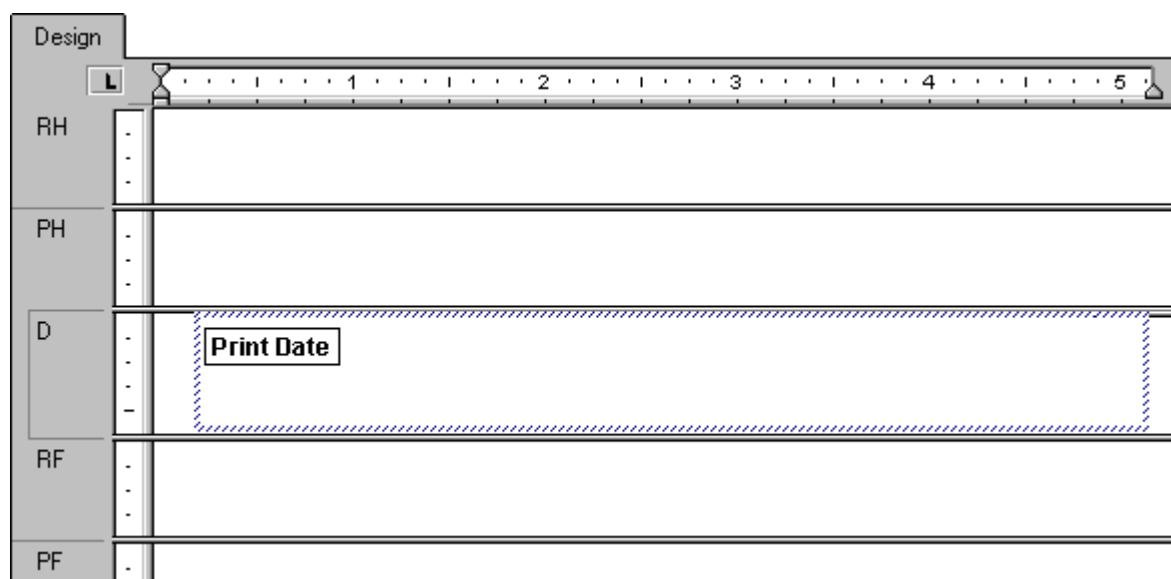
Note

To change the way the date is formatted in the letter, double-click the text object to select it. Then right-click the Print Date field and choose Format {Print Date} from the shortcut menu. Make your modifications on the Date tab of the Format Editor when it appears.

3. Press Enter twice to insert some white space between the date and the inside address and to move the insertion point down within the text object.

Tip

You might have to resize the Details section and the text object if you have not selected the Can Grow option for the text object in the Format Editor.



11.5.2.3 Creating an inside address

Procedure

1. To create the inside address, drag database fields into the text object from the *Customer* table in the Field Explorer dialog box.
2. Drag in the *Address 1* field and place it at the insertion point, and press *Enter*. The insertion point moves down to the line below.
3. Drag in the *City* field and place it at the insertion point.
4. Type a comma, followed by a space.
5. Drag in the *Region* field and place it at the insertion point.
6. Type in two spaces.
7. Finally, drag in the *Postal Code* field, place it at the insertion point, and press *Enter*. The insertion point moves down to the line below.
8. Press *Enter* one more time to move the insertion point down one more line to the position where the salutation should start. This completes the inside address.

Results

Note

When a field is placed within a text object, it is automatically trimmed on both the left and right sides, so that it contains no extra white space.

The screenshot displays the Design view of a report layout. On the left, a vertical pane lists report sections: Design, RH, PH, D, RF, and PF. The 'D' section is currently selected. A horizontal ruler at the top indicates positions 1 through 5. The main design area shows a form with several text boxes. The 'Print Date' box is at the top. Below it is the 'Address 1' box. Under 'Address 1', there are three boxes: 'City', 'Region', and 'Postal Code'. The 'City' and 'Region' boxes are grouped together, and the 'Postal Code' box is to their right. The form is enclosed in a dashed border.

11.5.2.4 Creating a salutation

Procedure

1. Press **Enter** four times to move the insertion point down.
2. Type in the word "Dear" and a space (do not include the quotation marks).
3. In the Field Explorer dialog box, highlight the **Contact Title** field from the **Customer** table and drag it into the text object, placing it immediately after the space.
4. Insert a space. The program positions the insertion point immediately after the space.
5. In the Field Explorer dialog box, drag the **Contact Last Name** field into the text object and place it at the insertion point. The insertion point moves to the right of the field.
6. Type a colon ":" at the insertion point (without the quotation marks) and press **Enter** twice to move the insertion point down two lines.

Design

RH

PH

D

RF

Print Date

Address 1

City, Region Postal Code

Dear Contact Title Contact Last Name :

11.5.2.5 Creating the letter body

Procedure

1. Now type "Your company" (do not include the quotation marks) and type a comma followed by a space.
2. Drag the **Customer Name** field into the text object and place it at the insertion point, just after the space.
3. Type a comma, followed by a space.

4. Type the following text (do not include the quotation marks): "helped make this year an outstanding year for Xtreme Mountain Bikes, Inc. I want to thank you and your staff for your support. I hope next year will be a banner year for you."
5. Press Enter twice.
6. Type "Sincerely yours" (without the quotation marks), followed by a comma, and then press [Enter](#) four times.
7. Finally, to complete the form letter, type your name.

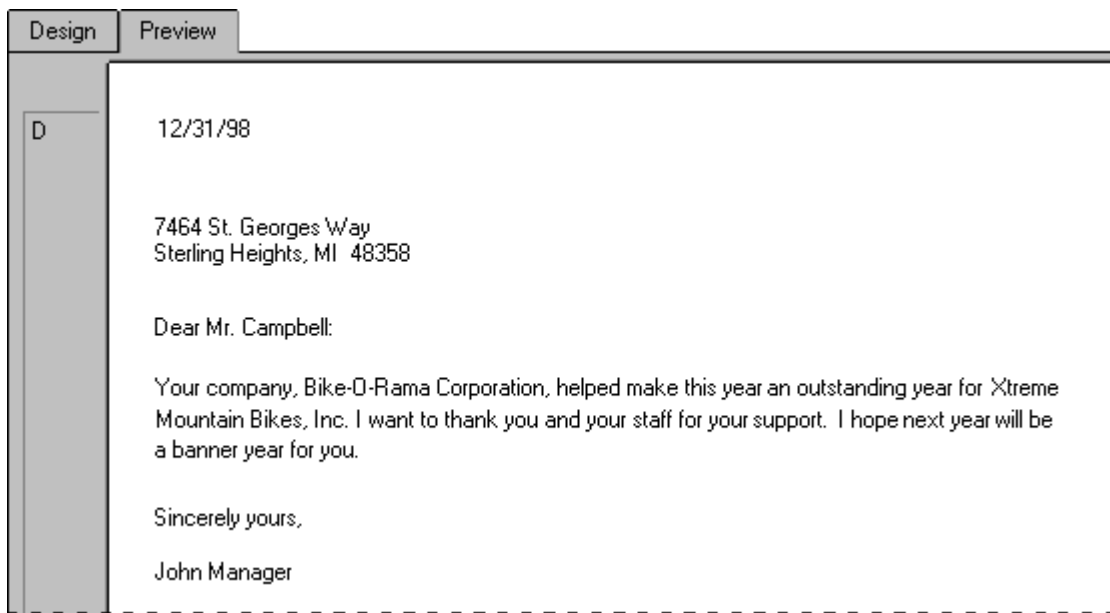
The Design tab should look similar to this:

The screenshot shows the Design tab of SAP Crystal Reports. The layout is as follows:

- Print Date** (text box)
- Address 1** (text box)
- City**, **Region**, **Postal Code** (three separate text boxes)
- Dear** (text) followed by **Contact Title** and **Contact Last Name:** (two separate text boxes)
- Body text: "Your company, **Customer Name**, helped make this year an outstanding year for Xtreme Mountain Bikes, Inc. I want to thank you and your staff for your support. I hope next year will be a banner year for you."
- Two blank lines (created by pressing Enter twice).
- Text: "Sincerely yours,"
- Four blank lines (created by pressing Enter four times).
- Text: "John Manager"

8.  Click [Print Preview](#) on the Standard toolbar to preview the form letter.

It should look similar to this:



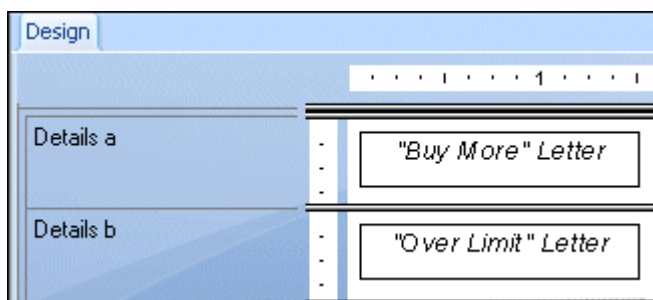
11.5.3 Printing conditional messages in form letters

It is likely that you will want to print conditional messages in form letters. For example, you may want to encourage customers with available credit to buy more and those who are over their credit limit to bring their accounts down below the limit once again. You can create both of these letters within a single report.

11.5.3.1 To create a conditional message

Procedure

1.  Use the Section Expert to insert a second Details section in the report. See [Working with sections \[page 218\]](#).
2. Create two form letters. Place a letter that encourages customers to buy more in the Details A section of the report; place a letter that encourages customers to bring their balance down in the Details B section of the report. See [Creating a form letter using a text object \[page 227\]](#).
3. Use the Section Expert to format the Details sections so that each is suppressed under certain conditions. For example, consider the situation where you want to print one of two letters.



- Format the first section to be suppressed when the balance is less than the credit limit.
- Format the second section to be suppressed when the balance is more than the credit limit.

Results

Now, when a record indicates available credit, the "buy more" letter will print. When the account is over the credit limit, the "over limit" letter will print. And when the customer is right at the credit limit, nothing will print at all.

Related Information

[Working with conditional formatting \[page 287\]](#)

12 Formatting

Formatting refers to changes in the layout and design of a report, as well as the appearance of text, objects, or entire report sections. This section details methods you can use to draw attention to data, change the presentation of dates, numbers, and other values, hide unwanted sections, and perform a variety of other formatting tasks to give a report a professional appearance.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

12.1 Formatting concepts

This section explains how to format a report. Formatting refers to changes you can make to the layout and design of a report, as well as the appearance of text, objects, or entire report sections.

You can use formatting to do many things, including:

- Dividing sections of a report.
- Calling attention to certain data.
- Changing the presentation of dates, numbers, Boolean values, currency values, and text strings.
- Hiding unwanted sections.
- Giving the report a professional appearance.

The following topics describe the types of formatting you can do with Crystal Reports, giving step-by-step instructions for performing a variety of formatting tasks.

Note

There are many date formats you can choose to use on an English report, but if you send the report to a Japanese system, there may be some formatting irregularities. Not all English date formats are viewable on a Japanese system, and the same is true going from Japanese to English. For instance, abbreviated English months do not appear on a Japanese system and Japanese eras in short format do not appear on an English system.

12.2 Using a template

A template is an existing report file whose formatting can be added to a new report. At the same time, the formatting of the template report's fields and report objects are applied to the new report. Use templates to give any number of reports a consistent look without having to format each one individually.

For additional information about templates, see [Template considerations \[page 239\]](#).

12.2.1 Applying a template

When you create a new report in the Standard Report Creation Wizard, you can apply a template as an optional step. You can also apply a template later by using the Template Expert. You can choose from a number of predefined templates, or you can use an existing Crystal report as a template.

12.2.1.1 To apply a template in the Standard Report Creation Wizard

Procedure

1.  Click [New](#) on the Standard toolbar.
2. Choose data, fields, grouping fields, and so on, until the wizard displays the Template screen.
3. In the [Available Templates](#) list, click a predefined template name to see an example of it in the Preview area.

By default, the sample templates shipped with Crystal Reports are installed at `\Program Files (x86)\SAP BusinessObjects\Crystal Reports\Templates`.

4. If you want to apply a template based on an existing Crystal report, click [Browse](#).
5. In the Open dialog box, select a Crystal report (.rpt) file and click [Open](#).

The report is added to the list of Available Templates.

Note

If a template name and preview picture were saved (in the Document Properties dialog box) with the report you selected as a template, you will see this information on the Template screen.

6. Click [Finish](#).

Your report appears with its data formatted to match the template you selected.

Note

Formatting is not applied if the report you chose does not meet the requirements for a template.

12.2.1.2 To apply a template to an existing report

Procedure

1. On the *Report* menu, click *Template Expert*.

The Template Expert appears.

→ Tip



Another way to do this is to click the Template Expert button on the Expert Tools toolbar.

As was the case with the Template screen, you can choose from a number of predefined templates, or you can click the Browse button to search for an existing report to use as a template.

2. Choose a template and click *OK*.

ⓘ Note

Any drill-down tabs, alerts, or analyzer views that are open will be closed before the template is applied.

12.2.2 Removing an applied template

Perhaps, after applying a template, you don't like the changes made to your report. As long as you haven't exited Crystal Reports since you applied the template, you can remove it from your report.

12.2.2.1 To remove an applied template

Procedure

1. On the *Report* menu, click *Template Expert*.

→ Tip



Another way to do this is to click the Template Expert button on the Expert Tools toolbar.

2. Choose *Undo the current template* and click *OK*.

The chosen template's features are removed and your report reverts back to the formatting it had when you first opened it.

ⓘ Note

To remove a template, you must use this option; the Undo command is not available from the Edit menu.

12.2.3 Reapplying the last template selected

If you want to reapply the last template you selected during a session of Crystal Reports, you can simply select an option on the Template Expert.

12.2.3.1 To reapply the last template selected

Procedure

1. On the *Report* menu, click *Template Expert*.

→ Tip



Another way to do this is to click the Template Expert button on the Expert Tools toolbar.

2. Choose *Re-apply the last template* and click *OK*.

12.2.4 Using Template Field Objects

You can use Template Field Objects to create more flexible report templates. These report objects do not refer to existing database fields; you simply put them in your template report and format them as you require. When you apply the template to another report, Crystal Reports displays that report's data with the formatting you specified. Therefore, when you design a template, you don't have to know what data might be in the report you'll eventually apply it to—you use Template Field Objects to take care of the possibilities for you.

Note

Template Field Objects are applied only to result fields: database fields, parameter fields, SQL statements, and formulas. Special Fields are not considered result fields.

12.2.4.1 To add a Template Field Object to a template report

Procedure

1. On the *Insert* menu, click *Template Field Object*.
A placement frame is attached to your cursor.
2. Place the Template Field Object frame on your template report.
A Template Field Object can be placed in any report section.

3. Right-click the object and click *Format Template Field* from its shortcut menu.

A list of formatting options appears. You can choose any of the options; when you do, the appropriate tab of the Format Editor appears.

→ Tip

You can select multiple Template Field Objects and apply your formatting choices to all of the objects.

4. Specify the formatting for your Template Field Object as you require.

For information about how to apply formatting, see [Working with absolute formatting \[page 271\]](#).

Results

For each Template Field Object you create, a special formula field is created. You can see this formula field in the Formula Workshop. If you want to use sample data in your report to see how your formatting will look, you can reference database fields in these formulas.

ⓘ Note

If you want to apply different formatting to a number of fields within the same report section, you should use a different Template Field Object for each field.

12.2.4.2 To add sample data to a Template Field Object formula

Procedure

1.  In the Field Explorer, expand the *Formula Fields* node, select a Template Field Object, and click *Edit*.

→ Tip

In the Formula Fields node of the Field Explorer and the Formula Workshop, Template Field Objects appear as <TemplateField>.

2. In the Formula Editor, replace the `Space(10)` section of the argument with a database field of the type you want to see in your sample, save your change, and close the Formula Workshop.
3. Refresh your report's data.

12.2.5 Template considerations

If an existing report is used as a template, these report objects will be applied to the new report:

- Fields
- Groups
- Group charts
- Summary fields
- Hyperlinks
- Bitmaps
- Lines, boxes, borders
- Static OLE objects

If an existing report is used as a template, these report objects will not be applied to the new report:

- Detail charts
- Subreports
- OLAP grids
- Cross-tabs
- Maps
- Embedded OLE objects
- BLOB fields
- Specified order grouping
- Advanced summaries (such as Top Ns, Percentages, and Running Totals)

The formatting and objects from an existing report used as a template can override choices you made in the Standard Report Creation Wizard. For example, if you had selected "No Chart" on the Chart screen of the wizard, but then applied a template report that includes a chart, the template report will override your selection and the new report will contain a chart.

Further, a chart in a template report overrides the charting summary value selected on the Chart screen of the wizard. The order in which you select your data fields in the wizard generally determines which summary field becomes the default value for your chart. Once the template has been applied, the first summary field in the report becomes the charting summary value even if you selected a different value in the wizard.

If the Template has...	If the target report has...	Result
Group Chart	Group Chart	Overwrite/Apply
Group Chart	No Chart • Group & summary	Overwrite/Apply
No Chart	Group Chart	Chart retained
Advanced Chart • Using group & summary in report	No Chart	Overwrite/Blank section
Advanced Chart • Not using group & summary in report	Advanced Chart	Overwrite/Blank section

If the Template has...	If the target report has...	Result
Advanced Chart Using group & summary in report	Advanced Chart	Overwrite/Apply
No Chart	Advanced Chart	Chart retained
Cross-Tab/OLAP grid	Cross-Tab/OLAP grid	Cross-Tab/OLAP Grid retained
No Cross-Tab/OLAP grid	Cross-Tab/OLAP grid	Cross-Tab/OLAP Grid retained
Cross-Tab/OLAP grid	No Cross-Tab/OLAP grid	Overwrite/Blank section
Map	Map	Map retained
No Map	Map	Map retained
Map	No Map	Overwrite/Blank section
Subreport	Subreport	Subreport retained
No Subreport	Subreport	Subreport retained
Subreport	No Subreport	Overwrite/Blank section

12.3 Using the Report Design Environment

This section describes several things to keep in mind when designing reports that are distributed to different environments.

12.3.1 Section characteristics

A report consists of several sections, including the Report Header, Page Header, Group Header, Details, Group Footer, Page Footer, and Report Footer.

Each report section is made up of a series of lines. When a text-based object is placed in a section, it is placed on a line in such a way that the text is aligned to the baseline. The line's height is then adjusted by the printer driver so that it is high enough to accommodate the object.

- If you place another text-based object on the same line with a font size larger than that of the first object, the line's height extends to accommodate the second object.
- If you place another text-based object on the same line with a font size even larger than the previous two objects, the line's height extends to accommodate the third object.

A line's height is determined by the text-based object with the largest font size on the line.

As you add text-based objects to a report, either in the same section or other sections, the line height adjusts to accommodate the various fonts. Since this vertical spacing is set up by the printer driver, it is difficult to create reports designed for pre-printed forms when they are printed in various environments.

When designing reports, you should do the following:

- Always print a test page.
- Keep all font sizes the same.
- Be sure to print pre-printed forms on the same machine.

12.3.2 Making an object underlay a following section

Using this example, you can make a logo underlay multiple sections. This procedure is similar to the one for inserting a company watermark to serve as a report background.

To make an object underlay a following section, first place the object in the section above the section you want it to underlay. Then select the Underlay Following Sections check box in the Section Expert for the section in which the object is placed.

12.3.2.1 To create a basic report

Procedure

1. Create a report using the Customer table in Xtreme.mdb.
Xtreme.mdb can be downloaded from the following link: [Direct download link for samples](#) .
2. Place {customer.CUSTOMER NAME} and {customer.LAST YEAR'S SALES} side-by-side in the Details section of the report.
3. To remove unnecessary objects from this example, delete the field titles that the program places in the Page Header section of each field.
4. On the *Insert* menu, click *Group* to break the data into regional groups.
5. On the *Common* tab of the Insert Group dialog box, select {customer.REGION}.
6. Click *OK*.

12.3.2.2 To insert a picture onto the report

Procedure

1. On the *Insert* menu, click *Picture*.

→ Tip



Another way to do this is to click the Insert Picture button on the Insert Tools toolbar.

2. Select an image file, and then place it in the Page Header section, to the right of the fields in the report body.

Note

In this example, the picture is placed to the right of the fields to avoid underlaying the text. When you are working with a watermark, a subdued picture designed to be nearly invisible, place it directly above the text.

3.  On the Standard toolbar, click *Print Preview*.

The picture prints in the Page Header section of each report page.

12.3.2.3 To make the picture underlay the following section

Procedure

1. On the *Report* menu, click *Section Expert*.

The Section Expert dialog box appears.

Tip



Another way to do this is to click the Section Expert button on the Expert Tools toolbar.

2. In the *Sections* list, click *Page Header*, then select the *Underlay Following Sections* check box.
3. Click *OK* to preview the report again.

The picture now prints in both the first Group Header and the following few Details sections, next to (instead of above) the text in the body of the report.

Note

Using the technique of placing a picture to the right of the body of the report, you can set up a chart or an employee picture to print beside the details pertaining to that chart or employee.

4. Once you are finished previewing the report, return to the *Design* tab.
5. Resize the picture vertically to make it two or three times larger, then preview the report again.

The bitmap file now underlays more sections.

Results

The area in which the picture underlays depends on the following conditions:

- The size of the picture.
- The section in which the picture was originally placed.

- The position of the picture in the section.

By modifying size and placement of an object, you can create a variety of visual effects, using the underlay feature.

12.3.3 Pre-printed forms

If you print on pre-printed forms, you will be able to:

- Scan a form.
- Place it in the report as a bitmap.
- Use the underlay feature to line up the bitmap and report, as well as move objects anywhere you want them to appear.
- Eliminate the need to print the forms separately by printing the report and the form as a single unit.

12.3.4 Multiple columns

Instead of having your data print straight down the page, you can set up multiple columns and have the data flow from column to column.

12.3.4.1 To create a multiple-column report

Procedure

1. Open the report you want to format with multiple columns.
2. On the *Report* menu, click *Section Expert*.

→ Tip



Another way to do this is to click the Section Expert button on the Expert Tools toolbar.

3. In the Section Expert, highlight *Details*, and then select *Format with Multiple Columns*.

A Layout tab is added to the Section Expert.

4. Click the *Layout* tab and set the *Width* you want your column to be.

Keep in mind the width of your paper when deciding your column width. For example, if you have three fields in your Details section, and they take up four inches of space, limit the width of the column to under four and a half inches so that all the field information can be seen.

5. Set the *Horizontal* and/or *Vertical* gap you want to maintain between each record in your column.
6. In the Printing Direction area, choose a direction.
7. If the report you're formatting contains grouping, select *Format Groups with multiple column*.

8. Click [OK](#).

Results

When you preview the report, you'll see that the field headers appear only for the first column. To have field headers for the second column, insert a text object.

12.3.5 Hiding report sections

Crystal Reports has three properties you can set in the Section Expert to hide report sections.

12.3.5.1 Hide (Drill-Down OK)

The Hide property hides a section whenever you run the report. For example, in a summary report, the Hide property can be used to display only the summaries, but not the details behind the summaries. When the Hide property is applied to a section, it becomes visible when the Drill-down cursor is used to drill down on the section contents. This property is absolute; it cannot be conditionally applied using a formula.

12.3.5.2 Suppress (No Drill-Down)

The Suppress property also hides a section when you run the report. Unlike the Hide property, however, you cannot apply the Suppress property, then drill down to reveal the section contents. This property can be applied absolutely, or conditionally using a formula. This is useful for writing form letters. For example, in a form letter, you might create two Details sections: one to suppress when sales are equal to or over \$X and one to suppress when sales are under \$X.

12.3.5.3 Suppress Blank Section

The Suppress Blank Section property hides a section whenever there is nothing in it. If something is placed within the section and it produces a value in your report, then it becomes visible.

12.3.6 Hiding report objects

Crystal Reports has three formatting options in the Format Editor for hiding individual objects.

12.3.6.1 Suppress If Duplicated (Common tab)

The Suppress If Duplicated property prevents a field value from printing if it is identical to a duplicate of the value that comes immediately before it in an iteration of the same section.

The value does not print, but the space in which it would have printed remains.

Cust ID	Orde
1	100.00
1	157.00
1	0.00
1	10.00
5	146.00
5	0.00
7	153.00
7	0.00
7	186.00

Cust ID	Orde
1	100.00
	157.00
	0.00
	10.00
5	146.00
	0.00
7	153.00
	0.00
	186.00

ⓘ Note

This option does not work for text fields that contain embedded fields.

ⓘ Note

This option compares record values, not formatted field values. The program ignores the option in the first Detail section of a formatted page.

12.3.6.2 Suppress If Zero (Number tab)

→ Tip

To find this option, click the Number tab of the Format Editor, then click the Customize button.

The Suppress If Zero property prevents a value from printing if it is a zero value. The value does not print, but the space in which it would have printed remains. To remove the blank space, select the Suppress Blank Section check box in the Section Expert.

Cust ID	Order
1	100.00
	157.00
	0.00
	10.00
5	146.00
	0.00
7	153.00
	0.00
	186.00

Cust ID	Order
1	100.00
	157.00
5	10.00
	146.00
7	153.00
	186.00

Note

This will only work if there are no other objects in the section.

12.3.6.3 Suppress (Common tab)

The Suppress property hides an object when you run the report. For example, it is common to apply this property to formulas that are needed to do some report calculations, but that you do not want to print when you run the report. When you select this property, the selected object does not print.

Product ID	Unit Price	SRP
1101	4.00	6.67
1102	8.00	13.33
1103	13.00	21.67
1104	2.00	3.33
1105	11.00	18.33
1106	16.00	26.67
1107	7.00	11.67
1108	4.00	6.67
1109	12.00	20.00

Product ID	SRP
1101	6.67
1102	13.33
1103	21.67
1104	3.33
1105	18.33
1106	26.67
1107	11.67
1108	6.67
1109	20.00

Note

You can click the Conditional Formula button for any of these properties and create a formula that will make the setting conditional on some event. See [Working with conditional formatting \[page 287\]](#).

To set these properties, select the object, then click Format from the Expert Tools toolbar to open the Format Editor dialog box. When the Format Editor appears, set the properties.

12.3.7 Placing text-based objects

When a text-based object is placed on a report, the object is represented by an object frame. The height of the object frame is based on the height of the font. The width, however, is determined differently, depending on the object you are working on.

- For database fields that are not memo fields, the width is initially determined by the width of the field as defined in the database, and by the average character width as provided by the selected font and font size. For example, you have a database field called {customer.LAST NAME} and your database defines this field as a text field with a length of 35 characters. When you place this field on your report, the width of the boundary is 35 times the average character width of the font and font size that the database field is formatted to. Remember that this is the initial default boundary width. The width can always be resized to increase or decrease as you see fit.
- For text-based objects, the default width is approximately 19 average character widths wide. Text objects are different from database fields in that their width automatically expands as you enter in text and/or database fields into the object. As with all other text-based objects, the width can be resized by the user.

- For different number fields (double, single, integer, long integer, and byte) the default widths are all different. As with all objects, the width can be resized by the user.

12.3.7.1 Preventing the truncation of text inside an object

Whether the default widths are accepted or the text-based objects are resized, a problem could arise if the text inside the object prints right to the edge of the object frame. While the report may look fine on the machine it was designed on, when the report is printed using another printer driver that measures the font wider, the length of the text grows, but the object frame remains fixed. The resulting text is cut-off or truncated.

12.3.7.1.1 To prevent the truncation of text inside an object

Procedure

1. Right-click the text object you want to format to bring up the shortcut menu.

→ Tip



Another way to do this is to click the Format button on the Expert Tools toolbar.

2. On the shortcut menu, click *Format Text*.

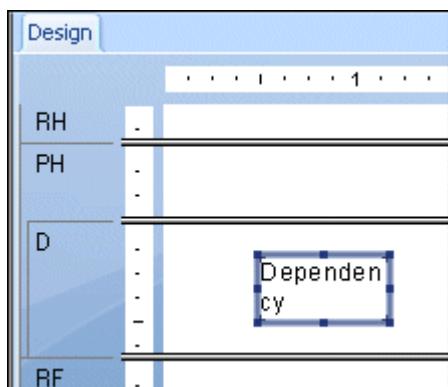
The Format Editor dialog box appears.

3. On the *Common* tab, select the *Can Grow* check box.
4. Click *OK* to save your changes.

The object is then formatted to print on multiple lines. If the text prints wider than the object, the text wraps onto additional lines.

12.3.7.2 Preventing breaks in non-spacing text inside an object

For text strings that do not contain spaces, such as single words, the text string is broken at the edge of the object frame before the line starts to wrap.



12.3.7.2.1 To prevent the breaks in non-spacing text inside an object

Procedure

1. Select the object you want to format.
2. Expand the object frame to make it wider than the widest block of text inside the frame.

Results

There are many times when the actual text in a database field is far less than the maximum amount the field can contain. For example, a {table.LAST NAME} field is designed with a field size of 80 and the longest name in the database is 28 characters. In this case, when you first place the field in your report, the field is 80 times the average character width. Reduce the width of the field, but include enough space to account for growth.

While each of these options offers an effective solution when dealing with a single text-based object in a section, there are still design considerations to take into account when placing more than one object in a section. When sizing an object, consider its placement with regard to other objects in the section.

Avoid designing reports where the space between each object is very tight. Leave room for growth by expanding the width of the object by approximately 5 per cent. Or, if this is not possible, consider reducing the size of the font or placing each object in its own subsection.

12.3.7.3 Suppressing blank lines in embedded fields

Because you can embed fields in text objects, you might encounter cases where an empty field causes a blank line in a text object. You can suppress blank instances of such embedded fields.

Note

The suppression of embedded field blank lines is designed to remove blank lines within a text object if the text object contains an entirely blank field alone on a line followed by a carriage return.

12.3.7.3.1 To suppress blank lines in embedded fields

Procedure

1. Open your report in the *Design* tab, and click the desired text object—that is, the text object that causes blank lines to show for some records.

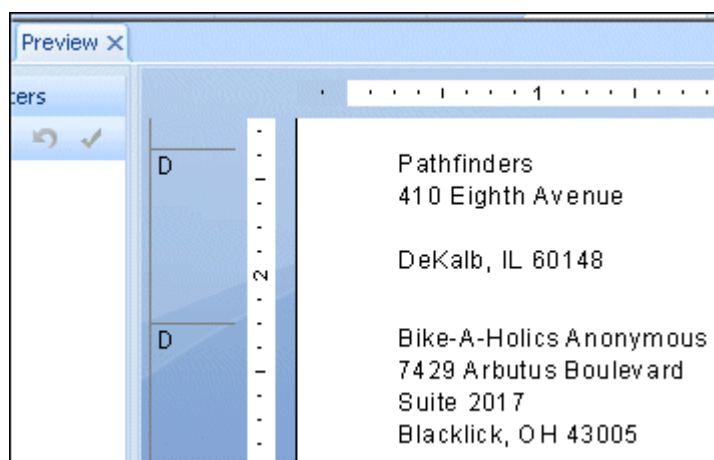
→ Tip

To ensure that you have clicked a text object, look for the word *Text* in the status bar at the bottom left corner of the screen.

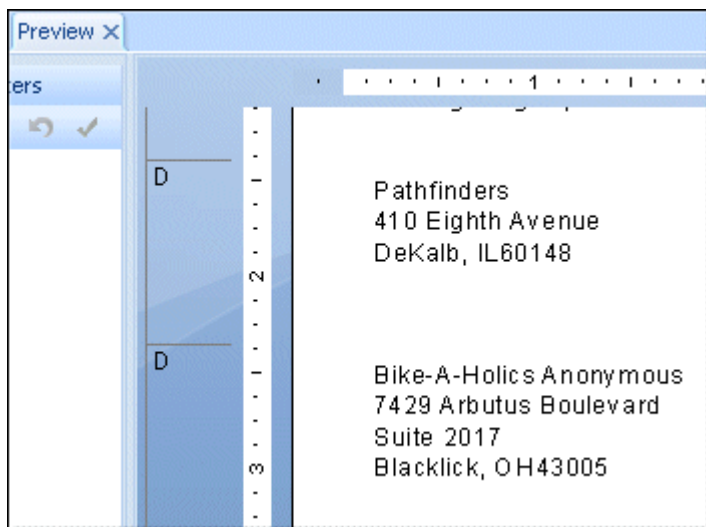
2. Right-click the text object and, on the shortcut menu, click *Format Text*.
3. Select the *Suppress Embedded Field Blank Lines* option in the Format Editor, and then click *OK*.

Results

Now, when the report prints, unwanted blank lines will no longer appear in place of empty embedded fields. You can confirm your changes in the Preview tab.



Before suppressing blank lines, the Address2 field prints a blank line if it is empty.



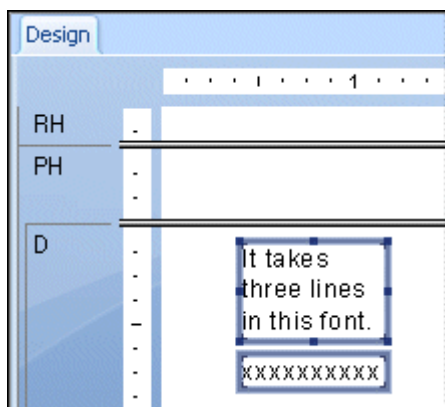
After suppressing blank lines, the Address2 field does not print if it is empty.

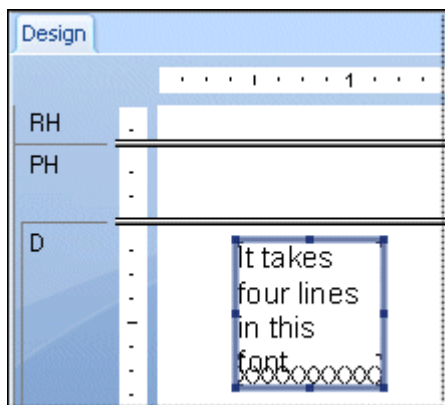
12.3.8 Placing multi-line, text-based objects

While text-based objects that are formatted to print on multiple lines follow the same design rules as other objects, they have one additional characteristic to be considered. If the printer driver expands or contracts the spacing of the text, word wrapping may differ, changing the number of lines necessary to print the object in order to accommodate growth or shrinkage.

When placing multi-line text-based objects, you could encounter problems if other objects in the same section are placed directly below them.

Unlike single-line text-based objects, expanding the object frame of a multi-line text-based object to accommodate growth is not a viable option. When you do this, the line width increases according to the expanded boundaries.





So, when possible, place multi-line text-based objects at the bottom of a section. If they require more lines to print, the section expands downward to accommodate the growth, and they do not endanger other objects.

12.3.9 Importing text-based objects from a file

Using Crystal Reports, you can import a formatted, text-based object from an existing file onto your report.

12.3.9.1 To import text-based objects from a file

Procedure

1. Double-click the text-based object you want to format to put it in edit mode, then right-click it to bring up the shortcut menu.
2. On the shortcut menu, click *Insert from file*.
3. In the Open dialog box that appears, select the file in which your text-based object is stored, then click *Open*.

The object is imported from the file into the text object on your report.

12.3.10 Spacing between text-based objects

Use the grid and guidelines options to help evenly align text-based objects.

You can select the Snap To Grid option, set the grid to a maximum of one inch, and make the grid visible or invisible on the Design tab, Preview tab, or both. For more information on working with grids, see [Using the grid \[page 254\]](#).

You can also work without a grid, placing objects wherever you want them on a report. You may want to work in a free-form environment while retaining the ability to align objects, or to move or resize them as a group. You can do this using guidelines. See [Designing with guidelines \[page 255\]](#).

12.3.10.1 Using the grid

The grid is a series of row and column coordinates. When the grid is selected and the Snap To Grid option is selected on the Layout tab of the Options dialog box, Crystal Reports enables you to place text-based objects only at these coordinates, not between them. You can then space data on your report and align objects as needed. If you attempt to place an object between grid coordinates, the object "snaps" to the grid; that is, the object automatically moves to the nearest set of row and column coordinates.

Each report contains a design grid. You can select the grid on or off, as well as set it to different sizes when required. By default, the grid is not selected. See [Selecting the grid \[page 254\]](#).

Once set, the grid remains the same size for all sections. It is measured from the upper-left corner of each section and continues down and to the right, until the end of the section. A new grid of the same size then starts from the upper-left corner of the next section, and so on, through the end of the report.

If you select the Snap To Grid option, the following conditions occur:

- The upper-left corner of all newly placed, text-based and OLE objects snap to a grid point.
- Objects placed on a report before the Snap To Grid option is selected do not snap to the nearest grid point. They remain in the same place.
- If you resize an object, the side or sides that you are resizing snap to the nearest grid point.

12.3.10.2 Selecting the grid

The Design and Preview tabs have an underlying grid structure that you can activate on the Layout tab in the Options dialog box.

12.3.10.2.1 To select the grid

Procedure

1. On the *File* menu, click *Options*.
The Options dialog box appears.
2. On the *Layout* tab, in the Grid options area, activate the snap to grid feature, or specify the grid size.
3. To show the underlying grid structure on the Design or Preview tab, select Grid in the Design View or Preview areas.
4. Click *OK* to save your changes.

Results

→ Tip

Another way to do this is to select the Grid commands from the View menu. You can also right-click an empty space on the Design or Preview tabs and select the command from the shortcut menu.

12.3.10.3 Designing with guidelines

Crystal Reports provides guidelines to help you align and size report objects with accuracy. Guidelines are non-printing lines that you can place anywhere on the Design and Preview tabs to aid in alignment. They have a snap property that automatically snaps objects to them.

12.3.10.4 Viewing guidelines

You can view guidelines on the Design and Preview tabs by selecting the view options in the Options dialog box.

12.3.10.4.1 Viewing guidelines

Procedure

1. On the *File* menu, click *Options*.
The Options dialog box appears.
2. On the *Layout* tab, in the Design View area, select the *Guidelines* check box and/or the *Guidelines* check box in the Preview area.
3. Click *OK* to save your changes.

Results

→ Tip

Another way to do this is to select a Guidelines command from the View menu.

12.3.10.5 Inserting guidelines

Although you can and should insert manual guidelines whenever necessary, Crystal Reports will automatically insert guidelines in certain situations:

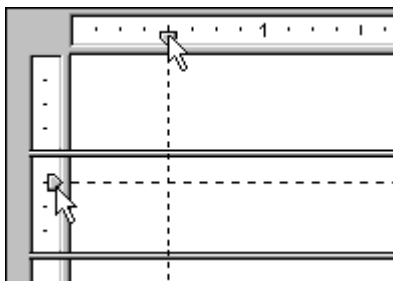
- Whenever you add a field or formula field to a report, the program creates a guideline at the left edge of the field frame and snaps the field and field title to it.
- When a field is summarized, the program snaps the summary to the same guideline to ensure proper alignment.
- When you right-click the shaded areas to the left of a section, then select the Arrange Lines option from the shortcut menu, the program creates one or more horizontal guidelines in the section and snaps the fields to them.

12.3.10.5.1 To insert, move, and remove guidelines manually

Procedure

1. In the Design or Preview tab, click the ruler at the top to activate a vertical guideline; click the ruler on the left to activate a horizontal guideline.

Notice that each guideline is attached to an arrowhead on its originating ruler.



Note

If guidelines don't appear, ensure that the appropriate Guideline option is selected on the View menu. On the Preview tab, you must select an object to see a guideline.

2. To position a guideline, drag its arrowhead along the ruler to the desired location.
3. To delete a guideline, drag its arrowhead away from the ruler.

Results

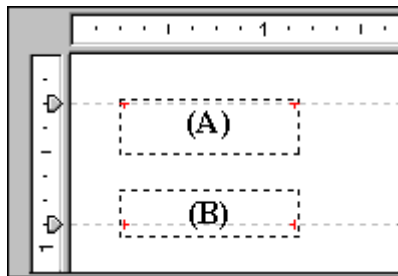
Note

If you select the Snap To Grid option, you can only insert or move guidelines in grid increments.

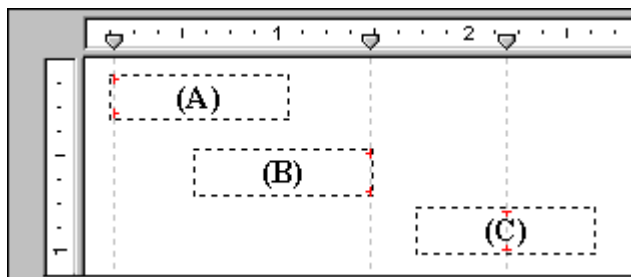
12.3.10.6 Snapping objects to guidelines

To snap an object to a guideline, drag the report object onto the guideline until the object's edge sits atop the guideline. Snapping enables you not only to align report objects accurately, but also to re-position and re-size multiple objects together. Once several objects are snapped to a guideline, you can move all the objects by moving the guideline.

You can snap either the top or the bottom of an object to a horizontal guideline.



You can snap an object's left side, right side, or vertical midline (the invisible line that bisects an object vertically) to a vertical guideline.



12.3.10.6.1 To snap objects to a guideline

Procedure

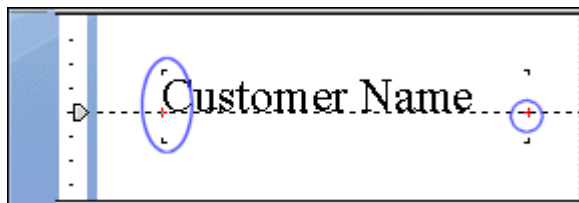
1. Insert a guideline by clicking one of the rulers.
2. Drag the report object onto the guideline, so that one of the object's edges is on the guideline.

Results

The snap property of guidelines works differently for text-based objects than for other objects, such as OLE objects. When a single-line text-based object snaps to a guideline, it is the baseline of the text, not the object

frame, that snaps to the guideline. When a multi-line text-based object snaps to a guideline, either the baseline of the text or the object frame can be snapped to the guideline.

You can tell if a text-based object is snapped to a horizontal guideline by looking for the special indicators positioned on either side of the object directly at the baseline (as circled in the image below) in Design view. If the object is snapped to a vertical guideline, the special indicators appear along the side of the object.



To place several text objects of different font sizes on one line with their baselines lined up, snap each object's baseline to the same horizontal guideline.

12.3.10.7 Positioning objects using guidelines

Once you've snapped one or more objects to a guideline, you can move all the objects at once by moving the guideline. To move the guideline, drag its arrowhead along the ruler.

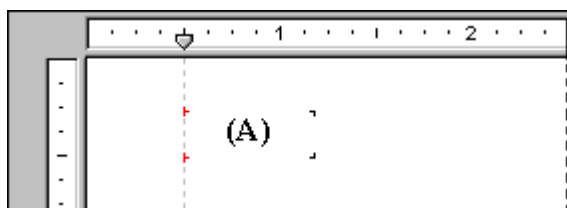
Note

When a guideline is moved, any object that is snapped to it is moved as well. But, if you move an object that is snapped to a guideline, the guideline does not move.

12.3.10.8 To resize objects using guidelines

Procedure

1. Create a guideline.
2. Snap one side of the object to that guideline.

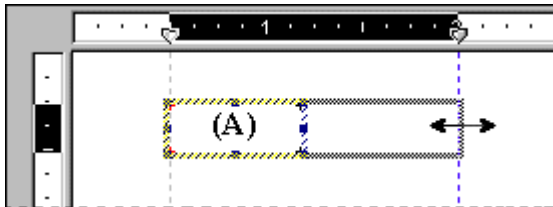


3. Create a second guideline to the right of the object.

Note

The guideline should not be touching the object.

- Click the object to activate the sizing handles.
- Drag the resizing handle over to the second guideline so that the object snaps to the guideline.

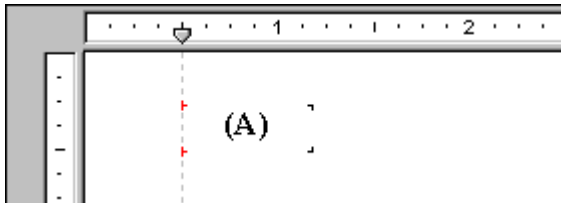


- For each additional object you want to snap to both guidelines, repeat Steps 2 through 5.
- If the objects are not the desired size, drag one or both of the guidelines until the objects are the correct size.

12.3.10.8.1 To resize objects using guidelines

Procedure

- Create a guideline.
- Snap one side of the object to that guideline.

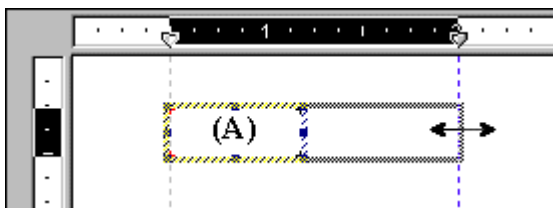


- Create a second guideline to the right of the object.

Note

The guideline should not be touching the object.

- Click the object to activate the sizing handles.
- Drag the resizing handle over to the second guideline so that the object snaps to the guideline.



6. For each additional object you want to snap to both guidelines, repeat Steps 2 through 5.
7. If the objects are not the desired size, drag one or both of the guidelines until the objects are the correct size.

12.3.10.9 Indenting lines

Using Crystal Reports, you can control line indentation for memo fields, string fields, and text-based objects. For objects, you have the option of indenting lines for a particular paragraph by positioning the cursor at the start of that paragraph. Or, if you select an object in its entirety, you can apply the same indenting specifications to all the paragraphs within that object.

Keep in mind that any line following a carriage return will be considered the first line of a new paragraph.

12.3.10.9.1 To indent lines

Procedure

1. Right-click the field or object you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format text*.
The Format Editor dialog box appears.
3. Click the *Paragraph* tab.
4. In the Indentations area, you can indent the first line of the paragraph; indent every paragraph line from the left edge of the object; and indent every paragraph line from the right edge of the object.

Note

Only indentation values within the range of the field or object width are accepted.

Note

When you select the "Right to Left" reading order, indents are measured from the opposite side of the object. That is, a left indentation is measured from the right side of the object.

5. Click *OK* to save your changes.

12.3.11 Deleting a field

Now that the report contains only records from the USA, displaying the Country field in the body of the report is not necessary. You can delete this field before continuing.

12.3.11.1 To delete a field

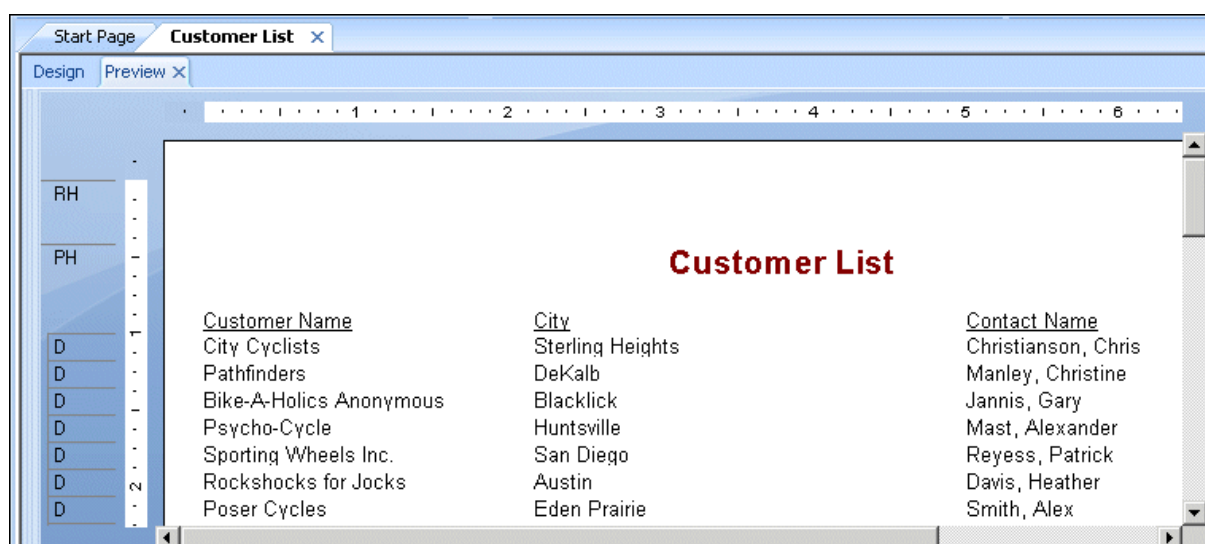
Procedure

1. Select the *Country* field and the *Country* column heading using the *Ctrl-click* combination.
2. Press *Delete* on your keyboard.

Results

That is all it takes to delete fields from the report.

The report should now look similar to this:



<u>Customer Name</u>	<u>City</u>	<u>Contact Name</u>
City Cyclists	Sterling Heights	Christianson, Chris
Pathfinders	DeKalb	Manley, Christine
Bike-A-Holics Anonymous	Blacklick	Jannis, Gary
Psycho-Cycle	Huntsville	Mast, Alexander
Sporting Wheels Inc.	San Diego	Reyess, Patrick
Rockshocks for Jocks	Austin	Davis, Heather
Poser Cycles	Eden Prairie	Smith, Alex

12.3.12 Balancing field spacing

Now that the Country field has been deleted, there is a large amount of white space between the City and Contact Name fields. You might be satisfied with the spacing as it stands, but it might be more readable if the columns were better balanced across the page.

12.3.12.1 To balance field spacing

Procedure

1. Return to the [Design](#) tab. Select the [Contact Name](#) field and its field heading by using the [Ctrl-click](#) combination.
2. Place the cursor over one of the two highlighted text objects and drag them to the left, closer to the [City](#) field.
3. Click the [Preview](#) tab and review your work again.

The report should look similar to this:

<u>Customer Name</u>	<u>City</u>	<u>Contact Name</u>
City Cyclists	Sterling Heights	Christianson, Chris
Pathfinders	DeKalb	Manley, Christine
Bike-A-Holics Anonymous	Blacklick	Jannis, Gary
Psycho-Cycle	Huntsville	Mast, Alexander
Sporting Wheels Inc.	San Diego	Reyess, Patrick
Rockshocks for Jocks	Austin	Davis, Heather
Poser Cycles	Eden Prairie	Smith, Alex

The spacing between the fields is much better, but it looks as if the report title is off-center.

4. Click the report title object to select it.
5. Position the cursor on the right handle of the object until the cursor turns into a Resizing cursor. Drag the right edge of the object frame until it is even with the right edge of the data in the [Contact Name](#) field object.

The report title automatically centers itself based on the size of the object.

12.3.13 To allow for overflow field representation

Procedure

1. Right-click the currency field or number field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click [Format Field](#).

The Format Editor dialog box appears with the Number tab open.

3. Click the *Customize* button.

The Custom Style dialog box appears with the Number tab open.

4. To allow overflow field representation, clear the *Allow Field Clipping* check box.

Note



You also have the option to click the Conditional Formatting button to enter a formula in the Format Editor. In the Format Formula Workshop, you can specify that field clipping will be disabled only when certain conditions are met.

5. Click *OK* to save your changes.

Results

To view the results, refresh the report. If you disabled field clipping, any numeric/currency field values that are larger than the field objects containing them will be represented by number signs (#####).

12.3.13.1 To allow for overflow field representation

Procedure

1. Right-click the currency field or number field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.

The Format Editor dialog box appears with the Number tab open.

3. Click the *Customize* button.

The Custom Style dialog box appears with the Number tab open.

4. To allow overflow field representation, clear the *Allow Field Clipping* check box.

Note



You also have the option to click the Conditional Formatting button to enter a formula in the Format Editor. In the Format Formula Workshop, you can specify that field clipping will be disabled only when certain conditions are met.

5. Click *OK* to save your changes.

Results

To view the results, refresh the report. If you disabled field clipping, any numeric/currency field values that are larger than the field objects containing them will be represented by number signs (#####).

12.3.14 Selecting multiple objects

You can select multiple objects, including text, field, chart, map, bitmap, OLAP grid, Cross-Tab and OLE objects, to format them together.

Once you have selected multiple objects, you can move, align, size, cut, and copy and paste them as a group. You can also change their font, color, and paragraph style.

Objects are moved, aligned, and sized based on a "main" object, which is usually the last object you select. You can change the main object to another by right-clicking the desired object.

12.3.14.1 To select multiple objects

Procedure

1. Click one object, and [Ctrl+click](#) the other objects you want to select.

Note

You can also lasso a number of objects by left-clicking and dragging your cursor over an area.

2. Right-click the main object.
3. On the short-cut menu, select the appropriate formatting option.

12.3.14.2 To select all objects in a section

Procedure

1. On either the Design or Preview tab, right-click the gray area to the left of the section you want to work with.
2. From the shortcut menu, select [Select All Sections Objects](#).

All report objects in that section are selected.

12.3.15 Vertical placement

On the Common tab of the Format Editor, you can use the text rotation options to vertically align the fields and text-based objects on your report.

When you select a text rotation of 90 degrees, the text shifts 90 degrees in a counter-clockwise direction.

When you select a text rotation of 270 degrees, the text shifts 270 degrees in a counter-clockwise direction.

❗ Note

If text rotation is left at 0 degrees, your report is horizontally formatted, left to right.

❗ Note

For text rotation of text-based objects, the Can Grow option that prevents the truncation of text inside an object is automatically cleared. For more information on the Can Grow option, see [Preventing the truncation of text inside an object \[page 249\]](#).

❗ Note

Vertically formatted text that spans over the edge of the page cannot be displayed as part of your report.

12.3.16 Inserting character and line spacing

With Crystal Reports, you can specify the amount of spacing between characters or lines for memo fields, string fields, and text-based objects.

12.3.16.1 To insert character and line spacing

Procedure

1. Right-click the field or object you want to format to bring up the shortcut menu.
2. On the shortcut menu, click [Format Field](#).

The Format Editor dialog box appears.

3. Click the [Font](#) tab.

You will use this tab to set up the character spacing values.

4. In the Spacing area, in the [Character spacing exactly](#) field, specify the value **<n>** that each character occupies.

The value n is defined as the distance in number of points measured from the start of one character to the start of the next. When you change the character spacing, you change only the spacing between adjacent characters, not the font size of the characters.

For example, if you specify a 14-point font with a character spacing of 14 points, each character will remain as a 14-point font size, occupying a space that is 14 points wide.

5. Click the [Paragraph](#) tab.

You will use this tab to set up the line spacing values.

6. In the Spacing area, in the [Line spacing](#) field, specify the line spacing as a multiple of the font size you are using, or as an exact number of points.
7. Click [OK](#) to save your changes.

12.3.17 Setting fractional font sizes

On the Font tab of the Format Editor, you can select a fractional font size for database fields and text-based objects on your report.

12.3.17.1 To set fractional font sizes

Procedure

1. Right-click the field or object you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.
The Format Editor dialog box appears.
3. Click the *Font* tab.
4. In the *Size* list, type the desired fractional font size for the field or object.

Note

The number you type must be between 1 and 1638. Crystal Reports will automatically round all fractional entries to the nearest 0.5. Consequently, in your report, you can use the fractional font sizes 1.5, 2.5, 3.5, and so on, up to 1637.5.

5. Click *OK* to save your changes.

Results

Note

When setting fractional font sizes for individual database fields and text-based objects that you've already placed on your report, you must make your changes manually—that is, by following these procedures. (This is because the existing font settings of objects in your report will override your default Options.) However, you can use the Fonts tab of the Options dialog box to adjust your default font settings: these default Options will affect the new reports that you create, along with any new objects that you add to an existing report.

12.3.18 Setting page size and page orientation

You can view and print your reports using either portrait or landscape orientation, and in a variety of page sizes.

You may also set different page orientation for each section in your report. This is useful for formatting certain sections to display charts or other graphics.

Specify these options using the *Page Setup* command from the *File* menu.

12.3.18.1 To set page size and page orientation

Procedure

1. On the *File* menu, click *Page Setup*.
The *Page Setup* dialog box appears.
2. In the *Page Size* box, select your page size.
3. In the *Page Orientation* box, select your page orientation.
4. In the *Margins* box, select your margins.
For information on conditional margins, see [Changing margins conditionally \[page 291\]](#).

12.3.18.2 To set page orientation by section

Procedure

1. On the *Report* menu, click *Section Expert*.

The *Section Expert* appears.

→ Tip



Another way to do this is to click the *Section Expert* button on the *Expert Tools* toolbar.

2. In the *Sections* list, select the section to be modified.
3. On the *Paging* tab, select the desired page orientation.

ⓘ Note

This option is not available for the Page Header and Pager Footer.

12.3.19 Setting page margins

Margins are the write spaces on the left, right, top, and bottom of the page. Crystal Reports gives you the option of setting margins to meet your specifications.

12.3.19.1 To set specific margins

Procedure

1. On the *File* menu, click *Page Setup*.
The *Page Setup* dialog box appears.
2. Change the default page margins to fit your needs.
3. Click *OK* to save your changes.

Results

Page margins can also be controlled by use of conditional formulas. See [Changing margins conditionally \[page 291\]](#).

ⓘ Note

All margins are calculated from the page edge. Thus, a left margin of .25 inches causes the printing to start exactly one quarter inch in from the edge of the page.

ⓘ Note

You can select the *Adjust Automatically* check box if you want Crystal Reports to adjust the report's margins automatically the next time you change the page size. If you choose a new page size that is large enough for the current printable area, Crystal Reports increases or decreases the margins by enlarging or reducing the left/right and top/bottom margins by the same factor. If you choose a new page size that is smaller than the current printable area, Crystal Reports fills the entire page by reducing the margins to 0. If you next choose a bigger page, this (reduced) printable area is kept and the left/right, top/bottom margin ratios become 1:1.

12.3.20 TrueType fonts

Designing your report with printer-specific fonts can lead to problems when using different printers. The fonts may not be supported by the other printers, or if they are supported, the fonts may not be installed on the printers.

During the printing process, if you encounter printer-specific fonts that are unrecognizable to the printer driver, Crystal Reports substitutes the fonts, creating inconsistent results. To avoid this situation, only common TrueType fonts should be used when designing reports.

12.3.21 Printer drivers

Click a link to view one of the topics in this section:

- [Updating printer drivers \[page 269\]](#)
- [Inconsistencies due to printer drivers \[page 269\]](#)

12.3.21.1 Updating printer drivers

Prerequisites

In order to maintain performance, Crystal Reports queries the printer driver for each of the font elements (font metrics), such as average character height, character width, height of the ascenders and descenders, internal leading, and so on. A problem may develop if using an older printer driver that does not return the font metrics accurately. If you are experiencing problems when printing (missing fields, incorrect formatting, and so on), it is recommended that you obtain and install the most recently updated drivers for your printer. In many cases, the newer printer drivers provide accurate font metrics and any printing issues are quickly resolved.

12.3.21.2 Inconsistencies due to printer drivers

Inconsistencies may occur if different printer drivers are used to create and print your reports. These inconsistencies are a result of the various methods that individual printer drivers use to measure text metrics such as font size. When printed, text-based objects may be misaligned, truncated, or overprint each other. Examples of text-based objects include string or character fields, text objects, memo fields, numeric fields, and formula fields.

Problems such as these may arise when you have:

- Two identical printers, but each one is using a different printer driver.
- Two different printers using the same printer driver.
- Two different printers using different printer drivers.
- One printer driver that uses the TrueType font and a second printer driver that maps TrueType fonts to PostScript fonts.
- Two identical printers using the same printer driver, but each one is printing from a different version of Microsoft Windows.
- Two identical printers using the same printer driver, but the printer drivers are different versions.
- Two identical printers, two identical printer drivers, and two identical operating systems, but the resolution of the video drivers is different.

Therefore, while a document using one printer driver may require six full lines to display a block of text:

- Using a second printer driver that measures fonts narrower could result in the same block of text requiring less than six full lines.
- Using a third printer driver that measures fonts wider could require more than six full lines.

The goal of the report distributor is to design reports that accommodate printer driver dependency and still print consistently using different printer drivers. To do this, Crystal Reports provides several design solutions. If taken into account when creating your report, these solutions can ensure proper printing and distribution for your report in almost any environment.

For reports that will be viewed online, you can avoid inconsistencies from printer drivers by disabling the report's reliance on a printer driver. For more information on disabling a printer driver, see [Formatting a report for web viewing \[page 270\]](#).

12.3.22 Formatting a report for web viewing

If your report will be viewed online, you may choose to format your report for web viewing. You can create custom page sizes and adjust margins for a viewer's screen without being restricted to printer-specific settings. Disabling reliance on a printer driver also prevents inconsistencies that can occur if your report is viewed with a different printer driver than the one you created it with.

12.3.22.1 To format a report for web viewing

Procedure

1. On the [File](#) menu, click [Page Setup](#).

The Page Setup dialog box appears.

2. In the Printer Options box, select the No Printer check box.
3. Select your desired page size, page orientation, and page margins.

ⓘ Note

Reports formatted for web viewing can also be printed. If the custom page size is larger than the printed page size, your report will be scaled down to fit the page. If the custom page size is smaller than the printed page size, a portion of your report will be enlarged to fit the printed page size.

ⓘ Note

Exported reports will retain custom page formatting except for reports exported to XLS format.

12.4 Formatting properties

You can set formatting properties using the Format Editor for objects and the Section Expert for report sections. In most cases, you can set one of two types of properties:

- Absolute (always apply the property).

- Conditional (apply the property only if certain criteria are met).

12.5 Working with absolute formatting

Absolute formatting is formatting that applies under any condition. This type of formatting property always follows a "select, then apply" procedure. For example, you select what it is that you want to format (object or section), then you apply the formatting to the selection using property settings.

You can use the following dialog boxes to format your reports:

- Format Editor to format field values.
- Section Expert to format entire sections.
- Highlighting Expert to conditionally format all types of fields.

Each of these dialog boxes contains a number of different formatting properties, as well as the tools for turning the properties on or off and specifying attributes.

12.5.1 Adding borders, color, and shading to a field

Crystal Reports allows you to add borders, color, and shading to fields on your report in order to emphasize important data and create professional-looking reports.

12.5.1.1 To add borders, color, and shading to a field

Procedure

1. Right-click the field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.
The Format Editor dialog box appears.
3. Click the *Border* tab.
4. Select the line style, color, and background color of the field.
5. Click *OK* to save your changes.

12.5.2 Making a report, section, area, or object read-only

You can make a report, section, area, or object read-only so it can't be formatted. When you set this option, choices in the Format Editor become inactive. The formatting options that are usually available on the toolbars or shortcut menus are also suppressed for the report or object.

Note

This feature is for your convenience in protecting report formatting; it is not intended to be used as report security.

12.5.2.1 To make a report read-only

Procedure

1. On the *File* menu, select *Report Options*.
2. Select *Read-only*.

12.5.2.2 To make a section or area read-only

Procedure

1. On the *Report* menu, select *Section Expert*.
2. Select the section or area that you want to make read-only.
3. On the *Common* tab, select *Read-only*.

Note

When a section is read-only, you can move it, cut it, and delete it, but you cannot make changes within the section. When an area is read-only, all sections in the area are read-only; therefore, you cannot insert another section, nor can you move, cut, or delete an existing section.

12.5.2.3 To make an object read-only

Procedure

1. Right-click the object you want to make read-only.
2. On the shortcut menu, click *Format Field*.
The Format Editor dialog box appears.
3. On the *Common* tab, select *Read-only*.

Note



You can also do this by clicking the Lock Format button on the Formatting toolbar.

Note

When a report object is read-only, you can move it, copy it, cut it, delete it, browse its data, and select it, but you cannot format it.

12.5.3 Locking an object's size and position

You can lock the position of the selected report object so it can't be moved. When you set this option, you cannot drag the object in the Report Designer and the Size and Position command becomes inactive.

12.5.3.1 To lock an object's size and position

Procedure

1. Select the object whose size and position you want to lock.
2.  On the Formatting toolbar, click [Lock Size/Position](#).

12.5.4 Changing your default field formats

Crystal Reports allows you to display database fields in almost any format on your report. This section describes how to use the Options command to control the default format settings that Crystal Reports uses when you add a field to any report. In the Options dialog box, you can set the default formats for database fields of the following type: String, Number, Currency, Date, Time, Date and Time, and Boolean.

Note

When you change default field formats, your new settings affect only the objects that you subsequently add to a report. To format fields that you've already added to a report, you must right-click the field in the report and select Format Field from the shortcut menu.

12.5.4.1 To specify default formats for fields

Procedure

1. On the [File](#) menu, click [Options](#).
2. In the Options dialog box, click the [Fields](#) tab.

3. Click the button appropriate to the type of field you want to format (String, Number, Currency, Date, Time, Date and Time, or Boolean).

The Format Editor appears.

4. Use the Format Editor's tabs to specify the formats you want.
5. Click [OK](#).

12.5.4.2 Setting default formats for Date, Time, and Date and Time fields

The following procedures first describe how to specify standard formats for Date, Time, and Date and Time fields, and then describe how to customize the formats for such fields.

Note

These default settings will affect only the objects that you subsequently add to a report. To format fields that you've already added to a report, you must right-click the field in the report and select Format Field from the shortcut menu.

12.5.4.2.1 To set standard default formats for Date, Time, and Date and Time fields

Procedure

1. On the [File](#) menu, click [Options](#).
2. In the Options dialog box, click the [Fields](#) tab.
3. To open the Format Editor, click the button appropriate to the field you want to format ([Date](#), [Time](#), or [Date and Time](#)).

Note

If you click the Date and Time button in the Format Editor, then any subsequent changes will affect "Date and Time" fields only. You must click Date or Time to format independent date fields or time fields.

4. In the Format Editor dialog box, click the [Date and Time](#) tab.
5. Select a predefined format from the list (or click [Customize](#) to create your preferred format). When you click a new format, you can preview the results in the Sample area of the Format Editor.

Note

From the list of predefined formats, you can choose the System Default settings to ensure that Crystal Reports uses the formats dictated by Windows. You can alter your system's settings in the Regional Settings Properties dialog box, located in the Control Panel.

6. Once you've selected a format, click [OK](#) in the Format Editor dialog box.
7. Click [OK](#) in the Options dialog box.

Results

Now, when you add Date, Time, or Date and Time fields to a report, Crystal Reports should use the format you specified.

12.5.4.2.2 To customize formats for Date, Time, and Date and Time fields

Procedure

1. On the [File](#) menu, click [Options](#).
2. In the Options dialog box, click the [Fields](#) tab.
3. Open the Format Editor by clicking the button appropriate to the field you want to format ([Date](#), [Time](#), or [Date and Time](#)).
4. In the Format Editor dialog box, click the [Date and Time](#) tab.
5. Click [Customize](#) to open the Custom Style dialog box.

Note

If you chose to format "Date and Time" fields at Step 3, then you will see three tabs in the Custom Style dialog box (Date and Time, Date, and Time). The formats specified in these tabs apply only to the two elements of "Date and Time" fields, and will not affect the formats specified for independent date fields or time fields.

6. Create your preferred format by adjusting the various options in the Custom Style dialog box.
7. Once you've finished designing your format, click [OK](#) in the Custom Style dialog box.
8. Click [OK](#) in the Format Editor dialog box.
9. To format another type of field, click the appropriate button in the Options dialog box. Otherwise, click [OK](#) to return to Crystal Reports.

Results

Now, when you add Date, Time, or Date and Time fields to your reports, Crystal Reports should use the customized format that you created.

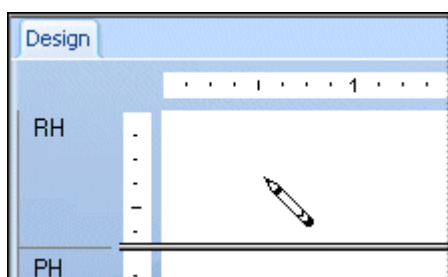
12.5.5 Adding and editing lines

Crystal Reports allows you to add lines to a report to emphasize important data and create professional-looking reports. Lines can run horizontally or vertically. For a vertical line to be broken across several pages, the report section that the line ends in must not be on the same page that it starts on. For example, if a line runs from a group header to the corresponding group footer, the line continues on the top of each subsequent page—just below the page header—until the group footer is encountered.

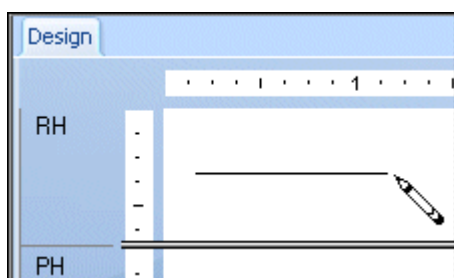
12.5.5.1 To add lines to a report

Procedure

1.  On the Insert Tools toolbar, click *Insert Line*.



2. Use the pencil cursor to draw the line where desired.



Note

You cannot draw diagonal lines.

12.5.5.2 To edit lines on a report

Procedure

1. Right-click the line you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Line*.

The Format Editor dialog box appears.

3. On the *Line* tab, make the desired changes to the line.
4. Click *OK* to save your changes.

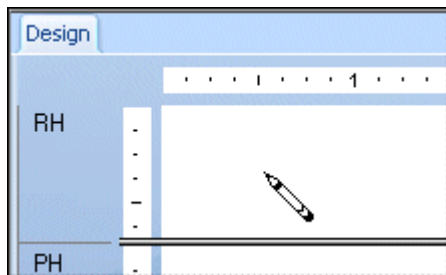
12.5.6 Adding and editing boxes

Crystal Reports allows you to add boxes to a report to emphasize important data and create professional-looking reports.

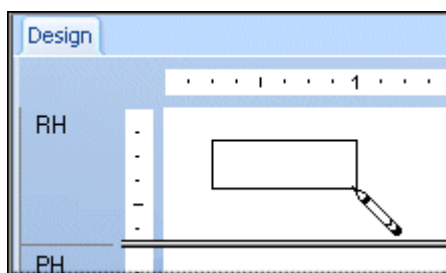
12.5.6.1 To add boxes to a report

Procedure

1.  On the Insert Tools toolbar, click *Insert Box*.



2. Use the pencil cursor to draw the box where desired.



12.5.6.2 To edit boxes on a report

Procedure

1. Right-click the box you want to format to bring up the shortcut menu.
2. On the shortcut menu, click [Format Box](#).
The Format Editor dialog box appears.
3. On the [Box](#) tab, make the desired changes to the box.
4. Click [OK](#) to save your changes.

12.5.7 Expected behavior of line and box formatting

This section lists the expected behavior of line and box formatting for some common cases.

Note

This table uses Short Section Names, for more information about this convention, see [Identifying and working with areas and sections \[page 92\]](#).

Between these sections	Start	End
• RH to PH • RH to RF • RH to PF • PH to RF • PH to PF • RH to PH	In the top section, or if that section is suppressed, at the top of the next visible section before the bottom section (including the bottom section).	In the bottom section, or if that section is suppressed, at the top of the next visible section.
• RH to GH/D/GF • PH to GH/D/GF	In the top section, or if that section is suppressed, at the top of the next visible header section (RH or PH). If these sections are suppressed, the first visible GH/D/GF section.	In the last instance of the GH/D/GF on a page, or if these sections are suppressed, before the first footer section.
• GH/D/GF to RF • GH/D/GF to PF	In the first instance of a GH/D/GF section on a page, or if these sections are suppressed, at the top of the first visible footer section (RF or PF)—provided that the section is not after the bottom section.	In the bottom section, or if that section is suppressed, at the top of the next visible section.

Between these sections	Start	End
GH2 to GH3	In every instance of the GH2 section, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the first instance of the GH3 section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
GH2 to D	In every instance of the GH2 section, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the first instance of the D section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
GH2 to GF3	In every instance of the GH2 section, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the first instance of the GF3 section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
GH2 to GF2	In every instance of the GH2 section, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the GF2 section that corresponds to the GH2 section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
GH2 to GF1	In the first instance of the GH2 section in group one, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the end section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
D to GF2	In the first instance of the D section in group two, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the end section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.
GF2 to GF1	In the first instance of the GH2 section in group one, or if that section is suppressed, at the top of any GH/D/GF section before the end section.	In the end section, or if that section is suppressed, at the top of the next visible GH/D/GF section. If these sections are suppressed, at the top of the first RH/PH/RF/PF.

12.5.8 Adding shapes to a report

When designing report formats in Crystal Reports, you can insert a variety of shapes such as circles, ellipses, and boxes with rounded corners, as part of your report. This is especially useful for formatting reports in languages that require these shapes to effectively communicate.

12.5.8.1 To add shapes to a report

Procedure

1. Add a box to your report.
See [Adding and editing boxes \[page 277\]](#).
2. Right-click the box to bring up the shortcut menu.
3. On the shortcut menu, click [Format Box](#).
4. In the Format Editor that appears, click the [Rounding](#) tab.
5. Select a number between 0 and 100 in the Rounding box, or move the slider to the right to increase the curvature of the box corners.

The box that you started with gradually changes to an ellipse or circle, depending on how far you move the slider to the right.

Note

If you have specified rounding for a box, you cannot use the Drop Shadow option that is usually available on the Box tab of the Format Editor.

6. Once the appropriate shape is created, click [OK](#) to save your changes.

12.5.9 Scaling, cropping, and sizing objects

When you create or modify a Crystal report, you can insert a variety of OLE objects. For more information about OLE objects and how to insert them in your report, see [OLE \[page 335\]](#). Once an OLE object is present in your report, you can scale, crop, or size it.

12.5.9.1 To scale, crop, or size an object

Procedure

1. Add a picture or other OLE object to your report.
See [Inserting OLE objects into reports \[page 337\]](#).

Note

This procedure also applies to BLOB field objects.

2. Right-click the object and select *Format Graphic* from the shortcut menu.
3. In the *Format Editor*, click the *Picture* tab.
4. To crop the object, specify the size of the piece that you want to cut from its top, bottom, left, and/or right side.

Note

Cropping begins at the outer edge of the object. Positive numbers cut into the object, while negative numbers add the specified amount of white space between the outer edge of the object and the frame.

5. To Scale the object as a percentage of the original height and width, enter the new scaling percentages.

For example, if your object is one inch wide, that width is automatically assigned a width scaling value of 100%. To double the width of the object, change Scaling Width to 200% (twice the size of the original). Likewise, to reduce the width of the object to half, change Scaling Width to 50%.
6. To resize the object, enter a new width and height.

For example, if your object is a one inch square, each of the Size settings is set at one inch. To double the length and width of the object (to make a two inch square), reset the Size settings to two inches each. To reduce the size of the object to a half inch square, reset the Size settings to a half inch each.
7. To keep aspect ratio on the *Format Editor* dialog, use the *Keep Aspect Ratio* checkbox. When *Keep Aspect Ratio* is checked, the
 - *Width %* and *Height %* in scaling box are always the same.
 - Width / Height ratio in size box is kept based on the object's original width / height ratio (and crop values).

Note

Some operations could break aspect ratio (for example: drag to resize in canvas, or align width / height with another report object). In these cases, *Keep Aspect Ratio* checkbox is automatically unchecked.

8. For dynamic pictures, for example, those from database (BLOB field) or from HTTP URL (defined in Graphic Location), the actual picture size often differs to the size in *Format Editor* based on the static sample picture. To generate a desired output, choose one of following display method:
 - *Stretch*: This option helps to stretch the picture to the defined width and height in Size section
 - *Respect Scaling (Can Grow)*: This option respects the width % and height % defined in Scaling section. Each formatted picture can be larger or smaller than the values defined in the Size section.
 - *Fit to box*: This new option keeps each actual picture's aspect ratio (which could differ to the static sample picture's aspect ratio), then resize to fit into the width and height defined in Size section (the box). Some white space (transparent) might appear on top/bottom, or left/right sides.
9. To control the automatic rotation of dynamic JPG pictures in the report, use the *Auto Rotate JPG* checkbox. When it is checked, JPG pictures are rotated according to the orientation tag (for example, rotate 90 degree Clockwise) in each JPG's EXIF data. Orientation tag is widely seen in JPG photos from most digital cameras and mobile phones.

Note

Orientation tags with mirror, e.g. mirror horizontal and rotate 90 degree Clockwise, are not supported in this release, consider mirroring is not widely used.

12.5.10 Using conventional accounting formats

As a way of supporting the conventions used in the accounting profession, Crystal Reports lets you decide on how to display the currency symbol, negative values, and zero values on your financial reports. You can also set up your report to reverse the signs for credit and debit amounts.

12.5.10.1 To use accounting conventions in a report

Procedure

1. Right-click the currency field or number field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.

The Format Editor dialog box appears with the Number tab open.

3. In the Style area, select how you want the system number format to appear for either positive or negative values.
4. In the Currency Symbol (system default) area, specify how you want the currency symbol to appear with the values on your report.
5. Click *OK* to save your changes.

12.5.10.2 To customize the accounting conventions for a report

Procedure

1. Right-click the currency field or number field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.

The Format Editor dialog box appears with the Number tab open.

3. Click the *Customize* button.

The Custom Style dialog box appears with the Number tab open.

4. Select the *Use Accounting Format* check box.

Once you select this option, the following conditions occur:

- In the Negatives list, how the negative values appear on your report are determined by the Windows locale settings. The negative values are represented by either a minus sign or brackets.
- In the Show Zero Values as list, the dash symbol is automatically selected to represent zero values on your report.
- On the Currency Symbol tab of the Custom Style dialog box, the currency symbol is positioned on the left-side of the currency and numeric values.

Note

Changes made to the Windows locale settings are implemented only after you exit and restart Crystal Reports.

5. Select the *Reverse Sign for Display* check box to reverse the signs for debit and credit amounts in your financial reports.
6. Click *OK* to save your changes.
7. Click *OK* again to return to your report.

12.5.11 Repeating report objects on horizontal pages

Some report objects, such as cross-tabs and OLAP grids, can expand horizontally over a number of pages. Crystal Reports lets you designate other report objects that don't expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, to be repeated on each additional horizontal page that the cross-tab or OLAP grid creates. For example, if your report's page footer includes an image, a text object, and a page number, you can use the Format Editor to make Crystal Reports repeat these objects on each horizontal page.

12.5.11.1 To repeat objects on horizontal pages

Procedure

1. Right-click the report object you want to repeat.
2. On the shortcut menu, click *Format Field*, or the appropriate format option.

Note

Formatting options are not available if the object you select is connected to the BusinessObjects Enterprise Repository. For information about modifying an object in the repository, see [Modifying objects in the repository \[page 129\]](#).

3. On the *Common* tab of the Format Editor, select the *Repeat on Horizontal Pages* check box.
For a line or a box, the Repeat on Horizontal Pages option appears on the Line or Box tab.
4. Click *OK* to save your changes.

Results

Now, if a cross-tab or OLAP grid makes your report expand horizontally, the object you formatted is repeated on each horizontal page.

- The repeated copies of a report object cannot be moved; you must move the original object to move the copies.
- You can format any copy of a repeated report object, but changes you make are applied to all copies.
- Report objects in a section with the Relative Positions option selected in the Section Expert cannot be repeated.

12.5.11.2 Numbering horizontal pages

A Special Field called Horizontal Page Number lets you number horizontal pages.

12.5.11.2.1 To number horizontal pages

Procedure

1. In the Field Explorer, scroll down to *Special Fields* and expand the node by clicking it.
2. Choose *Horizontal Page Number* from the list and insert it in your report.

Results

Note

The Repeat on Horizontal Pages option is automatically turned on when you insert the Horizontal Page Number field this way. If you cut and paste a Horizontal Page Number field, or if you insert such a field into another field or text object, the Repeat on Horizontal Pages option is not automatically turned on.

12.5.12 Using white space between rows

The height of a section in relation to the objects within it affects the amount of white space that appears between rows on the report.

The free-form Design tab lets you add and delete white space in two ways:

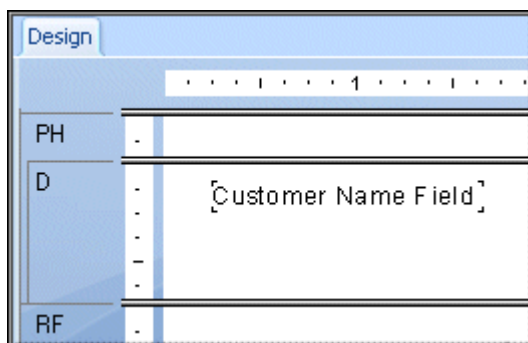
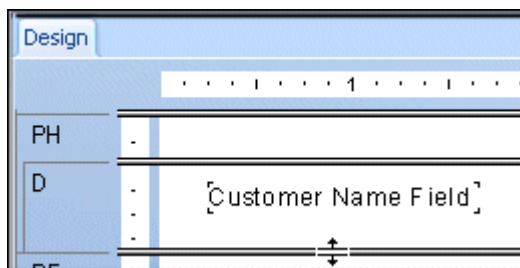
- Using the Resizing cursor to resize the area on the Design tab.
- Changing the option in the Section Expert.

Note

You can also right-click the shaded area to the left of the section and select Fit Section from the shortcut menu. The program automatically resizes the section so that the bottom boundary is even with the baseline of the bottom object in the section.

12.5.12.1 Adding white space by resizing

To add extra white space between rows in the report, move the pointer over the lower section boundary line. The pointer changes to a Resizing cursor. When the Resizing cursor appears, drag the section boundary downward to add extra white space.

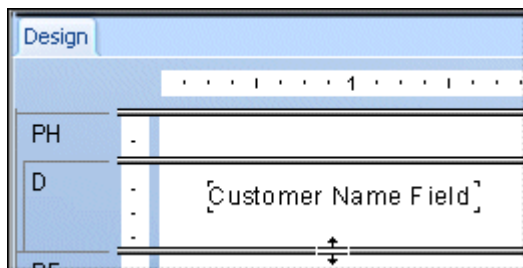
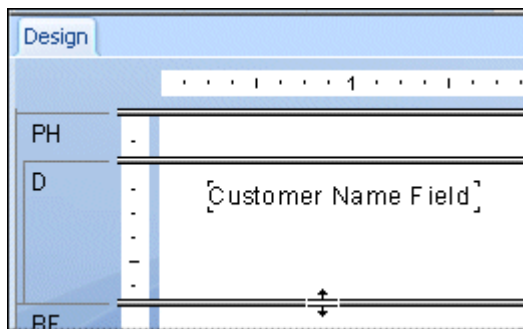


Note

White space can also be added to a section by right-clicking the shaded area to the left of the section and selecting Insert Line from the shortcut menu. The program resizes the section automatically, adding the amount of space necessary to hold a line of typical database fields.

12.5.12.2 Deleting white space by resizing

To delete unnecessary white space within a section, move the pointer over the lower section boundary line. The pointer changes to a Resizing cursor. When the Resizing cursor appears, drag the section boundary upward to remove extra white space.



12.5.12.3 Deleting white space by suppressing a section

If an entire section is blank, you can eliminate the unnecessary white space that the [Page Footer](#) would occupy by suppressing the section in the [Section Expert](#).

If you are not putting anything into the [Page Footer](#) section of the report, select the [Clamp Page Footer](#) option in the [Section Expert](#) to eliminate white space at the bottom of your report pages.

12.5.12.3.1 To delete white space by suppressing a section

Procedure

1. On the [Report](#) menu, click [Section Expert](#).

The [Section Expert](#) appears.

→ Tip



Another way to do this is to click the [Section Expert](#) button on the [Expert Tools](#) toolbar.

2. In the [Sections](#) area, click the section you want to suppress.
3. On the [Common](#) tab, select the [Suppress \(No Drill-Down\)](#) check box.
4. Click [OK](#) to return to your report.

The blank section will no longer be printed.

12.5.12.3.2 To delete white space by clamping the Page Footer

Procedure

1. On the [Report](#) menu, click [Section Expert](#).

The [Section Expert](#) appears.

→ Tip



Another way to do this is to click the [Section Expert](#) button on the [Expert Tools](#) toolbar.

2. In the [Sections](#) area, click [Page Footer](#).
3. Click [Clamp Page Footer](#).
4. Click [OK](#) to return to your report.

Results

The report eliminates white space by placing the [Page Footer](#) directly beneath the last visible section.

12.6 Working with conditional formatting

Conditional formatting is formatting that applies only under certain conditions. For example, in a report you may only want:

- Customer balances printed in red if they are past due.
- The dates to appear in Day, Month, Year format if the customer is Canadian.
- Background color to appear on every other line.

Crystal Reports makes it easy to apply conditional formatting in these and hundreds of other situations.

With absolute formatting, you follow the "select, then apply" procedure. For conditional formatting, you follow the same general procedure, but you go a step further and set up conditions that determine whether or not the formatting will be applied. You specify these conditions using simple formulas.

For more information on creating formulas using Crystal or Basic syntax, see the online help.

When a conditional formatting formula is set up, the formula overrides any fixed settings you have made in the Format Editor. For example, if you select the Suppress option, then set up a conditional formula for the Suppress option, the property will still apply only if the condition in the formula is met.

Crystal Reports enables you to set both on and off properties and set attribute properties conditionally. However, each of these requires a different kind of formula.

12.6.1 Conditional on or off properties

A conditional on or off property tests to see if a condition has been met. It is on if the condition is met, off if the condition is not met. There is no middle ground. Use Boolean formulas for this kind of formatting.

Crystal syntax example

```
condition
```

Basic syntax example

```
formula = condition
```

The program tests each value to see if it meets the condition and it returns a "yes" or "no" answer. It then applies the property to every value that returns a "yes" answer.

12.6.2 Conditional attribute properties

A conditional attribute property tests to see which of two or more conditions is met. The program then applies the formatting appropriate to the condition. For example, assume that you want values under quota printed in red and all other values printed in black. The program tests to see whether the value is under quota or not. If it is under quota, then it applies the red attribute; if it is not, then it applies the black attribute.

Use an If-Then-Else formula for this kind of conditional formatting.

Crystal syntax example

```
If Condition A Then
    crRed
Else
    crBlack
```

Basic syntax example

```
If Condition A Then
    formula = crRed
Else
    formula = crBlack
End If
```

When conditional attribute properties are set up, Crystal Reports loads a selection of attributes into the Functions list in the Formula Workshop. Double-click any of these attributes to add them to a formula. For example, if you are setting horizontal alignment conditionally, the Functions list contains attributes such as DefaultHorAligned, LeftAligned, and Justified. If you are setting borders conditionally, the Functions list contains attributes such as NoLine, SingleLine, and DashedLine.

Note

Always include the Else keyword in conditional formulas; otherwise, values that don't meet the If condition may not retain their original format. To retain the original format of values that don't meet your If condition, use the DefaultAttribute function.

Crystal syntax example

```
If Condition A Then
    crRed
Else
    DefaultAttribute
```

Basic syntax example

```
If Condition A Then
    formula = crRed
Else
    formula = DefaultAttribute
End If
```

You can take this kind of property one step further. You can specify a list of conditions and a property for each; you are not limited to two conditions. For example, if you have a number field on your report that contains sales

figures from countries around the world, you can specify the number attribute(s) that you want to apply to each country. In this case, your conditions would specify that if it is from Country A, the program should apply the Country A attribute; if it is from Country B, the Country B attribute; if it is from Country C, the Country C attribute, and so on.

With more than two alternatives, use this kind of formula:

Crystal syntax example

```
If Condition A Then
    crRed
Else If Condition B Then
    crBlack
Else If Condition C Then
    crGreen
Else
    crBlue
```

Basic syntax example

```
If Condition A Then
    formula = crRed
ElseIf Condition B Then
    formula = crBlack
ElseIf Condition C Then
    formula = crGreen
Else
    formula = crBlue
End If
```

Use a multi-condition If-Then-Else formula for this kind of conditional formatting.

12.6.2.1 To change fonts conditionally

Procedure

1. Right-click the field you want to format to bring up the shortcut menu.
2. On the shortcut menu, click *Format Field*.
The Format Editor dialog box appears.
3. Click the *Font* tab.
4. To change any of the font options, click the appropriate *Formula* button, located on the right side of the dialog box.
5. In the Formula Workshop, you can specify that conditional fonts will change only when certain conditions are met.

6. Click [Save and close](#).

ⓘ Note

If there is an error in the formula, a message box appears, asking if you want to save anyway. If you click No, a second message box is displayed, detailing the error.

ⓘ Note

If there is no error in the formula, you are returned to the Format Editor. Note that the Formula button has changed. This indicates that a formula has been entered for that property.

7. Click [OK](#) to return to your report.

12.6.3 Changing margins conditionally

Page margins can be controlled conditionally by use of formula.

Margins based on page number

The following formula checks whether a page number is even or odd and sets the margins accordingly: if the page is an even number, the margin is set to one inch; if the page is an odd number, the margin is set to two inches.

```
If Remainder(pagenum,2) = 0 then 1440 else 2880
```

Margins based on page orientation

The following formula checks for page orientation and sets the margins accordingly: if the orientation is portrait, the margin is set to one inch; if the orientation is landscape, the margin is set to two inches.

```
If CurrentPageOrientation = crPortrait then 1440 else 2880
```

ⓘ Note

Margin position is measured in twips; there are 1440 twips in an inch.

12.6.3.1 To change margins conditionally

Procedure

1. On the *File* menu, click *Page Setup*.

The *Page Setup* dialog box appears.

2.  Click the *Conditional Formula* button next to the margin you wish to change.
3. In the *Formula Workshop*, enter your conditional margin formula text.
4. Click *Save and close* to return to your report.
5. Click *OK* to apply your new settings.

12.6.4 Changing X position conditionally

You can change the X position (that is, the horizontal position from the left margin) for an object based on a condition. You might do this when you want objects to appear in different columns when their values meet a certain condition; for example, orders that shipped on time appear in the first column, while orders that shipped late appear in a second column.

Note

You cannot conditionally change the X position of line or box objects.

12.6.4.1 To conditionally change the X position of an object

Procedure

1. Right-click the field that you want to move conditionally, and select *Size and Position*.
2.  Click the *Conditional Formula* button next to the X position value field.
3. In the *Formula Workshop*, enter your conditional X position formula text.

For example, to move orders that were shipped late to a second column, enter formula text such as this:

```
If (Orders.Ship Date) < CDateTime (2004, 01, 01, 02, 10, 11) then 4320
```

Note

The number 4320 represents the new position that you want to define as the second column. The position is measured in twips; there are 1440 twips in an inch.

4. Click *Save and close* to return to your report.

5. Click [OK](#) to save your position setting.

Results

Crystal Reports moves objects that meet your condition to a new position, but leaves those objects that don't meet the condition where you originally placed them.

12.6.5 Creating footers after the first page

You may choose to print a page footer on all pages except the first page. You can do this by formatting the Page Footer section conditionally, using an on or off property.

12.6.5.1 To create footers after the first page

Procedure

1. Place the field you want displayed as a page footer in the Page Footer section of the report.
2. On the [Report](#) menu, click [Section Expert](#).

→ Tip



Another way to do this is to click the Section Expert button on the Expert Tools toolbar.

The Section Expert dialog box appears.

3. In the Sections area, click [Page Footer](#).
4. To open the Formula Workshop, click the [Formula](#) button, located to the right of the [Suppress \(No Drill-Down\)](#) check box.
5. Enter the following formula in the Format Formula Editor:

Crystal syntax example:

```
PageNumber = 1
```

Basic syntax example:

```
formula = PageNumber = 1
```

This formula suppresses the page footer on the first page, but not on any of the other pages.

6. Click [Save and close](#).

Note

If there is an error in the formula, a message box appears, asking if you want to save anyway. If you click No, a second message box is displayed, detailing the error.

Note

If there is no error in the formula, you are returned to the Section Expert. Note that the Formula button has changed. This indicates that a formula has been entered for that property.



7. On the Standard toolbar, click [Preview](#) to preview the report and ensure that the page footer appears on all pages but the first.

Results

Note

If you have a multi-line page footer and have inserted the lines into separate Page Footer sections, you will need to suppress each section conditionally, using the formula above.

Note

To create a page header that appears on all pages but the first, place the header information in the Page Header section and then suppress that section conditionally, using the same formula that was used for suppressing the Page Footer section.

12.6.6 Using the Highlighting Expert

The Highlighting Expert enables you to apply conditional formatting to all types of report fields (Number, Currency, String, Boolean, Date, Time, and Date and Time fields). With the expert, you format the selected field either by specifying a condition based on that field's value, or by specifying a condition based on the value of a different report field. In other words, the expert enables you create the following formula: If the value of field X meets condition A, then apply the specified formatting to the field selected on the report.

When used for conditional formatting, the Highlighting Expert allows you to:

- Modify several attributes at once, without writing a formula.
- Highlight all field types used in the report.
- Format font style, background color, font color, and border style.
- Format a field based on its own values or the values of another field.
- Highlight a cross-tab or OLAP cell based on row and column heading values.
- Enter values using your locale-specific number format (such as 1,224.23 for North American users).
- Enter dates numerically or textually (January 12, 2001, or Jan 12, 2001).
- Undo highlighting quickly.

When you need to conditionally format report fields, the Highlighting Expert is quicker and easier to use than the Formula Workshop. The Highlighting Expert is most commonly used to highlight field values that are in some way distinguished from other values in the report. You might, for example, highlight your key customers by printing the {Customer.Last Year's Sales} field with a red background whenever the sales value exceeds \$50,000. Alternatively, to draw attention to outstanding orders, you might bold the {Product.Product Name} field whenever the {Orders.Shipped} value is False.

The Highlighting Expert, however, is not as flexible as the Formula Workshop. To use the formatting capabilities of Crystal Reports to their full potential, create your own conditional formatting formulas with the Formula Workshop (accessible through the Format Editor dialog box). For complete details, see [Using Formulas \[page 687\]](#).

12.6.6.1 Conditionally formatting fields using the Highlighting Expert

The Highlighting Expert is a quick alternative to the Formula Workshop; it allows you to conditionally format any of your report fields. The Highlighting Expert essentially enables you create the following formula: If the value of field X meets condition A, then apply the specified formatting to the field selected on the report.

The dialog box is divided into two areas: the Item list area displays the formula; the Item editor area allows you to specify the formula. The Item editor area includes a Sample field that displays the formatting specifications being applied.

12.6.6.1.1 To conditionally format fields using the Highlighting Expert

Procedure

1. To open the Highlighting Expert, right-click the field you want to format and select [Highlighting Expert](#) from the shortcut menu.

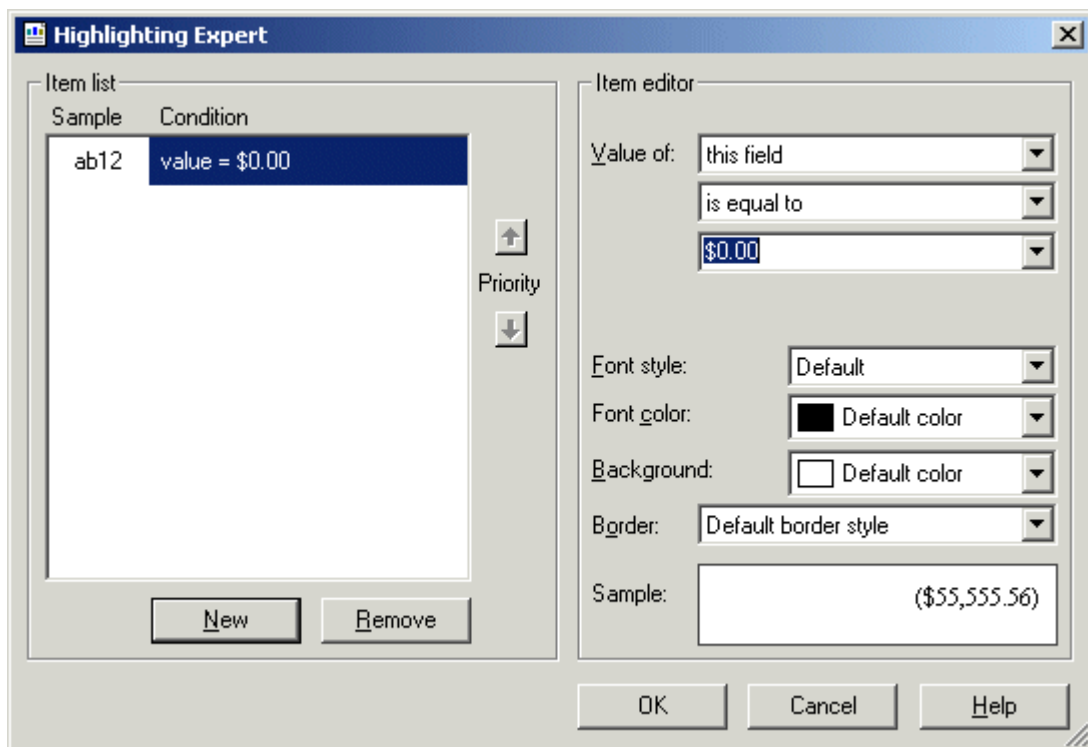
→ Tip



You can also start the expert by clicking the Highlighting button on the Expert Tools toolbar, or by clicking Highlighting Expert on the Format menu.

When opened, the expert is set to format the field that is currently selected on the report.

2. In the Highlighting Expert, click [New](#) to create a new conditional formula with default settings.



3. In the *Item editor* area, click the *Value of* list and select the field that you want to base your condition on.

The field chosen here is the field upon which your condition is based; this field need not be the field that is being formatted. To create a condition based on the values of the field that is being formatted, select "this field" from the list. To base your condition on a different report field, select it from the list of available fields.

Note

The "Value of" list displays only those fields that you have added to the report.

4. Select a comparison from the second list (is equal to, is less than, and so on).

This comparative statement works as the operator in the conditional formula created by the expert.

5. Complete the condition by entering the desired value in the box.

Note

If the field selected in the "Value of" list is not numeric, the text box turns into a list of available values, from which you must select one.

6. In the *Font style*, *Font color*, *Background*, and *Border* lists, specify the formatting changes that you want to apply to the selected field when your condition is met.
7. Repeat steps 3 and 4 if you want to apply multiple highlighting conditions to the selected field.

Note

You can use the expert's Remove button to delete highlighting formulas from the list.

8. Use the *Priority* arrows to specify the order in which you want Crystal Reports to apply your conditions. For details, see [Setting highlighting priorities \[page 297\]](#).

9. Click [OK](#) to return to your report.

12.6.6.2 Setting highlighting priorities

The Priority buttons in the Item list area of the Highlighting Expert allow you to set priorities for your formulas. This is useful when you have two or more formulas that could offer conflicting results in some situations.

For example, suppose that you highlight the Unit Price field on the report. You assign to this field a highlighting formula that shows a yellow background when a unit price is greater than \$100. Then, on this same report, you create another highlighting formula that shows a red background when a unit price is greater than \$200. Considering that 100 is a subset of 200, you could have Unit Price fields with yellow backgrounds when, in fact, those fields should have red backgrounds. In other words, a unit price of \$300 could receive either a red or a yellow background, depending on which formula has been assigned priority.

12.6.6.2.1 To set priorities for highlighting formulas

Procedure

1. On the [Format](#) menu, click [Highlighting Expert](#).
2. In the Item list area, select one of the conditional highlighting formulas that you have created.
3. Click the [Priority](#) arrows to move the selected formula to a position above or below the other formula(s).

Note

A formula has priority over another formula when it is higher in the Items list area.

4. Click [OK](#).
5. Click the [Preview](#) tab or refresh the report to see the highlighting changes.

12.6.7 Undo/Redo activities

Crystal Reports includes multiple levels of undo. With multiple levels of undo, you can undo any number of changes to an object, in reverse order, until you have your report in the condition you want it.

The program also has a redo feature that reverses an undo. If you move an object, for example, and do not like its new position, you can click Undo to move it back to its original position. If you then change your mind, you can click Redo to restore the latest change.

The Undo and Redo buttons have lists that allow you to undo or redo a number of changes at one time.

-  To undo an action, click [Undo](#) on the Standard toolbar.
The first time the button is clicked, it reverses the most recent change made to the report. Each additional time the button is clicked, it reverses the next most recent change.

To undo several actions at once, click the arrow button to display the list of actions. Select the series of actions you wish to undo.

-  To redo a change after you have undone it, click [Redo](#) on the Standard toolbar.
The program disables the Undo button and the Undo/Redo commands whenever there is nothing to undo/redo or when you have made a change that cannot be reversed.
To redo several actions at once, click the arrow button to display the list of actions. Select the series of actions you wish to redo.

Note

You can only undo or redo actions in order from the most recent backward. You cannot undo an action without undoing more recent actions.

12.7 Using the Format Painter

Use the Format Painter to copy absolute or conditional formatting properties from one report object to one or more target objects. The Format Painter button is activated on the Standard toolbar when you select a source object in your report. If you apply formatting to a target field that is not the same as your source field, only the common properties are applied. For example, if your source field is a Boolean field and your target field is a currency field, only the common font and border properties are changed; Boolean properties are not applied and currency properties are not set.

Note

The Format Painter does not copy hyperlink information to a target object or field.

Note

The Format Painter does not copy formatting that is applied through use of the Highlighting Expert.

Note

The Format Painter does not copy formatting from text/template objects to database fields.

Note

The Format Painter can use any object (including Repository objects, read-only objects, and objects contained in read-only sections) as a source of formatting (however, formatting cannot be applied to these objects).

Note

When using a "Date and Time" field as your source, a target field's date or time properties are changed; the reverse is also true (that is, a Date field or a Time field used as your source also affects the date and time properties of a "Date and Time" field).

The following is a list of source report objects and field and the applicable targets you can use with the Format Painter.

Source object or field	Target object or field
Report field (not in a cross-tab)	Report field (not in a cross-tab)
Report field in a cross-tab	Report field in a cross-tab
Template field	Template field
Text object (not in a cross-tab)	Text object (not in a cross-tab)
Text object in a cross-tab	Text object in a cross-tab
Object in an OLAP grid header	Object in an OLAP grid header
Line object	Line object
Box object	Box object
OLE or BLOB field	OLE or BLOB field
Subreport	Subreport
Chart, map, OLAP grid, or cross-tab	Chart, map, OLAP grid, or cross-tab

12.7.1 To copy and apply formatting

Procedure

1.  Select a source object or field in your report and click *Format Painter*.

→ Tip

You can also select Format Painter from the shortcut menu.

ⓘ Note

The Format Painter button is not available until you select an object or field.

ⓘ Note

Click the button a second time, or press ESC, to exit the Format Painter.

2. Click the target object or field you want to apply formatting to.

When you move your mouse over your report, the cursor changes to a Stop cursor if the object or field cannot be used as a target.

ⓘ Note

You cannot apply formatting to read-only objects or fields.

Note

Double-click the Format Painter button, or hold the ALT key down if you want to apply formatting to more than one object or field.

12.8 Working with barcodes

Barcodes allow companies to use computers to track products and sales. Specially designed scanners read the barcode and the computer translates it into a human-readable form.

The Crystal Reports install includes the formula and fonts necessary for supporting Code39 barcodes. You can add barcodes to your reports to be used for activities such as managing your inventory or creating package labels.

Code39 is a basic barcode format supported by most scanners on the market. For support of additional barcode types, such as UPC, please contact your preferred barcode vendor.

12.8.1 Adding a barcode

You can apply a barcode to an existing field or add a new field to the report specifically for the barcode.

You will need to consider the following information before adding a barcode:

- When a field is converted to a barcode, the default size changes to accommodate 16 characters at 24 pts. This may cause the new data to overlap existing fields if there is not enough space between fields.
- Barcodes can be applied to number and string fields only. The barcode cannot be applied to Currency, DateTime, or other field types.
- To undo changing a field to a barcode, you must use the [Undo](#) command right after you use [Change to Barcode](#).

If you perform any other actions after you change the field to a barcode, you must either delete the field item or use the Format Field option to reformat the appearance.

Related Information

[Changing the appearance of a barcode \[page 301\]](#)

[Removing a barcode \[page 302\]](#)

12.8.1.1 To add a barcode

Procedure

1. In the report, right-click the field to use for the barcode.
If the desired field is not in the report, use the Field Explorer to add the field to your report.
2. On the shortcut menu, select [Change to Barcode](#).
The Select Barcode Type dialog box appears.
3. Select the barcode type from the list.
4. Click [OK](#).

Results

The field is changed to display barcodes instead of characters.

12.8.2 Changing the appearance of a barcode

You can use the Format Field option to change the appearance of a barcode.

There are many reasons to change the appearance of a barcode including the following:

- To change the size so that the barcode is not too large or too small for a scanner to read.
- To display the barcode in a different color.

Related Information

[Adding a barcode \[page 300\]](#)

[Removing a barcode \[page 302\]](#)

12.8.2.1 To change the appearance of a barcode

Procedure

1. Right-click the barcode field that you would like to reformat.
2. From the shortcut menu, select [Format Field](#).
The Format Editor appears.
3. Set your desired border, font size and color, or other options.

4. Click [OK](#).

12.8.3 Removing a barcode

If you applied the barcode to the wrong field, or you decide that you do not want to have a barcode, you can use Format Field to change the field back to a regular font.

Related Information

[Adding a barcode \[page 300\]](#)

[Changing the appearance of a barcode \[page 301\]](#)

12.8.3.1 To change a barcode back to a regular font

Procedure

1. Right-click the barcode field that you would like to reformat.
2. From the shortcut menu, select [Format Field](#).
The Format Editor appears.
3. Select the [Common](#) tab.
4. In the [Display String](#) area, click the red formula editor button.
The Formula Editor appears.
5. Delete the string in the Formula text window and then click [Save and Close](#).
6. Select the [Font](#) tab
7. Change the font and size to the desired size, and then click [OK](#).
8. On the report, resize the field to an appropriate length.

13 Charting

Crystal Reports enables you to present summarized data in colorful, easy-to-read charts. This section demonstrates how to create charts and how to use them in reports to make report data more meaningful and easier to understand. You can choose from a number of chart layouts and types, as well as drill down to see the details behind the graphical summaries and format chart objects.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com>

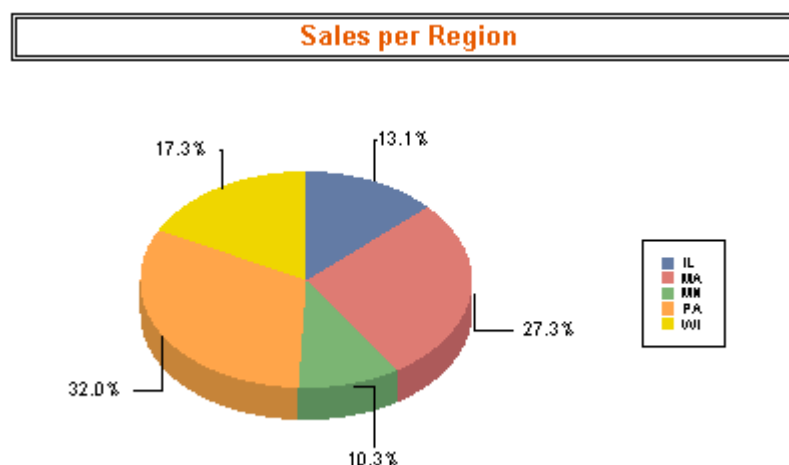
13.1 Charting concepts

Crystal Reports enables you to include sophisticated, colorful charts in your reports. You can use charts any time you want to improve the usefulness of a report.

13.1.1 Charting overview

Crystal Reports enables you to include sophisticated, colorful charts in your reports. You can use charts any time you want to improve the usefulness of a report.

For example, if you have a sales report grouped by Region with a subtotal of Last Year's Sales for each region, you can quickly create a chart that will display Sales per Region.



You can chart on the following:

- Summary and subtotal fields.
- Detail, formula, and Running Total fields.
- Cross-Tab summaries.
- OLAP data.

You will typically chart on summary and subtotal information at the group level. However, depending on the type of data you are working with, you can create an Advanced, Cross-Tab, or OLAP grid chart for your report.

13.1.2 Chart layouts

The Chart Expert provides four layouts that correspond to certain sets of data.

You can create charts with any of the following layouts, and depending on the data you are using, you can change the chart from one layout to another.

Advanced

Use the Advanced layout when you have multiple chart values or when you do not have any group or summary fields in the report.

The Advanced chart layout supports one or two condition fields: with these condition fields, you can create a 2-D, 3-D, or pie chart. Other specific functions with the Advanced layout include:

- Values can be grouped in ascending, descending, or specified order, as well as by Top N or Sort totals.
- Values can be plotted for each record.
- Values can be plotted as a grand total for all records.
- Charts can be based on formula and Running Total fields.

Group

The Group layout is a simplified layout in which you show a summary on change of field for topics such as Country.

Note

In order to create a chart using the Group layout, you must have at least one group and at least one summary field for that group.

Cross-Tab

Use the Cross-Tab layout to chart on a Cross-Tab object. A Cross-Tab chart uses the fields in the cross-tab for its condition and summary fields.

OLAP

Use the OLAP layout to chart on an OLAP grid. An OLAP chart uses the fields in the OLAP grid for its condition and summary fields.

Note

Your report must include an OLAP grid before you can create an OLAP chart.

13.1.3 Chart types

Different sets of data are particularly suited to a certain chart type. The following is an overview of the main chart types and their most common uses.

Bar

Most bar charts (also known as a column chart) display or compare several sets of data. Two useful bar charts are the Side-by-Side bar chart and the Stacked bar chart.

- Side-by-Side bar chart
A Side-by-Side bar chart displays data as a series of vertical bars. This type of chart is best suited for showing data for several sets over a period of time (for example, last year's sales figures for AZ, CA, OR, and WA).
- Stacked bar chart
A Stacked bar chart displays data as a series of vertical bars. This type of chart is best suited for representing three series of data, each series represented by a color stacked in a single bar (for example, sales for 1997, 1998, and 1999).

Line

A line chart displays data as a series of points connected by a line. This type of chart is best suited for showing data for a large number of groups (for example, total sales over the past several years).

Area

An area chart displays data as areas filled with color or patterns. This type of chart is best suited for showing data for a limited number of groups (for example, percentage of total sales for AZ, CA, OR, and WA).

Pie

A pie chart displays data as a pie, split and filled with color or patterns. Pie charts are typically used for one group of data (for example, the percentage of sales for the entire inventory); however, you have the option to choose multiple pie charts for multiple groups of data.

Doughnut

A doughnut chart is similar to a pie chart, displaying data as sections of a circle or doughnut. If, for example, you charted sales by region on a particular report, you would see the total number of sales (the figure) in the center of the doughnut and the regions as colored sections of the doughnut. As with the pie chart, you have the option to choose multiple doughnut charts for multiple groups of data.

3-D Riser

A 3-D Riser chart displays data in a series of 3-dimensional objects, lined up side-by-side, in a 3-dimensional plane. The 3-D Riser chart shows the extremes in your report data. For example, the differences between sales by customer by country are visually dynamic when presented in this chart.

3-D Surface

3-D Surface charts present a topographic view of multiple sets of data. If, for example, you need a chart to show the number of sales by customer by country, in a visually dynamic and relational format, you might consider using the 3-D Surface chart.

XY Scatter

An XY Scatter chart is a collective of plotted points that represent specific data in a pool of information. The XY Scatter chart allows the user to consider a larger scope of data for the purpose of determining trends. For example, if you input customer information, including sales, products, countries, months, and years, you would have a collective of plotted points that represents the pool of customer information. Viewing all of this data on

an XY Scatter chart would allow you to speculate as to why certain products were selling better than others or why certain regions were purchasing more than others.

Radar

A radar chart positions group data, such as countries or customers, at the perimeter of the radar. The radar chart then places numeric values, increasing in value, from the center of the radar to the perimeter. In this way, the user can determine, at a glance, how specific group data relates to the whole of the group data.

Bubble

A bubble chart (an extension of the XY Scatter chart type) displays data as a series of bubbles, where the size of the bubble is proportional to the amount of data. A bubble chart would be very effective with the number of products sold in a certain region; the larger the bubble, the greater the number of products sold in that region.

Stock

A stock chart presents high and low values for data. It is useful for monitoring financial or sales activities.

Note

Crystal Reports offers two possible formats for stock charts: High-Low and High-Low-Open-Close. Each of these types requires a series of values in the order specified in its name.

Numeric Axis

A numeric axis chart is a bar, line, or area chart that uses a numeric field or a date/time field as its "On change of" field. Numeric axis charts provide a way of scaling your X-axis values, thus creating a true numeric X-axis or a true date/time X-axis.

Gauge

A gauge chart presents values graphically as points on a gauge. Gauge charts, like pie charts, are typically used for one group of data (for example, the percentage of sales for the entire inventory).

Gantt

A Gantt chart is a horizontal bar chart often used to provide a graphical illustration of a schedule. The horizontal axis shows a time span, while the vertical axis shows a series of tasks or events. Horizontal bars on the chart represent event sequences and time spans for each item on the vertical axis. You should use only date fields when creating a Gantt chart. The field you choose for the data axis should be set to "For Each Record," and the start and end-date fields should be added to the "Show value(s)" area of the Chart Expert's Data tab.

Funnel

Funnel charts are often used to represent stages in a sales process. For example, the amount of potential revenue shown for each stage. This type of chart can also be useful in identifying potential problem areas in an organization's sales processes. A funnel chart is similar to a stacked bar chart in that it represents 100% of the summary values for the groups included in the chart.

Histogram

A histogram is a type of bar chart used to depict how measurements vary from the mean value. It can help to identify the cause of problems in a process by the shape of the distribution as well as the width (deviation) of the distribution. In a histogram, the frequency is represented by the area of a bar rather than the height of the bar.

13.1.4 Where to place a chart

The placement of a chart determines which data is displayed and where it is printed. For example, a chart placed in the Report Header section includes the data for the entire report, while a chart that is placed in a Group Header or in a Group Footer section displays only the group specific data.

📘 Note

If your report contains subreports, you can place charts in those subreports as well. See [Subreports \[page 870\]](#).

📘 Note

By default, when you insert a chart or chart object frame, it is placed in the Report Header.

13.1.5 Drill-down with charts

Not only is charting a means of presenting data—it is also an analysis tool. Move your cursor over a section of the group chart on the Preview tab, so that the pointer becomes a Drill-down cursor, then double-click to view the underlying details for that section of the chart.

13.1.6 Drill-down with legends

If the chart consists of one or more group fields, you can use the chart legend to drill down on individual groups. Double-click the drill-down cursor on the markers and text in the legend to view the details about that section of the chart.

13.2 Creating charts

When you insert a chart in a report, you may see one of the following options:

- A chart object frame in the Report Header.
Once you have placed the chart frame, the Chart Expert dialog box appears. For more information on where to place a chart in the report, see [Where to place a chart \[page 308\]](#).
- A chart automatically inserted in the Report Header.
In some situations, such as when there is at least one group and one summarized field in your report, a chart is automatically added to the Report Header, and the Chart Expert dialog box does not appear. This usually happens when you chart on Cross-Tab summaries or when you chart on an OLAP cube, but it may also happen when you chart on summary or subtotal fields.

13.2.1 Charting on details or formula fields (Advanced layout)

The Advanced layout allows you to create a chart based on specific values. Since charts are a good way to display summarized information, they are often based on a summary field in your report. With an Advanced layout, you can create a chart without the need for a summary field by using values that appear in the Details section of your report.

To create a chart based on the Advanced layout, you must specify two things:

- Conditions (there can be two).
- Values (there can be multiple values).

Condition

The condition is used to indicate when to plot the point. For example, a chart showing last year's sales for your customers uses the Customer Name field as the condition. Each time the condition changes (the customer name changes), a point is plotted.

You also have the option of plotting a point for each record, or plotting one point for all records.

Value

The Advanced chart uses the value to indicate what information is plotted as the points on your chart. For example, to create a chart showing last year's sales for your customers, the Last Year's Sales field would be the value.

13.2.2 Charting on summary or subtotal fields (Group layout)

Many of the charts you create are based on summary or subtotals within your report. In order to create these charts, you must have a summary or subtotal already inserted into your report in a group header or footer. For more information on inserting summaries and subtotals, see [Summarizing grouped data \[page 192\]](#) and [Subtotaling \[page 197\]](#).

13.2.2.1 To chart on a summary or subtotal field

Procedure

1. On the [Insert](#) menu, click [Chart](#).

An object frame appears in the Report Header area.

→ Tip



Another way to create a chart is to click the Insert Chart button on the Insert Tools toolbar.

2. Drag the frame to the desired position in the Report Header, Group Header, or Group Footer.

The Chart Expert dialog box appears.

ⓘ Note

You may see a default chart in the Report Header section rather than the Chart Expert dialog box. To select a different chart type, right-click the default chart, and then click Chart Expert. The Chart Expert dialog box appears.

3. On the [Type](#) tab, in the [Chart type](#) list, select a chart type.
Click the chart subtype that best illustrates your data. See [Chart types \[page 305\]](#).
4. Click the [Data](#) tab.
5. In the Layout area, click [Group](#), if it is not already selected.
6. In the Data area, in the [On change of](#) list, click the group field you want to base your chart on; then, in the [Show](#) list, click the summary field you want to display on your chart.
7. If the [Axes](#) and [Options](#) tabs appear, you can customize some of the chart's properties, such as the scaling of the axes, the legend, and the data points.
8. Click the [Text](#) tab.
9. Accept the default title information or add new titles to your chart.
10. Click [OK](#).

Results

Note

When your chart is inserted, it may cover a portion of the report. Move and resize the chart so that it fits properly within the report.

13.2.3 Charting on Cross-Tab summaries (Cross-Tab layout)

Crystal Reports allows you to include a chart based on summary values in your Cross-Tab report. For example, with a Cross-Tab that shows the amount of a certain product sold in each region of the United States, you may want to include a chart showing the percentage of total sales provided by each region for that product.

To create a Cross-Tab chart, you must first have a Cross-Tab in your report. For more information, see [Cross-Tab Objects \[page 346\]](#).

13.2.3.1 To chart on a Cross-Tab summary

Procedure

1. Select the Cross-Tab on which you want to chart.
2. On the [Insert](#) menu, click [Chart](#).

An object frame appears in the Report Header area.

→ Tip



Another way to create a chart is to click the Insert Chart button on the Insert Tools toolbar.

3. Drag the frame to the desired position in the Report Header.

The Chart Expert dialog box appears.

Note

You may see a default chart in the Report Header section rather than the Chart Expert dialog box. To select a different chart type, right-click the default chart, and then click Chart Expert. The Chart Expert dialog box appears.

4. On the **Type** tab, in the **Chart type** list, select a chart type.
Click the chart subtype that best illustrates your data. See [Chart types \[page 305\]](#).
5. Click the **Data** tab.
6. In the Layout area, click **Cross-Tab**, if it is not already selected.
7. In the Data area, in the **On change of** list, click the group field you want to base your chart on.
8. If necessary, in the **Subdivided by** list, click a secondary row or column you want to base your chart on.
9. In the **Show** list, click the summary field you want to display on your chart.
10. If the **Axes** and **Options** tabs appear, you can customize some of the chart's properties, such as the scaling of the axes, the legend, and the data points.
11. Click the **Text** tab.
12. Accept the default title information or add new titles to your chart.
13. Click **OK**.

Results

Note

When your chart is inserted, it may cover a portion of the report. Move and resize the chart so that it fits properly within the report.

13.2.4 Charting on an OLAP cube (OLAP layout)

The OLAP layout lets you chart on an OLAP grid. In order to create an OLAP chart, you must first have an OLAP grid in your report. For more information, see [Creating an OLAP report \[page 395\]](#).

13.2.4.1 To chart on an OLAP cube

Procedure

1. Select the OLAP grid on which you want to chart.

2. On the *Insert* menu, click *Chart*.

An object frame appears in the Report Header area.

→ Tip



Another way to create a chart is to click the Insert Chart button on the Insert Tools toolbar.

3. Drag the frame to the desired position in the Report Header.

The Chart Expert dialog box appears.

ⓘ Note

You may see a default chart in the Report Header section rather than the Chart Expert dialog box. To select a different chart type, right-click the default chart, and then click Chart Expert. The Chart Expert dialog box appears.

4. On the *Type* tab, in the *Chart type* list, select a chart type. Then click the chart subtype that best illustrates your data. See [Chart types \[page 305\]](#).
5. Click the *Data* tab.
6. In the Layout area, click the *OLAP* button, if it is not already selected.
7. In the Data area, in the *On change of* list, click the field you want to base your chart on.
8. If necessary, in the *Subdivided by* list, click a secondary row or column you want to base your chart on.

ⓘ Note

Be sure that the chart type selected in Step 3 supports a secondary charting field.

9. If the *Axes* and *Options* tabs appear, you can customize some of the chart's properties, such as the scaling of the axes, the legend, and the data points.
10. Click the *Text* tab.
Accept the default title information or add new titles to your chart.
11. Click *OK*.

Results

ⓘ Note

When your chart is inserted, it may cover a portion of the report. Move and resize the chart so that it fits properly within the report.

13.3 Working with charts

Once you have created a chart, you may want to add a new title, headings, or a legend, change fonts, or even change the type of chart. Crystal Reports provides many options for working with your existing charts.

13.3.1 Editing charts using the Chart Expert

Editing charts with the Chart Expert allows you to return to the expert in which you designed your chart. You can modify many of your original choices, such as the type of chart to display, the data on which the chart is based, and so on.

13.3.1.1 To edit a chart using the Chart Expert

Procedure

1. Right-click the chart to bring up the shortcut menu.
2. On the shortcut menu, click [Chart Expert](#).
3. In the Chart Expert dialog box, make the desired changes.
4. Click [OK](#).

13.3.2 Editing charts using the Chart Options menu items

Some of the editing options available in the Chart Expert are also available directly from the Chart Options menu. This menu, which is available when you right-click a chart, also contains many advanced formatting options.

The following procedures show you how to access the various options that are the Chart Options. For more information about how to use these features, click Help in the various dialog boxes to open the Chart Help. The Chart Help (Chart.chm) is installed by default in the \Program Files\SAP BusinessObjects\Crystal Reports\ChartSupport\Help\<language> directory.

13.3.2.1 To change chart formatting

Procedure

1. Right-click your chart, and then select [Chart Options](#).

2. Click the appropriate tab to change appearance, titles, data labels, and so on.

13.3.2.2 To change Numeric Axis Grid options

Procedure

1. Right-click your chart, and then select [Chart Options](#).
2. Click the appropriate tab to change gridlines, axes, and so on.

13.3.2.3 To change chart titles

Procedure

1. Right-click your chart, and then select [Chart Options](#).
2. Click the Titles tab and make your changes.

13.3.3 Editing charts using other menu items

Click a link to view one of the topics in this section:

- [To apply a new template \[page 315\]](#)
- [To format a selected chart object \[page 316\]](#)
- [To change series options \[page 316\]](#)
- [To choose a viewing angle for a 3D chart \[page 316\]](#)

13.3.3.1 To apply a new template

Procedure

1. Right-click your chart and select [Load Template](#) from the menu.
2. The [Custom](#) templates dialog box appears.

The options on the Custom tab represent directory locations under \Program Files (x86)\SAP BusinessObjects\Crystal Reports\Templates where custom chart files are stored.

Note

The custom charts are available only if you selected Custom Charting when installing Crystal Reports.

13.3.3.2 To format a selected chart object

Procedure

1. Select a line, area, or text object in your chart.
2. Right-click the specified object, and then click *Format <object>*.

For example, you will see Format Pie Slice if you select a pie slice, and Format Series Marker if you select an item within an area chart, bar chart, and so on.

3. Click the appropriate tab to make changes to the format.

13.3.3.3 To change series options

Procedure

1. Select a riser item (area, bar, line, marker, pie slice, and so on) or a legend marker.
2. Right-click the specified area, and then select *Series Options*.

Note

The Series option is not available unless you select a riser or marker as outlined in the previous step, and may not be available for some chart types.

3. Click the appropriate tab to change appearance, data labels, and so on.

13.3.3.4 To choose a viewing angle for a 3D chart

Procedure

Right-click your chart, then select *3D Viewing Angle* from the menu.

13.3.4 Using the zooming features with bar and line charts

On the Preview tab, you can find commands for zooming bar charts and line charts within your report. You have the ability to zoom in and out on these chart types at any time, with each time being referred to as instance-specific. If you decide to save the instance of the chart that has been zoomed in or out, you must save the data with the report.

13.3.4.1 To zoom in and out on a bar or line chart

Procedure

1. On the [Preview](#) tab, right-click the bar or line chart to bring up the shortcut menu.
2. On the shortcut menu, click [Zoom In](#).
3. Drag the Zoom In cursor around a section of the chart to enclose it within the tracking rectangle.

The chart zooms in to the section you selected.

4. Repeat the previous step to zoom in further.

Note

To see adjacent areas on a zoomed-in chart view (neighboring bars in a bar chart, for example), use the Pan option on the shortcut menu. Pull the Pan cursor to the left or right to move in that direction.

5. To zoom out on a chart, right-click the chart to bring up the shortcut menu.
6. On the shortcut menu, click [Zoom Out](#).
7. With the Zoom Out cursor, click the chart.

The chart zooms out one level of magnification.

8. Click the chart again to zoom out further.

13.3.5 Auto-arranging charts

If you move or resize chart objects on the Preview tab, select the Auto-Arrange Chart feature to reset the chart.

13.3.5.1 To auto-arrange a chart

Procedure

1. On the [Preview](#) tab, right-click the chart to bring up the shortcut menu.
2. On the shortcut menu, click [Auto-Arrange Chart](#).

Crystal Reports resets the chart to its original size and position.

13.3.6 Formatting charts

Click a link to view one of the topics in this section:

- [Changing the border of a chart \[page 318\]](#)

- [Conditionally formatting a chart \[page 318\]](#)
- [Changing the chart's legend text \[page 319\]](#)

13.3.6.1 Changing the border of a chart

Procedure

1. On the *Design* or *Preview* tab, right-click the chart to bring up the shortcut menu.
2. On the shortcut menu, click *Format Background*.
3. On the Format Editor dialog box, click the *Border* tab to see its options.
4. Change the line style, color, or thickness.
5. Click *OK* to save your changes.

Crystal Reports returns you to the report and implements your changes.

13.3.6.2 Conditionally formatting a chart

Procedure

1. On the *Design* or *Preview* tab, right-click the chart to bring up the shortcut menu.
2. On the shortcut menu, click *Chart Expert* and select the *Color Highlight* tab.

For information about the expert, search the online help for Color Highlight or Chart Expert.

Note

The option to apply conditional formatting is not available for every chart type.

Note

An area chart must have two "On change of" values for conditional formatting to appear.

Note

If your chart type is line, the chart must have data markers before you can see conditional formatting.

Note

After you apply conditional formatting, you must select "Color by Group" on the Look tab of the Chart Options dialog box before you will be able to see your formatting. To set this option, right-click your chart, point to Chart Options on the shortcut menu, and select General from the submenu. This note applies to line, 3-D Riser, and 3-D Surface charts with one "On change of" field, as well as to bar, numeric axis, 3-D Surface, radar, stock, and charts with two summaries.

3. Click [OK](#).

13.3.6.3 Changing the chart's legend text

Procedure

1. On the [Preview](#) tab, click the text in your chart's legend to select it.

→ Tip

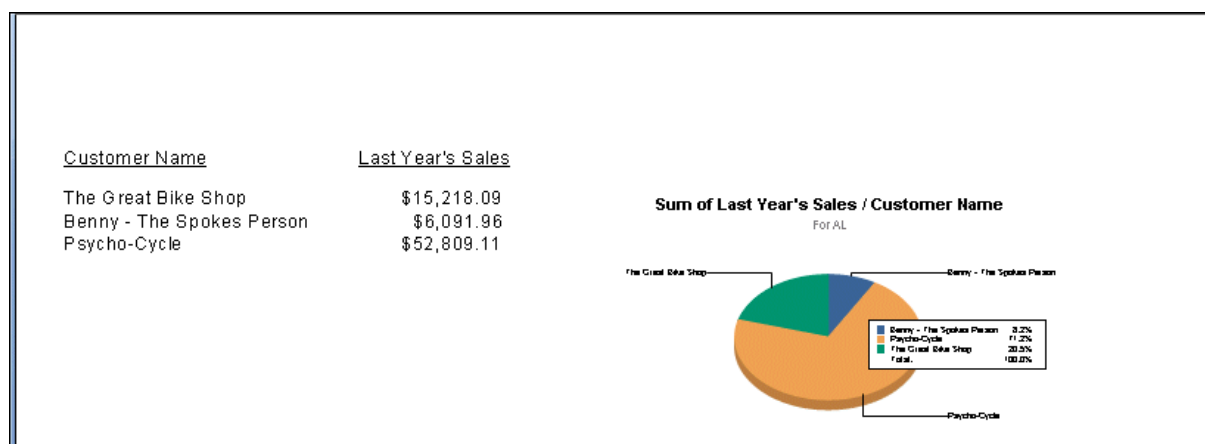
Be sure to select the text and not the entire legend.

2. Right-click the legend text and select [Edit Axis Label](#) from the shortcut menu.
3. In the Label Aliasing dialog box, add the text you want to see in the [Displayed Label](#) field.
4. Click [OK](#).

13.3.7 Using the underlay feature with charts

Since charts can print only in certain sections of your report, the underlay feature gives you more control in the overall look of your report. Instead of having a chart print ahead of the data it represents, it can appear next to the data for a more comprehensible report.

This is how your report looks when you underlay a chart with report data.



14 Mapping

Crystal Reports enables you to include maps with reports made up of geographic data. This section explains how to use maps in reports to make report data more meaningful and easier to interpret. You can customize and rearrange the appearance of a map and activate the drill-down mode to view the details behind the graphical summaries.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

Note

If necessary, Geo map function can be disabled in Crystal Reports designer using a registry key. For more information, please refer [SAP Note 3218545](#) .

You can do a similar setting in Web Intelligence as well.

14.1 Mapping concepts

Click a link to view one of the topics in this section:

- [Mapping overview \[page 320\]](#)
- [Map layouts \[page 321\]](#)
- [Map types \[page 322\]](#)
- [Where to place a map \[page 323\]](#)
- [Drill-down with maps \[page 324\]](#)

14.1.1 Mapping overview

With Crystal Reports, you can include geographic maps in reports. Maps help you analyze report data and identify trends more efficiently. For example, you could create a map that shows sales by region. You would then be able to:

- Use one of the three map types to analyze the data.
- Adjust the appearance and organization of the map (allowing you to better identify trends).

- Drill down on the map regions to view underlying data.

14.1.2 Map layouts

The Map Expert provides four layouts that correspond to certain sets of data.

You can create maps with any of the following layouts, and depending on the data you are using, you can change the map from one layout to another.

Advanced

Use the Advanced layout when using multiple map values or when you do not have any groups or summaries in the report.

Group

The Group layout is a simplified layout in which you show a summary on change of a geographic field (such as Region).

Note

In order to create a map using the Group layout, you must have at least one group and at least one summary field for that group.

Cross-Tab

Use the Cross-Tab layout when mapping on a Cross-Tab object. A Cross-Tab map does not require groups or summary fields.

OLAP

Use the OLAP layout when mapping on an OLAP grid. An OLAP map does not require groups or summary fields.

Note

If there is no map associated with the data you specify, then an empty block will appear unless the section that the map is placed in has been formatted to suppress if blank.

14.1.3 Map types

The Map Expert also provides five basic map types, each suitable for a different strategy of data analysis. When deciding which map type best fits your report, you should consider the type of data you want to analyze. For example, if you want the map to display one data item for each geographic division (city, state, country, and so on), then you might use a Ranged map. On the other hand, if you want the map to display more than one value for each new geographic division, then you could use a Pie Chart map or a Bar Chart map. The following is an overview of the main map types and their most common uses.

Ranged

A Ranged map breaks the data into ranges, assigns a specific color to each range, then color codes each geographic area on the map to display the range. For example, you could create a map that displays Last Year's Sales by Region. If you have sales ranging from zero to 100,000, you might give the map five ranges, with equal intervals of 20,000 each. You could use shades of red (going from dark to light red) to color code each region according to those sales figures. Then you could use this map to see where sales are the highest.

With equal intervals, you might end up with all your regions ranging between zero and 20,000, except perhaps one region (for example, California) that might have exceptionally high sales (such as 98,000). This map would be a very distorted representation of the data. A more useful map would have ranges like 0-5000, 5000-10000, 10000-15000, 15000-20000, and over 20000. It is important to carefully define your ranges.

Note

The end value for each division is repeated as the start value for the next division; the actual value is included in the group it starts. That is, the ranges in the previous example are actually: 0-4999, 5000-9999, and so on.

There are four different distribution methods for ranged maps.

- **Equal count**
This option assigns intervals in such a way that the same number of regions (or as close to the same number of regions as possible) appear in each interval. In other words, this option would assign intervals so that each color on the map is assigned to the same number of regions. The numeric quantity of the summary values in each interval may or may not be equal, depending on the individual regions and their summary values.
- **Equal ranges**
This option assigns intervals in such a way that the summary values in each interval are numerically equal. The number of regions in each interval may or may not be equal, depending on the individual regions and their summary values.
- **Natural break**
This option assigns intervals using an algorithm that attempts to minimize the difference between the summary values and the average of the summary values for each interval.
- **Standard deviation**
This option assigns intervals in such a way that the middle interval breaks at the mean (or average) of the data values, and the intervals above and below the middle range are one standard deviation above or below the mean. Standard deviation is the square root of the variance. It projects how various values in a set of values deviate from the mean for that set.

Pie Chart

A Pie Chart map displays a pie chart over each geographic area. The pie charts represent data items that make up a whole. Each slice of the pie represents an individual data item and shows that item's percentage in the whole. For example, you could create a Pie Chart map showing heating fuel types by region. You might have four types of heating fuel (four slices in each pie): electricity, gas, wood, and solar. Each region would then have a pie chart showing the breakdown of heating fuel types within that region. Washington state would probably use a high percentage (a large slice of the pie) of electricity because of the hydropower in that region, while Idaho would probably use a high percentage (a large slice of the pie) of natural gas.

You can use this map type to compare the distribution of several items within a particular region. You can also specify that the pie charts be sized proportionately so that, as with the symbols in a Graduated map, the pie charts will appear in various sizes, depending on the underlying data values. This will allow you to compare the totals between regions.

Bar Chart

A Bar Chart map works like a Pie Chart map, but may be more useful for certain sets of data. Typically, you would use a Bar Chart map for items that do not total 100%; that is, for data items that do not make a whole, or for data items that are unrelated. For example, you could create a Bar Chart map that displays use of heating fuel by region. You might choose to analyze use of three types of fuel: electricity, gas, and solar. Each bar chart on the map could contain individual bars for each of these types. In this example, the data items (electricity, gas, and solar) do not comprise a whole. There may be other types of fuel used in these regions, such as wood, but this map only focuses on three of them. Also, the purpose of the map is to compare each region's use of each fuel type with that of every other region. In a Pie Chart map, you could show these three fuel types as percentages of the entire fuel use within each region, and though you could compare the percentages for each region, you would probably not be able to compare the actual values for each region because each region would have the same total value (100%).

14.1.4 Where to place a map

When you choose where to place the map, you determine the amount of data that will be included in the map. For example, if you place the map in the Report Header section, the map includes data for the entire report. If you place it in a Group Header or Group Footer section, it displays group-specific data. This choice will also determine whether the map prints once for the entire report, or many times (once for each instance of a given group).

Note

If your report contains subreports, you can place maps in those subreports as well.

14.1.5 Drill-down with maps

Not only is mapping a means of presenting data—it is also an analysis tool. Move your cursor over a section of the map on the Preview tab, so that the pointer becomes a Drill-down cursor, then double-click to view the underlying details for that section of the map.

Note

If you drill down on a region that has no data associated with it, you will get a message saying, "There are no detail records for that {Region Name}."

14.2 Creating maps

The process for creating a map varies depending on the data you are mapping on. The following sections detail the mapping process for each map layout.

Mapping on details fields (Advanced layout)

The Advanced layout allows you to create a map based on specific values. With an Advanced layout, you can create a map without the need for a summary field by using values that appear in the Details section of your report.

To create a map based on the Advanced layout, you must specify two things:

- Conditions (there can be two).
- Value (there must be only one value).

Condition

The condition is used to indicate where to plot the data on a map. This condition must be a string field. In order for a map to be generated, the field should contain geographic information. For example, a map showing last year's sales for each country uses the Country field as the condition. Each time the condition changes (the country changes), that area on the map is highlighted.

Value

The Advanced layout uses the value to indicate what information is mapped when the area on the map is highlighted. For example, to create a map showing last year's sales for the countries, the Last Year's Sales field would be the value.

14.2.1 Mapping on group fields (Group layout)

To map on a group, you can use the Group layout, in which you show a summary (such as Last Year's Sales) on change of a geographic field (such as Region). In order to create a map using the Group layout, you must have at least one group and at least one summary field for that group.

14.2.1.1 To map on a group

Procedure

1. On the *Insert* menu, click *Map*.

→ Tip



Another way to do this is to click the Insert Map button on the Insert Tools toolbar.

The Map Expert dialog box appears.

2. On the *Data* tab, in the Layout area, click *Group*, if it is not already selected.
3. In the Placement area, in the *Place map* list, specify how often your map appears on the report, then click *Header* or *Footer* to specify where to place your map.
4. In the Data area, in the *On change of* list, click the group field you want to base your map on, then in the *Show* list, click the summary field you want to display on your map.
5. Click the *Type* tab.
6. Click the map type that best illustrates your data (Ranged, Dot Density, or Graduated). See [Map types \[page 322\]](#).
7. In the Options area, apply formatting options to your map.
8. Click the *Text* tab.
9. In the *Map title* field, enter a title for your map.
10. In the Legend area, you can click one of the following options:
 - *Full legend* to show a detailed legend on your map.
 - *Compact legend* to show a condensed legend on your map.
 - *No legend* to exclude the legend from your map.
11. If you click *Full legend*, click *Made by map* to have Crystal Reports create a legend title based on your map, or click *Specify* to enter your own legend title and subtitle.
12. Click *OK*.

Your map is placed in the Header or Footer section of the report, depending on your selection in Step 3.

14.2.2 Mapping on Cross-Tab summaries (Cross-Tab layout)

With the Cross-Tab layout, you can map on a Cross-Tab summary field. For example, with a Cross-Tab that shows the total number of items of a product sold in each region of the United States, you may want to include a map that shows the percentage of the total sales of the product provided by each region.

In order to create a Cross-Tab map you must first have a Cross-Tab in your report. For more information about Cross-Tabs, see [Cross-Tab Objects \[page 346\]](#).

14.2.2.1 To map on a Cross-Tab summary

Procedure

1. On the [Insert](#) menu, click [Map](#).

→ Tip



Another way to do this is to click the Insert Map button on the Insert Tools toolbar.

The Map Expert dialog box appears.

2. On the [Data](#) tab, in the Layout area, click [Cross-Tab](#), if it is not already selected.
3. In the Placement area, click [Header](#) or [Footer](#) to specify where to place your map.
4. In the Data area, in the [Geographic field](#) list, click a geographic field to supply the geographic areas for your map.
5. If necessary, in the [Subdivided by](#) list, click another field to map on.
Crystal Reports uses this field to subdivide the pie or bar charts appearing on your map.
6. In the [Map on](#) list, click a summary field to supply the numeric data for your map.
7. Click the [Type](#) tab.
8. Click the map type that best illustrates your data. See [Map types \[page 322\]](#).
 - If you plan to map on only one field, be sure to select None from the "Subdivided by" list, then choose either the Ranged, Dot Density, or Graduated map type.
 - If you plan to map on two fields, be sure to select an additional field from the "Subdivided by" list, then choose either the Pie Chart or Bar Chart map type.
9. In the Options area, apply formatting options to your map.
10. Click the [Text](#) tab.
11. In the [Map title](#) field, enter a title for your map.
12. In the Legend area, you can click one of the following options:
 - [Full legend](#) to show a detailed legend on your map.
 - [Compact legend](#) to show a condensed legend on your map.
 - [No legend](#) to exclude the legend from your map.
13. If you click [Full legend](#), click [Made by map](#) to have Crystal Reports create a legend title based on your map, or click [Specify](#) to enter your own legend title and subtitle.

14. Click [OK](#).

Your map is placed in the Header or Footer section of the report, depending on your selection in Step 3.

14.2.3 Mapping on an OLAP cube (OLAP layout)

Using the OLAP layout, you can map on an OLAP grid. In order to create an OLAP map, you must first have an OLAP grid in your report. For more information about OLAP grids, see [Creating an OLAP report \[page 395\]](#).

14.2.3.1 To map on an OLAP cube

Procedure

1. On the [Insert](#) menu, click [Map](#).

→ Tip



Another way to do this is to click the Insert Map button on the Insert Tools toolbar.

The Map Expert dialog box appears.

2. On the [Data](#) tab, in the Layout area, click [OLAP](#), if it is not already selected.
3. In the Placement area, click [Header](#) or [Footer](#) to specify where to place your map.
4. In the Data area, click the [On change of](#) list and select the field on which you want to base your map.

ⓘ Note

This field should contain geographic information, such as Country.

5. If necessary, click the [Subdivided by](#) list and select a secondary row or column to incorporate into your map.

ⓘ Note

Choosing to subdivide by a secondary mapping field, either row or column, will affect the map types available in Step 8.

6. Click the [Type](#) tab.
7. Click the map type that best illustrates your data. If there is no secondary mapping field, you can choose [Ranged](#), [Dot Density](#), or [Graduated](#); if there is a secondary mapping field, you can choose [Pie Chart](#) or [Bar Chart](#). See [Map types \[page 322\]](#).
8. In the Options area, apply formatting options to your map.
9. Click the [Text](#) tab.
10. In the [Map title](#) field, enter a title for your map.
11. In the Legend area, you can click one of the following options:

- *Full legend* to show a detailed legend on your map.
- *Compact legend* to show a condensed legend on your map.
- *No legend* to exclude the legend from your map.

ⓘ Note

If you click Full legend, click "Made by map" to have Crystal Reports create a legend title based on your map, or click Specify to enter your own legend title and subtitle.

12. Click *OK*.

Your map is placed in the Header or Footer section of the report, depending on your selection in Step 3.

14.3 Working with maps

Once you have created a map, you may want to add a new title, headings, and a legend, and you may want to change fonts, or even the map type. Crystal Reports provides many options for working with your existing maps.

14.3.1 Editing maps using the Map Expert

Editing maps with the Map Expert allows you to return to the expert in which you designed your map. You can modify many of your original choices, such as the type of map to display, the data on which the map is based, and so on. From either the Design or Preview tab, you can open the Map Expert and make your modifications.

14.3.1.1 To edit a map using the Map Expert

Procedure

1. Right-click the map to bring up the shortcut menu.
2. On the shortcut menu, click *Map Expert*.
3. In the Map Expert dialog box, make the desired changes.
4. Click *OK*.

14.3.2 Changing the map title

Procedure

1. On the [Preview](#) tab, right-click the map to bring up the shortcut menu.
2. On the shortcut menu, choose [Title](#).
3. In the Change Map Title dialog box, enter a new title.
4. Click [OK](#) to save your changes.

Crystal Reports returns you to the Preview tab and implements your changes.

14.3.3 Changing the map type

You can change the map type and set the properties for that map directly from the menu that appears when you right-click a map on the Preview tab. For example, if you want to see how the data of a Ranged map would look if it were presented in a Dot Density style, you can rearrange the map without having to return to the Map Expert and rework the format.

14.3.3.1 To change the map type

Procedure

1. On the [Preview](#) tab, right-click the map to bring up the shortcut menu.
2. On the shortcut menu, choose [Type](#).

The Customize Map dialog box appears.

3. In the [Map type](#) list, click the new map type.
4. In the Options area, apply formatting options to your new map.

The properties made available to you vary depending on the map you have selected.

5. Click [OK](#) to save your changes.

Crystal Reports returns you to the Preview tab and implements your changes.

14.3.4 Changing map layers

In Crystal Reports, maps are made up of a number of layers. You can stack these layers on top of each other to create a more detailed map. For example, you might want to see the names of major cities in every country; to do this, you could add a layer that contains city names. This layer sits on top of the basic world map to provide further information.

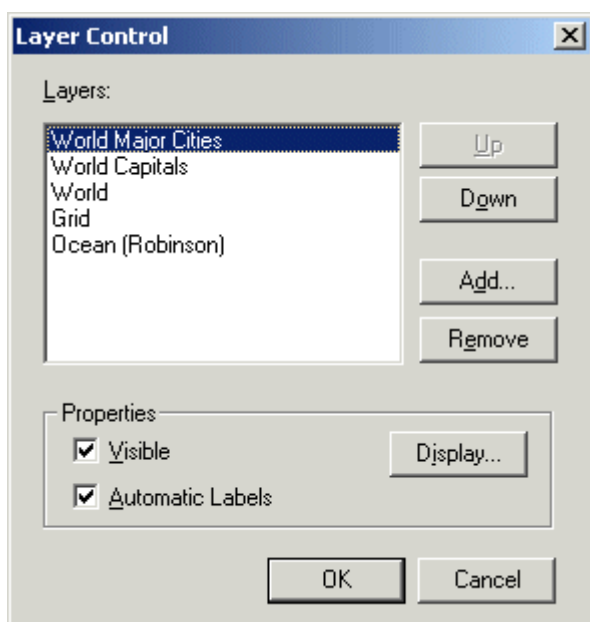
You can specify which of the provided map layers you want to see, and you can rearrange the order of the layers that you have selected.

14.3.4.1 To change map layers

Procedure

1. On the [Preview](#) tab, right-click the map to bring up the shortcut menu.
2. On the shortcut menu, and choose [Layers](#).

The Layer Control dialog box appears.



3. In the [Layers](#) list, click [Up](#) or [Down](#) to arrange the order of the map layers.

Be aware that some map layers contain non-transparent sections that can obscure the detail of other layers. For example, the Ocean layer is mostly non-transparent; if you place it higher in the layers list, it will block many of the other layers (the other layers seem to disappear, but they are merely obscured by the Ocean layer).

4. In the Properties area, set the properties for each map layer, specifying whether the layer is visible and whether it is automatically labeled.

The Visible option specifies whether the layer appears. The Automatic Labels option specifies whether a predefined label appears (for example, labels for the names of major world cities).

5. If necessary, click [Display](#) to open the Display Properties dialog box.

Use this dialog box to set the default display mode and zoom range (the minimum and maximum possible magnification) for the layer in question, then click OK to return to the Layer Control dialog box.

6. Click [OK](#) to save your changes.

Crystal Reports returns you to the Preview tab and implements your changes.

14.3.5 Resolving data mismatches

Sometimes the map you are working with uses a different geographic name from the one used in the database. For example, a map of Europe may use "United Kingdom" while the database uses the abbreviation "U.K." Until you resolve the data mismatch, the map will not be able to display any information for the selected geographic region.

14.3.5.1 To resolve data mismatches

Procedure

1. On the [Preview](#) tab, right-click the map to bring up the shortcut menu.
2. On the shortcut menu, and choose [Resolve Mismatch](#).

The Resolve Map Mismatch dialog box appears.

3. Click the Resolve Mismatch tab to see a listing the geographic names that do not correspond between the database and map.
4. In the [Assign this Field Name](#) list, click the name you want to resolve.
5. To assign the heading, click a keyword from the [To this Map Name](#) list.
6. If there is a match, click [Match](#).

The new assignment is displayed in the Matched results part of the Resolve Mismatch tab.

7. Once you are finished resolving the data mismatches, click [OK](#).

The map can now display data for the selected geographic region.

14.3.6 Changing the geographic map

If you prefer to have your data values presented with a different geographic map, specify your changes using the Change Map tab of the Resolve Map Mismatch dialog box.

14.3.6.1 To change the geographic map

Procedure

1. On the [Preview](#) tab, right-click the map to bring up the shortcut menu.

2. On the shortcut menu, click [Resolve Mismatch](#).

The Resolve Map Mismatch dialog box appears; the Change Map tab displays the name of the current map you are using, and a list of replacement maps that you can choose from.

3. In the [Available maps](#) list, click the new map you want to use.
4. Click [OK](#) to save your changes.

Crystal Reports returns you to the Preview tab and implements your changes.

14.3.7 Zooming in and out on a map

Procedure

1. On the [Preview](#) tab, right-click a map to bring up the shortcut menu.
2. On the shortcut menu, click [Zoom In](#).
3. Drag the selection cursor around the map section you want to see.

The map zooms in one level of magnification.

4. Drag another selection to zoom in further.
5. To zoom out on a map, right-click the map to bring up the shortcut menu.
6. On the shortcut menu, click [Zoom Out](#).
7. Click the map.

The map zooms out a level of magnification.

8. Click the map again to zoom out further.

Results

ⓘ Note

If a map occurs once for each instance of a group, any panning or zooming settings you specify are instance-specific. In other words, if you zoom in on the map in one group header, this setting is only for that group header. None of the headers have been changed. This way you can set each map to different settings.

ⓘ Note

If the Save Data with Report option is selected from the File menu, your panning and zooming settings will be saved with the report. If the option is not selected, then only the default map settings will be saved.

14.3.8 Panning a map

Procedure

1. On the [Preview](#) tab, right-click a map to bring up the shortcut menu.
2. On the shortcut menu, click [Pan](#).
3. Drag the panning cursor to the desired map section.

14.3.9 Centering a map

Procedure

1. On the [Preview](#) tab, right-click a map to bring up the shortcut menu.
2. On the shortcut menu, click [Center Map](#).

14.3.10 Changing the border of a map

Procedure

1. On the [Design](#) or [Preview](#) tab, right-click the map to bring up the shortcut menu.
2. On the shortcut menu, click [Format Map](#).
3. On the Format Editor dialog box, click the [Border](#) tab to see its options.
4. Change the line style, color, background color, and add or remove a drop shadow from the map border.
5. Click [OK](#) to save your changes.

Crystal Reports returns you to the report and implements your changes.

14.3.11 To underlay a map

Procedure

1. Create your map and place it in the Report Header section.

For more information, see [Creating maps \[page 324\]](#).

2. On the [Report](#) menu, click [Section Expert](#).

→ Tip



Another way to do this is to click the Section Expert button on the Expert Tools toolbar.

The Section Expert dialog box appears.

3. In the Sections area, click *Report Header*, then select the *Underlay Following Sections* check box.
4. Click *OK*.

Crystal Reports returns you to the report. Your map will now underlay the sections below it.

5. If necessary, move or resize the map.

15 OLE

This section explains how Object Linking and Embedding (OLE) can be used to edit graphics or other objects from within your report instead of opening an additional application.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com>

15.1 OLE overview

Object Linking and Embedding (OLE) enables you to insert objects (OLE objects) into a report from other applications (OLE server applications) and then use those applications from within Crystal Reports to edit the objects if necessary.

If you were not using OLE, you would have to exit Crystal Reports, open the original application, change the object, return to Crystal Reports, delete the object originally inserted, and then insert the newly revised object.

Note

If you intend to use your Crystal report in SAP BusinessObjects Business Intelligence platform or through the Report Application Server (RAS), copy the source file to a network folder that is accessible to the network account(s) under which the processing servers (Job Server, Page Server, RAS) are running. Ensure that report designers' accounts also have permission to the network folder. When you insert the linked OLE object in Crystal Reports, use a UNC path.

15.1.1 OLE terminology

Before you work with OLE objects in Crystal Reports, it might be helpful to review some terminology:

- **OLE**
OLE is an acronym for Object Linking and Embedding. It refers to the ability to create compound documents; that is, documents that contain elements from other applications that can be edited using the original application.
- **OLE Object**
An OLE object is, broadly speaking, a presentation of data that was created in another application and that maintains a relationship with the application that was used to create it. A bitmap created in Paint, an

Excel spreadsheet, or a graph from MS Graph may all be OLE objects if they are inserted in the receiving document as OLE objects. If they are not inserted as OLE objects, they retain no relationship with the original application.

- **OLE Container Application**
An OLE container application is one that can contain and process OLE objects created in other applications (such as Paint or Excel). Crystal Reports is a container application.
- **Container Document**
A container document is a document that was created using the container application and that contains one or more OLE objects.
- **OLE Server Application**
An OLE server application is an application that allows its documents to be inserted into an OLE container document as OLE objects. Microsoft Word and Excel are examples of applications that are both OLE servers and OLE containers. That is, they can both create new OLE objects and they can contain OLE objects created elsewhere.
- **Server Document**
The server document is a file created in the server application that stores the original OLE object.

15.1.2 Types of OLE objects

- **Static object**
A static OLE object is a picture of an object that can be displayed and printed, but not edited in place. It does not have a connection to a server application. There are two kinds of static objects: static bitmaps and static metafiles.
- **Embedded object**
An embedded object contains a representation of the object, as well as information that defines the content. It does not have a connection to the original object in the server document. As a general rule, use embedded objects when you need to edit the object independently of the original server document.
- **Linked object**
A linked object contains a representation of the data and a link to the file where it originated. As a general rule, use linked objects when the data in the server document is changing and you want the object in your report to be updated when you open the report.

15.1.3 General OLE considerations

There are several points to keep in mind when utilizing OLE functionality.

- When you double-click an embedded OLE object, the object is activated for editing and the Report Designer merges its menus and toolbars with those of the object's server application. If the OLE server application does not support this behavior, the object appears in a separate window. When you are finished editing, click outside the object and the Report Designer toolbars reappear.
- When you double-click a linked OLE object, the program opens the object's server application with the object displayed and ready for editing. You cannot edit a linked object in place in Crystal Reports because you are working on the original object. Since the object could be linked to multiple documents, displaying the original in the server application limits access to one editor at a time.

15.2 Inserting OLE objects into reports

There are several ways to insert OLE objects into an application.

- The OLE Object choice on the Insert menu can be used to import an existing object or create a new one. You can place either embedded or linked objects using this method.
- Paste Special on the Edit menu can be used to copy or cut the object from an OLE server application and paste it in a report. If the object can be pasted in multiple formats, you need to decide which format to use. For example, when inserting text from a Microsoft Word document, you can paste the text as Microsoft Word document text (which can be edited in Word), or as a metafile, which is simply a non-editable picture of the text. Use Paste Special to place either embedded or linked objects.

Note

When inserting selected information (portions of larger files), it is best to use Copy and Paste Special editing. When inserting entire files, click OLE Object from the Insert menu.

15.2.1 To copy and paste OLE objects

Context

This procedure assumes that you have Microsoft Excel or another spreadsheet program on your computer that is an OLE server application.

Procedure

1. Open both Crystal Reports and Microsoft Excel.
2. Open an existing report in Crystal Reports.
3. Create a spreadsheet in Microsoft Excel by typing the numbers one through ten (1-10) in cells A1 through A10.
4. Select cells A1 through A10, right-click and select *Copy* from the shortcut menu in Excel.
5. Go to Crystal Reports and select *Paste* from the *Edit* menu.
6. Place the cells into the Details section of the report.
7. Right-click the spreadsheet object in the report and observe that it is identified as an OLE object and that the third command from the last identifies it as a Worksheet Object.
8. Double-click the spreadsheet object.

The menus and toolbars change to a combination of those from the OLE server application and Crystal Reports. You can now edit the object in-place. The object is an embedded object. Any changes you make to the object will not affect the original.

9. Click outside the object when you are finished.

The menus and toolbars change back to those of Crystal Reports. Any changes you made while editing are saved.

15.3 How OLE objects are represented in a report

An OLE object is displayed in a report in any of several different formats depending on how you chose to insert the object.

- If you have chosen to display the object as an icon, the icon will appear in the report. Users can choose to edit the object later by double-clicking the icon.
- If the object was created from an existing file, the data from that file (or an icon) is displayed in the report. This data can be edited by double-clicking the object or its icon.
- If you are creating a new object, the application for the object type that you chose will open, and you can begin designing the object. When you are finished, close or exit the application. The object (or its icon) will be displayed in the report.

15.4 Editing OLE objects in reports

Click a link to view one of the topics in this section:

- [In-place editing \[page 338\]](#)
- [Dynamic OLE menu commands \[page 339\]](#)
- [OLE and the Insert Picture command \[page 339\]](#)

15.4.1 In-place editing

In-place editing is the ability to change an OLE object's properties from within an OLE container application (such as Crystal Reports). The container application's menu items change to provide the editing tools from the server application so that you can make changes easily.

When an OLE object is placed in a report, the object becomes part of the report. To edit the object, double-click it and then modify it using the editing tools found in the object's original application or from a similar application that allows in-place editing. You can edit an OLE object only if you have the corresponding server application installed on your computer. You can view and print a report containing an OLE object even if you don't have the server application installed.

15.4.2 Dynamic OLE menu commands

When you have an OLE object selected, there is a submenu on the Edit menu that provides commands for the object. The name of this submenu reflects the type of the OLE object. The commands on the submenu also vary according to the type of object.

The object may be described as a Document object, Bitmap Image object, Picture object, Worksheet object, or something similarly descriptive:

- If the object is embedded, the Edit menu displays those commands that are available to that type of embedded object.
- If the object is linked, the Edit menu displays commands for that type of linked object.

Commands on the context menus change in a similar fashion.

These dynamic commands are provided to give you more control when working with OLE objects.

15.4.3 OLE and the Insert Picture command

When the OLE Object command on the Insert menu is used to place pictures in a report, the picture will be converted to an embedded or linked object associated with Microsoft Paint (or another image editing application).

When the Picture command on the Insert menu is used to place pictures in a report, the Report Designer converts them into static objects.

Static objects cannot be edited. To edit a static object, it has to be converted into an editable type of object using the Convert command on the Edit menu.

15.5 Working with static OLE objects

You can insert a number of different types of static OLE objects into your report. Crystal Reports supports these pixel-based image formats:

- Windows Bitmap (BMP)
- JPEG
- PNG

As well, Crystal Reports supports this vector-based image format:

- Windows Metafiles (both WMF and the newer enhanced metafile format).

Note

Wide static OLE objects can span pages. This functionality may result in two or more horizontal pages in your report. If this do not want this effect, you should resize your static OLE object. As well, it is recommended that you do not use the Repeat on Horizontal Pages option for any static OLE object that will span several pages.

15.5.1 Inserting a static OLE object

Procedure

1. Open or create a report that you want to insert a static OLE object in.
2. On the *Insert* menu, click *Picture*.

→ Tip



Another way to do this is to click the picture button on the Insert Tools toolbar.

The Open dialog box appears.

3. Select a picture (.bmp, .jpg, .png, or .wmf) from a directory that you have access to.
4. Click *Open*.
5. When the object frame appears, place the picture in your report.
6. Right-click the picture, and notice that the picture is identified as an OLE object on the shortcut menu.

When you double-click the object, nothing happens. A static OLE object cannot be edited.

15.5.2 Making a static OLE object dynamic

A dynamic static OLE object is a picture that you access in Crystal Reports by use of a file path or URL. Typically, you use this option when you have the path to your pictures stored as a string field in a database (that is, you do not have the actual objects stored in the database). You can also use this option when you have pictures stored on a network share, and you know these objects change occasionally.

The functionality of these dynamic static OLE objects (or dynamic graphics) is created in Crystal Reports through the use of a conditional formatting formula that allows you to link to static OLE objects by reference.

ⓘ Note

This feature is activated when you refresh your report data; therefore, you may not notice a change in the static OLE object until you click the Refresh button in Crystal Reports.

ⓘ Note

When you schedule a report that contains a dynamic static OLE object in SAP BusinessObjects Business Intelligence platform, the instances that are created contain the version of the static OLE object as it existed in your Crystal report when you scheduled it. SAP BusinessObjects Business Intelligence platform does not update the object dynamically for each instance.

ⓘ Note

Reports that use an absolute or a relative path for a dynamic static OLE object are not supported in an SAP BusinessObjects Business Intelligence platform environment for this release. If you plan to publish reports that contain dynamic static OLE objects to SAP BusinessObjects Business Intelligence platform, it is recommended that you use URL links for your OLE objects.

15.5.2.1 To make a static OLE object dynamic

Procedure

1. Right-click the static OLE object that you inserted, and choose *Format Graphic* from the shortcut menu.

Note

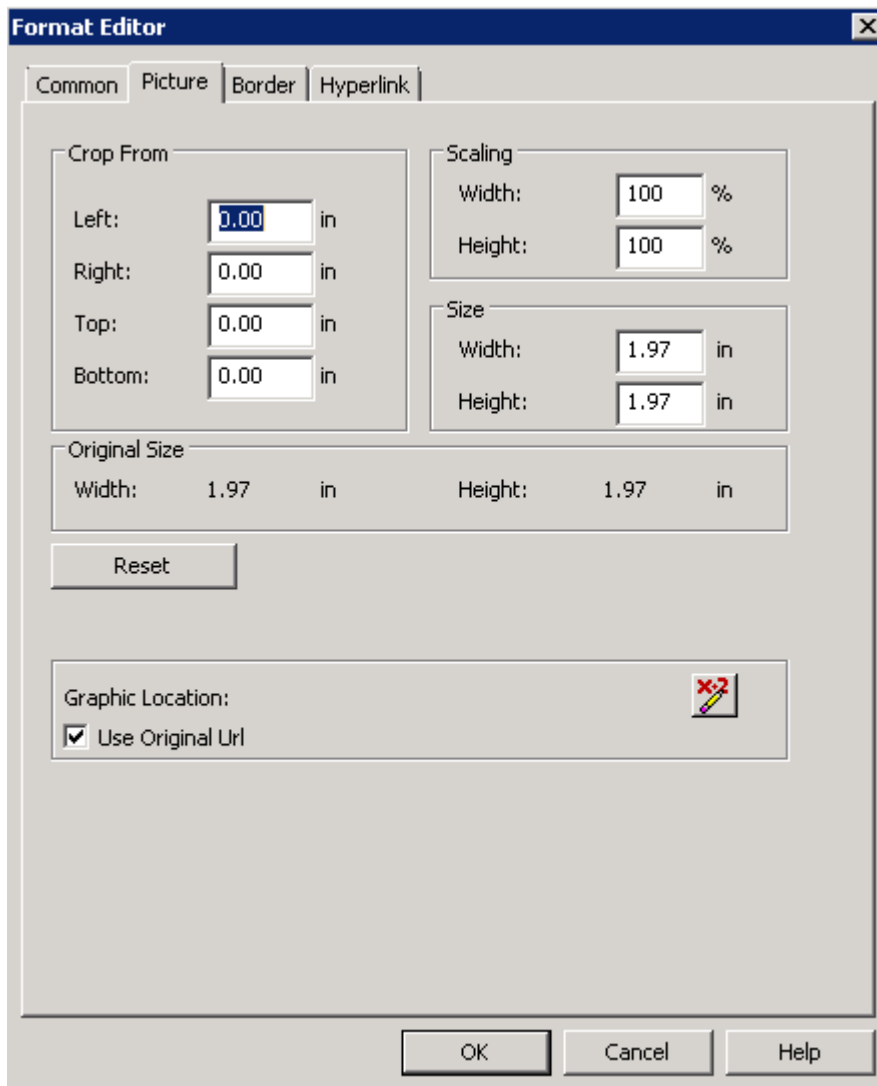
You begin the process of making a static OLE object dynamic by inserting the object into your report as usual. This object becomes the default object. If Crystal Reports cannot find the dynamic reference to the object, it uses the default object instead.

2. In the Format Editor window, click the *Picture* tab, and then click the  (*Conditional Formula*) button adjacent to the *Graphic Location* label.
3. In the Formula Workshop, create the path to your static OLE object.

For example, if your path is stored in a database field, add that field to the formula.

4. Click *Save and close* to return to the Format Editor.
5. (Optional) In the *Picture* tab, select the *Use Original URL* check-box to skip the URL encoding check.

This is sometimes necessary when your URL contains certain special characters (such as “%3d”) that are already encoded (to for instance “=”), while the URL destination expects the original special characters.



6. Click [OK](#) to return to your report.

7. Click  ([Refresh](#)) to update the reference link to your static OLE object.

15.5.3 Converting a static OLE object to a bitmap image object

Procedure

1. Right-click the static OLE object that you inserted, and choose [Convert Picture Object](#) from the shortcut menu.

The Convert dialog box appears.

2. Select [Paintbrush Picture](#) in the [Object Type](#) list, and click [OK](#).

You have just converted a static OLE object to a modifiable OLE object.

Note

This option does not appear if the original file was a metafile.

3. Right-click the object you have just converted.

The command on the shortcut menu is now Bitmap Image Object.

4. Double-click the object.

Microsoft Paint, or another graphics application installed on your machine, opens.

15.6 Working with embedded vs. linked objects

Since embedded and linked objects each have different properties, it is important for you to consider the capabilities of each when deciding which OLE format to use.

15.6.1 Embedded objects

An embedded object can be created from within Crystal Reports or by using a file that already exists. If you use an existing file, the object is copied to the report.

When changes are made to an embedded object it does not affect the original file and vice versa. The changes are made only to the object within your report.

15.6.1.1 To insert an embedded OLE object

Procedure

1. Click the *Design* tab.
2. On the *Insert* menu, click *OLE Object*.

The Insert Object dialog box appears.

3. Select *Create From File*.

The dialog box changes, allowing you to either type in an object name or browse.

4. Click *Browse* and choose a bitmap file (BMP).
5. Click *Open* to return to the Insert Object dialog box.
6. Click *OK* to return to the report.

An object frame appears.

7. Place the object in your report.

8. Right-click this picture and observe that it is also identified as an OLE Object on the shortcut menu.

The third command from the bottom of the shortcut menu identifies it as a Bitmap Image Object. It is an embedded OLE object.

9. Double-click the object.

You have remained in Crystal Reports but the menus and tools have merged with Microsoft Paint, or those of another graphics package installed on your machine. These tools can be used to edit the bitmap in place. Remember that even though you are working with the tools of the OLE server application, you are not working on the original file; you are working with a copy and nothing you do here will affect the original.

15.6.2 Linked objects

When a linked object is inserted into a report, Windows copies a snapshot of the data (not the data itself) from a file that already exists. The image of the object is added to your report along with a reference to the data used to create it. The actual data remains with the original file.

When the object is activated from within a report, the original file is opened inside the application that was used to create it. Any changes you make directly affect the original file also.

If you want the data in an object to remain available to other applications, and always reflect the most current changes to the data, link the object to the report. An automatic link is refreshed from the source file every time the report is loaded; a manual link is refreshed only when you request it. You can force a refresh, break a link, or redefine the link in the Edit Links dialog box.

When you have a linked object and you break the link using the Links dialog box, all connections to the original data in the server document are broken. A linked object in a container application is merely a representation of that object and a link between the object and the server document. When you break the link you're left with only the representation, with no relationship to the original data or to the editing capabilities of the server application. In this situation, Crystal Reports turns the object into a static object, a stand-alone object that can neither be edited using OLE capabilities nor converted into an editable OLE object.

15.6.2.1 To link a bitmap image object

Procedure

1. On the *Insert* menu, click *OLE Object*.

The Insert Object dialog box appears.

2. Select *Create From File*.

The dialog box changes, enabling you to enter an object name or browse.

3. Click *Browse* and choose a bitmap file (BMP).
4. Click *Open* to return to the Insert Object dialog box.
5. Select the *Link* check box.
6. Click *OK* to return to the report.

An object frame appears.

7. Place this object in your report.
8. Right-click this object and observe that this object is identified as an OLE object.

The third command from the bottom of the shortcut menu identifies this as a Linked Bitmap Image Object.

9. Double-click the object.

Microsoft Paint or the graphics application you are using opens, displaying the original file. Any changes you make to the original will be reflected in the object that appears in your report.

16 Cross-Tab Objects

A Cross-Tab object is a grid that displays values that are grouped and summarized in two directions. This section provides you with information about how you can use Cross-Tab objects in your report.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com>

16.1 What is a Cross-Tab object?

A Cross-Tab object is a grid that returns values based on the criteria you specify. Data is presented in compact rows and columns. This format makes it easy to compare data and identify trends. It is made up of three elements:

- Rows
- Columns
- Summary fields

The diagram illustrates a Cross-Tab object. A grid is shown with rows and columns. An arrow labeled 'Column' points down to the top row of the grid. An arrow labeled 'Row' points right to the first column of the grid. The grid contains the following data:

	USA	MEXICO	CANADA	UK	TOTAL
Gloves	4	0	4	0	8
Belts	0	1	1	1	3
Shoes	0	0	0	1	1
TOTAL	4	1	5	2	12

An arrow points to the value '4' in the 'Gloves' row and 'USA' column, with the text 'Sum of gloves in USA (intersection)' below it.

- The rows in a Cross-Tab run horizontally (from side to side). In the example above, "Gloves" is a row.
- The columns in a Cross-Tab run vertically (up and down). In the example above, "USA" is a column.
- The summary fields are found at the intersection of rows and columns. The value found at each intersection represents a summary (sum, count, and so on) of those records that meet the row and the column criteria. In the example above, the value at the intersection of "Gloves" and "USA" is four, the number of gloves sold in the USA.

A Cross-Tab also includes several totals:

	USA	MEXICO	CANADA	UK	TOTAL	
Gloves	4	0	4	0	8	← Total of products in row (gloves)
Belts	0	1	1	1	3	
Shoes	0	0	0	1	1	
TOTAL	4	1	5	2	12	← Grand total all products (row) all countries (column)

↑
Total of products in column (USA)

- At the end of each row is a total for that row. In the example above, this total represents a single product sold in all countries. At the end of the "Gloves" row is the value 8, the total number of gloves sold in all countries.

Note

The total column can appear at the beginning of each row.

- At the bottom of each column is a total for that column. In the example above, this total represents all products sold in a single country. The value at the bottom of the "USA" column is four, the total number of products (gloves, belts and shoes) sold in the USA.

Note

The total column can appear at the top of each column.

- At the intersection of the Totals column (totals for the products) and the Totals row (totals for the countries) is a grand total. In the example above, the value at the intersection of the Total Column and Total Row is 12, the total number of all products sold in all countries.

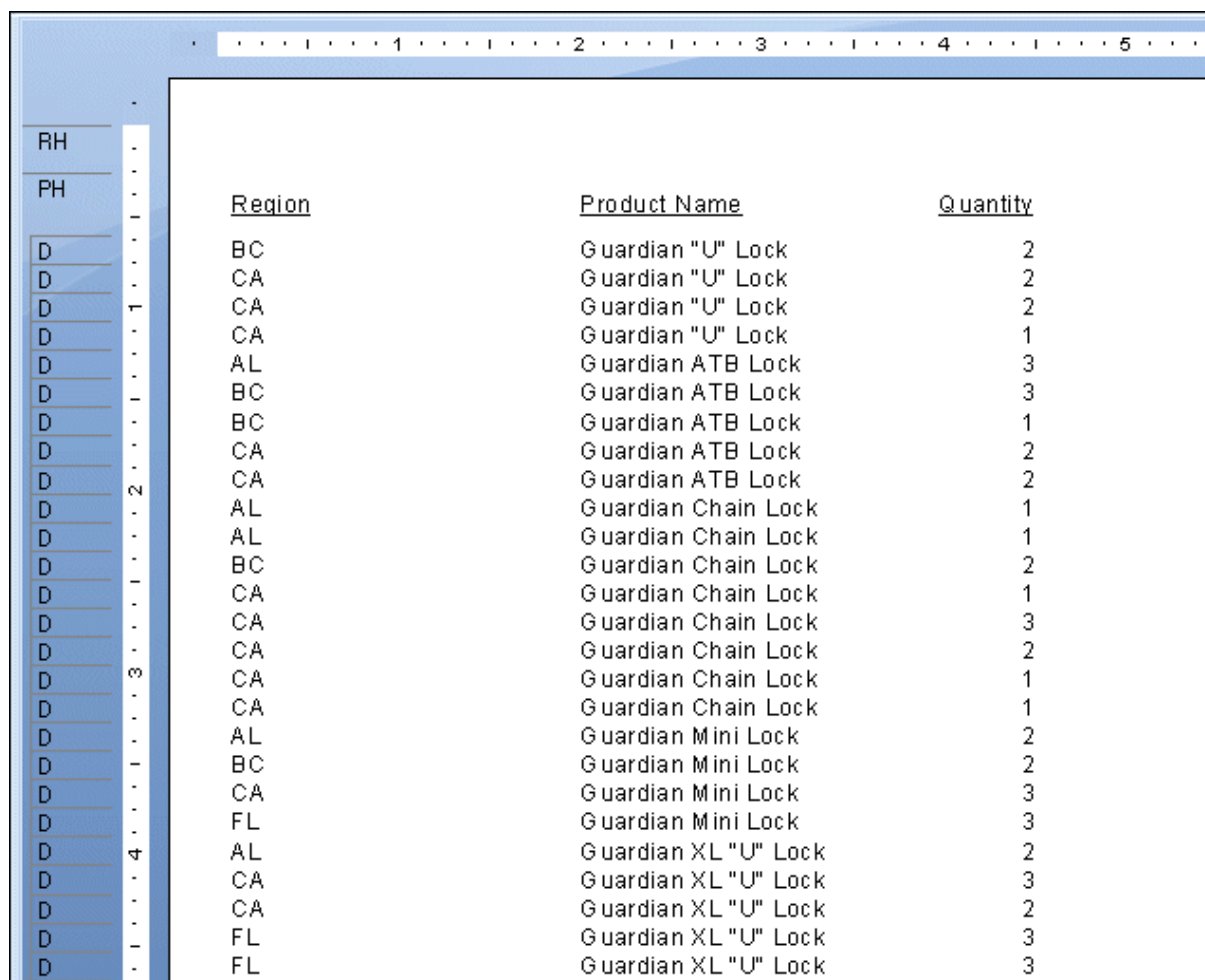
16.2 Cross-Tab example

This example demonstrates how you can use a Cross-Tab object to make it easier to understand complex data. The goal of this report is to analyze the unit sales of five different bicycle locks in four different regions (a unit sales of locks by region report). For greater clarity, only the most essential information in these reports has been included:

- The region from which the order came.
- The name of the lock.
- The quantity ordered.

The first way of looking at the data is in the most basic of all reports, a columnar report with no grouping or sorting.

16.2.1 Report of order data - no sorting/grouping



The screenshot displays a SAP Crystal Reports interface. On the left, there is a vertical pane with a list of 'D' entries, likely representing a drill-down menu. The main area shows a table with three columns: 'Region', 'Product Name', and 'Quantity'. The data is presented as a flat list of 20 rows, each representing an individual order. The regions listed are BC, CA, AL, and FL. The product names are 'Guardian "U" Lock', 'Guardian ATB Lock', 'Guardian Chain Lock', and 'Guardian Mini Lock'. The quantities range from 1 to 3. The table is not sorted or grouped, making it difficult to analyze the data by region or product type.

	<u>Region</u>	<u>Product Name</u>	<u>Quantity</u>
D	BC	Guardian "U" Lock	2
D	CA	Guardian "U" Lock	2
D	CA	Guardian "U" Lock	2
D	CA	Guardian "U" Lock	1
D	AL	Guardian ATB Lock	3
D	BC	Guardian ATB Lock	3
D	BC	Guardian ATB Lock	1
D	CA	Guardian ATB Lock	2
D	CA	Guardian ATB Lock	2
D	AL	Guardian Chain Lock	1
D	AL	Guardian Chain Lock	1
D	BC	Guardian Chain Lock	2
D	CA	Guardian Chain Lock	1
D	CA	Guardian Chain Lock	3
D	CA	Guardian Chain Lock	2
D	CA	Guardian Chain Lock	1
D	CA	Guardian Chain Lock	1
D	AL	Guardian Mini Lock	2
D	BC	Guardian Mini Lock	2
D	CA	Guardian Mini Lock	3
D	FL	Guardian Mini Lock	3
D	AL	Guardian XL "U" Lock	2
D	CA	Guardian XL "U" Lock	3
D	CA	Guardian XL "U" Lock	2
D	FL	Guardian XL "U" Lock	3
D	FL	Guardian XL "U" Lock	3

This report presents details. Each row represents an individual order. There are many orders from each of the regions for different locks. But because there is no summary information, it is nearly impossible to get any useful information out of a report like this.

The next logical step is to group the data in some way. You can group it by region, or by product line. The following section shows a look at both of these options.

16.2.2 Report of order data - grouped by region

This report uses the data seen in the first report, but here the data is grouped by region. All the orders in each region are grouped together, but each regional group contains orders for different types of locks. Because the groups contain different kinds of data, summarizing the Quantity field will determine the total number of locks sold per Region, but not the total of each type.

1 2 3 4 5				
RH				
PH				
GH1				
D	1	AL	Guardian ATB Lock	3
D		AL	Guardian Chain Lock	1
D		AL	Guardian Chain Lock	1
D		AL	Guardian Mini Lock	2
D		AL	Guardian XL "U" Lock	2
GF1				
GH1		2	BC	
D		BC	Guardian "U" Lock	2
D		BC	Guardian ATB Lock	3
D		BC	Guardian ATB Lock	1
D		BC	Guardian Chain Lock	2
D		BC	Guardian Mini Lock	2
GF1				
GH1		3	CA	
D		CA	Guardian "U" Lock	2
D		CA	Guardian "U" Lock	2
D		CA	Guardian "U" Lock	1
D		CA	Guardian ATB Lock	2
D		CA	Guardian ATB Lock	2
D		CA	Guardian Chain Lock	1
D	4	CA	Guardian Chain Lock	3
D		CA	Guardian Chain Lock	2
D		CA	Guardian Chain Lock	1
D		CA	Guardian Chain Lock	1
D		CA	Guardian Mini Lock	3

Each region group contains orders for different kinds of locks.

16.2.3 Report of order data - grouped by product

This report groups the data by product. Each group displays all the orders for a specific product. At first it appears that this might be useful, but then it becomes clear that each product group includes orders from several different regions.

The information is helpful, and it brings you closer to your goal, but you are still a long way from having the information you need.

1 2 3 4 5				
RH				
PH				
GH1		<u>Region</u>	<u>Product Name</u>	<u>Quantity</u>
D		Guardian "U" Lock		
D	1	BC	Guardian "U" Lock	2
D		CA	Guardian "U" Lock	2
D		CA	Guardian "U" Lock	2
D		CA	Guardian "U" Lock	1
GF1				
GH1		Guardian ATB Lock		
D		AL	Guardian ATB Lock	3
D	2	BC	Guardian ATB Lock	3
D		BC	Guardian ATB Lock	1
D		CA	Guardian ATB Lock	2
D		CA	Guardian ATB Lock	2
GF1				
GH1		Guardian Chain Lock		
D		AL	Guardian Chain Lock	1
D	3	AL	Guardian Chain Lock	1
D		BC	Guardian Chain Lock	2
D		CA	Guardian Chain Lock	1
D		CA	Guardian Chain Lock	3
D		CA	Guardian Chain Lock	2
D		CA	Guardian Chain Lock	1
D	4	CA	Guardian Chain Lock	1
GF1				
GH1		Guardian Mini Lock		
D		AL	Guardian Mini Lock	2
D		BC	Guardian Mini Lock	2

Each product group contains orders for many regions.

16.2.4 Report of order data - grouped by region and product

This report is the logical next step. If the By Region report contains multiple products in each region group, and the By Product report contains multiple regions in each product group, then it seems to make sense to combine the two. Doing that, you group first by Region and then by Product.

	<u>Region</u>	<u>Product Name</u>	<u>Quantity</u>
RH			
PH			
GH1	AL		
GH2	Guardian ATB Lock		
D	AL	Guardian ATB Lock	3
GF2			
GH2	Guardian Chain Lock		
D	AL	Guardian Chain Lock	1
D	AL	Guardian Chain Lock	1
GF2			
GH2	Guardian Mini Lock		
D	AL	Guardian Mini Lock	2
GF2			
GH2	Guardian XL "U" Lock		
D	AL	Guardian XL "U" Lock	2
GF2			
GF1			
GH1	BC		
GH2	Guardian "U" Lock		
D	BC	Guardian "U" Lock	2
GF2			
GH2	Guardian ATB Lock		
D	BC	Guardian ATB Lock	3
D	BC	Guardian ATB Lock	1
GF2			
GH2	Guardian Chain Lock		
D	BC	Guardian Chain Lock	2
GF2			

Each group contains orders for one product for one region. But the data is all spread out and remains difficult to analyze. This information is useful, and with a little work you can use a report like this to get the comparison information you need. However, a Cross-Tab offers a better solution.

16.2.5 Order data in a Cross-Tab object

With Cross-Tabs, all the information you need is provided in a compact format. The report shows the products sold in each region and what the unit sales were. It is easy to see, for example, that Guardian XL "U" Locks are not popular at all in BC but they are the biggest seller in Florida, or that Alabama is being outsold by California in every lock category.

	1	2	3	4	5
RH					
	AL	BC	CA	FL	Total
Guardian "U" Lock	0	2	5	0	7
Guardian ATB Lock	3	4	4	0	11
Guardian Chain Lock	2	2	8	0	12
Guardian Mini Lock	2	2	3	3	10
Guardian XL "U" Lock	2	0	5	6	13
Total	9	10	25	9	53
PH					

In this Cross-Tab:

- Product names make up the row headings.
- Regions make up the column headings.
- The value at each row/column intersection is the sum of all the orders for a particular product for a particular region; for example, the total number of Guardian Mini Locks purchased in British Columbia.
- The total at the end of each row is the total of all of the purchases for one product in all regions; for example, the total number of Guardian ATB Locks purchased in Alabama, British Columbia, California, and Florida combined.
- The total at the bottom of each column is the total number of all kinds of locks ordered in one region; for example, the number of locks of all kinds purchased in California.
- The total in the bottom right corner is the grand total showing the total unit sales of all five locks in all four regions.

The report is compact, and enables you to compare your customers' purchasing habits quickly.

16.3 Creating a Cross-Tab report

This section provides you with the steps to create a Cross-Tab object in a new report and how to add a Cross-Tab object to an existing report.

Things to keep in mind when using Cross-Tab objects:

- You can drag report fields into or out of Cross-Tab objects.
- You can have multiple rows, columns, and summarized fields.
- You can use print-time formulas as your rows or columns.
- You can use Running Total fields as your summarized field.
- You can include a group sort (top or bottom N) on the rows in your Cross-Tab.

Note

Group sorting cannot be applied to columns.

- You can insert as many Cross-Tab objects in a report as you need.
- You can insert the Cross-Tab into either the Report Header or Footer, or the Group Header or Footer.
- You can place Cross-Tab objects in subreports. This is useful when you want to refer to the results from another report.

16.3.1 To create a new cross-tab report

Procedure

On the Start Page, click [Cross-Tab Report Wizard](#).

The Cross-Tab Report Creation Wizard appears.

16.3.2 To add a Cross-Tab to an existing report

Procedure

1. Open your report.

This example uses the Group By Intervals.rpt included in the Feature Examples directory.

2. Click the [Design](#) tab.
3. On the [Insert](#) menu, click [Cross-Tab](#).

→ Tip



Another way to do this is to click the Insert Cross-Tab button on the Insert Tools toolbar.

An object frame appears with the Arrow cursor as you drag the Cross-Tab object into the report.

4. Place the object frame in an empty area in the [Group Header #1](#) and click to release it.

An empty Cross-Tab object appears in your report. You can drag fields from your report or from the Field Explorer into your Cross-Tab. This example uses Customer Name, Country, and Last Year's Sales from the Customer table.

5. In the Field Explorer, select [Customer Name](#) and drag it to the Cross-Tab.

This field will be your column value.

6. Drag the field to the upper-right-hand corner of the Cross-Tab object.

You'll see a placement arrow to show you where the field will appear when you drop it.

	Total
Total	Insert Summary

Once you have clicked to release the field, it appears to the right of the column that you selected.

	Total	Column #1
Total	Insert Summary	Insert Summary

- In the Field Explorer, select *Country* and drag it to the Cross-Tab.

This field will be your row value.

- Drag the field to the lower-left-hand corner of the Cross-Tab object, and click to release it.

	Total	Column #1
Total	Insert Summary	Insert Summary
Row #1 Name	Insert Summary	Insert Summary

- Finally, drag *Last Year's Sales* to the *Insert Summary* cells of the Cross-Tab, and click to release it.

→ Tip

Notice that all of the cells change color when you position the field over them.

	Total	Column #1
Total	Year's Sales	Year's Sales
Row #1 Name	Year's Sales	Year's Sales

- Click the *Preview* tab to see your Cross-Tab.

16.3.3 Specifying the data source

Procedure

- On the Data screen, locate the data source you want to use.

This example uses the Xtreme Sample Database.

For information on how to select a data source, see [Selecting the data source \[page 98\]](#).

- Select the tables that contain the fields to include in the report.

→ Tip

Use the Ctrl-click combination to pick a non-continuous range of fields and the Shift-click combination to pick a continuous list of fields.

This example uses Customer, Orders, Orders Detail and Product.

3. Click the > arrow to add the selected tables.
4. Click [Next](#).

The Link screen appears.

16.3.4 Modifying the links

Procedure

1. In this example, check to make sure that the Customer table is linked to the Product table via the Orders and Orders Detail tables.

If you do not like the linking that Crystal Reports has automatically suggested, change it by clicking Clear Links, and then select the fields that you would like to link together.

2. Click [Next](#).

The Cross-Tab screen appears.

16.3.5 Adding a chart

Procedure

1. Select the kind of chart you want to see in your report:
 - Bar Chart
 - Line Chart
 - Pie Chart

ⓘ Note

If you don't want to add a chart, leave No Chart selected and skip to [Selecting records \[page 356\]](#).

2. Change the default charting information to suit your report.

For more information about creating a group chart, see [Charting on summary or subtotal fields \(Group layout\) \[page 310\]](#).

3. Click [Next](#).

The Record Selection screen appears.

16.3.6 Selecting records

Context

Use this screen to apply selection conditions. This limits the results that are returned in your report to only those records that you are interested in.

Procedure

1. Select the field to apply selection to.

For this example choose Region.

2. Click the > arrow.

The field is added to the Filter Fields area and the filter types list appears below it.

3. From the filter types list choose a filter method.

For this example, choose "is one of" as the filter method.

4. In the filter values list that appears, select the value(s) you want to filter on and click [Add](#).

For this example, choose AZ, CA, and OR from the drop-down list.

The report will provide you with a breakdown and summary of accessory and bicycle sales for Arizona, California, and Oregon.

5. Click [Next](#).

The Grid Style screen appears.

16.3.7 Defining the structure of the Cross-Tab

Context

Add fields from the Available Fields area to the Rows, Columns, and Summary Fields areas.

Procedure

1. From the [Customer](#) table, select [Region](#); then click the > arrow next to the Rows area.

The Region field is added to the Rows area.

→ Tip

You can also add a field to the different Cross-Tab areas by selecting it and dragging it to the Columns, Rows, or Summary Fields areas.

2. From the *Product* table, select *Product Class*; then click the > arrow next to the Columns area.
The Product Class field is added to the Columns area.
3. From the *Orders* table, select *Order Amount*; then click the > arrow next to the Summary Fields area.
The Orders Amount is added to the Summary Fields area.
4. Select the summary operation you want to perform on *Order Amount* from the list beneath the Summary Fields area.
5. Click *Next*.
The Chart screen appears.

16.3.8 Applying a predefined style and finishing the report

Procedure

1. Select a style.
For this example choose Original.
2. Click *Finish*.
The Cross-Tab report appears in the Preview tab.
3. If you do not want the Cross-Tab to appear in the *Report Header*, click the *Design* tab, and move the Cross-Tab to the *Report Footer*.

Note

You can also place your Cross-Tab in the Group Header or Footer.

Note

If you place the Cross-Tab in the Report Footer, unsuppress the footer by right-clicking in the Report Footer's gray area and selecting Don't Suppress.

4. Click the *Preview* tab to view the report.

16.3.9 Adding a Cross-Tab to an existing report using the Cross-Tab Expert

16.3.9.1 To add a Cross-Tab to an existing report using the Cross-Tab Expert

Context

In this example, a Cross-Tab is added to an existing report to show the sales in North America compared to the rest of the world. However, instead of defining the report by dragging fields into it, you will use the Cross-Tab Expert.

Procedure

1. Open your report.

This example uses the Group By Intervals.rpt included in the Feature Examples directory.

2. Click the *Design* tab.
3. On the *Insert* menu, click *Cross-Tab*.

→ Tip



Another way to do this is to click the Insert Cross-Tab button on the Insert Tools toolbar.

An object frame appears with the Arrow cursor as you drag the Cross-Tab object into the report.

4. Place the object frame in an empty area in the *Report Header* and click to release it.

An empty Cross-Tab object appears in your report.

5. Right-click the Cross-Tab and select *Cross-Tab Expert* from the shortcut menu.

The Cross-Tab Expert appears. This dialog box is made up of three tabs: Cross-Tab, Style, and Customize Style.

16.3.9.2 Defining the structure of the Cross-Tab

Context

Add fields to the Rows, Columns, and Summarized Fields areas.

Procedure

1. From the *Available Fields* list, select *Country*; then click the > arrow next to the Rows area.

The Country field is added to the Rows area.

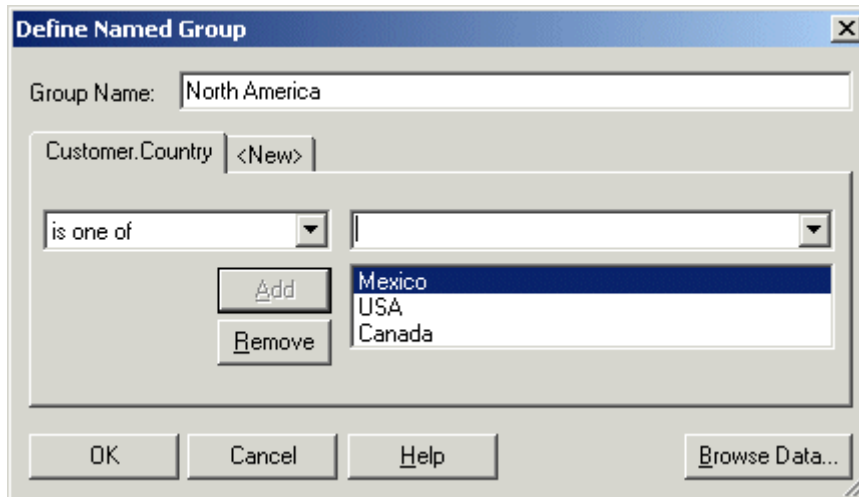
→ Tip

You can also add a field to the different Cross-Tab areas by selecting it and dragging it to the Columns, Rows, or Summarized Fields areas.

2. From the *Available Fields* list, select *Last Year's Sales*; then click the > arrow next to the Summarized Fields area.

The Last Year's Sales field is added to the Summarized Fields area.

3. Apply additional structure:
 - Click *Country* in the Rows area.
 - Click *Group Options*.
 - Select *in specified order* from the drop-down list.
 - Type *North America* for the name of the Named Group.
 - Click *New*.
 - Choose *is one of* from the drop-down list.
 - Select *Canada*, *Mexico*, and *USA* from the drop-down list.



- Click *OK* to close the Define Named Group dialog box.
- Click *OK* to close the Cross-Tab Group Options dialog box.

16.3.9.3 Choosing a predefined style

Procedure

1. Click the [Style](#) tab.
2. Select a style.

16.3.9.4 Applying a custom style

Procedure

1. Click the [Customize Style](#) tab.
2. Customize the style as desired.

16.3.9.5 Finishing the Cross-Tab

Procedure

1. Click [OK](#).
2. On the [Report](#) menu, click [Refresh Report Data](#).

The updated report appears.

16.4 Working with Cross-Tabs

This section describes ways you can work with a Cross-Tab once you've added it to your report.

16.4.1 Showing values as percentages

Procedure

1. Right-click the blank top-left area of a Cross-Tab and select [Cross-Tab Expert](#) from the shortcut menu.
The Cross-Tab Expert appears.
2. On the [Cross-Tab](#) tab, select a summarized field; then click [Change Summary](#).

3. In the Edit Summary dialog box, select the *Show as percentage of* option, and choose the summary field you want to base the percentage on.
4. Choose *Row* or *Column* depending on whether you want your percentage values to be calculated horizontally (row) or vertically (column).
5. Click *OK* to close the Edit Summary dialog box.
6. Click *OK* to return to your Cross-Tab.

Summarized field values are now displayed as percentages on your Cross-Tab.

Results

→ Tip

You can also access the Edit Summary dialog box directly from your Cross-Tab by selecting the summary field and choosing Edit Summary from the shortcut menu.

16.4.2 Abbreviating large summarized fields

Context

Because the values in a Cross-Tab's summarized fields are often very large, Crystal Reports lets you abbreviate such values.

ⓘ Note

To complete this procedure, the report you're working with must include the custom function called *cdFormatCurrencyUsingScaling*. The sample report called `Custom Functions.rpt` includes this function.

Procedure

1. If there is no cross-tab object in the report, insert one in the Report Header.
Use Customer.Country for the Row, and Orders.Order Amount for the summarized field.
2. Right-click the summarized field you want to abbreviate and choose *Format Field* from the shortcut menu.
The Format Editor appears.
3. On the Common tab, click the conditional formatting button adjacent to *Display String*.
4. In the Functions tree of the Format Formula Editor, select *cdFormatCurrencyUsingScaling* from the Custom Functions folder.
5. Complete the custom function's arguments as follows:

```
cdFormatCurrencyUsingScaling (CurrentFieldValue,1,"K","M")
```

- As the first argument, choose [CurrentFieldValue](#) from the Formatting Functions folder of the Functions tree. This argument defines the actual value to be summarized.
- As the second argument, enter the number of decimal places you want to display on your Cross-Tab.
- As the third and fourth arguments, enter the strings (surrounded by quotation marks) that you want to display as thousands and millions symbols.



6. Click [Check](#) to identify any errors in the formula.
7. Fix any syntax errors the Formula Checker identifies.
8. When the formula has the correct syntax, click [Save and close](#) on the Formula Workshop toolbar.

You return to the Format Editor dialog box.

9. Click [OK](#) to return to your Cross-Tab.

Your summarized field values appear abbreviated as defined.

Results

Note

You can create your own custom function, or you can copy and modify the provided sample to abbreviate a summarized field. In general, using a custom function is not a requirement of working with the Display String feature. You can write any Display String formula you need as long as its return value is a string.

16.4.3 Customizing row/column labels

Context

By default, row and column labels are derived from the data on which you base your Cross-Tab. You can, however, customize row and column labels.

Procedure

1. Right-click the row or column whose label you want to customize and choose [Format Field](#) from the shortcut menu.

The Format Editor appears.

2. On the Common tab, click the conditional formatting button adjacent to [Display String](#).
3. In the Format Formula Editor, create formula text that describes the custom names you want to use as well as the conditions under which these names should be applied.

For more information about how to do this, see [Working with conditional formatting \[page 287\]](#).

4.  Click [Check](#) to identify any errors in the formula.
5. Fix any syntax errors the Formula Checker identifies.
6. When the formula has the correct syntax, click [Save and close](#) on the Formula Workshop toolbar.
You return to the Format Editor dialog box.
7. Click [OK](#) to return to your Cross-Tab.
Your customized row and column names appear when the conditions you set are matched.

16.4.4 Using running totals in Cross-Tabs

Running Total fields can be useful as summary fields in Cross-Tab objects. Create your Cross-Tab as shown in [Creating a Cross-Tab report \[page 352\]](#), but choose a running total field as your Summary Field.

Note

You cannot use Running Total fields as columns or rows in a Cross-Tab.

If you want a running total that totals down the Cross-Tab's columns, you must sort the records in the Cross-Tab by Column and then Row.

16.4.4.1 To total down columns

Procedure

1. Right-click the blank top-left area of a Cross-Tab and select [Cross-Tab Expert](#) from the shortcut menu.
2. On the [Cross-Tab](#) tab of the Cross-Tab Expert, make sure you have added a Running Total field to the [Summarized Fields](#) area.
3. Click [OK](#) to return to the Cross-Tab in your report.
4.  Click [Record Sort Expert](#) and sort your Cross-Tab by the field you designated as a Column in the Cross-Tab Expert.
5. Create a second sort on the field you designated as a Row in the Cross-Tab Expert.

Results

If you want a running total that totals across the Cross-Tab's rows, you must sort the records in the Cross-Tab by Row and then Column.

16.4.4.2 To total across rows

Procedure

1. Repeat steps 1 to 3 of the previous procedure.
2.  Click *Record Sort Expert* and sort your Cross-Tab by the field you designated as a Row in the Cross-Tab Expert.
3. Create a second sort on the field you designated as a Column in the Cross-Tab Expert.

Results

If you add a Group Sort (top or bottom N sort) based on a Running Total field, the sort is carried out on the running total values as shown in the Cross-Tab's Row/Column Total areas—it is not based on the summary field's totals. Also, in this case, the running total results appear correctly only in the total data and not in the cell data.

16.4.5 Printing Cross-Tabs that span multiple pages

When you create a Cross-Tab that is wider or longer than the specified page size, the program automatically spans the printing across enough pages to accommodate the size of the Cross-Tab. In the Preview tab, a line will appear at each page break. For ease in reading, column headings will be repeated on subsequent pages. Row headings can also be repeated using the Keep Columns Together option.

16.4.5.1 To repeat row labels

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select *Cross-Tab Expert* from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the *Customize Style* tab.
3. Select the *Repeat Row Labels* check box, if desired.

Results

Crystal Reports lets you designate report objects that don't expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, to be repeated on each additional horizontal page that a Cross-Tab creates. For more information, see [Repeating report objects on horizontal pages \[page 283\]](#).

16.5 Formatting Cross-Tabs

Crystal Reports has powerful formatting capabilities that can be applied to Cross-Tabs. The following sections describe some key procedures.

By applying such formatting as background color, borders, and fonts, you can emphasize important data and create professional-looking, easy-to-understand Cross-Tabs. For more information, see [Formatting properties \[page 270\]](#).

You can also use the Highlighting Expert to apply conditional formatting to Cross-Tab cells. To access the Highlighting Expert, right-click the cell you want to format and, on the shortcut menu, click Highlighting Expert. For more details, see [Using the Highlighting Expert \[page 294\]](#).

16.5.1 Changing width, height, and alignment of Cross-Tab cells

Procedure

1. Click a cell within the Cross-Tab to activate the sizing handles.
2. Click and drag one of the sizing handles to expand the width or height of the cell.
3. Click an alignment option on the Formatting toolbar to change the alignment of the data in the cell.

You can choose between left, right, center, and justified alignment.

Results

Note

Changing one cell affects all like cells. For example, changing the width of one summary field cell changes all other summary field cells at the same time.

16.5.2 Formatting background color of entire rows/columns

Context

Use background colors to emphasize rows or columns within your Cross-Tab.

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select *Cross-Tab Expert* from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the *Customize Style* tab.
3. Click the row (in the *Rows* area) or column (in the *Columns* area), and select a color from the *Background Color* drop-down list.
4. Click *OK* to return to the Cross-Tab.
The row/column is formatted as specified.

16.5.3 Formatting fields individually

Procedure

1. Right-click the field you want to format and choose *Format Field* from the shortcut menu.
The Format Editor appears.
2. In the Format Editor, make font, background, borders, numbering, currency symbols, and printing characteristics choices.
3. Click *OK* to return to the Cross-Tab.
The field is formatted as specified.

16.5.4 Formatting several fields at one time

Procedure

1. Use the *Shift*-click method to highlight the desired fields.
2. Right-click any one of them and choose *Format Objects* from the shortcut menu.
The Format Editor appears.
3. In the Format Editor, make font, background, borders, numbering, currency symbols, and printing characteristics choices.

4. Click [OK](#) to return to the Cross-Tab.

The fields are formatted as specified.

16.5.5 Suppressing Cross-Tab data

This section provides you with the steps required to suppress data in your report. You can suppress:

- Empty rows and columns.
- Row and column grand totals.
- Subtotals and their labels.

16.5.5.1 To suppress empty rows and columns

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select [Cross-Tab Expert](#) from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the [Customize Style](#) tab.
3. Select either the [Suppress Empty Rows](#) or [Suppress Empty Columns](#) check box.
4. Click [OK](#).

Now, when you print the report, empty rows and/or columns will not appear.

16.5.5.2 To suppress row and column grand totals

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select [Cross-Tab Expert](#) from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the [Customize Style](#) tab.
3. Select either the [Suppress Row Grand Totals](#) or [Suppress Column Grand Totals](#) check boxes.
4. Click [OK](#).

16.5.5.3 To suppress subtotals and their labels

Context

If you have more than two groups in your Cross-Tab you can suppress the subtotal and label for one of them.

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select *Cross-Tab Expert* from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the *Customize Style* tab.
3. Click the field whose subtotal you want to suppress.
The Suppress Subtotal and the Suppress Label check boxes become active.
4. In the Group Options area, select the *Suppress Subtotal* check box.
5. Click the *Suppress Label* check box to suppress the label associated with subtotal.
6. Click *OK*.

16.5.6 Displaying summarized fields horizontally

Context

If you have two or more summarized fields in your Cross-Tab, you can display their values horizontally instead of vertically (the default).

Procedure

1. Right-click the blank top-left area of the Cross-Tab and select *Cross-Tab Expert* from the shortcut menu.
The Cross-Tab Expert appears.
2. Click the *Customize Style* tab.
3. In the Summarized Fields area, select *Horizontal*.
4. Click the *Show Labels* check box if you want to display labels for the summarized fields.
Labels are displayed in the direction you chose for the summarized fields.
5. Click *OK*.

16.6 Advanced Cross-Tab features

There are several ways to add even more information to your Cross-Tabs. This section covers several advanced Cross-Tab features.

Related Information

[Calculated Members \[page 369\]](#)

[Embedded Summaries \[page 373\]](#)

16.6.1 Calculated Members

You can add rows or columns to your Cross-Tab by inserting a [Calculated Member](#). These rows or columns can contain custom calculations (for example, the difference in sales between two regions) or can be used entirely for formatting purposes (for example, inserting a blank row every three lines to increase readability).

You have complete flexibility in designing your own calculation formulas and can choose whether they appear only once or repeat regularly in the Cross-Tab.

Related Information

[Calculation Formulas \[page 371\]](#)

[Header Formulas \[page 371\]](#)

[Insertion Formulas \[page 373\]](#)

[Value Formulas \[page 372\]](#)

16.6.1.1 To add a Calculated Member to your Cross-Tab

Procedure

1. In your Cross-Tab, right-click the row or column header you would like to use as the first value of your [Calculated Member](#).
For example, if you wanted to create a [Calculated Member](#) for the difference between Sales and Sales Returns, the desired formula would be Sales - Sales Returns. In this example, you would right-click the Sales header.
2. On the shortcut menu, point to [Calculated Member](#), and select the chosen value as your first value.
In this example, you would select Sales as your first value.

3. In your Cross-Tab, right-click the row or column header you would like to use as the second value of your *Calculated Member*.
In this example, you would select Sales Returns as your second value.
4. On the shortcut menu, point to *Calculated Member*, and select the desired calculation.
In this example, you would select *Difference of "Sales" and "Sales Returns"* as the desired calculation.

Results

A row or column that displays the results of the selected calculation appears in your Cross-Tab .

16.6.1.2 To add a blank row or column to your Cross-Tab

Procedure

1. In your Cross-Tab, right-click the header immediately preceding the desired location of your blank row or column.
2. On the shortcut menu, point to *Calculated Member*, and depending on whether you have clicked a row or a column, click either *Insert Row* or *Insert Column*.

Results

A blank row or column appears in your Cross-Tab.

16.6.1.3 To change the processing order of Calculated Members

Procedure

1. Right-click the top-left corner of your Cross-Tab.
2. On the shortcut menu, point to *Advanced Calculations*, and click *Calculated Member*.
The *Cross-Tab Calculated Members Expert* appears.
3. In the *Calculated Members* box, use the arrows to change the processing order of the *Calculated Members*.
4. Click OK to save your changes and return to your report.

Results

The Cross-Tab recalculates the values of the *Calculated Members*.

Note

Changing the processing order of *Calculated Members* does not cause a database refresh.

16.6.1.4 Calculation Formulas

Calculation Formulas are mathematical formulas that Crystal Reports uses to determine *Calculated Member* cell values. You can choose between several pre-defined formulas or design your own in the *Formula Workshop*.

The pre-defined formulas are as follows:

- Sum of
- Difference of
- Product of
- Quotient of

Crystal Reports is equipped with a variety of functions to help you design your own formulas.

16.6.1.4.1 To edit a Calculation Formula

Procedure

1. In the *Calculated Member* you would like to edit, right-click any cell except for the header.
2. On the shortcut menu, point to *Calculated Member*, and click *Edit Calculation Formula*.
The *Formula Workshop* appears.
3. In the *Formula Workshop*, enter the desired *Calculation Formula*.
4. Click *Save and close* to return to your report.

Results

The *Calculated Member* displays the results of the new *Calculation Formula*.

16.6.1.5 Header Formulas

Header Formulas determine the visible row or column header for a *Calculated Member*.

16.6.1.5.1 To edit a Header Formula

Procedure

1. Right-click the header of the *Calculated Member* you would like to edit.
2. On the shortcut menu, point to *Calculated Member*, and click *Edit Header Formula*.
The *Formula Workshop* appears.
3. In the *Formula Workshop*, enter your desired header name in quotations.

Note

You can also control your header formulaically. A *Header Formula* must return a string.

4. Click *Save and close* to return to your report.

Results

The *Calculated Member* header displays the result of the *Header Formula*.

16.6.1.6 Value Formulas

If you want to reference a *Calculated Member* in a formula, you must assign a value to it. *Value Formulas* assign values to *Calculated Members* for this purpose.

16.6.1.6.1 To edit a Group Value Formula

Procedure

1. Right-click the header of the *Calculated Member* you would like to edit.
2. On the shortcut menu, point to *Calculated Member*, and depending on whether you have clicked a row or a column, click *Edit Row Value Formula* or *Edit Column Value Formula*.
The *Formula Workshop* appears.
3. In the *Formula Workshop*, enter your desired value.

Note

You can also control your value formulaically. A *Value Formula* must be of the same type as the row/column grouping field that it is inserted into.

4. Click *Save and close* to return to your report.

Results

The *Calculated Member* retains the result of your *Value Formula*.

Note

If you have not edited the *Header Formula* for a *Calculated Member*, the visible header displays the result of the *Value Formula*. When you edit the *Header Formula*, it overrides the *Value Formula* as the visible header.

16.6.1.7 Insertion Formulas

Insertion formulas determine where a *Calculated Member* appears in a Cross-Tab. In most cases, a *Calculated Member* appears only once. However, you can set it to appear in multiple locations or in a repeating pattern.

16.6.1.7.1 To edit an Insertion Formula

Procedure

1. Right-click the header of the *Calculated Member* you would like to edit.
2. On the shortcut menu, point to *Calculated Member*, and click *Edit Insertion Formula*.
The *Formula Workshop* appears.
3. In the *Formula Workshop*, enter your desired formula.
For example, the following formula inserts a *Calculated Member* after the country field for Canada:

```
GetColumnGroupIndexOf(CurrentRowIndex) = 1 and  
GridRowColumnValue("Customer.Country") = "Canada"
```

4. Click *Save and close* to return to your report.

Results

The *Calculated Member* appears in the locations specified by the *Insertion Formula*.

16.6.2 Embedded Summaries

You can add an additional calculation to your Cross-Tab by inserting an *Embedded Summary*.

Unlike *Calculated Members*, *Embedded Summaries* do not add rows or columns to your Cross-Tab. Instead, they cause additional calculations to appear within each cell of the Cross-Tab.

For example, in a report showing regional sales, you could insert a calculation for each region that shows the percentage of total sales for the country.

16.6.2.1 To add an Embedded Summary to your Cross-Tab

Procedure

1. In your Cross-Tab, right-click any cell except for a header.
2. On the shortcut menu, point to *Embedded Summary*, and click *Insert Embedded Summary*.
A blank *Embedded Summary* appears in the Cross-Tab.
3. In the *Embedded Summary*, right-click the text that says "Edit this formula."
4. Point to *Embedded Summary*, and click *Edit Calculation Formula*.
The *Formula Workshop* appears.
5. In the *Formula Workshop*, enter the desired *Calculation Formula*.
6. Click *Save and close* to return to your report.

Results

The *Embedded Summary* displays the results of the new *Calculation Formula*.

16.6.2.2 To change the processing order of Embedded Summaries

Context

If you have multiple *Embedded Summaries* in your Cross-Tab, the order in which they are calculated can affect your results. You can change the processing order of *Embedded Summaries* in the *Embedded Summaries dialog box*.

Procedure

1. Right-click the top-left corner of your Cross-Tab.
2. On the shortcut menu, point to *Advanced Calculations*, and click *Embedded Summary*.
The *Embedded Summaries dialog box* appears.
3. In the *Summaries* box, use the arrows to change the processing order of the summaries.
4. Click *OK* to save your changes and return to your report.

Results

The Cross-Tab recalculates the values of the *Embedded Summaries* in the new processing order.

Note

Changing the processing order of *Embedded Summaries* does not cause a refresh from the database.

17 Reporting on Universes and managed connections

This section tells you how to access Business Objects universes and managed connections through Crystal Reports, how to build queries, and how to filter the data that the Business Objects Query Panel returns for you to use in Crystal Reports.

DSL data source types supported in Crystal Reports:

DSL data source type, and usage hints	Driver name	Supported since
Unx universes on relational DB 3371775 	Universe (unx)	BI 4.3 SP04
Managed OLAP connections to BW, BW4HANA, S4HANA 3556385 	SAP BW (DSL)	BI 2025, and BI 4.3 SP05

Other DSL data source types, e.g., OLAP connections to HANA, will also be investigated and could be supported in later releases.

17.1 Prerequisite: Install Microsoft WebView2

Microsoft WebView2 Runtime is required when creating / updating a report on Unx Universe or Managed OLAP connection, e.g., to display Query Panel, BW query selection dialog, or parameter prompts dialog in Crystal Reports Designer.

Simply search for “WebView2 Download”, download from **official Microsoft website**, and install.

Pick one of following installer:

- Evergreen Standalone Installer (choose “x64” version)
- Evergreen Bootstrapper

17.2 Connecting to a universe

You access the Business Objects Query Panel in Crystal Reports through the Database Expert. Once you have selected Universes from the Create New Connection node, you are prompted to log onto SAP BusinessObjects Business Intelligence platform, after which you can select a universe and design your query.

Crystal Reports can connect to multilingual universes and report off of them, but it cannot take advantage of the multilingual attributes. Crystal Reports will display multilingual field names and other metadata content only in the primary language of the universe.

You can only base new documents and queries on universes for which you have been granted the custom right, [Create / Edit Query](#).

Note

If you are a user of Web Intelligence and you are using the Query Panel in Crystal Reports, you will encounter some differences in behavior from what you might expect in Web Intelligence.

17.2.1 To connect to a universe

Procedure

1. In Crystal Reports, on the [File](#) menu, click [New](#), and then click [Blank Report](#).
2. In the Database Expert, double-click [Create New Connection](#).
3. Double-click [Universes \(unx\)](#) or [Universes](#) (for unv) to view a list of universes you have the right to base new reports on.

Note

- The [Universes](#) driver currently establishes connections with the older, legacy unv universes. Please be informed that support for unv Universe connections will be discontinued starting from the SAP BusinessObjects BI 2025 release.
- Use the new [Universes \(unx\)](#) driver whenever possible. For usage hint and restrictions, see <https://launchpad.support.sap.com/#/notes/3371775> 

4. Log on to SAP BusinessObjects Business Intelligence platform if you have not done so already.
5. Select the universe that you want to base your query on, and then click [Open](#).

The Business Objects Query Panel appears.

17.3 Connecting to a managed OLAP connection

Context

SAP BW (DSL) driver can connect to managed OLAP connections on BW, BW4HANA or S4HANA. Other managed OLAP connections are being investigated and could be added in later releases.

You access the Business Objects Query Panel in Crystal Reports through the Database Expert. Once you have selected SAP BW (DSL) driver from the Create New Connection node, you are prompted to log onto SAP BusinessObjects Business Intelligence platform. Then you can select a managed BW OLAP connection, choose a BW query, and design your DSL query.

Procedure

1. In Crystal Reports, on the *File* menu, click *New*, and then click *Blank Report*.
2. In the Database Expert, double-click *Create New Connection*.
3. Double-click *SAP BW (DSL)* driver.
4. Log on to SAP BusinessObjects Business Intelligence platform if you have not done so already.
5. **Select a BW query** dialog pops up, select a managed BW OLAP connection, then select the BW query that you want to base your query on, and click *OK*.

The Query Panel appears.

17.4 Defining the data selection for a query

You build queries in the Query Panel by using objects in a Business Objects universe. The objects in the universe are a graphical representation of the information available in a database. The objects in the universe are mapped to the table columns and rows in the database.

You can only base new documents and queries on universes for which you have been granted the custom right, *Create / Edit Query*.

The Query Panel is divided into several panes:

- The pane on the left shows a tree view of the objects that your universe contains. You cannot add new objects or edit existing objects in the Query Panel. For information about the types of objects that might appear in this pane, see [Quick reference to objects \[page 380\]](#).
- The Result Objects pane is where you place the objects that you want as part of your query.
- The Query Filters pane is where you place the objects that you want to use to filter the data that is returned from your universe. You can define your filter when you add an object to this pane.
- For queries on Unx universes, you can preview the query results in the Preview pane before using them in your report.

17.4.1 To build a simple query

Procedure

1. In the *Query Name* field, enter a name for your query.

Note

Crystal Reports uses the query name as the name of the SQL Command that it creates for your report. Once you have created your query in the Query Panel, you will see this name in the Available Data Sources and Selected Tables areas of the Database Expert.

2. Select an object in the left-hand pane and double-click it or drag it into the *Result Objects* pane.

Objects that appear in the Result Objects pane become fields that you report on in your Crystal report.

3. Repeat the previous step for each object that you want to include in the query.
4. Select an object in the left-hand pane, or in the *Result Objects* pane, and drag it into the *Query Filters* pane.

Add filters to the Query Filters pane to restrict the data your query returns based on the object that you selected. Filtering limits the data that a user can see and reduces the runtime of queries. For information about adding filters, see [Creating query filters \[page 383\]](#).

5. Click *OK*.
6. If your query contains prompts, select your prompt value(s) in the Enter Parameter Values dialog box.

For information about using the Enter Parameter Values dialog box, search for the dialog box by name in the Crystal Reports Online Help.

7. If your data source is secure, use the connection dialog box to log on to the database that your universe uses.

For information about using the supported connection dialog boxes, search for "Data source connectivity dialog boxes" in the Crystal Reports Online Help.

17.4.2 To sort objects

Procedure

1. On the top right corner of **Result Objects**, click *Sort* button.
The Sort dialog appears.
2. Click *Add*.
3. Select an object and then click *OK*.
4. Choose a Sort type: *Ascending* or *Descending*.
5. If you'd like to sort multiple objects, repeat step 2~3.
6. If there are multiple sortings defined, you can reorder them using up and down button.
7. Click *OK* to save and close the Sort dialog.
8. Specified sorting order will also show in *Result Objects* area, if relevant object is used.

17.4.3 To create a combined query

Procedure



Create a simple query and click *Add a combined query*.

A new pane is added to the Query Panel under the tree view. You can switch between your query definitions by clicking the nodes in this pane.

Note

The objects in the result panel of the first query are copied into the new query. The program does not copy existing query filters.

Note

When combining result objects of different datatypes from two queries into one query, the OK button may be grayed out, and you may not be able to generate a report. For example, Query1 contains a string and a number and Query2 contains a string and a date.

Before trying to combine the queries, ensure that the objects in both queries have the same datatype.

17.4.4 Quick reference to objects

An object is a named component that maps to data or a derivation of data in the database. For example, an object may represent a column in a database table, or it may be the summary of the values in a column.

You use objects in a query to retrieve data for your reports. For example, some of the objects in a human resources universe might be Names, Addresses, and Salaries.

Objects can represent different types of information.

Note

Object properties are defined in the BusinessObjects Designer, but are not taken into account by the Query Panel in Crystal Reports.

Note

Objects cannot be defined directly in the Query Panel. Use the BusinessObjects Designer to define objects in a universe.

Object	Examples	Description
<i>Dimension</i>	 Resort  Country  Service Line	This object retrieves the data that provides the basis for analysis in a report. Dimension objects typically retrieve character-type data (for example, customer names, resort names, or dates).

Object	Examples	Description
<i>Detail</i>	 Customer  Age  Phone Number  Address	This object provides descriptive data about a dimension. A detail is always attached to the dimension for which it provides additional information. For example, Age is a detail object that is associated with the Customer dimension. Address provides additional information on customers.
<i>Measure</i>	 Revenue  Number of guests  Future guests	This object retrieves numeric data that is the result of calculations on data in the database. For example, Revenue is the calculation of the number of items sold multiplied by item price. Measure objects are often located in a Measures class.

17.5 Editing an existing query

You can return to the Query Panel to change an existing query, provided you have been granted the custom right, [Create / Edit Query](#). You can add or remove an object, or you can add, edit, or remove a filter.

17.5.1 To edit an existing query

Procedure

1. In Crystal Reports, open a report created with a universe as a data source, go to the [Database](#) menu, and click [Query Panel](#).
2. Log on to SAP BusinessObjects Business Intelligence platform if you have not done so already.
For information about how to log on to SAP BusinessObjects Business Intelligence platform, see [Working with Enterprise folders \[page 628\]](#).
3. In the Query Panel, edit your query as necessary.
4. When you have finished editing your query, click [OK](#).
5. Click [OK](#) in the Database Expert to return to your report.

Changes that you made in your query are reflected in Crystal Reports.

Note

If you remove objects from your query that you have used in your Crystal report, you must remove these objects from the report.

17.6 Viewing the SQL behind a query

When you build a query, Crystal Reports automatically generates the SQL that corresponds to the query.

For query on Unv universe, equivalent SQL is saved as a Crystal SQL Command object.

For more information about SQL Command objects, see [Defining an SQL Command \[page 896\]](#).

17.6.1 To view the SQL when you create a query

Procedure

In the Query Panel, click [View SQL](#) (for Unv universe), or [View query script](#) (for Unx universe).

The SQL dialog box appears; it contains the SQL that constitutes your query. Use this option when you want to check the SQL as you create a query.

17.6.2 To view the SQL after you have created a query

Procedure

In Crystal Reports, open a report created with a universe as a data source, go to the [Database](#) menu, and click [Show SQL Query](#).

You can see the SQL in the Show SQL Query dialog box.

Or

In Crystal Reports, open a report created with a universe as a data source, go to the Database menu, and click Database Expert.

Once you are in the Database Expert, right-click your query in the Selected Tables list and select View Command. You can see the SQL in the View Command dialog box.

17.7 Query filters and prompts

Click a link to view one of the topics in this section:

- [Creating query filters \[page 383\]](#)
- [Building prompts \[page 384\]](#)
- [Combining query filters and prompts \[page 385\]](#)
- [Using And or Or to combine query filters \[page 385\]](#)

- [Quick reference to query filter operators \[page 386\]](#)
- [Editing and removing query filters \[page 387\]](#)

17.7.1 Creating query filters

You can use the following types of filters in a query:

- [Predefined filters](#)
These filters are created by your administrator.
- [Custom filters](#)
You define these filters while you are creating the query.
- [Prompts](#)
Prompts are dynamic filters that you define to display a question or a list of values; users can select different filter value(s) each time they refresh the report.

17.7.1.1 To add a predefined filter to a query

Procedure

1. Double-click the objects you want to use in your report, or drag them to the [Results Objects](#) pane.
For step-by-step instructions on selecting objects to build a query, see [Defining the data selection for a query \[page 378\]](#).
2. Drag a predefined filter to the [Query Filters](#) pane.

When you run the query, the data corresponding to the query filters you selected is returned to the report.

Results

Note

Predefined filters are created and edited by your administrator. As a user of the Query Panel, you cannot view the component parts of predefined filters, neither can you edit them.

17.7.1.2 To create a custom filter using the Filter Editor

Procedure

1. Select the object you want to filter and drag it to the [Query Filters](#) pane.

The Filter Editor appears. The name of the object you selected is displayed beneath the "Filtered object" text.

2. Select an Operator from the list.

For a description and example of each operator, see [Quick reference to query filter operators \[page 386\]](#).

3. Select [Constant](#) or [Value\(s\) from list](#).
4. Depending on the operator you selected, type a single value or multiple values in the [Type a value](#) field.
5. Click [OK](#) to confirm the properties of the custom filter.

The new filter displays in the Query Filters pane.

17.7.2 Building prompts

A prompt is a dynamic filter that displays a question every time you refresh the data in a report. You respond to prompts by typing or selecting the value(s) you want to view before you refresh the data. Crystal Reports retrieves only the values you specified from the database and returns those values to the report.

→ Tip

Prompts allow multiple users view a single report but specify a different sub-set of the database information. Prompts also reduce the time it takes for the data to be retrieved from the database.

17.7.2.1 To create a prompt

Procedure

1. Select the object you want to apply a prompt on and drag it to the [Query Filters](#) pane.

For example, if you want to allow users to specify the geographical region for the report, drag the Region field to the Query Filters pane.

The Filter Editor appears. By default, the name of the filter is the name of the filtered object.

2. Select an operator from the list.

See [Quick reference to query filter operators \[page 386\]](#).

ⓘ Note

You cannot use the following operators for prompts: Is null, Is not null, Both, and Except.

3. Under *Operator type*, select *Prompt*.

The Prompt text field shows the default message.

4. If you want to change the default message, edit it to word a question.

For example, you might ask "Which region do you want to see data for?"

5. Check *Select only from list* if you want your prompt to display a list of values from which users can select value(s).

Use this option to prevent users from typing a value that might not exist in the database.

6. Click *OK* to confirm the prompt.

The prompt appears at each report refresh.

17.7.3 Combining query filters and prompts

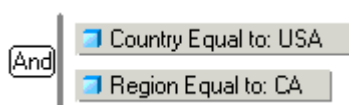
You can apply multiple filters and prompts on a single query.

17.7.3.1 To combine filters and/or prompts

Procedure

1. Create each query filter and/or prompt.

For step-by-step information on how to create filters and prompts, see [Creating query filters \[page 383\]](#) or [Building prompts \[page 384\]](#).



By default, the Query Panel combines the filters and prompts with the And operator. You can leave the And operator or change the operator to Or. See [Using And or Or to combine query filters \[page 385\]](#).

2. If necessary, change the operator to *Or* by double-clicking the *And* operator.

Or now displays as the operator. The prompts appear when you click OK, or when you refresh the report data.

17.7.4 Using And or Or to combine query filters

This table explains the difference between the And and the Or operators.

Retrieve this data	Example	Select
Data true for both filters.	Customers who ordered supplies in Q1 and in Q2 (the data you retrieve will include: customers who placed orders in both Q1 and Q2).	And
Data true for any one of the filters.	Customers who ordered supplies in: Q1 or Q2 (the data you retrieve will include: customers who placed orders in Q1 only; customers who placed orders only in Q2, and; customers who placed orders in both Q1 and Q2).	Or

17.7.5 Quick reference to query filter operators

The following table will help you select the operator you need to define a query filter.

Values to retrieve	Example	Select	Filter created
Values equal to a value you specify.	Retrieve data for the US only.	Equal to	<Country> Equal to US
Values different from a value you specify.	Retrieve data for all quarters except Q4.	Different from	<Quarter> Different from Q4
Values greater than a value you specify.	Retrieve data for customers aged over 60.	Greater than	<Customer Age> Greater than 60
Values greater than or equal to a value you specify.	Retrieve data for revenue starting from \$1.5M upward.	Greater than or equal to	<Revenue> Greater than or equal to 1500000
Values lower than a value you specify.	Retrieve data for exam grades below 40.	Less than	<Exam Grade> Less than 40
Values lower than or equal to a value you specify.	Retrieve customers whose age is 30 or less.	Less than or equal to	<Age> Less than or equal to 30
Values between two values you specify that includes those two values.	Retrieve the weeks starting at week 25 and finishing at 36 (including week 25 and week 36).	Between	<Weeks> Between 25 and 36
Values outside the range of two values you specify.	Retrieve all the weeks of the year, except for weeks 25 through 36 (week 25 and week 36 are not included).	Not between	<Weeks> Not Between 25 and 36

Values to retrieve	Example	Select	Filter created
Values that are the same as several values you specify.	Retrieve data for only the following countries: the US, Japan, and the UK.	In list	<Country> In list 'US; Japan; UK'
Values that are different from the multiple values you specify.	Don't retrieve data for the following countries: the US, Japan, and the UK.	Not in list	<Country> Not in list 'US; Japan; UK'
Values for which there is no value entered in the database.	Retrieve customers without children (the children column in the database has no data entry).	Is null	<Children> Is null
Values for which a value was entered in the database.	Retrieve customers with children (the children column in the database has a data entry).	Is not Null	<Children> Is not Null
Values that includes a specific string. Wildcard % can be used in the input string.	Retrieve customers whose date of birth is 1972.	Matches pattern	<DOB> Matches pattern, '72'
Values that don't include a specific string. Wildcard % can be used in the input string.	Retrieve customers whose date of birth is not 1972.	Different from pattern	<DOB> Different from pattern, '72'
Values that correspond to two values you specify.	Retrieve Telco customers who have both a fixed telephone and a mobile phone.	Both	<Account Type> Both "fixed" and "mobile". Unlike the "In List" operator, "Both" operator generates a combined query of type "Intersect"
Values that correspond to a single value and do not correspond to another value you specify.	Retrieve Telco customers who have a fixed telephone, but don't have a mobile phone.	Except	<Account Type> "fixed" Except "mobile". Unlike the "Not equal to" operator, "Except" operator generates a combined query of type "Minus"

17.7.6 Editing and removing query filters

You can edit and remove filters on queries.

17.7.6.1 To edit a query filter

Procedure

1. Double-click the filter you want to edit.
The Filter Editor appears.
2. Change the definition of the filter in the Filter Editor.
For information on how to define filters, see [Creating query filters \[page 383\]](#).
3. Click **OK** to confirm your changes.
The modified query appears in the Query Filters pane.

17.7.6.2 To remove a query filter

Procedure

Drag the filter you want to remove and drop it onto the left-hand pane.
The filter is removed from the query definition and no longer appears on the Query Filters pane.

17.8 Filtering data using subqueries and database ranking

This section explains how to work with more advanced types of query filters. It covers the following advanced query filters:

- Subqueries
- Database rankings

Subqueries and database rankings allow you to create filters that are much more powerful than standard query filters.

Note

Not supported yet when reporting on Unx universe, as of Crystal Reports 2020 SP04.

17.8.1 What is a subquery?

A subquery is a more flexible kind of query filter that allows you to restrict values in more sophisticated ways than with an ordinary query filter.

Subqueries are more powerful than ordinary query filters for the following reasons:

- They allow you to compare the values of the object whose values are used to restrict the query with values from other objects.
- They allow you to restrict the values returned by the subquery with a WHERE clause.

Subqueries allow you to pose complex questions that are difficult or impossible to formulate with simple query filters. For example: what is the list of customers and their associated revenue where the customer purchased a service that had previously been reserved (by any customer) in Q1 of 2003?

17.8.2 Building a subquery

You build a subquery in the Query Filters pane of the Query Panel.

Note

You can mix subqueries and other types of query filter in the Query Filters pane.

17.8.2.1 To build a subquery

Procedure

1. Add the objects that you want to appear in the query to the Result Objects pane.

2.  Select the object that you want to filter with a subquery, and then click [Add a subquery](#).

The subquery outline appears in the Query Filters pane. By default the object you selected appears as the Filter object and Filter By object. For more information on the Filter Object and Filter By object, see [Subquery parameters \[page 390\]](#).

3. To add a WHERE condition to the subquery, drag a query object to the white area of the subquery outline.

Note

You can use an existing subquery or standard query filter as a WHERE condition in a subquery. To do so, drag and drop the existing filter or subquery to the white area of the subquery outline.

4. Select the operator and values used to filter the object in the WHERE condition.

Note

For more information on query filter operators and values, see [Query filters and prompts \[page 382\]](#).

5. Click [Add a subquery](#) to add an additional subquery to the query filter.

By default the two subqueries are linked in an AND relationship. Click the AND operator to toggle between AND and OR.

In addition to linking subqueries in AND or OR relationships, you can nest them (create subqueries within subqueries) by dragging an existing subquery to the white area of the subquery outline. In this case the inner subquery becomes part of the WHERE condition of the outer subquery.

17.8.2.2 Subquery parameters

A subquery or set of subqueries contains the following parameters:

Parameter	Description
Filter Object(s)	The object whose values are used to filter the result objects. You can include more than one Filter Object. If you do, Crystal Reports concatenates the values of the objects you specify.
Filter By Object(s)	The object that determines which Filter Object values the subquery returns. You can include more than one Filter By object. If you do, Crystal Reports concatenates the values of the objects you specify.
Operator	The operator that specifies the relationship between the Filter object and the Filter By object. Because of database restrictions you cannot use certain combinations of operators and Filter By objects together. For example, if you use the Equal To operator with a Filter By object that returns multiple values, the database rejects the SQL because this type of subquery requires the Filter By object to return one value only. In cases where the generated SQL is rejected by the database, you see an error message showing the error description returned by the database.
WHERE condition (optional)	An additional condition that constrains the list of values of the Filter By object. You can use ordinary report objects, predefined conditions or existing query filters (including subqueries) in the WHERE condition.
Relationship operator	If there is more than one subquery, determines the relationship between the subqueries. AND - the conditions in all of the subqueries must be satisfied. OR - the conditions in any one of the subqueries must be satisfied.

17.8.3 What is a database ranking?

Database rankings allow you to answer questions like "what are the top three customers, based on the revenue they generated for each year?" at the query level, without the need to return data that falls outside the ranking to Crystal Reports. You can then filter this data using a ranking.

When you rank data you sort and filter it according to ranking criteria. This process is much like selecting Top or Bottom N values when sorting groups in Crystal Reports. For more information, see [Selecting top or bottom N groups or percentages \[page 194\]](#).

A database ranking allows you to specify a ranking at the query and database level so that the data returned to Crystal Reports by the query is already ranked. This has the following advantages:

- Ranking data can be processing-intensive. By ranking at the database level you allow the server, which is typically far more powerful than the client machine, to perform this processing.
- Pre-ranking data reduces the amount of data retrieved across the network and stored in Crystal Reports.

Note

You can perform a database ranking only if your database supports it. If this is not the case, the "Add a top bottom" button is disabled on the Query Panel toolbar. Databases that support ranking are Oracle, DB2, Teradata and Redbrick.

Note

Crystal Reports uses the SQL-99 Rank function in ranking SQL.

Note

SQL is the query language supported by all relational databases (RDBMS), although each database has its own syntax.

17.8.4 Creating a database ranking

You create a database ranking in the Query Filters pane of the Query Panel.

Note

You can mix database rankings and other types of query filter in the Query Filters pane.

17.8.4.1 To create a database ranking

Procedure

1. Add the objects that you want to appear in your query to the Result Objects pane of the Query Panel.

2. Select the object that you want to rank by.

3.  Click [Add a database ranking](#) on the toolbar.

The ranking outline appears in the Query Filters pane. The object that you selected appears as the ranking dimension, and the first measure in the query appears as the ranking measure in the "Based on" list.

Note

The "Add a database ranking" button is disabled if your database does not support ranking.

4. Select the ranking direction (Top or Bottom).
5. Type the number of records you want the ranking to return in the box next to Top/Bottom.

Note

You can specify a prompt instead of a constant by clicking the arrow next to the number. When you select a prompt, the user must enter the ranking number when they run the query. For more information on prompts, see [Building prompts \[page 384\]](#).

6. Drag the object that provides the calculation context for the measure to the [For each](#) box.

This dimension is optional.

Note

To display the "For each" box, click the arrow to the right of the "Based on" measure.

7. Drag any objects that you want to include in the WHERE restriction to the white area of the ranking outline. For more information on the WHERE restriction, see the explanation under [What is a subquery? \[page 388\]](#).

17.8.4.2 Ranking parameters

The following table describes the parameters from left to right in the ranking:

Parameter	Description
Top/Bottom	Ranking order. Top - ranks in descending order. Bottom - ranks in ascending order.
Number of records	The number of records to return in the ranking. For example, the top 10.
Ranking object	The object used in the ranking. For example, if the object is Region and the ranking is Top 10, the ranking returns the top 10 regions.

Parameter	Description
Based on	The measure by which the ranking object is ranked. For example, if the measure is Revenue and the object is Region, Crystal Reports ranks regions by the amount of revenue they generate.
For Each (optional)	The object that specifies additional calculation context for the ranking. For example, if the ranking object is Region, the measure is Revenue and the For Each dimension is Country, Crystal Reports ranks regions by revenue within each country.
WHERE condition (optional)	Additional restriction on the values returned in the ranking that appears below the other parameters. For example, a ranking of regions with a condition that restricts Country to "USA" ranks only those regions in the USA.

18 Creating and Updating OLAP Reports

This section describes how to create OLAP reports, how to update them when the location of your data changes, and how to work with the data displayed in an OLAP grid.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

18.1 OLAP reporting with Crystal Reports

Although relational databases such as SQL servers and PC databases are the most common sources of data, Online Analytical Processing (OLAP) and Multi-Dimensional Data are rapidly becoming the popular data-storage and analysis formats. Crystal Reports provides the same access and reporting features for OLAP data sources that it provides for relational data.

Note

The term OLAP is used in this topic to refer to all common forms of OLAP and Multi-Dimensional Data storage and access systems.

Crystal Reports supports access to OLAP data through direct connection and Open OLAP gateways. A direct connection requires an installed OLAP client and uses DLLs located on your local machine, whereas the Open OLAP gateway doesn't require either. With an Open OLAP connection, data access is achieved through a Name Server host that communicates with Crystal Reports and your OLAP data source. Set up an Open OLAP connection on the Advanced Settings tab of the Connection Properties dialog box.

For direct connections, Crystal Reports supports many OLAP server types. These types are displayed in the Server Type list of the Connection Properties dialog box. This list is dynamic and will show those server types for which you have an installed client.

Note

Crystal Reports can open OLAP reports that were created in previous versions. However, while Crystal Reports 9 can open version 10 OLAP reports with saved data, it cannot refresh them.

18.1.1 OLAP grid objects

When you design a report with OLAP data, Crystal Reports creates a primary report containing one or more OLAP grid objects. OLAP grid objects look and act much like Cross-Tab objects, but they are designed specifically for OLAP data.

OLAP grid objects provide true multi-dimensional reporting. Add dimensions to either axis to analyze three, four, or more dimensions in a single OLAP grid. Or instead of viewing multiple dimensions within one OLAP grid, create multiple OLAP grids within the same report.

Note

When Crystal Reports displays an OLAP grid, it can do so quickly if the grid is long (down many pages) instead of wide (across many pages). When the grid is long, the program processes it on a page-by-page basis. When the grid is wide, the program must gather all the data before it can display a page. This may require significantly more processing time.

18.2 Creating an OLAP report

OLAP reports are created using the OLAP Report Creation Wizard or the OLAP Expert. Before you start creating your report, ensure you have the following information:

- OLAP Type
- Server/Database name
- User ID
- Password
- Parameters (applies only to OLE DB for OLAP)

→ Tip

You can return to the OLAP Report Creation Wizard by selecting the OLAP Design Wizard option on the Report menu. Do this if you want to change your page dimensions, or if you want to change parameter values.

18.2.1 To create an OLAP report

Procedure

On the Start Page, click [OLAP Cube Report Wizard](#).

The OLAP Report Creation Wizard appears with the OLAP Data screen active.

18.2.2 Specifying the data source

Context

Use the OLAP Data screen to define the OLAP data source.

Procedure

1. Click [Select Cube](#).

The OLAP Connection Browser appears.

2. Browse your OLAP server for the cube you want to connect to.

If your server isn't in the list, click Add. In the Connection Properties dialog box, provide the server information; then click OK.

3. Select the desired cube and click [Open](#).

The OLAP Data screen reappears with the supplied data source information.

4. Click [Next](#).

Results

Note

You can also click "Select CAR File" on the OLAP Data screen. If you choose this option, the Open dialog box appears. Search for a file created in OLAP Intelligence Professional.

18.2.3 Defining the structure of the grid

Context

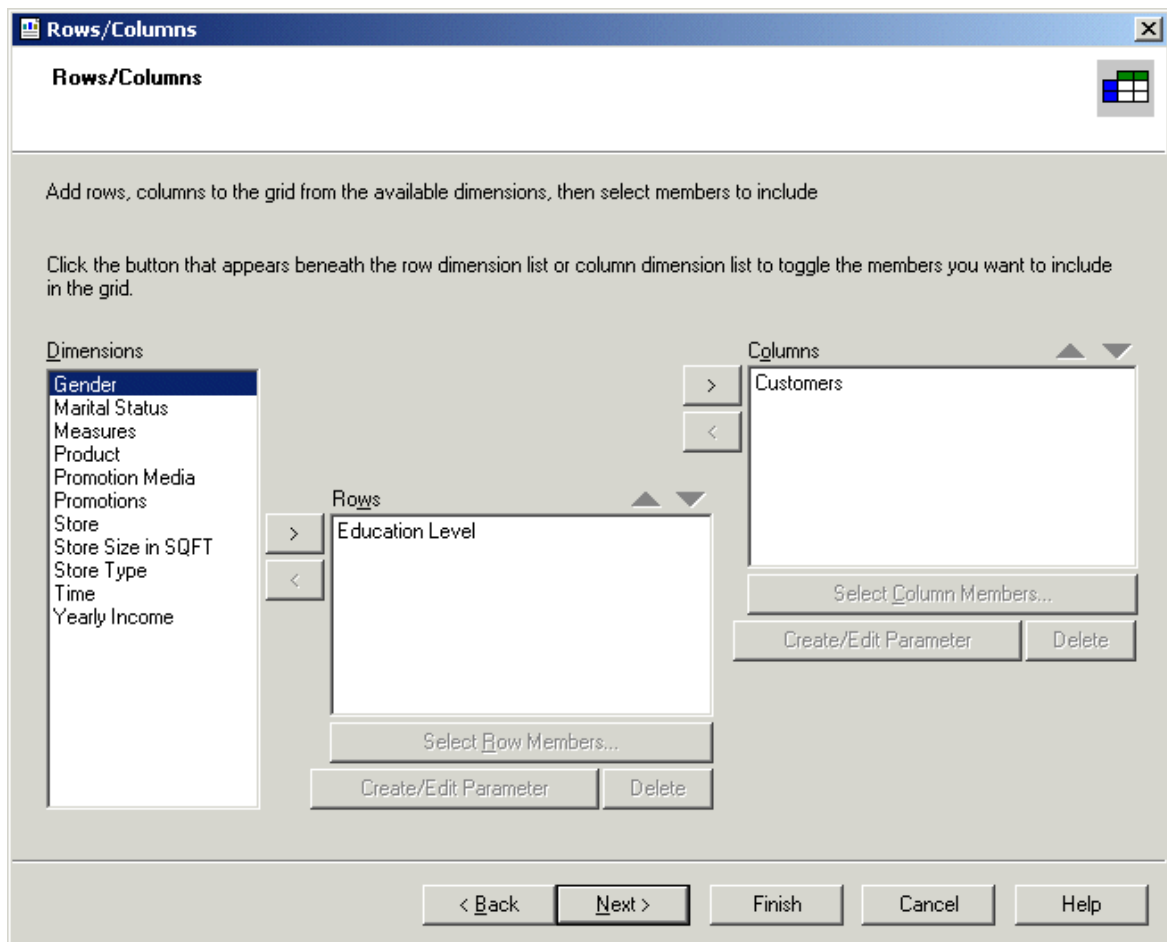
Use the Rows/Columns screen to define how your data is structured within the grid. You can place the dimensions into either the Columns or Rows areas.

Tip

It is also possible to drag and drop the dimensions into either the Rows or Columns area.

Procedure

1. Select a dimension to appear in the report.
2. Click the > arrow adjacent to either the Rows or the Columns areas, depending on where you would like the dimension to be located.
3. Continue adding dimensions to the Rows and Columns areas.



4. Click the *Up* and *Down* arrow buttons associated with the Rows and Columns areas to arrange the order of the dimensions.

Note

If you accidentally add a dimension to either the Rows or Columns area, click the < arrow to return it to the Dimensions list.

5. Select a dimension in either the Rows or the Columns areas and click *Select Row Members* or *Select Column Members* to specify the members to be included in your report.

The Member Selector dialog box appears.

6. Select the members you would like to include.
7. Click *OK*.

8. Select a dimension in either the Rows or the Columns areas and click [Create/Edit Parameter](#) to create a parameter for use with the dimension.

The Create Parameter Field dialog box appears.

9. Select default values, prompting text, and other options, and then click [OK](#).

Note

Once you have created a parameter, access to the Member Selector dialog box is disabled for the dimension until you delete the parameter.

Note

If you create a row/column parameter, the View Cube command and the Cube View tab are not available.

10. If you want to delete a parameter, select the appropriate dimension and click [Delete](#).

When you click OK in the Delete Link to Parameter Field dialog box, the program removes the parameter description from the dimension.

11. Click [Next](#).

The Slice/Page screen appears.

18.2.4 Setting sliced dimensions and specifying the number of grids

Use this screen to set sliced dimensions and to specify the number of grids that are required.

The Slice area determines the boundaries that will be placed on your report. For example, if you have a time dimension that is made up of fiscal quarter members, you could specify the report return the results for a specific quarter.

Note

If all of your members were used in the Rows/Column tab, nothing will appear here.

The Page area enables you to determine the number of grids and the subject of each. For example, if you have a product dimension, you could put this in the page area and specify two different products. This would result in two grids with the same rows, columns, and formatting, but each one would be based on a different product.

18.2.4.1 To specify a slice

Procedure

1. To determine the member that is going to be used as the slice, select a dimension in the [Slice](#) list and click [Select Slice](#), or double-click a dimension in the [Slice](#) list.

The Member Selector dialog box appears.

2. Select the appropriate member, expanding the structure if necessary.
3. Click [OK](#).
4. Click [Next](#) if you do not want to add a page. The Style screen appears.

18.2.4.2 To add a page

Procedure

1. Use the > arrow to add dimension(s) to the [Page](#) list.

The Member Selector dialog box appears.

2. Expand the dimension's structure and select the appropriate members.
3. Click [OK](#).

18.2.4.3 To add a parameter

Context

You can create a parameter field to link to any dimension. When you add a parameter to a dimension in your OLAP grid, users are prompted to select a value when refreshing report data.

Procedure

1. Click the [Create/Edit](#) button.
2. Select default values, prompting text, and other options, and then click [OK](#).
3. Click [Next](#).

The Style screen appears.

Results

ⓘ Note

Once you have created a parameter, access to the Member Selector dialog box is disabled for the dimension until you delete the parameter.

18.2.5 Applying a predefined style

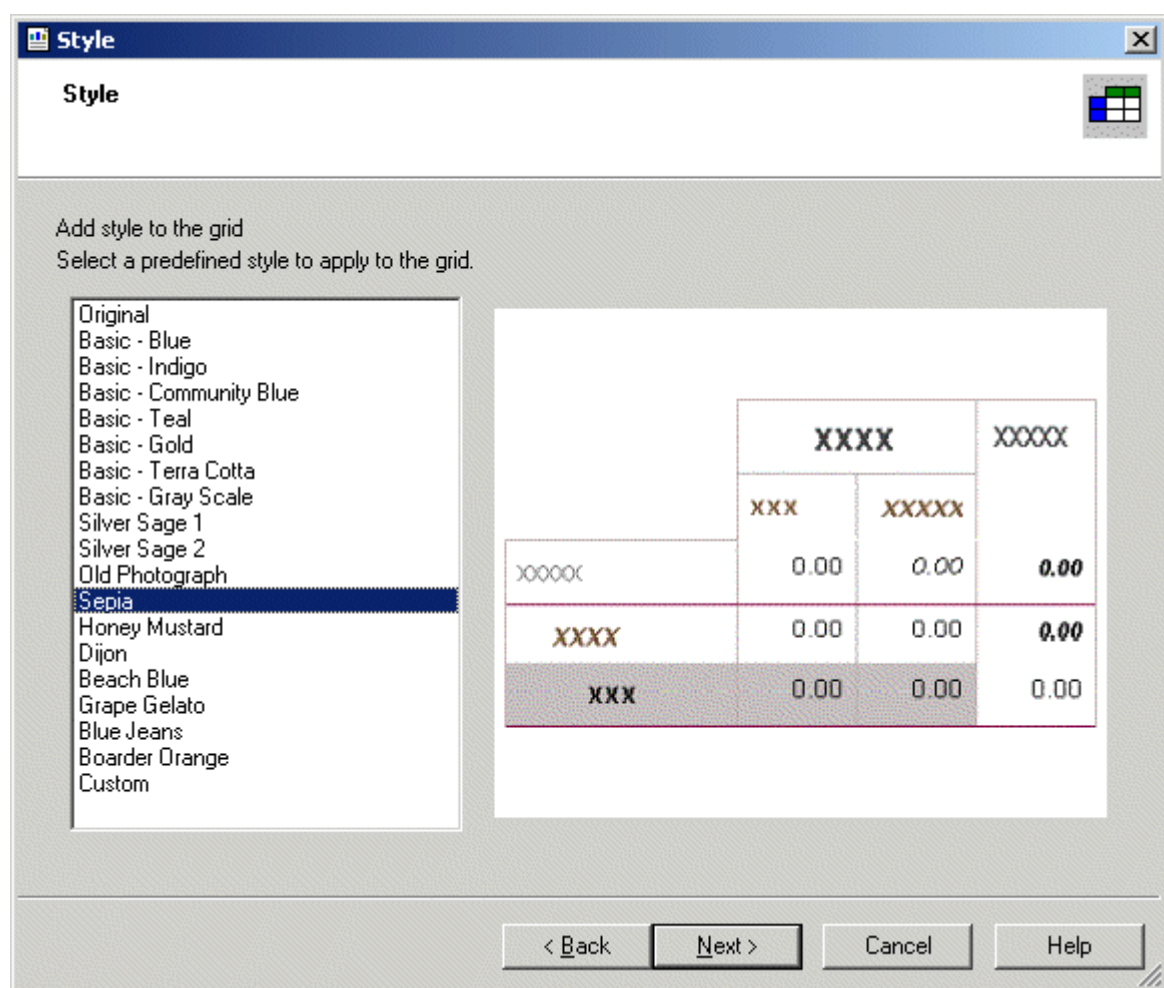
Procedure

1. Select a predefined style for the grid from the list.

→ Tip

If you do not want to use a predefined style, click Next.

A preview of the color scheme you select appears on the right.



Note

The color of the style you select may not appear correctly if your screen resolution is set to 256 colors. Increase the resolution to correct this.

2. Click *Next*.

The Chart screen appears.

18.2.6 Inserting a chart

Procedure

1. Select the kind of chart you want to add to your report from the options shown on the Chart screen.

→ Tip

If you do not want to insert a chart, click *Finish*.

2. Add a title for your chart.
3. In the *On change of* list, select the dimension you want to base your chart on.
4. If necessary, in the *Subdivided by* list, click a secondary row or column you want to base your chart on.

ⓘ Note

Be sure that the chart type selected in Step 1 supports a secondary charting field.

5. Click *Other Dimensions* to define field values for any dimensions you've used in your OLAP grid but not in your chart.

For more information refer to [Charting on an OLAP cube \(OLAP layout\) \[page 312\]](#) and [Creating charts \[page 309\]](#).

6. To finalize your report, click *Finish*.

18.3 Updating an OLAP report

The location of the OLAP data accessed by your report may change. Some possible scenarios include:

- The location of the OLAP server or database may change due to restructuring of Information System resources.
- Another cube instance may have been successfully processed and reflect more current information.
- New reports may be developed on a smaller database that represents a subset or production version of the primary OLAP database, then ported over to the actual working database to report on real data.

Before updating your report, it is important to ensure that it is compatible with the data. Some things to look for include:

- Removing dimensions contained in the report that are not in the cube.
- Removing fields referred to explicitly in the design of the report, such as a formula.

ⓘ Note

If a report contains subreports or grids, and the database used for them has changed name or location, you must update each subreport or grid.

18.3.1 To update a cube location in an OLAP grid object

Procedure

1. Select the grid by clicking on the border.
2. From the *Database* menu, click *Set OLAP Cube Location*.

→ Tip

You can also right-click the grid and select Set OLAP Cube Location.

The Confirm Command dialog box appears.

3. Click *Yes*.

The Set OLAP Cube Location dialog box appears.

4. Click *Select*.

The OLAP Connection Browser appears.

5. Browse your OLAP server for the cube you want to connect to.

If your server isn't in the list, click Add. In the Connection Properties dialog box, provide the server information; then click OK.

6. Select the desired cube and click *Open*.

The Set OLAP Cube Location dialog box reappears.

7. Click *OK*.

If your report contains multiple OLAP grids that are based on the same cube, the program prompts you to update the remaining grids to the new location.

Results

Finally, if the location of your report's relational database is the same as the location of your cube, you are prompted to change the location of the relational database to match the new location of the cube. Making sure that the location of the relational database and the OLAP cube match ensures that your data remains synchronized. The program can change the location automatically, or you can change it manually in the Set Datasource Location dialog box.

18.3.2 To update the database location

Procedure

1. In the Set Datasource Location dialog box, expand the *Create New Connection* folder in the "Replace with" area.

2. Expand the [OLAP](#) folder and search for the new cube location.

→ Tip

You can double-click the Make New Connection option to search for the cube in the OLAP Connection Browser.

3. Select the data source name, or an individual table, and click [Update](#).

The data source name is changed in the Current Data Source area.

4. Click [Close](#).

18.4 Formatting data in an OLAP grid

There are several ways to add formatting to rows and columns in an OLAP grid:

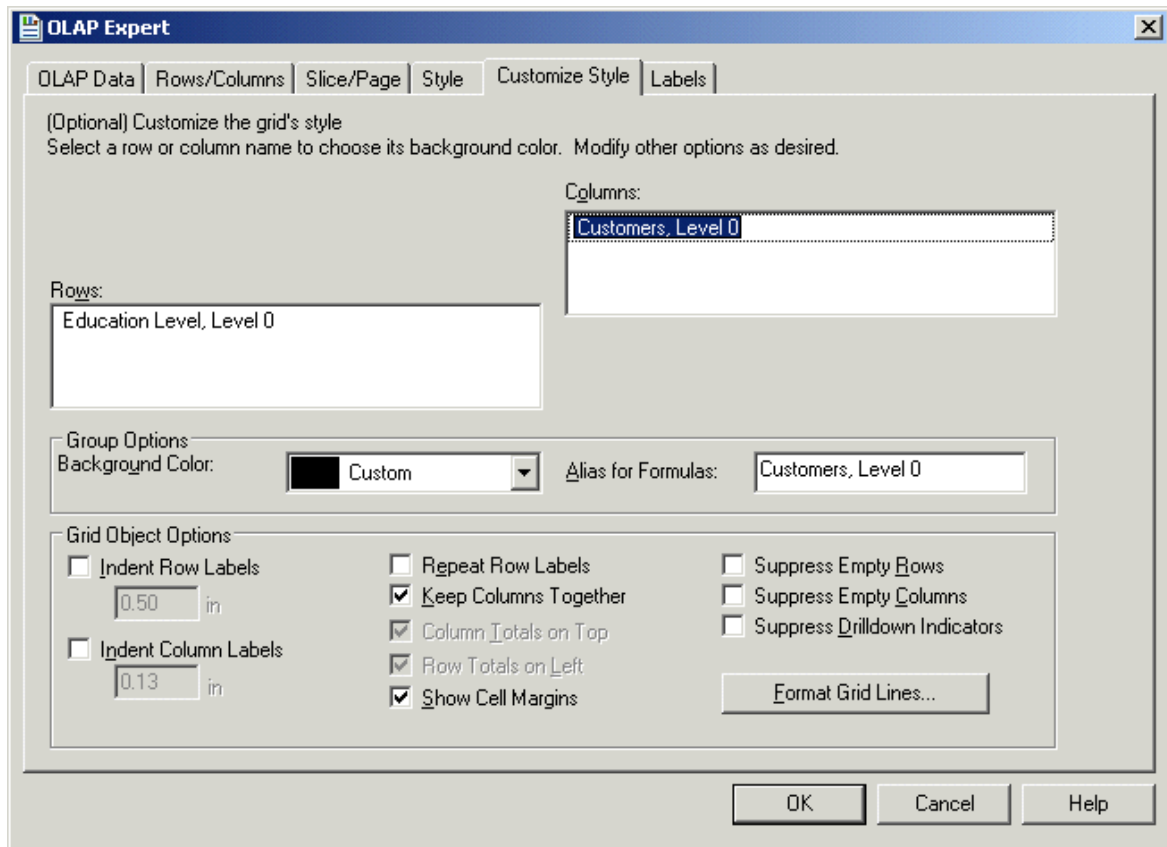
- Use the Highlighting Expert to apply conditional formatting to all types of report fields: Number, Currency, String, Boolean, Date, Time, and DateTime fields.
To access the Highlighting Expert, right-click the field you want to format and, on the shortcut menu, click Highlighting Expert. For more details, see [Using the Highlighting Expert \[page 294\]](#).
- Use the Format Editor to apply absolute formatting to fields in the grid. Absolute formatting is applied under all conditions, irrespective of the data values in the field.
To access the Format Editor, right-click the field you want to format; then, on the shortcut menu, click Format Field. For more details, see [Working with absolute formatting \[page 271\]](#).
- Use the Formula Workshop to write your own conditional formatting formulas with Crystal or Basic syntax. The Formula Workshop gives you maximum control over the formatting of the OLAP grid.
To access the Formula Workshop, open the Format Editor and click the appropriate Formula button. For more details, see [Working with conditional formatting \[page 287\]](#).
- Use the OLAP Expert to reformat the entire OLAP grid. This expert provides all of the formatting options that are available in the OLAP Report Creation Wizard.
To access the OLAP Expert, select the entire grid object by clicking one of its borders. Right-click the selected grid and, on the shortcut menu, click OLAP Grid Expert.
The OLAP Expert has two tabs not found in the OLAP Report Creation Wizard. Use the Customize Style and Labels tabs to format your OLAP grid.

As well as letting you format an OLAP grid's rows and columns, Crystal Reports lets you designate report objects that don't expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, to be repeated on each additional horizontal page that the OLAP grid creates. For more information, see [Repeating report objects on horizontal pages \[page 283\]](#).

18.4.1 Changing the background color of a dimension

Procedure

1. On the Customize Style tab of the OLAP Expert, select a dimension.



2. In the Group Options area, select the color from the [Background Color](#) list.
3. Click [OK](#) if you have finished customizing your grid.

18.4.2 Creating an alias for a dimension

Context

You can create aliases to shorten long dimension names. This is useful when you plan on referring to a dimension in a conditional formatting formula (by using the `GridRowColumnValue` function).

Procedure

1. On the Customize Style tab of the OLAP Expert, select the dimension.
2. In the Group Options area, enter an alias name in the [Alias for Formulas](#) field.
3. Click [OK](#) if you have finished customizing your grid.

18.4.3 Formatting grid lines

Context

In the Format Grid Lines dialog box, you can define whether the lines show or not, as well as the color, style, and width.

Procedure

1. On the Customize Style tab of the OLAP Expert, click [Format Grid Lines](#).
The Format Grid Lines dialog box appears.
2. From the list, choose the description of the area where you would like the lines to appear, or click the appropriate area in the Format Grid Line diagram.
3. Select the [Draw](#) check box to have your lines appear in the report.
4. Select the color, style, and width.
5. Click [OK](#).
6. Click [OK](#) if you have finished customizing your grid.

18.4.4 Labeling dimensions

Context

In the Labels tab, you are able to define which dimensions will be labeled and how they will be displayed in the grid. The dimensions available to you are the ones that have been designated as a page or slice.

Procedure

1. Using the arrow buttons, move the dimensions to the Unlabeled and Labeled areas as required.
2. Make changes to the appearance of the label by using the options available in the Label Position and Label Spacing areas.
3. Click [OK](#) if you have finished customizing your grid.

18.5 Changing the view of OLAP data

When analyzing OLAP data in your report, there are several basic ways to change how the data is displayed in the grid.

Note

The methods described here let you manipulate your OLAP grid directly from the Preview tab of Crystal Reports. You can also carry out the same and additional, more advanced, functionality on the Cube View tab.

Search the online help for "Using the OLAP Worksheet" to learn more about the functionality available on the Cube View tab.

18.5.1 To show or hide dimension members

Procedure

1. Right-click the dimension whose members you want to show or hide.
2. On the shortcut menu, click [Expand Member](#) or [Collapse Member](#).

The dimension expands to show its members, or collapses to hide them.

Tip

You can also double-click parent dimensions to drill down into the hierarchical data displayed in the OLAP grid.

18.5.2 To create asymmetry in an OLAP grid

Context

Using asymmetry, you can display different inner dimension members for any number of outer dimensions.

Note

You can create asymmetry only on stacked dimensions.

Procedure

1. Right-click the member that you want to remove.
2. On the shortcut menu, point to *Hide* or *Show* and then select the appropriate option.

For information about how to use these options, search the online help for "Creating asymmetry" and "Restoring symmetry."

18.5.3 To add totals to an OLAP grid

Procedure

1. Right-click the dimension you want to see totals for.
2. On the shortcut menu, select *Automatic Totals*.

A submenu appears with options for the kind of totals you can add.

3. Select the option you want.

Results

A "Total" row or column (or both) is added to your OLAP grid. To remove totals later, select No Totals.

18.5.4 To change the display format for member names

Procedure

1. Right-click the dimension name.
2. On the shortcut menu, point to *Display Members Using*, and then click one of the display options:
 - Caption
 - Name
 - Caption : Name
 - Unique Name

For information about names and captions, search the online help for "Changing member captions."

18.5.5 To alter the data displayed in the OLAP grid

Procedure

1. Drag and drop rows and columns to rearrange the data in the OLAP grid.
2. Drag and drop dimensions to and from the OLAP Labels area to add or remove data from the grid.

On the Preview tab, the OLAP Labels area is at the upper-left corner of the OLAP grid.

18.5.6 To define the order of fields in the OLAP grid

Procedure

1. Expand the row or column dimension whose fields you want to reorder.
2. Right-click the dimension and, on the shortcut menu, click *Reorder Displayed Members*.

Note

You cannot reorder members for dimensions that you have created row or column parameters for.

3. In the Reorder Displayed Members dialog box, select fields and click the *Up* and *Down* arrows to rearrange the members that are displayed in the grid.
4. Click *OK* to affect your changes and return to the OLAP grid.

18.6 Sorting and filtering OLAP grid data

The sorting capabilities of the OLAP grid enable you to order data by row and column values. If you want to limit the data based on particular field values, add one or more filters to the grid. Filters also allow you to perform top or bottom N analysis (by actual values and by percentage).

18.6.1 Sorting data in an OLAP grid

Use the OLAP grid's sorting capabilities to quickly arrange grid data in a useful manner. You can add, edit, and remove sorts by right-clicking the appropriate row or column member and selecting from the shortcut menu. You can sort data in ascending or descending order, and you can specify whether or not you want to break grid hierarchies.

In the following grid, for instance, an ascending sort has been added to the Budget column.

			Budget
All Products			2,449,052.00
	Frozen Goods		106,392.00
		Pastry	395.93
		Frozen Produce	97,490.98
	Bakery		173,531.50
		Cakes and Pies	72,723.45
		Loaves and Buns	86,326.31

In this case, the OLAP grid respects the parent/child relationships between grid members and sorts the data values accordingly. (Frozen Goods precedes Bakery, but Pastry follows Frozen Goods.)

In the next example, the Budget column remains sorted in ascending order; in this case, however, the Break Hierarchies option is selected.

			Budget
All Products	Frozen Goods	Pastry	395.93
	Bakery	Cakes and Pies	72,723.45
		Loaves and Buns	86,326.31
	Frozen Goods	Frozen Produce	97,490.98
	Frozen Goods		106,392.00
	Bakery		173,531.50
	All Products		2,449,052.00

Now the Report Designer disregards parent/child relationships between grid members and sorts on the basis of data value alone. (Frozen Goods still precedes Bakery, but Pastry precedes all others.)

You can add up to three sorts to grid rows and up to three sorts to grid columns. In each case, the first sort takes precedence, and each subsidiary sort serves to further differentiate between grid data. If you add a fourth sort to a row or to a column, your first three are removed from the grid, and the new sort becomes the single, primary sort.

→ Tip

To locate a sorted row or column, move the mouse pointer over the OLAP grid. When you reach a sorted member, the pointer turns into a double-arrow.

18.6.1.1 To sort data in the OLAP grid

Procedure

1. Right-click the row or column member that you want to sort by.
2. On the shortcut menu, point to [Add First Sort](#).
3. From the submenu, select from the available sort options:
 - Ascending
 - Descending
 - Ascending, Break Hierarchies
 - Descending, Break Hierarchies

Results

Note

An ascending sort on a grid row orders data values from lowest to highest, left to right. An ascending sort on a column orders data values from lowest to highest, top to bottom.

Search the online help for "Sorting data" in the "Using the OLAP Worksheet" section to learn more about sorting.

18.6.2 Filtering data in an OLAP grid

Use filters to exclude grid data that are not important, or to display only the data that you want to see. You can filter grid data by actual values, or you can choose to exclude or display the top or bottom N, or the top or bottom N%.

18.6.2.1 To add a filter

Procedure

1. Right-click the field whose values you want to filter.
2. On the shortcut menu, click [Add Filter](#).
3. In the Define Filter dialog box, use the [Filter Type](#) list to specify how you want to filter the data.

You can filter data by "Actual values," or by selecting "Top/bottom n" or "Top/bottom n%."

4. Use the [Filter Definition](#) options to specify which rows or columns you want to exclude or display.

5. Click [OK](#) to add the filter and return to the OLAP grid.

Results

→ Tip

To locate a filtered row or column, move the mouse pointer over the OLAP grid. When you reach a filtered row or column, the pointer turns into an X.

→ Tip

If you filter all the cells in your OLAP grid, right-click the empty grid and select Remove All Filters from the shortcut menu.

Search the online help for "Filtering data" in the "Using the OLAP Worksheet" section to learn more about filtering.

18.7 Adding calculations to OLAP grids

Calculated members enable you to perform specific calculations on OLAP data and to display the results in rows or columns that are added to the OLAP grid. You can create a quick calculation, such as a Variance member calculated by subtracting Budget from Sales. Or you can use functions to perform complex Statistical Analysis or Time Series calculations.

Search the online help for "Adding calculated members" in the "Using the OLAP Worksheet" section to learn more about calculated members.

19 Using the OLAP Worksheet

When you right-click an OLAP grid on the Preview tab of Crystal Reports and choose the View Cube command, the Cube View tab appears. The tab contains a Worksheet on which you can carry out simple and advanced data manipulations. This section describes how to use the Worksheet.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

19.1 Sample Reports

You can download sample reports and a sample database for Crystal Reports from the following link: [Direct download link for samples](#) .

19.2 Changing the view of your data

This section explains how to change the view of your business data by swapping and stacking dimensions, changing the current slice, and exploring hierarchical dimensions.

19.2.1 Swapping dimensions

You can swap dimensions with others to reorient the view of data in the Worksheet. You can drag and drop dimensions or use the commands on the shortcut menu.

You can swap a row or column dimension with one of the slice dimensions, or with another row or column.

19.2.1.1 Swapping dimensions containing sorts and filters

If you sort or filter a row dimension, and then swap the row with a column dimension, the filter moves with the dimension. The filter also moves with the dimension if you change a column to a row.

However, if you move a row or column dimension to a slice, then the sort or filter is removed.

See [Sorting data \[page 448\]](#) and [Filtering data \[page 444\]](#) for more information on sorting and filtering.

19.2.2 Stacking dimensions

Displaying two or more dimensions in a particular row or column in the Worksheet is known as stacking dimensions. When you stack dimensions, the one nearest the cells is called the inner dimension, and any others are called the outer dimensions.

You can stack dimensions using the following commands from the shortcut menu:

<i>Move To</i>	Choose Row, Column or Slice from the submenu to move the selected dimension.
<i>Add Before</i>	Choose a dimension member from this submenu. The dimension you are moving is inserted before this dimension.
<i>Add After</i>	Choose a dimension member from this submenu. The dimension you are moving is inserted after this dimension.

Alternatively, you can use the drag-and-drop method.

19.2.2.1 To stack dimensions

Procedure

1. Point to the dimension that you want to move and drag it to a row or column.

You can only place a dimension over a row or column, or over a slice at the bottom of the Worksheet. There must always be at least one dimension left in a row or column.

2. When the cursor changes to an arrow symbol, release the mouse button:



The arrow symbol indicates where the dimension appears: you can add the dimension above or below a column dimension, or on the left or right of a row dimension.

The data in the Worksheet updates to show the new orientation.

Results

When moving a dimension to the slice dimension area, you can set the slice member by dragging the appropriate row or column member heading. See [Changing the slice of data \[page 424\]](#) for more information on altering the active slice member.

❗ Note

You can only apply sorting, filtering, and exception highlighting to inner dimensions on stacked dimensions.

19.2.2.2 To swap rows with columns

Procedure

1. Right-click a row or column dimension.
2. Click *Swap Rows with Columns* on the shortcut menu.

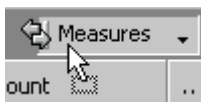
Results

The data in the Worksheet updates to show the new orientation.

19.2.2.3 To swap two dimensions

Procedure

1. Point to the dimension that you want to move, and drag it to the dimension that you want to swap.
You can place a dimension over a row or column, or over a slice dimension at the bottom of the Worksheet.
2. When the cursor changes to the swap symbol, release the mouse button to swap the two dimensions:



Results

→ Tip

You can also right-click the dimension you want to move, and click Swap With on the shortcut menu.

The data in the Worksheet updates to show the new orientation.

When swapping a row or column dimension with a slice dimension, you can set the new slice member by dragging the appropriate row or column member heading to the slice dimension area. See [Changing the slice of data \[page 424\]](#) for more information on altering the active slice member.

19.2.2.4 Removing a stacked dimension

If you no longer want to display a dimension as a stacked row or column, drag the dimension to the slice dimension area at the bottom of the Worksheet.

19.2.2.5 Changing the order of stacked dimensions

You can change the order of the dimensions in a row or column by swapping the dimensions.

19.2.3 Selecting row and column members

You can use the Member Selector to set the active members for your row and column dimensions. If you are only interested in a subset of your business data, use the Member Selector to display only those members.

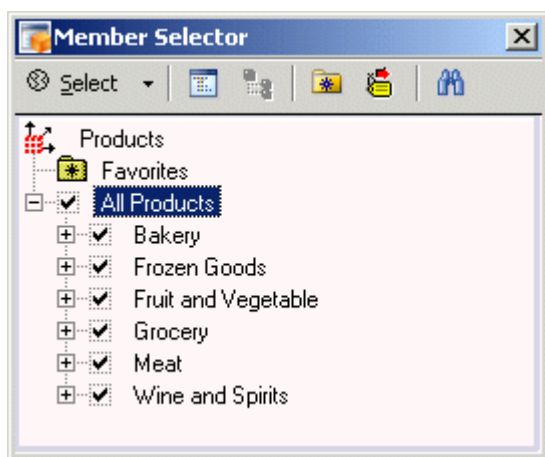
If you want your application to keep up to date with changes in the cube, we recommend that you use the Member Selector commands to select groups of members, rather than selecting members individually. For details, see [Optimizing the way you select members \[page 421\]](#).

19.2.3.1 To set row and column members

Procedure

1. Click the down-arrow button beside the dimension name to open the Member Selector.

The member list is automatically expanded to display all of the members that are currently displayed in the Worksheet.



2. Select the check boxes for each of the members that you want to see in your view.
 - Expand hierarchies by clicking the plus symbol, if required.
 - Glance down the levels to make sure that only the members you want are selected. If a parent member is not selected, this does not necessarily mean that child members are not selected. Child members can still be selected, even if the parent member is not selected.
 - Use the Member Selector commands to help you pick the members you want more quickly. The following section describes these commands.
3. Close the Member Selector when you have completed your selection.

19.2.3.2 Using the Member Selector toolbar commands

The toolbar contains a number of commands to help you select members quickly.

Toolbar

The Member Selector toolbar contains the following commands:

	<i>Selection menu</i>	Change the selected members.
	<i>Select Display Mode</i>	Select from the following: Hierarchy, Sort Ascending, and Sort Descending. See Displaying a hierarchical or a flat view [page 418] .
	<i>Select Hierarchy</i>	Display a list of hierarchies (if more than one hierarchy is available from the cube).
	<i>New Favorite Group</i>	Add a new favorite group to the Favorites folder of the associated dimension.
	<i>Display Members Using</i>	Select Caption, Name, or Caption : Name to specify the member display mode for the Member Selector.
	<i>Search</i>	Search for members.

Select menu

The following commands are available from the Selection menu on the Member Selector toolbar. Click the down-arrow button next to Select to display these commands.

<i>Select All Members</i>	Selects all the members in the dimension. This is useful if you want to select the majority of members. After you select all members, clear the members you don't want.
<i>Select None</i>	Clears all selections.
<i>Invert Selection</i>	Selects the members not selected, and clears the members that were selected.
<i>Select All Top Members</i>	Selects the members in the top level.
<i>Select All Members at Level x</i>	Selects all members at a certain level. Select the level you want from the Level Selector window. To select multiple levels, hold down <i>SHIFT</i> or <i>CTRL</i> and select the member levels you want.
<i>Select All Base Members</i>	Selects all members at the lowest level.
<i>Move Selection Down One Level</i>	Moves the selection from the parent member to the children.
<i>Move Selection Up One Level</i>	Moves the selection from the child member to the parent.
<i>Save As Favorite</i>	Creates a new favorite group containing the selected members. You can select more than one member by holding down the <i>SHIFT</i> or <i>CTRL</i> key.

Note

After you have used one of these commands, its symbol appears on the Select button on the toolbar so that you can choose it quickly.

19.2.3.3 Using the Member Selector shortcut menu

You can use the Member Selector shortcut menu to make selections quickly.

To display this menu, right-click a member in the Member Selector. The following commands are available:

<i>Select All Members at This Level</i>	Selects all the members on this level of the hierarchy.
---	---

Add Parent to Selection

Adds parent members to the selection. You can choose from:

- **One level:** Adds the immediate parents of the members you have selected.
- **All levels:** Adds all parents of the members you have selected.
- **Custom:** Displays the Level Selector. Select the level or levels of parent members you want to add. Use **CTRL** and **SHIFT** to select multiple levels.

Add Children to Selection

Adds child members to the selection. You can choose from:

- **One level:** Adds the immediate children of the members you have selected.
- **All levels:** Adds all children of the members you have selected.
- **Custom:** Displays the Level Selector. Select the level or levels of child members you want to add. Use **CTRL** and **SHIFT** to select multiple levels.

Add To

Adds your selections to your favorite groups. You can choose to:

- Create a favorite group.
- Add the selections to an existing favorite group.

You can select more than one member by holding down the **SHIFT** or **CTRL** key.

19.2.3.4 Displaying a hierarchical or a flat view

If a dimension contains a hierarchy, the Member Selector can either show the hierarchy, with its structure of groups and totals, or show all the members in the dimension. Displaying all members of a dimension is known as a flat view. The flat view is displayed alphabetically and can be shown in descending or ascending order.

19.2.3.4.1 To display a dimension in flat view

Procedure

1. Click the down-arrow button next to the dimension name to display the Member Selector.
2. Click **Select Display Mode**.
3. Select **Sort Ascending** from the menu to arrange the members into ascending alphabetical order. Select **Sort Descending** to reverse the order.

Results

The hierarchy is shown flattened—that is, with all the members in the dimension displayed at the same level.

19.2.3.4.2 To display a dimension in hierarchical view

Procedure

1. Click the down-arrow button next to the dimension name to display the Member Selector.
2. Click [Select Display Mode](#).
3. Select [Hierarchy](#) from the menu to display all the hierarchy levels.

Results

If a member is part of a hierarchy and its dimension is displayed in hierarchical view, a drill symbol appears next to the member name. A plus symbol enables you to expand the hierarchy under the member. A minus symbol enables you to collapse the hierarchy.

19.2.3.5 Switching between dimension hierarchies

Some cubes have more than one hierarchy. If this is the case, the Select Hierarchy button in the Member Selector is available.

If the cube has more than one hierarchy, click the select hierarchy button to switch between hierarchies:



Then select the hierarchy you want from the list.

19.2.3.6 Using the Favorites folder

If a dimension contains a large hierarchy with several levels, it can take time to find the members you want. If you use these members frequently, it saves time and effort to make a shortcut to them in the Favorites folder.

All dimensions have a Favorites folder. This folder can only contain members from the dimension associated with the folder. For information on creating a favorite, see [Working with favorite groups \[page 421\]](#).

You can use favorites in two ways:

- As a shortcut to groups of members you often use.

- As a tool to place members into organized groups.

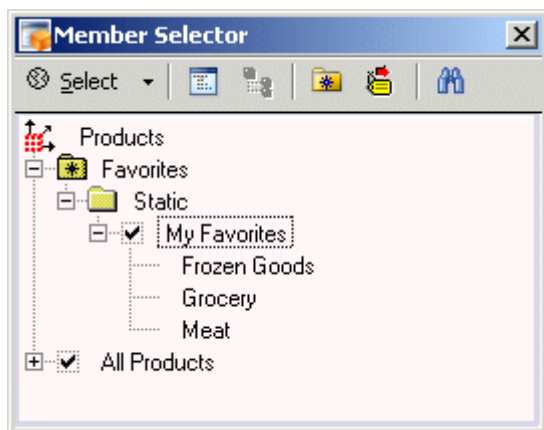
Favorite members

A favorite is a member that you choose from the Member Selector and add to the Favorites folder.

You cannot select an individual member from within a favorite group, you can only select the group itself. Selecting a favorite group also selects all the members in the group.

Favorite group

A favorite group is a selection of members that you can create and modify. Use it to quickly select a group of members you use on a regular basis. You can add and remove members from the favorite group, or create several favorite groups. Once you create a favorite group you can select it from the Favorites folder in the Member Selector.



There is no limit to the number of favorite groups you can create. There is no limit to the number of members you put into an individual favorite group.

Static and server favorite groups

There are two types of favorite groups: static and server.

Server favorite groups are created and maintained on the server. You can access and use these favorite groups from Crystal Reports, but you cannot create, edit, or delete them. If you want to create or change these groups, you must do so within the server environment.

You can only create and edit static groups using Crystal Reports.

19.2.3.7 Working with favorite groups

Click a link to view one of the topics in this section:

- [To add a member to a favorite group \[page 421\]](#)
- [To delete a member from a favorite group \[page 421\]](#)

19.2.3.7.1 To add a member to a favorite group

Procedure

1. Select the favorite group using the Member Selector.
2. Find and select the member you want.
3. Drag the member into the favorite group.

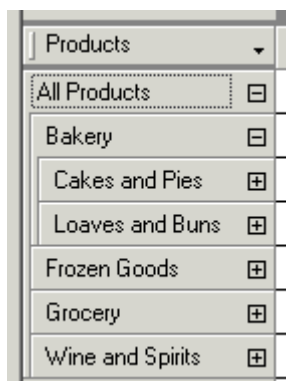
19.2.3.7.2 To delete a member from a favorite group

Procedure

1. Select the favorite group from the Member Selector and expand it.
2. Right-click the member, and click *Delete* on the shortcut menu.

19.2.3.8 Optimizing the way you select members

A dimension's active member set consists of the members that are currently selected in the Worksheet or chart. This example shows a selection of members that can make up the active member set for the **Products** dimension.



If you want your application to keep up to date with changes in the cube, we recommend that you use the Member Selector commands to select groups of members, rather than selecting members individually. When

you reload a report, Crystal Reports recreates the member sets to take into account any changes that have been made to the cube since the report was saved. For example, if you use the Add Children to Selection command to add the children of a member to the member set, this will ensure that any children added to the dimension in the future are also included in the member set. Alternatively, if you select all the children of a member individually, this member set will not include any children that are added to the member in the future.

19.2.3.9 Searching for members

The Member Selector allows you to search for members based on their properties.

19.2.3.9.1 To search for a member

Procedure

1. In the Member Selector, click the [Search](#) button.
The Search dialog box appears.
2. On the [Property](#) list, click a property:

Member Caption	The caption of the member that you want to find.
Member Name	The name of the member that you want to find.
Member Unique Name	The unique name of the member that you want to find.
Level	The level in the hierarchy that you want to search.

3. On the [Condition](#) list, click a condition:

=	Equal to	The search engine tries to find an exact match.
<>	Not equal to	The search engine tries to find all members that do not fulfil your criteria.

4. In the [Value](#) box, type the text that you want to base the search on, or select a level or property from the list.
5. Click [Search](#).
6. Click [New Search](#).

The results appear in a copy of the Member Selector. The members located by the search are selected in the hierarchy.

When you have defined a search, you can also use the Search within Results and Add to Search commands.

To search within the results of your search, redefine the criteria, click Search and then Search within Results.

To add to the results of a previous search, redefine the criteria, click Search then click Add to Search.

7. When you are satisfied with the results of your search, click one of the following:

Add to Selection	Add the search results to an existing selection of members.
Replace Selection	Replace your current selection of members with the results of your search

19.2.3.10 Using the advanced search options

Context

Use this feature if the standard search is not able to locate the members you want.

Procedure

1. In the Member Selector, click the [Search](#) button.
2. Click the [Advanced](#) tab to set the criteria for an advanced search.
3. Set your search criteria.

From left to right:

- The first cell sets a bracket on the left of your search string.
- The next cell sets the property you search on: Member Caption, Member Unique Name, or Level in the hierarchy.
- The next cell sets the operator; = (equal to) or <> (not equal to).
- The next cell is empty, use it to enter your search condition.
- The next cell is blank by default, you can use it to place a bracket on the right of your search string.
- The final cell in the row has an OR, AND setting. If you select one of these, a row of cells appears below the first row. You can use this second row of cells to enter another set of search criteria. The options are identical to the first row. Enter as many search criteria rows as you want.

Results

📘 Note

Do not forget to select the closing bracket when you finish your search string. With long search strings that extend over several rows, it is easy to forget to place a bracket at the end to complete the equation.

19.2.4 Changing the slice of data

Slice dimensions have an active slice member that determines the current view. To alter the active slice member, use the Member Selector on the corresponding dimension.

The slice Member Selector is similar to the Member Selector for row and column dimensions, but you can only select one slice member at a time.

19.2.4.1 To select a member from a slice

Procedure

1. Click the down-arrow button on the slice dimension.
2. In the Member Selector, select a member from the list.

The data in the Worksheet updates to show the view for the new slice.

Results

Alternatively, you can right-click the dimension to Drill Up, Drill Down, Drill Left, or Drill Right.

When a dimension is moved to the slice dimension area, it returns to the last-selected slice member, or to the default member if the original setting has not been altered.

If you drag a dimension heading from a row or column to the slice dimension area, the Worksheet automatically selects a slice member. To change this default behavior, drag the column or row heading of the member you want to set as the slice member. This member is automatically set as the active member for the dimension.

Note

If you select multiple members, the member that has the focus is used to set the active slice.

See also [Using the Member Selector toolbar commands \[page 416\]](#) for more information on selecting members.

19.2.4.2 Selecting the default member

The default member is the default active slice member. It is the member that the Worksheet selects when the dimension is moved to the slice dimension area. Default members are set by the administrator and may be different for individual users.

19.2.4.2.1 To revert to the default member

Procedure

Right-click the relevant slice dimension and click [Set to Default Member](#).

This slice member returns to the default member set by your administrator.

19.2.5 Reordering members

The default order for a dimension's displayed set of members is dictated by the data source. This is usually an alphabetical order that takes into consideration the hierarchical relationship between members. You can alter this arrangement by manually reordering the members within each dimension.

Note

Once you have reordered your members, you cannot return to the server-defined default order. This is not the case for Microsoft OLAP data: once the order has been redefined, any action you perform that causes the set of displayed members to change will result in a return to the server-defined default order.

19.2.5.1 To reorder members

Procedure

1. Right-click a member of the dimension that you want to reorder and click [Reorder Dimension Members](#).
2. Use the mouse to drag members, or use the arrow buttons to move the members up or down the list.

Results

If a row or column member contains a sort or filter, any reordering on the opposite row or column will not be displayed. The selected order will become apparent once the relevant sorts and filters have been removed.

19.2.5.2 Reordering large dimensions

Attempting to invoke the Reorder Members dialog box for a dimension that has an extremely large number of members can result in unstable system behavior. This may arise because your computer does not have enough memory to handle such large member sets. The following error message may appear:

Reordering a large dimension may cause unstable system behavior

Click Cancel to return to the Worksheet. Click OK if you want to continue the operation—this may cause unpredictable results.

To try and reduce the memory allocation required to deal with extremely large member sets, you can remove some of the members from the view before you reorder the members. Use the Member Selector to make sure you have only selected only the members that you want to reorder. For more Member Selector information, see [Selecting row and column members \[page 415\]](#).

19.2.6 Changing member captions

Members have names and captions. Names are defined by the server and cannot be modified in Crystal Reports. Captions are descriptions that can be modified in Crystal Reports.

You can rename a member in Crystal Reports by changing its caption. Alternatively, you can display member names instead of captions, or both names and captions.

19.2.6.1 Renaming a member

Procedure

1. Right-click the member heading that you want to rename and click [Change Caption](#).
2. Enter the new caption for the member and click [OK](#).

To return to the original member caption, click Default.

Results

Changing the caption of a member does not change the name of the member at the data source.

A calculated member uses the unique name to identify a member used in its expression. This unique name is set by the administrator and is not altered if you change the caption. Therefore, when viewing an expression it may be difficult to distinguish the members.

You can alter a caption by selecting the desired member heading on the Worksheet or a label on the chart.

19.2.6.2 Displaying member captions or names

There are three ways of displaying the names of member names in Crystal Reports: member captions only, member names only, and both member names and member captions.

Note

For some cubes, the member names and member captions are the same.

19.2.6.2.1 To override the default for an individual dimension

Procedure

1. Right-click the dimension name.
2. On the shortcut menu, point to *Display Members Using*, and then click one of the display options:
 - Caption
 - Name
 - Caption : Name

Results

This sets the display option for the selected dimension only.

19.2.7 Drilling down

Members in a dimension can be organized in a hierarchy of different levels (groups and subgroups, or parent and child members). For example, a dimension called **region** could have a member for Europe that is the total of UK, France, and Spain. UK could be the total of Scotland, England, Wales, and Northern Ireland, and so on. A plus symbol beside the description of a member indicates that there are child members beneath it.

Often you want to explore data in a hierarchical dimension more closely to find reasons for a performance that is unexpected. In the Worksheet, you can expand a member to show its constituent child members. This is called drilling down.

Moving back up to the parent member in a group is known as drilling up.

There are two methods of drilling down: expanded and focused. For example, drilling down on the member **Wine and Spirits** using the expanded method displays the member and its children:

Wine and Spirits	[-]
Beers	[+]
Other	[+]
Spirits	[+]
Wines	[+]

The focused method displays only the children of the member:

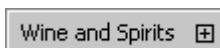
Beers	[+]
Other	[+]
Spirits	[+]
Wines	[+]

19.2.7.1 Expanding a parent member

19.2.7.1.1 To drill down a level

Procedure

Click the plus symbol next to the member name to display its child members:



You can continue drilling down into lower levels as long as a plus symbol appears beside the member name (indicating that there are lower levels).

Results

Alternatively, right-click the member, point to Drill on the shortcut menu, and click Down.

19.2.7.1.2 To drill up a level

Procedure

Click the minus symbol next to the member name.

The children no longer appear in the view and the minus symbol changes to a plus symbol.

Results

Alternatively, right-click the member, point to Drill on the shortcut menu, and click Up.

19.2.7.2 Focusing on child members

19.2.7.2.1 To drill down a level

Procedure

Double-click the member you want to drill down.

The child members replace their parent.

Results

Alternatively, right-click the member, point to *Drill* on the shortcut menu, and click *Focus down*.

19.2.7.2.2 To drill up a level

Procedure

Hold down *SHIFT* and double-click the member you want to drill up.

The parent member replaces its child members.

Results

Alternatively, right-click the member, point to *Drill* on the shortcut menu, and click *Focused up*.

19.2.8 Drilling through

If you want to find out more about the records that contributed to a particular cell value, you can drill through to the query result set.

❗ Note

Full drill-through capability is only available when running Microsoft SQL Server Analysis Services and a Microsoft SQL Server data source.

19.2.8.1 To drill through to the query records

Procedure

Right-click a cell and click *Drill Through* on the shortcut menu.

If suitable relational data is available, the underlying data appears in a Relational Table Viewer:

	store_sales	florist	frozen_sqft	last_remodel_date
1	1.50	False	3,560.91	04/06/1973
2	1.62	False	3,560.91	04/06/1973
3	2.40	False	3,560.91	04/06/1973
4	2.79	False	3,560.91	04/06/1973
5	5.18	False	3,560.91	04/06/1973
6	7.41	False	3,560.91	04/06/1973
7	5.84	False	3,560.91	04/06/1973
8	2.62	False	3,560.91	04/06/1973
9	3.96	False	3,560.91	04/06/1973
10	1.96	False	3,560.91	04/06/1973
11	3.88	False	3,560.91	04/06/1973
12	5.60	False	3,560.91	04/06/1973
13	4.84	False	3,560.91	04/06/1973
14	0.76	False	3,560.91	04/06/1973
15	0.83	False	3,560.91	04/06/1973
16	1.36	False	3,560.91	04/06/1973
17	4.40	False	3,560.91	04/06/1973
18	2.69	False	3,560.91	04/06/1973

Results

Using the toolbar at the top of the table, you can select this data and export it to Excel, or copy it to the clipboard for pasting into other applications.

You can refresh the data from its database by clicking the Refresh Result button.

19.2.9 Using asymmetry in your view

The Member Selector provides an easy and intuitive control to manipulate the view of data, but it works on individual dimensions, not on the row or column. If you have stacked dimensions on the row or column, the asymmetry commands provide more control over your view of data.

The asymmetry controls are available from the shortcut menu on the Worksheet. Right-click a member on the Worksheet to access the commands Hide and Show. These commands allow you to remove members from the current view without using the Member Selector, and also allow you to display an asymmetrical view of your data.

What is an asymmetrical view?

When you stack dimensions on a row or column, the view shows an instance of each inner dimension member for every outer dimension member. For example:

Year	Version
Last Year	Actual
	Budget
	Variance
This Year	Actual
	Budget
	Variance
Next Year	Actual
	Budget
	Variance

Actual, Budget, and Variance members appear for each member of Year.

Using an asymmetric view, you can display different inner dimension members. For example:

Year	Version
Last Year	Actual
This Year	Actual
	Budget
	Variance
Next Year	Budget

In this example, there are no Actual figures for Next Year, and no Budget figures for Last Year. You can hide these members to simplify the view of the data.

This is an asymmetrical view. You must use the commands on the Worksheet shortcut menu to hide individual member instances from the view. You cannot create asymmetrical views using the Member Selector.

19.2.9.1 To remove an instance of a member

Procedure

1. On the Worksheet, right-click the member.
2. Point to *Hide* on the shortcut menu, and click *Selected Member*.

Results

The member instance that you clicked on is removed from the view. All other instances of the member remain as they were.

19.2.9.2 To remove all instances of a member except the selected instance

Procedure

1. On the Worksheet, right-click the member.
2. Point to [Show](#) on the shortcut menu, and click [Selected Member](#).

Results

The instances of the member are removed from the view, except for the selected instance which is retained.

19.2.9.3 Creating asymmetry

You can only create asymmetry on stacked dimensions. There are two shortcut menu commands that can create asymmetry:

[Hide Selected Member](#)

Removes the selected member instance from the current view.

This is not the same as deselecting a member in the Member Selector—that would remove all instances of a member from the view.

For example, you could use Hide Selected Member to hide the **Budget** figures for **Last Year**.

[Show Selected Member](#)

Hides all instances of a member, except for the selected member instance.

For example, you could use Show Selected Member to show **Budget** figures for **Next Year** and hide the **Budget** figures for **This Year** and **Last Year**.

19.2.9.4 Restoring symmetry

You can restore symmetry with the Member Selector by clearing and then selecting a member check box. You can also restore symmetry using the following shortcut menu commands:

Hide All Occurrences

Hides all instances of a member from the current view. This is the same as clearing a member in the Member Selector.

Show All Occurrences

Shows all instances of a member in the current view, and hides all other members. This restores symmetry to the view and displays any hidden instances, but also hides all other members on the dimension.

19.2.9.4.1 To remove all instances of a member

Procedure

1. On the Worksheet, right-click the member.
2. Point to *Hide* on the shortcut menu, and click *All Occurrences*.

Results

All instances of the member are removed from the view. You must use the Member Selector to redisplay the member.

19.2.9.4.2 To show all instances of a member

Procedure

1. On the Worksheet, right-click the member.
2. Point to *Show* on the shortcut menu, and click *All Occurrences*.

Results

Any hidden instances of the member are restored. All other members are hidden.

19.2.9.5 Working with asymmetry

Reorienting dimensions

You can have asymmetry only on stacked dimensions, so reorienting dimensions with asymmetry can result in symmetry being restored. Expect the following behavior:

- Swapping rows with columns preserves asymmetry.
- Moving an asymmetric dimension to the slice dimension area removes asymmetry.
- Removing the outer dimension from an asymmetric row or column removes asymmetry.

Asymmetry and hierarchical dimensions

If you drill down on a member in an asymmetric row or column, the child members inherit the asymmetry from the parent.

For example, if you have two column dimensions, and set up asymmetry so that only **This Year** and **Next Year** are visible for the member **Meat**:

Grocery ⊕			Meat ⊕	
Last Year	This Year	Next Year	This Year	Next Year

Drilling down on the **Meat** member shows that **Meat** has two child members: **Poultry** and **Red Meat**. They inherit the asymmetry from **Meat**, and display only **This Year** and **Next Year**.

Grocery ⊕			Meat ⊖		Poultry ⊕		Red Meat ⊕	
Last Year	This Year	Next Year	This Year	Next Year	This Year	Next Year	This Year	Next Year

19.3 Changing the formatting

This section describes how to change the appearance of your data by adding formatting and by resizing rows and columns.

19.3.1 Formatting data in the Worksheet

You can choose different formats for data that meets specific conditions, but you can also set font formats unconditionally. For details, see [Highlighting exceptions \[page 437\]](#).

19.3.1.1 To set the format of displayed data

Procedure

1. Right-click a row or column member; point to *Format* and click *Add*.
2. On the *Type* list, click the number format that you want to use.

The formatting options depend on the type that you selected:

<i>Server Defined</i>	You cannot specify the number format.
<i>Number</i>	You can set the number of decimal places, change the thousands separator, show values as percentages, or scale values down.
<i>Currency</i>	You can set the number of decimal places, change the thousands separator, choose a currency symbol, or scale values down.
<i>Accounting</i>	You can set the number of decimal places, change the thousands separator, use a plus symbol for positive values and brackets for negative values, or scale values down.
<i>Scientific</i>	You can set the number of decimal places, change the thousands separator, or use scientific notation.

3. Click the *Font* tab.
4. Set the font formatting.

Foreground is the color of the text itself and Background is the color of the cell.

19.3.1.2 Notes on formatting

When you apply different formatting to columns and rows, there can be conflicts where columns and rows intersect. Such conflicts are resolved as follows:

- The last formatting applied takes precedence.
- Font attributes are additive, so, for example, Italic set in columns and Bold set in rows results in Bold Italic at the intersection.
- Where you don't want the column font and colors to overwrite those you set for rows, leave the color and font boxes blank in the columns format dialog box.

Formatting only changes the view of the data, not the value of the data itself. Displayed data may be rounded off. To see the real value of the data in a cell (usually to many decimal places), right-click the cell and click Show Cell Properties.

19.3.2 Resizing columns

When a Worksheet is rendered, it automatically sets the columns to an optimal width in order to view the data that has been retrieved. As this may obscure a member heading or restrict the amount of cells displayed in a single view, you can resize the columns.

19.3.2.1 To resize the column width

Procedure

1. Select the column or columns you want to resize.

You can select multiple columns by holding down CTRL and clicking multiple members.

2. Point to one of the selected column heading separators until the resize symbol appears:



3. Drag the separator to give the width you require.

19.3.3 Resizing rows

Cell data may span multiple lines. You can resize rows to show all the data within a cell.

19.3.3.1 To resize the row height

Procedure

1. Select the row or rows you want to resize.

You can select multiple rows by holding down CTRL and clicking multiple members.

2. Point to one of the selected row heading separators until the resize symbol appears:



3. Drag the separator to give the width you require.

19.3.4 Viewing cell and member properties

The data you see in a Worksheet cell is usually rounded up or formatted in some other way. You can view the original value held in the cube by using the Cell Properties commands.

Note

Filtering is performed on the original, unformatted values. This can lead to apparent small discrepancies in excluding or including cells. For example, a value displayed as 100.00 would be excluded after a filter has been applied excluding only numbers greater than 100, if its original value were 100.005. For details, see [Filtering data \[page 444\]](#).

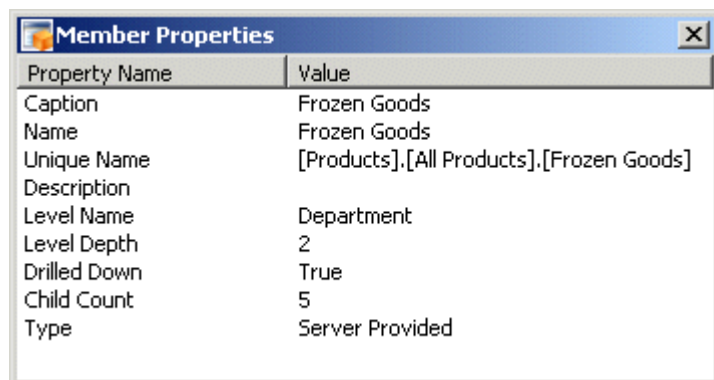
You can also view additional information for a specific member. The information includes its name, caption, unique name, and position in the hierarchy.

19.3.4.1 To view member properties

Procedure

Right-click a row or column member and click [Show Member Properties](#).

A dialog box appears revealing the member properties.



19.4 Highlighting exceptions

This section describes how to highlight important differences or unexpected results in your data. Typically the results that you want are either high or low absolute values, members that seem unusual compared to other members, or exceptions that result from calculations you specify.

19.4.1 Overview

You can apply conditions to the data in a Worksheet to highlight important differences or unexpected results.

				Measures ▾
Products ▾	Sales	Cost	Margin	↓
All Products	170,888,357.64	139,298,880.85	31,589,476.80	
Grocery	76,376,769.52	63,676,754.16	12,700,015.30	
Wine and Spirits	12,853,800.35	9,575,611.58	3,278,188.77	
Frozen Produce	4,908,030.29	4,317,634.95	590,395.34	
Speciality Rice	176,772.35	155,875.23	20,897.12	
Speciality Breads	173,823.78	153,299.46	20,524.32	
Dairy Produce	92,641.62	80,164.46	12,477.15	
Pastry	20,505.16	18,098.55	2,406.61	
Version . Actual ▾	Week . All Weeks ▾	Year . All Years ▾		

For example, the data might be above or below a predetermined value, or above or below budget. The Worksheet provides several types of exception reporting:

- [Highlighting high or low absolute values \[page 439\]](#).
- [Highlighting exceptions compared with other members \[page 440\]](#).
- [Highlighting exceptions based on a calculation result \[page 441\]](#).

The attribute settings are stored when you save the view, so the Worksheet looks the same when you reload it.

By default, values above the upper limit are displayed with a green background, those below the lower limit with a red background, and intermediate values with a yellow background. To change these colors, see [Using other formats to highlight exceptions \[page 442\]](#).

Note

Highlighting Exceptions colors all cells in a row or column. If you add other conditions to the Worksheet affecting the same members, the last condition applied is used.

19.4.2 Applying global highlighting

Usually highlighting is applied to individual row or column members, but you can also apply highlighting to the whole Worksheet.

19.4.2.1 To apply global highlighting

Procedure

1. Right-click the top left corner of the Worksheet.
2. Click [Global Highlighting Exceptions](#) on the shortcut menu.
3. Move the [Tolerance](#) sliders to set the upper and lower tolerance values. Alternatively, type the tolerance values in the boxes.
4. Click [Formatting](#) for more formatting options.

See [Using other formats to highlight exceptions \[page 442\]](#) for more details.

5. Click [OK](#).

Results

For information on changing the cell format unconditionally, see [Formatting data in the Worksheet \[page 434\]](#).

19.4.3 Highlighting high or low absolute values

You can color code data in the Worksheet according to its absolute value. This allows you to easily pick out numbers with exceptionally high or low values.

You must specify the high and low exception values. The Worksheet colors the cells based on the exception values you provide. By default, the Worksheet uses red, yellow and green to color cells:

- Cells below the low value are colored red.
- Cells between the two values are colored yellow.
- Cells above the high value are colored green.

To change these colors or use other formatting parameters to highlight exceptions, see [Using other formats to highlight exceptions \[page 442\]](#).

19.4.3.1 To highlight high or low absolute values

Procedure

1. Right-click a row or column member. Point to [Highlight Exceptions](#) and click [Add](#).
2. On the [Highlighting Method](#) list, click [Data Cell Value](#).
3. Click [Using the value displayed in each data cell](#).
4. Move the [Tolerance](#) sliders to set the upper and lower tolerance values. Alternatively, type the tolerance values in the boxes.

5. Click [Formatting](#) for more formatting options.

See [Using other formats to highlight exceptions \[page 442\]](#) for more details.

6. Click [OK](#).

19.4.4 Highlighting exceptions compared with other members

As well as highlighting exceptional absolute values, you can also draw attention to unusual relationships with other dimension members.

For example, you could highlight Sales variances between two Products by coloring variances of \$10,000 or more in green; Variances of less than \$10,000 but more than -\$10,000 in yellow; and Variances of -\$10,000 or greater in red.

Note

This option is not available when applying Global highlighting.

19.4.4.1 To highlight exceptions compared with other members

Procedure

1. Right-click a row or column member; point to [Highlight Exceptions](#) and click [Add](#).
2. On the [Highlighting Method](#) list, click [Data Cell Value](#).
3. Click [Compared to another dimension member](#).
4. Click [Choose](#).
5. Using the Member Selector, select a member for comparison.
6. Click [OK](#).
7. Move the [Tolerance](#) sliders to set the values above and below the comparison member. Alternatively, type the tolerance values in the boxes.

For example, if you consider 10,000 above and 10,000 below the base member to be exceptional, set the sliders to 10000 and -10000.

8. Click [Formatting](#) for more formatting options.

See [Using other formats to highlight exceptions \[page 442\]](#) for more details.

9. Click [OK](#).

19.4.5 Highlighting exceptions based on a calculation result

You can highlight exceptions from expected data once a calculation on the member has been performed.

Note

This option is not available when applying Global highlighting.

For example, you could use a calculation to determine the Budget variance of a product, then color code the results in three bands: 5% or more above Budget, less than 5% above but more than 5% below, and 5% or more below.

19.4.5.1 To highlight exceptions using the result of a calculation

Procedure

1. Right-click a row or column member. Point to *Highlight Exceptions* and click *Add*.
2. On the *Highlighting Method* list, click *Data Cell Value*.
3. Click *Use the result of a calculation*.
4. Click *Define* to display the Calculated Members dialog box.
5. Set up the calculation using the Calculation Expert, Data Analysis Expert, or as a calculation definition.
See [Adding calculated members \[page 450\]](#) for more information on defining calculations.
6. Move the *Tolerance* sliders to set the values above and below the result of the calculation. Alternatively, type the tolerance values in the boxes.
For example, if you consider 5 above and 5 below to be exceptional, set the sliders to 5 and -5.
7. Click *OK*.

19.4.6 Editing highlighting

Context

You can change the condition and highlighting format at any time using the Edit Highlighting command.

Procedure

1. Right-click the member whose highlighting you want to edit.
2. Point to *Highlight Exceptions* and click *Edit*.

- 3. In the *Highlight Exceptions* dialog box, make changes to the conditions and formatting.

19.4.7 Removing highlighting

Context

You can remove exception highlighting at any time using the *Remove Highlighting* command.

Procedure

- 1. Right-click the member whose highlighting you want to remove.
- 2. Point to *Highlight Exceptions* and click *Remove*.

19.4.8 Using other formats to highlight exceptions

By default, exceptions are highlighted using green, yellow, and red cell backgrounds. You can change the colors and fonts in which cells are displayed.

When you apply formatting to dimension members, the formatting moves with the dimensions.

19.4.8.1 To change the formatting

Procedure

- 1. Right-click the row or column member. Point to *Highlight Exceptions* and click *Add*.
- 2. Click *Formatting* to display the formatting options.
- 3. Click *Format* for the range that you want to edit.
- 4. Click the *Number* tab.
- 5. On the *Type* list, click the number format that you want to use.

The formatting options depend on the type that you selected:

Server Defined

You cannot specify the number format.

Number

You can set the number of decimal places, change the thousands separator, show values as percentages, or scale values down.

<i>Currency</i>	You can set the number of decimal places, change the thousands separator, choose a currency symbol, or scale values down.
<i>Accounting</i>	You can set the number of decimal places, change the thousands separator, use a plus symbol for positive values and brackets for negative values, or scale values down.
<i>Scientific</i>	You can set the number of decimal places, change the thousands separator, or use scientific notation.

6. Click the *Font* tab.
7. Change the font settings.
8. Click *OK*.

19.4.8.2 Notes on formatting

Foreground is the color of the text and Background is the color of the cell.

When you apply different formatting to columns and rows, there can be conflicts where columns and rows intersect. Such conflicts are resolved as follows:

- Column formatting generally takes precedence over row formatting.
- Font attributes are additive. For example, Italic set in columns and Bold set in rows results in Bold Italic at the intersection.
- Where you don't want the column font and colors to overwrite those you set for rows, leave the color and font boxes blank in the columns format dialog box.

Formatting only changes the view of the data, not the value of the data itself. However, your formatting may result in displayed data being rounded off. To see the real value of the data in a cell (usually to many decimal places), right-click the cell and click Show Cell Properties. See [Viewing cell and member properties \[page 437\]](#) for more information.

19.5 Filtering and sorting data

This section shows you how to filter and sort your data.

Filters enable you to cut through a great mass of data so you can concentrate on the information that is important to you.

Use sorting to order the data in the Worksheet. This can be useful, for example, if you want to rank sales regions according to their revenue. Once you sort your data, it is easy to see which sales region is the most successful.

19.5.1 Filtering data

You can apply filters to the data displayed in the Worksheet, using a number of different criteria to exclude members from or include members in the view. This helps you suppress data so that you can work only with the information that is important to you.

You can use the following criteria:

- Actual values
- Top / bottom n based on ranking
- Top / bottom n based on contribution.

If you specify a filter for a column member, any data value that satisfies the condition is affected, together with the rest of its row.

If you specify a filter for a row member, any data value that satisfies the condition is affected, together with the rest of its column.

If you have stacked dimensions in a row or column, you can only filter on the inner dimension members.

Note

Filtering is performed on the original, unformatted values of cells. For details, see [Viewing cell and member properties \[page 437\]](#). This can lead to apparent small discrepancies in excluding or including cells. For example, a value displayed as 100.00 would be excluded after a filter has been applied excluding only numbers greater than 100, if its original value were 100.005.

Examples of filtering

You can define filters to display:

- Products with Sales volumes of greater than 1000 units only.
- The ten most profitable products in absolute terms.
- The products that make up the top 10% of sales.

The following sections tell you how to define such filters.

19.5.1.1 Filtering by actual value

You can apply, to a member, filters that are based on the actual cell values with which the member is associated; in other words, no comparisons are made with other members.

The conditions you can use are the following:

- equal to
- not equal to
- less than

- less than or equal to
- greater than
- greater than or equal to
- between two numbers
- outside two numbers
- missing

19.5.1.1.1 To filter based on a value

Context

For example, display Products with Sales volumes greater than 1000 units.

Procedure

1. Right-click the row or column member you want to filter; point to *Filter*, and click *Add*.
2. On the *Filter Type* list, click *Actual values*.
3. Click the *show* or *hide* option.
4. On the *Cell value is* list, click the required filter (for example *is greater than*).
5. Enter the filter value in the box.
6. Click *OK*.

Results

A cross symbol appears next to the member name to show that it has a filter.

19.5.1.2 Filtering by ranking

You can choose to hide or only show the top or bottom *n* members in a dimension.

19.5.1.2.1 To filter based on ranking

Context

Display the ten top-selling products:

Procedure

1. Right-click the row or column dimension you want to filter; point to *Filter*, and click *Add*.
2. On the *Filter Type* list, click *Top / bottom n*.
3. Click the *show* or *hide* option.
4. Click *top* or *bottom*.
5. Type the number of members to exclude or display.
6. Click *OK*.

Results

A cross symbol appears next to the member name to show that it has a filter.

19.5.1.3 Filtering by contribution (Pareto analysis)

You can choose to hide or only show the top or bottom *n* percent of members in a dimension.

This function calculates the top or bottom *n* percent of a range, then ranks the members in order and shows (or hides) the members that make up that percentage.

19.5.1.3.1 To filter based on contribution

Context

Display the products which make up the top 10% of sales:

Procedure

1. Right-click the row or column dimension you want to filter; point to *Filter*, and click *Add*.
2. On the *Filter Type* list, click *Top / bottom n%*.
3. Click the *show* or *hide* option.
4. Click *top* or *bottom*.
5. Type the percentage you want the filter to apply to.
6. Click *OK*.

Results

A cross symbol appears next to the member name to show that it has a filter.

19.5.1.4 Changing a filter

Procedure

1. Right-click the member that has the filter.
2. Point to [Filter](#), and click [Edit](#).
3. Modify the filter settings.

19.5.1.5 Removing a filter

Procedure

1. Right-click the member that has the filter.
2. Point to [Filter](#) and click [Remove](#).

19.5.1.6 Swapping dimensions containing filters

If you specify a filter for a row, and then swap dimensions so that the row appears as a column, the filter moves with the dimension. The same occurs if you move a filtered column to a row. This only applies if you have a single dimension on the row and column axes.

If you swap a row or column dimension with a slice, however, the filter is lost.

Adding or removing dimensions on the row or column axes will remove any filters that have been added.

19.5.1.7 Filtering when Filter Null Values is also set

Filtering is applied to the underlying data, rather than the displayed data. Hence, if Filter Null Values is set, null values, even if not displayed, will still be taken into account when the bottom values are determined. This may mean that fewer of the values than you would expect are filtered. See [Excluding null values \[page 448\]](#) for further details.

19.5.1.8 Excluding null values

By default, null values are indicated in the Worksheet by the word *Null*.

You can remove Null values from the Worksheet by right-clicking the top left corner of the Worksheet and clicking Filter Null Values.

Note

This only applies to a complete row or column of null values.

19.5.2 Sorting data

You can use sorting to order the data in the Worksheet. This can be useful, for example, if you want to rank sales regions according to their revenue. Once sorting is applied, it is easy to see which sales region is the most successful.

19.5.2.1 Sorting on a member

When you sort a row member, the data values are sorted, together with their column headings. When you sort a column member, the data values are sorted, together with their row headings.

19.5.2.1.1 To sort the data values in a row or column

Procedure

Right-click a member; point to *Sort* and click *Add First/Ascending* or *Add First/Descending*.

The data values are sorted into ascending or descending numerical order, together with their column or row headings.

Results

An arrow symbol on the member indicates sorted data—the arrow symbol points in the direction of decreasing value.

Cost	↓
139,298,880.85	
63,676,754.16	
26,554,678.06	
26,261,723.40	
9,575,611.58	
8,505,040.99	

The hierarchy of the data is preserved—parent members are sorted in order, and child members below the parents are sorted in their own order. This is only possible if one sort is applied.

Cells that are uninitialized (null) or invalid are ranked in value below any other cells; they appear last in a descending and first in an ascending sort. They are shown as *Null* in the Worksheet.

19.5.2.1.2 To remove a sort

Procedure

1. Right-click the member.
2. Point to *Sort*, and click *Remove*.

19.5.2.2 Restrictions on sorting

If you have stacked dimensions in a row or column, you can only sort on the inner dimension or dimensions.

Only displayed members are sorted within each hierarchical group: if you required all the members in a dimension to be sorted on the same basis, you must show all members then choose a Break Hierarchies sort option.

19.5.2.3 Swapping sorted dimensions

If you sort a row, and then swap dimensions so that the row appears as a column, the sort moves with the dimension. The same applies if you move a sorted column to a row.

If you swap a row or column dimension with a slice, however, the sort is lost.

19.5.2.4 Preserving and breaking hierarchies

The option Add First/Ascending (or Descending) will preserve a data hierarchy on one sort only. See [Sorting on a member \[page 448\]](#).

Any subsequent sorts added to a Worksheet are not able to preserve a hierarchy: only the options Ascending/Descending, Breaks Hierarchies are available for subsequent sorts.

You can choose Add First Sort/Ascending (or Descending) for a subsequent sort, but this automatically removes all other sorts.

Similarly, if you clear Break Hierarchies on a sort, any other sorts are removed.

19.5.2.5 Changing the sort order

Click the arrow symbol next to the dimension or member name, or right-click the member and choose Change Direction from the Sort submenu.

The members or data values are sorted again, together with their rows or columns. The arrow symbol changes to show the direction of the sort—the arrow symbol points in the direction of decreasing value.

19.5.2.6 Sorting multiple members in a dimension

You can apply multiple sorts to the data in the Worksheet. The second sort will only order numbers that are still equal after the first sort.

For example, if you were analyzing the results of a soccer competition, and found after sorting on points scored that there was a tie for first place, you could apply a second sort on goal difference to determine the overall winner.

You can have up to three sorts on a row or column. These are referred to as the first, second, and third sorts. The sorts are applied in the order in which they are specified.

Multiple sorts are indicated by the numbers one to three.

If you delete the first sort on a row or column, the second sort becomes the first sort, and so on.

19.6 Adding calculated members

This section describes how to add the various types of calculated members to your Worksheet. You can add automatic totals to your data, add calculated members and data analysis calculations using experts, or define your own calculations using MDX or OLAP Syntax.

19.6.1 Adding Automatic Totals to rows and columns

You can insert a column or row into your view to display the sum of the data values for the corresponding row or column on the Worksheet.

19.6.1.1 To add automatic totals

Procedure

1. Right-click a member heading.

2. Point to [Automatic Totals](#), and click one of the following:

- Rows (Across)
- Columns (Down)
- Both
- No Totals

Selecting Both adds totals to both the rows and columns.

Year			
Products	Last Year	This Year	Total
Bakery	4,994,659.36	5,516,062.47	10,510,721.83
Frozen Goods	2,909,240.98	2,462,532.21	5,371,773.20
Fruit and Vegetable	18,769,419.00	14,666,980.40	33,436,399.40
Grocery	43,843,819.51	32,532,950.01	76,376,769.52
Meat	17,863,646.65	14,475,246.71	32,338,893.35
Wine and Spirits	6,838,943.17	6,014,857.17	12,853,800.35
Total	95,219,728.68	75,668,628.97	170,888,357.64
Version . Actual	Week . All Weeks	Measures . Sales	

Results

Totals are only displayed by the Worksheet, they do not appear in charts.

Note

When adding an automatic Total by Axis to a row or column displaying parent and child members, the child member is counted twice: once as a single member and again as a contribution to the total of its parent member.

19.6.2 Adding calculated members to the Worksheet

You can add calculated members to your Worksheet using the following calculation experts:

- Contribution (%)
- Growth
- Ranking
- Variance

You can have several calculated members in a Worksheet. They are calculated in the order in which they were added to the Worksheet. This is important if one calculation uses the results of a previous one.

19.6.2.1 To add a calculated member using an Expert

Procedure

1. Right-click any member in the dimension.
2. On the shortcut menu, point to *Calculated Member* and click *Add*.
3. Click the *Calculation Expert* tab.
4. On the *Calculation Type* list, click the type of calculation that you want:

<i>Contribution (%)</i>	Calculates the percentage contribution by each member in a hierarchical dimension.
<i>Growth</i>	Calculates how a value has grown from one point in time to another. The growth is expressed either in absolute or percentage terms.
<i>Ranking</i>	Ranks the members of a dimension according to specific criteria.
<i>Variance</i>	Compares the value of one dimension member with a target value; the resulting variance can be expressed as an absolute value or a percentage variance.

19.6.2.2 Adding calculated members that show contribution percentage

For example, to add a member showing the percentage contribution of the Bakery member to the overall total.

19.6.2.2.1 To add a calculated member that shows contribution percentage

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.
2. On the shortcut menu, point to *Calculated Member* and click *Add*.
3. Click the *Calculation Expert* tab.
4. On the *Calculation Type* list, click *Contribution%*.
5. Click *Calculate contribution to the overall total* or *Calculate contribution to each level*.
6. On the Member Selector, right-click the member whose contribution you are calculating.

7. On the shortcut menu, click [Add to 'Contribution of' field](#).

You can click the Calculation tab now to see the calculation definition.

8. Click [OK](#).

19.6.2.3 Adding calculated members showing growth

For example, to add a member showing how much the Bakery group has grown from one time period to another.

19.6.2.3.1 To add a calculated member showing Growth

Procedure

1. Right-click a member on the dimension where you want to add the calculated member. This must be the member whose growth you are calculating.
2. On the shortcut menu, point to [Calculated Member](#) and click [Add](#).
3. Click the [Calculation Expert](#) tab.
4. On the [Calculation Type](#) list, click [Growth](#).
5. If you want to show the result as a percentage, select [Calculate growth as a percentage](#).
6. On the Member Selector, right-click the member you are calculating the growth for.
7. On the shortcut menu, click [Add to 'Calculate Growth of' field](#).
8. On the [Time Dimension](#) list, click the dimension that you want to calculate the growth against.

You can click the Calculation tab now to see the calculation definition.

9. Click [OK](#).

19.6.2.4 Adding calculated members showing ranking

For example, to rank the sub-categories of the All Products member to detect which products are the most popular.

19.6.2.4.1 To add a calculated member showing Ranking

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.

This must be the member you want to rank.

2. On the shortcut menu, point to *Calculated Member* and click *Add*.
3. Click the *Calculation Expert* tab.
4. On the *Calculation Type* list, click *Ranking*.
5. Click the option specifying how you want to calculate the rank:

<i>Calculate rank across the dimension</i>	Calculate the rank for each member over the whole dimension.
<i>Calculate rank within each level</i>	Calculate the rank for each member according to the level that it is part of. This option breaks the hierarchical structure.
<i>Calculate rank within siblings</i>	Calculate the rank for each member in a sibling group. That is, those members related by a common parent member.

6. On the Member Selector, right-click the member that you want to rank.
7. On the shortcut menu, click *Add to 'Rank based on' field*.
8. Select the dimension that you want to calculate the rank against.

You can click the Calculation tab now to see the calculation definition.

9. Click *OK*.

19.6.2.5 Reversing the order of ranking

The ranking expert automatically arranges members into a descending order, with the greatest value receiving the highest rank. To change the arrangement to an ascending order, you can alter the calculation in the Calculation Definition box.

19.6.2.5.1 To reverse the order of ranking

Procedure

1. Right-click the calculated member.
2. On the shortcut menu, point to *Calculated Member* and click *Edit*.
3. Click the *Calculation* tab.
4. In the *Calculation Definition* box, edit the calculation string.

Use *BASC* to specify ascending, and *BDESC* to specify descending order.

5. Click *OK*.

19.6.2.6 Adding calculated members showing variance

For example, to add a member showing the variance between Budget and Actual sales.

19.6.2.6.1 To add a calculated member showing Variance

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.
2. On the shortcut menu, point to *Calculated Member* and click *Add*.
3. Click the *Calculation Expert* tab.
4. On the *Calculation Type* list, click *Variance*:
5. On the Member Selector, right-click the member to compare against the target member (in this case, Actual.)
6. On the shortcut menu, click *Add to 'Compare' field*.
7. On the Member Selector, right-click the target member for calculating the variance (in this case, Budget.)
8. On the shortcut menu click *Add to 'To target of' field*.
9. Click *Values less than the target are good* or *Values less than the target are bad*.

In this example, values less than Budget are bad.

10. Click *Calculate variance as a percentage* if required.

You can click the Calculation tab now to see the calculation definition.

11. Click *OK*.

19.6.2.7 To add a calculated member using an Expert

Procedure

1. Right-click any member in the dimension.
2. On the shortcut menu, point to *Calculated Member* and click *Add*.
3. Click the *Calculation Expert* tab.
4. On the *Calculation Type* list, click the type of calculation that you want:

Contribution (%)

Calculates the percentage contribution by each member in a hierarchical dimension.

<i>Growth</i>	Calculates how a value has grown from one point in time to another. The growth is expressed either in absolute or percentage terms.
<i>Ranking</i>	Ranks the members of a dimension according to specific criteria.
<i>Variance</i>	Compares the value of one dimension member with a target value; the resulting variance can be expressed as an absolute value or a percentage variance.

19.6.3 Adding data analysis calculations

In addition to the standard calculated members, you can also add three types of data analysis calculations. These are available from the Data Analysis Expert tab in the Calculated Member dialog box. Use this tab to choose one of these predefined calculations:

- Trend Line
- Moving Average
- Linear Regression (X and Y)

You can also use the Calculation tab to define a Custom Data Analysis calculation. This might be a modification of one of the predefined calculations, or you could define a completely new data analysis calculation.

19.6.3.1 Adding a trend line calculation

For example, to add a member showing the straight line trend of sales over the last twelve months.

19.6.3.1.1 Adding a trend line calculation

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.
2. On the shortcut menu, point to *Data Analysis*, and click *Add Calculation*.
This opens the Calculated Members dialog box on the Data Analysis Expert tab.
3. On the *Calculation Type* list click *Trend Line*.
4. On the Member Selector, right-click the member for which you want to calculate the trend line.
5. On the shortcut menu, click *Add to 'Trend of' field*.
6. On the *Series Dimension* list, click the dimension that you want to calculate the trend against.

You can click the Calculation tab now to see the calculation definition.

7. Click [OK](#).

19.6.3.2 Adding a moving average calculation

For example, to add a member showing a centered average of sales over a three month period.

19.6.3.3 Adding a linear regression calculation

For example, to add a member showing expected store sales based on the statistical relationship between store sales and store size.

19.6.3.3.1 To add a linear regression calculation

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.
2. On the shortcut menu, point to [Data Analysis](#), and click [Add Calculation](#).
This opens the Calculated Members dialog box on the Data Analysis Expert tab.
3. On the [Calculation Type](#) list, click [Linear Regression \(X and Y\)](#).
4. On the Member Selector, right-click the member you want to use for the Y values in the regression calculation (in this case, **Store Sales**).
5. On the shortcut menu, click [Add to 'Y Values' field](#).
6. On the Member Selector, right-click the member you want to use for the X values in the regression (in this case, **Store Size**).
7. On the shortcut menu click [Add to 'X Values' field](#).

You can click the Calculation tab now to see the calculation definition.

8. Click [OK](#).

19.6.4 Using calculation definitions

Calculations are defined in the OLAP Syntax language. For Microsoft Analysis Services, calculations are defined in the MDX query language.

You can create a custom calculation by typing in a calculation definition, or change an existing calculation by editing a calculation definition.

19.6.4.1 Adding a custom calculation using a calculation definition

You can add a custom calculation by typing a calculation definition.

19.6.4.1.1 To add a calculated member

Procedure

1. Right-click a member on the dimension where you want to add the calculated member.
2. On the shortcut menu, point to *Calculated Member*, and click *Add*.
3. In the Calculated Members dialog box, click the *Calculation* tab.

The Calculation tab consists of three areas:

- A Calculation Definition box where you define the calculation by using the OLAP Syntax or MDX query language.
 - A name box for the name of the new calculated member.
 - Tools to help you define the calculation (a Member Selector, keypad and Function button).
4. Check that you have the right dimension for the calculated member.
 5. Define the calculation.

Use the Member Selector to pick members that you want to use in the calculation. Right-clicking the member allows you to add it to the calculation. You can use all of the members in the specified dimension, including any calculated members that have been created.

Note

The calculation uses the unique name of a member rather than its caption. The Member Selector displays member captions.

You can also use functions in your definition. See [Adding functions to a calculation definition or MDX Query \[page 458\]](#) for information on adding functions from the library.

6. In the *Calculation Name* box, type a name for the calculated member.
7. Click *OK*.

19.6.4.2 Adding functions to a calculation definition or MDX Query

You can either type in a function, or select one from the Function Library dialog box.

19.6.4.3 OLAP Syntax

The OLAP Syntax provides a range of functions for defining your calculated members. The functions are divided into the following functional categories:

- Dimension Functions
- Level Functions
- Logical Functions
- Member Functions
- Numeric Functions
- Set Functions
- Statistical Functions
- String Functions
- Tuple Functions

For more information concerning the use of these functions, see [OLAP Syntax \[page 461\]](#).

19.6.4.4 MDX Query functions

You can add the standard range of functions to an MDX Query in a calculation. These are listed below:

- Array Functions
- Dimension Functions
- Hierarchy Functions
- Level Functions
- Logical Functions
- Member Functions
- Numeric Functions
- Set Functions
- String Functions
- Tuple Functions

For more information on the use of these functions, see the Microsoft SQL Server OLAP Services documentation.

19.6.4.5 Editing calculated members

Procedure

1. Right-click the calculated member.
2. Point to [Data Analysis](#) and click [Edit Calculation](#), or point to [Calculated Member](#) and click [Edit](#).
3. In the Calculated Members dialog box, edit the calculation.

19.6.4.6 Removing calculated members

Procedure

1. Right-click the calculated member.
2. Point to *Calculated Member* and then click *Remove*.

19.6.4.7 Switching between types of calculated members

You can switch between types of calculated members by clicking the appropriate tab in the Calculated Members dialog box.

You can replace a data analysis calculation with a standard calculated member, or replace a standard calculated member with a data analysis calculation.

19.6.4.7.1 To change to a standard calculated member

Procedure

1. From the *Data Analysis Expert* tab or the *Calculation* tab, click the *Calculation Expert* tab.
2. On the *Calculation Type* list, click the calculation you want.
3. Change the caption of the calculated member in the *Calculation Name* box as appropriate.
4. Click *OK*.

19.6.4.7.2 To change to a data analysis calculation

Procedure

1. From the *Calculation Expert* tab or the *Calculation* tab, click the *Data Analysis Expert* tab.
2. On the *Calculation Type* list, click the data analysis calculation you want.
3. Change the caption of the data analysis calculation in the *Calculation Name* box as appropriate.
4. Click *OK*.

19.7 OLAP Syntax

This section introduces the OLAP Syntax and provides a reference for the various functions provided by the language.

19.7.1 Introducing the OLAP Syntax

The OLAP Syntax provides a language to aid the manipulation of dimensions and members.

The syntax has a number of functions, you can use these functions to construct calculated members.

19.7.1.1 Specifying a dimension

To specify dimensions you can simply use the dimension name.

For Essbase and DB2 data sources, the dimension name takes the form:

```
dimension_name
```

For SAP data sources, the dimension name takes the form:

```
[DIMENSION_NAME]
```

19.7.1.2 Specifying a member

Essbase and DB2 unique names

To specify a member, use its unique name enclosed in curly brackets. Essbase and DB2 unique names are composed of the dimension name, followed by the subsequent ascendants of the required member. The components in the unique name are separated by an @:

```
{dimension_name@first_level_member@required_member}
```

For example, to specify the **Fruit** member in the **Products** dimension:

```
{Products@AllProducts@Fruit&Veg@Fruit}
```

You can also specify a member using just the member name, if that member name is unique to the cube:

```
{Fruit}
```

SAP unique names

SAP unique names are specified by the dimension name, followed by the member name:

```
{[DIMENSION_NAME].[REQUIRED_MEMBER]}
```

For example:

```
{[PRODUCTS].[8]}
```

Note

SAP dimension and member names are enclosed with [].

To find the unique name of any member, right-click the desired member, then click Show Member Properties on the shortcut menu.

19.7.1.3 Specifying a level

A level is specified by dimension name, followed by the level name.

For Essbase and DB2:

```
{dimension_name@level_name}
```

For SAP:

```
{[DIMENSION_NAME].[LEVEL_NAME]}
```

To find the name of a level, right-click a member from the desired level, then click Show Member Properties on the shortcut menu.

19.7.1.4 Notation

The OLAP Syntax is represented in a font like this:

```
Extract(<Set>,<Dimension>[,<Dimension>]...)
```

with parameters identified by italics.

The following special symbols are used:

[]

Encloses parameters that are optional and extend the functionality of the function. Do not include the brackets in your calculation definition.

	Separates two or more possible instances. You can include only one as the parameter.
...	Indicates repeated instances of the previous item.

19.7.2 Dimension functions

Click a link to view one of the topics in this section:

- [Dimension - Level \[page 463\]](#)
- [Dimension - Member \[page 464\]](#)
- [Dimensions \[page 465\]](#)

19.7.2.1 Dimension - Level

OLAP Syntax.

Returns the dimension that contains a specified level.

Syntax

```
Dimension(<Level>)
```

Parameters

<Level>	A level from the required dimension.
---------	--------------------------------------

Return value

Returns a dimension.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `Products`, which is the dimension containing the level containing `Fruit`.

```
Dimension ( Level({'Fruit'}) )
```

See also [Level - Member \[page 466\]](#).

19.7.2.2 Dimension - Member

OLAP Syntax.

Returns the dimension that contains a specified member.

Syntax

```
Dimension(<Member>)
```

Parameters

<Member>

A member from the required dimension.

Return value

Returns a dimension.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `Products`, which is the dimension containing the member `Apples`.

```
Dimension ( {'Apples'} )
```

19.7.2.3 Dimensions

OLAP Syntax.

Returns the dimension whose position within the cube is specified by a numeric expression.

Syntax

```
Dimensions(<NumericExpression>)
```

Parameters

<NumericExpression>

An expression that evaluates to a number, specifying the position of the dimension within the cube.

Return value

Returns a dimension.

Remarks

The first dimension within the cube has a position of 0. The second dimension has an position of 1, and so on.

Example

The following example returns the third dimension in the cube:

```
Dimensions(2)
```

19.7.3 Level functions

Click a link to view one of the topics in this section:

- [Level - Member \[page 466\]](#)
- [Levels - Numeric \[page 466\]](#)
- [Levels - String \[page 467\]](#)

19.7.3.1 Level - Member

OLAP Syntax.

Returns the level of a specified member.

Syntax

```
Level(<Member>)
```

Parameters

<Member>

A member from the required level.

Return value

Returns a level.

Example

The following example returns the level of the current member of the **Products** dimension:

```
Level ( CurrentMember({Products}) )
```

See also [CurrentMember \[page 492\]](#).

19.7.3.2 Levels - Numeric

OLAP Syntax.

Returns the level whose position in a dimension is specified by a numeric expression.

Syntax

```
Levels ( <Dimension>, <NumericExpression> )
```

Parameters

<Dimension>	Dimension name.
<NumericExpression>	An expression that evaluates to a number. This is the hierarchical level in the dimension.

Return value

Returns a level.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `level-2` the third level in the `Products` dimension:

```
Levels ( {Products}, 3 )
```

19.7.3.3 Levels - String

OLAP Syntax.

Returns the level whose name is specified by a numeric string expression.

Syntax

```
Levels ( <LevelString> )
```

Parameters

<LevelString>

A string expression specifying a level name.

Return value

Returns a level.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `level-1` from the `Products` dimension:

```
Levels ( {Products@level-1} )
```

19.7.4 Logical functions and operators

Click a link to view one of the topics in this section:

- [And \[page 469\]](#)
- [Between \[page 470\]](#)
- [Eqv \[page 471\]](#)
- [Imp \[page 472\]](#)
- [Is \[page 473\]](#)
- [Is NULL \[page 474\]](#)
- [IsAncestor \[page 474\]](#)
- [IsEmpty \[page 475\]](#)
- [IsGeneration \[page 476\]](#)
- [IsLeaf \[page 477\]](#)
- [IsSibling \[page 478\]](#)
- [Not \[page 479\]](#)
- [Or \[page 480\]](#)
- [Outside \[page 481\]](#)
- [Xor \[page 482\]](#)
- [<> \[page 482\]](#)
- [= \[page 483\]](#)

- [< \[page 484\]](#)
- [<= \[page 485\]](#)
- [> \[page 486\]](#)
- [>= \[page 487\]](#)

19.7.4.1 And

OLAP Syntax.

And operator.

The And operator tests two expressions. It evaluates TRUE when both expressions are TRUE, and FALSE if either expression is FALSE.

Syntax

<Expression1> And <Expression2>

Parameters

<Expression1>

An expression that evaluates to a boolean.

<Expression2>

An expression that evaluates to a boolean.

Return value

Returns TRUE or FALSE.

Example

The And operator tests for when two conditions are TRUE. The following example tests that the Budget and Actual values are greater than 200. If both values are greater than 200, And evaluates TRUE and the Actual value is displayed; if either value is less than 200, the cell is set to a null value (a cell with no value or an error value is set to null).

Iif (({'Budget'}>200) AND ({'Actual'}>200), {'Actual'}, NULL)

See also [lif \[page 519\]](#).

19.7.4.2 Between

OLAP Syntax.

Tests that a number is between a given range.

The function returns TRUE if the value of **<NumericExpression1>** lies within the range specified by **<NumericExpression2>** and **<NumericExpression3>**. The function returns FALSE if the value lies outside of the range.

Syntax

```
Between(<NumericExpression1>, <NumericExpression2>, <NumericExpression3>)
```

Parameters

<NumericExpression1>	An expression that evaluates to a number. This is the number being tested with the range.
<NumericExpression2>	An expression that evaluates to a numeric value. This is the lower number in the range.
<NumericExpression3>	An expression that evaluates to a numeric value. This is the upper number in the range.

Return value

Returns TRUE or FALSE.

Example

The following example demonstrates the Between function. We are only interested in products with a **Budget** value between 100 and 1000. Between returns TRUE if the **Budget** value lies within this range. In the example the **Actual** value is returned if Between evaluates TRUE, otherwise a null value is returned:

```
Iif ( Between ( {'Budget'}, 100, 1000 ) , {'Actual'} , NULL )
```

A cell with no value or an error value is set to null.

See also [lif \[page 519\]](#).

19.7.4.3 Eqv

OLAP Syntax.

Equivalence operator.

The Equivalence operator tests two expressions. It evaluates **TRUE** when both expressions are the same, and **FALSE** otherwise.

Syntax

```
<Expression1> Eqv <Expression2>
```

Parameters

<Expression1>

An expression that evaluates to a boolean.

<Expression2>

An expression that evaluates to a boolean.

Return value

Returns **TRUE** or **FALSE**.

Example

The following example tests whether the **Budget** and **Actual** values are greater than 200. If both values are greater than or less than 200, **Eqv** will evaluate **TRUE** and the **Actual** value is displayed. If only one value is greater than 200, the cell is set to a null value (a cell with no value or an error value is set to null).

```
Iif ( ( {'Budget'}>200 ) Eqv ( 'Actual'>200), {'Actual'}, NULL )
```

See also [lif \[page 519\]](#).

19.7.4.4 Imp

OLAP Syntax.

Implication operator.

The Implication operator returns the value of **<Expression2>** unless both expressions are FALSE, in which case TRUE is returned.

Syntax

```
<Expression1> Imp <Expression2>
```

Parameters

<Expression1>	An expression that evaluates to a boolean.
<Expression2>	An expression that evaluates to a boolean.

Return value

The Implication operator truth table is as follows:

<Expression1>	<Expression2>	Return value
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	TRUE
FALSE	FALSE	TRUE

Example

The following example tests whether the Budget and Actual values are greater than 200. If Actual is greater than 200, a null value is returned (a cell with no value or an error value is set to null). If Actual and Budget are

less than or equal to 200, a null value is returned. If **Actual** is less than or equal to 200 but **Budget** is greater than 200, **Actual** is returned.

```
Iif ( ( {'Budget'}>200 ) Imp ( {'Actual'}>200), NULL, {'Actual'} )
```

See also [Iif \[page 519\]](#).

19.7.4.5 Is

OLAP Syntax.

Tests if two specified objects are the same.

Returns **TRUE** if the objects are the same, returns **FALSE** otherwise.

Syntax

```
<Object1> Is <Object2>
```

Parameters

<Object1>	Any level, tuple, or member.
<Object2>	Any level, tuple, or member.

Return value

Returns **TRUE** or **FALSE**.

Example

The following example returns **TRUE** if **Fruit** is the current member of the **Products** dimension:

```
{'Fruit'} Is CurrentMember({Products})
```

See also [CurrentMember \[page 492\]](#).

19.7.4.6 Is NULL

OLAP Syntax.

Tests if a specified level, tuple, or member is a null value.

Returns TRUE if the specified level, tuple, or member is a null value; FALSE otherwise.

A cell with no value or an error value is set to null.

Syntax

```
Object Is NULL
```

Parameters

<Object>

Any level, tuple, member, or set.

Return value

Returns TRUE or FALSE.

Example

The Is NULL function can be used to test for null values. The following example demonstrates this: 0 is returned if a cell in the member is a null value, otherwise the cell value is returned.

```
Iif ( {'Amount'} Is NULL, 0, {'Amount'} )
```

See also [lif \[page 519\]](#).

19.7.4.7 IsAncestor

OLAP Syntax.

Tests that a specified member is a parent of another specified member.

Returns TRUE if <Member1> is a parent of <Member2> , and FALSE otherwise.

Syntax

```
IsAncestor (<Member1>, <Member2>)
```

Parameters

<Member1>	Any member.
<Member2>	Any member.

Return value

Returns TRUE or FALSE.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

`IsAncestor` can be used to determine if a member is a parent of another member. This example tests whether `Fruit` is a parent of `Apples`:

```
IsAncestor ( {'Fruit'}, {'Apples'} )
```

The example returns TRUE as `Fruit` is a parent of `Apples`.

19.7.4.8 IsEmpty

OLAP Syntax.

Tests whether an expression evaluates to the empty cell value.

Returns TRUE if the expression evaluates to a empty cell value; FALSE otherwise.

Syntax

```
IsEmpty (<Expression>)
```

Parameters

<Expression>	The expression being tested.
---------------------------	------------------------------

Return value

Returns TRUE or FALSE.

Example

The following example tests whether the Budget cells are empty. Empty cells are set to 0. Cells with values retain their value.

```
Iif ( IsEmpty({'Budget'}),0,{'Budget'} )
```

See also [lif \[page 519\]](#).

19.7.4.9 IsGeneration

OLAP Syntax.

Tests whether a specified member is of a specified generation.

Returns TRUE if the member is in the specified generation. Returns FALSE if the member is not of the specified generation.

Syntax

```
IsGeneration (<Member>, <Generation>)
```

Parameters

<Member>	The member being tested.
-----------------------	--------------------------

<NumericExpression>

An expression that evaluates to a number specifying the generation.

Return value

Returns TRUE or FALSE.

Example

This example tests if the member `Fruit` is a member of the second generation:

```
IsGeneration ( {'Fruit'}, 2 )
```

Remarks

A specific member can belong to more than one generation; this is due to the way generations are indexed.

19.7.4.10 IsLeaf

OLAP Syntax.

Tests whether a specified member is a member without children (a leaf member).

Returns TRUE if the specified member is a leaf member; FALSE if the member is not a leaf member.

Syntax

```
IsLeaf ( <Member> )
```

Parameters

<Member>

The member you are testing.

Return value

Returns TRUE or FALSE.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

This example tests if the member `Fruit` is a leaf member:

```
IsLeaf ( {'Fruit'} )
```

`Fruit` has children so it is not a leaf member. The example returns FALSE.

19.7.4.11 IsSibling

OLAP Syntax.

Tests if a member is a sibling of another specified member.

Returns TRUE if the members are siblings; FALSE if the members are not siblings.

Syntax

```
IsSibling (<Member1>, <Member2>)
```

Parameters

<Member1>	Any member.
-----------	-------------

<Member2>	Any member.
-----------	-------------

Return value

Returns TRUE or FALSE.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

Apples and Oranges are siblings therefore the example returns `TRUE`.

```
IsSibling ( {'Apples'}, {'Oranges'} )
```

19.7.4.12 Not

OLAP Syntax.

Not operator.

Returns `FALSE` if the expression evaluates `TRUE`; `TRUE` if the expression evaluates `FALSE`.

Syntax

```
Not <Expression>
```

Parameters

<Expression>

An expression that evaluates to a boolean.

Return value

Returns `TRUE` or `FALSE`.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The Not operator can be used as follows to invert the output of an expression:

```
Not ( IsLeaf({'Fruit'}) )
```

The example returns TRUE. The function IsLeaf returns FALSE because Fruit is not a leaf member; the example returns TRUE. See [IsLeaf \[page 477\]](#) for more details.

19.7.4.13 Or

OLAP Syntax.

Or operator.

The Or operator takes two expressions that evaluate to a boolean. If either expression evaluates TRUE, the operator returns TRUE. If both expressions evaluate FALSE, the operator returns FALSE.

Syntax

```
<Expression> Or <Expression>
```

Parameters

<Expression>

An expression that evaluates to a boolean.

Return value

Returns TRUE or FALSE.

Example

The Or operator tests if either of two conditions is true. The following example tests whether the Budget and Actual values are greater than 200; if either value is greater than 200, Or will evaluate TRUE and the Actual value is displayed; if both values are less than or equal to 200, the cell is set to a null value (a cell with no value or an error value is set to null).

```
Iif ( ( {'Budget'}>200 ) Or ( {'Actual'}>200), {'Actual'}, NULL )
```

See also [Iif \[page 519\]](#).

19.7.4.14 Outside

OLAP Syntax.

Tests if a number lies outside a given range.

Returns TRUE if the value of **<NumericExpression1>** lies outside the range specified by **<NumericExpression2>** and **<NumericExpression3>** . The function returns FALSE if the value lies within the range.

Syntax

```
Outside(<NumericExpression1>,< NumericExpression2>,<NumericExpression3>)
```

Parameters

<NumericExpression1>	An expression that evaluates to a number. This is the number being tested.
<NumericExpression2>	An expression that evaluates to a numeric value. This is the lower number in the range.
<NumericExpression3>	An expression that evaluates to a numeric value. This is the upper number in the range.

Return value

Returns TRUE or FALSE.

Example

The following example demonstrates the Outside function. We want to discard outlying values in our data: Budget values under 10 or above 10000 are considered outliers. Outside returns TRUE if the Budget value is an outlier. In the example, a null value is returned if Outside evaluates TRUE; the Actual value is returned if Outside evaluates FALSE (a cell with no value or an error value is set to null).

```
Iif ( Outside( {'Budget'}, 10, 10000 ), NULL , {'Actual'} )
```

See also [Iif \[page 519\]](#).

19.7.4.15 Xor

OLAP Syntax.

Exclusive Or operator.

Xor takes two expressions that evaluate to boolean values. It evaluates **FALSE** if both expressions are **TRUE** or both expressions are **FALSE**. Xor evaluates **TRUE** only when one expression is **TRUE** and the other expression is **FALSE**.

Syntax

```
<Expression> Xor <Expression>
```

Parameters

<Expression>

An expression that evaluates to a boolean.

Return value

Returns **TRUE** or **FALSE**.

Example

The following example uses the Xor statement to test the values of two measures: **ValueA** and **ValueB**. One of the values should be greater than 1000 at any given time, however both values should not exceed 1000 at the same time. Xor returns **TRUE** if just one of the values exceeds 1000, but returns **FALSE** if both values fall above or below the limit:

```
({'ValueA'}>1000) Xor ({'ValueB'}>1000)
```

19.7.4.16 <>

OLAP Syntax.

Not Equal To operator.

The operator compares two values and returns TRUE if the values are not equal. The operator returns FALSE if the values are equal.

Syntax

```
<NumericExpression1> <> <NumericExpression2>
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget and Actual are not equal, and FALSE if Budget and Actual are equal:

```
{'Budget'} <> {'Actual'}
```

19.7.4.17 =

OLAP Syntax.

Equal To operator.

The operator compares two values and returns TRUE if the values are equal, FALSE if they are not equal.

Syntax

```
<NumericExpression1> = <NumericExpression2>
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget and Actual are equal; FALSE if Budget and Actual are not equal:

```
{'Budget'} = {'Actual'}
```

19.7.4.18 <

OLAP Syntax.

Less Than operator.

The operator compares two values and returns TRUE if **<NumericExpression1>** is less than **<NumericExpression2>** ; FALSE if **<NumericExpression1>** is greater than or equal to **<NumericExpression2>** .

Syntax

```
<NumericExpression1> < <NumericExpression2>
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget is less than Actual, and FALSE if Budget is greater than or equal to Actual.

```
{'Budget'} < {'Actual'}
```

19.7.4.19 <=

OLAP Syntax.

Less Than or Equal To operator.

The operator compares two values and returns TRUE if **<NumericExpression1>** is less than or equal to **<NumericExpression2>** ; FALSE if **<NumericExpression1>** is greater than **<NumericExpression2>** .

Syntax

```
<NumericExpression1> <= <NumericExpression2>
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
-----------------------------------	--

<NumericExpression2>

An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget is less than or equal to Actual, and FALSE if Budget is greater than Actual.

```
{'Budget'} <= {'Actual'}
```

19.7.4.20 >

OLAP Syntax.

Greater Than operator.

The operator compares two values and returns TRUE if **<NumericExpression1>** is greater than **<NumericExpression2>** ; FALSE if **<NumericExpression1>** is less than or equal to **<NumericExpression2>** .

Syntax

```
<NumericExpression1> > <NumericExpression2>
```

Parameters

<NumericExpression1>

An expression that evaluates to a numeric value.

<NumericExpression2>

An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget is greater than Actual, and FALSE if Budget is less than or equal to Actual.

```
{'Budget'} > {'Actual'}
```

19.7.4.21 >=

OLAP Syntax.

Greater Than or Equal To operator.

The operator compares two values and returns TRUE if **<NumericExpression1>** is greater than or equal to **<NumericExpression2>** ; FALSE if **<NumericExpression1>** is less than **<NumericExpression2>** .

Syntax

```
<NumericExpression1> >= <NumericExpression2>
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns TRUE or FALSE.

Example

The following expression evaluates TRUE if Budget is greater than or equal to Actual, and FALSE if Budget is less than Actual.

```
{'Budget'} >= {'Actual'}
```

19.7.5 Member functions

Click a link to view one of the topics in this section:

- [Ancestor - Distance \[page 488\]](#)
- [Ancestor - Level \[page 489\]](#)
- [ClosingPeriod \[page 490\]](#)
- [Cousin \[page 491\]](#)
- [CurrentMember \[page 492\]](#)
- [DefaultMember \[page 492\]](#)
- [FirstChild \[page 493\]](#)
- [FirstSibling \[page 494\]](#)
- [Item - member \[page 495\]](#)
- [Lag \[page 496\]](#)
- [LastChild \[page 497\]](#)
- [LastSibling \[page 498\]](#)
- [Lead \[page 498\]](#)
- [NextMember \[page 499\]](#)
- [OpeningPeriod \[page 500\]](#)
- [ParallelPeriod \[page 501\]](#)
- [Parent \[page 502\]](#)
- [PrevMember \[page 503\]](#)

19.7.5.1 Ancestor - Distance

OLAP Syntax.

Returns the parent of a member that is a specified number of levels away in the hierarchy.

Syntax

```
Ancestor(<Member>, <Distance>)
```

Parameters

<Member>	Any member.
<Distance>	Number of levels away in hierarchy.

Return value

Returns a member.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns the member `Fruit&Veg`; this is the member two hierarchical levels above `Apples`:

```
Ancestor ( {'Apples'}, 2)
```

19.7.5.2 Ancestor - Level

OLAP Syntax.

Returns the ancestor of a member at a specified hierarchical level.

Syntax

```
Ancestor(<Member>, <Level>)
```

Parameters

<Member>	Name of a member.
-----------------------	-------------------

<Level>	Level in the hierarchy.
---------	-------------------------

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns the member **Fruit&Veg**, which is the ancestor of **Apples** at **level-1**:

```
Ancestor ( {'Apples'}, {Products@level-1} )
```

19.7.5.3 ClosingPeriod

OLAP Syntax.

Returns the last sibling among the descendants of a member at a specified level. If a level is unspecified, the last sibling at the level below is returned.

Syntax

```
ClosingPeriod ([<Level>], <Member>)
```

Parameters

<Level>	Level in the hierarchy.
<Member>	Any member.

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns **Peaches**, which is the last sibling of the descendants of **Fruit&Veg** at level-3:

```
ClosingPeriod ( {Products@level-3}, {'Fruit&Veg'} )
```

19.7.5.4 Cousin

OLAP Syntax.

Returns the member with the same relative position under a member as a specified member.

The function uses **<Member1>** as a reference, and returns the descendant of **<Member2>** whose position in the level is relative to **<Member1>**.

Syntax

```
Cousin (<Member1>, <Member2>)
```

Parameters

<Member1>	The reference member.
<Member2>	The member whose descendant you require.

Return value

Returns a member.

Example

Cousin can be used to find members related by their levels in a hierarchy. The following example returns the member for May 2001:

```
Cousin ( {Time@'2002'@'May'}, {Time@'2001'} )
```

19.7.5.5 CurrentMember

OLAP Syntax.

Returns the current member of a dimension during an iteration.

Syntax

```
CurrentMember ( <Dimension> )
```

Parameters

<Dimension>

Any dimension.

Return value

Returns a member.

Example

The following example returns the current member of the **Product** dimension.

```
CurrentMember ( {Product} )
```

19.7.5.6 DefaultMember

OLAP Syntax.

Returns the default member of a dimension.

Syntax

```
DefaultMember(<Dimension>)
```

Parameters

<Dimension>	Any dimension.
--------------------------	----------------

Return value

Returns a member.

Example

The following example returns the default member of the **Product** dimension.

```
DefaultMember ( {Product} )
```

19.7.5.7 FirstChild

OLAP Syntax.

Returns the first child of a member.

Syntax

```
FirstChild(<Member>)
```

Parameters

<Member>	Any member.
-----------------------	-------------

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns **Apples**, which is the first child of the **Fruit** member.

```
FirstChild ({'Fruit'})
```

19.7.5.8 FirstSibling

OLAP Syntax.

Returns the first child of a member's parent.

Syntax

```
FirstSibling(<Member>)
```

Parameters

<Member>	Any member.
-----------------------	-------------

Return value

Returns a member.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `Apples`, the first sibling of the `Bananas` member.

```
FirstSibling ( {'Bananas'} )
```

19.7.5.9 Item - member

OLAP Syntax.

Returns a member from a tuple. The position of the member in the tuple is specified by a numeric value.

Syntax

```
Item(<Tuple>,<NumericExpression>)
```

Alternative syntax

```
<Tuple>(<NumericExpression>)
```

Parameters

<Tuple>	The tuple from which the members should be selected.
<NumericExpression>	The position of the member in the tuple.

Return value

Returns a member.

Remarks

The first member in a tuple has a position of 0. The second member has a position of 1, and so on.

Example

The following example returns the third member from the current tuple.

```
Item( Current(), 2 )
```

See also [Current \[page 610\]](#).

19.7.5.10 Lag

OLAP Syntax.

Returns the member a number of places before the specified member along a dimension. The number of members along is specified by a numeric value.

Syntax

```
Lag(<Member>, <NumericExpression>)
```

Parameters

<Member>	The reference member.
<NumericExpression>	The number of members before the specified member.

Return value

Returns a member.

Example

The following example returns the member **June**, which is three members before **September** in the **Monthly** dimension.

```
Lag ( {Monthly@'September'}, 3 )
```

19.7.5.11 LastChild

OLAP Syntax.

Returns the last child of a member.

Syntax

```
LastChild(<Member>)
```

Parameters

<Member>	Any member.
----------	-------------

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns Peaches, which is the last child of the `Fruit` member.

```
LastChild({'Fruit'})
```

19.7.5.12 LastSibling

OLAP Syntax.

Returns the last sibling of a member.

Syntax

```
LastSibling(<Member>)
```

Parameters

<Member>	Any member.
----------	-------------

Return value

Returns a member.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns Peaches, the last sibling of the `Bananas` member:

```
LastSibling( {'Bananas'} )
```

19.7.5.13 Lead

OLAP Syntax.

Returns a member further along the specified member's dimension. The returned member is a specified number of members away from the specified member.

Syntax

```
Lead(<Member>, <NumericExpression>)
```

Parameters

<Member>	The reference member.
<NumericExpression>	The number of members along from the specified member.

Return value

Returns a member.

Example

The following example returns the member **September**, which is three members along from **June** in the **Monthly** dimension.

```
Lead ( {Monthly@'June'}, 3 )
```

19.7.5.14 NextMember

OLAP Syntax.

Returns the next member in the level containing a specified member.

Syntax

```
NextMember(<Member>)
```

Parameters

<Member>	The reference member.
-----------------------	-----------------------

Return value

Returns a member.

Example

The following example returns the member **July**, which is the next member along from **June** at that level.

```
NextMember( {Monthly@'June'} )
```

19.7.5.15 OpeningPeriod

OLAP Syntax.

Returns the first sibling among the descendants of a member at a specified level. If the level is unspecified, the first sibling of the member, one level down, is returned.

Syntax

```
OpeningPeriod ([<Level>,<Member>)
```

Parameters

<Level>	Level in the hierarchy.
<Member>	Any member.

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns **Red**, which is the first sibling of the descendants of **Fruit** at level-4:

```
OpeningPeriod ( {Products@level-3}, {'Fruit'} )
```

19.7.5.16 ParallelPeriod

OLAP Syntax.

Returns a member from a prior period in the same relative position as a specified member. If a level is unspecified, the member is returned from the level above the specified member. If a level is specified, the member is returned from this level. The optional numeric value specifies the lag. See [Lag \[page 496\]](#) for more details on lagging.

See also

[Lag \[page 496\]](#)

Syntax

```
ParallelPeriod([<Level>],[<LagValue>,<Member>])
```

Parameters

<Level>	The level containing the required member.
<LagValue>	The lag value.

<Member>

The reference member.

Return value

Returns a member.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns `Seafood`, which is a member from `level-1`, the reference member is `Bananas`, with a lag of 2:

```
ParallelPeriod ( {Products@level-1}, 2, {'Bananas'} )
```

19.7.5.17 Parent

OLAP Syntax.

Returns the parent of a member.

Syntax

```
Parent(<Member>)
```

Parameters

<Member>

Any member.

Return value

Returns a member.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns the member **Fruit**, which is the parent of **Apples**:

```
Parent ( {'Apples'} )
```

19.7.5.18 PrevMember

OLAP Syntax.

Returns the previous member in the level that contains a specified member.

Syntax

```
PrevMember(<Member>)
```

Parameters

<Member>

The reference member.

Return value

Returns a member.

Example

The following example returns the member **June**, which is the previous member to **July**:

```
PrevMember ( {'July'} )
```

19.7.6 Numeric functions

Click a link to view one of the topics in this section:

- [Abs \[page 505\]](#)
- [Acos \[page 506\]](#)
- [Asin \[page 506\]](#)
- [Atn \[page 507\]](#)
- [Avg \[page 508\]](#)
- [CoalesceEmpty \[page 509\]](#)
- [Cos \[page 510\]](#)
- [Count \[page 511\]](#)
- [Count - Dimension \[page 512\]](#)
- [Count - Dimensions \[page 512\]](#)
- [Count - Level \[page 513\]](#)
- [Count - Set \[page 514\]](#)
- [Count - Tuple \[page 515\]](#)
- [DistinctCount \[page 516\]](#)
- [Exp \[page 517\]](#)
- [Fix \[page 517\]](#)
- [Iif \[page 519\]](#)
- [Int \[page 520\]](#)
- [Log \[page 521\]](#)
- [Max \[page 521\]](#)
- [Median \[page 522\]](#)
- [Min \[page 523\]](#)
- [MOD \[page 524\]](#)
- [Native - Numeric \[page 525\]](#)
- [Ordinal \[page 526\]](#)
- [Rank \[page 526\]](#)
- [Remainder \[page 527\]](#)
- [RollupChildren \[page 528\]](#)
- [Round \[page 529\]](#)
- [Sgn \[page 531\]](#)
- [Sin \[page 532\]](#)
- [Sqr \[page 533\]](#)
- [Sum \[page 534\]](#)
- [Tan \[page 535\]](#)
- [Value \[page 536\]](#)

19.7.6.1 Abs

OLAP Syntax.

Returns the absolute value of a number.

Syntax

```
Abs (<NumericExpression>)
```

Parameters

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Examples

The following example returns the absolute value of `Fruit`:

```
Abs ( { 'Fruit' } )
```

The following examples demonstrate use of `Abs`:

```
Abs(1.50)
```

Returns 1.50.

```
Abs( -1.50)
```

Returns 1.50.

```
Abs(10 - 7)
```

Returns 3.

```
Abs(7 - 10)
```

Returns 3.

19.7.6.2 Acos

OLAP Syntax.

Returns the inverse cosine in radians.

Syntax

```
Acos ( <NumericExpression> )
```

Parameters

<NumericExpression>

A numeric value between 1 and -1.

Return value

Returns a number in radians.

Example

The following example returns the inverse cosine of the current member of the **Products** dimension:

```
Acos ( CurrentMember({Products}) )
```

See also [CurrentMember \[page 492\]](#).

19.7.6.3 Asin

OLAP Syntax.

Returns the inverse sine in radians.

Syntax

```
Asin ( <NumericExpression> )
```

Parameters

<NumericExpression>

A numeric value between 1 and -1.

Return value

Returns a number in radians.

Example

The following example returns the inverse sine of the current member of the **Products** dimension:

```
Asin ( CurrentMember({Products}) )
```

See also [CurrentMember \[page 492\]](#).

19.7.6.4 Atn

OLAP Syntax.

Returns the inverse tangent in radians.

Syntax

```
Atn ( <NumericExpression > )
```

Parameters

<NumericExpression>	A numeric value between 1 and -1.
----------------------------------	-----------------------------------

Return value

The function returns the inverse tangent as a number in radians.

Example

The following example returns the inverse tangent of the current member:

```
Atn ( CurrentMember({Products}) )
```

See also [CurrentMember \[page 492\]](#).

19.7.6.5 Avg

OLAP Syntax.

Returns the average value of a numeric expression, evaluated over a set.

Syntax

```
Avg(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which the expression should be evaluated.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the average value of `Profit after Tax`, for all members of the `Product` dimension:

```
Avg ( AllMembers({Product}) , {'Profit after Tax'} )
```

See also [AllMembers - Dimension](#) [page 552].

19.7.6.6 CoalesceEmpty

OLAP Syntax.

Returns the first non-empty value from the list of numeric expressions.

Syntax

```
CoalesceEmpty (<NumericExpression> [, <NumericExpression>]...)
```

Parameters

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

This example returns `Projected Sales` if `Sales` is empty, otherwise it returns the value of `Sales`.

```
CoalesceEmpty({'Sales'}, {'Projected Sales'})
```

19.7.6.7 Cos

OLAP Syntax.

Returns the cosine of a number given in radians.

Syntax

```
Cos(<NumericExpression>)
```

Parameters

<NumericExpression>

A numeric value in radians.

Return value

Returns a numeric value.

Example

The following example returns the cosine of the values in `Angle`.

```
Cos ( {Geometry@'Angle'} )
```

The following examples demonstrate the use of `Cos`:

```
Cos (1)
```

Returns 0.5403 (rounded to 4 decimals). This is the cosine of 1 radian.

```
Cos (60 * crPi / 180)
```

Returns 0.5. This is the cosine of 60 degrees. Before taking the cosine, the angle is converted to radians by multiplying by $\pi / 180$

19.7.6.8 Count

OLAP Syntax.

Returns the number of tuples in a set. Empty cells are excluded in the count if the **<EXCLUDEEMPTY>** flag is used.

Syntax

```
Count ( <Set> [ , <EXCLUDEEMPTY> | <INCLUDEEMPTY> ] )
```

Parameters

<Set>	Any set.
<EXCLUDEEMPTY>	A flag specifying that empty cells should not be included.
<INCLUDEEMPTY>	A flag specifying that empty cells should be included. This is the default.

Return value

Returns a numeric value.

Example

The following example returns the number of tuples in the set specified by the AllMembers function. Empty cells are excluded:

```
Count ( AllMembers({Products}) , EXCLUDEEMPTY )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.6.9 Count - Dimension

OLAP Syntax.

Returns the number of levels in a dimension.

Syntax

```
Count(<Dimension>)
```

Parameters

<Dimension>

The dimension name.

Return value

Returns a numeric value.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns 5, the number of levels in the dimension **Products**.

```
Count ( {Products} )
```

19.7.6.10 Count - Dimensions

OLAP Syntax.

Returns the number of dimensions in the current cube.

Syntax

```
Count(Dimensions)
```

Parameters

<Dimensions>

<Required flag.>

Return value

Returns a numeric value.

Example

The following example returns the number of dimensions in the current cube:

```
Count(Dimensions)
```

19.7.6.11 Count - Level

OLAP Syntax.

Returns the number of members in a specified level.

Syntax

```
Count(<Level>)
```

Parameters

<Level>	Any level.
----------------------	------------

Return value

Returns a numeric value.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns 6, which is the number of members in the **Products** dimension at **Level-1**.

```
Count ( {Products@level-1} )
```

19.7.6.12 Count - Set

OLAP Syntax.

Returns the number of cells in a specified set. Empty cells are included in the count.

Syntax

```
Count(<Set>)
```

Parameters

<Set>	Any set.
--------------------	----------

Return value

Returns a numeric value.

Example

The following example returns the number of cells in the set specified by the function AllMembers:

```
Count ( AllMembers({Products}) )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.6.13 Count - Tuple

OLAP Syntax.

Returns the number of dimensions in a tuple.

Syntax

```
Count(<Tuple>)
```

Parameters

<Tuple>

Any tuple.

Return value

Returns a numeric value.

Example

The following example returns the number of dimensions in the current tuple:

```
Count ( Current() )
```

See also [Current \[page 610\]](#).

19.7.6.14 DistinctCount

OLAP Syntax.

Returns the number of distinct tuples in a set. Empty cells are not included in the count.

Syntax

```
DistinctCount(<Set>)
```

Parameters

<Set>

Any set.

Return value

Returns a numeric value.

Example

The following example returns 3, the number of distinct tuples in the resulting set:

```
DistinctCount( Set({'January':{'March'}, {'February'}}) * Set({'Sales'}) )
```

See also [Set \[page 595\]](#).

19.7.6.15 Exp

OLAP Syntax.

Returns the exponential of a number.

Syntax

```
Exp(<NumericExpression>)
```

Parameters

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Examples

The following example returns the exponential of the cell values in **Quantity**:

```
Exp ( { 'Quantity' } )
```

The following example demonstrates use of Exp:

```
Exp ( 1.5 )
```

Returns the exponentiation of 1.5, which is approximately 4.48169.

19.7.6.16 Fix

OLAP Syntax.

Truncates a number to a specified number of decimal places. If the number of decimal places is not specified, zero decimal places is assumed.

Syntax

```
Fix(<NumericExpression>[, <Decimals>])
```

Parameters

<NumericExpression>	The numeric value to be truncated.
<Decimals>	The number of decimal places.

Return value

Returns a numeric value.

Example

The following example returns cell values in the measure Actual. The values are truncated to 3 decimal places:

```
Fix( {Measures@'Actual'}, 3 )
```

The following examples demonstrate use of Fix:

```
Fix (123.678)
```

Returns 123.

```
Fix (-123.678)
```

Returns -123.

```
Fix (123.678,1)
```

Returns 123.6.

```
Fix (123.678,2)
```

Returns 123.67.

19.7.6.17 Iif

OLAP Syntax.

Returns one of two numerical values determined by a logical test.

Iif returns the number specified by **<NumericExpression1>** if **<LogicalExpression>** evaluates TRUE. The function returns the number specified by **<NumericExpression2>** if **<LogicalExpression>** evaluates FALSE.

Syntax

```
Iif ( <LogicalExpression>, <NumericExpression1>, <NumericExpression2> )
```

Parameters

<LogicalExpression>	The logical expression you are testing.
<NumericExpression1>	The value to be returned when <LogicalExpression> evaluates TRUE.
<NumericExpression2>	The value to be returned when <LogicalExpression> evaluates FALSE.

Return value

Returns a numeric value.

Example

The following example returns the value of Actual if Budget is greater than 1000, otherwise a null value is returned:

```
Iif ( ({'Budget'}>1000 ) , {'Actual'}, NULL )
```

A cell with no value or an error value is set to null.

19.7.6.18 Int

OLAP Syntax.

Returns the integer value of a number.

Syntax

```
Int(<NumericExpression>)
```

Parameters

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the integer value of the Actual member:

```
Int( {'Actual'} )
```

The following examples demonstrate use of Int:

```
Int (123.678)
```

Returns 123.

```
Int (-123.678)
```

Returns -124.

```
Int (-123.1)
```

Returns -124.

19.7.6.19 Log

OLAP Syntax.

Returns the natural logarithm of a specified number.

Syntax

```
Log(<NumericExpression>)
```

Parameters

<NumericExpression>

A positive numeric value.

Return value

Returns a numeric value.

Example

The following example returns the natural logarithm of the Actual member:

```
Log( {'Actual'} )
```

The following example returns 0.4055 (rounded to 4 decimals):

```
Log (1.5)
```

19.7.6.20 Max

OLAP Syntax.

Returns the maximum value of a numeric expression evaluated over a set. If <NumericExpression> is not specified, the values for the current slice are used.

Syntax

```
Max(<Set>[, <NumericExpression>])
```

Parameters

<Set>	Any set.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the maximum value of **Profit after Tax** from the set specified by **AllMembers**:

```
Max ( AllMembers({Products}), {'Profit after Tax'} )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.6.21 Median

OLAP Syntax.

Returns the median value of a numeric expression evaluated over a set. If **<NumericExpression>** is not specified, the values for the current slice are used.

Syntax

```
Median(<Set>[, <NumericExpression>])
```


Parameters

<Set>	Any set.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the median value of Profit after Tax from the set specified by AllMembers:

```
Median( AllMembers({Products}) , {'Profit after Tax'})
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.6.22 Min

OLAP Syntax.

Returns the minimum value of a numeric expression evaluated over a set. If **<NumericExpression>** is not specified, the values for the current slice are used.

Syntax

```
Min(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which over which the minimum should be evaluated.
--------------------	--

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the minimum value of Profit after Tax from the set specified by AllMembers:

```
Min ( AllMembers({Products}) , {'Profit after Tax'})
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.6.23 MOD

OLAP Syntax.

Returns the modulo of two numbers.

Syntax

```
<NumericExpression1> MOD <NumericExpression2>
```

Parameters

<NumericExpression1>

An expression that evaluates to a numeric value.

<NumericExpression2>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

This example returns the value of Actual modulo Budget:

```
{Measure@'Actual'} MOD {Measure@'Budget'}
```

19.7.6.24 Native - Numeric

OLAP Syntax.

Evaluates a native expression.

Syntax

```
Native <StringExpression>
```

Parameters

<StringExpression>

A string specifying a member.

Return value

Returns the evaluated expression.

Example

This function example uses the sample dimension Products. See [Sample Dimension \[page 614\]](#) for more details.

The following expression returns the value of Fruit.

```
Native ({Products@'All Products'@'Fruit&Veg'@'Fruit'})
```

19.7.6.25 Ordinal

OLAP Syntax.

Returns the ordinal value associated with a level. The ordinal value of the first level is 0. The value of the second level is 1, and so on.

Syntax

```
Ordinal(<Level>)
```

Parameters

<Level>

Any level.

Return value

Returns a numeric value.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns 3, which is the ordinal value for **level-3** in the **Products** dimension:

```
Ordinal ( {Products@level-3} )
```

19.7.6.26 Rank

OLAP Syntax.

Returns the position of a tuple in a set.

The function returns the rank of the tuple in the set, based on its value of <NumericExpression> . If <NumericExpression> is not specified, the tuple's position in the set is returned.

Syntax

```
Rank(<Tuple>, <Set>[, <NumericExpression>])
```

Parameters

<Tuple>	Any tuple.
<Set>	Any set.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the rank of the current member of products in the specified set, and ranked according to Profit after Tax.

```
Rank (CurrentMember({Products}), Members({Products}), {'Profit after Tax'})
```

See also [CurrentMember \[page 492\]](#) and [Members - Dimension \[page 586\]](#).

19.7.6.27 Remainder

OLAP Syntax.

Returns the remainder after the numerator has been divided by the denominator.

Syntax

```
Remainder(<NumericExpression1>, <NumericExpression2>)]
```

Parameters

<code><NumericExpression1></code>	The numerator.
<code><NumericExpression2></code>	The denominator.

Return value

Returns a numeric value.

Example

The following example returns the remainder of **Actual** divided by **Budget**:

```
Remainder( {'Actual'}, {'Budget'} )
```

The following examples demonstrate use of Remainder:

```
Remainder(12,5)
```

Returns 2.

```
Remainder(16,5)
```

Returns 1.

19.7.6.28 RollupChildren

OLAP Syntax.

Returns a rollup of the values of a member's children.

Syntax

```
RollupChildren(<Member>,<Operator>)]
```

Parameters

<Member>	Any member.
<Operator>	Unary operator.
The following unary operators are valid:	
<+>	Returns total + current child.
<->	Returns total - current child.
<*>	Returns total * current child.
</>	Returns total / current child.
<%>	Returns total as a percentage of the current child.
<~>	The value of the current child is not included.

Return value

Returns a numeric value.

Example

The following example returns the rollup of the children of **Fruit**:

```
RollupChildren ( {'Fruit'}, + )
```

19.7.6.29 Round

OLAP Syntax.

Rounds a number to the specified number of decimal places. If the number of decimal places is not specified, the value is rounded to the nearest whole number.

Syntax

```
Round(<NumericExpression1>[, <NumericExpression2>])
```

Parameters

<NumericExpression1>	The number to be rounded.
<NumericExpression2>	The number of decimal places.

Return value

Returns a numeric value.

Example

The following example rounds the value of Apples to 2 decimal places:

```
Round ( { 'Apples' } , 2)
```

The following examples demonstrate use of Round:

```
Round(1.23456)
```

Returns 1.

```
Round(1.499)
```

Returns 1.

```
Round(1.5000)
```

Returns 2.

```
Round(2345.23456, 4)
```

Returns 2345.2346.

```
Round(2345.23456, 3)
```

Returns 2345.235.

```
Round(2345.23456, 2)
```


Returns 2345.23.

```
Round(2345.23456, 0)
```

Returns 2345.

```
Round(2345.23456, -1)
```

Returns 2350.

```
Round(2345.23456, -2)
```

Returns 2300.

```
Round(2345.23456, -3)
```

Returns 2000.

```
Round(1.234499, 3)
```

Returns 1.234.

```
Round(1.234500, 3)
```

Returns 1.235.

19.7.6.30 Sgn

OLAP Syntax.

Returns the sign of a specified number.

Returns 1 if the number is greater than 0; 0 if the number is 0; and -1 if the number is less than 0.

Syntax

```
Sgn(<NumericExpression>)
```

Parameters

<NumericExpression>

An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Examples

The following example returns the sign of **Actual**:

```
Sgn( {'Actual'} )
```

The following examples demonstrate use of Sgn:

```
Sgn ( -10)
```

Returns -1.

```
Sgn (0)
```

Returns 0.

```
Sgn (10)
```

Returns 1.

19.7.6.31 Sin

OLAP Syntax.

Returns the sine of an angle given in radians.

Syntax

```
Sin(<NumericExpression>)
```

Parameters

<NumericExpression>

A numeric value in radians.

Return value

Returns a numeric value.

Examples

The following example returns the sine of `Angle`:

```
Sin( {Geometry@'Angle'} )
```

The following examples demonstrate use of `Sin`:

```
Sin (1)
```

Returns 0.8415 (rounded to 4 decimals). This is the sine of 1 radian.

```
Sin (30 * crPi / 180)
```

Returns 0.5. This is the sine of 30 degrees. Before taking the sine, the angle is converted to radians by multiplying by `crPi / 180`.

19.7.6.32 Sqr

OLAP Syntax.

Returns the square root of a specified value.

Syntax

```
Sqr(<NumericExpression>)
```

Parameters

<NumericExpression>

A positive numerical value.

Return value

Returns a numeric value.

Example

The following example returns the square root of Area:

```
Sqr ( {Measures@'Area'} )
```

19.7.6.33 Sum

OLAP Syntax.

Returns the sum of a numerical expression evaluated over a set.

Syntax

```
Sum( <Set>[, <NumericalExpression> ] )
```

Parameters

<Set>	The set over which the operation is performed.
<NumericalExpression>	The values to be summed.

Return value

Returns a numeric value.

Example

The following example returns the sum of Sales for a set containing the members of Products:

```
Sum( Members({Products}), {profit_loss@'Sales'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.6.34 Tan

OLAP Syntax.

Returns the tangent of an angle given in radians.

Syntax

```
Tan( <NumericExpression> )
```

Parameters

<NumericExpression>

A numeric value in radians.

Return value

Returns a numeric value.

Example

The following example returns the tangent of Angle:

```
Tan( {Geometry@'Angle'} )
```

The following examples demonstrate use of Tan:

```
Tan (1)
```

Returns 1.5574 (rounded to 4 decimals). This is the tangent of 1 radian.

```
Tan (45 * crPi / 180)
```

Returns 1. This is the tangent of 45 degrees. Before taking the tangent, the angle is converted to radians by multiplying by `crPi / 180`.

19.7.6.35 Value

OLAP Syntax.

Returns the value of a member.

Syntax

```
Value(<Member>)
```

Parameters

<Member>

Any member.

Return value

Returns a numeric value.

Example

The following example returns the value of `Sales`:

```
Value( {profit_loss@'Sales'} )
```

19.7.7 Statistical functions

Click a link to view one of the topics in this section:

- [Correlation \[page 537\]](#)
- [Covariance \[page 538\]](#)
- [CovarianceN \[page 539\]](#)
- [LinRegIntercept \[page 540\]](#)
- [LinRegPoint \[page 541\]](#)
- [LinRegR2 \[page 542\]](#)
- [LinRegSlope \[page 543\]](#)
- [LinRegVariance \[page 544\]](#)
- [Stddev \[page 545\]](#)
- [StddevP \[page 546\]](#)
- [Stdev \[page 546\]](#)
- [StdevP \[page 547\]](#)
- [Var \[page 548\]](#)
- [Variance \[page 548\]](#)
- [VarianceP \[page 549\]](#)
- [VarP \[page 549\]](#)

19.7.7.1 Correlation

OLAP Syntax.

Returns the correlation of two series evaluated over a set.

Syntax

```
Correlation(<Set>, <NumericExpression> [, <NumericExpression>])
```

Parameters

<Set>	The set over which the expression should be evaluated.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the correlation of Sales and Profit for a set containing all members of the Products dimension.

```
Correlation ( Members({Products}), {'Sales'}, {'profit'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.2 Covariance

OLAP Syntax.

Returns the biased covariance (covariance normalized to n) of two series evaluated over a set.

Syntax

```
Covariance(<Set>, <NumericExpression1>[, <NumericExpression2>])
```

Parameters

<Set>	The set over which the covariance should be calculated.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the biased covariance of **Sales** and **Profit** for a set containing the members of the **Products** dimension.

```
Covariance ( Members({Products}), {'Sales'}, {'profit'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.3 CovarianceN

OLAP Syntax.

Returns the unbiased covariance (covariance normalized to $n-1$) of two series evaluated over a set.

Syntax

```
CovarianceN(<Set>, <NumericExpression1 >[, <NumericExpression2>])
```

Parameters

<Set>	The set for which the covariance should be calculated.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the unbiased covariance of **Sales** and **Profit** for a set containing all members of the **Products** dimension.

```
CovarianceN ( Members({Products}), {'Sales'}, {'Profit'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.4 LinRegIntercept

OLAP Syntax.

Calculates the linear regression of a set and returns the value of the y-axis intercept for the regression line.

The function uses the least squares method to calculate an equation of best fit for the specified data. The **<Set>** parameter specifies which members are used for the analysis. **<NumericExpression1>** specifies the x-axis data points. **<NumericExpression2>** specifies the y-axis data points. The least squares method fits a line to the equation:

$y = \text{<a> } x + \text{}$

where **<a>** is the gradient or slope, and **** is the y-axis intercept. The function returns the value of **** .

If **<NumericExpression2>** is not specified, the members of the specified set are used as values for the x-axis.

Syntax

```
LinRegIntercept(<Set>,<NumericExpression1>[,<NumericExpression2>])
```

Parameters

<Set>	The set over which to calculate the linear regression.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the y-axis intercept for the plot of Sales against Profit, for the members of the Product dimension.

```
LinRegIntercept( Members({Product}),'Sales',{'Profit'})
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.5 LinRegPoint

OLAP Syntax.

Calculates the linear regression of a set and returns a point on the regression line specified by its x-coordinate.

The function uses the least squares method to calculate an equation of best fit for the specified data. The **<Set>** parameter specifies which members are used for the analysis. **<NumericExpression2>** specifies the x-axis data points. **<NumericExpression3>** specifies the y-axis data points. The least squares method fits a line to the equation:

$$y = \text{<a>} x + \text{}$$

where **<a>** is the gradient or slope, and **** is the y-axis intercept. The function uses the calculated values of **<a>** and **** to find y when x is the value specified by **<NumericExpression1>**.

If **<NumericExpression3>** is not specified, the members of the specified set are used as values for the x-axis.

Syntax

```
LinRegPoint(<NumericExpression1>,<Set>,<NumericExpression2>[,<NumericExpression3>])
```

Parameters

<NumericExpression1>	An expression that evaluates to a numeric value.
<Set>	The set over which to calculate the linear regression.
<NumericExpression2>	An expression that evaluates to a numeric value.
<NumericExpression3>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the y-coordinate corresponding to the x-coordinate specified by **Projected** for the plot of **Sales** against **Profit**, for the members of the **Products** dimension.

```
LinRegPoint( {profit_loss@'Projected'} Members({Products} ),  
{profit_loss@'Sales'}, {profit_loss@'Profit'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.6 LinRegR2

OLAP Syntax.

Calculates the linear regression of a set and returns R^2 , the coefficient of determination for the line.

The function uses the least squares method to calculate an equation of best fit for the specified data. The **<Set>** parameter specifies which members are used for the analysis. **<NumericExpression1>** specifies the x-axis data points. **<NumericExpression2>** specifies the y-axis data points. The least squares method fits a line to the equation:

$$y = \text{<a> } x + \text{}$$

where **<a>** is the gradient or slope, and **** is the y-axis intercept. The function returns the coefficient of determination, R^2 , a measure of how well the regression line fits the data points.

If **<NumericExpression2>** is not specified, the members of the specified set are used as values for the x-axis.

Syntax

```
LinRegR2( <Set>, <NumericExpression1>[, <NumericExpression2>] )
```

Parameters

<Set>

The set over which to calculate the linear regression.

<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns R^2 for the plot of Sales against Profit, for the members of the Products dimension.

```
LinRegR2( Members({Products}), {'Sales'}, {'Profit'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.7 LinRegSlope

OLAP Syntax.

Calculates the linear regression of a set and returns the gradient of the regression line.

The function uses the least squares method to calculate an equation of best fit for the specified data. The **<Set>** parameter specifies which members are used for the analysis. **<NumericExpression1>** specifies the x-axis data points. **<NumericExpression2>** specifies the y-axis data points. The least squares method fits a line to the equation:

$$y = \text{<a> } x + \text{}$$

where **<a>** is the gradient or slope, and **** is the y-axis intercept. The function returns the value of **<a>**.

If **<NumericExpression2>** is not specified, the members of the specified set are used as values for the x-axis.

Syntax

```
LinRegSlope(<Set>, <NumericExpression1>[, <NumericExpression2>])
```

Parameters

<Set>	The set over which to calculate the linear regression.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the gradient for the plot of Sales against Profit, for the members of the Products dimension.

```
LinRegSlope( Members({Products}),'Sales','Profit' )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.8 LinRegVariance

OLAP Syntax.

Calculates the linear regression of a set and returns the variance in the regression line fit.

The function uses the least squares method to calculate an equation of best fit for the specified data. The **<Set>** parameter specifies which members are used for the analysis. **<NumericExpression1>** specifies the x-axis data points. **<NumericExpression2>** specifies the y-axis data points. The least squares method fits a line to the equation:

$$y = \text{<a> } x + \text{}$$

where **<a>** is the gradient or slope, and **** is the y-axis intercept. The function returns the value of the variance of the fit.

If **<NumericExpression2>** is not specified, the members of the specified set are used as values for the x-axis.

Syntax

```
LinRegVariance(<Set>, <NumericExpression1>[, <NumericExpression2>])
```

Parameters

<Set>	The set over which to calculate the linear regression.
<NumericExpression1>	An expression that evaluates to a numeric value.
<NumericExpression2>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the variance for the plot of Sales against Profit, for the members of the Products dimension.

```
LinRegVariance(Members({Products}), {'Sales'}, {'Profit'})
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.9 Stddev

OLAP Syntax.

An alias for [Stdev \[page 546\]](#).

See also

[Stdev \[page 546\]](#)

19.7.7.10 StddevP

OLAP Syntax.

An alias for [StdevP \[page 547\]](#).

See also

[StdevP \[page 547\]](#)

19.7.7.11 Stdev

OLAP Syntax.

Returns the unbiased standard deviation of a numeric expression evaluated over a set.

Syntax

```
Stdev(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which the standard deviation should be evaluated.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the unbiased standard deviation of the value of Sales for the members of the Products dimension.

```
Stdev( Members({Products}), {profit_loss@'Sales'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.12 StdevP

OLAP Syntax.

Returns the biased standard deviation of a numeric expression evaluated over a set.

Syntax

```
StdevP(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which the standard deviation should be evaluated.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the biased standard deviation of the value of Sales for the members of the Products dimension.

```
StdevP( Members({Products}), {profit_loss@'Sales'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.13 Var

OLAP Syntax.

Returns the unbiased variance (variance normalized to $n-1$) of a numeric expression evaluated over a set.

Syntax

```
Var(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which the variance is evaluated.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the unbiased variance of the value of Sales for the members of the Products dimension.

```
Var( Members({Products}), {profit_loss@'Sales'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.7.14 Variance

OLAP Syntax.

An alias for [Var \[page 548\]](#).

See also

[Var \[page 548\]](#)

19.7.7.15 VarianceP

OLAP Syntax.

An alias for [VarP \[page 549\]](#).

See also

[VarP \[page 549\]](#)

19.7.7.16 VarP

OLAP Syntax.

Returns the biased variance (variance normalized to n) of a numeric expression evaluated over a set.

Syntax

```
VarP(<Set>[, <NumericExpression>])
```

Parameters

<Set>	The set over which the variance is evaluated.
<NumericExpression>	An expression that evaluates to a numeric value.

Return value

Returns a numeric value.

Example

The following example returns the biased variance of the value of `Sales` for the members of the `Products` dimension.

```
VarP( Members({Products}), {profit_loss@'Sales'} )
```

See also [Members - Dimension \[page 586\]](#).

19.7.8 Set functions

Click a link to view one of the topics in this section:

- [AddCalculatedMembers \[page 551\]](#)
- [AllMembers - Dimension \[page 552\]](#)
- [AllMembers - Level \[page 553\]](#)
- [Ancestors - Distance \[page 554\]](#)
- [Ancestors - Level \[page 555\]](#)
- [Ascendants \[page 556\]](#)
- [Axis \[page 557\]](#)
- [BottomCount \[page 558\]](#)
- [BottomPercent \[page 559\]](#)
- [BottomSum \[page 560\]](#)
- [Children \[page 561\]](#)
- [Crossjoin \[page 562\]](#)
- [CurrentSet \[page 563\]](#)
- [Descendants - Distance \[page 564\]](#)
- [Descendants - Level \[page 565\]](#)
- [Distinct \[page 567\]](#)
- [DrilldownLevel - Index \[page 568\]](#)
- [DrilldownLevel - Level \[page 569\]](#)
- [DrilldownLevelBottom \[page 570\]](#)
- [DrilldownLevelTop \[page 571\]](#)
- [DrilldownMember \[page 572\]](#)
- [DrilldownMemberBottom \[page 573\]](#)
- [DrilldownMemberTop \[page 574\]](#)
- [DrillupLevel \[page 576\]](#)

- [DrillupMember \[page 577\]](#)
- [Except \[page 578\]](#)
- [Extract \[page 578\]](#)
- [Filter \[page 580\]](#)
- [Generate \[page 580\]](#)
- [GenerateSet \[page 581\]](#)
- [Head \[page 583\]](#)
- [Hierarchize \[page 584\]](#)
- [Intersect \[page 585\]](#)
- [LastPeriods \[page 585\]](#)
- [Members - Dimension \[page 586\]](#)
- [Members - Level \[page 587\]](#)
- [MTD \[page 588\]](#)
- [NameToSet \[page 589\]](#)
- [NonEmptyCrossjoin \[page 590\]](#)
- [Order \[page 591\]](#)
- [PeriodsToDate \[page 592\]](#)
- [QTD \[page 594\]](#)
- [Set \[page 595\]](#)
- [Siblings \[page 596\]](#)
- [StripCalculatedMembers \[page 596\]](#)
- [Subset \[page 597\]](#)
- [Tail \[page 598\]](#)
- [ToggleDrillState \[page 599\]](#)
- [TopCount \[page 600\]](#)
- [TopPercent \[page 601\]](#)
- [TopSum \[page 602\]](#)
- [Union \[page 603\]](#)
- [WTD \[page 604\]](#)
- [YTD \[page 605\]](#)

19.7.8.1 AddCalculatedMembers

OLAP Syntax.

Adds calculated members to a set.

The set specified by **<Set>** is returned, now containing any calculated members that are siblings to the members in the set. The **<ALL>** flag specifies that the current calculated member being evaluated are also added.

Syntax

```
AddCalculatedMembers(<Set>[, <ALL>])
```

Parameters

<Set>	The set to which you want to add calculated members.
<ALL>	Includes the member being calculated in the returned set.

Return value

Returns a set.

Example

The following example takes all the members that are currently on axis 1, and adds all the calculated members that have a sibling on that axis.

```
AddCalculatedMembers ( Axis (1, ALL) )
```

19.7.8.2 AllMembers - Dimension

OLAP Syntax.

Returns a set containing all members of the specified dimension. Calculated values are included. The optional <ALL> flag specifies that the current calculated member being evaluated are also added.

Syntax

```
AllMembers(<Dimension>[, <ALL>])
```

Parameters

<Dimension>	Any dimension.
<ALL>	Includes the member being calculated in the returned set.

Return value

Returns a set.

Example

The following example returns a set containing all the members in the **Products** dimension:

```
AllMembers ( {Products})
```

19.7.8.3 AllMembers - Level

OLAP Syntax.

Returns a set containing all members at the specified level in the hierarchy. Calculated values are included. The optional **<ALL>** flag specifies that the current calculated member being evaluated should also be added.

Syntax

```
AllMembers(<Level>[, <ALL>])
```

Parameters

<Level>	Any level.
<ALL>	Includes the member being calculated in the returned set.

Return value

Returns a set.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing all members at `level-1` in the `Products` dimension:

```
AllMembers ( {Products@level-1} )
```

The returned set would include the members: `Meat`, `Seafood`, `Fruit&Veg`, `Snacks`, `Drinks`, and `Canned`.

19.7.8.4 Ancestors - Distance

OLAP Syntax.

Returns a set containing all the parents of a member at a specified number of levels up the hierarchy.

Syntax

```
Ancestors(<Member>, <Distance>)
```

Parameters

<Member>	Any member.
<Distance>	Number of levels away in hierarchy.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing members two hierarchical levels above the member **Apples**:

```
Ancestors ( {'Apples'}, 2)
```

The returned set would include the members: **Meat**, **Seafood**, **Fruit&Veg**, **Snacks**, **Drinks**, and **Canned**.

19.7.8.5 Ancestors - Level

OLAP Syntax.

Returns a set containing all the parents of a member at a specified level in the hierarchy.

Syntax

```
Ancestors(<Member>, <Level>)
```

Parameters

<Member>	Any member.
<Level>	The level at which the required parents can be found.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing the parents of **Apples** at **Level-1**:

```
Ancestors ( { 'Apples' }, {Products@level-1} )
```

The returned set would contain the member **Fruit&Veg**.

19.7.8.6 Ascendants

OLAP Syntax.

Returns a set containing the ascendants of a specified member.

Syntax

```
Ascendants(<Member>)
```

Parameters

<Member>

Any member.

Return value

Returns a set.

Remarks

The function returns a set containing the parents of the member specified by Member. Unlike Ancestor, which returns members from a specific level, Ascendants returns the parents of Member from each ascendant level in the hierarchy; Member itself is also returned.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following code returns a set containing the parents of `Fruit`, from each ascendant level in the hierarchy:

```
Ascendants( {'Fruit'} )
```

The returned set contains the members: `All Products`, `Fruit&Veg`, and `Fruit`.

19.7.8.7 Axis

OLAP Syntax.

Returns a set associated with an axis, optionally including calculated members.

Syntax

```
Axis(<AxisNumber>[, <ALL>])
```

Parameters

<AxisNumber>	Numeric value specifying a set as defined in the axis.
<ALL>	Include calculated members.

Return value

Returns a set.

Remarks

An <AxisNumber> of 0 refers to the members found on the columns axis; 1 refers to the rows axis members, and 2 refers to the slice members.

Example

The following example returns a set containing the members on the columns axis:

```
Axis ( 0 )
```

19.7.8.8 BottomCount

OLAP Syntax.

Sorts a set, then returns a specified number of items from the bottom of the set. The set is sorted by the value of **<NumericExpression>** , if specified.

Syntax

```
BottomCount(<Set>, <Count>[, <NumericExpression>])
```

Parameters

<Set>	Any set.
<Count>	Number of items at the bottom of the set that are required.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

This example orders the members of **Products** by their value of **Sales**. A set containing the 5 members with the lowest values of **Sales** is returned.

```
BottomCount( AllMembers({Products}), 5, {profit_loss@'Sales'})
```

See also [AllMembers - Dimension \[page 552\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.9 BottomPercent

OLAP Syntax.

Sorts a set, then returns the lowest value elements to have a cumulative total greater than a specified percentage of the set total.

Syntax

```
BottomPercent(<Set>, <Percentage>, <NumericExpression>)
```

Parameters

<Set>	Any set.
<Percentage>	A percentage specifying the number of items to be returned.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

The members of Products are ordered by their value of Sales. A set is returned, containing a number of items with a cumulative total of Sales greater than 10% of the set total.

```
BottomPercent( AllMembers({Products}), 10, {profit_loss@'Sales'})
```

See also [AllMembers - Dimension \[page 552\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.10 BottomSum

OLAP Syntax.

Sorts a set, then returns the lowest value elements to have a cumulative total greater than a specified value.

Syntax

```
BottomSum(<Set>, <Value>, <NumericExpression>)
```

Parameters

<Set>	Any set.
<Value>	The cumulative value of the returned values.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

The members of Products are ordered by their value of Sales. A number of items are returned, whose cumulative total is greater than 1000.

```
BottomSum ( AllMembers({Products}), 1000, {profit_loss@'Sales'} )
```

See also [AllMembers - Dimension \[page 552\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.11 Children

OLAP Syntax.

Returns the children of a member. The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
Children(<Member>[, <ALL>])
```

Parameters

<Member>	Any member.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set containing the children of **Fruit**.

```
Children({'Fruit'})
```

The returned set would include the members: **Apples, Pears, Oranges, Bananas, Peaches**.

19.7.8.12 Crossjoin

OLAP Syntax.

Returns the cross product of two sets.

Syntax

```
Crossjoin(<Set1>, <Set2>)
```

Alternative syntax

```
<Set1>*<Set2>
```

Parameters

<Set1>	The first set for the cross product.
<Set2>	The second set for the cross product.

Return value

Returns a set.

Remarks

The function generates the cross product of two sets by this method:

```
Set A= {A1, A2, ..., An}  
Set B = {B1, B2, ..., Bn}
```

Then cross join of set A and set B is:

```
(A1, B1), (A1, B2), ..., (A1, Bn),  
(A2, B1), (A2, B2), ..., (A2, Bn),  
.  
.
```



```
(An, B1), (An, B2), ..., (An, Bn)
```

Example

This example returns the cross product of a set containing **January** and **March**, and a set containing the children of **Fruit**.

```
CrossJoin( Set({'January'}, {'March'}), Children({'Fruit'}) )
```

See also [Set \[page 595\]](#).

19.7.8.13 CurrentSet

OLAP Syntax.

Returns the current set, which contains all the members on the current viewpoint.

Syntax

```
CurrentSet()
```

Parameters

No parameters.

Return value

Returns a set.

Example

This example returns a set containing the members on the current viewpoint:

```
CurrentSet()
```

19.7.8.14 Descendants - Distance

OLAP Syntax.

Returns a set of descendants of a member at a specified number of levels away in the hierarchy. If the level is not specified, the function returns all descendants of the member. The optional flag **<ALL>** , specifies that calculated members are included. The optional **<Desc_flag>** modifier can be used to modify which descendants are returned.

Syntax

```
Descendants(<Member>[, <Distance>[, <Desc_flag>]][, <ALL>])
```

Parameters

<Member>	The reference member.
<Distance>	The number of levels away in the hierarchy that the descendants can be found.
<Desc_flag>	A flag modifying the returned set.
<ALL>	Include calculated members.

The Desc_flag modifier can take the following:

<SELF>	Returns only descendants from the specified level. <Member> is included only if it is a member of the specified level.
<AFTER>	Returns descendant members from all levels below the specified level.
<BEFORE>	Returns descendant members for all levels from <Member> up to, but not including, the specified level.
<BEFORE_AND_AFTER>	Returns descendant members from all levels below the level containing <Member> , excluding all members of the specified level.
<SELF_AND_AFTER>	Returns descendant members from the specified level and the levels below to it.

<SELF_AND_BEFORE>	Returns descendant members from the specified level and the levels between <Member> and the specified level.
<SELF_BEFORE_AFTER>	Returns descendant members from all levels below the level containing <Member> .
<LEAVES>	Returns only the leaf descendant members between the level containing <Member> and the specified level.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set containing the descendants of **Fruit** two levels down the hierarchy:

```
Descendants( {'Fruit'}, 2)
```

The returned set contains the members **Red** and **Green**.

19.7.8.15 Descendants - Level

OLAP Syntax.

Returns a set of descendants of a member at a specified level in the hierarchy. If a level is not specified, all descendants of the member are returned. The optional flag **<ALL>** , specifies that calculated members are included. The optional **<Desc_flag>** modifier can be used to modify which descendants are returned

Syntax

```
Descendants(<Member>[, <Level>[, <Desc_flag>]][, <ALL>])
```

Parameters

<Member>	The reference member.
<Level>	The level containing the required descendants.
<Desc_flag>	A flag modifying the returned set.
<ALL>	Include calculated members.

The Desc_flag modifier can take the following:

<SELF>	Returns only descendants from the specified level. <Member> is included only if it is a member of the specified level.
<AFTER>	Returns descendant members from all levels below the specified level.
<BEFORE>	Returns descendant members for all levels from <Member> up to, but not including, the specified level.
<BEFORE_AND_AFTER>	Returns descendant members from all levels below the level containing <Member> , excluding all members of the specified level.
<SELF_AND_AFTER>	Returns descendant members from the specified level and the levels below it.
<SELF_AND_BEFORE>	Returns descendant members from the specified level and the levels between <Member> and the specified level.
<SELF_BEFORE_AFTER>	Returns descendant members from all levels subordinate to the level containing <Member> .
<LEAVES>	Returns only the leaf descendant members between the level containing <Member> and the specified level.

Return value

Returns a set.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set containing the descendants of `Fruit` at the level specified by `level-4`:

```
Descendants( {'Fruit'}, {Products@level-4} )
```

The returned set contains the members `Red` and `Green`.

19.7.8.16 Distinct

OLAP Syntax.

Eliminates all duplicate tuples from a set, returning the new set.

Syntax

```
Distinct(<Set>)
```

Parameters

<Set>	Any set.
-------	----------

Return value

Returns a set.

Example

This example removes all duplicate tuples from a set containing the members of `Products`, and returns the new set:

```
Distinct( AllMembers({Products}) )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.8.17 DrilldownLevel - Index

OLAP Syntax.

Drills down on the members of a set to one level below the lowest represented member in the set, or one level below a specified level. The function organizes the resulting members under their parents, and the order of the original set is preserved.

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownLevel(<Set>[, <Index>][, <ALL>])
```

Parameters

<Set>	Any set.
<Index>	An optional reference level.
<ALL>	Include calculated members.

Return value

Returns a set.

Remarks

The index value of the first level is 0. The second level has an index of 1, and so on.

Example

This function example uses the sample dimension Products. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set composed of the descendants of **Vegetables** and **Apples** at the level below the level containing **Apples**. The resulting set also contains the members **Vegetables** and **Apples**, the descendants are organized under their parents in the set.

```
DrilldownLevel( Set({'Vegetables'}, {'Apples'}) )
```

See also [Set \[page 595\]](#).

19.7.8.18 DrilldownLevel - Level

OLAP Syntax.

Drills down on the members of a set to one level below the lowest represented member in the set, or one level below a specified level. The function organizes the resulting members under their parents, and the order of the original **<Set>** is preserved.

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownLevel(<Set>[, <Level>][, <ALL>])
```

Parameters

<Set>	Any set.
<Level>	An optional reference level.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set composed of the descendents of **Vegetables** and **Apples** at the level below **Level-3**. The resulting set also contains the members **Vegetables** and **Apples**, their descendants are organized under their parents in the resulting set.

```
DrilldownLevel( Set({'Vegetables'}, {'Apples'}, {Products@level-3} ) )
```

See also [Set \[page 595\]](#).

19.7.8.19 DrilldownLevelBottom

OLAP Syntax.

Drills down on the members of a set to one level below the lowest represented member in the set, or one level below a specified level. After drilling down, the function orders the members of the set according to the value of **<NumericExpression>** . The function returns a set composed of a number of the lowest value members for each of the original set members. Each of the original members is returned along a number of its descendants, the number of which is specified by **<Count>** .

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownLevelBottom(<Set>, <Count> [, <Level>] [, <NumericExpression>][, <ALL>])
```

Parameters

<Set>	Any set.
<Count>	The number of members to be returned per original member.
<Level>	The level containing the children.
<NumericExpression>	The value that the count is based on.
<ALL>	Includes calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set composed of the descendants of **Vegetables** and **Apples** at the level below **level-3**. The members of the resulting set are ordered by their value of **Sales**, and the 5 members with the lowest values returned.

```
DrilldownLevelBottom( Set({'Vegetables'}, {'Apples'}), 5,  
{profit_loss@level-3} , {profit_loss@'Sales'} )
```

This returns a set containing **Vegetables**, its 5 lowest valued descendants; and **Apples**, with its 5 lowest valued descendants.

See also [Set \[page 595\]](#).

19.7.8.20 DrilldownLevelTop

OLAP Syntax.

Drills down on the members of a set to one level below the lowest represented member in the set, or one level below a specified level. After drilling down, the function orders the members of the set according to their value of **<NumericExpression>** . The function returns a set composed of a number of the highest value members for each of the original set members. Each of the original members is returned along with a number of its descendants, specified by **<Count>** .

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownLevelTop(<Set>,<Count> [,<Level>] [,<NumericExpression>][,<ALL>])
```

Parameters

<Set>	Any set.
<Count>	The number of children to be returned per original member.
<Level>	The level containing the children.
<NumericExpression>	The value that the count is based on.

<ALL>

Include calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

This example returns a set composed of the descendants of **Vegetables** and **Apples** at the level below **level-3**. The members of the resulting set are ordered by their value of **Sales**, and the 5 members with the highest values returned.

```
DrilldownLevelTop( Set({'Vegetables'},{'Apples'}), 5, {profit_loss@level-3},  
{profit_loss@'Sales'})
```

This returns a set containing **Vegetables**, its 5 highest valued descendants from **level-4**, and **Apples**, with its 5 highest valued descendants from **level-4**.

See also [Set \[page 595\]](#).

19.7.8.21 DrilldownMember

OLAP Syntax.

Drills down on members in a set that are present in a second specified set.

The function drills down on members of **<Set1>** that are present in **<Set2>** . If **<RECURSIVE>** is specified, the new resulting set will again be compared with **<Set2>** and the drilldown continues. This recursive loop continues until none of the members in **<Set2>** match the members of **<Set1>** .

The optional flag **<ALL>** specifies that calculated members are included.

Syntax

```
DrilldownMember(<Set1>,<Set2> [, <RECURSIVE>][, <ALL>])
```

Parameters

<Set1>	The set on which you want to drill down.
<Set2>	The reference set. Usually a subset of <Set1> .
<RECURSIVE>	If specified, the drilldown will continue.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

This example drills down on all members of the current set that are present in the second set. So if Vegetables and Apples are present in the set, they are drilled down.

```
DrilldownMember( CurrentSet(), Set({'Vegetables'}, {'Apples'}) )
```

See also [CurrentSet \[page 563\]](#) and [Set \[page 595\]](#).

19.7.8.22 DrilldownMemberBottom

OLAP Syntax.

Drills down on the members in a set that are present in a second specified set; only a specified number of children from the bottom are returned.

The members of **<Set1>** that are present in **<Set2>** are drilled down. If **<RECURSIVE>** is specified, the new resulting set will again be compared with **<Set2>** and the drilldown continues. This recursive loop continues until none of the members in **<Set2>** match the members of **<Set1>** .

After drilling down, the function orders the members of the set according to their value of **<NumericExpression>** . The function returns a set composed of a number of the lowest value members for each of the original **<Set>** members. Each of the original members is returned along with a number of its descendants. The number of descendants returned is specified by **<Count>** .

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownMemberBottom(<Set1>,<Set2>,<Count>, [<NumericExpression>][,<RECURSIVE>]  
[,<ALL>])
```

Parameters

<Set1>	The set on which you want to drill down.
<Set2>	The reference set. Usually a subset of <Set1> .
<Count>	The number of children to be returned per member.
<NumericExpression>	The value by which the returned members are determined.
<RECURSIVE>	If specified, the drilldown continues.
<ALL>	Includes calculated members.

Return value

Returns a set.

Example

This example drills down on any members of the current set present in the second set, so if **Vegetables** or **Apples** are present in the current set, they are drilled down.

The function then returns for each of the original members, 5 drilled down members whose values of **Sales** are lowest.

```
DrilldownMember( CurrentSet(), Set({'Vegetables'}, {'Apples'}), 5,  
{profit_loss@'Sales'} )
```

See also [CurrentSet \[page 563\]](#) and [Set \[page 595\]](#).

19.7.8.23 DrilldownMemberTop

OLAP Syntax.

Drills down on the members in a set that are present in a second specified set; only a specified number of children from the top are returned.

The members of **<Set1>** that are present in **<Set2>** are drilled down. If **<RECURSIVE>** is specified, the new resulting set will again be compared with **<Set2>** and the drilldown continues. This recursive loop continues until none of the members in **<Set2>** match the members of **<Set1>** .

After drilling down, the function orders the members of the set according to their value of **<NumericExpression>** . The function returns a set composed of a number of the highest value members for each of the original **<Set>** members. Each of the original members is returned along with a number of its descendants. The number of descendants returned is specified by **<Count>** .

The optional flag **<ALL>** , specifies that calculated members are included.

Syntax

```
DrilldownMemberTop(<Set1>,<Set2>,<Count>,[<NumericExpression>][,<RECURSIVE>][,<ALL>])
```

Parameters

<Set1>	The set whose members will have drill down performed on them.
<Set2>	The reference set. Usually a subset of <Set1> .
<Count>	The number of children to be returned per member.
<NumericExpression>	The value by which the returned members are determined.
<RECURSIVE>	If specified, the drilldown continues.
<ALL>	Includes calculated members.

Return value

Returns a set.

Example

This example drills down on any members of the current set present in the second set, so if `Vegetables` or `Apples` are present in the current set, they are drilled down.

The function then returns for each of the original members, 5 drilled down members whose values of `Sales` are highest.

```
DrilldownMember( CurrentSet(), Set({'Vegetables'}, {'Apples'}), 5,
{profit_loss@'Sales'} )
```

See also [CurrentSet \[page 563\]](#) and [Set \[page 595\]](#).

19.7.8.24 DrillupLevel

OLAP Syntax.

Drills up on the members of a set that are below a specified level. If a level is not specified, the function drills up on the lowest level members of the set.

Syntax

```
DrillupLevel(<Set>[, <Level>])
```

Parameters

<Set>	Any set.
<Level>	The lowest permitted level.

Return value

Returns a set.

Example

The following example returns the current set, having drilled up on any members below level-3:

```
DrillupLevel( CurrentSet(), {Products@level-3} )
```

See also [CurrentSet \[page 563\]](#).

19.7.8.25 DrillupMember

OLAP Syntax.

Drills up on the members in a set that are present in a second specified set.

Syntax

```
DrillupMember( <Set1>, <Set2> )
```

Parameters

<Set1>	Any set.
<Set2>	The reference set.

Return value

Returns a set.

Example

This example drills up on any members of the current set present in the second set, so if **Vegetables** or **Apples** are present in the current set, they are drilled up.

```
DrillupMember( CurrentSet(), Set({'Vegetables'}, {'Apples'}))
```

See also [CurrentSet \[page 563\]](#) and [Set \[page 595\]](#).

19.7.8.26 Except

OLAP Syntax.

Returns all members of a set that are not found in a second specified set, with the option of retaining duplicates.

Syntax

```
Except( <Set1>, <Set2>[, <ALL>] )
```

Parameters

<Set1>	Any set.
<Set2>	Any set.
<ALL>	Retain duplicates.

Return value

Returns a set.

Example

The following example returns a set containing all members of **Products** that are not children of **Fruit**. Duplicates are not retained.

```
Except( AllMembers({Products}), Children({'Fruit'}) )
```

See also [AllMembers - Dimension \[page 552\]](#) and [Children \[page 561\]](#).

19.7.8.27 Extract

OLAP Syntax.

Returns a set of tuples that contain all elements from a specified set that belong to a specified dimension, or dimensions.

See also

[Crossjoin \[page 562\]](#)

Syntax

```
Extract(<Set>, <Dimension>[, <Dimension>]...)
```

Parameters

<Set>	Any set.
<Dimension>	The dimension specifying the elements to extract.

Return value

Returns a set.

Remarks

This function performs the opposite of Crossjoin.

Example

The following example returns all elements from the current set found in the **Products** dimension:

```
Extract( CurrentSet(), {Products} )
```

See also [CurrentSet \[page 563\]](#).

19.7.8.28 Filter

OLAP Syntax.

Applies a filter to a specified set using a search condition.

The logical condition specified by **<SearchCondition>** is evaluated for each member of **<Set>** . If **<SearchCondition>** evaluates TRUE for a member, that member is included in the returned set. If **<SearchCondition>** evaluates FALSE the member is not included in the set.

Syntax

```
Filter(<Set>, <SearchCondition>)
```

Parameters

<Set>	Any set.
<SearchCondition>	A logical expression defining the filter criteria.

Return value

Returns a set.

Example

The following example returns a set containing members from **Products**, whose **Sales** are greater than **Costs of Sale**:

```
Filter( AllMembers({Products}), {'Sales'} > {'Costs of Sale'} )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.8.29 Generate

OLAP Syntax.

Applies a set to each member within a second set, then joins the resulting set by a union. If the **<ALL>** flag is specified, duplicates in the resulting set are retained.

Syntax

```
Generate(<Set1>, <Set2>[, <ALL>])
```

Parameters

<Set1>	Any set.
<Set2>	Any set.
<ALL>	Retain duplicates.

Return value

Returns a set.

Example

The following example joins each member of **profit_loss** to the children of **Fruit**, duplicates in the resulting set are discarded.

```
Generate( AllMembers({profit_loss}), Children({'Fruit'}) )
```

See also [AllMembers - Dimension \[page 552\]](#), [Children \[page 561\]](#), and [Union \[page 603\]](#).

19.7.8.30 GenerateSet

OLAP Syntax.

Searches a specified dimension for members satisfying the search condition. The members satisfying the condition are returned in a set.

If **<Dimension>** is unspecified, the dimension containing the calculated member is used.

Syntax

```
GenerateSet( SetCondition [,Dimension])
```

Parameters

<SetCondition>	Specifies the criteria on which to base the search.
-----------------------------	---

<Dimension>	Any dimension.
--------------------------	----------------

<SetCondition> specifies the criteria on which to base the search. You can specify multiple conditions in your search using the AND and OR operators. Use brackets to set the precedence of the search criteria.

You can search for the following conditions:

Property	Syntax
Member Name	Name = <> { <searchtext> }
Unique Name	UniqueName = <> { <searchtext> }
Level	Level = <> > < <= >= { <searchtext> } <LevelDepth>

<searchtext> is the string used for the search condition. You can use the * wildcard in your search. For example the condition Name={Week *} specifies all members with names starting with Week.

<LevelDepth> specifies the level in a hierarchy. For example, Level=0 specifies the top level in a dimension, Level=1 specifies the next level.

Return value

Returns a set.

Example

The following examples show some uses of GenerateSet:

```
GenerateSet( UniqueName={*A*} OR UniqueName={*B*}, Products)
```

Returns a set containing all members in the dimension `Products` with the letters `A` or `B` in their unique names.

```
GenerateSet( UniqueName={'*car*'} AND Level=1 )
```

Returns a set containing all members from Level 1 in the hierarchy with `car` in the unique names.

19.7.8.31 Head

OLAP Syntax.

Returns a specified number of elements from the top of a set. If `<Count>` is not specified, only the first element from the set is returned.

See also

[Tail \[page 598\]](#)

Syntax

```
Head(<Set>[, <Count>])
```

Parameters

<Set>	Any set.
<Count>	The number of elements to be returned.

Return value

Returns a set.

Example

The following example returns the first five elements of a set containing all members of **Products**:

```
Head( AllMembers({Products}), 5 )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.8.32 Hierarchize

OLAP Syntax.

Returns a set containing the elements of a specified set ordered hierarchically.

Syntax

```
Hierarchize(<Set>[, <POST>])
```

Parameters

<Set>	Any set.
<POST>	Optional flag specifying that a post-natural ordering be used.

Return value

Returns a set.

Example

The following example orders the members of **Products** into a hierarchy, and returns the resultant set:

```
Hierarchize( AllMembers({Products}) )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.8.33 Intersect

OLAP Syntax.

Returns the elements common to two sets, optionally retaining duplicates.

Syntax

```
Intersect(<Set1>, <Set2>[, <ALL>])
```

Parameters

<Set1>	Any set.
<Set2>	Any set.
<ALL>	Retain duplicates.

Return value

Returns a set.

Example

The following example returns a set containing all members of the current set that are also children of **Fruit**:

```
Intersect( CurrentSet(), Children({'Fruit'}) )
```

See also [CurrentSet \[page 563\]](#) and [Children \[page 561\]](#).

19.7.8.34 LastPeriods

OLAP Syntax.

Returns a set containing members up to, and including, a specified member.

The function returns a set containing a number of members before **<Member>** . If **<Index>** is positive, members before are returned. If **<Index>** is negative, members after **<Member>** are returned. If **<Index>** is zero, the function returns an empty set.

Syntax

```
LastPeriods(<Index>, <Member>)
```

Parameters

<Index>	Number indicating the number of members to be returned.
<Member>	The reference member.

Return value

Returns a set.

Example

The following example returns a set containing February, March, April, May, and June, which are the 5 members up to and including June.

```
LastPeriods( 5, {time@'June'} )
```

19.7.8.35 Members - Dimension

OLAP Syntax.

Returns a set containing all members of a specified dimension. The optional flag **<ALL>** specifies that calculated members are included.

Syntax

```
Members(<Dimension>[, <ALL>])
```

Parameters

<Dimension>	Any dimension.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

The following example returns a set containing the members of the dimension **Products**:

```
Members( {Products} )
```

19.7.8.36 Members - Level

OLAP Syntax.

Returns a set containing all members of a specified level. The optional flag **<ALL>** specifies that calculated members are included.

Syntax

```
Members(<Level>)
```

Parameters

<Level>	Any level.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension **Products**. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing all members at **level-1**:

```
Members( {Products@level-1} )
```

The set would include the members **Meat**, **Seafood**, **Fruit&Veg**, **Snacks**, **Drinks**, and **Canned**.

19.7.8.37 MTD

OLAP Syntax.

Returns a set of members from the Month level in a time dimension, starting with the first member and ending with a specified member.

This is a shortcut function to the **PeriodsToDate** function that specifies the level to be the Month level in a dimension.

Syntax

```
MTD(<Member>)
```

Parameter

<Member>

The last member to be returned.

Return value

Returns a set.

Remarks

By default MTD assumes that level - 2 in your time dimension represents months. However, if your time dimension contains the word **month**, the top level is set as the Month dimension.

Alternatively you can use [PeriodsToDate \[page 592\]](#), and specify your own level.

Example

The following example returns all members of February, up to and including the member 5th.

```
MTD( {time@'2002'@'Q1'@'February'@'5th'} )
```

The function returns the members: 1st, 2nd, 3rd, 4th, and 5th.

See also [PeriodsToDate \[page 592\]](#).

19.7.8.38 NameToSet

OLAP Syntax.

Returns a set containing a single member specified by a string expression.

Syntax

```
NameToSet ( <MemberName> )
```

Parameters

<MemberName>

String specifying the member name.

Return value

Returns a set.

Remarks

If **<MemberName>** is not a valid member, an empty set is returned.

Example

The following example returns a set containing the member **Fruit**:

```
NameToSet( {'Fruit'} )
```

19.7.8.39 NonEmptyCrossjoin

OLAP Syntax.

Returns a set containing the cross product of two or more sets.

Empty tuples are automatically excluded. If **<CrossjoinCount>** is not specified, all the sets supplied as parameters are crossjoined. Alternatively **<CrossjoinCount>** can be supplied to modify the function. **<CrossjoinCount>** specifies the number of sets that should be crossjoined. For example if three sets are specified in the arguments, and **<CrossjoinCount>** is 2. **<Set1>** and **<Set2>** are crossjoined, and **<Set3>** is used to determine which cells in the resulting set are empty.

Syntax

```
NonEmptyCrossjoin( <Set1>, <Set2>[, <SetN>] ... [ <CrossjoinCount> ] )
```

Parameters

<Set1>	Any set.
<Set2>	Any set.
<SetN>	Any set.
<CrossjoinCount>	The number of sets you want to crossjoin.

Return value

Returns a set.

Example

The following example returns the cross product of three sets:

```
NonEmptyCrossjoin( AllMembers({Profit}), Children({'Fruit'}),  
Children('Fruit&Veg') )
```

See also [AllMembers - Dimension \[page 552\]](#), [Children \[page 561\]](#), and [Crossjoin \[page 562\]](#).

19.7.8.40 Order

OLAP Syntax.

Orders the members of a set.

The function returns a set containing the members of **<Set>** arranged by the value of **<NumericExpression>** . The optional ordering flag specifies how the returned set is ordered.

Syntax

```
Order( <Set>, <NumericExpression>[, <ASC> | <DESC> | <BASC> | <BDESC>] )
```

Parameters

<Set>	The set to be ordered.
<NumericExpression>	The value by which the members are ordered.
<ASC> <DESC> <BASC> <BDESC>	Ordering flag. <ASC> (ascending) is the default.
The ordering flag can be one of the following:	
<ASC>	Ascending, retaining hierarchy.
<DESC>	Descending, retaining hierarchy.
<BASC>	Ascending, breaking hierarchy.
<BDESC>	Descending, breaking hierarchy.

Return value

Returns a set.

Example

The following example returns a set containing all members of **Products** arranged according to the value of **Sales** in descending order:

```
Order( AllMembers({Products}), {profit_loss@'Sales'}, DESC )
```

See also [AllMembers - Dimension \[page 552\]](#).

19.7.8.41 PeriodsToDate

OLAP Syntax.

Returns a set of members from a specified level starting with the first member and ending with a specified member.

Syntax

```
PeriodsToDate([<Level>], [<Member>])
```

Parameters

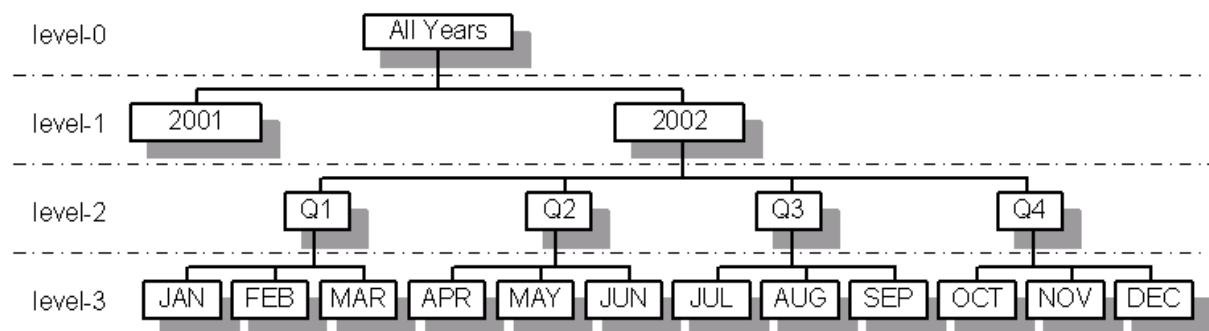
<Level>	The level containing the required members.
<Member>	The last member to be returned.

Return value

Returns a set.

Example

The examples for this function use the following dimension:



The following returns all members in Q2, up to and including the member MAY.

```
PeriodsToDate( {time@level-2}}, {time@'2002'@'Q2'@'MAY'} )
```

The function returns the members: APR and MAY.

Setting the level to level-1, returns all the months in 2002, up to and including the member MAY:

```
PeriodsToDate( {time@level-2}}, {time@'2002'@'Q2'@'MAY'} )
```

The function returns the members: JAN, FEB, MAR, APR, and MAY.

19.7.8.42 QTD

OLAP Syntax.

QTD stands for "quarter to date." Returns a set of members from the **Quarter** level in a time dimension, starting with the first member and ending with a specified member.

This is a shortcut function to the `PeriodsToDate` function that specifies the level to be the **Quarter** level in a dimension.

Syntax

```
QTD(<Member>)
```

Parameter

<Member>

The last member to be returned.

Return value

Returns a set.

Remarks

By default QTD assumes that `Level-1` in your time dimension represents quarters. However, if your time dimension contains the word **quarter**, the top level is set as the **Quarter** dimension.

Alternatively you can use [PeriodsToDate \[page 592\]](#), and specify your own level.

Example

The following example returns all members of **Q2**, up to and including the member **Week16**.

```
QTD( {time@'2002'@'Q2'@'Week16'} )
```

The function returns the members: **Week13**, **Week14**, **Week15**, and **Week16**.

See also [PeriodsToDate \[page 592\]](#).

19.7.8.43 Set

OLAP Syntax.

Returns a set containing specified tuples or members.

Syntax

```
Set ( <Tuple> | <Member> | <Tuple> : <Tuple> , <Tuple> | <Member> | <Tuple> : <Tuple> )
```

Parameters

<Tuple>	Any tuple.
<Member>	Any member.

Return value

Returns a set.

Remarks

The notation **<Tuple1> : <Tuple2>** specifies all tuples from **<Tuple1>** to **<Tuple2>** along a level. The tuples specified should be on the same level.

Example

This function example uses the sample dimension Products. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing the three members specified:

```
Set ( { 'Vegetables' }, { 'Fruit' }, { 'Apples' } )
```

19.7.8.44 Siblings

OLAP Syntax.

Returns a set containing all siblings of a member, optionally including calculated members.

Syntax

```
Siblings(<Member>[, <ALL>])
```

Parameters

<Member>	Any member.
<ALL>	Include calculated members.

Return value

Returns a set.

Example

This function example uses the sample dimension `Products`. See [Sample Dimension \[page 614\]](#) for more details.

The following example returns a set containing all siblings of the member `Fruit`:

```
Siblings( {'Fruit'} )
```

The set would contain the members `Vegetables` and `Fruit`.

19.7.8.45 StripCalculatedMembers

OLAP Syntax.

Removes calculated members from a set.

Syntax

```
StripCalculatedMembers(<Set>)
```

Parameters

<Set>	Any set.
-------	----------

Return value

Returns a set.

Example

The following example removes all calculated members from the row axis.

```
StripCalculatedMembers( Axis(1,ALL) )
```

See also [Axis \[page 557\]](#).

19.7.8.46 Subset

OLAP Syntax.

Returns a subset of elements from a set. The number of elements returned is specified by <Count> , and starts with the element specified by <Start> .

Syntax

```
Subset( <Set>, <Start>, [<Count>] )
```

Parameters

<Set>	Any set.
<Start>	Index specifying the first element to be included in the subset.
<Count>	Number specifying the number of tuples, starting with <Start> , that should be returned.

Return value

Returns a set.

Remarks

The first element in the set is specified by an index of 0. The second element in the set has an index of 1, and so on.

Example

The following example returns the second, third, and fourth elements from the current set:

```
Subset( CurrentSet(), 1, 3)
```

See also [CurrentSet \[page 563\]](#).

19.7.8.47 Tail

OLAP Syntax.

Returns a subset from the end of a set. The number of elements returned is specified by **<Count>** . If **<Count>** is not specified, the last element is returned.

See also

[Head \[page 583\]](#)

Syntax

```
Tail(<Set>[, <Count>])
```

Parameters

<Set>	Any set.
<Count>	The number of elements to return.

Return value

Returns a set.

Example

The following example returns 4 elements from the end of the current set:

```
Tail( CurrentSet(), 4 )
```

See also [CurrentSet \[page 563\]](#).

19.7.8.48 ToggleDrillState

OLAP Syntax.

Toggles the drill state of members that are common to two specified sets. If a member has children then it is drilled down. If a member has no children it is drilled up.

If **<RECURSIVE>** is specified, the new resulting set will again be compared with **<Set2>** and the drill continues. This recursive loop continues until none of the members in **<Set2>** match the members of **<Set1>** .

Syntax

```
ToggleDrillState(<Set1>, <Set2>[, <RECURSIVE>])
```

Parameters

<Set1>	Any set.
<Set2>	Any set.
<RECURSIVE>	If specified, the drillup or drilldown will continue.

Return value

Returns a set.

Example

This function example uses the sample dimension Products. See [Sample Dimension \[page 614\]](#) for more details.

This example toggles the drill state of any members of the current set present in the second set; so, if Apples is present in the current set, it is drilled down because it has children; if Bananas is present in the current set, it is drilled up because it has no children.

```
DrilldownMember( CurrentSet(), Set({'Apples'}, {'Bananas'}) )
```

See also [CurrentSet \[page 563\]](#) and [Set \[page 595\]](#).

19.7.8.49 TopCount

OLAP Syntax.

Sorts a set, then returns a specified number of items from the top of the set.

The set is first ordered by the value of <NumericExpression> . A set is then returned containing a number of items from the top of the set, as specified by <Count> .

Syntax

```
TopCount ( <Set>, <Count>[, <NumericExpression>] )
```

Parameters

<Set>	Any set.
<Count>	Number of items at the top of the set to return.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

This example orders the members of the current set by their value of **Sales**. A set containing the 5 members with the highest values of **Sales** is returned.

```
TopCount( CurrentSet(), 5, {profit_loss@'Sales'})
```

See also [CurrentSet \[page 563\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.50 TopPercent

OLAP Syntax.

Sorts a set, then returns the highest value elements to have a cumulative total greater than a specified percentage of the set total

Syntax

```
TopPercent(<Set>, <Percentage>, <NumericExpression>)
```

Parameters

<Set>	Any set.
<Percentage>	A percentage specifying the number of items to be returned.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

The current set is ordered by the member `Sales`. A set is returned, containing a number of items with a cumulative total of `Sales` greater than 10% of the set total.

```
TopPercent( CurrentSet(), 10, {profit_loss@'Sales'})
```

See also [CurrentSet \[page 563\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.51 TopSum

OLAP Syntax.

Sorts a set, then returns the highest value elements that have a cumulative total greater than a specified value.

Syntax

```
TopSum(<Set>, <Value>, <NumericExpression>)
```


Parameters

<Set>	Any set.
<Value>	The value determining which elements are returned.
<NumericExpression>	The value by which the set should be ordered.

Return value

Returns a set.

Example

The current set is ordered by the member **Sales**. A number of items are returned from the top of the set, whose cumulative total is greater than 1000.

```
TopSum( CurrentSet(), 1000, {profit_loss@'Sales'})
```

See also [CurrentSet \[page 563\]](#).

Remarks

This function breaks the hierarchy.

19.7.8.52 Union

OLAP Syntax.

Returns the union of two sets, optionally retaining duplicates.

The function merges the first set with the second set to form a new set. You can specify **ALL** to include multiple occurrences of the same elements in the new set.

Syntax

```
Union(<Set1>, <Set2>)
```

Alternative syntax

```
<Set1> + <Set2>
```

Parameters

<Set1>	The first set for the union.
<Set2>	The second set for union.
<ALL>	Retain duplicates.

Return value

Returns a set.

Example

This example returns the union of the current set and a set containing the children of **Fruit**.

```
Union( CurrentSet(), Children({'Fruit'}) )
```

See also [CurrentSet \[page 563\]](#) and [Children \[page 561\]](#).

Remarks

Duplicates are retained if using the alternative syntax.

19.7.8.53 WTD

OLAP Syntax.

WTD stands for "week to date." Returns a set of members from the Week level in a time dimension, starting with the first member and ending with a specified member.

This is a shortcut function to the PeriodsToDate function that specifies the level to be the Week level in a dimension.

Syntax

```
WTD( <Member> )
```

Parameter

<Member>

The last member to be returned.

Return value

Returns a set.

Remarks

By default WTD assumes that Level 1 - 2 in your time dimension represents weeks. However, if your time dimension contains the word week, the top level is set as the week dimension.

Alternatively you can use [PeriodsToDate \[page 592\]](#), and specify your own level.

Example

The following example returns all members of Week6, up to and including the member Thursday.

```
WTD( {time@'2002'@'Q1'@'Week6'@'Thursday'} )
```

The function returns the members: Monday, Tuesday, Wednesday, and Thursday.

See also [PeriodsToDate \[page 592\]](#).

19.7.8.54 YTD

OLAP Syntax.

YTD stands for "year to date." Returns a set of members from the Year level in a time dimension, starting with the first member and ending with a specified member.

This is a shortcut function to the PeriodsToDate function that specifies the level to be the Year level in a dimension.

Syntax

```
YTD(<Member>)
```

Parameter

<Member>	The last member to be returned.
----------	---------------------------------

Return value

Returns a set.

Remarks

YTD uses the top level in your dimension as the Year dimension.

Alternatively you can use [PeriodsToDate \[page 592\]](#), and specify your own level.

Example

The following example returns all members of 2002, up to and including the member May.

```
YTD( {time@'2002'@'Q2'@'May'} )
```

The function returns the members: January, February, March, April, and May.

See also [PeriodsToDate \[page 592\]](#).

19.7.9 String functions

Click a link to view one of the topics in this section:

- [GetName \[page 607\]](#)
- [GetUniqueName \[page 607\]](#)
- [GetCaption \[page 608\]](#)

19.7.9.1 GetName

OLAP Syntax.

Returns the name of a member.

Syntax

```
GetName(<Member>)
```

Parameters

<Member>

Any member.

Return value

Returns a string.

Example

The following example returns the string: 'Fruit'.

```
GetName ({Products@'AllProducts'@'Fruit&Veg'@'Fruit'})
```

See also [GetUniqueName \[page 607\]](#) and [GetCaption \[page 608\]](#).

19.7.9.2 GetUniqueName

OLAP Syntax.

Returns the unique name of a member.

Syntax

```
GetUniqueName(<Member>)
```

Parameters

<Member>	Any member.
----------	-------------

Return value

Returns a string.

Example

The following example returns the string: `Products@'AllProducts'@'Fruit&Veg'@'Fruit'`, which is the unique name of the member `Fruit`.

```
GetUniqueName ({'Fruit'})
```

See also [GetName \[page 607\]](#) and [GetCaption \[page 608\]](#).

19.7.9.3 GetCaption

OLAP Syntax.

Returns the caption of a member.

Syntax

```
GetCaption(<Member>)
```

Parameters

<Member>	Any member.
-----------------------	-------------

Return value

Returns a string.

Example

The following example returns the string: **Fruit**, which is the caption given to the member **Fruit**.

```
GetCaption ({Products@'AllProducts'@'Fruit&Veg'@'Fruit'})
```

See also [GetName \[page 607\]](#) and [GetUniqueName \[page 607\]](#).

19.7.10 Tuple functions

Click a link to view one of the topics in this section:

- [Array \[page 609\]](#)
- [Current \[page 610\]](#)
- [Item - Tuple \[page 610\]](#)
- [Tuple \[page 611\]](#)

19.7.10.1 Array

OLAP Syntax.

An alias for [Tuple \[page 611\]](#).

See also

[Tuple \[page 611\]](#)

19.7.10.2 Current

OLAP Syntax.

Returns the current tuple from a specified set during an iteration. If a set is not specified, the current calculated member is returned.

Syntax

```
Current ( [ <Set> ] )
```

Parameters

<Set>

Any set.

Return value

Returns a tuple.

Example

This example returns the current tuple from the current set:

```
Current ( CurrentSet() )
```

See also [CurrentSet \[page 563\]](#).

19.7.10.3 Item - Tuple

OLAP Syntax.

Returns a tuple from a specified position in a set.

Syntax

```
Item(<Set>, <NumericExpression>)
```

Alternative syntax

```
<Set>(<NumericExpression>)
```

Parameters

<code><Set></code>	Any set.
<code><NumericExpression></code>	The position of the tuple in the set.

Return value

Returns a tuple.

Remarks

The first tuple in the set has a position of 0. The second tuple has a position of 1, and so on.

Example

The following example returns the third tuple from the current set.

```
Item( CurrentSet(), 2 )
```

See also [CurrentSet \[page 563\]](#).

19.7.10.4 Tuple

OLAP Syntax.

Returns a tuple containing specified members.

Syntax

```
Tuple(<Member>[, <Member>] . . . )
```

Parameters

<Member>	Any member.
----------	-------------

Return value

Returns a tuple.

Example

This example returns a tuple containing the members `Vegetables` and `Cost of Sales`:

```
Tuple( {'Vegetables'}, {'Cost of Sales'} )
```

19.7.11 Operators

The OLAP Syntax supports a number of common operators.

Click a link to view one of the topics in this section:

- [Arithmetic \[page 613\]](#)
- [Relational \[page 613\]](#)
- [Boolean \[page 613\]](#)

19.7.11.1 Arithmetic

+	Addition. Also used to find the union of two sets.
–	Subtraction.
*	Multiplication. Also used to find the cross product of two sets.
/	Division.
%	Percentage.
^	Exponential.

19.7.11.2 Relational

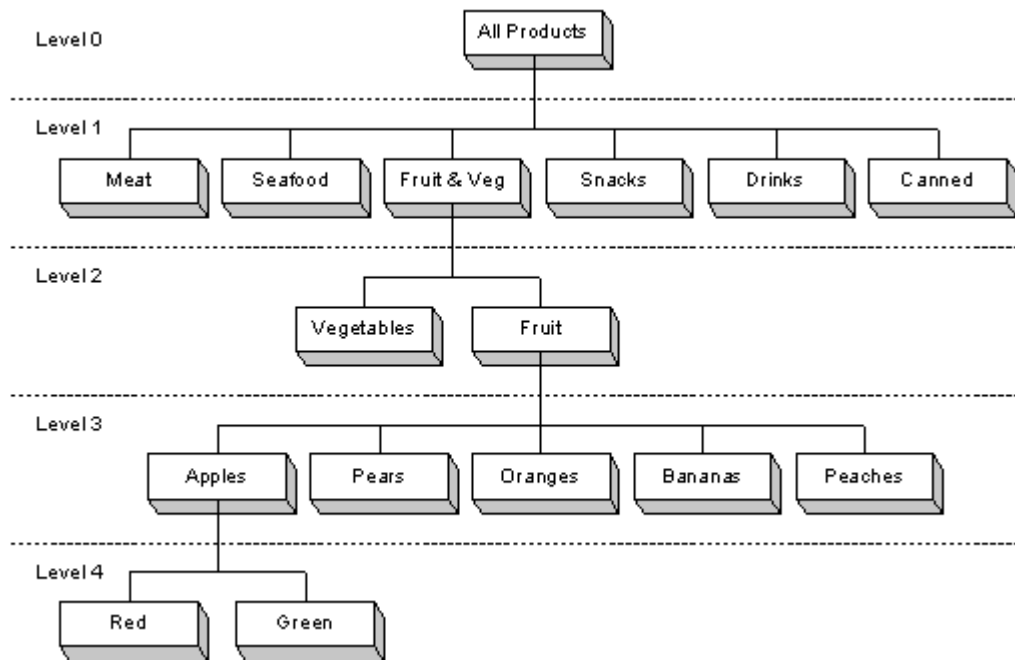
=	equal-to comparison.
<>	not-equal-to comparison.
<	less-than comparison.
>	greater-than comparison.
<=	less-than-or-equal comparison.
>=	greater-than-or-equal comparison.

19.7.11.3 Boolean

And	Combines two conditions, requiring both to be true.
Or	Combines two conditions, requiring either or both to be true.
Xor	Combines two conditions, requiring one to be true, and one to be false.
Not	Negates a condition. The condition must be false.
Eqv	Combines two conditions, requiring both to be the same.

19.7.12 Sample Dimension

The following dimension, **Products**, is used by a number of the function examples to clarify their use:



20 Printing, Exporting, and Viewing Reports

You can distribute finished reports using a variety of methods including printing, faxing, and exporting to different formats. You can also set up hyperlinks so that you can see Report Parts in various Report Viewers. Additionally, you can use smart tags with report objects in Microsoft Office products.

→ Tip

To ensure that the export to Word, Excel, PDF, or another format displays the way that you expect it to, use the [Preview tab \[page 93\]](#) option to preview the output as you are designing your report.

ⓘ Note

Not all operating systems render characters or fonts the same way; a report that displays properly on a Windows system may not display perfectly on a non-Windows (Linux, Unix, Mac) system. You may see either truncated or overlapping lines for multi-line fields, or the text lines may not wrap as expected.

To ensure that the report displays the way that you expect it to in a non-Windows system, verify the formatting results on the BI platform while you are designing the report. During the design phase, publish (or save) your report to the BI platform to test how it looks online. You can also test how a report exported to PDF from the BI platform is displayed.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

20.1 Distributing reports

Crystal Reports enables you to distribute your report using a variety of methods.

20.1.1 Printing a report

Procedure

1. On the [File](#) menu, select [Print](#).

→ Tip



Another way to do this is to click the Print button on the Standard toolbar.

The Print dialog box appears.

2. Choose the appropriate settings, and then click [OK](#).

The Printing Report dialog box appears showing the progress of your print job.

Results

For information about printers and printer drivers, see these topics:

Related Information

[Printer drivers \[page 269\]](#)

20.1.2 Faxing a report

Many fax applications, such as Microsoft Fax and Delrina WinFax, allow you to set up a printer driver that will fax documents over a modem. When using one of these applications, you can fax a report from Crystal Reports.

20.1.2.1 To fax a report

Procedure

1. On the [File](#) menu, click [Print](#).

The Print dialog box appears.

2. In the Print dialog box, click [Find Printer](#).

The Find Printers dialog box appears. Use this dialog box to select your fax driver.

3. Click [OK](#).

You are returned to the Print dialog box.

4. Choose the appropriate settings, and then click [OK](#).

Your fax application appears, prompting you to select a cover page and to fill in the appropriate fax information.

20.1.3 Exporting a report

Finished reports can be exported to a number of popular formats, such as spreadsheet, word processor, HTML, ODBC, and common data interchange formats. This makes the distribution of information easier. For example, you may want to use the report data to project trends in a spreadsheet package or to enhance the presentation of data in a desktop publishing package.

Note

Crystal Reports lets you insert objects anywhere on the report page. When you export to formats such as Microsoft Word, Microsoft Excel, and HTML, however, objects that you placed between lines are moved to the closest line in the output. To alleviate the formatting problems this behavior could cause, it is recommended that you use Guidelines when designing your reports. For more information, see [Designing with guidelines \[page 255\]](#).

The exporting process requires you to specify a format and a destination. The format determines the file type, and the destination determines where the file is located.

20.1.3.1 Export format types

The export formats supported by Crystal Reports can be broadly categorized in two groups: page-based formats and record-based formats.

Page-based formats tend to produce a more exact output. The emphasis of these formats is layout representation and formatting. Formatting refers to attributes such as font style, text color, text alignment, background color, and so on. Layout refers to object position, object size, and the relationship between these attributes and other objects. Depending on the format you choose, it may not be possible for the program to preserve all layout and formatting perfectly, but page-based formats, in general, preserve these properties as closely as possible.

With record-based formats, the emphasis is on data rather than the layout and formatting. However, in some formats—such as Microsoft Excel - Data only—you will notice that some formatting is exported. Some of the record-based formats are only data-exchange formats.

Crystal Reports (RPT)

Exporting to Crystal Reports format is much like using the Save As feature with the "Save Data with Report" option selected. This format exports (saves) your report with the current data without modifying the original report.

Crystal Reports Read-only (RPTR)

Crystal Reports Read-Only (RPTR) reports are Crystal reports that can be viewed with report viewer applications, but cannot be opened by the Crystal Reports designer. This enables you to protect your

intellectual property by preventing users from modifying the report or obtaining information about proprietary business logic that is stored in the report definition

HTML 3.2 and HTML 4.0

HTML export formats are page based. The HTML 4.0 format preserves the layout and formatting of the report by using DHTML. The HTML 3.2 format, however, cannot maintain all of the layout precisely. The HTML 3.2 format is intended for backward compatibility with older browsers that do not support HTML 4.0. All of the images in your report are saved externally and a hyperlink is inserted in the exported HTML output. Therefore, this export format generates more than one file in the output.

PDF

PDF format is a page-based format. The exported documents are intended for printing and redistribution. PDF format will export both layout and formatting in a manner that is consistent with how the report looks on the Preview tab. PDF format embeds the TrueType fonts that appear in the document. (Non-TrueType fonts are not supported.) This export format does not support Microsoft font-linking technology—which is used to provide support for some extended character sets such as Chinese HKCS. Therefore, the fonts used in the report must contain all of the required glyphs. These URI types are supported for hyperlinks: "http:", "https:" and "mailto:".

Microsoft Excel (XLSX)

Microsoft Excel (XLSX) is a page-based format that converts the report contents into Excel cells on a page-by-page basis. Content from multiple pages are exported to the same Excel worksheet. If a report element covers more than one cell, the export program merges cells to represent a report element. This export format retains most of the formatting, but does not export line and box elements from your report. This format is an enhancement to the existing Microsoft Excel (XLS) exporting type, and removes the 65536 rows and 256 columns restrictions.

Microsoft Excel (XLS)

Microsoft Excel format is a page-based format. This format converts your report contents into Excel cells on a page-by-page basis. Contents from multiple pages are exported to the same Excel worksheet. If a worksheet becomes full and there is more data to export, the export program creates multiple worksheets to accommodate the data. If a report object covers more than one cell, the export program merges cells to represent a report object. Microsoft Excel has a limit of 256 columns in a worksheet; therefore, any report object (or part of it) that is added to cells beyond 256 columns is not exported. This export format retains most of the formatting, but it does not export line and box objects from your report.

Microsoft Excel - Data-only (XLSX)

Microsoft Excel Data-only (XSLX) (.xlsx) is a record-based format that concentrates on data. However, this format does export most of the formatting as well. The data-only format does not merge cells; each object is added to only one cell. The data-only format can also export certain kinds of summaries in Crystal Reports as Excel functions. The summaries that are supported are SUM, AVERAGE, COUNT, MIN, and MAX.

Microsoft Excel - Data-only (XLS)

Microsoft Excel (XLS) Data-only is a record-based format that concentrates on data. However, this format does export most of the formatting as well. Unlike the Microsoft Excel format, this format does not merge cells; each object is added to only one cell. The data-only format can also export certain kinds of summaries in Crystal Reports as Excel functions. The summaries that are supported are SUM, AVERAGE, COUNT, MIN, and MAX.

To get the best output from Excel export formats, you should design your reports in an Excel-friendly way. For more information, refer to the Technical Brief called "Exporting to Microsoft Excel" found on the SAP Business Objects Support site.

Microsoft Word (RTF)

Microsoft Word is a page-based, exact format that produces an RTF (Rich Text Format) file. The exported file contains text and drawing objects to represent report objects. Individual objects are placed in text frames. This format is intended for use in applications such as fill-out forms where the space for entering text is reserved as empty text objects.

Almost all of the formatting is retained in this export format. Text objects cannot, however, be placed outside the left edge of the page. Therefore, if you have text objects in your report that are placed before the left edge of the printable area, they will be pushed right. (This is most noticeable in reports with wide pages.)

Microsoft Word - Editable (RTF)

The Microsoft Word - Editable format is different from the Microsoft Word format; it is a page-based format, but not all of the layout and formatting are preserved in the output. This format converts all of the report object contents to text lines. Unlike Microsoft Word format, this format does not use text frames. Text formatting is retained, but attributes such as background color, fill pattern, and so on may not be retained. All images in your report are inlined with the textual content and, therefore, the images automatically shift to accommodate text when the document is edited in Microsoft Word. This format does not export line and box objects from your report. As well, the special fields Page Number and Page N of M do not work when exported in this format.

The Microsoft Word - Editable format has an option to insert page breaks at the end of each report page. This option may not correspond to the page breaks created by Microsoft Word; the option is used mainly to separate your report-page contents.

Rich Text Format (RTF)

The Rich Text Format (RTF) format is similar to the Microsoft Word (97-2003) format.

ODBC

ODBC is a record-based, data-exchange format. When you use this format, you can export your report data to any ODBC-compliant database.

Record Style - Columns with spaces and Record Style - Columns without spaces

The record-style formats export report data as text. These formats export data only from Group and Details areas. The output contains one line per record in the database (for the report). The record-style formats are used mainly for data exchange.

Report Definition

The Report Definition format exports your report to a text file that contains a brief description of the report's design view. This format is maintained only for compatibility with Crystal Reports 5.0.

Separated Values (CSV)

The Separated Values format is a record-based, data-exchange format. It exports the report object contents as a set of values separated by separator and delimiter characters that you specify. (When a comma (,) is used to separate fields, the format is known as Comma Separated Values (CSV); this export format is popular among Microsoft Excel users.)

Like record-style formats, the Separated Values format also creates one line of values for each record in your report. A record contains all of the fields in each section of your report as seen in the Design view. That is, fields in the Report Header section are exported first, followed by the Page Header section, the Group Header section, the Details section, the Group Footer section, the Report Footer section, and finally, the Page Footer section.

The Separated Values format cannot be used to export reports with cross-tabs or OLAP grids; nor can it be used to export reports with subreports in Page Header or Page Footer sections.

Note

Standard Mode of the *Separated Values* format respects conditional suppression but only for report content. The data is suppressed, but a row is printed in the report with the field delimiters. For example, if

your field delimiter is a comma and your report has four columns, you will see a blank row that contains four commas.

Tab Separated Text (TTX)

The Tab Separated Text format is similar to the Text format. This format preserves the layout of your report but with some differences. In the Text format, multi-line text objects are exported to multiple lines. In this format, multi-line text objects are exported in a single line; all string values are surrounded by double quotes (""); and the values themselves are separated by tab characters. TTX files can be opened in Microsoft Excel.

Text

Text format is a record-based format. Its output is plain text; therefore formatting is not retained. However, this format can preserve some of your report's layout. Text format assumes that a font of a constant dimension is used throughout the export. The Characters Per Inch (CPI) option specifies the number of characters that can be fit in a linear inch of horizontal space, and thus determines the dimension of the font.

This format also provides an option for pagination. If you select this option, a page break is inserted in the output after every specified number of lines. Page breaks in this format may not correspond to the pagination of your report.

XML

XML is used primarily for data exchange. It is a record-based format that uses the Crystal XML Schema. The XML Expert in Crystal Reports can be used to customize the XML output.

XML Legacy

XML is used primarily for data exchange. It is a record-based format that uses the Crystal XML Schema.

20.1.3.2 Destination

The destination determines the export location of your report. Crystal Report enables you to choose one of six destinations. They include:

- Application
- Disk file

- Exchange Folder
- Lotus Domino
- Lotus Domino Mail
- MAPI

20.1.3.3 Exporting to an application

If you export to an application, the program exports the report to a temporary file in the specified format and then opens the file in the appropriate application.

❗ Note

When exporting to an application, the output opens with the program that is associated with its file extension as specified on the File Types tab of your operating system's Folder Options.

❗ Note

When exporting in ODBC format, Application and Disk file are the same.

❗ Note

You will need to specify a file path if you are exporting in HTML format.

❗ Note

The file name of the report and the file name of the temporary file cannot be the same.

The sections below provide instructions on how to export a report to the Microsoft Excel (XLS) format for each of the different destination types.

20.1.3.3.1 To export to an application

Procedure

1. Open the report you would like to export.
2. On the **File** menu, point to **Export**, and then click **Export Report**.

→ Tip



Another way to do this is to click the Export button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the **Format** drop-down list.

In this case, choose Microsoft Excel (XLS).

4. Select *Application* from the *Destination* drop-down list.
5. Click *OK*.

The Excel Format Options dialog box appears.

Note

The options in the "Column width" area let you define the width of the Excel cells in your output based on a point width (a constant width) or a width determined by objects in various sections of your Crystal report.

Note

Selecting "Whole Report" does not necessarily emulate the layout of your Crystal report. Rather, the Excel cell width is based on objects found in any section of your report. Likewise, for example, selecting "Details" creates cell widths based on the objects found only in the report's Details section.

6. Change the formatting options as needed.
7. Click *OK*.

The Exporting Records dialog box appears.

Tip

Click Cancel Exporting to cancel the export process.

Results

The program exports the report and opens it in the appropriate application. In this example, Microsoft Excel opens with the exported data.

20.1.3.4 Exporting to a disk file

If you export to a disk file, the program saves the report to the disk or diskette you have specified.

20.1.3.4.1 To export to a disk file

Procedure

1. Open the report you would like to export.
2. On the *File* menu, point to *Export*, and then click *Export Report*.

Tip

Another way to do this is to click the  (*Export*) button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the *Format* drop-down list.

In this case, choose Microsoft Excel (XLS).

4. Select *Disk file* from the *Destination* drop-down list.
5. Click *OK*.

The Excel Format Options dialog box appears.

For more information about this dialog box, see step 5 in [Exporting to an application \[page 622\]](#).

6. Change the formatting options as needed.
7. Click *OK*.

The Select Export File dialog box appears.

8. Select the appropriate directory.
9. Enter the File name.
10. Click *Save*.

→ Tip

Click Cancel Exporting to cancel the export process.

Results

The program exports the report and saves it in the designated directory.

20.1.3.5 Exporting to a Microsoft Exchange folder

Crystal Reports enables you to export a report file to a Microsoft Exchange folder. You select the folder, and the report is stored there in the format that you specify. A Microsoft Exchange folder can contain standard notes (mail), files, and instances of Microsoft Exchange forms.

20.1.3.5.1 To export to a Microsoft Exchange folder

Procedure

1. Open the report you would like to export.
2. On the *File* menu, point to *Export*, and then click *Export Report*.

→ Tip

Another way to do this is to click the  (*Export*) button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the *Format* drop-down list.

In this case, choose Microsoft Excel (XLS).

4. Select *Exchange Folder* from the *Destination* drop-down list.
5. Click *OK*.

The Excel Format Options dialog box appears.

For more information about this dialog box, see step 5 in [Exporting to an application \[page 622\]](#).

6. Change the formatting options as needed.
7. Click *OK*.

The Choose Profile dialog box appears.

Note

You will be prompted by the Microsoft Outlook Setup Expert if Microsoft Exchange, Microsoft Mail, or Internet E-mail is not set up on your machine.

8. Select the desired profile from the *Profile Name* drop-down list.

If the profile is not listed, click New to create it.

9. Click *OK* when finished.
10. When the Select a Folder dialog box appears, select the folder in the profile in which you want the report to appear, and click *OK*.

Results

The report is exported to the Microsoft Exchange folder you selected. The exported report can now be accessed through the Microsoft Exchange client.

20.1.3.6 Exporting to MAPI (Microsoft Mail)

Context

Note

This option works only if you have a mail client installed (Microsoft Outlook, Microsoft Mail, or Exchange). Because the exported report is attached to an email message, you must also have an email account configured properly.

Procedure

1. Open the report you would like to export.
2. On the *File* menu, point to *Export*, and then click *Export Report*.

→ Tip

Another way to do this is to click the  (*Export*) button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the *Format* drop-down list.

In this case, choose Microsoft Excel (XLS).

4. Select *Microsoft Mail (MAPI)* from the *Destination* drop-down list.
5. Click *OK*.

The Excel Format Options dialog box appears.

For more information about this dialog box, see step 5 in [Exporting to an application \[page 622\]](#).

6. Change the formatting options as needed.
7. Click *OK*.

The Send Mail dialog box appears.

8. Enter the address details, then click *Send*.

The Exporting Records dialog box appears.

→ Tip

Click *Cancel Exporting* to cancel the export process.

20.1.3.7 Exporting to HTML

By exporting reports in HTML format, Crystal Reports provides you with a new option for rapid, convenient distribution of important company data. Once exported, your reports become accessible with many of the most popular Web browsers, including Netscape and Microsoft Internet Explorer.

20.1.3.7.1 To export to HTML

Procedure

1. With the report you want exported active, go to the *File* menu, point to *Export*, and then click *Export Report*.

→ Tip



Another way to do this is to click the Export button on the Standard toolbar.

The Export dialog box appears.

2. From the *Format* drop-down list, select HTML 3.2 or HTML 4.0.
3. Select a destination from the *Destination* drop-down list.

The rest of this section assumes you select Disk file to store the HTML document in a directory on a web server.

4. Click *OK*.

The Select Export Files dialog box appears. When exported to HTML format, a report is saved as separate HTML pages. If you prefer, clear the check box to have the entire report saved to one file. Because the report is saved as separate HTML pages by default, the program asks you for the name of a directory for an export destination, and uses default names for the HTML files. The initial HTML page will be saved as Base File Name.html. This is the file you open if you want to view your report through your Web browser.

Note

When you are exporting to a single file (that is, you have cleared the Separate HTML Pages check box) all blank space in headers and footers, as well as top and bottom page margins, is included. If you don't want to see blank spaces in your HTML file, suppress empty header and footer sections and set the top and bottom page margins to zero in the Page Setup dialog box.

5. Select an existing directory, or create a new directory for your report.
6. To navigate through separate HTML files, click the Page navigator option.
7. Click *OK*.

The program exports the report to HTML format.

20.1.3.8 Exporting to an ODBC data source

Crystal Reports allows you to export reports to any ODBC data source. If you have an ODBC data source set up for a database or data format, you can export your report to that data format through ODBC.

For instance, you may have an ODBC data source set up through ODBC Administrator that you normally use to access database tables designed in Microsoft SQL Server. Using the Export dialog box, however, you can select your SQL Server data source and export your report as a new SQL Server database table.

Note

You must have an ODBC data source set up through ODBC Administrator in order for the program to export to a particular ODBC database format. See How to set up an ODBC data source.

Exporting to an ODBC data source allows you to:

- Change data from a centralized database format into a format compatible with a local DBMS application.
- Change data from a local database format into a format compatible with a centralized database.
- Create a new database table that can be used as a separate data set in future reporting.
- Create a mini data-warehouse.
- Manipulate database data by filtering records, adding formulas, and removing fields to create a new database table that provides the data you need most for your work.

20.1.3.8.1 To export to an ODBC data source

Procedure

1. With the report you want exported active, go to the *File* menu, point to *Export*, and then click *Export Report*.

→ Tip



Another way to do this is to click the Export button on the Standard toolbar.

The Export dialog box appears.

2. From the *Format* drop-down list, select *ODBC*.

ⓘ Note

Destination is ignored when you are exporting a report to an ODBC data source. You do not need to make any changes to the Destination edit box.

3. Click *OK* in the Export dialog box.
4. If your ODBC data source specifies a particular database, the report will be exported to that database. Otherwise, the ODBC Formats dialog box appears. Select the database to which this report will be added as a new table, and then click *OK*.
5. If the ODBC data source you selected requires a Logon ID and password, the Login or SQL Server Login dialog box appears. Enter your ID and Password, and then click *OK*.

The Enter ODBC Table Name dialog box appears.

6. Enter the name you want to give to the new table in the database, and then click *OK*.

Results

The program exports the report as a new table to the database you specified.

ⓘ Note

If your report contains a binary field, a Cross-Tab, or an OLAP grid, you will be unable to export it to an ODBC data source successfully.

20.1.4 Working with Enterprise folders

Another way to distribute your reports is through SAP BusinessObjects Business Intelligence platform. When you publish a report to SAP BusinessObjects Business Intelligence platform, you can deliver it to end users via any web application—intranet, extranet, Internet or corporate portal.

Crystal Reports facilitates the publication of reports through the Enterprise option found in the Open and Save As dialog boxes and through the Workbench. For more information about publishing through the Workbench, see [The Workbench \[page 115\]](#).

When you choose the Enterprise option in the Open dialog box, you can select any published report to make changes to it. The same option in the Save As dialog box lets you save a modified report back to its original location, or it lets you save a new report to an Enterprise folder, which, in essence, is the same as publishing the report to SAP BusinessObjects Business Intelligence platform.

20.1.4.1 Opening a report in an Enterprise folder

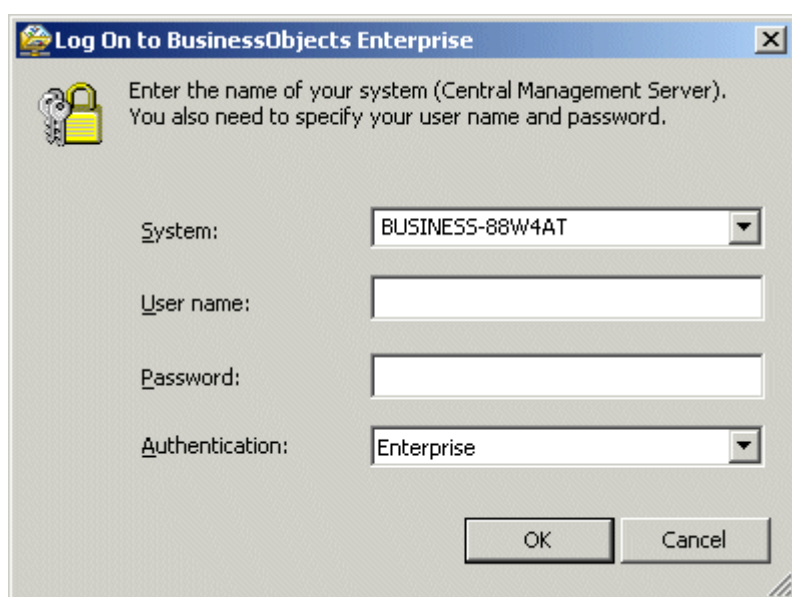
You can open reports from SAP BusinessObjects Business Intelligence platform folders in Crystal Reports. Crystal Reports displays the appropriate SAP BusinessObjects Business Intelligence platform folders in the Open dialog box.

20.1.4.1.1 To open a report in an Enterprise folder

Procedure

1.  On the *File* menu, click *Open*.
2. In the Open dialog box, click *Enterprise*.

If you have not already logged on to SAP BusinessObjects Business Intelligence platform, the [Log On to SAP BusinessObjects Enterprise](#) dialog box appears.



The dialog box titled "Log On to BusinessObjects Enterprise" contains a lock icon and the instruction: "Enter the name of your system (Central Management Server). You also need to specify your user name and password." It features four input fields: "System:" with a dropdown menu showing "BUSINESS-88W4AT", "User name:" with a text box, "Password:" with a text box, and "Authentication:" with a dropdown menu showing "Enterprise". At the bottom are "OK" and "Cancel" buttons.

3. In the [System](#) field, enter or select the name of the SAP BusinessObjects Business Intelligence platform system that you want to connect to.
4. Enter your user name and password.
5. Click the [Authentication](#) list to select the appropriate authentication type.

Enterprise authentication requires a user name and password that is recognized by SAP BusinessObjects Business Intelligence platform.

LDAP authentication requires a user name and password that is recognized by an LDAP directory server.

Windows Active Directory (AD) authentication requires a user name and password that is recognized by Windows AD.

Windows NT authentication requires a user name and password that is recognized by Windows NT.

6. Click [OK](#).

The Enterprise folders appear and you can select a report to open in Crystal Reports.

Results

Note

To find a specific report, highlight a folder and select Find from its shortcut menu. You can then use the Find dialog box to enter the report's name or a part of its name.

Note

To limit the risk of expanding a folder that contains many hundreds of reports, use the "batch" options on the folder's shortcut menu:

- First Batch displays the first 100 objects in the selected folder.
- Next Batch displays the next 100 objects in the selected folder.
- Previous Batch displays the previous 100 objects in the selected folder.
- Last Batch displays the last 100 objects in the selected folder.

Note

Select the "Update Repository Objects" option to ensure that the report's repository objects are updated when the report is opened in the future. For more information about the SAP BusinessObjects Enterprise Repository, see [SAP BusinessObjects Enterprise Repository \[page 120\]](#).

20.1.4.2 Saving a report to an Enterprise folder

You can save Crystal reports to SAP BusinessObjects Business Intelligence platform folders. Use the Save As dialog box to save a modified report back to its original location, or use it to save a new report to an SAP BusinessObjects Business Intelligence platform folder. (Saving a report to an SAP BusinessObjects Business

Intelligence platform folder is the same as publishing the report to SAP BusinessObjects Business Intelligence platform.)

Note

If your report uses a DataDirect DSN connection and you want to publish the report to a BI platform installation on Unix, verify that the BI platform administrator has added a DSN entry into the `odbc.ini` file. For more information about configuring BI platform to use DataDirect connections, see the *SAP BusinessObjects Business Intelligence Platform Installation Guide for Unix*.

20.1.4.2.1 To save a report to an Enterprise folder

Procedure

1. On the **File** menu, click **Save As**.
2. In the Save As dialog box, click **Enterprise**.
3. If you have not already logged onto SAP BusinessObjects Business Intelligence platform, do so now.

For information about how to log onto SAP BusinessObjects Business Intelligence platform, see [Opening a report in an Enterprise folder \[page 629\]](#).

4. When the Enterprise folders appear, select a folder to save your report in.
5. Enter a file name for your report.
6. Select **Enable Repository Refresh** if you want your report's repository objects to be updated when the report is opened again in Crystal Reports or scheduled in SAP BusinessObjects Business Intelligence platform.

For more information about the SAP BusinessObjects Enterprise Repository, see [SAP BusinessObjects Enterprise Repository \[page 120\]](#).

7. Click **Save** to publish your report to SAP BusinessObjects Business Intelligence platform.

20.2 Viewing reports

You can view Crystal reports using a number of report viewers available through the stand-alone Report Application Server (RAS) or SAP BusinessObjects Business Intelligence platform and the SAP BusinessObjects Business Intelligence platform Software Development Kit (SDK). For information about the Crystal Report Viewers, see the *Report Application Server Viewer Help* or the viewer online help in the SAP BusinessObjects Business Intelligence platform SDK documentation.

In general, the Crystal Report Viewers are page viewers that let you see complete pages of your Crystal reports. However, one viewer—the Report Part Viewer—lets you see specific report objects without viewing the entire page. Report objects displayed in such a way are referred to as Report Parts.

20.2.1 Report Parts and other Report objects

20.2.1.1 What are Report Parts?

Report objects displayed by themselves in a viewer—without the rest of the report page—are referred to as Report Parts. More precisely, however, Report Parts are objects that use hyperlinks to point from a home report object to a destination object.

Report Parts work with the DHTML viewer subset of the Crystal Report Viewers to expand the navigation possibilities within and between reports. Report Part hyperlinks can link to other objects in the current report or to objects in any other report. This linking lets you create a guided path through your reports that shows only specific information at each stop along the path.

Viewing Report Parts instead of the whole page is a powerful feature that allows you to seamlessly integrate reports into portal and wireless applications.

20.2.1.1.1 Report Part Viewer

The Report Part Viewer is a viewer that lets you display Report Parts without the rest of the report page. You can integrate this viewer into web applications so that your users see only specific report objects without having to see the rest of the report.

For the most part, you set up the Report Part hyperlinks in the Report Designer, but you take advantage of their functionality in the report viewers.

20.2.1.2 Navigating Report Parts and report objects

The navigation functionality in Crystal Reports lets you move to other report object(s) in the same report, or to object(s) in another report—with a specified data context. In this last case, the other report must be managed in SAP BusinessObjects Business Intelligence platform, or must be part of a stand-alone Report Application Server environment. This navigation is available only in the DHTML viewers (zero-client, server-side viewers). Its advantage is that you can link directly from one object to another; the viewer passes the required data context so you go to the object and data that is relevant.

Report Parts use this navigation functionality when linking between Report Part objects. The key difference between Report Part navigation and regular (page) navigation is that, when navigating with Report Parts (using the Report Part Viewer), you see only the objects identified as Report Parts. In regular navigation (using the page viewers or the Advanced DHTML Viewer), you go to the identified object(s), but you see the entire page.

20.2.1.2.1 Setting up navigation

Navigation (page or Report Part) is set up on the Hyperlink tab of the Format Editor dialog box.

In the DHTML Viewer Only area of the Hyperlink tab, you can select one of two options:

- [Report Part Drilldown](#)

The Report Part Drilldown option is available for summary fields, group charts and maps, and fields in your report's group header or group footer. This option is relevant only to Report Parts; regular navigation (using the page view) drills down by default. For more information, see [Report Part-specific navigation \[page 634\]](#).

- [Another Report Object](#)

You can use the Another Report Object option for both Report Part and page navigation. This option lets you specify the destination object(s) you want to navigate to and the data context to pass.

When you select the Report Part Drilldown option, the "Hyperlink information" area contains these options:

- [Available Fields](#)

The Available Fields area lists, as a tree view, all the sections in your report that contain report objects you can use for Report Part drill down. You select objects from this area and add them to the Fields to Display area.

- [Fields to Display](#)

The Fields to Display area lists, as a tree view, the section and objects that you have selected for drilling down on.

See [The Report Part Drilldown option \[page 634\]](#) for information about how to use these options.

When you select the Another Report Object option, the "Hyperlink information" area contains these options:

- [Paste the Report Part link](#)

Use the Past Link button to add detailed information about a report object that you previously selected and copied. The pasted information depends, in part, on the option you select from the list associated with the button.

- [Select From](#)

After you paste link information, the Select From field contains the path and file name of the report you selected and copied from (it is blank when the current report is referenced). You add information to this field by pasting a report part link; otherwise it is not accessible.

- [Report Title](#)

The Report Title field defaults to the title of the report specified in the Select From field (it contains the text <Current Report> when the current report is referenced). If a report title was not defined in the Document Properties dialog box, the field defaults to the report's file name. You add information to this field by pasting a report part link; otherwise it is not accessible.

- [Object Name](#)

By adding one or more objects to this field, you are identifying the destination objects for your navigation. In page navigation, this information determines what object you move to in the page. For Report Part navigation, this information determines what object(s) the viewer displays upon navigation (the viewer displays only the identified objects).

You can select any of the following report object types in the Object Name field (you can select one or more objects from the same report section):

- Field objects
- Charts or maps
- Bitmaps
- Cross-tabs
- Text objects

You cannot select the following report object types as destinations:

- Objects inside the Page Header or Page Footer.

- Lines or boxes.
- Subreports or any object inside a subreport.
- Entire sections (you must select the objects inside the section individually).

You can add specific information in one of two ways:

- You can type the object name(s) as they appear in the Report Explorer.
- You can copy report object(s) from the current report or another and paste its information into the object field.

- **Data Context**

The program normally completes the Data Context field when you paste information for the destination object(s). You can also customize this option by using the Conditional Formula button. For more information, see [Data context formats \[page 641\]](#).

See [The Another Report Object option \[page 637\]](#) for information about how to use these options.

Note

Report objects with hyperlinks defined are shown in the Report Explorer as blue text with a solid underline. Objects with Report Part hyperlinks are shown as green text with a dashed underline.

20.2.1.2.2 Report Part-specific navigation

Report Parts use the Crystal Reports navigation functionality. However, some special cases apply only to Report Parts:

- The viewer displays only the object identified as the destination object.
- You must specify Initial Report Part Settings for each report that is the first stop on the Report Part navigation path.

20.2.1.3 The Report Part Drilldown option

The Report Part Drilldown option lets you define a hyperlink so that the Report Part Viewer can emulate the drill-down functionality of Crystal Reports. The Report Part Viewer displays only destination objects; therefore, to make drill down work, you need to define a navigation path from a home object to one or more destination objects. When you have multiple destination objects, they must all reside in the same report section.

The Report Part Drilldown option does not affect the DHTML page viewers since the option emulates the default Crystal Reports behavior for drill down (which the page viewers already support). Page viewers, however, do not limit which objects are displayed—they always show all report objects.

Note

Use the Report Part Drilldown option only to navigate between objects in the same report.

Because the Report Part Viewer shows only destination objects, you must define Initial Report Part Settings (that is, a default home object) for a report before your Report Part Drilldown hyperlinks can work. A report's Initial Report Part Settings define the object that appears first in the Report Part Viewer. Think of this object as the place you'll begin your path of drill-down hyperlinks from.

20.2.1.3.1 To define Initial Report Part Settings

Procedure

1. Open the report whose default home object you want to define.
2. Right-click the object you want to set as the default home object and select [Copy](#) from its shortcut menu.
3. On the [File](#) menu, click [Report Options](#).
4. In the Initial Report Part Settings area of the Report Options dialog box, click [Paste Link](#).

The program pastes the name and data context of the report object you selected as your home object into the appropriate fields.

→ Tip

For more information about data contexts, see [Data context formats \[page 641\]](#).

Include another object from the same section of the report by entering a semi-colon (;) and typing the name of the object.

5. Click [OK](#).

20.2.1.3.2 To create a Report Part Drilldown hyperlink

Context

ⓘ Note

Before you begin this procedure, be sure to read [Setting up navigation \[page 632\]](#) to acquaint yourself with the limitations of creating this type of hyperlink.

Procedure

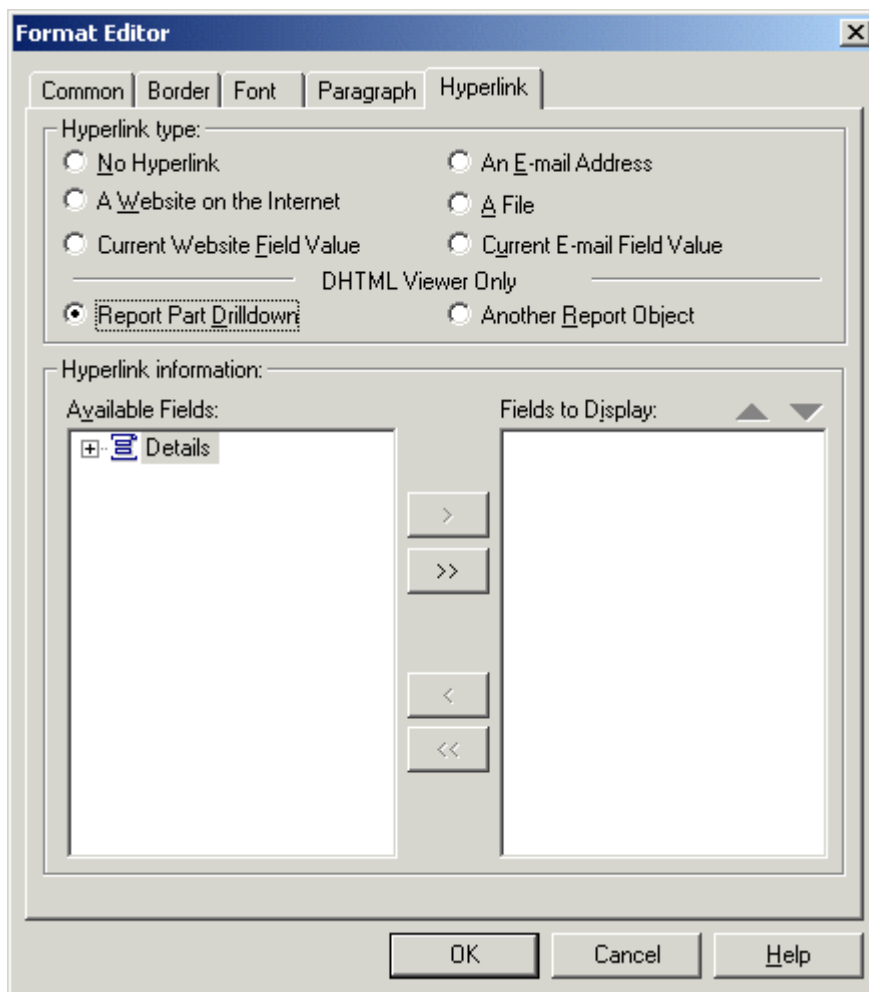
1.  Open a report and select the intended destination object; then click the Format button on the Expert Tools toolbar.

→ Tip

You can also do this by selecting Format Field from the Format menu.

2. In the Format Editor, click the [Hyperlink](#) tab.
3. In the DHTML Viewer Only area, select [Report Part Drilldown](#).

The "Hyperlink information" area changes to show the fields available for this type of hyperlink.



The Available Fields area shows only the sections and report objects you can select for drill down. In general, these objects include field objects, charts, maps, bitmaps, cross-tabs, and text objects contained in the next section down (that is, for example, objects in group two when the object you selected is in group one).

Note

The Available Fields area does not show suppressed report objects.

4. In the Available Fields area, select the section or report object(s) you want to use as your drill down destination.

Tip



Use the Report Explorer to quickly identify the default names assigned to each of your report objects. To open the Report Explorer, click its button on the Standard toolbar.

You can add all the objects in a section by selecting the section node. If you expand the section node, you can select one or more of the objects that it contains.

5. Use the arrow buttons to move the selected section or object(s) to the Fields to Display area.

For ease of identification, the program creates a section node in the Fields to Display area. This node contains the objects that you selected in the Available Fields area.

6. If you want to add another object to the Fields to Display area, select it in the Available Fields list and drag it into position.

Note

The position an object has in the Fields to Display area determines how it appears in the Report Part Viewer; top to bottom in the Fields to Display area equates to left to right in the viewer. Use the arrows above the Fields to Display area to change the order of objects. (Note that a programmer can alter this default behavior through the PreserveLayout property of the Report Part Viewer.)

Note

The Fields to Display area can contain only one section at a time. If you try to add a second section or an object from a second section, it replaces any existing section in the Fields to Display area.

7. Click [OK](#).

Results

You have established a hyperlink from your report's home object to a destination object or objects. In the Report Part Viewer, you will see the home object first, and when you click it for drill down, you will see the destination object(s).

20.2.1.4 The Another Report Object option

The Another Report Object option lets you define a hyperlink path for linking objects in the same or different reports. If you are defining a hyperlink path to a different report, that report must be managed in SAP BusinessObjects Business Intelligence platform, or it must be part of a stand-alone Report Application Server environment. The Report Part Viewer displays only the objects you specify. Like the Report Part Drilldown option, all destination objects must be from the same report section.

Note

When using a stand-alone Report Application Server environment, it is recommended that all reports navigated to and from using the Another Report Object option reside in the default Report Application Server report directory. Placing reports in this directory ensures that the linked reports can be easily migrated to an SAP BusinessObjects Business Intelligence platform environment.

Note

The DHTML page viewer also uses the Another Report Object option for navigation purposes.

Note

You cannot use the Another Report Object option for an entire cross-tab or OLAP grid object; you can, however, use it for any individual cell in a cross-tab or OLAP grid.

Work flow

Because the Another Report Object option allows you to create hyperlinks between objects in different reports managed in SAP BusinessObjects Business Intelligence platform, it requires more set up on the Hyperlink tab. This is an overview of the steps you have to complete to set up your hyperlink successfully.

- Open the report that contains the object you want to be the destination object and copy it.
- Open the report that contains the home object, select it, and open the Format Editor.
- On the Hyperlink tab, paste the information from the destination object into the appropriate fields.

The following procedure shows you how to do all of these steps.

20.2.1.4.1 To create an Another Report Object hyperlink

Context

Note

Before you begin this procedure, be sure to read [Setting up navigation \[page 632\]](#) to acquaint yourself with the limitations of creating this type of hyperlink.

Procedure

1. Open your source and target reports.

Your source report is the report you copy an object from; the target report is the report you add the hyperlink information to.

When you view your target report in the Report Part Viewer, you first see its home object. When you drill down on that object, you go to the destination object you selected in the source report.

2. In your source report, right-click the intended destination object and select [Copy](#) from its shortcut menu.

3.  In your target report, select the intended home object; then click the [Format](#) button on the [Expert Tools](#) toolbar.

→ Tip

You can also do this by selecting Format Field from the Format menu.

4. In the Format Editor, click the [Hyperlink](#) tab, and then select the [Another Report Object](#) option.
5. In the "Hyperlink information" area, click [Paste Link](#).

Note

The Paste Link button is unavailable if you have not copied a destination object as shown in step 2.

The program pastes the identifying information from the destination object you selected in the source report into the appropriate fields.

Format Editor

Common | Border | Font | Paragraph | **Hyperlink**

Hyperlink type:

☐ No Hyperlink ☐ An E-mail Address

☐ A Website on the Internet ☐ A File

☐ Current Website Field Value ☐ Current E-mail Field Value

DHTML Viewer Only

☐ Report Part Drilldown ☒ Another Report Object

Hyperlink information:

Paste the Report Part link: Paste

Select From:

C:\Program Files\Business Objects\Crystal Reports 11\Samples\En'

Report Title:

Regional Summary Q2

Object Name:

Sales1

Data Context:

/ChildIndex[3]

OK Cancel Help

The Paste Link button includes a list of options that you can use when pasting a link to the destination object. Depending on the type of link you are creating (for example, a link to a specific data context rather than a link to a generic data context), you may see these options:

- **Context Report Part**
This is the default option; it is selected when you click Paste Link without viewing the other list options. Use this option to paste a link that includes the most complete data context that the program can determine. For more information, see [Context Report Part option scenarios \[page 640\]](#).
- **Design View Report Part**
Use this option to paste a link that includes a generic data context (that is, a data context that does not point to a specific record). Use this option when you want a broad data context, or when data is not yet present in your source report.
- **Preview Report Part**
Use this option to paste a link that includes a specific data context (that is, a data context that points to a specific record). Use this option when you want a narrow data context, or when data is present in your source report.

Note

If you choose the Design View Report Part option (or if you choose the Context Report Part option for an object copied from a report's Design view) and your source and target reports have different data structures, you'll see the Report Part Map Fields dialog box. Each field in the Unmapped Fields area represents a group in your source report. Use this dialog box to map these groups to fields in your target report. When you've mapped all the groups, the OK button is available, and you can proceed.

6. If you want to add other fields from the same section of the source report, click the [Object Name](#) field and move the cursor to the end of the existing text. Enter a semi-colon (;) and then the name of the field(s).

Note

The order of objects in the Object Name field determines how they appear in the Report Part Viewer; left to right in the Object Name field equates to left to right in the viewer.

7. You can broaden your [Data Context](#) by using an asterisk (*) to represent all records in a group.

For example, if your Data Context is /USA/CA/Changing Gears (which will show only that specific detail record), you could change it to /USA/CA/* to show all detail records within that group.

→ Tip

You might find it more useful to copy your hyperlink information from the Design view because the data context will be more general (that is, you won't be copying only a specific record as you might on the Preview view).

→ Tip

If your report has a group selection formula, check the data context for your destination object to ensure it contains the correct child index.

→ Tip

For more information, see [Data context formats \[page 641\]](#).

8. Click [OK](#).

Results

You have established a hyperlink from your home object to a destination object or objects. In the Report Part Viewer, you see the home object first, and when you click it for drill down, you see the destination object(s).

20.2.1.5 Context Report Part option scenarios

When pasting linking information for a Report Part, the Context Report Part option mainly affects the data context of the link; the Select From field (the report URI) and the Object Name field are always the same for any of the choices available in the Paste Link list. This section provides various scenarios and describes how the program creates a data context for each.

Scenario 1

If your source report contains data (that is, if the report is in Preview view), the Context Report Part data context (in the target report) is the source report's Preview data context for all children of the selected field. For example, you might see this in the Data Context field: `/Country[USA]/Region[*]`. Otherwise, the program uses the source report's Design view data context as the Context Report Part data context in the target report.

Scenario 2

If your source and target reports are the same report, and the object you choose as your source is from the Preview view, when the target object you choose to link to is in a higher level than the source object, the data context is left empty. For example, when the source data context is `/Country[Australia]/ChildIndex[5]`, and the target data context is `/Country[Australia]`, the Context Report Part data context is left empty.

Scenario 3

If your source and target reports are the same report, and the object you choose as your source is from the Preview view, when the target object you choose to link to is in a lower level than the source object, the data context pasted is a Design view data context (in the form of a formula). For example, when the source data context is `/Country[Australia]`, and the target data context is `/Country[Australia]/ChildIndex[5]`, the Context Report Part data context is `"/"+{Customer.Country}`.

Cross-tabs

If the object you copy in your source report is part of a cross-tab, the Context Report Part data context is based on the selected object embedded in the cross-tab. Crystal Reports creates the data context by tracking if the selected object in the cross-tab is a row, column, or cell. Next, the program retrieves row and column names. Then it parses the Design view data context. Finally, it generates the Context Report Part data context by adding the `GridRowColumnValue` function if the group in the data context formula appears in the row or column names retrieved earlier. You might see, for example, a data context such as:

```
"/"+GridRowColumnValue("Customer.Country")
+"/"+GridRowColumnValue("Customer.Region")
```

20.2.1.6 Data context formats

- Normally, a data context formula appears as follows:

```
"/" + {Table.Field} + "/" + {Table.Field}
```

- You can also use these formats:
 - XPath-like format:
`/USA/Bicycle`
 - Strongly-Typed format:
`/Country[USA]/Product Class[Bicycle]`

ⓘ Note

You can add detail-level information in O-based format:

`/USA/Bicycle/ChildIndex[4]`

ⓘ Note

You can also use a wildcard to identify all instances:

`/USA/*`

- For cross-tab objects, you can use navigation on cells, columns, or rows. To define the data context, use the `GridRowColumnValue` formatting function. For example:

`"/" + GridRowColumnValue ("Supplier.Country") + "/" + GridRowColumnValue ("Product.Product Class").`

20.2.1.7 Hyperlinks displayed in the viewers

This section summarizes information about the hyperlink options and how they work in the page viewers and the Report Part Viewer.

Differences between the hyperlink options

Another Report Object	Report Part Drilldown
This option allows the selection of one or more report objects.	This option allows the selection of one or more report objects.
Destination objects must be from the same report section.	Destination objects must be from the same report section.
The location of the section that contains the destination objects has no restriction: - The section can be located anywhere in the report. - The section can be in a group level above or below the source object.	The location of the section that contains the destination objects must be in the immediately next, lower-group level from the source object.

Another Report Object	Report Part Drilldown
The object can be in a different report that is managed in SAP BusinessObjects Business Intelligence platform or is part of a stand-alone Report Application Server (RAS).	The object must be in the same report.

Differences between how the viewers display hyperlinks

Page viewers	Report Part Viewer
Another Report Object: • Navigates to destination objects and shows contents of the entire page.	Another Report Object: • Navigates to destination objects and shows only the destination objects.
Report Part Drilldown: • Navigates (drills down) to the group and shows the complete contents of the group.	Report Part Drilldown: • Navigates to destination objects and shows only the destination objects.
Default drill down (no hyperlink): • Group drill down from a group header or footer, chart, or map displays the group view. • Subreport drill down displays the subreport view.	Default drill down (no hyperlink): • Not available.

20.2.2 Creating an Enterprise Hyperlink

You can create hyperlinks from a report to enterprise documents such as Crystal Reports, Interactive Analysis and other documents.

Note

In order for this feature to work, SAP BusinessObjects Business Intelligence platform needs to be running.

20.2.2.1 To create an Enterprise Hyperlink

Procedure

1. Select a report object on the *Design* or *Preview* tab.
2. On the *Format* menu, click *Hyperlink*.

The *Format Editor* dialog box appears with the Hyperlink tab active.

3. On the *Hyperlink type* section, select the option *A Website on the Internet*.
4. Click *Create enterprise hyperlink*.
5. Click *Browse*.
The *Open* dialog box appears.
6. Click *My Connections* to add a new connection to the Enterprise.
If you already have a connection, click *Enterprise*.
7. Navigate to the enterprise document that you wish to open and click *Open*.
The *Document ID* and *Document Name* fields are filled out with the CUID and name of the selected document respectively.
8. On the *Hyperlink Settings* section change the different options to create the hyperlink with the specified settings.
9. On the *Parameter Settings* section select the parameter values, if the selected document has parameters.
10. Click *OK*.

20.2.3 Using Smart View to view a report in the CMC

Context

The *Smart View* setting in the Central Management Console (CMC) allows an administrator to change BI launch pad report viewing behavior from viewing the latest instance of a report to refreshing report content from the data source.

Procedure

1. In the Central Management Console, click *Applications*.
2. On the *Applications* page, double-click *Crystal Reports Configuration*.
The *Properties: Crystal Reports Configuration* dialog box appears.
3. Under *Properties*, select *BI launch pad*.
4. Under *Smart View*, perform one of the following actions:
 - To view the last successful instance when you double-click a publication in BI launch pad, select *View Latest Instance*. If no successful instance exists, the publication is refreshed against its data source. This is the default view.
 - To always refresh a publication against its data source (regardless of whether an instance exists) when you double-click the publication in BI launch pad, select *View Object*.
5. Click *Save & Close*.

20.3 Using smart tags

Crystal Reports lets you take advantage of smart tags in Office XP. When you paste a chart, a text object, or a field object into an Office XP application, you can view data from the host report after selecting a smart tag option. This table summarizes the options available to you.

Office XP application	Smart tag option	Crystal Reports object type
Word	View	Text object
	Refresh	Field object
		Chart
Excel	View	Text object
	Refresh	Field object
Outlook	View	Text object
	Refresh (only when creating a message)	Field object
		Chart

Before you can use smart tags, web server options must be configured on the Smart Tag tab of the Options dialog box. As well, an .asp or .jsp page must be created for viewing report details. Normally, these tasks should be carried out by your system administrator.

Note

The reports you want to use with smart tags must exist on the web server named in the Options dialog box in a directory that mirrors their real location, or the web server must be configured to accept a UNC path.

20.3.1 To use smart tags with a Crystal Reports object

Procedure

1. Open the Crystal report that contains the object you want to copy to an Office XP application.
2. On the **File** menu, click **Options** and ensure that the options in the Smart Tag Web Server Options area on the **Smart Tag & HTML Preview** tab have been configured:
 - Your web server must be named.
 - A virtual directory must be named (a default is provided).
 - A viewing page must be named (a default is provided).
3. On the **Preview** tab of Crystal Reports, right-click the text object, field object, or chart you want to copy and select **Copy Smart Tag** from its shortcut menu.
4. Open the appropriate Office XP application, and paste the report object into a document, worksheet, or email message.

Note

See the table in this section for limitations about which report objects can be pasted into each Office XP application.

5. In your Office XP application, choose the appropriate smart tag from the options for the pasted report object.

Note

See the table in this section for limitations about which smart tags are available in each Office XP application.

The .asp or .jsp page named in the Options dialog box appears showing the appropriate information from the host Crystal report.

21 Publishing

This section discusses publishing and the role of the report designer in the publishing process. It includes information on how to create a dynamic recipient distribution list and how to create delivery rules.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

21.1 Publishing overview

Publishing means making reports and documents publically available. The content of these reports can be distributed automatically by email or FTP. Content can also be saved to disk, or managed through the SAP BusinessObjects Enterprise platform for web viewing, archiving, retrieving, and scheduling.

The roles of the report designer (the person who creates the report) and the publisher (the person who schedules and distributes the reports in SAP BusinessObjects Enterprise) are often filled by the same person. However, this is not always the case. This section provides the report designer with important information about his or her role in the publishing process.

For more information about publishing, see the *SAP BusinessObjects Enterprise User's Guide*.

Related Information

[Dynamic recipient sources \(DRSs\) \[page 648\]](#)

[Delivery rules \[page 653\]](#)

21.2 Role of the report designer in publishing

Other than designing and exporting publication documents to SAP BusinessObjects Enterprise, report designers are responsible for preparing the following aspects of the publishing process:

- Creating dynamic recipient sources (DRSs). DRSs allow the publisher to distribute publications to recipients who do not have SAP BusinessObjects Enterprise user accounts.

- Adding Report Alerts to enable delivery rules. Delivery rules allow the publisher to specify criteria for the conditional delivery of publications.

21.3 Dynamic recipient sources (DRSs)

If your company wants to distribute a publication to recipients who are not users of SAP BusinessObjects Enterprise, but whose information you have stored in an external data source (such as a database, LDAP directory, or custom application), you can create a dynamic recipient source (DRS).

A DRS allows the publisher to distribute a publication to recipients without SAP BusinessObjects Enterprise user accounts. It obtains the list of recipients and their profile values directly from the external data source, thereby eliminating the need to create user accounts for the intended recipients.

For Crystal Reports, a DRS is a report, or a rowset, that lists data about recipients. The mapping between the columns of data in the rowset and their meaning in the publication (for example, recipient name, e-mail address, profile values) is managed by the publisher in SAP BusinessObjects Enterprise.

Note

There are several different types of DRSs. For instance, a DRS can be based on Universe queries defined in an SAP BusinessObjects BI interactive analysis report. For more information about using the different types of DRSs, see the *SAP BusinessObjects Enterprise Publisher's Guide*.

To provide an example, suppose that your company distributes a monthly regional brochure to customers, but several of your new customers have not yet been entered in the SAP BusinessObjects Enterprise system. You create a report that returns information for the new customers from your database. The report includes the following fields:

- CustomerID
- CustomerName
- EmailAddress
- Region

After you save this new report to SAP BusinessObjects Enterprise, the publisher specifies it as the DRS for the brochure publication. The publisher maps the CustomerName and EmailAddress fields from your new report to the user name and email address fields for the publication. The publisher then personalizes the brochure based on the Region field. When the report is published, the new customers receive the brochure that is appropriate for their region.

As report designer, your role is to create a report that lists data about the intended recipients. The publisher then uses this report to create a distribution list and personalize the publication.

For more information about publishing and dynamic recipients, see the *SAP BusinessObjects Enterprise Publisher's Guide*.

Related Information

[Design considerations \[page 649\]](#)

[Personalization with DRSs \[page 650\]](#)

[Active Directory recipients \[page 652\]](#)

21.3.1 Design considerations

When designing a DRS, you must create a column to be mapped to recipient ID. You can also include optional columns for email address and full name.

A DRS can return several different data types. If the publication is personalized, ensure that the data type of a column is compatible with the corresponding field in the personalized report. DRSs support the following data types:

- String
- Number
- DateTime
- Boolean

Additionally, DRSs must provide the following:

- Column Headings
The publishing system requests this information from a DRS to return a list of column names for the field mapping operation.
- Paging
A DRS returns recipient records to the publishing system in batches. The page size of the DRS controls how many rows it returns to the publishing system at one time. At publishing time, the publisher is able to preview and filter recipients from the data list, so you do not need to filter for intended recipients at design time.

Note

The maximum number of rows that the publisher can preview at publishing time is 100, regardless of the batch size of the DRS.

Multiple values for a single recipient

The publishing system supports a single recipient having multiple values for a profile (for example, a manager managing multiple regions, or a statement containing information for multiple accounts). In this case, a single recipient is associated with multiple rows in the rowset, each containing the same value in the recipient ID column. This is illustrated in the example below.

Recipient ID	Region	Role
RCPT123	California	Manager

Recipient ID	Region	Role
RCPT123	Florida	Manager

All rows with the same recipient ID value are processed as one recipient. If you plan to distribute a publication with multiple values for a recipient, it is best to sort the DRS by recipient ID, so that rowsets are grouped by recipient. Failure to follow this sorting guideline may result in the generation of separate publication instances and the delivery of documents with partial information.

If multiple rows in the rowset have the same recipient ID but different full names or email addresses, the publishing system treats this as a single recipient and results in one delivery. For that recipient, the system uses the first full name and email address to appear in the report.

Live data and saved data

In addition to choosing a report as the data source for recipient information, the publisher is also able to specify whether data is refreshed from the database live, or whether saved data from the latest instance is used.

If the DRS contains parameters, the default parameter value set in the Central Management Console (CMC) will be used. The publisher is not able to change parameter values at publishing time.

Sending to both SAP BusinessObjects Enterprise users and dynamic recipients

You can distribute a single publication to both dynamic recipients and SAP BusinessObjects Enterprise users/groups at the same time. However, a publication can only have one DRS.

If a dynamic recipient is also a SAP BusinessObjects Enterprise user, even one with the same recipient ID and email address, the system does not attempt to associate the dynamic recipient with the SAP BusinessObjects Enterprise user account.

21.3.1.1 Personalization with DRSs

For personalized publications, you can create columns that contain the values used for personalization. On a per document basis, for each document field that will be filtered on to achieve personalization, the publisher can specify which column from the dynamic recipient provides the data value for that field.

Each report can have zero to multiple fields to personalize on, and it is possible that multiple report fields are personalized by the same column from the DRS.

Note

The columns that contain user ID, email address, and full name may also be used for personalization.

Personalizing with multiple columns

When a report is personalized with multiple columns, the combination of the values across the columns is used, as illustrated in the example below.

Rowset:

Recipient ID	Region	Role
RCPT123	California	Manager
RCPT123	Florida	Team Lead
RCPT123	Florida	Manager

Column mapping:

- Report field : Region <--> Region column
- Report field : Role <--> Role column

The following is the resulting filter to personalize the document for recipient RCPT123:

- (State = 'California' and Role = 'Manager') or (State = 'Florida' and Role = 'Team Lead') or (State = 'Florida' and Role = 'Manager')

Personalizing multiple reports

When a publication contains multiple reports, column mapping for personalization is specified individually for each report. Across the reports, they may link to the same or different columns, as illustrated in the example below.

Rowset:

Recipient ID	Region	Role	CostCenter
RCPT123	California	Manager	9876
RCPT123	Florida	Team Lead	9876
RCPT123	Florida	Manager	9876

Column mapping:

- Report A: Report field : Dept <--> CostCenter column
- Report B: Report field : Title <--> Role column
- Report C: Report field : Region <--> Region column and Role <--> Role column

The following are the resulting filters for each report:

- Report A: Dept = 9876

- Report B: Title = 'Manager' or Title = 'Team Lead'
- Report C: (Region = 'California' and Role = 'Manager') or (Region = 'Florida' and Role = 'Manager') or (Region = 'Florida' and Role = 'Team Lead')

21.3.2 Creating a dynamic recipient source

If you want to distribute a publication to recipients who are not users of SAP BusinessObjects Enterprise, but whose information exists in an external data source (such as a database, LDAP directory, or custom application), you can create a dynamic recipient source.

Related Information

[Design considerations \[page 649\]](#)

[Personalization with DRSs \[page 650\]](#)

21.3.2.1 To create a dynamic recipient source

Procedure

1. Create a report that accesses the desired data source.
2. In the report, insert fields for Recipient ID, e-mail address, and full name.
3. Save the report, and export it to SAP BusinessObjects Enterprise.

Results

At publishing time, the publisher uses the dynamic recipient source to specify a list of recipients and their profile values.

21.3.3 Active Directory recipients

Windows users must create a special type of DRS for Active Directory recipients.

Unix users can use the ODBC Driver for eDirectory to perform SQL calls to Novell eDirectory. For more information about the ODBC Driver for eDirectory, see http://developer.novell.com/wiki/index.php/ODBC_Driver_for_eDirectory .

21.3.3.1 To create a DRS for Windows Active Directory recipients

Procedure

1. Create a new report with the *Standard Report Creation Wizard*.
2. In the *Available Data Sources* tree, expand the Create New Connection folder and double-click the *OLE DB (ADO)* folder.
The *OLE DB (ADO)* dialog box appears.
3. From the Provider list, select *OLE DB Provider for Microsoft Directory Services*.
4. In the *Data Source* field, type the IP address of your data source.

Note

Contact your server administrator to obtain this information.

5. Select the *Integrated Security* check box, and click *Finish*.
In the *Available Data Sources* tree, *Active Directory Service Interfaces* appears in the *OLE DB (ADO)* folder.
6. Under *Active Directory Service Interfaces*, double-click *Add Command*.
The *Add Command To Report* dialog box appears.
7. In the *Add Command To Report* dialog box, write an SQL query that obtains the desired information about your intended recipients, and click *OK*.
The new command appears in the *Selected Tables* tree.
8. In the *Standard Report Creation Wizard*, click *Finish*.

Results

The DRS report displays the desired information about your intended recipients. After creating this report, you need to run the Server Intelligence Agent under an account with access to your corporate network.

21.4 Delivery rules

If you use SAP BusinessObjects Enterprise to publish reports, you can use Report Alerts to create delivery rules. A delivery rule allows the publisher to specify criteria for the conditional delivery of a publication. The publisher specifies these rules in the CMC or BI launch pad, but they are based on Report Alerts added by the report designer during report creation.

Be aware of these considerations when you use delivery rules:

- Because delivery rules are based on Report Alerts, the criteria for conditional delivery are always evaluated in the context of the report content.
- The same delivery condition is evaluated for each recipient on the recipient list.

- For publications scheduled to be printed, the print-out is produced regardless of the delivery rule result. This happens because printing occurs at the report processing phase, which is before the evaluation of the delivery rule.

As report designer, your role is to create the Report Alert upon which a delivery rule is based. At publishing time, the publisher uses the Report Alert to design the delivery rule in SAP BusinessObjects Enterprise.

For more information about publishing and delivery rules, see the *SAP BusinessObjects Enterprise User's Guide*.

Related Information

[Report Alerts \[page 657\]](#)

[Recipient delivery rules \[page 654\]](#)

[Global delivery rules \[page 655\]](#)

21.4.1 Recipient delivery rules

In cases where each recipient receives a personalized publication, the publisher can create a recipient delivery rule. Recipient delivery rules are processed after personalization so that delivery to each recipient is conditionally based on relevant information.

The following is an example of when you would use a recipient delivery rule.

Your company distributes monthly account summaries to your customers. During the creation of the summary report, you add a Report Alert to notify you whether there has been any account activity during the month. At publishing time, the publisher personalizes the report and creates a delivery rule that sends the report to the customer only if the Report Alert indicates that the customer had account activity during the month.

21.4.1.1 To create a Report Alert for a recipient delivery rule

Procedure

1. Determine the publication documents to be distributed.
2. Determine the condition that distribution is based on.
3. In the report that contains the conditional data, create a Report Alert to evaluate the data.

Note

If the conditional data is not in a currently existing report, you must create a report that contains it.

Note

The conditional data does not need to be from a report included in the publication. The publisher can create a delivery rule based on any document stored in SAP BusinessObjects Enterprise.

Results

At publishing time, the publisher personalizes the publication and creates a recipient delivery rule based on your Report Alert. For more information about creating delivery rules in SAP BusinessObjects Enterprise, see the *SAP BusinessObjects Enterprise User's Guide*.

21.4.2 Global delivery rules

Publications, especially ones with many recipients, are computing intensive to process. For unpersonalized publications that depend on a single condition, the publisher can create a global delivery rule. Global delivery rules are checked before the publication begins to process, thereby eliminating unnecessary processing when the condition fails to distribute.

The following is an example of when you would use a global delivery rule.

Your company distributes a quarterly bonus statement to employees, but the bonus is only given if the company meets its quarterly revenue targets. You add a Report Alert in your revenue statement to notify you whether the quarterly revenue targets have been met. At publishing time, the publisher creates a global delivery rule that sends the report to employees only if the targets have been met.

21.4.2.1 To create a Report Alert for a global delivery rule

Procedure

1. Determine the publication documents to be distributed.
2. Determine the condition that distribution is based on.
3. In the report that contains the conditional data, create a Report Alert to evaluate the data.

Note

If the conditional data is not in a currently existing report, you must create a report that contains it.

Note

The conditional data does not need to be from a report included in the publication. The publisher can create a delivery rule based on any document stored in BOE.

Results

At publishing time, the publisher creates a global delivery rule based on the Report Alert. The global rule is evaluated before the publication is processed.

22 Report Alerts

This section provides information about creating and using alerts in your Crystal reports.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

22.1 About Report Alerts

Report Alerts are custom messages created in Crystal Reports that appear when certain conditions are met by data in a report. Report Alerts may indicate action to be taken by the user or information about report data.

Report Alerts are created from formulas that evaluate conditions you specify. If the condition is true, the alert is triggered and its message is displayed. Messages can be text strings or formulas that combine text and report fields.

Once a Report Alert is triggered, it's not evaluated again until you refresh your report's data.

Because Report Alerts are specific to each report, you decide when to use them and when not to. They can be useful to point out important information, such as sales that fall above or below a limit. And the message is created by you, so it can be specific to your data.

22.2 Working with Report Alerts

This section focuses on the tasks you'll need to undertake in order to use Report Alerts.

22.2.1 Creating Report Alerts

You must complete three steps when creating a Report Alert:

- Name the alert.
- Define the condition that triggers the alert.

- Create the message you want to appear when the alert is triggered (this step is optional).

22.2.1.1 To create a Report Alert

Procedure

1. On the Report menu, point to [Alerts](#) and then click [Create or Modify Alerts](#).

The Create Alerts dialog box appears.

2. Click [New](#).

The Create Alert dialog box appears.

3. Enter a name for your new alert in the [Name](#) box.
4. Enter your alert message in the [Message](#) box.

The Message box lets you enter a message to be used as a default. If you want the same message to appear every time your alert is triggered, enter it in the Message box.

If, however, you want to use a formula so the message is customized with data elements, see the next step.

5.  If you want to use a formula to create an alert message, click the formula button to the right of [Message](#).

The Formula Workshop appears. For information on how to use the editor, see [Working with the Formula Editor \[page 697\]](#).

6. Enter your alert message formula.

For example, if you want to see the message "Country is a star performer" (where Country is the name of a specific country), you might create the following formula:

```
{Customer.Country} + " is a star performer"
```

Note

The result of an alert message formula must be a string.

Note

The DefaultAttribute function can be used to refer to a message added in the Message box. For example, if the message in your Message box is " is a star performer," your alert message formula might be:

```
{Customer.Country} + DefaultAttribute
```

This formula relies on the text you enter in the Message box; that text becomes the DefaultAttribute.

Note

Message condition formulas can be created using either Crystal Syntax or Basic Syntax.

7. Click [Save and close](#) to return to the Create Alert dialog box.
8. In the Create Alert dialog box, click [Condition](#).

The Formula Workshop appears.

9. Enter your alert condition formula.

Alert formulas can be based on recurring records or on summary fields, but cannot be based on print-time fields, such as running totals or print time formulas. Alert formulas cannot have shared variables.

If an alert formula is based on a summary field, any recurring fields used must be constant over the summary field. For example, if you are grouping on Country and Region, you might create an alert such as:

```
Sum ({Customer.Last Year's Sales}, {Customer.Region}) > 10000
```

In this case, your formula can refer to either Country or Region, but not City or Customer Name since these are not constant.

Note

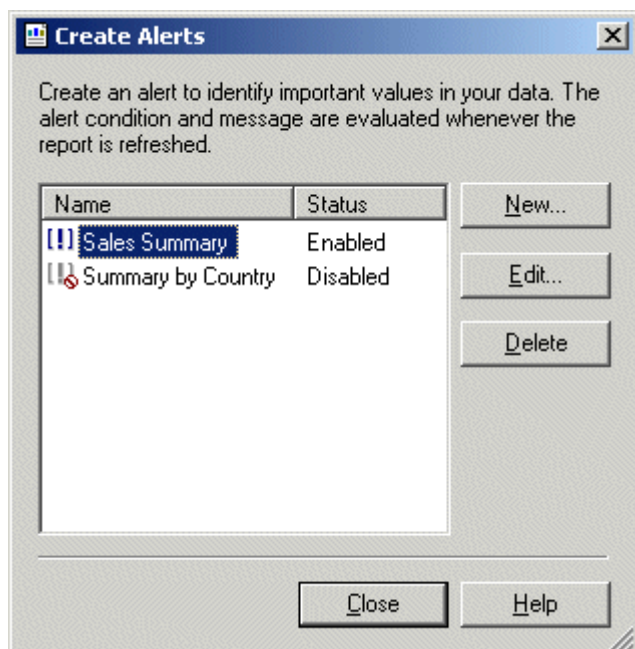
Alert condition formulas can be created using either Crystal Syntax or Basic Syntax.

10. Clear the [Enable](#) check box if you do not want the alert to be evaluated.

Otherwise, leave it selected.

11. Click [OK](#) to save your alert.

You are returned to the Create Alerts dialog box and your new alert is listed. You can see its name and status (Enabled or Disabled).



Only enabled and disabled alerts appear in the Create Alerts dialog box. If an alert is triggered, it is seen in the Report Alerts dialog box.

22.2.2 Editing Report Alerts

Procedure

1. On the Report menu, point to [Alerts](#) and then click [Create or Modify Alerts](#).
2. In the Create Alerts dialog box, select the alert you want to edit and click [Edit](#).

→ Tip

Double-clicking an alert also lets you edit it.

3. Make the changes you want in the Edit Alert dialog box.
4. Click [OK](#) to save your changes.

Results

ⓘ Note

If the alert has already been triggered, editing it removes it from the Report Alerts dialog box.

22.2.3 Deleting Report Alerts

Procedure

1. On the Report menu, point to [Alerts](#) and then click [Create or Modify Alerts](#).
2. In the Create Alerts dialog box, select the alert you want to delete and click [Delete](#).

The selected alert is removed from the Create Alerts dialog box.

Results

ⓘ Note

If the alert has already been triggered, deleting it removes it from the Report Alerts dialog box as well.

22.2.4 Viewing Report Alerts

You can view triggered Report Alerts by:

- Refreshing your report's data.
- Selecting Triggered Alerts from the Alerts submenu of the Report menu.

Note

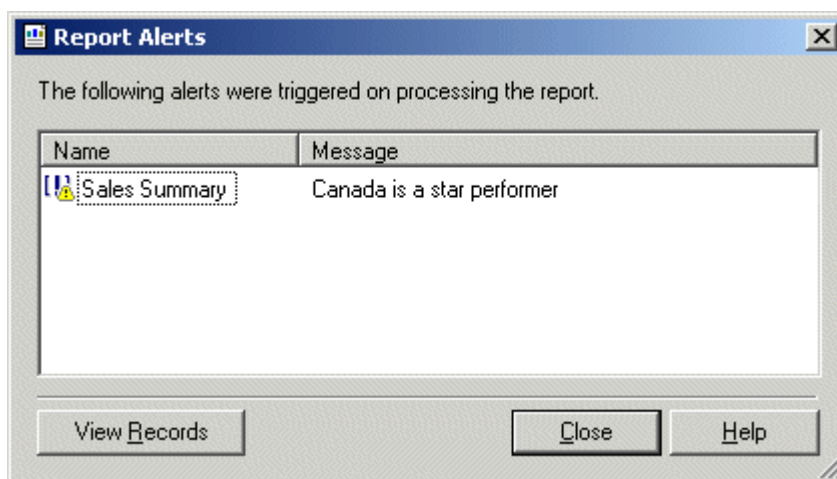
If you want to view alerts when report data is refreshed, you must select "Display Alerts on Refresh" on the Reporting tab of the Options dialog box (this option is also available on the Report Options dialog box).

22.2.4.1 To view Report Alerts

Procedure

1. On the Report menu, point to [Alerts](#) and then click [Triggered Alerts](#).

The Report Alerts dialog box appears.



2. Select the alert whose records you want to see.
3. Click [View Records](#).

A new report tab is opened showing the report record(s) that triggered the alert. If the record is hidden, the record's group is shown but drill down is not performed.

Note

If you select more than one triggered alert before clicking the View Records button, the results are generated by performing a Boolean AND operation on the selected alerts.

4. To return to the Report Alerts dialog box, click the [Preview](#) tab.
5. Click [Close](#) to remove the Report Alerts dialog box.

22.2.5 Referring to Report Alerts in formulas

Alerts can be referred to in formulas. Any formula that references an alert becomes a print time formula.

The following functions are available:

- IsAlertEnabled()
- IsAlertTriggered()
- AlertMessage()

These functions behave the same as alerts created in the Create Alerts dialog box:

- IsAlertTriggered("AlertName") is true only for the records on which the alert is triggered
- AlertMessage("AlertName") displays the message for a record when the alert is true.

Because alerts are not field objects (you cannot drop them into the report), alerts are represented differently from report fields in the Formula Workshop. In the Functions tree, you'll see an Alerts heading where the functions themselves appear. Available alerts are listed by name below that.

23 XML

This section describes the XML data access and export capabilities of Crystal Reports. It begins with a brief overview of XML technology and then provides procedures for setting up XML data access and using the XML exporting features.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

23.1 XML overview

XML (Extensible Markup Language) is a format for exchanging data. The format is not owned by any one organization, but is accepted by a host of companies as a standard for data interchange. XML facilitates moving structured data across the web by being independent of system or platform restrictions.

XML is the transmission format of B2B (Business to Business) and B2C (Business to Consumer), because it enables information exchange and makes it easy to publish report information on numerous platforms and devices.

Unlike HTML, which stores a limited amount of formatting with the content, XML content is stored without formatting—formatting information can be saved to a style sheet. XML content is structured to make it accessible to a wide range of users, applications, and platforms.

There are many advantages to using XML:

- XML files are reliable to use and easy to maintain because they contain only content.
- Multiple output formats can be supported simultaneously. From a single XML document, you can generate HTML for a browser, a PDF for printing, and so on.
- You can convert XML to new formats without affecting a file's content.
- Because XML uses style sheets to store formatting information, the look and feel of a web site can be changed easily.
- XML is easy to validate, so you're less likely to have incorrect information or corrupt files on your web site.
- XML makes searching through large numbers of files easier and more accurate due to its highly structured design.
- XML files can store content in multiple languages. An XML translator displays content for specific language preferences.

There are some drawbacks to consider as well:

- XML files can be very large and don't naturally support binary data, such as images.
- Existing content must be reworked to take advantage of XML's benefits and to reliably support the wide range of new devices.
- XML standards are still developing, so there's risk in selecting a particular methodology.
- Currently, there aren't many graphical content management tools available.

23.1.1 XML elements, tags, and structure

XML content is made up of elements that describe data. Each element usually contains two tags: one to open the element and one to close it (empty elements can be opened and closed in the same tag). You define the tags to suit your company's needs. For example, you might create a tag `<LastYearsSales>` to describe last year's sales totals. Any other tag is equally appropriate as long as it describes the data to your satisfaction and is used consistently throughout the XML file. XML tags are case sensitive and can contain additional information called attributes.

XML facilitates emulation of the hierarchical structure of documents by allowing elements to contain other elements. For example, you might show the structure of report data by grouping elements as follows:

```
<FormattedReportObject>
  <ObjectName>Field2</ObjectName>
  <FormattedValue>Australia</FormattedValue>
  <Value>Australia</Value>
</FormattedReportObject>
```

In this example, the `<FormattedReportObject>` element contains three other elements that describe the data within. This grouping of elements becomes, in essence, the structure of the XML content. You can add as many layers of grouping as needed to create an appropriate structure.

23.1.2 XML Schema

The XML Schema is a specification from the World Wide Web Consortium (W3C) that provides a way of defining the structure, content, and interpretation of XML documents. The XML Schema defines the data types that can be associated with XML elements and attributes. It also describes the structure of the contents of XML documents, and defines rules for governing schema validation. Based on these features, the XML Schema is able to enforce data integrity and compatibility among XML documents.

Because XML schemas are XML documents themselves, they can be managed by XML authoring tools.

23.1.3 Document Type Definition

The Document Type Definition (DTD), like the XML Schema, is a set of rules used to describe elements and attributes in an XML document. As a means of validation, the DTD originated in SGML. Because the DTD uses a syntax different from XML and has limitations in functionality, the XML Schema has begun to replace it.

A DTD can be included in an XML file, or can be a separate file referenced by an XML file.

23.1.4 Style sheets

Style sheets provide formatting information for the presentation and layout of XML content. Like the DTD, style sheets can be included in an XML file, or can be a separate file referenced by an XML file.

XSL (Extensible Stylesheet Language) is the preferred type of style sheet for XML. XSL provides both a method for formatting XML documents and a method for transforming XML documents into formats recognizable to a browser.

For example, you might use XSL to transform a report's XML output so it can be read as Wireless Markup Language (WML) for use with a hand-held device.

23.2 XML in Crystal Reports

In Crystal Reports, XML is both a data source and an export format.

You can create a Crystal report from an XML data source, thus turning raw XML data into report information.

You can also export a Crystal report to XML format using the new XML exporter in Crystal Reports. When exporting a report, you can use the Crystal Reports XML schema built in to Crystal Reports, or create new XML-based export formats by embedding XSLT transforms into your report.

23.3 Accessing XML data

Crystal Reports provides XML data access through a native XML driver.

23.3.1 Native XML driver

The native XML driver for Crystal products reads an XML stream and parses its contents based on the corresponding schema (XSD) to derive a hierarchical table representation of the data. Crystal Reports then uses this hierarchical table representation when you create a report.

Note

When Crystal Reports consumes the XML itself, the hierarchical table representation is mapped directly back to the XML based on an internal query definition. The XML is never deconstructed into a relational table model.

You must specify a schema or Document Type Definition (DTD) for the XML driver to use while parsing the XML stream; the driver uses this schema or DTD to determine how XML data types will be converted to Crystal data types. For more information about schemas and how data types are converted, see [Schema \[page 669\]](#).

For more information about the native XML driver, visit the SAP Community Network at:

<http://scn.sap.com> 

You'll find samples and other information at this site. For a general overview of XML, see [XML in Crystal Reports \[page 665\]](#).

23.3.1.1 XML data source access types

When you create a new connection in Crystal Reports by selecting the XML driver, you can choose from several possible data source types. This section describes each type.

Local data source

A local data source is an XML file on a local machine or a network drive that you can connect to by entering a path and file name. The XML driver lets you type a full path and file name, or you can select a file by searching for it in the Open dialog box. If you have a schema (XSD) for your XML file, you can specify it. For more information about schemas and how they are used, see [Schema \[page 669\]](#).

HTTP(S) data source

An HTTP(S) data source is an XML file or XML stream that you can access from a Servlet, an ASP page, a JSP page, or another type of dynamically created web page. The URL you provide to such a data source can be either HTTP or HTTPS. When using HTTPS, user IDs and passwords that you enter in Crystal Reports or Crystal Enterprise are used to authenticate against the underlying HTTP provider.

The XML driver lets you type a URL and later add authentication credentials. If you have a schema (XSD) for your XML file or stream, you can specify the schema and its authentication credentials. For more information about schemas and how they are used, see [Schema \[page 669\]](#).

HTTP(S) data sources also support HTTP parameters. For more information, see [HTTP parameters \[page 673\]](#).

Web Service data source

A Web Service data source can be a service on a local network or the Internet that returns an XML data set. Crystal Reports supports Web Services that communicate using SOAP. The URL you provide to such a source can be either HTTP or HTTPS. The schema for the response XML file is derived through the Web Services framework rather than specified manually.

Web Service data source interfaces are described using WSDL (Web Services Description Language), which is the standard format for describing a Web Service. After you specify a path to the WSDL, the driver lets you select which Service, Port, and Method you want to use. You may also have to enter authentication user ID and password.

Web Services can have parameters to control access and content. Crystal Reports supports Web Service parameters.

Web Services may be secured using the WS-Security Web Service standard. The XML driver supports this standard by allowing a security credentials policy file to be used to specify the security information required to access the Web Service. The underlying implementation of the XML driver makes use of the Apache Axis 2 Web Service components, which is helpful to know when implementing Web Service security and specifically when supplying security credentials in the WS-Security policy file.

23.3.1.1.1 To choose XML as your data source

Procedure

1. In Crystal Reports, on the [File](#) menu, point to [New](#), and then click [Standard Report](#).
2. On the Report Creation Wizard's Data screen, double-click [Create New Connection](#).
3. Double-click [XML](#).

The XML dialog box appears.

4. Choose an XML data source access type:
 - [Local data source](#)
 - [HTTP\(S\) data source](#)
 - [Web Service data source](#)
5. Identify the XML file you want to connect to by following the steps in one of these sections:
 - [To connect to a local data source \[page 667\]](#).
 - [To connect to an HTTP\(S\) data source \[page 668\]](#).
 - [To connect to a Web Service data source \[page 669\]](#).

Related Information

[XML data source access types \[page 666\]](#)

23.3.1.1.2 To connect to a local data source

Procedure

1. If you choose [Use Local Data Source](#), enter the full path and name of your XML file, or use the button adjacent to the field to search for your file.
2. Choose [Specify Schema File](#) if you want to apply a schema to the XML file you selected.

You must select this option and specify a schema unless the XML file contains embedded schema information.

Note

You define the location and name of your schema on the next screen.

3. Click [Next](#).

Depending on your selection in the previous step, you will see either a screen for specifying your schema, or you can click the Finish button.

4. To specify a schema file, choose either [Use Local Schema](#) or [Use HTTP\(S\) Schema](#) and complete the adjoining field.
 - For Use Local Schema, enter the full path and name of your schema file (XSD), or use the button adjacent to the field to search for it.
 - For Use HTTP(S) Schema, enter the URL that points to your schema file (XSD).
5. Click [Next](#), and then click [Finish](#).

23.3.1.1.3 To connect to an HTTP(S) data source

Procedure

1. If you choose [Use HTTP\(S\) Data Source](#), enter the URL that points to your XML file.

You can use HTTP or HTTPS to define your URL.

2. Choose [Specify Schema File](#) if you want to apply a schema to the XML file you selected.

You must select this option and specify a schema unless the XML file contains embedded schema information.

Note

You define the location and name of your schema on the next screen.

3. Choose [Validate XML](#) if you want the XML document validated against the schema.

Validation can only occur if both the XML instance and the schema are both specified.
4. Click [Next](#).
5. Enter authentication information if it is needed, and then click [Next](#) again.

Depending on your selection in step 2, you will see either a screen for specifying your schema, or a screen for defining HTTP parameters. If you see the HTTP Parameters screen, skip to step 6.

6. To specify a schema file, choose either [Use Local Schema](#) or [Use HTTP\(S\) Schema](#) and complete the adjoining field.
 - For Use Local Schema, enter the full path and name of your schema file (XSD), or use the button adjacent to the field to search for it.
 - For Use HTTP(S) Schema, enter the URL that points to your schema file (XSD).
7. On the HTTP Parameters screen, create parameters that can be added to your URL to limit the results that are returned in your XML file.
 - Click Add Property to define a new value. The Edit Property Value dialog box appears. Use this dialog box to define a parameter name and a formula. For more information about defining parameters, see [HTTP parameters \[page 673\]](#).

- Click **Edit Value** to change an existing parameter. Select the parameter you want to edit before you click this button.
 - Click **Remove Property** to delete an existing parameter. Select the parameter you want to remove before you click this button.
8. Click *Finish* when you are done with the HTTP Parameters screen.

23.3.1.1.4 To connect to a Web Service data source

Procedure

1. If you choose *Use Web Service Data Source*, click *Next* to continue.
2. To specify a web services file location, choose either *Use Local WSDL* or *Use HTTP(S) WSDL* and complete the adjoining field.
 - For *Use Local WSDL*, enter the full path and name of your web services file, or use the button adjacent to the field to search for it.
 - For *Use HTTP(S) WSDL*, enter the URL that points to your web services file.
 - If you are connecting to a Web Service secured using WS-Security, specify the policy file in this step as well.
3. Click *Next*.

Depending on your selection in the previous step, you will see either a screen for specifying your web service, ports, and methods, or a screen for defining authentication for your web service URL. If you see the web service, ports, and methods screen, skip to step 5.
4. Enter authentication information if it is needed, and then click *Next*.
5. On the Web Service, Port and Method screen, choose a service, a port, and a method from the lists.
6. Click *Finish*.

Results

Once you can see the table created from your XML data source, you can use it to create a Crystal report in the same way you'd use any other data source. For more information about creating Crystal reports, see the *Crystal Reports User Guide*.

23.3.1.2 Schema

When you use the XML driver, you can specify a schema (XSD) or Document Type Definition (DTD) of your choice, or you can use schema information embedded in the XML document. The schema is used to determine how to present the XML data within Crystal Reports.

Note

When you specify a schema to use with your XML file, the specified schema is used instead of schema information embedded in or referenced by the XML file itself.

The following table shows how the native XML driver maps XML data types to Crystal data types.

XML type	Crystal type
string	String
Boolean	Boolean
Decimal	Number
float	Number
Double	Number
duration	String (Duration: <duration>)
dateTime	DateTime
time	time
date	Date
gYearMonth	Date (First day of month)
gYear	Date (First day of year)
gMonthDay	Date (Within current year)
gDay	Date (Within current month/year)
gMonth	Date (Within current year—first day)
hexBinary	BLOB
base64Binary	BLOB
anyURI	String
QName	String
NOTATION	String (Notation: <notation>)
normalizedString	String
token	String
language	String

XML type	Crystal type
NMTOKEN	String
NMTOKENS	String
Name	String
NCName	String
ID	String
IDREF	String
IDREFS	String
ENTITY	String
ENTITIES	String
Integer	Number
nonPositiveInteger	Number
negativeInteger	Number
long	Number
int	Number
short	Number
byte	Number
nonNegativeInteger	Number
unsignedLong	Number
unsignedInt	Number
unsignedShort	Number
unsignedByte	Number
positiveInteger	Number
anySimpleType	String

Note

Names are case sensitive.

The XML driver deals with XML attributes, namespaces, specific data types, and tags as described in the following table.

XML	Resolved by the driver
Attributes	Attributes are included in the column name prefixed by the @ symbol (for example, tag <code></code> becomes field <code>A@attrib</code>).
Namespaces	Namespaces are included in the column name (for example, tag <code><abc:name></code> becomes field <code>abc:name</code>).
Complex data types	Complex type elements resolve into tables within Crystal Reports, whether or not they have repeating elements. An exception is where a complex type contains only a single simple type element; in this case, the element is returned as a long string containing a <CR>-separated list of the values of the elements.
List data types	List data types always resolve as String types. The string will contain the list.
Union data types	Union data types return a String containing the contents of the Union.
Mixed data types	Data types that are defined as Mixed are returned as an XML fragment that contains the entire contents of the tag (including the element's own tag) as a MEMO field.
Types defined with <code>processContents="skip"</code> attribute	Types that are defined with the attribute <code>processContents="skip"</code> are returned as an XML fragment that contains the entire contents of the tag (including the element's own tag) as a MEMO field.
Types defined with <code>processContents="lax"</code> attribute	Types that are defined with the attribute <code>processContents="lax"</code> are resolved through the schema; if resolution is not possible, the data is returned as an XML fragment that contains the entire contents of the tag (including the element's own tag) as a MEMO field.
Choice tags	Where Choice tags are used, the union of all combinations is offered along with an enumerated choice field that indicates the path chosen.
Tags containing no actual value	When a tag is a simple type and contains no actual value, a NULL value is returned. NULL is accepted for all data types.

Where it is not possible to determine the outermost (root) element of the schema from the XSD or DTD, all possible roots are displayed and you must select the root as part of your data connection.

23.3.1.3 HTTP parameters

When you select an HTTP(S) data source, you can also define HTTP parameters to pass to the underlying servlet or dynamically created web page. These parameters provide the servlet or web page with information for returning the correct XML stream. Parameters are passed to the underlying connection as Form variables. The Form variable name corresponds to the Crystal parameter name (for example, the Crystal {?REGION} parameter equals the Form variable 'REGION').

If parameters are passed with multiple values selected, they submit multiple Form variables with the same name. Consider the following partial example:

```
<p>What flavor has the largest sales?
<input type=checkbox name=flavor value=vanilla>Vanilla<br>
<input type=checkbox name=flavor value=strawberry>Strawberry<br>
<input type=checkbox name=flavor value=chocolate checked>Chocolate<br>
```

If you select both Chocolate and Vanilla, two form variables named flavor are submitted, each with a value of Vanilla and Chocolate, as in "http://server/servlet?flavor=Vanilla&flavor=Chocolate" in GET format. Similarly, {?flavor} with both Vanilla and Chocolate selected through Crystal Reports will create the same URL request.

23.3.1.3.1 To create HTTP parameters in the XML driver

Procedure

1. On the HTTP Parameters screen, click [Add Property](#).
2. On the Edit Property Value screen, enter a name for your parameter in the [Property](#) field.
3. Enter a parameter formula in the form of a @Prompt function in the [Value](#) field.

See [Parameter formulas \[page 673\]](#) for information about the syntax of the @Prompt function.

23.3.1.4 Parameter formulas

Parameter formulas are created by using the @Prompt function. This function is useful when you want to force a restriction in the XML stream but you do not want to preset the value of the condition.

The syntax of the function is as follows:

```
@Prompt( 'message', 'type', [lov], [MONO|MULTI], [FREE|CONSTRAINED])
```

Syntax	Description
'message'	This parameter is the text of the prompting message. The text must be enclosed between single quotation marks (for example, 'Choose a Region', 'Pick a time period', or 'Choose a showroom'). The text appears in the prompt box when the connection is completed.
'type'	This parameter describes the data type returned by the function. It can be one of the following: 'A' for alphanumeric 'N' for number 'D' for date The specified data type must be enclosed in single quotation marks.
lov	This parameter provides a list of values (LOV). The list is hard-coded; that is, you create it by entering the separate values. Each value is enclosed in single quotation marks and is separated by a comma. The whole list is enclosed in brace brackets. For example, {'Australia', 'France', 'Japan', 'United Kingdom', 'USA'}. This parameter is optional.
MONO	Users can select only one value from the list of values. MONO is the default of the MONO MULTI pair. This parameter is optional.
MULTI	Users can select multiple values from the list of values. MONO is the default of the MONO MULTI pair. This parameter is optional.
FREE	Users can enter a value of their choice, or they can select a value from the list of values. FREE is the default of the FREE CONSTRAINED pair. This parameter is optional.
CONSTRAINED	Users must select a value from the list of values. FREE is the default of the FREE CONSTRAINED pair. This parameter is optional.

Note

For each of the optional parameters, if you omit an argument, you must still enter the commas as separators.

23.4 Exporting to XML format

You can create XML data from your Crystal reports using Crystal Reports XML export. With this data, you can apply a style sheet or import it into another application. Because you can import your own schemas, you can specify any XML, HTML or other file format you need.

If you do not have a specific XML schema you want to use to format your report, or if you want to perform transformations after exporting the report, you can use the built in Crystal Reports XML schema, then transform the exported file.

You can customize the structure of your XML data when exporting a report. For information on how to do this, see [Customizing XML report definitions \[page 677\]](#).

23.4.1 To export a report to XML

Procedure

1. Open the report you want to export.
2. On the *File* menu, click *Export*.

→ Tip



Another way to do this is to click the Export button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the *Format* list. In this case, choose *XML* to choose the format to export the report as.
4. Choose the XML format and click *OK*.

The Choose export file dialog box appears if you are exporting to a file. If you are exporting to an application, the appropriate type of application for the format's file type is shown.

Results

The report is exported to the directory you selected. The exported report can now be accessed as raw XML data.

23.4.2 To export a report to XML (Legacy)

Procedure

1. Open the report you want to export.
2. On the *File* menu, click *Export*.

→ Tip



Another way to do this is to click the Export button on the Standard toolbar.

The Export dialog box appears.

3. Select the export format type from the *Format* list. In this case, choose *XML (Legacy)* to export the report in Crystal Reports Legacy Markup Language XML format.

The Export To Directory dialog box or the Choose export file dialog box appears.

4. Enter the directory and file name you want to store the XML file(s) in.

If you specified an external schema or DTD on the Options tab of the XML Format dialog box, both the output XML file and its XSD or DTD use the base file name.

ⓘ Note

The Validation Type box indicates the validation choice you made on the options tab of the XML Format dialog box. You cannot change the validation type on the Export to Directory dialog box.

5. Click *OK* when finished.

Results

The report is exported to the directory you selected. The exported report can now be accessed as raw XML data.

23.5 Managing XML exporting formats

Crystal Reports contains a built-in schema for exporting reports in XML format, Crystal Reports XML, a format to produce exported XML that is easy to transform into another format using XSLT.

Using the XML Exporting Format Manager, you can set this schema to be the default schema used when exporting reports in the XML format .

There may be times when you want to create a different file format (XML with a different schema or a different text-based file type) from a report. There are industry standard schemas, such as XBRL or HL7 you may need to apply to Crystal Reports report data. In this case you can use the XML Export Format Manager to import your XSLT into the report, and optionally set it as the default XML format when formatting the report during XML export.

23.5.1 To Import an XML Transform

Context

If the schema provided with Crystal Reports does not meet your needs for the exported XML format, you can import an XML transform to produce XML in a different schema or create a different file type altogether. In this case you can use the XML Export Format Manager to import your XSLT into the report, and optionally set it as the default schema for formatting the report during XML export. When exporting to custom XML formats, Crystal Reports first generates a Crystal Reports XML format file then applies your transform to it before providing the final results.

Procedure

1. On the [File](#) menu, point to [Export](#), then click [Manage XML Exporting Formats](#).
The Manage XML Exporting Formats dialog box opens.
2. From the Manage XML Exporting Formats dialog box, click [Add](#).
The Add a New XML Format dialog box opens.
3. Type a name for your format. This is the name that appears in the Manage XML Exporting Formats dialog box when you choose an XML format for exporting a report.
4. Click [Import XML Transform](#) to open the Open dialog box.
5. Browse through the folders to find a valid transformation (XSLT) and select it.
6. Indicate the extension for the exported report in the Specify the file type text area.

Note

This only affects the extension of the file name. Changing the extension does not affect the file format.

7. Optionally enter a description for the new format. This description appears in the Manage XML Exporting Formats dialog box when you choose an XML format for exporting a report.

Results

Once the schema is imported click [Save XML Transform As](#) to save a copy of the transform to another location.

23.6 Customizing XML report definitions

Although Crystal Reports provides a built in XML report definition for each report, and you can import any valid XSL stylesheet, you can also create custom report formats using the XML Expert Crystal Reports ML schema designer. The nature of the changes you can make to these default definitions depends on :

[XML\(Legacy\)](#)

The legacy XML format is the Crystal ML Schema. With it, you can create new attributes for existing XML elements, but you cannot make other kinds of changes.

The advantage of using this format is that you don't have to create your own schema or DTD. The Crystal ML Schema is Crystal Reports' standard schema and is published at the following address:

<http://www.businessobjects.com/products/xml/schema.xsd>

Custom Format

You can use the XML Expert to change element names, create attributes, and suppress XML tags. XML output created with this format is not controlled by the Crystal ML Schema; a custom DTD or schema is created on the fly.

The advantage of using this format is that you're free to create your own XML format. You can rename XML elements to suit an existing DTD or schema, and you can suppress XML tags to change the hierarchy of your output—which, in turn, helps minimize data output.

You are able to select between these output format options as you like. If, however, you change from one option to the other after you have made modifications to element names and/or attributes, you will lose these changes—element names and attributes will be reset to the default values.

Note

The element tags created for reports using the Crystal ML Schema and the Custom Format are not the same. For more information about the differences, see [Legacy XML output specifications \[page 683\]](#).

23.6.1 Why create your own XML format?

Although the Crystal ML Schema provides a default format for exporting to XML, and there are two built-in formats as well as functionality to import your own XML schema for formatting an exported report, you might want to create your own XML format for these reasons:

- You need to incorporate data from a Crystal report into an existing XML document that has its own schema/DTD and tags, but do not want to (or cannot) import an XML schema into the report. In this case, you would customize the Crystal report's XML data with tags that are consistent with your existing XML document.
- You have an application that needs to read XML data from Crystal Reports, but the application can identify only a specific set of tags, and you do not want to (or cannot) import an XML schema into the report. In this case, you would customize the Crystal report's XML tags to match the tags that the application can identify.
- You already use an industry standard XML schema/DTD that is required by your company, and you do not want to (or cannot) import an XML schema into the report. In this case, you would customize your Crystal Reports XML data so that it conforms to the standard you are required to use.

23.6.2 Customizing XML tags

Use the customization options in the XML Format dialog box when the legacy report definition created by Crystal Reports doesn't provide the XML information you need, and when you don't want to create an entirely

new XML schema to export your report in. Customization can range from defining a few new XML element attributes to renaming elements and suppressing tags for fields you don't want to export data for.

It is up to you to decide when modifying the existing legacy XML format is more desirable than creating a new schema to control the export format.

When you choose Crystal ML Schema, you can carry out these tasks in the XML Format dialog box:

- View the report definition tree.
- View XML element names and attributes.
- Add, edit, and delete XML tag attributes.

When you choose Custom Format, you can carry out these additional tasks:

- Rename XML tags.
- Suppress XML tags.

📘 Note

You cannot add new report objects (tree nodes) while using the XML Format dialog box. If you want to create a new node, add a report object to your Crystal report, and it will be added to the Report Definition Tree.

23.6.2.1 To customize XML tags

Procedure

1. Open a report in Crystal Reports.
2. On the *File* menu, click *Report Options*.
The Report Options dialog appears.
3. On the Report Options dialog, click *XML Expert*.

The XML Expert dialog box appears with the report's definition tree displayed.

→ Tip

The XML Output Format Used box displays the format your report definition is using. Click the Options tab to select a different option.

4. Expand the Report Definition Tree to find the report object whose XML tag you want to customize.

📘 Note

Parent nodes can represent areas or sections of the report. For example, each report group has its own parent node.

📘 Note

First-level child nodes can represent area pairs, such as a Report Header and a Report Footer.

23.6.2.2 To create a new XML element

Procedure

1. In the Report Definition Tree, select a report object that shares an XML element with another report object.
The New button is activated.
2. Click [New](#) and enter a new XML element name.
3. Click elsewhere in the Report Definition Tree to see the new element name.

23.6.2.3 Changing XML tag names

Context

This option applies only to report definitions that use Custom Format.

Procedure

1. Select a report object in the Report Definition Tree.
2. In the [XML Element Name](#) box, type your changes to the XML element name.
3. Click elsewhere in the Report Definition Tree to see the new element name.

Results

When you export the report in XML format, the new tag name you assigned will be used.

Note

Two or more report objects in the Report Definition Tree can be associated with the same XML element. Use the New button (next to the XML Element Name list) to create a new XML element for the selected report object.

23.6.2.4 Adding XML attributes

Context

This option applies to report definitions that use Crystal ML Schema or Custom Format.

Procedure

1. Select a report object in the Report Definition Tree.
2. Click [Create](#).

The XML Attribute dialog box appears.

3. In the [Attribute Name](#) box, enter a name for the attribute you are creating.

Note

Each attribute in an element must have a unique attribute name. Attribute names, however, don't have to be unique across elements.

4. In the Attribute Type list, click the appropriate option for the attribute you're creating:

- String
- Decimal
- Boolean
- Date
- Time
- Date Time

5. In the Instance Value area, select [Field](#) or [Text](#).

Field lets you select a report field of the specified type as your instance value. The Field option is available only if the corresponding report object is not Report (root node) or an Area Pair type.

Text lets you enter text in the associated box as an instance value for an element.

6. In the Default Value box, enter a value if you want to assure the attribute is not left empty when there is no instance value.
7. Click [OK](#).

The OK button is not available until an attribute name and instance or default value is entered.

Results

You are returned to the XML Expert dialog box. Your new attribute appears in the Attributes list.

23.6.2.5 Editing XML attributes

Context

This option applies to report definitions that use Crystal ML Schema or Custom Format.

Note

You can edit only those attributes you added using the XML Attribute dialog box.

Procedure

1. Select an element in the Report Definition Tree that has attributes.
2. Select the attribute you want to edit in the XML Tag Attribute List and click [Edit](#).

→ Tip

Attributes prefaced with the letters "ab" in blue are attributes you can edit. Those prefaced with "ab" in gray cannot be edited.

3. Make your changes in the XML Attribute dialog box.
4. Click [OK](#) to save changes.

23.6.2.6 Deleting XML attributes

Context

This option applies to report definitions that use Crystal ML Schema or Custom Format.

① Note

You can delete only those attributes you added using the XML Attribute dialog box.

Procedure

1. Select an element in the Report Definition Tree that has attributes.
2. Select the attribute you want to delete in the XML Tag Attribute List and click [Delete](#).

→ Tip

Attributes prefaced with the letters "ab" in blue are attributes you can delete. Those prefaced with "ab" in gray cannot be deleted.

Results

If you want to delete all attributes added using the XML Attribute dialog box for the selected element, click Delete All.

23.6.2.7 Suppressing XML tags

Context

This option applies only to report definitions that use Custom Format.

Procedure

1. Select a report object in the Report Definition Tree.
2. Select the *Suppress XML Tag* check box.

Results

The selected report object won't be exported, and the associated XML element is not available for editing.

Note

You can suppress all children of the selected report object by selecting the Suppress All Children check box.

To reverse this action, clear the Suppress XML Tag check box and/or the Suppress All Children check box.

23.7 Legacy XML output specifications

This section describes the output you can expect when exporting a Crystal report in the Legacy XML format. As stated in the [Exporting to XML format \[page 675\]](#) section, Crystal Reports creates raw XML data you can manipulate as you need. This data corresponds to the information in your Crystal report and includes elements for sections, areas, field objects, text objects, subreports, cross-tabs, and so on.

Note

Sections hidden or suppressed in a Crystal report are not exported as XML output even though they are displayed in the Report Definition Tree of the XML Format dialog box.

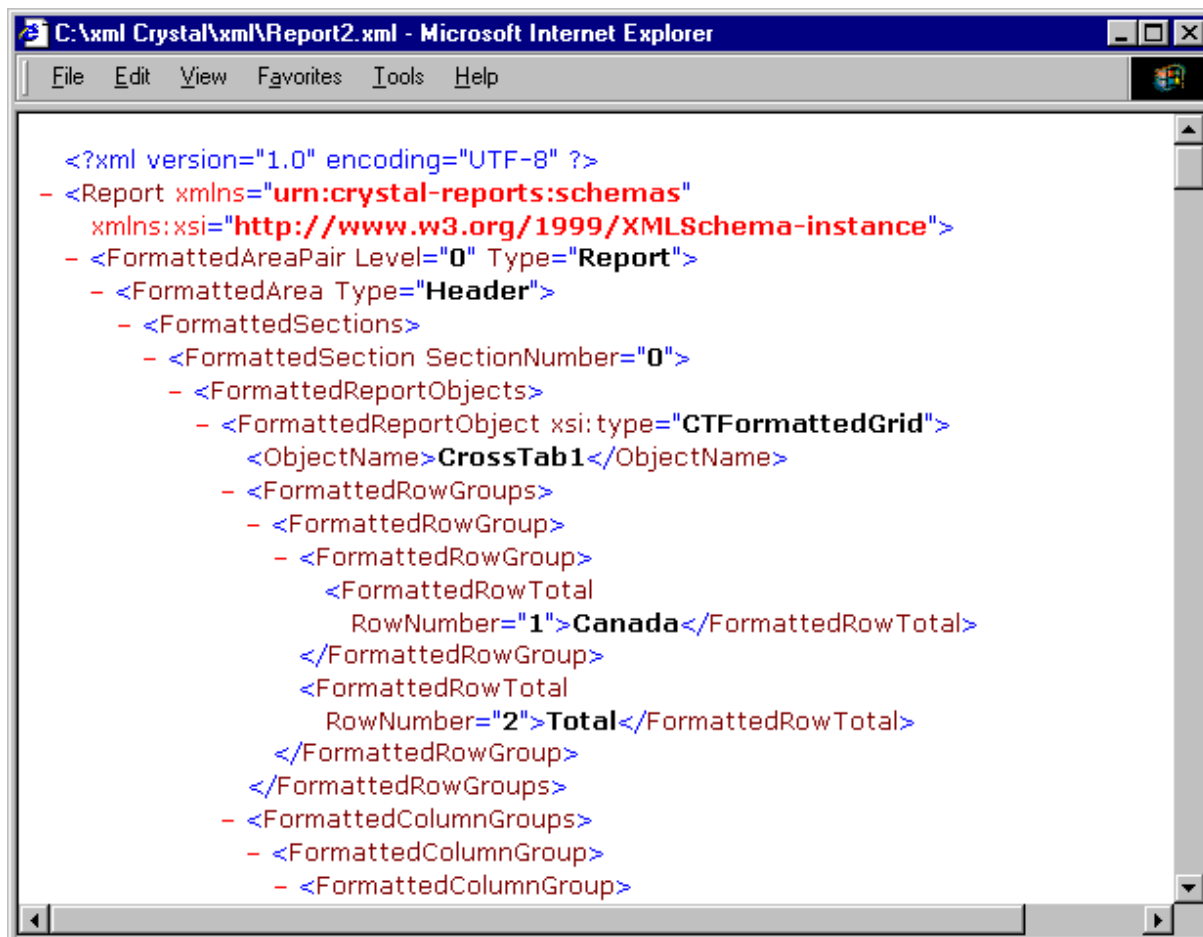
Note

Binary Large Object (BLOB) fields, such as charts, maps, and other types of graphics, are not exported.

The names and attributes of XML element tags depend on the output format you choose on the Others tab of the XML Format dialog box.

23.7.1 Crystal ML Schema

If you use a browser to look at an XML file exported using the Crystal ML Schema, you'll see something like this:



After the XML and document type declarations, there is a list of tags. These tags represent the contents of your report arranged sequentially, item by item. For example, the first few lines of the XML data shown above represent a report that begins with a header area containing a cross-tab object:

```
<Report xmlns="urn:crystal-reports:schemas" xmlns:xsi="http://www.w3.org/1999/
XMLSchema-instance">
  <FormattedAreaPair Level="0" Type="Report">
    <FormattedArea Type="Header">
      <FormattedSections>
        <FormattedSection SectionNumber="0">
          <FormattedReportObjects>
            <FormattedReportObject xsi:type="CTFormattedGrid">
              <ObjectName>CrossTab1</ObjectName>
```

- `<Report>` represents the report itself.
- `<FormattedAreaPair>` represents the generic report area pair (such as a header and footer).
- `<FormattedArea Type="Header">` represents the specific report area: the header.
- `<FormattedSections>` represents a subdivision of the report header area.
- `<FormattedSection SectionNumber="0">` represents an iteration of the subdivision.

- `<FormattedReportObject>` represents a report object (in this case, a formatted grid, or cross-tab).
- `<ObjectName>` represents the report object's (cross-tab's) name.

The entire XML file can be read in such a way. Each line in the XML data has a direct relation to your Crystal report and its Report Definition as defined in Crystal XML Schema and the XML Format dialog box.

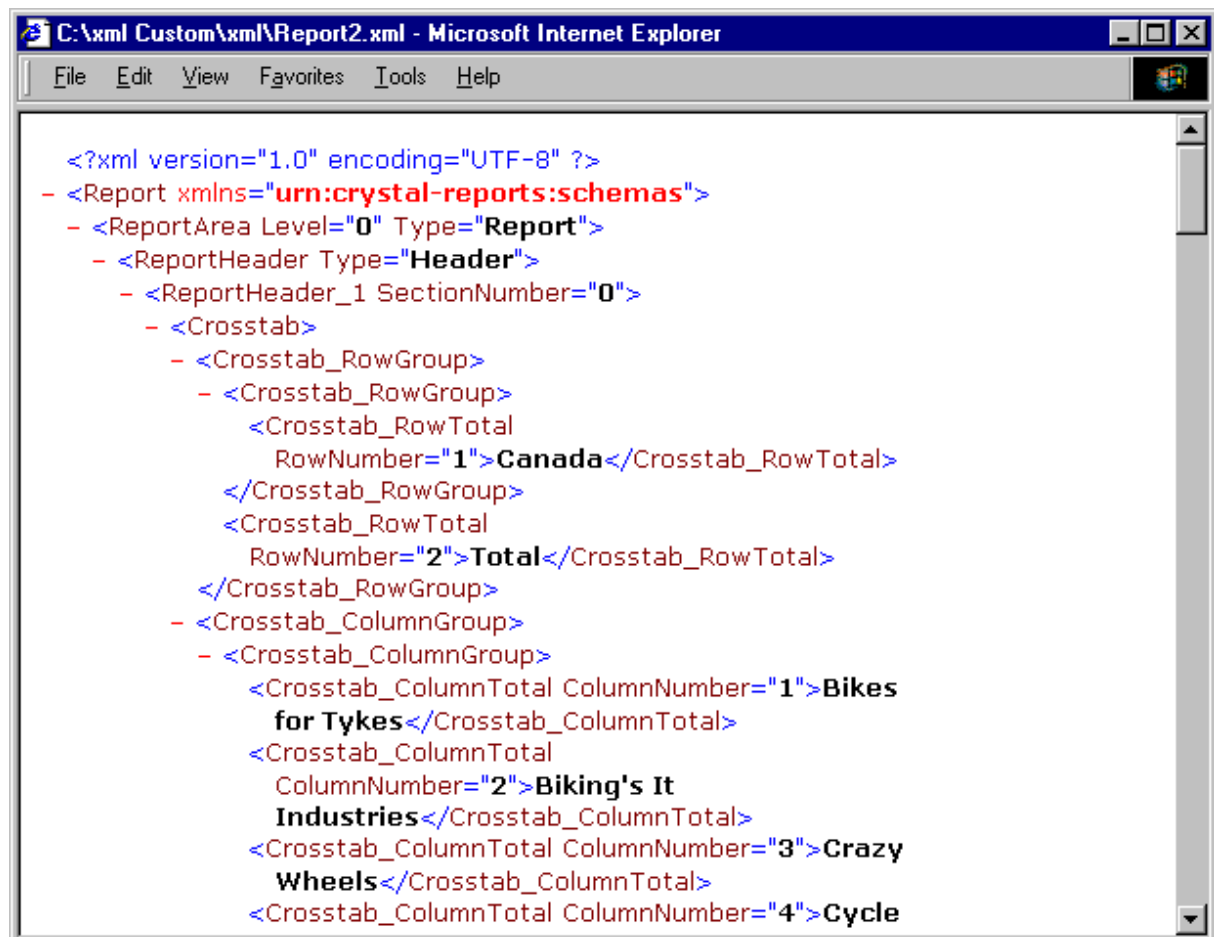
Note

For more detailed information about the XML elements and tags in this example, see the Crystal ML Schema published at the following address:

<http://www.businessobjects.com/products/xml/schema.xsd>

23.7.2 Custom Format

If you use a browser to look at an XML file exported using the Custom Format, you'll see something like this:



After the XML and document type declarations, there is a list of tags. These tags represent the contents of your report arranged sequentially, item by item. For example, the first few lines of the XML data shown above represent a report that begins with a header area containing a cross-tab object:

```
<Report xmlns="urn:crystal-reports:schemas">
```

```
<ReportArea Level="0" Type="Report">
  <ReportHeader Type="Header">
    <ReportHeader_1 SectionNumber="0">
      <Crosstab>
```

- <Report> represents the report itself.
- <ReportArea> represents the generic report area.
- <ReportHeader Type="Header"> represents the specific report area: the header.
- <ReportHeader_1 SectionNumber="0"> represents a subdivision of the report header area.
- <Crosstab> represents a cross-tab object.

The entire XML file can be read in such a way. Each line in the XML data has a direct relation to your Crystal report and its Report Definition as defined in the Customize XML dialog box.

Note

The element tag names in this example are the default names; they will be different if you changed them in the XML Format dialog box.

24 Using Formulas

This section explains the basics of formulas and introduces you to the Formula Workshop in order for you to begin to create formulas.

24.1 Formulas overview

In many cases, the data needed for a report already exists in database table fields. For example, to prepare an order list you would place the appropriate fields on the report.

Sometimes, however, you need to put data on the report that does not exist in any of the data fields. In such cases, you need to create a formula. For example, to calculate the number of days it takes to process each order, you need a formula that determines the number of days between the order date and the ship date. Crystal Reports makes it easy for you to create such a formula.

24.1.1 Typical uses for formulas

There are many uses for formulas. If you have a need for specialized data manipulation, you can do it with a formula.

Creating calculated fields to add to your report

To calculate a price discounted 15%:

Crystal syntax example:

```
{Orders_Detail.Unit Price}*.85
```

Basic syntax example:

```
formula = {Orders_Detail.Unit Price}*.85
```

Formatting text on a report

To change all the values in the Customer Name field to uppercase:

Crystal syntax example:

```
UpperCase ({Customer.Customer Name})
```

Basic syntax example:

```
formula = UCase ({Customer.Customer Name})
```

Pulling out a portion, or portions, of a text string

To extract the first letter of the customer name:

Crystal syntax example:

```
{Customer.Customer Name} [1]
```

Basic syntax example:

```
formula = {Customer.Customer Name} (1)
```

Extracting parts of a date

To determine what month an order was placed:

Crystal syntax example:

```
Month ({Orders.Order Date})
```

Basic syntax example:

```
formula = Month ({Orders.Order Date})
```

Using a custom function

To convert \$500 from U.S. currency to Canadian:

Crystal syntax example:

```
cdConvertUSToCanadian (500)
```

Basic syntax example:

```
formula = cdConvertUSToCanadian (500)
```

24.2 Formula components and syntax

Formulas contain two critical parts: the components and the syntax. The components are the pieces that you add to create a formula while the syntax is the rules that you follow to organize the components.

24.2.1 Formula components

Creating a formula in Crystal Reports is like creating one in any spreadsheet application. You can use any of the following components in your formula:

Fields

Example: {customer.CUSTOMER LAST NAME}, {customer.LAST YEAR'S SALES}

Numbers

Example: 1, 2, 3.1416

Text

Example: "Quantity", ":", "your text"

Operators

Example: + (add), / (divide), -x (negate)

Operators are actions you can use in your formulas.

Functions

Example: Round (x), Trim (x)

Functions perform calculations such as average, sum, and count. All functions available are listed with their arguments and are arranged by their use.

Custom functions

Example: `cdFirstDayOfMonth`, `cdStatutoryHolidays`

Custom functions provide a way to share and reuse formula logic. They can be stored in the SAP BusinessObjects Enterprise Repository and then added to a report. Once in the report, custom functions can be used in the Formula Expert when creating formulas.

Control Structures

Example: "If" and "Select", "For" loops

Group field values

Example: `Average (fld, condFld)`, `Sum (fld, condFld, "condition")`

Group field values summarize a group. For example, you could use group field values to find the percentage of the grand total contributed by each group.

Other formulas

Example: `{@GrossProfit}`, `{@QUOTA}`

24.2.2 Formula syntax

Syntax rules are used to create correct formula. Some basic rules are:

- Enclose text strings in quotation marks.
- Enclose arguments in parentheses (where applicable).
- Referenced formulas are identified with a leading @ sign.

24.2.2.1 Crystal and Basic syntax

When creating formulas, you have the option of using either Crystal or Basic syntax. Almost any formula written with one syntax can be written with the other. Reports can contain formulas that use Basic syntax as well as formulas that use Crystal syntax.

Crystal syntax is the formula language included in all versions of Crystal Reports.

If you are familiar with Microsoft Visual Basic or other versions of Basic, then Basic syntax may be more familiar to you. In general, Basic syntax is modeled on Visual Basic except that it has specific extensions to handle reporting.

If you are already comfortable with Crystal syntax, you can continue to use it, and benefit from the new functions, operators and control structures inspired by Visual Basic.

Note

Record selection and group selection formulas cannot be written in Basic syntax.

Note

Report processing is not slowed down by using Basic syntax. Reports using Basic syntax formulas can run on any machine that Crystal Reports runs on.

Note

Using Basic syntax formulas does not require distributing any additional files with your reports.

24.2.2.2 For more information

- To learn about Basic syntax, see [Creating Formulas with Basic syntax](#) in the online help.
- To learn about Crystal syntax, see [Creating Formulas with Crystal syntax](#) in the online help.

24.3 User Function Libraries in formulas

Crystal Reports lets developers create User Function Libraries (UFLs) that are recognized by the Formula Editor. A UFL is a library of functions that is created by a developer to address a specific need. UFLs can be programmed in a COM or a Java environment.

Crystal Reports lets you see COM UFLs or Java UFLs in the Formula Editor, but not both at the same time. (You can also choose to see no UFLs.)

Note

- Java UFLs are not supported in the Report Application Server (RAS) and the Crystal Page Server, so if a report has a formula in it, and that formula uses a Java UFL, the report may not run in SAP BusinessObjects Business Intelligence platform because the formula will fail to compile.
- SAP Crystal Reports 2025 requires 64-bit COM and .NET UFLs. 32-bit User Function Library (UFL) will not work in Crystal Reports 2025 since it is a 64-bit application. User Function Library (UFL) are created by third party developers to add additional functions to Crystal Reports. If your report is using a 32-bit UFL, contact the Third Party Provider that developed it to compile the UFL in 64-bit, or create a Custom Function that perform a similar function if the UFL did not require access outside the application.

For details about developing Java UFLs and configuring Crystal Reports to use a UFL, see the *Java Reporting Component Developer's Guide*.

Note

After you have completed the configuration outlined in the Developer's Guide, you must return to Crystal Reports, point to the File menu, and select Options. On the Formula Editor tab, go to UFL Support and select Java UFLs Only.

24.4 Specifying formulas

There are several different kinds of formulas in Crystal Reports: report, formatting, selection, search, running total condition, and alert formulas. The majority of formulas in a report are report formulas and conditional formatting formulas.

Report formulas

Report formulas are formulas that you create to stand alone in a report. For example, a formula that calculates the days between the order date and the shipping date is a report formula.

Conditional formatting formulas

Formatting formulas change the layout and design of a report, as well as the appearance of text, database fields, objects, or entire report sections. You format text through the Format Editor. If you need to create a formatting formula, you access the Formula Workshop from the Format Editor. See [Working with conditional formatting \[page 287\]](#).

Note

If you open the Formula Workshop from the Format Editor, you cannot create a different type of formula, nor can you modify or delete existing formulas. You can, however, view any formula for reference.

Selection formulas

Selection formulas specify and limit the records and groups that appear in a report. You can either enter these formulas directly or specify the selection using the Select Expert. Crystal Reports then generates the record selection and group selection formula. You have the option to manually edit these formulas, but you must use Crystal syntax. See [Selecting records \[page 155\]](#).

Search formulas

Search formulas help you locate data in your report. Like selection formulas, you normally do not enter these formulas directly, but instead specify the search criteria using the Search Expert. Crystal Reports generates the formula. You have the option to manually edit these formulas, but you must use Crystal syntax.

Note

If you already know Basic syntax, you need to know only a small amount of Crystal syntax to modify most selection and search formulas.

Running Total condition formulas

Running Total condition formulas let you define the condition upon which your running total will be evaluated or reset. See [Creating conditional running totals \[page 212\]](#).

Alerting formulas

Alerting formulas help you define conditions and messages for report alerts. See [About Report Alerts \[page 657\]](#).

24.4.1 Working with the Formula Workshop

You can create many different kinds of formulas in the Formula Workshop. The workshop consists of a toolbar, a tree that lists the types of formulas you can create or modify, and an area for defining the formula itself.

Note

Search formulas and Running Total condition formulas are created and maintained through the Search Expert and the Create (or Edit) Running Total Field dialog box. These features open the Formula Workshop so you can work with its familiar user interface, but you cannot add or maintain these kinds of formulas directly from the main Formula Workshop.

24.4.1.1 Accessing the Formula Workshop

There are many ways to access the Formula Workshop. You see it when you add new Formula Fields, when you define selection formulas, when you work with custom functions, and so on.

You can open the Formula Workshop by itself before you begin adding specific kinds of formulas.

24.4.1.1.1 To access the Formula Workshop

Procedure

1. On the *Report* menu, click *Formula Workshop*.

→ Tip



Another way to do this is to click the Formula Workshop button on the Expert Tools toolbar.

The Formula Workshop appears.

2.  Click *New* and select the kind of formula you want to create from the list that appears.

→ Tip

You can also select the appropriate folder in the Workshop Tree, and then click the New button.

The appropriate editor or dialog box appears.

24.4.1.2 Workshop Tree

The Workshop Tree contains folders for each type of formula you can create in Crystal Reports. It also contains folders for custom functions and SQL Expressions. If the workshop appears as the result of using a specific command (for example, you've selected the Record command on the Selection Formulas submenu), the appropriate folder in the tree is selected, and the appropriate version of the Formula Editor appears.

Expand any folder in the tree to see the formulas that already exist. New formulas can be added, and existing formulas can be edited or deleted as needed.

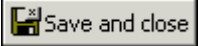
→ Tip

The Workshop Tree can be docked. By default, it appears docked on the left-hand side of the Formula Workshop, but you can manually dock it on the right-hand side. In free-floating mode, the Workshop Tree can be dragged to any location in the workshop.

24.4.1.3 Formula Workshop buttons

The main toolbar in the Formula Workshop is made up of three smaller toolbars. Each of these toolbars contains a set of buttons that is relevant to specific actions: working with the Formula Workshop as a whole, working with the Workshop Tree, or working with an editor. Individual buttons are available or unavailable based on the task you are undertaking. Each toolbar can be moved and docked as you require.

The buttons for the General Toolbar of the Formula Workshop perform the following functions:

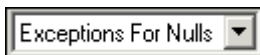
	Closes the Formula Workshop after prompting you to save changes. If you save changes, your formula is checked for errors.
	Saves the formula, SQL Expression, or custom function.
	Creates a new formula of the type selected from the list. Also used to create a new custom function or SQL Expression.
	Shows or hides the Workshop Tree.
	Toggles display between Custom Function Editor and Custom Function Properties dialog box. Custom functions and formulas (where possible) are displayed in the selected mode until this button is clicked again.
	Opens the Formula Expert. Use the Formula Expert to help you create a formula based on a custom function.
 Note This button is not available when creating a custom function.	
	Opens online help for the Formula Workshop dialog box.

The buttons for the Workshop Tree Toolbar of the Formula Workshop perform the following functions:

	Lets you rename the selected formula, custom function, or SQL Expression.
	Deletes the selected formula, custom function, or SQL Expression.
	Expands the selected node in the Workshop Tree. Also works with multiple node selections.
	Shows or hides report object nodes in the Formatting Formulas folder if the objects do not have formulas associated with them.
	Opens the Add Custom Function to Repository dialog box so you can select the repository to add the custom function to.
	Adds the selected repository custom function to the report.

The buttons for the Expressions Editor Toolbar of the Formula Workshop perform the following functions:

	Tests the syntax of the formula or custom function and identifies syntax errors if they are found.
	Undoes the last action performed.
	Redoes the last action.
	When you select a field from the Report Fields window and click the Browse Data button, a dialog box appears with a list of the values for the selected field. View values for the selected field and select individual values to insert into your formula. Note This button is not available for custom functions.
	Searches the Formula Editor for a specified word, number, or formula string.
	Inserts a bookmark at the beginning of a selected formula line. Click the button again to remove the bookmark.
	Jumps to the next bookmark.
	Jumps to the previous bookmark.
	Deletes all bookmarks in the current formula.
	Arranges all Report Fields, Functions, and Operators trees in alphabetical order.
	Hides or views the Report Fields tree. Note This button is not available for custom functions.
	Hides or views the Functions tree.
	Hides or views the Operators tree.
	Hides or views the result set of a global formula search.
	Allows you to select Crystal syntax or Basic syntax as your formula syntax.



Allows you to select Exceptions For Null or Default Values For Nulls as a method for dealing with null values in your data.



Comments out the highlighted selection of a formula. Commented lines are not evaluated as part of the formula.

24.4.2 Working with the Formula Editor

The Formula Editor is a component of the Formula Workshop. Use the Formula Editor to create and modify the content of formulas.

24.4.2.1 Understanding the sections of the Formula Editor

The Formula Editor contains four main windows.

Window	Description of contents
Report Fields	Report fields contain all database fields accessible for your report. They also contain any formulas or groups already created for the report.
Functions	Functions are prebuilt procedures that return values. They perform calculations such as average, sum, count, sin, trim, and uppercase. Custom functions are also listed in this window.
Operators	Operators are the "action verbs" you use in formulas. They describe an operation or an action to take place between two or more values. Examples of operators: add, subtract, less than, and greater than.
Formula text window	Area where you create a formula.

24.4.2.2 Choosing the syntax

The Expressions Editor Toolbar of the Formula Workshop contains the drop-down list where you choose either Crystal or Basic syntax for the formula you are creating.

Note

Changing the syntax from Crystal syntax to Basic syntax or vice versa will change the list of functions in the Functions window as well as the list of operators in the Operators window. The functions and operators differ from syntax to syntax.

The available report fields remain the same since the report fields are available to each syntax.

Setting the default syntax

When you open the Formula Editor, Crystal syntax appears as the syntax default. If you want to change the syntax default, select Options from the File menu, then click the Reporting tab. Choose the preferred syntax from the Formula Language drop-down list and click OK. When you access the Formula Editor, the syntax you selected appears as the default.

24.4.2.3 Entering formula components

The Report Fields, Functions, and Operators tree at the top of the Formula Editor contain the primary formula components. Double-click any component from these trees to add this component to your formula.

For example, if you set the syntax to Basic syntax and double-click the Operators > Control Structures > Multi-Line If in the Operators tree, the following text is transferred to the Formula text window with the cursor between the If and Then:

```
If | Then
ElseIf Then
Else
End If
```

The above text helps you organize the parts needed to write your formula.

24.4.2.4 Formula Editor buttons

For a list of the buttons you can use in the Formula Editor, see the Expressions Editor Toolbar section of [Formula Workshop buttons \[page 694\]](#).

24.4.2.5 Changing the font size of the Formula Editor

The Options dialog box (File menu) allows you to change the font size and the background/foreground color of text, comments, and keywords in the Formula Editor.

24.4.2.6 Formula Editor Key Controls

The Formula Editor accepts the following key controls:

Keyboard Combination	Action Performed
<i>Alt+B</i>	Opens Browse dialog box for highlighted field.
<i>Alt+C</i>	Checks formula for errors.
<i>Alt+F</i>	Opens or closes the Shows Field tree.
<i>Alt+M</i>	Comments out or removes comments from current line
<i>Alt+O</i>	Sorts contents of trees.
<i>Alt+P</i>	Opens or closes the Shows Operator tree.
<i>Alt+S</i>	Saves formula without closing Formula Editor.
<i>Alt+U</i>	Opens or closes the Shows Function tree.
<i>Ctrl+A</i>	Selects all.
<i>Ctrl+C</i>	Copies.
<i>Ctrl+End</i>	Goes to the end of the last line of the formula.
<i>Ctrl+F</i>	Opens the Find dialog box (same as clicking the Find or Replace button).
<i>Ctrl+F2</i>	Sets bookmark.
<i>Ctrl+Shift+F2</i>	Clears all bookmarks.
<i>Ctrl+Home</i>	Goes to the beginning of file.
<i>Ctrl+Left Arrow</i>	Goes to the beginning of the left word (+ <i>Shift</i> key will do the selection).
<i>Ctrl+N</i>	Opens Formula Name dialog box, or another appropriate dialog box, to create a new formula.
<i>Ctrl+S</i>	Saves formula and closes Formula Workshop.
<i>Ctrl+T</i>	Sets focus to the syntax name list box.
<i>Ctrl+Shift+Tab</i>	Changes focus to next control box (reverse order of <i>Ctrl-Tab</i>).
<i>Ctrl+Tab</i>	Changes focus to next control box.

Keyboard Combination	Action Performed
<i>Ctrl+V</i>	Pastes.
<i>Ctrl+X</i>	Cuts.
<i>Ctrl+Z</i>	Undoes an action.
<i>Ctrl+Shift+Z</i>	Repeats an action.
<i>Ctrl+Space</i>	Keyword Auto Complete—shows a list of the functions available.
<i>End</i>	Goes to end of line.
<i>Enter</i>	Copies a selected object from a list to the formula text box.
<i>F3</i>	Finds next item as defined in the Find dialog box.
<i>Shift+F2</i>	Goes to previous bookmark.

24.5 Creating and modifying formulas

Click a link to view one of the topics in this section:

- [To create a formula and insert it into a report \[page 700\]](#)
- [Creating a formula in the Formula Expert \[page 701\]](#)
- [Editing formulas \[page 703\]](#)
- [Searching for formula text \[page 703\]](#)
- [Copying formulas \[page 704\]](#)

24.5.1 To create a formula and insert it into a report

Procedure

1. On the *View* menu, click *Field Explorer*.
2.  In the Field Explorer dialog box, select *Formula Fields* and click *New*.
3. In the Formula Name dialog box, enter the name you want to identify the formula by, and then click *OK*.
The Formula Workshop appears with the Formula Editor active.
4. On the Expressions Editor Toolbar, choose either Crystal or Basic syntax.
If you are unsure which syntax to choose see [Formula syntax \[page 690\]](#).

5. Enter the formula by typing in the components or selecting them from the component trees.

→ Tip

Ctrl+Space will bring up a list of the available functions. If you've already started typing it will bring up a list of keywords that are possible matches for what you've already typed.

→ Tip

A list of available fields automatically appears when you enter an open brace bracket. Ctrl+Space will bring up the list again if you previously pressed Esc to clear it.

6.  Click [Check](#) to identify any errors in the formula.
7. Fix any syntax errors the Formula Checker identifies.
8. When the formula has the correct syntax, click [Save and close](#) on the Formula Workshop toolbar.
9. Select the new formula in the Field Explorer dialog box, and drag it to where you want it to appear on your report.

Results

📘 Note

A formula that is placed on a report is indicated by @ (for example, @ProcessTime) on the Design tab.

24.5.2 Creating a formula in the Formula Expert

The Formula Expert is a component of the Formula Workshop. Use the Formula Expert to create and modify formulas based on custom functions.

📘 Note

To learn about the Formula Expert's user interface, see Formula Expert in the online help.

24.5.2.1 To create a formula in the Formula Expert

Context

📘 Note

Before you begin this procedure, ensure you have a custom function in your report or in the SAP BusinessObjects Enterprise Repository you have access to.

Procedure

1. On the *Report* menu, click *Formula Workshop*.

→ Tip



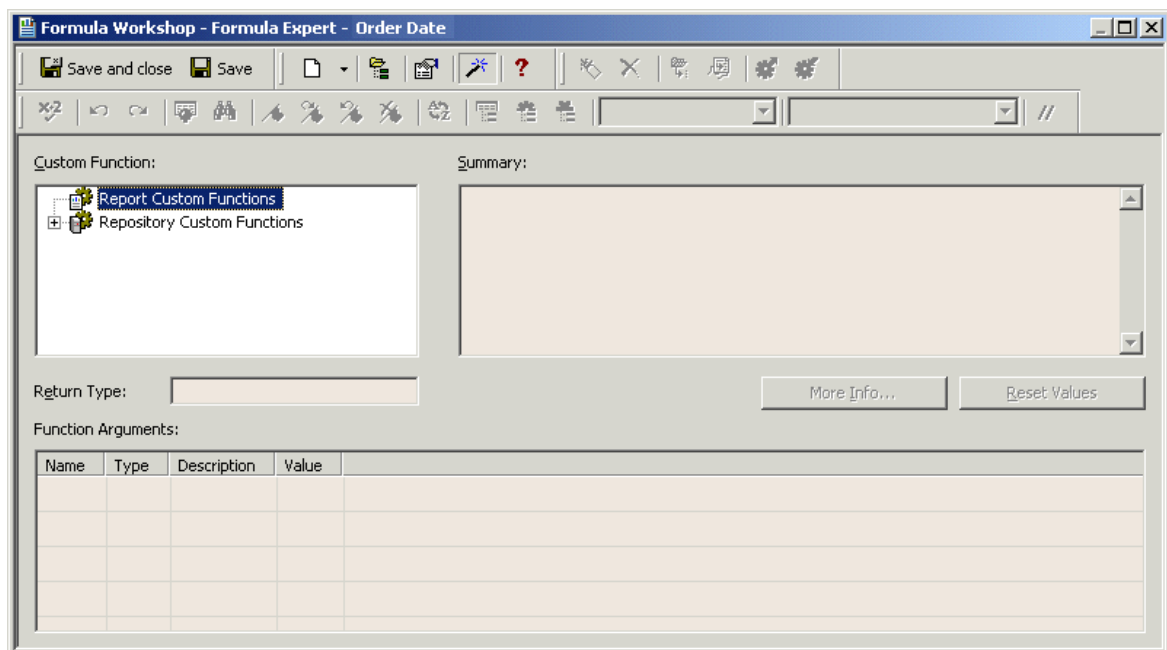
Another way to do this is to click the Formula Workshop button on the Expert Tools toolbar.

2.  Select *Formula Fields* in the Workshop Tree and click *New*.
3. In the Formula Name dialog box, enter the name you want to identify the formula by, and then click *OK*.

The Formula Expert appears with the Formula Editor active.

4.  Click *Use Expert/Editor* on the toolbar.

The Formula Expert appears.



5. In the Custom Function area, choose the custom function you want to base your formula on.
You can choose a Report Custom Function (a custom function that exists in the current report) or a Repository Custom Function (a custom function that is stored in the repository).

Note

If you select a Repository Custom Function, that custom function is added to the current report. If that custom function requires other custom functions from the repository, they can be added as well.

6. In the Function Arguments area, specify a value for each argument in the appropriate *Value* field.
You can enter constant values directly, or you can select predefined values or report fields from the associated list.

7. Click [Save](#) to save the formula to the Formula Fields folder of the Formula Workshop.

Results

You can now use this formula in your report just as you would use a formula you created in the Formula Editor.

24.5.3 Editing formulas

Procedure

1. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

2. Right-click the formula you want to edit and choose [Edit](#).

The Formula Workshop appears with the Formula Editor active.

3. In the Formula Editor, edit the formula.

4.  Click [Check](#) to identify any errors in the formula.

5. Fix any syntax errors the Formula Checker identifies.

6. When the formula has the correct syntax, click [Save and close](#) on the Formula Workshop toolbar.

24.5.4 Searching for formula text

You can use the Find dialog box to carry out a number of different activities:

- Search for text or characters in the Field, Function, or Operator trees.
- Search for text or characters that you want to mark or replace in the formula, Custom Function, or SQL Expression that you have open.
- Search for text or characters across all SQL Expressions in a report.
- Search for text or characters across all formulas and Custom Functions in a report.

When you search for text across all formulas, the results appear in a pane at the bottom of the Formula Workshop. (You can dock the results pane in other locations). When you click an item in the results pane, Crystal Reports opens the appropriate formula and highlights the matching text.

→ Tip

You can open the Formula Workshop to search across all formulas from the Field Explorer and from the body of a report. To do this, select a formula field, right-click it, and choose the "Find in Formulas" option.

24.5.4.1 To search and replace text

Procedure

1. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

2. Right-click the formula you want to edit and choose [Edit](#).

The Formula Workshop appears with the Formula Editor active.

3.  On the Expressions Editor Toolbar, click [Find or Replace](#) to open a Find dialog box.

From this dialog box, you can search and replace text within the Formula text box.

4. Enter the text that you want to search for.
5. Select [Edit Text](#) from the Search list.
6. If you want to mark all occurrences of the search text, click [Mark All](#).
7. If you want to replace all occurrences of the search text with the contents of the "Replace with" text box, click [Replace All](#).

24.5.5 Copying formulas

Click a link to view one of the topics in this section:

- [To copy an existing formula \[page 704\]](#)
- [To copy formulas from online help \[page 705\]](#)
- [To copy a formula from one report to another \[page 706\]](#)
- [Key points for editing a copy of a formula \[page 706\]](#)

24.5.5.1 To copy an existing formula

Context

Crystal Reports lets you copy an existing formula and then modify it to create a new formula.

Procedure

1. Choose [View](#) from the main menu and select [Field Explorer](#).

The Field Explorer dialog box appears.

2. Select an existing formula in the *Formula Fields* list, right-click it, and select *Duplicate* from the shortcut menu.

Crystal Reports creates a copy of the formula by appending a number to the end of the formula's name.

3.  Click *Edit* to modify the formula by changing its definition.

Note

You can also rename the formula in either the Formula Workshop or the Field Explorer.

4. When the formula has the correct syntax, click *Save and close* on the Formula Workshop toolbar.

24.5.5.2 To copy formulas from online help

Context

Since the formulas you develop using the Formula Editor are text, you can copy useful online formulas directly into the Formula Editor and then modify them to fit your needs.

Procedure

1. On the *Help* menu, click *Crystal Reports Help*.

The Crystal Reports online help appears.

2. Click the *Index* tab.
3. Enter **formulas** in the keyword field and click *Display*.
4. Scroll through the formula topics until you locate the formula you want to copy.
5. Highlight the formula, right-click, and choose *Copy* from the menu.

Windows places a copy of the selected text on the Clipboard.

6. Return to Crystal Reports, choose *View* from the main menu and select *Field Explorer*.

The Field Explorer dialog box appears.

7.  Select *Formula Fields* and click the *New* button.

The Formula Name dialog box appears.

8. Enter the name you want to identify the formula by, and then click *OK*.

The Formula Workshop appears with the Formula Editor active.

9. Place the insertion point where you want the text to appear in the *Formula* text box of the Formula Editor and press *Ctrl+V* to paste the text from the Clipboard.
10. Modify the formula by changing the fields, formulas, group fields, conditional statements, and text strings as necessary for use with the data in the new report.
11. When the formula has the correct syntax, click *Save and close* on the Formula Workshop toolbar.

24.5.5.3 To copy a formula from one report to another

Procedure

1. Select the formula field you want to copy in the report.
2. On the [Edit](#) menu, click [Copy](#).
3. Open the report you want to copy the formula to.
4. Choose [Paste](#) from the Edit menu.
5. When the program displays the object frame, drag the formula to the new location.
6. To make changes to the formula, right-click the formula and choose [Edit Formula](#) from the shortcut menu.
The Formula Workshop appears with the Formula Editor active.
7. Delete the old values and type in the new values, or select them from the Fields, Functions, and/or Operators tree.
8. Click [Save and close](#) on the Formula Workshop toolbar when finished.

24.5.5.4 Key points for editing a copy of a formula

When making changes, use the following points as a guide:

- All fields, formulas, and group fields referenced in the formula copy must actually exist in the new report. This means that any database referenced in the original formula (or a database with the same structure, field names, and alias) must be active in the new report.
- If such a database is not active, you must change the field, formula, and group field references in the formula copy to correspond to elements in your new report.
- If the formula contains conditional elements, make certain that the conditions apply to the data in the new report. For example, if the formula in your old report performed an action when the quantity was greater than 100, make sure that the greater than 100 condition makes sense in the new formula. When modifying a formula, you may find that greater than 10 or greater than 2000 makes more sense with your new data.
- If you are using the formula with new data, and if your report contains statements similar to the following:

```
If {file.FIELD} = "text string"
```

Make sure that the text strings used in the formula match values that actually exist in the new data.

24.6 Deleting formulas

When a formula is created and added to a report, the Report Designer:

- Stores the specification for creating the formula, using the name you assigned to it.
- Places a working copy of that formula at the point you specify in the report. A working copy is any occurrence of the formula in the report.

In order to completely delete formulas, you must delete the specification and all working copies of the formula.

ⓘ Note

You cannot delete the specification without deleting all working copies of the formula.

24.6.1 Removing the working formula from your report

Procedure

1. Right-click the formula you want to delete from the report.
2. Select *Delete*.

Results

ⓘ Note

Even after the working copies of a formula have been deleted from the report, the formula specification remains unchanged. The specification is listed in the Field Explorer dialog box. It is available if you wish to enter the formula in the report again.

24.6.2 Deleting the formula specification

Procedure

1. Choose *View* from the main menu and select *Field Explorer*.
The Field Explorer dialog box appears.
2. Right-click the formula you want to delete and choose *Delete*.

Results

ⓘ Note

A dialog box appears if this formula is currently in use in a report. If you delete this formula, you will delete all references of it in reports. Click Yes to delete.

24.7 Debugging formulas

For help on debugging formulas that occur when you save your formula, see the [Debugging tutorial \[page 708\]](#).

For help on debugging evaluation time errors that invoke the formula editor stack list and assisted debugging, see [Debugging evaluation time errors \[page 708\]](#).

24.7.1 Debugging evaluation time errors

When the Formula Workshop is being displayed as a result of an evaluation time error, the Workshop Tree will contain a call stack. The root of the tree provides a description of the error which occurred. The nodes in the tree provide the names of the custom functions and/or formulas which were being evaluated when the error occurred. The custom function/formula at the top of the call stack is where the error was detected. The custom function/formula next in the stack has invoked the custom function/formula above it in the stack. If you select a custom function/formula node in the tree, the text of the custom function/formula will be displayed in the editor window and the text of the expression being evaluated when the error occurred will be highlighted. If you expand a custom function/formula node in the tree, the variables being used in the custom function/formula will be shown along with the value they had at the time the error occurred.

24.7.1.1 Example of an evaluation time error

Using the sample report "Custom Functions.rpt", create a new formula that divides 1 by the result of the @Calendar Days Between function. The function would look like this in Basic syntax:

```
formula = 1/{@Calendar Days Between}
```

It would look like this in Crystal syntax:

```
1/{@Calendar Days Between}
```

Insert this formula into the details section of the report and preview it. You'll get a division by zero error and the formula editor will be invoked with the call stack on the left hand side.

24.7.2 Debugging tutorial

Follow the example below to learn the necessary steps for debugging a formula. After completing this exercise, use the same principles to debug your own formulas.

24.7.2.1 About this tutorial

- This tutorial uses the Xtreme.mdb sample database.
- This tutorial uses Crystal syntax.
- The following formula is the formula you will test for errors:

```
If ({customer.CUSTOMER NAME} [1 to 2 = "Bi" and  
ToText({customer.CUSTOMER ID}) [1] = "6") Or  
({customer.CUSTOMER NAME} [1] = 'Ro' and  
ToText({customer.CUSTOMER ID}) [1] = "5")  
    "PREFERRED CUSTOMER"  
Else  
    "DOES NOT FIT CRITERIA"
```

If correct, this formula should pick out all customers whose names begin with "Bi" and whose customer IDs begin with "6" as well as those companies whose names begin with "Ro" and whose customer IDs begin with "5". When printing the field, those selections will read "PREFERRED CUSTOMER", while the rest will read "DOES NOT FIT CRITERIA".

You will now break down the formula to check and see that each condition of the formula is working individually.

24.7.2.2 Formula1

Procedure

1. To get started, create a report using the Customer table in *Xtreme.mdb* and place the following fields from left to right in the Details section:

```
{customer.CUSTOMER ID}  
{customer.CUSTOMER NAME}
```

To test each portion of the formula, you will place a new formula field next to these two fields in the report.

2. Create a new formula called Formula1.
3. Type the following in the *Formula text* box of the Formula Editor:

```
If {customer.CUSTOMER NAME} [1 to 2 = "Bi" Then  
    "TRUE"  
Else  
    "FALSE"
```

4.  Click *Check* to test for errors. You will receive the following error message:

The] is missing.

5. Correct the formula by inserting the missing "] " after the 2.
6. Click *Check* again. You will receive the following message:

No errors found.

7. Click *Save and close* on the Formula Workshop toolbar.

8. Insert the corrected formula field to the right of the two data fields in the Details section of your report.
9.  Click [Print Preview](#) on the Standard toolbar to check the values in the report and compare the fields to see if the field values returned by @Formula1 are correct.

Results

You will find "TRUE" listed next to the customer names that begin with "Bi" and "FALSE" next to all the others. Now you will check the other portions of the formula. Create Formula2, Formula3, and Formula4, by following Steps 1 - 9, using the formulas specified below for each.

Insert each formula field on the same line of the Details section for easy comparison. Check each one for errors, fix as needed, and make sure the values returned are correct before moving on to Formula2.

24.7.2.3 Formula2

Procedure

1. Create a new formula called Formula2.
2. Type the following in the [Formula text](#) box of the Formula Editor:

```
If ToText({customer,CUSTOMER ID}) [1] = "6" Then  
    "TRUE"  
Else  
    "FALSE"
```

3.  Click [Check](#) to test for errors. You will receive the following error message:

This field name is not known.

4. Correct the formula by replacing the comma (,) in the field name with a period (.).
5. Click [Check](#) again. The formula should now be error-free.
6. Place the formula to the right of the @Formula1 field.
7.  Click [Print Preview](#) on the Standard toolbar to check the values in the report and compare the fields to see if the field values returned by @Formula2 are correct.

Results

You should see "TRUE" next to all customer numbers that begin with 6 and "FALSE" next to all customer numbers that do not begin with 6.

24.7.2.4 Formula3

Procedure

1. Create a new formula called Formula3.
2. Type the following in the *Formula text* box of the Formula Editor:

```
If {customer.CUSTOMER NAME} [1 to 2] = 'Ro" Then  
    "TRUE"  
Else  
    "FALSE"
```

3.  Click *Check* to test for errors. You will receive the following error message:

The matching ' for this string is missing.

4. Correct the formula by changing the single quote (') before Ro to a double quote (").
5. Click *Check* again. The formula should now be error-free.
6. Place the formula to the right of the @Formula2 field.
7.  Click *Print Preview* on the Standard toolbar to see the values in the report and compare the fields to check if the field values returned by @Formula3 are correct.

Results

You should see "TRUE" next to all Customer names that begin with "Ro" and "FALSE" next to all Customer names that do not begin with "Ro".

24.7.2.5 Formula4

Procedure

1. Create a new formula called Formula4.
2. Type the following in the *Formula text* box of the Formula Editor:

```
If ToText({customer.CUSTOMER ID}) [1] = "5"  
    "TRUE"  
Else  
    "FALSE"
```

3.  Click *Check* to test for errors. You will receive the following error message:

The word 'then' is missing.

4. Correct the formula by typing in the word "Then" at the end of the first line after "5".
5. Click [Check](#) again. The formula should now be error-free.
6. Place the formula to the right of the @Formula3 field.
7.  Click [Print Preview](#) on the Standard toolbar to see the values in the report and compare the fields to check if the field values returned by @Formula4 are correct.

Results

You should see "TRUE" next to all Customer IDs that begin with 5 and "FALSE" next to all Customer IDs that do not begin with 5.

Now that the formulas are error-free and the field values returned are correct, you will create a formula that links the separate components together. You will begin by linking the first two formulas (@Formula1 and @Formula2) and then you will add @Formula3 and @Formula4 to create the final formula @FinalFormula.

24.7.2.6 Formula1+2

Procedure

1. Create a new formula called Formula1+2.
2. Type the following in the [Formula text](#) box of the Formula Editor:

```
If {customer.CUSTOMER NAME} [1 to 2] = "Bi" and  
ToText({customer.CUSTOMER ID}) [1] = "6" Then  
    "TRUE"  
Else  
    "FALSE"
```

3. Place the formula to the right of the @Formula4 field.

Results

You should see "TRUE" next to each customer whose name begins with Bi and Id begins with 6, and "FALSE" next to all Customer IDs that do not meet this criteria.

If this formula is working correctly, you can create one last formula adding the code from @Formula3 and @Formula4.

24.7.2.7 FinalFormula

Procedure

1. Create a new formula called FinalFormula.
2. Type the following in the *Formula text* box of the Formula Editor:

```
If ({customer.CUSTOMER NAME} [1 to 2] = "Bi" and  
ToText({customer.CUSTOMER ID}) [1] = "6") or  
({customer.CUSTOMER NAME} [1 to 2] = "Ro" and  
ToText({customer.CUSTOMER ID}) [1] = "5") Then  
    "PREFERRED CUSTOMER"  
Else  
    "DOESN'T FIT CRITERIA";
```

3. Place the formula where you want it to appear in the Details section of the report. You can now delete all other formula fields from the report. See [Deleting formulas \[page 706\]](#).

Results

You can use this same process of condition-by-condition testing for any formulas as a means of systematically checking them.

24.8 Error Messages and Formula Compiler Warnings

A) is missing.

Parentheses must be used in pairs; each opening parenthesis must be matched with a closing parenthesis. One of your opening parentheses is not matched by a closing parenthesis. Insert the missing parenthesis and recheck.

A] is missing.

Brackets must be used in pairs; each opening bracket must be matched with a closing bracket. One of your opening brackets is not matched by a closing bracket. Insert the missing bracket and recheck.

A Boolean range variable is not allowed.

You have entered a Boolean range variable. Range variables are allowed in all data types other than Boolean. Either change the data type to something other than Boolean, or enter a Boolean item variable to replace the Boolean range variable.

A day number must be between 1 and the number of days in the month.

You have entered a day number that does not fit the month. The Formula Checker displays this warning if, for the month of January, for example, you enter a day number of zero (0) or a number 32 or greater. Change the day number to fit the month and recheck.

A field is required here.

You have entered something in your formula other than a field at a position where a field is expected. Correct the problem and recheck.

A formula cannot refer to itself, either directly or indirectly.

You cannot enter a formula that refers to itself. For example, in creating the formula @Profit, you cannot use @Profit as the argument to a function. Remove the reference and recheck.

A function is required here.

The Formula Editor is expecting a function but none was entered. Review your formula and enter the required function or correct the formula if it is in error.

A month number must be between 1 and 12.

You have entered a month number that falls outside the allowable range. Enter a month number between 1 and 12 and recheck.

A string can be at most 65534 characters long.

The program allows strings in formulas to be up to 65534 characters long. You have entered a string that exceeds that limit. Reduce the length of the string and recheck.

A subscript must be between 1 and the length of the string.

You have entered a subscript number that specifies a character that does not exist. If you enter a subscript that references the 6th or the 8th character in a five character string, for example, you will get this warning. Change the subscript to a value that exists and recheck.

Note

A subscript can be a negative number if that number is a negative between -1 and the negative length of the string. For example, if the string is three characters long, you can use -1, -2, or -3 as a subscript.

A subscript must be between 1 and the size of the array.

You have entered a subscript that specifies an array item that does not exist. If you enter a subscript that references the 6th or 8th item in a five item array, for example, you will get this warning. Change the subscript to a value that exists and recheck.

Note

A subscript can be a negative number if that number is a negative between -1 and the negative size of the array. For example, if the array is [1,2,3], you can use -1, -2, or -3 as a subscript.

A subtotal condition is not allowed here.

You have entered a subtotal condition for a subtotal that uses something other than a date or Boolean field as the sort and group by field. Your subtotal does not require a condition. Delete the condition and continue.

A subtotal condition must be a string.

You have entered a subtotal condition that is not in string format. Make certain when you enter the condition in the formula that it is surrounded by single or double quotation marks.

A variable cannot be redeclared with a different type.

You have declared a variable with the same name but a different data type than a variable already declared. This is not allowed. Either change the name of the variable or change the data type so it conforms with the original data type.

A variable is required here.

You have used the assignment operator (:=) in a formula without preceding it with a variable. The program expects to see a variable immediately before (to the left of) the assignment operator. Enter a variable and try again.

A variable name is expected here.

You have declared a variable data type without declaring a variable name. You must enter a variable name to complete the declaration. Enter the variable name and continue.

Access denied.

DOS will not allow access to a file specified. Make certain the file is not in use by another program (or another user on a network), and/or make certain you have the right network permissions and try again.

An error occurred when calling the custom function 'sampleFunction'.

An error occurred in your report when it tried to call the custom function specified. Please see the debugger in the Formula Workshop for help with the error.

Cannot allocate memory.

This message typically indicates that there is not enough memory available. Close any reports that are not needed, and exit any programs that are not essential. Then try again.

Cannot reallocate memory.

This message typically indicates that there is not enough memory available. Close any reports that are not needed, and exit any programs that are not essential. Then try again.

Custom function, a return value must be specified by assigning a value to the function name.

You must assign a return value to the function.

Dates must be between year 1 and year 9999.

You have entered a date that falls outside the allowable range. Enter a date that falls within the range of years 1 to 9999 (including the end values), and then recheck.

Disk full.

You have attempted to save a report to a disk that is full. Either save to a different disk, or delete unnecessary files from the current disk and try again.

Division by zero.

You have entered a formula that attempts a division by zero. The program does not allow such a division. Edit the formula so it does not attempt to divide by zero, and then recheck.

To avoid this type of problem, you can use a test such as this:

```
If {file.FORECAST} = 0 Then  
0  
Else  
{file.SALES} / {file.FORECAST}
```

Error in formula code. Please contact Business Objects, Information Management Group.

There is something unusual about the formula that was not foreseen. Please save the formula text that produced this warning and contact the company.

Error in parse tree. Please contact Business Objects, Information Management Group.

In parsing your formula, the program encountered a situation that the parse tree could not process. Please save the formula text that produced this warning and contact the company.

Field still in use.

The field you are requesting is currently in use. Try again once the field becomes available.

Fields cannot be used within a custom function.

You have tried to use a field within a custom function. Fields cannot be used in custom functions.

File name already in use. Please close the window for xxx before saving under this name.

You have tried to save a file under the name of a file already in use in an open report. Close that report first, and then try again.

File not found.

The file name you specified cannot be found. Either the filename or the path is incorrect. Enter the correct filename/path and try again. In some instances the file WBTRVDEF.DLL is missing from your directory. This file is required for reading Data Dictionary files along with WBTRCALL.DLL.

File permission error.

You have requested a file for which you do not have permission. You must gain the necessary permission before you can activate the file.

Incorrect Borland Custom Control DLL (BWCC.DLL) installed. Version m.n or higher required.

The program is finding and using a version of BWCC.DLL that is too old for proper program operation. Here's how this can happen:

BWCC.DLL is installed in the CRW directory (the same directory in which CRW32.EXE or CRW64.EXE resides) during program installation.

The CRW directory is added to the end of the path statement in AUTOEXEC.BAT during installation (if you allowed the installation program to update the path statement).

If an older version of BWCC.DLL has been installed in the Windows directory, the Windows System directory, or a directory that appears earlier in the path than the CRW directory (the result of an earlier installation), the program picks up that version, not the newer version in the CRW directory.

To correct this problem

The correct version of BWCC.DLL is shipped with Crystal Reports. To correct the problem, delete older versions of BWCC.DLL that reside in directories earlier in the path than CRW.

If this does not solve the problem, move the latest version of BWCC.DLL from the CRW directory to the Window's directory.

Internal Error: PrintDlg fail: 4100.

There is no printer driver installed in the Windows Control Panel. When Crystal Reports opens a report, it looks for the printer that was saved with the report. If that printer cannot be found, it looks for the default printer. If there is no default printer set, the error message results.

Insufficient memory available.

There is not enough memory available to do what you want the program to do. Free up memory and try again.

Invalid DOS version.

You are using a version of DOS earlier than Version 3.0. Install DOS Version 3.0 or higher and try again.

Invalid file handle.

You have specified a file handle that does not exist. Enter the correct file handle and continue.

No default printer selected. You may use the Control Panel to select a default printer.

Please use the Control Panel to select a printer and start the program again.

You cannot begin to utilize the program unless you have a default printer selected. Trying to start the program without a default printer results in this error message.

To select a default printer.

Click the Printers icon in the Windows Control Panel; the Printers dialog box appears with all installed printers listed in the Installed Printers box.

If you have not yet installed the printer, install it first, and then double-click its listing.

Note

A printer must first be given the status Active before it can be selected as the default printer.

Note

For additional information in installing printers and default printers, please refer to the documentation that came with Microsoft Windows.

Non unique table reference: tablename.

This is usually caused if a table name contains an underscore or is more than 15 characters long, or starts with a number.

Not enough arguments have been given to this function.

The function requires more arguments than you have entered. Enter the missing argument(s) and recheck.

Not enough memory.

There is not enough memory available to process the command. Close any reports that are not needed, and exit any programs that are not essential. Then try again.

Numeric overflow.

An intermediate result or the final result cannot be represented because it is too big. Restructure or subdivide the formula to create smaller results, and then recheck.

Only local variables can be used within a custom function.

A custom function cannot refer to variables not defined in the custom function.

Physical database not found.

The program is unable to locate either a DLL or the database. Check to make certain that the directories that hold these files are listed in the path statement.

Please cancel printing before closing.

Your Report Engine call is attempting to close a job while it is still in progress. Make certain that you cancel the printing before you close the print job.

Printer not available.

There are problems connecting to the selected printer. Reselect the printer through the Windows Control Panel and try again.

Report file already exists. Overwrite sample.rpt?

You are attempting to save a report under the same name as an existing report. This will overwrite the existing report and make it no longer available. Click Yes to overwrite the report, No to stop the saving process to give you a chance to select a different name.

Report has changed. Save changes to sample.rpt before closing?

You are attempting to close a report window without first saving it, even though you have made changes to the report since you opened it. The changes will be lost unless you save the report before closing. Click Yes to save the changes, No to close the report without saving the changes.

'sampleFunction' must be assigned a value within the custom function.

You must assign a value to the function within in the custom function.

Sorry, this feature is not yet implemented. Try again later.

You have attempted to use a feature that has not been implemented in the current release. Wait till an upgrade that implements the feature and try again.

The formula cannot be evaluated at the time specified.

You are trying to force a field, formula, or function to be evaluated earlier than is possible. Evaluation time functions can only force a later evaluation time, never an earlier one. Change the formula to accommodate the required evaluation time.

The formula is too complex. Try simplifying it.

The formula could not be evaluated because it exceeds the limit of 50 pending operations. Pending operations are operations that are on hold due to order of precedence rules; they will be performed once the operations with higher level precedence are finished.

Sometimes it is possible to rearrange the formula and calculate the same value without requiring as many pending operations. As a very simplified example, in the formula $2+3*4$, the addition cannot be performed until the multiplication has been done. The addition thus becomes pending, on hold until the multiplication is complete. If the formula is written as $3*4+2$ instead, the operations can be performed left-to-right with the same result, thus eliminating the pending operation.

Correct the formula and recheck.

The matching } for this field name is missing.

Field names must be enclosed in braces { }. You have entered one of the required braces but not the other. Insert the missing brace and recheck.

The matching ' for this string is missing.

A string that begins with a ' must end with a ' before the end of the line. You have used the ' in one of those positions but not the other. Insert the missing punctuation and recheck.

The matching " for this string is missing.

A string that begins with a " must end with a " before the end of the line. You have used the " in one of those positions but not the other. Insert the missing punctuation and recheck.

The number of copies of the string is too large or not an integer.

Using the ReplicateString function, you have requested too many copies or you are requesting a non-integer number of copies. Lower the number of copies requested or specify an integer number of copies and try again.

The number of days is too large or not an integer.

When adding days to dates, or subtracting days from dates, you can use only an integer number of days (a whole number); you cannot add or subtract non-integer numbers of days (1/2 days, 3.6 days, and so on). Additionally, once you add or subtract days from a date, the resulting date must fall within the allowable (year) date range, 0000-9999. If you enter a non-integer number of days or if your result falls outside the allowable range, the Formula Editor displays this warning. Correct the problem and recheck.

The number of decimal places is too large or not an integer.

The second argument to the Round(x, # places) or ToText(x, # places) functions must be a small integer (whole number). You have entered a number as the second argument (# places) that specifies too many decimal places or that is not an integer. Change the number to a small integer and recheck.

The record selection formula cannot include 'PageNumber', 'RecordNumber', 'GroupNumber', 'Previous', or 'Next'.

You cannot include the following Print State functions in your record selection formula: PageNumber, RecordNumber, GroupNumber, Previous, or Next. Eliminate the field(s) and recheck.

The record selection formula cannot include a summary field.

You have included a summary field in a record selection formula. The program does not allow this. Remove the summary field and recheck.

The remaining text does not appear to be part of the formula.

You have provided a formula operand (the item on which a formula operation is to be performed) where none is expected. Often this means that you have forgotten an operator, or an earlier part of a function, or some required syntax item. Correct the error and then recheck.

The result of a formula cannot be a range.

You have created a formula that results in a range. A formula must result in a single value. Correct the formula and recheck.

The result of a formula cannot be an array.

You have created a formula that results in an array. A formula must result in a single value. Correct the formula and recheck.

The result of the selection formula must be a Boolean.

You have created a selection formula that returns something other than a Boolean value. Reconstruct the formula using comparison operators (=, and so on) and recheck.

The special variable field could not be created.

This message typically indicates that there is not enough memory available. Close any reports that are not needed, and exit any programs that are not essential. Then try again.

The string is non-numeric.

The argument to the ToNumber function must be a number stored as a string (for example, a customer number, an ID number, and so on). The string may be preceded by a minus sign and may contain leading and trailing blanks. You have used an argument that is non-numeric and therefore cannot be converted to a number. Change the argument to numeric and recheck.

The summary field could not be created.

This message typically indicates that there is not enough memory available. Close any reports that are not needed, and exit any programs that are not essential. Then try again.

The variable could not be created.

The variable you declared could not be created. Check the spelling and syntax of your declaration statement and try again.

The word 'Else' is missing.

In an If-Then-Else expression, you have left out (or misplaced) the 'Else' component and the formula will not function. Insert (or reposition) the 'Else' component and recheck.

The word 'Then' is missing.

In an If-Then-Else expression, you have left out (or misplaced) the 'Then' component and the formula will not function. Insert (or reposition) the 'Then' component and recheck.

There are too many characters in this field name.

A field name may have at most 254 characters. You have entered a field name that exceeds that number. Enter a field name that has an allowable number of characters and try again.

There are too many characters in this string.

Strings in formulas are allowed to be up to 65534 characters long. You have entered a string that exceeds that limit. Reduce the length of the string (or break it into 2 or more concatenated strings) and recheck.

There are too many letters and digits in this name.

A variable name can have at most 254 characters. You have entered a name that exceeds that number. Shorten the name to conform to the limit and continue.

There is an error in this formula. Please edit it for more details.

You have tried to accept a formula (via the Accept button in the Formula Editor) that contains an uncorrected error. Correct the error that was indicated and try again.

There is an error in this custom function. Please edit it for more details.

You have tried to save a custom function that has an error in it. Correct the error that was indicated and try again.

There must be a subtotal section that matches this field.

You have entered a subtotal in a formula without there being a corresponding subtotal in the report itself. Any subtotal you enter in a formula must duplicate a subtotal already in your report. Add the required subtotal to the report and then re-enter the formula, or delete the formula, and then recheck.

This array must be subscripted. For example: Array [i].

You have entered an array without enclosing it in brackets. Enclose the array in brackets and recheck.

This field cannot be summarized.

You have entered a summary field that does not already exist in your report. Any summary field you enter in a formula must duplicate a summary field already in your report. Either enter the summary field in your report first and then re-enter it in the formula, or do not enter the summary field in the formula at all.

This field cannot be used as a subtotal condition field.

The field you are entering as a condition field causes the subtotal in the formula not to match any subtotal in the report. Any subtotal you enter in a formula must duplicate a subtotal already in your report. Either enter the subtotal in your report first and then re-enter it in the formula, or do not enter the subtotal in the formula at all.

This field cannot be used because it must be evaluated later.

You are trying to force a field, formula, or function to be evaluated earlier than is possible. Evaluation time functions can only force a later evaluation time, never an earlier one. Change the formula to accommodate the required evaluation time.

This field has no previous or next value.

You have used a field for which there is no "previous" value as the argument for the Previous or PreviousIsNull function, or you have used a field for which there is no "next" value as the argument for the Next or NextIsNull function. To use either of those functions, replace the argument with a field that contains the appropriate values.

This field must be in the same section as the current formula.

Since the field was put into the formula as an operand, it has been moved to a section where it is no longer a valid operand.

This field name is not known.

You have entered a field name that does not appear in any of the active databases. Correct the spelling of the field name and/or its alias, and then recheck. Or, to enter a field name from a database that is not currently active, activate the database first and then re-enter the field name.

This formula cannot be used because it must be evaluated later.

You are trying to force a field, formula, or function to be evaluated earlier than is possible. Evaluation time functions can only force a later evaluation time, never an earlier one. Change the formula to accommodate the required evaluation time.

This function cannot be used because it must be evaluated later.

You are trying to force a field, formula, or function to be evaluated earlier than is possible. Evaluation time functions can only force a later evaluation time, never an earlier one. Change the formula to accommodate the required evaluation time.

This function cannot be used within a custom function.

You have tried to use a function or formula that will not work in a custom function, that is it is not "stateless".

This group section cannot be printed because its condition field is non-existent or invalid.

Your report contains a group section that is based on a condition field that is either no longer in the report or changed so it is invalid for the group section. Review your grouping criteria to identify and correct the source of the problem.

This subtotal condition is not known.

You have entered a subtotal condition that does not appear anywhere in your report. Any subtotal you enter in a formula must duplicate a subtotal already in your report. Change the condition and recheck.

Too many arguments have been given to this function.

You have entered an array as the argument to a non-array function. This kind of problem can occur, for example, if you forget to use brackets (the required syntax items for an array) to enclose an array. The Formula Checker sees the array values as arguments to a non-array function and displays the error message.

Too many items have been given for this array.

The program allows up to 50 values in an array. You have exceeded this limit. Reduce the number of values in the array and recheck.

Too many open files.

You have too many open files (databases, reports) given the number of files you specified in the CONFIG.SYS FILES = statement. To prevent this error from recurring, either use fewer files or increase the number of files specified in the FILES = statement.

Missing or incorrect operand warnings.

The following warnings appear when the Formula Checker expects to find a specific kind of operand (the item on which a formula operation is to be performed), and finds something different. For example, the formula 5>a is comparing a number to text (the old comparing apples to oranges analogy). When the Formula Checker sees that the number five is being compared to something, it expects that something to be another number. If anything other than a number appears, it displays the warning: "A number is required here."

Depending on the formula that you enter, you might see one of the following messages:

- A number is required here.
- A Boolean array is required here.
- A Boolean is required here.
- A currency amount is required here.
- A currency array is required here.
- A currency range is required here.
- A date array is required here.
- A date is required here.
- A date range is required here.
- A number array is required here.
- A number array or currency array is required here.
- A number, currency amount, Boolean value, or string is expected here.
- A number, currency amount, Boolean, date, or string is required here.
- A number, currency amount, date, or string is required here.
- A number, currency amount, or date is required here.
- A number field or currency amount field is required here.
- A number is required here.
- A number or currency amount is required here.
- A number range is required here.
- A string array is required here.

- A string is required here.
- A string or an array of values is required here.
- An array of values is required here.
- The custom function name 'sampleFunction' is expected here.

25 Using Custom Functions

This section tells you what custom functions are, how to create and maintain them, and how to use them in your Crystal reports.

25.1 Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

25.2 Sample Reports

You can download sample reports and a sample database for Crystal Reports from the following link: [Direct download link for samples](#) .

25.3 What are custom functions?

Custom functions are procedures you create in Crystal Reports to evaluate, make calculations on, or transform data. When you use a custom function in a formula, all the operations in its definition are performed without you having to specify them individually in the formula itself. Thus, the custom function provides a way for you to share and reuse formula logic which, in turn, makes it easier and less time consuming for you and your users to create reports.

For example, suppose you are designing a series of reports from a database that stores department information with three-letter code names, but you want the actual department names to appear in your Crystal reports. Creating a formula to transform the data would work, but you'd have to copy the formula logic from report to report until you'd completed the series. It is much easier if you extract the logic that does the data transformation and save it as a custom function. You can add that custom function to each report in your series without having to reenter the text.

Note

A custom function cannot be placed directly in a report; it must be used as a part of a formula.

25.3.1 Advantages

Consider these advantages when deciding if you want to use a custom function:

- Custom functions can replace sections of formula logic common to many different formulas that, in turn, may be used in many different reports.
- Custom functions can be shared between reports by storing them to the BusinessObjects Enterprise Repository and adding them to reports as needed.
- Custom function can reduce the need for your users to write formulas themselves; you can provide them with the formula logic they need in a custom function.
- Existing formulas can be converted to custom functions using the [Extract Custom Function from Formula dialog box \[page 1239\]](#).
- Custom functions can have arguments, which makes them flexible.
- You can define help text and default argument values for custom functions, which makes it easier for your users to use them.
- Custom functions, unlike User Function Libraries (UFLs) to which they are related, are saved as part of the report file (.rpt) and don't have any external dependencies.
- Custom functions are stateless; the same input always yields the same result.
- When you modify a custom function, you can update the repository copy and refresh each report without having to find and open the formulas that use the function.

25.3.2 Sample work flow

Although there are different ways to work with custom functions in Crystal Reports, this sample work flow will acquaint you with how they might be used:

- Add a custom function to a report
There are three ways to add a custom function to a report:
 - Select the custom function from the SAP BusinessObjects Enterprise Repository and add it to the report.
 - Extract the custom function from an existing report formula using the [Extract Custom Function from Formula dialog box \[page 1239\]](#).
 - Create the custom function directly in the Custom Function Editor.
- Include a custom function in a report formula
There are also two ways to include a custom function in a formula:
 - Enter the custom function directly as part of your formula logic in the Formula Editor.
 - Choose a custom function to supply formula logic in the [Formula Expert \[page 1426\]](#).The custom function you include can be newly created for the report you're working on or added from the SAP BusinessObjects Enterprise Repository.
- Add a custom function to the repository
Once you create a custom function, it is added to the Report Custom Functions node of the Formula Workshop. At that point, the custom function is available for use in your current report. If, however, you want to use the custom function in other reports, you can share it by adding it to the SAP BusinessObjects Enterprise Repository.

Each of these tasks is described in detail in [Working with custom functions \[page 733\]](#).

25.4 Working with custom functions

Crystal Reports lets you create and modify custom functions, use them in formulas, and move them in or out of the repository. The remaining sections provide procedures for how to do these and other related tasks.

25.4.1 Creating custom functions

When creating a new custom function, you can enter its logic directly in the Custom Function Editor, or you can extract the logic from an existing report formula.

25.4.1.1 To use the Custom Function Editor

Procedure

1.  In the Formula Workshop, click [New](#) and select [Custom Function](#) from the list.
2. Enter a name for your custom function in the [Custom Function Name dialog box \[page 1189\]](#).
3. Click [Use Editor](#).
4. Define your custom function in the Custom Function Editor.

Using the Custom Function Editor is like using the Formula Editor to create a report formula. For information on how to use the Formula Editor, see [Working with the Formula Editor \[page 697\]](#). For more information on what syntax to use when writing a custom function, see [Custom function syntax \[page 741\]](#).

→ Tip

Ctrl+Space will bring up a list of the available functions. If you've already started typing it will bring up a list of keywords that are possible matches for what you've already typed.

5. Check and save your custom function.

Results

Once the custom function has been saved, you can modify it or use it in report formulas as needed.

25.4.1.2 Using the Extract Custom Function from Formula dialog box

When you extract a custom function from an existing formula, the program copies the formula and replaces its fields with arguments.

Note

A custom function created using the *Extractor* can only return simple types, like strings and integers. It cannot return arrays. To create a custom function that can return arrays, use the Custom Function Editor. For more information on return types and the syntax rules associated with custom functions, see [Custom function syntax \[page 741\]](#).

25.4.1.2.1 To extract a custom function from an existing formula

Procedure

1.  In the Formula Workshop, click *New* and select *Custom Function* from the list.
2. Enter a name for your custom function in the [Custom Function Name dialog box \[page 1189\]](#).
3. Click *Use Extractor*.
4. In the *Formula* list, select the report formula whose logic you want to use in your custom function.

Once you've selected a formula, the Return Type, Arguments, and Formula Text fields are completed with information that relates to that formula. These fields cannot be changed without selecting a different formula.

The remaining fields are optional. See [Extract Custom Function from Formula dialog box \[page 1239\]](#) for a description of each.

→ Tip

You will probably want to create meaningful names for the arguments in your custom function. The program assigns generic names (v1, v2, v3, and so on) by default.

→ Tip

Select "Modify formula to use new custom function" to replace the text of the formula you selected in this step with the new custom function you're creating.

5. Click *OK* to save the custom function to the Report Custom Functions node of the Formula Workshop.

Results

Once the custom function has been extracted and saved, you can modify it or use it in report formulas as needed.

Note

You cannot add new arguments to your custom function while you are creating it in the [Extract Custom Function from Formula](#) dialog box. To add new arguments, save the custom function and modify it in the [Custom Function Editor](#).

25.4.2 Viewing and modifying custom functions

Once you've created a custom function, you can view or modify it at any time using the Custom Function Editor.

When you view a repository custom function, the Custom Function Editor and the [Custom Function Properties dialog box \[page 1190\]](#) appear in read-only mode. (You cannot modify a repository custom function without first adding it to your report.)

25.4.2.1 To modify a custom function

Procedure

1. In the Formula Workshop, expand the [Report Custom Functions](#) node and select the custom function you want to modify.

Note



If the custom function appears in read-only mode, you may have it connected to the repository. Check the Workshop Tree for the "connected to repository" icon. Disconnect the custom function from the repository to make it editable.



You can modify the text of the custom function by using the Custom Function Editor. You can modify the custom function's properties by using the Custom Function Properties dialog box. Switch between the two interfaces by clicking the [Toggle Property Display](#) button. For more information on the syntax of a custom function and rules associated with custom functions, see [Custom function syntax \[page 741\]](#).

2. Make the necessary changes to your custom function.
3. Check and save your changes.

Results

Note

If the custom function you're modifying has been added to the SAP BusinessObjects Enterprise Repository, be sure to update the repository copy when you're done.

25.4.3 Defining default argument values for custom functions

You can define default argument values for a custom function in the [Custom Function Properties dialog box \[page 1190\]](#). Users can choose from the default values in the Function Arguments area of the [Formula Expert \[page 1426\]](#).

25.4.3.1 To define default argument values

Procedure

-  In the Formula Workshop, ensure [Toggle properties display](#) is selected on the toolbar.
- Expand the [Report Custom Functions](#) node of the workshop tree view and select the custom function you want to define default argument values for.

Note

The custom function must have arguments defined and must be in the report. You cannot define default argument values for custom functions stored only in the repository. As well, the custom function must be disconnected from the repository before you can edit it.

- Click the [Default Values](#) field for the appropriate argument.

The name and functionality of the dialog box that appears depends on the argument field you've selected. See [Default Values for dialog box \[page 1220\]](#) for more information.

Note

You cannot create default values for range, array, or range array argument types.

- Create your set of default values:
 - Click [Add](#) to include a new value.
 - Click [Remove](#) to delete a value.
 - Click the arrow buttons to arrange the order of the values.

Note

The dialog box won't let you add the same value twice.

Note

The value you put in the first position becomes the default value for the argument. For arguments that are not optional, this value is used by the Formula Expert if no value is entered or if the Reset Values button is clicked.

5. Click **OK** when you're done adding default values.

The default values are displayed separated by blanks in the Default Values field of the Custom Function Properties dialog box. In the Formula Expert, the default values appear at the top of the list associated with the value field.

Results

See [Optional Values \[page 737\]](#) for information about another type of default value you can create for your users.

25.4.4 Optional Values

An optional value can be specified for an argument when you create a custom function.

Note

Optional values are created in the Custom Function Editor and cannot be added through the [Custom Function Properties dialog box \[page 1190\]](#).

For example, the following formula text provides an optional value of the current date for the function called *FirstDayOfMonth*:

```
Function FirstDayOfMonth (Optional dayInMonth As DateTime = CurrentDate) As Date
```

The advantage of having your users select an optional value instead of a default value is that, for an optional value, the value used is not determined until the custom function is executed. The value is determined by the custom function itself, so if you change the value in the custom function, all the formulas based on that function change too.

In the [Formula Expert \[page 1426\]](#), optional values are displayed with their current value (the current value may be different at the time the formula is executed). The Formula Expert uses the optional value for an optional argument if no value is entered or if the Reset Values button is clicked.

For more information on specifying optional values, see [Custom function syntax \[page 741\]](#).

25.4.5 Calling custom functions

This section describes the ways to call a custom function.

25.4.5.1 Calling a custom function from within a formula

Custom functions can be selected for use in formulas just as you'd select any other function. They can be used in any formula in your report, including report formulas, formatting formulas, record selection formulas, and so on. A formula can call any custom function, even if the custom function was written in a different syntax. For example, a Basic syntax formula can call a Crystal syntax custom function.

Once a custom function has been created, you can select it from the Custom Functions folder of the Functions Tree in the Formula Editor (for more information, see [Working with the Formula Editor \[page 697\]](#)). Once you've selected a custom function for use in your formula, you must specify the argument values as defined when the custom function was created.

For the convenience of your users, custom functions can also be used through the Formula Expert. In this expert, the user chooses the custom function on which to base his or her formula and specifies the argument values as prompted for by the expert. You can set default values for your users to choose from. For information about how to create these defaults, see [Defining default argument values for custom functions \[page 736\]](#).

→ Tip

For a description of the choices that can appear in a Value field list, see the Function Arguments description in the [Formula Expert \[page 1426\]](#).

→ Tip

For a procedure that shows how to create a formula in the Formula Expert, see [Creating a formula in the Formula Expert \[page 701\]](#).

25.4.5.2 Calling a custom function from within another custom function

Custom functions can be used in formulas, but they can also be used in other custom functions. For an example, see the sample custom function called "cdExpandRegionAbbreviation" that ships with Crystal Reports; you can find it in the BusinessObjects Enterprise Repository.

ⓘ Note

The syntax of the custom function being called does not depend on the syntax of the custom function doing the calling. For example, a custom function written in Basic syntax can call a custom function written in Crystal syntax.

25.4.5.3 Calling a custom function with optional arguments

When calling a custom function with optional arguments, you can omit the optional arguments and the default value will be assumed for each optional argument left out.

Note

If you omit an optional argument, you must omit all arguments following.

25.4.6 Adding custom functions to the SAP BusinessObjects Enterprise Repository

Because reusability is one of the main virtues of custom functions, it makes sense to store your report custom functions in the SAP BusinessObjects Enterprise Repository. Once a custom function is in the SAP BusinessObjects Enterprise Repository, it becomes available to any Crystal Reports user who has access to that repository. Repository custom functions can easily be added to a report, as shown in the section called [Calling custom functions \[page 737\]](#). Repository custom functions can be edited in your report and then added back to the repository. Reports that use the updated custom functions can be refreshed with the latest version from the repository.

Note

You must add all custom functions to the Repository Custom Functions node that is provided with Crystal Reports.

Note

Adding a custom function to the repository connects the custom function to the repository. Custom functions that are connected to the repository appear in read-only mode in the [Custom Function Properties dialog box \[page 1190\]](#) and the Custom Function Editor.

25.4.6.1 To add a custom function to the SAP BusinessObjects Enterprise Repository

Procedure

1. In the Formula Workshop, expand the [Report Custom Functions](#) node and select the custom function you want to add to the SAP BusinessObjects Enterprise Repository.

2.  Click [Add to Repository](#).

If the selected custom function already exists in the repository you chose, you are prompted to add a new version.

→ Tip

You can also add a custom function to the SAP BusinessObjects Enterprise Repository by dragging it from the Report Custom Functions node—in the Workshop Tree—and dropping it on a Repository Custom Functions node.

25.4.7 Adding custom functions from the SAP BusinessObjects Enterprise Repository

As shown in the section called [Calling custom functions \[page 737\]](#), repository custom functions can be added to a report while creating a formula in the [Formula Expert \[page 1426\]](#). You can also add repository custom functions to a report without creating a formula.

Note

Report custom functions remain connected to the repository they are added from. To edit a custom function that is connected to the repository, you must first disconnect it by selecting the "Disconnect from Repository" option on the custom function's shortcut menu.

25.4.7.1 To add a custom function from the SAP BusinessObjects Enterprise Repository

Procedure

1. In the Formula Workshop, expand the [Repository Custom Functions](#) node and select the custom function you want to add.

2.  Click [Add to Report](#).

If the selected custom function already exists in your report, you are prompted to replace it with a new version. As well, if the custom function requires other custom functions from the repository, they can be added at the same time.

Results

→ Tip

You can also add a repository custom function to a report by dragging its Repository Custom Functions node—in the Workshop Tree—and dropping it on a Report Custom Functions node.

25.4.8 Refreshing a report's repository custom functions

Once a custom function has been modified and the updated version added to the BusinessObjects Enterprise Repository, you can update any Crystal report that uses the custom function by opening it in the Report Designer.

25.4.8.1 To refresh a report's repository custom functions

Procedure

1. On the *File* menu, click *Options* and ensure *Update Connected Repository Objects On Open* is selected on the *Reporting* tab.
2.  Click *Open* on the Standard toolbar in Crystal Reports and select the report whose custom functions need to be updated.
3. Click *Open*.

The report's custom functions and all other updated repository objects are refreshed as the report loads.

Results

If you don't want to use this global option, you can also update a report's custom functions by selecting the Update Repository Objects option on the [Open dialog box \[page 1314\]](#) for an individual report.

25.5 Custom function syntax

Custom functions can be written in both Basic and Crystal syntax.

There are some rules and guidelines you should be aware of when writing a custom function.

One of the most important characteristics of custom functions is that they are evaluated on demand.

Note

A report can contain custom functions written in Crystal syntax as well as custom functions written in Basic syntax.

25.5.1 Basic syntax for custom functions

The syntax for custom functions in Basic syntax is very similar to the syntax for functions in Visual Basic.

Function **<name>** [(argList)] [As **<type>**]

[statements]

[name=expression]

[Exit Function]

...

End Function

- **<name>**
Required. This is the name of the function. It must be the same as the name given to the function when you created it. The name cannot begin with a number, and cannot contain any spaces or punctuation marks other than the underscore. Also, it cannot be the same as a keyword in either Basic or Crystal syntax.
- **<type>**
Optional. Represents the type of the data value returned by the function. May be any of the simple Crystal Reports types (Number, Currency, String, Boolean, Date, Time, DateTime) or range types (Number Range, Currency Range, String Range, Date Range, Time Range, DateTime Range). If the return type is omitted, it is deduced by the type of the expression in the first assignment to name (implicit typing).

Note

If an array return type is desired, you must use the implicit typing since there is no explicit notation for this.

- **<expression>**
Assign expression to name to specify the value returned by the function. If type is specified, then this is optional, otherwise, it is required so that Crystal Reports will know the return type of the function through implicit typing.

The Exit Function statement causes Crystal Reports to immediately exit from the function. You can use this statement any number of times, anywhere in the body of the function.

Note

The only statements you can place before the start of the function, or after the end of the function declaration are comments—either with apostrophes or Rem statements.

argList syntax

[Optional] **<varName>** [()] As **<varType>** [= **<defaultValue>**]

- Optional
Optional. Indicates that the argument can be omitted when the custom function is called. If an argument is optional, all subsequent arguments must be optional as well. If you supply a value for an optional argument, then you must supply a value for each preceding argument.
- **<varName>**
Required. Specifies the name of the variable representing the argument.
- ()
Optional. If present, then it means that the argument variable is an array variable.
- **<varType>**
Required. Indicates the type of the argument variable. It can be any of the seven simple or six range types mentioned in the argList description above. If the optional parentheses are present, the type is an array type whose elements are of type varType.
- **<defaultValue>**
Required for optional arguments. defaultValue is a constant or constant expression, that is, an expression involving no variables that can be simplified at compile time to a constant value. If an optional argument is omitted when the function is called, then the default value for the argument will be used.

Note

Unlike Visual Basic, if an argument is optional, a default value must be supplied.

Passed by value

All arguments are passed by value. In Visual Basic, arguments can be passed either by value or by reference, and are passed by reference by default. However, custom functions do not support reference arguments.

25.5.2 Crystal syntax for custom functions

Crystal syntax custom functions are expression oriented, like Crystal syntax formulas. You can use either syntax formats listed below.

```
Function ([ <argList> ])
```

expressions

```
Function ([ <argList> ])
```

```
(
```

expressions

```
)
```

argList syntax

Optional. A list of variables representing arguments that are passed into the function when it is called. Multiple variables are separated by commas. An empty **<argList>** is allowed; however, the parentheses surrounding **<argList>** are not optional and must be used even when **<argList>** is empty.

```
[Optional] <simpleType> [Range] [Array] <varName> [:= <defaultValue> ]
```

- Optional
Optional. Indicates if the argument can be omitted when the custom function is called. If an argument is optional, all subsequent arguments must be optional as well. If you supply a value for an optional argument when calling the function, then you must supply values for each preceding argument.
- **<simpleType>**
Required. You can use one of:
 - NumberVar
 - CurrencyVar
 - StringVar
 - BooleanVar
 - DateVar

- TimeVar
- DateTimeVar
- Range
Optional. Specifies if the argument is a range variable.

Note

You cannot use Boolean Ranges.

- Array
Optional. Specifies if the argument is an array variable.
- **<varName>**
Required. The name of the variable representing the argument.
- **<defaultValue>**
Required for optional arguments. defaultValue is a constant or constant expression; that is, it's an expression involving no variables that can be simplified at compile time to a constant value. If an optional argument is omitted when calling a custom function, then the default value for the argument will be used.
- expressions
Required. The value returned by the Crystal syntax custom function is the value of the last expression evaluated in the expression sequence. The syntax is the same as that in Crystal syntax formulas.

Scope

Variable declarations within a custom function are local scope by default. This contrasts with variable declarations in Crystal syntax formulas, where global scope is the default.

25.5.3 Custom function syntax rules

Due to the independent nature of custom functions, there are some limitations:

- You cannot use report or database fields, including summary fields, in a custom function.
- You cannot use shared or global variables in a custom function, but you can pass a global or shared variable in as an argument when calling a custom function from within a formula.
- You cannot use recursion, either direct or indirect, in a custom function.
- You cannot directly use UFLs in a custom function.
- You cannot call Evaluation Time, Print State, or Document Properties functions. Also, you cannot use the functions: Rnd, CurrentFieldValue, DefaultAttribute and GridRowColumnValue.

These restrictions are designed to make custom functions independent of the report they are being used in.

Variables

You can use local variables and argument variables in custom functions. If you need to use a shared or global variable in your custom function, you can pass it in as an argument.

25.5.4 Evaluation time

Formulas are not evaluated "on demand" or called (that is, one formula cannot call another formula to do some processing, even through the use of global variables).

For example:

```
Rem Formula PrintSquare
Global x As Number
x = 5
formula = "The square of " & CStr(x,0) & " is " &
    ToText ({@square, 0}) & "."
Rem Formula Square
Global x As Number
formula = x * x
```

Using formula PrintSquare in a report, you get:

```
The square of 5 is 0.
```

In this example, formula PrintSquare uses formula Square, so Crystal Reports orders the evaluations. First Square is evaluated and then PrintSquare is evaluated. When Square is evaluated, the value of the global variable x is 0, since that is the value of an uninitialized Number variable.

In contrast, custom functions are evaluated on demand, or called. Using a custom function in the previous example:

```
Rem Formula PrintSquare
formula = "The square of " & CStr (5,0) & " is " &
    CStr(cf9Square (5),0) + "."
Rem Custom Function cf9Square
Function cf9Square (x As Number)
    cf9Square = x * x
End Function
```

Now if you use formula PrintSquare in your report, you'll get:

```
The square of 5 is 25.
```

25.6 Custom function examples

The following are some examples that show the syntax of custom functions. More typical examples are given in the sample report "Custom Functions.rpt."

25.6.1 Example of a custom function that returns the sum of two numbers

Click a link to view one of the topics in this section:

- [Sum of two numbers - Basic syntax \[page 746\]](#)
- [Sum of two numbers - Crystal syntax \[page 746\]](#)
- [Return values for SumTwoNumbers \[page 746\]](#)

25.6.1.1 Sum of two numbers - Basic syntax

```
Function SumTwoNumbers (Num1 as Number, Num2 as Number) as Number
    SumTwoNumbers = Num1 + Num2
End Function
```

Note

"as Number" at the end of the first line is optional.

25.6.1.2 Sum of two numbers - Crystal syntax

```
Function (NumberVar Num1, NumberVar Num2)
    Num1 + Num2
```

25.6.1.3 Return values for SumTwoNumbers

- SumTwoNumbers(7,10) returns 17
- SumTwoNumbers(0,55) returns 55

25.6.2 Example that capitalizes a string using an optional argument

This example takes in a string and returns that string with the first letters in each word capitalized if the Optional Boolean argument is True. If the Boolean argument is False or omitted, it returns the string in all capital letters.

25.6.2.1 Capitalize a string - Basic syntax

```
Function Capitalize (str as String, _  
    Optional firstLettersOnly as Boolean = False) as String  
If firstLettersOnly then Capitalize = ProperCase (str) _  
    Else Capitalize = UCase (str)  
End Function
```

25.6.2.2 Capitalize a string - Crystal syntax

```
Function (StringVar str, Optional BooleanVar firstLettersOnly:=False)  
if firstLettersOnly then ProperCase(str)  
else UpperCase(str)
```

25.6.2.3 Return values for Capitalize

- Capitalize("crystal reports") returns the string "CRYSTAL REPORTS"
- Capitalize("crystal reports", True) returns the string "Crystal Reports"
- Capitalize("crystal reports", False) returns the string "CRYSTAL REPORTS"

25.6.3 Example of a function that returns an array

Click a link to view one of the topics in this section:

- [Return an array of 10 numbers - Basic syntax \[page 747\]](#)
- [Return an array of 10 numbers - Crystal syntax \[page 748\]](#)
- [Return values for MakeNumberArray \[page 748\]](#)

25.6.3.1 Return an array of 10 numbers - Basic syntax

```
Function MakeNumberArray ()  
Dim x() as Number  
Dim i as Number  
For i = 1 to 10  
    x(i) = i  
Next  
MakeNumberArray = x  
End Function
```

25.6.3.2 Return an array of 10 numbers - Crystal syntax

```
Function ()  
NumberVar array x;  
Redim x[10];  
NumberVar i := 1;  
For i := 1 to 10 do  
    x[i]:=i;  
x
```

25.6.3.3 Return values for MakeNumberArray

- MakeNumberArray () returns the array consisting of the numbers: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

25.6.4 Example of a function that has only optional arguments

This example takes in two numbers and returns the sum. Both arguments are optional so the function can be called with no arguments.

25.6.4.1 Optional arguments example - Basic syntax

```
Function FunnySum (Optional Num1 as Number = 10, _  
    Optional Num2 as Number = 20)  
FunnySum = Num1 + Num2  
End Function
```

25.6.4.2 Optional arguments example - Crystal syntax

```
Function (Optional NumberVar Num1:= 0,  
    Optional NumberVar Num2 := 0)  
Num1 + Num2
```

25.6.4.3 Return values when calling FunnySum with different sets of arguments

- FunnySum(3, 2) returns 5

- FunnySum(3) returns 23
- FunnySum () returns 30

26 Creating Formulas with Basic Syntax

This section provides you with an overview of various parts of a formula. You learn about Basic syntax and the techniques you can use when creating a formula.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

26.1 Basic syntax overview

When creating formulas, you have the option of using either Crystal or Basic syntax. Syntax rules are used to create correct formulas. Almost any formula written with one syntax can be written with the other. Reports can contain formulas that use Basic syntax as well as formulas that use Crystal syntax.

If you are familiar with Microsoft Visual Basic or other versions of Basic, then Basic syntax may be more familiar to you. In general, Basic syntax is similar to Visual Basic except that it has specific extensions to handle reporting.

Note

Report processing is not slowed down by using Basic syntax. Reports using Basic syntax formulas can run on any machine that Crystal Reports runs on. Also, using Basic syntax formulas does not require distributing any additional files with your reports.

26.1.1 Tips on learning Basic syntax

Click a link to view one of the topics in this section:

- [If you have no programming experience \[page 751\]](#)
- [If you know Microsoft Visual Basic, VBScript, or another version of Basic \[page 751\]](#)
- [If you already know Crystal syntax \[page 752\]](#)

26.1.1.1 If you have no programming experience

In many cases you may not need to use the formula language. Crystal Reports includes several Experts that automatically handle situations where formulas could be used. These include the Select, Search, and Highlighting Experts, and the Insert Summary and Create Running Total Field dialog boxes. Before creating formulas, check to see if you can use one of these tools.

However, you may need to create formulas without the help of an Expert. Read this section to learn about Basic syntax and the rules you need to follow to create a formula.

26.1.1.2 If you know Microsoft Visual Basic, VBScript, or another version of Basic

In order to create formulas using Basic syntax, you need to understand the following:

- How a Basic syntax formula refers to other fields in the report such as database fields, parameter fields, summary fields, running total fields and other formula fields.
- How to return a value from a formula by setting the special variable named *formula*.
- How to use functions specific to report processing such as ReportTitle and OnFirstRecord.
- How to use the type system. Basic syntax is strongly typed, similar to using Option Explicit in Visual Basic and there is no Variant type. Basic syntax supports separate Date, Time and DateTime types unlike just the Date type in Visual Basic. It also supports range types such as 10 To 20 to allow for ranges of values which are common in many reporting applications.

Read this section to familiarize yourself with the above details.

Familiar features

- Many Basic syntax functions work in the same way as their counterparts in Visual Basic. This includes string functions such as Len, Mid and Filter, math functions such as Abs, Rnd and Sin, financial functions such as PV, programming shortcut functions such as IIF and date functions such as DateSerial, DateAdd and DateDiff.
- Most operators supported by Visual Basic are also in Basic syntax. For example, string concatenation (&) and date-time literals (#...#).
- Most statements and control structures use the same syntax as in Visual Basic. This includes the If, Select, Do While, Do Until, While and For/Next statements.
- The overall look of the formula will be unmistakably Basic like. For example, Basic style comments and line continuation characters are supported as is the Basic language use of new lines, colons, and the equal sign.

26.1.1.3 If you already know Crystal syntax

The main adjustment is getting used to the parts of the Basic language that are common to every version of Basic. Read this section for a detailed introduction.

26.2 Basic syntax fundamentals

Click a link to view one of the topics in this section:

- [The result of a formula \[page 752\]](#)

26.2.1 The result of a formula

The result of a formula, or the value that is printed when the formula is placed in a report, is called the value returned by the formula. Every formula in Crystal Reports must return a value. Basic syntax does this by setting the value of the special variable `formula`. For example, here is a simple Basic syntax formula that returns the value 10:

```
formula = 10
```

The value returned by a formula can be one of the seven simple data types supported: Number, Currency, String, Boolean, Date, Time and DateTime. Crystal Reports also supports range types and array types, but these cannot be returned by a formula.

26.2.1.1 Restrictions on changing formula variable data types

The `formula` variable can be set several times within a single formula. For example, suppose a company has a shipping policy in which orders over \$1,000 are insured, but orders below that amount are not insured:

```
Rem A formula that returns a String value
If {Orders.Order Amount} >= 1000 Then
    formula = "Insured shipping"
Else
    formula = "Regular shipping"
End If
```

→ Tip

The text following the keyword `Rem` is a comment for someone reading this formula and is ignored by the Basic syntax compiler.

The above formula returns the text string value "Insured shipping" if the value of the database field {Orders.Order Amount} is greater than or equal to 1000; otherwise, it returns the text string value "Regular

Shipping." Text strings are usually just referred to as strings. Notice that the *formula* variable appears twice in the above example.

If the *formula* variable is set to a value of one type, it cannot be set to a value of another type later in the same formula. For example, replacing the String "Regular shipping" in the above example with the Number 10 would result in an error since the special variable *formula* was first set to the String value "Insured shipping."

The reason for this restriction is that Crystal Reports needs to know in advance what the return type of a formula will be so that it can allocate enough storage for the returned values. This is because different types have different storage requirements. Another reason is that the formatting options available for a formula field depend on its type. For example, a Number field has Number formatting options, such as the number of decimals to display, which do not make sense for a String field.

Note

The special variable *formula* should not be declared, unlike other variables used in a Basic syntax formula. See [Variables \(Basic syntax\) \[page 762\]](#).

26.2.1.2 How formulas relate to functions in Microsoft Visual Basic

Consider the following Basic syntax formula:

```
Rem A formula that returns a String value
Rem The function Rnd returns a random number
Rem between 0 and 1
If Rnd > 0.9 Then
    formula = "You won!"
Else
    formula = "Sorry, try again."
End If
```

The above formula returns the text string value "You won!" if the random number returned by Rnd is greater than 0.9 and the text string value "Sorry, try again." otherwise.

The use of the *formula* variable is similar to writing a function named *formula* in Visual Basic.

For example, the above formula could be written as a Visual Basic function as follows:

```
Rem The following code is in Visual Basic
Function formula()
    If Rnd > 0.9 Then
        formula = "You won!"
    Else
        formula = "Sorry, try again."
    End If
End Function
```

26.2.1.3 The variable formula must be assigned a value

If the variable *formula* is not assigned a value, it is not a complete Basic syntax formula.

Note

Some examples in this section are not complete Basic syntax formulas but rather just fragments intended to explain a particular feature.

26.2.1.4 What if I'm not interested in using the value returned by a formula?

Sometimes you may want to write a formula that just declares and initializes some global variables. These formulas are commonly inserted into the report header section of a report. In such cases, assign any value to the special variable `formula`. Every formula must return a value, even if you are not interested in using that value.

For example:

```
Rem Some Global variable declarations
Rem Remember to set the value of 'formula'
Global x As String, y As Number, z As DateTime
x = "hello"
y = 10.5
z = #Aug 6, 1976#
formula = 10
```

26.2.1.5 The variable `formula` must be assigned a value

If the variable `formula` is not assigned a value, it is not a complete Basic syntax formula.

Note

Some examples in this section are not complete Basic syntax formulas but rather just fragments intended to explain a particular feature.

26.2.1.6 Basic syntax is not case-sensitive

What this means is that `formula`, `Formula`, and `FORMULA` are all considered to be the same. This is true of all variable names, functions, and keywords used in a Basic syntax formula.

Note

The only exception to this rule is for strings. The string `"Hello"` is not the same as the string `"hello"`.

26.2.1.7 Practice using the Xtreme sample database

Many of the examples in this section refer to the Xtreme sample database. The sample database can be downloaded from the following link: [Direct download link for samples](#) .

26.3 Comments (Basic syntax)

Formula comments are notes included with a formula to explain its design and operation. Comments do not print and they do not affect the formula; they appear only in the Formula Workshop. Use comments to explain the purpose of a formula or explain the steps involved in writing it.

Comments work as in Visual Basic. Begin comments with a Rem or an apostrophe.

Note

A comment beginning with a Rem is a separate statement and must either start on a new line or be separated from the previous statement by a colon.

```
Rem This is a comment
Rem This is another comment
formula = 10 'So is any text after an apostrophe
formula = 20 : Rem This is also a comment
'Comments can occur after the formula text
```

Related Information

[Comments \[page 1510\]](#)

26.4 Fields (Basic syntax)

Many of the fields you use when creating your report can also be referred to in your formulas. For example, database, parameter, running total, SQL expression, summary, and group name fields can all be used in a formula. You can also refer to other formula fields in your formula.

The easiest way to insert a field into your report is to double-click a field's name in the Report Fields tree. This ensures that the correct syntax for the field is used.

26.4.1 How fields appear in formulas

Database, parameter, formula, running total and SQL expression fields have their names surrounded by braces. Database field names are taken from the database. For example:

Database field: {Employee.Last Name}

Parameter, formula, running total, and SQL expression field names are specified when the fields are created:

- Parameter fields also includes a question mark: {?my parameter field}.
- Formula fields include an at sign: {@another formula}.
- Running totals fields include a pound sign: {#my running total}.
- SQL expression fields include a percent sign: {%my SQL expression}.

Summary and group name fields look like function calls. However, they are really shorthand notation for a report field:

- Sum summary field: Sum({Orders.Order Amount}, {Orders.Ship Via}).
- Group name field: GroupName({Orders.Ship Via}).

26.4.2 Sample formulas using fields

The formula in this example uses the Xtreme database. To find out how many days it takes to ship the product from the date when the order was placed, subtract the ship date database field from the order date database field:

```
Rem A formula that uses database fields
formula = {Orders.Ship Date} - {Orders.Order Date}
```

To find the total dollar amount of a given product that was ordered, multiply its unit price by the quantity ordered:

```
formula = {Orders Detail.Unit Price} * _
{Orders Detail.Quantity}
```

Note

The example uses the line continuation character " _ " (space underscore). See [Statements \(Basic syntax\) \[page 756\]](#).

To calculate a sale price of 80 percent of the original unit price:

```
formula = {Orders Detail.Unit Price} * 0.80
```

26.5 Statements (Basic syntax)

A Basic syntax formula consists of a sequence of statements. Each statement must be separated from the previous statement by either a new line or a colon. Typically, each statement takes one line, but you can

continue a statement onto the next line by using the line continuation character, which is a space followed by an underscore.

For example:

```
'Declare a variable x to hold a number
Dim x As Number
'Assign the value of 30 to x
x = 10 + 10 + 10
'This also assigns the value of 30 to x
x = 10 + _
    10 + 10
'Line continuation characters _
  can also be used in comments
Dim y as String
'Three statements separated by two colons
y = "Hello" : x = 30 : formula = True
```

26.6 Assignment (Basic syntax)

Use the equal sign (=) when making assignments. The keyword Let can be optionally included as well. For example:

```
x = 10
Let y = 20
```

26.7 Simple data types (Basic syntax)

The simple data types in Crystal Reports are Number, Currency, String, Boolean, Date, Time and DateTime.

26.7.1 Number (Basic syntax)

Enter numbers without any comma separators or currency symbols. (Generally, you would want to have formatted numbers appearing as the result of a formula and not in the formula itself.)

Examples

```
10000
-20
1.23
```

26.7.2 Currency (Basic syntax)

Use the CCur function to create a Currency value. The initial C in CCur stands for convert and it can be used to convert Number values to Currency values.

Examples

```
CCur (10000)  
CCur (-20)  
CCur (1.23)
```

26.7.3 String (Basic syntax)

Strings are used to hold text. The text must be placed between double quotation marks (") and cannot be split between lines. If you want to include double quotes in a string, use two consecutive double quotation marks.

```
"This is a string."  
"123"  
"The word ""hello"" is quoted."
```

You can extract individual elements or substrings from a string by specifying the character position or a range of character positions. Negative values are allowed; they specify the position starting from the end of the string.

```
"hello" (2) 'Equal to "e"  
"hello" (-5) 'Equal to "h"  
"604-555-1234" (1 to 3) 'Equal to "604"  
"abcdef" (-3 to -1) 'Equal to "def"
```

You can also extract substrings from a string using the Left, Right and Mid functions.

26.7.4 Boolean (Basic syntax)

The Boolean values are:

```
True  
False
```

Note

[Yes](#) can be used instead of True and [No](#) instead of False.

26.7.5 Date, Time, and DateTime (Basic syntax)

The DateTime type can hold date-times, dates only or times only. The Date type holds dates only and the Time type holds times only. The Date and Time types are more efficient than the DateTime type, and so can be used in situations where the added functionality and flexibility of the DateTime type is not needed.

Visual Basic does not support separate types for holding dates only or times only. The Basic syntax DateTime type is similar to Visual Basic's Date type.

You can create DateTime values directly using the date-time literal construction. It is formed by typing in the date-time between two pound (#) signs. Many different formats are supported, as in Visual Basic.

Note

Date-time literals cannot be split between lines.

Examples

```
#8/6/1976 1:20 am#  
#August 6, 1976#  
#6 Aug 1976 13:20:19#  
#6 Aug 1976 1:30:15 pm#  
#8/6/1976#  
#10:20 am#
```

Even though `#10:20 am#` looks like it could have the Time type and `#8/6/1976#` looks like it could have the Date type, they do not. They both have the DateTime type, as do all date-time literals. For example, you can think of `#10:20 am#` as a DateTime value with a null date part. To convert it to the Time type use `CTime` (`#10:20 am#`).

Instead of using date-time literals you can use `CDateTime` to convert a String to a DateTime. For example:

```
CDateTime ("8/6/1976 1:20 am")  
CDateTime ("10:20 am")
```

However, there is one key difference between using date-time literals and the above usage of `CDateTime`. Date-time literals always use U.S. English date formats rather than settings from the locale of the particular computer on which Crystal Reports is running. Thus, the date-time literal examples above would work on all computers. On the other hand, on a French system, you could use constructions like:

```
CDateTime ("22 aug 1997") 'Same as #Aug 22, 1997#
```

Date values can be constructed with `CDate` and Time values with `CTime`:

```
CDate ("Aug 6, 1969")  
CDate (1969, 8, 6) 'Specify the year, month, day  
'Converts the DateTime argument to a Date  
CDate (#Aug 6, 1969#)  
CTime ("10:30 am")  
CTime (10, 30, 0) 'Specify the hour, minute, second  
CTime (#10:30 am#)
```

26.8 Range data types (Basic syntax)

Ranges are designed to handle a spectrum of values. Range types are available for all the simple types except for Boolean. That is: Number Range, Currency Range, String Range, Date Range, Time Range and DateTime Range. You can generate ranges using the To, _To, To_, _To_, Is >, Is >=, Is < and Is <= keywords. In general, To is used for ranges with two endpoints, and Is is used for open ended ranges (only one endpoint). The underscores are used to indicate whether or not the endpoints are in the range.

Examples of Number Range values

The range of numbers from 2 to 5 including both 2 and 5

```
2 To 5
```

The range of numbers from 2 to 5, not including 2 but including 5

```
2 _To 5
```

All numbers less than or equal to 5

```
Is <= 5
```

All number less than 5

```
Is < 5
```

Examples of DateTime Range values

```
#Jan 5, 1999# To #Dec 12, 2000#  
Is >= #Jan 1, 2000#
```

26.8.1 Using ranges in formulas (Basic syntax)

There are twenty-seven functions in Crystal Reports that specify date ranges. For example, the function LastFullMonth specifies a range of date values that includes all dates from the first to last day of the previous month. So if today's date is September 15, 1999 then LastFullMonth is the same as the range value CDate (#Aug 1, 1999#) To CDate (#Aug 31, 1999#).

Ranges are often used with If or Select statements. The following example computes student letter grades based on their test scores. Scores greater than or equal to 90 receive an "A", scores from 80 to 90, not including 90 receive a "B" and so on.

```
Rem Compute student letter grades
```

```

Select Case {Student.Test Scores}
  Case Is >= 90
    formula = "A"
  Case 80 To_ 90
    formula = "B"
  Case 70 To_ 80
    formula = "C"
  Case 60 To_ 70
    formula = "D"
  Case Else
    formula = "F"
End Select

```

The above example uses the Select statement which is discussed in more detail in [Control structures \(Basic syntax\) \[page 780\]](#). You can check if a value is in a range by using the In operator. For example:

```

formula = 5 In 2 To 10 'True
formula = 5 In 2 To_ 5 'False
formula = 5 In 2 To 5 'True

```

The Maximum and Minimum functions can be used to find the endpoints of a range:

```

formula = Maximum (2 To 10) 'Returns 10

```

26.9 Array data types (Basic syntax)

Arrays in Crystal Reports are ordered lists of values that are all of the same type. These values are known as the array's elements. The elements of an array can be any simple type or range type. One way to create an array is using the Array function.

Examples

An array of three Number values. The first element is 10, the second is 5 and the third is 20.

```

Array (10, 5, 20)

```

An array of seven String values:

```

Array ("Sun", "Mon", "Tue", "Wed", "Th", "Fri", "Sat")

```

An array of two DateTime Range values (note the line continuation character which is used for readability):

```

Array (#Jan 1, 1998# To #Jan 31, 1998#, _
      #Feb 1, 1999# To #Feb 28, 1999#)

```

You can extract individual elements out of an array using parentheses containing the index of the element you want. This is called subscripting the array:

```

Array (10, 5, 20) (2) 'Equal to 5

```

Note

Arrays in Basic syntax are indexed from 1 (this means the first element has index 1). This is unlike in Visual Basic where arrays are indexed from 0 by default. However, in Visual Basic, arrays can be indexed from 1 by using the *Option Base* statement.

Number ranges can also be used to subscript arrays. The result is another array. For example:

```
Array (10, 5, 20) (2 To 3) 'Equal to Array (5, 20)
```

Arrays are most useful when used with variables. Using variables, you can change the individual elements of an array and resize the array to accommodate more elements. This capability significantly expands the capabilities of the formula language to do complex calculations.

For example, you can accumulate database field values into a global array variable in a detail level formula, and then use a formula in a group footer to perform a calculation based on those values. This allows you to perform a wide variety of customized summary operations.

26.10 Variables (Basic syntax)

This section describes the key components of variables and shows you how to create variables and assign values to them.

26.10.1 Variable overview

A variable represents a specific data item, or value, and acts as a placeholder for that value. When a formula encounters a variable, the formula searches for the value of the variable and uses it in the formula. Unlike a constant value, which is fixed and unchanging, a variable can be repeatedly assigned different values. You assign a value to a variable and the variable maintains the value until you later assign a new value. Because of this flexibility, it is necessary for you to declare variables before you use them so that Crystal Reports is aware of them and understands how you intend to use them.

Example of a variable

If you wanted to report on customers by area code, you could create a variable that extracts the area code from a customer fax number. The following is an example of a variable called `<areaCode>`:

```
Dim areaCode As String
areaCode = Left ({Customer.Fax}, 3)
Rem could also use: areaCode = {Customer.Fax} (1 To 3)
```

The first line of the variable example is the variable declaration; it gives the variable a name and type. The database field {Customer.Fax} is a String field and the Left function extracts the first three characters from its current value. The variable areaCode is then assigned this value.

26.10.2 Variable declarations using Dim (Basic syntax)

Before using a variable in a formula, you must declare it. A variable can hold values of a given type. The allowed types are the seven simple types (Number, Currency, String, Boolean, Date, Time and DateTime), the six range types (Number Range, Currency Range, String Range, Date Range, Time Range and DateTime Range) and variables that hold arrays of the previously mentioned types. This gives a total of 26 different types that a variable can have. When you declare a variable, you also specify its name. A variable cannot have the same name as any function, operator or other keyword that is valid for Basic syntax. For example, your variable cannot be named Sin, Mod or If because Sin is a built in function, Mod is a built in operator and If is a built in keyword. When typing formulas in the Formula Workshop, the names of the built-in functions, operators, and other keywords are highlighted in a different color. This makes it easy to check if the variable name conflicts.

Once a variable is declared, it can be used in the formula. For example, you might want to assign it an initial value:

```
Dim x As Number 'Declare x to be a Number variable
x = 10 'Assign the value of 10 to x
```

26.10.2.1 Variables can only hold values of one type (Basic syntax)

A variable can only hold values of one type. For example, if a variable holds a Number value you cannot later use it to hold a String.

Example

```
Dim y As String
y = "hello"
'OK - the Len function expects a String argument
formula = Len (y)
'Error- y can only hold String values
y = #Jan 5, 1993#
'Error- y can only hold String values
y = Array ("a", "bb", "ccc")
'Error- the Sin function expects a Number argument
formula = Sin (y)
```

You can declare more than one variable per statement by separating their declaration by commas:

```
Dim x As Number, y as String, z as DateTime Range
x = 10 : y = "hello"
z = #Jan 1, 1999# To #Jan 31, 1999#
```

26.10.2.2 Declaring variables without immediately specifying their type (Basic syntax)

In general, the type of a variable does not need to be explicitly given when declaring it. In such cases, the variable's type is determined by the first assignment that is made to it. This is similar to the special variable *formula*. This is different from in Visual Basic where a variable whose type is not given at declaration automatically has the Variant type. However, in practice, it means that you can write formulas in a similar style to what you would do if using a Variant in Visual Basic.

```
Dim p 'The type of p is not known yet
p = "bye" 'The type of p is now set to be String
Dim q 'The type of q is not known yet
q = Array ("hello", p) 'q is a String Array
'Error- p is a String variable and cannot hold a Number
p = 25
Dim r
'r is a Number variable, and holds the value 5
r = (10 + 5) / 3
'The types of a and c are not known yet
Dim a, b As Boolean, c
b = False
'The type of a is now set to Boolean
'and its value is False
a = b
'The type of c is now set to Number and its value is 17
c = 2 + 3 * 5
```

Examples of declaring and initializing range variables

```
Dim gradeA, quarter
'The type of gradeA is set to Number Range
gradeA = 90 To 100
'The type of quarter is set to Date Range
quarter = CDate (1999, 10, 1) To CDate (1999, 12, 31)
```

26.10.3 Variable Scope (Basic syntax)

Variable scopes are used to define the degree to which variables in one formula are made available to other formulas. There are three levels of scope in Crystal Reports: local, global and shared. Every variable has a scope, and this scope is specified when the variable is declared.

26.10.3.1 Local Variables

Variables with local scope, also known as local variables, are declared using either the Dim or Local keywords. For example, all the declarations in the previous section using Dim were declaring local variables. Another example:

```
Local x As Number 'equivalent to Dim x As Number
```

Local variables are restricted to a single formula and a single evaluation of that formula. This means that you cannot access the value of a local variable in one formula from a different formula.

Example

```
Rem Formula A
Local x as Number
x = 10
formula = x
Rem Formula B
EvaluateAfter ({@Formula A})
Local x as Number
formula = x + 1
```

The function call EvaluateAfter ({@Formula A}) ensures that Formula B will be evaluated after Formula A is evaluated. Formula A returns a value of 10 and Formula B returns a value of 1. Formula B does not have access to Formula A's x and thus cannot use the value of 10 and add 1 to it; instead, it uses the default value for the uninitialized local variable x found in Formula B, which is 0, and adds 1 to it to get 1.

You can also create local variables with the same name but different types in different formulas. For example, the type declarations in formulas A and B do not conflict with:

```
Rem Formula C
Local x as String
x = "hello"
formula = x
```

Local variables are the most efficient of the three scopes. In addition, they do not interfere with one another in different formulas. For these reasons, it is best to declare variables to be local whenever possible.

26.10.3.2 Global variables (Basic syntax)

Global variables use the same memory block to store a value throughout the main report. This value is then available to all formulas that declare the variable, except for those in subreports. Declare a global variable as in the following example:

```
Global y As String
```

Since global variables share their values throughout the main report, you cannot declare a global variable in one formula with one type and then declare a global variable with the same name in a different formula with a different type.

Example

```
Rem Formula A
Global z As Date
z = CDate (1999, 9, 18)
formula = 10
Rem Formula B
Global z As Number
formula = True
```

In this case, if you enter and save Formula A first, Crystal Reports will return an error when you check or try to save Formula B. This is because the declaration of the Global variable z as a Number conflicts with its earlier declaration in Formula A as a Date.

26.10.3.3 Using Global variables

Global variables are often used to perform complex calculations where the results of a formula depend upon the grouping and page layout of the actual printed report. This is accomplished by creating several formulas, placing them in different sections of the report, and having the different formulas interact via global variables.

Here is an example of the sort of effects that can be produced:

```
Rem Formula C
Global x as Number
x = 10
formula = x
Rem Formula D
'call the function WhileReadingRecords
WhileReadingRecords
Global x as Number
x = x + 1
formula = x
```

If Formula C is placed in the Report Header and then Formula D is placed in a detail section, Formula C will be evaluated before Formula D. Formula C will be evaluated once and then Formula D will be evaluated for each record appearing in the detail section. Formula C returns 10. For the first detail record, Formula D returns 11. This is because the value 10 of x is retained from when it was set by Formula C. Formula D then adds 1 to this value, setting x to 11 and then returns 11. For the second detail record, formula D return 12, adding 1 to the previously retained value of x which was 11. This process continues for the remaining detail records.

The call to WhileReadingRecords tells Crystal Reports to re-evaluate Formula D as it reads in each record of the report. Otherwise, since the formula does not contain any database fields, the program will evaluate it only once before reading the records from the database. The formula will then return the value 11 instead of 11, 12, 13, ... as the successive records are processed.

If the statement `x = x + 1` is replaced by `x = x + {Orders Detail.Quantity}`, you create the effect of a running total based on {Orders Detail.Quantity}, although it is one starting at 10 rather than 0 because of Formula C. In this case, you can omit the call to WhileReadingRecords, since it will automatically occur because the formula contains a database field.

26.10.3.4 Shared variables (Basic syntax)

Shared variables use the same memory block to store the value of a variable throughout the main report and all of its subreports. Thus shared variables are even more general than global variables. To use a shared variable, declare it in a formula in the main report as in the following example:

```
Shared x As Number  
x = 1000
```

and declare it in a formula in the subreport as in the following example:

```
Shared x as Number
```

To use shared variables, the variable must be declared and assigned a value before it can be passed between the main report and the subreport.

26.10.4 Declaring array variables (Basic syntax)

There are several different ways of declaring array variables. The first way is to use empty parentheses and explicitly specify the type of the array:

```
'Declare x to be a Global variable  
'of Number Array type  
Global x () As Number  
'Initialize x  
x = Array (10, 20, 30)  
'Declare y to be a Shared variable  
'of String Range Array type  
Shared y () As String Range  
'Initialize y  
y = Array ("A" To "C", "H" To "J")
```

The second way is to declare the variable without specifying that it is an array and without giving its type and waiting for the first assignment to the variable to completely specify its type:

```
'Declare y to be a Local variable  
'but do not specify its type  
Dim y  
'The type of y is now set to be a String Array  
y = Array ("Sun", "Mon", "Tue", "Wed", "Th", _  
          "Fri", "Sat")
```

The third way is to declare that the variable is an array but not specify its type fully until the first assignment. Assuming the declaration of y above:

```
'Declare z to be a Local variable that is an Array  
Local z()  
'z is set to Array ("Mon", "Tue") and is a String Array  
z = y(2 to 3)
```

The fourth way is to explicitly specify the size of the array during the declaration. If you use this technique, the array is automatically created and default values are used to fill the array. For example, for a Number Array, each element is initialized to 0 and for a String array each element is initialized to the empty string "". Since

this type of declaration actually creates the array, you must specify its type with the As clause so that Crystal Reports knows how much storage space to reserve for the array.

```
Dim a(2) As String
'Assign a value to the first element of the array a
a(1) = "good"
a(2) = "bye"
'The & operator can be used to concatenate strings
'the formula returns the String "goodbye"
formula = a(1) & a(2)
```

Arrays and For/Next loops

Arrays are commonly used with For/Next loops. The following example creates and then uses the array `Array` (10, 20, 30, ..., 100) using a For/Next loop. See [For/Next loops \(Basic syntax\) \[page 784\]](#) for more details.

```
Dim b (10) As Number
Dim i
For i = 1 To 10
    b(i) = 10 * i
Next i
formula = b(2) 'The formula returns the Number 20
```

26.10.4.1 Using array variables (Basic syntax)

You can assign values to elements of an array and also use the values of the elements for other computations:

```
Global x() As String
x = Array ("hello", "bye", "again")
'Now x is Array ("hello", "once", "again")
x (2) = "once"
'The statement below would cause an error if not
'commented out since the array has size 3
'x (4) = "zap"
'The formula returns the String "HELLO"
formula = UCase (x (1))
```

The `Redim` and `Redim Preserve` keywords can be used to resize an array, which is useful if you want to add extra information to it. `Redim` erases the previous contents of the array first before resizing it whereas `Redim Preserve` preserves the previous contents.

```
Dim x () As Number
Redim x (2) 'Now x is Array (0, 0)
x (2) = 20 'Now x is Array (0, 20)
Redim x (3) 'Now x is Array (0, 0, 0)
x (3) = 30 'Now x is Array (0, 0, 30)
Redim Preserve x (4) 'Now x is Array (0, 0, 30, 0)
formula = "finished"
```

26.10.5 Default values for the simple types (Basic syntax)

An uninitialized variable will have the default value for its type. In general, it is not a good programming practice to rely on the default values of types. For example, initialize all local variables in your formula, initialize all global variables in a formula placed in the Report Header, and initialize all shared variables in a formula placed in the Report Header of the main report.

When an array is resized using the Redim keyword, the entries are filled with default values for the type.

26.10.5.1 Default values

Number

```
0
```

Currency

```
CCur (0)
```

String

```
"" 'The empty string
```

Date

```
CDate (0, 0, 0) 'The null Date value
```

Time

The null Time value. Value held by an uninitialized Time variable.

DateTime

The null DateTime value. Value held by an uninitialized DateTime variable.

Note

It is not recommended that your formulas rely on the values of uninitialized range or array variables.

26.10.6 Automatic type conversions (Basic syntax)

Generally in Crystal Reports, values of one type cannot be used where values of another type are expected without explicitly supplying a type conversion function. For example:

```
Dim postalCode as String
'Error- assigning a Number value to a String variable
postalCode = 10025
'OK- use the type conversion function CStr
'to create "10025"
postalCode = CStr (10025, 0)
```

However, there are a few conversions that are made automatically:

- Number to Currency.
- Date to DateTime.
- Simple type to Range value of the same underlying simple type.

For example, the following assignments are correct:

```
Dim cost As Currency
'Same as: cost = CCur (10)
cost = 10
Dim orderDate As DateTime
'Same as: orderDate = CDateTime (1999, 9, 23, 0, 0, 0)
orderDate = CDate (1999, 9, 23)
Dim aRange As Number Range
'Same as: aRange = 20 To 20
aRange = 20
Dim aRangeArray ( ) As Number Range
'Same as :
'aRangeArray = Array (10 To 10, 20 To 25, 2 To 2)
aRangeArray = Array (10, 20 To 25, 2)
```

Note

The opposite conversions are not allowed. For example:

```
Dim num As Number
num = 5 + CCur (10) 'Error
'OK- convert to Number type using the CDb1 function
num = CDb1 (5 + CCur (10))
```

5 is converted to CCur (5) and added to CCur (10) to make CCur (15). However, this Currency value cannot be automatically assigned to the Number variable num since automatic conversions from Currency to Number are not allowed. Similarly, functions accepting a Currency argument can be supplied a Number argument

instead, and the Number argument will be converted to a Currency, whereas functions accepting a Number argument cannot be supplied a Currency argument without first explicitly converting the Currency to a Number using CDBL.

26.11 Functions (Basic syntax)

When using a function in a formula, type the name of the function and supply the arguments required. For example, the Len function requires a String argument and computes the length of the string.

```
Dim x As String
x = "hello"
formula = Len (x) 'The formula returns the Number 5
```

Supplying arguments of the incorrect type required by the function produces an error. For example, calling Len (3) would produce an error since Len does not accept a Number argument. Functions sometimes can accept different numbers of arguments or types of arguments. For example, the CDate function could accept a single String argument to form a Date value or 3 Number values holding the year, month and day respectively and form a Date value from them. See [Date, Time, and DateTime \(Basic syntax\) \[page 759\]](#).

Example with the Mid function

```
Dim x as String
x = "hello"
'Start at position 2, go to the end of the string
formula = Mid (x, 2) 'formula is now "ello"
'Start at position 2, extract 1 character
formula = Mid (x, 2, 1) 'formula is now "e"
```

The classes of functions are: Math, Summary, Financial, String, Date/Time, Date Range, Array, Type Conversion, Programming Shortcuts, Evaluation Time, Print State, Document Properties and Additional Functions. There are also some functions specific to conditional formatting formulas.

→ Tip

The functions that Crystal Reports supports are described in [Functions \[page 1524\]](#).

26.11.1 Functions similar to Visual Basic functions of the same name (Basic syntax)

The Math, Financial, String, Date/Time, Type Conversion and Programming Shortcuts groups consist mainly of functions that are familiar to Visual Basic users. Most of the functions are intended to work in the same way as the Visual Basic function of the same name.

Note

Sometimes functions will have more overloads than are available in Visual Basic.

For example, the CDate function supports the Visual Basic overload of creating a Date value from a String value, such as CDate ("Sept 18, 1999") but it also supports an overload of creating a Date value by supplying the year, month and day as Number arguments e.g. CDate (1999, 9, 18). The overloads are indicated in the Functions tree.

Note

Some functions that are supported by Basic syntax are not listed in the Basic syntax Functions tree. This is because they are equivalent to Basic syntax functions that are already listed in the tree.

For example, the Len function is the normal Basic syntax and Visual Basic function for finding the length of a string. However, the Length function is listed in the Functions tree and works the same as the Len function. Length is the traditional Crystal syntax function for this action and it is included for the convenience of Crystal syntax users migrating formulas to Basic syntax.

26.11.2 Summary functions (Basic syntax)

The Summary function group provides functions for creating summary fields such as:

```
Sum({Orders.Order Amount}, {Orders.Ship Via})
```

Summary fields are normally created using the Insert Summary or Insert Grand Total dialogs. They then appear in the Report Fields tree, and can be used in a formula by double clicking there. However, they do not need to be created in this way. You can create a summary field exclusively for use by your formula by filling in the arguments to one of the functions in the Summary functions section. However, any groups that refer to summary fields must already exist in the report.

→ Tip

The summary functions that Crystal Reports supports are described in [Summary \[page 1551\]](#).

26.11.3 Date ranges (Basic syntax)

For additional information refer to [Range data types \(Basic syntax\) \[page 760\]](#).

Date ranges produced by these functions depend on the current date. For example, if today's date is September 18, 1999, then LastFullMonth is the Date Range value:

```
CDate(#Aug 1, 1999#) To CDate(#Aug 31, 1999#)
```

This functionality is often useful, but if you want to determine a date range based on a database field such as {Orders.Order Date}? The Date/Time functions can be used instead.

Example

```
Dim d As Date
d = CDate ({Orders.Order Date})
Dim dr As Date Range
dr = DateSerial (Year(d), Month(d) - 1, 1) To _
    DateSerial (Year(d), Month(d), 1 - 1)
'At this point dr is the Date Range value holding
'the last full month before {Orders.Order Date}
```

The DateSerial function makes this easy because you don't have to worry about special cases. It never lets you create an invalid date. For example, DateSerial (1999, 1 - 1, 1) is December 1, 1998. Note that in the above example, {Orders.Order Date} is actually a DateTime field and so the CDate function is used to convert it to a date by truncating the time part.

Related Information

[Date Ranges \[page 1801\]](#)

26.11.4 Array functions (Basic syntax)

The array functions compute summaries of an array's elements. For example, the Sum function when applied to an array returns the sum of the elements of the array.

Example

The following formula returns 100:

```
formula = Sum (Array (10, 20, 30, 40))
```

Related Information

[Arrays \[page 1820\]](#)

26.11.5 Evaluation Time functions (Basic syntax)

These are the report specific functions: BeforeReadingRecords, WhileReadingRecords, WhilePrintingRecords and EvaluateAfter. You can use these functions to guide Crystal Reports as to when your formula should be evaluated.

Should the formula be evaluated before retrieving the records from the database, while reading the records from the database but before the records have been grouped, sorted and summarized, or while printing the report, when the records are grouped, sorted and summarized? In general, Crystal Reports sets an appropriate evaluation time for your formula, based on how much information the formula needs. For example, if a formula uses a database field, then it cannot be evaluated before the records are read from the database. However, you sometimes need to force a later evaluation time than normal to get the desired effect. See [Using Global variables \[page 766\]](#) for an example.

Normally, the returned value of a function is used further in a formula. However, evaluation time functions are called to change the internal behavior of Crystal Reports and their return value is not used. They can be called by just placing their name in a separate statement, optionally preceded by the keyword `Call`.

```
WhilePrintingRecords  
Call WhilePrintingRecords
```

Related Information

[Evaluation Time \[page 1851\]](#)

26.11.6 Print State functions (Basic syntax)

These are once again reporting-specific functions. For example, the notation `{Orders.Order Date}` refers to the value of the field in the current record where `PreviousValue ({Orders.Order Date})` refers to the value in the immediately preceding record. `NextValue ({Orders.Order Date})` refers to the value in the next record. `IsNull ({Orders.Order Date})` checks if the field's value is null.

Other examples are `PageNumber` and `TotalPageCount`. These can be used to access pagination information about your report.

Related Information

[Print State \[page 1855\]](#)

26.11.7 Document Properties functions (Basic syntax)

These are reporting specific functions that refer to the report document as a whole. For example, `PrintDate` and `ReportTitle`.

Related Information

[Document Properties \[page 1876\]](#)

26.11.8 Additional functions (Basic syntax)

These are functions that are in User Function Libraries (UFLs). A UFL is a separate dynamic link library or Automation server that you create and Crystal Reports uses to add your own customized functions to the formula language. Writing a UFL is more involved than writing a formula using Basic or Crystal syntax. See Crystal Reports Developer's Help for more information.

Note

Using UFLs makes your reports less portable because you must distribute your UFL along with the report.

Related Information

[Additional Functions \[page 1899\]](#)

26.11.9 Conditional formatting functions (Basic syntax)

When writing a conditional formatting formula, you may want to use the additional functions that appear at the top of the Functions tree.

Example

If you wanted to format the {Customer.Last Year's Sales} field so that sales of more than \$100,000 are printed in green and sales of less than \$15,000 are printed in red and all else are printed in black.

```
Rem Conditional formatting example 1
If {Customer.Last Year's Sales} > 100000 Then
    formula = crGreen
ElseIf {Customer.Last Year's Sales} < 15000 Then
    formula = crRed
Else
    formula = crBlack
End If
```

Since this is a font color formatting function, the list of Color Constants appears in the Functions tree. This example uses three: crGreen, crRed and crBlack. You could have used the actual numeric values of the color constants instead. For example, crRed is 255 and crGreen is 32768. However, the formula is easier to understand using the color constants. All constant functions in Basic syntax have the "cr" prefix.

Note

Some formatting attributes do not use constant functions. For example, if you wanted to not print {Customer.Last Year's Sales} values if the sales were less than \$50,000, you could write the following conditional formatting formula for the suppress attribute:

```
Rem Conditional formatting example 2
If {Customer.Last Year's Sales} < 50000 Then
    formula = True 'suppress the value
Else
    formula = False 'do not suppress the value
End If
```

Or more simply:

```
Rem Conditional formatting example 3 -
Rem equivalent to example 2
formula = {Customer.Last Year's Sales} < 50000
```

If the last year's sales are less than \$50,000, then the expression

```
{Customer.Last Year's Sales} < 50000
```

is True, and so the formula returns True. On the other hand, if the last year's sales are greater than or equal to \$50,000, then

```
{Customer.Last Year's Sales} < 50000
```

is False and so the formula returns False.

26.11.10 General purpose conditional formatting functions (Basic syntax)

There are three general purpose conditional formatting functions:

- CurrentFieldValue
- DefaultAttribute
- GridRowColumnValue

These functions are displayed at the top of the Functions tree when appropriate. DefaultAttribute can be used for any formatting formula. CurrentFieldValue can be used for any formatting formula where you are formatting a field value. GridRowColumnValue can be used for formatting a formula where you are formatting a field value in a Cross-Tab or OLAP grid.

CurrentFieldValue enables you to conditionally format Cross-Tab or OLAP grid cells based on their value. GridRowColumnValue enables you to conditionally format the cells of a Cross-Tab or OLAP grid based on row or column headings values. These two functions are essential in some situations as there is no other way in the formula language to refer to these fields.

→ Tip

The conditional formatting functions that Crystal Reports supports are described in [Conditional formatting functions \[page 1956\]](#).

Example

If you wanted Cross-Tab cells to be suppressed if the values are less than 50,000:

```
Rem Conditional formatting example 4  
formula = CurrentFieldValue < 50000
```

26.12 Operators (Basic syntax)

Click a link to view one of the topics in this section:

- [Arithmetic operators \[page 777\]](#)
- [Comparison operators \(Basic syntax\) \[page 778\]](#)
- [Boolean operators \(Basic syntax\) \[page 778\]](#)
- [Null fields and how to use IsNull \(Basic syntax\) \[page 779\]](#)

26.12.1 Arithmetic operators

The arithmetic operators are addition (+), subtraction (-), multiplication (*), division (/), integer division (\), modulus (Mod), negation (-) and exponentiation (^). Arithmetic operators are used to combine numbers, numeric variables, numeric fields and numeric functions to get another number.

Examples

```
'Outstanding preferred stock as a percent of  
'common stock  
formula = ({Financials.Preferred Stock} / _  
           {Financials.Common Stock}) * 100  
'The square root of 9, Sqr(9), is 3.  
'The formula returns 17.  
formula = 7 + 2 * 3 - 2 + Sqr(6 + 3) * Len("up")
```

→ Tip

The arithmetic operators that Crystal Reports supports are described in [Arithmetic operators \[page 1464\]](#).

26.12.1.1 Order of precedence

When you create arithmetic expressions that involve several operators, the order that the program evaluates the various elements of the expression becomes important.

In general, the program evaluates expressions in the following order:

- From left to right.
- Follows the rules of precedence from basic math.

Example

Multiplication and division are performed first from left to right. Then addition and subtraction are performed. For example, $5 + 10 * 3 = 5 + 30 = 35$.

You can change this order of precedence by using parentheses. For example, $(5 + 10) * 3 = 15 * 3 = 45$. If you are unsure of the order of precedence, it is a good idea to clarify your intentions with parentheses.

The arithmetic operators in Crystal Reports have the same order of precedence as in Visual Basic. Here is the list, from highest precedence to lowest:

- Exponentiation (^)
- Negation (-)
- Multiplication and division (*, /)
- Integer Division (\)
- Modulus (Mod)
- Addition and subtraction (+, -)

26.12.2 Comparison operators (Basic syntax)

The comparison operators are equal (=), not equal (<>), less than (<), less than or equal to (<=), greater than (>) and greater than or equal to (>=).

Comparison operators are usually used to compare operands for a condition in a control structure such as an If statement. Comparison operators as a group all have lower precedence than the arithmetic operators. For example, expressions like $2 + 3 < 2 * 9$ are the same as $(2 + 3) < (2 * 9)$.

Related Information

[Comparison operators \[page 1472\]](#)

26.12.3 Boolean operators (Basic syntax)

The Boolean operators are, in order of precedence from greatest to lowest: Not, And, Or, Xor, Eqv and Imp. Boolean operators are typically used with comparison operators to generate conditions for control structures. Boolean operators as a group have lower precedence than the comparison operators. Thus for example, the expression $2 < 3 \text{ And } 4 \geq -1$ is the same as $(2 < 3) \text{ And } (4 \geq -1)$.

Related Information

[Boolean operators \[page 1494\]](#)

26.12.4 Null fields and how to use IsNull (Basic syntax)

In general, when Crystal Reports encounters a null valued field in a formula, it immediately stops evaluating the formula and produces no value. If you want to handle null field values in your formula, you must explicitly do so using one of the special functions designed for handling them: IsNull, PreviousIsNull or NextIsNull.

Note

These special functions must be used first in a formula for them to be evaluated properly.

Example

The {Product.Color} field contains both basic colors such as "red" and "black" and more descriptive two word colors such as "steel satin" and "jewel green". Here's an example of a formula that writes out "basic" for the basic colors and "fancy" for the others.

```
If InStr({Product.Color}, " ") = 0 Then
    formula = "basic"
Else
    formula = "fancy"
End If
```

The function call to InStr searches the {Product.Color} string for a space. If it finds a space, it returns the position of the space, otherwise it returns 0. Since basic colors are only one word with no spaces, InStr will return 0 for them.

For some products, such as the Guardian Chain Lock, a color value was not recorded and so the {Product.Color} field has a null value in the database for that record. Thus, the Guardian Chain Lock record does not have any word printed beside it.

Here is an example of how to fix the above example using IsNull:

```
If IsNull({Product.Color}) Or _
    InStr({Product.Color}, " ") = 0 Then
    formula = "basic"
Else
    formula = "fancy"
End If
```

Relating to operators, when Crystal Reports evaluates the condition:

```
IsNull({Product.Color}) Or _
InStr({Product.Color}, " ") = 0
```

It first evaluates `IsNull ({Product.Color})`, and when it determines that this is `True`, it knows that the whole condition is `True`, and does not need to check whether

```
InStr({Product.Color}, " ") = 0
```

In other words, Crystal Reports will stop evaluating a Boolean expression when it can predict the results of the whole expression. In the following example, the formula guards against attempting to divide by zero in the case that `denom` is 0:

```
Dim num As Number, denom As Number  
...  
If denom <> 0 And num / denom > 5 Then  
...
```

Note

Visual Basic does not support this technique, since all parts of a Boolean expression in Visual Basic are evaluated, even if not necessary.

26.13 Control structures (Basic syntax)

Formulas without control structures execute each statement in the formula only once. When this happens the formula is evaluated. The statements are executed in a sequential fashion, from the first statement in the formula to the last. Control structures enable you to vary this rigid sequence. Depending upon which control structure you choose, you can skip over some of the statements or repeatedly evaluate some statements depending on certain conditions. Control structures are the primary means of expressing business logic and typical report formulas make extensive use of them.

Basic syntax supports many of the main control structures from Visual Basic with the same syntax. One of the advantages of the Basic language is it is easy to read block notation for control structures. This simplifies the writing and debugging of complex formulas.

Related Information

[Control structure operators \[page 1506\]](#)

26.13.1 If statements (Basic syntax)

The `If` statement is one of the most useful control structures. It enables you to evaluate a sequence of statements if a condition is true and evaluate a different sequence of statements if it is not true.

Note

When formatting with conditional formulas, always include the Else keyword; otherwise, values that don't meet the If condition may not retain their original format. To prevent this, use the DefaultAttribute function (If...Else formula = DefaultAttribute).

Note

When creating record selection formulas that include If statements, always include the Else keyword; otherwise, no records or unexpected records may be returned. For example, a record selection such as If {parameter field} = "less than 100" then {field} < 100 evaluates as False and returns no records. To correct this problem, complete the formula with Else True.

Example

A company plans to pay a bonus of 4 percent to its employees except for those who work in Sales who will receive 6 percent. The following formula using an If statement would accomplish this:

```
Rem Multi-line If example 1
If {Employee.Dept} = "Sales" Then
    formula = {Employee.Salary} * 0.06
Else
    formula = {Employee.Salary} * 0.04
End If
```

In this example, if the condition {Employee.Dept} = "Sales" evaluates as true, then the

```
formula = {Employee.Salary} * 0.06
```

statement is processed. Otherwise the statement following the Else, namely the

```
formula = {Employee.Salary} * 0.04
```

is processed.

Suppose another company wants to give employees a 4% bonus, but with a minimum bonus of \$1,000. The following example shows how. Notice that the Else clause is not included; it is optional, and not needed in this case.

```
Rem Multi-line If example 2
formula = {Employee.Salary} * 0.04
If formula < 1000 Then
    formula = 1000
End If
```

Now suppose that the previous company also wants a maximum bonus of \$5,000. You now need to use an Elself clause. Notice that Elself is all one word. The following example has only one Elself clause, but you can add as many as you need.

Note

There is a maximum of one Else clause per If statement.

The Else clause is executed if none of the If or ElseIf conditions are true.

```
Rem Multi-line If example 3
formula = {Employee.Salary} * 0.04
If formula < 1000 Then
    formula = 1000
ElseIf formula > 5000 Then
    formula = 5000
End If
```

Example

Suppose that a company wants to compute an estimate of the amount of tax an employee needs to pay and write a suitable message. Income below \$8,000 is not taxed, income between \$8,000 and \$20,000 is taxed at 20%, income between \$20,000 and \$35,000 is taxed at 29%, and income above \$35,000 is taxed at 40%.

```
Rem Multi-line If example 4
Dim tax As Currency, income As Currency
income = {Employee.Salary}
Dim message As String
If income < 8000 Then
    tax = 0
    message = "no"
ElseIf income >= 8000 And income < 20000 Then
    message = "lowest"
    tax = (income - 8000)*0.20
ElseIf income >= 20000 And income < 35000 Then
    message = "middle"
    tax = (20000 - 8000)*0.20 + (income - 20000)*0.29
Else
    message = "highest"
    tax = (20000 - 8000)*0.20 + (35000 - 20000)*0.29 + _
        (income - 35000)*0.40
End If
Dim taxStr As String
Rem use 2 decimal places
Rem and use the comma as a thousands separator
taxStr = CStr (tax, 2, ",")
formula = "You are in the " & message & _
    " tax bracket. " & _
    "Your estimated tax is " & taxStr & "."
```

Notice, the use of variables to simplify the logic of the computation. Also, notice that there are two statements that are executed when one of the conditions are met; one assigns the tax variable, and the other assigns the message variable. It is often useful to have multiple statements executed as a result of a condition.

26.13.1.1 Single-line and multi-line If statements (Basic syntax)

There are two kinds of If statement, the single-line if statement and the multi-line if statement. Starting on a new line after the first Then turns your If statement into a multi-line If statement. Otherwise it is a single-line If statement. The multi-line If statement always includes an End If whereas the single line If statement does not.

Note

Because of the use of line-continuation characters, single-line If statements do not need to be on a single line. In general, it is preferable to use multi-line If statements since they have a clearer layout. However, for simple situations, the single-line If statement is sometimes used.

```
Rem Single-line If example 1
Rem Same result as multi-line If example 1
If {Employee.Dept} = "Sales" Then _
    formula = {Employee.Salary} * 0.06 _
Else _
    formula = {Employee.Salary} * 0.04
```

Here is an example showing various forms of single-line If statements:

```
Rem Single-line If example 2
Dim per As Number, extra As Boolean
per = 2 : extra = False
'An example with no Else clause
If {Employee.Dept} = "Sales" Then per = 10
'More than 1 statement in the Then or Else part can
'be included by separating them with colons
If {Employee.Dept} = "R&D" Then _
    per = 5 : extra = True _
Else _
    per = 3
```

26.13.2 Select statements (Basic syntax)

The Select statement is similar to an If statement. Sometimes however, you can write formulas that are clear and less repetitive using the Select statement. This example evaluates the {Customer.Fax} field to determine if the area code is for Washington state (206, 360, 509) or British Columbia, Canada (604, 250):

```
Rem Select example 1
Select Case Left ({Customer.Fax}, 3)
    Case "604", "250"
        formula = "BC"
    Case "206", "509", "360"
        formula = "WA"
End Select
```

The expression right after the Select Case keywords is called the Select condition. In the above example it is Left ({Customer.Fax}[1 To 3]). The Select statement tries to find the first Case that matches the Select condition, and then executes the statements following it, up until the next Case.

```
Rem Same effect as Select example 1
Dim areaCode As String
areaCode = Left ({Customer.Fax}, 3)
If areaCode In Array ("604", "250") Then
    formula = "BC"
ElseIf areaCode In Array ("206", "509", "360") Then
    formula = "WA"
End If
```

Example

This formula groups the number of Oscar nominations a movie received into low, medium, high or extreme categories and in the process, shows some of the possibilities for the expression lists following the Case labels. Notice the optional Case Else clause. If none of the Case expression lists are matched by the preceding Case clauses, then the Case Else clause is matched. For example, in the following example, if {movie.NOM} is 11, then the formula returns "extreme".

```
Rem Select example 2
Select Case {movie.NOM}
  Case 1,2,3, Is < 1
    Rem Can have multiple statements in the
    Rem statement blocks
    formula = "low"
  Case 4 To 6, 7, 8, 9
    formula = "medium"
  Case 10
    formula = "high"
  Case Else
    formula = "extreme"
End Select
```

26.13.3 For/Next loops (Basic syntax)

For/Next loops enable you to evaluate a sequence of statements multiple times. This is unlike the If and Select statements where the program passes through each statement at most once during the formula's evaluation. For/Next loops are best when you know the number of times that the statements needs to be evaluated in advance.

26.13.3.1 The syntax of the For loop through examples

Sample 1

Suppose you want to reverse the {Customer.Customer Name} string. For example, "City Cyclists" becomes "stsilcyC ytiC".

```
Rem Reverse a string version 1
formula = ""
Dim strLen
strLen = Len ({Customer.Customer Name})
Dim i
For i = 1 To strLen
  Dim charPos
  charPos = strLen - i + 1
  formula = formula & _
    Mid({Customer.Customer Name}, charPos, 1)
Next i
```

Examine how this formula works assuming that the current value of the field {Customer.Customer Name} is "Clean Air". The variable strLen is assigned to be the length of "Clean Air", namely 9. At this time it is also

typed to be a Number variable. The variable `<i>` is known as a For counter variable since its value changes with each iteration of the For loop. In other words, it is used to count the iterations of the loop. The For loop will iterate 9 times, during the first time, i is 1, then i is 2, then i is 3 and so on until finally i = 9. During the first iteration, the ninth character of {Customer.Customer Name} is appended to the empty special variable `formula`. As a result formula equals "r" after the first iteration. During the second iteration, the eighth character of {Customer.Customer Name} is appended to formula and so formula equals "ri". This continues until after the ninth iteration, formula equals, "riA naelC" which is the reversed string.

Sample 2

Here is a simpler version of the above formula, that uses a Step clause with a negative Step value of -1. For the "Clean Air" example, i is 9 for the first iteration, 8 for the second, 7 for the third and so on until it is 1 in the final iteration.

```
Rem Reverse a string version 2
formula = ""
Dim i
For i = Len ({Customer.Customer Name}) To 1 Step -1
    formula = formula + _
                Mid({Customer.Customer Name}, i, 1)
Next i
```

Sample 3

The simplest version is to use the built in function StrReverse:

```
Rem Reverse a string version 3
formula = StrReverse ({Customer.Customer Name})
```

The built in String functions in Crystal Reports can handle many of the string processing applications which would traditionally be handled using a For/Next loop or some other kind of loop. However, For/Next loops provide the most flexibility and power in processing strings and arrays. This can be essential if the built-in functions do not cover your intended application.

26.13.3.2 For/Next loop example (Basic syntax)

Here is a more detailed example of Crystal Reports' string processing capabilities. The Caesar cipher is a simple code that is traditionally credited to Julius Caesar. In this code, each letter of a word is replaced by a letter five characters further in the alphabet. For example, "Jaws" becomes "Ofbx". Notice that "w" is replaced by "b". Since there are not 5 characters after "w" in the alphabet, it starts again from the beginning.

Here is a formula that implements applying the Caesar cipher to the field {Customer.Customer Name} in the Xtreme database:

```
Rem The Caesar cipher
Dim inString 'The input string to encrypt
```

```

inString = {Customer.Customer Name}
Dim shift
shift = 5
formula = ""
Dim i
For i = 1 To Len(inString)
    Dim inC, outC
    inC = Mid(inString, i, 1)
    Dim isChar, isUCaseChar
    isChar = LCase(inC) In "a" To "z"
    isUCaseChar = isChar And (UCase (inC) = inC)
    inC = LCase(inC)
    If isChar Then
        Dim offset
        offset = (Asc(inC) + shift - Asc("a")) Mod _
            (Asc("z") - Asc("a") + 1)
        outC = Chr(offset + Asc("a"))
        If isUCaseChar Then outC = UCase(outC)
    Else
        outC = inC
    End If
    formula = formula & outC
Next i

```

In the above example, there is a multi-line If statement nested within the statements block of the For/Next loop. This If statement is responsible for the precise details of shifting a single character. For example, letters are treated differently from punctuation and spaces. In particular, punctuation and spaces are not encoded. Control structures can be nested within other control structures and multiple statements can be included in the statement block of a control structure.

26.13.3.3 Using the Exit For statement (Basic syntax)

You can exit from a For/Next loop by using the Exit For statement. The following example searches the Global array names for the name "Fred". If it finds the name, it returns the index of the name in the array. Otherwise it returns -1. For example, if the names array is:

```
Array ("Frank", "Helen", "Fred", "Linda")
```

Then the formula returns 3.

```

Global names () As String
'The names array has been initialized and filled
'in other formulas
Dim i
formula = -1
'The UBound function returns the size of its array
'argument
For i = 1 to UBound (names)
    If names (i) = "Fred" Then
        formula = i
        Exit For
    End If
Next i

```


26.13.4 Do Loops (Basic syntax)

Another looping mechanism is the Do loop. A Do loop can be used to execute a fixed block of statement an indefinite number of time.

26.13.4.1 The 4 different types of Do loops

Type of Do Loop	Explanation	Example
Do While ... Loop	The Do While ... Loop evaluates the condition, and if the condition is true, then it evaluates the statements following the condition. When it has finished doing this, it evaluates the condition again and if the condition is true, it evaluates the statements again. It continues repeating this process until the condition is false.	<pre>Do While <condition> < statements> Loop</pre>
Do Until ... Loop	The Do Until ... Loop is similar to the Do While ... Loop except it keeps evaluating the statements until the condition is true rather than while it is true.	<pre>Do Until <condition> < statements> Loop</pre>
Do ... Loop While	The Do ... Loop While evaluates the statements only once. It then evaluates the condition, and if the condition is true, evaluates the statements again. This process continues until the condition is false.	<pre>Do <statements> Loop While <condition></pre>
Do ... Loop Until	Similar to Do ... Loop While except that it evaluates the statements until the condition is true.	<pre>Do <statements> Loop Until <condition></pre>

Note

The Do loops support an Exit Do statement to immediately jump out of the loop. The Exit Do statement is similar to the Exit For in For/Next loops.

26.13.4.2 Do While ... Loop formula example (Basic syntax)

The following example searches for the first occurrence of a digit in an input string. If a digit is found, it returns its position, otherwise it returns -1. In this case, the input string is set explicitly to a string constant. If preferred, it could be set to be equal to a String type database field instead.

For example, for the input String, "The 7 Dwarves", the formula returns 5, which is the position of the digit 7.

```
Dim inString
inString = "The 7 Dwarves"
Dim i, strLen
i = 1
strLen = Len (inString)
formula = -1
Do While i <= strLen And formula = -1
    Dim c As String
    c = Mid (inString, i, 1)
    If IsNumeric (c) Then formula = i
    i = i + 1
Loop
```

26.13.5 While loops (Basic syntax)

The While loop is similar to the Do While ... Loop except that it does not support an Exit statement. It uses While ... Wend instead of Do While ... Loop as its syntax.

```
While <condition>
    <statements>
Wend
```

26.13.6 Safety mechanism for loops (Basic syntax)

There is a safety mechanism to prevent report processing from hanging due to an infinite loop. Any one evaluation of a formula can have at most 100,000 loop condition evaluations per formula evaluation. For example:

```
Dim i
i = 1
Do While i <= 200000
    If i > {movie.STARS} Then Exit Do
    i = i + 1
Loop
formula = 20
```

If {movie.STARS} is greater than 100,000 then the loop condition (i <= 200000) will be evaluated more than the maximum number of times and an error message is displayed. Otherwise the loop is OK.

Note

The safety mechanism applies on a per formula base, not for each individual loop. For example:

```
Dim i
i = 1
For i = 1 To 40000
    formula = Sin (i)
Next i
i = 1
Do While i <= 70000
    i = i + 1
Loop
```

The above formula also triggers the safety mechanism since the 100,000 refers to the total number of loop condition evaluations in the formula and this formula will have 40001 + 70001 such evaluations.

26.13.7 Option Loop (Basic Syntax)

The Option Loop statement can be used to specify the maximum number of loop condition evaluations per evaluation of a formula. This statement should only be used when the default maximum loop count of 100,000 is not sufficient for the given formula.

Note

The Option Loop statement must be used before any other statements, except the REM statement.

26.13.7.1 The syntax of the Option Loop

The syntax of the Option Loop is:

```
Option Loop <maxIterations>
```

Where **<maxIterations>** = the maximum number of times you want the condition to be evaluated per evaluation of a formula. It must be a positive integer.

Example:

```
REM Option Loop example
Option Loop 300
REM make name lower case
Formula=""
dim i as number
i=1
While i<len({Customer.Customer Name})
Formula=formula+LCase(Mid({Customer.Customer Name},i,1))
i=i+1
Wend
```

If Customer.Customer Name is greater than 299, an error message will be shown: "A loop was evaluated more than the maximum number of times allowed."

26.14 Limitations (Basic syntax)

For reference purposes, here are the sizing limitations of the formula language:

- The maximum length of a String constant, a String value held by a String variable, a String value returned by a function or a String element of a String array is 65,534 characters.
- The maximum size of an array is 1000 elements.
- The maximum number of arguments to a function is 1000. (This applies to functions that can have an indefinite number of arguments such as Choose).
- The maximum number of loop condition evaluations per evaluation of a formula is 100,000. (See [Safety mechanism for loops \(Basic syntax\) \[page 788\]](#) for the precise meaning of this). Note that you can use the [Option Loop \(Basic Syntax\) \[page 789\]](#) statement to change this limit.
- Date-time functions modeled on Visual Basic accept dates from year 100 to year 9999. Traditional Crystal Reports functions accept dates from year 1 to year 9999.
- There is no limit on the size of a function.

27 Creating Formulas with Crystal Syntax

This section provides you with an overview of various parts of a formula. You learn about Crystal syntax and the techniques you can use when creating a formula.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

27.1 Crystal syntax overview

When creating formulas, you have the option of using either Crystal or Basic syntax. Syntax rules are used to create a correct formula. Almost any formula written with one syntax can be written with the other. Reports can contain formulas that use Basic syntax as well as formulas that use Crystal syntax. For information on whether to use Crystal or Basic syntax, see [Formula syntax \[page 690\]](#).

27.2 Crystal syntax fundamentals

Click a link to view one of the topics in this section:

- [The result of a formula \[page 791\]](#)

27.2.1 The result of a formula

The result of a formula, or the value that is printed when the formula is placed in a report, is called the value returned by the formula. Every formula in Crystal Reports must return a value. For example, here is a simple Crystal syntax formula that returns a value of 10:

10

The value returned by a formula can be one of the seven simple data types supported. These are Number, Currency, String, Boolean, Date, Time, and DateTime.

Note

Crystal Reports also supports range types and array types, but these cannot be returned by a formula.

For example, suppose a company has a shipping policy in which orders over \$1,000 are insured, but orders below that amount are not insured:

```
//A formula that returns a String value
If {Orders.Order Amount} >= 1000 Then
    "Insured shipping"
Else
    "Regular shipping"
```

→ Tip

The text following the two forward slashes is a comment for someone reading this formula and is ignored by the Crystal syntax compiler.

The formula returns the text string value "Insured shipping" if the value of the database field {Orders.Order Amount} is greater than or equal to 1000; otherwise, it returns the text string value "Regular Shipping" otherwise.

27.2.1.1 Crystal syntax is not case-sensitive

For example, the keyword *Then* could equivalently be typed in as *then* or *THEN*. This is true of all variable names, functions, and keywords used in a Crystal syntax formula.

Note

The only exception to this rule is for strings. The string "Hello" is not the same as the string "hello".

27.2.1.2 Practice using the Xtreme sample database

Many of the examples in this section refer to the Xtreme sample database. The sample database can be downloaded from the following link: [Direct download link for samples](#) .

27.3 Comments (Crystal syntax)

Formula comments are notes included with a formula to explain its design and operation. Comments do not print and they do not affect the formula, but they appear in the Formula Workshop. You can use comments to explain the purpose of a formula or explain the steps involved in writing it.

Comments begin with two forward slashes (//) and are followed by the text of the comment. Everything that follows the slashes on the same line is treated as being part of the comment:

```
//This formula returns the string "Hello"  
//This is another comment  
"Hello" //Comments can be added at the end of a line  
//Comments can occur after the formula text
```

Related Information

[Comments \[page 1510\]](#)

27.4 Fields (Crystal syntax)

Many of the fields used in the construction of a report can be referred to in your formulas. For example, database, parameter, running total, SQL expression, summary, and group name fields can all be used in a formula. You can also refer to other formula fields in your formula.

The easiest way to insert a field into your report is to double-click a field's name in the Report Fields tree. This ensures that the correct syntax for the field is used.

27.4.1 How fields appear in formulas

Database, parameter, formula, running total and SQL expression fields have their names surrounded by braces. Database field names are taken from the database. For example:

Database field: {Employee.Last Name}

Parameter, formula, running total, and SQL expression field names are specified when the fields are created.

- Parameter fields also includes a question mark: {?my parameter field}.
- Formula fields include an at sign: {@another formula}.
- Running totals fields include a pound sign: {#my running total}.
- SQL expression fields include a percent sign: {%my SQL expression}.

Summary and group name fields look like function calls. However, they are really shorthand notation for a report field.

- Sum summary field: Sum({Orders.Order Amount}, {Orders.Ship Via}).
- Group name field: GroupName({Orders.Ship Via}).

27.4.2 Sample formulas using fields (Crystal syntax)

The formula in this example uses the Xtreme database. To find out how many days it takes to ship the product from the date when the order was placed, you can just subtract the ship date database field from the order date database field:

```
//A formula that uses database fields  
{Orders.Ship Date} - {Orders.Order Date}
```

To find the total dollar amount of a given product that was ordered, multiply its unit price by the quantity ordered:

```
{Orders Detail.Unit Price} * {Orders Detail.Quantity}
```

To calculate a sale price of 80 percent of the original unit price:

```
{Orders Detail.Unit Price} * 0.80
```

27.5 Expressions (Crystal syntax)

An expression is any combination of keywords, operators, functions, and constant values that result in a value of a given type. For example:

```
//An expression that evaluates to the Number value 25  
10 + 20 - 5  
//An expression that evaluates to the String value  
/"This is a string."  
"This is a string."
```

A Crystal syntax formula consists of a sequence of expressions. The value of the final expression is the value returned by the formula and what gets printed. Each expression must be separated from the previous expression by a semicolon (;).

27.5.1 Multiple expressions (Crystal syntax)

Typically, each expression takes one line, but you can continue an expression onto the next line if you need more space.

The formula below consists of five expressions. It returns the Number value 25 since that is the value of the last expression in the formula.

Example

```
//Expressions example
```



```
//The first expression. Its value is the Number
//value 30
10 + 20;
//The second expression. Its value is the String
//"Hello World". It takes up two lines.
"Hello " +
"World";
//The third expression. Its value is of Number type
{Orders Detail.Quantity} * 2 - 5;
//The fourth expression. Its value is of String type
If {Orders Detail.Quantity} > 1 Then
    "multiple units"
Else
    "one unit";
//The fifth and final expression. Its value is the
//Number value 25
20 + 5
```

Placing a semicolon after the last expression in the formula is also allowed, but is optional. For example, the above formula could have ended with:

```
20 + 5;
```

Some of the sample formulas in the [Expressions \(Crystal syntax\) \[page 794\]](#) section do not have semicolons. This is because they consist of a single expression, and a semicolon is optional after the last expression. Many formulas in Crystal syntax can be written as a single expression.

Notice that there is no semicolon after the "multiple units" string. In fact, if you put a semicolon there, the program will report an error. This is because a semicolon separates expressions, and the

```
Else
    "one unit";
```

is not a separate expression. It does not stand alone apart from the If. In fact, it is an integral part of the If expression since it describes the value that the If expression will return under certain circumstances.

Note

The example is not a practical example because the first four expressions in the formula did not have any effect on the last expression.

27.5.2 How earlier expressions affect later expressions (Crystal syntax)

The fact that a Crystal syntax formula is a sequence of expressions whose result is the value of the final expression is the most important concept in understanding Crystal syntax. This expression-based syntax allows you to write very short formulas with a lot of functionality.

Example

```
//First expression. It declares the Number variable x
```

```
//and then returns the value of an uninitialized
//Number variable, which is 0.
NumberVar x;
//Second expression. It assigns the value of 30 to x,
//and returns 30.
x := 30
```

The above formula would give an error if the first expression were omitted. This is because the second expression refers to the Number variable x, and the program needs to have x declared before it understands expressions involving it.

In general, you use variables to get the earlier expressions in a formula to affect the final expression. See [Variables \(Crystal syntax\) \[page 802\]](#) for more information.

27.5.3 Using the If expression (Crystal syntax)

The *If* expression is one of the most widely used features of Crystal syntax. It also provides insight into the nature of expressions. Consider the earlier If expression as a separate formula. Notice that because this formula is a single expression, it does not need a semicolon:

```
If {Orders Detail.Quantity} > 1 Then
    "multiple units"
Else
    "one unit"
```

Suppose you wanted to modify this formula so that it either prints "multiple units" or the number 1.

```
//An erroneous formula
If {Orders Detail.Quantity} > 1 Then
    "multiple units"
Else
    1
```

This formula will result in an error. This is because the values in this expression are different types: "multiple units" is a String value and 1 is a Number value. Crystal Reports requires that the value of an expression always be of a single type.

Note

This example can be corrected by using the CStr function to convert the Number 1 to a String value. For example, the Number 1 is converted to the string "1" by calling CStr (1, 0).

```
//A correct formula
If {Orders Detail.Quantity} > 1 Then
    "multiple units"
Else
    CStr (1, 0) //Use 0 decimals
```

See [Control structures \(Crystal syntax\) \[page 819\]](#) for more information on the If expression.

27.6 Assignment (Crystal syntax)

The assignment operator is a colon followed by an equals sign (:=).

Example

```
//Assign the Number value of 10 to the variable x
x := 10;
//Assign the String value of "hello" to the
//variable named greeting
greeting := "hello";
```

The equality operator (=) is used to check when two values are equal. A common error is to use the equality operator when the assignment operator is actually intended. This can generate a mysterious error message or no error message at all since it is often syntactically correct to use the equality operator. For example:

```
greeting = "hello";
```

The above formula checks if the value held by the variable greeting is equal to the value "hello". If it is, then the expression's value is True, and if it is not then the expression's value is False. In any case, it is a perfectly correct Crystal syntax expression (assuming that the greeting is a String variable).

27.7 Simple data types (Crystal syntax)

The simple data types in Crystal Reports are Number, Currency, String, Boolean, Date, Time, and DateTime.

27.7.1 Number

Enter numbers without any comma separators or currency symbols (generally you would want to have formatted numbers appearing as the result of a formula and not in the formula itself).

Example

```
10000
-20
1.23
```

27.7.2 Currency (Crystal syntax)

Use the dollar sign (\$) to create a Currency value.

Example

```
$10000  
-$20  
$1.23
```

You can also use the CCur function. The initial C in CCur stands for convert and it can be used to convert Number values to Currency values:

```
CCur (10000)  
CCur (-20)  
CCur (1.23)
```

27.7.3 String (Crystal syntax)

Strings are used to hold text. The text must be placed between double quotation marks (") or apostrophes (') and cannot be split between lines. If you want to include double quotation marks in a string delimited by double quotation marks, use two consecutive double quotation marks. Similarly, if you want to include an apostrophe in a string delimited by apostrophes, use two consecutive apostrophes.

Example

```
"This is a string."  
"123"  
"The word ""hello"" is quoted."  
'This is also a string.'  
'123'  
'Last Year''s Sales'
```

If you use double quotes for the left side of the string, you must use double quotes on the right side. Similarly for apostrophes. The following example is incorrect:

```
'Not a valid string.'
```

You can extract individual elements or substrings from a string by specifying the character position or a range of character positions. Negative values are allowed; they specify the position starting from the end of the string.

```
"hello" [2] //Equal to "e"  
"hello" [-5] //Equal to "h"  
"604-555-1234" [1 to 3] //Equal to "604"  
"abcdef" [-3 to -1] //Equal to "def"
```

You can also extract substrings from a string using the Left, Right and Mid functions.

27.7.4 Boolean (Crystal syntax)

The valid Boolean values are:

```
True  
False
```

Note

Yes can be used instead of True and *No* instead of False.

27.7.5 Date, Time, and DateTime (Crystal syntax)

The DateTime type can hold date-times, dates only, or times only and thus is rather versatile. The Date type holds dates only and the Time type holds times only. The Date and Time types are more efficient than the DateTime type, and so can be used in situations where the added functionality and flexibility of the DateTime type is not needed.

You can create DateTime values directly using the date-time literal construction, which is formed by typing in the date-time between two pound (#) signs. Many different formats are supported.

Note

Date-time literals cannot be split between lines.

Examples

```
#8/6/1976 1:20 am#  
#August 6, 1976#  
#6 Aug 1976 13:20:19#  
#6 Aug 1976 1:30:15 pm#  
#8/6/1976#  
#10:20 am#
```

Even though #10:20 am# looks like it could have the Time type and #8/6/1976# looks like it could have the Date type, they do not. They both have the DateTime type, as do all date-time literals. For example, you can think of #10:20 am# as a DateTime value with a null date part. To convert it to the Time type use CTime (#10:20 am#).

Instead of using date-time literals, you can use CDateTime to convert a String to a DateTime. For example,

```
CDateTime ("8/6/1976 1:20 am")  
CDateTime ("10:20 am")
```

However, there is one key difference between using date-time literals and the above usage of CDateTime. Date-time literals always use U.S. English date formats rather than settings from the locale of the particular computer on which Crystal Reports is running. Thus, the date-time literal examples above would work on all computers. On the other hand, on a French system, you could use constructions like:

```
CDateTime ("22 aout 1997") //Same as #Aug 22, 1997#
```

Date values can be constructed with CDate and Time values with CTime:

```
CDate ("Aug 6, 1969")  
CDate (1969, 8, 6) //Specify the year, month, day  
//Converts the DateTime argument to a Date  
CDate (#Aug 6, 1969#)  
CTime ("10:30 am")  
CTime (10, 30, 0) //Specify the hour, minute, second  
CTime (#10:30 am#)
```

27.8 Range data types (Crystal syntax)

Ranges are designed to handle a spectrum of values. Range types are available for all the simple types except for Boolean. That is: Number Range, Currency Range, String Range, Date Range, Time Range and DateTime Range. You can generate ranges using the To, _To, To_, _To_, UpTo, UpTo_, UpFrom, and UpFrom_ keywords. In general, To is used for ranges with two endpoints, and UpTo and UpFrom are used for open ended ranges (only one endpoint). The underscores are used to indicate whether or not the endpoints are in the range.

Examples of Number Range values

The range of numbers from 2 to 5 including both 2 and 5

```
2 To 5
```

The range of numbers from 2 to 5, not including 2 but including 5

```
2 _To 5
```

All numbers less than or equal to 5

```
UpTo 5
```

All number less than 5

```
UpTo_ 5
```

Examples of DateTime Range values:

```
#Jan 5, 1999# To #Dec 12, 2000#  
UpFrom #Jan 1, 2000#
```

27.8.1 Using ranges in formulas (Crystal syntax)

There are twenty-seven functions in Crystal Reports that specify date ranges. For example, the function `LastFullMonth` specifies a range of date values that includes all dates from the first to last day of the previous month. So if today's date is September 15, 1999 then `LastFullMonth` is the same as the range value `CDate (#Aug 1, 1999#) To CDate (#Aug 31, 1999#)`.

Ranges are often used with `If` or `Select` expressions. The following example computes student letter grades based on their test scores. Scores greater than or equal to 90 receive an "A", scores from 80 to 90, not including 90, receive a "B" and so on.

```
//Compute student letter grades  
Select {Student.Test Scores}  
  Case UpFrom 90 :  
    "A"  
  Case 80 To_ 90 :  
    "B"  
  Case 70 To_ 80 :  
    "C"  
  Case 60 To_ 70 :  
    "D"  
  Default :  
    "F";
```

The above example uses the `Select` expression which is discussed in more detail in the control structures section. You can check if a value is in a range by using the `In` operator. For example:

```
5 In 2 To 10; //True  
5 In 2 To_ 5; //False  
5 In 2 To 5; //True
```

The `Maximum` and `Minimum` functions can be used to find the endpoints of a range:

```
Maximum (2 To 10) //Returns 10
```

27.9 Array data types (Crystal syntax)

Arrays in Crystal Reports are ordered lists of values that are all of the same type. These values are known as the array's elements. The elements of an array can be any simple type or range type. Arrays can be created using square brackets (`[ ]`).

Examples

An array of three Number values. The first element is 10, the second is 5, and the third is 20.

```
[10, 5, 20]
```

An array of seven String values:

```
["Sun", "Mon", "Tue", "Wed", "Th", "Fri", "Sat"]
```

An array of two DateTime Range values:

```
[#Jan 1, 1998# To #Jan 31, 1998#,  
#Feb 1, 1999# To #Feb 28, 1999#]
```

You can extract individual elements out of an array using square brackets containing the index of the element you want. This is called subscripting the array:

```
[10, 5, 20] [2] //Equal to 5
```

Number ranges can also be used to subscript arrays. The result is another array. For example:

```
[10, 5, 20] [2 To 3] //Equal to [5, 20]
```

Arrays are most useful when used with variables and so will be discussed in more detail in the [Variables \(Crystal syntax\) \[page 802\]](#) section. Using variables, you can change the individual elements of an array and resize the array to accommodate more elements. For example, you can accumulate database field values into a global array variable in a detail level formula, and then use a formula in a group footer to perform a calculation based on those values. This enables you to perform a wide variety of customized summary operations.

27.10 Variables (Crystal syntax)

This section describes the key components of variables and shows you how to create variables and assign values to them.

27.10.1 Variables overview

A variable represents a specific data item, or value, and acts as a placeholder for that value. When a formula encounters a variable, the formula searches for the value of the variable and uses it in the formula. Unlike a constant value, which is fixed and unchanging, a variable can be repeatedly assigned different values. You assign a value to a variable and the variable maintains the value until you later assign a new value. Because of this flexibility, it is necessary for you to declare variables before you use them so that Crystal Reports is aware of them and understands how you intend to use them.

Example

If you wanted to report on customers by area code, you could create a variable that extracts the area code from a customer fax number. The following is an example of a variable called `<areaCode>`:

```
Local StringVar areaCode;  
areaCode := {Customer.Fax} [1 To 3];
```

The first line of the variable example is the variable declaration; it gives the variable a name and type. The database field {Customer.Fax} is a String field and [1 To 3] extracts the first three characters from its current value. The variable areaCode is then assigned this value.

27.10.2 Variable declarations (Crystal syntax)

Before using a variable in a formula, you must declare it.

A variable can hold values of a given type. The allowed types are the seven simple types (Number, Currency, String, Boolean, Date, Time and DateTime), the six range types (Number Range, Currency Range, String Range, Date Range, Time Range and DateTime Range) and variables that hold arrays of the previously mentioned types. This gives a total of 26 different types that a variable can have.

When you declare a variable, you also specify its name. A variable cannot have the same name as any function, operator or other keyword that is valid for Crystal syntax. For example, your variable cannot be named Sin, Mod or If because Sin is a built in function, Mod is a built in operator and If is a built in keyword. When typing formulas in the Formula Workshop, the names of the built in functions, operators and other keywords are highlighted in a different color and so it is easy to check if the variable name conflicts.

Once a variable is declared, it can be used in the formula. For example, you might want to assign it an initial value:

```
Local NumberVar x; //Declare x to be a Number variable  
x := 10; //Assign the value of 10 to x
```

Note

The keyword for declaring the Number variable has a Var at the end. This is true for all the variable types in Crystal syntax.

A variable can only hold values of one type. For example, if a variable holds a Number value, you cannot later use it to hold a String.

Example

```
Local StringVar y;  
y := "hello";  
//OK- the Length function expects a String argument  
Length (y);  
//Error- y can only hold String values
```

```
y := #Jan 5, 1993#;
//Error- y can only hold String values
y := ["a", "bb", "ccc"];
//Error- the Sin function expects a Number argument
Sin (y);
```

You can combine declaring a variable and assigning it a value in a single expression. For example:

```
Local NumberVar x := 10 + 20;
Local StringVar y := "Hello" + " " + "World";
Local DateVar z := CDate (#Sept 20, 1999#);
Local NumberVar Range gradeA := 90 To 100;
```

This is a good practice because it is more efficient and helps prevent the common mistake of having incorrectly initialized variables.

Here are some more examples of declaring and initializing range variables:

```
Local NumberVar Range gradeA;
Local DateVar Range quarter;
gradeA := 90 To 100;
quarter := CDate (1999, 10, 1) To CDate (1999, 12, 31);
```

27.10.3 Variable Scope (Crystal syntax)

Variable scopes are used to define the degree to which variables in one formula are made available to other formulas. There are three levels of scope in Crystal Reports: local, global and shared. Every variable has a scope, and this scope is specified when the variable is declared.

27.10.3.1 Local Variables

Variables with local scope, also known as local variables, are declared using the Local keyword followed by the type name (with the Var suffix) followed by the variable name as in the above examples.

Local variables are restricted to a single formula and a single evaluation of that formula. This means that you cannot access the value of a local variable in one formula from a different formula.

Example

```
//Formula A
Local NumberVar x;
x := 10;
//Formula B
EvaluateAfter ({@Formula A})
Local NumberVar x;
x := x + 1;
```

The function call EvaluateAfter ({@Formula A}) ensures that Formula B will be evaluated after Formula A is evaluated. Formula A returns a value of 10 and Formula B returns a value of 1. Formula B does not have access

to Formula A's x and thus cannot use the value of 10 and add 1; instead, it uses the default value for the uninitialized local variable x found in Formula B, which is 0, and adds 1 to it to get 1.

You can also create local variables with the same name but different types in different formulas. For example, the type declarations in formulas A and B do not conflict with:

```
//Formula C  
Local StringVar x := "hello";
```

Local variables are the most efficient of the three scopes. They also do not interfere with one another in different formulas. For these reasons, it is best to declare variables to be local whenever possible.

27.10.3.2 Global variables (Crystal syntax)

Global variables use the same memory block to store a value throughout the main report. This value is then available to all formulas that declare the variable, except for those in subreports. You declare a global variable as in the following example:

```
Global StringVar y;
```

You can also omit the Global keyword which creates a Global variable by default:

```
StringVar y; //Same as: Global StringVar y;
```

However, even though global variables are easy to declare, it is recommended that you use them only when local variables do not suffice.

Since global variables share their values throughout the main report, you cannot declare a global variable in one formula with one type and then declare a global variable with the same name in a different formula with a different type.

Example

```
//Formula A  
Global DateVar z;  
z := CDate (1999, 9, 18)  
//Formula B  
NumberVar z;  
z := 20
```

In this case, if you enter and save Formula A first, Crystal Reports will return an error when you check or try to save Formula B. This is because the declaration of the Global variable z as a Number conflicts with its earlier declaration in Formula A as a Date.

27.10.3.3 Using Global variables

Global variables are often used to perform complex calculations where the results of a formula depend upon the grouping and page layout of the actual printed report. This is accomplished by creating several formulas, placing them in different sections of the report, and having the different formulas interact via global variables.

Example

```
//Formula C
Global NumberVar x;
x := 10;
//Formula D
//Call the function WhileReadingRecords
WhileReadingRecords;
Global NumberVar x;
x := x + 1
```

If Formula C is placed in the Report Header and then Formula D is placed in a detail section, Formula C will be evaluated before Formula D. Formula C will be evaluated once and then Formula D will be evaluated for each record appearing in the detail section. Formula C returns 10. For the first detail record, Formula D returns 11. This is because the value 10 of x is retained from when it was set by Formula C. Formula D then adds 1 to this value, setting x to 11 and then returns 11. For the second detail record, formula D return 12, adding 1 to the previously retained value of x which was 11. This process continues for the remaining detail records.

The call to WhileReadingRecords tells Crystal Reports to re-evaluate Formula D as it reads in each record of the report. Otherwise, since the formula does not contain any database fields, the program evaluates it only once before reading the records from the database. The formula will then return the value 11 instead of 11, 12, 13, ... as the successive records are processed.

If the expression `x := x + 1` is replaced by `x := x + {Orders Detail.Quantity}`, you create the effect of a running total based on `{Orders Detail.Quantity}`, although it is one starting at 10 rather than 0 because of Formula C. In this case, you can omit the call to WhileReadingRecords, since it will automatically occur because the formula contains a database field.

27.10.3.4 Shared variables (Crystal syntax)

Shared variables use the same memory block to store the value of a variable throughout the main report and all of its subreports. Thus shared variables are even more general than global variables. To use a shared variable, declare it in a formula in the main report as in the following example:

```
Shared NumberVar x := 1000;
```

and declare it in a formula in the subreport as in the following example:

```
Shared NumberVar x;
```

In order to use shared variables, the variable must be declared and assigned a value before it can be passed between the main report and the subreport.

27.10.4 Declaring array variables (Crystal syntax)

You can declare array variables by following the type name with the keyword Array.

Example

```
//Declare x to be a Global variable of
//Number Array type
Global NumberVar Array x := [10 , 20, 30];
//cost is a Global variable of Currency Array type
//It is automatically Global since the scope specifier
//(one of Local, Global or Shared) is omitted.
CurrencyVar Array cost := [$19.95, $79.50, $110.00,
                           $44.79, $223.99];
//payDays is a Global variable of Date Array type
Global DateVar Array payDays := [CDate(1999, 5, 15),
CDate(1999, 5, 31)];
//y is a Shared variable of String Range Array type
Shared StringVar Range Array y := ["A" To "C",
                                   "H" To "J"];
//days is a Local variable of String Array type
Local StringVar Array days;
days := ["Sun", "Mon", "Tue", "Wed", "Th",
         "Fri", "Sat"];
```

27.10.4.1 Using array variables (Crystal syntax)

You can assign values to elements of an array and also use the values of the elements for other computations.

Example

```
StringVar Array x := ["hello", "bye", "again"];
x [2] := "once"; //Now x is ["hello", "once", "again"]
//The expression below would cause an error if not
//commented out since the array has size 3
//x [4] := "zap";
//The formula returns the String "HELLO"
UpperCase (x [1])
```

The Redim and Redim Preserve keywords can be used to resize an array if you want to add extra information to it. Redim erases the previous contents of the array first before resizing it whereas Redim Preserve preserves the previous contents.

```
Local NumberVar Array x;
Redim x [2]; //Now x is [0, 0]
x [2] := 20; //Now x is [0, 20]
Redim x [3]; //Now x is [0, 0, 0]
x [3] := 30; //Now x is [0, 0, 30]
Redim Preserve x [4]; //Now x is [0, 0, 30, 0]
```

```
"finished"
Local StringVar Array a;
Redim a [2];
//Assign a value to the first element of the array a
a[1] := "good";
a[2] := "bye";
//The & operator can be used to concatenate strings
a[1] & a[2] //The formula returns the String "goodbye"
```

27.10.4.2 Using Arrays with For loops

Arrays are commonly used with For loops. The following example creates and then uses the Array [10, 20, 30, ..., 100] using a For loop. See [For loops \(Crystal syntax\) \[page 825\]](#) for more details.

```
Local NumberVar Array b;
Redim b[10];
Local NumberVar i;
For i := 1 To 10 Do
(
    b[i] := 10 * i
);
b [2] //The formula returns the Number 20
```

27.10.5 Default values for the simple types (Crystal syntax)

An uninitialized variable will have the default value for its type. In general, it is not a good programming practice to rely on the default values of types. For example, initialize all local variables in your formula, initialize all global variables in a formula placed in the Report Header and initialize all shared variables in a formula placed in the Report Header of the main report.

When an array is resized using the Redim keyword, the entries are filled with default values for the type. It is useful to know about default values when using If and Select expressions.

27.10.5.1 Default values

Number

0

Currency

\$0

String

```
"" //The empty string
```

Date

```
Date (0, 0, 0) //The null Date value
```

Time

The null Time value. Value held by an uninitialized Time variable.

DateTime

The null DateTime value. Value held by an uninitialized DateTime variable.

Note

It is not recommended that your formulas rely on the values of uninitialized range or array variables.

27.10.6 Automatic type conversions (Crystal syntax)

Generally in Crystal Reports, values of one type cannot be used where values of another type are expected without explicitly supplying a type conversion function. For example:

```
Local StringVar postalCode;  
//Error- assigning a Number value to a String  
postalCode := 10025;  
//OK - use the type conversion function CStr  
//to create "10025"  
postalCode := CStr (10025, 0);
```

However, there are a few conversions that are made automatically:

- Number to Currency.
- Date to DateTime.
- Simple type to Range value of the same underlying simple type.

For example, the following assignments are correct:

```
Local CurrencyVar cost;
```

```
//Same as: cost := $10
cost := 10;
Local DateTimeVar orderDate;
//Same as: orderDate := CDateTime (1999, 9, 23, 0, 0, 0)
orderDate := CDate (1999, 9, 23);
Local NumberVar Range aRange;
//Same as: aRange := 20 To 20
aRange := 20;
Local NumberVar Range Array aRangeArray;
//Same as : aRangeArray := [10 To 10, 20 To 25, 2 To 2]
aRangeArray := [10, 20 To 25, 2];
```

Note

The opposite conversions are not allowed. For example:

```
Local NumberVar num;
num := 5 + $10; //Error
//OK- convert to Number type using the CDbl function
num := CDbl (5 + $10) //Could also use ToNumber
```

5 is converted to \$5 and added to \$10 to make \$15. However, this Currency value cannot be automatically assigned to the Number variable num since automatic conversions from Currency to Number are not allowed. Similarly, functions accepting a Currency argument can be supplied a Number argument instead, and the Number argument will be converted to a Currency, whereas functions accepting a Number argument cannot be supplied a Currency argument without first explicitly converting the Currency to a Number using CDbl.

27.11 Functions (Crystal syntax)

When using a function in a formula, type the name of the function and supply the arguments required. For example, the Length function requires a String argument and computes the length of the string.

```
Local StringVar x := "hello";
Length (x) //The formula returns the Number 5
```

Supplying arguments of the incorrect type required by the function produces an error. For example, calling Length (3) would produce an error since Length does not accept a Number argument. Functions sometimes can accept different numbers of arguments or types of arguments. For example, the CDate function which could accept a single String argument to form a Date value or three Number values holding the year, month and day respectively and form a Date value from them. See [Date, Time, and DateTime \(Crystal syntax\) \[page 799\]](#).

Example with the Mid function

```
Local StringVar x := "hello";
Local StringVar y;
//Start at position 2, go to the end of the string
y := Mid (x, 2); //y is now "ello"
//Start at position 2, extract 1 character
y := Mid (x, 2, 1) //y is now "e"
```


These classes of functions are: Math, Summary, Financial, String, Date/Time, Date Range, Array, Type Conversion, Programming Shortcuts, Evaluation Time, Print State, Document Properties and Additional Functions. There are also some functions specific to conditional formatting formulas.

→ Tip

The functions that Crystal Reports supports are described in [Functions \[page 1524\]](#).

27.11.1 Non reporting-specific functions (Crystal syntax)

The Math, Financial, String, Date/Time, Type Conversion, and Programming Shortcuts groups consist mainly of functions that are not specific to reporting, but might be found in any full featured programming environment. Many of the functions are similar in functionality to Visual Basic functions of the same or similar name.

📘 Note

Some functions that are supported by Crystal syntax are not listed in the Crystal syntax Functions tree. This is because they are equivalent to Crystal syntax functions already listed in the tree.

For example, the Length function is the traditional Crystal syntax function for finding the length of a string. Crystal syntax also supports Len as a synonym. Len is the Visual Basic and Basic syntax function for this and its inclusion is for the convenience of Visual Basic and Basic syntax users who want to write or modify Crystal syntax formulas.

27.11.2 Summary functions (Crystal syntax)

The Summary function group provides functions for creating summary fields such as:

```
Sum({Orders.Order Amount}, {Orders.Ship Via})
```

Summary fields are normally created using the Insert Summary or Insert Grand Total dialogs. They then appear in the Available Fields tree, and can be used in a formula by double clicking there. However, they do not need to be created in this way. You can create a summary field exclusively for use by your formula by filling in the arguments to one of the functions in the Summary functions section appropriately. However, any groups in the report that the summary field refers to must already exist in the report.

→ Tip

The summary functions that Crystal Reports supports are described in [Summary \[page 1551\]](#).

27.11.3 Date Ranges (Crystal syntax)

This category of functions was discussed in the section [Range data types \(Crystal syntax\) \[page 800\]](#). One additional comment is that the Date ranges produced by these functions depend on the current date. For example, if today's date is September 18, 1999, then LastFullMonth is the Date Range value:

```
CDate(#Aug 1, 1999#) To CDate(#Aug 31, 1999#)
```

This functionality is often useful, but what if you want to determine a date range based on a database field such as {Orders.Order Date}? The Date/Time functions can be used instead.

For example:

```
Local DateVar d := CDate ({Orders.Order Date});  
Local DateVar Range dr;  
dr := DateSerial (Year(d), Month(d) - 1, 1) To  
      DateSerial (Year(d), Month(d), 1 - 1);  
//At this point dr is the Date Range value holding  
//the last full month before {Orders.Order Date}
```

The DateSerial function makes this easy because you don't have to worry about special cases. It never lets you create an invalid date. For example, DateSerial (1999, 1 - 1, 1) is December 1, 1998.

Note

In the above example, {Orders.Order Date} is actually a DateTime field and so the CDate function is used to convert it to a date by truncating the time part.

Related Information

[Date Ranges \[page 1801\]](#)

27.11.4 Array functions (Crystal syntax)

The array functions compute summaries of an array's elements. For example, the Sum function when applied to an array returns the sum of the elements of the array. For example, the following formula returns 100:

```
Sum ([10, 20, 30, 40])
```

Related Information

[Arrays \[page 1820\]](#)

27.11.5 Evaluation Time functions (Crystal syntax)

These are the reporting specific functions BeforeReadingRecords, WhileReadingRecords, WhilePrintingRecords and EvaluateAfter. You can use these functions to guide Crystal Reports as to when your formula should be evaluated.

Should the formula be evaluated before retrieving the records from the database, while reading the records from the database but before the records have been grouped, sorted and summarized, or while printing the report, when the records are grouped, sorted and summarized? In general, Crystal Reports sets an appropriate evaluation time for your formula, based on how much information the formula needs. For example, if a formula uses a database field, then it cannot be evaluated before the records are read from the database. However, you sometimes need to force a later evaluation time than normal to get the desired effect. See [Global variables \(Crystal syntax\) \[page 805\]](#) for an example.

Related Information

[Evaluation Time \[page 1851\]](#)

27.11.6 Print State functions (Crystal syntax)

These are once again reporting specific functions. For example, the notation {Orders.Order Date} is used to refers to the value of the field in the current record whereas Previous ({Orders.Order Date}) refers to the value in the immediately preceding record and Next ({Orders.Order Date}) in the next record. IsNull ({Orders.Order Date}) checks if the field's value is null.

Other examples are PageNumber and TotalPageCount that you can use to access pagination information about your report.

Related Information

[Print State \[page 1855\]](#)

27.11.7 Document Properties functions

These are specific functions, such as PrintDate and ReportTitle, that refer to the report document as a whole.

Related Information

[Document Properties \[page 1876\]](#)

27.11.8 Additional functions (Crystal syntax)

These are functions that are in User Function Libraries or UFLs. A UFL is a separate dynamic link library or Automation server that you create and that Crystal Reports can use to add your own customized functions to the formula language. Writing a UFL is more involved than writing a formula using Basic syntax or Crystal syntax. See Crystal Reports Developer's Help for details.

Note

Using UFLs makes your reports less portable because you must distribute your UFL along with the report.

Related Information

[Additional Functions \[page 1899\]](#)

27.11.9 Conditional formatting functions (Crystal syntax)

When writing a conditional formatting formula, certain additional functions appear at the top of the Functions tree to help you with this. For example, you can format the {Customer.Last Year's Sales} field to print sales of more than \$100,000 in green, sales of less than \$15,000 in red, and all other sales in black.

Example

```
//Conditional formatting example 1
If {Customer.Last Year's Sales} > 100000 Then
    crGreen
Else If {Customer.Last Year's Sales} < 15000 Then
    crRed
Else
    crBlack
```

Since this is a font color formatting function, the list of Color Constants appears in the Functions Tree. This example uses three: crGreen, crRed and crBlack. You could have used the actual numeric values of the color constants instead. For example, crRed is 255 and crGreen is 32768. However, the formula is more understandable using the color constants. All constant functions in Crystal syntax can have the "cr" prefix.

Crystal syntax still supports constant functions from previous versions without the "cr" prefix. For example, you can use "Red" instead of "crRed". However, using the "cr" prefix organizes constant functions and is recommended.

Note

Some formatting attributes do not use constant functions. For example, if you wanted to not print {Customer.Last Year's Sales} values if the sales were less than \$50,000, you could write the following conditional formatting formula for the suppress attribute:

```
//Conditional formatting example 2
If {Customer.Last Year's Sales} < 50000 Then
    True //suppress the value
Else
    False //do not suppress the value
```

Or more simply:

```
//Conditional formatting example 3 -
//equivalent to example 2
{Customer.Last Year's Sales} < 50000
```

If the last year's sales are less than \$50,000, then the expression

```
{Customer.Last Year's Sales} < 50000
```

is True, and so the formula returns True. On the other hand, if the last year's sales are greater than or equal to \$50,000, then

```
{Customer.Last Year's Sales} < 50000
```

is False and so the formula returns False.

27.11.10 General purpose conditional formatting functions (Crystal syntax)

There are three general purpose conditional formatting functions:

- CurrentFieldValue
- DefaultAttribute
- GridRowColumnValue

These functions are displayed at the top of the Functions tree whenever appropriate. DefaultAttribute can be used for any formatting formula, CurrentFieldValue for any formatting formula where you are formatting a field value, and GridRowColumnValue for any formatting formula where you are formatting a field value in a Cross-Tab or OLAP grid.

In particular, CurrentFieldValue allows you to conditionally format the cells of a Cross-Tab or OLAP grid based on their value while GridRowColumnValue lets you conditionally format the cells of a Cross-Tab or OLAP grid based on the values of the row or column headings. These two functions are essential in this situation since

there is no other way in the formula language to refer to the values of these fields. For example, if you wanted Cross-Tab cells to be suppressed if the values are less than 50,000:

```
//Conditional formatting example 4  
CurrentFieldValue < 50000
```

→ Tip

The conditional formatting functions that Crystal Reports supports are described in [Conditional formatting functions \[page 1956\]](#).

27.12 Operators (Crystal syntax)

Click a link to view one of the topics in this section:

Related Information

[Arithmetic operators \[page 816\]](#)

[Comparison operators \(Crystal syntax\) \[page 817\]](#)

[Boolean operators \(Crystal syntax\) \[page 818\]](#)

[Null fields and how to use IsNull \(Crystal syntax\) \[page 818\]](#)

27.12.1 Arithmetic operators

The arithmetic operators are addition (+), subtraction (-), multiplication (*), division (/), integer division (\), percent (%), modulus (Mod), negation (-) and exponentiation (^). Arithmetic operators are used to combine numbers, numeric variables, numeric fields and numeric functions to get another number.

Examples

```
//Outstanding preferred stock as a percent of  
//common stock  
{Financials.Preferred Stock} %  
{Financials.Common Stock};  
//The square root of 9, Sqr(9) is 3  
//The formula returns 17  
7 + 2 * 3 - 2 + Sqr(6 + 3) * Length("up");
```

→ Tip

The arithmetic operators that Crystal Reports supports are described in [Arithmetic operators \[page 1464\]](#).

27.12.1.1 Order of precedence (Crystal syntax)

When you create arithmetic expressions that involve several operators, the order that the program evaluates the various elements of the expression becomes important. In general, the program evaluates expressions from left to right. However, it also follows the rules of precedence from basic math.

Example

Multiplication and division are performed first from left to right and then addition and subtraction are performed.

For example, $5 + 10 * 3 = 5 + 30 = 35$. You can change this order of precedence by using parentheses. For example, $(5 + 10) * 3 = 15 * 3 = 45$. If you are unsure of the order of precedence, it is a good idea to clarify your intentions with parentheses.

27.12.1.2 List of arithmetic operators, from highest precedence to lowest

- Exponentiation (^)
- Negation (-)
- Multiplication, division, and percent (*, /, %)
- Integer Division (\)
- Modulus (Mod)
- Addition and subtraction (+, -)

27.12.2 Comparison operators (Crystal syntax)

The comparison operators are equal (=), not equal (<>), less than (<), less than or equal (<=), greater than (>) and greater than or equal (>=).

Comparison operators are usually used to compare operands for a condition in a control structure such as an If expression. Comparison operators as a group all have lower precedence than the arithmetic operators. Thus, expressions like $2 + 3 < 2 * 9$ are the same as $(2 + 3) < (2 * 9)$.

Related Information

[Comparison operators \[page 1472\]](#)

27.12.3 Boolean operators (Crystal syntax)

The Boolean operators are, in order of precedence from greatest to lowest: Not, And, Or, Xor, Eqv and Imp. Boolean operators are typically used with comparison operators to generate conditions for control structures. Boolean operators as a group have lower precedence than the comparison operators. Thus for example, the expression `2 < 3 And 4 >= -1` is the same as `(2 < 3) And (4 >= -1)`.

Related Information

[Boolean operators \[page 1494\]](#)

27.12.4 Null fields and how to use IsNull (Crystal syntax)

In general, when Crystal Reports encounters a null valued field in a formula, it immediately stops evaluating the formula and produces no value. If you want to handle null field values in your formula, you must explicitly do so using one of the special functions designed for handling them: `IsNull`, `PreviousIsNull` or `NextIsNull`.

Note

These special functions must be used first in a formula for them to be evaluated properly.

Example

The `{Product.Color}` field contains both basic colors such as "red" and "black" and more descriptive two word colors such as "steel satin" and "jewel green". Here's an example of a formula that writes out "basic" for the basic colors and "fancy" for the others.

```
If InStr({Product.Color}, " ") = 0 Then
    formula = "basic"
Else
    formula = "fancy"
```

The function call to `InStr` searches the `{Product.Color}` string for a space. If it finds a space, it returns the position of the space, otherwise it returns 0. Since basic colors are only one word with no spaces, `InStr` will return 0 for them.

For some products, such as the Guardian Chain Lock, a color value was not recorded and so the `{Product.Color}` field has a null value in the database for that record. Thus, the Guardian Chain Lock record does not have any word printed beside it.

Here is an example of how to fix the above example using `IsNull`:

```
If IsNull({Product.Color}) Or
    InStr({Product.Color}, " ") = 0 Then
    "basic"
```



```
Else  
    "fancy"
```

Relating to operators, when Crystal Reports evaluates the condition:

```
IsNull({Product.Color}) Or  
InStr({Product.Color}, " ") = 0
```

It first evaluates `IsNull ({Product.Color})`, and when it determines that this is `True`, it knows that the whole condition is `True`, and does not need to check whether

```
InStr({Product.Color}, " ") = 0
```

In other words, Crystal Reports will stop evaluating a Boolean expression when it can predict the results of the whole expression. In the following example, the formula guards against attempting to divide by zero in the case that `denom` is 0:

```
Local NumberVar num;  
Local NumberVar denom;  
...  
If denom <> 0 And num / denom > 5 Then  
...
```

Note

Visual Basic does not support this technique, since all parts of a Boolean expression in Visual Basic are evaluated, even if not necessary.

27.13 Control structures (Crystal syntax)

Formulas without control structures execute each expression in the formula exactly once when the formula is evaluated. The expressions are executed in a sequential fashion, from the first expression in the formula to the last. Control structures enable you to vary this rigid sequence. Depending upon which control structure you choose, you can skip over some of the expressions or repeatedly evaluate some expressions depending on if certain conditions hold. Control structures are the primary means of expressing business logic and typical report formulas make extensive use of them.

Related Information

[Control structure operators \[page 1506\]](#)

27.13.1 If expressions (Crystal syntax)

The `If` expression is one of the most useful control structures. It allows you to evaluate an expression if a condition is true and evaluate a different expression otherwise.

Note

The correct Crystal syntax for If statements is `if <condition> then <then> else <else>`, where `<condition>`, `<then>`, and `<else>` are all single expressions. If you have multiple expressions after the `<then>` or `<else>`, convert them into single expressions by surrounding them in parentheses. For example:

```
Global stringVar lastValue;
if {Branch_View.Branch_ID} = lastValue
then
    (lastValue := {Branch_View.Branch_ID};
    crRed;)
else
    (lastValue := {Branch_View.Branch_ID};
    crBlack;)
```

Note

When formatting with conditional formulas, always include the Else keyword; otherwise, values that don't meet the If condition may not retain their original format. To prevent this, use the DefaultAttribute function (`If...Else DefaultAttribute`).

Note

When creating record selection formulas that include If expressions, always include the Else keyword; otherwise, no records or unexpected records may be returned. For example, a record selection such as `If {parameter field} = "less than 100" then {field} < 100` evaluates as False and returns no records. To correct this problem, complete the formula with `Else True`.

Example

A company plans to pay a bonus of 4 percent to its employees except for those who work in Sales who will receive 6 percent. The following formula using an If expression would accomplish this:

```
//If example 1
If {Employee.Dept} = "Sales" Then
    {Employee.Salary} * 0.06
Else
    {Employee.Salary} * 0.04
```

In this example, if the condition `{Employee.Dept} = "Sales"` evaluates as true, then the

```
{Employee.Salary} * 0.06
```

expression is processed. Otherwise the expression following the Else, namely the

```
{Employee.Salary} * 0.04
```

is processed.

Suppose another company wants to give employees a 4% bonus, but with a minimum bonus of \$1,000. The following example shows how. Notice that the Else clause is not included; it is optional, and not needed in this case.

```
//If example 2
Local CurrencyVar bonus := {Employee.Salary} * 0.04;
If bonus < 1000 Then
    bonus := 1000;
//The final expression is just the variable 'bonus'.
//This returns the value of the variable and is the
//result of the formula
bonus
```

Another way of accomplishing example 2 is to use an Else clause:

```
//If example 3
Local CurrencyVar bonus := {Employee.Salary} * 0.04;
If bonus < 1000 Then
    1000
Else
    bonus
```

Now suppose that the previous company also wants a maximum bonus of \$5,000. You now need to use an Else If clause. The following example has only one Else If clause, but you can add as many as you need. Note, however, that there is a maximum of one Else clause per If expression. The Else clause is executed if none of the If or Else If conditions are true.

```
//If example 4
Local CurrencyVar bonus := {Employee.Salary} * 0.04;
If bonus < 1000 Then
    1000
Else If bonus > 5000 Then
    5000
Else
    bonus
```

27.13.1.1 If example (Crystal syntax)

Suppose that a company wants to compute an estimate of the amount of tax an employee needs to pay and write a suitable message. Income below \$8,000 is not taxed, income from \$8,000 to \$20,000 is taxed at 20% income from \$20,000 to \$35,000 is taxed at 29% and income above \$35,000 is taxed at 40%.

```
//If example 5
Local CurrencyVar tax := 0;
Local CurrencyVar income := {Employee.Salary};
Local StringVar message := "";
If income < 8000 Then
(
    message := "no";
    tax := 0
)
Else If income >= 8000 And income < 20000 Then
(
    message := "lowest";
    tax := (income - 8000)*0.20
)
Else If income >= 20000 And income < 35000 Then
(
```

```

    message := "middle";
    tax := (20000 - 8000)*0.20 + (income - 20000)*0.29
)
Else
(
    message := "highest";
    tax := (20000 - 8000)*0.20 + (35000 - 20000)*0.29 +
        (income - 35000)*0.40
);
//Use 2 decimal places and the comma as a
//thousands separator
Local StringVar taxStr := CStr (tax, 2, ",");
"You are in the " & message & " tax bracket. " &
"Your estimated tax is " & taxStr & "."

```

Note

The use of variables is to simplify the logic of the computation. Also, there are two expressions that are executed when one of the conditions are met; one assigns the tax variable, and the other assigns the message variable. It is often useful to have multiple expressions executed as a result of a condition.

27.13.1.2 More details on If expressions (Crystal syntax)

The If expression is an expression. In other words it evaluates to a value of a given type. If there is no Else clause, and the condition is not true, then the value is the default value for the type. For example:

```

If Length ({Employee.First Name}) < 5 Then
    "short"

```

The above If expression returns a String value. The string value is "short" if the Employee's first name has fewer than 5 letters and the empty String "" otherwise.

Consider the formula:

```

If Year({Orders.Order Date}) >= 1995 Then
    {Orders.Order Date}

```

For order dates before 1995, the above If expression returns the null DateTime value. It is a DateTime value rather than a Date value since {Orders.Order Date} is a DateTime database field. The null DateTime value is not printed by Crystal Reports so if the above formula is placed in a report, the formula field would be blank for order dates before 1995. Null Time values and null Date values behave similarly.

Here is an example that illustrates the use of parentheses to have more than one expression executed as the outcome of an If condition. A company charges a 5 percent fee for orders shipped within three days and a 2 percent fee otherwise. It wants to print messages such as "Rush shipping is \$100.00" or "Regular shipping is \$20.00" as appropriate.

```

Local StringVar message;
Local CurrencyVar ship;
If {Orders.Ship Date} - {Orders.Order Date} <= 3 Then
(
    message := "Rush";
    //A semicolon at the end of the next line
    //is optional
    ship := {Orders.Order Amount} * 0.05
) //A semicolon cannot be placed here

```

```

Else
(
    message := "Regular";
    ship := {Orders.Order Amount} * 0.02;
);
//The preceding semicolon is required to separate the
//If expression from the final expression below
message & " shipping is " & CStr (ship)

```

When expressions are grouped together with parentheses, the whole group is considered as a single expression, and its value and type are the value and type of the final expression inside the parentheses.

```

//The parentheses group expression as a whole has
//Currency type
(
    //The first expression in the parentheses has
    //String type
    message := "Rush";
    //The second and final expression in parentheses
    //has Currency type
    ship := {Orders.Order Amount} * 0.05;
)

```

Thus, for example, the following formula gives an error. The reason is that the Then part of the If expression returns a Currency value while the Else part returns a String value. This is not allowed, since the If expression is an expression and so must always return a value of a single type.

```

//An erroneous formula
Local StringVar message;
Local CurrencyVar ship;
If {Orders.Ship Date} - {Orders.Order Date} <= 3 Then
(
    message := "Rush";
    ship := {Orders.Order Amount} * 0.05
)
Else
(
    //The following 2 lines were interchanged
    ship := {Orders.Order Amount} * 0.02;
    message := "Regular";
);
message & " shipping is " & CStr (ship)

```

One way of fixing up the erroneous formula without being careful about expression order is just to make the If expression return a constant value of the same type in every branch. For example, the If expression now returns the Number value 0:

```

//Repaired the erroneous formula
Local StringVar message;
Local CurrencyVar ship;
If {Orders.Ship Date} - {Orders.Order Date} <= 3 Then
(
    message := "Rush";
    ship := {Orders.Order Amount} * 0.05;
    0
)
Else
(
    ship := {Orders.Order Amount} * 0.02;
    message := "Regular";
    0
);
message & " shipping is " & CStr (ship)

```

27.13.2 Select expressions (Crystal syntax)

The Select expression is similar to an If expression. Sometimes however, you can write clearer and less repetitive formulas using the Select expression. For example, to evaluate the {Customer.Fax} field to determine if the area code is for Washington state (206, 360, 509) or British Columbia, Canada (604, 250):

```
//Select example 1
Select {Customer.Fax}[1 To 3]
  Case "604", "250" :
    "BC"
  Case "206", "509", "360" :
    "WA"
  Default :
    "";
```

The expression right after the Select keyword is called the Select condition. In the above example it is {Customer.Fax}[1 To 3]. The Select expression tries to find the first Case that matches the Select condition, and then executes the expression following the colon for that Case. The Default case is matched if none of the preceding cases match the Select condition. Notice that there is also a colon after the Default.

```
//Same effect as Select example 1
Local StringVar areaCode := {Customer.Fax}[1 To 3];
If areaCode In ["604", "250"] Then
  "BC"
Else If areaCode In ["206", "509", "360"] Then
  "WA"
Else
  "";
```

Example

This formula groups the number of Oscar nominations a movie received into low, medium, high or extreme categories and in the process, shows some of the possibilities for the expression lists following the Case labels:

```
//Select example 2
Select {movie.NOM}
  Case 1,2,3, Is < 1 :
  (
    //Can have expression lists by using
    //parentheses
    10 + 20;
    "low"
  )
  Case 4 To 6, 7, 8, 9 :
    "medium"
  Case 10 :
    "high"
  Default :
    "extreme"
```

The Default clause of the Select expression is optional. If the Default clause is missing and none of the cases are matched, then the Select expression returns the default value for its expression type. For example, if in the above example the Default clause were omitted and {movie.NOM} = 11, it would return the empty string "". The Select expression is an expression, and similar comments as in the [More details on If expressions \(Crystal syntax\) \[page 822\]](#) section apply to it as well.

27.13.3 For loops (Crystal syntax)

For loops enable you to evaluate a sequence of expressions multiple numbers of times. This is unlike the If and Select expressions where the program passes through each expression at most once during the formula's evaluation. For loops are best when you know the number of times that the expressions needs to be evaluated in advance.

27.13.3.1 The syntax of the For loop through examples

Example 1

Suppose you want to reverse the {Customer.Customer Name} string. For example, "City Cyclists" becomes "stsilcyC ytiC".

```
//Reverse a string version 1
Local StringVar str := "";
Local NumberVar strLen :=
    Length ({Customer.Customer Name});
Local NumberVar i;
For i := 1 To strLen Do
(
    Local NumberVar charPos := strLen - i + 1;
    str := str + {Customer.Customer Name}[charPos]
);
str
```

Examine how this formula works assuming that the current value of the field {Customer.Customer Name} is "Clean Air". The variable strLen is assigned to be the length of "Clean Air", namely 9. The variable *i* is known as a For counter variable since its value changes with each iteration of the For loop. In other words, it is used to count the iterations of the loop. The For loop will iterate 9 times, during the first time, i is 1, then i is 2, then i is 3 and so on until finally i equals 9. During the first iteration, the ninth character of {Customer.Customer Name} is appended to the empty string variable str. Thus str equals "r" after the first iteration. During the second iteration, the eighth character of {Customer.Customer Name} is appended to str and so str equals "ri". This continues until after the ninth iteration, str equals, "riA naelC" which is the reversed string.

Example 2

Here is a simpler version of the above formula that uses a Step clause with a negative Step value of -1. For the "Clean Air" example, i is 9 for the first iteration, 8 for the second, 7 for the third and so on until it is 1 in the final iteration.

```
//Reverse a string version 2
Local StringVar str := "";
Local NumberVar strLen :=
    Length ({Customer.Customer Name});
Local NumberVar i;
For i := strLen To 1 Step -1 Do
(
    str := str + {Customer.Customer Name}[i]
```

```
);  
str
```

Example 3

The simplest version is to use the built in function StrReverse:

```
//Reverse a string version 3  
StrReverse ({Customer.Customer Name})
```

The built in String functions in Crystal Reports can handle many of the string processing applications that would traditionally be handled using a For loop or some other kind of loop. However, For loops provide the most flexibility in processing strings and also power in processing arrays, which can be essential if the built-in functions do not cover your intended application.

27.13.3.2 For loop example

Here is a more full featured example of Crystal Reports' string processing capabilities. The Caesar cipher is a simple code that is traditionally credited to Julius Caesar. In this code, each letter of a word is replaced by a letter five characters further in the alphabet. For example, "Jaws" becomes "Ofbx". Notice that "w" is replaced by "b"; since there are not 5 characters after "w" in the alphabet, it starts again from the beginning. Here is a formula that implements applying the Caesar cipher to the field {Customer.Customer Name} in the Xtreme database:

```
//The Caesar cipher  
//The input string to encrypt  
Local StringVar inString := {Customer.Customer Name};  
Local NumberVar shift := 5;  
Local StringVar outString := "";  
Local NumberVar i;  
For i := 1 To Length(inString) Do  
(  
    Local StringVar inC := inString [i];  
    Local StringVar outC;  
    Local BooleanVar isChar :=  
        LowerCase(inC) In "a" To "z";  
    Local BooleanVar isUCaseChar :=  
        isChar And (UpperCase (inC) = inC);  
    inC := LCase(inC);  
    If isChar Then  
    (  
        Local NumberVar offset :=  
            (Asc(inC) + shift - Asc("a")) Mod  
            (Asc("z") - Asc("a") + 1);  
        outC := Chr(offset + Asc("a"));  
        If isUCaseChar Then outC := UpperCase(outC)  
    )  
    Else  
        outC := inC;  
    outString := outString + outC  
);  
outString
```

In the above example there is an If expression nested within the expression block of the For loop. This If expression is responsible for the precise details of shifting a single character. For example, letters are

treated differently from punctuation and spaces. In particular, punctuation and spaces are not encoded. The general points here are that control structures can be nested within other control structures and that multiple expressions can be included in the (parentheses enclosed) expression blocks of other control structures.

27.13.3.3 Using Exit For (Crystal syntax)

You can exit from a For loop by using Exit For. The following example searches the Global array names for the name "Fred". If it finds the name, it returns the index of the name in the array. Otherwise it returns -1.

For example, if the names array is:

```
["Frank", "Helen", "Fred", "Linda"]
```

Then the formula returns 3.

```
Global StringVar Array names;  
//The names array has been initialized and filled  
//in other formulas  
Local NumberVar i;  
Local NumberVar result := -1;  
//The UBound function returns the size of its array  
//argument  
For i := 1 to UBound (names) Do  
(  
    If names [i] = "Fred" Then  
    (  
        result := i;  
        Exit For  
    )  
);  
result
```

When considered as an expression, the For loop always returns the Boolean value True. Thus you will almost never want a For loop to be the last expression in a formula, since then the formula will then just display the value True rather than your intended result.

27.13.4 While Loops (Crystal syntax)

Another looping mechanism is the While loop. A While loop can be used to execute a fixed block of statement an indefinite amount of time.

27.13.4.1 The 2 different types of While loops

Type of While Loop	Explanation	Example
While ... Do	The While ... Do loop evaluates the condition, and if the condition is true, then it evaluates the expression following the Do. When it has finished doing this, it evaluates the condition again and if the condition is true, it evaluates the expression following the Do again. It continues repeating this process until the condition is false.	<pre>While condition Do expression</pre>
Do ... While	The Do ... While loop evaluates the expression once no matter what. It then evaluates the condition, and if the condition is true, evaluates the expression again. This process continues until the condition is false.	<pre>Do expression While condition</pre>

Note

The While loops support an Exit While statement to immediately jump out of the loop. Its use is analogous to the use of Exit For in For loops.

Note

As with the For loop, the While loop when considered as an expression always returns the Boolean value True.

27.13.4.2 While ... Do loop example (Crystal syntax)

The following example searches for the first occurrence of a digit in an input string. If a digit is found, it returns its position, otherwise it returns -1. In this case, the input string is set explicitly to a string constant, but it could be set equal to a String type database field instead. For example, for the input String, "The 7 Dwarves", the formula returns 5, which is the position of the digit 7.

```
Local StringVar inString := "The 7 Dwarves";  
Local NumberVar strLen := Length (inString);  
Local NumberVar result := -1;  
Local NumberVar i := 1;  
While i <= strLen And result = -1 Do  
(  
    Local StringVar c := inString [i];  
    If NumericText (c) Then  
        result := i;
```

```
    i := i + 1;  
);  
result
```

27.13.5 Safety mechanism for loops (Crystal syntax)

There is a safety mechanism to prevent report processing from hanging due to an infinite loop. Any one evaluation of a formula can have at most 100,000 loop condition evaluations per formula evaluation. This will be explained by the example below.

For example:

```
Local NumberVar i := 1;  
While i <= 200000 Do  
(  
    If i > {movie.STARS} Then  
        Exit While;  
    i := i + 1  
);  
20
```

If {movie.STARS} is greater than 100,000 then the loop condition ($i \leq 200000$) will be evaluated more than the maximum number of times and an error message is displayed. Otherwise the loop is OK.

Note

The safety mechanism applies on a per formula base, not for each individual loop. For example:

```
Local NumberVar i := 1;  
For i := 1 To 40000 Do  
(  
    Sin (i);  
);  
i := 1;  
While i <= 70000 Do  
(  
    i := i + 1;  
)
```

The above formula also triggers the safety mechanism since the 100,000 refers to the total number of loop condition evaluations in the formula and this formula will have 40001 + 70001 such evaluations.

27.13.6 Option Loop (Crystal Syntax)

The Option Loop statement can be used to specify the maximum number of loop condition evaluations per evaluation of a formula. This statement should only be used when the default maximum loop count of 100,000 is not sufficient for the given formula.

Note

The Option Loop statement must be used before any other statements.

27.13.6.1 The syntax of the Option Loop

The syntax of the Option Loop is:

```
Option Loop <maxIterations>
```

Where **<maxIterations>** = the maximum number of times you want the condition to be evaluated per evaluation of a formula. It must be a positive integer.

For example:

```
//option loop example
option loop 256;
//make name upper case
Local StringVar outString;
Local StringVar inString := {Customer.Customer Name};
Local NumberVar i;
For i:=1 to Length(inString) do
  outString := outString + upperCase(inString[i]);
outString
```

If Customer.Customer Name is greater than 255, an error message will be shown: "A loop was evaluated more than the maximum number of times allowed."

27.14 Limitations (Crystal syntax)

For reference purposes, here are the sizing limitations of the formula language:

- The maximum length of a String constant, a String value held by a String variable, a String value returned by a function or a String element of a String array is 65,534 characters.
- The maximum size of an array is 1000 elements.
- The maximum number of arguments to a function is 1000. (This applies to functions that can have an indefinite number of arguments such as Choose).
- The maximum number of loop condition evaluations per evaluation of a formula is 100,000. (See [Safety mechanism for loops \(Crystal syntax\) \[page 829\]](#) for the precise meaning of this). Note that you can use the [Option Loop \(Crystal Syntax\) \[page 829\]](#) statement to change this limit.
- Date-time functions modeled on Visual Basic accept dates from year 100 to year 9999. Traditional Crystal Reports functions accept dates from year 1 to year 9999.
- There is no limit on the size of a function.

28 Parameter Fields and Prompts

This section explains what parameter fields are and how they can be applied to create a single report that can be used to access different types of data depending on the user's needs. The section also describes the kinds of parameter prompts that are available in Crystal Reports.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

28.1 Parameter and prompt overview

Parameters are Crystal Reports fields that you can use in a Crystal Reports formula. As a formula component, a parameter must have a value before the program can process the report. By using parameters in formulas, selection formulas, and in the report itself, you can create a single report that changes its behavior depending on the values entered by your users. Parameter fields can also be used in subreports.

Prompts are elements that help users set a value for report parameters. Prompts are different from parameters in several ways:

- Prompts are not used directly by a Crystal Reports formula.
- Prompts include user-interface settings that help you determine the appearance of the prompting dialog box that your users see.
- Prompts include an optional list of values that your users can choose from. This list of values can be a static list that is stored in each report, or a dynamic list that is retrieved from a database (this type of prompt includes multi-level cascading lists that are also retrieved from a database).
- Prompts can be set so that the parameter itself is optional.

When your users select values in the prompting dialog box, they are setting values for prompts. The Crystal Reports prompting engine then assigns that value to the corresponding parameter, which is finally used by the report.

When you create and edit a parameter field, you work with one parameter and one or more prompts.

28.1.1 Parameter field considerations

There are a number of things to keep in mind when working with parameter fields:

- Parameter fields support the following data types:
 - Boolean: Requires a yes/no or true/false answer.
Example: Include planned budget numbers in the summary?
 - Currency: Requires a dollar amount.
Example: Display customers with sales over XXXXX.
 - Date: Requires an answer in a date format.
Example: Enter the start and end dates of the quarter.
 - DateTime: Requires both date and time.
Example: Display statistics for 07/04/1999 between 1:00pm-2:00pm.
 - Number: Requires a numeric value.
Example: Enter the customer identification number.
 - String: Requires a text answer.
Example: Enter the region.
 - Time: Requires an answer using a time format
Example: Display the total number of calls from 1:00pm-2:00pm.
- Parameter field prompting text can be up to four lines long with approximately 60-70 characters per line (depending on character width, up to the 254 character limit). Text over one line in length will automatically word wrap.
- You can create a list of values from which users can choose the parameter value rather than having them enter it manually. Lists of values can be static (embedded in each report), dynamic (refreshed from a database on demand), scheduled (refreshed from a database on a recurring schedule), or partially scheduled (portions of a list of values can be scheduled, while other portions remain on-demand). Scheduled and partially scheduled lists of values are administered through the Business View Manager. For more information, see the *Business Views Administrator's Guide*.
- A parameter field does not have to be placed in a report in order to be used in a record or group selection formula. You create the parameter field and then enter it in your formula as you would any other field.
- A parameter field does not have to be placed in a report in order to be used in a record or group selection formula. You create the parameter field and then enter it in your formula as you would any other field.
- A parameter field can be set to be optional so that users are not forced to enter a value when prompted.

28.1.2 Prompt considerations

There are a number of things to keep in mind when working with prompts:

- Prompts can be static or dynamic. As well, a dynamic prompt can have a cascading list of values. For a description of each option, see these topics:
 - [Creating a parameter with a static prompt \[page 839\]](#).
 - [Creating a parameter with a dynamic prompt \[page 844\]](#).
 - [Creating a parameter with a cascading list of values \[page 846\]](#).
- Dynamic prompts contain lists of values that you can use for a particular report or for many reports. You share a list of values by adding it to the BusinessObjects Enterprise Repository.
- A list of values can be scheduled for automatic updates through the Business View Manager. For more information about scheduling a list of values, see the *Business Views Administrator's Guide*.
- A list of values can form a cascading prompt; that is, you can create a prompt whose final value is determined through a sequence of choices. For example, users might first be prompted to pick a country

before the choices for region appear. The users might then need to pick a region before the choices for city appear, and so on.

28.1.3 Data and non-data parameters

Depending on how parameters are used, they are marked as either data parameters or non-data parameters. How these parameters are used within a report will impact whether the report data will refresh upon changing parameter values.

Data parameters are used in:

- Record selection formulas, directly or indirectly (by way of being referenced within a formula)
- Command objects
- Stored procedures
- Universe parameters

Changes made to the value of a data parameter require a report refresh to fetch new data from the database. For example, a report will be refreshed when a data parameter is used to generate a query.

Non-data parameters are used in:

- Saved data selection formulas
- Conditional formulas
- Parameters placed on the report

Changes made to the value of a non-data parameter will filter the saved data of the report and will not require a refresh. This process minimizes the processing demand on the database. For example, a title is a non-data parameter.

Note

Non-data parameters change the presentation of existing data. It is best to use them only in a way that makes sense for the report. For example, do not filter using a saved data selection formula to display values that are below 100,000 if you have a record selection formula that only selects values above 100,000 from the database. Using selection formulas improperly may provide confusing results, or even blank pages.

28.1.4 Optional parameters

Crystal Reports supports optional parameters. An optional parameter is a prompted value that does not have to be supplied by the user.

Report designers should provide some guidance to the end user that a parameter is optional by adding this information to the prompt text.

Optional parameters are treated by Crystal Reports as optional anywhere within the report that they are used. You cannot set one instance of the parameter to optional and another to mandatory.

ⓘ Note

- After you set a parameter to optional, if you use the [Select Expert](#), the system will automatically add the selection formula for you using the `HasValue()` function. The [Formula Editor](#) does not automatically add the formula. Only the [Select Expert](#) automatically adds the formula for you.
- If you change the parameter from optional to mandatory, the `HasValue()` function is not automatically removed from your formula.

Handling a parameter with no value

When the report engine evaluates any formula which references an optional parameter that does not have a value, it generates a runtime error. All formulas that reference an optional parameter should first use the `HasValue()` function to check if that parameter has a value before evaluating it. This includes record selection formulas and saved data formulas.

ⓘ Note

The system will not automatically drop a statement from the SQL WHERE clause. It is the report designer's responsibility to decide which statement to leave out based on `HasValue()` testing.

28.2 Understanding dynamic prompts

Both dynamic prompts and cascading lists of values are available in Crystal Reports. These features let you populate the lists of values associated with a prompt from a data source outside of your report. (Static lists of values that are stored in your report are also available in Crystal Reports.)

This dynamic capability is available to all Crystal Reports users—whether or not they have the added capabilities of SAP BusinessObjects Business Intelligence platform. When Crystal report files are stored in SAP BusinessObjects Business Intelligence platform, additional dynamic prompting features are available. The following table shows which features are available with each product.

ⓘ Note

In the table, and elsewhere in this section, the term managed report is used to describe reports that are stored in a SAP BusinessObjects Business Intelligence platform environment, while unmanaged report is used to describe reports that are stored outside such an environment.

Feature	Available when Crystal reports are stored outside of SAP BusinessObjects Business Intelligence platform?	Available when Crystal reports are published to SAP BusinessObjects Business Intelligence platform?
Create dynamic prompts and cascading lists of values.	Yes	Yes
Reuse list of value definitions within a single report at design time.	Yes	Yes
Reuse list of value definitions within multiple reports at design time.	No	Yes
Schedule lists of values.	No	Yes
Populate lists of values from command objects.	Yes	No All lists of values that are used by managed reports are based on Business Views; however Business Views can themselves be based on command objects. In this way, lists of values can be indirectly based on command objects.
Populate lists of values from report fields.	Yes	No
Populate list of values from Business Views.	No	Yes
Cache lists of values at report run time, and share that list among multiple executing reports.	No	Yes
Schedule lists of values to update themselves on a regular basis.	No	Yes
Schedule portions of lists of values to update themselves on a regular basis, while the remaining portions retrieve their values from the database on demand.	No	Yes

Some components in the Crystal Reports and SAP BusinessObjects Business Intelligence platform products support the execution of reports with dynamic prompts and cascading lists of values:

- The supported viewers are as follows:
 - ActiveX
 - .NET Winform
 - .NET Webform
 - Java, COM, and JSF DHTML page viewers

- Java and COM Advanced DHTML Viewer

Note

The Java Applet viewer is not supported. Reports that use dynamic prompts and cascading lists of values will run in the Java Applet viewer, but they will not have pick lists where dynamic lists of values should appear.

- The BI launch pad and Central Management Console scheduling interfaces in SAP BusinessObjects Business Intelligence platform.
- The Report Designer Component (RDC).
- Crystal Reports XI R2 and later.

These components support the design of reports with dynamic prompts and cascading lists of values.

- Crystal Reports XI R2 and later.
- The integrated Visual Studio .NET designer.
- Business View Manager.

Note

While the dynamic capability is not available with SAP BusinessObjects BI Interactive Analysis or with full-client products, Crystal reports that are based on universes can use the dynamic prompting capabilities described throughout this section.

28.3 Understanding lists of values

List-of-values objects describe how to return a set of values from a data source:

- They provide the values for prompts in the prompting dialog box.
- They can be either static (the values are stored in the report), or dynamic (the values are stored outside the report in a database).
- They are used for both single-level dynamic prompts, and multi-level cascading prompts.
For example, a list of countries could be a dynamic prompt, while a hierarchical list of countries, regions, and cities could be a dynamic prompt with a cascading list of values.

Crystal Reports supports three data sources for list-of-values objects:

- Report fields (only for unmanaged reports).
- Command objects (only for unmanaged reports).
- Business Views (only for managed reports).

A list of values can have one or more levels. If there are multiple levels in the list of values, then each level constrains the next.

You can think of a list of values as the definition of the data needed to populate all levels of a cascading list. In Crystal Reports, a cascading relationship is defined by a single list-of-values object, not by multiple queries that are linked together by a common key. Although the list of values is defined as a single entity, data is not necessarily fetched from the data source in a single query.

Note

- List-of-values objects are not affected by report record selection or group selection formulas. These formulas affect the report data, not data used for prompt-time lists of values.
- If a data connection is broken or reset, Crystal Reports does not attempt to create or update the dynamic list-of-values object and instead reverts to a static list.

28.3.1 List-of-values types

There are two different types of list-of-values objects:

- Unmanaged lists of values are stored within each report file.
If you do not have SAP BusinessObjects Business Intelligence platform, or if you never publish your reports to SAP BusinessObjects Business Intelligence platform, you use an unmanaged list-of-values object. Unmanaged list-of-values objects can use report fields or command objects as a data source.
- Managed lists of values are stored within SAP BusinessObjects Business Intelligence platform.
Every report that you store in SAP BusinessObjects Business Intelligence platform uses managed list-of-values objects. All managed list-of-values objects are based on a Business View, even if the report itself does not use a Business View. Managed list-of-values objects have a number of features that are not available to unmanaged list-of-values objects.

Feature	Unmanaged list of values		Managed list of values
	Report fields	Command objects	Business Views
Database query per list-of-value level.	Yes The database is queried for each level as the user selects values in the prompting dialog box.	No The database is queried for all values in all levels of the list at one time.	Yes The database is queried for each level as the user selects values in the prompting dialog box (provided the Business View is not based on a command object).
At design time, specify a filter for the list of values.	No All of the values in the report fields that are also in the list of values definition are used.	Yes Any filter that can be represented in SQL can be used.	Yes Business Views support filters.
At report-view time, prompt for additional information needed by the list of values.	No	Yes If the command object contains parameters, the value(s) are prompted for at report-view time.	Yes If the Business View contains parameters, the value(s) are prompted for at report-view time.

Feature	Unmanaged list of values		Managed list of values
	Report fields	Command objects	Business Views
Display different values to different users.	No	No	Yes Because managed lists of values are based on Business Views, they inherit the view-time security capabilities of Business Views.
Schedule the list of values to update on a recurring schedule.	No	No	Yes The scheduling capability comes from the Business View Manager, not the Central Management Console (CMC) where reports are scheduled.
Partially schedule the list of values to update only certain portions of the list on a recurring schedule.	No	No	Yes

28.3.2 Determining which list-of-values type to use

Different reporting problems require different prompting solutions, depending on the amount of data in the lists of values. This table provides a rough guide to which list-of-values design you should use.

Description	Unmanaged list of values		Managed list of values
	Report fields	Command objects	Business Views
Single-level code tables. (A table that has hundreds of semi-static values in a single level.)	Not well suited. Because report fields cannot be filtered, any filtering of the list that you require must be done outside of Crystal Reports in a database view.	Well suited. You can define filtering within a command object. (That command objects return the data for all levels in the list is not a problem for single-level lists.)	Well suited. You can define filtering within the source Business View.

Description	Unmanaged list of values		Managed list of values
	Report fields	Command objects	Business Views
Multi level-cascading code tables. (A table that has hundreds of semi-static values in multiple levels.)	Well suited. Provided that the data filtering is done outside of Crystal Reports, report fields are well suited to multi-level cascading code tables because they retrieve data one level at a time.	Well suited. Provided that the total data volume for all levels in the list is not too large. If there is too much data, you could encounter performance issues because command-object-based lists of values retrieve data for all levels in one trip to the database.	Well suited. Business-View-based lists of values can support both filtering and level-by-level data retrieval.
Fact tables. (These tables tend to be very large, dynamic tables with millions of values in multiple levels.)	Well suited. Provided that the filtering is done outside of Crystal Reports in a database view, and provided that there is a multi-level hierarchy to the data.	Not well suited. Because command objects retrieve their data in one trip to the database, there could be performance issues when you use them against very large tables.	Well suited. Partially scheduled lists of values excel in this situation, where the semi-static part of the data can be scheduled, and the most dynamic part can be retrieved on-demand.

28.3.3 Lists of values and prompt groups contrasted

Lists of values are the data part of a prompt; the values from your data that your users will see and select from.

Prompt groups, on the other hand, are the presentation part of a prompt. Crystal Reports treats prompt groups as separate objects so that you can share the same list of values with different presentations. For example, you can have a Shipping City prompt, and a Customer City prompt. Perhaps you allow for multiple customer cities, but only a single shipping city in your report. You can design this report so that it uses a single city list of values, but with two different prompt groups (or presentations styles).

For more information about using prompt groups, see [Sharing common lists of values within a report \[page 850\]](#).

28.4 Creating a parameter with a static prompt

A static prompt is one that always contains the same values. For example, if your parameter prompts for a country value, you could create a static prompt because the country list represents a set of values that does not change often.

Use the following steps to create a parameter with a static prompt that lets users see a list of customers that they can select a specific country from.

This procedure is made up of two sets of steps. The first is creating the parameter, and the second is using the Select Expert to incorporate the parameter into the record selection filter.

28.4.1 To create a parameter with a static prompt

Procedure

1. Open the sample report called *Group.rpt*.
Sample reports are located on the SAP Business Objects Support web site.
2. On the *View* menu, click *Field Explorer*.

→ Tip



Another way to do this is to click the Field Explorer button on the Standard Toolbar.

The Field Explorer dialog box appears.

3.  Select *Parameter Fields* and click *New*.

The Create New Parameter dialog box appears.

4. Enter a name for the parameter in the *Name* field (up to 255 alphanumeric characters).

This example uses Country.

5. Select the appropriate *Type* from the list.

This example uses String.

ⓘ Note

When creating a parameter whose Type is either Date or DateTime, you can change the date format to suit your needs. For details, see [Changing your default field formats \[page 273\]](#).

6. From the *Value Field* list, select *Country*.
7. Click *Actions* and select *Append all database values* to move all of the countries in the sample database to the *Values* area.

This example will enable the user to choose from any of the countries. If you want to limit the selection, manually enter only the countries that you would like the user to choose from.

8. In the Value Options area, enter the desired prompting text in the *Prompt Text* field (up to 255 alphanumeric characters).

This is the text that appears in the "Enter prompt values" dialog box when the report is refreshed. This example uses the default value.

Create New Parameter

Create a new parameter and list of values.

Name: Type: List of Values:

Value Field: Description Field:

Value Description Actions

Value	Description
Argentina	
Aruba	
Australia	
Austria	
Bahamas	
Bangladesh	
Barbados	
Belgium	

Value Options:

Option	Setting
Prompt Text	Enter Country:
Prompt With Description Only	False
Default Value	
Allow custom values	True
Allow multiple values	False
Allow discrete values	True

OK Cancel Help

9. Click [OK](#).
10. Return to the Field Explorer dialog box, and drag the [Country](#) parameter into your report.

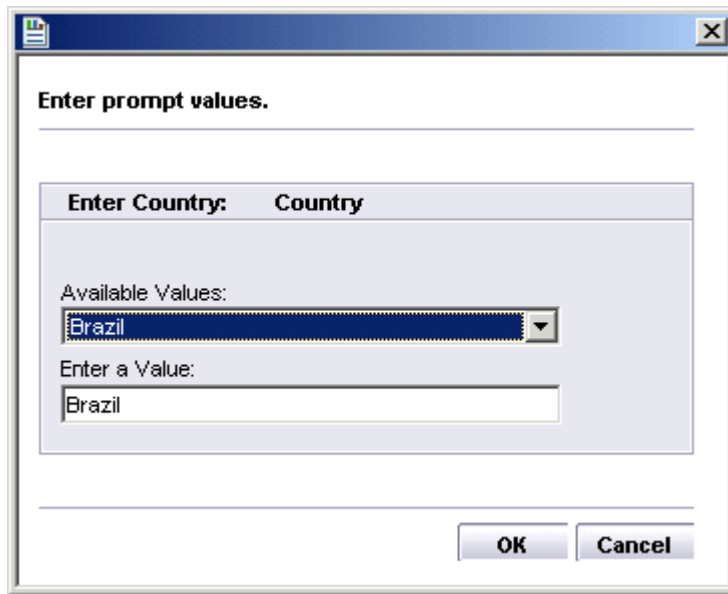
The "Enter prompt values" dialog box appears.

Note

If you don't want to see the parameter field you dropped in your report, place it in a section you can suppress, such as a report header or footer.

11. Select the country to base the report on.

This example uses Brazil.



12. Click [OK](#).

28.4.2 To incorporate the parameter into the record selection filter

Procedure

1. On the [Report](#) menu, click [Select Expert](#).

→ Tip



Another way to do this is to click the Select Expert button on the Expert Tools toolbar.

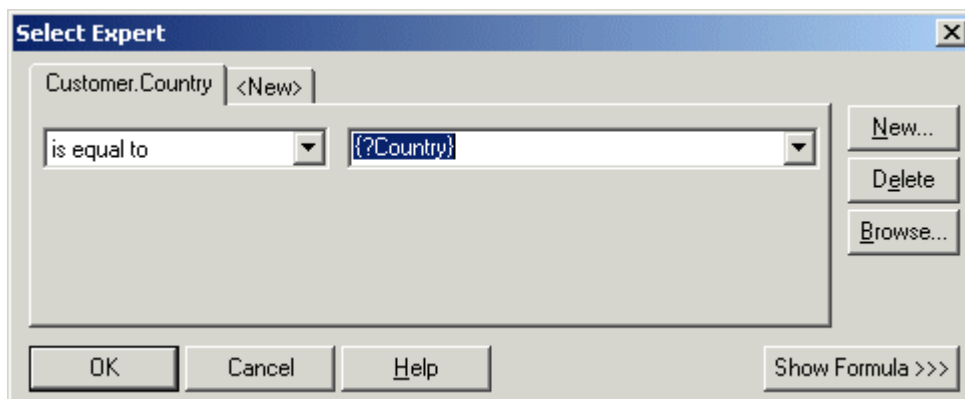
The Choose Field dialog box appears.

2. Select [Country](#) from the Customer table, then click [OK](#).

The Select Expert appears.

3. Choose [is equal to](#) from the drop-down list.
4. Choose the parameter from the adjacent drop-down list.

This example uses {?Country}.



5. Click [OK](#).

The report appears with the information for Brazil.

Results

With parameter fields, you can create a single report that can be customized quickly to meet a variety of needs.

28.4.3 To incorporate the parameter into a saved data selection formula

Procedure

1. On the [Report](#) menu, point to [Select Expert](#) and then click [Saved Data](#).
The [Choose Field](#) dialog box appears.
2. Highlight the field on which you want to base record selection and click [OK](#).
The [Select Expert](#) appears.
3. Use the drop-down lists to enter your selection criteria for the indicated field.
4. Choose the parameter from the adjacent drop-down list and click [OK](#).
If the parameter is being referenced for the first time, the [Enter Values](#) dialog box appears.
5. Select your initial values and click [OK](#).

Results

Crystal Reports immediately filters the report data according to your selection criteria.

28.5 Creating a parameter with a dynamic prompt

The values of a dynamic prompt are retrieved from the data source and are not stored within the report. For example, if you are prompting for customer name, you could create a dynamic prompt because the names in your customer database probably change frequently.

Dynamic prompts can be authored in Crystal Reports. You do not need SAP BusinessObjects Business Intelligence platform or Crystal Reports Server to author dynamic prompts. When Crystal Reports is installed on the same machine as Microsoft Visual Studio, you can use the embedded Crystal Reports designer to author dynamic prompts. When Crystal Reports is used with Crystal Reports Server or the SAP BI platform Server, additional features are available. For more information, see [Managed reports \[page 854\]](#).

Use the following steps to create a parameter that uses a dynamic prompt. As part of the procedure, you will create a list of values.

28.5.1 To create a parameter with a dynamic prompt

Procedure

1. Open the sample report called [Group.rpt](#).

Sample reports are located on the SAP Business Objects Support web site.

2. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

3.  Select [Parameter Fields](#) and click [New](#).

The Create New Parameter dialog box appears.

4. Enter a name for the parameter in the [Name](#) field (up to 255 alphanumeric characters).

This example uses Region.

5. In the List of Values area, click [Dynamic](#).
6. Ensure that [New](#) is selected in the Choose a Data Source area.

Note

If your report already contains a list of values, or if you want to pick a list of values from your repository, you can select Existing and choose an existing list of values from the tree structure.

7.  Click [Insert](#), and from the [Value](#) list, select [Region](#).

8. In the Value Options area, enter the desired prompting text in the [Prompt Text](#) field (up to 255 alphanumeric characters).

This is the text that appears in the "Enter prompt values" dialog box when the report is refreshed. This example uses "Select a Region."

Create New Parameter

Create a new parameter and list of values.

Name: Type: List of Values:

Prompt Group Text:

Choose a Data Source: ☐ New ☐ Existing

Value	Description	Parameters
Region	(None)	Region
Click here to add item		

Value Options - Level 1 (Region):

Option	Setting
Prompt Text	Enter Region:
Sort Order	Ascending by Value
Prompt With Description Only	False
Allow multiple values	False
Allow discrete values	True
Allow range values	False

OK Cancel Help

9. Click [OK](#).
10. Return to the Field Explorer dialog box, and drag the [Region](#) parameter into your report.

Note

If you don't want to see the parameter field you dropped in your report, place it in a section you can suppress, such as a report header or footer.

Results

When you look at your dynamic prompt within Crystal Reports, it does not seem to be much different from a static prompt. In the background, however, Crystal Reports stores the information about the list of values that you created for this prompt. When the report engine requires a parameter that has a dynamic prompt, it will access the data source and populate the list of values automatically. This will be displayed on the end user prompt page. If you choose to publish the report to SAP BusinessObjects Business Intelligence platform or Crystal Reports Server, the list of values becomes a separate entity that you can optionally schedule in Business Views Manager.

28.6 Creating a parameter with a cascading list of values

A dynamic prompt can include a cascading list of values, which adds additional levels of selection for your users. For example, if you are prompting for a city value, but you also need to know which country and region that city comes from, you could create a dynamic and cascading prompt. In this case, you first prompt for a country, and when that value has been selected, the program prompts for a region by showing only the regions that apply to the selected country. Finally, when a region value has been selected, the program prompts for a city by showing only the cities that apply to the selected region. In this way, you can be sure that your user picks the correct city (for instance, Vancouver, Washington, USA rather than Vancouver, British Columbia, Canada).

As with a dynamic prompt that has only one value, you are able to create or specify a cascading list of values that you can schedule for updates as often as you like through the Business View Manager. For more information about scheduling a list of values, see the *Business Views Administrator's Guide*.

Use the following steps to create a parameter that uses a dynamic prompt and a cascading list of values.

28.6.1 To create a parameter with a cascading list of values

Procedure

1. Open the sample report called [Group.rpt](#).

Sample reports are located on the SAP Business Objects Support web site.

2. On the [View](#) menu, click [Field Explorer](#).

The Field Explorer dialog box appears.

3.  Select [Parameter Fields](#) and click [New](#).

The Create New Parameter dialog box appears.

4. Enter a name for the parameter in the [Name](#) field (up to 255 alphanumeric characters).

This example uses Supplier City.

5. In the List of Values area, click [Dynamic](#).

6. Enter prompting text for your prompt group in the [Prompt Group Text](#) field.

This example uses "Choose the City that your supplier is located in."

Note

Prompt groups let you use cascading list of values for other cascading prompts. For example, if you set up such a list of values for an address prompt, you might want to use the list for both shipping and billing addresses. In this case, you could create two prompt groups for the same list of values; you might call one prompt group Shipping Address and the other Billing Address.

7. Ensure that [New](#) is selected in the Choose a Data Source area.

Note

If your report already contains a list of values, or if you want to pick a list of values from your repository, you can select Existing and choose an existing list of values from the tree structure.

8.  Click *Insert*.

The program automatically expands the Value list. You use this area to define the fields that make up your cascading list of values. This example uses a cascade of Country, Region, and City.

9. From the *Value* list, select *Country*.
10. Click the blank field under *Country* and select *Region*.
11. Click the blank field under *Region* and select *City*.
12. In the Parameters area, click *City* to bind the field that contains the City value to the parameter.

In this step, you are choosing the field in your cascading hierarchy that should be bound to the parameter field that you are creating. Because this particular parameter is intended to provide Crystal Reports with a city value, you bind the parameter to City.

13. In the *Prompt Text* field (in the Value Options area), enter additional prompting text for each of the fields in your cascading list of values.

This is the text that appears in the "Enter prompt values" dialog box when the report is refreshed.

Create New Parameter

Create a new parameter and list of values.

Name: Type: List of Values:

Prompt Group Text:

Choose a Data Source: ☐ New ☐ Existing

Value	Description	Parameters
Country	(None)	Click to create parameter
Region	(None)	Click to create parameter
City	(None)	? Supplier City - City

Value Options - Level 3 (City):

Option	Setting
Prompt Text	Enter City:
Sort Order	Ascending by Value
Prompt With Description Only	False
Allow multiple values	False
Allow discrete values	True
Allow range values	False

OK Cancel Help

14. Click [OK](#).

15. Return to the Field Explorer dialog box, and drag the [Supplier City](#) parameter into your report.

Note

If you don't want to see the parameter field you dropped in your report, place it in a section you can suppress, such as a report header or footer.

28.7 The Parameter Panel

The [Parameter Panel](#) lets users interactively format and filter report data by changing parameter values. It is located within the [Preview Panel](#).

In the [Create New Parameter](#) and [Edit Parameter](#) dialog boxes, you can specify parameters to display on the Parameter Panel via the [Show on \(Viewer\) Panel](#) option. You can choose one of several settings:

- [Do not show](#)
The parameter is not visible on the panel. Users must refresh the report to change the parameter values.

- *Editable*
Users can view and change the parameter values on the panel.
- *Read only*
Users can view the parameter values on the panel, but cannot change them.

Note

Do not show is the default setting for a new parameter.

The *Parameter Panel* displays all parameters that have been set to *Editable* or *Read only*. Depending on the type of parameter, users can add, delete, or modify parameter values either by using the dropdown menus and applying their changes or by clicking the *Advanced Dialog* button.

The following are some of the more commonly used types of parameters that must be modified by clicking the *Advanced Dialog* button:

- Parameters with ranged values.
- Dynamic parameters that use cascading prompts.

On the panel, data parameters are differentiated from non-data parameters with a database icon. When a user modifies the value of a data parameter, the report refreshes and accesses the database to make the corresponding changes. When a user modifies the value of a non-data parameter, the report is formatted or filtered based on saved data within the report. It does not access the database.

28.8 Working with lists of values

Dynamic prompts use lists of values. You can create a list of values in either Crystal Reports or in the Business View Manager. Crystal Reports refreshes the data in a list of values when you open a report that contains a dynamic prompt; however, you can also refresh the data in a list of values by scheduling it in the Business View Manager. For more information, see *Using Business Views*.

You can add lists of values to SAP BusinessObjects Business Intelligence platform or Crystal Reports Server in several ways:

- You can create a list of values when you design a report, and then you can save the report to an Enterprise folder. For more information, see [Saving a report to an Enterprise folder \[page 630\]](#).
- You can create a list of values in the Business View Manager. For more information, see *Using Business Views*.
- You can create a list of values when you design a report, and then you can add it to SAP BusinessObjects Business Intelligence platform through the Central Management Console (CMC). For more information, see the *SAP BusinessObjects Business Intelligence Platform Administrator Guide*.
- You can migrate lists of values from one environment to another by using the Business Views Manager Import/Export facility. For more information, see the *SAP BusinessObjects Business Intelligence Platform Administrator Guide*.

28.8.1 Sharing common lists of values within a report

You can use a single list-of-values object for many unrelated prompts within a single report. For example, in a report that shows shipments from suppliers to customers, shipments can go from a supplier in one city to a customer in another city. You would want your report to prompt for both supplier city and customer city; both have the same set of values. Because there are many hundreds of cities in the database, you can break up this long list of values into countries, regions, and cities. In that way, your users can navigate through hundreds of values by selecting from three shorter lists.

28.8.1.1 To share a common list of values within a report

Procedure

1. Create a report that contains fields for *Country*, *Region*, and *City*.

2.  In the Field Explorer, select *Parameter Fields* and click *New*.

The Create New Parameter dialog box appears.

3. Enter a name for the parameter in the *Name* field.

This example uses SupplierCity.

4. In the List of Values area, click *Dynamic*.

5. Enter prompting text for your prompt group in the *Prompt Group Text* field.

This example uses "Choose the City that your supplier is located in."

6.  Click *Insert*, and from the *Value* list, select *Country*.

7. Click the blank field under *Country* and select *Region*, and then click the blank field under *Region* and select *City*.

8. In the Parameter Binding area, ensure that only the *City* value is bound.

9. Click *OK*.

10. Return to the Field Explorer dialog box, select *Parameter Fields*, and click *New*.

11. Enter a name for the second parameter in the *Name* field.

This example uses CustomerCity.

12. In the List of Values area, click *Dynamic*.

13. Enter prompting text for your new prompt group in the *Prompt Group Text* field.

This example uses "Choose the City that your customer is located in."

14. Select *Existing*, and select the list of values that you created in steps 6 through 9.

15. Click *OK*.

Results

You have created two prompts: a Country, Region, City hierarchy for the {?SupplierCity} parameter, and a second Country, Region, City hierarchy for the {?CustomerCity} parameter. By reusing the list-of-values object, you saved time and improved performance time in Crystal Reports.

Note

You can also use an existing list of values if you have another parameter field that should have a country value. For example, if you add a {?DivisionCountry} parameter to indicate the corporate division that you want to report on, you could use the same list-of-values object in your report. You would bind this parameter to the top-level value in your hierarchy—that is, the Country level of the Country, Region, City hierarchy.

28.8.2 Using separate value and description fields

It is common in relational databases to make use of code fields that represent values. These codes are often numeric or text strings that cannot be read by your users. For such cases, you can create separate value and description fields in your list of values definition. You set the value field to the parameter; the description field appears in the prompting dialog box. How the description field appears in the prompting dialog box is controlled by the prompt option "Prompt with Description Only." When set to True, only the descriptions are visible; when set to False, both the values and descriptions are visible.

28.8.2.1 To set separate value and description fields

Procedure

1. Open the sample report called [Group.rpt](#).

Sample reports are located on the SAP Business Objects Support web site.

2.  In the Field Explorer, select [Parameter Fields](#) and click [New](#).

The Create New Parameter dialog box appears.

3. Enter a name for the parameter in the [Name](#) field.

This example uses Customer Name.

4. In the List of Values area, click [Dynamic](#).

5.  Click [Insert](#) and, in the [Value](#) field, select [Customer ID](#).

6. In the [Description](#) field, select [Customer Name](#).

7. In the Value Options area, set the [Prompt with Description Only](#) option to [True](#).

8. Click [OK](#).

Results

When your users see this prompt in the prompting dialog box, they do not see values from the Customer ID field (the field on which the parameter is based), but rather, they see a list of customer names.

28.8.3 Using command objects as list-of-values data sources

Crystal Reports supports command objects as data sources for unmanaged lists of values. To use this feature, you first create a command object using the Database Expert to return the values that you want in your list-of-values object.

For more information see "Defining an SQL Command" in the online help.

For more information see [Defining an SQL Command \[page 896\]](#).

The query you create should return at least one database field per level in the new list-of-values object. For example, if you want to build a Country, Region, City list-of-values object, your query should return at least three columns from your database: Country, Region, and City. You need to return all three values in a single select statement. There is no need for an order by clause, because the prompting engine sorts the values according to your specifications.

Note

It is not necessary to link your command object to your report fields. In general, command objects that you use only to populate lists of values do not need to be linked to your report. However, if you want to use a column from this command object in your report, you need to link the command object to the report.

All the command objects defined in your report appear in the value fields drop-down list, and in the description fields drop-down list in the Create New Parameter dialog box. You can then associate your list of values with the command object.

28.8.4 Null handling

Null values in the data returned by the list of values are displayed as "null" in the drop-down list for that prompt. Your users can select these values, and any parameters associated with that prompt receive the null value. You can test for this functionality in the Crystal Reports formula language with the IsNull function.

For more information, see "IsNull (fld)" in the online help.

28.8.5 Long lists of values

Because the number of items in a list of values (LOV) can be very large, numbering in the hundreds or thousands, Crystal Reports handles these long LOVs by retrieving smaller batches of values at a time.

Crystal Reports displays a single batch of LOVs, and provides two options to the user to control the list: the batch number and a simple text filter for limiting the number of items in the list.

The default number of items per batch retrieved in a long LOV is set to 1 000; however, you can change this default number by editing a registry key. For more info, please refer to [SAP Knowledge Base Article 1218588](#).

Changing the number of items in a long LOV

There are two methods for changing the number of items in a long LOVs.

When prompting the user, Crystal Reports retrieves the long LOV batch size from this Windows registry location:

```
HKEY_CURRENT_USER\Software\SAP BusinessObjects\Suite XI 4.0\Crystal Reports\ReportView
```

If it does not already exist, create the DWord value called `PromptingLOVBatchSize`, and set the value to the number of items in a batch.

Note

Be careful to select *Decimal* as the base value before entering the batch size.

You can create or change a parameter in the Tomcat configuration file to set the batch size for long LOVs. Open the following file:

```
{<CR_install_folder>}/Tomcat/webapps/CrystalReports/WEB-INF/web.xml
```

If it does not already exist, create this parameter:

```
<context-param>
  <param-name>crystal_max_number_list_of_values</param-name>
  <param-value><{number}></param-value>
</context-param>
```

Where `<{number}>` in the above example is the number of items in the batch.

28.9 Best practices for prompting

Click a link to view one of the topics in this section:

- [Unmanaged reports \[page 854\]](#)
- [Managed reports \[page 854\]](#)
- [Converting unmanaged reports to managed reports \[page 855\]](#)
- [Deploying managed reports with dynamic prompts \[page 855\]](#)

28.9.1 Unmanaged reports

Unmanaged reports are reports that you store outside of SAP BusinessObjects Business Intelligence platform. These reports can use lists of values that you define within the report, or lists of values that you have stored in the SAP BusinessObjects Enterprise Repository. (You create report lists of values in Crystal Reports when you use the [Create New Parameter](#) dialog box. You create repository lists of values in the Business Views Manager.)

Lists of values defined within a report have the following properties:

- They can be based on command objects or report fields.
- They are always stored within the .rpt file.
- They cannot be shared between reports.

If your unmanaged report also uses a repository-based list of values, the report will prompt you to log on to the repository whenever it is opened. This process is required so that Crystal Reports can locate the list-of-values object.

To maximize performance and scalability of unmanaged reports, you should reuse list-of-value objects within a single report wherever possible. For information about how to do this, see [Sharing common lists of values within a report \[page 850\]](#).

28.9.2 Managed reports

Managed reports are reports that you store within SAP BusinessObjects Business Intelligence platform.

Lists of values used in managed reports have the following properties:

- They are always stored in the repository.
- They can be shared between reports.
- They are based on Business Views.

To maximize performance and manageability of your managed reports, the following practices are recommended:

- Define your list-of-values objects in Business View Manager, not Crystal Reports.
- Build a separate Business View to provide lists of values for all of your reports. The only fields that you need in this Business View are those that you use for prompting. See the *Business Views Administrator's Guide* for information about how to create a Business View.
- Create managed list-of-values objects for each dynamic prompt that you intend to use in your report. These objects are visible to Crystal Reports users when they design parameters and prompts. If you need a list of values for a Country > Region > City hierarchy, and a Country pick list, you can satisfy both needs with a single list-of-values object.

Defining your list-of-values objects in Business View Manager has the following benefits:

- You have control over the metadata that is used to define the prompting list of values.
- You can apply row-level security (if desired) to the Business View that is used for prompting.
- You can separate metadata design from report design. A metadata designer can be responsible for authoring the list-of-values definitions, and the report designer can be responsible for authoring the report.
- This method minimizes the number of metadata objects you need to create to support prompting.

28.9.3 Converting unmanaged reports to managed reports

You can convert an unmanaged report to a managed report in the following ways:

- Publish one or more reports with the Publishing Wizard.
- Save the report to SAP BusinessObjects Business Intelligence platform using the Crystal Reports [Save As](#) command.
- Create a new report object from within the BI launch pad portal.
- Create a new report object from within the Central Management Console portal.

In all of these cases, these actions are carried out on the prompt object in your unmanaged report:

- List of values (LOV) objects that are defined in the report are converted to repository lists of values. A Business View, Business Element, Data Foundation, and Data Connection object is created.
- If the same LOV object already exists in the repository, no duplicate object is created. Instead, your report references the existing LOV object.
- LOV objects that are repository-based are unchanged.
- Prompt groups that are defined in the report are converted to repository prompt groups.
- Prompt groups that are repository-based are unchanged.
- All of the new repository-based Business View objects, LOV objects, and prompt-group objects are created in the repository folder called Dynamic Cascading Prompts.

28.9.4 Deploying managed reports with dynamic prompts

Repository-based lists of values and prompt groups are repository objects like any other. When managed reports are migrated from one repository to another using the Import Wizard, the list-of-values and prompt-group objects are imported as well. This functionality makes it easy to migrate reports from system to system. SAP BusinessObjects Business Intelligence platform tracks the objects that a report depends on, and imports these objects to the destination system if needed.

Business Views Manager has been enhanced to support the import and export of lists of values and prompt groups. For more information, see the *Business Views Administrator's Guide*.

28.10 Deleting parameter fields

There are several methods for deleting parameters within a report. The type of parameter you are deleting determines the method you can use.

28.10.1 To delete a parameter that is not used in a formula

Procedure

1. On the *View* menu, click *Field Explorer*.

The Field Explorer appears.

2. Expand the *Parameter Fields* folder and click the parameter you want to delete.
3. Click *Delete*.

If the parameter is used in your report, a Crystal Reports dialog box appears confirming whether you want to delete the parameter.

4. Click *Yes*.

This parameter is removed from the Parameter Fields folder and from your report.

28.10.2 To delete a parameter used with the Select Expert

Procedure

1. On the *Report* menu, click *Select Expert*.

→ Tip



Another way to do this is to click the Select Expert button on the Expert Tools toolbar.

The Select Expert dialog box appears.

2. Choose the tab whose selection criteria uses the parameter you want to delete.
3. Click *Delete*.
4. Click *OK* to close the Select Expert.
5. On the *View* menu, click *Field Explorer*.
6. Expand the *Parameter Fields* folder and click the parameter you want to delete.
7. Click *Delete*.

28.10.3 To delete a parameter that is used in a formula

Procedure

1. On the *View* menu, click *Field Explorer*.

The Field Explorer appears.

2. In the *Formula Fields* folder, select the formula that contains the parameter you want to delete.
3. Click *Edit* and delete the parameter field from the formula.

Note

If the parameter is used in more than one formula, it must be deleted from each formula.

4. Close the Formula Workshop.
5. Expand the *Parameter Fields* folder and click the parameter you want to delete.
6. Click *Delete*.

28.11 Responding to parameter field prompts

Click a link to view one of the topics in this section:

- [Previewing a report for the first time \[page 857\]](#)
- [Refreshing report data \[page 857\]](#)

28.11.1 Previewing a report for the first time

When you preview a report for the first time, the Enter Prompt Values dialog box appears, prompting you for a value.

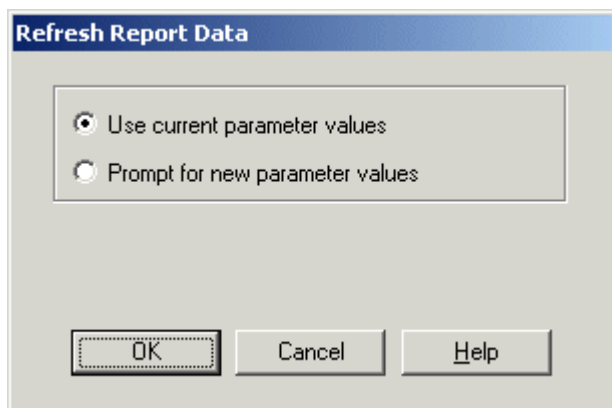
- If you specified a default value when you created the parameter field, the program will use that value unless you specify a new one.
- If you did not specify a default value, the program will not refresh the data until you supply a new value.

Note

If the parameter is a string value type, without a default value, and the "Allow discrete values" field is True, an empty string appears.

28.11.2 Refreshing report data

When you refresh data from the Preview tab, the Refresh Report Data dialog box appears.



Select the "Use current parameter values" option to use the current parameter value.

Select the "Prompt for new parameter values" option to enter a new parameter value. When you select this option and click OK, the Enter Prompt Values dialog box appears.

- Enter String values exactly as they will appear in the field. If the parameter field allows multiple values, you can enter additional parameter values using the Add button.
- Enter Boolean values using the following format: TRUE or FALSE.
- Enter Number values exactly as they will appear in the field.
- Enter Currency values exactly as they will appear in the field.
- Enter Date values to match the format used on-screen. If the format is unspecified, enter as Date (Year, Month, Day). For example, Date (1997, 5, 21). To access the calendar, click the drop-down arrow beside the date.
- Enter Time values to match the format used on-screen. If the format is unspecified, enter as Time (Hour, Minutes, Seconds AM/PM). For example, Time (4:32:12 PM). You can also select the unit of time and then use the up and down arrows to scroll through the numbers.
- Enter DateTime values using the following format: Date (Year, Month, Day), Time (Hour, Minutes, Seconds AM/PM). For example, Date (1997, 5, 21), Time (4:32:12 PM). You can also enter DateTime values by using the associated drop-down arrow to access the calendar, and the up and down arrows to scroll through the time.

Note

The drop-down arrow to access the calendar for dates and the up and down arrows to scroll for times will only be available if you have the correct version (4.70 or later) of comctl32.dll.

To use the current value displayed, click OK, or, to use a different value from the one displayed, type a new value in the text box, and click OK.

- If the parameter field is range limited, then you can only enter values within a certain range. The range limit is specified in the Set Default Value dialog box when creating or editing a parameter.
- If the parameter is a string value type, you can limit the values the user is able to enter by using the length limit option or by using an edit mask to restrict the format.

The program now runs the report using the new value(s) you specified.

28.12 Advanced parameter features

There are a variety of ways that parameters can be used within a report. This section covers some of the advanced methods of using parameters.

28.12.1 Creating a parameter with multiple prompting values

1. On the [View](#) menu, click [Field Explorer](#).
2.  Select [Parameter Fields](#) and click [New](#).
3. Enter a Name for your parameter.
4. Select a value type from the Type list.
5. From the [Value Field](#) list, select the table and field that you want to use to obtain your prompting values.
6. Add the values from the field you selected by clicking [Actions](#) and choosing [Append all database values](#). You can also import values, or you can create your own in the Value area under the Options button.
7. In the Value Options area, select the [Allow multiple values](#) option and set its value to [True](#) to enable your users to enter more than one prompting value for this parameter. Both discrete and range type values are allowed individually or in combination.

Note

For details on single or range values, see [Specifying single or ranges of values \[page 861\]](#).

8. Click [OK](#).

28.12.2 Applying conditional formatting using parameter fields

Parameter fields can be used to create conditional formatting formulas. You can customize these formulas whenever you refresh the report data. A conditional formatting formula could be used for color-flagging data that meets certain conditions. For example:

- Sales representatives who sell more than 10% over quota.
- Customers who have not ordered in the last quarter.
- Inventory items that have not had any movement in the last month.

If the conditions under which you flag these items never change, you do not need to use parameter fields. You can just use formulas (for text flags) or conditional formatting (for border flags). However, to change the conditions from report to report, you need to use parameter fields in formulas and conditional formatting formulas.

28.12.2.1 To apply conditional formatting using parameter fields

Procedure

1. Create the parameter field of the data type you need for the formula.
2. Create the formula and use the parameter field in place of the fixed value you would normally use.



For example, to be prompted for all the customers whose last year's sales were over a certain value, and to print their names in red, select the Last Year's Sales field and click Format from the Expert Tools toolbar.

The Format Editor appears.



3. Click the Conditional formula button next to the Color property on the *Font* tab, and format the field using a conditional formatting formula like this:

```
If {customer.LAST YEAR'S SALES} > {?SalesTarget} Then
    Red
Else
    Black
```

Results

Now, when you refresh the data, the program will prompt you for the value that triggers the color flag (known as the threshold value). It then runs the report and flags all the customers that had sales last year above the threshold figure. You can change the figure each time you run the report and the program will flag a different set of Customer Names.

For more information see "Conditional formatting functions" in the online help.

28.12.3 Creating a report title using parameter fields

Crystal Reports allows you to use parameter fields to create a report title that can be changed each time the report is refreshed.

28.12.3.1 To create a report title using parameter fields

Procedure

1. On the *View* menu, click *Field Explorer*.

The Field Explorer appears.

2.  Select *Parameter Fields* and click *New*.

The Create New Parameter dialog box appears.

3. Type a name for the parameter field in the *Name* field.
4. Select String from the *Type* list.
5. To specify a default title, click *Click here to add item* in the Value area and type the desired text.

Continue adding titles as required.

6. Click *OK*.
7. Return to the Field Explorer, and then place the parameter field in the Page Header section of the report to have the title appear on every page, or in the Report Header section if you want the title to appear on only the first page of the report.

Results

Now, when you refresh the data, the program will prompt you for a report title. If you wish, you can change the title each time you run the report.

28.12.4 Specifying single or ranges of values

You can create parameters that require users to enter a single (discrete) value, or a range of values. If you then include these parameters in your report's record selection, you can help users find specific information. For example, in the case of a record selection parameter that uses a discrete value, a user might enter a single country name to see sales figures for only that country. In the case of a record selection parameter that uses a range of values, a user might enter a range of countries (for example, Canada and France) to see sales figures for all the countries in the range (in this case, Chili, China, Denmark, England, and so on).

28.12.4.1 To specify single or range values

Procedure

1. Select the *Allow discrete values* option or the *Allow range values* option to specify whether the parameter field will accept a range of values.
 - If you select "Allow discrete values," the parameter field will accept single values (rather than ranges of values).
 - If you select "Allow range values," then when you are prompted for parameter values, you can enter a start value and an end value. For example, if you enter the values "5" and "10", the range is 5-10, and a report that uses this parameter for record selection will display all records with values between 5 and 10. This also works for string parameter fields. With a start value of "A" and an end value of "E", a report that uses this parameter for record selection will display all records within an alphabetical range of A-E.

Note

If the "Allow multiple values" and the "Allow discrete values" options are selected, the parameter field will accept multiple single values. In this case, you can enter more than one value, but these values will be evaluated individually and will not be interpreted as a range. If the "Allow multiple values" and "Allow range values" options are selected, the parameter field will accept multiple ranges.

- If applicable, select the *Min Length* and/or *Max Length* options to designate the length of the field.
 - For a Currency or Number parameter field, options for entering the "Min Value" and "Max Value" appear.
 - For a Date, DateTime, or Time parameter field, options for entering the "Start" and "End" values appear.

28.12.5 Incorporating a parameter into a formula

Procedure

- On the *View* menu, click *Field Explorer*.

The Field Explorer appears.

-  Select *Parameter Fields* and click *New*.

The Create New Parameter dialog box appears.

- Create a parameter field and save it.
- Select *Formula Fields* and click *New*.

The Formula Name dialog box appears.

- Enter the name of the formula, and then click *OK*.
- Create a formula using the parameter field as you would any constant value. For example, rather than creating a formula that hard-codes the country name:

```
{customer.COUNTRY} = "USA"
```

Use a parameter field instead of "USA".

```
{customer.COUNTRY} = {?Country}
```

To do this double-click the database field, press =, then double-click the parameter.

→ Tip

Identify parameter fields easily by looking for (?).

- Click *Save and Close* on the Formula Workshop.

The Field Explorer dialog box appears. The name of the formula you have just created is highlighted in the Formula list box.

- Drag and drop the formula into the report.

9.  Click [Refresh](#) to generate the report.
A dialog box appears, prompting you for values.

28.12.6 Defining sort order using parameter fields

Context

To set the sort order using parameter fields, you need to first create a formula that includes a parameter field and then sort based on that formula. For example, assume that you have a customer list report based on the Customer table. For each customer, you show the Customer Name, City, Region, Country, and Phone Number. You want to be able to sort the report by Country, by Region, or by City, depending on your needs at the time.

Procedure

1. Create a parameter field and call it [SortField](#).
2. Select [String](#) from the [Type](#) list.
You may want to limit the number of characters the user can type to one.
3. To do this, type the numeral 1 into the [Min Length](#) and [Max Length](#) text boxes and click [OK](#).
Now the parameter field will only accept single-character values. The field will accept "C" as a value, but not "City."
4. In the [Prompting Text](#) field, enter a prompt similar to this:

Type R to sort by Region or C to sort by City; otherwise, data will be sorted by Country.

5. Create a formula similar to this and call it [Sort](#):

```
If {?SortField} = "C" Then {customer.CITY}
Else
    If {?SortField} = "R" Then {customer.REGION}
    Else
        {customer.COUNTRY}
```

This formula prompts for a value for the parameter field {?SortField}. If you enter "C", the formula will sort by the City field. If you enter "R" it will sort by the Region field. If you enter anything else, or do not enter anything at all, the formula will sort by the Country field.

For more information see "If statements" in the online help.

6.  Place the formula in the Report Header section of the report and select [Suppress \(No Drill-Down\)](#) in the Section Expert so that it does not print.
7.  Click [Sort Records](#).

8. Choose your formula and click [Add](#).
9. Click [OK](#).

Results

Now when you run the report, the program will prompt you for a sort field, the formula will return a value based on your selection, and the sort facility will use that value as your sort field.

28.12.7 Defining entry type and format using the Edit Mask

Procedure

1. On the [View](#) menu, click [Field Explorer](#).



2. Select [Parameter Fields](#) and click [New](#).

The Create New Parameter dialog box appears.

3. Enter the Name and value Type.
4. For a string parameter field, you can choose to enter an edit mask in the [Edit mask](#) field, rather than specifying a range.

An edit mask can be any of a set of masking characters used to restrict the values you can enter as parameter values (the edit mask also limits the values you can enter as default prompting values).

You can enter any of the following masking characters, or any combination of them:

- "A" (allows an alphanumeric character and requires the entry of a character in the parameter value).
- "a" (allows an alphanumeric character and does not require the entry of a character in the parameter value).
- "0" (allows a digit [0 to 9] and requires the entry of a character in the parameter value).
- "9" (allows a digit or a space, and does not require the entry of a character in the parameter value).
- "#" (allows a digit, space, or plus/minus sign, and does not require the entry of a character in the parameter value).
- "L" (allows a letter [A to Z], and requires the entry of a character in the parameter value).
- "?" (allows a letter, and does not require the entry of a character in the parameter value).
- "&" (allows any character or space, and requires the entry of a character in the parameter value).
- "C" (allows any character or space, and does not require the entry of a character in the parameter value).
- ". , : ; - /" (separator characters). Inserting separator characters into an edit mask is something like hard coding the formatting for the parameter field. When the field is placed on the report, the separator character will appear in the field object frame, like this: LLLL/0000. This example depicts an edit mask that requires four letters followed by four numbers.
- "<" (causes subsequent characters to be converted to lowercase).

- ">" (causes subsequent characters to be converted to uppercase).
- "\" (causes the subsequent character to be displayed as a literal). For example, the edit mask "\A" would display a parameter value of "A." If the edit mask is "00\A00," then a valid parameter value would consist of two digits, the letter "A," and then two additional digits.
- "Password". Allows you to set the edit mask to "Password," you can create conditional formulas specifying that certain sections of the report become visible only when certain user passwords are entered.

Note

Some of the edit mask characters require that you enter a character in their place (when entering a parameter value), while others allow you to leave a space, if needed. For example, if the edit mask is 000099, you can enter a parameter value with four digits, five digits, or six digits, since the '9' edit mask character does not require the entry of a character. However, since '0' does require such an entry, you could not enter a parameter value with less than four digits.

5. Click [OK](#).
6. Return to the Field Explorer, and drag the parameter into the report.

28.12.8 Creating a saved-data record filter using parameter fields

You can design your report so that users can filter fields and records without refreshing information from the database. Saved-data record filters combine the use of saved data selection formulas and the [Parameter Panel](#).

This feature is particularly useful for several reasons:

- It allows users to filter and re-filter report data without leaving the canvas of the report.
- It eliminates processing demand on the database.
- It reduces the time users spend waiting for data to be filtered.
- It allows users to filter records in the report even if they do not have a connection or rights to access the database at the time of viewing.

When planning to include a saved-data record filter in your report, it is important to consider the following points:

- To prevent report refreshing to the database, only non-data parameters should be added to the [Parameter Panel](#).
- Exported reports retain all recently applied parameter changes.
- Sub-report parameters cannot be shown on the [Parameter Panel](#).

28.12.8.1 To create a saved-data record filter using parameter fields

Procedure

Combine the tasks below to create an interactive saved-data record filter.

- a. Create a parameter appearing on the Parameter Panel
- b. Incorporate a parameter into a saved data selection formula

Related Information

[To create a parameter appearing on the Parameter Panel \[page 866\]](#)

[To incorporate the parameter into a saved data selection formula \[page 867\]](#)

28.12.9 To create a parameter appearing on the Parameter Panel

Procedure

1. On the *View* menu, click *Field Explorer*.

→ Tip



Another way to do this is to click the *Field Explorer* button on the *Standard Toolbar*.

The *Field Explorer* dialog box appears.

2.  Select *Parameter Fields* and click *New*.

The *Create New Parameter* dialog box appears.

3. Enter a name for the parameter in the *Name* field (up to 255 alphanumeric characters).
4. Select the appropriate *Type* from the list.

ⓘ Note

When creating a parameter whose Type is either Date or DateTime, you can change the date format to suit your needs.

5. From the *Value Field* list, select a value.
6. Click *Actions*, and select *Append all database values* to move all of the applicable values to the *Values* area.

This example will enable the user to choose from any of the values. If you want to limit the selection, manually enter only the values that you would like the user to choose from.

7. Select *Editable* from the *Show on (Viewer) Panel* list.
This example will enable the user to edit the parameter value on the *Parameter Panel*. If you want to show the parameter on the panel, but do not want to allow the user to edit it, select *Read only* for this option.
8. Click *OK*.
9. Return to the *Field Explorer* dialog box, and drag the parameter onto your report.
The *Enter prompt values* dialog box appears.

Note

If you don't want to see the parameter field you dropped in your report, place it in a section you can suppress, such as a report header or footer.

Tip

You don't need to drag the parameter onto the report if you plan on referencing it in a conditional formatting formula or a selection formula. If you have set the parameter options to *Editable* or *Read only* on the *Parameter Panel*, using the parameter in a formula will automatically activate it on the panel.

Results

The parameter now appears on the *Parameter Panel*.

28.12.10 To incorporate the parameter into a saved data selection formula

Procedure

1. On the *Report* menu, point to *Select Expert* and then click *Saved Data*.
The *Choose Field* dialog box appears.
2. Highlight the field on which you want to base record selection and click *OK*.
The *Select Expert* appears.
3. Use the drop-down lists to enter your selection criteria for the indicated field.
4. Choose the parameter from the adjacent drop-down list and click *OK*.
If the parameter is being referenced for the first time, the *Enter Values* dialog box appears.
5. Select your initial values and click *OK*.

Results

Crystal Reports immediately filters the report data according to your selection criteria.

28.12.11 Adding dynamic grouping using parameter fields

You can design your report so that users can change the grouping presentation of the report without refreshing information from the database. For instance, users can move from a customer-focused view of the report to either a region-focused or an order-focused view. Dynamic grouping combines the use of group selection formulas and the [Parameter Panel](#).

28.12.11.1 To add dynamic grouping using parameter fields

Procedure

1. Create a report using the sample data, Xtreme.mdb, and place the following fields from left to right in the Details section:

```
{Customer.Customer Name}  
{Customer.Country}  
{Orders.Order ID}  
{Orders.Order Date}  
{Orders.Order Amount}
```

2. Create a new parameter field and call it [GroupBy](#).
3. Select [String](#) from the [Type](#) list.
4. Add the following values to your parameter:
 - Customer
 - Country
 - Order
5. Select [Editable](#) from the Show on (Viewer) Panel list, and click [OK](#).
6. Create a new formula field and call it [GroupField](#).
7. Type the following formula into the [Formula](#) text box.

```
If {?GroupBy} = 'Customer' then  
{Customer.Customer Name}  
Else if {?GroupBy} = 'Country' then  
{Customer.Country}  
Else if {?GroupBy} = 'Order' then  
ToText({Orders.Order ID})
```

Note

Crystal Reports™ formulas do not allow conditions to return different data types. Both the Customer Name and Country fields return strings, so the ToText function must be used to convert the Order ID from a number to a string as well.

8. Save your formula and close the [Formula Workshop](#).
9. In the [Group Expert](#), select the [GroupField](#) formula as your group field, and click [OK](#).
10. Customize summary fields and section formatting as you like.

Results

Users can now change the grouping presentation of the report by modifying the [GroupBy](#) parameter value on the [Parameter Panel](#).

Note

Changing parameter values will not cause a database refresh as long as the parameter is a non-data parameter.

Related Information

[Data and non-data parameters \[page 833\]](#)

29 Subreports

A subreport is a report within a report. With subreports, unrelated reports can be combined into a single report. You can coordinate data that otherwise cannot be linked and present different views of the same data in a single report. This section shows you how to create and use subreports.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

29.1 What are subreports?

A subreport is a report within a report. The process for creating a subreport is similar to the process of creating a regular report. A subreport can have most of the characteristics of a report, including its own record selection criteria. The only differences between a subreport and a primary report are that a subreport:

- Is inserted as an object into a primary report; it cannot stand on its own (although a subreport can be saved as a primary report).
- Can be placed in any report section and the entire subreport will print in that section.
- Cannot contain another subreport.
- Does not have Page Header or Page Footer sections.

There are four instances in which a subreport would typically be used:

- To combine unrelated reports into a single report. See [Combining unrelated reports by using subreports \[page 879\]](#).
- To coordinate data that cannot otherwise be linked. See [Using subreports with unlinkable data \[page 880\]](#).
- To present different views of the same data within a single report. See [Showing different views of the same data in a report \[page 883\]](#).
- To perform one-to-many lookups from a field that is not indexed on the lookup field.

Note

You can increase the performance of reports containing subreports by using on-demand subreports instead of regular, in-place subreports.

29.1.1 Unlinked vs. linked subreports

Click a link to view one of the topics in this section:

- [Unlinked \[page 871\]](#)
- [Linked \[page 871\]](#)
- [Database links vs. subreports in one-to-many situations \[page 871\]](#)

29.1.1.1 Unlinked

Unlinked subreports are free-standing; their data is not in any way coordinated with the data of the primary report.

In unlinked subreports, there is no attempt to match up the records in one report with records in the other. An unlinked subreport does not have to use the same data as the primary report; it can use the same data source or a different data source entirely. In addition, the subreport is not limited to reporting on a single table. An unlinked subreport can be based on a single table or on multiple tables. Regardless of the underlying data sources, the reports are treated as unrelated.

29.1.1.2 Linked

Linked subreports are just the opposite; their data is coordinated. The program matches up the records in the subreport with records in the primary report. If you create a primary report with customer information and a subreport with order information and link them, the program creates a subreport for each customer and includes in that subreport all the orders for that customer.

29.1.1.3 Database links vs. subreports in one-to-many situations

When two tables in a report have a one-to-many relationship, the program retrieves the data in different ways depending on:

- Data source.
- Index situation.
- Record selection criteria.
- Whether you are creating a single report based on linked tables or a primary report that contains a subreport.

When you are considering whether to use a subreport or linked tables, you need to understand the ramifications of each. These issues are discussed fully in [Performance considerations in one-to-many links \[page 903\]](#).

As a general rule, if you have indexed tables, linked indexed fields, or range limiting record selection criteria based on the indexed fields, the program needs to read the same number of records whether you are linking

tables in a single report or using subreports. Since each subreport is run as a separate report, linked tables may have a performance advantage. See [Indexed tables \[page 887\]](#).

29.1.2 How subreport linking works

When you link a subreport to a primary report, the program creates the link by using a parameter field.

When a subreport link field is selected, the program creates:

- A parameter field in the subreport which is then used to retrieve values passed to it by the primary report.
- A record selection formula for the subreport using the parameter field.
- The selection formula limits the subreport to those records in which the link field is equal to the parameter field value.

When the report is run, the program finds the first primary field record it needs and passes the value in the link field to the parameter field in the subreport. The program then creates the subreport with record selection based on the parameter field value. Here is an example:

- You create a report that shows customer data and a subreport that shows order data and then you link the two reports using the Customer ID field.
- When you run the report, the program finds the first customer record it needs and passes the Customer ID value from that record to the subreport parameter field.
- The program runs the Orders subreport. Since the subreport selection formula selects only those records in which the Customer ID value is equal to the parameter field value, and since that parameter field value is equal to the Customer ID in the first record in the primary report, the subreport contains only those records that have the same customer ID. Namely, those records that are orders for the first customer.
- When the subreport is finished, the program locates the second record it needs in the primary report, prints the customer data, and then passes this customer's ID number to the parameter field.
- The program then runs a subreport including only those order records for the second customer.
- The process continues until the report is finished.
- All of this parameter field manipulation takes place behind the scenes. You simply pick the fields that will link the primary report with the subreport and the program does the rest. The values are passed without the parameter field prompting you for a value.

Note



If you have a linked subreport and you click the Print Preview button on the Standard toolbar (from the Subreport Design tab), the program runs the subreport on its own, without waiting to receive a parameter field value from the primary report and without evaluating the tab text formula. In this case, the program displays the Enter Parameter Values dialog box and prompts you for a value.

The value entered in the box is the value the program uses to run the subreport.

29.2 Inserting subreports

Procedure

1. On the *Insert* menu, click *Subreport*.

→ Tip



Another way to do this is to click the Insert Subreport button on the Insert Tools Toolbar.

The Insert Subreport dialog box appears.

2. To choose an existing subreport, click *Choose an existing report* and type the name. If you do not know the name, click the *Browse* button and locate it in the dialog box that appears.

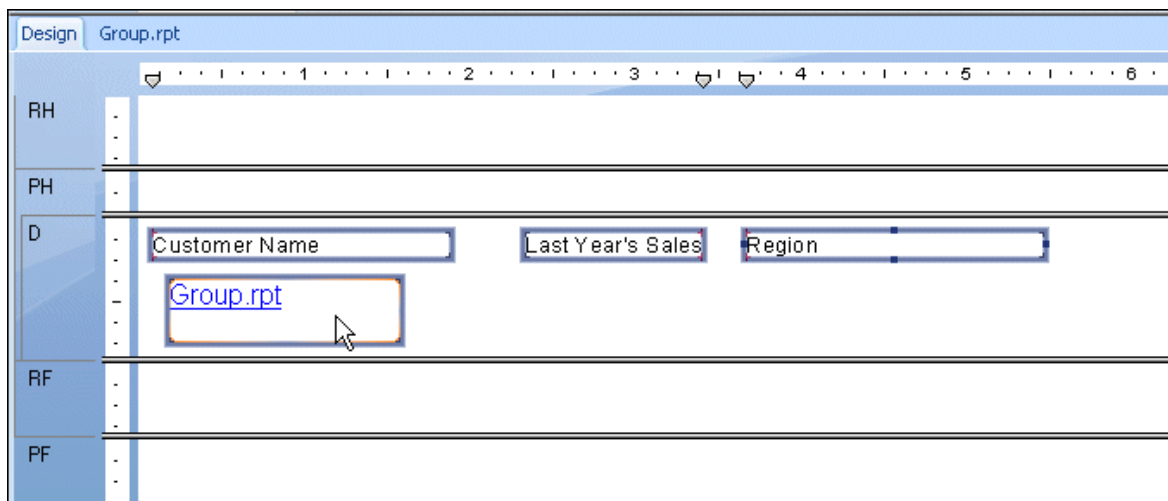
To create a new subreport, click "Create a subreport with the Report Wizard" and type a name; then click the Report Wizard button. See [Creating a new report \[page 98\]](#) for information about using a wizard to create a report.

3. Choose *On-demand subreport (similar to a hyperlink)* to have the ability to retrieve the data on the subreport when needed. Otherwise, all the subreport data will appear with the report.
4. Click *OK*.

The program displays an object frame.

5. Move the frame where you want it to appear in the report and click once to place it.

The program creates a Subreport Design tab, which is labeled with the subreport name. To edit the report, click the design tab and make your modifications.



6. Click the *Preview* tab to see your report.
7. If you chose *On-demand subreport (similar to a hyperlink)*, click the subreport preview tab to see your subreport.

This tab is labeled with the name of your subreport.

Results

Note

Using on-demand subreports will increase the performance of reports that contain subreports.

For information about creating a custom caption for the Subreport Preview Tab, see [Adding captions to on-demand subreports \[page 882\]](#).

29.2.1 Previewing subreports

There may be times when you want to preview a subreport on its own instead of previewing it as a part of the main report. For example, you may want to preview the subreport in order to view and analyze the data for a particular set of parameter values.

29.2.1.1 To preview a subreport

Procedure

1. Click the subreport's Design tab.

This tab is labeled with the name of your subreport.

2.  Click [Print Preview](#) on the Standard toolbar.

The program displays a preview of the selected subreport.

29.2.2 Saving a subreport as a primary report

You may find it advantageous to save a subreport as a primary report for the sake of distributing the information to a variety of audiences. For example, the primary report containing the subreport may be relevant for a stockholders meeting at the end of the fiscal year; however, the data contained in the subreport may be relevant for everyday use by your managers. In such cases, it is easy to save a subreport as a primary report.

29.2.2.1 To save a subreport as a primary report

Procedure

1. In the Design tab, right-click the subreport and click [Save Subreport As](#) from the shortcut menu.
The Save As dialog box appears.
2. Search for the appropriate directory in the Save As dialog box.
3. Type a new name for the subreport.
4. Click [Save](#).

The program saves the subreport as a primary report so that you can open it separately when necessary.

29.2.3 Updating subreports

In order to maintain the most up-to-date subreports, you can manually update the subreport data. However, you may want to re-import a subreport automatically when opening the main report.

Note

Re-importing is available only for subreports that were created from a report file.

Re-importing not only updates the data, but updates the formatting, grouping, and structure of the subreport if any changes have been made. If you change the report the subreport was originally based on, you can have these changes reflected in the report containing the subreport.

You can either globally specify that all subreports should be re-imported, or set the automatic re-import for an individual subreport.

29.2.3.1 To manually update the subreport data

Context

You can update your subreport data at any time.

Procedure

1. On the *Design* tab, right-click the subreport.
2. Click *Re-import subreport* from the shortcut menu.
3. Click *Yes* to update the subreport data.

29.2.3.2 To globally update subreports when opening a main report

Procedure

1. On the *File* menu, click *Options*.
2. In the Options dialog box, click the *Reporting* tab.
3. Click *Re-import Subreport on Open*.
4. Click *OK*.

Any subreport will be updated when its main report is opened and refreshed.

29.2.3.3 To update a specific subreport when opening a main report

Procedure

1. From the *Format* menu, click *Format Subreport*.
2. In the Format Editor dialog box, click the *Subreport* tab.
3. Click *Re-import When Opening*.

The current subreport will be updated when the main report is opened and refreshed.

29.3 Linking a subreport to the data in the primary report

Frequently, the data in a subreport supplements the data in the primary report. You might, for example, have customer data in a primary report and then use subreports to show the orders for each customer.

In such cases, you will need to coordinate the data in the primary report with the data in the subreport so that the orders in each subreport match up with the correct customer.

To do this, you need to specify a field that is common to both the subreport and the primary report. With the Subreport Links dialog box, you create a link between the two common fields. Crystal Reports uses the link to match up records from the primary report to those in the subreport. The link makes certain that the "orders" data in the subreport sits on the same row as the corresponding "customer" data in the primary report.

29.3.1 To link a subreport to the data in the primary report

Procedure

1. If you are creating a new subreport or importing an existing report as a subreport, from the [Insert](#) menu, click [Subreport](#). Choose or create a report and click the [Link](#) tab.

- or -

If you have already placed a subreport in the primary report, but did not create a link at setup, navigate to the Subreport Links dialog box by choosing Subreport Links from the Edit menu.

The Subreport Links dialog box appears.

2. Choose the subreport you want to link from the [For subreport](#) list (if it is not already selected).
3. Select the field you want used as a link field in the primary (containing) report from the [Available Fields](#) list.
4. Click the > arrow.

The field is added to the "Field(s) to link to" list box, and is now selected as a link field.

5. Repeat steps 3 and 4 for each additional link, as desired.
6. Use the [Field link](#) section (which will only appear if you have selected a link field) to set up the link for each link field:
 - Select the field you want linked to the primary report from the "Subreport parameter field to use."
 - Select the "Select data in subreport based on field" check box on and select a field from the adjacent drop-down list to organize the subreport data based on a specific field (this is the quick equivalent of using the Select Expert). If nothing is specified here, the subreport will adopt the organization of the primary report.
7. Click [OK](#).

Results

When you run the report, the program will coordinate the data in the primary report with the data in the subreport.

Note

The field type of the Containing Report field determines which subreport fields are visible. Because the Report Designer reads dates as either strings, dates, or date/time fields, you must make sure your subreport parameter field type matches the field type set up in Report Options in the main report for the field you want linked.

29.4 Linking a subreport to the main report without modifying the selection formula

Crystal Reports uses a parameter field mechanism for linking subreports to main reports.

When linking a main report field that is not a parameter field to a subreport field, the program:

- Automatically creates a parameter field to complete the link.
- Modifies the subreport record selection formula to select those records in which the subreport field is equal to the parameter field value.

The need for a parameter field is implied; it is called an "Implicit Link" situation.

At times, you may wish to use a linked parameter field in a subreport without using it as part of the selection formula for the subreport. For instance, you may want the main report to pass in a summary value that can be used in calculations by the subreport, or you may want the main report to pass in the title of the subreport.

When you link a field in the main report to a parameter field that you have created in the subreport, the program:

- Checks the link you have specified.
- Does not create any additional parameter fields.
- Does not modify the subreport record selection formula.

Specifying a link is called an "Explicit Link" situation.

29.4.1 To link a subreport to a main report without modifying the selection formula

Procedure

1. Create a parameter field in the subreport.

2. Link a field in the main report to that parameter field.

29.5 Combining unrelated reports by using subreports

At times, you may wish to combine unrelated reports into a single report. For example, you may want to create a single report that presents:

- Sales grouped by sales representative.
- Sales grouped by item.

While both reports deal with sales data, there is no real linear relationship between the reports.

Subreports can be used to combine unrelated reports into a single report like this. While the reports could be based on the same data set, they do not have to be. They could each be based on entirely different data sets.

Each of these reports is free-standing; the data in any of the reports is not linked in any way to data in another report. This is the easiest of the subreport options to work with.

29.5.1 To combine two unrelated reports

Procedure

1. Create the report you want printed first as the primary report.
2. Import an existing report for use as a subreport or create a new subreport.
3. Place the subreport into the Report Footer and it will print immediately after the primary report.

29.5.2 To combine three or more unrelated reports

Procedure

1. Create the report you want printed first as the primary report.
2. Import or create each of the other reports you want to use as subreports.
3. Use the Section Expert to insert enough Report Footer sections to match the number of subreports that you are using.

For example, if you want to use three subreports, insert two new Report Footer sections so that you have a total of three Report Footer sections.

4. In Report Footer A, place the subreport you want printed immediately after the primary report. In Report Footer B, place the subreport you want printed next, and so forth.

The primary report will print first and then the subreports in the order that you placed them in the report.

Note

Subreports can be placed side-by-side in the same Report Footer section. They will print next to each other at the end of the report.

5. Place the subreports into the Report Footer sections and they will print sequentially after the primary report.

29.6 Using subreports with unlinkable data

Tables can be linked in a report as long as the following criteria are met:

- The link fields are both database fields.
- The link fields contain similar data.
- The link fields are the same length.
- The link field in the link to (lookup) table is indexed (PC databases only).

Linking tables is rarely a problem. However, there are some circumstances in which you cannot coordinate data from different tables because the data does not meet the linking criteria.

For example, linking to or from a formula field, or linking two unindexed tables cannot be done in a single report. Subreports must be used.

29.6.1 Linking to/from a formula field

There are situations in which you may need to link to or from a formula (calculated) field. For example, an employee ID could be an 11 character value that consists of a two-character department code followed by the employee's nine-character Social Security Number (for example, HR555347487).

The formula language makes it easy to extract the Social Security Number from this field:

```
{employee.EMPLOYEE ID} [-9 to -1]
```

- or -

```
{employee.EMPLOYEE ID} [3 to 12]
```

For the value HR555347487, either formula would return the value 555347487.

While the return value is a valid Social Security Number, the fact that it comes from a formula prevents you from using the field to link to a Social Security Number field in another table. You can report on and coordinate the values in the two tables, however, by using a subreport.

29.6.1.1 To link to/from a formula field

Procedure

1. Create the primary report using a table that includes the Social Security Number field.
2. Create (or import) a subreport using the formula that extracts the Social Security Number from the Employee ID field (for this example, {@EXTRACT}).

See [Inserting subreports \[page 873\]](#).

3. Place the subreport where you want it to appear in the primary report.
4. Link the subreport to the primary report by linking the Social Security Number field in the primary report ({file.SSN}) to the formula that extracts the number in the subreport ({@EXTRACT}). See [Linking a subreport to the main report without modifying the selection formula \[page 878\]](#).

29.6.2 Linking unindexed tables

When using PC (not SQL or ODBC) databases, the link field in the lookup database needs to be indexed in order to create a valid link. You cannot link the tables in a single report when two tables contain related data yet neither is indexed on the field which you want to use as a link field, or when the primary table is indexed but the lookup table is not. You must use subreports if you want to coordinate the data in both tables.

Note

It is important to note that linking unindexed tables or linking from an indexed primary table to an unindexed lookup table may cause inefficient reporting. If your data set is large, this kind of report will take considerable time to run. Use this technique only if you do not have other options.

29.6.2.1 To link unindexed tables

Procedure

1. Create the primary report.
2. Create (or import) the subreport and insert it into the primary report.
See [Inserting subreports \[page 873\]](#).
3. Use the unindexed fields (or the indexed field in the primary table and the unindexed field in the lookup table) to link the subreport to the primary report. See [Linking a subreport to the data in the primary report \[page 877\]](#).

29.7 Creating an on-demand subreport

On-demand subreports can be especially useful when you want to create a report that contains multiple subreports. In this case, you can choose to have these subreports appear only as hyperlinks.

The actual data is not read from the database until the user drills down on the hyperlink. This way only data for on-demand subreports that are actually viewed will be retrieved from the database. This makes the subreports much more manageable.

Note

Data for an on-demand subreport is not saved unless the subreport is actually open in a preview window.

29.7.1 To create an on-demand subreport

Procedure

1. Place an ordinary subreport in your primary report.
2.  Click the *Format* button on the Expert Tools toolbar.
The Format Editor dialog box appears.
3. Click the *Subreport* tab and select the *On-demand subreport* check box.
4. Click *OK*.

29.8 Adding captions to on-demand subreports

To further organize a report, captions can be created for the Subreport Preview tab and for the placeholder frame of an on-demand subreport.

Captions are written by using formulas. Both placeholder frame captions and Subreport Preview tab captions can include field names from the main report.

A tab text caption replaces the subreport file name caption on the Subreport Preview tab.

Note

Placeholder frame captions only apply to on-demand subreports, while tab text captions apply to both on-demand subreports and regular subreports. Since the data from a regular subreport is visible on the Preview tab, there is no need for a frame caption when you format a regular subreport.

29.8.1 To add a caption

Procedure

-  Select the subreport and click the [Format](#) button on the Expert Tools toolbar.
The Format Editor dialog box appears.
- Click the [Subreport](#) tab.
- You can enter either an on-demand subreport caption or a preview tab caption by clicking the appropriate [Formula](#) button and opening the Formula Workshop.
- Enter your formula in the [Formula text](#) box.

Crystal syntax formula example:

```
"More Information About " + {Customer.Customer Name}
```

Basic syntax formula example:

```
formula = "More Information About" + {Customer.Customer Name}
```

Using the Xtreme.mdb sample database, these formulas would give you a caption like "More Information About Pathfinders" or "More Information About Rockshocks for Jocks."

-  Click [Check](#) to check the formula for errors. If the program finds an error, it will prompt you with a message box detailing the nature of the error.
- After fixing any errors, click [Save and close](#).
- Click [OK](#) to return to the report.

Results

Note

A tab text formula is only evaluated when you drill-down on a subreport. If you preview a subreport separately from the primary report, the formula will not be evaluated.

29.9 Showing different views of the same data in a report

Subreports can be used to provide a different view of the data in the primary report. For example, you might want to show summary values at the top of a report and details at the bottom.

This can be done in a variety of ways. The two easiest methods are:

- By creating the summary report as the primary report and the details report as the subreport. In this method, the details subreport would be placed in the Report Footer section.

- By creating the details report as the primary report and the summary report as the subreport. In this method, the summary report would be placed in the Report Header section.

Use the appropriate link fields to link the report and coordinate the data.

30 Understanding Databases

This section describes the essentials you'll need to know to understand database design and use. After describing relational databases (including those from ODBC data sources), indexing, and table linking, the section introduces the concepts of server-side processing (pushing report processing to the server level) and field mapping (re-establishing report and database field mappings after a database changes). Finally, the section focuses on common database-related tasks such as changing a database driver and creating an ODBC data source.

If you are new to databases in general, begin with the [Databases overview \[page 885\]](#).

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://community.sap.com> 

30.1 Databases overview

Though there are hundreds of Database Management Systems (DBMS) available, Crystal Reports eliminates many of the differences once it connects to the actual database files. The process of working with database files, tables, fields, and records is much the same, regardless of the actual type of data being accessed.

This section discusses several concepts and tasks common to working with database files. Using database aliases, locating moved or renamed database files, working with indexed tables, and linking tables are subjects common to anyone who designs reports in Crystal Reports. [Using SQL and SQL databases \[page 888\]](#), is especially important for anyone who accesses data in SQL databases and other database formats that are accessed through ODBC.

30.1.1 Relational database basics

The most popular architecture for database files used in the corporate world is based on the relational model. Applications that allow you to create databases with the relational model are, therefore, often referred to as Relational Database Management Systems (RDBMS).

In a relational database, data is organized in a system of rows and columns. The rows are called records, and the columns are called fields. Each record contains a collection of related data, all information relating to a specific customer, for example. Each field refers to a common type of data that exists in all records, the names of the customers, for example. Records and fields are stored in a database table. The following diagram illustrates the basic relational database model:

Customer Table

Customer ID	Customer Name	Address 1	City	Region	Postal Code
1	Bike-O-Rama	7464 St. Georges Way	Sterling Heights	MI	48358
2	The Pedellars Inc.	410 Eighth Avenue	Dekalb	IL	60148
3	Bikes 'R Us Enterprises	7429 First Boulevard	Blacklick	OH	4300
4	Cycle Sport Corporation	8287 St Georges Way	Huntsville	AL	35818
5	Sporting Wheels Inc.	480 Grant Way	San Diego	CA	92160
6	The Cyclists Company	1984 Sydney Street	Austin	TX	78770
7	Ride 'Em Cowboy Corp.	8194 Pender Avenue	Eden Prairie	MN	55360
8	XYZ Enterprises	3802 Georgia Court	Des Moines	LA	5030
9	Trail Blazer's Place	6938 Second Street	Madison	WI	53795
10	The Cyclists Incorporated	4861 Second Road	Newbury Park	CA	9134

Often, data in two different tables can be related by a common field. For example, a Customers table will have a Customer ID for each customer, and an Orders table will have the Customer ID of each customer who placed an order, demonstrating a relationship between tables. The two tables can be linked by a common field see [Linking tables \[page 902\]](#).

The following diagram displays how two tables can have a relationship:

Customer Table

Customer ID	Customer Name	Address 1	City	Region	Postal Code
1	Bike-O-Rama	7464 St. Georges Way	Sterling Heights	MI	48358
2	The Pedellars Inc.	410 Eighth Avenue	Dekalb	IL	60148
3	Bikes 'R Us Enterprises	7429 First Boulevard	Blacklick	OH	4300
4	Cycle Sport Corporation	8287 St Georges Way	Huntsville	AL	35818
5	Sporting Wheels Inc.	480 Grant Way	San Diego	CA	92160
6	The Cyclists Company	1984 Sydney Street	Austin	TX	78770
7	Ride 'Em Cowboy Corp.	8194 Pender Avenue	Eden Prairie	MN	55360
8	XYZ Enterprises	3802 Georgia Court	Des Moines	LA	5030
9	Trail Blazer's Place	6938 Second Street	Madison	WI	53795
10	The Cyclists Incorporated	4861 Second Road	Newbury Park	CA	9134

Order Table

Customer ID	Order ID	Order Amount	Order Date	Ship Via
1	1	\$41.90	94/12/02	UPS
41	1001	\$5,060.27	94/12/02	Pickup
77	1003	\$186.86	94/12/02	UPS
18	1004	\$823.05	94/12/02	Pickup
64	1005	\$0.00	94/12/03	Loomis
7	1006	\$64.90	94/12/03	Purolator
32	1007	\$49.50	94/12/03	Parcel Post
11	1008	\$2,214.93	94/12/03	Purolator
25	1009	\$29.00	94/12/03	Loomis
34	1010	\$14,872.28	94/12/03	UPS
71	1011	\$0.00	94/12/03	Purolator
66	1012	\$10,259.10	94/12/03	Loomis
28	1013	\$1,142.12	94/12/03	Parcel Post
8	1014	\$0.00	94/12/04	Purolator
72	1015	\$0.00	94/12/04	UPS
64	1016	\$563.70	94/12/04	FedEx
38	1017	\$72.00	94/12/05	Purolator
37	1018	\$115.50	94/12/05	Loomis
30	1019	\$0.00	94/12/05	Parcel Post
25	1020	\$67.80	94/12/05	FedEx

Relationships between tables based on a common field

30.1.2 Indexed tables

Creating indexes for database tables can increase the speed of data access and reduce the time it takes for the program to evaluate data. Some DBMS applications automatically index your database tables, while others require that you create an index yourself. For the best report generation performance, make sure each of your database tables has a corresponding index.

Note

Some DBMS applications do not support indexed tables. Refer to the documentation for your DBMS to find out if it supports indexes and how to create them. If your DBMS documentation does not mention indexed tables, it may not support them, and you should link tables based on common fields. The Links tab of the Database Expert can also help you determine if your tables include indexes.

Indexes organize the records in a relational database table so that data can be located easier. For example, assume you have a table with the following data:

Order#	Customer	Amount
10444	Allez Distribution	25141.50
10470	BG Mountain Inc.	19164.30
10485	Sierra Mountain	8233.50
10488	Mountain Toad	24580.50
10495	SFB Inc.	7911.80
10501	La Bomba de Bicicleta	1956.20
10511	BG Mountain Inc.	1683.60
10544	Sierra Bicycle Group	19766.20
10568	Mountain Tops Inc.	29759.55
10579	Sierra Bicycle Group	12763.95

The information in this table is organized according to the Order# field. This is fine anytime you want to look up information in the table based on order numbers. However, what if you want to look up information specific to a certain customer?

Say you want to look up all orders made by Sierra Bicycle Group. The database engine must begin by looking at the first order number in the list and checking to see if the customer name matches the request. If not, it goes to the second order number, and checks that customer name. When an order number is reached that contains the correct customer name, the database engine retrieves the information, then continues to the next order number. Using this technique, both the Order# field and the Customer field must be read for every single record in the table. This takes a long time and a large amount of computer processing effort for examining extensive database tables with thousands, or even millions of records.

Instead, you can create an index for the table based on the Customer field. Such an index might look like this:

Customer	Pointer to Order#
Allez Distribution	10444
BG Mountain Inc.	10470
BG Mountain Inc.	10511
La Bomba de Bicicleta	10501
Mountain Toad	10488
Mountain Tops Inc.	10568
SFB Inc.	10495
Sierra Bicycle Group	10544
Sierra Bicycle Group	10579
Sierra Mountain	10485

In this index, information is organized by customers, not order numbers. Also, notice that the second column actually contains pointers to specific order numbers in the original table. By using this index, the database engine can search just the information in the Customer column until it finds the customer you are interested in, Sierra Bicycle Group.

For each correct customer entry the database engine finds in the index, it looks up the matching order in the table according to the pointer in the second column of the index. Only the orders for the correct customer are read. Finally, since information in the index is organized according to the customer names, the database engine does not need to continue searching through the index or the table as soon as it finds an index entry that does not match the requested customer.

The advantage of this highly organized search through a database table according to an index is speed. Using indexes speeds up data retrieval and report generation, important factors when reporting on large database files.

30.2 Using SQL and SQL databases

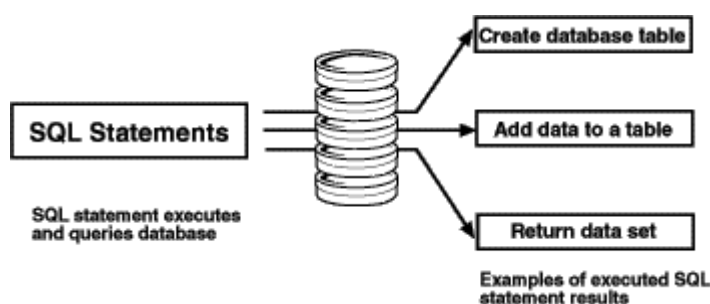
Perhaps the most popular and most powerful database formats are DBMS applications based on the Structured Query Language (SQL). SQL databases usually work over a client/server network architecture, providing an SQL Server to create, store, and manipulate database files, tables, fields and records, and an SQL Client interface allowing workstation users not only to design and work with database files, but also to retrieve useful and meaningful data that will help them in their everyday work.

30.2.1 What is SQL?

SQL is a query language designed for organizing, managing, developing and querying large relational databases over computer networks. SQL is a common language in the Information Science (IS) and Information Management industry. The language has been standardized by the American National Standards Institute (ANSI) and the International Standards Organization (ISO), meaning that there are specific features that must be present in any version of SQL produced by a software company in order for that version to be officially called SQL. Many software vendors add more advanced features to their version of SQL in an effort to improve the language and attract customers, but it must retain the original standards established by ANSI and ISO.

SQL is not a true computer language. It cannot be used to create stand-alone computer applications or operating systems. SQL is often referred to as a sub-language, since it can be used within other languages or applications. Most importantly, the purpose of the SQL language is specific to working with relational databases.

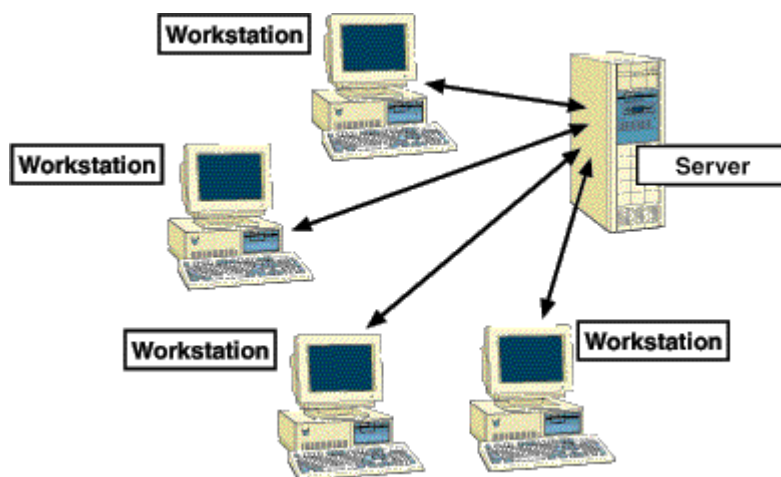
The syntax of the SQL language is built on a system of sending SQL statements to the SQL database server. Each statement is a request to perform a database operation, such as creating a database file, adding tables and fields to a database, adding records to tables, or retrieving data from databases. The SQL server analyzes the SQL statement and performs the required operation. For example, if the statement is a request for data, the server gathers the data and returns it to the client workstation for the user to view.



An SQL query is an SQL statement designed specifically to request data from one or more SQL databases. Some SQL applications require that you type in an SQL query directly using a text editor, while others provide graphical user interfaces that lead you through the process of querying an SQL database. In the latter case, the application must create an SQL statement based on the information you provide. This statement is the actual SQL query, and it is the SQL query that is used to request the data. Crystal Reports falls into both categories of SQL-compliant applications.

30.2.1.1 Client/server architecture

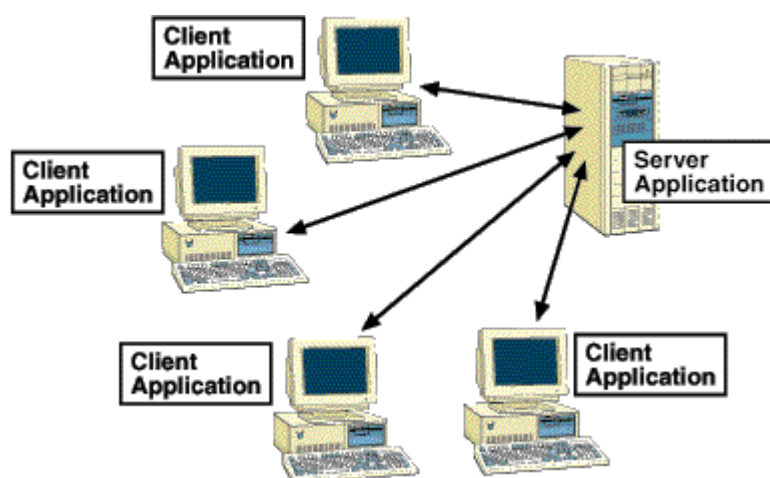
One of the most powerful features of SQL DBMS applications is their ability to efficiently use the client/server architecture of a network.



A simple network structure consists of one or more network servers that provide a common location where all users on the network can obtain data and applications. Many network servers also provide network security, automated services such as backing up data, and network resource monitoring to provide the best service possible to all workstations on the network. Because of the high processing demands required by a network server, the computer used as the server is often a high-powered, fast machine that may contain multiple processors, multiple hard drives, and multiple CD-ROM drives.

A network client is a single computer workstation that is used regularly by one or more company employees. A user works on the client and accesses data and applications from the server over the network. Large processing jobs that require a lot of time and resources are handled by the server, and the finished results are sent back to the client. This provides more efficient time management for users because the local workstation has less processing time and more "up" time available to the user.

Many modern computer applications are based on this client/server architecture. A simple client/server application has two parts: a server-based application that is located on a network server machine, and a client-based application that is located on a user's workstation. The server application handles complex, time-consuming, or power-demanding processes, taking advantage of the network server's power and resources, while the client application provides an easy-to-use user interface designed to help get the work done faster and better than it could otherwise be done.



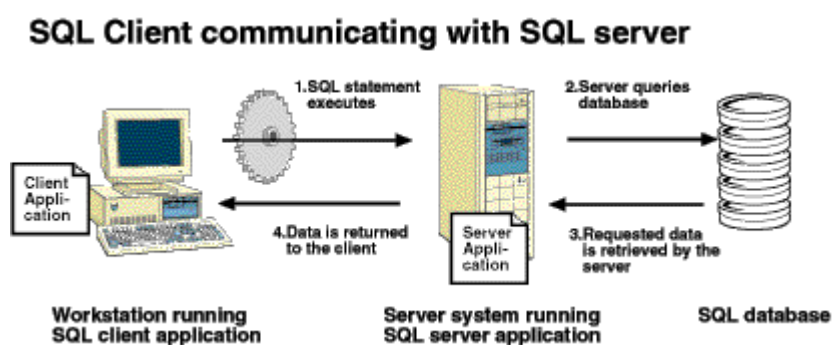
Often, a client/server application will be available with a certain number of seats, depending on the cost of the application. Each seat represents either a single client workstation, or a single client user (depending on the software vendors' specifications) that can be connected to the client/server software. Software vendors often sell additional seats for their applications, each seat coming with a complete set of client application software.

Do not confuse server applications with network server computers. Both are often referred to as servers. However, a server application resides on a network server, taking advantage of the hardware and operating system capabilities of the server machine, while a network server is a physical machine to which network clients are connected by cables or some other connection device.

30.2.2 SQL DBMS

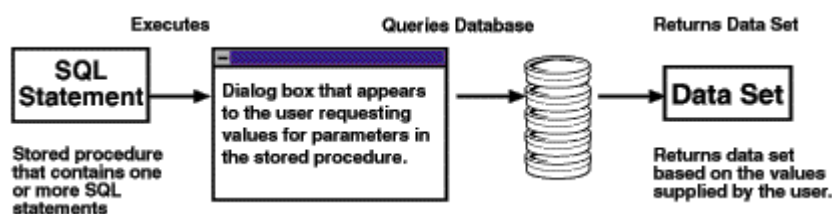
An SQL Database Management System is a common example of a client/server software package. A standard SQL DBMS will include an SQL server application that handles all the actual work of building and working with databases and database data. The DBMS will also include at least one set of SQL client software (one seat) that can connect to the SQL server over your network. SQL client software usually consists of, at the very least, an SQL statement editor that you can use to write and execute SQL statements, and an underlying communications layer that works with the SQL server application over the network.

When you execute an SQL statement, the client software passes the statement to the communications layer, which sends the statement over the network to the server software. The SQL server analyzes the statement, performs the requested operation, and returns any data requested to the client software. If the server returns any data, the client software displays the data to the user.



30.2.2.1 Stored procedures

In addition to the common relational database attributes (tables, fields, records, and so on) many SQL DBMS systems support stored procedures. A stored procedure is a compiled SQL program consisting of one or more SQL statements. A stored procedure can be used to define an SQL query that you can use over and over again. Furthermore, variables, conditional expressions, and variable arguments can be defined in the stored procedure so that you are prompted to provide information before the procedure is executed.



Since stored procedures can return a result set, they can provide a specific set of data when executed. In fact, Crystal Reports allows you to execute a stored procedure on an SQL database and use the returned data to design a report. If the stored procedure is designed to prompt a user for information to base its query on, Crystal Reports will prompt you for that information when you select the stored procedure for your report.

30.2.2.1.1 To select an SQL stored procedure

Procedure

1. Choose *Options* from the *File* menu.

The Options dialog box appears.

2. Click the *Database* tab and ensure that *Stored Procedures* is selected.

Selecting "Stored Procedures" automatically displays any available stored procedures when you log on to an SQL database.

3. In addition, you can specify *Table name LIKE* and *Owner LIKE* options if you wish.
 - "Table name LIKE" is based on the SQL LIKE clause. This option allows you to specify the kinds of table names you want to appear in the Choose SQL Table dialog box. You can use the underscore character (`_`) or the percent sign character (`%`) as wildcards with this function. The underscore character specifies any single character, while the percent sign signifies any character string. For example, `DAV_` matches `DAVE` only, while `DAV%` matches `DAVE` and `DAVID`. Table name LIKE `C%` would display only those tables that have a table name beginning with the letter `C`.
 - "Owner LIKE" is also based on the SQL LIKE clause. The Owner LIKE option allows you to select the Owner (or Creator or Alias) of the table, not the table name itself. For example Owner LIKE `C%` would display only those tables that have an owner beginning with the letter `C`.

Note

For more information on the SQL LIKE clause, refer to your SQL documentation.

4. Click *OK* to exit the Options dialog box.
5. On the Start Page, click *Blank Report*.
6. Locate and select the SQL Server data source that contains the stored procedure you want to use.
7. Click *Next* to go to the Connection Information dialog box.
8. Enter the required information to log in.

Note

You can also connect the SQL data source by selecting the *Integrated Security* checkbox. No user ID or password is required with this option, but it does require that Database Authentication is enabled on the server. Please consult your Database Administrator on how to enable and configure this function.

9. Click *Finish*.

30.2.2.1.2 To change a parameter in a stored procedure

Procedure

1. Highlight an SQL stored procedure in the Stored Procedures folder, and click the > arrow to add it to the *Selected Tables* list.

The Enter Parameter Values dialog box appears.

2. Highlight a parameter in the *Parameter Fields* list.
3. Assign a value by typing into the *Discrete Value* box and then click *OK*.

You are returned to the Database Expert.

4. Click *OK* and create your report using the fields in the stored procedure.

Results

The first time you refresh your report's data the parameter you entered is used. Only the records that satisfy that value are used in your report. Subsequent refreshing prompts for new parameter values.

30.2.3 How does Crystal Reports use SQL?

When you connect to an SQL database, Crystal Reports acts as an SQL client application, connecting to your SQL server through your network.

When you design a report that accesses SQL data, Crystal Reports builds an SQL query. This query can be seen by choosing Show SQL Query from the Database menu.

This SQL query is a representation of the SQL statement that Crystal Reports sends to the SQL server. By interpreting as much as possible from the report design into an SQL query, Crystal Reports can off-load much of the report processing onto the server machine. Rather than having to sift through an entire database to find the data you requested, Crystal Reports lets the server do the sifting and gets back a much smaller set of data, thus reducing the time and resources your workstation must use in order to finish the report.

30.2.4 The SQL language

Since Crystal Reports uses the SQL language to access client/server databases through ODBC, you can better understand the report generating process by understanding some of the SQL clauses (commands) used:

30.2.4.1 SELECT

The SELECT clause indicates specific data items to retrieve from the database tables. The item retrieved may be the values in a database field (column), or it may be the result of a calculation performed while gathering the data. For example:

```
SELECT
    TABLEA. 'CUSTNAME' ,
    TABLEA. 'STATE '
```

30.2.4.2 DISTINCT

The DISTINCT clause forces the query to retrieve only unique (distinct) sets of data. When using the DISTINCT clause, a row of results will be retrieved only once. The previous SELECT statement can be modified to use the DISTINCT clause:

```
SELECT DISTINCT
    TABLEA. 'CUSTNAME' ,
    TABLEA. 'STATE '
```

30.2.4.3 FROM

The FROM clause indicates the sources of the database fields specified in the SELECT clause. FROM lists actual database tables that include the fields and records containing the requested data. The FROM clause generated by Crystal Reports precedes the name of each table with the alias it uses to identify the table in your report. The following example illustrates the FROM clause used with the SELECT clause:

```
SELECT
    TABLEA. 'CUSTNAME' ,
    TABLEA. 'STATE '
FROM
    'TABLEA' TABLEA
```

30.2.4.4 WHERE

The WHERE clause has two purposes:

- To specify record selection criteria.
- To show how two database tables are joined.

When WHERE is used to specify record selection criteria, it includes a search condition in order to determine which records (rows of data) are to be retrieved. For example:

```
SELECT
    MYTABLE. ' SALESPERSON' ,
    MYTABLE. ' SALESTOTAL '
FROM
    'MYTABLE' MYTABLE
WHERE
    MYTABLE. ' SALESTOTAL ' < 10000.00
```

When WHERE is used to specify how two tables are linked, an SQL join operator sits between the two table names. See [Linking options \[page 911\]](#).

The following is an example of the WHERE clause joining two tables:

```
SELECT
    CUSTOMER. 'CUST_ID' ,
    CUSTOMER. 'CUST_NAME' ,
    ORDERS. 'AMOUNT'
FROM
    'CUSTOMER' CUSTOMER,
    'ORDERS' ORDERS
WHERE
    CUSTOMER. 'CUST_ID' = ORDERS. 'CUST_ID'
```

30.2.4.5 ORDER BY

The ORDER BY clause specifies that the database records retrieved be sorted according to the values in a specific field. If you do not use the ORDER BY clause, the program retrieves records in the order in which they appear in the original database. If you specify more than one field after the ORDER BY clause, the program sorts the records according to the values in the first field specified, then, within that sort, the program sorts the records by the values in the second field specified, and so on. The following SQL statement uses the ORDER BY clause:

```
SELECT
    MYTABLE. 'COMPANY' ,
    MYTABLE. 'CITY' ,
    MYTABLE. 'STATE'
FROM
    'MYTABLE' MYTABLE
ORDER BY
    MYTABLE. 'STATE' ASC,
    MYTABLE. 'CITY' ASC
```

📌 Note

ASC indicates that the values in the field are sorted in ascending order rather than descending order (DESC). Ascending order sorts letters from A to Z and numbers from 0 to 9.

30.2.4.6 GROUP BY

The GROUP BY clause retrieves a set of summary data. Instead of retrieving the data itself, GROUP BY groups the data and summarizes each group with an SQL aggregate function. The server returns only the summarization information for each group to Crystal Reports.

For example:

```
SELECT
    MYTABLE. 'STATE' ,
    MYTABLE. 'ZIPCODE' ,
    SUM (MYTABLE. 'SALES' )
FROM
    'MYTABLE' MYTABLE
GROUP BY
    MYTABLE. 'STATE' ,
    MYTABLE. 'ZIPCODE'
```

30.2.5 Defining an SQL Command

If the database you are using supports a query language such as SQL, you can write your own command which will be represented in Crystal Reports as a Table object. This allows database users complete control of the data processing that gets pushed down to the database server. A user who has experience with databases and the SQL language can write a highly optimized command that can considerably reduce the size of the set of data returned from the server.

You can write your own command by using the [Add Command](#) node in the [Database Expert](#) to create a virtual table which will represent the results of processing the command. Once the command has been created, you can store it in the SAP BusinessObjects Enterprise Repository so that it can be shared between many users.

When you use a virtual table that was created as a Command, or when you use a Command from the SAP BusinessObjects Enterprise Repository, Crystal Reports does not alter the syntax of the SQL submitted to the server (that is, it does not automatically add quoting or escape characters). This behavior also applies to parameters used in Commands. Therefore, you must add the quoting and escape characters that are necessary for your database driver.

ⓘ Note

Some of the Crystal Reports native database drivers do not support the Add Command feature:

- DB2 Server
- Sybase Server
- Informix Online Server

ⓘ Note

The Add command feature is susceptible to SQL Injection attacks.

Many web pages use SQL Injection to insert user parameters into SQL queries that run against the relational database. For example, a login webpage takes a user name and password and executes an SQL Injection to the database to authenticate the information. An attacker can put a value in the username or password field that adversely changes the SQL query that runs against the database.

30.2.5.1 To create a command object

Procedure

1. In the Database Expert, expand the [Create New Connection](#) folder and browse the subfolders to locate your data source.
2. Log on to your data source if necessary.
3. Under your data source, double-click the [Add Command](#) node.
4. In the [Add Command to Report](#) dialog box, enter an appropriate query/command for the data source you have opened.

For example:

```
SELECT
    Customer.`Customer ID`,
    Customer.`Customer Name`,
    Customer.`Last Year's Sales`,
    Customer.`Region`,
    Customer.`Country`,
    Orders.`Order Amount`,
    Orders.`Customer ID`,
    Orders.`Order Date`
FROM
    Customer Customer INNER JOIN Orders Orders ON
        Customer.`Customer ID` = Orders.`Customer ID`
WHERE
    (Customer.`Country` = 'USA' OR
    Customer.`Country` = 'Canada') AND
    Customer.`Last Year's Sales` < 10000.
ORDER BY
    Customer.`Country` ASC,
    Customer.`Region` ASC
```

Note

- Crystal Reports supports SQL query language for data retrieval like SELECT, FROM, WHERE, GROUP BY, ORDER BY and UNION. However, SQL query language data manipulation functions such as CREATE TABLE, ALTER TABLE, DROP TABLE, INSERT, UPDATE or DELETE are not supported.
- The use of double or single quotes (and other SQL syntax) is determined by the database driver used by your report. You must, however, manually add the quotes and other elements of the syntax as you create the command.

5. Optionally, you can create a parameter for your command by clicking [Create](#) and entering information in the [Command Parameter](#) dialog box.

For more information about creating parameters, see [To create a parameter for a command object \[page 898\]](#).

6. Click [OK](#).

You are returned to the Report Designer. In the Field Explorer, under Database Fields, a Command table appears listing the database fields you specified.

Note

To construct the virtual table from your Command, the command must be executed once. If the command has parameters, you will be prompted to enter values for each one.

Note

By default, your command is called Command. You can change its alias by selecting it and pressing F2.

30.2.5.2 To edit a command object

Procedure

1. In the Selected Tables area of the Database Expert, select the command you want to edit.
2. Right-click the command and select [Edit Command](#) from the shortcut menu.

Note

If the command is stored in the BusinessObjects Enterprise Repository, you must disconnect it before editing. For more information, see [Modifying objects in the repository \[page 129\]](#).

3. Make the changes you want in the Modify Command dialog box and click [OK](#) when you're done.

30.2.5.3 To create a parameter for a command object

Context

You can create a parameter field while working in the Add Command to Report dialog box or the Modify Command dialog box.

Procedure

1. In the Parameter List area of the appropriate command dialog box, click [Create](#).
2. In the Command Parameter dialog box, enter the following information in the fields provided:
 - [Parameter Name](#)
Enter the name you want to identify your parameter by.
 - [Prompting Text](#)
Enter the text you want to appear when the program prompts you.
 - [Value Type](#)
Select the data type of the parameter field.

- [Default Value](#)

Enter the value you want the program to use if you do not supply a new value. This is an optional step.

3. Click [OK](#).

Results

Your parameter is added to the Parameter List. You can modify or delete it by returning to the Modify Command dialog box.

30.2.5.4 To add a parameter to a command object

Procedure

1. Place the cursor in the query text in the spot where you want to insert the parameter.
2. Double-click the parameter's name in the [Parameter List](#).

The parameter is added where your cursor is sitting. It should look like this:

```
WHERE
Customer.`Country` = '{?CountryParam}'
```

Note

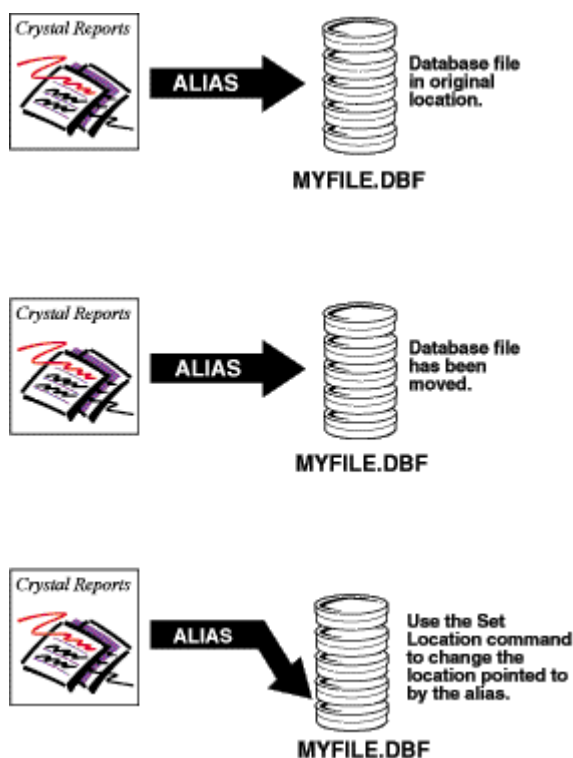
You must manually add quotes and other elements of syntax as you create your command.

Results

If you want to change the parameter field you've created, go to the Field Explorer dialog box and select it from the Parameter Fields node. See [Parameter Fields and Prompts \[page 831\]](#) for more information.

30.3 Changing data sources

When a database file is moved or renamed, Crystal Reports will not be able to find the data the next time the report is printed. On other occasions, a report may be created on one machine where all of the database data is stored in a certain directory, then the report is copied or moved to another machine that stores the same data in a different directory. In any of these events, you need to verify the location of the database files accessed by the report and reset the alias pointers to the new database location or name.



The Verify Database command on the Database menu checks the alias pointers stored in a report file to verify that the database files expected are located in the indicated directories. If the databases are not found in the specified location, the program notifies you of the discrepancies.

Use the Set Datasource Location command on the Database menu to change the alias pointers stored by Crystal Reports. The Set Datasource Location command provides a simple way to indicate the new name or location of database files. In addition, the Set Datasource Location command automatically converts your database driver to the data source you have chosen. For example, you can automatically convert a direct access data source to an ODBC data source using the Set Datasource Location command.

30.3.1 To change the data source

Procedure

1. On the *Database* menu, choose *Set Datasource Location*.

The Set Location dialog box appears with the database tables currently used in the report shown in the Current Data Source area. The main report is shown as the root node.

2. Highlight the database or table in the Current Data Source area that you want to replace.

→ Tip

If you want to set the database location to the location of the active report, expand the report's Properties node and right-click the Database Name field. Select "Edit" to manually enter a location, "Same as Report" to set the location to a local drive, or "Convert to UNC" to set the location to a mapped drive.

3. Highlight the database or table in the "Replace with" area that you want to use instead.
4. Click [Update](#).

This button is available only when like items are selected (that is, if you select a table in the upper list, you must select a table in the lower list).

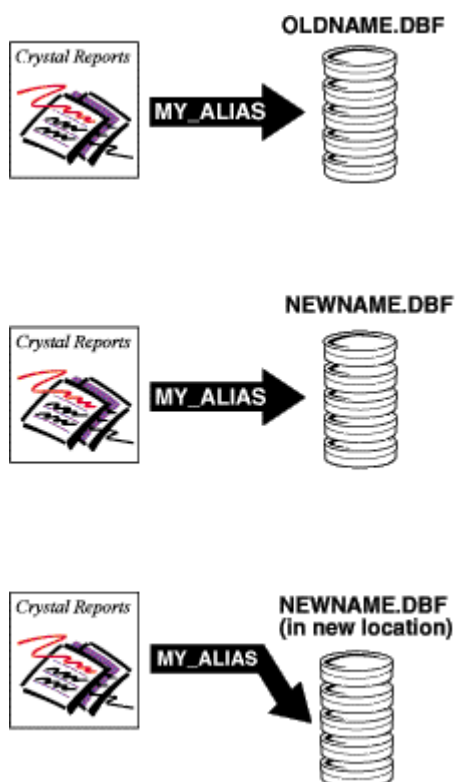
Once the Update button is clicked, the new data source information is updated in the Current Data Source list.

5. Repeat steps 2 to 4 if necessary.

30.4 Working with aliases

For a variety of reasons, database names and locations get changed. If you create a report, then change the name or location of a table or file, the Report Designer must be able to find the new name or location. This is especially important when you create formulas in your report that access a table that has been renamed or moved. To fix the reference for a single field would not be difficult, but to find every formula that uses that field could be a difficult and time consuming task.

To solve this problem, the Report Designer uses aliases to refer to database tables and files. Aliases are pointers, internal devices that tell the program where it should look for a database field. Now, if you change the name or location of the database, you simply reset the pointer. See [Changing data sources \[page 899\]](#). The name of the alias does not change, so your formulas are not affected. The Report Designer looks to the alias for the location and name, goes to the new location for the database field, and executes the formula without a problem.



The Report Designer automatically assigns default alias names to database tables when you first select the table or file. By default, an alias matches the original name of the table. In databases where the database table is a separate file (for instance, dBASE), the name of the database file is used without the file name extension. For example, if you are using the dBASE database file Company.dbf, the program will assign a default alias name of Company to the file. You can accept the default alias or assign a new one to the database table.

You can change an alias at any time using the Database Expert. However, if you have already created formulas in your report using the original alias name, you will need to edit the formulas to use the new alias.

30.5 Linking tables

You link tables so records from one table will match related records from another. For example, if you activate an Orders table and a Customers table, you link the tables so that each order (from the Orders table) can be matched up with the customer (from the Customer table) that made the order.

When you link, you are using a field that is common to both tables. Crystal Reports uses the link to match up records from one table with those from the other. In this example, the link assures that the data in each row of the report refers to the same order.

30.5.1 Link from and link to

When you link two tables, you link from one table to another table. The from table is used as a primary table, while the to table acts as a lookup table where records are looked up by the primary table. In a simple link, the Report Designer examines the first record in the primary table and finds all matching records in the lookup table. Once all matches have been found in the lookup table for the first record in the primary table, all matches in the lookup table for the next record in the primary table are found.

30.5.2 Link relationships

When you link records from one table to another table, the records will typically fall under one of two relationship types: one-to-one or one-to-many.

30.5.2.1 One-to-one relationships

In a one-to-one relationship between records in two linked tables, for every record in the primary table there is only one matching record in the lookup table (based on the linked fields). For example, in the Xtreme.mdb database, the Employee table can be linked to the Employee Addresses table based on the Employee ID field in each table. The Employee table contains information about employees at the company, the positions they hold, their salaries, hiring information, and so on. The Employee Addresses table contains each employee's home

address. There is only one record for each employee in each of these tables. Therefore, if the Employee table is linked to the Employee Addresses table, only one record will be found in the Employee Addresses table for each record in the Employee table. This is a one-to-one relationship.

30.5.2.2 One-to-many relationships

In a one-to-many relationship between records in two linked tables, for every record in the primary table, there may be more than one matching record in the lookup table, based on the linked fields. In the Xtreme.mdb database, the Customer table can be linked to the Orders table based on the Customer ID field in each table. The Customer table contains information about each customer that has placed an order with the company. The Orders table contains information about orders that customers have placed. Since customers can place more than one order, there may be more than one record in the Orders table for each customer record in the Customers table. This is a one-to-many relationship.

30.5.3 Performance considerations in one-to-many links

The information provided in this section is intended to help you maximize processing speed and minimize network traffic when you are running your reports. You will learn about the best ways to use selection formulas and indexes in one-to-many situations to make your reporting more efficient. If you do not use the information in this section, your reports may end up processing dozens or even thousands more records than necessary.

When a one-to-many situation exists between two database tables and the program matches up records from the tables, there are a number of factors that determine how many records the program reads and evaluates.

The tables that follow show the effects of the different factors on the number of records the program ultimately has to read. The charts are based on these assumptions:

- Table A contains 26 records (one for each letter in the alphabet).
- Table B contains 2600 records (100 matching records for every record in Table A).
- The scenario is to produce a report that finds two specific records in Table A and the 200 records (100+100) in Table B that match those two records in Table A.
In a best case scenario, the program would only have to read about 200 records to accomplish the task.
In a worst case scenario the program would have to read about 67,600 records to accomplish the same task.

Note

The performance considerations for data files are different from the considerations for SQL databases. A data file is any non-SQL database that is accessed directly from Crystal Reports. For the purpose of this discussion, an SQL database is any database capable of accepting SQL commands accessed directly from Crystal Reports or through ODBC as well as any other database types that are accessed through ODBC.

For a better understanding of the difference between direct access databases and ODBC data sources, see *Accessing Data Sources* in the online help.

For a better understanding of the difference between direct access databases and ODBC data sources, see the *Accessing Data Sources* chapter in the *Crystal Reports User Guide*.

30.5.3.1 Extended descriptions of chart columns

The performance charts use the following columns:

- **Linking or Subreport**
Are you creating a report from linked databases or are you inserting a subreport and binding it to the data in your primary report?
- **Selection Formula**
Does your primary report include a record selection formula that sets range limits on the key (indexed) field in Table A?
- **Index A**
Is Table A on the field you are going to use indexed to match up the records?
- **Index B**
Is Table B on the field you are going to use indexed to match up the records?
- **Reads A**
How many records does the program have to read out of Table A to find the two records it is looking for?
- **For each A reads in B**
How many records does the program have to read in Table B to find the 200 records it is looking for?
- **Total Records Read**
What is the total number of records the program has to process to complete the task?

PC Data

Linking/Subreport	Selection Formula	Index A	Index B	Reads A	For each A reads in B	Total Records Read
Linking	No	Yes or No	Yes	26	100 (26*100)	2600
Linking	Yes	No	Yes	26	100 (26*100)	2600
Linking	Yes	Yes	Yes	2	100 (2*100)	200
Subreport	No	No	No	26	2600 (26*2600)	67,600
Subreport	No	Yes	No	2	2600 (26*2600)	67,600
Subreport	No	Yes	Yes	26	100 (26*100)	2600
Subreport	Yes	No	No	2	2600 (2*2600)	5200
Subreport	Yes	No	Yes	26	100 (26*100)	2600
Subreport	Yes	Yes	Yes	2	100 (2*100)	200

SQL Data

Linking/Subreport	Selection Formula	Reads A	For each A reads in B	Total Records Read
Linking	No	26	100 (26*100)	2600
Linking	Yes	2	100 (2*100)	200
Subreport	No	26	100 (26*100)	2600
Subreport	Yes	2	100 (2*100)	200

30.5.4 Data file considerations

When working with data files, one-to-many links can occur when you link tables in a single report or when you add a subreport to your report.

30.5.4.1 Linking data files

When retrieving data from linked data files in one-to-many situations, the program uses the following process:

- If there is a selection formula, the program parses the selection formula and passes what it can down to the database DLL. This is generally range limit information. Consider the following record selection formula:

```
{customer.REGION} in "CA" to "IL" AND  
Remainder ({customer.CUSTOMER ID}, 2)=0
```

In this formula, the part before the "and" operator contains range selection criteria for the Region field. The region must fall alphabetically between "CA" and "IL." The program passes this kind of condition down to either the database DLL (for PC data) or the server (for SQL data). See [Record Selection \[page 155\]](#).

The second half of the selection formula, however, requires processing that must be done in the Report Engine. It uses a built-in function to manipulate and evaluate a field value and it cannot be done in the database DLL or the server. The program does not pass this condition to the database DLL.

- If there is an index on Table A, and the range limit selection condition is based on the indexed field ({customer.REGION} in this example), the program goes directly to the record it is seeking in Table A (the first CA record) and reads it.
 - For that record, the program locates the first matching record in Table B, using the Table B index.
 - The program passes this merged record (A+B) back to the Report Designer, which tests the record against the entire selection formula.
 - The program then reads the second matching record and passes the merged record on, and then reads the third matching record, and so on, until it has read all of the matching records.
 - The program then returns to Table A and reads the next record. There is no need to test the record to see if it meets the CA condition; the field is indexed and the records are in alphabetic order. But the program tests the record to see if it goes beyond the "IL" condition (for example, could the next record

be from Mississippi or Tennessee?). If the record is still within the specified range, the program begins the matching process again for that record.

- The program continues the process until it has located all targeted Table A records and the matching Table B records.
To find two records in Table A and the 100 records in Table B that match the Table A records, the program reads 200 records.
- If there is no index on Table A, or if there is an index but the range limit selection condition is not based on the indexed field, the program reads the first record it finds.
 - For that record, the program uses the Table B index to locate the first matching record in Table B.
 - The program passes this merged record (A+B) back to the Report Engine, which tests it against the entire selection formula.
 - The program then locates the second matching record in Table B and passes that merged record back, then the third record, and so on, until it has located, merged, and passed back all the records in Table B that match the first record in Table A.
 - The program then moves on to the next record in Table A and begins the matching and merging process all over again.
To find two records in Table A and the 100 records in Table B that match the Table A records, the program reads 2600 records.

30.5.4.2 Subreports and data files

If your primary report is based on Table A, the subreport is based on Table B, and the records are linked, your primary considerations are as follows:

- The number of subreports that are run by the program is determined by the index and the selection formula situation in the primary report:
 - If Table A is indexed, and if the primary report has a selection formula that passes down range limit conditions for the indexed field, the program runs two subreports.
 - If Table A is not indexed, or if Table A is indexed but the selection formula does not pass down range limit conditions for the indexed field, the program runs 26 subreports.
- The number of records read for each subreport is determined by the index situation on Table B:
 - If you have an index on Table B, the program will read only the matching records (100) when it runs a subreport.
 - If you do not have an index on Table B, the program will always read every record in Table B (2600) when it runs a subreport.

30.5.5 SQL database considerations

Since indexes are not critical with SQL data, the primary concern with both linked tables and subreports is whether or not there is a selection formula in the primary report that puts range limits on Table A. See [Linking data files \[page 905\]](#).

30.5.5.1 Linked SQL tables

If there are range limit conditions in the selection formula, the program passes those conditions down to the server.

- If there is a selection formula that puts range limits on Table A, the server locates the records in Table A that satisfy the selection criteria (2), matches them up with the appropriate records in Table B (100), and returns 200 merged records to the Report Engine.
- If there is no selection formula, or if there is a selection formula that does not put range limits on Table A, the server matches up each record in Table A (26) with the appropriate records in Table B (100), and returns 2600 merged records to the Report Engine.

In either case, the Report Engine will then apply the entire selection formula to the merged records.

30.5.5.2 Subreports and SQL databases

If you are creating a primary report based on Table A and a subreport based on Table B:

- The number of subreports that are run is determined by the selection formula situation in the primary report:
 - If there is a selection formula and it passes down range limits on Table A, the program runs a subreport only for those records that satisfy range limit conditions (2).
 - If there is no selection formula, or if the selection formula does not pass down range limits on Table A, the program runs a subreport for every record in Table A (26).
- The number of records read by each subreport remains the same regardless of whether there was range limit selection on Table A. Each subreport will read only those records in Table B that match each record read in Table A (100).

30.5.6 Performance considerations for all reports

Click a link to view one of the topics in this section:

- [Consideration 1 \[page 907\]](#)
- [Consideration 2 \[page 908\]](#)
- [Consideration 3 \[page 909\]](#)

30.5.6.1 Consideration 1

With both data files and SQL databases, the program parses the entire selection formula and passes down whatever parts of the criteria it is able to translate (pass), wherever they may physically appear in the formula. Thus, if the formula finds criteria it can pass, then criteria that it cannot, then criteria that it can, it passes down the first part, skips the second, and then passes down the third.

- In the case of data files, the program passes down the criteria that it can to the database translation layer.
- In the case of SQL databases, the program passes down to the server the criteria that it can in the form of a WHERE clause.

While there are exceptions, as a general rule the program can pass down any part of the record selection formula that compares a field with a constant. Typically, this means that it can pass down any kind of record selection criteria that can be set up in the Select Expert (equal to, one of, less than, greater than, less than or equal, greater than or equal, between, starting with, or like constant).

There are two special selection formula situations that you need to consider. In these situations, the record selection formula includes multiple conditions, some of which can be passed down while others cannot.

- AND situations

```
{customer.REGION} = "CA" and  
{customer.CUSTOMER ID}[3 to 5] = "777")
```

In this situation, the program sees that it can pass down the condition before the And operator but not the condition after. Since the only records that will meet the second condition will have to meet the first as well, the program passes down the first condition, retrieves the data set that satisfies the condition, and then applies the second condition only to the retrieved data. The rule for AND situations is that the program passes down whatever conditions it can.

Note

If all of the conditions in an AND situation can be satisfied on the server or in the database DLL, the program passes them all down.

- OR situations

```
{customer.REGION} = "CA" or  
{customer.CUSTOMER ID}[3 to 5] = "777")
```

In this situation, the program also sees that it can pass down the condition before the Or operator but not the condition after. Since there are records that can satisfy the second condition without satisfying the first, passing the first condition down does not make any sense because it will retrieve an incomplete data set. In other words, even if it retrieves all the data that satisfies the first condition, it will still have to retrieve all the data in the table(s) before it can apply the second condition in Report Designer. Thus, instead of duplicating parts of the data retrieval, the program passes nothing down. It retrieves all the data and then runs both tests in Report Designer. The rule for OR situations is that the program either passes down all the tests, or none of the tests.

Note

If all the tests in an OR situation can be performed on the server or in the database DLL, the program passes them all down.

30.5.6.2 Consideration 2

To make certain the program can use the index on Table A to enhance performance, make certain:

- There is a selection formula.

- There are range limits in the selection formula on the key (indexed) field in Table A.
- Use Indexes is selected in the Options dialog box.

30.5.6.3 Consideration 3

If the fields you are using from Table A are not indexed, but there is an indexed field that you can use in your record selection request, use that field. For example, assume that you have three products (Product 1, Product 2, and Product 3) and you want to identify all sales of Product 2 in the U.S. There is no index on the Product field but there is an index on the Order Date field. Since you know that Product 2 did not begin shipping until July of 1995, you can improve speed by limiting your report to orders placed in and after July 1995 using the selection formula. In such a case, the program uses the Order Date index to retrieve only those orders from July 1995 and afterward (a small subset of the entire database) and then searches for the occurrences of Product 2 in that subset, not in the entire database.

30.5.7 The Database Expert Links tab

The Database Expert Links tab lets you easily link two or more tables. Choose the Database Expert from the Database menu to display all current tables; then choose the Links tab to display all current links.

The easiest way to link database tables is to select Auto Link in the Database Expert Links tab. Auto Link automatically chooses links for your tables based on common fields in tables or indexed fields (if your database supports indexed fields).

30.5.8 Linking indexed tables

When you are linking direct-access database tables, you link from a primary table to a field in the lookup table. The link field in the primary table can be indexed, but does not have to be. The link field in the lookup table does not have to be indexed, either, unless you aren't using a native driver for your connection.

In addition, the fields used to link two tables must have the same data type. For example, you can link a string field in one table to a string field in another table, or a numeric field in one table to a numeric field in another table, but you cannot link a numeric field in one table to a string field in another table.

- Some DBMS applications allow you to convert the field value to another data type in the index. For instance, the field in the table can be numeric, while the index converts the field value to a string. However, if you choose to use that field to link to another table, you must link to a field of the original data type. You cannot link a string value to a numeric field that has been converted to a string in the index.
- If you are linking tables from two different ODBC data sources, MS SQL Server and Oracle, for example, you can only use string fields to link the tables. String fields are stored in databases in the same manner, regardless of the data source. Other types of values, however, may not be stored the same way in different data sources, so you cannot link different data sources in Crystal Reports using anything other than string values.
- When linking direct-access database tables using native drivers (non-SQL), the only join type available is Left Outer join.

30.5.8.1 Changing the index used in linking

When using the Smart Linking feature to link tables using a field that is a component of multiple indexes (two or more), Crystal Reports selects one of the indexes for the link. That index may or may not be the one you want to use. Delete the link made by the Smart Linking feature and manually link the tables. See [Linking multiple tables \[page 99\]](#) for more information.

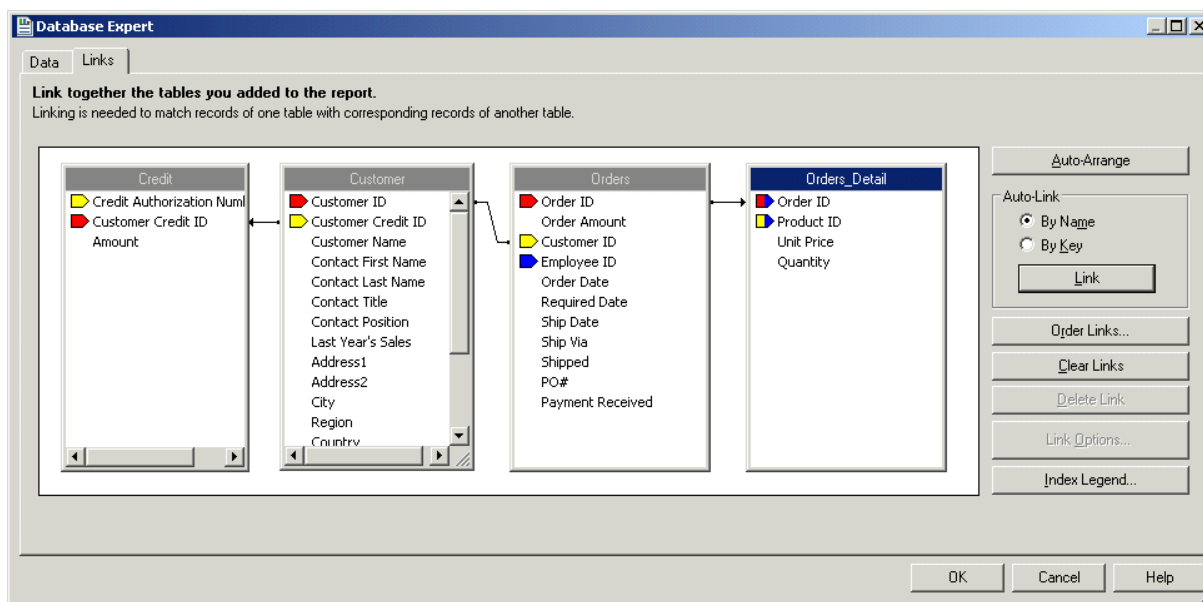
Note

Not all DBMS applications support indexed tables. Verify that your database uses indexes before trying to select an index for linking. Refer to your DBMS documentation to find out if your DBMS can use indexes, and to learn how to create them.

30.5.9 Link processing order

When there is more than one link, Crystal Reports needs to know in what order it should process the links. By default, the processing order matches the order in which the links appear on the Links tab. Use the arrow buttons in the Links Order dialog box to change the default order.

For example, if you have chosen the Credit, Customer, Orders, and Orders Details tables from the Xtreme sample database, the links tab shows the tables linked as follows.



In this case, the links will be processed first between the Credit/Customer tables, then between the Customer/Orders tables, and finally between the Orders/Orders Details tables. The Links Order dialog box shows you this default order and allows you to change the positions in the processing hierarchy as you require.

Note

Different link orders may result in different data sets returned for use in your report. As well, link order has a significant effect on performance.

30.5.10 Linking options

Crystal Reports enables you to specify the type of join and type of link you want to use when linking tables. You can also enforce the use of tables in your joins. Joins and links indicate how linked fields in two tables are compared when records are read. Join, enforce, and link options can be specified in the Link Options dialog box. Using the various join enforcement options can ensure that linked tables are included in the SQL query, even when none of the fields in the table are used in the report.

Note

When you link fields using joins, no indexed fields are required.

The join types are:

- Inner join
- Left Outer join
- Right Outer join
- Full Outer join

The enforce join options are:

- Not Enforced
- Enforced From
- Enforced To
- Enforced Both

The link types are:

- Equal [=] link
- Greater Than [>] link
- Greater Than Or Equal [>=] link
- Less Than [<] link
- Less Than Or Equal [<=] link
- Not Equal [!=] link

30.5.10.1 Inner join

An Inner join is the standard type of join. The result set from an Inner join includes all the records in which the linked field value in both tables is an exact match. For instance, you can use an Inner join to view all customers and the orders they have placed. You will not get a match for any customer who has not placed orders.

Customer Table	Customer Table	Orders Table
Customer ID	Customer Name	Order Amount
52	Allez Distribution	25141.50
53	BG Mountain Inc.	19164.30
53	BG Mountain Inc.	1683.60
57	Hansen MTB Inc.	15716.40
58	La Bomba de Bicicleta	1956.20
60	Mountain Toad	24580.50
62	SFB Inc.	7911.80
63	Sierra Bicycle Group	19766.20
63	Sierra Bicycle Group	12763.95
64	Sierra Mountain	8233.50

30.5.10.2 Left Outer join

The result set from a Left Outer join includes all the records in which the linked field value in both tables is an exact match. It also includes a row for every record in the primary (left) table for which the linked field value has no match in the lookup table. For instance, you can use a Left Outer join to view all customers and the orders they have placed, but you also get a row for every customer who has not placed any orders. These customers appear at the end of the list with blanks in the fields that would otherwise hold order information:

Customer Table	Customer Table	Orders Table
Customer ID	Customer Name	Order Amount
52	Allez Distribution	25141.50
53	BG Mountain Inc.	19164.30
53	BG Mountain Inc.	1683.60
57	Hansen MTB Inc.	15716.40
58	La Bomba de Bicicleta	1956.20
60	Mountain Toad	24580.50
62	SFB Inc.	7911.80

Customer Table	Customer Table	Orders Table
Customer ID	Customer Name	Order Amount
63	Sierra Bicycle Group	19766.20
63	Sierra Bicycle Group	12763.95
64	Sierra Mountain	8233.50
54	Bicicletas Aztecas	
55	Deely MTB Inc.	

Note

Left Outer and Right Outer joins are handled differently in the SQL language from other join types. If the database is accessed through ODBC, Crystal Reports uses ODBC syntax in the SQL statement. If you are connecting to an SQL database directly (not through ODBC), Crystal Reports uses a syntax native to the database. For more information about what an Outer join looks like in an SQL statement, refer to Microsoft ODBC documentation or to the documentation for your SQL database.

30.5.10.3 Right Outer join

The result set from a Right Outer join includes all the records in which the linked field value in both tables is an exact match. It also includes a row for every record in the lookup (right) table for which the linked field value has no match in the primary table. If you link the Customer table to the Orders table, you get one row in the table for each order a customer has placed. You also get a row for every order found that cannot be linked to a customer. Theoretically, this should not happen, but if an inexperienced sales person forgot to assign a customer ID to an order, you can quickly locate that order with a Right Outer join. The resulting table leaves a blank in any of the Customer fields for the order without a customer.

Customer Table	Orders Table	Orders Table
Customer ID	Order ID	Order Amount
52	6	25141.50
53	11	19164.30
53	21	1683.60
57	4	15716.40
58	20	1956.20
60	16	24580.50

Customer Table	Orders Table	Orders Table
Customer ID	Order ID	Order Amount
62	19	7911.80
63	28	19766.20
63	32	12763.95
64	14	8233.50
	25	10320.87

Note

Left Outer and Right Outer joins are handled differently in the SQL language from other join types. If the database is accessed through ODBC, Crystal Reports uses ODBC syntax in the SQL statement. If you are connecting to a SQL database directly (not through ODBC), Crystal Reports uses a syntax native to the database. For more information about what an Outer join looks like in an SQL statement, refer to Microsoft ODBC documentation or to the documentation for your SQL database.

30.5.10.4 Full Outer join

A Full Outer join is a bidirectional outer join where you can see all records in your linked tables. The result set from a Full Outer join includes all the records in which the linked field value in both tables is an exact match. It also includes a row for every record in the primary (left) table for which the linked field value has no match in the lookup table, and a row for every record in the lookup (right) table for which the linked field value has no match in the primary table. If you link the Customer table to the Orders table, you get one row in the table for each order a customer has placed. You also get a row for every order found that cannot be linked to a customer, and a row for every customer for whom an order cannot be found.

Customer Table	Orders Table	Orders Table
Customer ID	Order ID	Order Amount
52	6	25141.50
53	11	19164.30
53	21	1683.60
57	4	15716.40
58	20	1956.20
60	16	24580.50

Customer Table	Orders Table	Orders Table
Customer ID	Order ID	Order Amount
62	19	7911.80
63	28	19766.20
63	32	12763.95
64	14	8233.50
65		
66		
	25	10320.87

30.5.10.5 Not Enforced

When you select this option, the link you've created is used only if it's explicitly required by the Select statement. Your users can create reports based on the selected tables without restriction (that is, without enforcement based on other tables). This is the default option.

30.5.10.6 Enforced From

When you select this option, if the to table for the link is used, the link is enforced. For example, if you create a link from TableA to TableB using Enforce From and select only a field from TableB, the Select statement will still include the join to TableA because it is enforced. Conversely, selecting only from TableA with the same join condition will not cause the join to TableB to be enforced.

Note

For an explanation of from and to tables, see [Link from and link to \[page 902\]](#).

30.5.10.7 Enforced To

When you select this option, if the from table for the link is used, the link is enforced. For example, if you create a link from TableA to TableB using Enforce To and select only a field from TableA, the join to TableB will be enforced, and the Select statement that is generated will include both tables.

Note

For an explanation of from and to tables, see [Link from and link to \[page 902\]](#).

30.5.10.8 Enforced Both

When you select this option, if either the from table or the to table for this link is used, the link is enforced.

30.5.10.9 Equal [=] link

The result set from an Equal link includes all the records where the linked field value in both tables is an exact match. In the following example, the Customer table is linked to the Orders table by the Customer ID field. When the program finds a Customer ID in the Orders table that matches a Customer ID in the Customer table, it displays information for the corresponding records in both tables.

SQL uses the following syntax to describe an Equal link:

```
SELECT Customer.'Customer ID',  
       Customer.'Customer Name',  
       Orders.'Order Amount'  
FROM   'Customer' Customer,  
       'Orders' Orders  
WHERE  Customer.Customer ID =  
       Orders.Customer ID
```

This statement produces the following data:

Customer Table	Customer Table	Orders Table
Customer ID	Customer Name	Order Amount
52	Allez Distribution	25141.50
53	BG Mountain Inc.	19164.30
53	BG Mountain Inc.	1683.60
57	Hansen MTB Inc.	15716.40
58	La Bomba de Bicicleta	1956.20
60	Mountain Toad	24580.50
62	SFB Inc.	7911.80
63	Sierra Bicycle Group	19766.20
63	Sierra Bicycle Group	12763.95
64	Sierra Mountain	8233.50

30.5.10.10 Greater Than [>] link

The result set from a Greater Than link includes all records in which the linked field value from the primary table is greater than the linked field value in the lookup table. As an example, a company may want to compare the salaries made by all their sales representatives to the salaries made by all their sales managers. The company executives want to make sure no sales representative is making more money than any manager.

With this in mind, you can link the SalesRep table to the Manager table by the Salary field in each table using a Greater Than link:

```
SELECT SalesRep.'Last Name',
       SalesRep.'Salary',
       Manager.'Last Name',
       Manager.'Salary'
FROM 'SalesRep' SalesRep,
     'Manager' Manager
WHERE SalesRep.'Salary' >
      Manager.'Salary'
```

This SQL statement might produce data similar to this:

SalesRep Table		Manager Table	
Last Name	Salary	Last Name	Salary
Davolio	\$35,000.00	Fuller	\$32,000.00
Davolio	\$35,000.00	Brid	\$30,000.00
Davolio	\$35,000.00	Buchanan	\$29,500.00
Dodsworth	\$48,300.00	Hellstern	\$45,000.00
Dodsworth	\$48,300.00	Fuller	\$32,000.00
Dodsworth	\$48,300.00	Brid	\$30,000.00
Dodsworth	\$48,300.00	Buchanan	\$29,500.00
Dodsworth	\$48,300.00	Martin	\$35,000.00
Patterson	\$30,000.00	Buchanan	\$29,500.00

In this table, there is no relationship established between sales representatives and sales managers. Since all managers have seniority over all sales representatives, a company might find it necessary to check if any representatives make more money than any managers, evidence of a salary problem that needs to be remedied.

30.5.10.11 Greater Than Or Equal [>=] link

The result set from a Greater Than Or Equal link includes all records in which the linked field value in the primary table is greater than or equal to the linked field value in the lookup table. The example here is identical to the example for the Greater Than join, except that it uses the Greater Than Or Equal link:

```
SELECT SalesRep.'Last Name',
       SalesRep.'Salary',
       Manager.'Last Name',
       Manager.'Salary'
FROM 'SalesRep' SalesRep,
     'Manager' Manager
WHERE SalesRep.'Salary' >=
      Manager.'Salary'
```

This statement might produce data such as this:

SalesRep Table		Manager Table	
Last Name	Salary	Last Name	Salary
Davolio	\$35,000.00	Fuller	\$32,000.00
Davolio	\$35,000.00	Brid	\$30,000.00
Davolio	\$35,000.00	Buchanan	\$29,500.00
Davolio	\$35,000.00	Martin	\$35,000.00
Dodsworth	\$48,300.00	Hellstern	\$45,000.00
Dodsworth	\$48,300.00	Fuller	\$32,000.00
Dodsworth	\$48,300.00	Brid	\$30,000.00
Dodsworth	\$48,300.00	Buchanan	\$29,500.00
Dodsworth	\$48,300.00	Martin	\$35,000.00
Patterson	\$30,000.00	Brid	\$30,000.00
Patterson	\$30,000.00	Buchanan	\$29,500.00

30.5.10.12 Less Than [<] link

The result set from a Less Than link includes all records in which the linked field value in the primary table is less than the linked field value in the lookup table. Using the Less Than link, you can compare sales representative and manager salaries in a different direction. Once again, the Salary field in each table is used as

the link field. This time, though, you link from the Manager table to the SalesRep table using a Less Than link on the linked Salary fields:

```
SELECT Manager.'Last Name',
       Manager.'Salary',
       SalesRep.'Last Name',
       SalesRep.'Salary'
FROM 'Manager' Manager,
     'SalesRep' SalesRep
WHERE Manager.'Salary' <
       SalesRep.'Salary'
```

This SQL statement produces a slightly different table than that produced by the Greater Than link:

Manager Table	Manager Table	SalesRep Table	SalesRep Table
Last Name	Salary	Last Name	Salary
Fuller	\$32,000.00	Davolio	\$35,000.00
Fuller	\$32,000.00	Dodsworth	\$48,300.00
Brid	\$30,000.00	Davolio	\$35,000.00
Brid	\$30,000.00	Dodsworth	\$48,300.00
Buchanan	\$29,500.00	Davolio	\$35,000.00
Buchanan	\$29,500.00	Dodsworth	\$48,300.00
Buchanan	\$29,500.00	Patterson	\$30,000.00
Martin	\$35,000.00	Dodsworth	\$48,300.00
Hellstern	\$45,000.00	Dodsworth	\$48,300.00

30.5.10.13 Less Than Or Equal [<=] link

The result set from a Less Than Or Equal link includes all records in which the linked field value in the primary table is less than or equal to the linked field value in the lookup table. The example here is identical to the example for the Less Than link, except that it uses the Less Than Or Equal link:

```
SELECT Manager.'Last Name',
       Manager.'Salary',
       SalesRep.'Last Name',
       SalesRep.'Salary'
FROM 'Manager' Manager,
     'SalesRep' SalesRep
WHERE Manager.'Salary' <=
       SalesRep.'Salary'
```

This SQL statement produces data similar to the following:

Manager Table	Manager Table	SalesRep Table	SalesRep Table
Last Name	Salary	Last Name	Salary
Fuller	\$32,000.00	Davolio	\$35,000.00
Fuller	\$32,000.00	Dodsworth	\$48,300.00
Brid	\$30,000.00	Davolio	\$35,000.00
Brid	\$30,000.00	Dodsworth	\$48,300.00
Brid	\$30,000.00	Patterson	\$30,000.00
Buchanan	\$29,500.00	Davolio	\$35,000.00
Buchanan	\$29,500.00	Dodsworth	\$48,300.00
Buchanan	\$29,500.00	Patterson	\$30,000.00
Martin	\$35,000.00	Davolio	\$35,000.00
Martin	\$35,000.00	Dodsworth	\$48,300.00
Hellstern	\$45,000.00	Dodsworth	\$48,300.00

30.5.10.14 Not Equal [!=] link

The result set from a Not Equal link includes all records in which the linked field value in the primary table is not equal to the linked field value in the lookup table. This type of link can be used to find possible combinations of items when a table is joined to itself (a self-join). For example, a company can have a table listing all products they sell. When they decide to hold a sale where their customers buy one item and get the second item half price, they may need a list of all possible two item combinations:

```
SELECT Product1.'Product Name',
       Product2.'Product Name',
FROM 'Product' Product1
     'Product' Product2
WHERE Product1.'Product Name' !=
       Product2.'Product Name'
```

In this SQL statement, the Product table is opened twice. The first time, it is given the alias name Product1. The second time, it is given the alias name Product2. Then the Product Name field is used to link from the Product1 table to the Product2 table. This is the same table, but since it has been opened twice using different aliases, Crystal Reports considers it two separate tables. A Not Equal link is used to link the tables by the Product Name field. As a result, each product is paired with every other product offered, but is not paired with itself:

Product1	Product2
Product Name	Product Name
Xtreme Adult Helmet	Xtreme Mtn Lock
Xtreme Adult Helmet	InFlux Lycra Glove
Xtreme Adult Helmet	Roadster Micro Mtn Saddle
Xtreme Mtn Lock	Xtreme Adult Helmet
Xtreme Mtn Lock	InFlux Lycra Glove
Xtreme Mtn Lock	Roadster Micro Mtn Saddle
InFlux Lycra Glove	Xtreme Adult Helmet
InFlux Lycra Glove	Xtreme Mtn Lock
InFlux Lycra Glove	Roadster Micro Mtn Saddle
Roadster Micro Mtn Saddle	Xtreme Adult Helmet
Roadster Micro Mtn Saddle	Xtreme Mtn Lock
Roadster Micro Mtn Saddle	InFlux Lycra Glove

ⓘ Note

The symbol != is used to represent a Not Equal link, if the ODBC data source driver for the data being accessed supports this symbol. If not, the default symbol <> is used to represent a Not Equal link.

30.6 Server-side processing

Server-side processing allows you to set up a report that performs the majority of its processing on the server and pushes only relevant details to your computer.

Server-side processing provides you with a number of benefits:

- Less time connected to the server.
- Less memory needed to process the report on your computer.
- Lower transfer time from the server to the client.

Here's how server-side processing works: by using SQL pass-through technology to send an SQL statement to the database server and retrieve an initial set of data, Crystal Reports off-loads much of the data retrieval and sorting work onto the server system, thus freeing up local memory and resources for more important tasks. That is why server-side processing works only for reports that have been sorted and grouped; if a report has not been sorted and grouped (for example, if it is a simple list report), then there is no processing to push to the server. You should also note that server-side processing works only for reports based on SQL data sources.

Note

This description applies only to off-loading grouping and sorting to the server.

Note

Large amounts of server resources can be used for the temporary databases required when pushing grouping to the server.

Keep in mind that in order to perform the grouping on the server, your report must conform to the following conditions:

- The Perform Grouping on Server option (in the Options dialog box) is enabled. For more information, see [Enabling server-side processing \[page 923\]](#).
- The report uses some form of grouping.
- The report is at least partially hidden (at the very least, the Details section must be hidden). Since the server will process those sections that are hidden, the greater the portion of the report that is visible, the greater the amount of processing that must take place on the client side. If the Details section is shown, server-side processing will not be possible.
- In some cases, formula fields must be processed on the client side. If grouping is based on a formula field, or if a formula is used in a summary field, then all the records must be transferred to the client side before the formula can be evaluated. This will increase the amount of time required to run the report. Therefore, you may want to use SQL expressions as an alternative to formulas.

Note

Formulas used for record selection are an exception and can be pushed down to the server.

- For a report to be processed on the server, any running totals appearing in the report must be based on summary fields (since the data needed for the running totals will be pulled over to the client side).
- For a report to be processed on the server, the report must contain only summary fields of these types: Sum, Maximum, Minimum, Count.
- The report does not contain specified value grouping.

Note

When you drill-down on a hidden section of a report, with the processing being done on the server, connection to the server will be automatically initiated. If the client is disconnected from the server (for example, if you download a report onto your laptop and you work on it from a remote location), then drilling-down on data will produce an error since the database is not available.

Note

If you save a report that has been partially processed on the server using the Save Data with Report option, the program will save only those records that have been transferred to the client side. In other words, if you have drilled-down on a hidden section and there is a tab for that data in Crystal Reports (indicating the data has been transferred), those records will be saved with the report.

30.6.1 How server-side grouping affects the SQL query

When a report pushes most of its processing to the server, this by necessity alters the SQL query. Thus, when the Perform Grouping on Server option is enabled, individual aspects of server-side processing will modify the SQL statement in different ways.

- If you select Use Indexes Or Server For Speed (in the Report Options dialog box), the program adds an ORDER BY clause to the SQL statement and a WHERE clause for the record selection formula, if possible.
- If you group on a linkable data type in the DBMS, the program adds a GROUP BY clause to the SQL statement. The program uses the GROUP BY clause to perform the grouping on the server.
- If you summarize on a linkable data type, the program adds a summary field to the SELECT clause of the SQL statement.
- If you drill-down on a linkable data type, the program adds a WHERE clause to the SQL statement.
- If you group in descending order on a linkable data type, the program adds an ORDER BY clause to the SQL statement.

The statement also varies between tabs:

- If you are working in the Preview tab, the statement includes the GROUP BY clause, as well as any aggregates that the report is pushing to the server.
- If you are drilling-down, the statement varies depending on the underlying data and the level of drill-down. With each drill-down, the WHERE criteria changes. Also, if you drill-down to the details, the statement will not include a GROUP BY clause (since you no longer have any groups on that drill-down tab).

To view the current SQL statement for the active tab, choose Show SQL Query from the Database menu. The Show SQL Query dialog box appears, displaying the SQL statement.

Note

You can use the Formula Workshop to edit SQL expressions to be processed on the server.

30.6.1.1 Enabling server-side processing

Procedure

1. On the *File* menu, click *Report Options*.
2. Select *Perform Grouping on Server* on the Report Options dialog box.
This check box is inactive if Use Indexes or Server for Speed is not selected.
3. Click *OK*.

Results

Note

You can also enable or disable this option quickly by selecting or deselecting Perform Grouping on Server on the Database menu as needed. If Use Indexes or Server for Speed is not selected in the Report Options dialog box, this command is inactive.

30.7 Mapping database fields

The Map Fields dialog box allows you to link report fields to their corresponding database fields when you have made changes to the structure of the database, or when you have created a report based on one database from a template of another report based on some other database that has the same table and field structure. In this way, the dialog box helps you to make sure your reports print with the current version of the active database.

When you first create a report, the report draws its fields from the database as it exists at the time. If you change the structure of the database after you create the report, the program needs to adapt the report to the new structure.

30.7.1 About the Map Fields dialog box

The Map Fields dialog box contains four boxes:

- The upper-left box displays the names of all unmapped report fields (for which the program detects a change in the active database). The top name is selected by default.
- The upper-right box displays the names of unmapped database fields (in which the program detects a change). Since the Match Type box is selected by default, this box displays only the names of unmapped database fields of the same type as the unmapped report field you selected in the upper-left box. To display all unmapped database fields, regardless of type, clear the Match Type box.
- The lower-left box displays the names of mapped report fields. When you map fields in the upper boxes, they appear in the lower boxes.
- The lower-right box displays the names of mapped database fields. When you map fields in the upper boxes, they appear in the lower boxes.

For each database field that you have changed, highlight the report field and the database field in the upper sections and click Map. The field names move from the upper boxes to the lower boxes.

You do not have to remap every report field. For example, if you delete a database field, it is not necessary to remap its corresponding report field, since you will not need that field in your report. You can click OK to close the dialog box without remapping each field listed in the upper-left box.

On some occasions, you might want to unmap or unlink report and database fields that you have already mapped. For example, you might mistakenly map the wrong fields or want to map a report field to a database field that you have already mapped. In these cases, you can select a report field in the lower boxes (the program then automatically selects the database field) and click Unmap. The field names will move from the lower boxes to the upper boxes.

Note

Crystal Reports does not automatically refresh the report data when you close the Map Fields dialog box. To refresh your data, click Refresh on the Standard toolbar.

30.7.2 Remapping processes

If you make changes to the active database or universe fields that require you to remap the corresponding report fields, you can display the Map Fields dialog box by choosing any of these commands from the Database menu:

- Verify Database
- Verify on First Refresh
- Set Datasource Location

Note

There is also a global option called Verify When Database Driver Upgraded (set on Database tab of the Options dialog box) that can make the Map Fields dialog box appear when you first refresh a report's data after updating its database driver.

You can use each of these commands for a specific function; however, any of these commands will open the Map Fields dialog box if the program detects a mismatch between the field names in the report and the field names in the database or universe. In order to detect any possible mismatches, the program checks each field name in the report against the field names in the database or universe. If one of the field names does not match any of the field names in the database or universe, the Map Fields dialog box appears.

Note

Report fields that are left unmapped are removed from the report.

30.7.2.1 Using the Verify Database process

When you choose Verify Database from the Database menu, the program checks the active databases or universes, and reports. If it detects changes, the report must be adapted to prevent errors.

The program displays the Map Fields dialog box when it detects either of these types of changes to the database or universe:

- The name of a field that is used in the report has changed.
- The database or universe has been upsized from a PC data source to an SQL data source.

For a tutorial on the Map Fields dialog box, see [Remapping altered database fields \[page 926\]](#). Crystal Reports automatically adapts the report (and does not display the Map Fields dialog box) if it detects any of these changes:

- Fields have been added to the database or universe.

- Fields that are not used in the report have been deleted from the database or universe.
- Field positions have changed in the database or universe.
- Data types have changed for fields in the database or universe.

30.7.2.2 Using the Verify on First Refresh process

Verify on First Refresh triggers the Verify Database command the first time you refresh your report data per session:

- If there is a check mark beside Verify on First Refresh, the option is active (the option is active by default for new reports).
- If there is no check mark beside it, the option is inactive.

30.7.2.3 Using the Set Datasource Location process

When you choose Set Datasource Location from the Database menu and specify a new location for the active database or universe, the program checks the database or universe for changes.

Set Datasource Location displays the Map Fields dialog box when it detects any of the following changes in the database structure:

- A field has been deleted.
- A field has been renamed.
- The database or universe is completely new.

Note

The program checks for these changes only if the databases or universes have different names or if the name has changed. If the databases or universes have the same name, the Map Fields dialog box does not appear, and you need to verify the database or universe when you have finished setting the location. For more information, see [Using the Verify Database process \[page 925\]](#).

30.7.3 Remapping altered database fields

Use the Map Fields dialog box to remap existing report fields in the active database or universe if they have been altered.

30.7.3.1 To remap an altered database field

Procedure

1. With the report active in the *Design* tab, choose *Verify Database* from the *Database* menu.

The Verify Database message box appears.

- If the program detects no changes in the active database, the message box displays this message: "The database is up to date." In this case, click OK and return to your work.
- If the program detects a change(s) in the active database, the message box displays this message: "The database file ["table name"] has changed. Proceeding to fix up the report!"

2. Click *OK*.

If the program detects that a field name has been altered in the active database, the Map Fields dialog box appears.

Note

The program automatically adapts the report to changes in other data within the database (number of fields, field position, data type, and so on). It is not necessary to remap fields in which these changes have been made.

3. Highlight the first report field that you want to remap in the upper-left box.
4. In the upper-right box, highlight the unmapped database field to which you want to remap the selected report field.
5. Click *Map*.

The highlighted report and database fields no longer appear in the upper boxes. Instead, they appear in the corresponding lower boxes.

6. Repeat Steps 3 through 5 for each unmapped report field you want remapped.

Note

If the names of any report fields remain in the upper-left box when you exit the dialog box, the program removes them from your report.

7. Click *OK*.

The program remaps the report fields to the altered database fields.

30.8 Saved Data Indexes

You can increase the performance of a Crystal report by indexing its saved data. When you create a Saved Data Index on a particular field, Crystal Reports can more efficiently filter on that field. In particular, you will achieve considerable performance gains—especially in larger reports—by indexing fields that are referred to by record selection formulas.

Note

The benefits of Saved Data Indexes are largely unnoticeable in reports whose record selection returns fewer than 10,000 records.

Saved Data Indexes are especially useful when you schedule your Crystal reports for viewing through SAP BusinessObjects Business Intelligence platform. For example, you want to provide your users with information about yearly sales for North America, so you create a report. You also want sales representatives to see figures for their individual regions, but you don't want them to see figures for other regions, so you create a selection formula to limit the report. After you schedule the report in SAP BusinessObjects Business Intelligence platform and sales representatives view its instances, they see only figures for their own regions. If you had also created Saved Data Indexes, the representatives would see their records without having to wait for all records to be loaded.

In other words, the Saved Data Indexes allow a subset of the report's data to be accessed by a user. The indexes are invoked by applying a selection formula in SAP BusinessObjects Business Intelligence platform at view time (that is, the selection formula applies a filter to an indexed field). These selection formulas can be applied by setting them in a report viewer, or by setting them through a report processing extension. (For information about report processing extensions, see the *SAP BusinessObjects Business Intelligence Platform .NET SDK Developer Guide*.)

30.8.1 How Report Indexing works

With a non-indexed report, Crystal Reports has to look at every record in order to locate values that meet specified criteria. For instance, when a user requests a particular subset of the saved data, or when a user requests the report but only has rights to access certain records, Crystal Reports filters the saved data by checking each record for the appropriate values.

If you have indexed the saved data by one or more fields, however, Crystal Reports already knows which records contain particular values. Consequently, when a user accesses a particular subset of the saved data from the indexed field, Crystal Reports can locate and format the appropriate records more efficiently.

Once you have created Saved Data Indexes, they work entirely in the background. Users don't know that the saved data is indexed, and the grouping, sorting, or formatting of the report doesn't change at all. The indexes merely allow Crystal Reports to locate particular records quickly, without passing through the saved data in its entirety.

30.8.2 Considerations for using Saved Data Indexes

There are some things that you should consider before deciding to use Saved Data Indexes:

- Indexes work best in scenarios where a report contains a large set of data, but only small subsets of the data are viewed at one time.
- Large, complex reports can overload the Crystal Reports report engine.
For example, creating a large report with a selection formula that returns 90% of the report's data will probably take up more memory and disk space than a report file without indexes. In such a case, report processing will be slower.

- Indexes require disk space; this disk space can add up to more than the space required for the data itself. You trade the speed of applying selection formulas for the size of your report files.
- The Crystal Page and RAS servers are limited in how much caching occurs for different users when selection formulas are applied. Therefore, when a user views a report file with a selection formula and the Page Server takes up a certain amount of RAM, adding more users could cause the Page Server to use even more RAM.

30.8.3 Indexing the right field(s)

These guidelines describe the best ways to index saved data and what to avoid when indexing:

- Index fields that users frequently add to their record selection formulas.
- Index fields that are referred to by the report's record selection formula.
- Don't index all of the fields in the report.
Doing so can result in increased processing times. It is best to index only on the fields that meet the criteria specified above. If all of the fields meet these criteria, then you should prioritize the fields and index only some of them.
- Don't index fields that contain unique values only.
For instance, don't index a field such as "Last Year's Sales," whose values are likely to be distinct from one another. If you do so, a separate index is created for each and every value in the field.

30.8.3.1 To index saved data

Procedure

1. Open your report in the Crystal Reports.
2. On the [Report](#) menu, click [Report Bursting Indexes](#).
3. In the Saved Data Indexes dialog box, select the fields that you want indexed within the saved data.
4. Click [OK](#) to return to Crystal Reports.
5. If you want to create the index immediately, refresh and save the report.

30.9 Unicode support in Crystal Reports

Crystal Reports supports Unicode by converting data from non-Unicode databases as it accesses it (this data conversion happens within Crystal Reports; the data in your database is not affected). Conversion is done using the identifier of the non-Unicode data and the machine's locale setting (usually found in the Control Panel's Regional Settings). To take advantage of the Unicode support in Crystal Reports, ensure the locale setting is correct on each machine that uses Crystal Reports.

30.10 Working with databases

This section introduces several common procedures related to accessing database files from within Crystal Reports. Where practical, step-by-step instructions are provided.

30.10.1 Using Access queries

Click a link to view one of the topics in this section:

- [Opening Access queries through ODBC \[page 930\]](#)
- [Opening Access Parameter queries \[page 931\]](#)

30.10.1.1 Opening Access queries through ODBC

ODBC gives you more control over which parts of a database you can use. For this reason, using an Access query through ODBC may require a few extra steps.

30.10.1.1.1 To open an Access query through ODBC

Procedure

1. On the *File* menu, click *Options*.
2. Click the *Database* tab.
3. Ensure that the *Views* check box is selected.

Selecting "Views" automatically displays any available queries in your Access database.

4. In addition, you can specify *Table name LIKE* and *Owner LIKE* options, if you wish.
 - "Table name LIKE" is based on the SQL LIKE clause. This option allows you to specify the kinds of table names you want to appear in the Choose SQL Table dialog box. You can use the underscore character (_) or the percent sign character (%) as wildcards with this function. The underscore character specifies any single character, while the percent sign signifies any character string. For example, DAV_ matches DAVE only, while DAV% matches DAVE and DAVID. Table name LIKE C% displays only those tables that have a table name beginning with the letter C.
 - "Owner LIKE" is also based on the SQL LIKE clause. Owner LIKE allows you to select the Owner (or Creator or Alias) of the table, not the table name itself. For example Owner LIKE C% displays only those tables that have an owner beginning with the letter C.
5. Click *OK* to exit the Options dialog box.
6. Create a new report and choose ODBC as the data source for your Access database.
7. Locate and select the data source that contains the Access query you want to use.

Note

If your database requires a user name and password, or any other log on information, click Next to go to the Connection Information dialog box.

Tip

Choosing an ODBC data source and entering connection information automatically logs you onto the server.

8. Highlight your query in the Views folder, click the > arrow, and then click [OK](#).

The Design tab appears along with the Field Explorer dialog box. Your Access query, and all fields associated with that query, appear under "Database Fields."

Results

Note

You cannot use Access Action queries or Update queries in Crystal Reports. However, you can use Access Select queries and Cross-tab queries.

30.10.1.2 Opening Access Parameter queries

Access Parameter queries can be opened only when you open an Access database via ODBC. Make sure you have an ODBC data source set up for your Access database before attempting this procedure. See [Setting up an ODBC data source \[page 933\]](#).

Note

When you design a Parameter query in Access, you must provide a prompt for the query and specify a data type for the parameter. First, with your query open in Design View in Microsoft Access, enter a prompt in the Criteria cell for the field that will act as a parameter. Then, choose Parameters from the Query menu in Access, and specify a data type for the parameter you just created. Make sure the prompt appears exactly as it does in the Criteria cell. For complete instructions, refer to your Access documentation. If you do not set up your Parameter query correctly, Crystal Reports will not be able to use it.

30.10.1.2.1 To open an Access Parameter query

Procedure

1. In Crystal Reports, choose [Options](#) from the [File](#) menu.

The Options dialog box appears.

2. Click the [Database](#) tab.
3. Ensure that the [Stored Procedures](#) check box is selected.

Selecting "Stored Procedures" automatically displays any available stored procedures when you log on to an ODBC data source. Crystal Reports treats Access Parameter queries much like it treats SQL stored procedures. So, to use a Parameter query, the Stored Procedures check box must be selected.

4. In addition, you can specify [Table name LIKE](#) and [Owner LIKE](#) options if you wish.
 - "Table name LIKE" is based on the SQL LIKE clause. This option allows you to specify the kinds of table names you want to appear in the Choose SQL Table dialog box. You can use the underscore character (_) or the percent sign character (%) as wildcards with this function. The underscore character specifies any single character, while the percent sign signifies any character string. For example, DAV_ matches DAVE only, while DAV% matches DAVE and DAVID. Table name LIKE C% displays only those tables that have a table name beginning with the letter C.
 - "Owner LIKE" is also based on the SQL LIKE clause. Owner LIKE allows you to select the Owner (or Creator or Alias) of the table, not the table name itself. For example Owner LIKE C% displays only those tables that have an owner beginning with the letter C.
5. Click [OK](#) to exit the Options dialog box.
6. On the Start Page, click [Blank Report](#).
7. Locate and select the ODBC data source that contains the Access Parameter query you want to use.

Note

If your database requires a user name and password, or any other log on information, click Next to go to the Connection Information dialog box.

Tip

Choosing an ODBC data source and entering connection information automatically logs you onto the server.

8. Highlight your Parameter query in the Stored Procedures folder, click the > arrow, and then click [OK](#).
9. Create your report using the fields in the Parameter query.
10.  On the Standard toolbar, click [Refresh](#) to update the report's data.

The Enter Prompt Values dialog box appears.

11. Assign a value by typing into the field and then click [OK](#).

Your report appears. Only the records that satisfy the parameter values you specified in the Enter Parameter Values dialog box are used in your report.

Tip

You can change parameter values at any time by choosing Stored Procedure Parameters from the Database menu.

Results

Note

You cannot use Access Action queries or Update queries in Crystal Reports. However, you can use Access Select queries and Cross-tab queries.

30.10.2 Using ODBC data sources

Click a link to view one of the topics in this section:

- [Setting up an ODBC data source \[page 933\]](#)
- [Checking settings for an ODBC data source \[page 934\]](#)
- [Logging on to an ODBC data source \[page 934\]](#)
- [Adding an ODBC database table to a report \[page 935\]](#)
- [Logging off an ODBC data source \[page 935\]](#)

30.10.2.1 Setting up an ODBC data source

To set up an ODBC data source, you must first have an ODBC driver installed for the type of data you want to use. Many 64bit database clients automatically install and set up ODBC drivers. If you are not sure whether ODBC drivers have been installed for your data, refer to the documentation that came with your 64bit database client.

30.10.2.1.1 To set up an ODBC data source

Procedure

1. Open the *64bit ODBC Data Source Administrator*, which is usually found under Start > Programs > Administrative Tools > Data Sources (ODBC), or Start > Settings > Control Panel > Data Sources (ODBC).
2. Click *Add* to add a new ODBC data source.
The Create New Data Source dialog box appears.
3. Choose the ODBC driver appropriate for your data type from this list.
4. Click *Finish* when finished.

If a driver does not appear for your data type, then the ODBC driver has not been correctly installed. Refer to the documentation for your DBMS application.

An ODBC Data Source Setup dialog box that is specific to the ODBC driver you selected appears.

Note

If an error message appears instead of the Setup dialog box, you may not have the correct ODBC drivers installed on your system for the type of data you selected.

5. Type the name for your new ODBC data source in the *Data Source Name* box.

Note

The dialog box that appears may look different from the one shown here, depending on the type of data you are using. This dialog box is specific to the Access ODBC driver. For more information about using the dialog box that appears for your data, click Help.

6. Click *OK* when finished.

30.10.2.2 Checking settings for an ODBC data source

Procedure

1. Open the *ODBC Data Source Administrator (64 bit)*, which is usually found under Start > Programs > Administrative Tools > Data Sources (ODBC), or Start > Settings > Control Panel > Data Sources (ODBC).
2. Highlight the appropriate data source from the *User Data Sources* list (on the *User DSN* tab).
3. Click *Configure*.

The ODBC Data Source Setup dialog box that appears is specific to the data source you selected, and contains controls and information for setting up your data source.

4. Check the settings in this dialog box to make sure the information matches your system and database.
5. Make any changes necessary, and then click *OK*.
6. Click *OK* to close the ODBC Data Source Administrator dialog.

30.10.2.3 Logging on to an ODBC data source

Procedure

1. Choose *Log On or Off Server* from the *Database* menu.

If no report is open, choose Log On or Off Server from the File menu.

The Data Explorer dialog box appears.

2. Choose the ODBC database file you want to open and click *Log On*.

If the data source requires a user name and password, or any other log on information, a log on dialog box appears.

3. Type in the log on information you usually use to access this database, and click *OK*.

If you did not specify a database with the ODBC data source, the Select Database dialog box appears. Select the database file and then click OK.

30.10.2.4 Adding an ODBC database table to a report

Procedure

1. Choose [Database Expert](#) from the [Database](#) menu.

The Database Expert dialog box appears.

2. Expand the [Create New Connection](#) folder, and then expand the [ODBC \(RDO\)](#) folder.

The ODBC (RDO) dialog box appears.

3. Choose the data source you want to open by [Data Source Name](#) or [File DSN](#).
4. Click [Next](#) if the data source requires a user name and password, or other logon information.
5. Type in the log on information you usually use to access this database, and click [Finish](#).

If you did not specify a database with the ODBC data source, the Select Database dialog box appears. Select the database file and then click OK.

6. In the Database Expert, select and link database tables as usual.

30.10.2.5 Logging off an ODBC data source

Procedure

1. Choose [Log On or Off Server](#) from the [Database](#) menu.

If no report is open, choose Log On or Off Server from the File menu.

The Data Explorer dialog box appears.

2. Highlight the ODBC data source you want to log off from this list.
3. Click [Log Off](#).
4. Click [Close](#) when finished with the Data Explorer dialog box.

30.10.3 Using OData data sources

You can create reports on top of an OData V4 service.

Click a link to view one of the topics in this section:

- [Creating a report based on an OData service \[page 936\]](#)
- [Supported OData data types and structures \[page 937\]](#)

- [Best practices for OData data sources \[page 937\]](#)

30.10.3.1 Creating a report based on an OData service

Context

Procedure

1. In Crystal Reports, on the *File* menu, click ► *New* ► *Standard Report* ►.
2. On the Report Creation Wizard's Data screen, click *Create New Connection* node to expand.
3. Double-click *OData*.
4. Enter the OData service root URL
The service root URL = OData metadata URL without the ending '\$metadata' section. For example:
 - A sample OData metadata URL: `https://services.odata.org/V4/Northwind/Northwind.svc/$metadata`
 - A sample service root URL: `https://services.odata.org/V4/Northwind/Northwind.svc/`
5. If required, enter your credentials.
6. Click *Finish*.

Results

Once you can see the tables created from your OData data source, you can use them to create a Crystal Report, similar to how you'd use any other data source.

Note

OData services to be used in Crystal Reports must support json response (e.g. Accept application/json).

30.10.3.2 Supported OData data types and structures

The table below shows how the native OData driver maps OData data types to Crystal data types.

OData Types	Crystal Reports Types
Edm.Boolean	Boolean
Edm.String, Enumeration, Edm.GUID, Edm.Duration	String
Edm.Int16, Edm.Int32, Edm.Int64, Edm.Single, Edm.Double, Edm.Decimal	Number
Edm.DateTimeOffset	DateTime
Edm.Date	Date
Edm.TimeOfDay	Time
Edm.Binary	BLOB (Picture)
Edm.GeographyPoint - Not supported	-
Complex Type	Expanded in main table.
	<i>ⓘ</i> Note Expanded into main table, e.g. Location.Address, Location.City.Name
Collection (Primitive Type)	Table
Collection (Complex Type)	Table
Navigation	Table
	<i>ⓘ</i> Note Only one level of Navigation could be used in Crystal Reports.

30.10.3.3 Best practices for OData data sources

This sections lists a series of best practices when using OData data sources.

- For good performance, avoid using two or more irrelevant OData tables together, no matter they are from same OData service, or from different data sources (e.g. a table from OData + a table from CSV/ODBC). Following is recommended:
 - Just use one table in a report
 - It's also fine to use one table (e.g. People) together with its own collection tables (e.g. People/Emails) and navigation tables (People/Trips)
- Only one level of Navigation table could be seen and used in Crystal Reports.
- Avoid unsupported server side filters on 'Record Selection'.
 - Some OData service only support a portion of filters. When there's a failure during report refresh, Crystal Reports will spend some time automatically test each filter, then set the failed filters to run locally.

2. If your report refresh is much slower than expected, you can try to check and avoid such failure and retry time, with following steps:
 1. Enable crlogger. See [1653533](#)  for more information.
 2. Refresh your OData report and close Crystal Reports 2025 Designer.
 3. In `crw64_<pid>_<date_time>_runtime_java.log`, search for 'Failed Filter' (in English).
 4. Adjust failed filters, e.g. consider move failed filters from 'Record Selection' to 'SavedData Selection'.

30.11 Advanced database features

Click a link to view one of the topics in this section:

- [One-to-many links \[page 938\]](#)
- [Reporting on an NT Event Log \[page 939\]](#)

30.11.1 One-to-many links

Context

This tutorial demonstrates how to set up an A to B, A to C report using the Customer, Credit, and Orders tables in the Xtreme.mdb sample database. Use the instructions here as a guideline for creating A to B, A to C reports with your own database files.

Procedure

1. On the Start Page, click [Blank Report](#).
2. In the Database Expert dialog box, expand the [Create New Connection](#) folder, and then expand the [Database Files](#) folder.
3. In the [Database Files](#) folder, double-click [Find Database File](#).
4. Use the Open dialog box to locate and highlight the Xtreme.mdb database file and expand its Tables folder.
5. Highlight the Credit table and add it to the Selected Tables area.
6. Repeat the previous step for both the Customer and Orders tables.
7. When all three tables have been added, click the [links](#) tab on the Database Expert dialog box.
8. If the tables are linked by default, click [Clear Links](#).
9. Manually link the [Customer Credit ID](#) field in the Credit table to the [Customer ID](#) field in each of the other two tables.
10. Click [Order Links](#).

The Order Links dialog box appears.

11. Select a link and use the [Up](#) or [Down](#) arrow to arrange the links in the order you want.
12. Click [OK](#) to return to the Database Expert [Links](#) tab.
13. Click [OK](#) to close the Database Expert.

Results

You have now established an A to B, A to C relationship among the three tables.

30.11.2 Reporting on an NT Event Log

Context

If you are running on a Windows NT machine, Crystal Reports gives you the ability to generate reports based on the NT Event Log.

Procedure

1. On the Start Page, click [Blank Report](#).
2. In the Database Expert dialog box, expand the [Create New Connection](#) folder.
3. Double-click either [NT Archived Event Log](#) or [NT Current Event Log](#).
 - Choose NT Archived Event Log to report on a previously archived NT Event Log. If you choose this option the Select Archived NT Event Log dialog box appears, allowing you to navigate to the file you want to report on.
 - Choose NT Current Event Log to report on the most recent NT Event Log.
4. If you chose to report on the [NT Current Event Log](#), the Select Current Event Log dialog box appears:
5. The computer referenced in the [Computer\(s\)](#) box is the computer on which the sample report is based. To change the computer, expand the [Microsoft Windows Network](#) tree.

The directory structure expands to display the network group to which your computer is connected.

6. Expand the network group to display individual machines connected to the group.
7. Select the NT machine for which you would like to create an NT Event Log Report.

When you select a machine, that name appears in the Computer(s) box.

Three tables become available on the Database Expert dialog box:

- Application
- Security
- System

Use these tables to create your report based on the NT Current Event Log.

Results

Note

In order to report on the Security table, you need to have the User Right of "Manage auditing and security log" added to your Policy in NT.

Note

The data that you will see is the same data that is seen from the NT Event Viewer.

30.12 For additional information

This section has only touched on some of the more important aspects of database access, relational databases, and SQL. If you are interested in learning more about database topics, refer to the documentation provided with your DBMS application.

Note

In addition, there are hundreds of books available on the market that discuss database theory and design in depth. Look for the computer-related section at your local bookstore.

31 Integrating Business Applications with Crystal Reports

Crystal Reports is the program that you use to create and format Crystal reports, and SAP BusinessObjects Business Intelligence platform is the multi-tier system that allows you to distribute these reports over the Web in a secure and managed environment. The SAP BusinessObjects Business Intelligence Platform Integration with the following business applications provides components that enable you to combine the functionality of Crystal Reports and SAP BusinessObjects Business Intelligence Platform to work with your choice of business application:

- SAP
- JD Edwards (JDE) System
- Oracle E-Business Suite (EBS)
- PeopleSoft (PSFT)
- Siebel

Unlike in earlier releases, the integration for each application listed above has been integrated with the SAP BusinessObjects Business Intelligence Platform install. The SAP BusinessObjects Business Intelligence Platform install program provides integration for SAP, JDE, EBS, PSFT, and Siebel. You can use SAP BusinessObjects Business Intelligence platform to configure administrative tasks and use Crystal Reports to configure access to data in the applications. You can begin to work with the data from different applications in Crystal Reports and/or SAP BusinessObjects Business Intelligence Platform.

Refer to the following guides to learn how to perform administrative tasks for individual applications:

- *SAP BusinessObjects Business Intelligence Platform Administrator Guide*
- *SAP BusinessObjects Business Intelligence Platform User's Guide*
- *SAP BusinessObjects Business Intelligence Suite Master Guide*
- *SAP BusinessObjects Business Intelligence Platform Installation Guide*

31.1 Integrating with SAP

31.1.1 Introduction

Crystal Reports SAP Integration features enable you to extract additional value from your SAP Business Information Warehouse (BW) and R/3 data and share it with users across the enterprise.

The SAP integration components are:

- **Data Access**
This component provides database drivers that are designed specifically for reporting off SAP data. Crystal Reports drivers are included for OpenSQL, InfoSet, BW MDX, and ODS. Crystal Reports drivers need to be installed on the machines that host the Crystal Reports servers.

- **Crystal Reports SAP Toolbar**
Integrated within Crystal Reports, the SAP Tools toolbar facilitates tasks associated with reporting off BW queries. It allows you to log on to BW and work with its data sources, save reports to BW, and publish reports immediately to SAP BusinessObjects Business Intelligence platform via BW.

Note

SAP Business Explorer (BEx) should be installed on the same machine as SAP Crystal Reports, so that Crystal Reports SAP Toolbar can reuse BEx logon information and enable advanced workflows.

Related Information

[Reporting off InfoSets, Operational Data Stores, and MDX cubes \[page 947\]](#)

[Reporting off other SAP data sources \[page 980\]](#)

31.1.1.1 Reporting off SAP Data

This section shows how to connect to SAP from Crystal Reports, how to create reports off your SAP data with the Data Access Components, and how to access your reports through BI launch pad. The section also discusses the sample reports and the configuration options in Crystal Reports.

31.1.1.1.1 Overview of the Data Access Components drivers

The Data Access Components consist of the InfoSet Query driver, OLAP ABAPI driver, Open SQL driver, ODS driver, and the BW MDX Query driver. These drivers enable you to design reports off your SAP tables, ABAP functions, ABAP data clusters, ABAP queries, InfoSets, Operational Data Stores, and cubes. With the BW MDX Query driver, you can design formatted Crystal reports that are based on the data that is stored in SAP Business Information Warehouse (BW).

The Crystal reports that you create can be refreshed against the current SAP data by any user who has installed the drivers along with Crystal Reports. You can also schedule these reports within SAP BusinessObjects Business Intelligence platform, so your users can access customized reports with current data on a regular basis.

This section includes the following topics:

- [Connecting to SAP with Crystal Reports \[page 943\]](#)
- [Crystal Reports configuration options \[page 945\]](#)

Note

The procedures in this section are based on Crystal Reports.

31.1.1.1.2 Connecting to SAP with Crystal Reports

Once you have installed Crystal Reports, you can start Crystal Reports and select a driver for your report. There are two ways to select a driver in Crystal Reports: use one of the Report Wizards, or select [Log On Server](#) from the [File](#) or [Database](#) menu. Both methods are described below.

Note

Crystal Reports includes several integrated tools that allow you to report off your data.

31.1.1.1.2.1 Using the Report Wizards

Procedure

1. Start Crystal Reports.
2. Click the [Start Page](#) tab if the Start Page is not already visible in Crystal Reports.
3. In the New Reports area, click [Blank Report](#).

The [Database Expert](#) dialog box appears.

4. Expand [Create New Connection](#), then expand the folder that corresponds to the driver that you want to use:
 - To use the Info Set driver, expand "SAP Info sets."
 - To use the Open SQL driver, expand "SAP Table, Cluster, or Function."
 - To use the ODS driver, expand "SAP Operational Data Store."
 - To use the BW MDX Query driver, expand "SAP BW MDX Query."

Once you've selected a driver, the [SAP System Logon](#) dialog box appears. For more details, see [Logging on to the SAP server \[page 944\]](#).

31.1.1.1.2.2 Using the Log On Server command

Procedure

1. Start Crystal Reports.
2. On the [File](#) menu (or [Database](#) menu, if a report is already open), click [Log On or Off Server](#).
3. In the Data Explorer, expand [Create New Connection](#). Then, expand the folder that corresponds to the driver that you want to use:
 - To use the Info Set driver, expand "SAP Info sets."
 - To use the Open SQL driver, expand "SAP Table, Cluster, or Function."
 - To use the ODS driver, expand "SAP Operational Data Store."
 - To use the BW MDX Query driver, expand "SAP BW MDX Query."

Once you've selected a driver, the *SAP System Logon* dialog box appears. For more details, see [Logging on to the SAP server \[page 944\]](#).

31.1.1.1.2.3 Logging on to the SAP server

Regardless of which SAP driver you select, you must specify the SAP system that you want to report off and provide valid R/3 or BW user credentials for that system. Each driver prompts you for this logon information before it allows you to select the elements that you want to include in the report. Your user credentials are also requested when you run any existing reports that are built off SAP data sources.

31.1.1.1.2.3.1 To log on to the SAP server

Procedure

1. Once you have selected a driver, the SAP System Logon dialog box appears.
2. Select the SAP system that you want to report off and click *Next*.

Note

If your SAP administrator has not configured SAP Logon, your SAP system will not appear in the Available SAP systems list.

The next *SAP System Logon* dialog box prompts you for user logon credentials.

3. In the *Client* field, type your usual three-digit SAP client number.
4. In the *Username* and *Password* fields, type your usual SAP logon credentials, then click *Next*.

Note

When selecting Secure Network Communication (SNC) enabled entries, the password field will be disabled.

The next *SAP System Logon* dialog box prompts you for extended logon parameters.

5. If you want the SAP Communication Library to create trace files for all client/server communication (for troubleshooting purposes), select the *Generate RFC trace files* check box.
6. Click *Finish*.

Crystal Reports logs you onto the SAP system and displays the connection in the Data Explorer (or in the Available Data Sources list of a Report Wizard).

Results

Note

It is strongly recommended that you filter the tables that are displayed under the new connection by right-clicking the connection, selecting Options, and modifying the appropriate settings in the *Options*

dialog box. You can choose not to specify filter options for the tables that are returned to the Data Explorer; however, if the SAP system contains numerous tables, a significant wait may occur while Crystal Reports retrieves the list of data sources from your SAP system.

31.1.1.1.3 Crystal Reports configuration options

When you install Crystal Reports, you may need to change your global reporting options to improve the experience of reporting off SAP. The steps in this section show how to configure Crystal Reports to list tables and fields by both name and description. The section also shows how to filter the tables that are displayed in the Data Explorer.

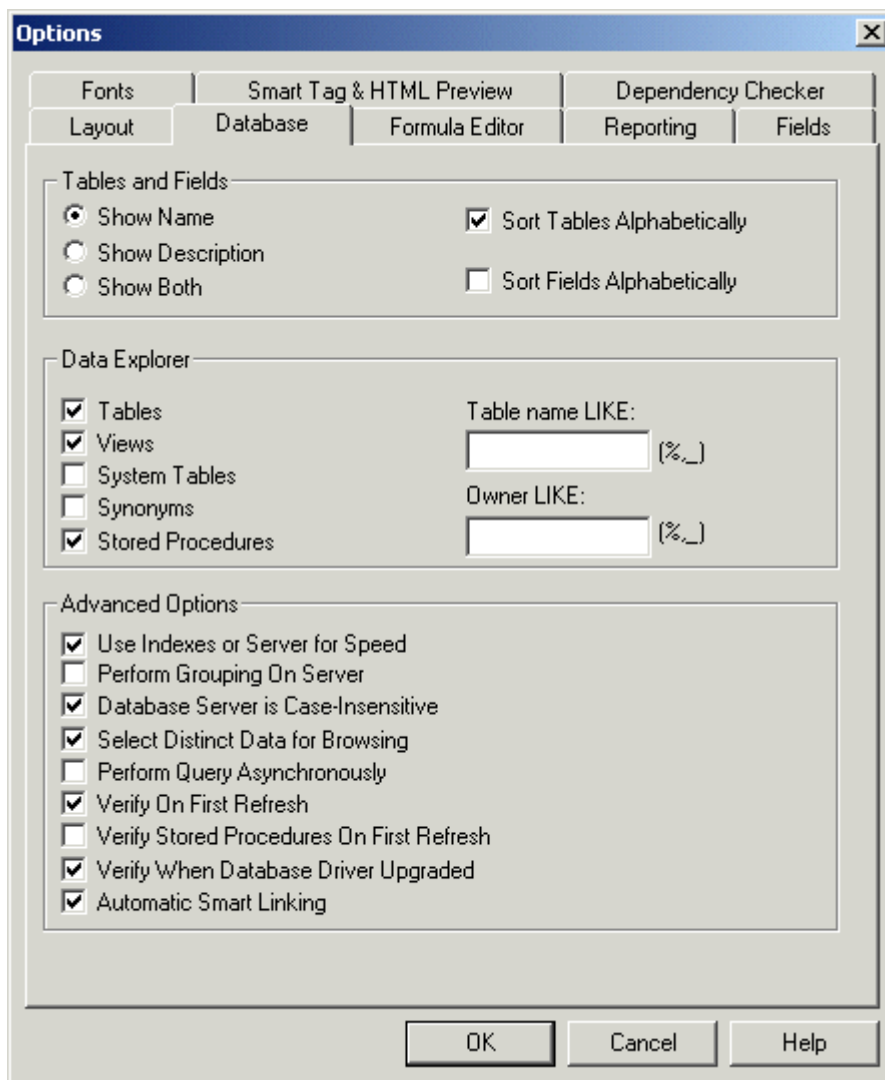
Your SAP system may contain thousands of tables, each with a predefined technical name. The options that are described in this section ensure that you can select specific tables for a report without having to search through a lengthy list.

31.1.1.1.3.1 To change configuration options

Procedure

1. On the *File* menu, click *Options*.

The *Options* dialog box appears.



2. Click the [Database](#) tab.
3. In the Data Explorer area, select the check boxes for the types of data that you want to make available for the report.
4. Use the [Table name LIKE](#) and/or the [Owner LIKE](#) fields to select only a subset of the available data types. Use any of the following techniques when filtering:
 - Type full or partial names of tables.
 - Add wildcards to select multiple queries or InfoSets: use a percentage sign (%) to denote any number of characters; use an underscore (_) to denote a single character. (The % and _ wildcards correspond respectively to the * and ? wildcards that are used within Windows.)
 - Clear both fields and click OK to proceed without filtering. Note, however, that there is a limit to the number of objects that can be displayed in the Data Explorer. If the number of queries or InfoSets in SAP exceeds this limit (which is by default 8000), you may receive an Out of Memory error or a blank Data Explorer.
5. In the Tables and Fields area, select [Show Both](#).
6. Click [OK](#).

31.1.2 Reporting with SAP

This section describes the key features provided by Crystal Reports to report off SAP systems.

31.1.2.1 Reporting off InfoSets, Operational Data Stores, and MDX cubes

Crystal Reports provides connectivity to your SAP data through three database drivers— the InfoSet driver, the ODS driver, and the MDX driver:

Note

The BW Query driver is not supported in the current and future releases. The MDX driver provides the same function set as the BW Query driver with enhancements. You can migrate the existing BW Query reports to MDX driver.

- The InfoSet driver provides Crystal Reports with another means of accessing SAP data sources: this driver can access R/3 InfoSets (previously known as Functional Areas) and ABAP Queries. You access this driver through the Data Explorer in Crystal Reports .
- The Operational Data Store (ODS) driver allows you to use existing ODS objects from your BW Data Warehouse as a data source in Crystal Reports.
- The SAP BW MDX Query driver helps you report off BW cubes and queries. By writing reports with the MDX Query driver, you gain direct access to BW cubes, display attributes, and multiple structures. You will be able to use SAP BW system as a data source through the MDX driver on Crystal Reports **C++** stack. By providing required SAP system information and credential, you will be able to create connection to the SAP system, achieve the common features of SAP System as a data source through MDX driver such as create report, view and schedule report etc in the SAP system.

One advantage of these drivers is that they allow business users to report off predefined views of your SAP data. Tables and fields are first logically grouped by an SAP content expert into queries, InfoSets, and so on. You then make these data sources accessible to users who are designing reports with Crystal Reports.

These drivers are also included in the add-ons for SAP BusinessObjects Business Intelligence platform (the Data Access feature), so the SAP BusinessObjects Business Intelligence platform processing servers can successfully run the your reports against your SAP systems. These drivers allow you to work with Crystal Reports.

Note

Before you can design or process reports with the InfoSet or OpenSQL drivers, you must install the corresponding transport file on the SAP system.

31.1.2.1.1 Reporting off BW Queries and Cubes: MDX Query Driver

This section introduces you to Crystal Reports and the integrated features of the MDX Query Driver. It shows how to format and build reports off a BW query and hierarchy.

31.1.2.1.1.1 MDX Query driver overview

The SAP BW MDX Query driver helps you report off BW cubes and queries. By writing reports with the MDX Query driver, you gain direct access to BW cubes. You can also now use display attributes, Multiple Structures, and Free Characteristics in your Crystal reports.

Additionally, the MDX Query driver allows you to create reports from queries that contain hierarchy variables and hierarchy node variables. The driver creates specific fields that allow you to specify pick lists for your variables in Crystal Reports.

31.1.2.1.1.2 Formatted reporting overview

With the BW MDX Query driver, you can design formatted Crystal reports that are based on the data that is stored in SAP Business Information Warehouse (BW). You base these reports either on the queries that you create with SAP's Business Explorer (BEx) Query Designer or directly on cubes. Once you have designed your Crystal reports, you can save them to SAP BusinessObjects Business Intelligence platform and, if required, target them for translation in order to make them available to users in multiple languages.

This section focuses on Crystal Reports and the task of using the MDX Query driver to select BW queries and cubes as data sources for Crystal reports. This section ends with a series of tutorials that guide you through the creation of reports off a sample query.

Note

It is recommended that you complete the tutorials in sequential order because each tutorial builds on the query and report you built in the previous tutorial.

SAP BusinessObjects Business Intelligence platform allows you to share Crystal reports over the Web so that all users can view your formatted content. For details about publishing reports to SAP BusinessObjects Business Intelligence platform, see [Publishing Reports \[page 1011\]](#).

Your BW queries may contain SAP variables that appear as parameters in SAP BusinessObjects Business Intelligence platform and Crystal Reports. To accommodate these variables in the BI platform, BusinessObjects Integration for SAP Solutions employs *dynamic pick lists*. A dynamic pick list provides you with a list of possible values to choose for a parameter (variable). Additionally, the values that are displayed in a dynamic pick list correspond to your user rights—you see only the pick list values that you have the rights to as defined by SAP BW security settings.

Note

In Crystal Reports, the pick lists are static but in SAP BusinessObjects Business Intelligence platform they are dynamic.

31.1.2.1.1.2.1 Accessing BW queries for reporting

This section outlines the ways in which you can create BW queries and select existing BW queries to use as data sources for your Crystal reports. To access the BEx Query Designer to create new queries you can use the Programs menu to navigate to it. To select existing queries you can use the Database Explorer in Crystal Reports.

31.1.2.1.1.2.2 Creating new queries for Crystal Reports

You can use [Programs](#) menu to open the [Query Designer](#).

31.1.2.1.1.2.2.1 To create a BW query directly from the BEx Query Designer

Procedure

1. Go to [Start > Programs > Business Explorer > Query Designer](#).
2. When prompted, log on to your SAP BW system.
3. On the Query Designer's toolbar, click [New Query](#).

The [New Query: Select InfoProvider](#) dialog box displays the InfoAreas and InfoProviders that are available on your system.

4. Select the InfoProvider on which you want to base your query, then click [OK](#).
5. Define your query by selecting measures and characteristics from the Key Figure and Dimensions lists and dragging them to the Columns area.

Note

When you create a new query for reporting purposes, it is recommended that you place Key Figures in the Columns area of the Query Designer and Characteristics in the Rows area.

6. Click [Query Properties](#).
7. Click the [Extended](#) tab and ensure that [Allow External Access to this Query](#) is selected.

This option allows other programs, like Crystal Reports, to access this query.

8. Click [Save Query](#).

9. In the [Save Query](#) dialog box, click [Roles](#) and select the role where you want to save the query.
10. Type a [Description](#) and a [Technical Name](#) for the query, then click [Save](#).

The Technical Name must uniquely identify the query—that is, the name can appear only once across all InfoProviders in BW. The Technical Name can be up to 30 characters long and must begin with a letter.

31.1.2.1.1.2.3 Selecting BW Queries for a report

There are two ways you can access BW queries to use as Crystal Reports data sources. You can use the SAP Tools toolbar in [Crystal Reports](#) or you can use the Database Explorer in Crystal Reports.

31.1.2.1.1.2.3.1 Using the SAP Tools toolbar to select an MDX query

Context

❗ Note

The tutorials in this section use the SAP Tools toolbar to create new queries for reporting. However, when creating new queries you can use whichever method you prefer.

❗ Note

Before you can see a query in the Database Explorer, you will need to set the “Allow External access to the Query” option in its properties.

Procedure

1. On the [SAP](#) menu click [Create New Report from a Query](#).
2. When prompted, log on to your SAP BW system.

The [Select Query for your Crystal Report](#) dialog box displays the queries that are available through your Favorites, Roles, and InfoAreas in BW.

3. Select the query with the data that you want to report off, then click [OK](#).

Crystal Reports generates a report that uses your query as its data source. The report opens in the [Design](#) tab of [Crystal Reports](#). You can now add fields, titles, charts, and other objects to the report.

31.1.2.1.1.2.3.2 Using the Database Expert to select an MDX query

Procedure

1. On the *File* menu in Crystal Reports, click *New* and then click *Blank Report*.

The Database Expert appears.

2. Under Available Data Sources, expand *Create New Connection* and then expand *SAP BW MDX Query*.

The *SAP System Logon* dialog box appears.

3. Select the appropriate BW system and click *Next*.

The next *SAP logon* dialog box prompts you for user credentials

4. Type your usual SAP user credentials in the *Client*, *Username*, and *Password* fields, then click *Next*.

→ Tip

To set the logon language, you must specify your Preferred Viewing Local in Crystal Reports under

► *View* ► *Preferred Viewing Local* ►.

The next *SAP logon* dialog box asks you whether you would like to generate RFC trace files.

5. If you want the SAP Communication Library to create trace files for all client/server communication (for troubleshooting purposes), select the *Generate RFC trace files* check box.
6. Click *Finish*.

You are returned to the *Database Expert*. There are now two entries under your SAP BW MDX Query connection. One is labeled CU (Cubes) and the other is labeled (QC) Queries.

7. Expand *QC (Queries)* to reveal the defined MDX BW Queries that are available as report data sources.

→ Tip

If no items are found, check the Options setting of the Data Explorer.

- a. Right-click *QC (Queries)* and click *Options*.

This opens the *Options* dialog box with only the *Database* tab visible.

- b. In the Data Explorer area, ensure that *Stored Procedures* is selected and then click *OK*.

You are returned to the Database Expert.

- c. Select *SAP BW MDX Query* and press the F5 key to refresh the Database Expert.
- d. Expand *Current Connections* and then expand *QC (Queries)*.

Now there are lists of available queries.

8. Select the query that you want to report off and click the right-arrow button.
9. Click *OK*.

Crystal Reports generates a blank report that uses your query as its data source. You can now add objects to the report.

31.1.2.1.1.2.4 Showing field descriptions and technical field names

In Crystal Reports you can choose to identify fields by their description, name, or both. In some cases you may prefer to show both and in other cases you may prefer to use just one. For example, when working with Multiple Structures, it is recommended that you show only the descriptions. The Global Unique Identifiers (GUID) that SAP assigns to Multiple Structures are reflected in Crystal Reports and can be difficult to work with.

31.1.2.1.1.2.4.1 To adjust the field name and description settings

Procedure

1. On the *File* menu, click *Options*.
The *Options* dialog box appears, with the Layout tab active.
2. Click the *Database* tab.
3. In the “Tables and Fields” area, select *Show Name*, *Show Description*, or *Show Both*.
4. Click *OK*.

Results

→ Tip

For more information about formatted reporting, see the SAP Library > Business Information Warehouse > Business Explorer > Formatted Reporting: Crystal Reports Integration.

31.1.2.1.1.2.5 Tips regarding field names in Crystal Reports

When you are viewing attribute names in Crystal Reports, the number that the attribute name is preceded by gives you information about what kind of information the field contains.

- 2 means that the attribute is the member key
- 1, 4, or 5 means that the attribute is a member description

When you are viewing characteristic names in Crystal Reports, if the characteristic name is followed by many spaces and then the name of a hierarchy, the characteristic contains that hierarchy.

For example, if the Material characteristic contains the HIGHTECH hierarchy, it appears as [0Material HIGHTECH].

The following fields are used for hierarchical grouping and parameter binding when there is a hierarchy node variable on a characteristic. These fields should not be used for reporting.

- NodeId
- Parent NodeId
- MemberUniqueName

Note

Putting the MemberUniqueName on a report will cause performance issues.

When a query contains a hierarchy variable, the MDX Query driver creates an extra field called Hierarchies in Crystal Reports that allows you to create a list of values for the variable.

The Hierarchies field has the following attributes:

- Hierarchy names
- Hierarchy descriptions

This field has no purpose other than to create lists of values for hierarchy variables. This field should not be used for reporting.

31.1.2.1.1.2.6 Selecting cubes for reports

If you or someone else in your organization has defined one or more cubes for reporting purposes within BW, you can select them as data sources for new Crystal Reports.

31.1.2.1.1.2.6.1 To select an existing query for a report

Procedure

1. On the *File* menu in Crystal Reports, click *New* and then click *Blank Report*.
The *Database Expert* appears.
2. Under “Available Data Sources”, expand *Create New Connection* and then expand *SAP BW MDX Query*.
The *SAP System Logon* dialog box appears.
3. Select the appropriate BW system and click *Next*.
The next *SAP logon* dialog box prompts you for user credentials
4. Type your usual SAP user credentials in the *Client*, *Username*, and *Password* fields, then click *Next*.

→ Tip

To set the logon language, you must specify your Preferred Viewing Local in Crystal Reports under  *View*  *Preferred Viewing Local* .

The next *SAP logon* dialog box asks you whether you would like to generate RFC trace files.

5. If you want the SAP Communication Library to create trace files for all client/server communication (for troubleshooting purposes), select the [Generate RFC trace files](#) check box.
6. Click [Finish](#).

You are returned to the [Database Expert](#). There are now two entries under your SAP BW MDX Query connection. One is labeled CU (Cubes) and the other is labeled (QC) Queries.

7. Expand [CU \(Cubes\)](#) to see the defined BW cubes that are available.

→ Tip

If no items are found, check the Options setting of the Data Explorer.

- a. Right-click [CU \(Cubes\)](#) and click [Options](#).
 - b. This opens the [Options](#) dialog box with only the Database tab visible.
 - c. In the "Data Explorer" area, ensure that [Stored Procedures](#) is selected and then click [OK](#).
 - d. You are returned to the [Database Expert](#).
 - e. Select [SAP BW MDX Query](#) and press the F5 key to refresh the Database Expert.
 - f. Expand [Current Connections](#) and then expand [CU \(Cubes\)](#).
 - g. Now there are lists of available cubes.
8. Select the cube that you want to report off and click the right-arrow button.
 9. Click [OK](#).

Crystal Reports generates a blank report that uses your cube as its data source. You can now add objects to the report.

31.1.2.1.1.2.7 Multiple Structures and Crystal Reports

The MDX Query driver recognizes Multiple Structures and allows you to use them in your Crystal reports. Multiple Structures show in Crystal Reports as a single dimension. That is, they show with no attributes. The structure consists only of the members that are in the query.

31.1.2.1.1.2.8 Saving reports to BW

After designing a report based on a query, you can save the report to a role in BW. At the same time, you can prepare the report for translation or automatically publish it to SAP BusinessObjects Business Intelligence platform.

Before saving your report, check its title by clicking Summary Info on the File menu. The title that appears here is displayed to users in SAP BusinessObjects Business Intelligence platform. The technical name of the query is used as the default title, but you can replace it with a descriptive title that is more meaningful to SAP users.

→ Tip

Use the Save As command on the File menu to save the report to disk.

31.1.2.1.1.2.8.1 To save a report to BW

Procedure

1. On the [SAP](#) menu, click [Save Report](#).

Note

You may be prompted to logon to the SAP system again to choose a role to publish the report to. This is necessary after accessing the SAP toolbar the first time after a report is created.

2. In the [Save a Crystal Report to BW](#) dialog box, select the role where you want to save the new Crystal report.
3. Type a [Description](#) for the report, then click [Save](#).

Depending on your connection settings, the [Save to BW options](#) dialog box may appear.

4. Select from the available options:
 - [Prepare this report for translation](#)
When you select this option, the strings in your report are extracted and made available for translation within SAP. For details, see [Designing reports for translation \[page 1009\]](#).
 - [Automatically publish to SAP BusinessObjects Business Intelligence platform](#)
For more information on publishing reports, see [Publishing Reports \[page 1011\]](#).
If you are unable to publish your report to SAP BusinessObjects Business Intelligence platform, you may need to consult with your administrator about licensing.
 - [Don't show this dialog again](#)
If you select this option now, you can modify your default settings later by clicking [Settings](#) on the [SAP](#) menu.
5. Click [OK](#).

31.1.2.1.1.3 Building a report off a BW query

This tutorial provides an introduction to reporting off a query using the BW MDX Query driver. You will perform the following tasks in this tutorial:

- Create a query
- Select the query as a data source for a Crystal report
- Insert database fields
- Group and sort data
- Summarize data
- Preview the report

31.1.2.1.1.3.1 How to create a simple BW query

This section guides you through the process of creating a query, saving it to BW, and releasing it for use by Crystal Reports.

31.1.2.1.1.3.1.1 To create a simple BW query

Procedure

1. Go to ► [Start](#) ► [Programs](#) ► [Business Explorer](#) ► [Query Designer](#) ►.
2. When prompted, log on to your SAP BW system.

The [Query Designer](#) opens.

3. On the Query Designer's toolbar, click [New Query](#).

The [New Query: Select InfoProvider](#) dialog box displays the InfoAreas and InfoProviders that are available on your system.

4. Select the [Customer-Cube](#) and click [OK](#).

Note

The location of the Customer-Cube varies, so you may need to browse to find it.

5. Select the following characteristics from the [Dimensions](#) list, and drag them to the “Rows” area:
 - Division
 - Material

→ Tip

You can find Division under the Sales area data dimension.

6. Select the following measures from the Key Figure list, and drag them to the Columns area:
 - Invoiced quantity
 - Sales Volume
7. Click [Query Properties](#).
8. Click the [Extended](#) tab and ensure that [Allow External Access to this Query](#) is selected.

This option allows other programs, like Crystal Reports, to access this query.

31.1.2.1.1.3.1.2 To save the query in BW

Procedure

1. Click [Save Query](#).
2. In the [Save Query](#) dialog box, click [Roles](#) and select the role where you want to save the query.
3. Type a [Description](#), such as Sample Material query
4. Type a [Technical Name](#), such as MATERIALSAMPLE01.

The Technical Name must uniquely identify the query—that is, the name can appear only once across all InfoProviders in BW. The Technical Name can be up to 30 characters long and must begin with a letter.

5. Click [Save](#).

31.1.2.1.1.3.1.3 To use the query in Crystal Reports

Procedure

Click [Quit and Use Query](#).

Crystal Reports generates a report that uses your query as its data source. The report opens in the Design tab of Crystal Reports. You can now add fields, titles, charts, and other objects to the report.

31.1.2.1.1.3.2 How to create a new report based on the query

This section guides you through the process of creating a new report based on the query you created in the last section, [How to create a simple BW query \[page 956\]](#).

31.1.2.1.1.3.2.1 To create a new report

Procedure

On the [View](#) menu, click [Field Explorer](#).

The [Field Explorer](#) appears. Expand Database Fields and Sample Material query to see the list of fields available for your report.

→ Tip

To change how the fields are identified in Crystal Reports, see [Showing field descriptions and technical field names \[page 952\]](#).

31.1.2.1.1.3.2.2 To select fields for the report

Procedure

1. Expand [Key Figures](#).
2. Drag Invoiced Quantity from the [Field Explorer](#) and place it in the Details section of the report.
3. Click the [Refresh](#) button to view the result set.

Only one value is returned on the report. This value represents the aggregated value of this Key Figure across all Characteristics in the cube. To display more detailed results, you must group the data on one or more dimensions.

31.1.2.1.1.3.2.3 To group and sort data

Context

Note

If you group on an attribute value, report performance is degraded. This occurs because the MDX query is slow to return data. To retain processing speed, it is recommended that you group on the Member Caption field whenever possible, as outlined in this tutorial.

Procedure

1. On the [Insert](#) menu click [Group](#).
The Insert Group dialog box appears.
2. On the first list click [Division](#).
This field is the Member Caption for Division.
3. On the second list click [in ascending order](#).
4. Click [OK](#).
5. On the [Insert](#) menu click [Group](#).
6. On the first list click [Material](#).
This field is the Member Caption for Material.
7. On the second list click [in ascending order](#).
8. Click the [Refresh](#) button to view the result set.

When you view this report, the records are first grouped according to the value in the Division characteristic. The resultant groups are then displayed in the report in ascending alphabetical order. The records in each Division group are further divided into secondary groups based on the values in the Material dimension. The resultant groups are also sorted in ascending alphabetical order.

Results

→ Tip

If you want to see the MDX query statement that the MDX Query driver is passing to the server, you can use the Show SQL Query option in CR. To see the query, on the [Database](#) menu, click [Show SQL Query](#).

This MDX statement can be tested directly against SAP BW using the transaction mdxtest.

31.1.2.1.1.3.3 How to summarize data and save report

This section guides you through the process of summarizing data and saving report based on the report you created in the last section, [How to create a new report based on the query \[page 957\]](#).

31.1.2.1.1.3.3.1 To summarize the data

Procedure

1. On the [Insert](#) menu click [Summary](#).
2. The [Insert Summary](#) dialog box appears.
3. In the [Choose the field to summarize](#) list, select [Invoiced Quantity](#).
4. In the [Calculate this summary](#) list, select [Sum](#).
5. In the [Summary Location](#) list, select [Group #1:\[Field Name\]](#).

In this case, [Field Name] indicates the name of the field that the group is based on.

6. Click the [Refresh](#) button to view the result set.

When you view the report, there is one summary in Group Footer 1. This summary represents a sum of the Invoiced Quantity for each Division.

31.1.2.1.1.3.3.2 To save your report

Procedure

On the [SAP](#) menu, click [Save Report](#), and save the report to a role in BW.

For details, see [Saving reports to BW \[page 954\]](#).

Results

You may want to resize or reposition report elements, or further modify the report's design. You can, for instance, select and drag the borders of columns and column headings in order to resize them in the Preview tab; or, you can quickly display only a subset of the data by filtering the records with the Select Expert.

The *Crystal Reports Online Help* provides extensive conceptual and procedural information, tutorials, and examples to help you use Crystal Reports to its fullest potential. Access the help by pressing [F1](#) anywhere within Crystal Reports.

31.1.2.1.1.4 Building a report off a BW hierarchy

This tutorial demonstrates how to report off a BW query with a hierarchy using the MDX Query driver in Crystal Reports.

In this tutorial you will:

- Create a simple BW query with a hierarchy.
- Create a new report and group the data hierarchically.
- Insert a summary.
- Adjust object positioning dynamically.

Note

This tutorial builds on the topics covered in [Building a report off a BW query \[page 955\]](#), which provides more details on steps such as logging on to your SAP system, selecting the Customer-Cube and saving queries.

31.1.2.1.1.4.1 How to create a simple BW query with a hierarchy

This section uses the query that you created in the last tutorial, [Building a report off a BW query \[page 955\]](#). You will be adding a hierarchy to the query on the Material dimension.

31.1.2.1.1.4.1.1 To create a BW query with a hierarchy

Procedure

1. Go to  [Start](#)  [Programs](#)  [Business Explorer](#)  [Query Designer](#) .
2. Click [Open Query](#).
3. Select the query you created in the last tutorial.

If you have not created a query, see [Building a report off a BW query \[page 955\]](#).

4. In the Rows area, right-click *Material* and click *Properties*.

The *Characteristic Properties* dialog box appears.

5. Next to the Hierarchy Name field, click the *Values* button.

The *Select Hierarchy* dialog box appears.

6. In the *Available Hierarchies* list, select *Material class* and click *OK*.

You are returned to the *Characteristic Properties* dialog box.

7. Click *OK* and then click *Save Query As...* to save your query to the appropriate location.

For this example, in the Description field, type Simple Hierarchy for MDX, and in the Technical Name field, type HIERARCHY_SIMPLE_MDX.

8. Click *Quit and Use Query*.

Results

You now have a simple query with three possible levels of hierarchy. You can display the query in either the SAP Business Explorer Analyzer or as a dataset in Crystal Reports.

31.1.2.1.1.4.2 How to build a report off the BW hierarchy

The report that you build in this section starts using the same field as the report you created in [Building a report off a BW query \[page 955\]](#). The difference comes with the grouping. Instead of grouping the data like you did in the previous tutorial, you will group the data hierarchically. This way, the hierarchy that exists in the data is reflected in the report.

This tutorial will use the following fields to create a hierarchical display:

- **<Hierarchy Name >** Node ID
This field reflects the child values.
- **<Hierarchy Name>** Parent ID
This field reflects the parent values. Combined with the Actual Node ID, Crystal Reports uses these fields to determine the hierarchy based on the information received from the SAP BW query.
- **<Hierarchy Name>**
This field contains the captions of the hierarchy members.

31.1.2.1.1.4.2.1 To build a hierarchy report off the BW hierarchy

Procedure

On the [View](#) menu, click [Field Explorer](#).

The [Field Explorer](#) appears. Expand Database Fields and Simple Hierarchy for MDX to see the list of fields available for your report.

→ Tip

You may want to set the field name and description display options to make it easier to select fields. To change how the fields are identified in Crystal Reports, see [Showing field descriptions and technical field names](#) [page 952]

31.1.2.1.1.4.2.2 To select fields for the report

Procedure

1. Expand [Key Figures](#).
2. Drag Invoiced Quantity from the Field Explorer and place it in the Details section of the report.

31.1.2.1.1.4.2.3 To group and sort data

Procedure

1. On the [Insert](#) menu click [Group](#).
The [Insert Group](#) dialog box appears.
2. On the first list click [Material Node ID](#).
3. On the second list click [in ascending order](#).
4. Click the [Options](#) tab and select the [Customize Group Name](#) field.
5. From the [Choose from Existing](#) field list, select [Material](#).

Material is the hierarchy name, or Member Caption.

6. Click [OK](#).
7. On the [Report](#) menu click [Hierarchical Grouping Options](#).

The [Hierarchical Group Options](#) dialog box appears.

8. In the Available Groups area, ensure that Material Node ID is selected.

Material Node ID is probably the only group listed.

9. Select the *Sort Data Hierarchically* check box.
10. In the Parent ID field, select *Material Parent Node ID*.
11. In the Group Indent: field, type the amount that you would like each hierarchy level to be indented.

This entry depends on how you want your report to look and on what unit of measurement you are using in CR.
12. Click the *Refresh* button to view the result set.

The result set for the report is now displayed hierarchically.

31.1.2.1.1.4.3 How to summarize data

This section guides you through the process of summarizing data based on the query you created in the last section, [How to build a report off the BW hierarchy \[page 961\]](#).

31.1.2.1.1.4.3.1 To summarize the data

Procedure

1. On the *Insert* menu click *Summary*.
2. The *Insert Summary* dialog box appears.
3. In the *Choose the field to summarize* list, select *Invoiced Quantity*.
4. In the *Calculate this summary* list, select *Sum*.
5. In the *Summary Location* list, select *Group #1:[Field Name]*.

In this case, [Field Name] indicates the name of the field that the group is based on.

Note

You do not need to select Summarize across hierarchy in order to have the data summarized correctly in this example because Crystal Reports can recognize the Hierarchy node summaries that the SAP BW data delivers.

Results

The report now displays the summary data across the hierarchy. However, the summary values are indented. To fix this see, [How to indent a hierarchy without affecting other fields \[page 964\]](#).

31.1.2.1.1.4.3.2 An important note about group summaries

If you mix hierarchical grouping with regular grouping and insert a summary at the regular group level, the summary may be incorrect. That is, the existing summaries are summarized again along with the detail records.

In this case, you must use server-side calculated summaries at the regular group level.

31.1.2.1.1.4.4 How to indent a hierarchy without affecting other fields

This procedure demonstrates how to indent a hierarchy while keeping all other fields on a report in their original positions.

31.1.2.1.1.4.4.1 To indent a hierarchy without affecting other fields

Procedure

1. On the *Report* menu, click *Hierarchical Grouping Options*.
2. Ensure that the *Group Indent* field is set to zero.
3. Click *OK*.

The report now displays the data without any indent.

4. Right-click the group header field and click *Size and Position*.
5. Click the *Conditional Formula* button beside the X position value field.
6. In the Formula Workshop, enter the formula:

```
HierarchyLevel (GroupingLevel({<[Hierarchy Name]> Node ID})) * 250
```

Note

The field {[Hierarchy Name] Node ID} appears with its technical name in the formula.

Note

Positions are measured in twips; there are 1440 twips in an inch.

Note

The report now displays the summary data in its original position and the group headers hierarchically.

31.1.2.1.1.5 Building a report off a BW query with a hierarchy node variable

When you design your reports, you might want to give users the opportunity to restrict the amount of data that is returned by Crystal Reports. To do this, you can create a hierarchy node variable in your query. This variable will be treated as a parameter in Crystal Reports.

You will perform the following tasks in this tutorial:

- Create a query with a hierarchy node variable
- Create a new report based on this query
- Bind the hierarchy node variable to the MemberUniqueName field in Crystal Reports.
- Preview the report

Note

This tutorial builds on the query that you built in the previous tutorial. If you have not built a query yet, follow the steps outlined in [How to create a simple BW query \[page 956\]](#) and [How to create a simple BW query with a hierarchy \[page 960\]](#).

31.1.2.1.1.5.1 How to create a query with a hierarchy node variable

This section guides you through the process of creating a hierarchy node variable on the Material hierarchy. When you report off this query using Crystal Reports, you will receive a prompt to select a hierarchy node to use for the report data.

31.1.2.1.1.5.1.1 To create a hierarchy node variable

Procedure

1. Go to ► **Start** ► **Programs** ► **Business Explorer** ► **Query Designer** ►.
2. Click **Open Query**.
3. Select the query you created in the last tutorial.

Note

If you have not created a query, complete the steps outlined in [How to create a simple BW query \[page 956\]](#) and [How to create a simple BW query with a hierarchy \[page 960\]](#).

4. In the Rows area, right-click **Material** and click **Restrict**.
The Selection for [Dimension] dialog box appears.
5. Ensure that the **Selection** field is set to Single Values, and that the **Hierarchy** field is set to the hierarchy you created for this dimension.

In this example, the hierarchy is Material class.

6. Click the *Variable* tab.
7. Right-click in the white space and click *New Variable*.

The SAP BW Variables Wizard opens.

8. Click *Next* on the Introduction window.
9. Ensure that the *Type of Variable* field says Hierarchy Node.
10. Fill in the rest of the fields as follows:
 - In the *Variable Name* field, type a name such as HNV01.
 - In the *Variable Description* field, type a name such as Test Hierarchy Node Variable.
 - In the *Processing by* field, ensure it says User Entry/Default Value.
 - In the *Characteristic* field, ensure it says Material.
11. Click *Next*.

The Details window appears.

12. On the *Variable Entry is* list click *Optional* or *Mandatory*.
13. Click *Next* to accept the other default values.

The *Default Values* window appears.

14. Click *Next* to accept the default values.
15. Click *Finish* to create the variable.

You are returned to the *Selection for Dimension* dialog box. Your new hierarchy node variable is visible in the variable list.

31.1.2.1.1.5.1.2 To add the hierarchy node variable to the query

Procedure

1. Click the hierarchy node variable named Test Hierarchy Node Variable.
2. Click the right arrow to move the variable to the Selection area of the dialog box.
3. Click *OK*.

You are returned to the BEx Query Designer. Your new hierarchy node variable is visible in the Rows area under the Material characteristic. You are now ready to save the query.

4. Click *Save Query As...* to save the query with a new name and description.

For this example, in the Description field, type Simple Hierarchy w Node Variable, and in the Technical Name field, type HIERARCHY_SIMPLE_HNV.

5. Click *Quit and Use Query*.

Results

You now have a query with a simple hierarchy on Material class and a hierarchy node variable based on the Material class hierarchy.

31.1.2.1.1.5.2 How to build a report off the query with a hierarchy node variable

This section guides you through the process of creating a list of default values for the parameter that appears in *Crystal Reports*.

1. Create the same report that you created in [How to build a report off the BW hierarchy \[page 961\]](#) except you are now using the query called `Simple Hierarchy w Node Variable` as the data source.
2. Click the [Refresh](#) button when you have finished designing the report.

Note

If your variable is optional, ensure that the [Set to Null](#) check box is clear when you select a parameter value. The [Set to Null](#) check box overrides any value you select from the list of default parameter values. This check box is not always cleared when you select a parameter value.

31.1.2.1.1.6 Building a report off a BW query with a hierarchy variable and a hierarchy node variable

This tutorial guides you through adding a hierarchy variable to the query you created in the last tutorial. The hierarchy variable allows users to choose what hierarchy they would like Crystal Reports to use when displaying the data. The hierarchy node variable allows users to then choose what node of the hierarchy they would like to see data from.

You will perform the following tasks in this tutorial:

- Add a hierarchy variable to your existing query
- Create a new report based on this query
- Bind the hierarchy node variable to the MemberUniqueName field.
- Bind the hierarchy variable to the Hierarchies field.
- Preview the report

Note

This tutorial builds on the query that you built in the previous tutorial. If you have not built a query yet, follow the steps outlined in [How to create a simple BW query \[page 956\]](#), [How to create a simple BW query with a hierarchy \[page 960\]](#), and [How to create a query with a hierarchy node variable \[page 965\]](#).

31.1.2.1.1.6.1 How to create a hierarchy variable

This section guides you through the process of creating a hierarchy variable on your existing query and resetting the hierarchy node variable to correspond to this new hierarchy variable.

In the last tutorial, you set the hierarchy node variable to the Material class hierarchy because that is the hierarchy that existed in the query. In this tutorial you will change the hierarchy of the query from the Material class hierarchy to a variable. To avoid unexpected behavior, the hierarchy node variable should be based on the same hierarchy as the query. Therefore, you will change the hierarchy node variable to reflect the new hierarchy variable in this query.

31.1.2.1.1.6.1.1 To create a hierarchy variable

Procedure

1. Go to ► [Start](#) ► [Programs](#) ► [Business Explorer](#) ► [Query Designer](#) .
2. Click [Open Query](#).
3. Select the query you created in the last tutorial.

This query already contains a hierarchy and a hierarchy node variable.

If you have not created a query, complete the steps outlined in [How to create a simple BW query \[page 956\]](#), [How to create a simple BW query with a hierarchy \[page 960\]](#) and [How to create a query with a hierarchy node variable \[page 965\]](#).

4. In the Rows area, right-click [Material](#) and click [Properties](#).

The [Properties for Characteristic Material](#) dialog box appears.

5. Click the [Values](#) icon.

The [Select Hierarchy](#) dialog box appears.

6. Select the [Variables](#) check box and then click the [New Variable](#) icon.

The SAP BW Variables Wizard appears.

7. Ensure that the [Type of Variable](#) field says Hierarchy.
8. Fill in the rest of the fields as follows:

- In the [Variable Name](#) field, type a name such as HV01.
- In the [Variable Description](#) field, type a name such as Test Hierarchy Variable.
- In the [Processing by](#) field, ensure it says User Entry/Default Value.
- In the [Characteristic](#) field, ensure it says Material.

9. Click [Next](#).

The [Details](#) window appears.

10. On the [Variable Entry is](#) list click [Optional](#) or [Mandatory](#).

The [Default Values](#) window appears.

11. Click [Next](#) to accept the default values.

12. Click [Finish](#) to create the variable.

You are returned to the [Select Hierarchy](#) dialog box. Your new hierarchy variable is visible in the Name list.

13. Select the new hierarchy variable from the Name list and click [OK](#).

In this example, the hierarchy variable name appears as HV01(Test Hierarchy Variable).

Results

Now that the display hierarchy is set to a variable rather than the Material class hierarchy, you must reset the hierarchy node variable to the new hierarchy variable.

31.1.2.1.1.6.1.2 To set the hierarchy node variable

Procedure

1. In the Rows area, click the Test Hierarchy Node Variable, and click [Restrict](#).

The [Selection for Material](#) dialog box appears.

2. In the Selection area, right-click the Test Hierarchy Node Variable.

3. Click [Select Variable Hierarchy](#).

4. Select the [Variables](#) check box to expose the variable hierarchies in the Name list.

5. From the Name list, click the hierarchy variable you created earlier in this tutorial.

In this example, the hierarchy name is HV01 (Test Hierarchy Variable).

6. Click [Save Query As...](#) and name the query Simple Hierarchy Variable w HN Variable.

7. Click [Quit and Use Query](#).

Results

You now have a simple query that will prompt the user to choose a particular hierarchy to use when displaying results and to restrict that result set to a particular hierarchy node.

31.1.2.1.1.6.2 How to build a report off the query with a hierarchy variable and hierarchy node variable

This section guides you through the process of creating a basic report off your new query and creating a list of default values for the hierarchy variable in Crystal Reports.

31.1.2.1.1.6.2.1 To build a report off the query with a hierarchy variable and hierarchy node variable

Procedure

1. Create the same report that you created in [Building a report off a BW query \[page 955\]](#) except you are now using the query called Simple Hierarchy Variable w HN Variable as the data source.
2. Click the [Refresh](#) button.

You are presented with two parameter prompts. One prompts you to enter a value for the hierarchy node variable and one prompts you to enter a value for the hierarchy variable.

3. Select a value for the Hierarchy parameter.

ⓘ Note

Be sure to select this value first because the Hierarchy Node values are set based on the selection you make here.

4. Select a value for the Hierarchy Node parameter.

31.1.2.1.1.7 Crystal Reports and variable pick lists

Crystal Reports does not create default pick lists for any BW variables that are in your query.

ⓘ Note

You do not need to do this when viewing the report in SAP BusinessObjects Business Intelligence platform. In SAP BusinessObjects Business Intelligence platform the pick list is dynamic.

31.1.2.1.1.7.1 Creating default values list for hierarchy node variable

Click a link to view one of the topics in this section:

- [To create a list of default values for the hierarchy node variable \[page 971\]](#)

31.1.2.1.1.7.1.1 To create a list of default values for the hierarchy node variable

Context

📘 Note

This procedure is based on the query you created in [Building a report off a BW query with a hierarchy variable and a hierarchy node variable \[page 967\]](#).

Procedure

1. In the Field Explorer, expand *Parameter Fields*, and right-click *[HNV01]*.
The parameter, [HNV01], in CR represents the hierarchy node variable you created for this query.
2. Click *Edit*.
The Edit Parameter dialog box opens.
3. Ensure that the *List of Values* field is set to *Static*.
4. From the *Value Field* list, select *Material MemberUniqueName*.
The attributes appear with their technical names in this list.
5. From the *Description Field* list, select *Material*, which is the Member Caption field.
The attributes appear with their technical names in this list.
6. Click *Actions* and then click *Append all database values*.
The Value and Description lists are populated with values from the BW system.
7. In the Values Options area, set the Prompt Text to the message you would like users to receive in the parameter prompt.
8. Ensure that Allow custom values and Allow multiple values are set to False.
9. Click *OK*.

Results

Now when you refresh the report and are presented with the parameter prompt, you are able to pick the parameter value from a list of values.

📘 Note

If your variable is optional, ensure that the Set to Null check box is clear when you select a parameter value. The Set to Null check box overrides any value you select from the list of default parameter values. This check box is not always cleared when you select a parameter value.

31.1.2.1.1.7.2 Creating list of default values for hierarchy variable

Context

Note

This procedure is based on the query you created in [Building a report off a BW query with a hierarchy variable and a hierarchy node variable \[page 967\]](#).

When a query contains a hierarchy variable, the MDX Query driver creates an extra field called Hierarchies in Crystal Reports that allows you to create a list of values for the variable.

Procedure

1. In the Field Explorer, expand [Parameter Fields](#), and right-click [\[HV01\]](#).

The parameter, [HV01], in CR represents the hierarchy variable you created for this query.

2. Click [Edit](#).

The Edit Parameter dialog box opens.

3. Ensure that the [List of Values](#) field is set to [Static](#).
4. From the Value list, select [Hierarchies-\[OMATERIAL\]](#).
5. From the Description list, select [Hierarchies-\[OMATERIAL\]-Description](#).
6. Click [Actions](#) and then click [Append all database values](#).

The Value and Description lists are populated with values from the BW system.

7. In the Values Options area, set the Prompt Text to the message you would like users to receive in the parameter prompt.
8. Ensure that Allow customer values and Allow multiple values set to False.
9. Click [OK](#).

31.1.2.1.1.7.2.1 To create a list of default values for the hierarchy node variable

Procedure

Repeat the steps outlined in the task [To create a list of default values for the hierarchy node variable \[page 971\]](#).

Now when you refresh the report, you are presented with two prompts, each with a list of default values.

31.1.2.1.2 Reporting off Operational Data Stores

This section introduces the Operational Data Store Driver and how to report off it.

31.1.2.1.2.1 Operational Data Store driver overview

The Operational Data Store (ODS) driver allows you to use existing ODS objects from your BW Data Warehouse as a data source in Crystal Reports. ODS objects are collections of info objects that act as smaller organizational units for the data in your BW Data Warehouse. Through this driver, the user can design multiple reports by quickly referencing the same ODS objects.

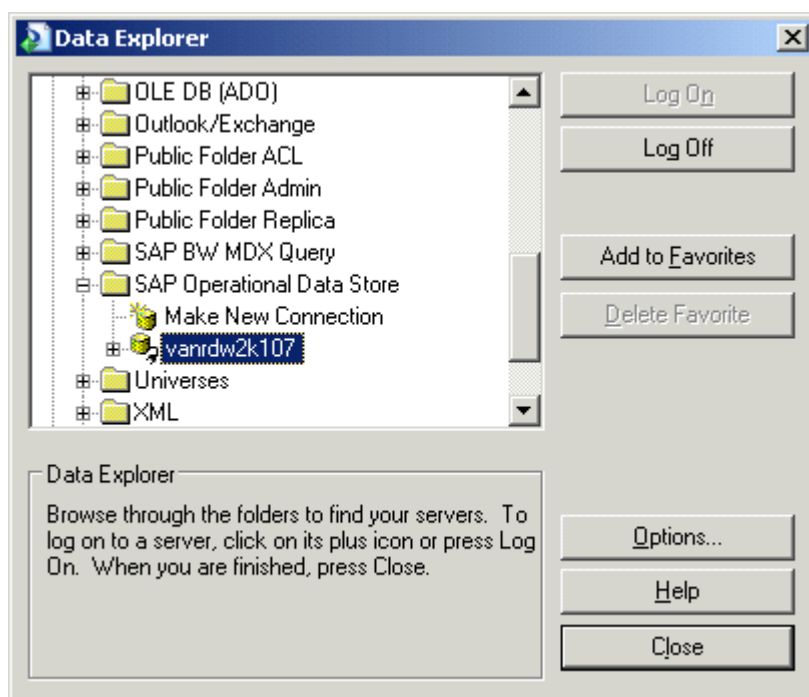
Reports that are based on ODS objects behave similarly to reports that are based on stored procedures in mainstream databases. Because the data stored in the ODS objects is not multidimensional and contains no parameters, it can be accessed and reported from quickly. This driver also pushes record selection to the server, which further speeds up report processing.

31.1.2.1.2.2 Selecting an ODS object

Procedure

1. Start Crystal Reports.
2. Connect to SAP as outlined in [Using the Log On Server command \[page 943\]](#).

Select SAP Operational Data Store in the Data Explorer.



3. Click [Options](#).

The [Options](#) dialog box appears with only the Database tab displayed.

The screenshot shows the 'Options' dialog box with the 'Database' tab selected. The dialog is divided into three main sections: 'Tables and Fields', 'Data Explorer', and 'Advanced Options'. In the 'Tables and Fields' section, 'Show Name' is selected with a radio button, and 'Sort Tables Alphabetically' is checked. In the 'Data Explorer' section, 'Tables', 'Views', and 'Stored Procedures' are checked, while 'System Tables' and 'Synonyms' are not. There are input fields for 'Table name LIKE:' and 'Owner LIKE:', both with wildcards '(%,_)'. The 'Advanced Options' section contains several checked options: 'Use Indexes or Server for Speed', 'Database Server is Case-Insensitive', 'Select Distinct Data for Browsing', 'Verify On First Refresh', 'Verify When Database Driver Upgraded', and 'Automatic Smart Linking'. At the bottom are 'OK', 'Cancel', and 'Help' buttons.

→ Tip

If you chose to use the report wizard, right-click SAP Operational Data Stores and click Options.

4. In the Data Explorer area, select the options for the types of data that you want to make available for the report.

In this case, ensure that Stored Procedures is selected.

5. Use the [Table name LIKE](#) and [Owner LIKE](#) fields to select only a subset of the available data types.

Use any of the following techniques when filtering:

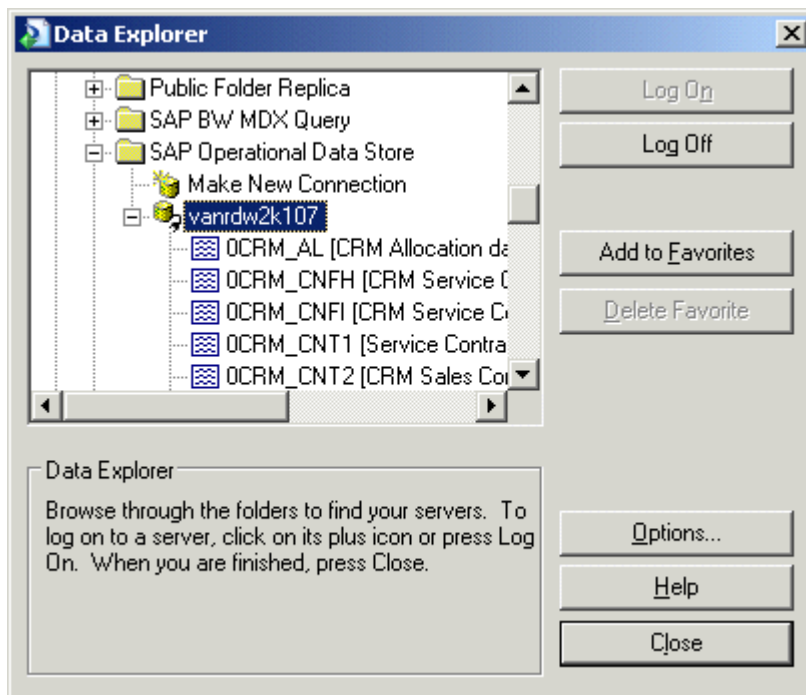
- Type full or partial names of tables.
- Add wildcards to select multiple ODS objects.
 - Use a percentage sign (%) to denote any number of characters.
 - Use an underscore (_) to denote a single character.

Note

The % and _ wildcards correspond respectively to the asterisk (*) and question mark (?) wildcards used in Windows. The % and _ wildcards correspond respectively to the asterisk (*) and plus sign (+) wildcards in SAP.

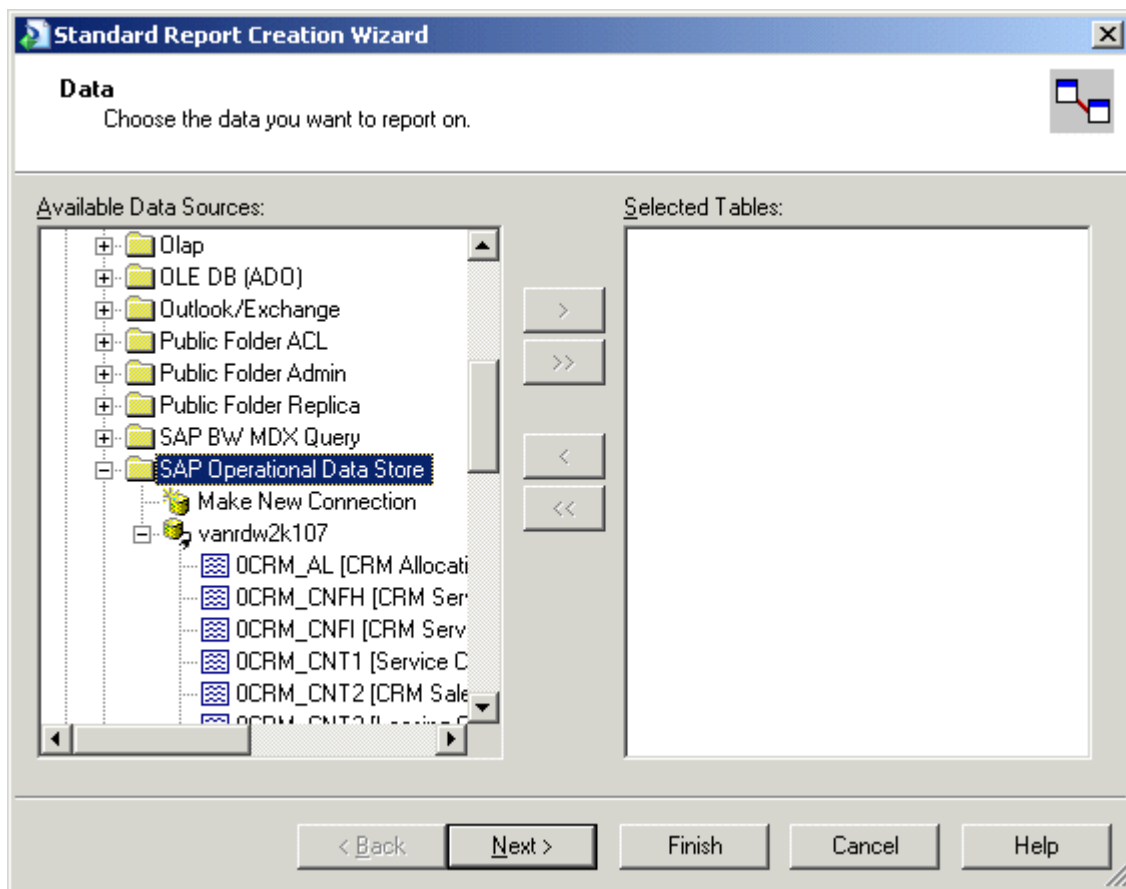
- Clear both fields and click OK to proceed without filtering.
6. In the Tables and Fields area, click [Show Both](#).
 7. When you have specified the ODS that you want to see, click [OK](#).

The Data Explorer displays the selected tables with descriptions defined by the SAP administrator.



8. Click [Close](#).
9. Open a new, blank report.

The Database Expert opens.



10. In the Available Data Sources area, double-click the ODS that you want to use as a data source for the report.

The ODS moves to the Selected Tables area.

11. Click [OK](#).

Results

You can now add fields and continue to design your report.

31.1.2.1.3 Reporting off InfoSets and SAP Queries

This section describes the combined InfoSet/ABAP Query driver and shows how to select an InfoSet or an SAP query as a data source for a Crystal report.

31.1.2.1.3.1 InfoSet Query driver overview

The combined InfoSet/ABAP Query driver allows you to use an existing SAP query or InfoSet (previously known as a Functional Area) as a data source for a Crystal report. The SAP query or InfoSet is built within SAP by a content expert, who then makes the data set available to the business user. Through this driver, the user can design multiple reports by quickly referencing the same query or InfoSet.

Because an InfoSet can be built on top of a logical database in SAP, it can inherit all of the business logic and security that is programmed into the logical database. However, the report that is designed through the driver can contain only as much data as the SAP query or InfoSet has been designed to retrieve.

Reports that are based on SAP queries and InfoSets behave similarly to reports that are based on stored procedures in mainstream databases. You can insert subreports that are based on different queries or InfoSets into the main report.

📘 Note

If you want to join two or more InfoSets or SAP queries, you should consider creating a brand new InfoSet or SAP query instead. The new join will be processed on the SAP server allowing the report to respond faster.

📘 Note

Your SAP queries and InfoSets may contain parameters that you can use to limit the data that is returned from the SAP system. For example, you might specify a date range to get data for only a certain period. If parameters are included in the SAP query or InfoSet, Crystal Reports prompts you to specify the values that you want to include in the query.

→ Tip

You can use transaction SQ02 to create an InfoSet in SAP, and transaction SQ01 to create an SAP query. For more information about designing SAP queries and InfoSets, refer to your SAP documentation.

31.1.2.1.3.2 Accessing an InfoSet or SAP query

User groups are groups that are managed in SAP. They allow you to control user access to InfoSets and SAP queries

An SAP query can belong only to a single user group (which must be shared with the InfoSet that the SAP query is based on). In contrast, InfoSets can belong to multiple user groups. For a user to report off an InfoSet (or SAP query based on an InfoSet), the user and the InfoSet/SAP query must belong to a common user group.

→ Tip

To associate existing InfoSets and users with additional user groups in SAP, use transaction SQ03.

31.1.2.1.3.3 Selecting an InfoSet or SAP query in Crystal Reports

Procedure

1. Start Crystal Reports.
2. Connect to SAP as instructed in [Using the Log On Server command \[page 943\]](#). Select the SAP InfoSet driver, and provide your user credentials.

The *Data Explorer* dialog box appears.

3. Click *Options*.

The *Options* dialog box appears with only the *Database* tab displayed.

The screenshot shows the 'Options' dialog box with the 'Database' tab selected. The dialog is divided into three main sections: 'Tables and Fields', 'Data Explorer', and 'Advanced Options'. In the 'Tables and Fields' section, 'Show Name' is selected with a radio button, and 'Sort Tables Alphabetically' is checked. In the 'Data Explorer' section, 'Tables', 'Views', and 'Stored Procedures' are checked, while 'System Tables' and 'Synonyms' are unchecked. There are input fields for 'Table name LIKE:' and 'Owner LIKE:', both with wildcards. The 'Advanced Options' section contains several checked options: 'Use Indexes or Server for Speed', 'Database Server is Case-Insensitive', 'Select Distinct Data for Browsing', 'Verify On First Refresh', 'Verify When Database Driver Upgraded', and 'Automatic Smart Linking'. At the bottom are 'OK', 'Cancel', and 'Help' buttons.

4. In the Data Explorer area, select the check boxes for the types of data that you want to make available for the report.
5. Use the *Table name LIKE* field to select only a subset of the available data types. Use any of the following techniques when filtering:

- Type full or partial names of tables.
- Add wildcards to select multiple queries or InfoSets: use a percentage sign (%) to denote any number of characters; use an underscore (_) to denote a single character. (The % and _ wildcards correspond respectively to the * and ? wildcards that are used within Windows. The % and _ wildcards also correspond respectively to the * and + wildcards that are used within SAP.)
- Clear both fields and click OK to proceed without filtering. Note, however, that there is a limit to the number of objects that can be displayed in the Data Explorer. If the number of queries or InfoSets in SAP exceeds this limit (which is by default 8000), you may receive an Out of Memory error or a blank Data Explorer.

Note

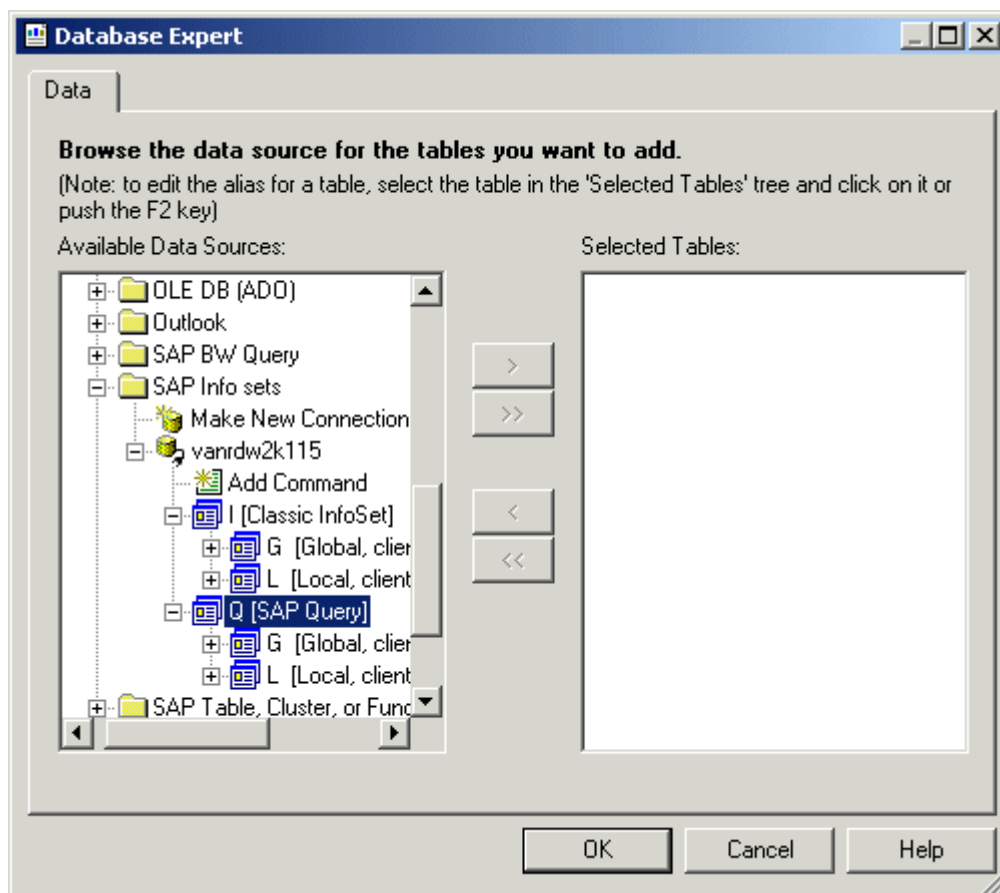
The InfoSet driver does not make use of the Owner LIKE field.

6. In the Tables and Fields area, select *Show Both*.
7. When you have specified the data sources that you want to see, click *OK*.

The Data Explorer displays the selected queries or InfoSets, along with the descriptions defined by the SAP administrator.

8. Click *Close*.
9. Open a new, blank report.

The Database Expert appears.



→ Tip

You can also add InfoSets or SAP queries to existing reports. To do so, open the desired report and, on the *Database* menu, select *Database Expert*.

10. In the Available Data Sources area, locate the query or InfoSet that you want to use as a data source for the report:

- Expand either I (for Info Set) or Q (for SAP query).
- Expand either G (to search in the Global area) or L (to search in the Local area).

ⓘ Note

Queries and InfoSets in the Local area are client-dependent.

11. Double-click the query or InfoSet that you want to use as a data source for the report.

ⓘ Note

If the query or InfoSet contains parameters, the Enter Parameter Values dialog box will prompt you for information. Use the dialog box to specify the data that you want to include, and then click OK.

12. In the *Data Explorer* dialog box, click *OK*.

Results

→ Tip

For additional information about parameters and general report design topics, see the *Crystal Reports Online Help*.

31.1.2.2 Reporting off other SAP data sources

In addition to the previously mentioned drivers, Crystal Reports provides the Open SQL driver, which allows you to report off additional SAP data sources from within Crystal Reports.

31.1.2.2.1 Reporting off tables, views, clusters, and functions

The Open SQL driver is the most powerful of the drivers due to the flexibility it provides for accessing data in SAP. This flexibility, however, results in a higher level of complexity than with the InfoSet driver.

When you create reports that use the Open SQL driver, you gain easy access to SAP's transparent tables, pool tables, cluster tables, and views. Users who are currently involved with the production of ABAP reports will find it easy to build reports quickly with this driver.

Advanced functionality in the driver also allows reporting against ABAP functions and ABAP data clusters. ABAP developers can use these features to perform advanced tasks such as reporting against data clusters in HR.

31.1.2.2.1.1 Open SQL driver overview

The Open SQL driver is a powerful driver because of the flexibility that it provides for accessing data in SAP. By writing reports with the Open SQL driver, you gain easy access to SAP's transparent tables, pool tables, cluster tables, and views. If you are currently involved with the production of ABAP reports, you will find it easy to quickly build reports with this driver.

The advanced functionality of the driver allows you to report against ABAP functions and ABAP data clusters. ABAP developers can therefore perform advanced tasks such as reporting against data clusters in HR. The Cluster Definition tool is provided as part of the server installation for users who need to add, edit, and delete data cluster definitions. Developers can also utilize ABAP to augment report processing.

Additionally, the Open SQL driver provides the ability to secure the data to which users have access. You can place limitations either on individual tables, or, at a more granular level, on the individual rows within a table. For more information about using the Security Definition Editor to customize your own set of data access authorizations, see the "Defining Security for the Data Access Kit" section in the *SAP BusinessObjects Business Intelligence Platform Installation Guide*.

31.1.2.2.1.1.1 Objects supported by the Open SQL driver

The Open SQL driver provides access to several low-level objects within SAP: transparent tables, pool and cluster tables, views, and ABAP data clusters and functions.

31.1.2.2.1.1.2 Transparent tables

This type of database object is similar to the traditional database table. Transparent tables contain the majority of the SAP application data from which you can derive Business Intelligence. Most SAP applications use one or more transparent tables to store and retrieve business transactions. You can link these tables to other transparent tables in order to pull information from other application areas of the SAP system.

31.1.2.2.1.1.3 Pool and cluster tables

Pool and cluster tables are made up of a logical grouping of other database tables. They are primarily used to store application configuration and control information rather than the actual application data. Because these tables are only a logical view of several tables combined, they are accessible only through the Open SQL driver. Native database access does not provide the same view of this data.

31.1.2.2.1.1.4 Views

A view appears as one table with a list of fields, but actually consists of several tables linked together. Views provide a more abstract view of the data and allow you to build reports without having to worry about linking multiple tables in a report. Views are maintained within SAP and need to be configured before you use them for reporting. For more information about views, see your SAP documentation.

31.1.2.2.1.1.5 ABAP data clusters and functions

Reporting off ABAP data clusters and functions are advanced topics that usually require some understanding of the ABAP programming language. For details, see [Reporting off ABAP functions \[page 985\]](#) and [Reporting off ABAP data clusters \[page 989\]](#).

31.1.2.2.1.2 Selecting tables, views, functions, and clusters

Once you have selected the Open SQL driver and provided valid SAP user credentials, use the Options dialog box to choose the tables, views, functions, or clusters that you want to add to the report. You can filter by the type of table and by the actual table name. Note that, in this situation, table also refers to views, ABAP data clusters, and ABAP functions.

31.1.2.2.1.2.1 To select tables, views, functions, and clusters

Procedure

1. Start Crystal Reports.
2. Click the [Start Page](#) tab if the Start Page is not already visible in Crystal Reports.
3. Click [Blank Report](#) in the New Reports area.

The [Database Expert](#) dialog box appears.

4. In the [Create New Connection](#) folder, expand [SAP Table, Cluster, or Function](#).

The [SAP System Logon](#) dialog box appears.

5. Log on to an SAP system, provide your user logon credentials, and specify extended logon parameters as prompted. Then, click [Finish](#).

You return to the [Database Expert](#) dialog box.

6. Right-click the connection you created and select [Options](#).

The [Options](#) dialog box appears, with only the [Database](#) tab displayed.

7. In the Data Explorer area, select the check boxes for the types of data that you want to make available to the report.

8. Use the [Table name LIKE](#) field to select only a subset of the available data types. Use any of the following techniques when filtering:
 - Type full or partial names of tables.
 - Add wildcards to select multiple queries or InfoSets: use a percentage sign (%) to denote any number of characters; use an underscore (_) to denote a single character. (The % and _ wildcards correspond respectively to the * and ? wildcards that are used within Windows. The % and _ wildcards also correspond respectively to the * and + wildcards that are used within SAP.)
 - Clear the field and click [OK](#) to proceed without filtering. Note, however, that there is a limit to the number of objects that can be displayed in the Data Explorer. If the number of queries or InfoSets in SAP exceeds this limit (which is by default 8000), you may receive an Out of Memory error or a blank Data Explorer.
9. In the Tables and Fields area, select [Show Description](#).
10. Click [OK](#).

The Database Expert displays the selected clusters, tables, or functions, along with the descriptions that were defined by the SAP administrator. Three branches appear under the data connection that you created:

- ABAP Function modules
 - Data Clusters
 - Data Dictionary
11. In the Available Data Sources area, double-click the tables, clusters, or functions that you want to add to the report.

→ Tip

You can also CTRL-click multiple tables and then click >.

12. Click [OK](#).

Results

If you selected multiple tables for the report, you must provide table-linking information. For more details, see [Linking tables and views \[page 983\]](#). If you selected a single table, you can now add fields to the report.

31.1.2.2.1.3 Linking tables and views

Reports that are based on SAP tables (transparent, pool, and cluster tables, as well as views) behave similarly to reports that are based on mainstream databases. If you include more than one table or view in a report, you must describe the relationship between the tables by using the Links tab in the Database Expert in Crystal Reports.

Due to the standardized field-naming conventions that are used by SAP, you may find that the Auto-Link feature (on the Links tab of the Database Expert) generates unnecessary links between tables.

31.1.2.2.1.3.1 To clear Auto-Links

Procedure

1. On the [Database](#) menu, click [Database Expert](#).
2. On the [Links](#) tab, click [Clear Links](#).
3. Click [OK](#).

Results

Note

The [Links](#) tab appears in the Database Expert only when two or more tables are selected for the report.

The following client field and language field features are specific to SAP tables.

31.1.2.2.1.3.2 Client field

Most tables in SAP contain a client field (native field name is MANDT). Because individual SAP systems can be configured to hold data for multiple clients, the client field specifies which rows of data in a table are for which clients. The Open SQL driver automatically handles this configuration. When you connect to SAP, you are prompted to provide your client number: this information eliminates the need to create a selection formula that filters the clients. As well, if a report contains multiple tables, you do not need to include a link on the client field—this, too, is automatically handled by the Open SQL driver.

31.1.2.2.1.3.3 Language field

Some tables in SAP, specifically those that contain text descriptions, have multiple entries—one for each language that is installed on the SAP system. Consequently, with these tables you must create a selection formula on the Language Key (native field name is SPRAS) and specify the language that you want to display. For example:

```
{table.spras} = "E"
```

For additional help on table-linking and on general report design topics, see the *Crystal Reports Online Help*.

31.1.2.2.1.4 Reporting off ABAP functions

Reporting off ABAP Functions is an advanced topic that requires an understanding of the ABAP programming language.

While technically not a table, an ABAP function is often used in SAP reporting to provide calculations, and to retrieve data that is based on parameters that are passed into the function (for example, an employee number passed to a function that returns the employee's address). For this reason, the Open SQL driver provides the ability to call ABAP functions that have defined return types and to display their results. ABAP functions are also useful in situations where complex pre-processing of the data must occur before viewing, or in circumstances where you may want to further optimize the performance of a query.

31.1.2.2.1.4.1 How it works

Using the steps listed in [Selecting tables, views, functions, and clusters \[page 982\]](#), select a function. In the Database Expert's Available Data Sources area, right-click ABAP Function modules and select *Options*. Then, type the full or partial function name in the Table name LIKE field. When you add it to the report, the function appears on the Links tab of the Database Expert just like a table (assuming there is more than one table or function already added to the report). All the input and output parameters are displayed as fields in the table.

Specify the input parameters either by linking them to another table's fields or by adding them to a record selection formula. If the output of the function is one or more tables, the "virtual table" consists of rows that are the cross-product of the values in all the tables. If the output of the function is one or more values or structure instances, the "virtual table" consists of a single row that contains all the values/structure values.

31.1.2.2.1.4.2 Limitations

For functions to be used in a report, they must meet two conditions:

- Functions must have defined return types for each of the output parameters.
- Functions must not have whole tables as input parameters.

Note

If a function does not meet the first condition, you can write a "wrapper" function with defined return types, and then call the original function from within the wrapper function.

Finally, the ability to call entire programs is not supported—you may call individual functions only.

31.1.2.2.1.4.3 Examples

The following examples illustrate the different types of ABAP functions that can be called by the Open SQL driver and show how they appear in Crystal Reports.

31.1.2.2.1.4.3.1 Functions with values as inputs and outputs

```
Function F
  Importing
    VALUE(NAME) TYPE C
    VALUE(DEPARTMENT) TYPE C
  Exporting
    VALUE(Years_of_service) TYPE I
    VALUE(Age) TYPE I
```

This function would appear as a table called "F" with the following schema:

Field	Type
Name	String
Department	String
Years_of_service	Integer
Age	Integer

The first two fields, Name and Department, are input parameters. You must either link to them from another table (or function), or specify them with a record selection formula (for example, {F.Name} = "Richard").

For example, if table "T" contains departments that are over budget, you can link {T.Department} to {F.Department} and have a record selection formula that specifies {F.Name} = "Richard". This formula will get the years of service and age of all people named Richard in departments over budget.

31.1.2.2.1.4.3.2 Functions with values as inputs and outputs and in/out (CHANGING) parameters

```
Function F
  IMPORTING
    VALUE(NAME) TYPE C
    VALUE(DEPARTMENT) TYPE C
  EXPORTING
    VALUE(Years_of_service) TYPE I
    VALUE(Age) TYPE I
  CHANGING
    DATEFIELD TYPE D
```

This function would appear as a table called "F" with the following schema:

Field	Type
Name	String
Department	String

Field	Type
Years_of_service	Integer
Age	Integer
DateField_In	Date
DateField_Out	Date

When working with the Name, Department, and DateField_In fields, you must either link them to another table, or specify them in a record selection formula.

For example, if table "T" contains departments that are over budget, you can link {T.Department} to {F.Department} and have a record selection formula that specifies {F.Name} = "Richard" and DateField = Date(1999,04,14). This formula will get the years of service, age, and start date of all people named Richard in departments over budget on April 14, 1999.

31.1.2.2.1.4.3.3 Functions with structures as inputs and outputs

```
Function F
  IMPORTING
    VALUE(ADDRESS) LIKE S_ADDR STRUCTURE S_ADDR
  EXPORTING
    VALUE(CONTACTINFO) LIKE S_CINFO STRUCTURE S_CINFO
S_CINFO is defined as:
PHONE(20) TYPE C
EMAIL(100) TYPE C
S_ADDR is defined as:
STREET(100) TYPE C
ZIPCODE(6) TYPE N
```

This function would appear as a table called "F" with the following schema:

Field	Type
Address.street	String
Address.zipcode	Integer
ContactInfo.Phone	String
ContactInfo.email	String

As before, you must either link the street and zip code fields to or from another table, or define them in a record selection formula.

31.1.2.2.1.4.3.4 Functions with a table as their output

```
Function F
  IMPORTING
    VALUE(NAME) TYPE C
  EXPORTING
    VALUE(GENDER) TYPE C
  TABLES
    POSITIONS STRUCTURE S_POSN
  S_POSN is defined as:
    TITLE(20) TYPE C
    STARTDATE TYPE D
```

In this case, the function takes a name as an argument and returns the age and position of that person. Suppose that Tom's gender is male, and that the Positions table returned by F for Tom contains:

Title	StartDate
Mail Clerk	1/1/93
Manager, Mail	3/4/95
VP, Mail	7/12/98

When the function is called, the result will be a table called "F" that contains:

Name	Gender	Positions.Title	Positions.StartDate
Tom	M	Mail Clerk	1/1/93
Tom	M	Manager, Mail	3/4/95
Tom	M	VP, Mail	7/12/98

31.1.2.2.1.4.3.5 Functions with multiple tables as their output

```
Function F
  IMPORTING
    VALUE(NAME) TYPE C
  EXPORTING
    VALUE(GENDER) TYPE C
  TABLES
    POSITIONS STRUCTURE S_POSN
    OPTIONS STRUCTURE S_OPTN
  S_POSN is defined as before and S_OPTN is defined as:
    GRANTDATE TYPE D
    NUMBER TYPE I
```

Suppose that the data from Positions is as before, and the Options table looks like:

GrantDate	Number
4/4/93	1000
5/6/97	15300

When the function is called, the result will be a table called "F" that contains:

Name	Gender	Positions.Title	Positions. StartDate	Options. GrantDate	Options. Number
Tom	M	Mail Clerk	1/1/93	4/4/93	1000
Tom	M	Manager, Mail	3/4/95	4/4/93	1000
Tom	M	VP, Mail	7/12/98	4/4/93	1000
Tom	M	Mail Clerk	1/1/93	5/6/97	15300
Tom	M	Manager, Mail	3/4/95	5/6/97	15300
Tom	M	VP, Mail	7/12/98	5/6/97	15300

31.1.2.2.1.5 Reporting off ABAP data clusters

Reporting off ABAP data clusters is an advanced topic that requires an understanding of the ABAP programming language.

31.1.2.2.1.5.1 What is a data cluster?

Unlike most of the SAP system, the Human Resources module stores much of its data in ABAP data clusters, rather than in data dictionary tables such as transparent, pool, or cluster tables. Data clusters are actually stored in some transparent tables. Any such transparent table can contain many different types of clusters. Each cluster of data is stored in encoded binary fields. A cluster can contain any number of individual components or pieces of data, which may range from single values to structures to tables.

You can access a data cluster in an ABAP program by running the ABAP command IMPORT. This command takes parameters—such as cluster database name, cluster ID, and so on—as well as a list of pieces of data to import. The IMPORT command also copies the contents of the data cluster into local variables within the calling code. A typical transparent table that contains cluster data has a structure similar to the following:

client	area	key	line counter	user data	data length	data cluster
--------	------	-----	--------------	-----------	-------------	--------------

The combination of transparent table name and area identifies which type of cluster to import, while the key identifies the specific cluster. For HR, the cluster key is typically some combination of personnel number and

something else. The key is stored in a single text field, but may actually represent more than one logical key field (for example, personnel number + a sequence number for HR Payroll data).

31.1.2.2.1.5.2 How it works

The difficulty in working with clusters from a generic, relational data access point of view is that the definition of the clusters is not stored in a repository such as the ABAP dictionary (unlike the definitions of transparent, pool, and cluster tables, for instance). Furthermore, the format of these clusters may change from site to site due to customizations. Finally it is fundamentally difficult to map these as tables in the relational sense because, although the data represented is relational, it is stored in independent clusters in an almost hierarchical form.

In order to overcome these issues, the individual components of data clusters must be mapped as tables. You can do this by creating a dictionary of the clusters on a particular SAP system.

31.1.2.2.1.6 Mapping ABAP data clusters

The Cluster Definition tool (transaction ZCDD) enables you to create a dictionary of the data clusters on your SAP system. Once you have added a data cluster to the dictionary, you can select the cluster in Crystal Reports and report off its data with the Open SQL driver. You will usually require prior experience with ABAP programming because you need to find the cluster definitions within R/3 before you enter them with the Cluster Definition tool.

The following procedures show how to map the SALDO table (Time balances per period) to your dictionary of data clusters with the Cluster Definition tool. The SALDO table is located in the B2 (PDC Data (Month)) cluster area of the ABAP data cluster that is stored in the PCL2 (HR cluster 2) transparent table. In this case, PCL2 is the cluster database, B2 is the cluster area or type, and SALDO is the table. The procedures listed below correspond to SAP R/3 4.6b:

31.1.2.2.1.6.1 Creating a new cluster entry in the dictionary

This section guides you through the process of defining a new cluster entry in the dictionary.

31.1.2.2.1.6.1.1 To create a new cluster entry in the dictionary

Procedure

1. From the SAPGUI, execute transaction ZCDD.

The Cluster Definition tool displays its [View Cluster Id Table](#) screen.

2. Click [Add](#) to create a new entry.

The *Edit Cluster ID Record* screen appears.

3. In the *Cluster Database* field, type the name of the table in which the ABAP data cluster is stored.

For this example, type PCL2.

Note

To find out which transparent table contains a required ABAP data cluster, ask a subject matter expert about the module on which you are working. Or, if you are trying to locate the data from a particular program (such as an SAP report like RPC11XEO), look for the table in the ABAP code. When you find the table, look at the data in the RELID field for a list of cluster areas.

4. In the *Cluster Area* field, type the name of the cluster area or RELID.

For this example, type B2.

5. Click [Save](#) and exit the Cluster Definition tool.

Results

You now need to find the key to the cluster area. Specifically, you need the name of the dictionary structure or table in which the key fields are stored, and you need the names of those key fields. To locate the cluster area's key fields, go to the next section, [Locating the cluster area's key fields \[page 991\]](#).

31.1.2.2.1.6.2 Locating the cluster area's key fields

This section guides you through the process of locating the cluster area's key fields based on the new cluster entry you created in the last section, [Creating a new cluster entry in the dictionary \[page 990\]](#).

There are several ways to find this information within SAP. This example uses the Object Navigator to locate the key information for the B2 cluster area.

31.1.2.2.1.6.2.1 To locate the cluster area's key fields

Procedure

1. Execute transaction SE80 to access the Object Navigator.
2. Select [Program](#) from the list and type the name of the program that holds the key information related to the cluster area.

For this example, type RPCLSTB2, which is the program that holds the key information for the B2 cluster area.

3. Press [Enter](#).

The Object Navigator displays the information for the specified program.

4. In the *Object name* list, expand the *Fields* folder.
5. Double-click the entry that corresponds to your cluster area's key.

For this example, double-click the B2-KEY entry.

The relevant ABAP code is displayed.

6. In the ABAP code, look for the heading that denotes the beginning of the cluster area's key, and note the structure that is included in the following line.

For this example, look under the heading DATA: BEGIN OF B2-KEY for the line INCLUDE STRUCTURE PC2B0 (highlighted in the image above). This shows that the PC2B0 structure contains the fields of the key.

Now that you've located the name of the structure that holds the key fields, you need to determine the names of those key fields.

7. Create a new session in R/3 and execute transaction SE11 to access the ABAP/4 Dictionary.

The *ABAP Dictionary: Initial* Screen appears.

8. Select *Database table* and type the name of the structure that contains the key fields.

For this example, type PC2B0.

9. Click *Display*.

The *Dictionary: Display Structure* screen appears.

10. Make a note of the names of the key fields displayed in the *Component* column.

In this case, the four key fields are PERNR, PABRJ, PABRP, CLTYP.

Results

You now need to add the key fields to your new dictionary entry in the Cluster Definition tool. To add these key fields to the dictionary entry, go to next section, [Adding the key fields to the dictionary entry \[page 992\]](#).

31.1.2.2.1.6.3 Adding the key fields to the dictionary entry

This section guides you through the process of adding the key fields to the dictionary entry based on the key fields that you noted in the last section, [Locating the cluster area's key fields \[page 991\]](#).

Following are the steps involved in adding the four key fields to the dictionary entry that you created for the B2 cluster area.

31.1.2.2.1.6.3.1 To add the key fields to the dictionary entry

Procedure

1. In the SAP Easy Access screen, execute transaction ZCDD to access the Cluster Definition tool.
2. In the [View Cluster Id Table](#) screen, select the new entry that you created.

For this example, select the PCL2 B2 entry.

3. Click [Edit](#).

The [View Cluster Record](#) screen appears.

4. Click [New Cluster Key](#).

The [Edit Cluster Key Record](#) screen appears.

5. In the [Key Number](#) field, type the sequence number of the key field that you want to specify.

For this example, type 001 (this is the first entry for this cluster area's key).

6. In the [Key Name](#) field, type the name that you want to appear when you use this key field in Crystal Reports.

For this example, type K_PERNR.

→ Tip

In general, follow a naming convention. For instance, start a key field name with K_ to specify it as a key field, as has been done here.

7. Use the [Reference Table](#) field to define the key field's data type by referencing an existing field of the same type in another table.

For this example, type PC2B0, because it is the name of the structure that holds the Reference Field.

8. In the [Reference Field](#) field, type the name of the field to which you want to refer. This field must exist in the Reference Table.

For this example, type PERNR.

ⓘ Note

If the key field is not already defined in the data dictionary, and if you do not define it by referencing an existing field, you can manually specify the ABAP type and Length in order to define the key field's data type. In this example, the key field is defined by reference, so you can ignore the ABAP type and Length fields.

9. Click [Save](#); then click [Back](#) to return to the View Cluster Record screen.
10. Repeat steps 4 to 9 for each field in the key. In the [Key Number](#) field, be sure to increment the sequence number by one every time you define a new key field for this cluster area.

The values for each field in this example are:

Key Number	Key Name	Reference Table	Reference Field
002	PABRJ	PC2B0	PABRJ
003	PABRP	PC2B0	PABRP
004	CLTYP	PC2B0	CLTYP

Once you have entered all of the definitions, the [View Cluster Record](#) screen looks like this.

Results

Now you need to determine the structure of the table that you want to access. In this example, the table in question is SALDO. There are several ways to determine the SALDO table's structure within SAP. The next section [Determining the structure of the cluster table \[page 994\]](#) explains one way.

31.1.2.2.1.6.4 Determining the structure of the cluster table

This section guides you through the process of determining the structure of the cluster table based on the cluster entry you created in the previous section. This example below again uses the Object Navigator to locate the required information about the SALDO table.

31.1.2.2.1.6.4.1 To determine the structure of the cluster table

Procedure

1. In the SAP Easy Access screen, execute transaction **SE80** to access the Object Navigator.
2. Select [Program](#) from the list and type the name of the program that holds the key information related to the cluster area.

For this example, type RPCLSTB2, which is the program that holds the key information for the B2 cluster area and its SALDO table.

3. In the [Object name](#) list, expand the [Fields](#) folder.
4. Double-click the entry that corresponds to the table that you want to access.

For this example, double-click the SALDO entry.

The relevant ABAP code is displayed.

5. In the ABAP code, look for the heading that denotes the beginning of the table, and note the structure that is included in the following line.

For this example, look under the heading DATA: BEGIN OF SALDO OCCURS 50 for the line INCLUDE STRUCTURE PC2B5 (highlighted in the image above). This shows that the PC2B5 structure defines the fields of the SALDO table.

Results

Now that you've determined which structure defines the fields of the table you want to report off, you must add this information to your new dictionary entry by following the steps in section, [Adding the table to the dictionary entry \[page 995\]](#).

31.1.2.2.1.6.5 Adding the table to the dictionary entry

This section guides you through the process of adding the table to the dictionary entry based on the cluster table's structure you created in the last section, [Determining the structure of the cluster table \[page 994\]](#).

31.1.2.2.1.6.5.1 To add the table to the dictionary entry

Procedure

1. In the SAP Easy Access screen, execute transaction ZCDD to access the Cluster Definition tool.
2. In the [View Cluster Id Table](#) screen, select your new entry.

For this example, select the PCL2 B2 entry.

3. Click [Edit](#).
4. In the [View Cluster Record](#) screen, click [New Cluster Data](#).

The [Edit Cluster Data Record](#) screen appears.

5. In the [Piece Name](#) field, type the name of the table as it appears in the cluster area.

For this example, type SALDO.

6. In the [Display Name](#) field, type the name of the table as you want it to appear in Crystal Reports .

For this example, type B2SALDO.

→ Tip

In general, use a naming convention. For instance, combine the table name with the name of the cluster area, as has been done here (in case there is another SALDO table in a different cluster area).

7. In the [Piece Class](#) field, type the abbreviation appropriate to the data piece: (T)able, (S)tructure, or (B)asic type.

For this example, type T.

8. Use the [Reference Table](#) field to define the structure of the new data piece by referencing an existing structure of the same type in another table.

For this example, type PC2B5.

Note

In this example, you need not complete the remaining fields (Reference Field, ABAP Type, and Length). Use these fields to add specific information if the structure of the table is not defined in the data dictionary.

9. Click [Save](#); then click [Back](#) to return to the [View Cluster Record](#) screen.

Now that you have entered all of the definitions, you have finished creating your new dictionary entry in the Cluster Definition tool. The [View Cluster Record](#) screen appears.

Results

When you create a new report with the Open SQL driver, you can report off your customized ABAP data cluster by searching for the table named B2SALDO.

Although it takes some time to define each data cluster, keep in mind that once the definition is complete all users can access the cluster—regardless of whether or not they have ABAP experience.

31.1.2.2.1.7 Defining Security for the Open SQL driver

This section shows how to use the Security Definition Editor to define table-level and row-level security for specific SAP tables when you access them using the Open SQL driver. The Security Definition Editor enables you to maintain your current SAP security levels when you deploy the Open SQL driver.

31.1.2.2.1.7.1 Security Definition Editor overview

When you deploy the Data Access component and distribute the database drivers, you allow SAP users to access the data stored within your SAP R/3, mySAP ERP, and BW systems. By default, the Open SQL driver provides an open reporting environment where there are no data access restrictions. (You can, however, easily reverse this default behavior with the Security Definition Editor's Global Lock feature, which initially restricts users from accessing all SAP tables.

If you need to deploy a security model that regulates users' access to SAP data, use the Security Definition Editor (transaction /CRYSTAL/RLS) to restrict data access at the table level and at the row level. The Security Definition Editor defines security for transparent tables, pool tables, cluster tables, and tables containing data clusters (for instance, PCL2). The restrictions that you specify are enforced whenever a user attempts to connect to SAP with the Open SQL driver, whether to design, refresh, or schedule a Crystal report. Thus, in order to design, refresh, or schedule a Crystal report against R/3, mySAP ERP or BW, users require access to all of the tables referred to by the report.

Data access restrictions that you define using the Security Definition Editor are applied on a per-client basis. That is, the restrictions you apply affect all users who access SAP using the same client used to define the restrictions.

In previous versions of BusinessObjects Enterprise, data access restrictions were client-independent. To preserve restrictions that you defined using earlier versions of this tool, follow the instructions in [Migrating your existing security model \[page 997\]](#) before continuing.

Note

The Security Definition Editor is provided by the Security Definition Editor transport, which you import into your SAP system when you set up the Data Access.

31.1.2.2.1.7.2 Migrating your existing security model

To continue to use the data access restrictions that you defined using a previous version of SAP BusinessObjects XI Integration (using the ZRLS transaction), you can import these restrictions into client-dependent form using /CRYSTAL/RLS. Import these data restrictions before using /CRYSTAL/RLS to create or modify additional restrictions. Any changes you made using /CRYSTAL/RLS will be overwritten when you import the global restrictions.

To apply your global data access restrictions to additional clients, import these restrictions once for each client.

31.1.2.2.1.7.2.1 To migrate data access restrictions

Procedure

1. Enter the transaction /CRYSTAL/RLS in your SAP system.
2. Under the *Extras* menu, select *Import Global Data*.

A dialog appears, warning you that importing these restrictions will overwrite any restrictions you have defined using /CRYSTAL/RLS.

3. Click *Yes*.

Your global, client-independent data access restrictions are imported.

31.1.2.2.1.7.3 Choosing your table-level security model

For enhanced flexibility, the Security Definition Editor provides a Global Lock feature, which allows you to define the basis of your data access security model in one of two ways:

- **Global Lock disabled**
This open security model initially provides all users with access to all tables. To customize the model to match your current SAP security configuration, you can prevent users from accessing particular tables on a table-by-table basis. When you first deploy the Data Access, the Global Lock is disabled by default.

- Global Lock enabled

This closed security model initially denies all users access to all tables. To customize the model to match your current SAP security conventions, you can authorize users to access particular tables on a table-by-table basis. Additionally, you can specify a list of custom functions that users can still report off when the Global Lock is enabled.

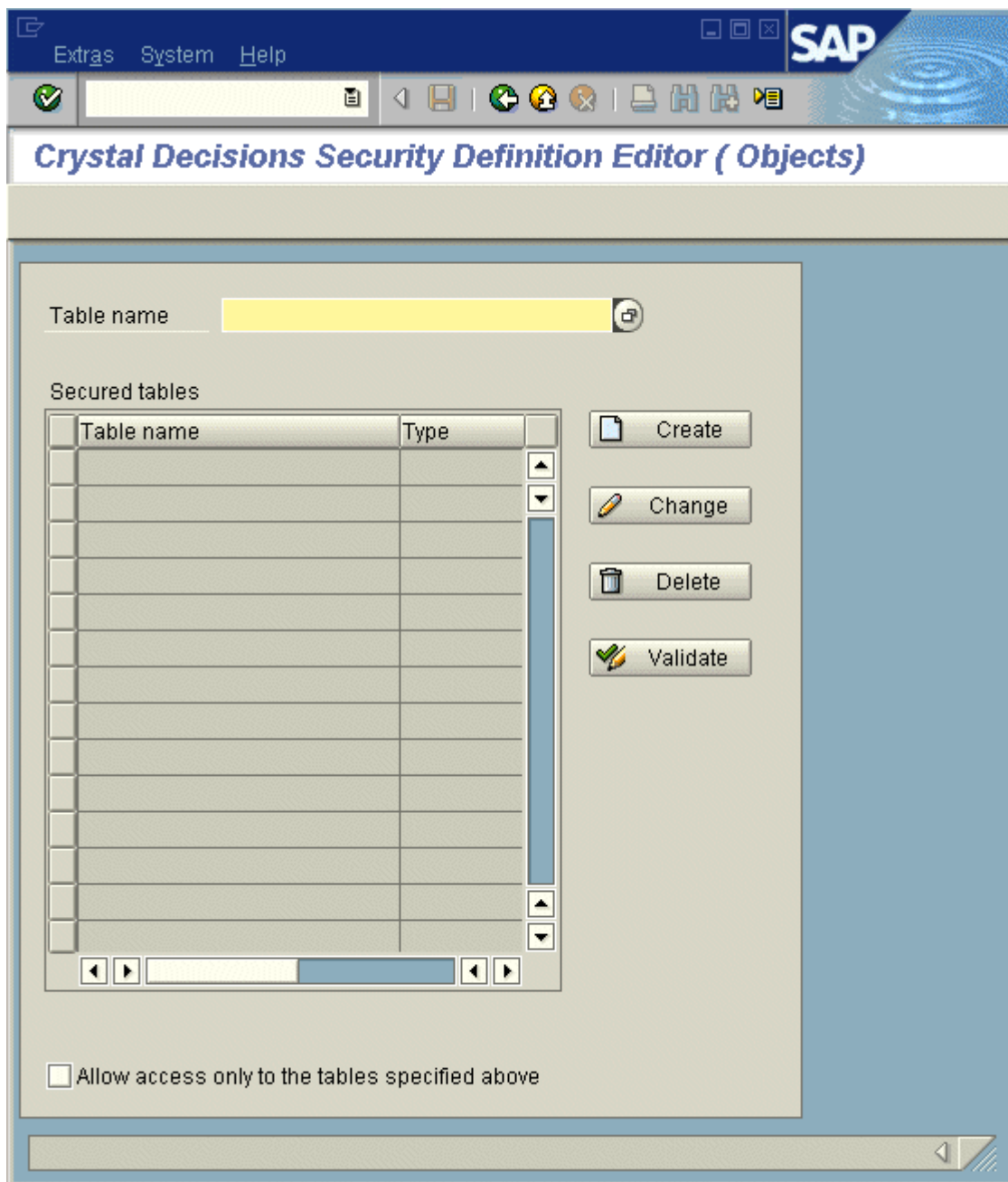
Once you have enabled or disabled the Global Lock, you can refine the security model by customizing data access authorizations for particular tables.

31.1.2.2.1.7.3.1 To enable/disable the Global Lock

Procedure

1. From the SAPGUI, execute transaction /CRYSTAL/RLS.

The *Security Definition Editor* appears.



2. You now have two options:

- If you want to enable the Global Lock, select the *Allow access only to the tables specified above* option.
- If you want to disable the Global Lock, ensure that the *Allow access only to the tables specified above* option is not selected.

Results

Now that you've chosen an open or closed basis for your security model, you can customize authorizations for particular users and tables, thereby allowing or disallowing data access in order to match your SAP security conventions.

31.1.2.2.1.7.4 Customizing authorizations for specific tables

Whether you have enabled or disabled the Global Lock feature, you may need to customize the data access authorizations for one or more SAP tables. If the Global Lock is enabled, you will need to allow all or some users to access particular tables. (Otherwise, users will be unable to design or refresh reports against the SAP system.) Alternatively, if the Global Lock is disabled, you may need to prevent all or some users from accessing tables that contain sensitive data.

In either case, whether you want to secure a table or to permit access to a table, you must use an authorization object for that purpose. (You can customize a new authorization object, or reuse an existing one.) Then, using the Security Definition Editor, you associate the authorization object with the table in question. Finally, you incorporate the new authorization object into your current configuration of user profiles or roles.

Essentially, by associating an authorization object with an SAP table, you define that table as an exception to the Global Lock settings that you have made within the Security Definition Editor. In other words, once you've associated an authorization object with a table, that table is withheld from all users—regardless of your Global Lock settings. You must then use the authorization object according to your usual SAP security conventions in order to provide particular users with access to the SAP table.

31.1.2.2.1.7.4.1 To customize authorizations for a specific table

Procedure

1. Create a new authorization object.

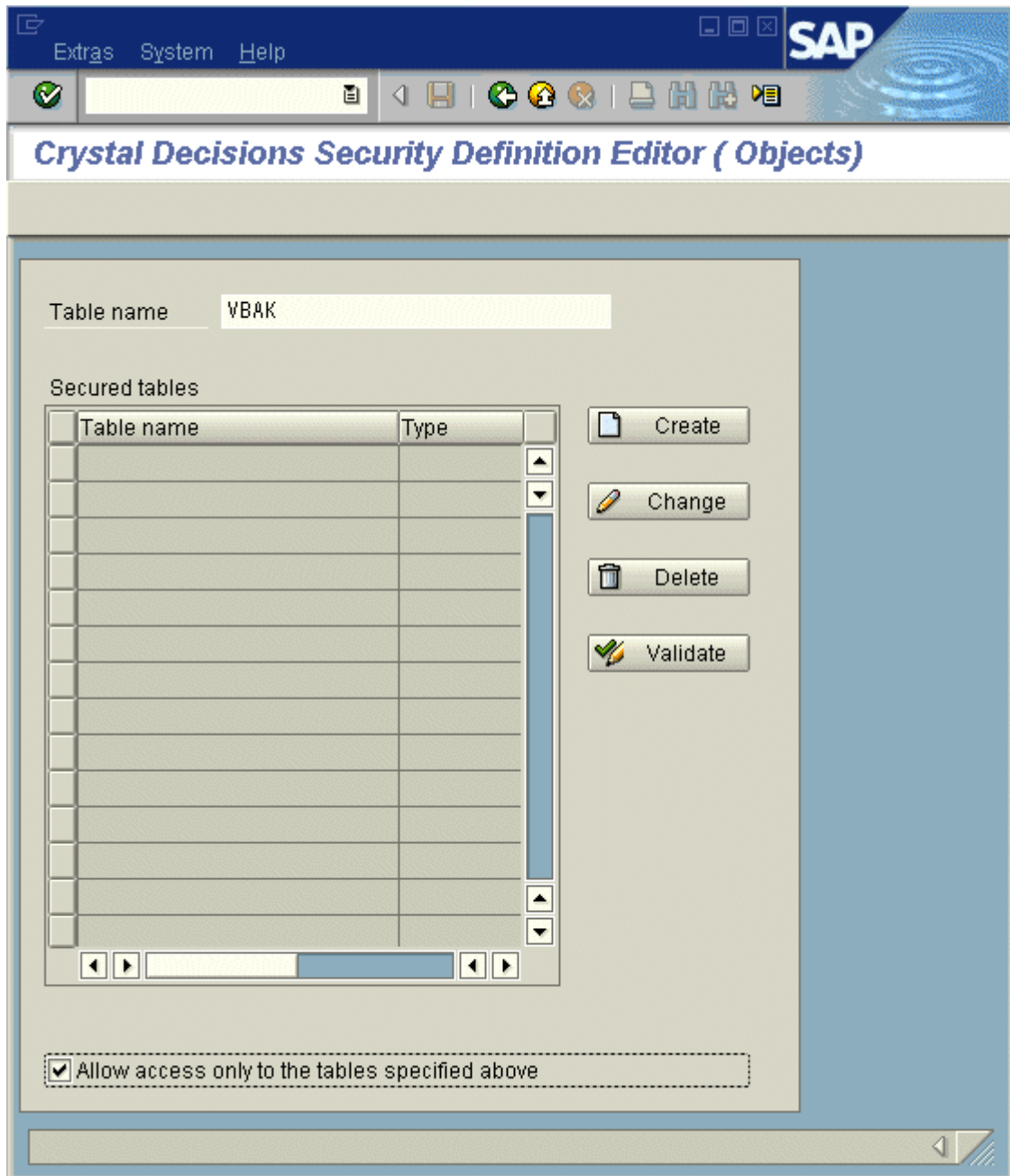
For this example, create an authorization object called ZTABCHK that consists of one authorization field called TABLE. The ZTABCHK authorization is being used to secure the VBAK table from all but a particular subset of users.

Note

Use transaction SU21 to create the authorization object, or see the SAP Help for more information on creating authorization objects.

2. From SAP, execute transaction /CRYSTAL/RLS.

The *Security Definition Editor* screen appears.



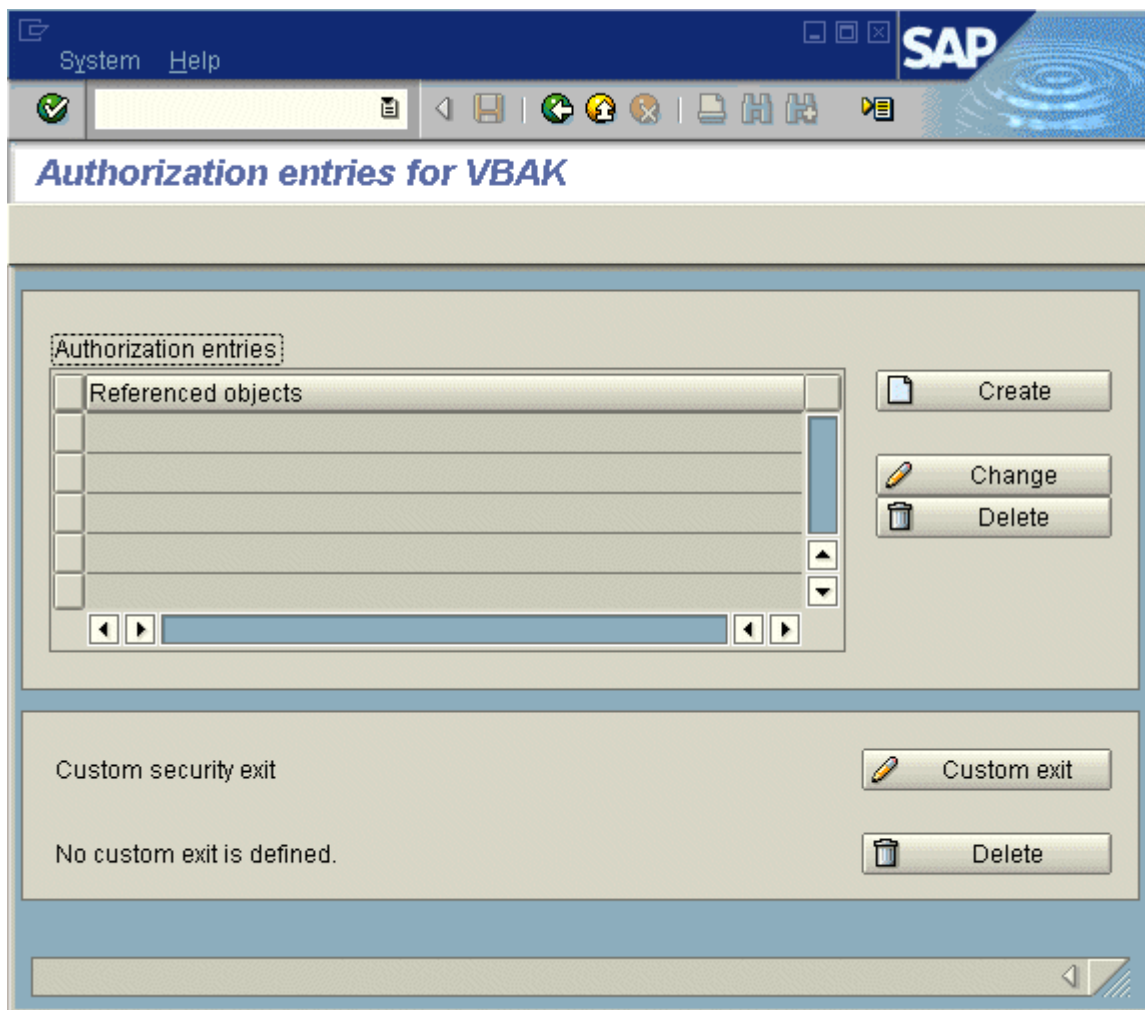
You will now associate your new authorization object with an SAP table.

3. In the *Table name* field, type the name of the table whose security definition you will customize.

For this example, type **VBAK**.

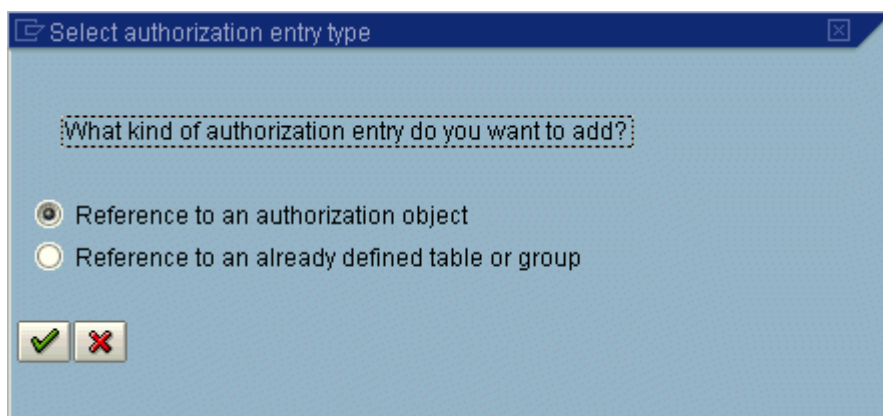
4. Click *Create*.

The *Authorization entries* screen appears



5. Click *Create*.

The *Authorization Object entry* dialog box appears.



6. Click *Reference to an authorization object*.

The *Authorization Object entry* screen appears.

System Help

SAP

Authorization Object entry for VBAK

Authorization object name

Referenced authorization objects

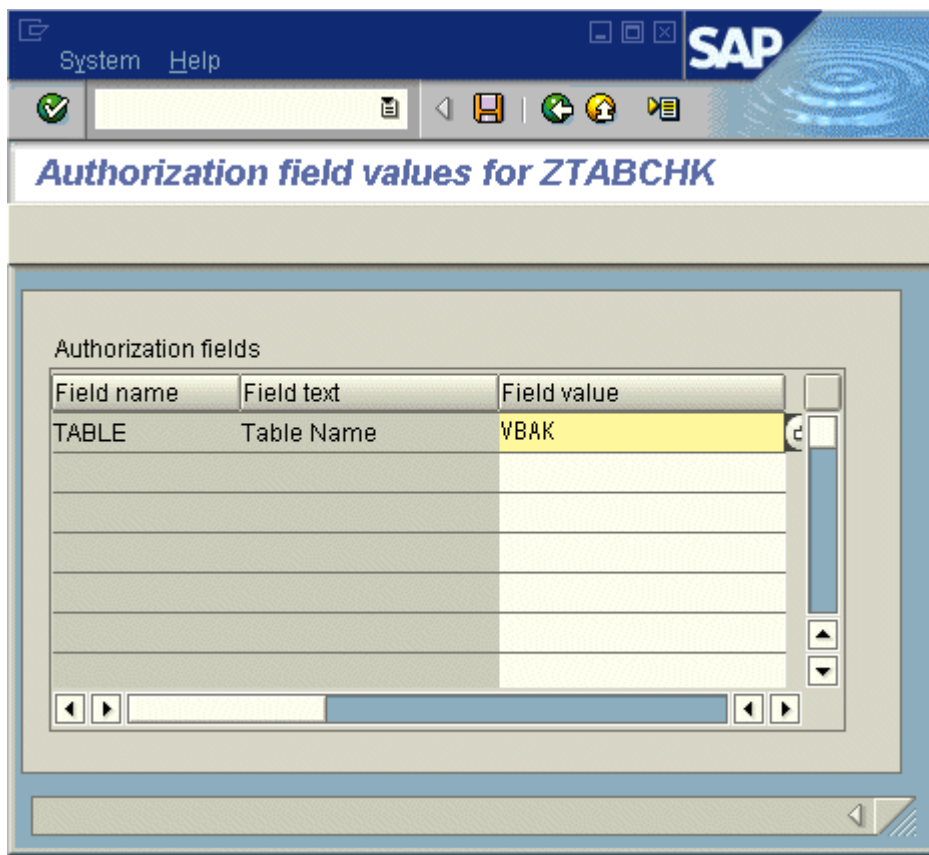
Object name	Object description

Create

Change

Delete

7. In the *Authorization object name* field, type the name of the authorization object that you created in step 1.
For this example, type **ZTABCHK**.
8. Click *Create*.
The *Authorization field values* screen appears.



9. In the *Field value* list, type the name of the table whose security definition you want to customize.

For this example, type **VBAK**.

10. Click *Save*.

You have now associated your customized authorization object with a specific SAP table.

11. Exit the Security Definition Editor.
12. Following your usual procedure, incorporate the new authorization object into your configuration of user profiles or roles.

For instance, following this example, you would apply the ZTABCHK authorization object to the subset of users who need to access the VBAK table.

Depending on the version of R/3 or mySAP ERP that you are using, and the SAP authorization model that you have already deployed, you can accomplish this in various ways:

- Attach the new authorization object to a new role that you in turn add to each user's security profile.
- Attach the new authorization object to a new profile that you in turn add to each user's security profile.

The end result is that the authorization is added to the master user data, so the database drivers can adhere to your customized data access restrictions.

Note

Regardless of the method you use, ensure that your new authorization holds the name of the correct database table in its TABLE field value. In this example, the VBAK table must be specified.

31.1.2.2.1.7.5 Customizing authorizations for specific functions

When the Global Lock feature is disabled, report designers essentially have the freedom to call any function on the SAP system. To restrict the functions available to report off, enable the Global Lock. This restricts users from calling any functions on the SAP system. In the `/CRYSTAL/AUTHFCN` table, you can specify a list of functions that are exceptions to the Global Lock. That is, you can specify a list of functions that users are able to call. To access the list of functions, use the `/CRYSTAL/RLS` tool and go to [Extras > Function modules](#).

31.1.2.2.1.7.6 Defining table authorizations by group

The Security Definition Editor provides two ways for you to quickly define table authorizations for tables that require similar levels of security:

- You can reference a table to any other table whose data access authorizations you have already defined. The [Reference to an already defined table or group](#) option is provided in the [Select authorization entry type](#) screen, which appears in the Security Definition Editor when you create an authorization entry for a table.
- You can reference multiple tables to a group table that acts as a security template. To create a group table, execute transaction `/CRYSTAL/RLS` and specify a nonexistent table name in the Editor. (When the specified table does not exist, the Editor assumes instead that you are creating a group or template.) The main screen listing all the definitions will show the table type as group. You can now share security levels by referencing existing tables to this security template.

31.1.2.2.1.7.7 Applying row-level security

Sometimes table-level security does not allow you to secure data in a way that corresponds with the security privileges of your SAP users: you may need to grant all users access to a particular table, while at the same time limiting a subset of those users to a certain portion of the table's data. To accomplish this, use the Security Definition Editor to define an additional level of security for the individual rows of data.

In general, row-level security is based on a particular field within a table. For instance, you might need to prevent users from seeing data for all customers within a Sales table. To enable this type of restriction, you create an authorization object and use the Security Definition Editor to associate the object with the Customer field in the Sales table. Then, when you apply the new authorization object to each user's profile or role, you specify the Customer values for those customers whose Sales records the user is permitted to view. When the user designs or refreshes reports against the Sales table, data is returned only for those customers that you have specified.

Essentially, by associating an authorization object with a field within an SAP table, you define that table as an exception to the Global Lock settings that you have made within the Security Definition Editor. In other words, once you've associated an authorization object with a field, the parent table is secured from all users—regardless of your Global Lock settings. You must then use the authorization object according to your usual SAP security conventions in order to provide particular users with access to rows of data within the SAP table.

You can secure rows in a table by using authorization objects, custom functions, or both.

31.1.2.2.1.7.7.1 Securing rows using authorization objects

In order to define row-level security for tables, you can create an authorization object and use the Security Definition Editor to associate the object with one of the table's fields. You then apply the new authorization object to the roles or profiles of SAP users who need to access any of the table's data. When you apply the authorization object, you specify the rows of data that each user is permitted to access.

31.1.2.2.1.7.7.1.1 To define row-level security using an authorization object

Procedure

1. Create a new authorization object.

For this example, create an authorization object called Z_BUKRS that consists of one authorization field called BUKRS. This authorization is being used to secure the GLT0 table based on the values in the BUKRS (Company Codes) field.

Note

Use transaction SU21 to create the authorization object, or see the SAPGUI Help for more information on creating authorization objects.

2. Execute transaction /CRYSTAL/RLS to access the Security Definition Editor.
3. In the *Table name* field, type the name of the table that contains the rows you want to secure.

For this example, type **GLT0**.

Note

You can now use wildcards to specify the table name. This is an efficient way to apply an authorization to multiple tables with similar names simultaneously. Use ***** to specify zero or more characters and **+** to specify one character.

4. Click *Create*.
5. In the *Authorization entries* screen, click *Create*.
6. In the *Select authorization entry type* screen, click *Reference to an authorization object*.
7. Click *OK* to access the *Authorization Object entry* screen.
8. In the *Authorization object name* field, type the name of the authorization that you created in step 1.

For this example, you would type **Z_BUKRS**.

9. Click *Create*.

The *Authorization field values* screen appears.

10. In the *Field value* list, type the equals sign (=) and the name of the field that you want to secure.

For this example, you would type **=BUKRS**.

→ Tip

To view a list of available fields, right-click the Field value list and, on the shortcut menu, click Possible entries. Double-click the desired field (in this case, BUKRS) to insert it along with the required equals sign (=).

11. Click [Save](#).
12. Exit the Security Definition Editor.
13. Following your usual procedure, apply the new authorization object to the profiles or roles of the appropriate users.

For instance, following this example, you would apply the Z_BUKRS authorization object to each user who needs to access rows of data within the GLTO table. When you apply the authorization object, you enter the appropriate Company Code values into the BUKRS authorization field, thereby specifying the Company Codes that the specified user is authorized to access.

Depending on the version of R/3 or mySAP ERP that you are using, and the SAP authorization model that you have already deployed, you can accomplish this in various ways:

- Attach the new authorization to a new role that you in turn add to each user's security profile.
- Attach the new authorization to a new profile that you in turn add to each user's security profile.

The end result is that the authorization is added to the master user data, so the database drivers can adhere to your customized data access restrictions.

31.1.2.2.1.7.2 Securing rows using custom security exits

In addition to, or instead of using authorization-based restrictions for tables, you can secure rows using custom security exits. That is, you can map table fields to function parameters so that you can define the call to one function to handle the post filtering for any table.

ⓘ Note

You must give a value (either constant or field-mapped) for all non-optional parameters.

ⓘ Note

Make mappings between compatible types. For example, do not link a function parameter for a date to a number field. Mapping incompatible types results in a failure at runtime.

The following example shows how to map table fields to function parameters in a custom security exit.

31.1.2.2.1.7.7.2.1 To define row-level security using a custom security exit

Procedure

1. Execute transaction /CRYSTAL/RLS.
2. In the *Table name* field, type the name of the data table whose rows you want to secure.

For this example, type PA0000 to secure the rows in that table.

Note

You can now use wildcards to specify the table name. This is an efficient way to apply an authorization to multiple tables with similar names simultaneously. Use * to specify zero or more characters and + to specify one character.

3. Click *Create*.

The *Authorization entries* screen appears.

This screen lists the authorization entries (if any) that are currently applied to the table.

4. Click *Custom exit*.

The *Edit custom function security exit mappings* window appears.

5. In the *Function name* field, type the name of the function that you want to use to secure the table, and then click *Transfer*.

For this example, type HR_CHECK_AUTHORITY_INFTY.

6. For each necessary parameter, specify a value or field to map to that parameter.

In this example make the entries listed in the table below:

Imported parameter name	Field name
INFTY	'0000'
PERNR	=PERNR
SUBTY	=SUBTY

7. Click *Save*.

You are returned to the *Authorization entries* screen, which now indicates that there is an active custom security exit for this table.

When a user attempts to access a table secured by a custom security exit, the driver checks the appropriate field(s) in the table. The field values you mapped are passed as parameters to the function you selected, in this example HR_CHECK_AUTHORITY_INFTY. When authorization restrictions are combined with the values you mapped, the authorization restrictions filters the results first, and then the mapped values further filter the results.

31.1.2.3 Reporting with SAP BW

Click a link to view one of the topics in this section:

- [Designing and Translating Reports \[page 1009\]](#)
- [Publishing Reports \[page 1011\]](#)

31.1.2.3.1 Designing and Translating Reports

This section introduces you to the Report Wizards in Crystal Reports and provides reporting details that are relevant within SAP environments. The section also shows how to prepare reports for translation and how to conditionally modify the report's design based on the user's logon language.

31.1.2.3.1.1 Designing reports for translation

If your BW system supports users in multiple languages, you may want to prepare your reports for translation within BW. Once you publish the translated reports to SAP BusinessObjects Business Intelligence platform, SAP users can view the reports that correspond to their preferred viewing locale. This section describes how to prepare your report for translation, and how to use a report's LanguageCode% formula to make language-specific modifications to reports.

31.1.2.3.1.1.1 Preparing reports for translation

When you save a report to BW, you are given the option to prepare the report for translation. When you select this option, Crystal Reports extracts the string contents from all the text objects that you have inserted into the report (and replaces the original strings with placeholders in the .rpt file). Use the standard translation transactions within BW to access the language table that contains the original strings and translate them into all the languages that you require. Once you finish this process, the translated strings will be inserted back into the report when you publish to SAP BusinessObjects Business Intelligence platform.

When you publish a translated report from BW to SAP BusinessObjects Business Intelligence platform, the BW Report Publisher receives a single .rpt file, the original report strings, a list of destination languages, and all the translated report strings from BW. For each language, the BW Report Publisher copies the original .rpt file and replaces the placeholders with the corresponding report strings. The BW Report Publisher also marks each report with a language code (for details, see [Conditional formatting with the LanguageCode% formula \[page 1010\]](#)). The BW Report Publisher then generates multiple .rpt files (one for each language), and publishes each of these objects to SAP BusinessObjects Business Intelligence platform.

Note

When users refresh or schedule translated reports, new data is retrieved from BW, but the report strings are not refreshed. To update the translations of your report strings, publish the reports again from BW to SAP BusinessObjects Business Intelligence platform.

31.1.2.3.1.1.1 To prepare all reports for translation

Procedure

1. Start Crystal Reports and open a report that you want to prepare for translation.
2. On the [SAP](#) menu, click [Settings](#).

The Settings dialog box appears.

3. Ensure that [Prepare reports for translation on server](#) is selected.

This option changes your default settings, so reports are always prepared for translation on the server.

If you prefer to select this option for individual reports, clear the [Suppress save dialog](#) option. You are then prompted with a [Prepare this report for translation](#) option every time you save a report to BW.

4. Click [OK](#).
5. On the [SAP](#) menu, click [Save Report](#) and proceed as usual.

31.1.2.3.1.1.2 Conditional formatting with the LanguageCode% formula

When you first create a report that is based on a BW query, Crystal Reports automatically generates a formula named LanguageCode%. You can find this formula in the Field Explorer. Rather than performing a calculation, this formula consists of a string value—a single letter—that serves to associate the report with a particular logon language.

When you first create and design your report, the formula contains the code for your current language ("E" for English, "D" for German, and so on). This is the report's master language. When you prepare reports for translation and subsequently publish them to SAP BusinessObjects Business Intelligence platform, the BW Report Publisher modifies the LanguageCode% formula in each translated report. When an SAP user views a Crystal report or logs on to BI launch pad, the application filters the available reports: if a translated report is available in the user's logon language, it is displayed for the user; if a translated report is unavailable, the report appears in its master language.

The LanguageCode% formula is intended to provide you, the report designer, with a tool for conditionally formatting reports in ways that accommodate users with different logon languages. For example, suppose that your report displays values from a Product Category field, the string values of which are translated within SAP. As a report designer, the difficulty lies in that the German strings are longer than the equivalent English strings.

To solve this problem, you could use the LanguageCode% to conditionally reduce the font size for the translated version of the report (as demonstrated in the following steps). Alternatively, you could insert two Details sections—each containing the same database fields—and increase the width of the fields in the second section to accommodate for the longer German strings. Your conditional formatting formula would then show/hide the Details sections according to the report's language.

Note

For complete instructions on conditionally formatting fields and using the Format Editor, see the *Crystal Reports Online Help*.

31.1.2.3.1.1.2.1 To reduce a field's font size conditionally

Procedure

1. Open the report in Crystal Reports, and click the [Design](#) tab.
2. In the Details section of the report, right-click the field that contains the font that you want to conditionally format.
3. On the shortcut menu, click [Format Field](#).

The [Format Editor](#) dialog box appears.

4. Click the [Font](#) tab.
5. Click the [Formula](#) button that corresponds to the Size field.

The Formula Workshop opens a new formula and names it Font Size.

6. In the text area, type this formula (which uses Crystal Syntax):

```
if {@LanguageCode%} = "D"  
    then 10  
    else 12
```

This formula ensures that the font size of the field that is currently selected is reduced from 12 pts to 10 pts when the user's logon language is German (or when the user schedules a German instance of the report).

7. Click [Save and close](#).

31.1.2.3.2 Publishing Reports

This section describes how to publish Crystal reports to SAP BusinessObjects Business Intelligence platform. It also describes how to view those reports from SAP BusinessObjects Enterprise BI launch pad and through other applications.

31.1.2.3.2.1 Publishing predefined reports to SAP BusinessObjects Business Intelligence platform

There are several ways to publish valid Crystal report files to SAP BusinessObjects Business Intelligence platform:

- When you open a report in Crystal Reports, you can simultaneously save it to BW and publish it to SAP BusinessObjects Business Intelligence platform.
- If you have already added a number of reports to BW, you can publish them in batches to SAP BusinessObjects Business Intelligence platform. For details, see [Batch publishing reports from BW \[page 1012\]](#).
- You can add Crystal reports to the system through the Publishing Wizard, the Central Management Console, or the Import Wizard. For details, see the *SAP BusinessObjects Business Intelligence Platform Administrator Guide* (admin.pdf in the docs directory of your SAP BusinessObjects Business Intelligence platform product distribution).

31.1.2.3.2.1.1 Batch publishing reports from BW

You can publish large numbers of Crystal reports using the Content Administration Workbench. For more information on the Content Administration Workbench, see the “Configuring publishing in the Content Administration Workbench” section of the *SAP BusinessObjects Business Intelligence Platform Administrator Guide*.

31.1.2.3.2.1.2 Migrating development content to a production BW system

If you have deployed SAP BusinessObjects Enterprise Integration for SAP Solutions in your development BW environment, you can import your reporting content to the SAP BusinessObjects Business Intelligence platform system that is configured for use with your production BW environment.

Before importing your content, consider the following:

- BW treats Crystal reports (.rpt files) as native objects. If the Crystal reports are stored in the repository of your development BW system, you can transport the BW content and then batch publish the reports to SAP BusinessObjects Business Intelligence platform. (For details about transporting content between BW systems, see the SAP Library.) This procedure ensures that the database information for each report is updated by the BW Report Publisher.
- If you have deleted some or all of the Crystal reports from the repository of your development BW system, you can use the Upgrade Manager to import the report objects from one SAP BusinessObjects Enterprise installation to another. When using the Upgrade Manager, be sure to set the correct database information on each of the report files that you import.
- If the number of report files that you want to migrate is small, you may find it easier to change each report's database information in the Central Management Console (CMC). (Go to the Objects management area, locate the report, and click the Database link on its Process tab.)

After you migrate your content, use the Content Administration Workbench to perform maintenance on your reports. Report maintenance tasks include synchronizing information about reports between SAP BusinessObjects Business Intelligence platform and BW (Update status), deleting unwanted reports (Delete reports), and updating reports migrated from previous versions of SAP BusinessObjects Enterprise (Post-migration).

For more information about maintaining your reports, refer to the *SAP BusinessObjects Business Intelligence Platform Administrator Guide*.

31.2 Integrating with PeopleSoft

31.2.1 Overview

If you installed the Data Connectivity feature of SAP BusinessObjects Business Intelligence platform 4.x, you can work with PeopleSoft data in SAP Crystal Reports.

31.2.2 To access data in SAP Crystal Reports

Procedure

1. Start SAP Crystal Reports.
2. On the Start Page, click Blank Report.
3. In the Database Expert, expand [Create New Connection](#).
4. Double-click the data source type that corresponds to your installation:
 - If you installed the integration product on a PeopleSoft Enterprise (PeopleTools 8.46-8.49) environment, click PeopleSoft Enterprise, and provide the following information:

Field	Action
Server	Type the appropriate URL to connect to the Query Access Services through the PeopleSoft Integration Gateway. (For example, type <code>http://<webserver>/PSIGW/QueryListeningConnector.</code>)
ⓘ Note This URL is specified when you configure the Query Access Services in PeopleSoft.	
User	Type your user name.
Password	Type your password.

- If you installed the integration product on a PeopleSoft Enterprise (PeopleTools 8.21-8.45) environment, click PeopleSoft Query, and provide the following information:

Field	Action
Server	Type <code><///server:port></code> , where <code><server></code> is the name of your PeopleSoft Application Server, and <code><port></code> is the JSL port number that the server uses for the Jolt listener.
User	Type your user name.
Password	Type your password.

- If you installed the integration on a PeopleTools 8.50 environment, click PeopleSoft Query, and provide the following information:

Field	Action
Server	Type the appropriate URL to connect to the Query Access Services through the PeopleSoft Integration Gateway. (For example, type http://<webserver>/PSIGW/PeopleSoftServiceListeningConnector.)
User	Type your user name.
Password	Type your password.

- Click *Finish*.

You can now create Crystal reports with data from your PeopleSoft Enterprise system.

Note

Private queries in PeopleSoft Enterprise are marked with an asterisk (*) to distinguish them from public queries.

Results

Once you have connected to a data source, you can save the connection in your Favorites folder. In the Database Expert, right-click the connection, and click Add to Favorites.

For details on using Crystal Reports please see the *Crystal Reports User's Guide*.

To enable Unicode support, you must configure your system as described in the *Crystal Reports User's Guide* and the *SAP BusinessObjects Business Intelligence platform 4.0 Installation Guide*.

32 Creating Accessible Reports

Click a link to view one of the topics in this section:

- [About accessibility \[page 1015\]](#)
- [Improving report accessibility \[page 1017\]](#)
- [Designing for flexibility \[page 1025\]](#)
- [Improving data table accessibility \[page 1028\]](#)
- [Accessibility and BusinessObjects Business Intelligence platform \[page 1034\]](#)
- [Accessibility and customization \[page 1035\]](#)
- [Resources \[page 1036\]](#)

32.1 About accessibility

When you create Crystal reports for a large audience across the organization—and around the world—you need to account for the diverse needs of that audience. Report designers often create reports for specific languages, countries, job tasks, or work groups, but it is also important to consider the accessibility requirements of users.

Report users may have physical, sensory, or cognitive limitations that affect their ability to access the Web. They may not be able to see, move, or hear. They may have low vision or limited movement. Some people have dyslexia, color-blindness, or seizure disorders; others have difficulty reading or understanding text. They may have a combination of disabilities, with varying levels of severity.

People with disabilities often use assistive technologies: products or techniques that help people perform tasks they cannot perform otherwise. Assistive technologies include adaptive software programs such as screen readers (which translate text into audible output), screen magnifiers, and speech-recognition software. People with disabilities may also use special browsers that allow only text or voice-based navigation. They may use assistive devices such as refreshable Braille displays, or alternative keyboards that use "sip-and-puff" switches or "eyegaze" technology.

To meet the reporting needs of people with disabilities, your reports should be designed to work with as many assistive technologies as possible.

Despite the wide range of potential accessibility issues, you can use the techniques described in this section to create reports that are useful for everyone.

32.1.1 Benefits of accessible reports

As more business and government leaders adopt new standards for delivering web content to people with disabilities, accessible design is becoming critical to information management and delivery.

Accessible design provides many benefits:

- Accessible reports are easier for everyone to use.
Many accessibility guidelines result in improved usability. An accessible report must provide logical and consistent navigation. Its content must be clearly written and easy to understand.
- Accessible reports are more compatible with a variety of technologies, new and old.
Accessible content is easier to export to simple formats that are more compatible with mobile phone browsers, personal digital assistants (PDAs), and other devices with low-bandwidth connections. Some people may not have a keyboard or a mouse. They may have a text-only screen, a small screen, or a slow Internet connection. Accessible design makes it easier for people with limited technology to access information.
- Accessible content is easier to reuse for other formats.
In the viewers, accessible reports are more accurately copied or exported to other formats.
- Accessible reports improve server efficiency.
You may reduce the number of HTTP requests on the server, by providing clear navigation so people can find what they need faster. Providing text-only alternatives can reduce the number of graphics, which take up valuable bandwidth.
- Recent initiatives indicate a worldwide trend towards providing accessible web content.
More companies are making accessibility a requirement for their web content, especially in the United States, where the government introduced section 508 of the Rehabilitation Act. Accessibility is quickly becoming an essential part of web content delivery.
- You may be legally required to provide accessible content.
Each year, more countries introduce anti-discrimination laws that ensure equal opportunities for people with disabilities. Even if you are not legally required to meet accessibility guidelines, you may want to do business with an organization that is required to adhere to them.
- Creating accessible reports is easier than modifying existing reports to make them accessible.
If you build accessible features into your reports now, it will be significantly less expensive than to redesign existing reports later.

32.1.2 About the accessibility guidelines

The most comprehensive accessibility guidelines are the Web Content Accessibility Guidelines (WCAG), developed by the international World Wide Web Consortium (W3C). The WCAG is widely considered the definitive set of recommendations for delivering web content to people with disabilities. The WCAG has influenced the development of similar web content standards around the world.

Organizations and governments worldwide are adopting the accessibility recommendations of the W3C. In Australia, the Disability Discrimination Act includes standards for web site accessibility. Similar guidelines have been introduced in the United Kingdom and throughout Europe. In Canada, all government web content is now developed according to the Common Look and Feel (CLF) initiative, which is largely based on the W3C's Web Content Accessibility Guidelines. Taking web accessibility a step further, the United States government introduced legislation in the form of Section 508 of the Rehabilitation Act, which ensures the right to accessible government web content.

Common to all guidelines is a focus on providing web content that is useful for all people, regardless of disability or impairment. For reports, accessible design is focused on the same key concepts:

- Content must be easy to understand and navigate.
- Text equivalents or alternatives should be provided for non-text objects.

- Objects should be logically organized to clarify relationships between objects.
- Reports must not rely on any one specific type of hardware, such as a mouse, a keyboard, or a color screen.

For more information on specific accessibility guidelines, see [Resources \[page 1036\]](#).

32.1.3 Accessibility and Business Objects products

Business Objects products allow you to design accessible reports and deliver them to your users via the Web. By observing accessibility guidelines, you can use Crystal Reports to create reports that are accessible to users with disabilities. You can then publish these reports to the BI platform Server, where people with disabilities can view them on the Web using the BusinessObjects Business Intelligence platform web desktop and the DHTML viewers.

The reports in this section were created in Crystal Reports and tested using screen readers (including JAWS 4.5).

However, Crystal Reports does not currently provide complete accessibility for report designers with disabilities.

The BusinessObjects Business Intelligence platform management components, including the BusinessObjects Business Intelligence platform Central Management Console (CMC) and the Central Configuration Manager (CCM), do not currently provide access for people with disabilities. The ActiveX and Java viewers are also not accessible.

In the BusinessObjects Business Intelligence platform web desktop, the main user interface for working with reports through BusinessObjects Business Intelligence platform, the ability to log on and view reports is accessible for most users. However, other areas, such as new account sign up and scheduling, may not be accessible.

Note

Although you can use many of the same design guidelines to improve the accessibility of OLAP Intelligence Professional reports, Worksheets are difficult to format for accessibility. Crystal Reports is the recommended option for delivering reports to people with disabilities.

32.2 Improving report accessibility

To begin improving the accessibility of your Crystal reports, start with accessibility guidelines that are quick and easy to implement. A small change in your design conventions or company template may have a significant impact on accessibility.

Simple navigation and clearly-written content are critical for accessibility, but they are easy to implement and useful for all report users.

The following sections provide tips and guidelines on how to design reports with accessibility in mind. Click the appropriate link to jump to that section:

- [Placing objects in reports \[page 1018\]](#)
- [Text \[page 1019\]](#)
- [Color \[page 1023\]](#)
- [Navigation \[page 1024\]](#)
- [Parameter fields \[page 1025\]](#)

32.2.1 Placing objects in reports

There are a few general guidelines to keep in mind when you place objects on a report.

32.2.1.1 Organizing objects logically

When you place objects on reports, make sure their placement is clear and logical, especially when you need to imply a relationship between two objects in a report. For example, if you include a text description of a chart, ensure that it is close enough to the chart to make the connection clear.

Many assistive technologies read from left to right and from top to bottom; therefore, if you include a text description and title for a chart, you should decide which one you want the user to read first. This will ensure that the objects in a report are read in the correct order.

32.2.1.2 Placing objects in order

When you publish a report to SAP BusinessObjects Business Intelligence platform, the HTML version organizes the objects in the report according to the consecutive order that you added them in Crystal Reports, not according to where they were positioned on the report. The report appears the same on the screen, but the underlying HTML code lists the reports objects in the order they were inserted. Instead of reading the report from left to right and top to bottom, screen readers and other assistive devices may follow the order specified in the HTML. To make a report accessible, you must add objects to reports in the order that you want a screen reader to read them.

For example, you place Quarter, Year, and Invoice fields in the Details section and then add the report title "Invoices by Quarter" to the Report Header. When you publish the report to the BI platform Server, it looks the same as it did in Crystal Reports, but the underlying HTML displays the database field headings first, followed by the title. Instead of reading the report title first, a screen reader reads the headings first: "Quarter, Year, Invoice, Invoices by Quarter."

To avoid this, insert the "Invoices by Quarter" title first. Before you add the data table, you could provide an introductory text object that describes the table. Finally, add the fields to the Details section. The report will now make more sense in a screen reader, which will read "Invoices by Quarter. The following table lists our invoices for each quarter. Quarter, Year, Invoice.", and then read the data. (For details on providing accessible data tables, see [Improving data table accessibility \[page 1028\]](#).)

Therefore, to create accessible reports, you must plan the order of your report before you begin working in Crystal Reports. Plan it on paper. Make sure you know which objects you want to add and where you want

them. Include all calculations, images, and charts on your plan. When you create a new report based on your plan, you can start adding objects from the upper left corner and work your way to the bottom right corner of the report. Once the objects are placed, you can make changes to them afterwards without affecting their order.

Note

If you create a text-only alternative of your report, add it to your report as a subreport and, most importantly, add the subreport before you add any other object to your report. For further details, see [Text \[page 1019\]](#).

After you add all objects to the report, you can test their placement order by tabbing through the objects.

32.2.1.2.1 To test the placement order of objects in a report

Procedure

1. Make sure no objects in the report are selected.
2. Press the Tab key.

Crystal Reports selects the object that was placed on the report first.

3. Tab through the remaining objects.

The order that Crystal Reports uses to tab through the objects is the same order adopted by a screen reader that views the published version of the report.

32.2.2 Text

The most common accessibility issue encountered by report designers is also one of the easiest to resolve: providing text-only versions of non-text objects. A non-text object is an object that conveys meaning through a picture or sound. Non-text objects include pictures, charts, graphical buttons, graphical representations of text, sounds, animations, and audio or video clips.

People who use assistive technologies are accustomed to text-only substitutes and, therefore, will respond well to the text-only alternatives you provide.

There are a number of ways you can use text to substantially improve your reports' accessibility:

- Provide text equivalents for objects in reports.
- Provide text alternatives for reports.
- Ensure that text is written and formatted clearly.

Text is a useful tool for creating accessible reports. Most assistive technologies require text input, including screen readers, speech synthesizers, and Braille displays. You can easily resize and format text, and text is the most flexible medium for import and export.

32.2.2.1 Providing text equivalents

When you create reports, there are many opportunities to use text equivalents to clarify non-text objects.

- Place a descriptive text object next to a non-text object, and be sure to add them to the report in consecutive order (for more details see [Placing objects in order \[page 1018\]](#)).
Whenever possible, a text equivalent should communicate the same information as its corresponding object in the report. If a report displays data in a pie chart, for example, include a text box next to the chart that summarizes its contents.
Describe the purpose of the non-text object. For example, if an image performs an action when you click it, describe the action. For a button that opens your web site, provide a text box labeled "Click to view our web site".
- If a report includes audio links, provide a transcript for significant audio clips.
- If a report links to a multimedia or video presentation, provide a transcript. You may also want to provide captioning for the audio portion and an audio description of the visual portion. Captioning should be synchronized with the audio.

32.2.2.2 Providing text-only alternatives

If there are too many non-text objects on a report, or if you do not have the resources to integrate accessible design into all of your reports, then you can provide complete text-only alternatives. For reports that represent data using only charts and graphics, for example, you can provide a link to a text-only alternative that provides the same data in data tables and text objects.

Whenever possible, a text-only alternative should provide the same information as the original report. The information conveyed through images in the main report should also be described using text objects on the alternative report.

Note

If you cannot produce a complete text-only version of the report, you can still improve accessibility by providing a descriptive summary of key information or conclusions illustrated by the report.

It is good practice to provide the text-only alternative on a subreport, linked from the top left corner of the main report, so the user has the opportunity to switch to the text-only version as soon as possible. Add the subreport to the report before any other object to ensure that a screen reader will read it first. If you want the subreport link to appear only for people using screen readers or similar software, you can create a subreport link that is the same color as the background color. The link will appear as a small blank space, but a screen reader will read the text for the link.

32.2.2.2.1 To add a text-only alternative to a subreport

Procedure

1. Create a text-only version of the report and save it.
2. Open a new report.
3. On the *Insert* menu, click *Subreport*.
4. In the Insert Subreport dialog box, select *Choose an existing report* and click *Browse* to locate the report you created in step 1.
5. Click the subreport, then choose *Format Subreport* from the *Format* menu.
6. In the Format Editor, on the *Subreport* tab, select *On-demand Subreport*.
7. To hide the subreport link, on the *Font* tab, choose the color that matches the background color of the report.

Note

Instead of hiding the subreport link, you can conditionally suppress the section that contains the subreport. For details, see [Accessibility and subreports \[page 1028\]](#).

32.2.2.3 Using punctuation

To improve the logical flow of spoken text, you may need to add extra punctuation to create pauses. Without extra punctuation, screen readers may read several text objects as one continuous sentence, making the content difficult to understand. For example, information in data tables may be read without stop. To prevent this, you can break up information in data tables by inserting periods between fields.

Certain punctuation marks are read aloud, which may be distracting if used too frequently. For example, when a screen reader reads a colon ":", it may read it aloud as "colon" instead of a pause. You can change the amount of spoken punctuation in your screen reader's settings.

To troubleshoot your report's punctuation, it is good practice to read the report using a screen reader. Do objects run together too quickly? Or are there too many pauses? Are any punctuation marks read aloud? Does this improve or deter from the usability of the report?

32.2.2.4 Formatting text

After you create text equivalents or alternatives for non-text objects, ensure that the text is clearly written and easy to read. Observe the following design guidelines:

- Use a larger font.
Although people with visual impairments can use the Zoom feature to increase the size of the report, they will not need to magnify the report as much if the font size is larger. For example, chart labels or legends can appear in a small font by default. For general legibility, it is good practice to use a font larger than 8 point. For accessibility, ensure that text is larger than 11 point.

- Use a "sans serif" font.
Simple fonts such as Arial and Helvetica can be easier to read than serif fonts like Times or Palatino.
- Choose left or justified alignment.
Left-aligned or justified text is easier to read than centered or right-aligned text.
- Ensure that text follows the guidelines for color usage. For details, see [Color \[page 1023\]](#).

Note

You can allow users to choose different font settings using a parameter and conditional formatting. For details, see [Accessibility and conditional formatting \[page 1026\]](#).

32.2.2.5 Finding the right balance between text and non-text objects

Text equivalents are very flexible and often the best solution for accessibility, but they are not always necessary or preferred.

Not all non-text objects require text equivalents. You need to include text alternatives only for non-text objects that provide information or navigation elements that the user cannot do without. Images used for decorative purposes do not need a text description. If a report has a watermark image that acts as a background for the data, you do not need to provide a text equivalent. Adding text descriptions for decorative objects can produce unnecessary clutter.

Text versions of visual or auditory objects in reports should be used as a complement to the object—not as a replacement. You do not need to remove non-text objects. Visual objects in reports can be very helpful, especially for people with learning disabilities such as attention deficit disorder, or for people who are deaf. People with hearing impairments may be accustomed to visual communication such as sign language, and may find images more useful than text.

No one presentation method can meet the needs of all users. Audio clips can be very useful for people with visual impairments, but people with hearing impairments will be unable to use them. To help both groups, provide a combination of audio and text. Multimedia presentations may provide audio information for people with visual impairments, as well as video information for people who are deaf or hard of hearing. Multimedia presentations are particularly effective for people with attention deficit disorder. However, people with certain mental health disabilities may be distracted by visual or audio objects.

The best approach is to communicate the same information with both text and non-text objects. Add descriptive text to support the images, and add images that support the text.

If text objects begin to overwhelm your report, you may want to provide a complete text-only version in a separate report or a subreport. For details, see [Providing text-only alternatives \[page 1020\]](#).

To learn more strategies on how to choose presentation methods that meet the needs of a variety of audiences, see [Designing for flexibility \[page 1025\]](#).

32.2.3 Color

The colors you choose for objects in reports can have a significant impact on accessibility for people with visual impairments, low vision, or color blindness. Ensure that your reports can be understood when viewed without color.

32.2.3.1 Contrasting colors

Users with limited vision may be unable to distinguish between colors. To test the color contrast in your report, print or view a black and white copy. You should be able to distinguish between values or fields displayed in different colors (in a pie chart, for example).

If you cannot distinguish between colors on the report, try different colors or use gray shading. If this does not resolve the issue, you can change other characteristics.

For text, use the Format Editor to change the font, size, or style. You can add borders, underlining, or background shading to differentiate text objects from each other.

For charts, use a combination of shading and patterns. You can automatically convert a color chart to a black and white one using the Chart Expert, or you can select values individually and choose your own patterns.

32.2.3.1.1 To convert a chart into black and white

Procedure

1. Select the chart and choose [Chart Expert](#) from the [Format](#) menu.
2. In the Chart Expert, click the [Options](#) tab.
3. In the "Chart color" area, select [Black and white](#), and then click [OK](#).

The chart colors convert to a variety of high-contrast pattern and color fills.

32.2.3.1.2 To change the fill for a chart value

Procedure

1. Select the chart, then click the shaded area you want to change.
2. On the [Chart](#) menu, point to [Chart Options](#), and then click [Format Background](#).
3. In the Format Background dialog box, on the [Fill](#) tab, choose a color and click [Pattern](#).
4. In the Choose A Pattern dialog box, click a pattern, then click [OK](#).

Results

Note

You can also select a texture, gradient, or picture as a fill for the chart value. See the *Chart Help* for more information.

32.2.3.2 Using color to convey information

Do not use color as the only identifying characteristic for critical information in a report.

For example, a text object may instruct users to "click the green button" to open a subreport. Users with limited vision cannot tell which button is green. The button should be recognizable by another defining characteristic besides its color. For example, you can change the button graphic to a shape that is not used elsewhere on the report, and instruct users to "click the green arrow button". This solution provides color information for people who can distinguish colors, and extra information for people who cannot.

Other common situations where color may be used to provide important information include:

- **Highlighting**
To highlight particular values in a table, do not change only the color of the value. If you highlight outstanding invoices in red, for example, they may look the same as the paid invoices to someone with limited vision. In the Highlighting Expert dialog box, change a font characteristic other than color, such as font style.
- **Hyperlinks**
Using color as the only method for identifying hyperlinks may also cause problems for color-blind users. When you print your report in black and white, check the hyperlinks to ensure that they are still visible.
- **Identifying important areas of the report**
Do not organize a report by using color as a background or as a separator between different sections or areas. Instead of using color to identify sections, establish clear and consistent navigation for the entire report.

32.2.4 Navigation

As with other aspects of accessible design, providing several alternative navigation methods can help you meet the reporting needs of more people. The W3C recommends including several different navigation methods. On the other hand, simplicity is critical for intuitive navigation. Section 508 recommends simple navigation that uses the least number of navigation links possible. Either approach can be effective for your reports, as long as you maintain clarity and consistency.

You may want to use report parts to navigate a report (or to connect several reports). If you provide a series of links in a page header, keep in mind that screen-reading software will reread the navigation information every time the user refreshes the page or views a new page. In this case, simple navigation is preferable.

For a large report, you could provide a list of navigation links as a table of contents in the report header. More extensive navigation can be useful when you have a large volume of data. To allow users to skip the list, you could start with a "Skip the table of contents" link that jumps ahead to the first page header.

In general, report navigation should follow these guidelines:

- Identify the target of each link.
- Provide information at the start of the report that describes the layout and navigation.
- Use navigation consistently.
- Provide the opportunity to skip repetitive navigation links.

32.2.5 Parameter fields

When you include parameter fields in a report, make sure they are clear and simple. Although parameter fields can be a useful tool for providing accessible content, they can also introduce several accessibility concerns. It is important to test all parameter fields for accessibility.

Parameter fields should follow these guidelines:

- Provide a list of default values for the user to choose from.
Avoid requiring the user to type a value for a parameter. When users provide their own values, they need to make sure the format of the value will be recognized by the parameter field. A list of default values is easier to use, and it ensures that the user chooses from values with valid formats.
- Try to avoid complex parameter fields.
A complex parameter field may be more accessible when it is broken down into multiple parameters. When you test the accessibility of your parameter fields, pay particular attention to parameters that require a range. It may be easier to understand if you provide two parameter fields that prompt for discrete values for the top and bottom of the range, rather than ask the user to choose both values in the same parameter field.
- For date fields, do not allow users to choose their own values.
The calendar used to select date values is not currently accessible. Provide a pick-list of default date values. Using a list of default values also helps avoid invalid date formats.

32.3 Designing for flexibility

Flexibility is the key to providing accessible reports. Because different users require different levels of accessibility, it is good practice to provide a variety of presentation styles and methods to meet the needs of as many people as possible. For a detailed report, however, you may not be able to provide multiple presentation styles without cluttering the report with extra objects.

To address this problem, plan the degree to which you want to integrate accessible formats into your reports. You can provide accessible formatting for each object, for each section, or as a subreport. You can then allow users to choose their own accessibility options using a parameter field that prompts them to choose whether or not to display accessible formats.

Using this parameter field, you can conditionally format objects, or conditionally suppress sections that address different access needs. Or you can provide different display options by using subreports.

32.3.1 To create an accessibility parameter field

Procedure

1. In Crystal Reports, on the [View](#) menu, click [Field Explorer](#).
2. In the Field Explorer, right-click [Parameter Fields](#) and click [New](#).
3. In the Create New Parameter dialog box, type the parameter name (**Access** , for example).
4. Ensure that the [Type](#) is set to [String](#).
5. In the Value area, create **Yes** and **No** values.
6. In the Value Options area, add prompting text in the [Prompt Text](#) field (for example, **Do you want to enable accessible formatting for this report?**).
7. Click [OK](#) in the Create New Parameter dialog box.

32.3.2 Accessibility and conditional formatting

Using the accessibility parameter field in simple formulas, you can provide multiple formats for any object in a report. If a user chooses "Yes" when prompted by the parameter, the conditional formulas will ensure that the objects are modified with accessible formatting conventions. If a user chooses "No", then the report appears without accessible formatting, perhaps in the standard company template.

For accessible text formatting, you can follow the guidelines suggested by this section and by the W3C, or you can survey your report users to determine the formats that work best for them. After you determine the formatting options you want to use, you can create conditional formulas that define the options. For example, you can display all database fields in a large Arial font, in white text on a black background, with the Can Grow option enabled.

The following procedure creates a conditional formatting formula based on the ?Access parameter field. The formula increases the font size if the ?Access parameter field is set to "Yes". You can use similar formulas to change colors, add borders, or enable the Can Grow setting. For complete instructions on conditionally formatting fields and using the Format Formula Editor, see the *Crystal Reports Online Help*.

Note

If text objects are too small to accommodate the enlarged font, you can use a similar conditional formatting formula to enable the Can Grow setting, which appears on the Common tab of the Format Editor.

32.3.2.1 To apply accessible settings to font size conditionally

Procedure

1. Open the report in the [Design](#) tab of Crystal Reports.

2. In the Details section, right-click the field you want to conditionally format, and select [Format Field](#).
3. In the Format Editor, click the [Font](#) tab.

4.  Click the [Formula](#) button that corresponds to the Size list.

The Format Formula Editor opens a new formula named Font Size.

5. In the Formula text window, type this formula (which uses Crystal Syntax):

```
if {?Access} = "Yes"
  then 20
else 10
```

This formula ensures that the font size for the currently selected field is increased from 10 point to 20 point when the user chooses to display accessible formatting.

6. Click [Save and close](#).
7. Click [OK](#) in the Format Editor.

32.3.3 Accessibility and suppressing sections

Instead of formatting individual objects conditionally, you can create separate sections for accessible versions of the report content, then use the accessibility parameter field to conditionally suppress sections. The accessible and non-accessible sections can be suppressed or shown, based on the parameter value the user selects.

Creating separate sections for accessible versions of report content may be more time-consuming, but there are a few situations where suppressing sections conditionally can be more practical than formatting on the object level:

- If a report contains many objects, suppressing sections may require fewer conditional formulas.
- Not all settings and features can be formatted conditionally. By suppressing sections, however, you can make any formatting changes you want.
- You may want to provide completely different types of information for people viewing the accessible version of the report. For example, you may want to split visual and audio objects into two different sections and conditionally suppress them based on the parameter value the user chooses.

32.3.3.1 To suppress an accessible section

Procedure

1. Right-click the left boundary of the section you want to suppress conditionally, and click [Section Expert](#).
2.  In the Section Expert, click the [Formula](#) button that corresponds to the [Suppress \(No Drill-Down\)](#) setting.

The Format Formula Editor opens a new formula named Suppress (No Drill-Down).

3. In the Formula text window, type this formula (which uses Crystal Syntax):

```
if {?Access} = "No" then True
```

This formula selects the Suppress option if the user chooses not to view accessible report content.

4. Click [Save and close](#).
5. Click [OK](#) in the Section Expert.

32.3.4 Accessibility and subreports

Accessible report design may become too cumbersome using conditionally formatted objects and suppressed sections. Two situations in particular may be problematic:

- To make the report accessible, you may need to change the overall organization of the report sections, or you may need to provide different objects.
- If the report contains a large number of objects or sections, it may take too much time to create conditional formulas for all of them.

For example, if a report contains many non-text objects displayed in a complex series of groups and sections, you may want to provide a text-only version that uses different objects and a simplified group structure to meet accessibility guidelines. The easiest way to address this problem is to create a subreport that displays the accessible version of the report and place the subreport at the beginning of the main report. For details on creating a text-only accessible subreport, see [Providing text-only alternatives \[page 1020\]](#).

If you want only screen readers to be able to see the subreport, you can hide it by changing the subreport link to the same color as the background. Alternatively, you can use the ?Access parameter field to allow users to choose whether or not the subreport appears in the report. Place the subreport in its own section and conditionally suppress the section based on the ?Access parameter field. For details, see [Accessibility and suppressing sections \[page 1027\]](#).

32.4 Improving data table accessibility

Large tables of data can be difficult to interpret if a person is using a non-visual means of accessing the web, such as a screen reader. People using screen magnifiers or the Zoom feature may also find data tables hard to navigate because they cannot see the table headings at all times. It can easily become difficult to associate the value that a screen reader is reading with the corresponding column and row headings. Users need to be able to understand the data value's position in the table and its relationship to other values.

To improve data table navigation, you can use text objects to provide contextual information with each value. Using conditional formatting or suppression, you can create a report that displays these objects only if the user chooses to view them. Other design guidelines can help make large tables of data easier to understand, such as providing summary paragraphs and expanded column headings.

Note

This section uses terminology consistent with the W3C accessibility guidelines. In these guidelines, the term data table refers to values arranged in columns and rows. In Crystal Reports, data tables take the form

of group or page headings combined with database fields in the Details section. Do not confuse data tables with database tables, which are data sources used by Crystal Reports.

32.4.1 Text objects and data table values

You can make a large table easier to understand and navigate by adding text objects that provide information about each value in the table.

Include whatever information is necessary to establish the meaning and context of the value displayed. When appropriate, include information that describes column headings or neighboring fields. For example, if a report displays employee names and salaries, you can add a text object before the Salary database field that reads "{Last Name}'s salary is ". The user can determine the context and meaning of the value by reading the accompanying text object.

Ensure that your text objects use punctuation that will make the content easier to understand when read aloud by a screen reader. Without accessibility-orientated punctuation, data tables may be read as one long sentence, making navigation and interpretation very difficult. For example, you can add periods after values so a screen reader will pause between columns and rows. For details, see [Using punctuation \[page 1021\]](#).

As with all objects in reports, the order in which you place text objects on the report can affect accessibility. Screen readers read the objects in the order they were originally added. (For details, see [Placing objects in order \[page 1018\]](#).) The correct placement order is critical when you add a text object that identifies the contents of a particular column in a data table. If you add the text objects at the end of the design process, they may be read after the columns that they refer to. When you add text objects that describe values in a report, ensure that you place them on the report in the order that you want them to be read.

Before you can create an accessible data table, you must plan your report in advance, determining which objects and database fields you want to include. Because objects must be placed in the order you want them to be read, planning your content for accessibility is essential. As part of this planning, it is good practice to choose how you will use text objects to identify data table values. You can simply add text objects before each database field. Or you can conditionally suppress text objects or use formulas to combine text objects and values.

32.4.1.1 Labeling data tables with text objects

Before each field, add a text object that describes the field's position in the table. In the following example, the text box provides information about the Employee ID number. When the report is read with a screen reader, each number is preceded by the brief explanation in the text box.

Employee ID	Last Name	Salary
Employee ID number: 1	Last name is: Davolio	Davolio's salary is: \$40,000.00
Employee ID number: 2	Last name is: Fuller	Fuller's salary is: \$90,000.00
Employee ID number: 3	Last name is: Leverling	Leverling's salary is: \$33,000.00
Employee ID number: 4	Last name is: Peacock	Peacock's salary is: \$35,000.00
Employee ID number: 5	Last name is: Buchanan	Buchanan's salary is: \$50,000.00
Employee ID number: 6	Last name is: Suyama	Suyama's salary is: \$30,000.00
Employee ID number: 7	Last name is: King	King's salary is: \$37,000.00
Employee ID number: 8	Last name is: Callahan	Callahan's salary is: \$45,000.00
Employee ID number: 9	Last name is: Dodsworth	Dodsworth's salary is: \$35,000.00
Employee ID number: 10	Last name is: Hellstern	Hellstern's salary is: \$60,000.00

Providing extra information for each value can make a data table appear cluttered for people without vision impairments, so you may want to hide the extra text objects by changing the font color to the same color as the background. The extra text is invisible, but is still detected and read by screen readers.

Employee ID	Last Name	Salary
1	Davolio	\$40,000.00
2	Fuller	\$90,000.00
3	Leverling	\$33,000.00
4	Peacock	\$35,000.00
5	Buchanan	\$50,000.00
6	Suyama	\$30,000.00
7	King	\$37,000.00
8	Callahan	\$45,000.00
9	Dodsworth	\$35,000.00
10	Hellstern	\$60,000.00

32.4.1.2 Labeling data tables conditionally

Although adding text objects is relatively easy to implement, it does not address all accessibility concerns. Invisible text is read by screen readers, but does not help people with limited vision. You can allow the user to choose whether or not to display text descriptions in the data table by conditionally formatting or suppressing text objects.

Make sure your report includes an accessibility parameter field. For instructions on how to create the ?Access accessibility parameter field, see [Designing for flexibility \[page 1025\]](#).

You can use the parameter field to suppress the text objects conditionally. While it has the same effect as changing the font color to the background color, conditionally-suppressed text also allows you to use the parameter field to specify other formatting options such as font size and style.

To display the text objects only when the user chooses **Yes** for the ?Access parameter field, the following report uses a simple conditional formula to enable the Suppress option on the Common tab of the Format Editor.

```
{?Access}="No"
```

The formula must be added for each text object you want to suppress.

When the user chooses **Yes** for the ?Access parameter field, the text objects are not suppressed; the data table displays text descriptions.

Employee ID	Last name is:	Last Name
1	Davolio	
2	Fuller	
3	Leverling	
4	Peacock	
5	Buchanan	

Note

The report shown also uses the ?Access parameter field to enable the Can Grow option (also on the Common tab of the Format Editor) and increase the font size for people with visual impairments.

When the user chooses **No** for the ?Access parameter field, the conditional formula suppresses the text objects, leaving spaces in the report in place of the text objects.

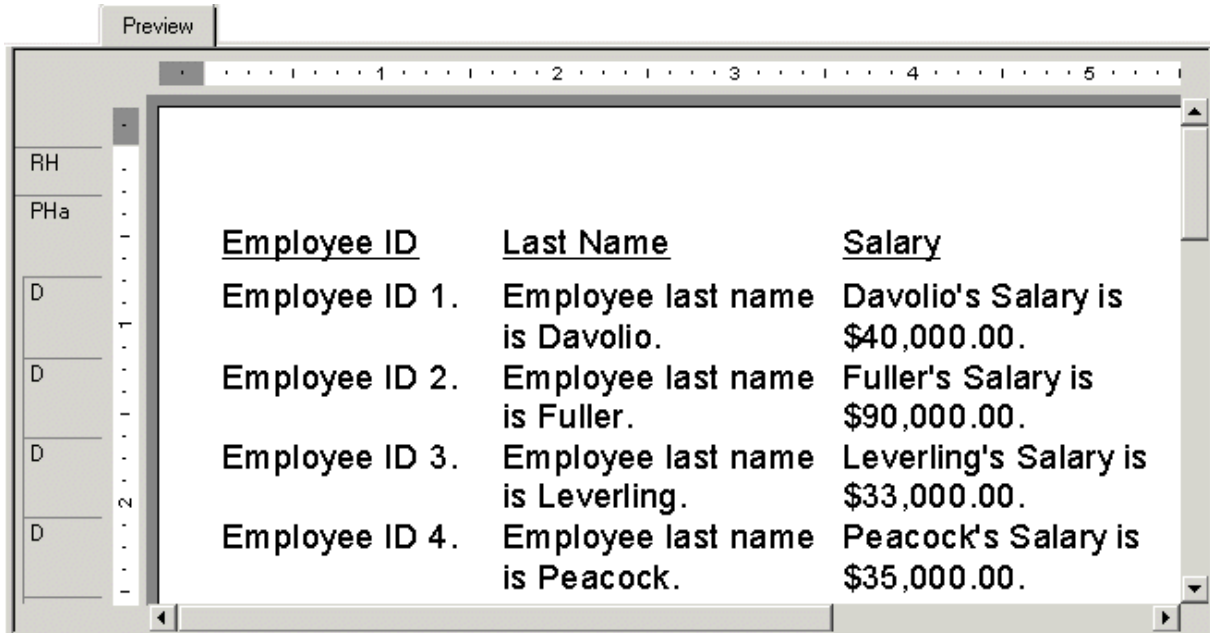
32.4.1.3 Labeling data tables with formulas

Another method for adding explanatory text to a data table is to create formulas that combine text, database fields, and conditional formatting. By adding the text and the database fields together in a conditional formula based on the ?Access parameter, you can provide optional text for values in a table without leaving blank spaces in the report. Using formulas also reduces the number of objects on the report, making it easier to maintain the proper placement order.

Note

Do not use this method if the report has summary fields or calculated fields. Although formulas provide the best display of data, they can interfere with calculations because the data is converted to text.

The following report uses formulas placed in the Details section that combine the database fields and the extra text. When the user chooses **Yes** for the ?Access parameter field, each formula builds a string that includes the description and the value.



Employee ID	Last Name	Salary
Employee ID 1.	Employee last name is Davolio.	Davolio's Salary is \$40,000.00.
Employee ID 2.	Employee last name is Fuller.	Fuller's Salary is \$90,000.00.
Employee ID 3.	Employee last name is Leverling.	Leverling's Salary is \$33,000.00.
Employee ID 4.	Employee last name is Peacock.	Peacock's Salary is \$35,000.00.

This report uses the following formulas:

@Employee ID

```
If {?Access}="Yes" then "Employee ID "  
+ ToText({Employee.Employee ID},0) + ". "  
else ToText({Employee.Employee ID},0)
```

@Last Name

```
If {?Access}="Yes" then "Employee last name is "  
+ {Employee.Last Name} + ". "  
else {Employee.Last Name}
```

@Salary

```
If {?Access}="Yes" then {Employee.Last Name} + "'s Salary is " +  
ToText({Employee.Salary}) + ". "  
else ToText({Employee.Salary})
```

Notice the added punctuation. The periods at the end of each formula improve screen reader legibility by creating a pause between fields.

Note

The report also uses the ?Access parameter field to enable the Can Grow option and increase the font size.

Note

In @Employee ID, ?Access parameter field has been set to "0" to enable the Can Grow option and increase the font size.

When the user chooses **No** for the ?Access parameter field, the formula returns only the data. The report does not display blank spaces in place of the conditional text objects. Both versions of the report are easy to read.

The screenshot shows a SAP Crystal Reports preview window. The window has a 'Preview' tab at the top. On the left side, there is a vertical pane with a tree view showing the report structure: 'RH' (Report Header), 'PHb' (Page Header), and 'RF' (Report Footer). The main area displays a table with three columns: 'Employee ID', 'Last Name', and 'Salary'. The table contains 15 rows of data. The 'Employee ID' column is underlined. The 'Salary' column is formatted with dollar signs and commas. The table is surrounded by a scroll bar on the right and a status bar at the bottom.

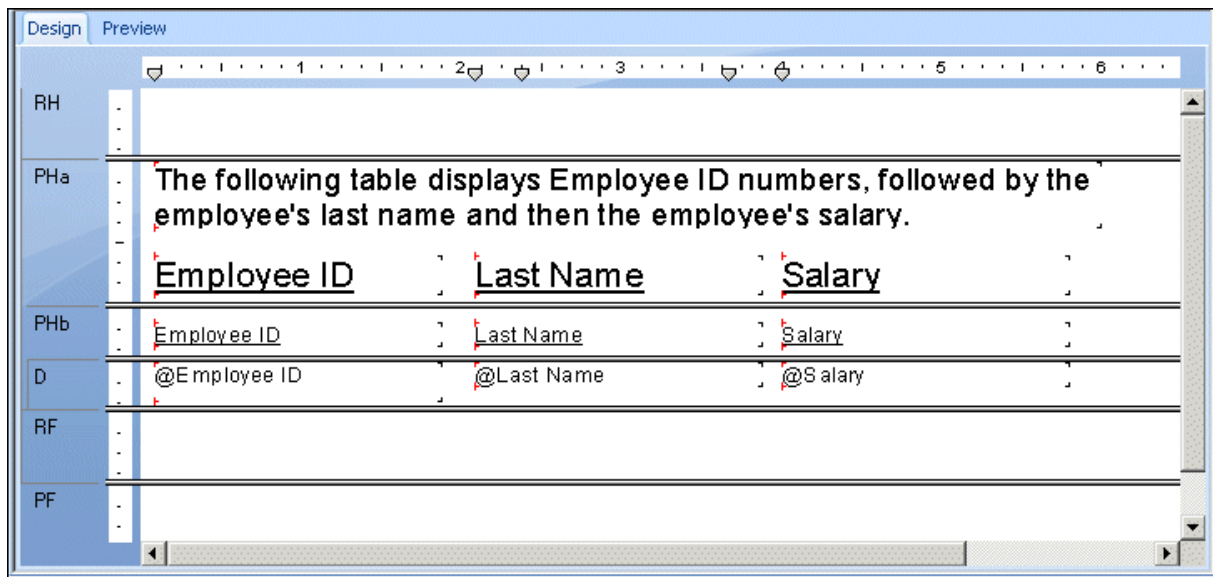
<u>Employee ID</u>	<u>Last Name</u>	<u>Salary</u>
1	Davolio	\$40,000.00
2	Fuller	\$90,000.00
3	Leverling	\$33,000.00
4	Peacock	\$35,000.00
5	Buchanan	\$50,000.00
6	Suyama	\$30,000.00
7	King	\$37,000.00
8	Callahan	\$45,000.00
9	Dodsworth	\$35,000.00
10	Hellstern	\$60,000.00
11	Smith	\$18,000.00
12	Patterson	\$25,000.00
13	Brid	\$75,000.00
14	Martin	\$50,000.00
15	Pereira	\$45,000.00

32.4.2 Other data table design considerations

In addition to labeling data values with text objects, other report design techniques can help you create data tables that are easier to understand and navigate.

- Include an introductory paragraph that summarizes the content of the table. The summary should be brief: one or two sentences if possible.
- Ensure that headings provide enough information to clearly identify the values that they label.
- To test a table's accessibility, read its headings and values in a linear fashion from left to right and from top to bottom. For example, if a report displays last and first name fields for each customer, it may read better if it displays first name followed by last name. Whenever possible, test the report using assistive technologies such as screen reading software.

The final accessible report includes a summary of the data table.



To display the table summary conditionally, the report designer divided the Page Header into two sections. The first page header is suppressed when the ?Access parameter field is set to **No**. The second page header is suppressed if the user chooses **Yes**. For details, see [Accessibility and suppressing sections \[page 1027\]](#).

32.5 Accessibility and BusinessObjects Business Intelligence platform

Designing accessible reports is only part of the solution. You need to make sure that you deliver reports through an accessible interface that follows the same design guidelines.

Although the administrative components of SAP BusinessObjects Business Intelligence platform and the scheduling functionality in the BI platform Server web desktop are not currently accessible to everyone, the SAP BusinessObjects Business Intelligence platform web desktop and the DHTML viewer allow for accessible access to reports over the Web.

Several enhancements have been made to the BI platform Server to account for accessibility issues. Text descriptions are now provided in ALT tags for the toolbar buttons and other images. Descriptions for text boxes are clearer, and shortcut links are provided in the DHTML viewer so you can navigate past the toolbar and group tree.

32.5.1 Setting accessible preferences for SAP BusinessObjects Business Intelligence platform

For the best accessibility support in SAP BusinessObjects Business Intelligence platform, you need to set certain display preferences.

For the BI platform Server web desktop, display objects in the Action view. The Action view is more accessible because it provides a text list of the available reports and does not use shortcut menus for report commands. Depending on your users' needs, you may also want to reduce the number of reports displayed on each page.

For viewing reports, choose the DHTML viewer as the default viewer in your preferences.

Note

You must have your own account on the system in order to set preferences.

32.5.1.1 To set accessible preferences for SAP BusinessObjects Business Intelligence platform

Procedure

1. Log on to SAP BusinessObjects Business Intelligence platform.
2. On the title bar, click [Preferences](#).
3. On the General Preferences page, in the "On my desktop, show me" area, select [Action view](#).
4. To reduce the number of reports displayed on each page, type a number in the text box next to the Action view option.
5. Click the Crystal Report Preferences link.
6. In the "View my reports using the" area, select the [DHTML viewer](#).
7. Click [Apply](#).

32.6 Accessibility and customization

When you customize Crystal reports or the SAP BusinessObjects Business Intelligence platform web desktop, or if you incorporate the BI platform Server into an existing web site, ensure that your changes follow the accessibility guidelines set forth by the U.S. Access Board in section 508, or the W3C's Web Accessibility Initiative.

If you customize Crystal reports or the SAP BusinessObjects Business Intelligence platform web desktop extensively, you may encounter other accessibility issues. For online resources that provide comprehensive accessibility guidelines, see [Resources \[page 1036\]](#). The following list provides some common accessibility issues that may cause problems when you customize Crystal Reports or SAP BusinessObjects Business Intelligence platform content.

- **Frames**
Frames should be clearly labeled, for easier identification and navigation. Provide text at the top of the frame that describes its purpose. For example, if a frame provides a list of links to different countries, you can clarify its purpose clear by adding text to the frame, such as a title ("Countries") or short instructions ("Click a country for details").
- **Style sheets**

If you have a visual impairment, you can create a style sheet with specific viewing preferences to accommodate the disability. For example, you could create a style sheet that displays all web pages in a large font with white characters on a black background. Users cannot apply personalized style sheets to Crystal reports, but the viewers provide a Zoom button that enables people with visual impairments to increase the magnification to suit their needs. You can also allow users to choose from different formatting options using conditional formatting. For details, see [Accessibility and conditional formatting \[page 1026\]](#).

- **Scripts**
If you modify Crystal content to include a script that displays content or an interactive object, ensure that the script is identified by text that conveys the purpose of the script. Make sure that pages with scripts are still usable when the scripts are turned off or unsupported. For more information about scripts and accessibility, see [Resources \[page 1036\]](#).
- **Image maps**
Server-side image maps identify active regions using coordinates, which are not meaningful to a screen reader. Client-side image maps provide better accessibility because you can assign a link or URL to each active region within the image map.
- **Electronic forms**
Electronic forms can present difficulties for screen readers, and must be set up carefully. When you label a component in a form, ensure the label is clearly located next to the form component. For example, for a Search box, ensure that the "Search" title appears alongside the appropriate text box.
- **Applets and plug-ins**
If a report needs an applet, plug-in, or other application on the client machine in order to interpret page content, the plug-in or applet must follow accessibility guidelines.
If you attach multimedia or other additional resource files to your report, such as PDF or Real Audio files, provide a link to install the required plug-ins or software, and ensure that the required software also meets accessibility design standards.
- **Flickering**
Flickering images can trigger seizures for people with seizure disorders. The W3C recommends to avoid use of images that flicker or flash between four and 59 times per second.
- **Search engine placement**
Do not use hidden text to enhance your web site's placement in search engines. Hidden text reduces readability, because it is read by the screen readers. Also, hidden text is actively discouraged by popular search engines such as Google, and thus offers little benefit.

32.7 Resources

This section focuses on how you can create and distribute accessible reports with Crystal software. The report design techniques in the section were tested using JAWS 4.5. It is good practice to test all accessible reports using JAWS and other assistive technologies whenever possible.

To make all of your Web communications accessible, consult the detailed guidelines available through the W3C or from your government's web site.

- World Wide Web Consortium's Web Accessibility Initiative:
<http://www.w3.org/WAI/>
- the United States Access Board's web site for Section 508:
<http://www.access-board.gov/sec508/guide/>

- the Government of Canada Internet Guide:
<http://www.tbs-sct.gc.ca/clf2-nsi2/index-eng.asp>

33 Accessing Data Sources

33.1 Introduction

Crystal Reports can access data stored in almost any common database format, as well as many uncommon formats. This section discusses the many different types of data that Crystal Reports can access, and explains the data access layers involved in connecting to the data. If you are not sure what Database Management System (DBMS) your company uses, contact your IT manager or your network administrator.

The principal purpose of Crystal Reports is simply to access data stored in databases and to produce reports on that data. This goal is one of the oldest uses of computers and remains one of the most common and most necessary. Crystal Reports is designed to make that task easier, less time-consuming, and more powerful.

This idea of accessing data remains at the root of every report produced. By understanding how Crystal Reports accesses data, you will gain a better knowledge of the reporting process, as well as a better knowledge of the type of data with which the program can work.

In addition, understanding the data access process will help you troubleshoot problems you may encounter while trying to open a particular database file. This is especially useful for IT managers, or for anyone who provides data access support for a group of users.

Most of the information in this section is designed for experienced Crystal Reports users and IT managers, and covers technical aspects of Database Management Systems (DBMS) and data storage techniques. A familiarity with computers, the Windows operating system, and at least one DBMS application is assumed.

33.1.1 Four types of data

The data that Crystal Reports can access falls into four general categories:

- [Direct access database files \[page 1039\]](#)
- [ODBC data sources \[page 1045\]](#)
- [OLE DB \[page 1051\]](#)
- [Business Views \[page 1055\]](#)

Each type of data must be accessed using a specific set of Dynamic Link Libraries (DLLs) and other data access-related files. Once you understand the process the program uses to access each type of data, you will have a better understanding of the report creation process and the elements used to create powerful reports with your data.

Note

When accessing any type of data, Crystal Reports relies on the database drivers to provide field names, field types, and field lengths. This information is provided by either the database engine or the ODBC database driver.

33.2 Direct access database files

Crystal Reports can access many of the most common PC database formats directly. In other words, the program has the built-in capabilities needed to directly open database files and tables designed in Microsoft Exchange server, Outlook files, Local file system, IIS log files, among others. This functionality exists as soon as you install Crystal Reports. Once the program is installed on your system, you can immediately begin creating reports based on these databases by selecting the appropriate file.

33.2.1 Advantages

Accessing the database directly is the fastest way to read the data. Crystal Reports only needs to talk to a single data access layer that provides contact with the data. Report results can be obtained quickly in almost any system environment.

In addition, data access is simple. Direct access database files are point-and-click data sources. You need only select the required database files, and Crystal Reports will read all the stored data.

33.2.2 Disadvantages

When you access a database directly through Crystal Reports, only that database type can be used by the report. You cannot switch to a different type of database or table without creating a new report.

If you access data through ODBC, on the other hand, the syntax used is always the SQL language, regardless of the actual database type. See [ODBC data sources \[page 1045\]](#).

33.2.3 Three layers

Direct access of database files from Crystal Reports requires three layers:

- [Crystal Reports \[page 1039\]](#)
- [Data translation \[page 1040\]](#)
- [Database \[page 1040\]](#)

Crystal Reports uses the data translation layer to talk to the database and access its data.

33.2.3.1 Crystal Reports

Crystal Reports operates as an interface through which you can format, arrange, select, and sort the data stored in database files. It obtains data by communicating with one or more files in the data translation layer that can actually read the database. Since Crystal Reports can work with many forms of data, it must

rely on other files in order to work directly with the data. Crystal Reports can then use a native method of communication to talk to the translation files.

33.2.3.2 Data translation

Data is translated through a set of DLLs specific to Crystal Reports. The program uses the DLLs specific to a certain data type to understand how data is organized for that type and to present it correctly when your report is printed, previewed, or exported.

📘 Note

Crystal Reports comes with all the data translation files for each of the direct access database types that it supports.

33.2.3.3 Database

The database file consists of one or more tables. Different DBMS applications store database information differently.

When Crystal Reports accesses a database file directly, it automatically retrieves information about all of the tables and fields in that file. You may not use all of the tables or fields, but the program will make them available to you.

📘 Note

Crystal Reports will also open queries in an Access database through ODBC and will allow you to report on query fields, as with table fields. See [Access \[page 1049\]](#).

33.2.4 Common database formats

Although Crystal Reports uses the same three-tiered system for obtaining data from all direct access database file formats, each format requires a different set of DLLs. However, some formats expand the basic three-tiered structure.

The following sections cover the systems used by Crystal Reports to access data from some of the most popular database formats.

33.2.4.1 Microsoft Access

Microsoft Access provides several means for opening its database files. Each method has its advantages and disadvantages, and the technique that you should use may depend on how your data is set up. You can use Microsoft's Open Database Connectivity (ODBC) standard, which is described in [Access \[page 1049\]](#).

33.2.4.2 Exchange

Exchange is Microsoft's successor to MS Mail. Exchange, however, includes not only e-mail, but also management of group scheduling, electronic forms, groupware, and Internet connectivity. An Exchange folder can contain standard notes (mail), files, and instances of Exchange forms. All of this data is stored in the Exchange Information Store.

Note

Exchange 2000 can be accessed through ADO.

Crystal Reports can report on data contained in the Exchange Information Store. Exchange data sources available for reporting include:

- Message Tracking Logs
- Address Lists
 - Personal Address Books
 - Global Address Lists
 - Distribution Lists
- Exchange Folder Contents
 - Mail messages
 - Exchange Form applications
 - Properties of OLE documents
- Exchange Administrator
 - Properties of Exchange mailboxes on the Exchange Server
 - Properties of public folders on the Exchange Server
 - Replica list of public folders
 - ACL (Access Control List) of public folders

You can use each Exchange data source like a database table and you can link each Exchange data source to other data sources. For example, you can join the Message Tracking Log to an Address List by using an e-mail address as the link field.

The data translation file used to access the Exchange data source depends on the data source that is being accessed. The following table lists each of the Exchange data translation files and describes their purpose:

File name	Data source
crdb_p2strack.dll	Exchange Server Message Tracking Logs
crdb_p2smapi.dll	Exchange Folders and Address Book
crdb_p2sexsrn.dll	Exchange Server Mailbox Admin
crdb_p2sexsrp.dll	Exchange Server Public Folders Admin
crdb_p2srepl.dll	Exchange Public Folder Replica
crdb_p2sacl.dll	Public Folders Access Control Lists (ACL)

File name	Data source
crdb_p2soutlk.dll	Physical Server DLL for Microsoft Outlook
p2soutlk.dll	

Exchange translation files work directly with the Microsoft Messaging API (MAPI). MAPI acts as a database engine for Exchange data.

How Crystal Reports treats Exchange data

Crystal Reports treats Exchange Administrator data (Public Folder Admin, Public Folder ACL, Public Folder Replica, and Mailbox Admin server types) as a physical database server. To access this data, you must log on using a SQL/ODBC server and select the appropriate tables from the Choose SQL Table dialog box.

When you log on to any of the Exchange Administrator servers, you must select a profile (or profiles). Each profile represents an Exchange Server.

Note

Refer to your Microsoft Exchange documentation for more information about how to set up and create Exchange profiles.

33.2.4.3 Outlook

Microsoft Outlook provides powerful organization and information management tools in the form of e-mail, calendars, task lists and other popular tools.

Crystal Reports lets you report on your Outlook data by providing the translation files crdb_p2soutlk.dll and p2soutlk.dll. Because Outlook data is stored in a flat file system, the translation file accesses it directly.

Note

You must have Microsoft Outlook installed, or available through UNC (universal naming convention) read-access. UNC is a standard format for paths that include a local area network file server. The basic syntax is:
`\\server\share\path\filename`

33.2.4.4 Oracle

Crystal Reports supports direct access to Oracle SQL databases. This accessibility is provided by the crdb_oracle.dll translation file. This file communicate with the Oracle database driver, which works directly with Oracle databases and clients, retrieving the data you need for your report.

Note

In order for Crystal Reports to use Oracle SQL databases, the Oracle client software must be installed on your system, and the location of the Oracle client must be in the PATH statement of your Autoexec.bat file.

Note

When logging on to Oracle using crdb_oracle.dll, the date format is changed to match the default date/time format of Crystal Reports.

33.2.4.5 Sybase Adaptive Server

Crystal Reports opens SQL data created by Sybase Adaptive Server directly through the crdb_p2ssyb10.dll, installed with Crystal Reports. This translation file works with the Sybase database drivers to read Sybase Adaptive Server data. If your Sybase server is correctly configured, you will be able to read Sybase data as soon as Crystal Reports is installed.

33.2.4.6 DB2 Server

IBM's DB2 Server is an SQL compliant, client/server application that is part of IBM's popular database management suite.

Crystal Reports opens SQL data created by DB2 through the crdb_p2sdb2.dll, installed with Crystal Reports. This translation file works with the DB2 database drivers to read DB2 data. You must have the DB2 client installed and configured properly before using this translation file.

33.2.4.7 Active Data

Crystal Reports can be used to report off ActiveX data sources, including ActiveX Data Objects (ADO), and Remote Data Objects (RDO). Reports created off these data sources can be used in Visual Basic applications that use these ActiveX data sources.

Crystal Reports also offers the ability to create reports without specifying an actual data source. This is supported through the use of a Data Definition file, which is an ASCII text file with placeholders to represent database fields. By using reports created off a Data Definition File, an application developer has the flexibility of specifying the actual data source at runtime.

Crystal Reports provides these translation files for Active Data reporting:

File	Description
crdb_fielddef.dll	For Microsoft ActiveX Data Objects

File	Description
crdb_ado.dll	For Microsoft ActiveX Data Objects
crdb_odbc.dll	For ODBC

These translation files works with a different set of drivers for each data source.

33.2.4.8 Local file system

You can use Crystal Reports to report on files located on your system or network drives including information such as file name, version number, date, and so on.

The translation file `crdb_filesystem.dll` is provided by Crystal Reports for reporting on your local file system.

Note

You must have UNC (universal naming convention) read-access to your local file system. UNC is a standard format for paths that include a local area network file server. The basic syntax is: `\server\share\path\filename`

33.2.4.9 NT Event Log

If using Windows NT version 3.51 or later, you can use Crystal Reports to report on the NT Event Log. The Event Log is a database used by network administrators to record and keep track of different types of events that can occur on a Windows NT Workstation or on a Windows NT server.

Crystal Reports provides the translation files `crdb_p2sevt.dll` (archived log) and `crdb_p2sevtc` (current log) for working with NT Event Log data. These files communicate with the Event Logging API in `Advapi32.dll`, a part of the Windows NT operating system.

33.2.4.10 Microsoft IIS/Proxy log file

If you use Microsoft Internet Information Server (MS IIS), or Microsoft Proxy, you can use Crystal Reports to report on the log files. These log files keep track of different types of events that occur when using a web server and browser.

The translation file Crystal Reports provides for MS IIS and Proxy log files is `crdb_p2smsiis.dll`. Because log file data is normally stored in a flat file system, the translation file accesses it directly.

Note

You must have UNC (universal naming convention) read-access to your local file system. UNC is a standard format for paths that include a local area network file server. The basic syntax is: `\server\share\path\filename`

33.2.4.11 Web/IIS log files

You can use Crystal Reports to report on Web log files. These log files keep track of different types of events that occur when using a web server and browser.

The translation file Crystal Reports provides for Web/IIS log files is `crdb_p2swblg.dll`. Because log file data is normally stored in a flat file system, the translation file accesses it directly.

Note

You must have UNC (universal naming convention) read-access to your local file system. UNC is a standard format for paths that include a local area network file server. The basic syntax is: `\server\share\path\filename`.

33.3 ODBC data sources

Open Database Connectivity (ODBC) is a standard developed by the Microsoft Corporation through which many different types of data can be accessed by a single application. An application need only communicate with one set of files (ODBC) to be able to work with any source of data that can be accessed by ODBC.

There are hundreds of Database Management Systems (DBMS) available for personal computers, and thousands of applications that access DBMS data. Normally, a company that designs an application that accesses data, such as Crystal Reports, must develop a means for the application to communicate with each type of data that a customer might want to use. Crystal Reports does this with the databases that it can access directly.

On the other hand, if a DBMS simply provides a means by which ODBC can access its data, the DBMS data becomes an ODBC data source. Any application that can communicate with ODBC (such as Crystal Reports), will instantly have access to any ODBC data source. With ODBC drivers available for most common DBMS products, the range of data types that Crystal Reports can use is almost unlimited.

33.3.1 Advantages

Perhaps the biggest advantage to accessing data through ODBC is the ability to access a wide range of data with just one interface. Since most popular Database Management Systems now offer ODBC drivers, with more appearing every day, Crystal Reports can use any type of data you have.

Because of the extreme flexibility built into ODBC, you can use the same report file with different ODBC data sources. For example, you might design a report using an Oracle data source, and later, if your company switches to Microsoft SQL Server, you can simply change the ODBC data source used by your report. The only requirement is that the new data source must have the same structure (tables and fields) that the original data source had (although table names can be different). For more information, search for the topic called "Changing the data source accessed by a report" in the *Crystal Reports Online Help*.

Experienced SQL (Structured Query Language) programmers also benefit from the ODBC standard. Since Crystal Reports uses SQL to communicate with ODBC, SQL programmers and Database Administrators can view and edit the SQL statements sent to ODBC, controlling exactly how data is retrieved from the data source.

Finally, by using SQL pass-through technology to send an SQL statement to ODBC and retrieve an initial set of data, Crystal Reports off-loads much of the data retrieval and sorting work on to the server system, freeing up local memory and resources for more important tasks. In addition, only the data specified by the SQL statement is returned to Crystal Reports, reducing network traffic and the use of network resources. By working more efficiently with the original data, Crystal Reports saves you time and effort, and lets you concentrate on the design process and other more important work.

33.3.2 Disadvantages

There are many layers involved in passing data through ODBC from a database to an application. First, Crystal Reports must request some data. The request must be translated by the ODBC translation layer into a format that ODBC understands (an SQL statement). ODBC must determine where the requested data exists, and pass the request on to the ODBC data source. For more information, see [DBMS translation \(ODBC data source\) layer \[page 1047\]](#). The data source must analyze the request and translate it again into a format that can be understood by the DBMS. This complex process can fail at any of several possible levels.

In addition, ODBC data sources must be correctly configured and set up in the Odbc.ini and Odbcinst.ini files before they can be used. If you create a report on one system and try to open it on another system that does not have the same ODBC data source set up, Crystal Reports will not be able to connect to the data.

When working with ODBC, you should also be aware that the SQL language used by ODBC is based on the standards set for the SQL language by the American National Standards Institute (ANSI). Some SQL-based DBMS applications, however, provide additional features to the SQL language that are specific to that DBMS. If your data uses features unique to your DBMS, ODBC will not be able to translate those features (though in many cases it will still retrieve most of the data). See [The SQL language \[page 894\]](#).

33.3.3 Five layers

The process by which Crystal Reports accesses data from an ODBC data source consists of five layers:

- [Crystal Reports layer \[page 1046\]](#)
- [ODBC translation layer \[page 1047\]](#)
- [ODBC layer \[page 1047\]](#)
- [DBMS translation \(ODBC data source\) layer \[page 1047\]](#)
- [Database layer \[page 1048\]](#)

By using the Structured Query Language (SQL), all five layers can conveniently pass data from the database to your report.

33.3.3.1 Crystal Reports layer

When working with ODBC data, Crystal Reports generates an SQL statement that requests the appropriate data from ODBC. The powerful SQL generator built into Crystal Reports is designed to create an SQL

statement that will let the ODBC data source or ODBC itself do as much of the report generation as possible, returning only the data needed to produce the report.

33.3.3.2 ODBC translation layer

Crystal Reports uses the Dynamic Link Library `crdb_odbc.dll` to communicate with ODBC. This file is unique to Crystal Reports environment and provides your report with access to any ODBC data source. This is the driver that actually passes data to and from ODBC.

33.3.3.3 ODBC layer

ODBC is a set of several DLLs and INI files built into the Windows environment that act as a gateway through which database requests and data can pass. Any database file or format that is to be used via ODBC must be set up as an ODBC data source.

Note

For more information on ODBC and the ODBC files, see the Microsoft ODBC documentation.

ODBC uses the SQL language for all transactions between Crystal Reports and ODBC. Even if the database does not normally use SQL to create and work with tables, the ODBC driver provided by the database (the DBMS translation layer) must communicate with ODBC using SQL. For most users, this feature of ODBC is transparent, but more advanced users often take advantage of the features of the SQL language used by ODBC.

33.3.3.4 DBMS translation (ODBC data source) layer

This layer consists of one or more drivers provided by a DBMS that allow ODBC to communicate with the database. Crystal Reports ships with several ODBC drivers for many of the most common database formats. If you are unsure whether you can use an ODBC driver to access the data in your database, refer to the documentation for your DBMS application. Most DBMS applications that run on a Windows-based platform offer an ODBC driver.

When a DBMS provides an ODBC driver, it must register the driver with ODBC on the machine on which it has been installed. It does this by assigning a name to the driver and recording the filename in the `Odbcinst.ini` file. Usually, this step is handled automatically when the DBMS application is installed on the system. However, your network system or DBMS application may require that you register the ODBC driver manually with the ODBC Setup application. For more information about registering an ODBC driver with ODBC, refer to the documentation for your DBMS.

Once an ODBC driver is registered, you need to establish an ODBC data source using that driver. The ODBC data source is the object that you connect to when accessing data from Crystal Reports through ODBC. Data sources are recorded in the `Odbc.ini` file. The data source keeps track of the DBMS translation files (ODBC drivers) and, sometimes, the database itself. An ODBC data source can specify just a database format, such as

Oracle, Sybase SQL, or MS SQL Server. Some users, however, prefer to actually specify a particular database. In this case, the ODBC data source extends across both the DBMS translation layer, and the database layer.

If you are using a client/server database, such as an SQL server, the ODBC drivers communicate with the database server through the Database Communication layer, the same layer that your database client uses to communicate with the database server.

Note

- ODBC drivers find their specific DBMS client files on the local machine mainly through key directories that the DBMS client has installed in the search path. The important thing to remember is that a workstation client on a local PC must be able to connect to its server successfully. If you are not sure how to verify this, contact your IT manager.
- Crystal Reports 2025 is a 64bit application. Make sure database client and ODBC driver are both 64bit.

For Crystal Reports and the ODBC drivers, it does not matter what kind of platform your database server exists on. It is the DBMS client that connects and communicates with the server; Crystal Reports and the drivers need only communicate with the DBMS client. For more information, see [Using SQL and SQL databases \[page 888\]](#).

33.3.3.5 Database layer

The database file which the ODBC data source refers to can be located anywhere on a system. Once the ODBC data source is set up, Crystal Reports does not need to know the actual location and format of the data. Thus, the database can have any format and can be located anywhere on a network, as long as ODBC can communicate with it through the ODBC data source.

33.3.4 Installing database clients

Ideally, your database client should be installed and configured before you install Crystal Reports. If, however, you've installed Crystal Reports first, you can ensure the proper configuration of the Crystal Reports ODBC Data Source Name (DSN) by following this procedure.

Crystal Reports 2025 is a 64bit application and requires the 64bit database client and 64bit ODBC driver to be installed and configured. For information on installation and configuring Database Client, see your database documentation or your database administrator.

33.3.4.1 To ensure correct ODBC DSN configuration

Procedure

1. On the [Start](#) menu, select [Settings](#), select [Control Panel](#), then double-click [Add or Remove Programs](#).
2. Select [Crystal Reports](#) and click [Change](#).

3. In the Crystal Reports Setup dialog box, click [Add/Remove](#).
4. Expand the [Data Access](#) folder and select the appropriate data access option.

For example, if you have installed the Informix database client, select Informix from the Data Access folder.
5. Click the selected data access option and choose [Entire feature will be unavailable](#).

An X appears next to the option and the Feature Description area includes a remark that the feature will be completely removed.
6. Click [Next](#) on this dialog box and the following one.

The data access option you selected is removed.
7. Repeat steps 2 through 4.
8. Click the selected data access option and choose [Will be installed on local hard drive](#).

The X next to the option is removed and the Feature Description area includes a remark that the feature will be installed.
9. Click [Next](#) on this dialog box and the following one.

The data access option you selected is reinstalled and its ODBC DSN is configured properly.

33.3.5 Common ODBC database formats

33.3.5.1 Access

Although Microsoft Access files can be opened directly by Crystal Reports (see [Microsoft Access \[page 1040\]](#)), you may wish to use ODBC for these files instead. When communicating with Access databases, ODBC uses a translation file that communicates with the Microsoft Jet Database Engine. The Jet Database Engine is an example of a DBMS-specific ODBC driver.

The Jet engine is the part of the Access DBMS that does all of the actual work with an Access database. Jet is a required component for working with Access data. Since the Jet engine is an integral part of all Access databases, it is shown here as part of the database layer.

33.3.5.2 Excel

You can convert Microsoft Excel spreadsheets into databases that can be read by Crystal Reports through ODBC. In Excel 4.0 and earlier, use the Set Database command on the Data menu. In Excel 5.0 and later, use Define on the Name submenu of the Insert menu. Once converted, spreadsheet rows become records, and spreadsheet columns become fields. (For more information on converting your spreadsheets to database format, refer to your Excel documentation). Once the spreadsheet is converted, you can set up an ODBC data source for the file, and then select it from Crystal Reports.

Note

If you are using Excel 7 or later, you can export your spreadsheets as Access database tables, and read them from Crystal Reports as you would read any other Access tables. For more information, refer to your Excel documentation.

33.3.5.3 Informix

Crystal Reports accesses Informix databases through ODBC. The Informix client should be installed on your machine. Otherwise, Crystal Reports provides the DataDirect driver that ODBC uses to communicate with the Informix database engine.

ⓘ Note

If your report uses a DataDirect DSN connection and you want to publish the report to a BI platform installation on Unix, verify that the BI platform administrator has added a DSN entry into the `odbc.ini` file. For more information about configuring BI platform to use DataDirect connections, see the *SAP BusinessObjects Business Intelligence Platform Installation Guide for Unix*.

33.3.5.4 SQL databases through DataDirect

There are many different SQL Database Management Systems available on the market, and most of them can be accessed through ODBC. Crystal Reports provides several DataDirect ODBC drivers, allowing you to access many of the most popular SQL databases, including:

- Informix
- Oracle
- Sybase
- Microsoft SQL Server
- DB2

Crystal Reports still communicates with ODBC through the `crdb_odbc.dll` translation file, but the DataDirect drivers provide ODBC with easy access to the actual databases.

ⓘ Note

If you are using an SQL database not accessed by the DataDirect drivers, refer to your DBMS documentation for more information about the required ODBC drivers.

Do not confuse SQL databases with the SQL language. Structured Query Language (SQL) is a specialized computer language used to design, build, and read database files. See [Using SQL and SQL databases \[page 888\]](#).

SQL databases are any collection of computer applications that depend on the SQL language for database creation and manipulation. SQL DBMS applications are usually designed to run as large client/server applications spread across a network.

To open an SQL database, ODBC uses a set of DataDirect ODBC support drivers (these are the same for every SQL database format), a DataDirect ODBC driver specific to the database format, and the Database Communication layer that actually communicates directly to the database file. The files provided for this DBMS translation layer are all DataDirect drivers.

ⓘ Note

ODBC does not have to use the DataDirect drivers to access the SQL databases shown here. Most SQL DBMS applications provide their own ODBC drivers.

Note

If your report uses a DataDirect DSN connection and you want to publish the report to a BI platform installation on Unix, verify that the BI platform administrator has added a DSN entry into the `odbc.ini` file. For more information about configuring BI platform to use DataDirect connections, see the *SAP BusinessObjects Business Intelligence Platform Installation Guide for Unix*.

33.3.5.5 OLE DB

OLE DB is a database connectivity entity from Microsoft similar in conception to ODBC.

Note

OLE DB is a connectivity methodology and not a database. Like ODBC, it facilitates the exchange of data created in other database management systems by providing an interface layer.

Note

OLE DB requires the use of an OLE DB provider, which is the OLE DB equivalent of an ODBC driver.

Note

There are two ways to connect using an OLE DB provider. You can connect using a username and password, and depending on the provider, you may also be able to connect using *Integrated Security*. No user ID or password is required with *Integrated Security*, but it does require that Database Authentication is enabled on the server. Please consult your Database Administrator on how to enable and configure this function.

33.4 JDBC data sources

Java Database Connectivity (JDBC) is a Java API developed by Sun Microsystems that acts as an interface between a developer's Java code and a database. JDBC provides a mechanism for the developer to use to connect to a specified database, request information about the database, and then select information from it.

JDBC is similar to Microsoft's Open Database Connectivity (ODBC) and is fast becoming the standard for database-independent connectivity between Java applications and SQL-based databases.

JDBC shares many of the advantages of ODBC, but it has a unique advantage in its ability to provide platform-independent access to any database that has a JDBC driver available to it.

Note

The Crystal Reports JDBC driver requires version 1.4 of the JDK.

33.4.1 JNDI

The Java Naming and Directory Interface (JNDI) provides an interface that you can use to seamlessly connect to diverse naming and directory services within your company. Most importantly for reporting purposes, JNDI contains a repository for holding JDBC connection strings; when you provide these predefined connection strings through JNDI, your users will not be called on to provide connection URLs and class names when connecting to a data source.

JNDI can also be used to employ the more advanced connection-pooling type connections to a database. In either case, JNDI returns a valid connection to your database.

Note

Check the `Platforms.txt` file in the Crystal Reports product distribution for a list of supported JNDI servers.

33.4.2 Five layers

The process by which Crystal Reports accesses data from a JDBC data source consists of five layers.

- [Crystal Reports layer \[page 1052\]](#)
- [JDBC translation layer \[page 1052\]](#)
- [JDBC layer \[page 1052\]](#)
- [DBMS translation \(JDBC data source\) layer \[page 1053\]](#)
- [Database layer \[page 1053\]](#)

33.4.2.1 Crystal Reports layer

The Crystal Reports layer is database-driver independent, but it is used to drive all of the remaining layers.

33.4.2.2 JDBC translation layer

Crystal Reports uses the Dynamic Link Library `crdb_jdbc.dll` to communicate with JDBC. This file is unique to Crystal Reports environment and provides your report with access to any JDBC data source. This is the driver that actually passes data to and from JDBC.

33.4.2.3 JDBC layer

JDBC acts as a gateway through which database requests and data can pass.

Note

For more information on JDBC and the JDBC files, see the Sun Microsystems web site.

JDBC uses the SQL language for all transactions between Crystal Reports and JDBC. Even if the database does not normally use SQL to create and work with tables, the JDBC driver provided by the database (the DBMS translation layer) must communicate with JDBC using SQL. For most users, this feature of JDBC is transparent, but more advanced users often take advantage of the features of the SQL language used by JDBC.

33.4.2.4 DBMS translation (JDBC data source) layer

This layer consists of one or more drivers provided by a DBMS that allow JDBC to communicate with the database. If you are unsure whether you can use an JDBC driver to access the data in your database, refer to the documentation for your DBMS application.

33.4.2.5 Database layer

The database file which the JDBC data source refers to can be located anywhere on a system. Once the JDBC data source is set up, Crystal Reports does not need to know the actual location and format of the data. Thus, the database can have any format and can be located anywhere on a network, as long as JDBC can communicate with it through the JDBC data source.

33.4.3 Supported JDBC database formats

There are many different SQL Database Management Systems available on the market, and many of them can be accessed through JDBC. Crystal Reports communicates with JDBC through the `crdb_jdbc.dll` translation file, which has been designed to work with any fully JDBC-compliant driver. Crystal Reports officially supports third-party JDBC-compliant database drivers provided with:

- IBM DB2
- Oracle
- Microsoft SQL Server
- BEA LiquidData for WebLogic

Note

Check the Release Notes in the Crystal Reports product distribution for specific version requirements and for updates to this list; more database drivers may be added as testing progresses.

33.4.4 Configuring the Crystal Reports JDBC driver

When you install Crystal Reports, the following Crystal Reports JDBC driver files are added to your system:

- In the \Program Files (x86)\SAP BusinessObjects\SAP BusinessObjects Enterprise XI 4.0\win64_x64 directory:
 - crdb_jdbc.dll
 - crdb_jdbc_res_en.dll
- In the \Program Files (x86)\SAP BusinessObjects\SAP BusinessObjects Enterprise XI 4.0\java directory:
 - CRConfig.xml
- In the \Program Files (x86)\SAP BusinessObjects\SAP BusinessObjects Enterprise XI 4.0\java\lib directory:
 - CRDBJavaServer.jar

33.4.4.1 CRConfig.xml configuration for JDBC/JNDI

After installing the Crystal Reports JDBC driver, you must configure the CRConfig.xml file before the driver will work. See the CRConfig.xml Tag Reference in the *Crystal Reports Online Help* for a complete list of tags. The main tags that you must configure are **<JavaDir, Classpath>** , and **<IORFileLocation>** .

33.5 Business Objects universes

You can create a Crystal report from a Business Objects universe. Crystal Reports lets you select a universe and design a query from it using a tool called the Business Objects Query Panel. This query forms the basis of a SQL statement that is saved as a Crystal SQL Command object. For more information about SQL Command objects, search for the topic called "Defining an SQL Command" in the *Crystal Reports Online Help*.

ⓘ Note

When you report off a universe in Crystal Reports, the query that is produced is based on the structure of the database the universe references as it was when you designed the report. If the universe changes, you must return to the Query Panel to propagate the changes in the report. See [Editing an existing query \[page 381\]](#) for more information.

ⓘ Note

If the universe contains security that is created and managed through BusinessObjects Supervisor, the rights applied to the current report are always the rights of the person designing the report.

ⓘ Note

When you report off a universe in Crystal Reports, Interactive Analysis server groups are not supported (PServer, JServer groups are supported).

This version of the Query Panel in Crystal Reports supports ODBC database connections (including Sybase and Informix ODBC) as well as OLE DB, DB2 native, and Oracle native connections.

To learn how to begin reporting off a universe in Crystal Reports, see [Connecting to a universe \[page 376\]](#).

33.6 Managed OLAP connections

You can create a Crystal report from a managed OLAP connection stored in the SAP Business Objects Business Intelligence Platform. For example, Crystal Reports lets you select a managed OLAP connection on BW, select a BW query, and then design a DSL query from it using a tool called the Business Objects Query Panel.

To learn how to begin reporting off a managed OLAP connection in Crystal Reports, see [Connecting to a managed OLAP connection \[page 377\]](#).

33.7 Business Views

Business Views are collections of components created in Business View Manager—a flexible and reliable multi-tier system. Business View Manager lets companies build detailed and specific Business Views that help report designers and end users access the information they require.

Using Business Views, you can integrate data from disparate sources. You can also bring together data from multiple data collection platforms and application boundaries so that the differences in data resolution, coverage, and structure between collection methods are eliminated.

Business View Manager, a thick-client application, enables administrators to create and modify Data Connections, Dynamic Data Connections, Data Foundations, Business Elements, and Business Views.

Note

A Business View is defined as the following collection of components:

- Data Connections
- Dynamic Data Connections
- Data Foundations
- Business Elements

Users can access Business Views through products such as Crystal Reports and the Report Application Server.

For detailed information about Business Views and how to create them, see the *Business Views Administrator's Guide*.

Note

When you save a report that is based on a Business View, the Save As dialog box contains an option called "Disconnect view security." Selecting this option disconnects the report from the Central Management Server (CMS) so that users are not prompted for logon when they open the report. Once a report is disconnected from the CMS, you cannot reconnect to BusinessObjects Business Intelligence platform and

its security features, nor can you refresh the report's data. This is a useful option for times when you might want to share data that is normally secured, but you don't want those who receive the report to be able to refresh the data later.

Note

It is the responsibility of the System Administrator, or the report designer who has Administrator rights, to save reports based on Business Views to a secure location. If the Administrator creates a report based on a Business View that has row- or column-level security, saving the report with saved data to a non-secure location and then disconnecting from SAP BusinessObjects Business Intelligence platform makes the report and all of its data available to any user.

34 Report Processing Model

This section provides you with in-depth information about the Report Processing model. This model determines the order in which data is accessed and manipulated during report generation.

34.1 Overview

Crystal Reports uses a three-pass reporting method to generate reports. The sections below describe what happens during each step of this process. To see a visual representation, refer to the flow-chart at the end of this section.

34.1.1 What is a "pass"?

A pass is a process that Crystal Reports uses each time the data is read or manipulated. Depending on the complexity of the report Crystal Reports may make 1, 2, or 3 passes over the data. This feature allows for complex reporting and formula manipulation.

34.1.2 Pre-pass 1

When previewing a report, the first elements to be evaluated are "constant" formulas. Constant formulas are those that have a constant value for the entire report. They do not change from record to record. For example, $100 * 30$ would be a constant formula. Constant formulas are evaluated at the beginning of the print generation process and are never evaluated again. This process is known as "BeforeReadingRecords." If you were to place a constant formula field (i.e., $100 * 30$) in the Details section, the result would be 3000 for each record displayed.

34.1.3 Pass 1

After the "BeforeReadingRecords" process has taken place, Crystal Reports begins reading the database records. During the record reading process, the following will occur:

- Record retrieval. Where possible record selection and sorting are pushed down to the database in this step.
- Evaluation of recurring formulas. These formulas are those that contain database fields but do not contain references to subtotals or summary information. This evaluation time is known as "WhileReadingRecords." Formulas that contain references to subtotals or summary information are processed in the second pass.

- Application of the record selection locally. If the record selection is too complex to be pushed down to the database, it is applied by Crystal Reports in this step.
- Application of the saved data record selection formulas. When records return to the report, they are further filtered through any existing saved data formulas.
- Sorting, grouping, and totaling. In this step, Crystal Reports sorts the records, separates them into groups, and then calculates the subtotals and summaries needed for each group.
- Cross-Tab, chart, and map generation. Only Cross-Tabs, charts, and maps that are based entirely on database fields and recurring formulas are generated in Pass 1. If these objects include running totals and/or PrintTime formulas, they are generated in Pass 2.
- Storage of saved data. After the totaling process is complete, all of the records and totals are stored in memory and to temporary files. Crystal Reports does not read the database again, but instead uses this saved data during all subsequent processing.

34.1.4 Pre-pass 2

During Pre-Pass 2, Crystal Reports orders the groups in the report for Top/Bottom N or Hierarchical Grouping. The records are not read in this process, instead Crystal Reports only looks at group instances from Pass 1, and takes the Top N as appropriate, or orders the groups based on the Hierarchical Grouping settings specified.

34.1.5 Pass 2

Crystal Reports enters the second pass through the data to format pages. The pages are formatted on demand. This means that Crystal Reports will not format a page until it is requested by the user, or until it is required for the total page count in Pass 3.

During page formatting, Crystal Reports does the following:

- Group selection formulas.
- Running totals.
- Calculation of formulas marked "WhilePrinting Records."
These are formulas that contain references to subtotals or summary information, also known as "PrintTime" formulas. This evaluation time is known as "WhilePrinting Records."
- Cross-Tabs, charts, and maps.
Cross-Tabs, charts, and maps that include running totals and/or PrintTime formulas, and charts that are based on Cross-Tabs are generated in Pass 2.
- OLAP grids.
- Subreports.
- Generation of Pages on Demand.

Note

Subtotals, grand totals, and summaries may appear to be incorrect if the report has a group selection formula. This occurs because the grand totals and summaries are calculated during Pass 1, but the group selection formula filters the data again in Pass 2. Running total fields can be used instead of summaries to total data in reports with a group selection formula.

Note

You can use on-demand subreports to ensure that your main report remains a single-pass report. With on-demand subreports, Crystal Reports must still make a second pass through the data; however, this second pass does not begin until you drill down on the subreport. Consequently, you can increase the performance of reports that contain subreports by using on-demand subreports.

34.1.6 Pass 3

In the third, and final pass, the total page count is determined. This applies to reports that use the total page count, or Page N of M special fields.

35 Crystal Reports Error Messages

This section provides you with information and resolutions for some common error messages that you might encounter in Crystal Reports.

35.1 Drive:\filename.extension

This document could not be opened. It does not appear to be a Crystal Reports document.

Context

Occurs when attempting to open a Crystal report file that does not have an RPT extension.

Reason

This error occurs because the report has become corrupted.

Resolution

A backup copy of the report must be used in place of the corrupted copy.

35.2 Drive:\test.rpt

This document could not be opened. It does not appear to be a Crystal Report document.

The document has the expected extension (RPT), but it seems to be corrupted. If this report used to work, try opening it with a different version of Crystal Reports; if that does not work, contact your administrator.

Context

Occurs when attempting to open a Crystal report file that has an RPT extension.

Reason

This error occurs because the report has become corrupted.

Resolution

A backup copy of the report must be used in place of the corrupted copy.

35.3 Failed to load database connector

Reason

The database connector is not available on the machine that is processing the report.

The database client software required by the database connector is not available on the machine that is processing the report.

Context

Occurs when refreshing or previewing a Crystal report that does not contain saved data.

Resolution

Ensure that the database connector used by the report is installed on all machines where the report can be run.

Ensure that the necessary database client software required by the database connector is installed and in the PATH environment system variable on all machines where the report can be run.

35.4 Failed to open the connection. Details: [Database Vendor Code: <vendor code number>]

Reason

- The data source required by the report is unavailable or improperly configured on the machine that is processing the report.

- The database server required by the report is unavailable.
- The database client is installed but improperly configured on the machine that is processing the report. For example a DB2 database alias, Oracle *tnsnames* entry or Sybase *DSEDT* entry is missing or improperly configured.

Context

Occurs when refreshing or previewing a Crystal report that does not contain saved data.

Resolution

- Ensure that the data source required by the report is available and properly configured on all machines where the report can be run. For example, the ODBC data source or JNDI data source exists and connectivity can be proven from a client test tool.
- Ensure that the database server required by the report is running and that connectivity can be proven from a client test tool on all machines where the report can be run.

35.5 Failed to retrieve data from the database. Details: [Database Vendor Code: <vendor code number>]

Context

Occurs when refreshing or previewing a Crystal report that does not contain saved data.

Reason

The report SQL queries that have been sent to the server cannot be executed, because an error was returned by the database server after it processed the queries. For example, a field/table/procedure required by the report is missing, or the queries contain syntax errors.

Resolution

Ensure the report is using the correct connection information. For example, the data source name, server name, database name, and user name are correct.

In a database client test tool, ensure that the fields, tables, and procedures used by the report exist, and that SQL queries can be executed against them.

In the Crystal Reports designer, verify that the SQL query, as shown under the Database menu Show SQL Query option does not contain syntax errors.

If the SQL query shown in Crystal Reports is valid, execute the same SQL query through a database client test tool.

36 CRConfig.xml Tag Reference

This section provides you with a description of each tag in the CRConfig XML file. Use this reference to configure CRConfig.xml for use with your Crystal data drivers.

36.1 Parameter reference

The tags that appear in CRConfig.xml depend on the drivers you install. For example, if you do not install the native XML driver, the <XML> tags do not appear. Optimally, the CRConfig.xml file contains these settings.

Note

For Crystal Reports, the CRConfig.xml file is installed by default in the \Program Files\SAP BusinessObjects\SAP BusinessObjects Enterprise XI 4.0\java directory.

Tag Name	Description
reportlocation	When creating a report source using the Java Reporting Component, the report's path can be specified by using either a relative or absolute path. When using relative paths, the path is specified relative to the location of the Java Reporting Component on the web server. For example, ../reports/sample.rpt resolves to /WEB-INF/reports/sample.rpt where the Java Reporting Component JAR file is in /WEB-INF/lib. The value of the <reportlocation> tag becomes the new root directory that is used to determine the location of a report. If the <reportlocation> tag is not present, the Java Reporting Component uses absolute paths. When using absolute paths, the Java Reporting Component will also look in the location of your web application's resources. For example, you can place your report into the classes folder and then specify the name of the report directly in your JSP file (with no path). In this case, the Java Reporting Component will find the report. To use absolute paths, ensure that the CRConfig.xml file does not contain a <reportlocation> tag.

Tag Name	Description
timeout	The time out interval determines (in minutes) when inactive report sources are disposed. By default, the <timeout> interval is 10 minutes. You can configure the Java Reporting Component to have no <timeout> by setting the value to 0. The <timeout> interval only applies to inactive reports—reports that are being processed are not timed out as a result of exceeding this value. Each time a report source request is successfully completed, the <timeout> timer is reset. If a report source is not used within the <timeout> interval, it is disposed and its resources are made available to other processes.
<i>ExternalFunctionLibraryClassNames</i>	
classname	The <classname> tag value is the fully qualified class-name to the first Java function library. Repeat this tag for each Java function library that you want to reference.
keycode	The <keycode> tag specifies the key code used when running the Java Reporting Component.
<i>DataDriverCommon</i>	
JavaDir	The <JavaDir> tag contains the directory of your Java executable. If you have multiple versions of the Java Runtime Environment installed, this tag points to the version that you want to use; for example: <pre><JavaDir>C:\apps\j2sdk1.4.2\bin</JavaDir></pre> Alternatively, the path information may be configured as a user or system environment variable. In this case, if JAVA_HOME is defined as a variable, you can use {JAVA_HOME} as in this example: <pre><JavaDir>\${JAVA_HOME}\bin</JavaDir></pre>

Tag Name	Description
Classpath	The <Classpath> tag contains a list of the classpaths to the following .jar files: CRDBJavaServer.jar All database-specific JDBC driver .jar files For the Oracle JDBC driver, for example, classpath to ojdbc14.jar is required. Alternatively, the classpath information may be configured as a user or system environment variable. If this variable is not defined, it is equivalent to <Classpath>\${CLASSPATH}</Classpath>. Environment variables can also be included in a mixed path, as shown in this example: <pre><Classpath>c:\java\lib\xerces.jar;\$ {CommonProgramFiles}"/Business Objects/4.0/bin/CRDBJavaServer.jar";\$ {CLASSPATH}</Classpath></pre> When using classpaths with long folder and/or file names that contain spaces, ensure that they are enclosed within quotation marks. For example, "C:\Program Files\Business Objects\Common\4.0\bin\CRDBJavaServer.jar".
IORFileLocation	The <IORFileLocation> tag provides a temporary directory for use by the JDBC driver. This location must exist and be accessible by the system; otherwise, the driver will fail to work.
JavaServerTimeout	The <JavaServerTimeout> tag specifies the maximum amount of time your Java server will continue processing your request until it shuts itself down. The default value is 1800 seconds (30 minutes).
JVMMaxHeap	The <JVMMaxHeap> tag specifies the maximum amount of heap space that the JVM can allocate for running the Java server. This tag is set to 64 MB of heap by default. Change this number to a higher value if you need to process a large amount of data.
JVMMinHeap	The <JVMMinHeap> tag specifies the minimum amount of heap space that the JVM can allocate for running the Java server. This tag is set to 32 MB of heap by default.

Tag Name	Description
NumberOfThreads	The <NumberOfThreads> tag specifies the number of concurrent instances that can be run. This tag is set to 50 by default.
<i>JDBC</i>	
CacheRowSetSize	The <CacheRowSetSize> tag specifies the number of rows to cache in memory. This tag is set to 100 rows by default.
JDBCURL	The <JDBCURL> tag is the default JDBC connection URL that will be displayed in Crystal Reports when you create a new JDBC data connection. The exact format of the connection URL is specific to the database driver and is provided by the database driver vendor. For example, the connection URL for the Oracle JDBC driver is: <pre>jdbc:oracle:thin:@<hostname>:<port>:<sid></pre> Where: <hostname> is the TCP/IP address or TCP/IP host name of the server to which you are connecting. <port> is the number of the TCP/IP port. <sid> is the Oracle database ID.
JDBCClassName	The <JDBCClassName> is the default full classname of the JDBC driver that will be displayed in Crystal Reports when creating a new JDBC data connection. For example, the full classname of the Oracle JDBC driver is: <pre>oracle.jdbc.driver.OracleDriver</pre>
JDBCUserName	The <JDBCUserName> is the default user ID that will be displayed in Crystal Reports when creating a new JDBC data connection. The JDBC driver uses the user ID to connect to the database.
JNDIURL	The <JNDIURL> tag is the default JNDI connection URL that will be displayed in Crystal Reports when you create a new JNDI data connection. The exact format of the connection URL is specific to the database driver and is provided by the database driver vendor.

Tag Name	Description
JNDIConnectionFactory	The <JNDIConnectionFactory> tag is the name of the connection factory of the JNDI server. For WebLogic, it is <code>weblogic.jndi.WLInitialContextFactory</code> . For WebSphere, it is <code>com.ibm.websphere.naming.WsnInitialContextFactory</code> .
JNDIInitContext	The <JNDIInitContext> tag is the starting point for where to look for JNDI context on the JNDI server. For example, you can set <code>JNDIInitContext=/</code> for WebLogic. The tag starts to search from the top of the tree. You can set this tag to <code>cell/nodes/localhost/servers/server1/jdbc</code> for WebSphere if you created the connection in this node.
JNDIUserName	The <JNDIUserName> is the default user ID that will be displayed in Crystal Reports when creating a new JNDI data connection. The JDBC driver uses the user ID to connect to the database.
<i>GenericJDBCDriver</i>	
Default	These settings are used when the program cannot find a corresponding configuration section for the database server that you are trying to connect to.
ServerType	The <ServerType> tag specifies a keyword that represents a database server type. Its value is usually UNKNOWN for the Default settings.
QuoteIdentifiedOnOff	The <QuoteIdentifiedOnOff> tag can be the value ON or OFF. If the value is ON, the quote characters will be added to the SQL statements generated for the connections that use the Default settings; otherwise, quote characters are not added.
StoredProcType	The <StoredProcType> tag describes the type of Stored Procedure that the database uses. Valid values are Standard and Oracle.
LogonStyle	If you are using a generic JDBC driver instead of a specific driver, the <LogonStyle> tag specifies the driver whose logon style you want to emulate. Valid values are DB2, SQLServer, Oracle, MySQL and Standard.
Sybase	These settings are used when you are trying specifically to connect to a Sybase server.

Tag Name	Description
ServerType	The <ServerType> tag specifies a keyword that represents the Sybase server type. Its value is SYBASE.
QuoteIdentifiedOnOf	The <QuoteIdentifiedOnOff> tag can be the value ON or OFF. If the value is ON, the quote characters will be added to the SQL statements generated for the Sybase connections; otherwise, quote characters are not added.
DriverClassName	The <DriverClassName> tag describes the JDBC driver class name of the database specified in the current configuration section. For the Sybase section, the tag's value is the Sybase JDBC driver class name.
StoredProcType	The <StoredProcType> tag describes the type of Stored Procedure that the database uses. Valid values are Standard and Oracle.
LogonStyle	If you are using a generic JDBC driver instead of a specific driver, the <LogonStyle> tag specifies the driver whose logon style you want to emulate. Valid values are DB2, SQLServer, Oracle, MySQL and Standard.
XML	
CacheRowSetSize	The <CacheRowSetSize> tag specifies the number of rows to cache in memory. This tag is set to 100 rows by default.
PreReadNBytes	The <PreReadNBytes> tag determines the number of bytes to read from the XML doc for each logical read. This parameter is used to improve efficiency and to avoid reading one byte at a time; it is equivalent to block read size. The size should not be too small (at least 4096). Setting the value too high affects memory usage. This parameter is set to 4096 by default.
XMLLocalURL	The <XMLLocalURL> tag is the default connection URL for a local XML file. This default will be displayed in Crystal Reports when you create a new XML data connection.
SchemaLocalURL	The <SchemaLocalURL> tag is the default connection URL for a local schema file. This default will be displayed in Crystal Reports when you create a new XML data connection.

Tag Name	Description
XMLHttpURL	The <XMLHttpURL> tag is the default connection URL for an HTTP XML file. This default will be displayed in Crystal Reports when you create a new XML data connection using HTTP(S).
SchemaHttpURL	The <SchemaHttpURL> tag is the default connection URL for an HTTP schema file. This default will be displayed in Crystal Reports when you create a new XML data connection using HTTP(S).
<i>JavaBeans</i>	
CacheRowsetSize	The <CacheRowsetSize> tag specifies the number of rows to cache in memory. This tag is set to 100 rows by default.
JavaBeansClassPath	The <JavaBeansClassPath> tag value is the fully qualified classname to the Java Bean.

37 Command Reference

Menu commands are used to create, customize, print, and save your reports. Each menu contains a list of commands that are related by their function. For example, all of the commands related to files, such as opening, closing, saving, and so on, are found on the File menu.

37.1 File menu

The File menu commands allow you to open, close, and save files, to save files under a different file name, to send the file to a printer, and to create new reports.

Additionally, it contains commands that enable you to change page margins, preview a report before printing, select a printer if you want the report to print on something other than the default printer, and export a report to a disk file in a variety of formats. You can also add summary information to help you identify a report, as well as change the default settings so that the program works the way that is the most efficient for you.

37.1.1 New submenu

Use the commands on the New submenu to create a report by using a report wizard. You can also click the Blank Report command to open a blank report that you can work on without the help of a wizard.

Note

The New Report command is available as a button on the [Standard toolbar \[page 1145\]](#). When you use the command from the toolbar, the Standard Report Creation Wizard appears.

Related Information

[Report Creation Wizards \[page 1380\]](#)

[Introduction to Reporting \[page 85\]](#)

37.1.1.1 Standard Report command (New submenu)

Use this command to open the [Standard Report Creation Wizard \[page 1380\]](#). This wizard guides you through the creation of a typical report.

37.1.1.2 Blank Report command (New submenu)

Use this command to create a new report without the help of a wizard. When you select this option, the [Database Expert \[page 1420\]](#) appears so that you can choose a data source.

37.1.1.3 Cross-Tab Report command (New submenu)

Use this command to open the [Cross-Tab Report Creation Wizard \[page 1381\]](#). This wizard guides you through the creation of a report that contains a summarized grid.

37.1.1.4 Mailing Label Report command (New submenu)

Use this command to open the [Mailing Labels Report Creation Wizard \[page 1381\]](#). This wizard guides you through the creation of a report with multiple columns. The wizard is commonly used to create a sheet of address labels.

37.1.1.5 OLAP Cube Report command (New submenu)

Use this command to open the [OLAP Report Creation Wizard \[page 1382\]](#). This wizard guides you through the creation of a report that contains a summarized grid based on an OLAP data source.

37.1.2 Open command

Use the Open command to open an existing report. When you choose the Open command, the [Open dialog box \[page 1314\]](#) appears. Use this dialog box to select the report file you wish to open.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Open dialog box \[page 1314\]](#)

37.1.3 Close command

Use the Close command to close the current report while keeping the application active.

When you choose the Close command:

- If you have already saved your work, the program closes the report.
- If you have not yet saved your work, the program asks you if you want to save your changes before closing.
 - If you click Yes and you have already given your report a name, the program saves your changes under that name and closes the report.
 - If you click Yes and you have not already given your report a name, the [Save As dialog box \[page 1356\]](#) appears. Assign a name to your report. When you finish, the program saves your changes under the new name and closes the report.
 - If you click No, the program closes the report. All of your changes since you last saved are lost.
 - If you click Cancel, the program cancels the close request and returns you to the report.

37.1.4 Save command

Use the Save command to save the active report to disk under its current name.

- All changes you have made while working on the report will be saved.
- The previous version of the report will be overwritten.

If you have not previously saved your report, the program will prompt you to do so.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.1.5 Save As command

Use the Save As command to save the active report to disk under a new name. All changes you have made while working on the report will be saved to the new file. Your original report file will remain unchanged. See [Save As dialog box \[page 1356\]](#).

37.1.6 Save Data with Report command

Use the Save Data with Report command if you plan to work again with the report you are creating in the near future (for example, fine tuning it) or if you plan to share the report with someone else who would not have access to the original data.

Typically, when you save a report, the program saves a report definition and the underlying data. The definition contains all of the information needed for printing your report. The data is a snapshot of the original data as it existed when you ran the report. If you clear the Save Data with Report option via the [Options command \[page](#)

[1077](#)] on the File menu, the program saves only the report definition. The data must be refreshed each time the report is run. See [Refresh Report Data command \[page 1120\]](#). To select the option for the current report, use this command.

There are some things to consider before using the Save Data With Report command.

- If you save only the definition, your report will require somewhat less disk space (enough for the template only), but the program will need to retrieve the data before it prints the report.
- If you save your data with the report your report will require somewhat more disk space (enough for the report template and the data), but the program will not need to retrieve the data before it prints the report.

Switch this command on or off to suit your needs. This option is switched on by default. To change the default, choose the [Options command \[page 1077\]](#) from the File menu and select the Save Data With Report option on the [Reporting tab \(Options dialog box\) \[page 1326\]](#) when it appears.

Note

When the program saves data with the report, it compresses the data to take up less disk space. It decompresses the data when it opens the report.

Related Information

[Options dialog box \[page 1314\]](#)

37.1.7 Save Subreport As command

The Save Subreport As command allows you to save a subreport as a primary report. Before you can use the Save Subreport As command, the subreport must be selected either on the Design tab or on the Preview tab.

37.1.8 Log On or Off Server command

Use the Log On or Off Server command to display the [Data Explorer dialog box \[page 1205\]](#) in order to log on to or log off a data source server.

To log on, choose the database file you want to open and click Log On.

To log off, choose the database file you want to open and click Log Off.

37.1.9 Print command

Use the Print command to print a hard copy of your report to the currently selected printer.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.1.10 Page Setup command

Use the Page Setup command to set page orientation, page size, and page margins for your report. Margins define the white space between your report and the edges of the page. You can also use the Page Setup to format your report for web viewing.

Related Information

[Setting page margins \[page 267\]](#)

37.1.11 Export submenu

Use the commands on the Export submenu to export your report and to save default export settings.

37.1.11.1 Export Report command (Export submenu)

Use the Export command to export your report in one of several popular word processing, database, and spreadsheet formats, in HTML or ODBC format, or in one of a number of standard data interchange formats. For example, you can export your report in Microsoft Excel format and then open it in Excel as a spreadsheet file.

You can export to a disk file, to an application, to a Lotus Notes database, to an Exchange Folder, or to email.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Export dialog box \[page 1236\]](#)

[Exporting a report \[page 617\]](#)

37.1.11.2 Report Export Options command (Export submenu)

Use the Report Export Options command to set default export options for the current report. When a user exports the report through Crystal Reports, the various report viewers, or SAP BusinessObjects Business Intelligence platform, the default options automatically appear; the user can accept these options or make modifications as needed.

Related Information

[Set Default Export Options dialog box \[page 1360\]](#)

[Excel Format Options dialog box \[page 1232\]](#)

37.1.11.3 Manage XML Exporting Formats (Export submenu)

Opens the [Manage XML Exporting Formats dialog box \[page 1305\]](#). From this dialog box, you can add and set a schema to be the default schema used when exporting reports in the XML format.

Related Information

[Managing XML exporting formats \[page 676\]](#)

37.1.12 Send To submenu

Use the commands on the Send To submenu to send your report to others.

37.1.12.1 Mail Recipient command (Send To submenu)

Use the Mail Recipient command to export your report to email. You must choose a recipient name before your report is sent.

37.1.12.2 Exchange Folder command (Send To submenu)

Use the Exchange Folder command to send your report to an Exchange folder. You must choose a profile and a folder before your report is sent.

37.1.13 Options command

Use the Options command to modify settings that apply to the reporting environment. Using these settings, you can customize the program to fit the way you work.

This command takes you to the [Options dialog box \[page 1314\]](#). This dialog box serves as a control panel for configuring the program in a number of different areas.

Once there you can change default settings in the following tabs:

- [Layout tab \(Options dialog box\) \[page 1315\]](#)
- [Database tab \(Options dialog box\) \[page 1319\]](#)
- [Formula Editor tab \(Options dialog box\) \[page 1323\]](#)
- [Reporting tab \(Options dialog box\) \[page 1326\]](#)
- [Fields tab \(Options dialog box\) \[page 1329\]](#)
- [Fonts tab \(Options dialog box\) \[page 1331\]](#)
- [Smart Tag & HTML Preview tab \(Options dialog box\) \[page 1332\]](#)
- [Dependency Checker tab \(Options dialog box\) \[page 1334\]](#)

Your settings here affect new reports and fields added to existing reports, but they do not affect the work done before you changed the settings.

Related Information

[Options dialog box \[page 1314\]](#)

37.1.14 Report Options command

Use the Report Options command to display the [Report Options dialog box \[page 1345\]](#) to set a number of technical options that apply only to the active report. Your specifications are saved with the report and do not affect any other reports. This option is useful when creating reports for distribution.

Related Information

[Report Options dialog box \[page 1345\]](#)

37.1.15 Summary Info command

Use the Summary Info command to:

- Add summary data that can help you and other users identify the report.
- View the statistical information the program generates for each report.

Summary Data

You can supply any or all of the following kinds of information about your report: author, keywords, comments, title, subject, and template. Summary information does not print with the report; it is simply an accessible part of the electronic report file. You can use this command to view and modify summary information as well.

Statistical Information

The program tracks statistical information (such as who last worked on the report, when it was created and last saved, and so forth). This information is read-only; you cannot modify it directly.

Related Information

[Document Properties dialog box \[page 1224\]](#)

[Adding summary information to the report \[page 113\]](#)

37.1.16 Recently Opened Reports command

Use the Recently Opened Reports command to reopen a report you've recently had open.

37.1.17 Exit command

Use the Exit command to end your reporting session and return to the Windows environment.

37.2 SAP menu

Integrated within Crystal Reports, the SAP menu facilitates tasks associated with reporting off BW queries. It allows you to log on to BW and work with its data sources, save reports to BW, and publish reports immediately to SAP BusinessObjects Business Intelligence platform via BW.

The SAP menu provides the following commands:

Command	Description
Create New Report from a Query	This command can create a new report based on a query that is already defined in BW
Open report	This command can open a Crystal report that has already been saved in BW.
Save Report	This command can save the report that is currently open to BW. You have the option to prepare the report for translation and publish it to SAP BusinessObjects Business Intelligence platform.
Save Report As	This command can save the report to BW with a different description and/or in a different role.
Disconnect	This command can disconnect from SAP BW.
Setting	This command can change your connection settings.
SAP mode	This command can change a predefined SAP report into a regular Crystal report.

37.3 Edit menu

The Edit menu includes commands used to modify aspects of a report. It includes commands to undo and redo actions, to cut, to copy, to delete, and to paste report and OLE objects. You can also use the Edit menu commands to change subreport links and to convert static OLE objects into editable bitmaps.

37.3.1 Undo command

The Undo command is a dynamic command. The command name (as seen on the menu) changes based on the last action you performed. Undo reverses the effects of an action and restores your document to the way it appeared before you performed the action. The program supports multiple levels of Undo.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Undo/Redo activities \[page 297\]](#)

37.3.2 Redo command

The Redo command is a dynamic command. The text changes on the menu based on the last Undo you performed. Redo restores the effects of an Undo and returns your document to the way it was before using the Undo command. The program supports multiple levels of Redo.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Undo/Redo activities \[page 297\]](#)

37.3.3 Cut command

Use the Cut command any time you wish to cut (delete) a selected object while holding it on the Clipboard for later use.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.3.4 Copy command

Use the Copy command any time you wish to hold a copy of a selected object on the Clipboard for later use without disturbing the object as it currently appears in the report.

You can copy entire report objects, including charts and maps, to the Clipboard. This allows you to copy the data values from any report and paste them into other applications as bitmap images, text files, metafiles, and so on.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.3.5 Paste command

Use the Paste command any time to paste (insert) an object from the Clipboard into your report.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.3.6 Paste Special command

The Paste Special command allows you to paste an OLE object into your report from the Clipboard, and to choose how the object is pasted.

Related Information

[OLE \[page 335\]](#)

[Paste Special dialog box \[page 1338\]](#)

[OLE Object command \[page 1100\]](#)

37.3.7 Delete command

The Delete command allows you to remove an object from your report.

37.3.8 Select All command

Use the Select All command to select all the objects on your report. If you are on the Preview screen, the first occurrence of a repeated database field is selected, the other occurrences are highlighted.

37.3.9 Find command

Use the Find command to find text or particular values of a field in the report.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Find dialog box \[page 1252\]](#)

37.3.10 Go To Page command

Use the Go To Page command as a shortcut for maneuvering within your report.

When you choose this command, the [Go To dialog box \[page 1283\]](#) appears. Use this dialog box to jump to:

- A specific page number.
- The first page of the report.
- The last page of the report.

37.3.11 Edit Report Object command

Use the Edit Report Object command to edit the currently selected text object, formula, SQL Expression, Parameter, or Running Total field. When you choose this command, the appropriate Editor or dialog box appears:

- Formula fields: Formula Workshop
- SQL Expression fields: SQL Expression Editor
- Parameter fields: Edit Parameter Field dialog box
- Running Total Field: Edit Running Total dialog box

Related Information

[Field Explorer dialog box \[page 1241\]](#)

[Formula Editor dialog box \[page 1280\]](#)

[Using Formulas \[page 687\]](#)

[Edit Parameter dialog box \[page 1227\]](#)

[Create Running Total Field dialog box \[page 1186\]](#)

[Text objects \[page 105\]](#)

[Working with text objects \[page 226\]](#)

37.3.12 Subreport Links command

Use the Subreport Links command to link the data in a subreport to data in the parent report. This enables you, for example, to create a subreport that provides the details behind summary data in the parent report. When you choose this command, the [Subreport Links dialog box \[page 1369\]](#) appears.

Related Information

[Subreport Links dialog box \[page 1369\]](#)

[Linking a subreport to the data in the primary report \[page 877\]](#)

37.3.13 Object command

Note

The name of this command and its submenu change to match the object you have selected.

Use the appropriate Object command, or the appropriate submenu command, to modify the selected embedded object.

Related Information

[OLE overview \[page 335\]](#)

37.3.13.1 Edit command (Object command variant)

Use the Edit command to edit the selected object in place using the tools from the object's original (server) application. To use the original application to edit the object, use the [Open command \(Object command variant\) \[page 1083\]](#).

Related Information

[OLE overview \[page 335\]](#)

37.3.13.2 Open command (Object command variant)

Use the Open command to open the selected object's original (server) application with the object open and ready for editing.

Related Information

[OLE overview \[page 335\]](#)

37.3.13.3 Properties command (Object command variant)

Use the Properties command to change the properties for the selected linked or embedded object.

Related Information

[OLE overview \[page 335\]](#)

37.3.13.4 Activate Contents command (Object command variant)

Use the Activate Contents command to open the package that created the linked or embedded object.

Related Information

[OLE overview \[page 335\]](#)

37.3.13.5 Edit Package command (Object command variant)

Use the Edit Package command to open the Object Packager. See the Object Package help for details.

Related Information

[OLE overview \[page 335\]](#)

37.3.13.6 Convert command (Object command variant)

Use the Convert command to convert the selected linked or embedded object to another kind of object, for example to convert a document file to a picture.

Related Information

[Convert dialog box \[page 1170\]](#)

[OLE overview \[page 335\]](#)

37.3.13.7 Convert Picture Object command (Object command variant)

Use the Convert Picture Object command to convert the selected picture object.

When you use the [OLE Object command \[page 1100\]](#) on the Insert menu to place a picture in a report, the picture can be treated as either an embedded or a linked object.

When you use the [Picture command \[page 1098\]](#) on the Insert menu to insert a picture into the report, the Report Designer converts the picture into a static object.

Static objects do not support editing. To edit a static object, it has to be converted into an editable type of object using the Convert Picture Object command on the Edit menu.

Related Information

[Convert dialog box \[page 1170\]](#)

[OLE overview \[page 335\]](#)

37.3.14 Links command

Use the Links command to make changes in existing OLE object links. Using this command, you can change the source data for a link and also the way in which that link is updated. This command is only active when you have a linked object in the report. When you insert an OLE object, you can choose to insert it as an embedded object or as linked object. See [Insert Object dialog box \(Create from File version\) \[page 1294\]](#).

When you choose the Links command, the [Links dialog box \[page 1302\]](#) appears.

Note

This command is not related to editing a database link. To edit a database link you must choose the [Links tab \(Database Expert\) \[page 1422\]](#).

Related Information

[Insert Object dialog box \(Create from File version\) \[page 1294\]](#)

[Links dialog box \[page 1302\]](#)

[OLE overview \[page 335\]](#)

37.4 View menu

The View menu includes commands used to:

- Modify the user interface of the program.
- Open or close various report and subreport views.
- Open Explorers.

37.4.1 Design command

Use the Design command to go to the Design tab of a primary report. The current view is indicated by a check mark.

37.4.2 Preview command

Use the Preview command to go to the Preview tab of a primary report. The current view is indicated by a check mark.

37.4.3 Other Views submenu

When your report has subreports, this submenu appears. Use the submenu to go to the Design or Preview tab of any subreport. The current view is indicated by a check mark.

37.4.4 Print Preview command

Use the Print Preview command to display the report in the Preview Tab before you print it, to see what it will look like and to fine tune the report while working with the actual data.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Preview tab \[page 93\]](#)

37.4.5 Preview Sample command

Use the Preview Sample command to print a draft report to a window using only a subset of the data you will use in your final report. This feature can save you a great deal of time during the report creation process.

37.4.6 HTML Preview command

Use the HTML Preview command to ensure that your report design is appropriate and correct over the web in a zero-client environment. This feature displays your report in HTML format instead of true RPT format; you can return to the Design tab to make adjustments for the best results when viewing the report over the web.

37.4.7 Close Current View command

Use the Close Current View command to close the active tab. You cannot use this command if the active tab is the Design tab of the primary report.

37.4.8 Field Explorer command

Use the Field Explorer command to open the [Field Explorer dialog box \[page 1241\]](#). If the Field Explorer is already open, this command changes focus to the explorer's root node. You can add the following types of fields to your report from this dialog box:

- Database Fields
- Formula Fields
- SQL Expression Fields
- Parameter Fields
- Running Total Fields
- Group Name Fields
- Special Fields

With the exception of Database Fields and Special Fields, you can add, edit, and delete all types of fields listed before you insert them in your report.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Field Explorer dialog box \[page 1241\]](#)

[Creating and modifying formulas \[page 700\]](#)

[Formula Editor dialog box \[page 1280\]](#)

[SQL Expression Name dialog box \[page 1367\]](#)

[Parameter field considerations \[page 831\]](#)

[Creating group headers \[page 202\]](#)

37.4.9 Report Explorer command

Use the Report Explorer command to open the [Report Explorer dialog box \[page 1344\]](#). If the Report Explorer is already open, this command changes focus to the explorer's root node. The Report Explorer displays a tree view of the sections in your report and lists the report objects contained in each section. You can work with report objects directly from the explorer.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.4.10 Repository Explorer command

Use the Repository Explorer command to open the [Repository Explorer dialog box \[page 1353\]](#). If the Repository Explorer is already open, this command changes focus to the explorer's root node. The Repository Explorer displays a tree view of the folders and report objects the repository contains. You can use report objects from the repository in your current report.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.4.11 Dependency Checker command

Use the Dependency Checker command to open the [Dependency Checker dialog box \[page 1223\]](#). The Dependency Checker displays errors found in your reports, projects, and/or object packages.

Note

Selecting this command opens the dialog box, but it does not run the validation process. You run the Dependency Checker from the Workbench or from the command on the Report menu.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.4.12 Workbench command

Use the Workbench command to open the [Workbench dialog box \[page 1377\]](#). The Workbench displays a tree view of the projects, reports, and object packages that you have previously created.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.4.13 Toolbars command

Use the Toolbars command to open the [Toolbars dialog box \[page 1374\]](#) to control the appearance of the toolbars.

There are several toolbars to choose from:

- Standard
- Formatting
- Insert Tools
- Expert Tools
- Navigation Tools
- External Command

Use the check boxes provided to show any or all of the toolbars.

Tooltips are windows that appear with a name when your cursor rests on top of a button. To view tooltips on your toolbars, select the Show ToolTips check box.

Related Information

[Standard toolbar \[page 1145\]](#)

[Formatting toolbar \[page 1147\]](#)

[Insert Tools toolbar \[page 1148\]](#)

[Expert Tools toolbar \[page 1149\]](#)

[Navigation Tools toolbar \[page 1150\]](#)

[External Command toolbar \[page 1151\]](#)

37.4.14 Status Bar command

Use the Status Bar command to show or hide the status bar. By default the status bar is shown (a check mark appears beside the command on the menu).

37.4.15 Preview Panel command

Use the Preview Panel command to show or hide the Preview Panel in the Preview tab. When the Preview Panel view is active, you can alternate between the Group Tree and the Parameter Panel.

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

Related Information

[Group Tree \[page 95\]](#)

[The Parameter Panel \[page 848\]](#)

37.4.16 Zoom command

Use the Zoom command to open the [Zoom dialog box \[page 1379\]](#) to zoom in and out on your report. With this command you can view your report at anywhere from 25% to 400% of its actual size, make it fit within the page or reset the size to 100%.

At the lower magnifications you will see more emphasis on layout; at the higher magnifications, you will see more emphasis on details.

Note

These options are also available using the Zoom Control on the [Standard toolbar \[page 1145\]](#).

Related Information

[Using the zoom feature \[page 111\]](#)

37.4.17 Rulers submenu

Use the Rulers submenu to show or hide the rulers in the Design or Preview tabs.

37.4.17.1 Design command (Rulers submenu)

Use the Design command to show or hide the rulers in the Design tab. By default the rulers are shown on the Design tab (a check mark appears beside the command on the menu).

Related Information

[Comparisons with the Design tab \[page 96\]](#)

37.4.17.2 Preview command (Rulers submenu)

Use the Preview command to show or hide the rulers in the Preview tab. By default the rulers are shown on the Preview tab (a check mark appears beside the command on the menu).

Related Information

[Comparisons with the Design tab \[page 96\]](#)

37.4.18 Guidelines submenu

Use the Guidelines submenu to show or hide the guidelines in the Design or Preview tabs.

37.4.18.1 Design command (Guidelines submenu)

Use the Design command to show or hide the guidelines in the Design tab. By default, the guidelines are shown on the Design tab (a check mark appears beside the command on the menu).

Related Information

[Designing with guidelines \[page 255\]](#)

[Viewing guidelines \[page 255\]](#)

37.4.18.2 Preview command (Guidelines submenu)

Use the Preview command to show or hide the guidelines in the Preview tab. By default, the guidelines are shown on the Preview tab (a check mark appears beside the command on the menu).

Related Information

[Designing with guidelines \[page 255\]](#)

[Viewing guidelines \[page 255\]](#)

37.4.19 Grid submenu

Use the Grid submenu to show or hide the grids in the Design or Preview tabs.

37.4.19.1 Design command (Grid submenu)

Use the Design command to show or hide the grid lines in the Design tab. By default the grid is not shown on the Design tab (no check mark appears beside the command on the menu).

Related Information

[Using the grid \[page 254\]](#)

[Selecting the grid \[page 254\]](#)

37.4.19.2 Preview command (Grid submenu)

Use the Preview command to show or hide the grid lines in the Preview tab. By default the grid is not shown on the Preview tab (no check mark appears beside the command on the menu).

Related Information

[Using the grid \[page 254\]](#)

[Selecting the grid \[page 254\]](#)

37.4.20 Tooltips submenu

Use the Tooltips submenu to allow tooltips on the Design or Preview tabs.

37.4.20.1 Design command (Tooltips submenu)

Use the Design command to allow tool tips on the Design tab.

37.4.20.2 Preview command (Tooltips submenu)

Use the Preview command to allow tool tips on the Preview tab.

37.4.21 Product Locale command

Use the *Product Locale* command to change the language used to display the Crystal Reports menus and commands. The language choices are restricted to officially supported languages.

Note

You must restart Crystal Reports for the change to take effect.

37.4.22 Preferred Viewing Locale command

Use the *Preferred Viewing Locale* (PVL) command to change the viewing language to a language other than the one used when the product was installed.

The PVL displays the language used to view the reports in the Preview Panel.

When the PVL is changed, the numeric separators and layout of a report are changed to display the symbols used by the new language locale. The currency symbol is not changed.

Note

- When you change the PVL for legacy reports, the currency symbols will change to match the new locale.
For example, if a report was created in Euros and the PVL is set to English(US), the currency symbol will change to a dollar sign.
- You may need to install additional character sets to view reports in some languages

37.5 Insert menu

The Insert menu includes commands used to insert subtotals, grand totals, summaries (counts, averages, and so on), and groups. This menu also includes commands that enable you to insert OLAP grids, cross-tab objects, subreports, graphics, lines, boxes, charts, and OLE objects into a report.

37.5.1 Text Object command

Use the Text Object command to insert a text object anywhere on your report. A text object is an object that holds text (a single character, a single word, entire sentences, or an entire letter). A text object can hold database fields and formula fields as well as text. Thus, a text object can be used for a variety of reporting needs such as labels, titles, annotations, callouts, disclaimers, footnotes, and even mail merge form letters.

ⓘ Note

While the effective limit on text object size is 32K (including carriage return and line feed characters), it is recommended that text objects be used only for more manageably sized blocks of text.

ⓘ Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Text objects \[page 105\]](#)

37.5.2 Summary command

Use the Summary command to summarize data and print the summary in your report. The program sorts, groups and summarizes in a single step. You can use summaries:

- To count the number of values in a group.
- To calculate the sum, average, standard deviation, or variance value in a group.
- To identify the minimum or maximum value in a group.

This command is only active after you have selected a field to summarize. When you choose the Summary command, the [Insert Summary dialog box \[page 1298\]](#) appears.

ⓘ Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Insert Summary dialog box \[page 1298\]](#)

[Grouping, sorting, and summarizing data \[page 109\]](#)

37.5.3 Field Heading command

Use the Field Heading command to add a heading for a field you've selected in the Details section—if one doesn't already exist. The heading duplicates the field name in a text object that is inserted in the Page Header.

ⓘ Note

You cannot use this command to add a heading for a formula field or an SQL Expression.

ⓘ Note

You cannot add field headings to the BusinessObjects Enterprise Repository.

ⓘ Note

You cannot add an embedded field to a field heading.

37.5.4 Group command

Use the Group command to sort your data and break it into groups. For example, you could group the information in a customer list into state or zip codes. When you choose the Group command, the [Insert Group dialog box \[page 1287\]](#) appears.

ⓘ Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Insert Group dialog box \[page 1287\]](#)

[Grouping, sorting, and summarizing data \[page 109\]](#)

[Grouping data \[page 173\]](#)

37.5.5 OLAP Grid command

Use the OLAP Grid command to insert an OLAP Grid into your report. When you choose the OLAP Grid command, the [OLAP Expert \[page 1442\]](#) appears. The term OLAP is used to refer to all common forms of OLAP (Online Analytical Processing) and multi-dimensional data storage and access systems.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[OLAP Report Creation Wizard \[page 1382\]](#)

[OLAP grid objects \[page 395\]](#)

[Creating and Updating OLAP Reports \[page 394\]](#)

37.5.6 Cross-Tab command

Use the Cross-Tab command to insert a cross-tab object into your report. A cross-tab is a spreadsheet-like report used for identifying trends. You can create a report in which the cross-tab object is the entire report, or you can insert one or more cross-tab objects into a primary report, as subreports. When you choose the Cross-Tab command, the [Cross-Tab Expert \[page 1412\]](#) appears.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Cross-Tab Objects \[page 346\]](#)

[Cross-Tab Expert \[page 1412\]](#)

37.5.7 Subreport command

Use the Subreport command to create a subreport object that you can place on your report. A subreport object is a self-contained report that has all of the usual report characteristics and abilities. You can import an existing report to use as a subreport object, or you can create your subreport from scratch as you would any other report.

You can insert multiple subreport objects into your parent report if you wish. You can coordinate the subreport data with the data in the primary report, or you can leave your subreport freestanding and unrelated to any data in the parent report.

When you choose the Subreport command, the [Insert Subreport dialog box \[page 1295\]](#) appears.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Subreports \[page 870\]](#)

[Inserting subreports \[page 873\]](#)

[Insert Subreport dialog box \[page 1295\]](#)

37.5.8 Line command

Use the Line command to draw horizontal and vertical lines anywhere on your report. You can use these lines to enclose field data or to create other graphic effects.

While you can create lines with the Outside Borders button, the lines you draw via the Line command, differ from those in several important ways:

- Border lines cannot stand alone (they are always attached to a field or a picture) while graphic lines can stand alone.
- A border line is limited to a very small selection of sizes, while a graphic line can be any length or height you wish.
- A border line increases the height of the adjacent text line (taking up more space), while a graphic line does not.
- A border (field border) cannot span multiple sections; whereas, a graphic line can.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Line tab \(Format Editor dialog box\) \[page 1266\]](#)

[Adding and editing lines \[page 276\]](#)

37.5.9 Box command

Use the Box command to draw free-form boxes anywhere on your report. You can use these boxes to enclose field data or to create other graphic effects.

While you can create boxes via the Outside Borders button, the boxes you draw via the Box command differ from those in several important ways:

- Borders come in fixed sizes; whereas, boxes can be created in any size you need.
- A border can be used to enclose individual field values; whereas, a box placed around a field encloses all the values in the field.
- Borders cannot stand alone (they must surround a field or a picture), while boxes can.
- Boxes do not increase the height of a text line whereas borders do.
- A border (field border) cannot span multiple sections, while a box can.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Box tab \(Format Editor dialog box\) \[page 1264\]](#)

[Adding and editing boxes \[page 277\]](#)

37.5.10 Picture command

Use the Picture command to insert bitmapped artwork (graphics, logos, and so on) into your report. The program helps you enhance the visual impact of your report using artwork in the following popular formats: bitmap, TIFF, JPEG, and PNG.

When you use the [OLE Object command \[page 1100\]](#) on the Insert menu to place a picture in a report, the picture can be treated as either an embedded or a linked object.

When you use the Picture command on the Insert menu to insert a picture into the report, the Report Designer converts the picture into a static object.

Static objects do not support editing. To edit a static object, it has to be converted into an editable type of object using the [Convert Picture Object command \(Object command variant\) \[page 1085\]](#) on the Edit menu.

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Working with static OLE objects \[page 339\]](#)

37.5.11 Chart command

Use the Chart command to insert a chart into your report. When you choose the Chart command, the [Chart Expert \[page 1400\]](#) appears. Use this Expert to choose from among four types of Charts:

- Advanced chart
- Group chart
- Cross-Tab chart
- OLAP chart

Follow the step-by-step instructions in the Chart Expert to create your chart.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Charting \[page 303\]](#)

37.5.12 Map command

Use the Map command to open the [Map Expert \[page 1433\]](#) and insert a map into your report. Maps help you analyze your report data and identify trends more efficiently. For example, you could create a map that shows sales by region. You would then be able to:

- Drill down on the map regions to view the underlying data.
- Analyze the data using one of the five map types.
- Adjust the appearance and organization of the map (allowing you to better identify trends).

In order to place a generic, group-based map on your report, you must first have a group and a summary or subtotal field present in the report. However, Crystal Reports does provide several specialized map layouts (Advanced, Cross-Tab, and OLAP) that do not require groups and summaries.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#).

Related Information

[Map Expert \[page 1433\]](#)

[Mapping \[page 320\]](#)

37.5.13 Flash command

Use the Flash command to add a Shockwave Flash (SWF) object to your report.

Note

This command is also available as a button on the [Insert Tools toolbar \[page 1148\]](#) menu.

37.5.14 OLE Object command

Use the OLE Object command to link or embed an OLE object in your report.

Note

You can also insert a picture using the [Picture command \[page 1098\]](#) on the Insert menu. In that case, the Report Designer converts the picture into a static object, which you cannot edit. To convert a static object to an editable object, use the [Convert Picture Object command \(Object command variant\) \[page 1085\]](#) on the Edit menu.

Related Information

[Insert Object dialog box \[page 1293\]](#)

[Inserting OLE objects into reports \[page 337\]](#)

[OLE \[page 335\]](#)

37.5.15 Template Field Object command

Use the Template Field Object command to insert a Template Field Object into your report.

Template Field Objects do not refer to existing database fields; you put them in your template report and format them as you require. Then, when you apply the template to another report, Crystal Reports displays that report's data with the formatting you specified.

Related Information

[Using Template Field Objects \[page 238\]](#)

37.6 Format menu

The Format menu includes commands used to change the look of the elements in a report. It has commands for formatting fields that are embedded in text objects, for formatting individual paragraphs in those text objects, and for entire sections of the report as well. There are also commands for formatting charts, as well as formatting and pivoting cross-tabs. Finally, Format menu commands can be used to have the program automatically arrange report objects, apply professionally designed styles to the entire report, and move individual objects within a stack of objects forward and backward.

37.6.1 Format command

Note

The format command you see on the [Format menu \[page 1101\]](#) changes to match the object you have selected (for example, Format Field, Format Box, Format Graphic, Format Map, Format Chart, and so on).

Use the displayed command to format the selected object.

The Format commands open the [Format Editor dialog box \[page 1256\]](#), which presents you with the various formatting properties that you can apply to the object. The properties vary for each kind of object, but they include:

- Applying borders and colors.
- Changing the font for text or numbers.
- Suppressing the object so that it does not print.
- Cropping the size of the object.
- Maintaining the object size or letting it grow.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Formatting \[page 235\]](#)

[Format Editor dialog box \[page 1256\]](#)

37.6.2 Format Painter command

Use the Format Painter command to copy absolute or conditional formatting properties from one report object or field to one or more target objects or fields.

Related Information

[Using the Format Painter \[page 298\]](#)

37.6.3 Hyperlink command

Use the Hyperlink command to create a hyperlink from the selected object to a web site, a file, an email address, or a report object in another report.

The hyperlink is saved with your report and is available to other users as a way of viewing additional information. Hyperlinks are also used to define Report Parts to be viewed in web browsers.

When you choose the Hyperlink command, the [Hyperlink tab \(Format Editor dialog box\) \[page 1270\]](#) appears.

Note

If you create a hyperlink to another report object, you are creating an on-demand subreport. Just as one subreport cannot contain another subreport, you cannot create a hyperlink between two subreports. See [How subreport linking works \[page 872\]](#).

Note

To select a hyperlinked object, use *Shift-click* or *Ctrl-click*. Clicking on a hyperlink, without using the *Shift* or *Ctrl* key, activates the link.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Format Editor dialog box \[page 1256\]](#)

[Hyperlink fields \[page 106\]](#)

[Setting up navigation \[page 632\]](#)

37.6.4 Use Expert command

The name of the Use Expert command changes to reflect the object you've selected in your report. Use the command to open the appropriate expert for the object you've selected. For example, if you've selected a chart, the Use Expert command opens the [Chart Expert \[page 1400\]](#).

37.6.5 Highlighting Expert command

Use the Highlighting Expert command to display the [Highlighting Expert \[page 1431\]](#) to apply conditional formatting to currency and number fields.

Use the Highlighting Expert to highlight field values that are of particular interest. For example, to highlight customers whose sales are greater than \$25,000, you could use the Highlighting Expert to apply red background color to the {customer.LAST YEAR'S SALES} field, if the value is greater than \$25,000.

You can also use the dialog box to set conditional font color and border style.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Using the Highlighting Expert \[page 294\]](#)

37.6.6 Text Formatting command

Use the Text Formatting command to format text in a text object. You can set the font characteristics, indentation and spacing, and tabs. In order to activate this command, you have to double-click the text object.

When you choose the Text Formatting command the [Text Format dialog box \[page 1373\]](#) appears with the [Font tab \(various dialog boxes\) \[page 1274\]](#) selected.

Note

When you select a field, or select a text object by clicking it, you can move, or resize the object, or apply formatting that affects all the text in the object.

Note

When you select a text object by double-clicking it, you can edit the text itself, and apply formatting to sections of the text.

Related Information

[Formatting \[page 235\]](#)

[Format Editor dialog box \[page 1256\]](#)

37.6.7 Move Submenu

The commands on the Move submenu allow you to give prominence to particular items in a stack of multiple items.

37.6.7.1 Backward command (Move submenu)

When you have multiple objects in a stack, use the Backward command to move the selected object "backward" (down) one layer in your stack. In order to access an object you have moved backward, you can do one of the following:

- Move objects that are above it out of the way.
- Move those objects backward.

Related Information

[To Back command \(Move submenu\) \[page 1104\]](#)

[Forward command \(Move submenu\) \[page 1105\]](#)

[To Front command \(Move submenu\) \[page 1105\]](#)

37.6.7.2 To Back command (Move submenu)

When you have multiple objects in a stack, use the To Back command to move the selected object to the bottom of the stack.

Related Information

[Backward command \(Move submenu\) \[page 1104\]](#)

[Forward command \(Move submenu\) \[page 1105\]](#)

[To Front command \(Move submenu\) \[page 1105\]](#)

37.6.7.3 Forward command (Move submenu)

When you have multiple objects in a stack, use the Forward command to move the selected object "forward" (up) one layer in the stack.

Related Information

[Backward command \(Move submenu\) \[page 1104\]](#)

[To Back command \(Move submenu\) \[page 1104\]](#)

[To Front command \(Move submenu\) \[page 1105\]](#)

37.6.7.4 To Front command (Move submenu)

When you have multiple objects in a stack, use the To Front command to move the selected object to the front (top) of your stack.

Related Information

[Backward command \(Move submenu\) \[page 1104\]](#)

[To Back command \(Move submenu\) \[page 1104\]](#)

[Forward command \(Move submenu\) \[page 1105\]](#)

37.6.8 Align submenu

Use the commands on the Align submenu to align the currently selected objects with the last object you select. Click an object to select it and then *Shift*-click or *Ctrl*-click to select further objects.

Once you have selected the objects that you want to align, choose a command from the submenu.

37.6.8.1 Tops (Align submenu)

Use the (Align) Tops command to align the tops of the selected objects with the top of the last object selected.

37.6.8.2 Middles (Align submenu)

Use the (Align) Middles command to align the (vertical) middles of the selected objects with the (vertical) middle of the last object selected.

37.6.8.3 Bottoms (Align submenu)

Use the (Align) Bottoms command to align the bottoms of the selected objects with the bottom of the last object selected.

37.6.8.4 Baseline (Align submenu)

Use the (Align) Baseline command to align the baseline of the selected objects with the baseline of the last object selected.

37.6.8.5 Lefts (Align submenu)

Use the (Align) Lefts command to align the left of the selected objects with the left of the last object selected.

37.6.8.6 Centers (Align submenu)

Use the (Align) Centers command to align the center of the selected objects with the center of the last object selected.

37.6.8.7 Rights (Align submenu)

Use the (Align) Rights command to align the right of the selected objects with the right of the last object selected.

37.6.8.8 To Grid (Align submenu)

Use the (Align) To Grid command to align the selected objects with the grid.

37.6.9 Make Same Size submenu

Use the commands on the Make Same Size submenu to resize an object or objects to match the size of another object.

Click the first object you wish to resize, then [Shift](#)-click or [Ctrl](#)-click to select further objects, finally selecting the object whose size you wish to match.

Choose the appropriate size command from the submenu to resize the selected objects to match the size of the last object you selected.

37.6.9.1 Width (Make Same Size submenu)

Resizes the selected objects to the same width as the last object you selected.

37.6.9.2 Height (Make Same Size submenu)

Resizes the selected objects to the same height as the last object you selected.

37.6.9.3 Both (Make Same Size submenu)

Resizes the selected objects to the same size (both width and height) as the last object you selected.

37.6.10 Size and Position command

Use the Size and Position command to increase or decrease the size of the selected object, or to adjust the position of the selected object, in relation to the top and side margins. When you choose the Size and Position command, the [Object Size and Position dialog box \[page 1311\]](#) appears.

37.6.11 Pivot OLAP Grid command

Use the Pivot OLAP Grid command to pivot an OLAP grid.

Related Information

[OLAP Report Creation Wizard \[page 1382\]](#)

37.6.12 Pivot Cross-Tab command

Use the Pivot Cross-Tab command to pivot a cross-tab on its axis, turning columns into rows and rows into columns. Use the [Cross-Tab command \[page 1096\]](#) on the Insert menu to create a cross-tab object.

Related Information

[Cross-Tab Objects \[page 346\]](#)

37.7 Database menu

The Database menu includes commands used to work with databases (adding and deleting tables for use with reports, changing the alias used to identify a table, and linking and unlinking tables) and to set the location of your report's data source. It has commands for logging on and off SQL and ODBC servers, showing SQL queries, and modifying parameters for stored procedures. The Database menu also has commands that allow you to change database drivers used in a report, to browse field data, and to push grouping down to the server level.

37.7.1 Database Expert command

When creating a new report, you select the first database you want to use in your report from the [Database Expert \[page 1420\]](#) which appears when you begin a new report. Use the Database Expert command to select an additional database table or tables for use in your report.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[The Database Expert \[page 86\]](#)

[Selecting the data source \[page 98\]](#)

37.7.2 Set Datasource Location command

Use the Set Datasource Location command to change the location of a database that is active in a report. The Set Datasource Location command provides a simple way to indicate the new name or location of database files. This is especially useful if someone sends you a report that uses a database that has a different location on your system, or if you have changed the directory or disk location of a database. In addition, the Set Datasource Location command automatically converts your database driver to the data source you have chosen. For example, you can automatically convert a direct access data source to an ODBC data source using the Set Location command.

Note

This command does not physically move the database or databases. It simply directs Crystal Reports to look for the database or databases in another location than the one originally specified when setting up the report.

Related Information

[To change the data source \[page 900\]](#)

[Using the Set Datasource Location process \[page 926\]](#)

37.7.3 Log On or Off Server command

Use the Log On or Off Server command to display the [Data Explorer dialog box \[page 1205\]](#) in order to log on to or log off an SQL or ODBC data source server.

To log on, choose the database file you want to open and click Log On.

To log off, choose the database file you want to open and click Log Off.

37.7.4 Browse Data command

Use the Browse Data command to view the values in a report field.

Note

This command is only active if you have selected a field on your report.

37.7.5 Set OLAP Cube Location command

Use the Set OLAP Cube Location command to enter the file path for an OLAP cube. This tells Crystal Reports where to find the cube.

This command is only available when you have selected an OLAP grid.

Related Information

[Updating an OLAP report \[page 401\]](#)

[Set OLAP Cube Location dialog box \[page 1363\]](#)

37.7.6 Verify Database command

Use the Verify Database command to make certain your report prints with the current version of the active database and resets its record buffers to the current record size.

When you first create a report, the report draws its fields from the database as it exists at that time. It uses the structure of the database (number of fields, field position, data type, and so on) to identify and select those fields you want placed on the report. If you change the structure of the database (by adding or deleting fields) after you create the report, Crystal Reports needs to adapt the report to the new structure. This command is the tool you can use for adapting the report to the new database structure.

- If the database has more fields now than it had when the report was first created, Crystal Reports attempts to identify and use the correct fields from the new database. The aim is to print an unchanged version of the report even though the underlying database has changed.
- If the database has fewer fields now than it had when the report was first created, it uses those fields that are still available when it prints the report and ignores those that are no longer available.

Note

If your report is using ADO, RDO, or DAO to access data, then Verify Database can also be used to modify the command you wish to execute.

The [Map Fields dialog box \[page 1306\]](#) appears if the report database has changed since you last ran the report.

Related Information

[Using the Verify Database process \[page 925\]](#)

37.7.7 Show SQL Query command

Use the Show SQL Query command to view the SQL query that Crystal Reports is sending to your SQL data source.

Related Information

[Show SQL Query dialog box \[page 1364\]](#)

[SQL database considerations \[page 906\]](#)

[Using SQL and SQL databases \[page 888\]](#)

37.7.8 Perform Grouping On Server command

Use the Perform Grouping On Server command to perform server-side processing for a designated report. Server-side processing allows you to set up a report that performs the majority of its processing on the server and pushes only relevant details to your computer. Server-side processing provides you with a number of benefits:

- Less time connected to the server.
- Less memory needed to process the report on your computer.
- Lower transfer time from the server to the client.

Related Information

[Server-side processing \[page 921\]](#)

37.7.9 Select Distinct Records command

Use the Select Distinct Records command to refine your report's SQL query and prevent it from returning duplicate records from the database.

When this command is enabled, SQL SELECT statements are treated as SELECT DISTINCT statements. Consequently, the query looks for identical records and ensures that they are returned only once. (To be considered identical, records must match completely—not just one or two fields.)

This command is only available for ODBC, OLE DB, and SQL native drivers; it is not available for Stored Procedures. Note also that your server may automatically sort returned values if no sort is specified in Crystal Reports.

37.7.10 Query Panel command

Use the Query Panel command to open the Business Objects Query Panel in Crystal Reports. In the Query Panel, you can build and modify queries and filter universe data that the program returns to Crystal Reports.

Related Information

[Reporting on Universes and managed connections \[page 376\]](#)

37.8 Report menu

The Report menu includes commands used to select the records or groups to be included in a report; you can use either an expert or the Formula Editor to specify your record or group selection. You can use commands to open various other experts to help you work with report sections, groups, XML definitions, report templates, and to select the order in which report data is to be sorted (by record or by group). It has commands for updating the data used in a report and for specifying a print date for a report.

37.8.1 Select Expert command

Use the Select Expert to select the records or groups you want to include in your report, and to filter out those you don't want to include. For example, you may have a customer database that contains records for customers from every state and Canadian province, but you may want to do a report only on Texas customers. This command lets you restrict your report so that only Texas customers are included.

When you choose the Select Expert, if you have selected a field the Select Expert appears. If you have not selected field, you have to choose a field from the [Choose Field dialog box \[page 1167\]](#) before the Select Expert appears. Crystal Reports automatically generates a record or group selection formula based on settings you specify in the dialog box.

This command allows you to set up reasonably complex selection criteria, but to have more control over the formula that is generated, use the [Selection Formulas submenu \[page 1114\]](#) on the Report menu and set up your selection formula using the Group Selection or Record Selection folders of the [Formula Editor dialog box \[page 1280\]](#).

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Select Expert \[page 1457\]](#)

[Record Selection \[page 155\]](#)

[Grouping data \[page 173\]](#)

37.8.1.1 Record command (Select Expert submenu)

Use the Record command to create a record selection formula or to edit an existing record selection formula from within the [Select Expert \[page 1457\]](#).

Related Information

[Record Selection \[page 155\]](#)

37.8.1.2 Group command (Select Expert submenu)

Use the Group command to create a group selection formula or to edit an existing group selection formula from within the [Select Expert \[page 1457\]](#).

Related Information

[Grouping data \[page 173\]](#)

37.8.1.3 Saved Data command (Select Expert submenu)

Use the Saved Data command to create a saved data selection formula or to edit an existing saved data selection formula from within the [Select Expert \[page 1457\]](#).

Note

Saved data selection formulas do not cause a refresh to the database.

Related Information

[Saved data selection formulas \[page 160\]](#)

37.8.2 Selection Formulas submenu

Use the commands on the Edit Selection Formula submenu to select the records or groups that you want included in your report (if you do not want them all included).

37.8.2.1 Record command (Selection Formulas submenu)

Use the Record command to create a record selection formula or to edit an existing record selection formula. When you choose this command, the [Formula Editor dialog box \[page 1280\]](#) appears with the Record Selection folder selected. Use this editor to build a Boolean selection formula.

Related Information

[Record Selection \[page 155\]](#)

37.8.2.2 Group command (Selection Formulas submenu)

Use the Group command to create a group selection formula or to edit an existing group selection formula. When you choose this command, the [Formula Editor dialog box \[page 1280\]](#) appears with the Group Selection folder selected. Use this editor to build a Boolean selection formula.

Related Information

[Grouping data \[page 173\]](#)

37.8.2.3 Saved Data command (Selection Formulas submenu)

Use the Saved Data command to create a saved data selection formula or to edit an existing saved data selection formula. When you choose this command, the [Formula Editor dialog box \[page 1280\]](#) appears with the Saved Data Selection folder selected. Use this editor to build a Boolean selection formula.

Note

Saved data selection formulas do not cause a refresh to the database.

Related Information

[Saved data selection formulas \[page 160\]](#)

37.8.3 Formula Workshop command

Use the Formula Workshop command to open the [Formula Workshop dialog box \[page 1282\]](#). The Formula Workshop allows you to create or modify any type of formula used in Crystal Reports:

- Report Custom Functions
- Repository Custom Functions
- Formula fields
- SQL Expression fields
- Selection formulas
- Formatting formulas
- Alerting formulas

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Using Formulas \[page 687\]](#)

[Using Custom Functions \[page 731\]](#)

[Selecting records \[page 155\]](#)

[Working with conditional formatting \[page 287\]](#)

[Working with Report Alerts \[page 657\]](#)

37.8.4 Alerts submenu

Use the Alerts submenu to create, modify, or view report alerts.

37.8.4.1 Create or Modify Alerts command (Alerts submenu)

Use the Create or Modify Alerts command to open the [Create Alerts dialog box \[page 1171\]](#). Use this dialog box to specify a condition that, when met, displays a message about the report data.

Related Information

[Report Alerts \[page 657\]](#)

37.8.4.2 Triggered Alerts command (Alerts submenu)

Use the Triggered Alerts command to open the [Report Alerts dialog box \[page 1344\]](#). Use this dialog box to view the name, message, and records associated with a triggered report alert.

Related Information

[Viewing Report Alerts \[page 660\]](#)

37.8.5 Report Bursting Indexes command

Use the Report Bursting Indexes command to open the [Saved Data Indexes dialog box \[page 1356\]](#). When you create a Saved Data Index on a particular field, Crystal Reports can more efficiently filter on that field, which will help performance in large reports.

37.8.6 Section Expert command

Use the Section Expert command to display the [Section Expert \[page 1449\]](#). Using the Section Expert you can:

- Insert, delete, merge, and move report sections.
- Hide a section (keeping it from printing) while retaining drill down capabilities.
- Suppress a section (keeping it from printing and eliminating drill down capabilities).
- Control page orientation for each section.
- Print subtotals or group values only at the bottom of the page.
- Insert a page break before the section is printed.
- Insert a page break after the section is printed.
- Insert a page break after a specific number of records.

- Reset the page number to one (1) after a group value prints.
- Prevent page breaks from spreading data from a single record over two pages.
- Suppress empty space at the bottom of a page and prevent blank lines from printing.
- Cause an object to overlay the following section when it prints.
- Format the section with multi-column capabilities.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Working with sections \[page 218\]](#)

[Formatting \[page 235\]](#)

[Section Expert \[page 1449\]](#)

37.8.7 Group Expert command

Use the Group Expert command to change the sorting and grouping specifications (sort and group by field, sort direction, and so on) for any of the groups in your report. You can also use this command to insert and remove groups and to change their processing order. If you insert a group section, a subtotal, or a summary and later want to modify it in any way, use this command to do so.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Sorting, Grouping, and Totaling \[page 167\]](#)

[Grouping data \[page 173\]](#)

[Change Group Options dialog box \[page 1162\]](#)

37.8.8 Group Sort Expert command

Use the Group Sort Expert command to set up reports that print only the Top N or Bottom N groups (N being a number you specify). Before you can use the [Group Sort Expert \[page 1429\]](#), your data must first be subtotaled or summarized. You can also use this command to print Top or Bottom percentages. Top or Bottom groups can also be used with Cross-Tabs.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Sorting, Grouping, and Totaling \[page 167\]](#)

[Grouping data \[page 173\]](#)

[Group Sort Expert \[page 1429\]](#)

37.8.9 Record Sort Expert command

Use the Record Sort Expert command to change the order in which records appear in your report. You can add and remove sort fields and you can define the sort order (ascending or descending) for the data in your report. You cannot, however, remove group sorts with this command.

When you choose this command, the [Record Sort Expert \[page 1446\]](#) appears.

Note

This command is also available as a button on the [Expert Tools toolbar \[page 1149\]](#).

Related Information

[Sorting, Grouping, and Totaling \[page 167\]](#)

37.8.10 Template Expert command

Use the Template Expert command to apply one of several professionally-designed styles to your report.

When you choose the Template Expert command, the [Template Expert \[page 1459\]](#) appears. Use this dialog box to select a report style and apply it to your report.

Related Information

[Formatting \[page 235\]](#)

37.8.11 OLAP Design Wizard command

When you choose this command, the [OLAP Expert \[page 1442\]](#) appears. Use the expert to change the settings of your OLAP grid.

Note

Once you change your grid settings, the expert replaces the OLAP grid in your report with one based on the new settings.

37.8.12 Hierarchical Grouping Options command

The Hierarchical Grouping Options command enables you to group data in a report to show hierarchical relationships. When you group data hierarchically, you sort information based on the relationship between two fields.

When you choose this command, the [Hierarchical Group Options dialog box \[page 1285\]](#) appears.

Related Information

[Grouping data hierarchically \[page 185\]](#)

37.8.13 Show Current Parameter Values command

The Show Current Parameter Values command lets you see all parameter values in the current report.

When you choose this command, the [Current Parameter Values dialog box \[page 1189\]](#) appears.

Related Information

[Parameter Fields and Prompts \[page 831\]](#)

37.8.14 Check Dependencies command

The Check Dependencies command lets you check the report that you currently have open for problems related to report part hyperlinks, repository objects, and formula compilation.

When you choose this command, the [Dependency Checker dialog box \[page 1223\]](#) appears.

Related Information

[The Dependency Checker \[page 116\]](#)

37.8.15 Refresh Report Data command

Use the Refresh Report Data command to retrieve fresh data for your report when you display it in the Preview Tab.

When you refresh a report that doesn't have saved data, Crystal Reports retrieves data from the database; otherwise, the program discards the report's saved data and retrieves new data.

Note

Refreshing a report's data does not necessarily refresh a subreport's data at the same time. Crystal Reports generates subreport data only when the subreport is encountered. For example, if a subreport is located on the second page of your report, the data for that subreport is not retrieved until you view the second page.

By design, Crystal Reports only retrieves data when necessary. When you preview your report in the Preview tab, print the report, or export it to a file, the program runs your report and retrieves the required data. Once you are working with the data in the Preview tab, the program will run the report again and retrieve fresh data if you:

- Add fields to the report after the report was run.
- Add a formula that references a field that was not in the report when you ran it the first time.
- Expand your record selection criteria to include more records than were needed when you first ran the report.

Aside from these circumstances, the program will not automatically refresh a report.

However, there may be times when you want to retrieve new data for the report. For example, you may have created a report several days earlier and you want to see it run with the most current data. The Refresh Report Data command is intended for times such as this.

Note

This command is also available as a button on the [Navigation Tools toolbar \[page 1150\]](#).

Related Information

[Refreshing report data \[page 857\]](#)

37.8.16 Set Print Date and Time command

Use the Set Print Date and Time command if your report contains a Print Date field that you would like to change. This enables you, for example, to post-date a report that you need to print prior to the date when it will be reviewed in a meeting or conference.

37.8.17 Performance Information command

Use the Performance Information command to open the [Performance Information dialog box \[page 1339\]](#). This dialog box displays detailed information about your report, which helps you make decisions about possible performance enhancements.

37.9 Chart menu

The Chart menu appears when you select a chart in a report. The menu includes commands used to customize your chart.

Depending on the type of chart you select, the Chart menu includes commands for zooming in and out on bar and line charts, applying changes to all instances of the chart, discarding any custom changes made to the chart, saving the chart template to a file, applying a new template to the chart, changing the template specifications of the chart, changing the chart titles, changing the numeric axis grids and scales of the chart, and auto-arranging the appearance of the chart.

37.9.1 Format Background command (Chart menu)

Use the Format Background command to format the fill (background color, texture, and so on) and border specifications of your chart.

37.9.2 Chart Options command (Chart menu)

Use the Chart Options command to change the template specifications of your chart. You can change the appearance of your chart, as well as its title and legend when you use this command.

37.9.3 Axis Options command (Chart menu)

Use the Axis Options command to change the numeric axis grids and scales of your chart.

37.9.4 Series Options command (Chart menu)

Use the Series Options command to show how you want the data to be presented on your chart, and to include data labels, numbers, and trendlines with your chart.

37.9.5 3D Viewing Angle command (Chart menu)

Use the 3D Viewing Angle command to open the Choose a Viewing Angle dialog box. From here, you can make viewing adjustments to your 3D chart, including options to rotate, pan, move, and define the length and thickness of the frame walls surrounding your 3D chart.

In the Advanced Options section of the Choose a Viewing Angle dialog box, use the features on the Rotate tab to rotate your 3D chart to the specified angle.

In the Advanced Options section of the Choose a Viewing Angle dialog box, use the features on the Pan tab to pan and zoom your 3D chart inside the chart frame.

In the Advanced Options section of the Choose a Viewing Angle dialog box, use the features on the Walls tab to increase or decrease the length and thickness of the frame walls surrounding your 3D chart.

In the Advanced Options section of the Choose a Viewing Angle dialog box, use the features on the Move tab to move the 3D chart within the chart frame.

37.9.6 Edit Axis Label command (Chart menu)

Use the Edit Axis Label command to open the [Label Aliasing dialog box \[page 1299\]](#). Use this dialog box to create an alias for the selected axis label in your chart.

This command is available only if you have chosen to show labels for the data points on your chart, and if you have selected one of these labels. See [Options tab \(Chart Expert\) \[page 1408\]](#).

37.9.7 Trendlines command (Chart menu)

Use the Trendlines command to fit one or more statistical lines or curves to a series of data on the chart.

❗ Note

This command is only available when you select a data series on a chart, by clicking one of the data items for the series.

Choosing the Trendlines command opens the [Trendlines dialog box \[page 1375\]](#) for you to select the type of curve to display for the chosen data series.

You can set the color of the curves for a data series to the color used for that data series on the chart. (If you select this option for a chart that colors each item differently, such as a bar chart, the color for the first data item is used.)

If you do not select Show Color Same as Series for a data series, you can set the color for its curves selecting one of them and using the Change Color list.

37.9.8 Auto-Arrange Chart command (Chart menu)

When moving or resizing a chart object, select the Auto-Arrange Chart command from the Chart menu to reset your chart with the latest changes.

37.9.9 Apply Changes to All Charts command (Chart menu)

Each change made to your chart is instance-specific. For example, if you rename the title of your selected chart, the title change applies to only that chart instance. However, if you want to apply the title change to all chart instances, select the Apply Changes to All Charts command from the Chart menu.

37.9.10 Discard Custom Changes command (Chart menu)

Select the Discard Custom Changes command to restore the chart that you have customized back to its original appearance (as specified in the [Chart Expert \[page 1400\]](#)).

This command is available only if you have altered the original settings.

37.9.11 Load Template command (Chart menu)

Use the Load Template command to apply a new template to your chart.

Note

You must have saved your chart template to the directory for user-defined templates in order for it to appear in the "Choose a Chart type" dialog box. By default, the designated directory is located at:

`\Documents and Settings\<Profile Name>\Application Data\Business Objects\Crystal Reports\ChartSupport\Templates\User Defined`

37.9.12 Save as Template command (Chart menu)

Use the Save as Template command to save the selected chart as a template. To apply a template to a chart, select the [Load Template command \(Chart menu\) \[page 1123\]](#) from the Chart menu.

Note

You must save your chart template to the directory for user-defined templates, or you won't be able to apply the template later. By default, the designated directory is located at:

`\Documents and Settings\<Profile Name>\Application Data\Business Objects\Crystal Reports\ChartSupport\Templates\User Defined`

37.9.13 Select Mode command (Chart menu)

Use the Select Mode command to disable all movement options. The Select Mode command prevents accidental zooming or panning of a chart or map.

37.9.14 Zoom In command (Chart menu)

Select the Zoom In command to get a closer view of your chart or map. Once you activate the Zoom In mode, your cursor changes to represent a magnifying glass. The magnifying glass includes a plus sign to inform you that the selected area increases in size as you click the chart or map.

37.9.15 Zoom Out command (Chart menu)

Select the Zoom Out command to get a more distant view of your chart or map. Once you activate the Zoom Out mode, your cursor changes to represent a magnifying glass. The magnifying glass includes a minus sign to inform you that the selected area decreases in size as you click the chart or map.

37.9.16 Pan command (Chart menu)

Select the Pan command to move through the data in your chart or map:

- When you activate this command for a chart, you can click and drag through the chart data to the desired location.
- When you activate this command for a map, you can click and drag the map to the desired position on your screen.

37.10 Map menu

The Map menu appears when you select a map in a report. The menu includes commands used to customize your map.

Depending on the type of map you select, the Map menu includes commands for zooming, panning, and centering maps; changing the map title, map type, and setting the overall style of the map; reorganizing the layers of report elements (for example, you can display the Major Cities layer above the Highways layer); resolving data mismatches; changing the geographic map; and hiding and showing the Map Navigator.

37.10.1 Select Mode command (Map menu)

Use the Select Mode command to disable all movement options. The Select Mode command prevents accidental zooming or panning of a chart or map.

37.10.2 Zoom In command (Map menu)

Select the Zoom In command to get a closer view of your chart or map. Once you activate the Zoom In mode, your cursor changes to represent a magnifying glass. The magnifying glass includes a plus sign to inform you that the selected area increases in size as you click the chart or map.

37.10.3 Zoom Out command (Map menu)

Select the Zoom Out command to get a more distant view of your chart or map. Once you activate the Zoom Out mode, your cursor changes to represent a magnifying glass. The magnifying glass includes a minus sign to inform you that the selected area decreases in size as you click the chart or map.

37.10.4 Pan command (Map menu)

Select the Pan command to move through the data in your chart or map:

- When you activate this command for a chart, you can click and drag through the chart data to the desired location.
- When you activate this command for a map, you can click and drag the map to the desired position on your screen.

37.10.5 Center Map command (Map menu)

Use the Center Map command to center the map in its frame on the report. This is useful to restore the map to its original position after panning.

37.10.6 Title command (Map menu)

Use the Title command to change the name of your map. This command opens the [Change Map Title dialog box \[page 1164\]](#).

Related Information

[Changing the map title \[page 329\]](#)

37.10.7 Type command (Map menu)

Use the Type command to open the [Customize Map dialog box \[page 1204\]](#) and change the map type and properties for that map. For example, if you are analyzing a Ranged map, and you want to see how the data would look if it were presented in a Dot Density style, you can rearrange the map without having to return to the Map Expert and rework the format.

Related Information

[Changing the map type \[page 329\]](#)

37.10.8 Layers command (Map menu)

Use the Layers command to open the [Layer Control dialog box \[page 1300\]](#). Use this dialog box to rearrange the layers of your map. For example, you can rearrange the layers so that the Major Cities layer appears above the Highway layer, or so that the Capitals layer appears above the Top 20 Cities layer.

Related Information

[Changing map layers \[page 329\]](#)

37.10.9 Resolve Mismatch command (Map menu)

Sometimes the map you are working with uses a different geographic name from the one used in the database. For example, a map of Europe may use "United Kingdom" while the database uses the abbreviation "U.K." Until

you resolve the data mismatch, the map will not be able to display any information for the selected geographic region.

Select the Resolve Mismatch command to open the [Resolve Map Mismatch dialog box \[page 1354\]](#). In this dialog box, you can change the type of map used in your report, or you can see a list of the geographic names that do not correspond between the database and map.

Note

This command is available only when you are working with a map on the Preview tab.

Related Information

[Resolving data mismatches \[page 331\]](#)

37.10.10 Map Navigator command (Map menu)

Use the Map Navigator command to show or hide the Map Navigator. The Map Navigator provides a small-scale version of the currently displayed map. It includes commands for zooming, panning, and centering geographic maps. You can zoom in and out on your maps whenever you need to view more specific or general information. By panning, you can quickly drag your map anywhere you please. You can then return your map to the center of the tab by using the center command.

37.11 Window menu

The Window menu includes commands used to rearrange icons and windows. It also lists the report windows that are open and includes a command that lets you close all report windows at once.

37.11.1 Tile Vertically command

Use the Tile Vertically command to display your windows side by side on screen. With this command, you can resize the windows so they all fit on screen; all windows are visible.

37.11.2 Tile Horizontally command

Use the Tile Horizontally command to display your windows one above the other on screen. With this command, you can resize the windows so they all fit on screen; all windows are visible.

37.11.3 Cascade command

Use the Cascade command to stack and overlap your windows. When you choose this command, the entire top window is visible but only the title bars of the remaining windows are visible.

37.11.4 Arrange Icons command

If you have minimized any document windows, use the Arrange Icons command to arrange the document icons neatly at the bottom of the application window.

37.11.5 Close All command

Use the Close All command to close all open windows. Crystal Reports gives you an opportunity to save any unsaved work in the open windows.

When you choose the Close All command:

- If you have saved all your reports, the program closes all the windows.
- If you have not saved all your reports, the program prompts you to save them, report by report:
 - Click Yes to save the report before closing the window.
 - Click No to close the window without saving the report.

37.11.6 Active Report command

Use the Active Report command to select the report that you want to work with in Crystal Reports.

37.12 Help menu

The Help menu includes commands used to access Crystal Reports online help system. It has commands for registering Crystal Reports, accessing technical information about your computer system, and creating a technical support request. You can switch the [Start Page \[page 1367\]](#) on and off, and if connected to the Internet, you can access several key pages of the Business Objects web site using commands on the Help menu. One other command provides information about the version you are using.

37.12.1 Crystal Reports Help command

Use the Crystal Reports Help command to start the online help system.

37.12.2 Context Help command

Use the Context Help command to activate the context help pointer that you can use to identify elements of the interface.

Note

This command is also available as a button on the [Standard toolbar \[page 1145\]](#).

37.12.3 Show Start Page command

Use the Show Start Page command to display the [Start Page \[page 1367\]](#) where you can choose from a number of activities:

- Open a recently opened report.
- Open an existing report.
- Create a new report using a report wizard.
- Create a new report as a blank report.
- Search the index of the help system.
- See the sample reports.
- Check for updates to Crystal Reports.
- Read a selection of online articles and content about Crystal Reports and other products from Business Objects.

37.12.4 Check for Updates on Start Up command

Select the Check for Updates on Start Up option if you want the Business Objects Update Service dialog box to appear each time you open Crystal Reports.

Related Information

[Check for Updates command \[page 1129\]](#)

37.12.5 Check for Updates command

Select the Check for Updates option if you want Crystal Reports to check for product updates so that you can install them. When you choose this option, the Business Objects Update Service dialog box appears. Use this dialog box to see the updates that are available before you install them.

Related Information

[Check for Updates on Start Up command \[page 1129\]](#)

37.12.6 Register or Change Address command

Use the Register or Change Address command to register and/or notify the company of a change of address.

Note

You must send in the registration form in order to receive technical support for your product.

37.12.7 License Manager command

Use the License Manager command to start the License Manager—a tool that lets you manage your product activation keycodes.

37.12.8 About Crystal Reports command

Use the About Crystal Reports command to learn the version number and other pertinent information about Crystal Reports.

37.13 Shortcut menus

Everywhere in Crystal Reports you can display shortcut menus, by clicking the right mouse button. These menus enable you to work more quickly by providing quick access to the commands that are relevant to what you are doing at the time. For example if you have a text object selected right-clicking brings up a shortcut menu listing all the text editing and formatting commands.

This section lists, in alphabetical order, the commands that are unique to the shortcut menus.

37.13.1 Add Calculated Member command

Use the Add Calculated Member command to open the [Calculated Members dialog box \[page 1158\]](#). Use this dialog box to add a member that calculates Contribution (%), Growth, Variance. You can also define your own calculation with MDX query language.

37.13.2 Add Current Report command

Use the Add Current Report command to add your active report to the [Workbench dialog box \[page 1377\]](#). The program adds the report to the selected folder.

37.13.3 Add Filter command

Use the Add Filter command to open the [Define Filter dialog box \[page 1221\]](#). Use this dialog box to create a filter that removes data you don't want to see in your OLAP grid.

37.13.4 Add First Sort submenu

Click a link to view one of the topics in this section:

- [Ascending command \(Add First Sort submenu\) \[page 1131\]](#)
- [Descending command \(Add First Sort submenu\) \[page 1131\]](#)
- [Ascending, Break Hierarchies command \(Add First Sort submenu\) \[page 1132\]](#)
- [Descending, Break Hierarchies command \(Add First Sort submenu\) \[page 1132\]](#)

Related Information

[Sorting data in an OLAP grid \[page 408\]](#)

37.13.4.1 Ascending command (Add First Sort submenu)

Use the Ascending command to sort data values into ascending numerical order. Data hierarchy is preserved for the first sort.

37.13.4.2 Descending command (Add First Sort submenu)

Use the Descending command to sort data values into descending numerical order. Data hierarchy is preserved for the first sort.

37.13.4.3 Ascending, Break Hierarchies command (Add First Sort submenu)

Use the Ascending, Break Hierarchies command to sort data values into ascending numerical order without preserving data hierarchy.

37.13.4.4 Descending, Break Hierarchies command (Add First Sort submenu)

Use the Descending, Break Hierarchies command to sort data values into descending numerical order without preserving data hierarchy.

37.13.5 Add to Repository command

Use the [Add to Repository](#) command to open the [Add Item dialog box \[page 1155\]](#) so you can add the object that you have selected to the SAP BusinessObjects Enterprise Repository.

ⓘ Note

You might need to connect to SAP BusinessObjects Business Intelligence platform before you can access the repository. For information about connecting to SAP BusinessObjects Business Intelligence platform, see [Working with Enterprise folders \[page 628\]](#).

ⓘ Note

For information about how to use the repository, see [SAP BusinessObjects Enterprise Repository \[page 120\]](#).

37.13.6 Arrange Lines command

Use the Arrange Lines command to evenly distribute guidelines in a report section.

37.13.7 Bind Sort Control command

Use the [Bind Sort Control](#) command to open the [Sort Control](#) dialog box. The dialog box allows you to add, remove, and modify sort controls.

37.13.8 Break Hierarchies command

Use the Break Hierarchies command to remove subsequent sorts from OLAP data values and change the current sort into a first sort.

37.13.9 Calculated Member command

Use the *Calculated Member* command to open the *Cross-Tab Calculated Member Expert*. The *Cross-Tab Calculated Member Expert* allows you to add, remove, and modify *Calculated Members*. You can also use it to specify the order in which *Calculated Members* are processed. The order can have an important effect on the results of your formulas.

Related Information

[Calculated Members \[page 369\]](#)

[Cross-Tab Calculated Members Expert \[page 1419\]](#)

37.13.10 Cancel Menu command

Use the Cancel Menu command to cancel the shortcut menu.

37.13.11 Change Caption command

Use the Change Caption command to change the caption name in a row or column of an OLAP grid. You can also use this command to reset a caption to its default value.

When you choose this command, the [Change Caption dialog box \[page 1162\]](#) appears.

37.13.12 Change Direction of Sort command

Use the Change Direction of Sort command to re-sort data values in an OLAP grid.

Related Information

[Changing the sort order \[page 450\]](#)

37.13.13 Change Group command

Use the Change Group command to open the [Change Group Options dialog box \[page 1162\]](#). Use this dialog box to modify an existing report group.

37.13.14 Change to Barcode command

Use the Change to Barcode command to open the [Select Barcode Type dialog box \[page 1358\]](#) so that you can convert the field value to a barcode.

37.13.15 Collapse Member command

Use the Collapse Member command to collapse the visible hierarchy of an OLAP grid member that has previously been expanded.

37.13.16 Column Grand Totals submenu

These options apply to cross-tab objects.

37.13.16.1 Totals on Left command (Column Grand Totals submenu)

Use the Totals on Left command to ensure that column grand totals appear on the left of the cross-tab rather than on the right (the default position).

37.13.16.2 Suppress Grand Totals command (Column Grand Totals submenu)

Use the Suppress Grand Totals command to have the program remove the grand totals from columns in the cross-tab.

37.13.17 Column Options submenu

These options apply to cross-tab objects.

37.13.17.1 Keep Columns Together command (Column Options submenu)

Use the Keep Columns Together command to have the program keep columns together on every page for cross-tabs that print across more than one page.

37.13.17.2 Suppress Empty Columns command (Column Options submenu)

Use the Suppress Empty Columns command to have the program remove empty columns from the cross-tab. When you select this option, any columns whose fields do not contain any records will be hidden.

37.13.17.3 Indent Labels command (Column Options submenu)

Use the Indent Labels command to indent column labels on your cross-tab.

37.13.17.4 Group Options command (Column Options submenu)

Use the Group Options command to open the [Cross-Tab Group Options dialog box \[page 1189\]](#). Use the Cross-Tab Group Options dialog box to create or modify group options for any column in your cross-tab.

37.13.18 Copy Smart Tag command

Use the Copy Smart Tag command to copy an object from Crystal Reports to the clipboard before you paste it into an Office XP application. For information about what objects you can use as smart tag objects, see [Using smart tags \[page 645\]](#).

37.13.19 Delete Group command

Use the Delete Group command to remove an existing report group.

37.13.20 Delete Last Line command

Use the Delete Last Line command to remove the last guideline you added to a report section.

37.13.21 Delete Section command

Use the Delete Section command to remove the report section you have selected.

Note

You can delete only additional sections that you added with the [Insert Section Below command \[page 1140\]](#) or in the [Section Expert \[page 1449\]](#).

37.13.22 Disconnect from Repository command

Use the Disconnect from Repository command to log off the SAP BusinessObjects Enterprise Repository after you have connected to it.

Related Information

[Working with Enterprise folders \[page 628\]](#)

37.13.23 Drill-down command

Use the Drill-down command in a report's Group Tree to open a drill-down view for the selected group.

Related Information

[Group Tree \[page 95\]](#)

[Drilling-down on group headers \[page 206\]](#)

37.13.24 Duplicate command

Use the Duplicate command to make a copy of the formula that you have selected in the [Field Explorer dialog box \[page 1241\]](#). When the copy is created, you can rename it and edit it through the Field Explorer.

37.13.25 Edit Filter command

Use the Edit Filter command to open the [Define Filter dialog box \[page 1221\]](#). Use this dialog box to modify an existing data filter in your OLAP grid.

37.13.26 Embedded Summary command

Use the *Embedded Summary* command to open the *Embedded Summaries* dialog box. The *Embedded Summaries* dialog box allows you to add, remove, and modify *Embedded Summaries*. You can also use it to specify the order in which *Embedded Summaries* are processed. The order can have an important effect on the results of your formulas.

Related Information

[Embedded Summaries \[page 373\]](#)

[Embedded Summaries dialog box \[page 1228\]](#)

37.13.27 Expand Member command

Use the Expand Member command to display the underlying data hierarchy of an OLAP grid member.

37.13.28 Expert command

Use the Expert command to show or hide the Expert Tools toolbar.

37.13.29 Find in Field Explorer command

Use the Find in Field Explorer command to find the selected field in the [Field Explorer dialog box \[page 1241\]](#).

37.13.30 Find in Formulas command

Use the Find in Formulas command to find all of the formulas in the current report that reference the selected field. The program opens the [Formula Workshop dialog box \[page 1282\]](#), which shows a list of formulas that you can select from.

37.13.31 Find in Grouping condition command

Use the [Find in Grouping condition](#) command to find all the grouping conditions in the current report that reference the selected field. If found, the program opens the [Group Expert \[page 1428\]](#) dialog box, which shows the usage of selected field.

37.13.32 Find in Sorting condition command

Use the [Find in Sorting condition](#) command to find all the sorting conditions in the current report that reference the selected field. If found, the program opens the [Record Sort Expert \[page 1446\]](#) dialog box, which shows the usage of selected field.

37.13.33 Find in Subreport links command

Use the [Find in Subreport links](#) command to find all the Subreport links in the current report that reference the selected field. If found, the program opens the [Subreport Links dialog box \[page 1369\]](#), which shows the usage of selected field.

37.13.34 Fit Section command

Use the Fit Section command to resizes a report section, moving the bottom boundary of the section to the baseline of the bottom object in the section.

Related Information

[Resizing a section \[page 222\]](#)

37.13.35 Formatting command

Use the Formatting command to show or hide the Formatting toolbar.

37.13.36 Go to Hyperlink command

Use the Go to Hyperlink command to open the web site or link associated with a specific report object.

37.13.37 Grid Options submenu

These options apply to cross-tab objects.

37.13.37.1 Show Cell Margins command (Grid Options submenu)

Use the Show Cell Margins command to have the program automatically format the cells of the cross-tab. When this option is not selected, any extra space (white space) within the cells of the cross-tab is eliminated, and the contents of the cells are adjusted to use the least amount of space in your report as possible.

37.13.37.2 Format Grid Lines command (Grid Options submenu)

Use the Format Grid Lines command to open the [Format Grid Lines dialog box \[page 1278\]](#). In this dialog box, you can define whether the grid lines are shown or not, as well as the color, style, and width you'd like the grid lines to have.

37.13.38 Hide command

Use the Hide command to close a report's Group Tree.

Related Information

[Group Tree \[page 95\]](#)

37.13.39 Hide (Drill-Down OK) command

Use the Hide (Drill-Down OK) command if you do not want a report section to be displayed, but you want the section contents to be available for drill-down.

Related Information

[Drilling-down on group headers \[page 206\]](#)

37.13.40 Insert command

Use the Insert command to show or hide the Insert Tools toolbar.

37.13.41 Insert Line command

Use the Insert Line command to add a guideline to a report section.

37.13.42 Insert Section Below command

Use the Insert Section Below command to insert a new report section immediately below and of the same type as the section you have selected. For example, if you have a Details section selected and select this command, the program will insert a new Details section immediately below the selected section.

37.13.43 Merge Section Below command

Use the Merge Section Below command to merge the current report section with the one immediately below it.

Note

You can merge only sections of the same type as the section you have selected. See [Merging two related sections \[page 220\]](#) for more information.

37.13.44 Remove All Horizontal Guidelines command

Use the Remove All Horizontal Guidelines command to remove all horizontal guidelines from your report view.

37.13.45 Remove All Vertical Guidelines command

Use the Remove All Vertical Guidelines command to remove all vertical guidelines from your report view.

37.13.46 Remove Filter command

Use the Remove Filter command to remove a data filter you created in your OLAP grid.

37.13.47 Remove Sort command

Use the Remove Sort command to remove a data sort you created in your OLAP grid.

37.13.48 Reorder Displayed Members command

Use the Reorder Displayed Members command to open the [Reorder Displayed Members dialog box \[page 1343\]](#). Use this dialog box to customize the order in which the dimension members appear in your report's OLAP grid.

37.13.49 Row Grand Totals submenu

These options apply to cross-tab objects.

37.13.49.1 Totals on Top command (Row Grand Totals submenu)

Use the Totals on Top command to ensure that row grand totals appear on the top of the cross-tab rather than on the bottom (the default position).

37.13.49.2 Suppress Grand Totals command (Row Grand Totals submenu)

Use the Suppress Grand Totals command to have the program remove the grand totals from rows in the cross-tab.

37.13.50 Row Options submenu

These options apply to cross-tab objects.

37.13.50.1 Repeat Labels on Page Break command (Row Options submenu)

Use the Repeat Labels on Page Break command to have the program repeat row labels on every page for cross-tabs that print across more than one page.

37.13.50.2 Suppress Empty Rows command (Row Options submenu)

Use the Suppress Empty Rows command to have the program remove empty rows from the cross-tab. When this check box is selected, any rows whose fields do not contain any records will be hidden.

37.13.50.3 Indent Labels command (Row Options submenu)

Use the Indent Labels command to indent row labels on your cross-tab.

37.13.50.4 Group Options command (Row Options submenu)

Use the Group Options command to open the [Cross-Tab Group Options dialog box \[page 1189\]](#). Use the Cross-Tab Group Options dialog box to create or modify group options for any row in your cross-tab.

37.13.51 Select All Section Objects command

Use the Select All Section Objects command to select all report objects located in the section with which you are working.

37.13.52 Set Prompt Order command

Use the Set Prompt Order command to open the [Set Prompt Order dialog box \[page 1364\]](#) so that you can rearrange the order of the prompts in your report.

37.13.53 Show Hidden Sections in Design command

Use the Show Hidden Sections in Design command to display sections to which you've applied [Hide \(Drill-Down OK\)](#) or [Suppress \(No Drill-Down\)](#) as shaded areas in your report's Design view.

Note

If you do not select this command, hidden or suppressed sections appear collapsed on the Design tab and you aren't able to see their contents.

37.13.54 Show Section Names command

On the Design tab, this command can appear as Show Short Section Names or Show Long Section names depending on the option selected on the [Layout tab \(Options dialog box\)](#) [page 1315].

On the Preview tab, this command can appear as Show Section Names or Hide Section Names.

Use this command to display the report's section names appropriately.

37.13.55 Snap to Grid command

Use the Snap to Grid command to have new database fields you insert snap to the nearest grid positions. When you don't use this command, new database fields are left where you place them on your report.

37.13.56 Sort Fields Alphabetically command

Use the Sort Fields Alphabetically command to display table fields in alphabetical order (instead of the order they appear in the table).

37.13.57 Sort Tables Alphabetically command

Use the Sort Tables Alphabetically command to display database tables in alphabetical order (instead of the order they appear in the database).

37.13.58 Standard command

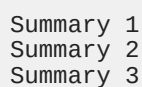
Use the Standard command to show or hide the Standard toolbar.

37.13.59 Summarized Field Labels submenu

These options apply to cross-tab objects.

37.13.59.1 Summarize Horizontally command (Summarized Field Labels submenu)

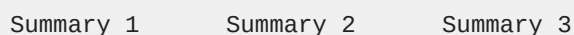
Use the Summarize Horizontally command to have the program display multiple summarized fields in a vertical list:



```
Summary 1  
Summary 2  
Summary 3
```

37.13.59.2 Summarize Vertically command (Summarized Field Labels submenu)

Use the Summarize Vertically command to have the program display multiple summarized fields in a horizontal list:



```
Summary 1      Summary 2      Summary 3
```

37.13.59.3 Show Summarized Field Labels command (Summarized Field Labels submenu)

Use the Show Summarized Field Labels command to display the names of multiple summarized fields. Labels can be displayed for both vertical and horizontal summaries. These labels can also be modified by editing them as you would any text field.

37.13.60 Suppress (No Drill-Down) command

Use the Suppress (No Drill-Down) command if you do not want the section to be displayed, and you do not want the section contents to be available for drill-down.

37.13.61 View Cube command

Use the View Cube command to open the Cube View of your report's OLAP grid. Use the worksheet on the Cube View to carry out advanced analysis of your OLAP grid data.

Related Information

[Using the OLAP Worksheet \[page 412\]](#)

37.14 Standard toolbar

The standard toolbar contains several commonly used commands. You can switch the toolbars on and off using the [Toolbars command \[page 1089\]](#), or from the shortcut menu that appears when you right-click the toolbar or menu area.

If you are not sure about the function of a button on the Standard toolbar, place the pointer over the button and hold it there until the tool tip appears.

The buttons on the Standard toolbar perform the following functions:

	Creates a new report. Shortcut to the New submenu [page 1071] .
	Opens an existing report. Shortcut to the Open command [page 1072] .
	Saves the report. Shortcut to the Save command [page 1073] .
	Sends the report to a printer. Shortcut to the Print command [page 1074] .
	Displays a preview of the report on the Preview tab. Shortcut to the Print Preview command [page 1086] .
	Displays a preview of the report in HTML format on the HTML Preview tab. Shortcut to the HTML Preview command [page 1087] .
	Exports the report in one of several popular formats. You can export to a disk file, to an application, to a Lotus Notes database, to an Exchange Folder, or to email. Shortcut to the Page Setup command [page 1075] .

	Cuts selected object/data to the clipboard. Shortcut to the Cut command [page 1080] .
	Copies the selected object/data to the clipboard. Shortcut to the Copy command [page 1080] .
	Pastes object/data from the clipboard to the report. Shortcut to the Paste command [page 1080] .
	Lets you copy absolute or conditional formatting properties from one report object or field to one or more target objects or fields. Shortcut to the Format Painter command [page 1102] .
	Undoes an action. Shortcut to the Undo command [page 1079] . Use the drop down list to undo more than one action.
	Redoes the latest action that was undone. Shortcut to the Redo command [page 1080] . Use the drop down list to redo more than one action.
	Toggles the Preview Panel on and off on the Preview Tab. Shortcut to the Preview Panel command [page 1090] on the View menu.
	Opens the Field Explorer for you to insert database fields. Shortcut to the Field Explorer command [page 1087] on the View menu.
	Opens the Report Explorer for you to see the contents of your report in tree view. Shortcut to the Report Explorer command [page 1088] on the View menu.
	Opens the Repository Explorer for you to see the contents of your repository. Shortcut to the Repository Explorer command [page 1088] on the View menu.
	Opens the Dependency Checker for you to see the results of running the Check Dependencies command [page 1119] . Shortcut to the Dependency Checker command [page 1088] on the View menu.
	Opens the Workbench for you to see the contents of your projects. Shortcut to the Workbench command [page 1089] on the View menu.
	Finds a specific record. Shortcut to the Find command [page 1081] .

37.15 Formatting toolbar

The Formatting toolbar contains commands for formatting the selected field.

Use the lists to the left of the Formatting toolbar to change the font and font size of the selected field.



The buttons on the Formatting toolbar perform the following functions:

Regional Settings area

The program refers to the settings in the Regional Settings section of the Control Panel.

	When a number field is selected, places a currency symbol with the number.
	When a number field is selected, places a thousands separator in the number.
	When a number field is selected, places a percentage sign with the number.
	When a number field is selected, adds one decimal place to the number.
	When a number field is selected, removes one decimal place from the number.

Other formatting options

	Increases the font size of the selected data one point each time the button is clicked.
	Decreases the font size of the selected data one point each time the button is clicked.
	Changes the selected data to boldface.
	Italicizes the selected data.
	Underlines the selected data.
	Aligns the selected data flush left.

	Centers the selected data.
	Aligns the selected data flush right.
	Fully justifies the selected data.
	Applies the chosen color to the selected data. This button allows you to choose or define colors in the color dialog box.
	Applies the chosen border to the selected data. You can choose from six border configurations in the adjacent list.
	Suppresses the selected field or report object.
	Locks the formatting of the selected data so it can't be changed accidentally.
	Locks the size and position of the selected field or report object in relation to another object to its right.

37.16 Insert Tools toolbar

The Insert Tools toolbar provides quick access to many advanced, report-enhancing features.

The buttons on the Insert Tools toolbar perform the following functions:

	Inserts a text object. Shortcut to the Text Object command [page 1094] on the Insert menu.
	Inserts a group. Shortcut to the Group command [page 1095] on the Insert menu.
	Inserts a summary. Shortcut to the Summary command [page 1094] on the Insert menu.
	Inserts a cross-tab object. Shortcut to the Cross-Tab command [page 1096] on the Insert menu.
	Inserts a OLAP grid object. Shortcut to the OLAP Grid command [page 1096] on the Insert menu.
	Inserts a subreport. Shortcut to the Subreport command [page 1096] on the Insert menu.
	Inserts a line. Shortcut to the Line command [page 1097] on the Insert menu.

	Inserts a box. Shortcut to the Box command [page 1098] on the Insert menu.
	Inserts a picture. Shortcut to the Picture command [page 1098] on the Insert menu.
	Inserts a chart. Shortcut to the Chart command [page 1099] on the Insert menu
	Inserts a map. Shortcut to the Map command [page 1099] on the Insert menu
	Inserts a flash object. Shortcut to the Flash command on the Insert menu.

37.17 Expert Tools toolbar

The Expert Tools toolbar provides quick access to many advanced experts.

The buttons on the Expert Tools toolbar perform the following functions:

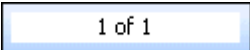
	Opens the Database Expert [page 1420] . Select a data source and tables on the Data tab. Click the Links tab to define links between tables. Shortcut to the Database Expert command [page 1108] .
	Opens the Group Expert [page 1428] . Use the expert to create, modify, and delete groups. Shortcut to the Group Expert command [page 1117] .
	Finds Top or Bottom N records or sorts on summary information. Shortcut to the Group Sort Expert command [page 1117] .
	Sets record sort order. Shortcut to the Record Sort Expert command [page 1118] .
	Activates the Select Expert [page 1457] for you to set record/group selection criteria. Shortcut to the Select Expert command [page 1112] .
	Activates the Section Expert [page 1449] for you to format any section of the report. Shortcut to the Section Expert command [page 1116] .

	Activates the Formula Workshop. Shortcut to the Formula Workshop command [page 1115].
	Activates the OLAP Report Creation Wizard [page 1382]. Shortcut to the OLAP Design Wizard command [page 1119].
	Activates the Template Expert [page 1459]. Shortcut to the Template Expert command [page 1118].
	Activates the appropriate editor to modify object properties. Shortcut to the Format command [page 1101].
	Lets you insert a hyperlink. Shortcut to the Hyperlink command [page 1102] on the Format menu.
	Activates the Highlighting Expert [page 1431]. Shortcut to the Highlighting Expert command [page 1103] on the Format menu.

37.18 Navigation Tools toolbar

The Navigation Tools toolbar provides buttons for refreshing your report's data and for moving around your report.

The buttons on the Navigation Tools toolbar perform the following functions:

	Refreshes report data. Shortcut to the Refresh Report Data command [page 1120].
	Stops gathering data from your data source after you have clicked Refresh.
	Goes to the first page in your report.
	Goes to the previous page in your report.
	Displays the number of the current page. Type a number and click Enter to go to that page.
	Goes to the next page in your report.
	Goes to the last page in your report.
	Returns to the previous page in your report preview (HTML Preview tab).



Goes to the next page in your report preview (HTML Preview tab).

37.19 SAP toolbar

Crystal Reports includes several integrated tools that allow you to report off your BW data. You can access many of these tools from two locations: click the appropriate button on the SAP Tools toolbar, or click the corresponding command on the SAP menu. This section describes the available commands and shows how to change the related settings in Crystal Reports.

→ Tip

To show/hide the SAP Tools toolbar in Crystal Reports, on the [View](#) menu, click [Toolbars](#). Select the [SAP Tools](#) check box, and click [OK](#). Commands without icons are available only on the SAP menu.

Command	Function
Create New Report from a Query	Create a new report based on a query that is already defined in BW.
Open Report	Open a Crystal report that has already been saved in BW.
Save Report	Save the report that is currently open to BW. You have the option to prepare the report for translation and publish it to SAP BusinessObjects Business Intelligence platform.
Save Report As	Save the report to BW with a different description and/or in a different role.
Start BW Query Designer	Launch the BW Query Designer to create a new query, or to modify a query that is already defined in BW.
Disconnect	Disconnect from SAP BW.
Settings	Change your connection settings.
SAP mode	Change a predefined SAP report into a regular Crystal report.

37.20 External Command toolbar

The [External Command](#) toolbar provides quick access to applications that are created by a third party. Once an application has been added to Crystal Reports, you can access it through an icon on this toolbar. Some applications, such as wizards, can also be accessed from the [Add-ins](#) menu.

Note

External DLLs can be registered with Crystal Reports by installing them in the Addins directory in the Business Objects install folder.

For more information about using this feature, and for help with creating applications, see the *Crystal Reports .Net Developers Guide*.

38 Dialog Box Reference

This section describes the dialog boxes available in Crystal Reports.

38.1 Add a New XML Format dialog box

Note

This dialog box may also be named *Modify an XML Exporting Format*.

Use this dialog box to add or modify the details of an existing schema to be used during report export.

Name

Enter a name to identify the export format.

Note

This name is used internally when choosing the report export format; it is not the file names of the XSLT file.

Include images and binary data fields (BLOBs)

Choose this check box to include binary data in the exported report format.

See XML Transform samples on the Crystal Reports Web Site

Click this link to open your web browser and view examples of XML transformations available on the Crystal Reports web site. You can choose any of these samples from the web site and use or modify them as you require.

Import XML Transform

Click this button to open the [Open dialog box \[page 1314\]](#) and select the transform to import into the report.

Save XML Transform As

Click this button to save the imported XML Transform with a different name or to a different location.

Note

You can only save formats that were imported; you cannot save the default schema.

Specify the file type of the output file format

This text indicates the extension of the file name to be used when the exported file is created. The default is XML.

The description of this export format

This optional text is used to display information about the report on the [Manage XML Exporting Formats dialog box \[page 1305\]](#).

38.2 Add Command to Report dialog box

The Add Command to Report dialog box appears when you double-click Add Command in the [Database Expert \[page 1420\]](#). The Add Command option is found in the connection folder for an active Database Files or ODBC (RDO) data source.

Use this dialog box to write your own SQL command (query) which will be represented in Crystal Reports as a Table object.

Note

This dialog box also appears as Modify Command or View Command when you select the appropriate option from the command's right-click menu in the Database Expert.

SQL Query

Enter the text of your SQL command in this box. See [Defining an SQL Command \[page 896\]](#) for more information.

Parameter List

Create, remove, or modify parameters for your SQL command in this box. Clicking Create or Modify opens the [Command Parameter dialog box \[page 1168\]](#).

38.3 Add Custom Function to Repository dialog box

The Add Custom Function to Repository dialog box appears when you select a Report Custom Function and click Add to Repository in the [Formula Workshop dialog box \[page 1282\]](#), or when you choose Add to Repository from the custom function's shortcut menu.

Use this dialog box to select the repository to store your custom function in. Custom functions stored in the repository can be used in other reports.

38.4 Add Item dialog box

The Add Item dialog box appears when you right-click a text object or bitmap and choose "Add to Repository," when you select the "Add to Repository" check box on the [Add Command to Report dialog box \[page 1154\]](#) and click OK, or when you select the "Add to Repository" option from the Actions list when you are working with a dynamic and cascading prompt.

Use this dialog box to describe the object you want to add and to choose the repository location.

Name

Enter a name for the repository object. This is a mandatory field; if you don't enter a name, the object won't be added to the repository.

Author

Enter the name of the person who is adding the object to the repository. This field is optional.

Description

Add an optional description that you can see when you're looking at the repository object's properties.

Location

Choose the repository you want to add the object to. If you double-click the root node of the repository in the Location area, you can see its folders and objects.

Related Information

[Properties dialog box \[page 1340\]](#)

[Object Information dialog box \[page 1310\]](#)

38.5 Add or Update Object dialog box

The Add or Update Object dialog box appears when you drag an object from your report to the same object in the [Repository Explorer dialog box \[page 1353\]](#).

Use this dialog box to indicate how you want the new version of the report object to be processed.

Update the report object in the repository

Select this option to replace the repository's object with the newer version from your report.

Add the report object to the repository

Select this option to add the new version of the report object as a unique repository object. When you click OK, the [Object Information dialog box \[page 1310\]](#) appears so you can define new information (including a new location) for the object.

38.6 Browse and Paste Data dialog box

This dialog box appears when you click the Browse Data button in the [Formula Editor dialog box \[page 1280\]](#).

Use the Browse and Paste Data dialog box to preview the data type, length, and content of any field and to paste a value into your formula or selection criteria.

- The source of the field (alias) and the field name appear as the title for the dialog box.
- The data type for the field (string, number, and so on) appears just below the title.

- A list of field values appears in the scroll box.

When you are finished reviewing the field data, click the Done button or select a value and click the Paste Data button to paste the value into your formula.

Note

If you need to select multiple values (for example, if you want to include records in which the field value is one of three values), the insertion point automatically moves to the next box after you paste data in the current box.

Note

When you browse a field, by default you see the first 500 unique values of the field. You can change this for the current report by clearing the Select Distinct Data for Browsing option in the [Report Options dialog box \[page 1345\]](#). You can change the default setting for all reports by clearing the Select Distinct Data for Browsing option on the [Database tab \(Options dialog box\) \[page 1319\]](#) of the [Options dialog box \[page 1314\]](#).

If Select Distinct Data for Browsing is not selected, browsing returns the unique values in the first 500 records.

Note

Note that since most SQL servers do not support selecting distinct records for long data types such as memo and blob fields, selecting and browsing distinct records are not supported for such fields.

If you fail to connect to a database because it is unavailable or password-protected, you can still browse on saved data.

Related Information

[Browse Data dialog box \[page 1157\]](#)

38.7 Browse Data dialog box

This dialog box appears when you click the Browse Data button in a dialog box or when you choose the Browse Field Data option from the shortcut menu on a selected field.

You can browse data for example:

- From the [Choose Field dialog box \[page 1167\]](#).
- In the [Select Expert \[page 1457\]](#).
- From the [Field Explorer dialog box \[page 1241\]](#).

Use the Browse Data dialog box to preview the data type, length, and content of any field.

- The source of the field (alias) and the field name appear as the title for the dialog box.

- The data type for the field (string, number, and so on) appears just below the title.
- The list contains a list of field values.

Note

When you browse a field, by default you see the first 500 unique values of the field. You can change this for the current report by clearing the Select Distinct Data for Browsing option in the [Report Options dialog box \[page 1345\]](#). You can change the default setting by clearing the Select Distinct Data for Browsing option on the [Database tab \(Options dialog box\) \[page 1319\]](#) of the [Options dialog box \[page 1314\]](#).

If Select Distinct Data for Browsing is not selected, browsing returns the unique values in the first 500 records.

Note

Note that since most SQL servers do not support selecting distinct records for long data types such as memo and blob fields, selecting and browsing distinct records are not supported for such fields.

If you fail to connect to a database because it is unavailable or password-protected, you can still browse on saved data.

Related Information

[Browse Data command \[page 1109\]](#)

38.8 Calculated Members dialog box

The Calculated Members dialog box appears when you right-click an OLAP grid row or column member and select Add Calculated Member from the shortcut menu.

38.8.1 Calculation Expert tab (Calculated Members dialog box)

Use the Calculation Expert tab to add a member that calculates Contribution (%), Growth, Ranking, or Variance. For details, see [Adding calculated members to the Worksheet \[page 451\]](#). The Calculation Expert tab consists of the following items:

Dimension

The Dimension list displays the name of the dimension that you selected on the OLAP grid. Use this dimension's members in your calculation.

Calculation Name

Use this box to enter a name for your calculation. In some cases, a default name is provided; accept or modify this name as you need.

Note

The program also modifies the Calculation Name as you define your calculation. For example, if you create a Growth calculation, the program adds the Calculation Name "Growth." When you select a member from the tree view, the program appends the name of that member to the default name.

Calculation Type

Use this list to specify the type of calculation you want to perform. Choose from the predefined calculations: Contribution (%), Growth, Ranking, and Variance. Or, select Custom if you want to specify your own MDX calculation on the Calculation tab.

Calculation Definition

This area provides the tools with which you define your calculation. The available options vary, depending upon the Calculation Type that you selected. Select dimension members from the tree view to add them to the calculation.

Related Information

[Adding calculated members that show contribution percentage \[page 452\]](#)

[Adding calculated members showing growth \[page 453\]](#)

[Adding calculated members showing ranking \[page 453\]](#)

[Adding calculated members showing variance \[page 455\]](#)

38.8.2 Data Analysis Expert tab (Calculated Members dialog box)

In addition to the standard calculated members, you can also add three types of Data Analysis calculations. Use the Data Analysis Expert tab to choose one of these predefined calculations:

- Trend Line
- Moving Average

- Linear Regression (X and Y)

For details, see [Adding data analysis calculations \[page 456\]](#). The Data Analysis Expert tab consists of the following items:

Dimension

The Dimension list displays the name of the dimension that you selected on the OLAP grid. Use this dimension's members in your data analysis calculation.

Calculation Name

Use this box to enter a name for your data analysis calculation. In some cases, a default name is provided; accept or modify this name as you need.

Note

The program also modifies the Calculation Name as you define your calculation. For example, if you create a Trend Line calculation, the program adds the Calculation Name "Trend Line." When you select a member from the tree view, the program appends the name of that member to the default name.

Calculation Type

Use this list to specify the type of calculation you want to perform. Choose from the predefined calculations: Trend Line, Moving Average, and Linear Regression (X and Y). Or, select Custom Data Analysis if you want to specify your own MDX calculation on the Calculation tab.

Calculation Definition

This area provides the tools with which you define your calculation. The available options vary, depending upon the Calculation Type that you selected. Select dimension members from the tree view to add them to the calculation.

Related Information

[Adding a trend line calculation \[page 456\]](#)

[Adding a moving average calculation \[page 457\]](#)

[Adding a linear regression calculation \[page 457\]](#)

38.8.3 Calculation tab (Calculated Members dialog box)

Use the Calculation tab to define your own calculation with MDX query language. Before using the Calculation tab, select Custom in the Calculation Type list on the Calculation Expert tab. For details, see [Adding a custom calculation using a calculation definition \[page 458\]](#). The Calculation tab consists of the following items:

Dimension

The Dimension list displays the name of the dimension that you selected on the OLAP grid.

Calculation Name

Use this box to enter a name for your calculation.

Calculation Definition

This area displays your MDX calculation as you write it. Type your calculation into this area directly, or use the dialog box's selection tools to add functions and dimensions to the calculation.

Dimension

Use this list to select a particular dimension, and then use the tree view to select the specific dimension members that you want to include in your calculation.

Functions

Click the Functions button to access the Function Library. The [Function Library dialog box \[page 1283\]](#) enables you to incorporate functions into your calculation quickly.

38.9 Change Caption dialog box

The Change Caption dialog box appears when you right-click a row or column caption in an OLAP grid and select the [Change Caption command \[page 1133\]](#) from the shortcut menu.

Use the Change Caption command to change the caption name in a row or column of an OLAP grid. You can also use this command to reset a caption to its default value.

Enter a new caption for

This box contains the current caption name for the row or column you selected. Type a new caption and click OK to save it.

Default

This button resets the current caption name to the default value found in your OLAP data source.

Related Information

[Renaming a member \[page 426\]](#)

38.10 Change Group Options dialog box

This dialog box appears when you click the Options button in the [Group Expert \[page 1428\]](#), or when you select the [Change Group command \[page 1134\]](#) from the gray area in the report designer that is to the left of a group header or footer.

Use the Change Group Options dialog box to edit an existing group. For details, see the [Insert Group dialog box \[page 1287\]](#), which has the same functionality.

38.11 Change Icon dialog box

This dialog box appears when you click the Change Icon button in the [Convert dialog box \[page 1170\]](#). (The Change Icon button appears after you select the Display As Icon option.)

Use the Change Icon dialog box to change the appearance of the icon you are using to represent an OLE object in your report.

Current

Displays the current icon in your report.

Default

Displays the default icon in your report.

From File

Enables you to select an icon from the specified file. You can choose any icon in an icon file, a library file, or an executable file. Select the icon from the scroll box below the file name. Click the Browse button to select a different file.

Label

Specifies the icon label that appears in the report.

Browse

Click the Browse button to select a different file containing icons. You can use choose one of the icons in an icon file, a library file, or an executable file.

Related Information

[Convert dialog box \[page 1170\]](#)

[Inserting OLE objects into reports \[page 337\]](#)

38.12 Change In Record Selection Formula dialog box

This dialog box appears when Crystal Reports detects a change in your report's record selection.

Use Saved Data

Click this option to run the report using the current data (in other words, fresh data is not retrieved from the database). The dialog box shows you the date and time that the data was last refreshed.

Refresh Data

Click this option if you want to run the report with fresh data from the database.

38.13 Change Map Title dialog box

The Change Map Title dialog box opens when you choose the [Title command \(Map menu\) \[page 1126\]](#) from the Map menu or the map's right-click menu. Change Map Title is available only when you are working with a map on the Preview tab.

Use this dialog box to specify or change the title you'd like to use for your map. The dialog box is empty if you did not specify a map title on the Text tab of the Map Expert. Otherwise, the title you specified earlier appears in the Change Map Title dialog box.

Enter a new title, or edit an existing one in the dialog box and click OK when you are done. The new title appears on your map on the Preview tab.

Related Information

[Map Expert \[page 1433\]](#)

38.14 Character-Separated Values dialog box

This dialog box appears when you use the [Export Report command \(Export submenu\) \[page 1075\]](#) to export a file in character-separated values format. From the [Export dialog box \[page 1236\]](#), select Separated Values (CSV) from the Format list and click OK; the Character-Separated Values dialog box appears.

Use this dialog box to specify the character(s) you want used to separate fields and to specify single or double quotes as a delimiter to surround non-numeric fields.

Note

In earlier versions, you could choose whether to have the output use the number formats and date formats that you defined in your report. The default behavior has changed so that your number and date formats are always used.

Delimiter

When you export a report to a character separated file, the program encloses all non-numeric fields in quotation marks. You need to specify whether the program should use single (') or double (") quotes. Using the keyboard, type the kind of quotation marks you want in this box.

Separator

When you export a report to a character separated file, you need to specify the character that will be used to separate one field from the next (the field delimiter) in that file. Enter any keyboard character in this box and the program will use that character as a separator.

When you import the file into another application, that application will be able to separate the data into fields whenever it sees the character you specify.

Note

It is best not to specify quotation marks as your separator, since they are already used to surround non-numeric fields.

Mode

Standard mode has the following features:

- Lets you control how report/page and group header/footers display in the CSV output.
- Respects conditional suppression.

Note

Conditional suppression applies to the report content only. The data is suppressed, but a row is printed in the report with the field delimiters. For example, if your field delimiter is a comma and your report has four columns, you will see a blank row that contains four commas.

- Retains line breaks in the text.
- Uses UTF-8 encoding.

Note

Standard Mode is the default format for exporting reports.

Legacy Mode is provided specifically to help customers transition to new CSV exports gradually by providing older style CSV export behavior (for example, MBCS output not UTF-8). Legacy Mode is deprecated, meaning it will not be available in future versions. All customers are recommended to transition all CSV exports to Standard CSV.

Report and Page Sections

When you export a report to a character separated file, you can choose to export the Report and Page Sections. The Report and Page headers and footers will be printed for each record.

To print Report and Page headers and footers as separate records, select Isolate Report\Page Sections.

Group Sections

When you export a report to a character separated file, you can choose to export the Group Sections. The Group headers and footers will be printed with each record.

To print the Group headers and footers as separate records, select Isolate Group Sections.

Related Information

[Export dialog box \[page 1236\]](#)

38.15 Chart Sort Order dialog box

This dialog box appears when you click the Order button on the Data tab of the [Chart Expert \[page 1400\]](#) for a chart with the Advanced layout type.

Use the Chart Sort Order dialog box to sort and group your chart data. For details, see the [Insert Group dialog box \[page 1287\]](#), which has similar functionality.

38.16 Choose Export File dialog box

This dialog box appears when you click the OK button in the [Export dialog box \[page 1236\]](#).

Use the Choose Export File dialog box to specify the file name (and path) to which you want to export the file.

- Using the Save in list, specify the folder in which you want to save the file.
- In the File name box, specify the file name you want to use.
- Click Save. The program exports the data to the file you specified.

Related Information

[Exporting a report \[page 617\]](#)

38.17 Choose Field dialog box

This dialog box appears when you choose the [Select Expert command \[page 1112\]](#) on the Report menu if you do not have a field selected and you have not already created a record selection formula. The dialog box also appears when you click Choose Other Field in the Function Arguments area of the [Formula Expert \[page 1426\]](#).

Use the Choose Field dialog box to select a field for record selection or to select a field as a function argument in the Formula Expert.

Fields

This list displays the report fields and database fields available in your report for use in record selection or the Formula Expert.

Browse

Click Browse to browse the data in the selected field.

Related Information

[Select Expert \[page 1457\]](#)

[Formula Expert \[page 1426\]](#)

38.18 Choose Profile dialog box

This dialog box appears when you export to Microsoft Exchange. Please refer to your Microsoft Exchange documentation for more information.

38.19 Color dialog box

The Color dialog box opens when you choose More from any list of colors.

Use this dialog box to specify the color you'd like to use on your report if one of the default values will not do. This dialog box is the same as the Color dialog box available in many Microsoft Office products.

Basic colors

This grid displays an assortment of predefined colors and shades. These choices allow you to use color shades to represent data ranges in your reports. For example, a best selling item might be bright blue, while subsequent items are represented by darker shades of blue.

Custom colors

This grid displays colors you have created by using Define Custom Colors (until you define custom colors, the grid contains only white sample boxes).

Define Custom Colors

Click Define Custom Colors to see the custom color palette. Click and drag in the color and/or solid palettes to see a sample of the color as you create it. You can also define custom colors by entering settings in the boxes below the palettes.

Once you have defined a custom color, click Add to Custom Colors to add your new color to the Custom colors grid. Crystal Reports saves custom colors between sessions.

38.20 Command Parameter dialog box

The Command Parameter dialog box appears when you click Create or Modify in the [Add Command to Report dialog box \[page 1154\]](#).

Use this dialog box to create or modify parameters for your SQL command. See [Creating a parameter with a static prompt \[page 839\]](#) for information about how to complete the fields in this dialog box.

38.21 Connection Properties dialog box

The Connection Properties dialog box allows you connect to an OLAP server that hasn't yet been set up in Crystal Reports. This dialog box appears when you click Add in the [OLAP Connection Browser dialog box \[page 1312\]](#).

Note

The Connection Properties dialog box also appears when you refresh a report and the data source needs to be reset or logged on to, or when you select the [OLAP Design Wizard command \[page 1119\]](#) from the Report menu. In these cases, you see only the Server Options section of the dialog box.

Specify the name and location of your OLAP server, along with any additional log on information that may be requested.

Server Type

From this list, select the type of OLAP provider that is compatible with your OLAP server. For the majority of OLAP servers, connect using Microsoft OLE DB Provider for OLAP Services.

Caption

The Caption is the name used to represent the server in the OLAP Connection Browser. You must specify a Caption before you can add the server. The Caption does not have to be the same as the Server Name.

Server Options

The options in this section change depending on the server type you selected from the Server Type list. In general, use these options to identify your OLAP server and select the cube you want to work with. If access to your server is restricted, you can also enter user and password credentials.

Note

Additional Server Options appear when you see the Connection Properties dialog box after refreshing a report or after selecting the OLAP Report Settings command from the Report menu. In these cases, only the options that you can change are available—the other options are for reference.

Test Connection

Use this button to test if the server options that you entered are valid. An error message is returned if the program encounters a problem (the program also tries to provide details of the error).

38.22 Convert dialog box

This dialog box appears when you choose the [Convert Picture Object command \(Object command variant\) \[page 1085\]](#) from the Edit menu if you have a picture object, or when you choose the [Convert command \(Object command variant\) \[page 1084\]](#) from the Linked Bitmap Image Object submenu of the Edit menu if you have a linked bitmap.

Use the Convert dialog box to convert the selected picture object from its current format to a new format. If your report contains a picture object in a format that you can't edit in one of your applications, use this command to convert the picture to a format that you can edit.

Convert to

Click this to convert the embedded information to a different type. This is useful if you no longer have the program that was used to create the information, or if you want to display the information in a different format than the original type allows. Select the new object type from the list.

Object Type

Lists all the available file types that the object can be converted to.

Activate as

Click this to open the embedded object by using a different program that is compatible with the original object type. This does not permanently convert the object.

Display As Icon

Specifies whether the embedded object will appear in the report as an icon or whether the actual contents of the object will always be visible. Note that some object applications only display objects as icons.

If you select Display as Icon, you can open the object in your document by double-clicking its icon.

Change Icon

Click this button to change the icon that represents the embedded object in your report. The [Change Icon dialog box \[page 1162\]](#) appears.

Related Information

[Convert Picture Object command \(Object command variant\) \[page 1085\]](#)

38.23 Create Alerts dialog box

The Create Alerts dialog box appears when you go to Alerts on the Report menu and choose the [Create or Modify Alerts command \(Alerts submenu\) \[page 1116\]](#) from the submenu.

Use the Create Alerts dialog box to specify a condition that, when met, displays a message about the report data.

New

Use this button to create a new Report Alert. The New button opens the [Create Alert dialog box \[page 1172\]](#).

The New button is always available.

Edit

Use this button to modify an existing Report Alert. The Edit button opens Edit Alert dialog box.

The Edit button is available when you select an existing Report Alert.

Delete

Use this button to delete an existing Report Alert. The Delete button is available when you select an existing Report Alert.

38.24 Create Alert dialog box

The Create Alert dialog box appears when you choose the New button from the [Create Alerts dialog box \[page 1171\]](#).

Note

The Edit Alert dialog box appears when you choose the Edit button from the [Create Alerts dialog box \[page 1171\]](#). The Create Alert dialog box and the Edit Alert dialog box contain the same functionality.

Use the Create Alert dialog box to specify the details of a condition that, when met, displays a message about the report data.

Name

Type the name you want to use for the Report Alert.

Message

You can type a message directly in the Message box, or you can open the Alert Message Formula Workshop and enter a message there.

The Alert Message formula is limited to 254 characters.

Condition

Click this button to open the Alert Condition Formula Workshop. Use the editor to create the alert condition you want. For example, if you want to know which countries are making sales over \$150,000 a year, you'd enter a formula like:

```
Sum ({Customer.Last Year's Sales}, {Customer.Country}) > 150000
```

Enabled

This check box must be selected if you want the Report Alert to be evaluated; otherwise, the alert is ignored.

38.25 Create New Parameter dialog box

This dialog box appears when you select Parameter Fields in the [Field Explorer dialog box \[page 1241\]](#) and click the New button.

Use the Create New Parameter dialog box to create a parameter field that you can use in a formula, in a record selection formula, or in the body of the report itself. From this dialog box, you can create static and dynamic prompts for your parameter; you can also create a new list of values.

Note

Dynamic prompts can include cascading lists of values, where one value in the prompt constrains values in subsequent pick lists.

38.25.1 Parameter name area

Name

Enter the name you want to assign to the parameter field in this box (the name My Parameter is added by default). This is the name that appears in the Parameter Fields list of the Field Explorer dialog box and in the Fields list in the [Formula Editor dialog box \[page 1280\]](#). This name is also used to identify the field when the field appears in the report or in a formula.

Note

When a parameter field name appears in a report, the program puts a question mark at the beginning of the name, to identify it as a parameter field. Thus, if you name a parameter field "My Parameter," the name appears in the field list in the Formula Workshop as "?My Parameter."

Type

Select the data type of the parameter field from this list. For example, if you are going to be prompting for a value in a numeric field, select "Number."

If you select Date or DateTime for your parameter's value type, you can alter the default date/time formats by using the Options command from the File menu. For details, see [Setting default formats for Date, Time, and Date and Time fields \[page 274\]](#).

Note

This option is available only for parameters with static prompts.

Continue to the next area by clicking [List of values area \[page 1174\]](#).

38.25.2 List of values area

This area changes depending on the type of prompt you create for your parameter field:

- **Static**
A static prompt is one that always contains the same values. For example, if your parameter prompts for a country value, you could create a static prompt because the country list represents a set of values that does not change often.
- **Dynamic**
The values of a dynamic prompt are retrieved from the data source and are not stored within the report. For example, if you are prompting for customer name, you could create a dynamic prompt because the names in your customer database probably change frequently.
Dynamic prompts can be authored in Crystal Reports. You do not need SAP BusinessObjects Enterprise or Crystal Reports Server to author dynamic prompts. When Crystal Reports is installed on the same machine as Microsoft Visual Studio, you can use the embedded Crystal Reports designer to author dynamic prompts. When Crystal Reports is used with Crystal Reports Server or SAP BusinessObjects Enterprise, additional features are available.

Note

A dynamic prompt can include a cascading list of values, which adds additional levels of selection for your users. For example, if you are prompting for a city value, but you also need to know which country and region that city comes from, you could create a cascade of values. In this case, you first prompt for a country, and when that value has been selected, the program prompts for a region by showing only the regions that apply to the selected country. Finally, when a region value has been selected, the program prompts for a city by showing only the cities that apply to the selected region. In this way, you can be sure that your user picks the correct city (for instance, Vancouver, Washington, USA rather than Vancouver, British Columbia, Canada).

38.25.2.1 Static

Value Field

Click this list to pick a database field that provides the prompt values for your parameter field.

Description Field

Click this list to pick a database field that provides the descriptions for the values in your parameter field. For example, if you choose Employee ID as your value, you might choose Employee Name as a description so that your users are able to identify ID numbers by the associated name.

Insert

Click this button to manually add a new value to the end of the Value list that appears under the Insert button.

Delete

Click this button to delete the value that you have selected in the Value list that appears under the Delete button.

Move Up

Click this button to move the value that you have selected in the Value list to a higher position in the list.

Move Down

Click this button to move the value that you have selected in the Value list to a lower position in the list.

Actions

Click this button to see these options:

- [Append all database values](#)
Use this option to load the values from the database field that you selected in Value Field. Crystal Reports adds the values to the list below the Options button. If you have specified a database field for descriptions, the descriptions are added at the same time.

Note

This option is available only if you have selected a Value Field.

- [Clear](#)
Use this option to clear any values and descriptions that appear in the list below the Options button.
- [Import](#)
Use this option to import a static list of values to the list below the Options button. The Open dialog box appears so you can search for a text file that contains the values to import.
- [Export](#)
Use this option to export the list of values below the Options button to a text file. The Save As dialog box appears so you can enter a name and choose a location for your file.

Value

This list contains static prompt values that you have imported, loaded from a database field, or entered manually. To add values manually, click the cell that contains the phrase "Click here to add item," and type the value that you want; press [Enter](#) to move to the next cell to type another value.

→ Tip

You can also click Insert to add values to this list.

You can sort the values in this list by clicking the header that contains the word Value; click once to sort in ascending order and twice to sort in descending order.

Description

This list contains static prompt descriptions that you have imported or loaded from a database field. You cannot add descriptions manually.

You can sort the values in this list by clicking the header that contains the word Description; click once to sort in ascending order and twice to sort in descending order.

Continue to the next area by clicking [Options area \[page 1179\]](#).

38.25.2.2 Dynamic

Prompt Group Text

Enter the text that you want to show users as a prompt for your group. This text appears on the "Enter prompt values" dialog box when you refresh your report. For example, if you are prompting for country, region, and city values, you might enter a prompt like "Choose the country, region, and city that you want to see data for."

ⓘ Note

If your dynamic prompt does not include a cascading list of values (that is you have entered only one field in the Value area), Crystal Reports skips this field and uses prompting text that it finds in the [Options area \[page 1179\]](#).

ⓘ Note

If you do not enter Prompt Group Text for a cascading list of values, the program creates a default name made up of the names of the fields that you picked in the Value area. For example: Country > Region > City – Prompt Group.

Note

You can enter both Prompt Group Text and prompting text (from the Options area) for a cascading list of values. The Prompt Group Text appears at the top of the "Enter prompt values" dialog box, while the prompting text from the Options area appears above the individual fields in the prompting dialog box.

New

Select this option to create a new list of values; when you do so, you can go to the Value area and select fields from the data source that you are currently connected to.

Existing

Select this option to choose an existing list of values. The tree view that appears contains nodes for any existing list of values that your report already contains. The tree view also contains a node for the Repository, which shows lists of values that you have saved to BusinessObjects Enterprise.

→ Tip

If the list is empty (you cannot connect to a repository, or your report does not contain an existing list of values), this option is not available.

When you pick an existing list of values, you see the values that make up that list, but you can change only the Sort Order, the Parameter Binding, or the fields in the Options area.

Insert

Click this button to manually add a new value to the end of the Value list that appears under the Insert button. This option is available only when you create a new list of values.

Delete

Click this button to delete the value that you have selected in the Value list that appears under the Delete button. This option is available only when you create a new list of values.

Move Up

Click this button to move the value that you have selected in the Value list to a higher position in the list. This option is available only when you create a new list of values.

Move Down

Click this button to move the value that you have selected in the Value list to a lower position in the list. This option is available only when you create a new list of values.

Value

Use this list to choose database fields to base your list of values on. If you want to create a cascading list of values, select more than one field. For the cascade to work logically, the values you pick should have some relationship between them. For example, you might choose the Department, Manager, and Employee fields (where Employee is the value you want for your parameter). In this case, you can ensure that employees with the same name are correctly identified through their departments and managers.

→ Tip

A dynamic prompt does not have to also be a cascading prompt. If you choose only one field in the Values area, the prompt will be dynamic but will not show your users a cascading pick list.

Description

Use this list to choose database fields to provide the descriptions for your list of values. Use this field only when you need to associate the values of the fields that you chose in the Value list with more descriptive database fields.

Sort Order

Use this list to choose a sort order. You can sort in ascending or descending order by field value or description.

Parameter Binding

Each field that you add to the Values area is automatically bound to the parameter that you are creating. Click the Parameter Binding field for any value that you want to unbind; likewise, click this field to add a binding after you have removed it.

Note

You bind a value to your parameter to indicate which value in your cascading parameter is the one that the program selects data for. For example, if your cascade consists of Country, Region, and City, you would bind the City value to indicate to Crystal Reports that your parameter is based on that value (as opposed to Country or Region, which are used only to narrow the values returned for City). If you leave all of the Value fields bound, Crystal Reports creates a new parameter for each. For example, the Parameter Fields node of the Field Explorer might contain: My City Parameter, My City Parameter – Level 1, and My City Parameter – Level 2.

Continue to the next area by clicking [Options area \[page 1179\]](#).

38.25.3 Options area

The Prompt Options are different depending on the type of parameter that you create.

38.25.3.1 All static parameter types except Boolean

Note

When you work with the Value fields that make up a cascading list of values, these options apply to the individual field that you have selected; you can set separate options for each Value field in the cascade.

Show on (Viewer) Panel

Choose whether the parameter will display on the parameter panel and whether it will be editable or read-only.

- Do not show: The parameter is not visible on the panel. Users must refresh the report to change the parameter's values.
- Editable: Users can view and change the parameter's values on the panel.
- Read only: Users can view the parameter's values on the panel, but cannot change them.

By default, the program sets this field to Editable.

Prompt Text

Enter the text that you want the program to show your users as a prompt. For example, if you are prompting for a state value, you might want to enter a prompt like "Enter the state that you want sales data for." For a Static prompt, the program adds the phrase "Enter < **<Parameter Name>** >:" by default.

Prompt With Description Only

Choose whether your user should be prompted with only a description (True) or both a value and its description (False). By default, the program sets this field to False.

Optional Prompt

Set this to true if you want to allow the user to bypass entering a value for this parameter.

Default Value

Add a value to this field if you want a default to appear when the program prompts your users to select a value.

Note

This option is available only for parameters with static prompts.

Allow custom values

If you set this option to True, your users can choose values that you entered or values of their own choice. Otherwise, your users can choose only values that you entered in the List of Values area.

Allow multiple values

Use the "Allow multiple values" field to select more than one default value for your parameter field when you click the Refresh button on the Navigation Tools toolbar.

When this field is set to True, the prompt allows multiple values. When set to True, you can set both "Allow discrete values" and "Allow range values" to True. Otherwise, only one of these two fields can be set to True at any given time. The field defaults to False.

Allow discrete values

Use the "Allow discrete values" field to select only discrete, or singular, values for your parameter field. If you have "Allow multiple values" selected, you will be able to select more than one discrete value for your parameter field, though those discrete values will continue to function singularly as opposed to a range value. You can choose discrete values for your parameter field after you choose the [Refresh Report Data command \[page 1120\]](#).

When this field is set to True, the prompt allows only discrete values. The field defaults to True.

Allow range values

Use the "Allow range values" field to select a range of values for your parameter field. For example, if you've selected a currency field, your parameter field (with range value selected) could equal any number that falls between \$10,000 and \$100,000.

When this field is set to True, the prompt allows range values. The field defaults to False.

Min length

Use the "Min length" field to limit the minimum number of characters that can appear in your parameter field. For example, if you choose the number 4 as your minimum length limit, the name of customer Dan could not be used because it is fewer than 4 characters long.

Max length

Use the "Max length" field to limit the maximum number of characters that can appear in your parameter field. For example, if you choose the number 5 as your maximum length limit, the name of customer Margaret could not be used because it is more than 5 characters long.

Edit mask

Use the "Edit mask" field to create an Edit Mask for your parameter. The Edit Mask can be any of a set of masking characters used to restrict the values you can enter as parameter values (the edit mask also limits the values your users can enter as prompting values). For more information on using the Edit Mask, see [Defining entry type and format using the Edit Mask \[page 864\]](#).

Note

When you enter an Edit Mask, the length options become inactive.

38.25.3.2 Static Boolean parameter type

Show on (Viewer) Panel

Choose whether the parameter will display on the parameter panel and whether it will be editable or read-only.

- Do not show: The parameter is not visible on the panel. Users must refresh the report to change the parameter's values.
- Editable: Users can view and change the parameter's values on the panel.
- Read only: Users can view the parameter's values on the panel, but cannot change them.

By default, the program sets this field to Editable.

Prompt Text

Enter the text that you want the program to show your users as a prompt. For example, if you are prompting for a state value, you might want to enter a prompt like "Enter the state that you want sales data for." For a Static prompt, the program adds the phrase "Enter < **<Parameter Name>** >:" by default.

Prompt With Description Only

Choose whether your user should be prompted with only a description (True) or both a value and its description (False). By default, the program sets this field to False.

Optional Prompt

Set this to true if you want to allow the user to bypass entering a value for this parameter.

Default Value

Add a value to this field if you want a default to appear when the program prompts your users to select a value.

Boolean group

Use the "Boolean group #" field to enter the number of the group you want to add the current Boolean parameter to. If this field is blank, a Boolean group is not used.

Note

Boolean groups appear when the program requires users to enter a prompt value. Boolean groups can contain multiple Boolean parameter fields. When users select a group of Boolean parameters, they can assign the same or different values to each item in the group (this behavior is controlled by the "Exclusive group" field).

Exclusive group

Use the "Exclusive group" field to ensure that users can choose only one Boolean parameter from a given group to have a value of True. All other Boolean parameters in the same group automatically become False.

If this field is set to True, users can add only a single Boolean value for the entire group. If this field is blank, users can combine Boolean values from several different groups and select which are to be True.

38.25.3.3 Parameters with dynamic prompts

Show on (Viewer) Panel

Choose whether the parameter will display on the parameter panel and whether it will be editable or read-only.

- Do not show: The parameter is not visible on the panel. Users must refresh the report to change the parameter's values.
- Editable: Users can view and change the parameter's values on the panel.
- Read only: Users can view the parameter's values on the panel, but cannot change them.

By default, the program sets this field to Editable.

Prompt Text

Enter the text that you want the program to show your users as a prompt. For example, if you are prompting for a state value, you might want to enter a prompt like "Enter the state that you want sales data for." For a dynamic prompt, the program adds the phrase "Enter < <Parameter Name> >:" by default.

Note

You can enter both Prompt Group Text (in the List of Values area) and Prompt Text for a cascading list of values. The Prompt Group Text appears at the top of the "Enter prompt values" dialog box, while the Prompt Text appears above the individual fields in the prompting dialog box.

Prompt With Description Only

Choose whether your user should be prompted with only a description (True) or both a value and its description (False). By default, the program sets this field to False.

Optional Prompt

Set this to true if you want to allow the user to bypass entering a value for this parameter.

Allow multiple values

Use the "Allow multiple values" field to select more than one default value for your parameter field when you click the Refresh button on the Navigation Tools toolbar.

When this field is set to True, the prompt allows multiple values. When set to True, you can set both "Allow discrete values" and "Allow range values" to True. Otherwise, only one of these two fields can be set to True at any given time. The field defaults to False.

Allow discrete values

Use the "Allow discrete values" field to select only discrete, or singular, values for your parameter field. If you have "Allow multiple values" selected, you will be able to select more than one discrete value for your parameter field, though those discrete values will continue to function singularly as opposed to a range value. You can choose discrete values for your parameter field after you choose the [Refresh Report Data command \[page 1120\]](#).

When this field is set to True, the prompt allows only discrete values. The field defaults to True.

Allow range values

Use the "Allow range values" field to select a range of values for your parameter field. For example, if you've selected a currency field, your parameter field (with range value selected) could equal any number that falls between \$10,000 and \$100,000.

When this field is set to True, the prompt allows range values. The field defaults to False.

38.26 Create Parameter Field dialog box

This dialog box appears when you click the Create/Edit Parameter button on the Rows/Column or Slice/Page screens of the [OLAP Report Creation Wizard \[page 1382\]](#), or when you click Create/Edit Parameter button on the Rows/Columns tab of the [OLAP Expert \[page 1442\]](#).

Note

A variation of this dialog box, called Delete Link to Parameter Field, appears when you delete an existing parameter in the OLAP Report Creation Wizard or the OLAP Expert.

Use the Create Parameter Field dialog box to create a parameter field that you can use in your OLAP grid.

Name

Enter the name you want to assign to the parameter field in this box. This is the name that will be used to identify the field in the Rows or Columns lists.

Note

When a parameter field name appears, the program puts a question mark at the beginning of the name, to identify it as a parameter field. Thus, if you name a parameter field "My Parameter," the name will appear in the Rows or Columns lists as "?My Parameter."

Prompting Text

Enter the text you want the program to use for your prompt into this box. For example, if you are prompting for a state value, you might want to enter a prompt like "Enter state of interest." You can use approximately 20 characters/spaces in your prompt.

Value Type

This field cannot be edited when creating or editing parameters in the OLAP Report Creation Wizard.

38.26.1 Options

Allow multiple values

By selecting the "Allow multiple values" check box, you can choose more than one default value for your parameter field.

Discrete value(s)

By selecting the "Discrete value(s)" option, you can select only discrete, or singular, values for your parameter field. If you have "Allow multiple values" selected, you will be able to select more than one discrete value for your parameter field, though those discrete values will continue to function singularly as opposed to a range value.

Range value(s)

This option is not available when creating or editing parameters in the OLAP Report Creation Wizard.

Discrete and Range Values

This option is not available when creating or editing parameters in the OLAP Report Creation Wizard.

Default Values

Click the Default Values button to set the default prompt settings for the parameter field you are creating. The [Set Default Values dialog box \[page 1361\]](#) appears.

38.27 Create Running Total Field dialog box

The Create Running Total Field dialog box appears when you select Running Total Fields in the [Field Explorer dialog box \[page 1241\]](#) and then click the New button. This is the same as the [Edit Running Total Field dialog box \[page 1227\]](#) that appears when you edit a running total field in the Field Explorer or directly from the report. This description applies to both dialog boxes.

Use these dialog boxes to create or edit running total fields. Running totals are totals that are displayed on a record by record basis. Running Totals total all records (in the report, in the group, and so on) up to and including the current record. For example, if your first three records have values of 2, 4, and 7, the running total for each of the three records would be as follows:

$$0 + 2 = 2$$

$$2 + 4 = 6$$

$$6 + 7 = 13$$

Available Tables and Fields

Lists all tables and fields in the report. Choose the field you want to summarize (the field on which a running total will be based) from this list.

Running Total Name

Type the name of your running total, for example Last Year's Sales, into the Running Total Name box.

Field to summarize (Summary area)

The Field to summarize list will display a field you select from the Available Tables and Fields list. The running total you are creating will be based on the field in the Field to summarize list.

Note

Use the Add button (>) to copy a field from the Available Tables and Fields list to the Field to summarize list.

Type of summary (Summary area)

Use the Type of summary list to select from a number of summary options. The summary type you choose will determine the running total calculation. For example, if you chose Customer.Last Year's Sales as the field to summarize, you can choose to have your running total field perform sum, average, or sample variance calculations on the selected field, to name only a few summary types.

Note

Summary-type options change depending on the field you chose to summarize. For example, you will have one set of summary types when creating a running total for Customer.Last Year's Sales, and you will have a different set of summary types when creating a running total for Customer.Region.

For each record (Evaluate area)

Use the For each record option to have your running total evaluate each record in a field.

On change of field (Evaluate area)

Use the On change of field option to have your running total perform its evaluation only when a specified field has changed.

Note

Use the Add button (>) to copy a field from the Available Tables and Fields list to the On change of field list.

On change of group (Evaluate area)

Use the On change of group option to have your running total perform its evaluation only when a specified group has changed.

Use a formula (Evaluate area)

Select the Use a formula option to have your running total use a formula when performing its calculations.

Never (Reset area)

Use the Never option to lock your running total field. With the Never button selected, your running total will perform its calculations on that field throughout the entire report.

On change of field (Reset area)

Use the On change of field list to have your running total evaluate or reset when a designated field changes.

Note

Use the Add button (>) to copy a field from the Available Tables and Fields list to the On change of field list.

On change of group (Reset area)

Use the On change of group list to have your running total evaluate or reset when a designated group changes.

Use a formula (Reset area)

Select the Use a formula option to have your running total use a formula to determine when to reset.

Related Information

[Creating running totals \[page 209\]](#)

38.28 Cross-Tab Group Options dialog box

This dialog box appears when you click Group Options in the [Cross-Tab Expert](#) [page 1412].

Use the Cross-Tab Group Options dialog box to create or modify group options for any row or column field used in the cross-tab report. The Group Options button is not available for summarized fields. For details, see the [Insert Group dialog box](#) [page 1287], which has similar functionality.

38.29 Current Parameter Values dialog box

This dialog box appears when you choose the [Show Current Parameter Values command](#) [page 1119] from the Report menu. It is a read-only version of the [Enter Parameter Values dialog box](#) [page 1230].

Use the Current Parameter Values dialog box to see all the parameter values in your current report:

- If your report has a subreport, the dialog box shows all parameter values in the main report and in the subreport.
- If no parameters have values, the Parameter Fields list displays the message: <No parameters have any values>.
- If your reports has a linked subreport, the dialog box does not display the parameter for that link (that is, it does not show the field link parameter). To see that parameter, go to the subreport tab and select the Show Current Parameter Values again.

38.30 Custom Function Name dialog box

This dialog box appears when you select Report Custom Functions in the [Formula Workshop dialog box](#) [page 1282] and then click the New button.

Use the Custom Function Name dialog box to specify a name for your function.

New Name

Use this box to type in the name of your new function.

Note

The name you assign a custom function can contain only letters, numbers, and the underscore character. Custom function names cannot begin with a number.

Use Extractor

Click the Use Extractor button if you want to use the [Extract Custom Function from Formula dialog box \[page 1239\]](#) for creating your function.

Use Editor

Click the Use Editor button if you want to use the Custom Function Editor dialog box for creating your formula.

38.31 Custom Function Properties dialog box

The Custom Function Properties dialog box appears when you:

- Select an existing custom function when the "Toggle properties display" button is selected on the Formula Workshop toolbar.
- Click Enter More Info on the [Extract Custom Function from Formula dialog box \[page 1239\]](#).
- Click More Info in the [Formula Expert \[page 1426\]](#) (in this case, the content of the Custom Function Properties dialog box is only for reference).

Use this dialog box to specify additional information and default values for your custom function.

Name

This field is only for reference. It displays the name specified when the custom function was created.

Summary

Use this area to add additional information about your custom function to help identify its purpose, intended use, and so on.

Category

Use this area to define a category for your custom function. Categories are a way of grouping certain custom functions; they appear as subfolders in the Report Custom Functions folder of the Formula Workshop tree view and in the Functions window of the [Formula Editor dialog box \[page 1280\]](#).

If you want to create further levels, use a forward slash in the category definition. For example, if you enter Crystal/Format in the Category field, your custom function will appear in the tree as follows:



Repository

This field is only for reference. It displays the name of the repository to which the custom function is connected or was last connected, if any. The custom function must be connected to the repository for this field to contain information.

Display in Experts

Select this option if you want your function to be displayed in the Custom Function Supplying Logic list of the [Formula Expert \[page 1426\]](#).

Author

Use this field to add the name of the creator of the custom function.

Return Type

This field is only for reference. It displays the data type of the function.

Help Text

Clicking this button opens the [Help Text dialog box \[page 1285\]](#). Use the dialog box to specify a message to help others use the function. Help text is available only in this dialog box.

Arguments

The fields in this area display the definition of your custom function.

Name

This field is only for reference and shows the argument name.

Type

This field is only for reference and shows the data type of the corresponding database field.

Description

This field lets you add an optional description for an argument. This description appears in the [Formula Expert \[page 1426\]](#).

Default Values

This field lets you define default values for an argument. Clicking a Default Values field opens the [Default Values for dialog box \[page 1220\]](#). Default values created here can be used in the Formula Expert.

Note

You cannot create default values for range, array, or range array argument types.

For information about creating a set of default values, see [Defining default argument values for custom functions \[page 736\]](#).

38.32 Custom Style dialog box

The Custom Style dialog box appears when you click the Customize button on various tabs of the Format Editor.

38.32.1 Currency Symbol tab (Custom Style dialog box)

Click the Customize button on the [Number tab \(Format Editor dialog box\) \[page 1262\]](#) to see the Currency Symbol tab.

Use the Currency Symbol tab to set detailed formatting options for currency fields in your report.

Enable Currency Symbol

Select this check box if you want to display currency symbols in your currency fields.

Fixed

Click this option if you want the program to place the currency symbol in the first (far left) position in the field. The program will leave extra space between the symbol and the currency if the field is justified to the right.

For example:

\$ 325.00

\$ 401,325.00

Floating

Click this option if you want the program to place the currency symbol as close to the currency amount as possible so that there are never any extra spaces between the symbol and the amount.

For example:

\$401,325.00

\$325.00

One Symbol Per Page

Select this check box if you want the program to displays/print a currency symbol only once per page, beside the first number/currency value that appears on the page for each field. No currency symbols will appear beside any of the other values for that field on the page.

Position

This list displays all the available positions for the currency symbol.

Currency Symbol

Use this box to enter the desired currency symbol.

38.32.2 Number tab (Custom Style dialog box)

Click the Customize button on the [Number tab \(Format Editor dialog box\) \[page 1262\]](#) to see the Number tab.

Use the Number tab to set detailed formatting options for number fields in your report.

Use Accounting Format

When you select this option:

- The negative symbol used and its position is determined by the Windows Regional Settings (it will be either the minus sign or the brackets).
- The dash will be automatically chosen for "Show Zero Values as" (zero values are automatically displayed as dashes ('-')) in your report.
- The currency symbol is fixed on the left side.

Suppress if Zero

When you select this check box, nothing is printed in a column of number field data if the value is zero.

Decimals

When you are setting up a custom format for a number field object, use this list to set the Decimals property for that object. This option allows you to specify the number of decimal places you want to print. You can specify from 0 to 10 decimal places. When you choose the number of decimal places you want, the program changes the sample number at the bottom of the dialog box to show you the results of your selection.

Rounding

When you are setting up a custom format for a number field object, use this list to set the Rounding property for that object. This property allows you to specify the number of places you want to round to. When you choose the number of places you want, the program changes the sample number at the bottom of the dialog box to show you the results of your selection.

Negatives

When you are setting up a custom format for a number field object, this property enables you to specify the way you want negative values to appear. Your options are:

-1.23	Prints negative values with a leading minus sign (-100.00, -225.73, -1,000,000).
1.23-	Prints negative values with a trailing minus sign (100.00-, 225.73-, 1,000,000.00-).
(1.23)	Prints negative values in parentheses [(100.00), (225.73), (1,000,000)].

Set the Negatives property to fit your needs. The program changes the sample number at the bottom of the dialog box to reflect the property you choose.

Reverse Sign for Display

You can reverse the sign for debit and credit amounts in financial reports by selecting this check box.

Allow Field Clipping

The Allow Field Clipping check box determines whether overflow field representation is enabled for the selected field(s).

- Select this check box to disable overflow field representation. This means that if the value in the field is too large for the field, the value will be truncated. For example, if you have a field that is sized to allow 5 characters, and the value in the field is 3708.19, the field will print the value: 3708.1 (assuming that the field values are left-justified).
- Clear this check box to disallow field clipping and enable overflow field representation. With overflow field representation enabled, values that are too large for their fields will be represented by a string of # signs. For example, if you have a field that is sized to allow 5 characters, and the value in the field is 3708.19, the field will print: #####. This lets you know that the field needs to be resized (or that you need to select the Can Grow property).

Decimal separator

When you are setting up a custom format for a number field object, the Decimal box allows you to type in the character(s) you want to use as a decimal separator. The default is a decimal point (.). If you want to change the separator, replace the character in the edit box with the separator of your choice. The program modifies the sample number at the bottom of the dialog box as you type in your characters.

Note

The box does not expand as you type in your separation characters; you are limited to the number of characters the box will hold.

Thousands Separator

The Thousands separator property allows you to choose the way you want numbers over 999 to appear on your reports.

- When you select the property (check mark), the program inserts a comma (,) or another separator of your choice as a thousands separator character for amounts over 999. Your numbers are printed like this: 1,000.00, 10,000.00, 999,000.00.
- When you clear the property you deactivate the thousands separator character. Your numbers are printed like this: 1000.00, 10000.00, 999000.00.

Symbol

This box displays the current thousands separator. The default is a comma (,). If you would like to use a different thousands separator, type it in this box. The changes you make in this box do not apply unless you select the Thousands Separator check box.

Leading Zero

Leading Zero allows you to include a zero, if you wish, before the decimal point in decimal amounts less than one.

- Select this property if you want the program to include a zero before the decimal point whenever you have a decimal amount less than one (0.001, 0.9999, 0.755).
- Clear this property if you do not want the leading zero (.001, .999, .755).

Show Zero Values as

This list contains the options available for displaying values of zero (0) in the selected field.

Related Information

[Format Editor dialog box \[page 1256\]](#)

38.32.3 Date and Time tab (Custom Style dialog box)

Click the Customize button on the [Date and Time tab \(Format Editor dialog box\)](#) [page 1264] to see the Date and Time tab.

Use the Date and Time tab to set detailed formatting options for date and time fields in your report.

Order

From this list, select the order in which you want the elements of your date/time field to appear. You can select:

Date Time	03/01/99 1:23:45AM (Default)
Time Date	1:23:45AM 03/01/99
Date	03/01/99
Time	1:23:45AM

Separator

Type the character(s) you want to separate the date and time elements of your date/time field. For example, typing an "X" in the Separator box produces the following formatting for a date/time field sorted in Date Time order:

03/01/99 X 1:23:45AM

 Note

The Separator character has no effect if you select Date or Time as your sort order.

38.32.4 Date tab (Custom Style dialog box)

Click the Customize button on the [Date and Time tab \(Format Editor dialog box\)](#) [page 1264] to see the Date tab.

Use the Date tab to set detailed formatting options for date fields in your report.

Date Type

The Regional Settings dialog box in the Windows Control Panel allows you to specify default date formats for use with any Windows program. You can specify the month/day/year separator and the order in which the date elements appear. These dialog boxes also allow you to specify a short date style (like 12/24/96 or 16 Mar, 1996) and a long date style (like Sunday, April 21, 1996).

This list lets you:

- Choose one of the two default styles.
- Choose Custom to create your own style in the Format Editor.

Select the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to demonstrate the style you have chosen.

Calendar Type

If you are using Windows NT/95/98 your operating system may support more than one calendar type. For instance, your operating system may support a native calendar (Japanese), a Gregorian calendar (in the local language) and a Gregorian English calendar. If your operating system supports multiple calendar types you may use the Calendar Type list to select a calendar type for the date field.

Month

If you want to create a custom format for a date field object, use this list to specify the way you want the program to print/display the month part of the date. Your options are:

3	Prints a month as a number. If the month number is only a single digit, it prints the number without a leading zero.
03	Prints the month as a number. If the month number is only a single digit, it prints the number with a leading zero.
Mar	Prints a three letter abbreviation for the month.
March	spells out the full name of the month.

Choose the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to reflect your choice.

Day

If you want to create a custom format for a date field object, use this list to specify the way you want the program to print/display the day part of the date. Your options are:

1	Prints the day as a number. If the day number is only a single digit, it prints the number without a leading zero.
01	Prints the day as a number. If the day number is only a single digit, it prints the number with a leading zero.

Choose the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to reflect your choice.

Year

If you want to create a custom format for a date field object, use this list to specify the way you want the program to print/display the year part of the date.

99	Prints the year in its short, two digit format.
1999	Prints the year in its long, four digit format.

Choose the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to reflect your choice.

Era/Period Type

Use the Era/Period Type list to select either a short or long era/period of time.

YMD, DMY, MDY options

If you want to create a custom format for a date field object, use these options to select the order in which you want the elements of the date to appear. Your options are:

MDY	Prints the date in the order Month, Day, Year.
DMY	Prints the date in the order Day, Month, Year.
YMD	Prints the date in the order Year, Month, Day.

Select the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to demonstrate the sequence you have chosen.

Type

When you are setting up a custom format for a date field object, if you want the name of the day (or an abbreviation of that name) to appear before the rest of the date, use this option to set the format for that name. Your options are:

Short	Prints an abbreviation of the day name before the date (for example, Fri).
Long	Prints the full day name before the date (for example, Friday).
None	Does not print a day name before the date.

Choose the option that fits your needs. The program modifies the sample date at the bottom of the dialog box to reflect your choice.

If you set this property to Short or Long, you need to specify the character(s) you want to use to separate the name from the date itself. You do that in the Separator box.

Sep

When you are setting up a custom format for a date field object, if you set the Leading Day Type property to short or long (so a day name or abbreviated day name appears before the date), you need to specify the character(s) you want to use to separate the name from the date itself. You do that in this box. Within practical limits, you can use as many characters as you need. For example,

- To separate a date name from a date, you may use Comma+Space (,) as your separators.
- To separate a date name abbreviation from a date, you may use Period+Comma+Space (.,) as your separators.

Type in the characters you want to use. The program modifies the sample date at the bottom of the dialog box as you do.

Encl

Use the Encl list to select parentheses or square brackets as an enclosure for date/time fields on your report.

Position

Use the Position list to select either a Leading or a Trailing position.

Link separators to date order

Select this option if you want any separators you specify to be considered when sort date fields.

Prefix

Use the Prefix box to create a prefix for dates in your date field, for example, DATE.

First

When you are setting up a custom format for a date field object, the program enables you to specify the separators, if any, that you want to use to separate the date elements. Since each date has three elements (YMD, DMY, or MDY), you can specify two different separators. The first separator goes between the first and second elements, the second separator goes between the second and third elements. Type the character(s) you want to use for the first separator in this box. Within practical limits, you can use as many characters as you need. For example,

- If you are using a short format, all numbers, you may want to specify a forward slash (/) as the first separator and as the second separator as well (like 12/25/96).
- If you are using a long format with the month spelled out, you may not want to use any separator between the first and second elements and then use a comma between the second and third (like March 22, 1998).

Second

When you are setting up a custom format for a date field object, the program enables you to specify the separators, if any, that you want to use to separate the date elements. Since each date has three elements (YMD, DMY, or MDY), you can specify two different separators. The first separator goes between the first and second elements, the second separator goes between the second and third elements. Type the character(s) you want to use for the second separator in this box. Within practical limits, you can use as many characters as you need. For example,

- If you are using a short format, all numbers, you may want to specify a forward slash (/) as both the first and second separators (for example, 12/25/96).
- If you are using a long format with the month spelled out, you may not want to use any separator between the first and second elements and then use a comma between the second and third (like March 22, 1998).

Suffix

Use the Suffix box to create a suffix that follows the dates in your date field.

38.32.5 Time tab (Custom Style dialog box)

Click the Customize button on the [Date and Time tab \(Format Editor dialog box\) \[page 1264\]](#) to see the Time tab.

Use the Time tab to set detailed formatting options for time fields in your report.

Use System Default Format

When you select this check box, the program, by default, uses the date formats from the Regional Settings dialog box in the Windows' Control Panel.

Note

The program automatically clears this property whenever you make date formatting changes in this dialog box.

12 Hour

Select this check box if you want the time to be displayed in a 12 hour format.

24 Hour

Select this check box if you want the time to be displayed in a 24 hour (military time) format.

00:00-11:59

Use this box to specify the characters you want to use to identify the time between midnight and noon if you want to use something other than the default, AM.

12:00-23:59

Use this box to specify the character(s) you want to use to identify the time between noon and midnight if you want to use something other than the default, PM.

Symbol position

Use the Symbol position list to select the placement for the AM or PM symbols in your time field. For example, you can choose to have the AM symbols appear before or after the time.

Note

Symbol position is only available for the 12-hour time format.

Hour

This list shows all the possible formats for the hour display. Choose the format you want. When you do, the program formats the sample time at the bottom of the dialog box to reflect your choice.

Minute

This list shows all the possible formats for the minute display. Choose the format you want. When you do, the program formats the sample time at the bottom of the dialog box to reflect your choice.

Second

This list shows all the possible formats for the seconds display. Choose the format you want. When you do, the program formats the sample time at the bottom of the dialog box to reflect your choice.

Hr/Min Separator

Use this box to enter the character(s) you want to use to separate hours and minutes in your time field.

Min/Sec Separator

Use this box to enter the character(s) you want to use to separate minutes and seconds in your time field.

38.33 Customize Map dialog box

The Customize Map dialog box appears when you choose the [Type command \(Map menu\) \[page 1126\]](#) from the Map menu or the map's right-click menu. The Type command appears only when viewing a map on the Preview tab.

This dialog box allows you to change your map type and to set the options for the map type you select. The Customize Map dialog box contains much the same functionality as the [Type tab \(Map Expert\) \[page 1438\]](#).

The Customize Map dialog box provides five basic map types (all types may not be available for every map), each suitable for a different strategy of data analysis. When deciding which map type best fits your report, you should consider the type of data you want to analyze. For example, if you want the map to display one data item for each geographic division (city, state, country, and so on), then you might use a Ranged, Dot Density, or Graduated map. On the other hand, if you want the map to display more than one value for each new geographic division, then you could use a Pie Chart map or a Bar Chart map.

Map Type

Use this list to select a map type and to activate the related mapping options.

The map types are:

- Ranged
- Dot Density
- Graduated
- Pie Chart
- Bar Chart

Options

Use the Options area to select the appearance of the map values displayed on the map. The options available in the Customize Map dialog box vary depending on the map layout you selected.

For more information about the options available for each map type, see:

- [Ranged option view \[page 1438\]](#)
- [Dot Density option view \[page 1440\]](#)
- [Graduated option view \[page 1440\]](#)
- [Pie Chart Option view \[page 1440\]](#)
- [Bar Chart Option view \[page 1441\]](#)

Related Information

[Map Expert \[page 1433\]](#)

38.34 Data Explorer dialog box

The Data Explorer lets you select the server you want to log on to or off of from a tree view like that in the [Database Expert \[page 1420\]](#).

Log On

Logs you onto the data source you've selected from the data source list.

Log Off

Logs you off the data source you've selected from the data source list.

Add to Favorites

Adds the selected data source to your Favorites folder.

Delete Favorite

Removes the selected data source from your Favorites folder.

Options

Provides access to the options you can set globally on the Database tab of the [Options dialog box \[page 1314\]](#).

38.35 Data source connectivity dialog boxes

To create a report, you have to select a data source from the list of Available Data Sources on the [Data tab \(Database Expert\)](#) [page 1421]. Depending on the data source, you might also have to provide additional information. The descriptions in this section pertain to the various connectivity dialog boxes available in Crystal Reports.

38.35.1 Access/Excel (DAO) dialog box

Database Name

Enter the full path and name of your database, or use the button adjacent to this field to search for your database.

Database Type

Select your database type from the list. The database types that are supported include Access, Dbase, Excel, HTML, Lotus, Paradox, and text files.

Secure Logon

If your data source requires secure logon, select this check box. Use the remaining fields in this dialog box to specify connection information.

Note

Access supports file-level and user-level security both individually and in combination. If you are accessing a database that uses both of these security types in combination, you must use the Crystal Reports DAO or OLE DB driver to enter the necessary security information (you cannot use ODBC in this case because ODBC drivers cannot handle multiple passwords).

Database Password

Enter the file-level security password for your Access database, if applicable.

Session UserID

Enter the user-level security ID for your Access database, if applicable.

Session Password

Enter the password that corresponds to your user-level security ID, if applicable.

System Database Path

Enter the path of the security file (.mdw) associated with your Access database. You can also use the button adjacent to this field to search for your security file.

For more information about the security features available in Access databases, see your Access documentation.

38.35.2 ADO.NET (XML)

This option supports pushing data into your report from an ADO.NET dataset; it also supports pulling data from a DLL that returns a dataset (this DLL is normally created by a developer). When a DLL has been created to pull data from, users can create reports from the data that is returned.

Note

The *Crystal Reports for Visual Studio .NET* online help includes information about generating ADO.NET dataset objects.

File Path

Enter the full path and name of your XML file, XML Schema file, or .NET DataSet Provider. You can also use the button adjacent to this field to search for your file or provider. When you search, choose the appropriate file type from the "Files of type" list in the Open dialog box:

- XML Files
- XML Schema Files
- .NET DataSet Provider

Use Classes from Project

Select this option if you want to use classes from an existing .NET project.

Class Name

This option appears after you've selected "Use Classes from Project." The list is populated with the classes contained in the file you selected. Select the class whose DataSet you want to report on.

Use DataSet from Class

Select this option if you want to use a DataSet that is returned from the file you selected.

DataSet Names

This option appears after you've selected "Use DataSet from Class." The list is populated with the DataSets contained in the file you selected. Select the DataSet that you want to report on.

38.35.3 COM Connectivity dialog box

Use COM Connectivity to report off any COM object that you (or someone else in your organization) has developed for this purpose. To serve as a data source for a Crystal report, the object must provide an interface that returns an ADO recordset, and the object must be registered on your local machine. Once you type the object's ProgID in this dialog box, Crystal Reports allows you to select from all methods that return an ADO recordset.

For details on developing compatible COM objects and using them in Crystal reports and custom reporting applications, see the sections related to the COM data provider in the *Crystal Reports Developer's Help* (CrystalDevHelp.chm).

38.35.4 Crystal Data Object dialog box

Crystal Data Object (CDO) connectivity is provided mainly for backward compatibility. This feature allows application developers to pass non-persistent data sources to Crystal reports at runtime.

Note

Using COM connectivity or Field Definitions connectivity is recommended instead of Crystal Data Object connectivity.

If you are designing a Crystal report for an application that uses this technology, you have two choices:

- You can design your base Crystal report using this dialog box by specifying the ProgID of a COM object that implements the ICRDataSource interface. At runtime, the report calls into this interface to retrieve data.
- You can create your base Crystal report off a schema, rather than a Crystal Data Object, using the [Field Definitions Only dialog box \[page 1209\]](#). At runtime, your application code must pass the report a recordset that adheres to the schema.

For details on developing reports and applications with Crystal Data Objects, see the *Crystal Reports Developer's Help* (CrystalDevHelp.chm).

38.35.5 DB2 Unicode dialog box

Crystal Reports opens SQL data created by DB2 through `crdb_p2sdb2.dll`, installed with Crystal Reports. It can also open SQL data through a Unicode version of the driver, called `crdb_db2cli.dll`. Both of these drivers work with the DB2 database drivers to read DB2 data.

38.35.6 Field Definitions Only dialog box

Field Definitions are special drivers used to create reports based on only a schema without actual data. This driver is included mainly for backward compatibility and is only used during the report creation process.

At runtime, a report created with this driver must be passed a recordset or dataset through a Crystal Data Object. For details on developing reports and applications with Crystal Data Objects, see the *Crystal Reports Developer's Help* (CrystalDevHelp.chm).

Field Definition File

Enter the name and full path of your Field Definition File, or use the find button adjacent to the Field Definition File field to search for your definition file (.ttx).

Create File

Click this button to create a new field definition file in the Database Definition Tool dialog box.

38.35.7 File System Data dialog box

You can use Crystal Reports to report on your File System Data. File System Data is a collection of information that represents the files, folders, and shortcuts of your product installation.

Starting Directory

Enter the name and full path of your File System Data, or use the button adjacent to the Starting Directory field to pick the directory that contains the File System Data that you want to use.

Include All Files

Select this option to include all files in the directory that you specified above. If you clear this option, you are able to enter criteria for the files that you want to include.

File Mask

If Include All Files is not selected, you can use this field to indicate which file types you want to include in your File System Data.

Include All Subdirectories

Select this option to include all subdirectories in the directory that you specified above. If you clear this option, you are able to enter the maximum number of subdirectories to include.

Max Number of Subdirectory Levels

If Include All Subdirectories is not selected, you can use this field to enter the maximum number of subdirectory levels that you want to include in your File System Data.

Include Empty Directories

Select this option to include empty subdirectories in your File System Data.

38.35.8 Java Beans Connectivity dialog box

Use Java Beans Connectivity to report off Java Beans that you (or someone else in your organization) has developed for this purpose. You can use this feature to report off any compiled Java class that has a public method returning `java.sql.ResultSet`.

This feature requires that you first install the Java Runtime Environment (JRE) or Java Development Kit (JDK) on your local machine. In addition, specific registry entries must be created and configured correctly, so Crystal Reports knows where to find your Java data sources. Developers who create Java Beans for reporting purposes within your organization may distribute programs that configure these registry entries for you.

For complete details on installing prerequisite Java software, developing compatible Java Beans, creating required registry entries, and using Java Beans in Crystal Reports, see the *Creating a Java Bean to Use as a Crystal Reports Data Source* document available from the SAP Community Network: <http://scn.sap.com>.

38.35.9 JDBC (JNDI) dialog box

Note

You can configure some of the connection information on this screen in the `CRConfig.xml` file. For more information, see [CRConfig.xml Tag Reference \[page 1064\]](#).

JDBC Connection

Select this option to use a URL and class name to connect to a Java Database Connectivity (JDBC) data source.

For details about JDBC, connection URLs, and class names, see your Java and/or data source documentation, or search the Sun Microsystems web site.

Connection URL

Enter a JDBC URL to specify a data source so that the appropriate database driver (class) can establish a connection; you'll identify the class in the next field.

Database Classname

Enter the name of the class that you want to use as your database driver for this connection. You must know the class name, which is provided by your database client software.

JNDI Connection

Select this option to use a data source (a combination of a connection URL and a class name) that has already been specified and saved through the Java Naming and Directory Interface (JNDI).

JNDI Provider URL

Enter the JNDI connection URL for your database driver. This information is provided by the database driver vendor.

JNDI Username

Enter the appropriate user name to connect to your JNDI server. The JDBC driver uses this information to connect to the database.

JNDI Password

Enter the appropriate password to connect to your JNDI server.

Initial Context

Enter a directory path that indicates where in the directory service the JNDI should start a recursive search for your predefined connections (data sources, or URL and class name combinations).

Note

All tokens that appear below the directory location of the initial context are returned. Therefore, it is useful to be as specific as possible when entering initial context information.

After you enter the appropriate directory path and click Next, all data sources that you can choose are listed by name in the Data Source Name list. Select the data source that you want to use and click Next to go to the Connection Information screen.

38.35.10 ODBC (RDO) dialog box

Data Source Selection screen

Select Data Source

Select this option to connect to ODBC data sources that you have already created with the ODBC Data Source Administrator on your local machine (any User DSN, File DSN, or System DSN).

For details about ODBC, data source names (DSNs), and the ODBC Data Source Administrator, see your Microsoft documentation.

Data Source Name

Use this list to select a particular DSN. This list shows all System DSNs on your machine, all User DSNs created for your Windows user account, and all File DSNs located in your default ODBC data sources folder (typically C:\Program Files\Common Files\ODBC\Data Sources).

Find File DSN

Select this option to use a File DSN that's located on your computer or network (that is, a File DSN in a location other than your default ODBC data sources folder).

File DSN

Enter the full path and name of your File DSN, or use the button adjacent to this field to search for your File DSN.

Enter Connection String

Select this option to connect to your data source by passing an ODBC connection string directly to the ODBC Driver Manager. This option allows you to connect to a data source through ODBC without first creating a DSN (so long as the connection string provides all relevant server information).

Connection String

Enter an ODBC connection string that contains all attribute/value pairs (separated by semi-colons) that are required in order to connect successfully to your data source through ODBC. Then click Finish.

Note

If you include your database password in the connection string, the information is saved with your report and is consequently visible to any other Crystal Reports user who opens your report in the designer.

Check your database and/or ODBC driver documentation for a complete list of attribute/value pairs that may be required. To view an example of the attribute/value pairs relevant to your data source, first use the ODBC Data Source Administrator to create a File DSN that allows you to connect to your data source. Save the File DSN to your local drive, and open the resultant text file (with a .dsn extension) in a text editor such as Notepad.

Connection Information screen

The fields on this screen are completed based on the DSN that you selected on the previous screen. In general, use the fields you see to specify the connection information (database identification and logon information) for your data source.

38.35.11 OLE DB (ADO) dialog box

OLE DB Provider screen

Provider

Use this list to select a configured OLE DB provider.

Use Data Link File

Select this check box if you have previously configured and saved a Data Link File. A Data Link File is OLE DB connection information saved as a file; it is similar to a File DSN that you might create in the ODBC Data Source Administrator.

Note

See your Microsoft programmer's documentation for information about creating a Data Link File.

Microsoft Data Link File

This field is available when you've selected the check box above it. Enter the full path and name of your Data Link File, or use the button adjacent to this field to search for your Data Link File.

Connection Information screen

The fields you see on this screen are different for each Provider. In general, use the fields you see to specify the connection information (database identification and logon information) for your data source.

Note

For some of the OLE DB providers, you can connect by selecting the *Integrated Security* checkbox. No user ID or password is required with this option, but it does require that Database Authentication is enabled on the server. Please consult your Database Administrator on how to enable and configure this function.

Advanced Information screen

Use this screen to edit various property values for the Provider that you chose in the OLE DB Provider screen. Select the property that you want to change, and click the Edit Value button to make the change.

38.35.12 Oracle 8 Server dialog box

Service

Enter the service name of your Oracle database. (You must create a service name in your Oracle client before you use this connection dialog box.)

User ID

Enter the user name required to connect to the data source.

Password

Enter the password for the appropriate user.

OS Authentication

Select this option if you want to use operating system authentication (such as NT authentication) to log on with instead of a separate User ID and password. When you select this option, User ID and Password become unavailable.

Note

This option may not be compatible with older versions of Oracle. See your Oracle documentation for further information about server and client configuration to enable OS authentication.

38.35.13 XML dialog box

XML (data source) type and location screen

Use Local Data Source

Select this option to connect to an XML file that you have on a local machine or a network drive.

Local XML File

Enter a full path and file name for your local data source, or select a file by searching for it in the Open dialog box.

Use HTTP(S) Data Source

Select this option to connect to an HTTP(S) data source (an XML file or XML stream that you can access from a Servlet, an ASP page, a JSP page, or another type of dynamically created web page).

HTTP(S) XML URL

Enter either an HTTP or an HTTPS URL for your HTTP(S) data source. When using HTTPS, user IDs and passwords that you enter in Crystal Reports or Crystal Enterprise are used to authenticate against the underlying HTTP provider.

Specify Schema File

Select this option if you want to apply a schema to the XML file you selected.

Note

You must select this option and specify a schema unless the XML file contains embedded schema information.

Validate XML

Select this option if you want the XML document validated against the schema, and you have indicated the XML file and schema.

Use Web Service Data Source

Select this option if you want to use a Web Service data source. This type of data source can be a service that you have on a local machine or a network drive that you can connect to by entering a path and file name, or it can be a service that you access from a Servlet or a dynamically created web page. The URL you provide to such a source can be either HTTP or HTTPS.

Schema file type and location screen

Use Local Schema

Select this option if you want to use a schema file that you have on a local machine or a network drive.

Local Schema File

Enter the full path and name of your schema file (XSD), or use the button adjacent to the field to search for it.

Use HTTP(S) Schema

Select this option if you want to use an HTTP(S) schema file.

HTTP(S) Schema URL

Enter the URL that points to your schema file (XSD).

HTTP Authentication for XML file screen

HTTP(S) XML User ID

Enter the user name required to connect to the XML file.

HTTP(S) XML Password

Enter the password for the appropriate user.

HTTP Parameters screen

Add Property

Click this button if you want to create a new HTTP parameter. The [Edit Property Value dialog box \[page 1227\]](#) appears. See [HTTP parameters \[page 673\]](#) for information about using this dialog box to create a new parameter.

Edit Value

Click this button if you want to edit an existing HTTP parameter. (You must select a parameter from the list before the button will work.) The Edit Property Value dialog box appears.

Remove Property

Click this button if you want to delete an existing HTTP parameter. (You must select a parameter from the list before the button will work.)

Web Services Location screen

Use Local WSDL

Select this option if you want to use a web services file that you have on a local machine or a network drive.

Local WSDL File

Enter the full path and name of your web services file, or use the button adjacent to the field to search for it.

Use HTTP(S) WSDL

Select this option if you want to use an HTTP(S) web services file.

HTTP(S) WSDL URL

Enter the URL that points to your web services file.

Web Service, Port and Method screen

Services

Choose a service from the list.

Ports

Choose a port from the list.

Methods

Choose a method from the list.

Web Services User ID

If required, enter the user ID for basic authentication.

Web Services Password

If required, enter the password for the User ID for basic authentication.

Related Information

[Native XML driver \[page 665\]](#)

[XML data source access types \[page 666\]](#)

[Schema \[page 669\]](#)

[HTTP parameters \[page 673\]](#)

[XML data source access types \[page 666\]](#)

38.36 Default Values for dialog box

This dialog box appears when you click a Default Values field for an argument in the [Custom Function Properties dialog box \[page 1190\]](#).

The complete dialog box name appends the selected argument's name and type. For example, if you select a DateTime argument called "DayInQuarter," the dialog box name appears as "Default Values for DayInQuarter (Date Time)."

The "Default Values for" dialog box lets you create a set of default values to be used in the Formula Expert. For detailed information about creating default values, see [Defining default argument values for custom functions \[page 736\]](#).

For information about the version of this dialog box that users will see in the Formula Expert, see [Enter Value for dialog box \[page 1231\]](#).

Use the upper list to create the default values you want your users to see. You can enter values directly or, as in the case of date values, select from a calendar or list.

The lower list contains the values you have defined. That is, it contains the values your users will see in the Formula Expert.

Add

Use the Add button to add values from the upper list to the lower list.

Remove

Use the Remove button to remove values from the lower list.

Up and Down

Use the Up and Down buttons to reorder values in the lower list.

38.37 Define Description dialog box

This dialog box enables you to define a description for a default parameter value. It appears when you click the Set Description button in the [Set Default Values dialog box \[page 1361\]](#).

Enter the description in the box and click OK.

38.38 Define Filter dialog box

This dialog box appears when you right-click an OLAP grid row or column member and select Add Filter from the shortcut menu. For details on filtering data in an OLAP grid, see [Filtering data in an OLAP grid \[page 410\]](#).

Filter Type

From this list, select how you want to filter the data that is displayed in the OLAP grid. You can filter data by "Actual values," or you can select only the "Top/bottom n" or the "Top/bottom n%."

Filter Definition

Specify your filter condition using the options and lists provided. For instance, if you selected "Actual values" as the Filter Type, you might choose to "Show columns" where the cell value is "greater than" a specified number.

Related Information

[Filtering data \[page 444\]](#)

[Filtering by actual value \[page 444\]](#)

[Filtering by ranking \[page 445\]](#)

[Filtering by contribution \(Pareto analysis\) \[page 446\]](#)

[Changing a filter \[page 447\]](#)

[Removing a filter \[page 447\]](#)

38.39 Define Named Group dialog box

This dialog box appears when you click either the New button or the Edit button on the Specified Order tab of various grouping dialog boxes. For more information about the Specified Order tab, see [Group Expert \[page 1428\]](#).

When you use specified order grouping, you have to use the Define Named Group dialog box to create the groups and to specify the values in each when you group in a specific order.

Group Name

Type the name you want to use for the group into this box. By default the name will appear in the group header section.

For example, if you have a group that contains customers from Arizona, Washington, and California, you can call that group "Western Region" by typing that name in this box.

There is a tab for the named group in which you can specify the selection criterion. Select a type of selection criterion from the list. To add another selection criterion, click the New tab.

Browse Data

Click this button to browse the field data. You can select data from the dialog box that opens and click the Paste button to paste data to the Selection list. Click the Done button to return to the Define Named Group dialog box.

Tabs

The tabs display the fields whose values you are using to create the group. The tabs are all related to the same group, and each tab relates to a specific portion of the query used to create the group. When you click a tab, the appropriate portion of the query appears in the dialog box. If you want to expand the query, click the New tab:

- To edit a named group, click the tab for that group.
- To create a new group, click the New tab.

Related Information

[Specified Order tab \(various dialog boxes\) \[page 1291\]](#)

[Group Expert \[page 1428\]](#)

[Grouping data \[page 173\]](#)

[Sorting data \[page 167\]](#)

[Creating custom groups \[page 174\]](#)

38.40 Dependency Checker dialog box

The Dependency Checker appears when you select the [Dependency Checker command \[page 1088\]](#) from the View menu, when you choose the [Check Dependencies command \[page 1119\]](#) from the Report menu, or when you click the Dependency Checker button on the Standard toolbar.

By default, the explorer appears docked in the Report Designer, but you can move it to another location. For more information, see [Opening and docking explorers \[page 117\]](#).

For more information about the this explorer, see [The Dependency Checker \[page 116\]](#).

38.41 Display Properties dialog box

The Display Properties dialog box appears when you click the Display button on the [Layer Control dialog box \[page 1300\]](#). The Display Properties dialog box allows you to further manipulate the mode and Zoom layering of a selected layer.

Display within Zoom Range

By clearing this check box, the selected layer is shown at all zoom levels of the map. By selecting this check box, the selected layer will only be shown within the range set in the Min Zoom and Max Zoom boxes.

Min Zoom

Use this box to set a minimum zoom level for the selected layer.

Max Zoom

Use this box to set a maximum zoom level for the selected layer.

38.42 Document Properties dialog box

This dialog box appears when you choose the [Summary Info command \[page 1077\]](#) from the File menu.

Use the Document Properties dialog box to:

- Add summary data to your report (to help you and other users identify the report).
- View the statistical information the program generates for each report.

The Document Properties dialog box has two tabs.

38.42.1 Summary tab (Document Properties dialog box)

Use the Summary tab to add summary information to your report file. You and other users can later use this information to help identify the content of the report.

Author

Type the name(s) of the report author(s) in this box.

Keywords

Type keywords that will help identify the content of the report in this box.

Comments

Type comments that you want saved with the report in this box. These comments might help describe the content of the report, the reason for the report's creation, and so on.

Note

If you include these comments on a report using the Special Field called [Report Comments](#), only the first 256 characters will be displayed.

Title

Type the report title in this box. A title entered here can be added to the report by using the Special Field called Report Title.

Subject

Type the overall subject of your report in this box.

Template

If you want to use this report as a template for formatting other reports, enter a identifying name here.

Save preview picture

Select this check box if you'd like to create a picture of the first page of the report so you can see it when selecting a report in the [Open dialog box \[page 1314\]](#). This check box is not selected by default.

Note

Preview Pictures are saved with the current data in your report. Ensure you do not have sensitive data in the Preview Picture you save.

Related Information

[Summary Info command \[page 1077\]](#)

[Statistics tab \(Document Properties dialog box\) \[page 1225\]](#)

38.42.2 Statistics tab (Document Properties dialog box)

Use the Statistics tab to view the document management information that the program generates for the report.

Last Saved By

This tab displays the name found in the Author box on the [Summary tab \(Document Properties dialog box\) \[page 1224\]](#) the last time you saved the report. If there was no name present when you last saved the report, this line is blank.

Revision number

This line displays the number of times the report has been revised. Each new report begins with the revision number set to zero (0). That value increases by one each time you save the report, even if you made no changes before saving.

Total Editing Time

This line displays the total number of minutes since you opened the report for editing, regardless of whether or not you have actually edited the report during that time.

Note

This value does not refresh while the dialog box is open. Each time you open the Document Properties dialog box, the program updates the value.

Last Printed

This line displays the date and time your report was sent to a printer.

Note

The value does not update when you export or preview the report.

Created

This line displays the date and time that your report was initially created.

Last Saved

This line displays the date and time that your report was last saved.

Related Information

[Summary Info command \[page 1077\]](#)

[Summary tab \(Document Properties dialog box\) \[page 1224\]](#)

38.43 Edit Parameter dialog box

This dialog box appears when you select a parameter from the Parameter Fields list in the [Field Explorer dialog box \[page 1241\]](#) and then click the Edit button. It also appears when you select a parameter in a report and choose the [Edit Report Object command \[page 1082\]](#) from the Edit menu, or when you right-click a parameter and choose Edit Parameter from the shortcut menu.

Use the Edit Parameter dialog box to change the list of values, binding, and/or other options for the selected parameter field.

For details about the options available in this dialog box, see the [Create Parameter Field dialog box \[page 1184\]](#).

Related Information

[Field Explorer dialog box \[page 1241\]](#)

38.44 Edit Parameter Field dialog box

This dialog box appears when you select a Row or Column for which you created a parameter in the [OLAP Expert \[page 1442\]](#) or the [OLAP Report Creation Wizard \[page 1382\]](#), and then click the Create/Edit Parameter button.

Use the Edit Parameter Field dialog box to change the prompting text, value type, default value(s), and/or option for the selected parameter field.

For details about the options available in this dialog box, see the [Create Parameter Field dialog box \[page 1184\]](#).

38.45 Edit Property Value dialog box

This dialog box appears when you add or edit an HTTP Parameter with the [Add Property](#) or [Edit Property](#) buttons on the HTTP Parameters screen.

In the Property field, enter a name for your parameter. In the Value field, enter a parameter formula in the form of a @Prompt function. See [Parameter formulas \[page 673\]](#) for information about the syntax of the @Prompt function.

38.46 Edit Running Total Field dialog box

The Edit Running Total Field dialog box appears when you select a running total field from the list in the [Field Explorer dialog box \[page 1241\]](#) and then click the Edit button. It also appears when you select a running total

field in a report and choose the [Edit Report Object command \[page 1082\]](#) from the Edit menu, or when you right-click a running total field and choose Edit Report Object from the shortcut menu.

For details about the options available in this dialog box, see the [Create Running Total Field dialog box \[page 1186\]](#).

Related Information

[Field Explorer dialog box \[page 1241\]](#)

38.47 Edit Summary dialog box

This dialog box appears when you select a summary field on a report and use the [Edit Report Object command \[page 1082\]](#). The command is also available when you:

- Right-click a summary field and choose the Edit Report Object command from the shortcut menu.
- When you click the Change Summary button for a summarized field in the [Cross-Tab Expert \[page 1412\]](#).

Note

For information about the Row and Column options, see [Showing values as percentages \[page 360\]](#).

- When you click the Set Summary Operation button on the [Data tab \(Map Expert\) \[page 1433\]](#) of the [Chart Expert \[page 1400\]](#) or [Map Expert \[page 1433\]](#).

Note

The Set Summary Operation button is available for charts and maps with an Advanced layout type. You must move a field to the "values" list and select it in order to activate the button.

Use the Edit Summary dialog box to set or change the summary operation for the selected field.

For details about the options available in this dialog box, see the [Insert Summary dialog box \[page 1298\]](#).

Related Information

[Summary \[page 1551\]](#)

38.48 Embedded Summaries dialog box

The [Embedded Summaries](#) dialog box appears when you choose the [Embedded Summary](#) command.

Use this dialog box to add, remove, and modify *Embedded Summaries*. You can also use it to specify the order in which *Embedded Summaries* are processed. The order can have an important effect on the results of your formulas.

New Embedded Summary

Click this button to create a new *Embedded Summary*.

Remove Embedded Summary

Click this button to remove an existing *Embedded Summary*.

Edit Embedded Summary

Click this button to edit the formula that determines the cell values of the *Embedded Summary*.

Description

Use this field to provide a description of the *Embedded Summary*.

38.49 Enter ODBC Table Name dialog box

The Enter ODBC Table Name dialog box appears when you export a report to ODBC format, choose your data source in the [ODBC Formats dialog box \[page 1312\]](#), and click OK.

Use the Enter ODBC Table Name dialog box to specify a name for the table that you want the program to add to the selected ODBC data source. This table will contain the information from the report that you export.

Related Information

[Exporting to an ODBC data source \[page 627\]](#)

38.50 Enter Parameter Values dialog box

This dialog box appears when you run a report that includes a parameter field, or when you refresh the data for that report.

Use the Enter Parameter Values dialog box to supply a parameter value if you want the program to run the report using a value other than the default value. This dialog box includes a list that includes each parameter field in the report.

Parameter Fields

When the Enter Parameter Values dialog box appears, it displays a list of all the parameter fields in your report. You can select any parameter field in the list to set a new value for it.

If prompting text was specified when the parameter field was created, that text appears below the Parameter Fields list on the Enter Parameter Values dialog box. The text changes as you select new parameter fields from the list.

Value options

The bottom part of the Enter Parameter Values dialog box contains a field or fields for you to enter the value(s) you want to use when running your report.

The fields that appear in this area vary depending on a number of things such as:

- The data type of the field the parameter is based on (String, Currency, Boolean, Date and Time, and so on).
- Whether the parameter is range limited (in other words, if it allows you to enter a start and end value).
- Whether the parameter allows the use of multiple values.

For example, if you created a parameter based on a date field and allowed a range to be specified, the value options on the Enter Parameter Values dialog box would include:

- *Start of range*
Select a value from a calendar to represent the first date in the range (including the date specified if you want).
- *End of range*
Select a value from a calendar to represent the last date in the range (including the date specified if you want).
- *No lower bound*
Select this check box to specify a date range with no start date.
- *No upper bound*
Select this check box to specify a date range with no end date.
- *Optional Prompt*
Set this to true if you want to allow the user to bypass entering a value for this parameter.

Related Information

[Refresh Report Data command \[page 1120\]](#)

[Refreshing report data \[page 857\]](#)

[Creating a parameter with a static prompt \[page 839\]](#)

38.51 Enter Search Name dialog box

The Enter Search Name dialog box opens when you choose Find Field from the Fields screen of any Report Creation Wizard except the OLAP Report Creation Wizard.

Use this dialog box to specify the name of the field you want to find.

38.52 Enter Value for dialog box

This dialog box appears when you select the Enter Constant Value option from the Value list in the Function Arguments area of the [Formula Expert \[page 1426\]](#).

Note

Not every argument type requires the use of this dialog box; simple types that don't have formatting restrictions (such as you'd encounter with a date field, for example), can be entered directly.

The complete dialog box name appends the selected argument's name and type. For example, if you select a DateTime argument called "DayInQuarter," the dialog box name appears as "Enter Value for DayInQuarter (DateTime)."

There are different versions of this dialog box to accommodate each of the following:

- The selection of a single value.
- The selection of a range of values.
- The selection of an array of values.
- The selection of an array of ranged values.

Single value

Use the options in this dialog box to select a single value to use in your formula. You can create a formula with a value such as `CDateTime (2001, 12, 07, 15, 50, 18)` by using this version of the dialog box.

Range of values

Use the options in this dialog box to select a starting and ending value to use as the bounds of your formula's range. You can create a formula with a range such as (10000 To 50000) by using this version of the dialog box.

Note

The options in the Value list let you define more exactly how the formula's range is to be calculated. The list includes options such as "is between, including endpoints," "is between, excluding endpoints," and so on.

Array of values

Use the options in this dialog box to select an array of values to use in your formula. You can create a formula with an array such as (["Canada", "England", "USA"]) by using this version of the dialog box.

Array of ranged values

Use the options in this dialog box to select an array of ranged values to use in your formula. You can create a formula with an ranged array such as (["AL" To "AZ", "CA" To "CT"]) by using this version of the dialog box.

Note

The options in the Value list let you define more exactly how each range in the formula's array is to be calculated. The list includes options such as "is between, including endpoints," "is between, excluding endpoints," and so on.

38.53 Excel Format Options dialog box

This dialog box appears when you choose an MS Excel option from the [Export dialog box \[page 1236\]](#).

38.53.1 Microsoft Excel

Column Width

The options in this area define the width of the columns in your output Excel worksheet based on a width determined by objects in various sections of your Crystal report or a point width (a constant width).

Note

Selecting "Whole Report" does not necessarily emulate the layout of your Crystal report. Rather, the Excel cell width is based on objects found in any section of your report. Likewise, for example, selecting "Details" creates cell widths based on the objects found only in the report's Details section.

Export page headers and page footers

You can choose to export your report's page headers and footers once for the entire report or repeatedly for each page in the report. If you choose Once Per Report, the header appears once at the top of your spreadsheet and the footer appears once at the bottom. If you choose On Each Page, the header and footer appear repeatedly throughout the spreadsheet in relation to the pagination of your report's data. If you choose None, page headers and footers are withheld.

Create page breaks for each page

Choose this option to maintain page breaks from your report that you can see in Excel's Page Break Preview. If this option is not selected, Excel determines the location of page breaks.

Note

Selecting this option does not guarantee absolute alignment with the page breaks Excel creates in the Page Break Preview.

Convert date values to strings

Choose this option to convert date fields in your report to string fields in Excel. This can be useful if you don't want Excel to change the formatting of your report's date information.

This option addresses the difficulty in handling dates prior to January 1, 1900 (or 1904). If your report contains dates prior to this, Excel displays an error and shows invalid values in the date cell.

Show Gridlines

Choose this option to see gridlines in Excel. Otherwise, the grid will be invisible in your Excel spreadsheet.

Page range

Use the options in this area to limit the number of report pages that you want to export.

38.53.2 Microsoft Excel - Data only

Excel Format

Choose a predefined option from this area:

- *Typical: Data is exported with default options applied.*
Choosing Typical automatically selects a number of options from the Options area and exports using those:
 - Use worksheet functions for summaries
 - Maintain column alignment
 - Export page header and footer
 - Simplify page headers
- *Minimal: Data is exported with no formatting applied.*
Choosing Minimal automatically selects "Use worksheet functions for summaries" from the Options area and exports using only that option.
- *Custom: Data is exported according to selected options.*
Choosing Custom exports using only options that you select from the Options area.

Options

Select Options to open the Options area of the dialog box.

Column width

The options in this area define the width of the columns in your output Excel worksheet based on a width determined by objects in various sections of your Crystal report or a point width (a constant width). For example, selecting "Details" creates cell widths based on the objects found only in the report's Details section.

Export object formatting

Choose this option to maintain formatting that you applied to report objects (such as font color, font size, background color, and so on) in Crystal Reports. The formatting that is exported is limited to the formatting that Excel supports.

Export images

Choose this option to export any images that are present in your Crystal report.

Use worksheet functions for summaries

Choose this option to create an Excel function for any summary field found in your Crystal report. This option applies only to those summaries for which a worksheet function can be created.

Maintain relative object position

Choose this option to retain the relative position of the report elements in your Crystal report. For example, if your report fields are placed two inches from the left margin of your Crystal report, the exported worksheet may contain several empty columns in lieu of the margin space; the data appears after the empty columns.

If this option is not selected, the program places all report objects from a section in a single row. For example, all report objects in the Report Header section would be exported to one row at the start of your Excel spreadsheet; all report objects in the Details section would be exported to another row (one row for each record), and so on.

Maintain column alignment

Choose this option to ensure that all related fields and their summaries from different sections align in the same Excel column. For instance, you may have a summarized field in the Details section and its summary in a Group Footer section. When you export such a report with this option selected, the program ensures that the values for the summarized field and the summary field export to the same column.

Note

Selecting this option makes the column 10 characters wide, regardless of the settings.

Note

This option is superseded by "Maintain relative object position." It is recommended that you use that option over "Maintain column alignment."

Export page header and page footer

Choose this option to export your Crystal report's page headers and footers for the entire report.

Simplify page headers

Choose this option to export the simplest version of your Crystal report's page headers. For example, if your page header includes two or more rows of objects, the program exports only the bottom row. If this option is not selected, the program exports all objects in the page header (to a single row in the spreadsheet).

Show group outlines

Choose this option to include group information from your Crystal report as outline information in the Excel spreadsheet. The group information appears as Excel outline symbols.

Note

Excel supports up to eight levels of outline information. All data is exported to Excel, but only the first eight levels of group information appear as Excel outline symbols.

38.54 Export dialog box

This dialog box appears when you choose the [Export Report command \(Export submenu\) \[page 1075\]](#) from the File menu.

Use the Export dialog box to export your report to an electronic file in a format that can be read by another software application. For example, you could export your report in Microsoft Excel format and then open it in Excel as a normal spreadsheet file.

The Export facility supports several popular word processing, database, and spreadsheet formats, as well as a number of standard data interchange formats.

Format

This is a list of all the possible export formats for your report. Select a format from this list.

Destination

Lists all the possible destinations for your report. Select the destination you want from this list.

You can export to:

- An application
- A disk file

- An Exchange folder
- Lotus Domino database
- Lotus Domino Mail
- MAPI

Related Information

[Exporting a report \[page 617\]](#)

38.55 Export Options dialog box

This dialog box appears when you export a report to one of these formats:

- Adobe Acrobat (PDF)
- Microsoft Word (RTF)
- Microsoft Word - Editable (RTF)
- Rich Text Format (RTF)

You can specify whether you want to export the entire report or a page range.

Note

In the case of the Adobe Acrobat (PDF) format, you can also specify whether you want to create bookmarks from the group tree. With this option, when you click an item in the PDF bookmark list, that page of the PDF file is displayed.

Note

In the case of the Microsoft Word - Editable (RTF) format, you can also specify whether you want the program to insert a page break after each report page. Such a page break is in addition to any page break that Word adds automatically. For example, if your Crystal Report page is slightly longer than a page in Word, choosing this option yields export output that takes an entire page in Word, plus a bit of the next page. The rest of this second page is left blank, and what was the second page of your Crystal report will start on page three of the Word document.

Note

Drill-down features of Crystal reports are not supported in the exported files. The first page of the report is exported, but none of the drill-down information is exported.

38.56 Export to Directory dialog box

The Export to Directory dialog box appears when you export a report and select the XML (Legacy) format in the [Export dialog box \[page 1236\]](#).

Directory Name

Enter a directory name for your XML page(s) and image(s), and then select the path location for where you want the program to create this directory. You must specify a directory for XML export because the program can produce more than one file for this export format.

Drives

Select any of your mapped drives as the location in which you want the program to create the XML file(s).

Base File Name

Specify a name to use as the base name for the XML file(s) that the program creates. If only one XML file is created, the base name is the name of that file. If several XML files are created, the base name is used as the root file name; a number is appended to this name for each page that is needed. For example, MyReport1, MyReport2, MyReport3, and so on.

Validation Type

You cannot change the validation type in this dialog box.

Related Information

[Exporting to XML format \[page 675\]](#)

38.57 Export to Text dialog box

The Export to Text dialog box appears when you export a report to Text (TXT) format.

Characters per inch

Use this option to specify the number of characters per inch that you want to see in the text of your output file. The characters-per-inch setting specifies the number of characters that are used in the text file to represent one inch of your report.

For example, if you specify eight characters per inch, but your report uses a font that is small enough to display the word "characters" in one inch, the output file would show you "characte". If you specified 16 characters per inch, the output file would show you "characters ". The larger the number you enter, the more spacing there is in your text; the smaller the number you enter, the more condensed your text becomes. Values in the range 8-16 are suitable for the most commonly used font sizes.

Number of Lines Per Page

Use this option to specify the number of lines per page that you want your output file to contain. When you make this calculation, you must include at least one line for each the Page Header and the Page Footer because those report areas are included on each output page. If your Page Header and Footer contain more than one line (including blank lines), you must add that number, as well. For example, if your Page Header contains a blank line and a field description, and your Page Footer contains a page number, you have three lines in those areas; you add this number to the number of lines in one Details section.

If you do not want output pages that contain only one Details section (plus the Page Header and Footer), you can specify a larger number. If you want only one Page Header and Footer, set the lines per page as zero.

38.58 Extract Custom Function from Formula dialog box

The Extract Custom Function from Formula dialog box appears—in the [Formula Workshop dialog box \[page 1282\]](#)—when you enter a name in the [Custom Function Name dialog box \[page 1189\]](#) and click the Use Extractor button.

Use this dialog box to create a custom function from an existing report formula. Doing so lets you migrate formula logic from a formula field to a custom function. You can then replace the appropriate part of your formula with the new custom function. Like all custom functions, those created through the Extract Custom Function from Formula dialog box can be added to the repository for use in other reports.

Formula

Use this list to choose an existing report formula to base your new custom function on. When you select a formula, information from it appears in various fields in the Extract Custom Function from Formula dialog box.

Custom Function Name

Use this field to change the name you assigned your custom function in the Custom Function Name dialog box, if you wish.

Note

Names for custom functions can be alphanumeric, but cannot contain spaces and cannot begin with a number.

Enter More Info

Click this button to see the [Custom Function Properties dialog box \[page 1190\]](#). You can add further information such as default argument values and help text while you are defining your custom function. If you don't add additional information now, you can right-click your custom function and select Properties later.

Return Type

This field displays your custom function's return type. The field cannot be changed because the return type is based on the report formula you selected in the Formula list.

Summary

Use this area to add additional information about your custom function to help identify its purpose, intended use, and so on.

Arguments

This area displays field-based arguments from the report formula you selected in the Formula list. While the Field Name and Type cannot be changed, you can enter a name and description for the argument. If you don't assign an Argument Name, the default names (v1, v2, v3, and so on) are used to describe the variables.

Arrow buttons

Use the arrow button adjacent to the Arguments area to change the default order (that is, the order in which the fields are used in the report formula) of the arguments.

Formula Text

This area displays the report formula as it was defined in the [Formula Editor dialog box \[page 1280\]](#). The text is only for information and cannot be edited.

Modify formula to use new custom function

Select this option if you want to use your new custom function in the report formula you are basing it on. In other words, the appropriate formula text is replaced by a call to the custom function with the fields used in the formula supplied as argument values.

38.59 Field Explorer dialog box

The Field Explorer is used to insert, modify or delete fields on the Design and Preview tabs of Crystal Reports. You'll see the Field Explorer after you have:

- Clicked the Field Explorer button on the Standard toolbar.
- Selected Field Explorer from the View menu.

The Field Explorer displays a tree view of database fields and special fields that you can add to your report. It also shows formula fields, SQL expression fields, parameter fields, group name fields, and running total fields that you have defined for use in your report.

Fields that have already been added to the report, or fields that have been used by other fields (such as formula fields, groups, running total fields, summaries, and so on) have a check mark in front of them.

Note

You can drag items from the Field Explorer to replace other items in your report. For example, if you have formatted a field in your report and you want to replace that field while maintaining its formatting, you can switch the fields by holding the *Shift* key while you drag the replacement field from the Field Explorer.

The Field Explorer can be docked. For more information about docking explorers, see [Opening and docking explorers \[page 117\]](#).

For a description of the toolbars and shortcut menu items available in the Field Explorer, see [Toolbar and Shortcut Menu \[page 89\]](#).

38.59.1 Database Fields (Field Explorer dialog box)

Use the Database Fields branch of the tree to add database fields to your report. Fields are listed by table.

If there are many active tables, or if the number of fields in a table is very large, only the table name for each of the tables appears. When you click the + sign in front of the table, or double-click the table name itself, the

program displays the fields in that table. Double-click the table name again and the program displays only the table name.

You insert database fields by dragging them into the report, or by selecting them and then clicking Insert to Report. You can select multiple fields using the [Shift](#)-click or [Ctrl](#)-click combinations.

Each field added to the report has a check mark added in front of its name in the Field Explorer.

Insert to Report

Click this button to insert the selected field(s) in your report. When you do, the program displays a placement frame. Move that frame where you want it on the report and click to place it.

Browse

Click this button to browse the data in the selected field.

38.59.2 Formula Fields (Field Explorer dialog box)

Use the Formula Field branch of the tree as a gateway for editing, renaming, or deleting formulas, creating new formulas, or inserting formula fields into your report.

The formula list displays the names of all the formula fields currently in your report. You insert formula fields by dragging them into the report, or by selecting them and then clicking Insert to Report.

Each formula field added to the report has a check mark added in front of its name in the Field Explorer.

Insert to Report

Use this button to insert the formula field selected in the Formula Fields list into your report.

New

This button opens the [Formula Name dialog box \[page 1282\]](#) where you can specify a name for your formula. This option is available from the root of the Formula Fields branch so you can create a new formula for your report even when there are no existing formulas.

Edit

This button opens the Formula Workshop and displays the currently selected formula for you to edit.

Rename

This button lets you rename the formula you select in the Formula Fields list. Formula Fields can be renamed in the standard Windows fashion.

Delete

This button deletes the formula field selected in the Formula Field list. If the formula is already on the report, you are asked to confirm the deletion as it will also be deleted from the report.

Related Information

[Working with the Formula Workshop \[page 693\]](#)

38.59.3 SQL Expression Fields (Field Explorer dialog box)

Use the SQL Expression Fields branch of the tree as a gateway for editing, renaming, or deleting SQL Expression fields, creating new SQL Expression fields, or inserting SQL Expression fields into your report.

The SQL Expression Fields list displays the names of all the expression fields currently in your report. You insert SQL Expression fields by dragging them into the report, or by selecting them and then clicking Insert to Report.

Each SQL Expression field added to the report has a check mark added in front of its name in the Field Explorer.

Insert to Report

Use this button to insert the SQL Expression field selected in the SQL Expression Fields list into your report.

New

This button opens the [SQL Expression Name dialog box \[page 1367\]](#) where you can specify a name for your expression. This option is available from the root of the SQL Expression Fields branch so you can create a new SQL Expression for your report even when there are no existing expressions.

To add a new SQL Expression, add a name in the SQL Expression Name dialog box, click OK, and create a expression in the [Formula Editor dialog box \[page 1280\]](#).

Edit

This button opens the SQL Expression Editor and displays the currently selected expression for you to edit.

Rename

This button lets you rename the expression you select in the SQL Expression Fields list. SQL Expression Fields can be renamed in the standard Windows fashion.

Delete

This button deletes the expression field selected in the SQL Expression Fields list. If the expression is already on the report, you are asked to confirm the deletion as it will also be deleted from the report.

38.59.4 Parameter Fields (Field Explorer dialog box)

Use the Parameter Fields branch of the tree as a gateway for editing, renaming, deleting, creating new, or inserting parameter fields and prompts into your report.

The Parameter Fields list displays the names of all the parameter fields currently in your report. You insert Parameter fields by dragging them into the report, or by selecting them and then clicking Insert to Report.

Each Parameter field added to the report has a check mark added in front of its name in the Field Explorer.

Insert to Report

Use this button to insert the Parameter field selected in the Parameter Fields list into your report.

New

This button opens the [Create Parameter Field dialog box \[page 1184\]](#) where you can specify the details for your parameter. This option is available from the root of the Parameter Fields branch so you can create a new parameter for your report even when there are no existing parameters.

To add a new Parameter Field, add a name, prompting text, and value type in the Create Parameter Field dialog box. After choosing options and setting default values, click OK.

Edit

This button opens the [Edit Parameter dialog box \[page 1227\]](#) and displays the currently selected parameter for you to edit.

Rename

This button lets you rename the parameter you select in the Parameter Fields list. Parameter Fields can be renamed in the standard Windows fashion.

Delete

This button deletes the parameter field selected in the Parameter Fields list. If the parameter is already on the report, you are asked to confirm the deletion as it will also be deleted from the report.

Related Information

[Creating a parameter with a static prompt \[page 839\]](#)

38.59.5 Running Total Fields (Field Explorer dialog box)

Use the Running Total Fields branch of the tree as a gateway for editing, renaming, deleting, creating new, or inserting Running Total fields into your report.

The Running Total Fields list displays the names of all the Running Total fields currently in your report. You insert Running Total fields by dragging them into the report, or by selecting them and then clicking Insert to Report.

Each Running Total field added to the report has a check mark added in front of its name in the Field Explorer.

Insert to Report

Use this button to insert the Running Total field selected in the Running Total Fields list into your report.

New

This button opens the [Create Running Total Field dialog box \[page 1186\]](#) where you can specify the details for your Running Total. This option is available from the root of the Running Total Fields branch so you can create a new Running Total for your report even when there are none existing.

To add a new Running Total Field, add a name, field to summarize, and Evaluate/Reset criteria in the Create Running Total Field dialog box and click OK.

Edit

This button opens the [Edit Running Total Field dialog box \[page 1227\]](#) and displays the currently selected Running Total for you to edit.

Rename

This button lets you rename the Running Total you select in the Running Total Fields list. Running Total Fields can be renamed in the standard Windows fashion.

Delete

This button deletes the Running Total field selected in the Running Total Fields list. If the Running Total is already on the report, you are asked to confirm the deletion as it will also be deleted from the report.

Related Information

[Creating running totals \[page 209\]](#)

38.59.6 Group Name Fields (Field Explorer dialog box)

You can insert an existing Group Name field shown in the Field Explorer by selecting Insert to Report. Unlike a formula field, parameter field, or running total field, however, you cannot create a Group Name field through the Field Explorer. (A Group Name field is created when you insert a group.)

The Group Name Fields list displays the Group Name fields currently in your report.

Insert to Report

Click this button to insert the Group Name field selected in the Group Name Fields list.

Related Information

[Grouping data \[page 173\]](#)

38.59.7 Special Fields (Field Explorer dialog box)

To display information such as Page Numbers, Print Date, and Report Comments use the commands in the Special Fields list in the Field Explorer.

Content Locale

Use the Content Locale Field to include a field that displays the current *Preferred Viewing Locale* (PVL) setting.

Current CE User ID

Use the Current CE User ID Field to include a field that displays the ID number of the current SAP BusinessObjects Enterprise user (if one exists).

Current CE User Name

Use the Current CE User Name Field to include a field that displays the name of the current SAP BusinessObjects Enterprise user (if one exists).

Current CE User Time Zone

Use the Current CE User Time Zone Field to include a field that displays the time zone of the current SAP BusinessObjects Enterprise user (if one exists).

Data Date

Use the Data Date Field to include a field that contains the date the data was last retrieved (refreshed). This field can be placed in any section of your report, depending on how often you want it to print.

Data Time

Use the Data Time Field to include a field that contains the time the data was last retrieved (refreshed). This field can be placed in any section of your report, depending on how often you want it to print.

Data Time Zone

Use the Data Time Zone Field to include a field that contains the time zone in which the data was last retrieved (refreshed). This field can be placed in any section of your report, depending on how often you want it to print.

Note

Use this field in conjunction with Data Date and Data Time.

File Author

Use the File Author Field command to include a field that displays the name of the author of the report (as specified in the [Document Properties dialog box \[page 1224\]](#)).

File Creation Date

Use the File Creation Date Field command to include a field that displays the date when you created the report.

File Path and Name

Use the File Path and NameField command to insert a field that displays the file path and file name for the report.

Group Number

Use the Group Number Field to number each group in your report. You can place this field in either the Group Header or Group Footer section of your report.

Group Selection Formula

Use the Group Selection Formula Field to insert a group selection formula field into your report. (Use the [Select Expert \[page 1457\]](#) to create a group selection formula for your report.)

Horizontal Page Number

Use the Horizontal Page Number Field to number the horizontal pages created by report objects, such as cross-tabs and OLAP grids, that can expand over a number of horizontal pages.

Modification Date

Use the Modification Date Field to include a field that contains the date the report was last modified. Modified refers to any modification (including something as simple as moving a field). When you modify the report and print it, the program prints the modification date even if you have not saved the report before printing. This field can be placed in any section of your report, depending on how often you want it to print.

Modification Time

Use the Modification Time Field to include a field that contains the time the report was last modified. This field works the same way as the Modification Date. This field can be placed in any section of your report, depending on how often you want it to print.

Page N of M

Use the Page N of M Field to insert a field that specifies the page number and the total number of pages in the report. For example, a field value of Page 1 of 8 would indicate the first page in an eight-page report.

Page Number

Use the Page Number Field to insert a field that prints the current page number. These fields are most often placed in the Page Header or Page Footer sections.

Print Date

Use the Print Date Field to include the current date when the report prints. This field can be placed in any section of your report, depending on how often you want it to print. The date can be changed by using the [Set Print Date and Time command \[page 1121\]](#) on the Report menu.

Print Time

Use the Print Time Field to include a field that contains the time when the report prints. This field can be placed in any section of your report, depending on how often you want it to print.

Print Time Zone

Use the Print Time Zone Field to include a field that contains the time zone in which the report prints. This field can be placed in any section of your report, depending on how often you want it to print.

Note

Use this field in conjunction with Print Date and Print Time.

Record Number

Use the Record Number Field to number each record printed in the Details section of your report.

Record Selection Formula

Use the Record Selection Formula Field to insert a record selection formula field into your report. (Use the [Select Expert \[page 1457\]](#) to create a record selection formula for your report.)

Report Comments

Use the Report Comments Field to include a field that contains the comments you specified using the [Summary Info command \[page 1077\]](#) on the File menu. This field can be placed in any section of your report, depending on how often you want it to print.

Note

In version 8.5 of Crystal Reports, only the first 254 characters of the comments you entered on the Summary tab of the [Document Properties dialog box \[page 1224\]](#) are displayed. In later versions, Crystal Reports can display up to 30,000 characters.

Report Title

Use the Report Title Field to include a field that contains the title you specified using the [Summary Info command \[page 1077\]](#) on the File menu. This field can be placed in any section of your report, depending on how often you want it to print.

Selection Locale

Use the Selection Locale Field to include a field that displays the locale setting of the machine that your report is running on (the machine's locale is usually found in the Control Panel's Regional Settings).

Note

This special field is based on a constant function that you can use in the Crystal Reports formula language (including record and group selection filters). The Selection Locale Field is updated each time you refresh your report's data.

Total Page Count

Use the Total Page Count Field to print the total number of pages in the report. You can use this field in a variety of instances. For example, you can use it in combination with the Page Number Field to create a text object that reads "Page x of y" where x is the Page Number Field and y is the Total Page Count Field. These fields can be placed in any section of your report, depending on how often you want them to print.

38.60 Find dialog box

The Find dialog box allows you to locate text in a report. This dialog box appears when you choose the [Find command \[page 1081\]](#) from the Edit menu.

Find what

Type the text you want to search for, paste it from the clipboard or click a recent entry from the list.

Find Next

Click this button to find and select the next occurrence of the text specified in the Find what box.

Match case

Select or clear this check box to control whether the exact sequence of upper and lower case letters in the search text should be considered.

Match whole word only

Select or clear this check box to control whether only whole words are to be found instead of searching for text within words.

Direction

Specifies which direction to search starting from the current cursor position. Click Up to search toward to start of the report or Down to search toward the end of the report.

Advanced Find

Click this button to call the [Search Expert \[page 1447\]](#).

This button is only available in the Preview tab.

38.61 Find dialog box (Formula Editor)

The Find dialog box allows you to locate text in the Formula Workshop and replace it if required. This dialog box appears when you click the Find or Replace button on the [Formula Editor dialog box \[page 1280\]](#) toolbar.

Find what

Type the text you want to search for, paste it from the clipboard or click a recent entry from the list.

Find All

Click this button to find all occurrences of the text specified in the Find what box. Crystal Reports provides you with a list of the formulas that contain the text that you specified. Double-click the formula's name to go to the text in the Formula Editor.

Replace with

Type the text you want to replace the find text with.

Mark All

Click this button to mark all occurrences of the find text.

This option is only available when Edit Text is selected.

Replace

Click this button to search for text that matches the find text, then replace it with the text in the Replace with field.

This option is only available when Edit Text is selected.

Replace All

Replaces all occurrences of the find text with the text in the Replace with field.

This option is only available when Edit Text is selected.

Search

Click an option to specify the type of text to search for.

Click either All SQL Expressions or All Formulas to carry out a global search of all SQL Expressions or formulas.

Direction

Specifies which direction to search starting from the current cursor position. Click Up to search toward to start of the report or Down to search toward the end of the report.

Match Case

Select or clear this check box to control whether the exact sequence of upper and lower case letters in the search text should be considered.

38.62 Font dialog box

The Font dialog box appears when you click any of the buttons on the [Fonts tab \(Options dialog box\) \[page 1331\]](#) in the [Options dialog box \[page 1314\]](#), or when you click the Font button on the [Text tab \(Chart Expert\) \[page 1410\]](#) of the [Chart Expert \[page 1400\]](#).

Use the Font dialog box to change the fonts, the font size, and/or the font style for text and data fields on your reports.

Font

Lists all the available fonts. Click a font name to select a different font.

Font style

Lists the available styles for the specified font. Click a font style to select a different style.

Size

Lists the available point sizes for the specified font. Click a size to change the size of the font.

Color

Displays the colors you can choose to apply to your font. Click More to see the [Color dialog box \[page 1168\]](#) so you can define further font colors.

ⓘ Note

This option does not appear in the Chart Expert.

Strikeout

Specifies whether the font should appear with strikeout marks.

Underline

Specifies whether the font should appear with underlines.

Preview

Shows a sample of how the text will appear with the specified font settings.

Related Information

[Options dialog box \[page 1314\]](#)

[Formatting \[page 235\]](#)

38.63 Format dialog box (OLAP Worksheet)

The Format dialog box appears when you right-click a row or column, point to Format, and select Add.

Use this dialog box to set formatting for data in your Worksheet.

Number tab

For a procedure about using this tab and its fields, see [Formatting data in the Worksheet \[page 434\]](#).

Note

The formatting options on this tab change for each item that you select in the Type list.

Font tab

For a procedure about using this tab and its fields, see [Formatting data in the Worksheet \[page 434\]](#).

Note

Options similar to those on this tab are described in [Font tab \(various dialog boxes\) \[page 1274\]](#).

38.64 Format Editor dialog box

With the Format Editor, you can change the appearance of report fields and other report objects. At its simplest, the Format Editor has one tab. Other tabs are added as you select other field types (Date and Time, Boolean, Hyperlink, and so on).

The Format Editor appears when:

- You select a report object and select the [Format command \[page 1101\]](#) from the Format menu or the right-click menu.
- You select a report object and select the [Hyperlink command \[page 1102\]](#) from the Format menu.
- You click any of the buttons on the Fields tab of the [Options dialog box \[page 1314\]](#).

The Format Editor contains tabs specific to the type of field specified on the fields tab of the [Options dialog box \[page 1314\]](#), or to the type of field you selected.

38.64.1 Boolean tab (Format Editor dialog box)

Use the Boolean tab to select the format for the return values of Boolean fields in your report.

Boolean Text

This list includes all the formats in which Boolean fields can be displayed. Select the desired format from this list.

38.64.2 Border tab (Format Editor dialog box)

Use the Border tab to format the borders, background fill, and drop shadows for selected fields on your report. Using the options on the border tab, you can create a variety of effects that can enhance both the look of your report and highlight important data.

Left

This list contains the different line styles available. Your choices include single, double, dashed, and dotted line borders. Select the desired style and the program will insert the specified border on the left side of the field. The default value is None.

Right

This list contains the different line styles available. Your choices include single, double, dashed, and dotted line borders. Select the desired style and the program will insert the specified border on the right side of the field. The default value is None.

Top

This list contains the different line styles available. Your choices include single, double, dashed, and dotted line borders. Select the desired style and the program will insert the specified border on the top of the field. The default value is set to None.

Bottom

This list contains the different line styles available. Your choices include single, double, dashed, and dotted line borders. Select the desired style and the program will put the specified border on the bottom of the field. The default value is None.

Tight Horizontal

Select this check box to trim the border to the size of the field.

- When this check box is not selected, the border will be the same size for each record.
- When this check box is selected, the border will be trimmed to match the size of each individual record.

Drop Shadow

Select this check box if you want to print a drop shadow below and to the right of the field.

Border

This list contains the colors available for use as borders.

Background

Select this check box if you want to change the background color. When you select this check box, a list appears to the right, allowing you to select a new background color.

Related Information

[Formatting \[page 235\]](#)

38.64.3 Common tab (Format Editor dialog box)

Use the Common tab to set properties like suppress, horizontal alignment, and keep object together, for a desired field.

Object Name

Use this field to define a name for the object you are formatting. This name is used when dragging and dropping items to the repository, when viewing objects in the [Report Explorer dialog box \[page 1344\]](#), and when defining Report Part hyperlinks.

A default name is created for each field in a report. If the default name suits you, you can ignore this field.

Tool Tip Text

You can add Tool Tip text for the selected field by clicking the Format Formula Workshop button and entering text within quotation marks. Tool Tip text appears when you leave your mouse pointer over a field for a few seconds.

Note

Tool Tips must be selected in the Preview area of the [Options dialog box \[page 1314\]](#) in order for Tool Tips to appear on the Preview tab.

Read-only

Use this option to make the selected report object read-only so it can't be formatted. When you select this option, all other choices in the Format Editor except "Repeat on Horizontal Pages" and "Lock Position and Size" become inactive. The formatting options that are usually available on the toolbars or shortcut menus are also suppressed for the object.

Note

This feature is for your convenience in protecting report formatting; it is not intended to be used as report security.

Lock Position and Size

Use this option to lock the position of the selected report object so it can't be moved. When you select this option, you cannot drag the object in the Report Designer and the Size and Position command becomes inactive.

Suppress

Select this check box if you want to suppress the selected field. The field values won't appear on the Preview tab, nor will they print.

Suppress if Duplicated

Select this check box if you don't want the program to show field values that are duplicated when it displays or prints the report.

This option compares record values, not formatted field values. The program ignores the option in the first Detail section of a formatted page.

Suppress Embedded Field Blank Lines

Select this check box if you want the program to avoid printing blank fields—that is, fields for which the database returns no information. This option prevents blank lines from appearing in your report when some fields are empty.

Can Grow

This check box is useful for objects that contain variable length strings, such as memo fields.

- Select this check box if you want the program to expand the object frame vertically when the text within the object is larger than the frame.
- Clear this check box to truncate the data when the value is larger than the frame.

By default, the Can Grow option is off.

Max number of lines

If you have selected the Can Grow check box, you can use this box to specify the maximum number of lines by which the program can expand an object frame. Simply enter the desired number of lines into this box or enter a 0 (zero) if you do not want to set a limit.

Text Rotation

This list contains the rotation options available for the selected field. Use the Text Rotation options to vertically align the fields and text-based objects on your report.

Horizontal Alignment

This list contains the horizontal alignment types available for the field (for example, left, center, and right).

Display String

Click the conditional formatting button to enter a formula that lets you display number, currency, date, time, date and time, Boolean, or even string fields as a custom string. This type of conditional formatting is not limited to database fields; you can use it for running totals, formulas, parameter fields, summaries, and so on.

For example, suppose you want to use a notation in your report that represents millions as M and thousands as K. You can create a display string formula for your report by selecting the appropriate numeric field and entering this formula in the Format Formula Editor:

```
if CurrentFieldValue >= 1000000
then ToText(CurrentFieldValue/1000000, 2) + "M"
else if CurrentFieldValue >= 1000
then ToText(CurrentFieldValue/1000, 2) + "K"
else ToText(CurrentFieldValue, 2)
```

CSS Class Name

Use this field to specify a class name for the report object you're formatting. The class name can be used in your CSS to indicate a unique style for the selected report object.

You can also create a conditional name for the class by using the [Formula Editor dialog box \[page 1280\]](#). Doing so allows you to indicate in your CSS that certain repetitions of an object have different formatting from others.

Repeat on Horizontal Pages

Use this option to repeat a report object that does not expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, on each additional horizontal page that a cross-tab or OLAP grid might create. For more information, see [Repeating report objects on horizontal pages \[page 283\]](#).

Keep Object Together

Select this check box to keep an object on a single page. If there is enough room, the program prints the object on the current page. If there is not enough room, the program prints the object on the next page.

Note

If you have objects that are larger than a page, this property will not work. Such objects are always split across pages.

Close Border on Page Break

If an object has a border and the object is split over two pages, you have two options: put a border around each part of the split object, or display each part of the object with a partial border. Select this check box to display a full border.

Related Information

[Formatting \[page 235\]](#)

[Vertical placement \[page 264\]](#)

38.64.4 Number tab (Format Editor dialog box)

Use the Number tab to set the formatting options for number or currency fields in your report.

Style

From this predefined list, choose a number/currency style to apply to the field selected on your report. When you choose System Default Currency Format, the program uses the number and currency formats from the Regional Settings Properties dialog box in the Windows Control Panel.

Note

You can create a custom number/currency format by clicking the Customize button. Your customized style appears in the style list as Custom Style.

Display currency symbol

Select the Display currency symbol check box if you want to display currency symbols in your currency fields.

Fixed

Click this option if you want the program to place the currency symbol in the first (far left) position in the field. The program will leave extra space between the symbol and the currency if the field is justified to the right.

For example:

\$ 325.00

\$ 401,325.00

Floating

Click this option if you want the program to place the currency symbol as close to the currency amount as possible so that there are never any extra spaces between the symbol and the amount.

For example:

\$401,325.00

\$325.00

Customize

Use this button to see more detailed formatting options on the [Custom Style dialog box \[page 1192\]](#).

Prefix

Use this field to insert text in front of values displayed by a Horizontal Page Number field.

Suffix

Use this field to append text to values displayed by a Horizontal Page Number field.

Suppress if No Horizontal Pages

Select this check box to suppress a Horizontal Page Number field when the report is only one page wide.

Note

The Prefix, Suffix, and Suppress if No Horizontal Pages options appear only when you format the Special Field called Horizontal Page Number.

Related Information

[Currency Symbol tab \(Custom Style dialog box\) \[page 1192\]](#)

[Number tab \(Custom Style dialog box\) \[page 1194\]](#)

38.64.5 Date and Time tab (Format Editor dialog box)

Use the Date and Time tab to set the formatting options for the date and time fields in your report.

Style

From this predefined list, choose a date/time style to apply to the field selected on your report. When you choose System Default Long Format or System Default Short Format, the program uses the time and date formats from the Regional Settings Properties dialog box in the Windows Control Panel.

Note

You can create a custom date/time format by clicking the Customize button. Your customized style appears in the style list as Custom Style.

Customize

Use this button to see more detailed formatting options on the [Custom Style dialog box \[page 1192\]](#).

Related Information

[Date and Time tab \(Custom Style dialog box\) \[page 1197\]](#)

[Date tab \(Custom Style dialog box\) \[page 1197\]](#)

[Time tab \(Custom Style dialog box\) \[page 1202\]](#)

38.64.6 Box tab (Format Editor dialog box)

Use the Box tab to change the fill, border, width and style of a box object in your report.

Style

This list shows all the available line styles for your box; such as: single, double, dashed, and so on.

Width

Click the button which represents your desired border line width. This option is available only for Single border lines, and not for Dashed or Dotted border lines.

Color

This palette shows all the available colors for your box. Select the desired color from the palette or click More to see the [Color dialog box \[page 1168\]](#).

Drop Shadow

Select this check box if you want to have a drop shadow below and to the right of your box.

Fill

If you select the Color check box, a list appears showing all the available colors to fill your box.

Close Border on Page Breaks

Select this check box if you want the program to convert box segments into finished boxes when the program splits a box due to a line break. For example, if you have a one inch square box which is split in half, the segment on the first page is missing a bottom, the segment on the second page is missing a top. When this property is selected, the program adds the bottom to the first box and the top to the second, creating a finished half inch box on each page.

Extend to Bottom of Section when Printing

Select this check box if you want the program to allow the box to extend to the bottom of the section.

Suppress

Select this check box if you do not want the box to print.

Read-only

Use this option to make the box read-only so it can't be formatted. When you select this option, all other choices on the Format Editor become inactive.

Note

This feature is for your convenience in protecting report formatting; it is not intended to be used as report security.

Repeat on Horizontal Pages

Use this option to repeat a report object that does not expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, on each additional horizontal page that a cross-tab or OLAP grid might create. For more information, see [Repeating report objects on horizontal pages \[page 283\]](#).

Lock Position and Size

Use this option to lock the position of the box so it can't be moved. When you select this option, you cannot drag the box in the Report Designer and the Size and Position command becomes inactive.

Related Information

[Format Editor dialog box \[page 1256\]](#)

[Box command \[page 1098\]](#)

[Adding and editing boxes \[page 277\]](#)

38.64.7 Line tab (Format Editor dialog box)

Use the Line tab to set the formatting specification for a line object in your report.

Style

This list shows all the available styles for your line, such as: Single, Dashed, Dotted, and so on.

Width

Click the button that represents the line width you want to use. This option is available only for Single lines, and not for Dashed or Dotted lines.

Color

This palette shows all the available colors for your box. Select the desired color from the palette or click More to see the [Color dialog box \[page 1168\]](#).

Move to bottom of Section when Printing

This option is typically selected when you are printing a horizontal line next to a field that may produce variable length output (a memo field, for example) and you format that field using the [Can Grow](#) option (on the Common tab of the Format Editor). In this case, if the program needs to grow the section to accommodate field output, it will move the line to the bottom of the section so that it remains at the bottom of the field.

- Select this check box if you want the line to always move to the bottom of the section regardless of the length of the section.
- Clear the check box if you want the line to be in the same vertical location in each section, regardless of section size.

Extend to Bottom of Section when Printing

This option is typically selected when you are printing a vertical line next to a field that may produce variable length output (a memo field, for example) and you format that field using the [Can Grow](#) option (on the Common tab of the Format Editor). In this case, if the program needs to grow the section to accommodate field output, it will lengthen the line to the bottom of the section as well.

- Select this check box if you want the line to always extend to the bottom of the section regardless of the length of the section.
- Clear the check box if you want the line to be the same length in each section, regardless of section size.

Suppress

Select this check box if you do not want the line to print.

Read-only

Use this option to make the line read-only so it can't be formatted. When you select this option, all other choices on the Format Editor become inactive.

Note

This feature is for your convenience in protecting report formatting; it is not intended to be used as report security.

Repeat on Horizontal Pages

Use this option to repeat a report object that does not expand horizontally, such as text objects, field objects, OLE objects, charts, maps, lines, boxes, and so on, on each additional horizontal page that a cross-tab or OLAP grid might create. For more information, see [Repeating report objects on horizontal pages \[page 283\]](#).

Lock Position and Size

Use this option to lock the position of the line so it can't be moved. When you select this option, you cannot drag the line in the Report Designer and the Size and Position command becomes inactive.

Related Information

[Format Editor dialog box \[page 1256\]](#)

[Line command \[page 1097\]](#)

[Adding and editing boxes \[page 277\]](#)

38.64.8 Picture tab (Format Editor dialog box)

Use the Picture tab to format bitmap graphics in your report.

Crop From edit boxes

Cropping refers to "cutting" away those portions of your graphic that you do not want to print (although you can use cropping to add white space between the graphic and the frame that surrounds it as well). Using the cropping box, specify the size of the piece you want to cut off the top, bottom, left, and/or right side of your graphic:

- Positive numbers cut into the graphic the amount specified.
- Negative numbers add the specified amount of white space between the outer edge of the graphic and the frame.

Note

All cropping activities begin at the outer edge of the graphic.

Scaling edit boxes

Scaling refers to the length and width of a graphic as a percentage of the original length and width. For example, if the original graphic is one inch wide, that width is automatically assigned a width scaling value of 100% by the program. To double the width of the graphic using the Scaling options, you would change Scaling Width to 200% (twice the size of the original). Likewise, to reduce the width of the graphic to one-half inch (half the size of the original), you would change Scaling Width to 50%. Scaling Height works in the same way.

If you want to resize the graphic as a percentage of the original height and width, enter the new scaling percentages.

The program stores a copy of each graphic in its original size. All scaling settings refer back to that original size. For example, if you have a graphic that was originally a four inch square and you have resized it to a two inch square, the Scaling box will show settings of Width = 50% and Height = 50%. Those percentages refer back to the original. If you want to resize the graphic again to a one inch square, you will have to enter scaling values that again refer back to the original Width and Height, not the current values. Entering values of Width = 25% and Length = 25% will reduce the original four inch square to a one inch square.

Size edit boxes

Size refers to the absolute (measured) length and width of a graphic. For example, if a graphic is originally a one inch square, each of the Size settings will initially be set at one inch. To double the length and width of the graphic (to make a two inch square), reset the Size settings to two inches each. To reduce the size of the graphic to a half inch square, reset the Size settings to a half inch each.

Note

When you change the Size settings, the program automatically recalculates the Scaling settings, and when you change the Scaling settings, the program automatically recalculates the Size settings. The recalculated settings appear when you next open the Picture tab of the [Format Editor dialog box \[page 1256\]](#).

Original size information

This box displays the original dimensions of the graphic (its dimensions when first inserted into the report). Sizing, scaling, and cropping do not affect this figure nor can you change it directly; it is simply provided as a reference.

Dynamic picture options

This box controls how dynamic pictures are displayed (Stretch / Respect Scaling / Fit to box), e.g. those from database (BLOB field) or from HTTP URL (defined in Graphic Location). It also controls whether dynamic JPG pictures are rotated, based on orientation tag stored in each JPG picture's EXIF data.

For more information, check [To scale, crop, or size an object \[page 280\]](#).

Reset

When you click this button, any changes you have made to your graphic will be reset to the defaults displayed in the Original size display box.

Related Information

[Format Editor dialog box \[page 1256\]](#)

38.64.9 Hyperlink tab (Format Editor dialog box)

Use this tab to create a hyperlink to a web site, an e-mail address, a file, or a report object. The hyperlink is saved with your report and is available to other users as a way of viewing additional information.

Hyperlink type

Click the type of hyperlink you want to create. Hints about how to use each option are provided on the bottom of the tab in the "Helpful hint" section.

ⓘ Note

Not all hyperlink types are available at all times. The object you select and its location on the report determine which types are available.

ⓘ Note

For information about how to use the Report Part Drilldown or Another Report Object options, see [Setting up navigation \[page 632\]](#).

Hyperlink Information

Use this section of the tab to enter the appropriate hyperlink information (the URL of a web site, for example). Hyperlink information is available for all Hyperlink types except No Hyperlink, Current Website Field Value, and Current E-mail Field Value.

Browse

Use Browse to search for files using the [Open dialog box \[page 1314\]](#). Browse is available only for the "A File" option.

Helpful hint

Use these hints to learn how each Hyperlink type works and what it will add to your report.

Related Information

[Hyperlink fields \[page 106\]](#)

[Setting up navigation \[page 632\]](#)

[Create Enterprise Hyperlink dialog box \[page 1271\]](#)

38.64.9.1 Create Enterprise Hyperlink dialog box

The [Browse](#) button allows you to create a hyperlink to an enterprise document. Once you select a document, the [Document ID](#) and [Document Name](#) fields are filled out with the CUID and name of the selected document respectively.

Hyperlink Settings

Use this section to change the settings of the hyperlink you want to create.

- [Target Window](#) allows you to create the hyperlink in a new window or in the current window.
- [Output Format](#) allows you to create the hyperlink in one of the following formats:
 - HTML
 - PDF
 - Excel

- Word
- [Refresh on Open](#) allows you to set the refresh mode to On or Off for the hyperlink.
- [Instance](#) allows you to select from the following options:
 - Do not use
 - Latest Instance
Selects the latest instance of the document.
 - Latest Instance By User
Selects the latest instance of the document that the user has worked on.
 - Latest Instance By Parameter Values
Selects the latest instance that has been run with the parameter set input.

Parameter Settings

Crystal Reports will detect any parameters in the target report and allow you fill them in appropriately. You will be able to enter a constant, a field or let your user be prompted.

Related Information

[Creating an Enterprise Hyperlink \[page 643\]](#)

38.64.10 Rounding tab (Format Editor dialog box)

Use the Rounding tab to create boxes with rounded corners as part of your report.

Rounding

Choose a percentage or move the slider to the right to increase the curvature of the box corners. The box that you started with gradually changes to an ellipse or circle, depending on how far you move the slider to the right.

Once the appropriate shape is created, the program remembers the position of the slider and applies it unless you discard your changes in the Format Editor.

38.64.11 Subreport tab (Format Editor dialog box)

Use the Subreport tab to select or clear options specific to a subreport.

Subreport Name

Use this field to rename a subreport from within your containing report.

On-demand Subreport

Select this check box to create a hyperlink to a subreport.

On-demand Subreport Caption

Use the Formula Workshop button to create a caption that appears on your subreport object within the containing report.

For example, the actual name of your subreport might be "sales". This name appears on the Design tab when you insert your on-demand subreport. If this name is not appropriate to be displayed on the Preview tab, you might create a caption that reads "Sales Figures for the Last Quarter."

Subreport Preview Tab Caption

Use the Formula Workshop button to create a caption that appears on the subreport Design and Preview tabs when you preview the subreport.

Re-import When Opening

Select this check box to re-import this subreport along with any changes that you have made to it each time the subreport opens. If the "Re-import Subreport on Open" option is selected on the [Reporting tab \(Options dialog box\)](#) [page 1326], the option is selected by default in this dialog box.

Note

For this option to work correctly, the subreport must still be located where you first inserted it in your report.

Suppress Blank Subreport

Select this check box to withhold blank subreports. Because blank subreports are treated as visible report objects, this option is especially useful when you want to suppress blank report sections that contain blank subreports.

File name for linked subreport

When you have a linked subreport, you can see the source location of that subreport and - if necessary -you can change the location. If you have an embedded subreport, this option is grayed out.

38.64.12 Font tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Format Editor dialog box \[page 1256\]](#)
- [Text Format dialog box \[page 1373\]](#)

Use the Font tab to change the fonts, the font size, and the font style for text and data fields on your reports.

Font

This list contains all the fonts that you have installed for your system. The list appears with the font that is currently in use selected.

Style

This list contains additional attributes that you can assign to the selected font. The list appears with the style that is currently in use selected.

Size

This list includes common point sizes for the selected font. The list appears with the point size that is currently in use selected.

If you enter your own font size, the number you type must be between 1 and 1638. Crystal Reports will automatically round all fractional entries to the nearest 0.5. Consequently, in your report, you can use the fractional font sizes 1.5, 2.5, 3.5, and so on, up to 1637.5.

Color

This palette shows all the available colors for your box. Select the desired color from the palette or click More to see the [Color dialog box \[page 1168\]](#).

Strikeout

When you select this check box, the program prints the strikeout character across the font.

Underline

When you select this check box, the program underlines characters typed in the selected font.

Character spacing exactly

Use this option to specify the value that each character in your selected font occupies. The value is defined as the distance in number of points measured from the start of one character to the start of the next. When you change the character spacing, you change only the spacing between adjacent characters, not the font size of the characters.

For example, if you specify a 14-point font with a character spacing of 14 points, each character will remain as a 14-point font size, occupying a space that is 14 points wide.

Note

If you set the character spacing to zero, no spacing is applied—the program follows the character spacing of the font.

38.64.13 Paragraph tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Format Editor dialog box \[page 1256\]](#)
- [Text Format dialog box \[page 1373\]](#)

Use the Paragraph tab to set formatting options for paragraphs in a text object.

First Line

Enter the distance in inches (or the default measurement for your report) that you want the first line of the paragraph to be indented from the rest of the paragraph.

Left

Enter the distance in inches (or the default measurement for your report) that you want the paragraph to be indented from the left margin of the page.

Right

Enter the distance in inches (or the default measurement for your report) that you want the paragraph to be indented from the right margin of the page.

Line spacing

Enter the spacing you want for lines as a multiple of the font size you are using, or as an exact number of points.

Of

Use this box to enter the multiple you want to use (if you selected Multiple for Line Spacing), or the number of points (if you selected Exact for Line Spacing).

Reading Order

Use the options in this area to define whether your text is to be read from left to right or from right to left.

Text interpretation

Use this list to select the type of pre-formatted data (string or memo fields only) or text you are inserting in a field. Crystal Reports extracts information about the font, size, style, color, and paragraph and displays the data or text with the specified attributes.

Your choices are:

- "none" to interpret the string or memo field or imported text as if it contains plain text.
- "RTF Text" to interpret the string or memo field or imported text as if it contains Rich Text Format data.
- "HTML Text" to interpret the string or memo field or imported text as if it contains HTML data.

The supported HTML tags are:

- html

- body
- div (causes a paragraph break)
- tr (causes only a paragraph break; does not preserve column structure of a table)
- span
- font
- p (causes a paragraph break)
- br (causes a paragraph break)
- h1 (causes a paragraph break, makes the font bold & twice default size)
- h2 (causes a paragraph break, makes the font bold & 1.5 times default size)
- h3 (causes a paragraph break, makes the font bold & 9/8 default size)
- h4 (causes a paragraph break, makes the font bold)
- h5 (causes a paragraph break, makes the font bold & 5/6 default size)
- h6 (causes a paragraph break, makes the font bold & 5/8 default size)
- center
- big (increases font size by 2 points)
- small (decreases font size by 2 points if it's 8 points or larger)
- b
- i
- s
- strike
- u
- strong
- ol (ordered list)
- ul (unordered list)
- li (list item)

The supported attributes are:

- align
- face
- size
- color
- style
- font-family
- font-size
- font-style
- font-weight

38.64.14 Flash tab (Format Editor dialog box)

Use the Flash tab to change the size of the flash object, pick a new flash object, or set the image that will be displayed for printing and exporting.

Scaling

Use this section to set the width and height as a percentage of the size of the original flash object.

Size

Use this section to set the new width and height for the flash object.

SWF Location

Use this field to select a different flash object to embed.

Image for printing and exporting

Use this option to change the image that is displayed in printed and exported reports.

38.65 Format Grid Lines dialog box

The Format Grid Lines dialog box allows you to control the appearance of grid lines. This dialog box appears when you click Format Grid Lines on the [Customize Style tab \(Cross-Tab Expert\) \[page 1415\]](#) or the [Customize Style tab \(OLAP Expert\) \[page 1443\]](#). It will also appear when you select ► [Grid Options](#) ► [Format Grid Lines](#) ► from the context menu.

Grid line diagram

Shows the grid layout based on the current settings. Click a section then specify the grid display settings.

Grid section

Click a section then specify the grid display settings.

Show Grid Lines

Select or clear this check box to show or hide all grid lines.

Color

Click a color for the item selected in the Grid line location list.

Style

Click a line style for the item selected in the Grid line location list.

Width

Click a line width for the item selected in the Grid line location list.

Draw

Select or clear this check box to show or hide the item selected in the Grid line location list.

Draw Grand Total Line Only

Select or clear this check box to show or hide the grand total grid line. When the grand total grid line is shown, the lines usually displayed at the grid line location are hidden.

38.66 Format Other Dimensions dialog box

This dialog box appears when you click the Other Dimensions button on the Data tab of the Map Expert or the Chart screen of the OLAP Report Creation Wizard.

Use the Format Other Dimensions dialog box to set field values for your other dimensions.

Other Dimensions

This list displays the remaining dimensions in the OLAP grid. Use the list to select the dimensions for which you want to set the field values.

Select a field value

Use this button to select the fields you want displayed in an OLAP grid object, for the dimension you have selected in the Other Dimensions list. When you click this button, the [Member Selector dialog box \[page 1307\]](#) appears.

The field value you select in that dialog box will appear in the list to the right of the other dimension name.

38.67 Formula Editor dialog box

The Formula Editor is a major component of the [Formula Workshop dialog box \[page 1282\]](#). The editor has a variety of names depending on the kind of formula you are working with:

- Formula Editor.
- Format Formula Editor.
- Record Selection Formula Editor.
- Group Selection Formula Editor.
- Running Total Condition Formula Editor.
- Alert Condition Formula Editor.

ⓘ Note

The Formula Editor is similar to the Custom Function Editor and the SQL Expression Editor. These editors are included in this description.

The Formula Editor contains four main windows.

Window	Description of contents
Report Fields	Report fields contain all database fields accessible for your report. They also contain any formulas or groups already created for the report. ⓘ Note This window does not appear when you create a custom function.

Window	Description of contents
Functions	Functions are prebuilt procedures that return values. They perform calculations such as average, sum, count, sin, trim, and uppercase.  Note The contents of this window change depending on the kind of formula you are working with.
Operators	Operators are the "action verbs" you use in formulas. They describe an operation or an action to take place between two or more values. Examples of operators: add, subtract, less than, and greater than.  Note The contents of this window change depending on the kind of formula you are working with.
Formula text window	Area where you create or edit a formula.

Create a formula by double-clicking on any of the components listed in the Report Fields, Functions, or Operators trees. When you choose a component, the required syntax is also inserted. You can also manually enter a formula.

When you are finished creating or editing a formula, click the Check button to identify any errors in the formula. Click the Save button to save the formula.

For a list of shortcut keys available in the Formula Editor, see [Formula Editor Key Controls \[page 699\]](#).

Related Information

[Creating and modifying formulas \[page 700\]](#)

[Formula Editor buttons \[page 698\]](#)

[Formula syntax \[page 690\]](#)

[Operators \[page 1464\]](#)

[Functions \[page 1524\]](#)

[Specifying formulas \[page 692\]](#)

[Find dialog box \(Formula Editor\) \[page 1253\]](#)

[Formula Workshop dialog box \[page 1282\]](#)

38.68 Formula Name dialog box

This dialog box appears when you select Formula Fields in the [Field Explorer dialog box \[page 1241\]](#) and then click the New button.

Use the Formula Name dialog box to specify a name for your formula.

Related Information

[Field Explorer dialog box \[page 1241\]](#)

38.69 Formula Workshop dialog box

The Formula Workshop is the place in Crystal Reports where you can work with all types of formulas, custom functions, and SQL Expressions. The workshop consists of a toolbar, a tree that lists the types of formulas you can create or modify, and an area for defining the formula itself.

Toolbars

See [Formula Workshop buttons \[page 694\]](#) for a description of each of the buttons on the various Formula Workshop toolbars.

Tree view

See [Workshop Tree \[page 694\]](#) for an overview of the tree view.

Definition area

The definition area of the Formula Workshop is where you define or modify your formula, custom function, or SQL Expression. A version of the [Formula Editor dialog box \[page 1280\]](#) is the primary editor you will encounter in the definition area, but you might also see the [Formula Expert \[page 1426\]](#) or one of a number of dialog boxes used for working with custom functions.

For details about the specific editor or dialog box you need to use in the definition area, click the appropriate link in the following list of related topics:

- [Working with the Formula Workshop \[page 693\]](#)
- [Working with the Formula Editor \[page 697\]](#)
- [Formula Expert \[page 1426\]](#)
- [Formula Editor dialog box \[page 1280\]](#)
- [Group Selection Formula Editor dialog box \[page 1284\]](#)
- [Record Selection Formula Editor dialog box \[page 1342\]](#)
- [Extract Custom Function from Formula dialog box \[page 1239\]](#)
- [Custom Function Properties dialog box \[page 1190\]](#)

38.70 Function Library dialog box

This dialog box appears when you click Functions on the Calculation tab of the [Calculated Members dialog box \[page 1158\]](#).

The Function Library dialog box displays the MDX functions that are available for you to use when you add a calculated member to an OLAP grid.

In the Function Library list, select the type of function that you want to see (Dimension, Level, Logical, and so on), or select All to view all available functions.

In the Functions list, select the function that you are interested in. A brief description of the function appears at the bottom of the dialog box.

Related Information

[Adding functions to a calculation definition or MDX Query \[page 458\]](#)

38.71 Go To dialog box

This dialog box appears when you choose the [Go To Page command \[page 1082\]](#) from the Edit menu.

Use the Go To Page dialog box to jump a particular page while previewing the report.

Page

Click this option to jump to a specific page in your report. In the box, enter the number of the page that you want to jump to.

First page

Click this option to jump to the first page of the report.

Last page

Click this option to jump to the last page of the report.

38.72 Group Selection Formula Editor dialog box

This dialog box appears within the [Formula Workshop dialog box \[page 1282\]](#) whenever you add or modify a group selection formula. You can open the Formula Workshop by itself, or you can select Group from the Selection Formulas submenu of the Report menu.

Use the Group Selection Formula Editor to edit a group selection formula that limits the groups the program uses when preparing your report.

Note

You can only use Crystal syntax when creating a group selection formula.

Note

No field placement box appears when you create a selection formula; the formula is simply stored in the report.

Note

Your group selection formula must be Boolean, that is, it must result in a Yes or No answer.

Note

You cannot use commas in numbers you enter in a formula. Enter only the number itself.

Related Information

[Formula Editor dialog box \[page 1280\]](#)

[Formula Workshop dialog box \[page 1282\]](#)

[Formula Editor buttons \[page 698\]](#)

[Formula syntax \[page 690\]](#)

[Operators \[page 1464\]](#)

38.73 Help Text dialog box

The Help Text dialog box appears when you click the Help Text button on the [Custom Function Properties dialog box \[page 1190\]](#).

Use this dialog box to specify a message to help others use the custom function you're creating. Help text is available only in this dialog box.

38.74 Hierarchical Group Options dialog box

This dialog box appears when you choose the [Hierarchical Grouping Options command \[page 1119\]](#) from the Report menu.

Available Groups

Select the group that you want to sort hierarchically from the list.

Sort Data Hierarchically

Select this check box to sort the data in the selected group hierarchically.

Instance ID Field

This field is only for display; it shows the value for the group selected from the Available Groups list.

Parent ID Field

Select the field by which you want the Instance ID field organized.

For example, for a company hierarchal report, select the data field listing whom the employee reports to.

The type of the Instance ID and the Parent ID must be the same. For example, they both must be strings (text) or numbers.

Group Indent

Enter the amount you want to indent for each subgroup.

Note

The top level of the hierarchy is determined by group instances that match the Instance ID and Parent ID.

Note

If a group instance is not connected to any Parent ID, it appears at the top of the hierarchy.

Related Information

[Grouping data hierarchically \[page 185\]](#)

38.75 Highlight Exceptions dialog box

The Highlight Exceptions dialog box appears when you right-click a row or column on an OLAP Worksheet, point to Highlight Exceptions on the shortcut menu, and select Add from the submenu.

Use the dialog box to highlight important differences or unexpected results in your data. For more information, see [Overview \[page 438\]](#) in the "Highlighting exceptions" section.

Highlighting Method

Currently, you can select only Data Cell Value as the method on which to base highlighting.

Options

Use the options in this area to determine which values you want to use for highlighting. You can choose absolute values as they appear in your Worksheet, values as they compare to other dimension members, or values based on a calculated member. For more information about each of these options, see [Highlighting high or low absolute values \[page 439\]](#), [Highlighting exceptions compared with other members \[page 440\]](#), or [Highlighting exceptions based on a calculation result \[page 441\]](#).

Tolerance

Use the sliders in this area to set the benchmarks above and below the base member (the one you are using for comparison) that will be used to flag exceptional data. For example, if you consider 10,000 above and 10,000 below the base member to be exceptional, set the sliders to 10000 and -10000.

Note

By default, values above the upper limit are displayed with a green background, those below the lower limit with a red background, and intermediate values with a yellow background. To change these colors, use the options in the Formatting area.

Formatting

Note

Display this area by clicking the Formatting button; hide the area by clicking the Shrink button.

Use the options in this area to set the colors and fonts for your highlighting benchmarks. Click the Reverse Ranges button to reverse the current settings. For more information and considerations about setting colors and fonts, see [Using other formats to highlight exceptions \[page 442\]](#).

38.76 Index Legend dialog box

This dialog box appears when you click the Index Legend button on the Links tab of the [Database Expert \[page 1420\]](#).

Use the Index Legend dialog box to see a key to the index indicators used in the Links display area.

38.77 Insert Group dialog box

This dialog box appears when you choose the [Group command \[page 1095\]](#) from the Insert menu.

→ Tip

The information in this section also applies to the [Change Group Options dialog box \[page 1162\]](#), the [Chart Sort Order dialog box \[page 1166\]](#), and the [Cross-Tab Group Options dialog box \[page 1189\]](#).

Use the Insert Group dialog box to group your data without summarizing the data in the groups. This is a useful process, for example, for dividing a customer list into state or ZIP code groups.

Related Information

[Group command \[page 1095\]](#)

[Group Expert \[page 1428\]](#)

[Chart Expert \[page 1400\]](#)

[Cross-Tab Expert \[page 1412\]](#)

[Sorting, Grouping, and Totaling \[page 167\]](#)

38.77.1 Common tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Insert Group dialog box \[page 1287\]](#)
- [Change Group Options dialog box \[page 1162\]](#)
- [Chart Sort Order dialog box \[page 1166\]](#)
- [Cross-Tab Group Options dialog box \[page 1189\]](#)

Use the Common tab to specify the field and sort order for your grouping.

Field

Use this list to select a database field to group by.

Order

This list contains four sort direction options. The default is in ascending order. To choose a different sort direction, click the box to reveal the options and make your selection from the list.

Use a Formula as Group Sort Order

Select this option if you want to create a formula for a conditional sort order. When you select this option, the Conditional Formula button becomes active, and the Order list is grayed out. For information about creating group sort order formulas, see [Sorting groups conditionally \[page 175\]](#).

Note

This option is not available if you choose "in specified order" as the sort order for your group.

Date Options

This lists appears only when you are working with a date field. The list contains choices for grouping by time periods (each week, each month, and so on).

Boolean Options

This lists appears only when you are working with a Boolean field. The list contains choices for grouping by changes in Boolean value (that is, changes from yes to no, or True to False in your report data).

38.77.2 Options tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Insert Group dialog box \[page 1287\]](#)
- [Change Group Options dialog box \[page 1162\]](#)
- [Cross-Tab Group Options dialog box \[page 1189\]](#)

Note

The Options tab does not appear in the [Chart Sort Order dialog box \[page 1166\]](#).

Customize Group Name Field

The group name field is displayed in the group tree and in the group header. By default, the group name field has the same values as the group field. Select this check box to define the group name field as:

- A database field.
- A group name formula that returns a string value, to generate values for a group name field.

For example, you could group on the Customer ID, but display the Customer Name as the group name. To do this, select the Choose From Existing Field option and choose the appropriate field from the list.

Alternatively, you could group on the Customer ID, but use the following formula to display both the Customer ID and the Customer Name in the group name:

```
ToText ({Customer.CustomerID}) + " : " + {Customer.CustomerName}
```

To do this, select Use a Formula as Group Name and click the Formula button to enter your formula in the Formula Workshop.

Note

The Customize Group Name Field option is not available for groups sorted "in specified order."

Note

You cannot include a field that is evaluated at runtime, such as a running total field, summary field or another group name field, in a group name formula.

Keep Group Together

Select this check box to keep your groups from breaking across pages. (If a group extends beyond a single page, Crystal Reports begins printing the group at the top of the next empty page, but then breaks the group across pages as necessary.)

If the first group in a report extends beyond a single page, and this check box is selected, Crystal Reports begins printing the group on the second page of the report, instead of the first. To avoid this behavior on the first page of the report, clear the *Respect Keep Group Together On First Page* check box in the [Report Options dialog box \[page 1345\]](#).

Note

This option does not appear for the [Cross-Tab Group Options dialog box \[page 1189\]](#).

New Page After Visible Groups

Select this check box to regularly insert page breaks after a specific number of visible groups.

Note

Suppressed groups may occur on a page, but will not count as visible groups.

Note

This option does not appear for the [Cross-Tab Group Options dialog box \[page 1189\]](#).

Repeat Group Header On Each Page

If you want your groups to break across pages, select this check box to repeat the group header for every successive page.

Note

This option does not appear for the [Cross-Tab Group Options dialog box \[page 1189\]](#).

38.77.3 Specified Order tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Insert Group dialog box \[page 1287\]](#)
- [Change Group Options dialog box \[page 1162\]](#)
- [Chart Sort Order dialog box \[page 1166\]](#)
- [Cross-Tab Group Options dialog box \[page 1189\]](#)

The Specified Order tab appears only after you select "in specified order" as your sort order on the Common tab.

Use the Specified Order tab whenever you want to set up custom groups. Custom groups are freeform groups that are not necessarily triggered by any one field value. You define the groups and you specify the individual values that belong in each.

Named Group

This list enables you to browse a list of the values in the field you are grouping. You can select a value from that list if you want to use it as a group name. For example, if you are setting up a California group and the value California appears among the field values, you can select that name from the list.

The box below displays the fields you have selected and the groups you have created as specified order groups.

Up arrow

Click this button to move the highlighted group higher in the list.

Down arrow

Click this button to move the highlighted group lower in the list.

New

Click this button to open the [Define Named Group dialog box \[page 1222\]](#) where you can create a new named group. If you entered a new name in the Named Group list on the Specified Order tab, it appears as the Group Name on the Define Named Group dialog box.

Edit

Click this button open the [Define Named Group dialog box \[page 1222\]](#) where you can edit a group you've highlighted in the Named Group list.

Delete

Click this button to delete the highlighted group.

38.77.4 Others tab (various dialog boxes)

This tab can appear as part of several dialog boxes:

- [Insert Group dialog box \[page 1287\]](#)
- [Change Group Options dialog box \[page 1162\]](#)
- [Chart Sort Order dialog box \[page 1166\]](#)
- [Cross-Tab Group Options dialog box \[page 1189\]](#)

The Others tab appears only in connection with the Specified Order tab.

Use the Others tab to set up what you want the program to do with all values not included in any of the specified groups.

Discard all others

Select this option if you do not want to print the unselected fields in the groups for your report. If you summarize groups, values in the discarded groups will not be summarized nor will they be included in any grand total if you have one.

Put all others together, with the name

Select this option and enter a name if you want to put all unselected fields in one group. The default name for this group is Others. If you summarize groups, the values in the group will be summarized and will be included in any grand total.

Leave in their own groups

When you click this option, your specified groups are shown in the order you specified. Remaining groups are displayed normally. If you summarize the groups, the values in the group will be summarized and will be included in any grand total.

38.78 Insert Object dialog box

This dialog box appears when you choose the [OLE Object command \[page 1100\]](#) from the Insert menu.

Use the Insert Object dialog box to insert an object into your report. You can either insert an existing object, or create a new one.

Create New

Click this option to create a new object and embed it in your report. When the Create New option is active, the Object Type list is displayed.

Create from File

Click this option if you would like to embed or link an object that already exists. For example, if you have created and saved a spreadsheet file in Microsoft Excel and you want to use it as an object, click Create from File and then select the file name via the File box.

The program displays the File box as well as the Browse button and Link check box whenever you select Create from File. See [Insert Object dialog box \(Create from File version\) \[page 1294\]](#).

Object Type

This box lists all of the object types available on your system.

Note

This list is based on the registration database. When OLE server applications are installed on your system, they are responsible for registering themselves with that database. If you have server applications on your system but they do not appear on this list, the applications may not have registered themselves properly.

Select the type of object you would like to embed in your report.

Display As Icon

Specifies whether the embedded object will appear in the report as an icon or whether the actual contents of the object will always be visible. Note that some object applications only display objects as icons.

If you select Display as Icon, you can open the object in your document by double-clicking its icon.

Related Information

[OLE Object command \[page 1100\]](#)

[Inserting OLE objects into reports \[page 337\]](#)

[Insert Object dialog box \(Create from File version\) \[page 1294\]](#)

38.79 Insert Object dialog box (Create from File version)

This dialog box appears when you choose the Create from File option in the Insert Object dialog box.

Use the Create from File versions of the Insert Object dialog box to insert an already existing object.

File

Use this box to type in the path and file name of the file you want to use as an OLE object. Be sure that the file you select can be used as an OLE object. If you are not sure, check the documentation for the application that was used to create the file to see if the application is an object application.

Browse

If you are not sure of the path and file name of the object file, click the Browse button. The Browse dialog box appears. Use the Drives, Directories, and File lists to locate the file you want to use. When you click OK, the program automatically writes the path and file name into the File box.

Link

Select the Link check box if you want to create a linked object. When a linked object is activated from within a report, the original file is opened inside the application that was used to create it. Any changes you make directly affect the original file also.

If you want the data in an object to remain available to other applications, and always reflect the most current changes to the data, remember to link the object to the report.

Display As Icon

Specifies whether the embedded object will appear in the report as an icon or whether the actual contents of the object will always be visible. Note that some object applications only display objects as icons.

If you select Display as Icon, you can open the object in your document by double-clicking its icon.

Change Icon

Click this button to change the icon that represents the embedded object in your report. The [Change Icon dialog box \[page 1162\]](#) appears.

Related Information

[Insert Object dialog box \[page 1293\]](#)

38.80 Insert Subreport dialog box

This dialog box appears when you choose the [Subreport command \[page 1096\]](#) from the Insert menu, or when you click the Insert Subreport button on the Insert Tools toolbar.

Use the Insert Subreport dialog box to insert a subreport into your report. Once the dialog box appears, you can either choose an already existing report to insert as a subreport or create a new report from scratch.

38.80.1 Subreport tab (Insert Subreport dialog box)

Choose an existing report

When you choose this option, the "File name" box under the option is activated. Enter the name of the report you want to use as a subreport. If you do not know the name of the report, click Browse to find the file.

File name

Enter the full path for the report file you want to insert as a subreport. If you do not know the exact path click Browse to find the desired file.

Browse

Click the Browse button to find a report you'd like to include as a subreport. Double-click the report, or select Open and you are returned to the Insert Subreport dialog box. Notice that the file path to the report you just selected is now in the Report file name box.

Create a subreport with the Report Wizard

When you choose this option, the "New report name" edit box under the option is activated. Enter a name for the subreport and then click the Report Wizard button to create the new subreport.

New report name

Enter the name you would like to assign to your new subreport and then click the Report Wizard button to create your new report.

Report Wizard

Click this button to open the [Standard Report Creation Wizard \[page 1380\]](#). Follow the directions on the tabs to create a new subreport.

On-demand subreport (similar to a hyperlink)

Select this check box if you want your subreport to be On-demand. Data for an On-demand subreport is not read from the database until you drill down on the hyperlink used to represent it. Otherwise, subreport data is collected at the same time as the data for the container report.

Related Information

[Subreports \[page 870\]](#)

[Link tab \(Insert Subreport dialog box\) \[page 1297\]](#)

38.80.2 Link tab (Insert Subreport dialog box)

The Link tab cannot be accessed until you enter a report name into the Report File name box, or until you create and save a new report using the Create a subreport option.

Note

The Link tab has the same functionality as the [Subreport Links dialog box \[page 1369\]](#). For more information, see the Subreport Links dialog box.

Available fields

The Available fields box lists all of the fields available in the container report to which you can link your subreport.

Add (>)

Click the Add button (>) to copy a field from the Available Fields box into the Field(s) to link to box.

Remove (<)

Click the Remove button (<) to remove a field from the Field(s) to link to box.

Field(s) to link to

The Field(s) to link to box displays the currently selected fields to which you intend to have your subreport link.

Subreport parameter field to use

The Subreport parameter field to use list appears when the Field(s) to link to box contains at least one field name. Use the Subreport parameter field to use list to determine the subreport field on which the container report and the subreport are linked.

Select data in subreport based on field

Use the Select data in subreport based on field check box to activate the Select data in subreport based on field list.

After you select this check box, choose a field from the list below to organize the subreport data based on a specific field. Clear this check box to have the subreport adopt the organization used in the primary report.

38.81 Insert Summary dialog box

This dialog box appears when you choose the [Summary command \[page 1094\]](#) from the Insert menu, or when you click Insert Summary on the Insert Tools toolbar.

Use the Insert Summary dialog box to summarize the data in a field in your report. You can insert summaries, subtotals, and grand totals while using this dialog box.

Choose the field to summarize

This list displays the fields and formulas used in the report along with the fields available in the report's data source tables. Select the field that you want to summarize.

Calculate this summary

This list contains all the summary operations available for use in your report. Select the desired operation.

Note

Not all the summary types available apply to all fields. For example, you cannot sum or average a text, Boolean, date/time, time, or date field.

Summary location

This list contains the locations in your report in which you can place a summary. By default, every report can contain a summary field in the Report Footer (this is a Grand Total). If you want to add a different kind of summary, such as a subtotal, add a group using the Insert Group button.

Add to all group levels

Select this check box to add the summary you define to all group levels in your report.

Insert Group

Use this button to add a group to your report. The [Insert Group dialog box \[page 1287\]](#) appears. After you save your group, it appears in the Summary location list and can be selected.

Show as a percentage of

Select this check box if you want to calculate the percentage total of one group within a broader grouping. For example, you can show the percentage of sales in each city based on the total sales for each country.

After you select the check box, you can choose the total or group you want the comparison to be based on.

Note

The percentage option is available only when the result of a summary operation is numeric.

Summarize across hierarchy

This option enables you to calculate a summary across hierarchical groupings. To set or edit Hierarchical Options for existing groups, click Hierarchical Grouping Options on the Report menu. For details, see [Grouping data hierarchically \[page 185\]](#).

Related Information

[Summary command \[page 1094\]](#)

38.82 Label Aliasing dialog box

The Label Aliasing dialog box appears when you choose the [Edit Axis Label command \(Chart menu\) \[page 1122\]](#) from the Chart menu.

Use this dialog box to create an alias for the selected axis label in your chart.

Note

This command is available only if you have chosen to show labels for the data points on your chart, and if you have selected one of these labels.

Original Label

This field displays the name of the selected chart axis label.

Displayed Label

Use this field to specify a new name (that is, an alias) to display as the chart axis label.

38.83 Layer Control dialog box

The Layer Control dialog box appears when you choose the [Layers command \(Map menu\)](#) [page 1126] from the Map menu or the map's right-click menu.

In Crystal Reports, maps are made up of a number of layers. You can stack these layers on top of each other to create a more detailed map. Use this dialog box to add, remove or change layers on your map. For example, you could rearrange the layers so that the World Major Cities layer appears above the World layer, or so that the Europe Highways layer appears above the Europe layer.

Note

It may be necessary to zoom in on the map to view layer changes.

Layers

The Layers list contains the layers options for your map. The possible options are determined by Crystal Reports; use the Add button to select additional layer options.

Up

Use the Up button to move a layer up in the Layers list. Be aware that some map layers contain non-transparent sections that can obscure the detail of other layers.

Down

Use the Down button to move a layer down in the Layers list. Be aware that some map layers contain non-transparent sections that can obscure the detail of other layers.

Add

Use the Add button to add a layer to the Layers list. Additional layer files are located at \Program Files\SAP BusinessObjects\MapInfo MapX\Maps when you install Crystal Reports to the default location.

Remove

Use the Remove button to remove a layer from the Layers list.

Visible

Use the Visible check box to make a layer visible on the map. If this check box is not selected, the selected layer is hidden.

Automatic Labels

Use the Automatic Labels check box to specify whether the layer is labeled automatically on the map.

Display

Click the Display button to bring up the [Display Properties dialog box \[page 1223\]](#). The Display Properties dialog box allows you to further manipulate the mode and zoom layering of a selected layer.

38.84 Link Options dialog box

This dialog box appears when you have a link line selected and you click the Link Options button on the [Links tab \(Database Expert\) \[page 1422\]](#).

Use the Link Options dialog box to specify join and link types.

Link display

This box shows the selected link.

Join Type

The program enables you to specify the type of join you want it to use when joining tables. For more information on the Join types, see [Linking options \[page 911\]](#).

Enforce Join

The program enables you to enforce the use of tables while you're specifying joins. For more information on the Enforce Join types, see [Linking options \[page 911\]](#).

Link Type

The program enables you to specify the type of link you want it to use when joining tables. For more information on the Link types, see [Linking options \[page 911\]](#).

Related Information

[Linking indexed tables \[page 909\]](#)

[Link relationships \[page 902\]](#)

[Links tab \(Database Expert\) \[page 1422\]](#)

[Linking options \[page 911\]](#)

38.85 Links dialog box

This dialog box appears when you have a linked OLE object selected and you choose the [Links command \[page 1085\]](#) from the Edit menu.

Use the Links dialog box to update and modify existing object links in your report.

Links

Displays the current links used in the report, their type and the update status. To change the status of a link, click the link and then click Automatic or Manual.

Update

Specifies whether you want the selected link to be updated automatically or manually.

If you click Automatic, whenever you change the linked information in the original file, this report will be updated.

If you click Manual, you need to click Update Now when you want the information to be updated.

Update Now

Click this to update the information in the selected link.

Open Source

Click this to open the source file for the selected link.

Change Source

Click this to change the source file for the selected link.

Use this button to specify a new location for an object file, for example. When you click this button, the Change Source dialog box appears. Using this dialog box, select your new source. For example, if you have linked to Red.bmp and you have moved the bitmap from one directory to another, locate the new directory in this dialog box and select Red.bmp in that directory as your new source.

Break Link

Click this to disconnect the selected link. This is useful if you no longer want changes in the source file to be reflected in the current report. The object remains in the report as an unlinked, static object that cannot be edited via OLE.

Related Information

[Links command \[page 1085\]](#)

[OLE \[page 335\]](#)

38.86 Log On to SAP BusinessObjects Enterprise dialog box

This dialog box appears the first time you try to connect to the Repository in a session, or when you click the Enterprise Folders option in the [Open dialog box \[page 1314\]](#).

Use this dialog box to log on to SAP BusinessObjects Enterprise .

System

Select the name of the repository and/or SAP BusinessObjects Enterprise system that you want to connect to.

User Name

Enter your user name for the SAP BusinessObjects Enterprise system and the authentication type you selected.

Password

Enter your password for the SAP BusinessObjects Enterprise system that you selected.

Authentication

Select Enterprise Authentication or Windows NT Authentication:

- Enterprise Authentication requires a user name and password that is recognized by BusinessObjects Enterprise.
- LDAP authentication requires a user name and password that is recognized by an LDAP directory server.
- Windows Active Directory (AD) authentication requires a user name and password that is recognized by Windows AD.
- Windows NT authentication requires a user name and password that is recognized by Windows NT.

38.87 Manage XML Exporting Formats dialog box

This dialog box appears when you choose the [Manage XML Exporting Formats \(Export submenu\)](#) [page 1076] from the [Export](#) submenu of the [File](#) menu.

Show Schema

Click this button to open the [XML Export Schema dialog box](#) [page 1379] which shows the currently selected schema to be used to format the exported report.

XML Exporting Formats

This is a list of schemas that were imported into the report. The default schema to be used during export is indicated in parentheses.

Add

Click the [Add](#) button to open the [Add a New XML Format dialog box](#) [page 1153].

Delete

Click the [Delete](#) button to delete the currently selected schema. If this button is disabled, then the highlighted schema is a built-in Crystal Reports schema which cannot be deleted.

Modify

Click the [Modify](#) button to open the [Add a New XML Format dialog box](#) [page 1153].

Set Viewer Default

Click the [Set Viewer Default](#) button to set the currently selected schema in the [XML Exporting Formats](#) to be the default schema used during report export.

Description

Displays a description of the schema. This text was entered at the time the schema was added to the report. It may be blank if no description was created.

38.88 Map Fields dialog box

The Map Fields dialog box is used to link report fields to their corresponding database fields when changes are made to the structure of the database, or when moving a database from a direct access type to an SQL type. The Map Fields process helps you to make sure your reports print with the current version of the active database.

This dialog box opens when you choose any of the following Database menu commands (if the report database has changed since you last ran the report):

- Verify Database command
- Verify on First Refresh command
- Set Datasource Location command

Unmapped fields (report fields)

The Unmapped fields list (upper left) shows the unmapped report fields for which the program detects a change in the active database.

Unmapped Fields (database fields)

The Unmapped fields list (upper right) shows the unmapped database fields for which the program detects a change.

Match Type

When the Match Type check box is selected, the Unmapped fields list (upper right) shows only the names of unmapped database fields that are of the same type as the field selected in the Unmapped fields list (upper left). When the Match Type check box is not selected, the Unmapped fields list (upper right) shows all unmapped database fields, regardless of the field type.

Map

Click Map to map the report field selected in the Unmapped fields list (upper left) to the database field selected in the Unmapped fields list (upper right).

Mapped fields (report fields)

The Mapped fields list (lower left) displays the names of mapped report fields. When you map report fields, those fields appear in this list.

Mapped fields (database fields)

The Mapped fields list (lower right) shows the mapped database fields. When you map database fields, those fields appear in this list.

Unmap

Click Unmap to unmap the report field selected in the Mapped fields list (lower left) and the database field selected in the Mapped fields list (lower right). After you click this button, the two fields are no longer mapped.

Related Information

[Remapping processes \[page 925\]](#)

38.89 Member Selector dialog box

This dialog box appears when you click the Select button for an OLAP row or column dimension. Use the Member Selector to specify which fields you want to include. Expand the tree structure to view available members and to select them manually. Or, use the toolbar (or the shortcut menu) to select from predefined commands.

Select

Click the Select list to display a number of commands that can help you select particular members or add them to your Favorites. Using the Select list is often much quicker than browsing through all of the available fields. The Select list also appears when you right-click a member in the tree view.

Select Display Mode

Click this button to view members in their hierarchical arrangement, or to sort them in ascending or descending order.

Select Hierarchy

Click this button to display a number of commands that can help you select a hierarchy if more than one hierarchy is available in your data cube.

New Favorite Group

Click this button to create a new Favorite group and add the selected member to it. When you have added multiple Favorite groups, their names are displayed here so you can choose which folder to add the member to.

Display Members Using

Click this button to choose between Caption, Name, or Caption : Name. Your choice determines the description you will see for members displayed in this dialog box.

Note

Names are defined by the sever and cannot be modified in Crystal Reports. Captions are descriptions that can be modified in Crystal Reports.

Search

Click this button to open the [Search dialog box \(OLAP Worksheet\) \[page 1357\]](#). Use the Search dialog box to create search criteria for locating specific members.

Related Information

[Selecting row and column members \[page 415\]](#)

[Using the Member Selector toolbar commands \[page 416\]](#)

[Using the Member Selector shortcut menu \[page 417\]](#)

38.90 New Formatting Formula dialog box

The New Formatting Formula dialog box appears when you select the Formatting Formula option from the New list in the [Formula Workshop dialog box \[page 1282\]](#), or when you select an item in the Formatting Formulas folder and select New Formatting Formula from the shortcut menu.

Use this dialog box to determine the kind of formatting formula you want to create for the selected item (the choices differ depending on which item you've selected in the Formatting Formulas folder). For example, for the Page Footer, you have options to create these formatting formulas:

- Background Color
- Reset Page Number After
- Suppress (No Drill-Down)
- Suppress Blank Section
- Underlay Following Sections

Select the appropriate option and click the Use Editor button to create your formula in the [Formula Editor dialog box \[page 1280\]](#).

38.91 Number and Date Format dialog box

This dialog box appears when you export a report to either of the Record Style formats.

Use the Number and Date Format Settings dialog box to:

- Set the number formats you have defined in your report to be used to export your data to the new disk file.
- Set the date formats you have defined in your report to be used to export your data to the new disk file.

Same number format as in report

Select this check box if you want the number formats you have defined in your report to be used when exporting your data.

Same date format as in report

Select this check box if you want the date formats you have defined in your report to be used when exporting your data.

Related Information

[Exporting a report \[page 617\]](#)

38.92 Object Information dialog box

The Object Information dialog box appears when you select the "Add the report object to the repository" option from the [Add or Update Object dialog box \[page 1156\]](#).

Use this dialog box to describe the object you're adding to the repository so it can be identified later (when using the Properties option on the Repository Explorer's shortcut menu).

- This dialog box can also be called Modify Item. This version appears when you drag and drop a report object onto an existing version of that object in the repository.
- If the object type and the information you assign in the Object Information dialog box are the same as an existing repository object, the new object overwrites the existing one. The object's version number is increased by one.

Name

Enter a name for the repository object. This is a mandatory field; if you don't enter a name, the object won't be added to the repository.

Note

This field is read-only in the Modify Item version of this dialog box.

Author

Enter the name of the person who is adding the object to the repository. This field is optional.

Description

Add an optional description that you can see when you're looking at the repository object's properties.

Related Information

[Properties dialog box \[page 1340\]](#)

[Add Item dialog box \[page 1155\]](#)

38.93 Object Size and Position dialog box

Use the Object Size and Position dialog box to change the height and width of the selected object, and to adjust the object's position from the top and side margins.

X

Use the X box to change an object's horizontal position from the margin. You can create a conditional formula in the Formula Workshop to adjust the X position. Do this when you want to move an object to a different position in the report based on its value.

Y

Use the Y box to change an object's vertical position from the margin.

Height

Use the Height box to change an object's vertical height.

Width

Use the Width box to change an object's horizontal width. You can create a conditional formula in the Formula Workshop to adjust width. Do this when you want an object to have a different width based on its value.

Note

The type of units you enter in these boxes is determined by your operating system settings.

38.94 ODBC Formats dialog box

The ODBC Format dialog box appears when you export a report and select ODBC from the Format list in the [Export dialog box \[page 1236\]](#).

Use the list of ODBC data sources to choose the data source that you want to export your report to. The list contains all of the User and System DSNs that are available in the ODBC Data Source Administrator on your machine. When you choose a data source and click OK, the [Enter ODBC Table Name dialog box \[page 1229\]](#) appears.

Related Information

[Exporting to an ODBC data source \[page 627\]](#)

38.95 OLAP Connection Browser dialog box

The OLAP Connection Browser allows you to specify the type of OLAP data source that you want to work with in your report. The main window of the dialog box provides an integrated tree view of the OLAP data sources that you have set up to use with Crystal Reports. If the data source that you want is not listed, click Add Server to access the [Connection Properties dialog box \[page 1169\]](#).

The OLAP Connection Browser appears when you click Select Cube on the Data tab of the OLAP Expert.

The following buttons are available in the OLAP Connection Browser:

Open

Opens the object currently selected in the tree view (OLAP server, database, or cube).

Add Server

Opens the [Connection Properties dialog box \[page 1169\]](#), which enables you to connect to an OLAP Server, a Local Cube file, or an HTTP Cube. Once added, the new server appears in the OLAP Connection Browser's tree view.

Cancel

Closes the OLAP Connection Browser without selecting an OLAP cube, and returns you to the expert's Data tab.

38.95.1 Shortcut menu commands

Right-click any item in the OLAP Connection Browser to see a shortcut menu with one or more of the following commands:

Remove Server

Removes the selected server from the list in the OLAP Connection Browser.

Rename

Allows you to change the caption that is displayed for the selected OLAP server or the selected Favorites cube.

Refresh

Updates the list of available databases and OLAP cubes for the selected OLAP server.

Add to Favorites

Adds the selected cube to the list of Favorites displayed in the OLAP Connection Browser. Use this command so that you can quickly locate frequently used cubes. You can rename cubes once you add them to the list of Favorites.

Remove from Favorites

Removes the selected cube from the Favorites list, but does not remove the connection to the corresponding OLAP server.

Add Server

Opens the [Connection Properties dialog box \[page 1169\]](#), which enables you to connect to an OLAP Server, a Local Cube file, or an HTTP Cube. Once added, the new server appears in the OLAP Connection Browser's tree view.

38.96 Open dialog box

The Open dialog box appears when you choose the [Open command \[page 1072\]](#) from the File menu.

Use this dialog box to open the file with the name, file type, and location you specified.

If you want to update the report's repository objects when you open it, select the Update Repository Objects option.

38.97 Open Hyperlink dialog box

This dialog box appears when you choose the Prompt For Hyperlink option in the [Report Options dialog box \[page 1345\]](#), and then you click a hyperlink in a report that points to a file on your local machine (that is, the hyperlink is not a web link).

Click Yes to open the link. Click Always to open this link and any other such link that you encounter within the same session in Crystal Reports. Click No if you decide not to open the link.

38.98 Options dialog box

This dialog box appears when you choose the [Options command \[page 1077\]](#) from the File menu.

Use the Options dialog box to configure the program—that is, to set up the program to suit your needs. This dialog box offers a wide range of options for helping you work faster and easier.

Note

Many of these options also appear in the [Report Options dialog box \[page 1345\]](#). Use the Options dialog box to change the default values for the program, use the Report Options dialog box to change the options for the current report.

38.98.1 Types of setting options

Some of the setting options in this dialog box affect the processing of data, and some affect the formatting of report objects.

38.98.1.1 Processing data

The first class of setting options often appear in both the [Options dialog box \[page 1314\]](#) and the [Report Options dialog box \[page 1345\]](#). When this happens, the setting in Report Options is used when the report is run, and the setting in Options is used to determine the initial value of the corresponding setting in Report Options when a new report is created.

For example, if the check box "Convert Database NULL Values to Default" is selected, then while data is being read from the database, the conversion from null to default values will occur for any null values. This conversion is done before the record set is returned to Crystal Reports for formatting.

38.98.1.2 Formatting report objects

The second class of setting options often appear in the Options dialog box and in the Format settings for a report object. In this case, the setting for the report object affects how it is formatted, and the setting in Options is used to determine the initial value of the corresponding setting on an object when that object is created.

For example, if all fields are defined in Options as 10 pt. Arial, then when you place a field on a report it will have that font. However, if you individually format a field and change it to 12 pt. Times New Roman, then that individual field will be altered, but all other current and future field objects will have the default font selection.

38.98.2 Layout tab (Options dialog box)

Use the Layout tab to set the default grid, ruler, preview page, and object format options.

38.98.2.1 Design View

Rulers

When you select this check box, the Ruler is displayed on the Design tab. This option is selected by default.

Guidelines

When you select this check box, the program displays guidelines on the Design tab. This option is not selected by default.

Grid

When you select this check box, a grid is displayed on the Design tab. This option is not selected by default.

Tool Tips

When you select this check box, the program displays tool tips when you are working on the Design tab. Tool tips show the name of a button when you leave your mouse pointer over it for a few seconds. This option is selected by default.

Short Section Names

When you select this check box, the program abbreviates the section names that appear in the gray area on the left side of the Design tab (the gray area is reduced in size). This adds extra width to the report page and allows you to work with more of your report on the screen.

When you clear this check box, the program displays full section names and maintains the gray area at full width.

This option is not selected by default.

Show Hidden Sections

When you select this check box, hidden and suppressed sections in your report are displayed on the Design tab. Report objects in these sections are shown on a gray background. This option is selected by default.

38.98.2.2 Grid options

Snap To Grid

When you select this check box, new database fields you insert snap to the nearest grid positions. When you clear this check box, new database fields are left where you place them. This option is selected by default.

Grid Size

This box lets you specify the size of the grid increments. The unit of measure used for grid increments is based on your settings in the International section of the Windows Control Panel.

Related Information

[Preview tab \[page 93\]](#)

38.98.2.3 Field options

Show Field Names

When you select this check box, field names are displayed for each report field. When you clear the check box, symbolic characters (XXXX, 555, and so on) are displayed instead of field names. This option is selected by default.

Insert Detail Field Headings

When you select this check box, a default field title appears in the Page Header section whenever you insert a field in the Details section of the Design tab. The default title is the name of the field, underlined. This option is selected by default.

Insert Group Name with Group

When you select the check box, the program automatically enters the Group Name in the Group Header whenever you insert a new group. This option is selected by default.

38.98.2.4 Preview

Rulers

When you select this check box, the Ruler is displayed on the Preview tab. This option is selected by default.

Guidelines

When you select this check box, the program displays guidelines on the Preview tab. This option is not selected by default.

Grid

When you select this check box, a grid is displayed on the Preview tab. This option is not selected by default.

Tool Tips

When you select this check box, the program displays tool tips when you are working on the Preview tab. Tool tips show the name of a button when you leave your mouse pointer over it for a few seconds. This option is selected by default.

Section Names

When you select this check box, the program displays abbreviations of the section names in a gray area on the left side of the Preview tab.

When you clear this check box, no section names are displayed and the entire width of the Preview tab is used to display the report page.

This option is selected by default.

Page Breaks in Wide Pages

When you select this check box, indicators are displayed on the Preview tab for page breaks in wide pages. This option is selected by default.

Pages Start With

These three options let you specify which level of magnification you want the program to use as a default when displaying reports on the Preview tab. You have the following choices:

- **Full Size**
Displays the report at full size and fits as much of the report into the Preview tab as possible. This option is selected by default.
- **Fit Width**
Adjusts the width of the report to match the width of the Preview tab.
- **Fit Page**
Adjusts the width and the height, in order to be able to display the entire page in the Preview tab.

See also the [Report Options dialog box \[page 1345\]](#).

Display Preview Panel

When you select this check box, the Preview Panel is shown on the Preview tab. This option is selected by default. See also the [Report Options dialog box \[page 1345\]](#).

38.98.3 Database tab (Options dialog box)

Use the Database tab to set SQL/ODBC database options, the method you want to use to sort and display table and field names, and other advanced options.

38.98.3.1 Tables and Fields

By setting the options in this section of the Database tab, you can specify the text that you want the program to use for tables and fields.

Show Name

If you select this option, the program identifies tables and fields using the actual name, for example, "Customer" table and "Customer Name" field. This option is selected by default.

Show Description

If you select this option, the program identifies tables and fields using the description you have specified, for example, "Our Clients" table and "The names of all our customers" field.

Show Both

If you select this option, the program identifies tables and fields using both the name and the description you have assigned, for example, "Customer - Our Clients" table and "Customer Name - The names of all our customers" field.

Sort Tables Alphabetically

When you select this check box, tables are displayed throughout the program sorted in alphabetical order (instead of the order they appear in the database). This option is selected by default.

Sort Fields Alphabetically

When you select this check box, fields are displayed throughout the program sorted in alphabetical order (instead of the order they appear in the database table). This option is not selected by default.

38.98.3.2 Data Explorer

Tables

When you select this check box, the program allows reporting on database tables in your SQL/ODBC data sources. This option is selected by default.

Views

When you select this check box, the program allows reporting on virtual tables in your SQL/ODBC data sources. This option is selected by default.

System Tables

When you select this check box, the program allows reporting on system tables. These tables are typically used by the system administrator only but are available for use if you have the appropriate permissions. This option is not selected by default.

Synonyms

When you select this check box, the program allows reporting on virtual tables that are available on some hosts. This option is not selected by default.

Stored Procedures

When you select this check box, the program will allow reporting on the result sets from stored procedures if you are using SQL systems that support stored procedures. This option is selected by default.

Table name LIKE

This box allows you to enter the SQL LIKE function to specify the kinds of table names you want to appear in the [Database Expert \[page 1420\]](#). You can use the underscore character (_) and the percent sign character (%) as wildcards with this function. The underscore character specifies any single character, and while the percent sign signifies any character string. For example, DAV_ matches DAVE only, while DAV% matches both DAVE and DAVID. Table name Like C% would display only those tables that have a table name beginning with the letter C.

Owner LIKE

This box works exactly like the Table name Like box except that the LIKE function here is used to select the Owner (or Creator or Alias) of the table, not the table name itself. For example, Owner Like C% would display only those tables that have an owner beginning with the letter C.

38.98.3.3 Advanced Options

Use Indexes or Server for Speed

Select this check box to use the available indexes/servers to speed the record selection process. If this check box is cleared, the record section process may be much slower. This option is selected by default.

Note

If an index has a name that is different from the database it indexes, you need to identify the index for the program. You do this using the [Link Options dialog box \[page 1314\]](#), which you can access via the [Links tab \(Database Expert\) \[page 1422\]](#).

Perform Grouping on Server

Select this check box to have the program perform grouping on the server (server-side processing). Server-side processing allows you to set up a report that performs the majority of its processing on the server and pushes only relevant details to your computer. Server-side processing provides you with a number of benefits:

- Less time connected to the server.
- Less memory needed to process the report on your computer.
- Lower transfer time from the server to the client.

Server-side processing works only for reports based on SQL data sources (you cannot, for example, use server-side processing for a report based on a query, since a query is not an SQL data source).

This option is not selected by default. See also the [Report Options dialog box \[page 1345\]](#).

For more information, see [Server-side processing \[page 921\]](#).

Database Server is Case-Insensitive

Select this check box if you want to search for strings in your SQL data without checking the case. For example, if your report contains SQL data and the data is of mixed case (red, RED, Red), a case-sensitive search for "red" returns only "red." If you select this check box, the same query will return red, RED, and Red when using red as a record selection value. This option is selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Select Distinct Data for Browsing

When Select Distinct Data for Browsing is selected, you can see the first 500 distinct (unique) records when you browse the contents of a database field.

When Select Distinct Data for Browsing is not selected, browsing returns the unique values in the first 500 records.

Note

Note that since most SQL servers do not support selecting distinct records for long data types such as memo and blob fields, selecting and browsing distinct records are not supported for such fields.

See also the [Report Options dialog box \[page 1345\]](#).

Perform Query Asynchronously

Retrieving data from the database server can be divided into two parts: executing the SQL statement and fetching the rowset from the database server. Selecting this option allows Crystal Reports to execute the SQL statement asynchronously, which means that instead of waiting for the SQL statement to finish executing, the program checks intermittently to see if the user wants to cancel the process. See also the [Report Options dialog box \[page 1345\]](#).

Verify On First Refresh

When this check box is selected, the report refreshes with the current version of the active database (unless it is a stored procedure) and resets its record buffers to the current record size the first time you refresh it per session. This option is selected by default.

For more information, see [Using the Verify Database process \[page 925\]](#).

Verify Stored Procedures On First Refresh

When this check box is selected, the report refreshes with the current version of the stored procedure and resets its record buffers to the current record size the first time you refresh it per session. This option is not selected by default.

Verify When Database Driver Upgraded

When this check box is selected, the report refreshes with the current version of the active database and resets its record buffers to the current record size when the database driver is changed. This option is selected by default.

Automatic Smart Linking

Use this check box if you want the program to automatically link your tables when you are using the [Links tab \(Database Expert\) \[page 1422\]](#). This option is selected by default.

38.98.4 Formula Editor tab (Options dialog box)

The Formula Editor tab allows you to customize formatting options for the text you create in the Formula Editor or the SQL Expression Editor.

Font

Use the Font list to select from font options for text created in the Editor. Changes to the font affect all text that you enter in the Editor.

Font size

Use the Font size list to select a font size for text created in the Editor. Changes to font size affect all text that you enter in the Editor.

Tab size

Use the Tab size box to enter the number of spaces that the program will use to indicate a <tab> in the text.

Element

The Element list contains four kinds of elements that you can use in the editor. By selecting the Comment or Keyword elements, you can change either the foreground or the background colors. You can change the foreground or background colors of the Text and Text Selection elements only if you clear the "Use Windows system color" option first.

Foreground

Use the Foreground palette to change the text color of an Editor element. Click More to open the Color dialog box for a greater selection of foreground colors.

Note

The Foreground list becomes active for Text and Text Selection elements when the "Use Windows system color" check box is not selected.

Background

Use the Background palette to change the background color of an Editor element. Click More to open the Color dialog box for a greater selection of background colors.

Note

The Background list becomes active for Text and Text Selection elements when the "Use Windows system color" check box is not selected.

Use Windows system color

Clear the Use Windows system color check box to activate the Foreground and Background lists for the Text and Text Selection elements.

Default Formula Language

Use this list to choose the formula language you want the program to use. You can choose between Crystal Syntax and Basic Syntax. Crystal Syntax is selected by default.

For more information about syntax, see [Crystal and Basic syntax \[page 690\]](#).

Null Treatment

Use this list to choose how you want the program to treat Null values. You can choose between Exceptions For Nulls and Default Values For Nulls. Exceptions For Nulls is selected by default.

If you choose Exceptions For Nulls, the program treats a null value as an exception and halts formula evaluation when it encounters such a value.

If you choose Default Values For Nulls, the value that the program assigns varies depending on the data type that you are using in your formula. (These default values are the same values that are used for uninitialized variables.)

Data type	Default value
Number	0
Currency	\$0
Boolean	False
Date	NullDate
Time	NullTime
DateTime	(NullDate, NullTime)

Data type	Default value
String	""
Array	0-length array
Range	Upper and lower bounds are both default values, and both are included

UFL Support

Use this list to choose which User Function Libraries (UFLs) you want the program to show in the [Formula Editor dialog box \[page 1280\]](#). You can choose between C/COM UFLs Only, Java UFLs Only, and No UFLs. C/COM UFLs is selected by default.

Sample

The Sample box shows an example of how the Font, Font Size, Foreground Color, and Background Color changes you made will appear on the Editor.

Reset All

Click [Reset All](#) to return all formatting changes you have made on the Formula Editor tab back to the original default settings.

38.98.5 Reporting tab (Options dialog box)

The reporting options on this tab let you choose global settings for how data is read and saved for all reports.

Update Enterprise Report Properties on Save

If you have reports that are published to a SAP BusinessObjects Enterprise system, select this check box to ensure report properties, such as parameters, are updated in SAP BusinessObjects Enterprise after you have changed and saved them in your report in Crystal Reports. This option is selected by default.

Note

If you choose to republish your modified report (that is, if you choose to save your report back to SAP BusinessObjects Enterprise by using the [Save As](#) option in Crystal Reports), the original report and its

properties are overwritten. In this case, the [Update Enterprise Report Properties on Save](#) option does not apply.

Update Connected Repository Objects on Open

Select this check box to update repository objects used in your report with the latest version from the repository when opening a report in the designer.

Convert Database NULL Values to Default

Select this check box to have the program convert null values from the database to a 0 (for numeric fields) or a blank (for non-numeric fields). This option is not selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Note

This option affects only null values found in the database used for a report.

Note

This option is not available when you have selected [Perform Grouping On Server](#) on the [Database](#) menu.

Convert Other NULL Values to Default

Select this check box to have the program convert non-database null values in the report to a 0 (for numeric fields) or a blank (for non-numeric fields). This option is not selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Note

This option affects only non-database null values in the report.

Always Sort Locally

Select this check box to ensure your records are sorted locally even if they were pre-sorted when retrieved from your data source.

Save Data With Report

When you select this check box, the program automatically saves database data with your report. Otherwise, the program saves the report itself, but it will need to retrieve the report data once again when you run the report. This option is selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Discard Saved Data on Open

Select this check box to discard saved data when opening reports. When you open a report, this function removes that report's current set of data from the database, so that you can enter new data while retaining the report's format and settings. This option is not selected by default.

Re-import Subreport on Open

Select this check box to re-import all subreports along with any changes that you have made to them each time the subreports open. This option is not selected by default; if you select it, the Re-import When Opening option is selected on the [Subreport tab \(Format Editor dialog box\) \[page 1272\]](#).

Note

For this option to work correctly, subreports must still be located where you first inserted them in your reports.

Suppress Printing if No Records Selected

Use this check box to ensure reports cannot be printed if the database does not return any records. This option is not selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Show All Headers on Drill Down

Use this check box if you would like report drill-down views to include headers for all sections above the drill-down records. This option is not selected by default. See also the [Report Options dialog box \[page 1345\]](#).

Autosave Reports After ... Minutes

Use this check box if you want reports to be saved automatically as you work on them. Use the Minutes box to specify the interval before and between autosaves. This option is not selected by default.

Note

The report is autosaved with the name <<FileName>>.AutoSave.rpt. When you manually save the report, or close Crystal Reports, the autosave file is deleted.

Display Alerts on Refresh

Select this check box if you want to see relevant [Report Alerts \[page 657\]](#) when you refresh your report. This option is selected by default.

Save Preview Picture

Use this check box to globally set the Save Preview Picture option found in the [Document Properties dialog box \[page 1224\]](#).

Note

Because the Save Preview Picture option is not selected by default in the Document Properties box, selecting this check box ensures each report you create, preview, and save is saved with a preview picture (or thumbnail image).

Note

Preview Pictures are saved with the current data in your report. Ensure you do not have sensitive data in the Preview Picture you save.

Retain Original Image Color Depth

If you want images in your report to keep their original color depth when they are viewed, select this option. Otherwise, all images are converted to 8 bits per pixel before being displayed. The advantage of selecting this option is that it lets Crystal Reports render images as exact copies of the originals. Crystal Reports, however, will then store more information about the images, which may lead to larger files and slower response time in loading reports.

38.98.6 Fields tab (Options dialog box)

Use the Fields tab to set the default format for string, number, currency, date, time, date/time, and Boolean fields. The program will use the default formats for all field objects that have not been custom formatted.

String

Click this button to set default formatting properties for string field objects. When you click this button, the program takes you to the Common tab of the Format Editor.

Number

Click this button to set default formatting properties for number field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Number tab is also present in the Format Editor.

Currency

Click this button to set default formatting properties for currency field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Number tab is also present in the Format Editor.

Date

Click this button to set default formatting properties for date field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Date/Time tab is also present in the Format Editor.

Time

Click this button to set default formatting properties for time field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Date/Time tab is also present in the Format Editor.

Date and Time

Click this button to set default formatting properties for Date and Time field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Date and Time tab is also present in the Format Editor.

Boolean

Click this button to set default formatting properties for Boolean field objects. When you click this button, the program takes you to the Common tab of the Format Editor. A Boolean tab is also present in the Format Editor.

38.98.7 Fonts tab (Options dialog box)

Use the Fonts tab to change the fonts for any field in your report.

Fields

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the fields in your report.

Summary Fields

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the summary fields in your report.

Group Name Fields

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the group name fields in your report.

Text Objects

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the text objects in your report.

Field Titles

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the field titles in your report.

Charts

When you click this button, the [Font dialog box \[page 1254\]](#) appears. You can then set the font for all the charts in your report.

38.98.8 Smart Tag & HTML Preview tab (Options dialog box)

Use Smart Tag and HTML Preview options to define settings to use when viewing reports on the web.

38.98.8.1 Smart Tag Web Server Options

Use the Smart Tag options to define the web server and viewing page you want to use when selecting Office XP smart tags for Crystal report objects.

Web Server Name

Enter the HTTP address of the web server you want to use. When you choose the View or Refresh smart tag for a Crystal report object, this web server is queried for the appropriate report.

Note

The reports you want to use with smart tags must exist on the web server in a directory that mirrors their real location, or the web server must be configured to accept a UNC path.

Virtual Directory

Enter the name of the virtual directory you've created on your web server. This directory contains your viewing page. The field defaults to `crystalreportviewers12` (a directory created at `\Program Files\Business Objects\Common\4.0`).

Viewing Page

Enter the name of the .asp or .jsp page you'll use to display your Crystal report. The pages `crsmarttag.asp` and `crsmarttag.jsp` are created by default if the Report Application Server (RAS) is detected when you install; you can find them at `\Program Files\Business Objects\Common\4.0\crystalreportviewers12`.

38.98.8.2 HTML Preview Options

Use the HTML preview options to define the configuration needed by Crystal Reports to display an HTML rendering of your report.

Note

These options are necessary when your report is unmanaged. In such cases, Crystal Reports must be able to access certain information about your Report Application Server and web server. For managed reports, your BusinessObjects Enterprise system controls the HTML preview requirements. (The guest account in BusinessObjects Enterprise must be enabled before you can use the HTML Preview feature.)

Enable HTML Preview

Select this option only if your reports are stored outside of BusinessObjects Enterprise (that is, if your reports are unmanaged). When you select this option, you are required to enter information about your RAS and web server.

Web Server Name

Enter the name of the web server that you want to use. When you choose HTML Preview for a report that is open in Crystal Reports, this web server is contacted to communicate with your Report Application Server. The default value for the web server is `localhost`.

Platform

Choose the platform of your web server installation. The choice that you make in this area sets the extension of the preview web page in the next field:

- Choosing Java changes the extension to `.jsp`.
- Choosing .Net changes the extension to `.aspx`.

URL

Enter the HTTP address of the virtual folder on your web server that contains the preview web page. Crystal Reports uses this page to display the viewer it needs for the HTML preview. By default, this field is constructed from the information that you add in the previous fields.

RAS Server Name

Enter the name of your Report Application Server. The default value for the RAS name is `localhost`.

38.98.9 Dependency Checker tab (Options dialog box)

Use the Dependency Checker options to set which conditions you want to verify when you check for errors in the Workbench.

38.98.9.1 Report Checking Options

Compile formulas

Crystal Reports compiles all of the formulas in your report to generate a list of errors. The process is not interrupted when the first error is found; you must double-click the error message in the Dependency Checker to see the error in the Formula Workshop.

Compile SQL expressions

Crystal Reports compiles all of the SQL expressions in your report to generate a list of errors. The process is not interrupted when the first error is found; you must double-click the error message in the Dependency Checker to see the error in the Formula Workshop.

Compile custom functions

Crystal Reports compiles all of the custom functions in your report to generate a list of errors. The process is not interrupted when the first error is found; you must double-click the error message in the Dependency Checker to see the error in the Formula Workshop.

Validate hyperlink URLs

Crystal Reports validates all hyperlink URLs and returns an error message when a hyperlink cannot be accessed.

Verify database

Crystal Reports verifies the report's database and returns an error message if the database cannot be accessed, or if tables and/or fields in the database have changed.

38.98.9.2 Multi-Report Checking Options

Check repository custom functions

Crystal Reports checks all of the custom functions used in the report(s) and ensures that they are present in the BusinessObjects Enterprise Repository.

- If you have not yet logged onto the Central Management Server (CMS), a warning is generated and no further checking is done.
- The URI of the custom function is validated to ensure that it still exists in the repository. A warning is generated if the custom function no longer exists.
- The version of the local copy is compared and a warning is generated if a newer copy is available in the repository.

Check repository objects (text and bitmap)

Crystal Reports checks all of the repository-linked text objects and bitmaps.

- If you have not yet logged onto the Central Management Server (CMS), a warning is generated and no further checking is done.
- The URI of the object is validated to ensure that it still exists in the repository. A warning is generated if the object no longer exists.
- The version of the local copy is compared and a warning is generated if a newer copy is available in the repository.

Check report part hyperlinks

Crystal Reports checks for several different error conditions in report part hyperlinks.

Errors with the target file

Note

If the target file is derived from a formula, Crystal Reports does not check the hyperlink.

If the target file is a RUID or a CUID, these errors are checked (in order):

- Is the user logged on to the CMS? (If No, there is no further checking).
- Does the report that the RUID/CUID identifies exist?

In all other cases, the target file is validated to ensure that it exists and can be opened.

If you are checking an entire project, a warning message appears if the target is not in the same project or report package.

Errors with the target report object(s)

Crystal Reports checks for these errors:

- The report objects all exist.
- The report objects are located in the same report section.

Errors in the data context

Note

If the data context is a formula, Crystal Reports does not check the hyperlink.

The length of the group path must correspond to the group identified by the target report objects. For example, if the target is a field in group three, the data context must pass down to group one and two; you can specify group three or a wildcard ("*").

Check subreports which are re-imported on open

Crystal Reports checks that all subreports that are to be re-imported exist in the specified location. For this validation to succeed, subreports must still be located where you first inserted them in your reports.

Related Information

[The Workbench \[page 115\]](#)

[The Dependency Checker \[page 116\]](#)

38.99 Order Links dialog box

The Order Links dialog box appears when you click the Order Links button on the [Links tab \(Database Expert\)](#) [\[page 1422\]](#).

Note

The Order Links button is available only when there is more than one link displayed in the [Database Expert](#) [\[page 1420\]](#).

Use this dialog box to specify the order in which the program should process the table links. By default, the processing order matches the order in which the links appear on the Links tab. Use the arrow buttons in the Links Order dialog box to change the default order.

For example, if you have chosen the Credit, Customer, Orders, and Orders Details tables from the Xtreme sample database, the links tab shows the tables linked in alphabetical order.

In this case, the links will be processed first between the Credit/Customer tables, then between the Customer/Orders tables, and finally between the Orders/Orders Details tables.

Note

Different link orders may result in different data sets returned for use in your report.

38.100 Page Setup dialog box

This dialog box appears when you choose the [Page Setup command](#) [\[page 1075\]](#) from the File menu.

Use the Page Setup dialog box to select printer options, select page size and orientation, and set margins. You can also use the Page Setup dialog box to optimize your report for web viewing.

No Printer

Select the No Printer check box to format your report for web viewing.

Page Size options

Use the Page Size list to choose from a variety of paper- or envelope-size options.

Use the radio buttons to choose pixels, inches, or centimeters, and then set the vertical and horizontal dimensions.

Changes to the page size will be reflected in the visual representation of your report as seen on the Page Setup dialog box.

Orientation options

Select the Portrait option to print your report in a standard-letter orientation.

Select the Landscape option to print your report in landscape orientation.

Margins

You can specify your left, right, top, and bottom page margins

38.101 Paste Special dialog box

This dialog box appears when you choose the [Paste Special command \[page 1081\]](#) from the Edit menu.

Use the Paste Special dialog box to paste an OLE object into your report from the clipboard.

Paste

Click this to embed the copied information into your report.

Changes you make to the embedded object will not be reflected in the original file, and changes to the original file will not be reflected in this report.

Paste Link

Click this to link the copied information to your report.

Changes you make to the embedded object will be reflected in the original file, and changes to the original file will also appear in this report.

As

Enables you to select how the copied information is pasted into your report. This option is not available for all types of object. See [OLE \[page 335\]](#) for details.

Display As Icon

Specifies whether the embedded object will appear in the report as an icon or whether the actual contents of the object will always be visible. Note that some object applications only display objects as icons.

If you select Display as Icon, you can open the object in your document by double-clicking its icon.

Change Icon

Click this button to change the icon that represents the embedded object in your report. The [Change Icon dialog box \[page 1162\]](#) appears.

Related Information

[OLE \[page 335\]](#)

[Paste Special command \[page 1081\]](#)

38.102 Performance Information dialog box

The Performance Information dialog box appears when you choose the [Performance Information command \[page 1121\]](#) from the Report menu.

Use this dialog box to gather information that will help you streamline the current report for optimal performance. (For information about ways to improve your report's performance, see "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the *Crystal Reports User Guide*.)

The left side of the dialog box lists nodes for various areas you can see information about:

- [Report Definition](#)
Select this node to see information about the content of the report: the number of fields, the number of summaries, and so on.
- [Saved Data](#)
Select this node to see information about the data in your report: the number of data sources used, the total number of records, and so on.
- [Processing](#)
Select this node to see information about how your report will be processed as a result of how you created it: if the grouping and sorting is done on the server, the number of passes requires, and so on.
- [Latest Report Changes](#)
Select this node to see information about recent changes to the report (listed on as changed or unchanged).
- [Performance Timing](#)
Select this node to see statistics related to the time it took your report to open and to format pages.

Note

Only the nodes that apply to the current state of the report appear. For example, until you preview a newly created report, only the Report Definition and Performance Timing nodes appears.

If you want to save the current Performance Information, select the Save to File button and enter a file name (.txt file) and destination.

Note

In the case of a subreport, the Performance Information dialog box displays only information relevant to the subreport itself.

38.103 Preview Sample dialog box

This dialog box appears when you choose the [Preview Sample command \[page 1087\]](#) from the View menu.

Use the Preview Sample dialog box to select how many records you want to preview.

All Records

Select this option to preview all the records available in the report.

First n records

Select this option to preview a specified number of records available in the report.

Related Information

[Preview tab \[page 93\]](#)

38.104 Properties dialog box

The Properties dialog box appears when you select an object in the [Repository Explorer dialog box \[page 1353\]](#) or [Database Expert \[page 1420\]](#) and choose Properties from its shortcut menu.

Use this dialog box to see detailed information about the repository or database object.

Related Information

[Add Item dialog box \[page 1155\]](#)

[Object Information dialog box \[page 1310\]](#)

38.105 Publish Object dialog box

The Publish Object dialog box appears when you select a folder in the [Workbench dialog box \[page 1377\]](#) and choose Publish to SAP BusinessObjects Enterprise from its shortcut menu.

Note

You must log onto SAP BusinessObjects Enterprise before you can see the Publish Object dialog box. For more information, see [Opening a report in an Enterprise folder \[page 629\]](#).

Use this dialog box to see the folders and objects that are stored on the SAP BusinessObjects Enterprise system that you have connected to.

Choose Location

Use this area to explore the folders on your SAP BusinessObjects Enterprise system. You can choose a folder to publish the reports that you have selected in the Workbench. In the Choose Location area, you can expand folders or refresh the view by selecting the appropriate option from the shortcut menu.

Add as Object Package

Use this option to publish the reports that you have selected in the Workbench as individual reports or as an object package. You can schedule individual reports in SAP BusinessObjects Enterprise as separate entities; you can schedule all of the reports in an object package as a single entity. Object packages allow users to view synchronized data across report instances. For more information about object packages, see the *SAP BusinessObjects Enterprise Administrator's Guide*.

Clear this option if you do not want to publish your folder as an object package.

Related Information

[Working with Enterprise folders \[page 628\]](#)

38.106 Record Selection Formula Editor dialog box

This dialog box appears within the [Formula Workshop dialog box \[page 1282\]](#) whenever you add or modify a record selection formula. You can open the Formula Workshop by itself, or you can select Record from the Selection Formulas submenu of the Report menu.

Use the Record Selection Formula Editor to set up a record selection formula that limits the records the program uses when preparing your report.

Interaction of the Record Selection Formula Editor and the Select Expert

The [Select Expert \[page 1457\]](#) and the Record Selection Formula Editor are interactive. That is, record selection criteria you enter via the Select Expert (from the Report menu) automatically generates a record selection formula which can be reviewed and modified. Likewise, record selection formulas and modifications to existing record selection formulas automatically update the selection criteria in the Select Expert.

Note

You can only use Crystal syntax when creating record selection formulas.

Note

No field placement box appears when you create a selection formula; the formula is simply stored in the report.

Note

Your record selection formula must be Boolean, that is, it must result in a Yes or No answer.

Note

You cannot use thousands separators in numbers you enter in a formula. Enter only the number itself.

Note

You can use only the period as a decimal separator.

Related Information

[Formula Editor dialog box \[page 1280\]](#)

[Formula Workshop dialog box \[page 1282\]](#)

[Formula Editor buttons \[page 698\]](#)

[Formula syntax \[page 690\]](#)

[Operators \[page 1464\]](#)

38.107 Refresh Report Data dialog box

There are two versions of the Refresh Report Data dialog box. This dialog box appears when you click the Refresh button on the Standard toolbar, or when you choose the [Refresh Report Data command \[page 1120\]](#) from the Report menu.

If your report does not contain any parameter fields, a simple message box appears. Click OK to refresh the report data.

If your report does contain parameter fields, the Refresh Report Data dialog box enables you to run the report using the current parameter values or to enter new parameter values.

Use current parameter values

Click this option to run the report using the current parameter values.

Prompt for new parameter values

Click this option if you want to run the report with different parameter values. When you choose this option the [Enter Parameter Values dialog box \[page 1230\]](#) appears.

Related Information

[Parameter Fields and Prompts \[page 831\]](#)

38.108 Reorder Displayed Members dialog box

This dialog box appears when you right-click a row or column member in an OLAP grid.

Use this dialog box to customize the order in which the dimension members appear in the OLAP grid. Select one or more dimensions and click the up and down arrows to define your own order.

Note

You must expand a parent member in order to reorder its children. To expand a member, right-click it in the OLAP grid and select Expand Member from the shortcut menu.

Related Information

[Reordering members \[page 425\]](#)

38.109 Report Alerts dialog box

The Report Alerts dialog box appears when you refresh your report's data.

Note

If you want to view alerts when refreshing report data, you must select "Display Alerts on Refresh" on the [Reporting tab \(Options dialog box\) \[page 1326\]](#).

Use the Report Alerts dialog box to view the name, message, and records associated with a triggered alert.

Name

This part of the alerts list displays the name of the triggered alert.

Message

This part of the alerts list displays the message you created for the triggered alert.

View Records

The record that triggered an Alert is shown in a tab with the name of the alert. If the record is hidden, the record's group is shown but drill down is not performed.

38.110 Report Explorer dialog box

The Report Explorer appears when you select the [Report Explorer command \[page 1088\]](#) from the View menu, or when you click the Report Explorer button on the Standard toolbar.

By default, the explorer appears docked on the right side of the Report Designer, but you can manually dock it on the left side, or the bottom of the designer. For more information about docking explorers, see [Opening and docking explorers \[page 117\]](#).

For more information about this explorer, see [The Report Explorer \[page 114\]](#).

38.111 Report Options dialog box

This dialog box appears when you choose the [Report Options command \[page 1077\]](#) from the File menu.

ⓘ Note

These options also appear in the [Options dialog box \[page 1314\]](#). Use the Options dialog box to change the default values for the program, use the Report Options dialog box to change the options for the current report.

Convert Database NULL Values to Default

Select this check box to have the program convert null values from the database to a 0 (for numeric fields) or a blank (for non-numeric fields).

ⓘ Note

This option affects only null values found in the database used for a report.

ⓘ Note

This option is not available when you have selected *Perform Grouping On Server* on the *Database* menu.

Convert Other NULL Values to Default

Select this check box to have the program convert non-database null values in the report to a 0 (for numeric fields) or a blank (for non-numeric fields).

ⓘ Note

This option affects only non-database null values in the report.

Show All Headers on Drill Down

Use this check box if you would like report drill-down views to include headers for all sections above the drill-down records. This option is not selected by default.

Always Sort Locally

Select this check box to ensure your records are sorted locally even if they were pre-sorted when retrieved from your data source.

Database Server is Case-Insensitive

This check box specifies whether or not the SQL data you use in the report becomes case sensitive. If the data you are using contains fields where the data is in both upper case and lower case, and you do not wish to categorize the data by this aspect, select this check box.

Note

Case Sensitive SQL data is not supported in the 32-bit standard edition of the program.

Perform Grouping On Server

This check box is only available if the Use Indexes or Server For Speed check box is selected.

Note

This option is not available for reports created from OLAP data.

Select this check box to have the program perform grouping on the server (server-side processing). Server-side processing allows you to set up a report that performs the majority of its processing on the server and pushes only relevant details to your computer. Server-side processing provides you with a number of benefits:

- Less time connected to the server.
- Less memory needed to process the report on your computer.
- Lower transfer time from the server to the client.

Server-side processing works only for reports based on SQL data sources (you cannot, for example, use server-side processing for a report based on a query, since a query is not an SQL data source).

This option is not selected by default.

For more information, see [Server-side processing \[page 921\]](#).

Use Indexes Or Server for Speed

When you select this check box, the program uses available indexes/servers to speed the record selection process.

When you clear this check box, the program selects records without the use of indexes/specified servers (a much slower process).

Note

If an index has a different name than the database it indexes, you need to identify the index for the program. You do this using the [Link Options dialog box \[page 1301\]](#), which is accessed from the [Links tab \(Database Expert\) \[page 1422\]](#).

Verify on First Refresh

When this check box is selected, the report refreshes with the current version of the active database (unless it is a stored procedure) and resets its record buffers to the current record size the first time you refresh it per session. This option is not selected by default.

Verify Stored Procedures on First Refresh

When this check box is selected, the report refreshes with the current version of the stored procedure and resets its record buffers to the current record size the first time you refresh it per session. This option is not selected by default.

Respect Keep Group Together On First Page

Clear this check box to ensure that Crystal Reports begins printing your first group on the first page of the report, even when you have selected the [Keep Group Together](#) check box.

Select this check box along with the [Keep Group Together](#) check box to have Crystal Reports begin printing the first group on the second page of the report, when the first group extends beyond a single page.

Save Lock Report Design

When you select this check box you are prompted to create a password. That password must be entered before you can save changes to the report.

Save Data With Report

Select this check box to save the current data with the report.

Suppress Printing If No Records

Select this option to suppress the printing of a report if that report does not contain any records.

Perform Query Asynchronously

Retrieving data from the database server can be divided into two parts: executing the SQL statement and fetching the rowset from the database server. Selecting this option allows Crystal Reports to execute the SQL statement asynchronously, which means that instead of waiting for the SQL statement to finish executing, the program checks intermittently to see if the user wants to cancel the process. This option is not selected by default.

Show Preview Panel

Select this check box to create a Preview Panel for use in the Preview tab. The Preview Panel includes a Group Tree, a Parameter Panel, and a Find option. For more information, see [Preview Panel \[page 94\]](#).

Display Alerts on Refresh

Select this check box if you want to see relevant [Report Alerts \[page 657\]](#) when you refresh your report. This option is selected by default.

Read-only

Select this check box to make the current report read-only. Those viewing the report won't be able to change its formatting in any way; they won't be able to add fields or other report objects, change font sizes or colors, and so on. They will, however, still be able to perform drill down, refresh data, export, save, and other options that don't change the physical layout of the report.

Note

This feature is for your convenience in protecting report formatting; it is not intended to be used as report security.

Select Distinct Records

Use this option to select only unique records from SQL tables in your database. By selecting this option in the Report Options dialog box, you set the option for the current report. Alternatively, you can set the option for all reports by switching on the [Select Distinct Records command \[page 1111\]](#) on the Database menu.

Note

This option is not available for reports created from OLAP data.

Select Distinct Data for Browsing

When you browse a field, by default you see the first 500 unique values of the field. You can change this for the current report by clearing this option. If Select Distinct Data for Browsing is not selected, browsing returns the unique values in the first 500 records.

Note

Note that since most SQL servers do not support selecting distinct records for long data types such as memo and blob fields, selecting and browsing distinct records are not supported for such fields.

Retain Original Image Color Depth

If you want images in your report to keep their original color depth when they are viewed, select this option. Otherwise, all images are converted to 8 bits per pixel before being displayed. The advantage of selecting this option is that it lets Crystal Reports render images as exact copies of the originals. Crystal Reports, however, will then store more information about the images, which may lead to larger files and slower response time in loading reports.

Prompt For Hyperlinks

When a hyperlink points to a file on your local machine (that is, the hyperlink is not a web link), choose this option if you want to be prompted with an alert that tells you a local file will be opened or executed. This is an optional security feature in Crystal Reports. For more information, see [Open Hyperlink dialog box \[page 1314\]](#).

Note

This option is always enabled when you open a report, even if you clear the option before you save the report.

Limit size of result set to

Enter a number to specify the maximum number of records that Crystal Reports returns from the database. The ability to specify this limit lets you control the size of data returned to your report, and it helps improve the predictability of report size and performance.

Note

This option is available only for database drivers that support the functionality.

Legacy XML Expert

Click the [Legacy XML Expert](#) button to display the [XML Expert \[page 1460\]](#) dialog box. The Legacy XML Expert is designed for you to format XML tags and attributes on one screen. An XML element is created for each section and field in your Crystal report.

Initial Report Part Settings

If you want to use objects from this report in the Report Part Viewer, define the home object in this area. The home object provides the base for a path of Report Part hyperlinks; it also becomes the first object from the report that appears in the Report Part Viewer.

Paste the Report Part Link

Use the Paste Link button to paste the name and data context of a report object that you selected and copied earlier. The button has these options: Context Report Part and Preview Report Part. These options affect the kind of data context that is pasted. For more information about pasting report part links and the options, see [The Another Report Object option \[page 637\]](#).

Object Name

Enter the name of the report object that you want to use as the home object.

Data Context

Enter the data context (that is, the specific record or records) that you want to use in your home object.

For more information, see [Report Part-specific navigation \[page 634\]](#).

Preview Pages Start With

This list enables you to select the default level of magnification for displaying reports on the Preview tab.

- **Full Size**
Full size allows you to view your report at 100 percent magnification for easy reading. When you resize the Preview tab, the report size remains unchanged. All that changes is the amount of the report you can view at one time. You may need to use scroll bars to see all sections of your report.
- **Fit Width**
Fit Width adjusts the width of the report to the width of the Preview tab. When you resize the Preview tab (within realistic limits), the page image resizes as well. To see a larger image, you can expand the window.
- **Fit Page**
Fit Page adjusts the width and the length of the report to let you see a full page of your report in the Preview tab. When you resize the Preview tab (within realistic limits), the page image resizes as well. To see a larger image, you can expand the window.

By default, the selector is set to Full Size.

Note

All of the functions available in the Preview tab are available at every magnification.

Convert Date-Time Field

This is a legacy option. This option is not available, nor is it needed for reports created in Crystal Reports version 9 or higher (versions of Crystal Reports released after 2001). In these versions, date-time fields are always saved as date-time types.

This option is available only when you open a report created in a version of Crystal Reports that was released prior to version 9 (released prior to 2002).

Using this list, specify if you want Date/Time string fields to be automatically converted to one of the following options:

- String format
- Date format
- Date-time format: This is the recommended option.

Note

Once you switch to this format, you cannot switch back to the original format.

Related Information

[Report Options command \[page 1077\]](#)

[Options dialog box \[page 1314\]](#)

38.112 Report Part Map Fields dialog box

This dialog box opens when you are creating an Another Report Object hyperlink and choose the Design View Report Part option (or if you choose the Context Report Part option for an object copied from a report's Design view) and your source and target reports have different data structures. Each field in the Unmapped Fields area represents a group in your source report. Use this dialog box to map these groups to report fields in your target report.

Unmapped Fields

The Unmapped Fields list shows the unmapped groups in your source report.

Report Fields

The Report Fields list shows the database fields in your target report that you can select for mapping.

Map

Click Map to associate the group selected in the Unmapped Fields list with the database field selected in the Report Fields list. After you click this button, the selected group and field are mapped, and both are moved to the Mapped Fields area.

Original Fields

The Original Fields list displays the groups in your source report that you have mapped.

Mapped Fields

The Mapped Fields list shows the database fields in your target report that you have mapped.

Unmap

Click Unmap to unmap the report groups and fields selected in the Mapped Fields area. After you click this button, the selected group and field are no longer mapped.

Related Information

[The Another Report Object option \[page 637\]](#)

38.113 Repository Explorer dialog box

The *Repository Explorer* appears when you select the [Repository Explorer command \[page 1088\]](#) from the *View* menu, or when you click the *Repository Explorer* button on the Standard toolbar.

By default, the explorer appears docked on the right side of the Report Designer, but you can manually dock it on the left side, or the bottom of the designer. For more information about docking explorers, see [Opening and docking explorers \[page 117\]](#).

Use the Repository Explorer to view many types of objects commonly used in Crystal Reports:

- Text objects
- Bitmaps
- SQL Commands (queries)

Storing objects in the SAP BusinessObjects Enterprise Repository provides a method for managing and using these objects in many different reports without having to recreate the objects from report to report.

Text objects and bitmaps can be added to the *Repository Explorer* by dragging them from your report to the appropriate repository folder. The [Object Information dialog box \[page 1310\]](#) or the [Add or Update Object dialog box \[page 1156\]](#) appears so you can add identifying information about the object.

Note

Custom functions are added to the repository through the [Formula Workshop dialog box \[page 1282\]](#). See [Using Custom Functions \[page 731\]](#) for more information. Commands are added to the repository through the Database Expert. See [Adding items to the repository \[page 124\]](#) for more information. Business Views are added to the repository through the Business View Manager.

Likewise, repository report objects can be dragged from the Repository Explorer into your report. Using the "Update Connected Repository Objects On Open" option on the [Reporting tab \(Options dialog box\) \[page 1326\]](#) ensures that repository objects in your report are updated with the latest version from the repository.

Shortcut menu

Shortcut menu options are available when you right-click folders or objects in the Repository Explorer:

- At the top level (the repository node), or at the subfolder level, you can insert a folder or refresh the display.
- For folders you've added, you can also rename and delete.
- For objects you've added, you can also see properties.

Toolbar

For a description of the toolbar options in the Repository Explorer, click [Repository Explorer toolbar \[page 123\]](#).

38.114 Resolve Map Mismatch dialog box

The Resolve Map Mismatch dialog box appears when you choose the [Resolve Mismatch command \(Map menu\)](#) [\[page 1126\]](#) from the Map menu, or from the right-click menu after you've selected a map in your report.

Use this dialog box to change map types, or to resolve mismatches between the names used for geographic regions.

38.114.1 Change Map tab (Resolve Map Mismatch dialog box)

Use this tab to specify the map you'd like to see if you want to have your data values presented with a different geographic map. For example, if the data used on your map applies only to Asia, you can choose the map of Asia instead of the default World map. This change gives you a more detailed map of Asia for a better presentation of your data.

Current map

This box displays (for viewing) the geographic map that is currently used for the map selected in your report. The same geographic map is highlighted in the Available maps box.

Available maps

This box displays a list of the possible geographic maps you can select for your report. Highlight the geographic map of your choice and click OK. The map in your report changes to the appropriate geographic map.

38.114.2 Resolve Mismatch tab (Resolve Map Mismatch dialog box)

Use this tab when the map you are working with uses a different geographic name from the one used in the database. For example, a map of Europe may use "United Kingdom" while the database uses the abbreviation "U.K." Until you resolve the data mismatch, the map will not be able to display any information for the selected geographic region.

Note

The first time you choose this command while creating or editing a report, a processing gauge appears briefly.

Assign this Field Name

This list displays all the geographic field names used in your database that do not match a default map name. Select items from this list (one at a time) and resolve the mismatch by choosing a name from the To this Map Name list.

To this Map Name

Use this list to select and assign a name from the list of defaults. When you have made a choice, the mismatched heading in the Assign this Field Name list is linked to the default name you selected in the To this Map Name list.

Match

Click Match to resolve the mismatch and save the link between the database field name and the default map name. The new assignment is shown in the list in the Matched results list.

Unmatch

Click Unmatch to delete the link between the database field name and the default map name. After removing the link, the database heading is unresolved. You can make another link for it if you choose.

38.115 SAP BusinessObjects Enterprise Explorer dialog box

The SAP BusinessObjects Enterprise Explorer dialog box opens when you select Universes or Repository as your data source.

Use this dialog box to choose a universe or items that are stored in the SAP BusinessObjects Enterprise Repository. Specifically, when on the Data screen, use this dialog box to choose a command or Business View to base your report on.

For information about the toolbar, see [Repository Explorer toolbar \[page 123\]](#).

38.116 Save As dialog box

This dialog box appears when you choose the [Save As command \[page 1073\]](#) from the File menu, you are working on a new report and have not yet saved it and use this dialog box to specify the name (and path) under which you want to save your report file, or you choose the [Save command \[page 1073\]](#) from the File menu while working on a new report that you have not yet saved.

Use the [Save As](#) dialog box if you have made changes to an existing report file and you want to save the changed version of your file under a different name. All changes you have made while working on the report will be saved to the new file and your original report file will remain unchanged.

Specify the name, location and type of the file that you are saving.

Note

When you save a report that is based on a Business View, the [Save As](#) dialog box contains an option called [Disconnect view security](#). Selecting this option disconnects the report from the Central Management Server (CMS) so that users are not prompted for logon when they open the report. Once a report is disconnected from the CMS, you cannot reconnect to SAP BusinessObjects Enterprise and its security features, nor can you refresh the report's data. This is a useful option for times when you might want to share data that is normally secured, but you don't want those who receive the report to be able to refresh the data later.

Related Information

[Save As command \[page 1073\]](#)

38.117 Saved Data Indexes dialog box

The Saved Data Indexes dialog box appears when you select the [Report Bursting Indexes command \[page 1116\]](#) from the Report menu.

Use this dialog box to select the fields you want indexed within your report's saved data. You can increase the performance of a Crystal report by indexing its saved data. Indexes you specify are created when you refresh the report's data.

Available Fields

This list shows all of the report and data source fields available for indexing. Some fields are better than others to choose for indexing. See [Indexing the right field\(s\) \[page 929\]](#) for more information about which fields to choose.

Indexed for Bursting

This list show the fields you have chosen to create indexes for.

Auto

Click this button to automatically add fields that are used in Record Selection formulas.

Arrow buttons

Use the > arrow between the two lists to add fields to the Indexed for Bursting list from the Available Fields list. Use the < arrow to remove fields you've already added. The double arrow removes all fields.

Browse Data

Click this button to browse the field data. Click the Close button to return to the Saved Data Indexes dialog box.

38.118 Search dialog box (OLAP Worksheet)

The Search dialog box appears when you select the Search button in the [Member Selector dialog box \[page 1307\]](#).

Use this dialog box to search for members within the Member Selector dialog box.

Search tab

For a procedure about using this tab and its fields, see [Searching for members \[page 422\]](#).

Advanced tab

For a procedure about using this tab and its fields, see [Using the advanced search options \[page 423\]](#).

38.119 Select Barcode Type dialog box

This dialog box appears when you choose the [Change to Barcode command \[page 1134\]](#) from the shortcut menu.

When a barcode type is selected, the information for that barcode is displayed in the Selected Barcode Type area, including details such as the Vendor Name, Font Name, and a Sample of what the barcode would look like.

For more information about the desired barcode type, click the "View Vendor's Web Site" button.

38.120 Select Export File dialog box

The Select Export File dialog box appears when you export a report and select either of the HTML formats in the [Export dialog box \[page 1236\]](#).

Directory Name

Enter a directory name for your HTML page(s) and image(s), and then select the path location for where you want the program to create this directory. You must specify a directory for HTML export because the program can produce more than one file for this export format.

Drives

Select any of your mapped drives as the location in which you want the program to create the HTML file(s).

Base File Name

Specify a name to use as the base name for the HTML file(s) that the program creates. If only one HTML file is created, the base name is the name of that file. If several HTML files are created, the base name is used as the root file name; a number is appended to this name for each page that is needed. For example, MyReport1, MyReport2, MyReport3, and so on.

Page navigator

Select this check box if you want the program to include navigation link (First Page, Previous Page, Next Page and Last Page) on each HTML page.

Separate HTML Pages

Select this check box if you want the program to create separate HTML pages for each page in your report.

Note

When you export to a single file (that is, you clear the Separate HTML Pages check box), all blank space in headers and footers, as well as top and bottom page margins, is included. If you don't want to see blank spaces in your HTML file, suppress empty header and footer sections in your report, and set the top and bottom page margins to zero in the Page Setup dialog box.

Page Range

Use the options in this area to limit the number of report pages that you want to export.

Related Information

[Exporting to HTML \[page 626\]](#)

38.121 Set Datasource Location dialog box

The Set Datasource Location dialog box appears when you choose the [Set Datasource Location command \[page 1109\]](#) from the Database menu.

Use this dialog box to change the location of the data source currently used in your report.

Current Data Source

This list displays the database and tables currently used in the report. The main report is shown as the root node.

Note

All subreports are located under one node so that the data sources for them are grouped together. A data source used in two or more subreports is indicated by a different icon at the data source level.

Note

If a table is used in two or more subreports but the alias name is different, the real table name is appended to the end.

Expanding the Properties node for a data source or for a table displays detailed information about the item. Some entries in the Properties node, such as Database Name, have shortcut menu options; any property that can be edited has, at least, an Edit option on its shortcut menu.

Replace with

This list displays the same options as the Data tab of the Database Expert. See [Data tab \(Database Expert\) \[page 1421\]](#) for information about how to select a new data source or table.

Update

Click the Update button to update the new data source information in the Current Data Source list. If the new data source is different from the one you're replacing, the [Map Fields dialog box \[page 1306\]](#) appears. See [Remapping altered database fields \[page 926\]](#) for information about how to use the Map Fields dialog box.

Related Information

[Using the Set Datasource Location process \[page 926\]](#)

[To change the data source \[page 900\]](#)

38.122 Set Default Export Options dialog box

The Set Default Export Options dialog box appears when you choose the [Report Export Options command \(Export submenu\) \[page 1076\]](#) from the File menu.

Use this dialog box to set default export options for the report that you have open. When a user exports the report through Crystal Reports, the various report viewers, or BusinessObjects Enterprise, the default options automatically appear; the user can accept these options or make modifications as needed.

Format

This list displays the supported formats for which you can set default export options. Select an option and click OK to open the appropriate dialog box. For example, if you choose Microsoft Excel, the [Excel Format Options dialog box \[page 1232\]](#) appears. You would then select default options for Microsoft Excel export format; any user who exports this report sees these default options.

Note

You can set default export options for each supported format in the Set Default Export Options dialog box.

38.123 Set Default Values dialog box

Use the Set Default Values dialog box to specify the default prompts you want to appear with your parameter field. This dialog box appears when you click the Default Values button in the [Create Parameter Field dialog box \[page 1184\]](#) or the [Edit Parameter Field dialog box \[page 1227\]](#).

Browse table

This field shows the name of the Dimension that you are creating a parameter for. It is not editable.

Select Default Values

Click Set Default Values to open the Member Selector dialog box. Use this dialog box to pick your default values from the tree view for your dimension. For information about using the Member Selector, see [Member Selector dialog box \[page 1307\]](#).

Order arrow buttons

Use the Order arrow buttons to move a value up or down in the Description list.

Set Description

Click this button to edit the description of any value in the Description list. Clicking this button opens the [Define Description dialog box \[page 1221\]](#).

Display

Use the Display list to select how you'd like the default values to appear. You can choose to display values with or without their descriptions.

Order

In the Order list, you can specify whether you want the default values sorted. The sorting order can be alphabetically ascending, alphabetically descending, or in the hierarchy of the dimension.

Order based on

In the Order based on list, you can specify whether to sort default values based on value or description.

Note

This option is available only for alphabetical sorts.

38.124 Set Line Height dialog box

The Set Line Height dialog box enables you to change the line height of the selected text object. It appears when you choose the Change Line Height command from the Format menu.

Use this dialog box to change the line height to the default or by specifying an exact height and descent.

Note

You can also change the line height by dragging the handles of the selected text object.

38.125 Set OLAP Cube Location dialog box

This dialog box appears when you click the [Set OLAP Cube Location command \[page 1110\]](#) on the Database menu.

Click Select and then choose an OLAP cube from the [OLAP Connection Browser dialog box \[page 1312\]](#).

Related Information

[Updating an OLAP report \[page 401\]](#)

38.126 Set Print Date and Time dialog box

This dialog box appears when you choose the [Set Print Date and Time command \[page 1121\]](#) from the Report menu.

Use the Set Print Date and Time dialog box to set the date for your report. You can choose either today's date or specify another date.

Today's Date and Time

Select this option to use today's date and time as your report date.

Other

Select this option if you want to use a date other than today's as the report date. When you select this option, the Date and Time edit boxes are activated where you can specify the desired print date.

Date

Use the Date list to select a future print date.

Time

Use the Time box to set the print time.

38.127 Set Prompt Order dialog box

This dialog box appears when you choose the [Set Prompt Order command \[page 1142\]](#) from the shortcut menu in the Parameter Fields area of the [Field Explorer dialog box \[page 1241\]](#).

Note

You must have more than one parameter field in the Parameter Fields area before this option appears.

Use the up and down arrow buttons to change the order of parameter fields in the list area. The order of the parameters in this list corresponds to the order you see when the program prompts you to choose parameter values.

38.128 Show SQL Query dialog box

This dialog box appears when you choose the [Show SQL Query command \[page 1111\]](#) from the Database menu.

The Show SQL Query dialog box displays the SQL query that the program is sending to your SQL server.

The query can have up to four parts:

SELECT

SELECT lists all of the fields used in the report, including fields used in formulas, totals, the sort order, the selection formula, and ranges. For example:

```
SELECT  
DIV_INFO.DIVISION_ID, DIV_INFO.DIVISION_NAME,  
DEP_INFO.DEPARTMENT_ID, DEP_INFO.DEPARTMENT_NAME
```

Shows that the report is using the DIVISION_ID, DIVISION_NAME, DEPARTMENT_ID, and DEPARTMENT_NAME fields.

FROM

FROM lists all the tables used in the report, and each table name is followed by an alias name. For example:

```
FROM
SYSADM.DIV_INFO DIV_INFO,
SYSADM.DEP_INFO DEP_INFO
```

Shows that the report is using the DIV_INFO and DEP_INFO tables from the SYSADM database, and in each case, the alias name is the same as the table name.

WHERE

WHERE specifies any links between tables and field ranges for the report. For example:

```
WHERE
DIV_INFO.DIVISION_ID = DEP_INFO.DIVISION_ID
```

Shows that the DIV_INFO and DEP_INFO tables are linked via the DIVISION_ID fields.

ORDER BY

ORDER BY sorts the data in alphabetic or numeric ascending or descending order. For example:

```
ORDER BY
DIV_INFO.DIVISION_ID ASC
```

Sorts the records in ascending order by the values that appear in the DIV_INFO.DIVISION_ID field.

Related Information

[Show SQL Query command \[page 1111\]](#)

38.129 Sort Control dialog box

This dialog box appears when you click *Bind Sort Control* from the shortcut menu.

Select a field from this box to enable a *Sort Control*. For example, if you select the country field from this box, the resulting *Sort Control* will sort all affected records by country.

Select the *< Not Interactive >* option to disable a *Sort Control*.

Note

The [Sort Control](#) dialog box shows all the record and group sorts, but not all of them can be bound to a sort control:

All record sorts can be bound to sort controls.

Top N, Bottom N, Top Percentage, and Bottom Percentage group sorts cannot be bound to sort controls.

Groups which contain any group sort cannot be bound to sort controls.

Related Information

[Sort controls \[page 170\]](#)

38.130 SQL Expression Editor dialog box

The SQL Expression Editor appears as part of the [Formula Workshop dialog box \[page 1282\]](#) when you select SQL Expression Fields in the [Field Explorer dialog box \[page 1241\]](#) and click New or Edit.

Note

When creating a new expression, you have to enter a name in the [SQL Expression Name dialog box \[page 1367\]](#) before the workshop will appear.

The SQL Expression Editor is used to combine components into SQL expressions. The editor allows you to type the components into the SQL expression directly, or, in the case of fields, functions, operators, group field values, and formulas, to select them from lists of those items that are available.

The SQL Expression Editor requires you to enter the various components according to a specific set of rules called syntax. Some examples of rules:

- Text enclosed in quotation marks.
- Arguments enclosed in parentheses (where applicable).
- Referenced formulas identified with a leading @ sign, and so on.

Note

Crystal Reports supports ANSI standard functions and operators. Check your Database Management System's documentation for a SQL syntax reference, or search the Internet for a references of the ANSI standard functions and operators.

Note

Some functions may use a different syntax in Crystal Reports than they do in other programs. For example, the syntax for the CONVERT function is often something like `CONVERT(data_type [(length)], expression [, style])`. In the Crystal Reports SQL Expression Editor, however, the expected syntax is `CONVERT(expression, <SQL datatype>)`.

The SQL Expression Editor checks the syntax and helps you debug (fix) problems before you enter the SQL expression field into the report. For more information about the editor, see the [Formula Editor dialog box \[page 1280\]](#), which the SQL Expression Editor resembles.

38.131 SQL Expression Name dialog box

The SQL Expression Name dialog box appears when you select SQL Expression Fields in the [Field Explorer dialog box \[page 1241\]](#) and click New.

Use the dialog box to enter a name for a SQL expression field you are creating.

New Name

Use this box to enter a name for a new SQL expression field you are creating.

Then click OK to open the [Formula Workshop dialog box \[page 1282\]](#). You can use the Formula Workshop to enter the SQL expression.

38.132 Start Page

This page appears as a tab in the designer whenever you run Crystal Reports. You can also open this page by choosing the [Show Start Page command \[page 1129\]](#) from the Help menu.

Use the Start Page for typical start-up tasks such as opening an existing report, using a wizard to design a new report, looking at sample reports, searching the online help for a topic, and so on. The Start Page also provides a place to search for and read a wide selection of online resources that pertain to Crystal Reports and other products from Business Objects.

38.132.1 Getting Started

Use the link at the top of this area to register your copy of Crystal Reports. Clicking this link opens the [SAP BusinessObjects Registration Wizard \[page 1399\]](#).

Recent Reports

This area contains a list of recently opened reports as well as a link that calls the [Open dialog box \[page 1314\]](#) so you can search for an existing report to open.

New Reports

This area contains a number of options for creating a new report and for using other features that are available in Crystal Reports.

Wizards

Click any link that includes the word Wizards to create a new report using one of the [Report Creation Wizards \[page 1380\]](#). The appropriate wizard appears so that you can start creating your report.

Blank Report

Click this link to create a new report from scratch. Choose the data on which to base your report from the [Database Expert \[page 1420\]](#).

Help System

Search the index of the Crystal Reports Online Help for a topic that you'd like to know more about. When you click Go, the help appears with the nearest match selected on the Index tab. Click Display to see this topic, or search the index to find a different term.

Sample Reports

Click this link to see a selection of sample reports that illustrate many of the features that are available in Crystal Reports. The link calls the [Open dialog box \[page 1314\]](#), which displays folders that contain the sample reports.

Check for Updates

Click this link to check for updates to Crystal Reports. The link calls the Business Objects Update Service dialog box, which you can use to see the updates that are available and to install those that you want.

If you want to disable this option, clear the [Check for Updates on Start Up command \[page 1129\]](#) on the Help menu.

Note

The machine that you are using must have a connection to the Internet before you can use this option.

Show available online resources

Select this option if you want to see a selection of online resources that pertain to Crystal Reports.

38.132.2 Online Resources

This area is available when you select the "Show available online resources" option. You can search the SAP BusinessObjects web site by entering a word or phrase and clicking Go in the section above the content links.

The main area of this part of the Start Page offers a selection of online resources that are of interest to report creators:

- Product announcements.
- Top-ranked content from the Business Objects web site.
- Web resources that pertain to Crystal Reports and other products from SAP BusinessObjects.
- Featured articles about new and interesting features.
- Featured articles that are aimed at a developer audience.

Note

If you are looking for general information about using Crystal Reports, click [Introduction to SAP Crystal Reports 2025 \[page 47\]](#) or [Introduction to Reporting \[page 85\]](#).

38.133 Subreport Links dialog box

This dialog box appears when you choose the [Subreport Links command \[page 1082\]](#) from the Edit menu, or when you right-click the subreport object and choose the Change Subreport Links command from the shortcut menu that appears.

Use the Subreport Links dialog box to specify the field in the parent report that will serve as the link field to the data in the subreport.

For Subreport

Use this list to select the subreport that you want linked to the data in the container report.

Available fields

The Available fields box lists all of the fields available in the container report to which you can link your subreport.

Add (>)

Click the Add button (>) to copy a field from the Available Fields box into the Field(s) to link to box.

Remove (<)

Click the Remove button (<) to remove a field from the Field(s) to link to box.

Field(s) to link to

The Field(s) to link to box displays the currently selected fields to which you intend to have your subreport link.

Subreport parameter field to use

The Subreport parameter field to use list appears when the Field(s) to link to box contains at least one field name. Use the Subreport parameter field to use list to determine the subreport field on which the container report and the subreport are linked.

Select data in subreport based on field

Use the Select data in subreport based on field check box to activate the Select data in subreport based on field list.

After you select this check box, choose a field from the list below to organize the subreport data based on a specific field. Clear this check box to have the subreport adopt the organization used in the primary report.

Related Information

[Subreport Links command \[page 1082\]](#)

[Subreports \[page 870\]](#)

38.134 Symbol Style dialog box

The Symbol Style dialog box appears when you click Graduated on the [Type tab \(Map Expert\) \[page 1438\]](#) and then click Customize. The Symbol Style dialog box offers a variety of formatting and style options for your Graduated map symbol, including:

- Font type
- Font size
- Symbol options
- Color
- Rotation angle
- Background options
- Effects options

Font

Use this list to change the font of your Graduated map symbol. The default font is Map Symbols.

Font size

Use this list to change the font size of your Graduated map symbol. The default font size is 18.

Symbol

Use this list to change the symbol for your Graduated map.

Color

Use this palette to change the color of the symbol for your Graduated map.

Sample

The sample to the right of Font, Symbol, and Color shows you how the symbol for your Graduated map is affected by changes you make in the Symbol Style dialog box.

Rotation Angle

Use this box to change the rotation angle of your symbol as it appears on the Graduated map.

None

Use this option to display your Graduated map symbol without a background.

Halo

Use this option to display your Graduated map symbol with a Halo background.

Note

Halo backgrounds are most effective when the background color greatly contrasts the symbol color. For example, a white or yellow halo background will give a true halo effect to a navy blue symbol.

Box

Use this option to display your Graduated map symbol in a colored-background box.

Color

Use this list to select the background color for your Graduated map symbol.

Effects options

Use these check boxes to apply the following effects to your Graduated map symbol:

- Bold
- Italic
- Drop Shadow

38.135 Text Format dialog box

The Text Format dialog box appears when you have a text object in edit mode and select the [Text Formatting command](#) [page 1103] from either the Format menu or the shortcut menu.

Note

If you do not have your text object in edit mode, the option is not available.

Use this dialog box to format text in a text object. The Text Format dialog box contains three tabs:

- [Font tab \(various dialog boxes\)](#) [page 1274]
- [Paragraph tab \(various dialog boxes\)](#) [page 1275]
- [Tabs tab \(Text Format dialog box\)](#) [page 1373]

The Font and Paragraph tabs are shared with the [Format Editor dialog box](#) [page 1256], but the Tabs tab is unique to the Text Format dialog box.

38.135.1 Tabs tab (Text Format dialog box)

Use the tab to set, delete, modify and edit tabs in a text object.

Tab Stop Position box

Enter the desired tab stop position in this box.

Alignment options

Choose from four alignment option for the selected tab stop: left, right, center, and decimal.

Set button

Once you have entered a tab stop position into the Tab Stop Position box, click this button to set it.

Delete button

When you click this button, the selected tab stop is deleted.

Delete All button

When you click this button all the defined tab stops are deleted.

38.136 Toolbars dialog box

Use the Toolbars command on the View menu to show or hide toolbars. There are several toolbars to choose from:

- Standard
- Formatting
- Insert Tools
- Expert Tools
- Navigation Tools
- External Command

Use the check boxes provided to show any or all of the toolbars.

You can also choose a number of display options from this dialog box.

Show ToolTips

Select this check box to have the program show tool tips for the buttons on the toolbar. Tool tips appear with the name of the button when you leave your mouse pointer over a button for a few seconds. This check box is selected by default.

Autohide Animation

Select this option to activate a scrolling effect for a dockable window for which you have chosen the Auto Hide option.

Reset all toolbars and explorers on next restart

Use this option to revert all toolbar settings back to their defaults. The reset action takes effect the next time you open Crystal Reports.

Related Information

[Standard toolbar \[page 1145\]](#)

[Formatting toolbar \[page 1147\]](#)

[Insert Tools toolbar \[page 1148\]](#)

[Expert Tools toolbar \[page 1149\]](#)

[Navigation Tools toolbar \[page 1150\]](#)

[External Command toolbar \[page 1151\]](#)

38.137 Trendlines dialog box

This dialog box appears when you right-click a data series item in a chart and then choose Trendlines from the Chart menu.

Use the Trendlines dialog box to fit one or more statistical lines (trendlines) to a series of data on the chart.

Available Types

Shows the available trendline types.

Show Trendlines

Shows the trendline types that will be displayed on the chart.

Add

Click the Add button (>) to copy the selected trendline type from the Available Types list to the Show Trendlines list.

Remove

Click the Remove button (<) to remove the selected trendline from the Show Trendlines list.

Remove All

Click the Remove All button (<<) to remove all trendlines from the Show Trendlines list.

Show Color same as Series

Use this option to show the curve in the same color as the selected series item on the chart.

Show Formula

Use this option to display the formula of a regression line (linear, log, exponential, and so on) on the graph. However, this function is not available for simple curves such as mean, standard deviation and connected lines.

This option is only available when a regression or polynomial curve is selected in the Show Trendlines list.

Show Correlation Coefficient

Use this option to display how closely a trendline fit line intersects with the data-points it is charting.

This option is only available when a regression or polynomial curve is selected in the Show Trendlines list.

Smooth Factor

Type a value (between 10 and 1000) in this box to control the smoothness of the curve or regression line. The higher the number, the smoother the curve.

This option is only available when a Smooth, Regression or Polynomial curve is selected in the Show Trendlines list.

Order

Type a value to specify the order of the trendline's curve. The options vary, depending on the trendline.

Moving Average

Order determines the number of values within a period. If you enter 3 for the number of periods, the first 3 values (P1, P2, P3) will be averaged. One value is then shifted over to take another 3-value average (P2, P3, P4). This continues (in groups of 3) until the final value is reached.

Polynomial

Order 0	Draw line at the mean.
Order 1	Draw linear regression line.
Order 2	Draw second order curve.
Higher order	Draw higher order polynomials.

Financial/Scientific

Click Financial to plot the average of each period at the right end of the period.

Click Scientific to plot the average of each period in the middle of the period.

These options are only available when a Moving Average curve is selected in the Show Trendlines list.

38.138 Workbench dialog box

The Workbench appears when you select the [Workbench command \[page 1089\]](#) from the View menu, or when you click the Workbench button on the Standard toolbar.

By default, the explorer appears docked on the upper-right side of the Report Designer, but you can manually dock it elsewhere in the designer. For more information, see [Opening and docking explorers \[page 117\]](#).

For more information about the this explorer, see [The Workbench \[page 115\]](#).

38.139 XML Attribute dialog box

The XML Attribute dialog box appears when you click the Create button or the Edit button on the [XML Expert \[page 1460\]](#). Use this dialog box to create or modify attribute names, types, and values. Values saved in the XML Attribute dialog box appear in the Attribute list of the associated element.

Attribute Name

This box contains the name you want to use for your attribute. Each attribute in an element must have a unique attribute name. Attribute names do not have to be unique across elements.

Attribute Type

This list contains content type options for the attribute you're creating:

- String
- Decimal
- Boolean
- Date
- Time
- Date Time

Instance Value

Use the options in this area to set an instance value. Instance values can be set as a field, or as text.

Field

This option lets you select a report field of the specified attribute type as your instance value. The Field option is available only if the corresponding report object is not Report (root node) or an Area Pair type. When the option is selected, the associated box displays the fields of the specified type available in the report.

Text

This option lets you enter text in the associated box as an instance value.

Default Value

Enter a value in this field to assure the attribute is not left empty when there is no instance value.

38.140 XML Export Schema dialog box

This dialog box appears when you click the [Show Schema](#) button on the [Manage XML Exporting Formats dialog box \[page 1305\]](#).

As well as viewing the schema that is used to format the report at the time it is exported, you can also highlight all or a portion of the text using the mouse and copy it onto the clipboard using the mouse controls or keyboard shortcuts.

38.141 Zoom dialog box

This dialog box appears when you choose the [Zoom command \[page 1090\]](#) from the View menu.

Use the Zoom dialog box to set the magnification factor for zooming in or out of your report.

Magnification Factor

Click this option to set the magnification to a particular percentage. Type the magnification factor into the box, for example 50%.

Fit One Dimension

Click this option to fit the page according to the width of the report in the Design or Preview tabs.

Fit Whole Page

Click this option to fit the whole page of the report in the Design or Preview tabs.

Related Information

[Zoom command \[page 1090\]](#)

[Design tab \[page 90\]](#)

[Preview tab \[page 93\]](#)

[Using the zoom feature \[page 111\]](#)

39 Wizard and Expert Reference

This section describes the wizards and experts available in Crystal Reports.

39.1 Overview

Crystal Reports includes a number of wizards that help you create reports with step-by-step instructions. Using these wizards, you can prepare graphs, set up the overall report format, link related database tables, and select records and groups for inclusion in the report.

Wizards do not use a system of tabs; instead they present you with a series of screens. Simply follow the instructions in the screens that appear and the wizard leads you through otherwise complex procedures with a minimum of effort.

Crystal Reports also includes many experts to help you with common tasks you might want to undertake after you've created a report, such as selecting records, creating groups, applying templates, and so on.

Experts use a series of tabs. Begin on the first tab and proceed to the last (if necessary—you can exit an expert on any tab).

39.2 Report Creation Wizards

A Report Creation Wizard appears when you go to the File menu and choose one of the wizards from the New submenu.

Note

You can also choose a wizard from the [Start Page \[page 1367\]](#).

Each Report Creation Wizard consists of several screens that lead you through step-by-step instructions for creating the specified report.

There are four Report Creation Wizards.

39.2.1 Standard Report Creation Wizard

The Standard Report Creation Wizard is the most generic of the experts. It provides many screens that are common to the other Report Creation Wizards. The Standard Report Creation Wizard guides you through choosing a data source and linking database tables. It also helps you add fields and specify the grouping,

summarization (totals), and sorting criterion you want to use. Finally, the Standard Report Creation Wizard leads you through chart creation and record selection.

The Template screen contains predefined layouts for you to apply to your report to give it more impact.

The Standard Report Creation Wizard has the following screens:

- [Data screen \(Report Creation Wizard\) \[page 1385\]](#)
- [Link screen \(Report Creation Wizard\) \[page 1393\]](#)
- [Fields screen \(Report Creation Wizard\) \[page 1387\]](#)
- [Grouping screen \(Report Creation Wizard\) \[page 1390\]](#)
- [Summaries screen \(Report Creation Wizard\) \[page 1397\]](#)
- [Group Sorting screen \(Report Creation Wizard\) \[page 1389\]](#)
- [Chart screen \(Report Creation Wizard\) \[page 1382\]](#)
- [Record Selection screen \(Report Creation Wizard\) \[page 1393\]](#)
- [Template screen \(Report Creation Wizard\) \[page 1398\]](#)

39.2.2 Cross-Tab Report Creation Wizard

The Cross-Tab Report Creation Wizard guides you through the creation of a report in which your data is displayed as a cross-tab object. Two special screens (Cross-Tab and Grid Style) help you create and format the cross-tab itself.

The Cross-Tab Report Creation Wizard has the following screens:

- [Data screen \(Report Creation Wizard\) \[page 1385\]](#)
- [Link screen \(Report Creation Wizard\) \[page 1393\]](#)
- [Cross-Tab screen \(Report Creation Wizard\) \[page 1384\]](#)
- [Chart screen \(Report Creation Wizard\) \[page 1382\]](#)
- [Record Selection screen \(Report Creation Wizard\) \[page 1393\]](#)
- [Grid Style screen \(Report Creation Wizard\) \[page 1388\]](#)

39.2.3 Mailing Labels Report Creation Wizard

The Mailing Labels Report Creation Wizard lets you create a report that is formatted to print on any size mailing label. You can use the Label screen to select a commercial label type, or you can define your own layout of rows and columns for any multi-column style report.

The Mail Label Report Creation Wizard has the following screens:

- [Data screen \(Report Creation Wizard\) \[page 1385\]](#)
- [Link screen \(Report Creation Wizard\) \[page 1393\]](#)
- [Fields screen \(Report Creation Wizard\) \[page 1387\]](#)
- [Label screen \(Report Creation Wizard\) \[page 1391\]](#)
- [Record Selection screen \(Report Creation Wizard\) \[page 1393\]](#)

39.2.4 OLAP Report Creation Wizard

The OLAP Report Creation Wizard lets you create a report in which your OLAP data is displayed as a grid object. Although similar to the Cross-Tab Report Creation Wizard in several ways, the OLAP Report Creation Wizard appears to be different due to the requirements of working with OLAP data sources. You first specify the location of your OLAP data, and then you choose the dimensions you want to include in the grid. Next you filter the report data and choose the style of the grid object. Finally, you can insert a chart if you wish.

The OLAP Report Creation Wizard has the following screens:

- [OLAP Data screen \(OLAP Report Creation Wizard and OLAP Expert\)](#) [page 1386]
- [Rows/Columns screen \(OLAP Report Creation Wizard and OLAP Expert\)](#) [page 1394]
- [Slice/Page screen \(OLAP Report Creation Wizard and OLAP Expert\)](#) [page 1396]
- [Style screen \(OLAP Report Creation Wizard\)](#) [page 1397]
- [Chart screen \(Report Creation Wizard\)](#) [page 1382]

39.2.5 Screen reference

The following sections describe the screens available in the Report Creation Wizards.

39.2.5.1 Chart screen (Report Creation Wizard)

Use the Chart screen to insert a graph or chart into a report. This screen appears with slight variations in all Report Creation Wizards except the Mailing Labels wizard.

In the Standard Report Creation Wizard, the Chart screen appears only if you've specified a group on the [Grouping screen \(Report Creation Wizard\)](#) [page 1390] and a summary on the [Summaries screen \(Report Creation Wizard\)](#) [page 1397]. Creating a chart is an optional step in all wizards.

Chart types

Click the type of chart you'd like to add to your report. You can choose from these types:

- No Chart
- Bar Chart
- Line Chart
- Pie Chart

Chart title

Add a title for your chart.

Chart off entire grid

This option appears only when you use the [OLAP Report Creation Wizard \[page 1382\]](#). Select the option to chart on all grid column and row fields. When you select this option, all other fields in the area are unavailable. Charting off an entire grid is useful when the grid has asymmetric filters based on members from two or more dimensions.

On change of

Select the group field you want to use as a condition for plotting values in a graph or chart. Points will be plotted whenever the value in the field changes. You can choose from the groups you created on the [Grouping screen \(Report Creation Wizard\) \[page 1390\]](#).

Subdivided by

Use this list to choose the secondary row or column to base your chart on. Clicking this list displays the row/columns fields in your report that you can choose from.

Show summary

Use this list to select the summary field whose values you want to display in your graph. You can choose from the groups you created on the [Summaries screen \(Report Creation Wizard\) \[page 1397\]](#).

Other Dimensions

Click the Other Dimensions button (available only in the OLAP wizard) to call the [Format Other Dimensions dialog box \[page 1279\]](#).

Click a dimension (shown with its field setting) in the list then click "Select a field value" to specify a new field for the dimension to be fixed to.

Related Information

[Charting \[page 303\]](#)

39.2.5.2 Cross-Tab screen (Report Creation Wizard)

Use the Cross-Tab screen to create and format a cross-tab object for your report. This screen appears only in the [Cross-Tab Report Creation Wizard \[page 1381\]](#).

Available Fields

This list displays all the tables and fields that you have activated for use in the report.

Select the field you want to use and drag it into the appropriate list, or click the appropriate arrow button. Repeat this process to add additional fields.

Browse Data

Click the Browse Data button to view the values for a field you have selected in the Available Fields list.

Find Field

Click this button to enter the name of a field you'd like to locate in the Available Fields list.

Add or remove arrow buttons

Use the > arrows adjacent to each list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added.

Up or down arrow buttons

Use the up and down arrows adjacent to each list to change the order of fields you've added from the Available Fields list.

Columns

This list displays the fields that you have selected as column headings for your cross-tab. If you have multiple column headings, they will appear on your report in the order in which they appear on this list.

To add a column heading to this list, select a field from the Available Fields list and then drag it onto the list, or click the > arrow adjacent to the Columns list.

If you choose a Date field or a Boolean field, the adjacent list becomes available with options that apply to that type of field. For example, if you pick a date field, you can select options such as "for each day," "for each week," "for each two weeks," and so on.

Rows

This list displays the fields that you have selected as row headings for your cross-tab. If you have multiple row headings, they will appear on your report in the order they appear on this list.

To add a row heading to this list, select a field from the Available Fields list and then drag it onto the list, or click the > arrow adjacent to the Rows list.

If you choose a Date field or a Boolean field, the adjacent list becomes available with options that apply to that type of field. For example, if you pick a Boolean field, you can select options such as "on any change," "on change to yes," "on change to no," and so on.

Summary Fields

This list displays the fields that you have selected as summarized field headings for your cross-tab. If you have multiple summarized field headings, they will appear on your report in the order they appear on this list.

To add a summarized field heading to this list, select a field from the Available Fields list, then drag it onto the list, or click the > arrow adjacent to the Summary Fields list.

If you add your summary field, the adjacent list becomes available with summary options. Choose the kind of summary you'd like to apply to the field.

Related Information

[Cross-Tab Objects \[page 346\]](#)

39.2.5.3 Data screen (Report Creation Wizard)

The Data screen appears in the Report Creation Wizard when you select any of the wizards on the [Start Page \[page 1367\]](#) when creating a new report.

Note

The OLAP Report Creation Wizard uses a different data screen. See [OLAP Data screen \(OLAP Report Creation Wizard and OLAP Expert\) \[page 1386\]](#).

Use the Data screen to select the data source and tables for your new report.

Available Data Sources

The available data sources tree is where you select your data source. For information about how to use this list, see [Data tab \(Database Expert\) \[page 1421\]](#).

Selected Tables

Once you have chosen the desired data source, use the > arrow to add the tables you want to use in your report.

39.2.5.4 OLAP Data screen (OLAP Report Creation Wizard and OLAP Expert)

The OLAP Data screen is common to the OLAP Report Creation Wizard and the OLAP Expert:

- In the OLAP Report Creation Wizard, it appears when you select OLAP Cube Report Wizard on the [Start Page \[page 1367\]](#).
- In the OLAP Expert, it appears when you select the [OLAP Grid command \[page 1096\]](#) from the Insert menu, or when you click the Insert OLAP Grid button on the Insert Tools toolbar.

Use the Data screen to give specific information about your OLAP data source:

- The specific cube you intend to report on.
- The connection to the cube.
- The name of the database.
- The type of OLAP data source.

Note

You will not be able to proceed beyond the OLAP Data screen until you have properly selected an OLAP cube. Furthermore, working with OLAP data requires a previous understanding of the OLAP concept. Make sure you are familiar with OLAP concepts and your OLAP software before attempting to design OLAP-based reports. If you have questions, please contact your System Administrator.

Select Cube

When you click this button, the [OLAP Connection Browser dialog box \[page 1312\]](#) appears, prompting you to choose the OLAP cube you want to build your report on.

Select CAR File

Note

This option appears only in the OLAP Report Creation Wizard.

When you click this button, the [Open dialog box \[page 1314\]](#) appears. Use the Open dialog box to search for a CAR file to build your report on.

Note

CAR files (that is, files with the extension of .car) are application files created in OLAP Intelligence Professional.

Cube

This information box specifies which cube you are currently working on.

Type

This information box specifies what kind of data you are using.

Server

This information box tells you the server on which the cube data is located.

39.2.5.5 Fields screen (Report Creation Wizard)

Use the Fields screen to select the fields you want included in your report.

Available Fields

This list displays all of the fields available for use in your report. The list is based on the tables you selected on the [Data screen \(Report Creation Wizard\) \[page 1385\]](#).

Browse Data

Click this button to browse the data of the field that is selected in the Available Fields list.

Find Field

Click this button to enter the name of a field you'd like to locate in the Available Fields list.

Add or remove arrow buttons

Use the > arrow adjacent to the Fields to Display list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added. Double arrows add or remove all fields.

Fields to Display

This list displays all of the report fields presently active in the report.

Up or down arrow buttons

Use the up and down arrows adjacent to the Fields to Display list to change the order of fields you've added from the Available Fields list.

39.2.5.6 Grid Style screen (Report Creation Wizard)

Use the Grid Style screen to choose a predefined style for your cross-tab.

Available Styles

This box displays the formatting styles available for use with your cross-tab. Select the desired style from the list.

Sample Preview

This box displays a sample of the style selected in the Available Styles list.

39.2.5.7 Group Sorting screen (Report Creation Wizard)

Use the Group Sorting screen to sort groups you created on the [Grouping screen \(Report Creation Wizard\) \[page 1390\]](#). You can sort all groups, or you can choose the top or bottom 5 groups. When sorting by the top or bottom 5 groups, you can also choose the summarized field you want to base the sorting on.

This screen appears only if you've specified a group on the Grouping screen and a summary on the [Summaries screen \(Report Creation Wizard\) \[page 1397\]](#). Creating a group sort is an optional step in the wizard.

Group that will be sorted

Select the group to be sorted from a list of groups you created on the [Grouping screen \(Report Creation Wizard\) \[page 1390\]](#).

Group ordering options

Select a group sorting option:

- All groups
- Top 5 groups
- Bottom 5 groups

If you choose Top 5 or Bottom 5, your report displays data for only those groups; no other groups appear on your report.

Comparing summary values

If you've chosen Top 5 or Bottom 5 as your group ordering option, you must select a summarized field to base the sorting on. The choices that appear in this list are the summaries you created on the [Summaries screen \(Report Creation Wizard\) \[page 1397\]](#).

Related Information

[Sorting, Grouping, and Totaling \[page 167\]](#)

39.2.5.8 Grouping screen (Report Creation Wizard)

Use the Group screen to specify how fields will be grouped on the report. Creating groups is an optional step in the wizard.

Available Fields

This list displays all of the fields available for use in your report. The list is based on the fields you chose on the [Fields screen \(Report Creation Wizard\) \[page 1387\]](#) and the tables you selected on the [Data screen \(Report Creation Wizard\) \[page 1385\]](#).

Browse Data

Click this button to browse the data of the field that is selected in the Available Fields list.

Find Field

Click this button to enter the name of a field you'd like to locate in the Available Fields list.

Add or remove arrow buttons

Use the > arrow adjacent to the Group By list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added. Double arrows add or remove all fields.

Group By

This list displays all of the fields you've selected for grouping. As you add fields, they are added in a hierarchy. This hierarchy establishes the order the groups will appear in the report. You can change the order of the hierarchy by using the up and down arrow buttons.

You can also set the sorting order for each field you add to the Group By list. Select a sort order from the adjacent list:

- Click in ascending order to sort the values for the field selected in the Group By list in ascending (A to Z, 1 to 9) order.
- Click in descending order to sort the values for the field selected in the Group By list in descending (Z to A, 9 to 1) order.

Up or down arrow buttons

Use the up and down arrows adjacent to the Group By list to change the order of fields you've added from the Available Fields list.

39.2.5.9 Label screen (Report Creation Wizard)

Use the Label screen to set up mailing label options. The Label screen lets you to print data on virtually any labels that are commercially available for line printers (dot matrix) or page printers (laser, ink jet). Use the settings and formatting commands to fine-tune your printing so the labels come out just the way you want them.

Mailing Label Type

You can define your own label type by selecting User-Defined Label from this list. However, to make your work easier, the program comes with preset templates for many other label types. Those labels are listed in the Mailing Label Type list.

When you choose a preset label template, the program automatically sets up the specifications for that label.

Label Size

These edit boxes enable you to specify the dimensions (height and width) of one label. This information can usually be found on the box or folder the labels come in. Simply enter the height and width values in the respective edit boxes.

Note

The unit of measurement used in the dialog box (inches or centimeters) is based on settings in the Regional Options of the Windows Control Panel.

Gap Between Labels

These edit boxes enable you to specify the empty area (gap, gutter, and so on) between labels. Simply measure the gaps and then enter the horizontal and vertical values in the respective edit boxes.

- **Horizontal**
The gaps between labels going across the page. One-up (one across) labels don't have any horizontal gaps.
- **Vertical**
The gaps between labels going down the page.

Note

The unit of measurement used in the dialog box (inches or centimeters) is based on settings in the Regional Options of the Windows Control Panel.

Page Margins

These edit boxes enable you to set the top, bottom, left, and right margins of your page. Simply measure the margins and enter the values in the respective edit boxes.

Note

All margins are figured from the edge of the paper. For example, a setting of .5 inches sets a half-inch margin.

Note

You cannot use page margin settings to set the paper size; you must set the paper size via the Paper Size list in the [Page Setup dialog box \[page 1337\]](#).

Note

The unit of measurement used in the dialog box (inches or centimeters) is based on settings in the Regional Options of the Windows Control Panel.

Printing Direction

The Printing Direction section enables you to specify the path that the program follows when printing the data on your labels.

- [Across then Down](#)
Prints data left to right across the first row of labels, then across the second row, and so on.
- [Down then Across](#)
Prints data down the first column of labels, then down the second column, and so on.

Number of Labels

This information box displays the maximum number of labels that can print across the page and the maximum number that can print down the page based on your settings in the Page Margins, Mailing Label Size, and Gap Between Labels edit boxes and paper size (which is set up in Windows Control Panel).

Note

The program displays only the number of complete labels it can print. For example, if it determines that there is space available to print 2.75 labels across the page, it displays 2 as the Across Page setting and prints only two labels across the page.

Note

If you want to print fewer labels across than the number calculated (fewer columns), increase the size of the right margin, left margin, or horizontal gap until the calculation changes to the number across that you want.

39.2.5.10 Link screen (Report Creation Wizard)

The Link screen appears in the Report Creation Wizard when you have selected two or more tables on the [Data screen \(Report Creation Wizard\)](#) [page 1385].

Note

The Link screen is not used with the OLAP Report Creation Expert.

Use the Link screen to link tables in your new report.

The Link screen has the same functionality as the [Links tab \(Database Expert\)](#) [page 1422].

39.2.5.11 Record Selection screen (Report Creation Wizard)

Use the Record Selection screen to choose fields to select (or filter) records in a report. By selecting records in this manner, you can narrow the scope of your report and improve processing speed. Creating a record selection is an optional step in the wizard.

Available Fields

This list displays the report fields currently available for selection in your report.

Filter Fields

This list displays the fields currently selected as filter for your report.

Add or remove arrow buttons

Use the > arrow adjacent to the Filter Fields list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added.

Selection options

A list of Selection options appears under the Filter Fields list when you add a field. The options in the list are dynamic and depend on the field you've chosen.

Use the first list box and the others that might appear to select the condition that best finishes this sentence:

```
"I want to select all records where the value in the field I have selected is..."
```

Using the selection options, you might create a record selection such as:

```
"I want to select all records where the value of the Sales field is between  
10,000 and 20,000."
```

Related Information

[Select Expert \[page 1457\]](#)

39.2.5.12 Rows/Columns screen (OLAP Report Creation Wizard and OLAP Expert)

The Rows/Columns screen is common to the OLAP Report Creation Wizard and the OLAP Expert.

The Rows/Columns screen allows you to choose dimensions and members for display in the grid.

Dimensions

Use this list to select dimensions for the object.

Columns

This list displays the column dimensions.

Select Column Members

Use this button to select the members you want displayed for a column dimension. When you click this button, the [Member Selector dialog box \[page 1307\]](#) appears.

Rows

This list displays the row dimensions.

Select Row Members

Use this button to select the members you want displayed for a row dimension. When you click this button, the [Member Selector dialog box \[page 1307\]](#) appears.

Add or remove arrow buttons

Use the > arrow adjacent to the Columns and Rows lists to add dimensions from the Dimensions list. Use the < arrows to remove dimensions you've already added.

Order arrow buttons

Click the up and down arrow buttons to change the order of the dimensions in the Columns and Rows lists.

Create/Edit Parameter

Select a column or row dimension and click this button to see the [Create Parameter Field dialog box \[page 1184\]](#). Use the Create Parameter Field dialog box to create parameter fields for the selected dimension in either the Columns or Rows lists.

Delete

Click this button to delete the parameter selected in either the Columns or Rows lists.

39.2.5.13 Slice/Page screen (OLAP Report Creation Wizard and OLAP Expert)

The Slice/Page screen is common to the OLAP Report Creation Wizard and the OLAP Expert.

The Slice/Page screen allows you to choose what to do with other dimensions that are not rows/columns in the grid (the list displays all of the dimensions you did not use in the Rows/Columns screen). The Slice/Page screen lets you specify how dimensions that do not appear in the grid will affect the data for the dimensions that do appear in the grid.

Slice

Choose one member for each of the dimensions by double-clicking the dimension and choosing the member from the [Member Selector dialog box \[page 1307\]](#). The dimensions in the grid will be set to the member you selected.

Add or remove arrow buttons

Use the > arrow to add dimensions from the Slice list to the Page list. Use the < arrow to remove dimensions you've already added. When you are moving dimensions to the Page list, the [Member Selector dialog box \[page 1307\]](#) appears so you can select multiple members.

Order arrow buttons

Click the up and down arrow buttons to change the order of the Page dimensions.

Page

Choose multiple members from each of the dimensions by double-clicking the dimensions and choosing members from the [Member Selector dialog box \[page 1307\]](#). A separate grid will be created for each member.

Select Slice

Click this button to open the [Member Selector dialog box \[page 1307\]](#) for a dimension you have highlighted in the Slice list.

Select Page Values

Click this button to open the [Member Selector dialog box \[page 1307\]](#) for a dimension you have highlighted in the Page list.

Create/Edit

Click this button to see the [Create Parameter Field dialog box \[page 1184\]](#). Click the appropriate button and use the Create Parameter Field dialog box to create parameter fields for either the Slice or Page lists.

Delete

Click this button to delete the parameter selected in either the Slice or Page lists.

39.2.5.14 Style screen (OLAP Report Creation Wizard)

Use this screen to apply a predefined style to your OLAP grid.

Style

This box displays the formatting styles available for use in your report. Select the desired style from the list.

Sample Preview

This box displays a sample of the style selected in the Style list.

39.2.5.15 Summaries screen (Report Creation Wizard)

Use the Summaries screen to choose fields to be subtotals, counted, and so on. This screen appears only if you've specified a group on the [Grouping screen \(Report Creation Wizard\) \[page 1390\]](#). Creating a summary field is an optional step in the wizard.

Available Fields

This list displays the report fields available for totaling in the report.

Summarized Fields

This list displays the total fields currently in use in the report.

Add or remove arrow buttons

Use the > arrow adjacent to the Summarized Fields list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added. Double arrows add or remove all fields.

Summary options

Click a summary option for the field selected in the Summarized list. You can choose options to total a field, or to count its entries, and so on.

Related Information

[Summary \[page 1551\]](#)

[Sorting, Grouping, and Totaling \[page 167\]](#)

39.2.5.16 Template screen (Report Creation Wizard)

Use the Template screen to choose from predefined formatting templates for use in your report. Adding a template is an optional step in the wizard.

Available Templates

This area displays the templates available for use in your report. Select the desired template from the list.

Browse

Click this button to use the [Open dialog box \[page 1314\]](#) to search for a template.

Preview

This box displays a sample of the template selected in the Available Templates area.

Related Information

[Using a template \[page 236\]](#)

[Template considerations \[page 239\]](#)

39.3 SAP BusinessObjects Registration Wizard

The Registration Wizard appears when you start Crystal Reports, or when you choose the [Register or Change Address command \[page 1130\]](#) from the Help menu. Use the Registration Wizard to register your copy of Crystal Reports.

Once you have registered your copy of the program with SAP BusinessObjects, you will receive a registration number. Once you have entered the serial number, the Registration Wizard will no longer appear on startup.

Registration assures you that you will be notified whenever the product is upgraded to offer new features, benefits, and efficiencies.

Related Information

[Register or Change Address command \[page 1130\]](#)

39.4 Calculation Expert

The Calculation Expert assists in creating calculated members that you can add to rows or columns in an OLAP grid. The expert is available as a tab in the [Calculated Members dialog box \[page 1158\]](#).

For details on calculated members, see [Adding calculated members to the Worksheet \[page 451\]](#).

39.5 Chart Expert

The Chart Expert appears when you take any one of the following actions:

- You click the Insert Chart button on the [Insert Tools toolbar \[page 1148\]](#) and drop the placement rectangle where you want the chart to appear.

Note

If your report already contains the correct grouping and summary information, Crystal Reports creates a chart without showing you the Chart Expert. For more information about this option, see [Charting \[page 303\]](#).

- You choose the [Chart command \[page 1099\]](#) from the Insert menu and drop the placement rectangle where you want the chart to appear. (The previous note applies in this case as well.)
- When you right-click an existing chart and choose Chart Expert.

Use the Chart Expert to prepare graphs for use in a report. When the Chart Expert appears, Type is usually the active tab.

Related Information

[Editing charts using the Chart Expert \[page 314\]](#)

39.5.1 Type tab (Chart Expert)

The Type tab offers a choice of many different chart types to insert in a report. Select the type of chart you want, and the program begins the construction of a chart of that type which you can further customize.

Choose your chart type

Use this list to select the chart type you want to use. Once a chart type is selected, you can use the icons to the right of the list to select the chart type format to use for the chart. For example, if you select the Pie Chart as the chart type, you can then select from among the following formats: "Pie chart," "Multiple pie chart," and "Proportional multiple pie chart."

Additional options

Certain chart types offer additional options, which are found under the chart-type description area:

- Vertical
- Horizontal
- Use depth effect

Use the Vertical and Horizontal options to specify whether the chart is to represent increasing values in a vertical or a horizontal direction. For example, if you create a Vertical bar chart, the bars will rise upward from the bottom of the chart. If you create a Horizontal bar chart, the bars will rise from left to right, from the side of the chart (by default).

Use the "Use depth effect" option to add a 3D effect to your chart.

39.5.2 Data tab (Chart Expert)

The Data tab contains these sections:

- Layout
- Data

In the Layout section, you determine the actual layout of the chart, depending on the data you wish to chart.

39.5.2.1 Advanced option

Available fields

This list displays all of the fields available for use in your chart.

Browse

Click this button to browse the values for the field that is selected in the Available fields list.

On change of

Use the options in the "On change of" list to determine what values you want to see in your chart:

- Choose "On change of" to see values summarized by the field you choose for the area below the list. For example, if you choose "Country," you'll see summarized values for each country.
- Choose "For each record" to see values for each record in your report.

Note

For this option, you don't choose a field for the area below the list.

- Choose "All records" to see a summarized value that is a grand total for all the records in your report.

Note

For this option, you don't choose a field for the area below the list.

If it's necessary to choose a field for the area below the "On change of" list, use the arrow buttons to add or remove the field(s) selected in the Available fields list.

Order

Click the Order button to change the sort order for the selected field. When you click this button, the [Chart Sort Order dialog box \[page 1166\]](#) appears.

This button is active only when a single field in the "On change of" list is selected.

Top N

Click the Top N button to sort the selected field by All, Top N, Bottom N, Top Percentage, or Bottom Percentage. When you click this button, the [Group Sort Expert \[page 1429\]](#) appears.

This button is active only when a single field in the "On change of" list is selected, and you have already added a value field to the "Show value(s)" list.

Show value(s)

This list displays the field chosen as a value for your chart. Points will be plotted for these values on the chart whenever the condition selected in the "On Change of" list changes. For example, if you create a chart of last year's sales for all your customers, you will select the Customer Name field as the condition and the Last Year's Sales field as the value. Whenever the Customer Name changes a point will be plotted according to the value of their last year's sales.

Use the arrow buttons to add or remove the field(s) selected in the Show value(s) list. Click the double-arrow button to remove all the fields from the Show value(s) list.

Don't summarize

Use the "Don't summarize" option to determine if the program should automatically summarize the values that will appear on the chart.

Note

This option is available only for formula fields.

- If you select this check box, the program will not summarize the values.
- If you clear this check box, the program will summarize the values, and the Set Summary Operation button will become available. The program adds a summary field to the Show value(s) list.

Set Summary Operation

Use this button to set or change the summary operation for the selected field. When you click this button, the [Edit Summary dialog box \[page 1228\]](#) appears. This button is only active when you have cleared the Don't summarize values check box, and when you have selected a field in the Map Values list.

Related Information

[Editing charts using the Chart Expert \[page 314\]](#)

39.5.2.2 Group option

On change of

Select the field that you want to use as a condition for plotting values in a chart. Points will be plotted whenever the value in the condition field changes. For example, if you create a chart of last year's sales for all your customers, you will select the Customer Name field as the condition and the Last Year's Sales field as the value. Whenever the Customer Name changes a point will be plotted according to the value of their last year's sales.

Show

Use this list to select the summary field whose values you want to display in your graph.

39.5.2.3 Cross-Tab option

On change of

Select the field that you want to use as a condition for plotting values in a chart. Points will be plotted whenever the value in the condition field changes. For example, if you create a chart of last year's sales for all your customers, you will select the Customer Name field as the condition and the Last Year's Sales field as the value. Whenever the Customer Name changes a point will be plotted according to the value of their last year's sales.

Subdivided by

Use this list to choose the secondary row or column to base your chart on. Clicking this list displays the row/columns fields in your report that you can choose from.

Show

Use this list to select the summary field whose values you want to display in your chart.

39.5.2.4 OLAP option

Chart off entire grid

Select the option to chart on all grid column and row fields. When you select this option, all other fields in the area are unavailable. Charting off an entire grid is useful when the grid has asymmetric filters based on members from two or more dimensions.

On change of

Select the dimension that you want to use as a condition for plotting values in a chart. Points will be plotted whenever the value in the condition field changes.

Subdivided by

Use this list to choose the secondary row or column to base your chart on. Clicking this box displays a list of the row/columns fields in your report that you can choose from.

Other Dimensions

Click the Other Dimensions button to call the [Format Other Dimensions dialog box \[page 1279\]](#).

Click a dimension (shown with its field setting) in the list, and then click "Select a member value" to specify a new field for the dimension to be fixed to.

39.5.3 Color Highlight tab (Chart Expert)

The Color Highlight tab lets you conditionally apply color based on chart value fields. You apply a color to the selected item by specifying a condition.

The tab is divided into two areas: the Item list area displays the formula conditions; the Item editor area lets you create the formula conditions.

- If your chart type is line, the chart must have data markers before you can see conditional formatting.
- An area chart must have two "On change of" values for conditional formatting to appear.

39.5.3.1 Item list

The Item list displays the color highlighting conditions created for the currently selected chart.

Up and down arrows

Use the up and down arrows above the Item list area to set the priority order for your formulas.

New

Click the New button to create a new formula with the default settings.

Remove

Use the Remove button to discard unwanted formulas.

39.5.3.2 Item editor

Value of

Use this list to select the field upon which the condition is based. That is, choose a field for X in the following phrase: "If the value of field X meets condition A, then apply the selected color."

Comparison list

From this second list, select the comparison for your conditional formula. This comparative phrase works as the operator in your conditional formula.

Field values

Use this box to enter the value to which you want to apply color formatting. For example, to apply the color red when the value is equal to \$25,000, type 25,000 in this box.

Note

If the field selected in the "Value of" list is nonnumeric, then you can select from a list of available values.

Format color

Use this list to select a color for the field value to which you want to apply conditional color formatting. You can define custom colors, which the program saves for new sessions of Crystal Reports, and which you can use in report objects other than charts.

Related Information

[Conditionally formatting a chart \[page 318\]](#)

39.5.4 Axes tab (Chart Expert)

The Axes tab lets you set the properties for each of the chart axes. The Axes tab changes slightly depending upon the type of chart you are creating.

Note

There is no Axes tab for Pie, Doughnut, Gantt, or Funnel charts.

Major

Use the Major check boxes to determine whether major guidelines will be shown for the chart axes. Depending on the type of chart you are creating, there may be a Major check box for the Group, Series, Data, and/or Data2 axes.

- Select a Major check box to show the guideline for the corresponding axis.
- Clear a Major check box to hide the guideline for the corresponding axis.

Minor

Use the Minor check boxes to determine whether minor guidelines will be shown for the chart axes. Depending on the type of chart you are creating, there may be a Minor check box for the Group, Series, Data, and/or Data2 axes.

- Select a Minor check box to show the guideline for the corresponding axis.
- Clear a Minor check box to hide the guideline for the corresponding axis.

Auto Scale

When you select the Auto Scale check box, the number of labels is adjusted to ensure a clear display along the Data axis.

Auto Range

When you select the Auto Range check box, the data values for the corresponding axis are assigned a default range.

Min

Use the Min text box to enter the range minimum for the data values on the corresponding Data axis. There is a Min text box for the Data axis, and there may also be a Min text box for the Data2 axis. The second Min text box is only active if the chart is a cross-tab or OLAP chart.

If the corresponding Auto Range check box is selected, the Min text box is inactive.

Max

Use the Max text box to enter the range maximum for the data values on the corresponding Data axis. There is a Max text box for the Data axis, and there may also be a Max text box for the Data2 axis. The second Max text box is only active if the chart is a cross-tab or OLAP chart.

If the corresponding Auto Range check box is selected, the Max text box is inactive.

Number Format

Use the Number Format list to select a number format for the data values on the corresponding Data axis. There is a Number Format list for the Data axis (y), and there may also be a list for the Data2 axis. The second Number Format list is only active if the chart is a cross-tab or OLAP chart.

If the corresponding Auto Range check box is selected, the Number Format list is inactive.

Automatic

If you select the Automatic option, the program automatically sets the number of divisions for the corresponding Data axis. There is an Automatic option for the Data axis, and there may also be an Automatic option for the Data2 axis. The second Automatic option is only active if the chart is a cross-tab or OLAP chart.

Manual

Use the Manual option to manually set the number of divisions for the corresponding Data axis. There is a Manual option for the Data axis, and there may also be a Manual option for the Data2 axis. The second Manual option is only active if the chart is a cross-tab or OLAP chart.

When you click the Manual option button, the adjacent text box becomes active. Use the text box to enter the number of divisions.

39.5.5 Options tab (Chart Expert)

The Options tab offers choices for displaying a chart. The Options tab changes slightly depending upon the type of chart you are creating.

Chart color

Use the Chart Color option buttons to specify whether the chart will be color or black and white.

Data points

Use the Data Points option buttons to specify the type of label that will appear for each data item (that is, for each chart value). You can click Show label or Show value. If you click Show value, the Number format list becomes active. Use the Number format list to select a format for your chart values. For example, you might select a format of 1 (numeral) or you might select a format of \$1.00 (currency).

Transparent background

When this check box is selected, the chart background will become transparent so objects positioned behind it are visible.

Marker size

Use this list to specify the marker size for the chart value markers.

Marker shape

Use this list to specify the marker shape for the chart value markers.

Viewing angle

Use this list to specify the angle from which the chart is viewed (Standard Angle, From The Top, and so on). This option is available for 3D charts.

Pie size

Use this list to select the pie size for a Pie or Doughnut chart (Minimum, Small, Average, Large, or Maximum).

Bar size

Use this list to select a bar size for a Bar chart (Minimum, Small, Average, Large, or Maximum).

Detach pie slice

When this check box is selected, a pie slice is detached from a Pie or Doughnut chart. You can click Smallest slice or Largest slice to specify which pie slice to pull out.

Show legend

When this check box is selected, a legend is displayed for your chart. When the check mark is not visible, the option is not selected, and no legend will be displayed for your chart. By default, the option is selected.

Placement

Use the Placement list to specify where you want the chart legend to appear (Right, Left, or Bottom).

This list is only active when the Show Legend check box is selected.

Layout

Use this list to display percentage values, amount values, amount and percentage values or no values.

Related Information

[Editing charts using the Chart Expert \[page 314\]](#)

39.5.6 Text tab (Chart Expert)

The Text tab lets you specify titles, footnotes and axis titles for a chart.

Auto-Text

When these check boxes are selected, chart titles are generated automatically. If they are cleared you can specify titles for the chart manually.

Title

Use this text box to enter a title for the chart. The title will appear just above the chart.

Subtitle

Use this text box to enter a subtitle for the chart. The subtitle will appear just below the title, above the chart.

Footnote

Use this text box to enter text for a footnote. The footnote will appear just below the chart on your report.

Group Title (x-axis)

Use this text box to enter a title for the chart's x axis.

Data Title (y-axis)

Use this text box to enter a title for the chart's y axis.

Data2 Title (y2-axis)

Use this text box to enter a title for the chart's second y axis (if you are creating a cross-tab or OLAP chart).

Series Title

Use this text box to enter a title for the chart series.

Staggered text

Use this check box to stagger the text in your labels.

This check box is only active when you have Group labels, Data labels or Series labels selected in the Format list.

Font

Click the Font button to set the font options for the set of text selected in the Format list. When you click this button, the [Font dialog box \[page 1254\]](#) appears.

Format

Use this list to select the text for which you are setting the font options. For example, you could select the chart title, the chart subtitle, the chart legend, and so on.

Related Information

[Editing charts using the Chart Expert \[page 314\]](#)

39.6 Cross-Tab Expert

This expert appears when you choose the [Cross-Tab command \[page 1096\]](#) from the Insert menu, or when you select an existing cross-tab and choose the Cross-Tab Expert command (see the [Use Expert command \[page 1103\]](#)) from the Format or right-click menus.

Use the Cross-Tab Expert to create a cross-tab quickly and easily. Cross-tabs are used in reports to present summarized data in a row/column structure, similar to that of a spreadsheet. These reports are easy to read, and they are particularly useful for making rapid comparisons and identifying trends.

Note

In order to print a Cross-Tab report, you must enter at least one summarized field in the Cross-Tab Expert. That is the only field required.

Related Information

[Cross-Tab command \[page 1096\]](#)

[What is a Cross-Tab object? \[page 346\]](#)

39.6.1 Cross-Tab tab (Cross-Tab Expert)

The Cross-Tab tab contains the basic functionality you need to create a cross-tab. You can define rows, columns, and summarized fields. As well, you can create groups and formulas directly from this tab.

Available Fields

This list displays all the tables and fields that you have activated for use in the report.

Select the field you want to use and drag it into the appropriate list, or click the appropriate Add button. Repeat this process to add additional fields.

Browse Data

Click the Browse Data button to view the values for a field you have selected in the Available Fields list.

Find Field

Click this button to enter the name of a field you'd like to locate in the Available Fields list.

New Formula

When you click this button, the [Formula Name dialog box \[page 1282\]](#) appears. Enter a name for your new formula and then click OK. The [Formula Workshop dialog box \[page 1282\]](#) appears with the Formula Editor active so you can create the new formula.

Edit Formula

When you click this button, the [Formula Workshop dialog box \[page 1282\]](#) appears with the Formula Editor active. Use the editor to edit the selected formula. Edit Formula is available only when a formula field is selected in one of the lists.

Columns

This list displays the fields that you have selected as column headings for your cross-tab. If you have multiple column headings, they will appear on your report in the order in which they appear on this list.

To add a column heading to this list, select a field from the Available Fields list and then drag it onto the list, or click the > arrow adjacent to the Columns list.

Rows

This list displays the fields that you have selected as row headings for your cross-tab. If you have multiple row headings, they will appear on your report in the order they appear on this list.

To add a row heading to this list, select a field from the Available Fields list and then drag it onto the list, or click the > arrow adjacent to the Rows list.

Summarized Fields

This list displays the fields that you have selected as summarized field headings for your cross-tab. If you have multiple summarized field headings, they will appear on your report in the order they appear on this list.

To add a summarized field heading to this list, select a field from the Available Fields list, then drag it onto the list, or click the Add Summarized Field button.

Add or remove arrow buttons

Use the > arrows adjacent to each list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added.

Group Options

When you click this button, the [Cross-Tab Group Options dialog box \[page 1189\]](#) appears. Use this dialog box to enter a group for your cross-tab. Group Options is available only for rows and columns.

Change Summary

When you click this button, the [Edit Summary dialog box \[page 1228\]](#) appears. Use this dialog box to change the summary type of the cross-tab's summarized fields. Change Summary is available only for summarized fields.

Related Information

[Formula Editor dialog box \[page 1280\]](#)

[Group Expert \[page 1428\]](#)

[Edit Summary dialog box \[page 1228\]](#)

39.6.2 Style tab (various experts)

This tab can appear as part of several experts:

- [Cross-Tab Expert \[page 1412\]](#)
- [OLAP Expert \[page 1442\]](#)

The Style tab allows you to choose a predefined style for your cross-tab or OLAP grid. An example of each style appears on the right side of the screen as you highlight its name.

Note

The color of the style you select may not appear correctly if your screen resolution is set to 256 colors. Increase the resolution to correct this.

If you select any of the predefined grid styles, the settings for it appear on the Customize Style tab. You can create a custom style based on any of the predefined styles by choosing it and modifying the settings on the Customize Style tab. Once you have created a custom style, a new option, called Custom, is added to the list on the Style tab.

39.6.3 Customize Style tab (Cross-Tab Expert)

Use the Customize Style tab to format your cross-tab's grid with background color, gridlines, and cell margins.

Columns

This list displays the columns you selected for your cross-tab. Select the field for which you want to customize the style.

Rows

This list displays the rows you selected for your cross-tab. Select the field for which you want to customize the style.

Summarized Fields

Use the options in this area to determine how multiple summarized fields are to be displayed.

Vertical

Choose this option to display multiple summarized fields in a vertical list:

Summary 1
Summary 2
Summary 3

Horizontal

Choose this option to display multiple summarized fields in a horizontal list:

Summary 1	Summary 2	Summary 3
-----------	-----------	-----------

Show Labels

Choose this option to display the names of multiple summarized fields. Labels can be displayed for both vertical and horizontal summaries. These labels can also be modified by editing them as you would any text field.

	AL		AR		AZ	
	Sales	Discount	Sales	Discount	Sales	Discount
USA	\$74,119.16	\$59,295.33	\$5,879.70	\$4,703.76	\$725.40	\$580.32
Total	\$74,119.16	\$59,295.33	\$5,879.70	\$4,703.76	\$725.40	\$580.32

Suppress Subtotal

Use this check box to suppress the subtotal for a selected field. You must first have a field selected in either the Rows list or the Columns list.

Suppress Label

Use this check box to suppress the label of a suppressed subtotal field. This check box is inactive if the Suppress Subtotal check box is not selected.

Alias for Formulas

Enter an alias name for the selected row or column field. Creating an alias shortens long field names, which is useful later when you refer to a field in a conditional formatting formulas. The field's original name is shown in this box by default.

Background Color

Use this list to select a color for the selected row or column field.

Indent Row Labels

Select this check box to indent row labels on your cross-tab. Use the adjacent box to specify the size of the indent you want to see.

Indent Column Labels

Select this check box to indent column labels on your cross-tab. Use the adjacent box to specify the size of the indent you want to see.

Repeat Row Labels

Select this check box to have the program repeat row labels on every page for cross-tabs that print across more than one page.

Keep Columns Together

Select this check box to have the program keep columns together on every page for cross-tabs that print across more than one page.

Column Totals on Top

Select this check box if you want column totals to appear at the top of the cross-tab rather than at the bottom (the default position).

Row Totals on Left

Select this check box if you want row totals to appear on the left of the cross-tab rather than on the right (the default position).

Show Cell Margins

Select this check box to have the program automatically format the cells of the cross-tab. When this check box is not selected, any extra space (white space) within the cells of the cross-tab will be eliminated and the contents of the cells will be adjusted to utilize the least amount of space in your report as possible.

Suppress Empty Rows

Select this check box to have the program remove empty rows from the cross-tab. When this check box is selected, any rows whose fields do not contain any records will be hidden.

Suppress Empty Columns

Select this check box to have the program remove empty columns from the cross-tab. When this check box is selected, any columns whose fields do not contain any records will be hidden.

Suppress Row Grand Totals

Select this check box to have the program remove the grand total from rows in the cross-tab.

Suppress Column Grand Totals

Select this check box to have the program remove the grand total from columns in the cross-tab.

Format Grid Lines

When you click this button, the [Format Grid Lines dialog box \[page 1278\]](#) appears. In this dialog box, you can define whether the grid lines are shown or not, as well as the color, style, and width you'd like the grid lines to have.

39.7 Cross-Tab Calculated Members Expert

The *Cross-Tab Calculated Members Expert* appears when you choose the *Calculated Member* command.

Use the *Cross-Tab Calculated Members Expert* to add, remove, and modify *Calculated Members*. You can also use this Expert to specify the order in which *Calculated Members* are processed. The order can have an important effect on the results of your formulas.

New

Click this button to create a new *Calculated Member*.

Remove

Click this button to remove an existing *Calculated Member*.

Edit Insertion Formula

Click this button to edit the formula that determines where a *Calculated Member* appears in a Cross-Tab. By default, a *Calculated Member* appears only once, but you can set it to appear in multiple locations or a repeating pattern.

Edit Group Value Formula

Click this button to edit the value of a *Calculated Member*. You can use this value to reference the *Calculated Member* in other formulas.

Edit Header Formula

Click this button to edit the visible header of the *Calculated Member*.

Properties

Use this section to modify the following properties of the *Calculated Member*:

- *Description*
Enter a unique description to identify the *Calculated Member*.
- *Type*
Select *Row* or *Column*. This determines whether the *Calculated Member* will be inserted as a row or as a column.
- *Insert Evaluation*
Set the insertion position to *Before* or *After*. This determines whether the new row or column will be inserted before or after the one being evaluated.
- *Group level*
Allows you to specify the group level of the *Calculated Member*.

Edit Value Formula

Click this button to edit the formula that determines the cell values of the *Calculated Member*.

39.8 Data Analysis Expert

The Data Analysis Expert assists in creating three types of predefined data analysis calculations:

- Trend Line
- Moving Average
- Linear Regression (X and Y)

The expert is available as a tab in the *Calculated Members dialog box* [page 1158].

For details on data analysis calculations, see *Adding data analysis calculations* [page 456].

39.9 Database Expert

The Database Expert provides an integrated tree view of all data sources you can use with Crystal Reports. With the Database Expert, you can:

- Create a report using one of the currently connected data sources.
- Create a report based on a Business View or SQL command that's stored in the SAP BusinessObjects Enterprise Repository.
- Create a report based on a customized "Favorites" list of data sources (which you also maintain within the Database Expert).
- Create a report based on a list of recently accessed data sources (that the Database Expert automatically maintains for you).
- Create a report by connecting to an existing data source (for example, a data file residing locally, or an ODBC data source that has already been set up).
- Create a report by first creating a new connection (for example, a new ODBC data source, or an OLE DB File Data Link).
- View information about a database connection by right-clicking its icon and selecting Properties from the shortcut menu.

39.9.1 Data tab (Database Expert)

Use the Data tab to browse for the data source you want to add to your report.

Available Data Sources

Available data sources are shown in a tree view. The tree is made up of folders for:

- [My Connections](#)
This folder shows a list of data sources you are currently connected to.
- [Create New Connection](#)
This folder shows subfolders for various data sources you can connect to. For more information about the options in this list, see the related topics at the bottom of this page. You can also search the online help for topics related to the specific type of connection that you want to make.
The data source options available in the Create New Connection folder depend on the data access components selected during installation.
- You can right-click any item in the Database Expert to see a shortcut menu.

Add or remove arrow buttons

Use the > arrow adjacent to the Selected Tables list to add tables from the Available Data Sources list. Use the < arrows to remove tables you've already added. Double arrows add or remove all tables.

Selected Tables

This list shows the tables you've selected for your report. If more than one table has been added, the [Links tab \(Database Expert\)](#) [page 1422] is added to the expert.

Note

For data sources that require you to enter additional information, see [Data source connectivity dialog boxes](#) [page 1206].

39.9.2 Links tab (Database Expert)

Use the Links tab to link two or more database tables together. When you choose more than one table on the [Data tab \(Database Expert\)](#) [page 1421], the Links tab appears.

You link tables so records from one table will match related records from another. For example, if you activate a Movie table and a Studio table, you link the tables so that each movie (from the Movies table) can be matched up with the studio that made the movie (from the Studio table).

When you link, a field is used that is common to both database tables. The program uses the link to match up records from one table with those from the other. In this example, the link assures that the data in each row of the report refers to the same movie.

Links display area

The display area shows all the tables currently in your report along with links and indexes.

→ Tip

A key to the index indicators is displayed when you click the Index Legend button.

Auto-Arrange

When you click this button, the program arranges the tables to fit in the most efficient manner inside the Links display area.

Auto-Link Tables

Use this area to choose a method of linking tables. Click By Name to link tables by their names or By Key to use foreign key information. Click the Link button to link tables automatically (by name or foreign key, as selected).

Order Links

To activate this button, you must have more than two tables available for linking. When you click the button, the [Order Links dialog box \[page 1337\]](#) appears. Establish the link processing order you want for the available linked tables.

Clear Links

Click this button to clear all the current table links.

Delete Link

Click a link then click this button to remove it.

Link Options

To activate this button, you must have a link line selected. Once you have a link selected, click this button to open the [Link Options dialog box \[page 1301\]](#) where you can choose the join and link type for the selected link.

Index Legend

Click this button to open the [Index Legend dialog box \[page 1287\]](#) so you can see a key to the index indicators used in the Links display area.

Related Information

[Linking multiple tables \[page 99\]](#)

[Link relationships \[page 902\]](#)

[Linking indexed tables \[page 909\]](#)

[Link processing order \[page 910\]](#)

[Linking options \[page 911\]](#)

39.10 Flash Data Binding Expert

This Expert appears:

- When you choose the [Flash command \[page 1100\]](#) command from the Format menu.
- When you right-click a flash object and choose [Flash Data Expert](#).

The [Flash Data Binding Expert](#) consists of tabs that allow you to add fields to a flash object or define flash variables.

39.10.1 Xcelsius Data tab (Flash Data Binding Expert)

Available Fields

Available Fields are shown in a tree view. They can be dragged to the rows and columns of the cross-tab.

The shape of the rows and columns in the cross-tab area matches the shape of the bindings that were designed in Xcelsius 2008.

Data-binding options

The buttons are used to perform the following tasks:

-  X deletes a field from the row, column, or heading of the cross-tab.
-  The arrows change the order of the columns.
- [Options](#) allows you to change sort order or summary information for a column or row.
- [Group Sorting](#) displays the [Group Sort Expert](#) for the SWF data.

Use Existing Cross-Tab

The [Use Existing Cross-Tab](#) checkbox is available when you add an Xcelsius 2008 SWF to a report that contains a cross-tab.

Preview

The data that appears in the preview area is not the report data. The preview is provided to ensure that the integration between Crystal Reports data and the flash object is synchronized.

Description

The description area displays a description of how the SWF author should bind data to the SWF. This description is entered when the user creates a Crystal Reports data connection in Xcelsius 2008, and it provides hints on how to bind data properly. For example, it may mention what type of data to send (number, currency, date, and so on), or how many values are expected and how the values will be used within the SWF.

Related Information

[Group Sort Expert \[page 1429\]](#)

39.10.2 Flash Vars tab (Flash Data Binding Expert)

Variable names are extracted from a SWF only if the variables are defined as Crystal Reports data connections.

To add a new variable, either click the "+" button  or double-click in the list area.

To remove a variable, select it and click the X button .

You can change the value associated with a variable name by typing in a new value, selecting from the drop-down list, or creating a formula.

Preview

The data that appears in the preview area is not the report data. The preview is provided to ensure that the integration between Crystal Reports data and the flash object is synchronized.

Description

The description area displays a description of how the user should bind data to the SWF. This description is entered when the user creates a Crystal Reports data connection in Xcelsius 2008, and it provides hints on how to bind data properly. For example, it may mention what type of data to send (number, currency, date, and so on), or how many values are expected and how the values will be used within the SWF.

39.11 Formula Expert

Use the Formula Expert to create a Formula based on an existing custom function. The expert appears when you click Use Expert in the [Formula Name dialog box \[page 1282\]](#), or when you select a simple formula in the Formula Workshop without selecting the Toggle Properties Display button.

Note

The Formula Expert also appears when you edit an empty formula field or a formula field that makes a simple call to a custom function.

For information about how to create a formula in this dialog box, see [Calling custom functions \[page 737\]](#).

Custom Function Supplying Logic

This list includes all the custom functions available in your report (Report Custom Functions) or your repository (Repository Custom Functions). Select a custom function to base your new formula on. If you select a Repository Custom Function that does not exist in your report, that function can be added (along with any other functions it relies on) when you save your formula.

When you select a custom function, the Summary, Return Type, and Function Arguments areas are filled in based on the function's definition.

Summary

This area contains the summary that has been defined for the custom function in the [Custom Function Properties dialog box \[page 1190\]](#). The summary information is only for reference and cannot be modified.

Return Type

This area contains the data type that the selected custom function will return. The return type information is only for reference and cannot be modified.

Reset Values

Click this button to reset values you've entered in the Value fields of the Function Arguments area. If an argument has a set of default values, the value is reset to the first default value. If an argument has no default values assigned, the value is reset to "Select Field or Enter Value" (that is, the value is unbound). If an argument is optional, it is set to "Use Optional Value:" followed by the current value of the optional value defined in the custom function.

More Info

Click this button to see more information about the selected custom function. The [Custom Function Properties dialog box \[page 1190\]](#) appears.

Function Arguments

This area contains the arguments defined for the custom function. Most of the argument information is only for reference and cannot be modified.

However, the Value field is where you specify the argument value for the formula you are creating. You can enter a value directly, or you can click the Value field and select from the list that appears. You can choose from possible values (the choices you see are specific to the argument's data type):

- A set of default values created in the [Custom Function Properties dialog box \[page 1190\]](#). You can save the formula without selecting a value; the first default value from the set is used.
- An optional value as specified in the custom function. This choice appears as the text "Use Option Value" followed by the current value of the optional value defined in the custom function. You can save the formula without selecting a value; the optional value is used.
- A field of the type that matches the selected argument's type (that is, if the argument's type is string, you can pick a string field). The field must be used in the current report for it to appear in this list.
- True or False for arguments with a Boolean data type.
- A formula field or SQL Expression whose return type matches the selected argument's type. The formula or expression must exist in the [Formula Workshop dialog box \[page 1282\]](#), but does not have to be used in the report for it to appear in this list.
- A running total whose summary field's type matches the selected argument's type. The running total must exist in the [Field Explorer dialog box \[page 1241\]](#), but does not have to be used in the report for it to appear in this list.
- A parameter field whose value type matches the selected argument's type. The parameter must exist in the [Field Explorer dialog box \[page 1241\]](#), but does not have to be used in the report for it to appear in this list.
- A group name. The group must exist in the current report for it to appear in this list.
- A constant value. For some types, such as string or number, you can enter a simple constant value directly. For others that require specific formatting, such as date, date and time, and so on, a version of the [Enter Value for dialog box \[page 1231\]](#) appears so you can choose a value or range.
- Another field. Click Choose Other Field to see the [Choose Field dialog box \[page 1167\]](#), which displays only the report and database fields of the type that matches the selected argument's type.

Use Editor

Click this button to see the [Formula Editor dialog box \[page 1280\]](#). The definition area of the [Formula Workshop dialog box \[page 1282\]](#) is changed to the Formula Editor so that you can create your formula in the editor instead of the expert.

Note

If Crystal Reports encounters an error in your formula after you save it, the Formula Editor is displayed so you can debug the formula. For more help, see [Debugging formulas \[page 708\]](#).

Save

Click this button to save changes you've made in the Formula Expert.

39.12 Group Expert

This expert appears when you choose the [Group Expert command \[page 1117\]](#) from the Report menu.

Use the Group Expert to specify how fields will be grouped on your report. You can also use the dialog box to reposition groups and to edit the criteria used for triggering and sorting groups.

Available Fields

This list displays all of the fields available for use in grouping. The list is based on the report fields you chose in the [Field Explorer dialog box \[page 1241\]](#) and the tables you selected on the [Data tab \(Database Expert\) \[page 1421\]](#).

Browse Data

Click this button to browse the data of the field that is selected in the Available Fields list.

Find Field

Click this button to enter the name of a field you'd like to locate in the Available Fields list.

Add or remove arrow buttons

Use the > arrow adjacent to the Group By list to add fields from the Available Fields list. Use the < arrows to remove fields you've already added. Double arrows add or remove all fields.

Group By

This list displays all of the fields you've selected for grouping. As you add fields, they are added in a hierarchy. This hierarchy establishes the order the groups will appear in the report.

When you enter a group, that group appears directly above and below the Details section. When you add a second group, it appears between the Details section and the first group. The third group is between the Details section and the second group, and so on. Thus, when you look at the Design tab, the Group Header for the first group is the top group header, and the Group Footer for the first group is the bottom group footer.

This Group By list reflects the group footer order of groups; the bottom group in the list is the first group, the group next to the bottom is the second group, and so forth.

When you select a group in this list, the program activates those buttons that apply to the selected group.

Up or down arrow buttons

Use the up and down arrows adjacent to the Group By list to change the order of fields you've added from the Available Fields list.

Options

When you click this button the [Change Group Options dialog box \[page 1162\]](#) appears. Use this dialog box to edit the criteria used for triggering and sorting groups.

Related Information

[Group Expert command \[page 1117\]](#)

39.13 Group Sort Expert

This Expert appears when you choose the [Group Sort Expert command \[page 1117\]](#) from the Report menu.

Note

A report must contain a summary value in order to be able to perform a Group Sort. See [Summarizing grouped data \[page 192\]](#).

Use the Group Sort Expert to identify top or bottom groups (the states with the most/fewest customers in a customer report, for example). You can also identify top or bottom percentages.

For this group sort

Use this list to choose one of the following sorting options for your group:

- No Sort
- All
- Top N
- Bottom N
- Top Percentage
- Bottom Percentage

Depending on which option you choose, the expert displays different options.

based on

This list displays a list of summary fields in your report. Select the field you want to select top or bottom values or percentages for from this list. When All is selected, use the scroll box beneath the "based on" list to see the fields chosen for sorting.

Where N is

Enter the number of top or bottom values (where N is the number of values) you want to select into this edit box.

Where Percentage is

Enter the percentage of top or bottom values you want to select into this edit box.

Include ties

Select this option to accommodate groups whose summarized values are equal.

Include Others, with the name

Select this option if you want to include all fields that do not fit into your top or bottom selection under a specified group name. Enter the group name you want all fields that do not fit into your selection to appear under.

Related Information

[Sorting, Grouping, and Totaling \[page 167\]](#)

[Selecting top or bottom N groups or percentages \[page 194\]](#)

39.14 Highlighting Expert

The Highlighting Expert enables you to apply conditional formatting to all types of report fields (Number, Currency, String, Boolean, Date, Time, and DateTime fields). With the expert, you format the selected field either by specifying a condition based on that field's value, or by specifying a condition based on the value of a different report field. In other words, the expert enables you create the following formula: If the value of field X meets condition A, then apply the specified formatting to the field selected on the report.

When used for conditional formatting, the Highlighting Expert allows you to:

- Modify several attributes at once, without writing a formula.
- Highlight all field types used in the report.
- Format font style, background color, font color, and border style.
- Format a field based on its own values or the values of another field.
- Highlight a cross-tab or OLAP cell based on row and column heading values.
- Enter values using your locale-specific number format (such as 1,224.23 for North American users).
- Enter dates numerically or textually (January 12, 2001, or Jan 12, 2001).
- Undo highlighting quickly.

The dialog box is divided into two areas: the Item list area displays the formula; the Item editor area allows you to specify the formula. The Item editor area includes a Sample field that displays the formatting specifications being applied.

Item list

The Item list displays the highlighting formulas created for the currently selected field.

Priority

Use the Priority buttons in the Item List list of the Highlighting Expert to set the priorities for your formulas.

New

Click this button to create a new formula with default settings.

Remove

Use the Remove button to discard unwanted highlighting formulas for the currently selected field.

Value of

Use this list to select the field upon which the condition is based. That is, choose a field for X in the following phrase: "If the value of field X meets condition A, then format the selected field."

Comparison list

From this second list, select the comparison for your conditional formula. This comparative phrase works as the operator in your conditional formula.

Field values

Use this text box to enter the value to which you want to apply highlighting conditions. For example, to apply a red background color when the value contained in a field is equal to \$25,000, type 25,000 into the Field values text box.

Note

If the field selected in the Value of list is nonnumeric, then you can select from a list of available values.

Font style

Use this list to select a font style for the field value to which you want to apply a highlighting condition.

Font color

Use this list to select a font color for the field value to which you want to apply a highlighting condition.

Background

Use this list to select a background color for the field value to which you want to apply a highlighting condition.

Border

Use this list to select a border style for the field value to which you want to apply a highlighting condition.

Sample

Shows a sample using the settings selected in the Expert.

Related Information

[Using the Highlighting Expert \[page 294\]](#)

[Conditionally formatting fields using the Highlighting Expert \[page 295\]](#)

[Setting highlighting priorities \[page 297\]](#)

39.15 Map Expert

This Expert appears:

- When you choose the [Map command \[page 1099\]](#) from the Insert menu.
- When you click the Insert Map button from the [Insert Tools toolbar \[page 1148\]](#).
- When you right-click a Map and choose Map Expert.

The Map Expert consists of several tabs that lead you through step-by-step instructions for creating a map in your report.

Related Information

[Creating maps \[page 324\]](#)

[Mapping concepts \[page 320\]](#)

39.15.1 Data tab (Map Expert)

The Data tab contains three sections:

- Placement
- Layout

- Data

In the Placement section, you determine where you would like your map to be placed on the report, as well as how often the map will appear on the report.

In the Layout section, you determine the actual layout of the map, depending on the data you wish to map.

In the Data section, you determine the data that will be mapped.

Place map

The Place Map list gives you the option to place a map only "Once per report" or "For each" instance of a group.

Header and Footer

Use the Header and Footer option buttons to determine where in the report your map will be placed.

39.15.1.1 Advanced option view

Use the Advanced option view when you have multiple map values, rather than only one, or when you do not have any groups or summaries in the report. With the Advanced Layout you determine which fields are used for the Geographic breakdown, the On change of value, and the Map value. You can also set summary options.

ⓘ Note

You must map on a geographic field, such as Country, Region, or City. The program cannot generate a map based on a non-geographic field.

Available fields

The Available fields list allows you to select which fields to use as the data for your map.

Browse

Click the Browse button to view the data contained in the selected field in the Available fields list.

Add

Click the Add button (>) to copy fields from the Available fields list to the Geographic Field text box, the On change of text box, or the Map values list.

Remove

Click the Remove button (<) to remove a selected field from the Map values list.

Remove All

Click the Remove All button (<<) to remove all fields from the Map values list.

Geographic field

Use the Geographic field text box to determine the geographic breakdown of the map. For example, if you insert the field Customer.Country into the Geographic field text box, your map will display values for each country. If you insert the field Customer.Region, your map will display values for each region.

On change of

Use the on change of list to specify a field to use as a condition for displaying a summary field. Summary values will be displayed whenever the value in the condition field changes.

Map values

Use the Map values list to create a list of all the fields you want shown as values on the map.

Don't summarize values

Use the Don't summarize values check box to determine whether the program should automatically summarize the values that will appear on the map.

- If you select this check box, the program will not summarize the values.

- If you clear this check box, the program will summarize the map values, and the Set Summary Operation button will activate. The program adds a summary field to the Map Values list.

Set Summary Operation

Use this button to set or change the summary operation for the selected field. When you click this button, the [Edit Summary dialog box \[page 1228\]](#) appears. This button is only active when you have cleared the Don't summarize values check box, and when you have selected a field in the Map Values list.

39.15.1.2 Group option view

The Group option view allows you to show a summary field (such as Last Year's Sales) on change of a geographic field (such as Region). In order to create a map using the Group layout, you must have at least one group and at least one summary field in your report.

On change of

Use the on change of list to specify a field to use as a condition for displaying a summary field. Summary values will be displayed whenever the value in the condition field changes.

Show

The Show list displays the value options for your map. In other words, the data contained in the field in the Show list will be used by the program as map values. If the Show list contains the Sum of Customer.Last Year's Sales field, your map values will be last year's sales for each customer.

39.15.1.3 Cross-Tab option view

Use the Cross-Tab layout button when mapping on a cross-tab object. The cross-tab information can be used as the basis of the map. A Cross-Tab map does not require groups or summary fields. With the Cross-tab layout, you determine the Map on value, the Geographic field breakdown, and whether or not the data will be subdivided.

Geographic Field

Use the Show Geographic Field list to determine the geographic breakdown of your map. For example, if you select the Customer.Country field, your map will display values for each country. If you select the Customer.Region field, your map will display values for each region.

Subdivided by

Use the Subdivided by list to select an additional field to map on, if desired. The program uses this field to subdivide the pie or bar charts appearing on your map.

Map on

The Map on list displays the value options for your map. In other words, the data contained in the field in the Show list will be used by the program as your map values. If the Show list contains the Sum of Customer.Last Year's Sales field, your map values will be last year's sales for each customer.

Related Information

[Cross-Tab Objects \[page 346\]](#)

39.15.1.4 OLAP option view

The OLAP option view allows you to map to an OLAP object. The OLAP information can be used as the basis of the map.

Map off entire grid

Select the option to map on all grid column and row fields. When you select this option, all other fields in the area are unavailable. Mapping off an entire grid is useful when the grid has asymmetric filters based on members from two or more dimensions.

On change of

Use the on change of list to specify a field to use as a condition for displaying a summary field. Summary values will be displayed whenever the value in the condition field changes.

Subdivided by

Use the Subdivided by list to select an additional field to map on, if desired. The program uses this field to subdivide the pie or bar charts appearing on your map.

Other Dimensions

Click the Other Dimensions button to call the [Format Other Dimensions dialog box \[page 1279\]](#).

Click a dimension (shown with its field setting) in the list then click Select a field value to specify a new field for the dimension to be fixed to.

39.15.2 Type tab (Map Expert)

The Map Expert provides five basic map types, each suitable for a different strategy of data analysis. When deciding which map type best fits your report, you should consider the type of data you want to analyze. For example, if you want the map to display one data item for each geographic division (city, state, country, and so on), then you might use a Ranged, Dot Density, or Graduated map. On the other hand, if you want the map to display more than one value for each new geographic division, then you could use a Pie Chart map or a Bar Chart map.

There is also a separate Options section that allows you to decide the appearance of the map values when they are displayed on the map. For example, you can choose a color for the lowest interval on your map.

The Type tab options vary, depending on the type of map you are creating.

39.15.2.1 Ranged option view

A Ranged map breaks data into ranges, assigns a specific color to each range, and then codes geographic areas on the map by color in order to display the range. You can use a ranged map to display sales figures in a range of equal intervals. You can then use shades of color to code sales regions according to those figures. This type of map is useful for determining the regions of highest sales simply by looking at the colors it displays.

Number of intervals

Use this box to enter the number of intervals you want to use as a range. Your map and its legend change accordingly. You can enter a number between 1 and 16. The default is 5.

Distribution method

Use this list to assign range intervals in such a way that the values fit one of the available criteria:

- [*Equal count*](#)
Assigns intervals so that the same number of regions (or as close to the same number of regions as possible) appear in each interval.
- [*Equal ranges*](#)
Assigns intervals so that the summary values in each interval are numerically equal.
- [*Natural break*](#)
Assigns intervals using an algorithm that attempts to minimize the difference between the summary values and the average of the summary values for each interval.
- [*Standard deviation*](#)
Assigns intervals so that the middle interval breaks at the mean (or average) of the data values, and the intervals above and below the middle range are one standard deviation above or below the mean. Standard deviation is the square root of the variance. This option projects how various values in a set of values deviate from the mean for that set.

Color of highest interval

Use this list to select the color you'd like to use for the highest (largest) interval in your range. This color becomes the top end of your color range.

Color of lowest interval

Use this list to select the color you'd like to use for the lowest (smallest) interval in your range. This color becomes the bottom end of your color range.

Allow empty intervals

Select this check box if you want to see empty intervals on your ranged map. This option is selected by default.

39.15.2.2 Dot Density option view

A Dot Density map displays a dot for each occurrence of a specified item. For example, you might create a United States map that shows one dot for each national park. The purpose of a Dot Density map, then, is to provide an overall impression of the distribution of the specified item. Such a map is not a very accurate means of showing detailed information, but is a good way to give an impression of distribution.

Size of dots

Click Small or Large to define the size of the dots you want on your Dot Density map.

39.15.2.3 Graduated option view

A Graduated map is much like a Ranged map in that it shows one symbol per instance of a specified item. Use a Graduated map for displaying data that is linked to points rather than precise geographical areas (as is the case with Ranged map types). For example, a map of an individual region could use graduated circles to represent the sales for each office. The size of each circle would be proportional to the sales (or to the sales range) of the office it represents. A Graduated map provides a more efficient representation of point data than a Ranged map does, and it uses sized symbols rather than colors to distinguish variations in the values of the items it represents.

You can choose the symbol you want to use on your Graduated map. Each symbol is proportional in size to the value of the item it represents (within a range of three sizes).

Symbol style

Click Customize to open the [Symbol Style dialog box \[page 1371\]](#). You can use this dialog box to choose and format a symbol.

39.15.2.4 Pie Chart Option view

A Pie Chart map displays a pie chart over each geographic area. Each slice of the pie represents an individual data item and shows that item's percentage in the whole. You can use this map type to compare the distribution of several items within a particular region. You can also specify that the pie charts be sized proportionately, so that, as with the symbols in a Graduated map, the pie charts will appear in various sizes, depending on the underlying data values. This allows you to compare the totals between regions.

Size of pies

Use the slider to choose Large, Medium, or Small as the size of the pies to display on your map.

Proportional sizing

Select this check box to specify that the pie charts on your map will be sized proportionately depending on the underlying data values.

39.15.2.5 Bar Chart Option view

A Bar Chart map works like a Pie Chart map, but may be more useful for certain sets of data. Typically, you would use a Bar Chart map for items that do not total 100% (that is, for data items that do not make a whole, or for data items that are unrelated).

Size of bars

Use the slider to choose Large, Medium, or Small as the size of the bars to display on your map.

39.15.3 Text tab (Map Expert)

The Text tab allows you to determine the title of your map, legend information, and the legend title.

Map title

Use this text box to create a title for your map. Creating a map title is optional.

Full Legend

Select the Full Legend option button for a detailed breakdown of how the data has been represented in the map.

Compact Legend

Select the Compact Legend option button for a minimal breakdown of how the data has been represented in the map.

No Legend

Select this option button if you prefer not to have a legend with your map.

Made by map

Select the Made by map option button to create a legend title for you.

Specify

Select the Specify option button to create your own legend title.

Title

Use this text box to create a legend title for your map.

Subtitle

Use this text box to create a legend subtitle for your map.

39.16 OLAP Expert

The OLAP Expert appears when you select the [OLAP Grid command \[page 1096\]](#) from the Insert menu.

This expert lets you create an OLAP grid object that you can add to an existing report. Although similar to the Cross-Tab Expert in several ways, the OLAP Expert appears to be different due to the requirements of working with OLAP data sources.

For the first three tabs of the OLAP Expert, refer to the descriptions of their counterpart screens in the [OLAP Report Creation Wizard \[page 1382\]](#):

- [OLAP Data screen \(OLAP Report Creation Wizard and OLAP Expert\) \[page 1386\]](#)
- [Rows/Columns screen \(OLAP Report Creation Wizard and OLAP Expert\) \[page 1394\]](#)
- [Slice/Page screen \(OLAP Report Creation Wizard and OLAP Expert\) \[page 1396\]](#)

See [Style tab \(various experts\) \[page 1415\]](#) for a description of the Style tab.

39.16.1 Customize Style tab (OLAP Expert)

Use the Customize Style tab to format an OLAP grid with background color, gridlines and cell margins.

Note

The Customize Style screen only formats the primary grid dimensions you have selected in the Rows/Columns screen.

Columns

This list displays the dimension and levels used as the column aspect in the grid.

Rows

This list displays the dimension and levels used as the row aspect in the grid.

Background Color

Use this list to select a color for the selected row or column level.

Alias for formulas

Type the alias for the selected dimension. Shortened dimension names can be useful when building conditional formatting formulas.

Indent Row Labels

Select this check box then type the indentation measurement for row labels in the box. This can be as long as two pages.

When you specify an indentation, all row labels, except the label for the input member of the first level will be indented by the same amount.

Indent Column Labels

Select this check box then type the indentation measurement for column labels in the box. This can be as long as two pages.

When you specify an indentation, all column labels, except the label for the input member of the first level will be indented by the same amount.

Repeat Row Labels

Select this check box to repeat row labels on every page for grids that print across more than one page.

Keep Columns Together

Select this check box to keep columns together on every page for grids that print across more than one page.

Column totals on Top

Select this check box to display totals on the top (relative to the corresponding column input member).

Row totals on Left

Select this check box to display row totals on the left (relative to the corresponding row input member).

Show Cell Margins

Select this check box to expand column widths to show all the grid. If you clear the check box, column widths will be uniform and some data will be hidden behind the grid.

Suppress Empty Rows

Select this check box to suppress empty rows for the OLAP Grid object. If you clear the check box, empty rows will be shown in the OLAP Grid object.

Suppress Empty Columns

Select this check box to suppress empty columns for the OLAP Grid object. If you clear the check box, empty columns will be shown in the OLAP Grid object.

Suppress Drilldown Indicators

Select this check box to suppress the drilldown indicators in your OLAP Grid. The drilldown indicators are the small plus and minus signs that appear in the upper-left-hand corner of the cells in your grid; these signs are used to indicate that further levels of detail are available.

Format Grid Lines

Click this button to specify settings for the grid lines in the Format grid lines dialog box.

Type the amount of space between the left edge of the report page and the label in the Horizontal gap box.

39.16.2 Labels tab (OLAP Expert)

Use the Labels screen to define which dimensions are labeled and how they are displayed in the grid.

Unlabeled Dimensions

Displays the dimension that will not be labeled on the grid.

Labeled Dimensions

Displays the dimensions that will be labeled on the grid.

Add or remove arrow buttons

Use the > arrow add fields from the Unlabeled Dimensions list to the Labeled Dimensions list. Use the < arrows to remove fields you've already added. Double arrows add or remove all fields.

Order arrow buttons

Click the up and down arrow buttons to change the order of the Labeled dimensions.

Label Position

Click Top to display labels at the top of the grid object.

Click Left to display labels to the left of the grid object.

Click Don't display labels to hide labels.

Label Spacing

Type the amount of space between the top edge of the grid object and the label in the Vertical gap box.

Display dimension name

Select this option to display a dimension's name instead of its description.

39.17 Record Sort Expert

The Record Sort Expert appears when you choose the [Record Sort Expert command \[page 1118\]](#) from the Report menu.

Use the Record Sort Expert to define how you want the records in your report to be sorted for printing. You can add and remove a sort field and define the sort direction (ascending or descending) for the data in your report.

Available Fields

Displays a list of the report fields currently in the report as well as those available in the report's data source tables.

Sort Fields

Displays a list of the sort fields currently in the report.

Add (>)

Adds the selected report field to the Sort Fields list.

Remove (<)

Removes the selected sort field from the Sort Fields list.

Sort Direction

Sets the sort direction to ascending or descending.

Related Information

[Record Sort Expert command \[page 1118\]](#)

[Sorting, Grouping, and Totaling \[page 167\]](#)

[Sorting single and multiple fields \[page 169\]](#)

39.18 Search Expert

This expert appears when you click the Advanced Find button on the [Find dialog box \[page 1252\]](#).

Use the Search Expert to search for specific values in a report.

Note

The Search facility is only available in the [Preview tab \[page 93\]](#).

Search tab settings

Use the lists on any search tab to select the condition that best finishes this sentence:

"I want to select all records where the value in the field I have selected is"

You can select primary conditions, date conditions or Boolean conditions, depending on the type of field you have selected.

New

Click the New tab or button to create a new search formula. When you click this tab/button, the [Choose Field dialog box \[page 1167\]](#) appears where you select a field to create the search for.

Delete

Click this button to delete the active search and its associated tab.

Browse

The Browse button has been included as an aid in selecting the values that define your search. When you click this button, the program displays a list of field values for the records in the database. If the value you want is on the list, select the value and click the Paste Data button. The program enters the value selected in the active text box (the text box displaying the insertion point).

Note

If you need to select multiple values (for example, if you want to include records in which the field value is one of three values), the insertion point automatically moves to the next text box as soon as it finishes pasting data in the previous box.

Search Direction

Choose the Forward option to search the records from your current position in the report forward to the end of the report.

Choose the Backward option to search the records from your current position in the report backward to the beginning of the report.

Note

If you do not have a field selected, the Search will begin at the top of the current page.

Search

Click this button to begin a search. Once you have begun the search, click [F3](#) to find the next record.

Show/Hide Formula

Search criteria that you enter via the expert automatically generates a record selection formula. You can review and modify this formula by clicking the Show button. When you click the button the expert expands to show the formula. When you are finished editing/reviewing, click the Hide button to hide the formula from view. The Select Expert is automatically updated with any modifications you make to the formula.

Formula Workshop

When you click this button, the [Format Editor dialog box \[page 1256\]](#) appears with the selection formula shown in the Formula text box.

39.19 Section Expert

This expert appears when you choose the [Section Expert command \[page 1116\]](#) from the Report menu.

Use the Section Expert to make formatting changes that affect entire sections of your report.

Sections

This box lists all of the sections in the active report. When you highlight a section in this list, the program sets the dialog box buttons to indicate the actions you can take and the properties you can set for the selected section.

Insert

Use this button to insert a new report section immediately below and of the same type as the section you have selected in the Sections list. For example, if you have a Details section selected and click the Insert button, the program will insert a new Details section immediately below the selected section.

Delete

Use this button to delete the section you have selected in the Sections list. You can only delete sections that have been added to the originating sections; you cannot delete any of the originating sections.

Merge

Use this button to merge the current section with the companion section that appears immediately below it. For example, if you have three sections, Details A, Details B, and Details C, and you highlight Details B and click the Merge button, the program merges Details B and Details C into a single section.

Note

You cannot merge sections of different types; for example, you cannot merge a Details section with a Group Header section.

Arrow buttons

Use these buttons to move a section up or down with respect to other related sections. For example, if you have three sections, Details A, Details B, and Details C, if you highlight Details B and click the up arrow, the program moves the selected section to the top of the three Details sections. Conversely, if you highlight Details B and click the down arrow, the program moves the selected section to the bottom of the three.

Note

You cannot move a section past a section of a different type. For example, you cannot move a Details section past a Group Header section.

Conditional Formula

Click this button if you want to set up a conditional formatting formula for the corresponding property. See [Working with conditional formatting \[page 287\]](#) for more information.

Related Information

[Common tab \(Section Expert\) \[page 1451\]](#)

[Color tab \(Section Expert\) \[page 1455\]](#)

[Layout tab \(Section Expert\) \[page 1456\]](#)

39.19.1 Common tab (Section Expert)

Use the Common tab to set properties such as: visible, keep together, formatting with multiple columns, and so on for a desired section.

Hide (Drill-Down OK)

Select this check box if you do not want the section to print, but you want the section contents to be available for drill-down.

Suppress (No Drill-Down)

Select this check box if you do not want the section to print, and you do not want the section contents to be available for drill-down.

Print at Bottom of Page

When this check box is selected, each group value prints only at the bottom of a page. (Details continue to print in their normal positions.) The command is useful for printing invoices and other reports where you want a single group (for example, line items grouped by order number) to appear on a page and the value for that group (subtotal, summary field, and so on) to print only at the bottom of the page.

Keep Together

When this check box is selected, the program keeps all the lines of the section together, either on the current page (if there is room) or on the next (if not).

In a customer list, for example, data on a single customer (from a single record or linked records) may extend over several lines. If the standard page break falls within the data for a customer, the data will be split, part on one page, part on the next. But with Keep Together, the program inserts the page break before the record begins so that all the data will be printed together, on the following page.

Note

This option does not work with multi-line string or memo fields. A page break may occur in the middle of a multi-line field.

Suppress Blank Section

When this check box is selected, the program hides the report section if it is blank, and prints it if it is not.

Underlay Following Sections

Select this check box if you want the selected object to underlay the following section(s) when it prints. For an example, see [Making an object underlay a following section \[page 242\]](#).

Format with Multiple Columns

When this check box is selected, the [Layout tab \(Section Expert\) \[page 1456\]](#) will appear.

Note

This option appears only when you've selected Details in the Sections list.

Clamp Page Footer

When this check box is selected, the program removes any extra white space at the bottom of a report. This minimizes unnecessary scrolling for reports viewed online.

Note

This option appears only when you have selected Page Footer in the Sections list.

Reserve Minimum Page Footer

This option enables you to minimize the space reserved for your Page Footer sections, thereby maximizing the space available for valuable report information on each page. This option only affects a Page Footer area with multiple sections.

By default, Crystal Reports reserves space at the bottom of each page for your Page Footer sections. This space is reserved whether or not the Page Footer sections are conditionally suppressed. Normally, the space reserved is equivalent to the combined height of all Page Footer sections that are shown or conditionally suppressed.

You can, however, minimize the space reserved. To do so, first conditionally suppress individual Page Footers as desired. Then select Page Footer in the Sections area of the Section Expert, and select the Reserve Minimum Page Footer option on the Expert's Common tab. When you do this, Crystal Reports will only reserve space on each page for the tallest of your Page Footer sections.

Note

This option appears only when you've selected Page Footer in the Sections list.

Read-only

This option makes a section read-only and locks the formatting and position of all the report objects it contains so they can't be formatted or moved. When you select this option, all other choices in the Section Expert become inactive. The formatting options that are usually available on the toolbars or shortcut menus are also suppressed for the section.

Relative Positions

This option allows you to lock the relative position of a report object next to a grid object within a section. For example, if you place a text object one centimeter to the right of a cross-tab object, during report generation the program will push the text object to the right, so that the one centimeter space is maintained regardless of the width of the cross-tab object.

Note

Only horizontal locking is possible. If you need to preserve relative vertical positioning, you should create a new report section.

Note

Once Relative Positions is selected for a section, you can no longer drag objects to new positions within that section. To change an object's position, use the [Object Size and Position dialog box \[page 1311\]](#).

Note

Relative positioning applies only to objects placed to the right of grid objects (that is, objects that can grow horizontally).

39.19.2 Paging tab (Section Expert)

Use the Paging tab to set properties such as: page breaks, page numbers, orientation, and so on for a desired section.

New Page Before

New Page Before is an available format property for Group (header and footer) and Details sections. When you select this option, the program inserts a page break before it prints the section. The page break thus comes before:

- The group (if using the option with a Group footer section).
- Each report record (if using the option with a Details section).

The Page Header and Page Footer appear on each page. If you have a subtotal or summary field in a Group Footer section, you can use the New Page Before option to put these values on pages following the value being totaled.

Reset Page Number After

Resets the page number to one (1) for the following page, after it prints a group total. When this option is used in conjunction with Print at Bottom of Page, the program prints a single group on a page, prints the group value at the bottom of the page, and resets the page number to 1 for the next page. This option is useful whenever you are printing multiple reports from a single file (for example, invoices), and you want each report to be numbered beginning with Page 1.

New Page After

When the End of Section check box is selected, the program inserts a page break after the section.

- The Page Header and Page Footer appear on each page.
- Use New Page After in the Group footer section to print each group on a separate page.

When the Visible Records check box is selected, the program regularly inserts a page break after a specific number of records.

Note

This feature applies only to visible records. Suppressed records may be contained on a page, but will not count toward visible records.

Note

This option appears only when you have selected the Details section in the Sections list.

Orientation

Allows you to set page orientation for each section in the report except for the Page Header and Page footer. The default setting will be the same as the report page orientation in the Page Setup dialog box.

For more information on setting page orientation, see [Setting page size and page orientation](#) in the Related Links.

Note

Once changed from the default setting, section page orientation will not be affected by any further changes to the report page orientation.

Related Information

[Setting page size and page orientation \[page 266\]](#)

39.19.3 Color tab (Section Expert)

Use the Color tab to set the background color for the desired section.

Background Color

To add a background color to the selected section, select this check box, then choose a color from the list.

Conditional Formula

Click this button if you want to set up a conditional formatting formula for the corresponding property. See [Working with conditional formatting \[page 287\]](#) for more information.

39.19.4 Layout tab (Section Expert)

The Layout tab appears when you have the details section selected and the Format with Multiple Columns box is checked on the Common tab.

Use the Layout tab to set up your report in a multi-column format. That is, instead of having the data print straight down the page, you can set up multiple columns and have the data flow from column to column. You can also have your data print across then down the page, printing one record in each column, then printing a second record in each column, then a third, and so on. The dialog box is divided into four smaller boxes.

Detail size

This box enables you to specify the dimensions (height and width) of one detail. Determine how wide you want each detail to be (based on number of characters, font size, and so on) and enter that value in the Width edit box.

Determine how high you want each detail to be (based on number of lines in the detail, font size, and so on) and enter that value in the Height edit box.

Gap between details

This box enables you to specify the empty area (gap, gutter, and so on) you want to allow between details. Horizontal = the gaps between details going across the page, Vertical = the gaps between details going down the page.

Determine the gaps you want to allow. Enter the horizontal gap in the Horizontal edit box and enter the vertical gap in the Vertical edit box.

Printing direction

The Printing direction box enables you to specify the path the program follows when printing the details on a report page. Your options are:

- *Across then Down*
Prints details across the columns, one detail in the first column, one in the next, one in the next, and so on. Then, when all the columns have a detail, the program moves down the page and prints a second detail in the first column, then in the second, and so on.
- *Down then Across*
Prints details down the first column, then down the second column, and so on.

Format groups with multiple column

Select this check box if you want the program to format groups with multiple columns using the Width, Gap between Details, and Printing Directions specified for the selected section.

39.20 Select Expert

This Expert appears:

- When you choose the [Select Expert command \[page 1112\]](#) from the Report menu.
- When you click the Select Expert button on the [Expert Tools toolbar \[page 1149\]](#).

Use the Select Expert to select the records or groups you want to include in your report (if you do not want them all included). For example, you may have a customer database that contains records for customers from every state and Canadian province but you want to do a report only on Texas customers. Select Records enables you to restrict your report so that only Texas customers are included. When using the Select Expert, the program automatically generates a record or group selection formula based on settings you specify in the dialog box.

Note

This Expert allows you to set up reasonably complex selection criteria, but to have more control over the formula that is generated, use the [Formula Editor dialog box \[page 1280\]](#).

Select tab settings

Use the lists on any select tab to select the condition that best finishes this sentence:

"I want to select all records where the value in the field I have selected is"

You can select primary conditions, date conditions or Boolean conditions, depending on the type of field you have selected.

New

Click the New tab or button to create a new selection formula. When you click this tab, the [Choose Field dialog box \[page 1167\]](#) appears where you select a field to create the selection formula on.

Delete

Click this button to delete the active selection formula and its associated tab.

Browse

The Browse button has been included as an aid in selecting the values that define your selection formula. When you click this button, the program displays a list of field values in the database. If the value you want is on the list, select the value and click the Paste Data button. The program enters the value selected in the active text box (the text box displaying the insertion point).

Note

If you need to select multiple values (for example, if you want to include records in which the field value is one of three values), the insertion point automatically moves to the next text box as soon as it finishes pasting data in the previous box.

Show/Hide Formula

Record selection criteria that you enter via the Expert automatically generates a record selection formula. You can review and modify this formula by clicking the Show button. When you click the button the Expert expands to show the formula. When you are finished editing/reviewing, click the Hide button to hide the formula from view. The Select Expert is automatically updated with any modifications you make to the formula.

Record Selection/ Group Selection

Select the Record Selection option to create a record selection formula.

Select the Group Selection option to create a group selection formula.

Formula Workshop

When you click this button, the [Formula Editor dialog box \[page 1280\]](#) appears with the selection formula shown in the Formula text box.

Related Information

[Record Selection \[page 155\]](#)

39.21 Template Expert

This expert appears when you choose the [Template Expert command \[page 1118\]](#) from the Report menu.

Use the Template Expert to choose from predefined formatting templates for use in your report.

Apply a new template

This option is selected by default each time you open the Template Expert. It is cleared only when you select "Undo the current template" or "Re-apply the last template."

Available Templates

This box displays the templates available for use in your report. Select the desired template from the list.

Browse

Click this button to use the [Open dialog box \[page 1314\]](#) to search for a template.

Preview

This box displays a sample of the template selected in the Available Templates list.

Note

A sample is available only for those reports that have the Save Preview Picture option selected in the [Document Properties dialog box \[page 1224\]](#).

Undo the current template

Use this option to remove a template you applied in your current session with Crystal Reports. This option provides an undo for template application.

Re-apply the last template

Use this option to reapply a template you applied previously during your current session with Crystal Reports. You might select this option to update changes made to your report (newly added fields for example) after initially applying a template.

Related Information

[Formatting \[page 235\]](#)

39.22 XML Expert

The XML Expert appears when you select the *Legacy XML Expert* button in the [Report Options dialog box \[page 1345\]](#). The expert is designed for you to format XML tags and attributes on one screen. An XML element is created for each section and field in your Crystal report.

Note

The functionality you have access to in the XML Expert depends on the output format that you choose on the expert's Options tab. For more information, see [Customizing XML report definitions \[page 677\]](#).

39.22.1 Format tab (XML Expert)

XML Output Format Used

This box displays the name of the output format your report definition is using. This information is not editable; it can be changed only by selecting a different output format on the Options tab.

Report Definition Tree

This box lists the XML definition for your report in tree format. Tree nodes can be expanded to show their contents. The selected node is active and its associated XML element definition is displayed in the other fields on the dialog box.

XML Element Name

This box contains the name of the XML element associated with the selected report object. The XML element name can be changed only for output using the Custom Format.

New

Use this button to create a new XML element when two or more report objects in the Report Definition Tree are associated with the same XML element.

Attributes

This box lists the attributes for the XML element associated with a selected node in the Report Definition Tree. Attributes describe additional information about an element.

Name

Attributes must have unique names within an element's attribute list. Attribute names do not have to be unique across elements.

Instance Value

This field shows the instance value for the selected XML element attribute. This value is used in the XML output.

Default Value

Enter a value in this field to assure the attribute is not left empty when there is no instance value.

Create

Use this button to create a new attribute. The Create button opens the [XML Attribute dialog box \[page 1377\]](#).

The Create button is available for any selected node that is not suppressed in the Report Definition Tree.

Edit

Use this button to modify an existing attribute. The Edit button opens the [XML Attribute dialog box \[page 1377\]](#).

The Edit button is available only when you select an attribute you created in the XML Attribute dialog box.

Delete

Use this button to delete an existing attribute. The Delete button is available only when you select an attribute you created in the XML Attribute dialog box.

Delete All

Use this button to delete all existing attributes for a selected element. The Delete All button is available only when you select a node that has attributes you created in the XML Attribute dialog box.

Note

You cannot delete the default attributes. The Delete All button removes only attributes you created in the XML Attribute dialog box.

Suppress XML Tag

Select this check box if you don't want to export the selected node in the Report Definition Tree. This option applies only to the selected node and not its children.

Suppress All Children

Select this check box if you want to suppress a node and all of its children.

39.22.2 Options tab (XML Expert)

Use the options on this tab to choose the format you want to use for XML output, and to specify the type of validation (if any) you want to apply.

Choose XML Output Format

This area contains two options: Crystal ML Schema, and Custom Format. Click the appropriate link for more information:

- [Crystal ML Schema \[page 684\]](#)
- [Custom Format \[page 685\]](#)

Create External Validation

Select this check box if you want the program to create an external XML Schema Definition (XSD), or an external Document Type Definition (DTD).

Note

If you choose the Crystal ML Schema format, you cannot choose to have an external DTD.

Customize Document Prolog

Select this check box if you want to add a Document Prolog to your XML output.

A Document Prolog can be used to reference an external DTD. For example, if you want to validate your Crystal XML output against your own DTD, you can enter its path as a Document Prolog.

Note

You can also use the Document Prolog for an XSL transform. For example, you can reference an HTML style sheet to your XML output so that your browser transforms your XML into HTML.

40 Operators

Operators are special symbols or words that describe an operation or an action to take place between two or more values. Operators are used in formulas. The program reads the operators in a formula and performs the actions specified.

Online support, samples, and technical briefs

For more information about topics covered in this online help, visit the SAP Community Network where you'll find a large collection of samples, downloads, technical briefs, and other useful information.

<http://scn.sap.com> 

40.1 Arithmetic operators

Arithmetic operators are used to calculate number or dollar values.

40.1.1 Add operator

Basic and Crystal syntax.

Usage

```
x+y
```

Adds values x and y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
5 + 6
```

Returns 11.

```
{file.QTY1} + {file.QTY2}
```

Returns 1487 where {file.QTY1} = 366 and {file.QTY2} = 1121.

```
{file.AMT1} + {file.AMT2} + {file.AMT3} + {file.AMT4}
```

Returns 20 where {file.AMT1} = 2, {file.AMT2} = 4, {file.AMT3} = 6, {file.AMT4} = 8.

```
{file.CLASS1} + 25
```

Returns 37 where {file.CLASS1} = 12.

```
CDate (1991, 04, 05) + 12
```

Returns the date value April 17, 1991.

40.1.2 Subtract operator

Basic and Crystal syntax.

Usage

```
x - y
```

Subtracts value y from value x.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
244 - 112
```

Returns 132.

```
{file.SALES} - {file.COGS} - {file.S&A}
```

Returns 214,972, where {file.SALES} = 455,031, {file.COGS} = 188,213, and {file.S&A} = 51846.

```
{file.ONHAND} - 877
```

Returns 114, where {file.ONHAND} = 991.

```
CDate (1991, 04, 15) - 12
```

Returns Apr 03 91.

40.1.3 Multiply operator

Basic and Crystal syntax.

Usage

```
x*y
```

Multiplies value x by value y.

Examples

The following examples are is applicable to both Basic and Crystal syntax:

```
2883 * 1999
```

Returns 5,763,117.

```
{file.AMOUNT} * {file.QTY} * {file.DISCOUNT}
```

Returns 26.25 where {file.AMOUNT} = 25.00, {file.QTY} = 7, and {file.DISCOUNT} = .15

```
{file.EXMPT} * 356.00
```

Returns 152,012 where {file.EXMPT} = 427.

40.1.4 Divide operator

Basic and Crystal syntax.

Usage

```
x/y
```

Divides value x by value y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
25/5
```

Returns 5

```
1/3
```

Returns .333333

```
{file.SALES} / {file.FORECAST}
```

Returns .875, where {file.SALES} = 52533, {file.FORECAST} = 60000

```
{file.DAYSUE} / 5
```

Returns 22, where {file.DAYSUE} = 110

Note

If the denominator = 0, the report will be halted with a divide by zero protection. If you want to avoid this type of problem, you should put a test in.

The following example is applicable to Crystal syntax:

```
If {file.FORECAST} = 0 Then  
    0  
Else  
    {file.SALES} / {file.FORECAST}
```

40.1.5 Percent operator

Crystal syntax.

Usage

```
x % y
```

Calculates value x as a percentage of value y. That is (x/y) * 100.

Examples

The following examples are applicable to Crystal syntax:

```
{file.BALANCE OUTSTANDING} % {file.CREDIT LIMIT}
```

Returns 30.00, where {file.BALANCE OUTSTANDING} = \$1500 and {file.CREDIT LIMIT} = \$5000.

```
{file.AMOUNT} % {file.CREDIT LIMIT}
```

Returns 32.26, where {file.AMOUNT} = 2257.87 and {file.CREDIT LIMIT} = 7000.

Note

If the denominator equals 0, the report will be halted with a divide by zero protection. If you want to avoid this type of problem, you should put a test in.

```
If {file.FORECAST} = 0 Then  
    0  
Else  
    {file.SALES} % {file.FORECAST}
```

40.1.6 Integer divide (x \ y)

Basic and Crystal syntax.

Usage

```
x \ y
```

Divides Number value x by Number value y and returns a quotient that is a whole number.

Note

If x or y is not a whole number, then it is first rounded before the operation.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
23 \ 5
```

Returns 4.

```
3 \ 2
```

Returns 1.

40.1.7 Modulus (x Mod y)

Basic and Crystal syntax.

Usage

```
x Mod y
```

Divides x by y and returns a remainder that is a whole number. The Number values x and y can be positive or negative, and they can be fractional. If x or y is fractional, it will be first rounded before the modulus is taken.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
83 Mod 10
```

Returns 3.

```
10 Mod 3
```

Returns 1.

```
-10 Mod 3
```

Returns -1.

```
10 Mod -3
```

Returns 1.

```
10 Mod 3.5
```

Returns 2 since 3.5 is first rounded to 4 before the modulus is taken.

40.1.8 Negate operator

Basic and Crystal syntax.

Usage

```
[ -(x) ]
```

Multiplies the value inside the parentheses by -1.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
- (-1)
```

Returns 1 since negative times negative equals positive.

```
-(1)
```

Returns -1 since negative times positive equals negative.

```
-(0-14)
```

Returns 14.

```
-( {file.QTYONHND} )
```

Returns 144 where {file.QTYONHND} = -144.

```
- ( - (15-18) )
```

Returns -3 since $15 - 18 = -3$, $-(-3) = +3$, $-(+3) = -3$.

40.1.9 Exponentiate (x ^ y)

Basic and Crystal syntax.

Usage

```
x ^ y
```

Raise x to the power of y.

Note

The Number value y can be fractional, and it can be positive or negative. The Number value x can be fractional, but can be negative only if y is a whole number.

Examples

The following examples are applicable to both Crystal and Basic syntax:

```
3 ^ 2
```

Returns 9.

```
3 ^ .5
```

Returns 1.73205....

```
(-3.5) ^ 2
```

Returns 12.25. Note that the Exponentiation operator has a higher precedence than Negation, so -3.5^2 is different from $(-3.5)^2$.

```
3 ^ (-2)
```

Returns .111...

Note

To express a number to a negative power, enclose the power in parentheses.

40.2 Conversion operators

Conversion operators are used to convert one data type to another.

40.2.1 To currency operator

Crystal syntax.

Usage

```
$x
```

Converts x from number to currency.

Examples

The following examples are applicable to Crystal syntax.

Note

Each example assumes you have selected the following format options on the Number tab (Custom Style dialog box) of the Format Editor dialog box: Decimal places = (1.00), Negative sign = (345.00-), Currency symbol = (Float), and Thousands Separator = (1,000.00).

```
$12345678
```

Returns \$12,345,678.00.

```
$(123 * 456)
```

Returns \$56,088.00

```
${file.QUANTITY} * 3)
```

Returns \$42.00, where {file.QUANTITY} = 14.

```
${file.MILES} * {file.PLEDGE})
```

\$27.95, where {file.MILES} = 13 and {file.PLEDGE} = 2.15.

Note

You cannot multiply two currency values, doing so will result in an error.

40.3 Comparison operators

Comparison operators are used to compare data in a data field with a constant, with the content of another data field, or with a formula result.

Note

A constant is a fixed value (for example, a text string, date, or a number or dollar value you enter) that remains the same when a calculation is performed.

40.3.1 Equal operator

Basic and Crystal syntax.

Usage

```
x = y
```

x is equal to y

The equal operator tells the program to evaluate an expression (x=y) and return a TRUE (if x is equal to y) or FALSE (if x is not equal to y).

Examples

The following examples are applicable to both Crystal and Basic syntax:

```
{file.QUANTITY} = 3
```

TRUE, where {file.QUANTITY} has a value of 3.

```
{file.QUANTITY} = 3
```

FALSE, in all other situations.

```
{file.YTD} = {file.LASTYEARYTD}
```

TRUE, where the value of the field {file.YTD} is identical to the value of the field {file.LASTYEARYTD}.

```
{file.YTD} = {file.LASTYEARYTD}
```

FALSE, in all other situations.

```
({file.SALES} - {file.COGS}) = 22,654
```

TRUE, where calculating the expression {file.SALES}-{file.COGS} produces the value 22,654, for example {file.SALES} = 109,986 and {file.COGS} = 87,332.

```
({file.SALES} - {file.COGS}) = 22,654
```

FALSE, in all other situations.

```
{customer.LAST NAME} = "Johnson"
```

TRUE, where the text string in the {customer.LAST NAME} field is "Johnson".

```
{customer.LAST NAME} = "Johnson"
```

FALSE, in all other situations.

Comments

This operator is often used in expressions with If-Then-Else operators. For example in Crystal syntax:

```
If {file.PURCHASES} = 0 Then
    "Your account had no activity this month."
Else
    "";
```

Which prints the "Your account..." statement if the {file.PURCHASES} field has a zero value, and prints nothing (indicated by the empty string "") if the {file.PURCHASES} field has something other than a zero value.

40.3.2 Not Equal operator

Basic and Crystal syntax.

Usage

```
x<>y
```

x is not equal to y.

The Not Equal operator tells the program to evaluate an expression (x<>y) and return a TRUE (if x is not equal to y) or FALSE (if x is equal to y).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{orders.ORDER AMOUNT} <> 400
```

TRUE, where {orders.ORDER AMOUNT} is equal to 200 or {orders.ORDER AMOUNT} is equal to 401, and so on.

```
{orders.ORDER AMOUNT} <> 400
```

FALSE, where {orders.ORDER AMOUNT} is equal to 400.

```
{file.DAY} <> "Thursday"
```

TRUE, when {file.DAY} = "Friday".

```
{file.DAY} <> "Thursday"
```

FALSE, when {file.DAY} = "Thursday".

```
{file.ONHAND} <> 0
```


TRUE, where the value of {file.ONHAND} is 10 or -5.

```
{file.ONHAND} <>0
```

FALSE, where the value of {file.ONHAND} is zero.

```
{file.AVAILABLE} - {file.USED} <>10
```

TRUE, where the value of the {file.AVAILABLE} field less the value of the {file.USED} field gives a result other than 10.

```
{file.AVAILABLE} - {file.USED} <>10
```

FALSE, where it gives a value of 10.

Comments

This operator is often used with If-Then-Else operators. For example in Crystal syntax:

```
If {file.SEX} <> "M" Then  
    "FEMALE"  
Else  
    "MALE"
```

Prints the word "FEMALE" if the value in the {file.SEX} field is not equal to "M", and prints the word "MALE" in all other situations.

40.3.3 Less Than operator

Basic and Crystal syntax.

Usage

```
x < y
```

x is less than y

The less than operator tells the Formula Editor dialog box to evaluate an expression ($x < y$) and returns a TRUE (if x is less than y) or FALSE (if x is equal to or greater than y).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{file.WEIGHT} < 200
```

TRUE, where {file.WEIGHT} = 150, or {file.WEIGHT} = 199.

```
{file.WEIGHT} < 200
```

FALSE, where {file.WEIGHT} = 200 or {file.WEIGHT} = 400.

```
{file.COST} < {file.PRICE}
```

TRUE, where {file.COST}=350, {file.PRICE} = 400.

```
{file.COST} < {file.PRICE}
```

FALSE, where {file.COST} = 350 and {file.PRICE} = 350, or where {file.COST} = 350 and {file.PRICE} = 325.

40.3.4 Greater Than operator

Basic and Crystal syntax.

Usage

```
x > y
```

x is greater than y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{file.WEIGHT} > 200
```

FALSE, where {file.WEIGHT}= 150, {file.WEIGHT} = 199, or {file.WEIGHT} = 200.

```
{file.WEIGHT} > 200
```

TRUE, where {file.WEIGHT} = 400 or {file.WEIGHT} = 201.

```
{file.COST} > {file.PRICE}
```

FALSE, where {file.COST} = 350, {file.PRICE} = 400, or where {file.COST} = 350 and {file.PRICE} = 350.

```
{file.COST} > {file.PRICE}
```

TRUE, where {file.COST} = 350 and {file.PRICE} = 325.

40.3.5 Less or equal operator

Basic and Crystal syntax.

Usage

```
x <= y
```

x is less than or equal to y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{file.WEIGHT} <= 200
```

TRUE, where {file.WEIGHT} = 150, {file.WEIGHT} = 200, or {file.WEIGHT} = 199.

```
{file.WEIGHT} <= 200
```

FALSE, where {file.WEIGHT} = 400.

```
{file.COST} <= {file.PRICE}
```

TRUE, where {file.COST} = 350, {file.PRICE} = 400, or where {file.COST} = 350 and {file.PRICE} = 350.

```
{file.COST} <= {file.PRICE}
```

FALSE, where {file.COST} = 350 and {file.PRICE} = 325.

40.3.6 Greater or Equal operator

Basic and Crystal syntax.

Usage

```
x >= y
```

x is greater than or equal to y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{file.WEIGHT} >= 200
```

FALSE, where {file.WEIGHT} = 150 or {file.WEIGHT} = 199.

```
{file.WEIGHT} >= 200
```

TRUE, where {file.WEIGHT} = 400, {file.WEIGHT} = 200, or {file.WEIGHT} = 201.

```
{file.COST} >= {file.PRICE}
```

FALSE, where {file.COST} = 350, {file.PRICE} = 400.

```
{file.COST} >= {file.PRICE}
```

TRUE, where {file.COST} = 350 and {file.PRICE} = 325, or where {file.COST} = 350 and {file.PRICE} = 350.

40.4 String operators

String operators are used to concatenate (join) text strings, to extract substrings from text strings, or to test for the presence of substrings in text strings.

Note

Operators are case-sensitive. For example, "ABC" is not equal to "abc". Nor is "abc" in the string "ABCDEF".

40.4.1 Concatenate operator (x + y)

Basic and Crystal syntax.

[Concatenate \(x & y\) \[page 1479\]](#) and Concatenate (x + y) are synonymous if both operands x and y are Strings.

Usage

```
x + y
```

Concatenates (combines) string x to string y to make one contiguous string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
"Bread" + " and " + "butter"
```

Returns "Bread and butter".

```
"Your customer number is " + ({customer.CUSTOMER ID}) + " and your company  
contact person is " + ({customer.CONTACT FIRST NAME}) + "."
```

Returns "Your customer number is 64 and your company contact person is Bob." Where CustomerID="64" and First Name = "Bob".

Comments

- You can use this operator only if all the elements you are connecting are strings.
- If you want to include a value from a numeric field (for example, an account balance), you must first convert that value to a text string using the [ToText \[page 1716\]](#) function:

```
"Your account balance is " + ToText({file.BALANCE}) + "."
```

See Also

- [Concatenate \(x & y\) \[page 1479\]](#)

40.4.2 Concatenate (x & y)

Basic and Crystal syntax.

Concatenate (x & y) and [Concatenate operator \(x + y\) \[page 1478\]](#) are synonymous if both operands x and y are strings.

Usage

```
x & y
```

Concatenates the values of x and y, and returns the result as a String. x and y can be of one of the types Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to both Crystal and Basic syntax:

```
10 & 20
```

Returns the String "10.0000020.00000", where the number formatting for this formula field specifies 5 decimal places. Use [CStr \[page 1834\]](#) if you want to specify how to format a String when converting from a different type. In this example, you can use CStr on the Number values a and b to convert and format each of them to a String, and then concatenate the resultant Strings.

```
True & False
```

Returns the String "TrueFalse".

```
"The meeting is " & #Nov 15. 1999#
```

Returns the String "The meeting is 11/15/99 12:00:00 AM".

```
"Boy" & "Girl"
```

Returns the String "BoyGirl".

Comments

The format of the returned value is determined by the format that has been set up for that formula field of the corresponding type of operands. However, if you want to convert to a String from a value of a different type and be able to specify formatting for the resultant string, you can use [CStr \[page 1834\]](#).

See Also

- [Concatenate operator \(x + y\) \[page 1478\]](#)

40.4.3 Subscript for arrays and strings

Click a link to view one of the topics in this section:

- [Subscript \(x\(y\)\) Basic Syntax \[page 1481\]](#)
- [Subscript \(x\[y\]\) Crystal Syntax \[page 1482\]](#)

40.4.3.1 Subscript (x(y)) Basic Syntax

Usage

```
x(y)
```

Extracts the y element from string x.

```
x(y To z)
```

Extracts the y to z range of elements from string x.

```
x(n)
```

Extracts the n element of array x.

Note

The subscript ranges are 1-based; they start at 1 rather than 0.

Subscript is used to extract one or more characters from a text string or to extract an element from an array.

Examples

The following examples are applicable to Basic syntax:

```
LNAME (1)
```

Returns "S", where LNAME = "Smith".

```
Postal (6)
```

Returns "V", where Postal Code = "T5A 9V2" (the space between A and 9 counts as an element).

```
{customer.POSTAL CODE} (5 to 7)
```

Returns "9V2", where Postal Code = T5A 9V2.

```
{file.ITEMNUMBER} (4 to 5)
```

Returns "40", where ItemNumber is A1/4020/B10.

Comments

- You must subscript an array by first declaring an array, then referencing the array variable name in Basic syntax. For example:

```
'declare array MyName
Dim MyName () As String
MyName = Array ("Company A", "Company B", "Company C")
'reference array by its name
formula = MyName (1)
```

- Do not confuse Subscript with the [In String operator \[page 1483\]](#). While Subscript tests a target string for the presence of an element and extracts the element (if found) from the string, In String only tests the target string for the presence of the element.

See Also

- [Subscript \(x\[y\]\) Crystal Syntax \[page 1482\]](#)

40.4.3.2 Subscript (x[y]) Crystal Syntax

Usage

```
x[y]
```

Extracts the y element from string x.

```
x[y to z]
```

Extracts the y to z range of elements from string x.

```
x[n]
```

Extracts the n element of array x.

Note

The subscript ranges are 1-based; they start at 1 rather than 0.

Subscript is used to extract one or more characters from a text string or to extract an element from an array.

Examples

The following examples are applicable to Crystal syntax:

```
[100, 233, 466, 998] [3]
```

Returns 466; 466 is the third element in the array.

```
LNAME [1]
```

Returns "S", where LNAME = "Smith".

```
Postal [6]
```

Returns "V", where Postal Code = "T5A 9V2" (the space between A and 9 counts as an element).

```
{customer.POSTAL CODE} [5 to 7]
```

Returns "9V2", where Postal Code = "T5A 9V2".

```
{file.ITEMNUMBER} [4 to 5]
```

Returns "40", where ItemNumber is A1/4020/B10.

Comments

Do not confuse Subscript with [In String operator \[page 1483\]](#).

While Subscript tests a target string for the presence of an element and extracts the element (if found) from the string, In String only tests the target string for the presence of the element.

See Also

- [Subscript \(x\(y\)\) Basic Syntax \[page 1481\]](#)

40.4.4 In String operator

Basic and Crystal syntax.

Usage

```
x in y
```

Tests for the presence of string x in string y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
"Elm" in {customer.ADDRESS1}
```

TRUE, where {customer.ADDRESS1} is "1335 Elm Street".

```
"Elm" in {customer.ADDRESS1}
```

TRUE, where {customer.ADDRESS1} is "1335 Elmer Street".

```
"elm" in {file.MOTTO}
```

FALSE, where {file.MOTTO} = "Feel more energy".

(The "el" ending "feel" and the "m" beginning the word "more" are separated by a space which itself counts as an element.)

```
"el m" in {file.MOTTO}
```

TRUE, where {file.MOTTO} = "Feel more energy".

The search string "el m" this time contains the space between the l and the m, which allows for a perfect match.

Note

The "in" operator can also be used to test for the presence of a string in a text range. For example, "V5B" in "VOA" to "V9Z".

40.4.5 Insert Empty String

Basic and Crystal syntax.

Usage

Inserts an empty string.

Examples

You could insert an empty string to check and see if customer name is blank or empty. If customer name is empty, then you could return the contact name for that customer instead of customer name.

```
if {Customer.Customer Name}="" then
({Customer.Contact First Name}+" "+{Customer.Contact Last Name})''
else
{Customer.Customer Name}
```

40.5 Range operators

Range operators are used to create ranges and to see if a value is within the range created. These operators test for consecutive values such as dates, text, or amounts which fall within a range.

40.5.1 In Range operator

Basic and Crystal syntax.

Usage

```
x in y
```

Tests a range of value (y) to see if a value (x) falls within the range specified.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CurrentDate in CDate(1990, 09, 01) to CDate(1990, 09, 20)
```

Returns True, if today's date is September 15, 1990.

```
CurrentDate in CDate(1990, 09, 01) to CDate(1990, 09, 20)
```

Returns False, if today's date is September 21, 1990.

```
{file.QTY} in {file.ONHAND} to ({file.BACKORDER} + {file.ONORDER})
```

Returns True, where {file.QTY} = 20, {file.ONHAND} = 10, {file.BACKORDER} = 5, {file.ONORDER} = 25 (Is 20 in the range that begins with 10 and ends with the sum of 5 and 25?).

```
{file.QTY} in {file.ONHAND} to ({file.BACKORDER} + {file.ONORDER})
```

Returns False, where {file.QTY} = 31, {file.ONHAND} = 10, {file.BACKORDER} = 5, {file.ONORDER} = 25 (Is 31 in the range that begins with 10 and ends with the sum of 5 and 25?).

Comments

The combination of [Make Range operator \(x To y\) \[page 1486\]](#) and In Range operators is often used with the If-Then-Else operator. For example the Crystal syntax formula:

```
If ({file.AMOUNT} in (100.00 to 250.00)) Then  
    (.10 * {file.AMOUNT})  
Else  
    0;
```

If the value of {file.AMOUNT} falls within the range 100.00 to 250.00, multiply .10 times {file.AMOUNT}. If it does not, it returns zero.

See Also

- [DateTime \[page 1762\]](#) function

40.5.2 Make Range operator (x To y)

Basic and Crystal syntax.

Usage

```
x to y
```

Creates the range x to y.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
100.00 to 250.00
```

The range of consecutive numeric values beginning with 100.00 and ending with 250.00, including the end values.

```
CDate(1990, 09, 01) to CDate(1990, 09, 20)
```

The range of consecutive dates beginning with September 1, 1990, and ending with September 20, 1990. Both September 1 and September 20 are included in the range.

```
"Aaron" to "Lusk"
```

The range of consecutive text values beginning with "Aaron" and ending with "Lusk", including the beginning and end values.

Comments

- You cannot create a formula that has a range as a result. Thus, Make Range is always used in conjunction with other operators such as the [In Range operator \[page 1485\]](#). The combination of Make Range and In Range produces a formula that gives a single TRUE or FALSE value, not a range.
- The program comes with a number of [Date Ranges \[page 1801\]](#), such as YearToDate, preset for your convenience.

40.5.3 Left end point excluded range (x _To y)

Basic and Crystal syntax.

Usage

```
x _To y
```

x _To y is used to specify a range of values greater than but not including the value x, and less than or equal to the value y. x and y can be of one of the types: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following example is applicable to Basic syntax:

```
Dim BlackoutDates As Date Range
Rem Blackout period is Jan. 1, 1999 to March 31, 1999, inclusive
BlackoutDates = CDate(#12/31/1998#) _To CDate(#3/ 31/1999#)
If CurrentDate In BlackoutDates Then
    formula = "Blackout"
Else
```

```
    formula = "Free to trade"  
End If
```

If today's date is Dec. 31, 1998, then "Free to trade" is returned since it is before the Blackout period.

If today's date is Jan. 1, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is March 31, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is Oct. 30, 1999, then "Free to trade" is returned since it is after the Blackout period.

40.5.4 Right end point excluded range (x To_ y)

Basic and Crystal syntax.

Usage

x To_ y

x To_ y is used to specify a range of values greater than or equal to the value x, and less than but not including the value y. x and y can be of one of the types: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following example is applicable to Basic syntax:

```
Dim BlackoutDates As Date Range  
Rem Blackout period is Jan. 1, 1999 to March 31, 1999, inclusive  
BlackoutDates = CDate(#1/1/1999#) To_ CDate(#4/1/1999#)  
If CurrentDate In BlackoutDates Then  
    formula = "Blackout"  
Else  
    formula = "Free to trade"  
End If
```

If today's date is Dec. 31, 1998, then "Free to trade" is returned since it is before the Blackout period.

If today's date is Jan. 1, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is March 31, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is April 1, 1999, then "Free to trade" is returned since it is after the Blackout period.

40.5.5 Both end points excluded range (x _To_ y)

Basic and Crystal syntax.

Usage

`x _To_ y`

`x _To_ y` is used to specify a range of values greater than but not including the value `x`, and less than but not including the value `y`. `x` and `y` can be of one of the types: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following example is applicable to Basic syntax:

```
Dim BlackoutDates As Date Range
Rem Blackout period is Jan. 1, 1999 to March 31, 1999, inclusive
BlackoutDates = CDate(#12/31/1998#) _To_ CDate(#4/1/1999#)
If CurrentDate In BlackoutDates Then
    formula = "Blackout"
Else
    formula = "Free to trade"
End If
```

If today's date is Dec. 31, 1998, then "Free to trade" is returned since it is before the Blackout period.

If today's date is Jan. 1, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is March 31, 1999, then "Blackout" is returned since it is within the Blackout period.

If today's date is April 1, 1999, then "Free to trade" is returned since it is after the Blackout period.

40.5.6 Is > x

Basic and Crystal syntax.

`Is > x` is preferred in Basic syntax, whereas [Up from but not including \(upFrom_ x\) \[page 1492\]](#) is preferred in Crystal syntax.

Usage

`Is > x`

`Is > x` is used to specify a range of values greater than the value `x`. `x` can be of any type: Number, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
Is > 250
```

The range of all numbers greater than 250.

```
Is > CDate (2000, 3, 17)
```

The range of dates from March 18, 2000 onwards, including March 18, 2000.

40.5.7 Is < x

Basic and Crystal syntax.

Is < x is preferred in Basic syntax, whereas [Up to but not including \(upTo_ x\) \[page 1494\]](#) is preferred in Crystal syntax.

Usage

```
Is < x
```

Is < x is used to specify a range of values less than the value x. x can be of any type: Number, Currency, Date, Time, DateTime, and String.

Examples

The following example is applicable to Basic syntax:

```
Dim BlackoutDates() As Date Range
Rem Blackout period is any day before 1999 or any date within March 1999
BlackoutDates = Array (Is < CDate(1999, 1, 1), _
                      CDate(1999, 3, 1) To CDate (1999, 3, 31))
If CurrentDate In BlackoutDates Then
    formula = "Blackout"
Else
    formula = "Free to trade"
End If
```

If today's date is Dec. 30, 1998, then "Blackout" is returned since it is within the Blackout period before 1999.

If today's date is Jan. 1, 1999, then "Free to trade" is returned since it is not within any blackout period.

If today's date is March 31, 1999, then "Blackout" is returned since it is within the Blackout month of March.

If today's date is Oct. 30, 1999, then "Free to trade" is returned since it is not within any blackout period.

40.5.8 Is >= x

Basic and Crystal syntax.

Is >= x is preferred in Basic syntax, whereas [Up from \(upFrom x\) \[page 1492\]](#) is preferred in Crystal syntax.

Usage

```
Is >= x
```

Is >= x is used to specify a range of values greater than or equal to the value x. x can be of any type: Number, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
Is >= 250
```

The range of all numbers greater than or equal to 250.

```
Is >= CDate (2000, 3, 17)
```

The range of dates from March 17, 2000 onwards including March 17, 2000.

40.5.9 Is <= x

Basic and Crystal syntax.

Is <= x is preferred in Basic syntax, whereas [Up to \(upTo x\) \[page 1493\]](#) is preferred in Crystal syntax.

Usage

```
Is <= x
```

Is <= x is used to specify a range of values less than or equal to the value x. x can be of any type: Number, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
Is <= 250
```

The range of all numbers less than or equal to 250.

```
Is <= CDate (2000, 3, 17)
```

The range of dates before March 17, 2000, up to and including March 17, 2000.

40.5.10 Up from (upFrom x)

Basic and Crystal syntax.

upFrom x is preferred in Crystal syntax, whereas `Is >= x` [\[page 1491\]](#) is preferred in Basic syntax.

Usage

```
upFrom x
```

upFrom x is used to specify a range of values greater than or equal to the value x. x can be of any type: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
upFrom 250
```

The range of all numbers greater than or equal to 250.

```
upFrom CDate (2000, 3, 17)
```

The range of dates from March 17, 2000 onwards including March 17, 2000.

40.5.11 Up from but not including (upFrom_ x)

Basic and Crystal syntax.

UpFrom_ x is preferred in Crystal syntax, whereas `Is > x` [\[page 1489\]](#) is preferred in Basic syntax.

Usage

```
upFrom_ x
```

upFrom_ x is used to specify a range of values greater than but not including the value x. x can be of any type: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
upFrom_ 250
```

The range of all numbers greater than 250.

```
upFrom_ CDate (2000, 3, 17)
```

The range of dates from March 18, 2000 onwards, including March 18, 2000.

40.5.12 Up to (upTo x)

Basic and Crystal syntax.

upTo x is preferred in Crystal syntax, whereas `ls <= x` [\[page 1491\]](#) is preferred in Basic syntax.

Usage

```
upTo x
```

upTo x is used to specify a range of values less than or equal to the value x. x can be of any type: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
upTo 250
```

The range of all numbers less than or equal to 250.

```
upTo CDate (2000, 3, 17)
```

The range of dates before March 17, 2000, up to and including March 17, 2000.

40.5.13 Up to but not including (upTo_ x)

Basic and Crystal syntax.

upTo_ x is preferred in Crystal syntax, whereas `ls < x` [\[page 1490\]](#) is preferred in Basic syntax.

Usage

```
upTo_ x
```

upTo_ x is used to specify a range of values less than but not including the value x. x can be of any type: Number, Boolean, Currency, Date, Time, DateTime, and String.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
upTo_ 250
```

The range of all numbers less than 250.

```
upTo_ CDate (2000, 3, 17)
```

The range of dates before March 17, 2000, not including March 17, 2000.

40.6 Boolean operators

Boolean operators are used to create conditions that require a logical relationship between two or more values. Conditions that use Boolean operators are called Boolean expressions.

A and B means that both A and B must be true for the condition to be satisfied (to return a TRUE value).

A or B means that either A or B (or both) must be true for the condition to be satisfied (to return a TRUE value).

40.6.1 Not operator

Basic and Crystal syntax.

Usage

```
not (x)
```

Reverses the True or False value of x.

Value of x	Not x
True	False
False	True

- Not (Not(False)) = False
- Not (Not(True)) = True

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
not (A>B and B>C)
```

If A=5, B = 4, C = 3, the expression (A>B and B>C) is TRUE. Both conditions tied together by the Boolean operator And are TRUE; thus, the entire statement has a value of TRUE. The Not operator changes the value of the expression to FALSE.

```
not (A>B and B>C)
```

If A=3, B = 4, C = 3, the expression (A>B and B>C) is FALSE. One of the two conditions tied together by the Boolean operator And is FALSE; thus, the entire statement has a value of FALSE. The Not operator changes the value of the expression to TRUE.

```
not ({file.ONHAND} - {file.ORDER} > 0)
```

Returns TRUE if {file.ONHAND} = 10 and {file.ORDER} = 11.

```
not ({file.ONHAND} - {file.ORDER} > 0)
```

Returns FALSE if {file.ONHAND} = 10 and {file.ORDER} = 9.

40.6.2 And operator

Basic and Crystal syntax.

Usage

x And y

Value of x	Value of y	x And y
True	True	True
True	False	False
False	True	False
False	False	False

Examples

The following example is applicable to Crystal syntax:

```
If {file.CREDIT LIMIT} = 5000 and {file.SALESMAN} = "SP" Then
    {file.AMOUNT}
Else
    0
```

If the credit limit is 5000 and the salesman is SP (both conditions true), then returns the value in the {file.AMOUNT} field, otherwise return zero.

A > B and B > C

Returns TRUE, where A = 10, B = 6, and C = 3 (both conditions are true).

Returns FALSE, where A=10, B=6, and C=7 (only one of the two conditions are true).

(A>B) and (A * C - D > E) and (E / D <= B)

Returns TRUE, where A = 7, B = 5, C = 3, D = 2, E = 10 (all three of the conditions are true).

40.6.3 Or operator

Basic and Crystal syntax.

Usage

x Or y

Value of x	Value of y	x Or y
True	True	True
True	False	True
False	True	True
False	False	False

Examples

The following example is applicable to Crystal syntax:

```
If {file.CREDIT LIMIT} = 5000 or {file.SALESMAN} = "SP"
Then
    {file.AMOUNT}
Else
    0;
```

This means that if the credit limit is 5000, or, if the salesman is SP, (either of the conditions are true), then it returns the value in the {file.AMOUNT} field; otherwise it returns nothing.

The following examples are applicable to both Basic and Crystal syntax:

```
A > B or B > C
```

TRUE, where A = 10, B = 6, and C = 3 (both conditions true).

```
A > B or B > C
```

TRUE, where A=10, B=6, and C=7 (either one of the two conditions true).

```
A > B or B > C
```

FALSE, where A=5, B=6, and C=7 (neither of the two conditions true).

```
(A>B) or (A * C - D > E) or (E / D<= B)
```

TRUE, where A = 5, B = 5, C = 3, D = 2, E = 12 (At least one of the three conditions is true. In this case only (A * C - D > E) is true).

40.6.4 Xor (Logical exclusion)

Basic and Crystal syntax.

Usage

`x Xor y`

Value of x	Value of y	x Xor y
True	True	False
True	False	True
False	True	True
False	False	False

Examples

The following example is applicable to both Basic and Crystal syntax:

```
{Customer.SELL} Xor {Customer.GOVt_APPROVE}
```

Returns True if one of {Customer.SELL} and {Customer.GOVt_APPROVE} is True, and the other is False.

40.6.5 Eqv (Logical equivalence)

Basic and Crystal syntax.

Usage

`x Eqv y`

Value of x	Value of y	X Eqv y
True	True	True
True	False	False
False	True	False
False	False	True

Examples

The following example is applicable to Basic and Crystal syntax:

```
{Customer.SELL} Eqv {Customer.GOVt_APPROVE}
```

Returns True if both {Customer.SELL} and {Customer.GOVt_APPROVE} are True, or both {Customer.SELLs} and {Customer.GOVt_APPROVE} are False.

40.6.6 Imp (Logical implication)

Basic and Crystal syntax.

Usage

```
x Imp y
```

Value of x	Value of y	X Imp y
True	True	True
True	False	False
False	True	True
False	False	True

Examples

The following example is applicable to both Basic and Crystal syntax:

```
{Customer.SELL} Imp {Customer.GOVt_APPROVE}
```

Returns True if both {Customer.SELL} and {Customer.GOVt_APPROVE} are in agreement (that is, both are True or both are False), or only {Customer.GOVt_APPROVE} agrees (that is, only {Customer.GOVt_APPROVE} is True) and overrides.

40.7 Array operators

Array operators are used to build a list of data fields, constants, or text string. These lists can then be used for checking to see if a field exists in the list, or for extracted elements by their position. Whereas the [Range operators \[page 1485\]](#) are used to see if an item exists in a range of values, these array operators allow you to see if an item exists in a set of non-contiguous values.

40.7.1 Make Array

Crystal syntax.

The Make Array operator can be used with Crystal syntax only; the Array function can be used with Basic syntax only. See [Arrays \[page 1820\]](#) (Basic Syntax).

Usage

```
[x, y, z, ...n]
```

Build an array containing the elements x, y, z,...n.

Examples

The following examples are applicable to Crystal syntax:

```
[100, 200, 300, 400]  
[{file.QTYA}, {file.QTYB}, {file.QTYC}]  
[( {file.AMT1} * .5), ( {file.AMT2} * .5), ( {file.AMT3} * .25)]  
[500, ( {file.QTY} / 3)]
```

Note

You cannot have more than one data type in an array.

40.7.2 Subscript (x[y])

See [Subscript for arrays and strings \[page 1481\]](#).

40.7.3 In Array

Basic and Crystal syntax.

Usage

```
x in [y]
```

Is x in the array y?

Examples

The following example is applicable to both Basic and Crystal syntax:

```
Rem Basic syntax  
{customer.REGION} In Array ("CA", "HI", "AK")  
//Crystal syntax  
{customer.REGION} in ["CA", "HI", "AK"]
```

Is the value of the {customer.REGION} field in the array of state abbreviations?

The following examples are applicable to Crystal syntax:

```
{file.COLOR} in ["Red", "White", "Blue"]
```

Is the value of the {file.COLOR} field in the array of colors listed in the brackets?

```
DayofWeek({file.DATE}) in [2, 4, 6]
```

Is the value of the {file.DATE} field, converted to a number that represents the [DayOfWeek \[page 1784\]](#) in the array of numbers listed in the brackets? (Sunday = 1, Saturday = 7)

40.7.4 Redim

Click a link to view one of the topics in this section:

- [Redim Basic syntax x\(n\) \[page 1501\]](#)
- [Redim Crystal syntax x\[n\] \[page 1502\]](#)

40.7.4.1 Redim Basic syntax x(n)

Use Redim x[n] (with the square brackets) only in Crystal syntax, and Redim x(n) (with the round brackets) only in Basic syntax.

Usage

```
Redim x(n)
```

Re-dimension the array x to size n, where x is an array and n is a positive whole number specifying the new size of x.

Examples

The following example is applicable to Basic syntax:

```
Dim x() As String
'Initialize first three array elements
x = Array ("a", "bb", "ccc")
'Re-size x to 4; old values are ignored
'x is now equal to Array ("", "", "", "")
Redim x(4)
'x is now equal to Array ("", "", "", "dddd")
x(4) = "dddd"
formula = x(4)
```

Comments

When an array is re-dimensioned with Redim, elements in the array are filled with default values for that type.

40.7.4.2 Redim Crystal syntax x[n]

- Use Redim x[n] (with the square brackets) only in Crystal syntax, and Redim x(n) (with the round brackets) only in Basic syntax.
- Unlike Redim x(n) in Basic syntax, a Redim statement in Crystal syntax cannot have multiple arrays.

Usage

```
Redim x[n]
```

Re-dimension the array x to size n, where x is an array and n is a positive whole number specifying the new size of n.

Examples

The following example is applicable to Crystal syntax:

```
Local StringVar array x:= ["a", "bb", "ccc"];  
// resize the array to size 4; old values are ignored  
// and filled with default values of empty strings  
Redim x [4];  
x [4] := "dddd"; // only x[4] is initialized
```

Comments

When an array is re-dimensioned with Redim, elements in the array are filled with default values for that type.

40.7.5 Redim Preserve

Click a link to view one of the topics in this section:

- [Redim Preserve Basic syntax x\(n\) \[page 1503\]](#)
- [Redim Preserve Crystal syntax x\[n\] \[page 1504\]](#)

40.7.5.1 Redim Preserve Basic syntax x(n)

Usage

Redim Preserve x(n)

Re-dimension the array x to size n, while preserving the initial values in x. x is an array and n is a positive whole number specifying the new size of x.

Examples

The following example is applicable to Basic syntax:

```
Dim x() As String  
'Initialize first three array elements  
x = Array ("a", "bb", "ccc")  
'Re-size x to 4; old values are retained  
Redim Preserve x(4)  
'initialize x(4)  
x(4) = "dddd"  
'return "dddd"
```

```
formula = x(4)
```

40.7.5.2 Redim Preserve Crystal syntax x[n]

- Use Redim Preserve x[n] (with the square brackets) only in Crystal syntax, and Redim Preserve x(n) (with the round brackets) only in Basic syntax.
- Unlike Redim Preserve x(n) in Basic syntax, you may not have multiple arrays in one Redim Preserve statement in Crystal syntax.

Usage

```
Redim Preserve x[n]
```

Re-dimension the array x to size n, while preserving the initial values in x. x is an array and n is a positive whole number specifying the new size of n.

Examples

The following example is applicable to Crystal syntax:

```
Local StringVar array x := ["a", "bb", "ccc"];  
// resize the array to size 4 while preserving the old values.  
Redim Preserve x [4];  
// now x = ["a", "bb", "ccc", "dddd"]  
x [4] := "dddd";  
x[4] // returns "dddd"
```

40.8 Pattern operators

Pattern operators allow you to compare strings to a given pattern. They are useful operators for selecting records based on part of a string ([Like pattern operator \[page 1505\]](#)) or based on a partly unknown string ([Starts with operator \[page 1504\]](#)).

40.8.1 Starts with operator

Basic and Crystal syntax.

The startsWith operator is useful for selecting records to include or exclude from your report.

Usage

```
x startsWith y  
{fieldname} startsWith "abc"
```

This operator tests to see if the contents of {fieldname} start with a character string that you specify: "abc". If the contents of the field do start with "abc", then the formula returns the value True. If the field starts with anything else, the formula returns False.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{customer.CUSTOMER NAME} startsWith "A"
```

TRUE, where {customer.CUSTOMER NAME} = ABC Ltd.

```
{customer.CUSTOMER NAME} startsWith "C"
```

FALSE, where {customer.CUSTOMER NAME} = ABC Ltd.

```
{customer.CUSTOMER NAME} startsWith "ABC"
```

TRUE, where {customer.CUSTOMER NAME} = ABC Inc. or ABC Ltd.

```
{customer.CUSTOMER NAME} startsWith "ABC"
```

FALSE, where {customer.CUSTOMER NAME} = XYZ Inc.

40.8.2 Like pattern operator

Basic and Crystal syntax.

The Like operator is useful for selecting records to include or exclude from your report.

Usage

```
x like y  
{fieldname} like "c?n*"
```

This operator tests to see if the contents of {fieldname} matches a pattern that you specify in a character string "c?n*". If the contents of the field do fit the pattern "c?n*", then the formula returns the value True. If the field starts with anything else, the formula returns False.

Use the wildcard symbols ? and * to stand for variable characters. The ? stands for a single character. The * symbol stands for any number of characters.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
{customer.FIRST NAME} like "D?n"
```

TRUE, where {customer.FIRST NAME} = Dan or Don.

```
{customer.FIRST NAME} like "D?n"
```

FALSE, where {customer.FIRST NAME} = Doug or Rob.

```
{customer.LAST NAME} like "*s?n*"
```

TRUE, where {customer.LAST NAME} = Johnson or Olson or Olsen.

```
{customer.LAST NAME} like "*s?n*"
```

FALSE, where {customer.LAST NAME} = Johnston or Smith.

40.9 Control structure operators

Formulas without control structures execute each expression in the formula exactly once when the formula is evaluated. The expressions are executed in a sequential fashion, from the first expression in the formula to the last. Control structures enable you to vary this rigid sequence. Depending upon which control structure you choose, you can skip over some of the expressions or repeatedly evaluate some expressions depending on if certain conditions hold. Control structures are the primary means of expressing business logic and typical report formulas make extensive use of them.

Related Information

[Control structures \(Basic syntax\) \[page 780\]](#)

[Control structures \(Crystal syntax\) \[page 819\]](#)

40.10 Other operators

Other operators are used to assign values to variables, indicate an order of precedence in which calculations are to be performed, separate comments from formulas, define date constants, or call a function.

40.10.1 Parentheses operators

Basic and Crystal syntax.

Usage

```
(x + y) * z
```

Performs the calculations inside the parentheses first.

Parentheses are used to control the order of precedence in which the Format Editor dialog box calculates a formula. In Basic syntax, they are also used as a string and array subscript.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
8 + 6 * 3 - 6 / 2
```

Returns 23.

```
(8 + 6) * 3 - 6 / 2
```

Returns 39.

```
(8 + 6) * (3 - 6 / 2)
```

Returns 0.

```
(8 + 6 * 3 - 6) / 2
```

Returns 10.

```
{file.SALES} - {file.COGS} - {file.T&E} * .8
```

Returns 1000, where SALES=25,000, COGS=12,000, and T&E=15,000.

```
{file.SALES} - (({file.COGS} - {file.T&E}) * .8)
```

Returns 27,400, where SALES=25,000, COGS=12,000, and T&E=15,000.

```
{file.SALES} - {file.COGS} - {file.T&E}) * .8
```

Returns -1,600, where SALES=25,000, COGS=12,000, and T&E=15,000.

40.10.2 Assignment operators

Click a link to view one of the topics in this section:

- [Basic syntax \(x = y\) \[page 1508\]](#)
- [Crystal syntax \(x := y\) \[page 1509\]](#)

40.10.2.1 Basic syntax (x = y)

You can use the assignment operator (=) and the assignment keyword "Let" (Let x=y) with only Basic syntax. Crystal syntax uses the other assignment operator (:=).

Usage

```
x = y
```

Assigns y to the variable x. x must have already been declared before the assignment. y must be of the same type as x, or of a type compatible with x.

Let x = y

Assigns y to the variable x. x must have already been declared before the assignment. y must be of the same type as x, or of a type compatible with x.

Examples

The following example is applicable to Basic syntax:

```
Dim n As Number
Dim b As Boolean
Dim c As Currency
Dim d As Date
Dim t As Time
Dim s As String
Dim numArray () As Number
'n is assigned 100
n = 100
'b is assigned the Boolean value True
b = True
```

```
'c is assigned the Currency value 5000
c = CCur (5000)
'd is assigned the Date value of Oct. 10, 1999
d = CDate ("10, 10, 1999")
't is assigned the Time value 12:50am
t = CTime ("12:50am")
's is assigned a string which is a concatenation of 2 substrings and a field
value
s = "The total sales is " & {Orders.Order Amount} & " billion."
' numArray is assigned an array of Number values
numArray = Array (10, 20, 30)
formula = s
```

Comments

- A variable must be declared in a separate statement before it is assigned any value in Basic syntax.
- Assignment only assigns a value to a variable. When a formula returns, the value assigned to the variable Formula is then formatted according to the format specified for that formula field of the corresponding type.

40.10.2.2 Crystal syntax (x := y)

You can use the assignment operator (:=) with only Crystal syntax.

Usage

```
x := n
```

Assigns the value n to the variable x. (x must have already been declared in the same formula.)

Examples

The following examples are applicable to Crystal syntax:

```
Amount := 0
```

Initializes (sets to zero) the variable named Amount.

```
Amount := 100
```

Assigns the value 100 to the variable named Amount.

```
Amount := Amount + {file.QTY}
```

Assigns the result of a calculation to the variable named Amount. The calculation adds the value of the quantity field ({file.QTY}) to the current value of the Amount variable.

```
Amount:= {file.QTY1} + {file.QTY2} + {file.QTY3}
```

Totals the three quantity fields and assigns the total to the variable named Amount.

```
Customer:= "Westside Motors"
```

Assigns the string "Westside Motors" to the variable named Customer.

```
Customer:= {customer.FIRST NAME} + {customer.LAST NAME}
```

Concatenates two fields and assigns the concatenated value of both fields to the variable named Customer.

```
Customer:= TrimRight({customer.FIRST NAME}) + {customer.LAST NAME}
```

Trims the trailing blanks from the first name field ({customer.FIRST NAME}), concatenates that field to the last name field ({customer.LAST NAME}), and assigns the concatenated value of both fields to the variable named Customer.

```
Customer:= "Mr. " + {customer.LAST NAME}
```

Concatenates the string "Mr. " with the value of the last name field {customer.LAST NAME} and assigns the concatenated value to the variable named Customer.

```
Amount:= 100; Customer:= "Westside Motors"
```

Assigns the constant 100 to the number variable named Amount and assigns the string "Westside Motors" to the string variable named Customer. You can assign values to multiple variables by separating the assignment statements with semicolons.

40.10.3 Comments

Formula comments are notes that are included with a formula to explain its design and operation. Comments do not print, and they do not affect the formula; they appear only in the Formula Workshop.

40.10.3.1 Basic syntax (Rem)

Usage

```
Rem text
```

The text that follows the operator is a comment.

Examples

The following examples are applicable to Basic syntax:

```
Rem This formula returns the total number of days worked in a year.
```

Comment is on a new line on its own.

```
formula = TotalDays : Rem Total number of days worked in a year.
```

Comment follows a formula statement and a colon.

```
formula = TotalDays : Rem This formula returns the total number of days _  
                        worked in a year.
```

Comment follows a formula statement and a colon, and it uses the line continuation character to continue onto the next line.

Comments

- The comment can be above or below the formula. It can begin on a new line on its own, or it can follow the formula on the same line if it is preceded by a colon (:).
- Can use a line continuation character ("_") within comment to carry on to next line.
- You can use the comment operator (//) only with Crystal syntax, and the other comment operators (') and (Rem) with Basic syntax.

40.10.3.2 Basic syntax (')

Usage

```
' text
```

The text that follows the operator is a comment.

Examples

The following examples are applicable to Basic syntax:

```
' This formula returns the total number of days worked in a year.
```

Comment is on a new line on its own.

```
formula = TotalDays 'Total number of days worked in a year.
```

Comment follows a formula statement.

```
formula = TotalDays 'This formula returns the total number of days _  
                      worked in a year.
```

Comment follows a formula statement, and it uses the line continuation character to continue onto the next line.

Comments

- The comment can be above or below the formula. It can begin on a new line on its own, or it can follow the formula on the same line if it is preceded by the apostrophe (').
- Can use a line continuation character ("_") within comment to carry on to next line.
- You can use the comment operator (//) only with Crystal syntax, and the other comment operators (') and (Rem) with Basic syntax.

40.10.3.3 Crystal syntax (//)

Usage

```
// text
```

The text that follows the operator is a comment; it is not treated as part of a formula, and it does not print.

Examples

The following examples are applicable to Crystal syntax:

```
{file.SALES} - {file.COST}  
// calculates the gross profit
```

{file.SALES} - {file.COST} is the formula; the Formula Editor dialog box ignores everything else.

```
({file.SALES} - {file.QUOTA}) *.06  
// calculates sales commission
```

({file.SALES} - {file.QUOTA}) *.06 is the formula; the Formula Editor dialog box ignores everything else.

```
//what follows is  
//a formula. Note that when  
//we force the line break we have  
//to begin each new line with a  
//comment operator.  
If {file.COST} > {file.SELL PRICE} Then "Loss" Else "";  
//That was a formula
```

All commented lines that appear before or after the formula are ignored.

Comments

- The comment can be above or below the formula, or it can follow the formula on the same line if it is preceded by two slashes.
- You can use the comment operator (//) with only Crystal syntax, and the other comment operators (') and (Rem) with only Basic syntax.

40.10.4 Date-time literal (#...#)

Basic and Crystal syntax.

Usage

```
# alpha-numeric string #
```

This operator converts the enclosed alpha-numeric string to a DateTime value. The alpha-numeric should contain a date, a time, or a date and time representation that can be commonly recognized. Many formats are supported.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
#10:10am#
```

Returns a DateTime value of 10:10:00am.

```
#Oct. 19, 1999 #
```

Returns a DateTime value of October 19, 1999 12:00:00am.

```
#Oct. 19, 1999 10:10am#
```

Returns a DateTime value of October 19, 1999 10:10:00am.

Comments

If a formula returns a DateTime value, the format of the returned DateTime value is determined by the DateTime formatting applicable to that formula field.

40.11 Variable declaration operators

All variable names in the formula language must be declared before you can use them in a formula.

40.11.1 Variable declarators for Basic syntax

Variable declarators let you define a variable by giving the variable a name. You can also assign a value to a variable when you declare it by using the Assignment operator. The following are variable declarators in Basic syntax.

Usage

- Dim x
- Dim x ()
- Dim x As Boolean
- Dim x As Number
- Dim x As Currency
- Dim x As Date
- Dim x As Time
- Dim x As DateTime
- Dim x As String
- Dim x As Number Range
- Dim x As Currency Range
- Dim x As Date Range
- Dim x As Time Range
- Dim x As DateTime Range
- Dim x As String Range
- Dim x () As Boolean
- Dim x () As Number
- Dim x () As Currency
- Dim x () As Date
- Dim x () As Time

- Dim x () As DateTime
 - Dim x () As String
 - Dim x () As Number Range
 - Dim x () As Currency Range
 - Dim x () As Date Range
 - Dim x () As Time Range
 - Dim x () As DateTime Range
 - Dim x () As String Range
- Declares a variable x that can hold data of a type corresponding to the variable declarator used.

Comments

- A variable declarator must be used to declare a variable before the variable can be used in a formula.
- Dim specifies a variable with local scope. You can use the keywords Local, Global, and Shared instead of Dim to specify different scopes. Local and Dim are equivalent keywords.

40.11.2 Variable declarators for Crystal syntax

Variable declarators let you define a variable by giving the variable a name. Optionally, you can also assign a value to a variable when you declare it by using the Assignment operator. The following are variable declarators in Crystal syntax.

Usage

- Local BooleanVar x
- Local NumberVar x
- Local CurrencyVar x
- Local DateVar x
- Local TimeVar x
- Local DateTimeVar x
- Local StringVar x
- Local NumberVar range x
- Local CurrencyVar range x
- Local DateVar range x
- Local TimeVar range x
- Local DateTimeVar range x
- Local StringVar range x
- Local BooleanVar array x

- Local NumberVar array x
 - Local CurrencyVar array x
 - Local DateVar array x
 - Local TimeVar array x
 - Local DateTimeVar array x
 - Local StringVar array x
 - Local NumberVar range array x
 - Local CurrencyVar range array x
 - Local DateVar range array x
 - Local TimeVar range array x
 - Local DateTimeVar range array x
 - Local StringVar range array x
- Declares a local variable x that can hold data of a type corresponding to the variable declarator used. Local variables retain their value only for the given evaluation of the formula in which they occur.

Comments

- A variable declarator must be used to declare a variable before the variable can be used in a formula.
- Local specifies a variable with local scope. You can use the keywords Global, and Shared instead of Local to specify different scopes. You can also omit the scope keyword and start the variable declaration with the type name. This is the same as declaring a Global variable.

40.12 Scope operators

The scope of a variable is an attribute of that variable specifying where you can use or reference it, e.g., within the same formula where it is defined, or within and outside of the formula where it is defined.

40.12.1 Variable scopes for Basic syntax

Declaring a scope for your variable regulates its availability through different parts of your report.

The variable scopes available are:

- Dim
- Local
- Global
- Shared

Dim specifies a variable with local scope. Local and Dim are equivalent keywords.

40.12.2 Variable scopes for Crystal syntax

Declaring a scope for your variable regulates its availability through different parts of your report.

The variable scopes available are:

- **Global**
The variable is available to formulas throughout the entire current report.
- **Shared**
The variable can be shared with a subreport as well as the entire current report.
- **Local**
The variable is specific and can only be used in the formula in which it is defined.

To use variable scopes you add the scope that you want to declare with your variable at the beginning of your variable declaration statement. You can also omit the scope keyword and start the variable declaration with the type name. This is the same as declaring a Global variable.

Examples

```
NumberVar Amount;
```

Declares a global variable named Amount that can hold any value of an integer or decimal data type.

```
Global NumberVar Amount;
```

Declares a global variable named Amount that can hold any value of an integer or decimal data type. This variable would be available to formulas throughout the current report

```
BooleanVar Outstanding;
```

Declares a global variable named Outstanding that can hold a TRUE or FALSE value.

```
Shared BooleanVar Outstanding;
```

Declares a shared variable named Outstanding that can hold a TRUE or FALSE value. This variable would be available in both the main report and any subreports contained within the main report.

```
StringVar LastName := "Adams";
```

Declares a global variable named LastName that can hold a string value and assigns the string "Adams" to that variable.

```
Local StringVar LastName : = "Adams";
```

Declares a local variable named LastName that can hold a string value and assigns the string "Adams" to that variable. This variable would be available only within the formula which it was declared.

Comments

- Your variable declaration statement is used to set aside a block of memory to hold the value assigned to the variable, and to assign a default value to that memory block. The default value assigned depends on the data type stored by the particular variable declarator used and on whether or not a value was assigned when the variable was declared.
- A variable declared in two formulas uses the same block of memory (that is, it refers to the same value.) For example:

```
//Formula #1  
NumberVar x;  
x:=5  
//Formula #2  
NumberVar x;  
x:=x+15
```

x now has the value of 20.

40.13 Expression separator

Crystal syntax.

Operator (Symbol/Word)

```
;
```

Usage

```
1+1; "abc"
```

1+1 and "abc" are two different formula expressions in a multiple expression formula. The semicolon between the expressions specifies where one expression ends and the next one begins. Without the semicolon, the two expressions would be treated as an individual expression.

Comments

This operator is used in only Crystal syntax to separate consecutive expressions. In Basic syntax, a line break or a colon is used to separate consecutive statements.

40.14 Constants

This section contains information about several different types of constants. Constants can be used in the formula language with:

- date functions
- conditional date formatting
- conditional font formatting
- text interpretation in the Report Designer

40.14.1 Date constants

Click a link to view one of the topics in this section:

- [Day of Week constants \[page 1519\]](#)
- [First Week of Year constants \[page 1520\]](#)
- [Day of Week enclosure constants \[page 1520\]](#)
- [Date and time constants \[page 1521\]](#)

40.14.1.1 Day of Week constants

The following constants are defined with the corresponding numeric values. You may use them with date functions that accept firstDayOfWeek as a parameter:

Constant	Description
crUseSystem	0 (Use the NLS API setting.)
crSunday	1
crMonday	2
crTuesday	3
crWednesday	4
crThursday	5
crFriday	6
crSaturday	7

40.14.1.2 First Week of Year constants

The following constants are defined with the corresponding numeric values. You may use them with date functions that accept firstWeekOfYear as a parameter:

Constant	Value	Description
crUseSystem	0	Use the NLS API setting.
crFirstJan1	1	Start with week in which January 1 occurs (default).
crFirstFourDays	2	Start with the first week that has at least four days in the new year.
crFirstFullWeek	3	Start with first full week of the year.

40.14.1.3 Day of Week enclosure constants

The following constants are defined to support international formatting of day of the week, such as enclosing the day of the week in round or square brackets, or not enclosing it.

Constant	Description
crDayOfWeekNotEnclosed	Do not enclose the day of a week in a Date or DateTime field with brackets.
crDayOfWeekInParentheses	Enclose the day of a week in a Date or DateTime field with round brackets. This is the default with English fonts.
crDayOfWeekInFWParentheses	Enclose the day of a week in a Date or DateTime field with full-width round brackets which are available in Japanese fonts.
crDayOfWeekInSquareBrackets	Enclose the day of a week in a Date or DateTime field with square brackets. This is the default with English fonts.
crDayOfWeekInFWSquareBrackets	Enclose the day of a week in a Date or DateTime field with full-width square brackets which are available in Japanese fonts.

40.14.1.4 Date and time constants

Constant	Description
crDateThenTime	Display the date and then the time.
crTimeThenDate	Display the time and then the date.
crDateOnly	Display only the date.
crTimeOnly	Display only the time.

40.14.2 Formatting constants

Click a link to view one of the topics in this section:

- [Font Style constants \[page 1521\]](#)
- [Text Interpretation constants \[page 1521\]](#)
- [Orientation constants \[page 1522\]](#)
- [Color constants \[page 1522\]](#)
- [Alignment constants \[page 1523\]](#)
- [Line Style constants \[page 1523\]](#)
- [Mathematics constant \[page 1523\]](#)

40.14.2.1 Font Style constants

Constant	Description
crRegular	Format font in regular type face.
crBold	Format font in bold type face.
crItalic	Format font in italic type face.
crBoldItalic	Format font in bold italic type face.

40.14.2.2 Text Interpretation constants

The following constants are defined to support importing data which is pre-formatted in Rich Text Format or HTML, and inserting it into a report as string or memo fields.

Constant	Description
crUninterpretedText	Interpret data as plain text.
crRTFText	Interpret data as Rich Text Format data.
crHTMLText	Interpret data as HTML data.

40.14.2.3 Orientation constants

Constant	Description
crPortrait	Portrait orientation
crLandscape	Landscape orientation

40.14.2.4 Color constants

Constant	Description
crBlack	The color black.
crMaroon	The color maroon.
crGreen	The color green.
crOlive	The color olive.
crNavy	The color navy.
crPurple	The color purple.
crTeal	The color teal.
crGray	The color gray.
crSilver	The color silver.
crRed	The color red.
crLime	The color lime.
crYellow	The color yellow.
crBlue	The color blue.

Constant	Description
crFuchsia	The color fuchsia.
crAqua	The color aqua.
crWhite	The color white.
crNoColor	No color.

40.14.2.5 Alignment constants

Constant	Description
crDefaultHorAligned	Set the default alignment to horizontal.
crLeftAligned	Align to the left.
crRightAligned	Align to the right.
crCenteredHorizontally	Center horizontally.
crJustified	Set alignment to be justified.

40.14.2.6 Line Style constants

Constant	Description
crNoLine	No border line.
crSingleLine	One single border line.
crDoubleLine	Double border line.
crDashedLine	Dashed border line.
crDottedLine	Dotted border line.

40.14.2.7 Mathematics constant

crPi – the Mathematical value pi which is 3.14 (if rounded to 2 decimal places).

41 Functions

Functions are built-in procedures or subroutines used to evaluate, make calculations on, or transform data.

When you specify a function, the program performs the set of operations built into the function without you having to specify each operation separately. In this way, a function is a kind of shorthand that makes it easier and less time consuming for you to create reports.

Simple, straightforward examples are provided for each function. In addition, for many functions, references are provided to specific formulas. These formulas demonstrate more advanced examples of solving real-world reporting problems.

41.1 About number formatting in the examples

In the descriptions of functions in this section, there are many examples of the values that the program returns when you use a particular function. Bear in mind that field formatting affects how formula results are displayed in your report; this is especially important when your function returns a numeric value. For example, if you use the Truncate function on the value 1.599, the formula returns 1, but what Crystal Reports displays depends on field formatting. If the formula field in your report is formatted to show two decimal places, the display in Crystal Reports is 1.00. Therefore, what you see in your report and the value shown in the function example can be different.

41.2 Math

Mathematical functions are used for a variety of mathematics-oriented calculations and operations.

41.2.1 Abs (x)

Basic and Crystal syntax.

Arguments

- x is the Number or Currency for which you want the Absolute Value returned.

Returns

Number value

Action

Abs (x) returns the absolute value of x.

Examples

```
Abs(1.50)
```

Returns 1.50.

```
Abs(-1.50)
```

Returns 1.50.

```
Abs(10 - 7)
```

Returns 3.

```
Abs(7 - 10)
```

Returns 3.

```
Rem Basic syntax
If Abs(37 - {file.FIELD}) > 1 Then
    formula = "Maintenance, Temperature Check"
End If
//Crystal syntax
If Abs(37 - {file.FIELD}) > 1 Then
    "Maintenance, Temperature Check"
Else
    "" ;
```

This example flags instances where a laboratory heat block has a temperature variation greater than plus or minus 1 degree C.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.2.2 Sgn (number)

Basic and Crystal syntax.

Argument

- number is the value you want to know the sign of.

Returns

Returns 1, 0 or -1

Action

Sgn returns the sign of a given number: 1 if number > 0, 0 if number is 0, and -1 if number < 0.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Sgn (-10)
```

Returns -1.

```
Sgn (0)
```

Returns 0.

```
Sgn (10)
```

Returns 1.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.2.3 Int (number)

Basic and Crystal syntax.

Argument

- number is value you want to round down to the nearest integer.

Returns

Whole number value

Action

Int returns the integer portion of a given number by rounding it down to the next smallest integer.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Int (123.678)
```

Returns 123.

```
Int (-123.678)
```

Returns -124.

```
Int (-123.1)
```

Returns -124.

Comments

- This function is designed to work like the Visual Basic function of the same name.

- `Fix (n)` (or `Truncate (n)`) and `Int (n)` are the same except when `n` is negative, in which case `Fix` and `Truncate` return the first integer greater than or equal to `n`, and `Int` returns the first integer smaller than or equal to `n`. For example,

```
Fix (-10.2)
```

```
Truncate (-10.2)
```

Both return -10.

```
Int (-10.2)
```

Returns -11.

See Also

- [Fix \[page 1539\]](#)
- [Truncate \[page 1533\]](#)

41.2.4 Round

Basic and Crystal syntax.

Overloads

- `Round (x)`
- `Round (x, #places)`

Arguments

- `x` is a Number or Currency to be rounded off.
- `#places` is a whole number indicating the number of decimal places `x` is to be rounded off to.

Note

The value of `#places` can range from -20 to 10 (inclusive).

Returns

Number

Action

Round rounds to the nearest whole number if the `#places` argument is excluded. If the value to the right of the decimal point is .499 or below, the program rounds to the next lowest number. If the value to the right of the decimal point is .5 or above, it rounds to the next highest number.

If the `#places` argument is used, the value in `x` is rounded to the nearest decimal place indicated by `#places`. Specifying `#places` as 0 works identical to leaving the `#places` argument off. You can also specify a negative number for `#places`; the number will be rounded to the nearest tenth (.1), hundredth (.01), or thousandth (.001), and so on.

Note

The value of `#places` can range from -20 to 10 (inclusive).

Typical uses

Use Round any time rounding off a value to a particular decimal place is more appropriate than using the original value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Round(1.23456)
```

Returns 1.

```
Round(1.499)
```

Returns 1.

```
Round(1.5000)
```

Returns 2.

```
Round(2345.23456,4)
```

Returns 2345.2346.

```
Round(2345.23456,3)
```

Returns 2345.235.

```
Round(2345.23456, 2)
```

Returns 2345.23.

```
Round(2345.23456, 0)
```

Returns 2345.

```
Round(2345.23456, -1)
```

Returns 2350.

```
Round(2345.23456, -2)
```

Returns 2300.

```
Round(2345.23456, -3)
```

Returns 2000.

```
Round(1.234499, 3)
```

Returns 1.234.

```
Round(1.234500, 3)
```

Returns 1.235.

```
Round({file.AMOUNT}, 1)
```

Returns 1854.5 where Amount = 1854.49.

```
Round({file.AMOUNT})
```

Returns 1854.00 where Amount = 1854.49.

```
Round({file.AMOUNT})
```

Returns 1855.00 where Amount = 1854.51.

```
Round({file.WEIGHT} / 100)
```

Returns 4 where Weight = 424.

```
Round({file.WEIGHT} / 100)
```

Returns 5 where Weight = 451.

```
Round((A * B) / C)
```

Returns 11 where A = 25, B = 3, and C = 7.

```
Round(file.AMOUNT, 1)
```


Returns 1854.5 where Amount = 1854.51.

```
Round({file.WAGE} * {file.HOURS WORKED}, 2)
```

Returns \$146.63 where Wage = \$5.75 and Hours worked = 25.5.

Comments

Rounding is also a feature available as a formatting option for numerical values in fields.

Be aware that using field formatting features may affect how values are used in formulas. See [ToNumber \[page 1714\]](#) and the [To currency operator \[page 1471\]](#).

See Also

- [Truncate \[page 1533\]](#)
- [Fix \[page 1539\]](#)
- [Int \(number\) \[page 1527\]](#)

41.2.5 RoundUp

Basic and Crystal syntax.

Overloads

- RoundUp (x)
- RoundUp (x, #places)

Arguments

- x is real number to be rounded.
- #places is a whole number indicating the number of decimal places x is to be rounded to.

Returns

Number

Action

RoundUp returns a number that has been rounded up. This function rounds up (away from zero).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
RoundUp( 4 . 1, 0 )
```

Returns 5.

```
RoundUp( 5 . 34 )
```

Returns 6.

```
RoundUp( 6 . 36521, 3 )
```

Returns 6.366.

```
RoundUp( - 6 . 36521, 3 )
```

Returns -6.366.

```
RoundUp( 50 . 43, -1 )
```

Returns 60.

Comments

- If #places is greater than 0 (zero), your number is rounded up to the specified number of decimal places.
- If #places is 0 or unspecified, your number is rounded up to the next integer.
- If #places is less than 0, your number is rounded up to the left of the decimal point.

See Also

- [Round \[page 1528\]](#)

41.2.6 Truncate

Truncate and [Fix \[page 1539\]](#) are equivalent functions. However, Truncate is preferred in Crystal syntax whereas Fix is preferred in Basic syntax.

Overloads

- Truncate (x)
- Truncate (x, #places)

Arguments

- x is a Number or Currency to be truncated.
- #places is a whole number indicating how many decimal places are to remain after the value is truncated. (This argument is optional.)

Returns

Number

Action

Truncate(x) returns either a fractional or whole number by truncating the number at the decimal point. If the #places argument is specified, the number is truncated to the decimal place indicated and the function returns a fractional Number. If #places is negative, the number is rounded to the first ten, hundred, and so on.

Typical uses

Use this function whenever the characters to the right of the decimal are not needed for a report or calculation.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Truncate(1.23456)
```

Returns 1.

```
Truncate(1.499)
```

Returns 1.

```
Truncate(1.599)
```

Returns 1.

```
Truncate(1.999)
```

Returns 1.

```
Truncate(12346.33, 1)
```

Returns 12346.3.

```
Truncate(12345.33, -2)
```

Returns 12300.00.

If you have 147 golf balls in stock and want to know how many dozen are available for sale, your calculation is $147/12=12.25$. 12.25 (truncated) = 12 dozen available for sale. If you sell balls only by the dozen, the .25 dozen you truncated is unimportant.

```
Truncate({file.BALL INVENTORY} / 12)
```

Returns 12 where Ball inventory = 147 ($147 / 12 = 12.25$, 12.25 truncated = 12).

```
Truncate({file.BALL INVENTORY} / 12)
```

Returns 12 where Ball inventory = 155 ($155/12 = 12.92$, 12.92 truncated = 12).

```
Truncate({file.BALL INVENTORY} / 12)
```

Returns 13 where Ball inventory = 157 ($157/12 = 13.08$, 13.08 truncated = 13).

Comments

- This is not a rounding function; Truncate simply deletes all characters to the right of the decimal point. See the [Round \[page 1528\]](#) for an explanation on rounding.

- Truncate (n) and [Int \(number\) \[page 1527\]](#) are synonymous except when n (number) is negative, in which case Truncate returns the first integer greater than or equal to n, and Int returns the first integer smaller than or equal to n. For example,

```
Truncate (-10.2)
```

Returns -10.

```
Int (-10.2)
```

Returns -11.

- Truncating is also a feature available as a formatting option for numerical values in fields. Be aware that using field formatting features may affect how values are used in formulas. For more information on converting to Numbers and Currency, see the [ToNumber \[page 1714\]](#) and [To currency operator \[page 1471\]](#).

See Also

- [Int \(number\) \[page 1527\]](#)

41.2.7 MRound

Basic and Crystal syntax.

Overloads

- MRound (x)
- MRound (x, multiple)

Arguments

- x is a value to be rounded.
- multiple is a multiple that you want to round the x value to.

Returns

Number

Action

MRound returns a number that has been rounded to the multiple that you specify. This function rounds up (away from zero) if the remainder of the division is greater than or equal to half of the value of the multiple.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
MRound(10, 3)
```

Returns 9.

```
MRound(-10, -3)
```

Returns -9.

```
MRound(1.3, 0.2)
```

Returns 1.4.

```
MRound(5, -2)
```

Returns 6.

```
MRound(-5, 2)
```

Returns -6.

Comments

- Crystal Reports ignores the sign of the multiple, and the result of the function takes the sign of the first parameter.

41.2.8 Ceiling

Basic and Crystal syntax.

Overloads

- Ceiling (x)

- Ceiling (x, multiple)

Arguments

- x is a value to be rounded.
- multiple is a multiple that you want to round the x value to.

Returns

Number

Action

Ceiling returns a number that has been rounded to the multiple that you specify. This function rounds up (away from zero).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Ceiling(3.5,1)
```

Returns 4.

```
Ceiling(-2.5,-2)
```

Returns -4.

```
Ceiling(-2.5,2)
```

Returns -4.

```
Ceiling(5.43,.05)
```

Returns 5.45.

```
Ceiling(.43,-1)
```

Returns 1.

Comments

- Crystal Reports ignores the sign of the multiple, and the result of the function takes the sign of the first parameter.
- Values are always rounded up when they are adjusted away from zero (without regard to the sign of the value). If number is an exact multiple of multiple, no rounding occurs.

41.2.9 Floor

Basic and Crystal syntax.

Overloads

- Floor (x)
- Floor (x, multiple)

Arguments

- x is a value to be rounded.
- multiple is a multiple that you want to round the x value to.

Returns

Number

Action

Floor returns a number that has been rounded to the multiple that you specify. This function rounds down (away from zero).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Floor(3.5, 1)
```

Returns 3.

```
Floor(-2.5, -2)
```

Returns -2.

```
Floor(-2.5, 2)
```

Returns -2.

```
Floor(5.43, .05)
```

Returns 5.40.

```
Floor(.43, -1)
```

Returns 0.

Comments

- Crystal Reports ignores the sign of the multiple, and the result of the function takes the sign of the first parameter.
- Values are always rounded down when they are adjusted away from zero (without regard to the sign of the value). If number is an exact multiple of multiple, no rounding occurs.
- 0 is not allowed as the multiple argument. Using 0 in this way results in an error. However, `Floor(0, 0)` is allowed, and it returns 0.

41.2.10 Fix

Fix and [Truncate \[page 1533\]](#) are equivalent functions. However, Fix is preferred in Basic syntax whereas Truncate is preferred in Crystal syntax.

Overloads

- Fix (number)
- Fix (number, #places)

Arguments

- number is the Number value to be truncated; it can be positive, 0 or negative.
- #places is an optional Number indicating the number of decimal places to be truncated to. If omitted, 0 is assumed.

Returns

Whole number value which can be positive, 0 or negative.

Action

Fix truncates a number to the specified number of decimal places and returns it. If #places is omitted, 0 is assumed.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Fix (123.678)
```

Returns 123.

```
Fix (-123.678)
```

Returns -123.

```
Fix (123.678,1)
```

Returns 123.6.

```
Fix (123.678,2)
```

Returns 123.67.

Comments

- This function with a single parameter, Fix (number), is designed to work like the Visual Basic function of the same name.

- `Fix (n)` and `Int (number)` [\[page 1527\]](#) work the same except when `n` is negative, in which case `Fix` returns the first integer greater than or equal to `n`, and `Int` returns the first integer smaller than or equal to `n`. For example,

```
Fix (-10.2)
```

Returns -10.

```
Int (-10.2)
```

Returns -11.

41.2.11 Remainder (num, denom)

Basic and Crystal syntax.

Arguments

- Numerator is a fractional value.
- Denominator is a fractional value.

Returns

Fractional Value

Action

`Remainder` returns the remainder after the numerator (dividend) has been divided by the denominator (divisor). In a typical division situation, the program expresses a quotient as a whole number (if any) and up to six decimal places. When using `Remainder`, however, the program performs the division internally, determines the whole number quotient and the remainder, and returns only the remainder.

Typical uses

You can use this function in making conversions (feet to miles, units to grosses, and so on). You can also use it to select every `n`th item out of an array.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
Remainder(12,5)
```

Returns 2.

```
Remainder(16,5)
```

Returns 1.

```
Rem Basic syntax
If Remainder ({file.EXAM#}, 7) = 0 Then
    formula = "*****"
End If
//Crystal syntax
If Remainder({file.EXAM#}, 7) = 0 Then
    "*****"
Else
    ""
```

This flags every 7th exam for grading by a second party.

```
ToText(Truncate({file.DAYS}/7)) + " week(s), " +
ToText(Remainder({file.DAYS},7)) + " day(s)"
```

Returns "9 week(s), 1 day(s)". Converts days to weeks and days. For example, if the field had a value of 64 days, the formula would return the value of "9 week(s), 1 day(s)."

See Also

- [Modulus \(x Mod y\) \[page 1469\]](#)

41.2.12 Sin (number)

Basic and Crystal syntax.

Arguments

- Number value that represents an angle in radians.

Returns

Number value between -1 and 1

Action

Sin returns a number specifying the sine of an angle given in radians. It takes a right-angle triangle, and returns the length of the side opposite to the specified angle divided by the length of the hypotenuse.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Sin (1)
```

Returns 0.8415 (rounded to 4 decimals). This is the sine of 1 radian.

```
Sin (30 * crPi / 180)
```

Returns 0.5. This is the sine of 30 degrees. Before taking the sine, the angle is converted to radians by multiplying by `crPi / 180`.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The range of values returned by Sin is between -1 and 1.

41.2.13 Cos (number)

Basic and Crystal syntax.

Arguments

- Number value which is an angle in number of radians.

Returns

Number value between -1 and 1

Action

Cos returns a Number specifying the cosine of an angle given in radians. It takes a right-angle triangle, and returns the length of the side adjacent to the specified angle divided by the length of the hypotenuse.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Cos (1)
```

Returns 0.5403 (rounded to 4 decimals). This is the cosine of 1 radian.

```
Cos (60 * crPi / 180)
```

Returns 0.5. This is the cosine of 60 degrees. Before taking the cosine, the angle is converted to radians by multiplying by `crPi / 180`.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The range of values returned by Cos is between -1 and 1.

41.2.14 Tan (number)

Basic and Crystal syntax.

Arguments

- Number value that represents an angle in radians.

Returns

Number value

Action

Tan returns a Number specifying the tangent of an angle given in radians. It takes a right-angle triangle, and returns the length of the side opposite the specified angle divided by the length of the side adjacent to the angle.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Tan (1)
```

Returns 1.5574 (rounded to 4 decimals). This is the tangent of 1 radian.

```
Tan (45 * crPi / 180)
```

Returns 1. This is the tangent of 45 degrees. Before taking the tangent, the angle is converted to radians by multiplying by crPi / 180.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.2.15 Atn (number)

Basic and Crystal syntax.

Arguments

- Number value

Returns

Number value which is an angle specified in number of radians

Action

Atn returns a Number specifying the arctangent of the given Number argument. In other words, it returns the angle whose tangent is the given Number argument.

Example

The following example is applicable to both Basic and Crystal syntax:

```
Atn (1)
```

Returns an angle of 0.7854 radians (rounded to 4 radians). To convert this angle to degrees, multiply by 180 / crPi. For instance, Atn (1) * 180 / crPi is 45 degrees.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The range of values returned by Atn is between $-\pi/2$ and $\pi/2$ radians.

41.2.16 Pi

Basic and Crystal syntax.

Returns the Mathematical value pi, which is 3.14 (if rounded to 2 decimal places).

41.2.17 Sqr (number)

Basic and Crystal syntax.

Arguments

- Number value greater than or equal to 0.

Returns

Number value

Action

Sqr returns the square root of a given number.

Example

The following example is applicable to both Basic and Crystal syntax:

```
Sqr (100)
```

Returns 10.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.2.18 Exp (number)

Basic and Crystal syntax.

Arguments

- Number value that specifies a power.

Returns

Number value.

Action

Exp returns a Number specifying *e* (the base of natural logarithms) raised to a power. The value of *e* is approximately 2.718282.

Example

The following example is applicable to both Basic and Crystal syntax:

```
Exp (1.5)
```

Returns the exponentiation of 1.5, which is approximately 4.48169.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- Numeric overflow occurs if the given number argument is larger than approximately 705.

41.2.19 Log (number)

Basic and Crystal syntax.

Arguments

- Number value

Returns

Number value

Action

Log returns a Number specifying the natural logarithm of a given number. The natural logarithm is the logarithm to the base *e*, where *e* is approximately 2.718282.

Example

The following example is applicable to both Basic and Crystal syntax:

```
Log (1.5)
```

Returns 0.4055 (rounded to 4 decimals).

Comments

This function is designed to work like the Visual Basic function of the same name.

This function can calculate a log with any base when you employ this logic:

```
LogAnyBase (x, base) = log(x)/log(base)
```

(*x* represents the number for which you want to calculate the log; *base* represents the base you want to use.)

For example, for log base 10, you would enter this formula text:

```
log(x)/log(10)
```

41.2.20 Rnd

Basic and Crystal syntax.

Overloads

- Rnd ()
- Rnd (seed)

Arguments

- seed is an optional Number value argument.

Returns

A Number value.

Action

Rnd returns a random number greater than or equal to 0 and less than 1. If seed is equal to 0, Rnd returns the random number that was returned from the previous call to Rnd. If seed is not supplied or is greater than 0, then Rnd returns the next random number in the internally generated sequence of random numbers. If seed is less than 0, then Rnd uses this value of seed to start a new random number sequence and returns the first value in this sequence.

Typical uses

Used when your formula requires a randomly generated number such as for statistical calculations or for selecting records at random to limit the data in a report.

Comments

- This function is similar to the Visual Basic function of the same name.
- You can call Rnd without ever starting a new random number sequence by specifying a negative seed argument. If you do so, an internal seed will be generated using the system clock.
- The reason to start a new random number sequence by calling Rnd with a negative seed argument, and then making subsequent calls to Rnd without an argument (or with a positive argument) is so that the report will look exactly the same every time it is previewed. In other words, it allows you to make use of random numbers, but get a reproducible result.

41.3 Miscellaneous

41.3.1 GetValueDescriptions (x)

This function can be used in both Basic and Crystal syntax formulas. It returns the description(s) of selected value(s) for the input parameter.

Arguments

x is a parameter field, it can be of any data type.

Returns

- String value, if the relevant parameter field only allow single discrete input.
- Array of string, if the relevant parameter field allow multiple discrete input.
- String range, if the relevant parameter field allow range input.
- Array of string range, if the relevant parameter field allow multiple range input.

Example

```
GetValueDescriptions ({?MyCountryVariable})
```

Returns ["Canada", "China", "India"] (An String Array)

41.4 Summary

The Summary functions are all used to summarize field data in several different ways. Because of their unique summarization features, they have been grouped together as summary functions.

Summary functions can be designed to perform operations on group data.

ⓘ Note

Not all arithmetic group functions are available for use with all data types.

ⓘ Note

In order to use a summary function to insert a group field in a formula, you must have already entered a group field in your report with identical parameters: same field, same sort and group by (trigger) field, and same action (for example, average, count).

41.4.1 Conditions for summary functions

Here is a list of the conditions for each summary function:

Boolean conditions	• on any change • on change to yes • on change to no • on every yes • on every no • on next is yes • on next is no
Date conditions	• for each day • for each week • for each two weeks • for each half month • for each month • for each quarter • for each half year • for each year
Time conditions	• for each second • for each minute • for each hour • for AM/PM

41.4.2 Sum

Basic and Crystal syntax.

Overloads

- Sum (fld)
- Sum (fld, condFld)
- Sum (fld, condFld, cond)
- Sum (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to add the values that appear in your report. For example:

- If a sales report includes a field that shows the amount of each order, you can compute the sum of all the orders that appear on the report (a grand total sum).
- If you break orders into groups (for example, orders grouped by the state that they come from), you can compute the sum of the order amounts per group (in this case, per state).
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can compute the sum of the order amounts per group based on a specific change in the date or Boolean field (in this case, per month).
- If you specify a set of individual values, you can also compute the sum of the values in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Sum({file.QTY})
```

Calculates the sum of all values in the QTY field.

```
>Sum({orders.AMOUNT}, {orders.CUSTOMER ID})
```

Sums (totals) the orders in each group of orders in the Amount field. The orders are separated into groups whenever the value in the Customer ID field changes.

```
Sum({orders.AMOUNT}, {orders.DATE}, "monthly") % Sum({orders.AMOUNT })
```

Groups values in the Amount field by month, and calculates the sum of the values for each month group as a percentage of the sum of the values for the entire report.

```
Sum([file.AMOUNT], {file.PRICE}, {file.COST})
```

Sum of values in the Amount, Price, and Cost fields.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.4.3 Average

Basic and Crystal syntax.

Overloads

- Average (fld)
- Average (fld, condFld)
- Average (fld, condFld, cond)
- Average (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to average the values that appear in your report. For example:

- If a sales report includes a field that shows the amount of each order, you can compute the average of all the orders that appear on the report (a grand total average).
- If you break orders into groups (for example, orders grouped by the state that they come from), you can compute the average order per group (in this case, per state).
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can compute the average order per group based on a particular change in the date or Boolean field (in this case, per month).
- If you specify an array of individual values, you can also compute the average value in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Average( {orders.ORDER AMOUNT} )
```

Calculates the average of all values in the Amount field.

```
Average ( {orders.ORDER AMOUNT}, {customer.CUSTOMER ID} )
```

Finds the average amount of a sale made to each customer.

The following example is applicable to Crystal syntax:

```
Average ( [10,12,32,48] )
```

Calculates the average of the values in an array of constants.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.4 StdDev

Basic and Crystal syntax.

Overloads

- StdDev (fld)
- StdDev (fld, condFld)
- StdDev (fld, condFld, cond)
- StdDev (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to find the standard deviation of a set of values in your report. For example:

- You can calculate the grand total standard deviation for all values in a field.
- You can calculate the standard deviation for all values within a group (for example, sales grouped by the state that they come from).
- You can calculate the standard deviation for all values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).
- If you specify a set of individual values, you can compute the standard deviation of the values in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
StdDev({file.RESULTS})
```

Calculates the standard deviation of all values in the Result field.

```
StdDev({orders.AMOUNT}, {orders.CUSTOMER ID})
```

Calculates the standard deviation for the orders in each group of orders in the Orders field. The orders are separated into groups whenever the value in the Customer field changes.

```
StdDev({file.RESULTS}, {file.DATE}, "daily")
```

Calculates the variation for each group of laboratory test results in the Results field. The test results are separated into groups whenever the value in the Date field changes to a new day.

```
StdDev({file.SCORES}, {file.NATURALIZED}, "any change")
```

Groups values in the Scores field based on whether or not the test taker is a naturalized citizen, and calculates the standard deviation for each group of scores.

The following example is applicable to Crystal syntax:

```
StdDev([(file.QTY1} * {file.PRICE1}),({file.QTY2} * {file.PRICE2}),({file.QTY3} * {file.PRICE3}), ({file.QTY4} * {file.PRICE4})])
```

Returns 36.60 where Qty1 = 2, Price1 = 10.00, Qty2 = 2, Price2 = 2.00, Qty3 = 10, Price3 = 3.00, and Qty4 = 8, Price4 = 11.00.

Comments

Standard deviation is calculated using the following technique:

- It calculates the average (mean) value for the items in the sample.
- It subtracts the average value from the value of each item.
- It squares the difference for each item.
- It adds the squared differences for all of the items in the sample.
- It divides the sum by one less than the number of items in the sample (N - 1). The result is the variance. Compare this step to the [PopulationStdDev \[page 1557\]](#).
- It calculates the square root of the variance to arrive at the standard deviation.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.5 PopulationStdDev

Basic and Crystal syntax.

Overloads

- PopulationStdDev (fld)
- PopulationStdDev (fld, condFld)
- PopulationStdDev (fld, condFld, cond)
- PopulationStdDev (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field.
For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to find the population standard deviation of a set of values in your report. For example:

- You can calculate the grand total population standard deviation for all values in a field.
- You can calculate the population standard deviation for all values within a group (for example, sales grouped by the state that they come from).
- You can calculate the population standard deviation for all values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).
- If you specify a set of individual values, you can compute the population standard deviation of the values in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
PopulationStdDev({file.Scores})
```

Calculates the population standard deviation of all values in the Scores field for the entire report.

```
PopulationStdDev({orders. AMOUNT}, {orders.CUSTOMER ID})
```

Calculates the population standard deviation for the orders in each group of orders in the Amount field. The orders are separated into groups whenever the value in the Customer ID field changes.

```
PopulationStdDev({file.RESULTS}, {file.DATE}, "daily")
```

Calculates the variation for each group of laboratory test results in the Results field. The test results are separated into groups whenever the value in the Date field changes to a new day.

The following example is applicable to Crystal syntax:

```
PopulationStdDev([file.QTY1], {file.QTY2}, {file.QTY3}, {file.QTY4})
```

Returns 3.57 where Qty1 = 2, Qty2 = 2, Qty3 = 10, and Qty4 = 8.

Comments

Population standard deviation is calculated using the following technique:

- It calculates the average (mean) value for the items in the population.
- It subtracts the average value from the value of each item.
- It squares the difference for each item.
- It adds the squared differences for all of the items in the population.
- It divides the sum by the number of items in the population (N). The result is the population variance. Compare this step to the [StdDev \[page 1555\]](#).
- It calculates the square root of the population variance to arrive at the population standard deviation.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.4.6 Variance

Basic and Crystal syntax.

Overloads

- Variance (fld)
- Variance (fld, condFld)

- Variance (fld, condFld, cond)
- Variance (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to find the variance of a set of values in your report. For example:

- You can calculate the grand total variance for all values in a field.
- You can calculate the variance for all values within a group (for example, sales grouped by the state that they come from).
- You can calculate the variance for all values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).
- If you specify a set of individual values, you can compute the variance of the values in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Variance({file.AMOUNT})
```

Calculates the variance of all values in the Amount field.

```
Variance({file.RESULTS}, {file.DATE}, "daily")
```

Calculates the variation for each group of laboratory test results in the Results field. The test results are separated into groups whenever the value in the Date field changes to a new day.

The following examples are applicable to Crystal syntax:

```
Variance({file.Scores}, {file.Teachers}) % Variance({file.Scores})
```

Groups values in the Scores field by teacher, and calculates the variance for each teacher group as a percentage of the variance of the values for the entire report (for all teachers listed).

```
Variance([file.QTY1], {file.QTY2}, {file.QTY3}, {file.QTY4})
```

Returns 17.00 where Qty1 = 2, Qty2 = 2, Qty3 = 10, and Qty4 = 8.

Comments

Variance is calculated using the following technique:

- It calculates the average (mean) value for the items in the sample.
- It subtracts the average value from the value of each item.
- It squares the difference for each item.
- It adds the squared differences for all of the items in the sample.
- It divides the sum by one less than the number of items in the sample (N - 1). The result is the variance. Compare this final step to the [PopulationVariance \[page 1561\]](#).

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.7 PopulationVariance

Basic and Crystal syntax.

Overloads

- PopulationVariance (fld)
- PopulationVariance (fld, condFld)
- PopulationVariance (fld, condFld, cond)
- PopulationVariance (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of Number or Currency values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to find the population variance of a set of values in your report. For example:

- You can calculate the grand total population variance for all values in a field.
- You can calculate the population variance for all values within a group (for example, sales grouped by the state that they come from).
- You can calculate the population variance for all values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).
- If you specify a set of individual values, you can compute the population variance of the values in the set. For information on this kind of summary, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

For examples of each PopulationVariance function, click on the corresponding summary function.

```
PopulationVariance({file.CENSUS})
```

Calculates the variance of all values in the Census field.

```
PopulationVariance({file.RESULTS}, {file.PROCEDURE})
```

Calculates the population variation for each group of laboratory test results in the Results field. The test results are separated into groups whenever the value in the Procedure field changes.

```
PopulationVariance({file.SCORES}, {file.NATURALIZED}, "any change")
```

Groups values in the Scores field based on whether or not the test taker is a naturalized citizen, and calculates the population variance for each group of scores.

The following examples are applicable to Crystal syntax:

```
PopulationVariance({file.SCORES}, {file.TEACHERS}) %  
PopulationVariance({file.SCORES})
```

Groups values in the Scores field by teacher, and calculates the population variance for each teacher group as a percentage of the population variance of the values for the entire report (for all teachers listed).

```
PopulationVariance([2,4,6,8,10])
```

Returns 8.

Comments

Population variance is calculated using the following technique:

- It calculates the average (mean) value for the items in the population.
- It subtracts the average value from the value of each item.
- It squares the difference for each item.
- It adds the squared differences for all of the items in the population.
- It divides the sum by the number of items in the population (N). The result is the population variance. Compare this final step to the [Variance \[page 1559\]](#).

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.8 Maximum

Basic and Crystal syntax.

Overloads

- Maximum (fld)
- Maximum (fld, condFld)
- Maximum (fld, condFld, cond)
- Maximum (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array or range of values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to find the maximum value that appears in a set of values. For example:

- If a sales report includes a field that shows the amount of each order, you can find the highest order amount of all the orders that appear on the report (a grand total maximum amount).
- If you break orders into groups (for example, orders grouped by the state that they come from), you can find the highest order amount per group (in this case, per state).
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can find the highest order amount per group based on a specific change in the date or Boolean field (in this case, per month).
- If you specify a set of individual values, you can find the highest value in the set. For information on finding this kind of maximum, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Maximum({orders.ORDER AMOUNT})
```

Returns the highest value in the Amount field.

```
Maximum({orders.AMOUNT}, {orders.CUSTOMER ID})
```

Identifies the largest order in each group of orders in the Amount field (the largest order for each customer). The orders are separated into groups whenever the value in the Customer ID field changes.

```
Maximum({orders.AMOUNT}, {orders.ORDER DATE}, "monthly")
```

Identifies the largest order in each group of orders in the Amount field (the largest order for each month). The orders are separated into groups whenever the value in the Date field changes to a new month.

The following example is applicable to Crystal syntax:

Maximum also allows you to set a floor on a calculation. For example: `Maximum([file.BALANCE], 500)` sets a floor of 500 on the calculation. The expression will always return the balance unless the balance drops below 500. Then it will return the floor amount of 500. Thus:

```
Maximum([file.PROFITS], 500) = {file.PROFITS}
```

Where Profits > 500.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

Note

When using this function, you can use the [HasUpperBound \(x\) \[page 1827\]](#) function to test if the range will have a Maximum before taking the Maximum.

41.4.9 Minimum

Basic and Crystal syntax.

Overloads

- Minimum (fld)
- Minimum (fld, condFld)
- Minimum (fld, condFld, cond)
- Minimum (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array or range of values that can be evaluated by the function being used.

Returns

Fractional Number

Action

Enables you to find the minimum value that appears in a set of values. For example:

- If a sales report includes a field that shows the amount of each order, you can find the lowest order amount of all the orders that appear on the report (a grand total minimum amount).
- If you break orders into groups (for example, orders grouped by the state that they come from), you can find the lowest order amount per group (in this case, per state).
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can find the lowest order amount per group based on a specific change in the date or Boolean field (in this case, per month).
- If you specify a set of individual values, you can find the lowest value in the set. For information on finding this kind of minimum, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Minimum({file.QTY})
```

Returns the lowest value in the QTY field.

```
Minimum({orders.AMOUNT}, {orders.CUSTOMER ID})
```

Identifies the smallest order in each group of orders in the Amount field (the smallest order for each customer). The orders are separated into groups whenever the value in the Customer ID field changes.

```
Minimum({orders.AMOUNT}, {orders.ORDER DATE}, "monthly")
```

Identifies the smallest order in each group of orders in the Amount field (the smallest order for each month). The orders are separated into groups whenever the value in the Date field changes to a new month.

The following example is applicable to Crystal syntax:

Minimum also allows you to set a ceiling on a calculation. For example: `Minimum([file.COMMISSION], 2500)` returns the commission accrued up to a cap or ceiling of 2500. Once accrued commission passes the \$2500 mark, this expression returns 2500. Thus:

```
Minimum([file.COMMISSION], 2500)
```

Returns 1575 where commission = 1575.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

Note

When using this function, you can use the [HasLowerBound \(x\) \[page 1826\]](#) function to test if the range will have a Minimum before taking the Minimum.

41.4.10 Count

Basic and Crystal syntax.

Overloads

- Count (fld)
- Count (fld, condFld)
- Count (fld, condFld, cond)
- Count (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to count the values that appear in your report (for a specified field). For example:

- If a sales report includes all orders made and the amount of each order, you can compute the total number of orders that appear on the report (a grand total count).
- If you break orders into groups (for example, orders grouped by the state that they come from), you can compute the number of orders per group (in this case, per state).
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can compute the number of orders per group based on a particular change in the date or Boolean field (in this case, per month).
- If you specify a set of individual values, you can compute the number of values in the set. For information on this kind of counting, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Count({orders.AMOUNT}, {orders.CUSTOMER ID})
```

This formula counts the number of orders in each group of orders in the Amount field (the total orders for each customer). The orders are separated into groups whenever the value in the Customer ID field changes.

```
Count({orders.ORDER AMOUNT}, {orders.ORDER DATE}, "monthly")
```

Counts the number of orders in each group of orders in the Amount field (the total orders for each month). The orders are separated into groups whenever the value in the Date field changes to a new month.

The following examples are applicable to Crystal syntax:

```
If Count({orders.ORDER ID}) >= 100 Then
    "Congratulations on meeting your quota!"
Else
    ""
```

Prints the congratulatory message if the number of orders is 100 or more, and prints nothing if the number of orders is less than 100.

```
Count([1, 2, 3, 4, 5])
```

Returns 5. Counts the total number of values in the array.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.11 DistinctCount

Basic and Crystal syntax.

Overloads

- DistinctCount (fld)
- DistinctCount (fld, condFld)
- DistinctCount (fld, condFld, cond)
- DistinctCount (x)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- x is an array of values that can be evaluated by the function being used.

Returns

Number

Action

Enables you to get a distinct count of the values that appear in your report. For example:

- If a sales report includes all orders made by customers, you can compute the total number of distinct customers that appear in the report (a grand total distinct count), excluding any duplicate records. If a customer made more than one order, the duplicate occurrences of that customer are ignored.
- If you break orders into groups (for example, orders grouped by the state that they come from), you can compute the number of distinct customers per group (in this case, per state). Any customers that made more than one order and appear more than once in a group are only counted once.
- If you break orders into date or Boolean groups (for example, orders grouped by the month in which they were placed), you can compute the number of distinct customers in each group based on a particular change in the date or Boolean field (in this case, the number of customers that placed orders each month). If a customer placed more than one order within the month, duplicate instances of that customer are ignored.
- If you specify a set of individual values, you can compute the number of distinct values in the set. Duplicate values in the set are only counted once. For information on this kind of distinct count, see [Array summary functions \[page 1824\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DistinctCount({customer.REGION})
```

Counts the number of different States represented in the Region field, ignores any repetitions.

```
DistinctCount({customer.CITY}, {customer.REGION})
```

Calculates the number of different cities in each State group.

```
DistinctCount({orders.CUSTOMER ID}, {orders.ORDER DATE}, "weekly")
```

Counts the number of different customers contacted, follow up contacts are excluded. Customers are separated into groups whenever the value in the Date field changes to a new week.

The following example is applicable to Crystal syntax:

```
DistinctCount([1,3,5,3,2,5])
```

Returns 4. Counts the number of distinct values in the array. Duplicate values are ignored.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

Comments

DistinctCount counts the number of distinct values in a field. Duplicate values are ignored.

41.4.12 Correlation

Basic and Crystal syntax.

Overloads

- Correlation (fld, fld)
- Correlation (fld, fld, condFld)
- Correlation (fld, fld, condFld, cond)

Arguments

- fld is any numeric field.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Enables you to calculate the correlation of the specified fields (that is, the degree to which the fields vary in the same manner). For example:

- You can calculate the correlation of two fields, for all the records in the report.
- You can calculate the correlation of two fields for all values within a group (for example, sales grouped by the state that they come from).
- You can calculate the correlation of two fields for all the values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Correlation({Customer.CUSTOMER ID}, {Customer.CREDIT ID})
```

Calculates the correlation between the Customer ID and Credit ID fields.

```
Correlation({orders.CUSTOMER ID}, {orders.CREDIT ID}, {Customer.REGION})
```

Calculates the correlation between the Customer ID and Credit ID fields, for each region.

```
Correlation({orders.CUSTOMER ID}, {orders.CREDIT ID}, {Customer.REGION},  
"monthly")
```

Calculates the correlation between the Customer ID and Credit ID fields for all the values within each Region group, for each month.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

Comments

The correlation will always be a number between -1 and 1, unless the correlation is undefined, in which case the function is assigned a null value (0). If the correlation is 0, the fields are uncorrelated.

41.4.13 Covariance

Basic and Crystal syntax.

Overloads

- Covariance (fld, fld)
- Covariance (fld, fld, condFld)
- Covariance (fld, fld, condFld, cond)

Arguments

- fld is any numeric field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Covariance is the measure of the linear relation between paired variables (that is, the tendency of two fields to vary together). Fields are covariant when they vary according to a specific mathematical relationship. The circumference of a circle and the radius of a circle are covariant.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Covariance({table.FIELD1}, {table.FIELD2})
```

Calculates the covariance of the two fields and returns the covariance as a fractional number.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

Comments

Covariance is calculated using the following technique:

- It calculates the average (mean) value for the items in each sample.
- It subtracts the average value from the value of each item, in both samples.
- It multiplies the differences for each pair of items.
- It calculates the average (mean) value for all the products of that multiplication. The result is the covariance.

41.4.14 WeightedAverage

Basic and Crystal syntax.

Overloads

- WeightedAverage (fld, fld)
- WeightedAverage (fld, fld, condFld)
- WeightedAverage (fld, fld, condFld, cond)

Arguments

- fld is any valid numeric database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Enables you to calculate the weighted average of the specified fields. When you calculate a weighted average, you are actually calculating the average of one field and then using the values in another field to "weigh" the contribution of each value in the first field to the average. In a normal average, all the weights are equal to 1.

For example:

- You can calculate the weighted average of two fields.
- You can calculate the weighted average of two fields, for all the values within a group (for example, sales grouped by the state that they come from).
- You can calculate the weighted average of two fields, for all the values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).

Examples

The following example is applicable to both Basic and Crystal syntax:

```
WeightedAverage({table.FIELD1}, {table.FIELD2})
```

Returns 3.5, where the two fields have four values each. Field1 has values of 3, 2, 5, and 1. Field2 has values of 0, 1, 1, and 0. WeightedAverage calculates as follows: $(3*0 + 2*1 + 5*1 + 1*0) / \text{sum}\{\text{table.FIELD2}\} = 3.5$.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.4.15 Median

Basic and Crystal syntax.

Overloads

- Median (fld)
- Median (fld, condFld)
- Median (fld, condFld, cond)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Calculates the median of the given numeric fields. The median is the middle value in a sequence of numeric values (or the average of the two middle values in an even-numbered sequence of values).

For example:

- You can calculate the median of all the values in a field.
- You can calculate the median of all the values in the field, within a group (for example, sales grouped by the state that they come from).
- You can calculate the median of all the values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Median({Customer.LAST YEAR'S SALES})
```

Returns \$29087, where the values of the Last Year's Sales field are \$27014, \$28000, \$29087, \$34500, and \$48260.

```
Median({Customer.LAST YEAR'S SALES}, {Customer.REGION})
```

Groups the Last Year's Sales field by region and returns the median of the Last Year's Sales field per region.

```
Median({orders.ORDER AMOUNT}, {Customer.REGION}, "monthly")
```

Groups values in the Amount field by region, then returns the median of the order amount for each region per month.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.16 PthPercentile

Basic and Crystal syntax.

Overloads

- PthPercentile (P, fld)
- PthPercentile (P, fld, condFld)
- PthPercentile (P, fld, condFld, cond)

Arguments

- P is any integer between 0 and 100.
- fld is any Number or Currency field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field.

For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number or Currency value

Action

Calculates the value for a specified percentile (P) in a Number or Currency field.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
PthPercentile(20, {Customer.LAST YEAR'S SALES})
```

Returns \$2302 as the value at the 20th percentile (if 20% of {Customer.LAST YEAR'S SALES} are less than \$2302)

```
PthPercentile(P, {Customer.LAST YEAR'S SALES}, {Customer.REGION})
```

Groups the Last Year's Sales field by region and returns the value at percentile P in the Last Year's Sales field per region.

```
PthPercentile(P, {orders.ORDER AMOUNT}, {Customer.REGION}, "monthly")
```

Groups values in the Amount field by region, then returns the value at percentile P in the Amount field for each region, per month.

Comments

P = 50 (the 50th Percentile) is the same value returned by the [Median \[page 1575\]](#) function (the median value).

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.17 NthLargest

Basic and Crystal syntax.

Overloads

- `NthLargest (N, fld)`
- `NthLargest (N, fld, condFld)`
- `NthLargest (N, fld, condFld, cond)`

Arguments

- N is any integer from 1 to 100 (inclusive).
- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Determines the Nth largest value in a given field, either for the entire report or for each instance of the (condFld) group.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
NthLargest(1, {Customer.CUSTOMER ID})
```

Returns 50, where the Customer ID field contains numerical values ranging from 12 to 50.

```
NthLargest(5, {Customer.CUSTOMER NAME}, {Customer.REGION})
```

Returns the fifth largest value in the Customer Name field, per region.

```
NthLargest(1, {orders.ORDER AMOUNT}, {Customer.REGION}, "monthly")
```

Groups values in the Amount field by region, then returns the largest value in the Amount field for each region, per month.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.18 NthSmallest

Basic and Crystal syntax.

Overloads

- NthSmallest (N, fld)
- NthSmallest (N, fld, condFld)
- NthSmallest (N, fld, condFld, cond)

Arguments

- N is any integer from 1 to 100 (inclusive).
- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Determines the Nth smallest value in a given field, either for the entire report or for each instance of the (condFld) group.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
NthSmallest (1, {Customer.CUSTOMER ID})
```

Returns 12, where the Customer ID field contains numerical values ranging from 12 to 50.

```
NthSmallest(1, {Customer.CUSTOMER NAME}, {Customer.REGION})
```

Returns the least value in the Customer Name field, per region.

```
NthSmallest({orders.ORDER AMOUNT}, {Customer.REGION}, "monthly")
```

Groups values in the Amount field by region, then returns the least value in the Amount field for each region, per month.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.19 Mode

Basic and Crystal syntax.

Overloads

- Mode (fld)
- Mode (fld, condFld)
- Mode (fld, condFld, cond)

Arguments

- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Fractional Number

Action

Identifies the most frequently occurring value.

For example:

- You can calculate the mode of all the values in a field.
- You can calculate the mode of all the values in the field, within a group (for example, sales grouped by the state that they come from).
- You can calculate the mode of all the values within a group in which grouping is controlled by changes in a date or Boolean field (for example, sales grouped by the month in which they were made).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Mode({Customer.CUSTOMER NAME})
```

Returns the mode (the most frequently occurring value) for the Customer Name field.

```
Mode({Customer.CUSTOMER NAME}, {Customer.REGION})
```

Groups values in the Customer Name field by region, and then returns the mode for the Customer Name field, per region.

```
Mode({Customer.CUSTOMER NAME}, {Customer.REGION}, "monthly")
```

Groups values in the Customer Name field by region, then returns the mode of the Customer Name field for each region, per month.

Comments

Mode works like NthMost Frequent when N = 1.

📘 Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.20 NthMostFrequent

Basic and Crystal syntax.

Overloads

- NthMostFrequent (N, fld)
- NthMostFrequent (N, fld, condFld)
- NthMostFrequent (N, fld, condFld, cond)

Returns

Fractional Number

Arguments

- N is any integer from 1 to 100 (inclusive).
- fld is any valid database or formula field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Action

Determines the Nth most frequent value in a given field, either for the entire report or for each instance of the (condFld) group. If no values in the field appear more than once, the function will return the minimum value, by default.

Typical Uses

You can use this function to spotlight exceptionally high or exceptionally low values in a particular field.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
NthMostFrequent(1, {Customer.LAST YEAR'S SALES})
```

Returns \$29087, where the value of \$29087 appears most frequently in the Last Year's Sales field.

```
NthMostFrequent(2, {Customer.LAST YEAR'S SALES})
```

Returns \$34700, where \$34700 is the value that appears with the second highest frequency in the Last Year's Sales field.

```
NthMostFrequent(1, {Customer.LAST YEAR'S SALES}, {Customer.REGION})
```

Returns \$48000 for one instance of the Customer.Region group, where \$48000 is the value that appears most frequently in that group; returns \$34000 for another instance of the group, where \$34000 is the most frequent value for that occurrence of the group; and returns \$9000 for a third instance of the group, where none of the values in this occurrence of the group are repeated, and \$9000 is the lowest value.

Comments

When more than one value appears with the same frequency, the least value is considered the most frequent. For example, if

```
NthMostFrequent(1, {Customer.LAST YEAR'S SALES})
```

returns a value of \$29087, and a value greater than \$29087 (for example, \$35000) appears with the same frequency, NthMostFrequent with N = 1 will still return \$29087, since \$29087 is the lesser value.

Note

Using this function in a formula forces the formula to be evaluated at print time.

For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.4.21 PercentOfSum

Basic and Crystal syntax.

Overloads

- PercentOfSum (fld, condFld)
- PercentOfSum (fld, condFld, cond)

- `PercentOfSum (fld, innerCondFld, outerCondFld)`
- `PercentOfSum (fld, innerCondFld, innerCond, outerCondFld)`
- `PercentOfSum (fld, innerCondFld, outerCondFld, outerCond)`
- `PercentOfSum (fld, innerCondFld, innerCond, outerCondFld, outerCond)`

Arguments

- `fld` is a Number or Currency field that can be evaluated by the function.
- `condFld` is a field used to group the values in `fld` by.
- `cond` is a String indicating the type of grouping for `condFld`. You only specify this argument when `condFld` is a Date, Time, DateTime or Boolean field.
For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- `innerCondFld` is a field used to group the values in `fld` by.
- `innerCond` is a String indicating the type of grouping for `innerCondFld`. You only specify this argument when `innerCondFld` is a Date, Time, DateTime or Boolean field.
- `outerCondFld` is a field used to group the values in `fld` by.
- `outerCond` is a String indicating the type of grouping for `outerCondFld`. You only specify this argument when `outerCondFld` is a Date, Time, DateTime or Boolean field.
For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value

Action

[PercentOfSum \(fld, condFld\)](#) has the same effect as: $100 * \text{Sum}(\text{fld}, \text{condFld}) / \text{Sum}(\text{fld})$. It expresses the sum of the values of the field `fld` for the group determined by `condFld` as a percentage of the grand total sum.

[PercentOfSum \(fld, condFld, cond\)](#) has the same effect as: $100 * \text{Sum}(\text{fld}, \text{condFld}, \text{cond}) / \text{Sum}(\text{fld})$. The String argument `cond` expresses some additional information to precisely specify the grouping. For example, if `condFld` is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

[PercentOfSum \(fld, innerCondFld, outerCondFld\)](#) has the same effect as: $100 * \text{Sum}(\text{fld}, \text{innerCondFld}) / \text{Sum}(\text{fld}, \text{outerCondFld})$. It expresses the sum of the values of the field `fld` for the group determined by `innerCondFld` as a percentage of the sum for the group determined by `outerCondFld`.

[PercentOfSum \(fld, innerCondFld, innerCond, outerCondFld\)](#) has the same effect as: $100 * \text{Sum}(\text{fld}, \text{innerCondFld}, \text{innerCond}) / \text{Sum}(\text{fld}, \text{outerCondFld})$.

PercentOfSum (fld, innerCondFld, outerCondFld, outerCond) has the same effect as: $100 * \text{Sum (fld, innerCondFld)} / \text{Sum (fld, outerCondFld, outerCond)}$.

PercentOfSum (fld, innerCondFld, innerCond, outerCondFld, outerCond) has the same effect as: $100 * \text{Sum (fld, innerCondFld, innerCond)} / \text{Sum (fld, outerCondFld, outerCond)}$.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
PercentOfSum ({Orders.Order Amount}, {Orders.Order Date}, "annually")
```

Returns the total value of the orders that were ordered in a given year expressed as a percent of the total value of all orders.

```
PercentOfSum ({Orders.Order Amount}, {Orders.Ship Via})
```

Returns the total value of the orders shipped via a given shipping company expressed as a percent of the total value of all orders.

```
PercentOfSum ({Product.Price (SRP)}, {Product.Size}, {Product.Product Class})
```

Returns the total price of the products of a given size and class expressed as a percent of the total price of the products of the same given class.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.4.22 PercentOfAverage

Basic and Crystal syntax.

Overloads

- `PercentOfAverage (fld, condFld)`
- `PercentOfAverage (fld, condFld, cond)`
- `PercentOfAverage (fld, innerCondFld, outerCondFld)`
- `PercentOfAverage (fld, innerCondFld, innerCond, outerCondFld)`
- `PercentOfAverage (fld, innerCondFld, outerCondFld, outerCond)`

- `PercentOfAverage (fld, innerCondFld, innerCond, outerCondFld, outerCond)`

Arguments

- `fld` is a Number or Currency field that can be evaluated by the function.
- `condFld` is a field used to group the values in `fld` by.
- `cond` is a String indicating the type of grouping for `condFld`. You only specify this argument when `condFld` is a Date, Time, DateTime or Boolean field.
- `innerCondFld` is a field used to group the values in `fld` by.
- `innerCond` is a String indicating the type of grouping for `innerCondFld`. You only specify this argument when `innerCondFld` is a Date, Time, DateTime or Boolean field.
- `outerCondFld` is a field used to group the values in `fld` by.
- `outerCond` is a String indicating the type of grouping for `outerCondFld`. You only specify this argument when `outerCondFld` is a Date, Time, DateTime or Boolean field.

For more information on the valid strings for these arguments, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value.

Action

[PercentOfAverage \(fld, condFld\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{condFld}) / \text{Average}(\text{fld})$. It expresses the average of the values of the field `fld` for the group determined by `condFld` as a percentage of the average of all the values of `fld`.

[PercentOfAverage \(fld, condFld, cond\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{condFld}, \text{cond}) / \text{Average}(\text{fld})$. The String argument `cond` expresses some additional information to precisely specify the grouping. For example, if `condFld` is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

[PercentOfAverage \(fld, innerCondFld, outerCondFld\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{innerCondFld}) / \text{Average}(\text{fld}, \text{outerCondFld})$. It expresses the average of the values of the field `fld` for the group determined by `innerCondFld` as a percentage of the average for the group determined by `outerCondFld`.

[PercentOfAverage \(fld, innerCondFld, innerCond, outerCondFld\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{innerCondFld}, \text{innerCond}) / \text{Average}(\text{fld}, \text{outerCondFld})$.

[PercentOfAverage \(fld, innerCondFld, outerCondFld, outerCond\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{innerCondFld}) / \text{Average}(\text{fld}, \text{outerCondFld}, \text{outerCond})$.

[PercentOfAverage \(fld, innerCondFld, innerCond, outerCondFld, outerCond\)](#) has the same effect as: $100 * \text{Average}(\text{fld}, \text{innerCondFld}, \text{innerCond}) / \text{Average}(\text{fld}, \text{outerCondFld}, \text{outerCond})$.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.4.23 PercentOfMaximum

Basic and Crystal syntax

Overloads

- PercentOfMaximum (fld, condFld)
- PercentOfMaximum (fld, condFld, cond)
- PercentOfMaximum (fld, innerCondFld, outerCondFld)
- PercentOfMaximum (fld, innerCondFld, innerCond, outerCondFld)
- PercentOfMaximum (fld, innerCondFld, outerCondFld, outerCond)
- PercentOfMaximum (fld, innerCondFld, innerCond, outerCondFld, outerCond)

Arguments

- fld is a Number or Currency field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- innerCondFld is a field used to group the values in fld by.
- innerCond is a String indicating the type of grouping for innerCondFld. You only specify this argument when innerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- outerCondFld is a field used to group the values in fld by.
- outerCond is a String indicating the type of grouping for outerCondFld. You only specify this argument when outerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value.

Action

PercentOfMaximum (fld, condFld) has the same effect as: $100 * \text{Maximum (fld, condFld)} / \text{Maximum (fld)}$. It expresses the maximum of the values of the field fld for the group determined by condFld as a percentage of the maximum of all the values of fld.

PercentOfMaximum (fld, condFld, cond) has the same effect as: $100 * \text{Maximum (fld, condFld, cond)} / \text{Maximum (fld)}$. The String argument cond expresses some additional information to precisely specify the grouping. For example, if condFld is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

PercentOfMaximum (fld, innerCondFld, outerCondFld) has the same effect as: $100 * \text{Maximum (fld, innerCondFld)} / \text{Maximum (fld, outerCondFld)}$. It expresses the maximum of the values of the field fld for the group determined by innerCondFld as a percentage of the maximum for the group determined by outerCondFld.

PercentOfMaximum (fld, innerCondFld, innerCond, outerCondFld) has the same effect as: $100 * \text{Maximum (fld, innerCondFld, innerCond)} / \text{Maximum (fld, outerCondFld)}$

PercentOfMaximum (fld, innerCondFld, outerCondFld, outerCond) has the same effect as: $100 * \text{Maximum (fld, innerCondFld)} / \text{Maximum (fld, outerCondFld, outerCond)}$.

PercentOfMaximum (fld, innerCondFld, innerCond, outerCondFld, outerCond) has the same effect as: $100 * \text{Maximum (fld, innerCondFld, innerCond)} / \text{Maximum (fld, outerCondFld, outerCond)}$.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.4.24 PercentOfMinimum

Basic and Crystal syntax.

Overloads

- *PercentOfMinimum (fld, condFld)*
- *PercentOfMinimum (fld, condFld, cond)*
- *PercentOfMinimum (fld, innerCondFld, outerCondFld)*
- *PercentOfMinimum (fld, innerCondFld, innerCond, outerCondFld)*
- *PercentOfMinimum (fld, innerCondFld, outerCondFld, outerCond)*
- *PercentOfMinimum (fld, innerCondFld, innerCond, outerCondFld, outerCond)*

Arguments

- fld is a Number or Currency field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- innerCondFld is a field used to group the values in fld by.
- innerCond is a String indicating the type of grouping for innerCondFld. You only specify this argument when innerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- outerCondFld is a field used to group the values in fld by.
- outerCond is a String indicating the type of grouping for outerCondFld. You only specify this argument when outerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value.

Action

PercentOfMinimum (fld, condFld) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{condFld}) / \text{Minimum}(\text{fld})$. It expresses the minimum of the values of the field fld for the group determined by condFld as a percentage of the minimum of all the values of fld.

PercentOfMinimum (fld, condFld, cond) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{condFld}, \text{cond}) / \text{Minimum}(\text{fld})$. The String argument cond expresses some additional information to precisely specify the grouping. For example, if condFld is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

PercentOfMinimum (fld, innerCondFld, outerCondFld) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{innerCondFld}) / \text{Minimum}(\text{fld}, \text{outerCondFld})$. It expresses the minimum of the values of the field fld for the group determined by innerCondFld as a percentage of the minimum for the group determined by outerCondFld.

PercentOfMinimum (fld, innerCondFld, innerCond, outerCondFld) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{innerCondFld}, \text{innerCond}) / \text{Minimum}(\text{fld}, \text{outerCondFld})$.

PercentOfMinimum (fld, innerCondFld, outerCondFld, outerCond) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{innerCondFld}) / \text{Minimum}(\text{fld}, \text{outerCondFld}, \text{outerCond})$.

PercentOfMinimum (fld, innerCondFld, innerCond, outerCondFld, outerCond) has the same effect as: $100 * \text{Minimum}(\text{fld}, \text{innerCondFld}, \text{innerCond}) / \text{Minimum}(\text{fld}, \text{outerCondFld}, \text{outerCond})$.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.4.25 PercentOfCount

Basic and Crystal syntax.

Overloads

- PercentOfCount (fld, condFld)
- PercentOfCount (fld, condFld, cond)
- PercentOfCount (fld, innerCondFld, outerCondFld)
- PercentOfCount (fld, innerCondFld, innerCond, outerCondFld)
- PercentOfCount (fld, innerCondFld, outerCondFld, outerCond)
- PercentOfCount (fld, innerCondFld, innerCond, outerCondFld, outerCond)

Arguments

- fld is a Number, Currency, String, Boolean, Date, Time or DateTime field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- innerCondFld is a field used to group the values in fld by.
- innerCond is a String indicating the type of grouping for innerCondFld. You only specify this argument when innerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- outerCondFld is a field used to group the values in fld by.
- outerCond is a String indicating the type of grouping for outerCondFld. You only specify this argument when outerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value.

Action

PercentOfCount (fld, condFld) has the same effect as: $100 * \text{Count (fld, condFld)} / \text{Count (fld)}$. It expresses the count of the values of the field fld for the group determined by condFld as a percentage of the count of all the values of fld.

PercentOfCount (fld, condFld, cond) has the same effect as: $100 * \text{Count (fld, condFld, cond)} / \text{Count (fld)}$. The String argument cond expresses some additional information to precisely specify the grouping. For example, if condFld is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

PercentOfCount (fld, innerCondFld, outerCondFld) has the same effect as: $100 * \text{Count (fld, innerCondFld)} / \text{Count (fld, outerCondFld)}$. It expresses the count of the values of the field fld for the group determined by innerCondFld as a percentage of the count for the group determined by outerCondFld.

PercentOfCount (fld, innerCondFld, innerCond, outerCondFld) has the same effect as: $100 * \text{Count (fld, innerCondFld, innerCond)} / \text{Count (fld, outerCondFld)}$.

PercentOfCount (fld, innerCondFld, outerCondFld, outerCond) has the same effect as: $100 * \text{Count (fld, innerCondFld)} / \text{Count (fld, outerCondFld, outerCond)}$.

PercentOfCount (fld, innerCondFld, innerCond, outerCondFld, outerCond) has the same effect as: $100 * \text{Count (fld, innerCondFld, innerCond)} / \text{Count (fld, outerCondFld, outerCond)}$.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
PercentOfCount ({Orders.Order ID}, {Orders.Order Date}, "annually")
```

Returns the total number of orders that were ordered in a given year expressed as a percent of the total number of all orders.

```
PercentOfCount ({Orders.Order ID}, {Orders.Ship Via})
```

Returns the total number of orders shipped via a given shipping company expressed as a percent of the total number of all orders.

```
PercentOfCount ({Product.Product ID}, {Product.Size}, {Product.Product Class})
```

Returns the total number of products of a given size and class expressed as a percent of the total number of products of the same given class.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.4.26 PercentOfDistinctCount

Basic and Crystal syntax.

Overloads

- PercentOfDistinctCount (fld, condFld)
- PercentOfDistinctCount (fld, condFld, cond)
- PercentOfDistinctCount (fld, innerCondFld, outerCondFld)
- PercentOfDistinctCount (fld, innerCondFld, innerCond, outerCondFld)
- PercentOfDistinctCount (fld, innerCondFld, outerCondFld, outerCond)
- PercentOfDistinctCount (fld, innerCondFld, innerCond, outerCondFld, outerCond)

Arguments

- fld is a Number, Currency, String, Boolean, Date, Time or DateTime field that can be evaluated by the function.
- condFld is a field used to group the values in fld by.
- cond is a String indicating the type of grouping for condFld. You only specify this argument when condFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- innerCondFld is a field used to group the values in fld by.
- innerCond is a String indicating the type of grouping for innerCondFld. You only specify this argument when innerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).
- outerCondFld is a field used to group the values in fld by.
- outerCond is a String indicating the type of grouping for outerCondFld. You only specify this argument when outerCondFld is a Date, Time, DateTime or Boolean field. For more information on the valid strings for this argument, see [Conditions for summary functions \[page 1552\]](#).

Returns

Number value.

Action

PercentOfDistinctCount (fld, condFld) has the same effect as: $100 * \text{DistinctCount (fld, condFld)} / \text{DistinctCount (fld)}$. It expresses the distinct count of the values of the field fld for the group determined by condFld as a percentage of the distinct count of all the values of fld.

PercentOfDistinctCount (fld, condFld, cond) has the same effect as: $100 * \text{DistinctCount (fld, condFld, cond)} / \text{DistinctCount (fld)}$. The String argument cond expresses some additional information to precisely specify the grouping example: if condFld is a Date field, is the grouping "daily" or "weekly" or "monthly" and so on.

PercentOfDistinctCount (fld, innerCondFld, outerCondFld) has the same effect as: $100 * \text{DistinctCount (fld, innerCondFld)} / \text{DistinctCount (fld, outerCondFld)}$. It expresses the distinct count of the values of the field fld for the group determined by innerCondFld as a percentage of the distinct count for the group determined by outerCondFld.

PercentOfDistinctCount (fld, innerCondFld, innerCond, outerCondFld) has the same effect as: $100 * \text{DistinctCount (fld, innerCondFld, innerCond)} / \text{DistinctCount (fld, outerCondFld)}$.

PercentOfDistinctCount (fld, innerCondFld, outerCondFld, outerCond) has the same effect as: $100 * \text{DistinctCount (fld, innerCondFld)} / \text{DistinctCount (fld, outerCondFld, outerCond)}$

PercentOfDistinctCount (fld, innerCondFld, innerCond, outerCondFld, outerCond) has the same effect as: $100 * \text{DistinctCount (fld, innerCondFld, innerCond)} / \text{DistinctCount (fld, outerCondFld, outerCond)}$.

Comments

This function and other percentage functions are intended to support Percentage Summary fields. The groupings supplied as arguments must actually exist in the report for the summary function call to succeed.

41.5 Financial

Click a link to view one of the topics in this section:

- [ACCRINT \[page 1595\]](#)
- [ACCRINTM \[page 1596\]](#)
- [AmorDEGRC \[page 1598\]](#)
- [AmorLINC \[page 1600\]](#)
- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)
- [CumIPMT \(rate, nPeriods, presentValue, startPeriod, endPeriod, type\) \[page 1611\]](#)

- [CumPrinc \(rate, nPeriods, presentValue, startPeriod, endPeriod, type\) \[page 1612\]](#)
- [DB \[page 1614\]](#)
- [DDB \[page 1615\]](#)
- [Days360 \[page 1617\]](#)
- [DISC \[page 1618\]](#)
- [DollarDE \(fractionNumber, fractionBase\) \[page 1620\]](#)
- [DollarFR \(decimalNumber, fractionBase\) \[page 1621\]](#)
- [Duration \[page 1622\]](#)
- [Effect \(rate, nCompoundingPeriods\) \[page 1624\]](#)
- [FV \[page 1625\]](#)
- [FVSchedule \(value, rates\) \[page 1626\]](#)
- [IntRate \[page 1627\]](#)
- [IPmt \[page 1629\]](#)
- [IRR \[page 1631\]](#)
- [ISPMT \(rate, period, nPeriods, presentValue\) \[page 1632\]](#)
- [MDuration \[page 1633\]](#)
- [MIRR \(values, financeRate, reinvestRate\) \[page 1635\]](#)
- [Nominal \(rate, nCompoundingPeriods\) \[page 1636\]](#)
- [NPer \[page 1637\]](#)
- [NPV \(rate, values\) \[page 1638\]](#)
- [OddFPrice \[page 1640\]](#)
- [OddFYield \[page 1641\]](#)
- [OddLPrice \[page 1643\]](#)
- [OddLYield \[page 1645\]](#)
- [Pmt \[page 1647\]](#)
- [PPmt \[page 1649\]](#)
- [Price \[page 1651\]](#)
- [PriceDisc \[page 1652\]](#)
- [PriceMat \[page 1654\]](#)
- [PV \[page 1656\]](#)
- [Rate \[page 1657\]](#)
- [Received \[page 1658\]](#)
- [SLN \(cost, salvage, life\) \[page 1660\]](#)
- [SYD \(cost, salvage, life, period\) \[page 1661\]](#)
- [TBillEq \(settlementDate, maturityDate, discountRate\) \[page 1662\]](#)
- [TBillPrice \(settlementDate, maturityDate, discountRate\) \[page 1663\]](#)
- [TBillYield \(settlementDate, maturityDate, price\) \[page 1665\]](#)
- [VDB \[page 1666\]](#)
- [XIRR \[page 1668\]](#)
- [XNPV \(rate, values, dates\) \[page 1670\]](#)
- [YearFrac \[page 1671\]](#)
- [Yield \[page 1672\]](#)

- [YieldDisc \[page 1674\]](#)
- [YieldMat \[page 1676\]](#)

41.5.1 ACCRINT

Basic and Crystal syntax.

Overloads

- ACCRINT (issueDate, firstInterestDate, settlementDate, rate, parValue, frequency)
- ACCRINT (issueDate, firstInterestDate, settlementDate, rate, parValue, frequency, basis)

Arguments

- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- firstInterestDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security makes its first interest payment.
- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- rate is a positive number specifying the annual interest rate for the security.
- parValue is a positive number or currency specifying the par value of the security.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

ACCRINT returns the total accrued interest for a security with periodic interest payments. The value returned is the parValue multiplied by the interest rate multiplied by the number of years or portions of years between the issue date and the maturity date.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose a \$10,000 bond issued on July 1, 1995 pays interest semiannually at 9.5% starting on September 1, 1995. This bond has a short odd period since the time between July and September is less than six months. The bond is then purchased on February 15, 2000 (the settlement date). A 30/360 basis will be used.

```
ACCRINT (DateValue(1995,7,1), DateValue(1995,9,1),DateValue(2000,2,15), 0.095,  
10000, 2, 0)
```

This example returns \$4391.11 (rounded to the nearest cent) as the total accrued interest. This value can be checked by adding up the number of full coupon periods and the fractional periods. There are eight six-month periods starting on September 1, 1995 and ending on September 1, 1999. The partial periods are the 60 days (using our basis) in the initial odd period and the 164 days leading up to the settlement date. So $\$10,000 \times (8 + 224/180) = \4391.11 .

Comments

This function is similar to the Excel function of the same name.

See Also

- [ACCRINTM \[page 1596\]](#)
- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.2 ACCRINTM

Basic and Crystal syntax.

Overloads

- ACCRINTM (issueDate, maturityDate, rate, parValue)
- ACCRINTM (issueDate, maturityDate, rate, parValue, basis)

Arguments

- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the issue date specifying when the security matures.
- rate is a positive number specifying the annual interest rate for the security.
- parValue is a positive number or currency specifying the par value of the security.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

ACCRINTM returns the total accrued interest for a security that pays interest at maturity. This function calculates the interest by multiplying the par value, the interest rate, and the number of years and partial years between issue and maturity.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose a short-term \$5,000 note, with a 10.5% annual interest rate, is issued on September 25, 1999 with a maturity on March 31, 2000. An actual/360 basis will be used.

```
ACCRINTM(DateValue(1999,9,25),DateValue(2000,3,31),0.105, 5000, 2)
```

This example returns \$274.17 for the accrued interest of this security. There are 188 days between the issue and maturity dates, so $\$5000 \times 0.105 \times (188/360) = \274.17 .

Comments

This function is similar to the Excel function of the same name.

See Also

- [ACCRINT \[page 1595\]](#)

41.5.3 AmorDEGRC

Basic and Crystal syntax.

Overloads

- AmorDEGRC (cost, purchaseDate, firstPeriodEndDate, salvage, period, rate)
- AmorDEGRC (cost, purchaseDate, firstPeriodEndDate, salvage, period, rate, basis)

Arguments

- cost is a positive number or currency specifying the initial cost of the asset.
- purchaseDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the asset was purchased.
- firstPeriodEndDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying the end of the first period. This date must be after the purchase date.
- salvage is a non-negative number or currency specifying the value of the asset after its useful life. This number must be equal to or less than the cost of the asset.
- period is a non-negative number specifying which period the depreciation should be calculated for.
- rate specifies the rate at which the asset depreciates.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365

- 4 - European 30/360

Returns

Currency value

Action

AmorDEGRC returns the depreciation of an asset in the accounting year specified. The accounting years are aligned with the firstPeriodEndDate. A depreciation factor is applied to the rate depending on the lifetime (1/rate) of the asset. The answer is rounded to the nearest whole number.

lifetime (1/rate)	depreciation factor applied
less than or equal to 4	1.5
greater than 4 but less than 6	2
greater than or equal to 6	2.5

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a machine is purchased for \$1500 on April 15, 2000. The machine can be sold for scrap at a price of \$250 and depreciates at 10% per year. If the accounting periods start on January 1st of each year, then the first period after the purchase date is January 1, 2001. An actual/actual basis is used.

```
AmorDEGRC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 0, .1, 1)
```

Returns \$267 as the depreciation for the period of time between April 15, 2000 and December 31, 2000. Since the lifetime is ten years, the depreciation rate is $2.5 \times 0.1 = 0.25$.

```
AmorDEGRC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 1, .1, 1)
```

Returns \$308 as the depreciation during the first full year.

```
AmorDEGRC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 6, .1, 1)
```

Returns \$43 as the depreciation for the sixth full year.

Comments

This function is similar to the Excel function of the same name.

See Also

- [AmorLINC \[page 1600\]](#)
- [DB \[page 1614\]](#)
- [DDB \[page 1615\]](#)
- [VDB \[page 1666\]](#)

41.5.4 AmorLINC

Basic and Crystal syntax.

Overloads

- AmorLINC (cost, purchaseDate, firstPeriodEndDate, salvage, period, rate)
- AmorLINC (cost, purchaseDate, firstPeriodEndDate, salvage, period, rate, basis)

Arguments

- cost is a positive number or currency specifying the initial cost of the asset.
- purchaseDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the asset was purchased.
- firstPeriodEndDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying the end of the first period. This date must be after the purchase date.
- salvage is a non-negative number or currency specifying the value of the asset after its useful life. This number must be equal to or less than the cost of the asset.
- period is a non-negative number specifying which period the depreciation should be calculated for.
- rate specifies the rate at which the asset depreciates.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360Returns

Returns

Currency value

Action

AmorLINC returns the linear depreciation in a specified year with the accounting years aligned with the firstPeriodEndDate. The depreciation during each year is constant, except for the zeroth and last periods which depend on how far through the year the purchase occurred. The sum of all of the depreciations equals the difference between the purchase price and the salvage price.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a machine is purchased for \$1500 on April 15, 2000. The machine can be sold for scrap at a price of \$250 and depreciates at 10% per year. If the accounting periods start on January 1st of each year, then the first period after the purchase date is January 1, 2001. An actual/actual basis is used.

```
AmorLINC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 0, .1, 1)
```

Returns \$106.97 (rounded to the nearest cent) as the depreciation for the period of time between April 15, 2000 and December 31, 2000.

```
AmorLINC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 1, .1, 1)
```

Returns \$150 as the depreciation during the first full year. Note that this is 10% of the \$1500 cost.

```
AmorLINC(1500, DateValue(2000, 4, 15), DateValue(2001, 1, 1), 250, 8, .1, 1)
```

Returns \$93.03 (rounded to the nearest cent) as the depreciation for the eighth full year. Note that this value added to the depreciation in the initial partial year equals \$150.

Comments

This function is similar to the Excel function of the same name.

See Also

- [AmorDEGRC \[page 1598\]](#)

- [DB \[page 1614\]](#)
- [DDB \[page 1615\]](#)
- [VDB \[page 1666\]](#)

41.5.5 CoupDayBS

Basic and Crystal syntax.

Overloads

- CoupDayBS (settlementDate, maturityDate, frequency)
- CoupDayBS (settlementDate, maturityDate, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360Returns

Returns

Number value

Action

CoupDayBS returns the number of days between the latest coupon date before the settlement, and the settlement date. Coupon dates are evenly spaced, aligning with the maturity date. If the settlement date falls on a coupon date then CoupDayBS returns 0.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date, or March 15, 2025. This example uses the actual/actual basis.

```
CoupDayBS(DateValue(2000, 8, 25), DateValue(2025, 3, 15), 2, 1)
```

Returns 163. The coupon days for this bond occur on March 15 and September 15 of each year. 163 is the number of days between March 15, 2000 and August 25, 2000.

Comments

- This function is designed to work like the Excel function of the same name.
- For the actual/actual basis, $\text{CoupDays} = \text{CoupDayBS} + \text{CoupDaysNC}$ for all valid inputs. This is not necessarily true when using a different basis, since CoupDays is derived from the number of days in the year while CoupDayBS and CoupDaysNC are calculated directly.

See Also

- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.6 CoupDays

Basic and Crystal syntax.

Overloads

- `CoupDays (settlementDate, maturityDate, frequency)`
- `CoupDays (settlementDate, maturityDate, frequency, basis)`

Arguments

- `settlementDate` is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- `maturityDate` is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.
- `frequency` is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- `basis` is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360Returns

Returns

Number value

Action

`CoupDays` returns the number of days between the coupon dates surrounding the settlement date. Coupon dates are evenly spaced, aligning with the maturity date. If the settlement date occurs on a coupon date, then the following period is used in the calculation.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date or March 15, 2025. This example uses the actual/actual basis.

```
CoupDays (DateValue (2000, 8, 25), DateValue (2025, 3, 15), 2, 1)
```

Returns 184. The coupon days for this bond occur on March 15 and September 15 of each year. 184 is the number of days between March 15, 2000 and September 15, 2000 since these days are the first coupon days before and after the settlement date.

Comments

- This function is designed to work like the Excel function of the same name.
- Note that for any basis, other than actual/actual, CoupDays returns the same number for all settlement dates.
- For the actual/actual basis, $\text{CoupDays} = \text{CoupDayBS} + \text{CoupDaysNC}$ for all valid inputs. This is not necessarily true when using a different basis, since CoupDays is derived from the number of days in the year while CoupDayBS and CoupDaysNC are calculated directly.

See Also

- [CoupDayBS \[page 1602\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.7 CoupDaysNC

Basic and Crystal syntax.

Overloads

- CoupDaysNC (settlementDate, maturityDate, frequency)
- CoupDaysNC (settlementDate, maturityDate, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.

- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360Returns

Returns

Number value

Action

CoupDaysNC returns the number of days between the settlement date and the next coupon date after the settlement date. Coupon dates are evenly spaced, aligning with the maturity date. If the settlement date falls on a coupon date, then CoupDaysNC returns the number of days to the next coupon date.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date or March 15, 2025. This example uses the actual/actual basis.

```
CoupDaysNC(DateValue(2000,8,25),DateValue(2025,3,15),2,1)
```

Returns 21. The coupon days for this bond occur on March 15 and September 15 of each year. 21 is the number of days between August 25, 2000 and September 15, 2000.

Comments

- This function is designed to work like the Excel function of the same name.
- For the actual/actual basis, $\text{CoupDays} = \text{CoupDayBS} + \text{CoupDaysNC}$ for all valid inputs. This is not necessarily true when using a different basis, since CoupDays is derived from the number of days in the year while CoupDayBS and CoupDaysNC are calculated directly.

See Also

- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.8 CoupNCD (settlementDate, maturityDate, frequency)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).

Returns

Date value

Action

CoupNCD returns the date of the next coupon after the settlement date. Coupon dates are evenly spaced, aligning with the maturity date and occur either once, twice or four times per year.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date, or March 15, 2025.

```
CoupNCD(DateValue(2000,8,25),DateValue(2025,3,15),2)
```

Returns September 15, 2000. The coupon days for this bond occur on March 15 and September 15 of each year, and the first of these dates, falling after the settlement date, is September 15, 2000.

Comments

- This function is designed to work like the Excel function of the same name.
- The number of days (according to the basis) between the value returned by CoupNCD and the settlement date is equal to the value returned by CoupDaysNC.

See Also

- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.9 CoupNum (settlementDate, maturityDate, frequency)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).

Returns

Number value

Action

CoupNum returns the number of coupon periods between the settlement date and maturity. The partial period between the settlement date and the first coupon date after the settlement date, is counted as one.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date, or March 15, 2025.

```
CoupNum(DateValue(2000,8,25),DateValue(2025,3,15),2)
```

Returns 50. This is the number of six-month periods between August 25, 2000 and March 15, 2025, rounded up. Working backwards from maturity, there are 48 periods in the 24 years between March 15, 2001 and March 15, 2025 and a full period between September 15, 2000 and March 15, 2001. The partial period (only 21 days) between settlement date (August 25, 2000) and September 15, 2000 is also counted.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupPCD \(settlementDate, maturityDate, frequency\) \[page 1610\]](#)

41.5.10 CoupPCD (settlementDate, maturityDate, frequency)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#), after the settlement date, specifying the date when the security matures.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).

Returns

Date value

Action

CoupPCD returns the date of the last coupon date before the settlement date. Coupon dates are evenly spaced, aligning with the maturity date. If the settlement date falls on a coupon date, then CoupPCD returns the settlement date.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a 30-year bond was issued on March 15, 1995 with semiannual coupon dates. The bond was purchased on the secondary market on August 25, 2000 (the settlement date). The maturity date is 30 years after the issue date, or March 15, 2025.

```
CoupPCD(DateValue(2000, 8, 25), DateValue(2025, 3, 15), 2)
```

Returns March 15, 2000. The coupon days for this bond occur on March 15 and September 15 of each year. The last of these dates, falling before the settlement date, is March 15, 2000.

Comments

- This function is designed to work like the Excel function of the same name.
- The number of days (according to the basis) between the value returned by CoupPCD and the settlement date is equal to the value returned by CoupDayBS.

See Also

- [CoupDayBS \[page 1602\]](#)
- [CoupDays \[page 1603\]](#)
- [CoupDaysNC \[page 1605\]](#)
- [CoupNCD \(settlementDate, maturityDate, frequency\) \[page 1607\]](#)
- [CoupNum \(settlementDate, maturityDate, frequency\) \[page 1608\]](#)

41.5.11 CumIPMT (rate, nPeriods, presentValue, startPeriod, endPeriod, type)

Basic and Crystal syntax.

Arguments

- rate is a positive, non-zero number that specifies the interest rate per period.
- nPeriods is a positive number specifying the total number of payment periods.
- presentValue is a positive number or currency representing the total value of the loan.
- startPeriod is a positive number specifying the first period to be included in the calculation.
- endPeriod is a number, greater than or equal to startPeriod, specifying the last period to be included in the calculation. The end period cannot be greater than the number of periods.
- type is a number specifying when the payments are made. 0 = end of the payment period, 1 = start of the payment period.

Returns

Currency value

Action

CumIPMT returns the total interest paid on a loan between the start and end periods specified. This function sums the value returned from IPMT for the periods specified. The value will be negative since it represents a sum of payments.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose you have taken out a \$5000, six-year loan with an annual interest rate of 8.3% with semiannual payments paid at the end of the month.

```
CumIPMT(0.083/2, 6*2, 5000, 3, 4, 0)
```

Returns -\$344.25 as the total interest paid in the second year of the loan. Note that both the interest rate and the duration have to be written on a per period basis.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [CumPrinc \(rate, nPeriods, presentValue, startPeriod, endPeriod, type\) \[page 1612\]](#)

41.5.12 CumPrinc (rate, nPeriods, presentValue, startPeriod, endPeriod, type)

Basic and Crystal syntax.

Arguments

- rate is a positive, non-zero number that specifies the interest rate per period.
- nPeriods is a positive number specifying the total number of payment periods.
- presentValue is a positive number or currency representing the total value of the loan.

- startPeriod is a positive number specifying the first period to be included in the calculation.
- endPeriod is a number, greater than or equal to startPeriod, specifying the last period to be included in the calculation. The end period cannot be greater than the number of periods.
- type is a number specifying when the payments are made. 0 = end of the payment period, 1 = start of the payment period.

Returns

Currency value

Action

CumPRINC returns the total paid towards the principal on a loan, between the start and end periods specified. This function sums the value returned from PPMT for the periods specified. The value will be negative since it represents a sum of payments.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose you have taken out a \$5000, six-year loan with an annual interest rate of 8.3% with semiannual payments paid at the end of the month.

```
CumPRINC(0.083/2, 6*2, 5000, 3, 4, 0)
```

Returns -\$730.57 as the total principal paid off during the second year of the load. Note that both the interest rate and the duration have to be written on a per period basis.

```
CumPRINC(0.083/2, 6*2, 5000, 1, 12, 0)
```

Returns -\$5000 since over the duration of the loan you have paid off the entire principal. Note that both the interest rate and the duration have to be written on a per period basis.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [CumIPMT \(rate, nPeriods, presentValue, startPeriod, endPeriod, type\) \[page 1611\]](#)

41.5.13 DB

Basic and Crystal syntax.

Overloads

- DB (cost, salvage, life, period)
- DB (cost, salvage, life, period, month)

Arguments

- cost is a Number or Currency that specifies the initial cost of the asset. The value is non-negative and greater than or equal to salvage.
- salvage is a Number or Currency that specifies the value of the asset at the end of its useful life. The value is non-negative.
- life is a positive Number that specifies the length of the useful life of the asset.
- period is a Number that specifies the period for which asset depreciation is calculated. The value is positive and less than or equal to life. The arguments life and period must have the same units.
- month is an optional, positive Number that specifies the number of months in the first year. The default value is 12 months. Month must be greater than zero and less than or equal to 12.

Returns

Number value

Action

DB returns a number specifying the depreciation of an asset for a specific time period, using the fixed-declining balance method.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a business purchases a car for \$25,000 at the beginning of March. The car is to be depreciated using the fixed-declining balance method and is to have a five year lifetime and a resell value, after that time, of \$5,000.

```
DB(25000, 5000, 5, 1, 10)
```

Returns 5729.17. The depreciation on the car over the remaining ten months of the year is \$5729.17.

```
DB(25000, 5000, 5, 5, 10)
```

Returns 2019.52. The depreciation in the last full year is \$2019.52.

```
DB(25000, 5000, 5, 6, 10)
```

Returns 244.02. The depreciation in the last two months is \$244.02.

Suppose a database table contains the following fields: {table.cost}, {table.salvage}, {table.purchaseDate}, and {table.lifetime}. The following formula can be used to calculate the depreciation using the fixed-declining balance method for the remainder of the first year for each record in the table.

```
DB ({table.cost}, {table.salvage}, {table.lifetime}, 1, 12-Month({table.purchaseDate}))
```

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [DDB \[page 1615\]](#)
- [VDB \[page 1666\]](#)

41.5.14 DDB

Basic and Crystal syntax.

Overloads

- DDB (cost, salvage, life, period)
- DDB (cost, salvage, life, period, factor)

Arguments

- cost is a Number or Currency that specifies the initial cost of the asset. The value is non-negative and greater than or equal to salvage.
- salvage is a Number or Currency that specifies the value of the asset at the end of its useful life. The value is non-negative.
- life is a positive Number that specifies the length of the useful life of the asset.
- period is a Number that specifies the period for which asset depreciation is calculated. The value is positive and less than or equal to life. The arguments life and period must have the same units.
- factor is an optional positive Number that specifies the rate at which the balance declines. If omitted, 2 (double-declining method) is assumed.

Returns

Number value

Action

DDB returns a Number specifying the depreciation of an asset for a specific time period, using the double-declining balance method or some other method as specified by the factor argument.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a company purchases a fleet of cars for \$500,000. The cars have a lifetime of 10 years and a salvage value of \$60,000. They are depreciated using the double-declining method.

```
DDB (500000, 60000, 10, 1)
```

Returns 100000. The first year's depreciation is \$100,000.

```
DDB (500000, 60000, 10, 4)
```

Returns 51200. The fourth year's depreciation is \$51,200.

```
DDB (500000, 60000, 10, 10)
```

Returns 7108.86 (rounded to the nearest cent). The final year's depreciation is \$7,108.86.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.15 Days360

Basic and Crystal syntax.

Overloads

- Days360 (startDate, endDate)
- Days360 (startDate, endDate, method)

Arguments

- startDate is a Date or DateTime for the start of an interval of time.
- endDate is a Date or DateTime for the end of an interval of time.
- method is an optional Boolean value specifying the type of basis to use. FALSE is the default and implies the American 30/360 day basis while TRUE implies the European 30/360 day basis.

Returns

Number value

Action

Days360 returns the number of days between two dates using a calendar that has 30 days in a month, and 360 days in a year, commonly used for financial accounting. If the end date is earlier than the start date, then a negative answer is returned.

The formula is $(Y2-Y1) \times 360 + (M2-M1) \times 30 + (D2-D1)$ where:

- Y1, M1, and D1 are the year, month and day of the start date, and
- Y2, M2, and D2 are the year, month and day of the end date.

For the American 30/360 basis, if D1 is 31 then it is changed to 30 and if D2 is 31, it is changed to 30 only if D1 is 30 or 31. For the special case of February, '31' in the above formula is replaced by 'the number of days in the month'.

For the European 30/360 basis, D1 is changed to 30 if D1 is 31, and similarly D2 is changed to 30 if D2 is 31.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Days360(DateValue(2000,1,1),DateValue(2000,2,1),FALSE)
```

Returns 30 since each month is assumed to have 30 days. The same value is returned if the European system is used.

```
Days360(DateValue(2000,2,28),DateValue(2000,3,31),FALSE)
Days360(DateValue(2000,2,28),DateValue(2000,3,31),TRUE)
Days360(DateValue(2000,2,29),DateValue(2000,3,31),FALSE)
Days360(DateValue(2000,2,29),DateValue(2000,3,31),TRUE)
```

The first line returns 33, the second returns 32, the third returns 30, and the fourth returns 31. Note that February 2000 contains 29 days.

Comments

- This function is similar to the Excel function of the same name.
- To calculate the difference between two dates using the normal (actual/actual) calendar, subtract the earlier date from the later date.

```
DateValue(2000,3,31) - DateValue(2000,2,29)
```

Returns 31.

41.5.16 DISC

Basic and Crystal syntax.

Overloads

- DISC (settlementDate, maturityDate, price, redemptionPrice)
- DISC (settlementDate, maturityDate, price, redemptionPrice, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- price is a positive number or currency specifying the value of the security when it was purchased.
- redemptionPrice is a positive number or currency specifying the value of the security at maturity.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

DISC returns the discount rate for a security, given the settlement and maturity dates, and the security's value at those times.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a security was purchased on April 15, 2000 for \$96.25. The security matures on November 1, 2000 with a redemption value of \$100. A 30/360 basis is used as it is common for many securities.

```
DISC (DateValue(2000, 4, 15), DateValue(2000, 11, 1), 96.25, 100, 0)
```

Returns 0.0689 (rounded to four decimal places) corresponding to a discount rate of 6.89%.

Comments

This function is designed to work like the Excel function of the same name.

41.5.17 DollarDE (fractionNumber, fractionBase)

Basic and Crystal syntax.

Arguments

- fractionNumber is a number representing the fraction to be converted.
- fractionBase is a positive number which, truncated, specifies the base of the fraction that is to be created.

Returns

Number value of the fraction.

Action

DollarDE takes a number representing a fraction and converts it into a normal decimal number. See the examples below for how fractions are represented.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DollarDE(1.1, 8)
```

Returns 1.125. The fraction 1.1 was interpreted as 1 and 1/8 which equals 1.125.

```
DollarDE(2.13, 16)
```

Returns 2.8125. The fraction was interpreted as 2 and 13/16.

```
DollarDE(2.45, 16)
```

Returns 4.8125. The fraction was interpreted as 2 and 45/16 which reduces to 4 and 13/16.

```
DollarDE(1.16, 8)
```

Returns 1.2. This was interpreted as 1 and 1.6/8.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [DollarFR \(decimalNumber, fractionBase\) \[page 1621\]](#)

41.5.18 DollarFR (decimalNumber, fractionBase)

Basic and Crystal syntax.

Arguments

- decimalNumber is a number representing the actual decimal value that is to be converted.
- fractionBase is a positive number which, truncated, specifies the base of the fraction that is to be created.

Returns

Number value which represents a fraction.

Action

DollarFR takes a decimal number and returns a number representing a fraction. See the example below for how numbers are represented in fractional form.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DollarFR(1.125, 8)
```

Returns 1.1. This is interpreted as 1 and 1/8. The decimal portion of the fraction is taken to be the numerator for the fraction base.

```
DollarFR(1.2,8)
```

Returns 1.16. This is interpreted as 1 and 1.6/8. The decimal portion is the numerator. Note that this is not interpreted as 16/8.

```
DollarFR(1.8125,16)
```

Returns 1.13. This is interpreted as 1 and 13/16.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [DollarDE \(fractionNumber, fractionBase\) \[page 1620\]](#)

41.5.19 Duration

Basic and Crystal syntax.

Overloads

- Duration (settlementDate, maturityDate, couponRate, yield, frequency)
- Duration (settlementDate, maturityDate, couponRate, yield, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- couponRate is a non-negative number specifying the interest rate for the security.
- yield is a non-negative number specifying the security's yield.

- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

Duration returns a number representing the duration of a bond. Duration is the weighted average of the coupon payment cash flows and is a measure of the bond's interest rate risk.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a ten-year bond with a coupon rate of 12%, a yield of 13%, and semiannual payments, is purchased on January 25, 2000 and has a maturity date of October 1, 2003. The default basis of 30/360 will be used.

```
Duration(DateValue(2000,1,25),DateValue(2003,10,1),0.12,0.13,2)
```

Returns 2.961918 as the bond's duration, representing 2.96 years.

Comments

This function is similar to the Excel function of the same name.

See Also

- [Duration \[page 1622\]](#)

41.5.20 Effect (rate, nCompoundingPeriods)

Basic and Crystal syntax.

Arguments

- rate is a number that specifies the annual interest rate.
- nCompoundingPeriods is a positive number that specifies the number of compounding periods per year.

Returns

Number value

Action

Effect returns the effective annual interest rate based on the specified interest rate and the given number of compounding periods.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose you have a savings account that is advertising at 5% annual interest rate and pays interest monthly.

```
Effect (0.05, 12)
```

Returns 0.0512 (rounded to four decimal places) which corresponds to an effective interest rate of 5.12%.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [Nominal \(rate, nCompoundingPeriods\) \[page 1636\]](#)

41.5.21 FV

Basic and Crystal syntax.

Overloads

- FV (rate, nPeriods, payment)
- FV (rate, nPeriods, payment, presentValue)
- FV (rate, nPeriods, payment, presentValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate and nPeriods must consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate.
- payment is a Number or Currency that specifies the payment to be made each period.
- presentValue is an optional Number or Currency that specifies the present value of a series of future payments.
- Type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

FV returns a Number specifying the future value of an annuity based on periodic, fixed payments and a fixed interest rate.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose that you put \$1000 per month into a retirement savings plan that pays 6 percent annual interest, compounded monthly. How much will the account be worth after 20 years?

```
FV (0.06 / 12, 20 * 12, -1000)
```

Returns 462041 (rounded to the nearest integer). So your account will have \$462,041. The payment (-1000) is negative since you are paying out the money to the plan.

The above example assumes that you made your payments into the plan at the end of the month. Thus after the first month, your plan would have only \$1000 in it, since there was no time for interest to accrue. Suppose that instead you make your payments at the start of the month:

```
FV (0.06 / 12, 20 * 12, -1000, 0, 1)
```

Returns 464351 (rounded to the nearest integer). So your account will have \$464,351. You will save \$2,310 more by depositing at the beginning of the month.

Now suppose that in addition to making payments at the start of the month, you start your plan with an initial deposit of \$20,000.

```
FV (0.06 / 12, 20 * 12, -1000, -20000, 1)
```

Returns 530555 (rounded to the nearest integer). Your account will have \$530,555 after 20 years.

You can also use the FV function to calculate the future value of a lump sum deposit. For example, if you deposit \$20,000 in to a plan that pays 6 percent annual interest compounded monthly for 20 years:

```
FV (0.06 / 12, 20 * 12, 0, -20000)
```

Returns 66204 (rounded to the nearest integer). Thus you would have \$66,204 in your account. This is also equal to the difference of the previous 2 examples (\$530,555 - \$464,351).

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.22 FVSchedule (value, rates)

Basic and Crystal syntax.

Arguments

- value is a number or currency that specifies the present value of the investment.
- rates is a number type array that specifies the interest rate for equal length periods.

Returns

Currency value

Action

FVSchedule returns the future value of an investment given the interest rates for over several compounding periods.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a \$50,000 investment has a guaranteed interest rate of 5% for the first year, 5.5% for the second year and 6% for the third year.

```
Rem Basic syntax
formula = FVSchedule(50000, Array(0.05,0.055,0.06))
//Crystal syntax
FVSchedule(50000, [0.05,0.055,0.06])
```

Returns \$58,710.75 as the value of the investment after the third year.

Suppose that you have a database table containing information on several types of five-year savings bonds. The table contains the following fields: {table.denomination}, {table.rateYear1}, {table.rateYear2}, {table.rateYear3}, {table.rateYear4}, {table.rateYear5}. To compare the different bond types fairly, you decide to use a per \$100 calculation of the final value of each bond type. The following formula (in Crystal Syntax) might be a good way to start comparing the different bond types.

```
FVSchedule (100, [{table.rateYear1}, {table.rateYear2},{ table.rateYear3},
{table.rateYear4}, {table.rateYear5}])
```

Comments

This function is designed to work like the Excel function of the same name.

41.5.23 IntRate

Basic and Crystal syntax.

Overloads

- IntRate (settlementDate, maturityDate, price, redemptionValue)
- IntRate (settlementDate, maturityDate, price, redemptionValue, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- price is a positive number or currency specifying the value of the security when it was purchased.
- redemptionValue is a positive number or currency specifying the amount to be received for the security at maturity.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

IntRate returns the interest rate for an investment in a security. The formula used is:

```
= (redemptionValue - presentValue) / presentValue * (portion of the year according to the basis)
```

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a security is purchased on January 15, 2000 for \$1000, and it matures on April 30, 2000 with a redemption value of \$1010. Using a 30/360 basis (common for U.S. Government bonds), the following formula can be used to calculate the interest rate.

```
IntRate(DateValue(2000,1,15),DateValue(2000,4,30),1000,1010,0)
```

Returns 0.0343 (rounded to four decimal places) corresponding to an interest rate of 3.43%.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [Received \[page 1658\]](#)

41.5.24 IPmt

Basic and Crystal syntax.

Overloads

- IPmt (rate, period, nPeriods, presentValue)
- IPmt (rate, period, nPeriods, presentValue, futureValue)
- IPmt (rate, period, nPeriods, presentValue, futureValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- period is a Number that specifies the payment period in the range 1 through nPeriods.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate, period and nPeriods must be consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate and period specifies a month.
- presentValue is a Number or Currency that specifies the present value, or value today, of a series of future payments or receipts.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.

- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

IPmt returns a Number specifying the interest payment for a given period of an annuity based on periodic, fixed payments and a fixed interest rate.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose that you want to take out a \$250,000 loan payable monthly over 15 years at an annual interest rate of 7 percent. The following formula returns the amount of interest that you pay in your first loan payment. Note that the monthly interest rate is $0.07 / 12$ and the number of months of the loan is $15 * 12$.

```
IPmt (0.07 / 12, 1, 15 * 12, 250000)
```

Returns the Number value -1458.33 (rounded to 2 decimals). The value is negative because it represents a payment out from you whereas the loan amount of \$250,000 is positive because it represents a payment in to you.

The following formula returns the amount of interest that you pay in your 121st payment (after 10 years of payments):

```
IPmt (0.07 / 12, 10*12 + 1, 15 * 12, 250000)
```

Returns -661.98 (rounded to 2 decimals). You've made progress on the loan and so less of your monthly payment is for paying interest.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.25 IRR

Basic and Crystal syntax.

Overloads

- IRR (values)
- IRR (values, guess)

Arguments

- values is a Number or Currency type array that specifies cash flow values. The array must contain at least one negative value (a payment) and one positive value (a receipt). The cash flows must occur at regular intervals such as monthly or yearly.
- guess is an optional Number value that is estimated to be returned by IRR. If omitted, guess is 0.1 (10 percent).

Returns

Number value

Action

IRR returns a Number specifying the internal rate of return for a series of periodic cash flows (payments and receipts).

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose that you can choose one of two offers: \$20,000 now or guaranteed payments of \$5,000 after 1 year, \$10,000 after 2 years and \$15,000 after 3 years. What is the better offer? One way to quantify this is to calculate the internal rate of return. If you take the second offer, you can't take the first so that is like experiencing an initial payment of \$20,000 followed by the receipts:

```
Rem Basic syntax
formula = IRR (Array(-20000, 5000, 10000, 15000))
//Crystal syntax
```

```
IRR ([-20000, 5000, 10000, 15000])
```

Returns 0.194 (rounded to 3 decimals) or 19.4 percent interest. So all other things being equal, if you think that 19.4 percent is a good rate of return, you would prefer the second offer.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The NPV and IRR functions are related since $\text{NPV}(\text{IRR}(\text{values}), \text{values}) = 0$. That is, the internal rate of return of a sequence of cash flows is the interest rate for which that sequence of cash flows has a net present value of 0.
- There is no direct formula for the IRR function and so Crystal Reports calculates the value by iteration. The process depends on the initial guess for the internal rate of return. If the program reports an error, try changing the value of the guess argument to be closer to what you expect the internal rate of return should be.

41.5.26 ISPMT (rate, period, nPeriods, presentValue)

Basic and Crystal syntax.

Arguments

- rate is a number specifying the interest rate per period.
- period is a number specifying the period of interest.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate and nPeriods must consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate.
- presentValue is the value of the investment or loan.

Returns

Currency value

Action

ISPMT returns the interest paid during a specified period. This function assumes that equal installments are paid off against the principal each period.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose \$1000 is borrowed for ten years at an interest rate of 4%, and the \$1000 is to be paid back in equal installments per year.

```
ISPMT(0.04, 0, 10, 1000)
```

Returns -\$40. This is the interest due for the first year (zeroth period). For the first year the full \$1000 is outstanding at 4% interest, so $\$1000 \times 0.04 = \40 . The value is negative since it represents a payment of \$40.

```
ISPMT(0.04, 3, 10, 1000)
```

Returns -\$28. This is the interest due for the fourth year (third period). By this time, three installments of \$100 have been paid so the interest due is $(\$1000 - 3 \times \$100) \times 0.04 = \$28$.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [IPmt \[page 1629\]](#)
- [PPmt \[page 1649\]](#)
- [Pmt \[page 1647\]](#)

41.5.27 MDuration

Basic and Crystal syntax.

Overloads

- MDuration (settlementDate, maturityDate, couponRate, yield, frequency)
- MDuration (settlementDate, maturityDate, couponRate, yield, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- couponRate is a non-negative number specifying the interest rate for the security.
- yield is a non-negative number specifying the security's yield.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

MDuration returns a number representing the modified duration of a bond. Duration is the weighted average of the coupon payment cash flows and is a measure of the bond's interest rate risk and MDuration divides the duration by $(1 + \text{yield}/\text{frequency})$.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a ten-year bond with a coupon rate of 12%, a yield of 13% and semiannual payments is purchased on January 25, 2000 and has a maturity date of October 1, 2003. The default basis of 30/360 will be used.

```
MDuration(DateValue(2000,1,25),DateValue(2003,10,1),0.12,0.13,2)
```

Returns 2.781144 as the bond's modified duration.

Comments

This function is similar to the Excel function of the same name.

See Also

- [Duration \[page 1622\]](#)

41.5.28 MIRR (values, financeRate, reinvestRate)

Basic and Crystal syntax.

Arguments

- values is a Number or Currency type array that specifies cash flow values. The array must contain at least one negative value (a payment) and one positive value (a receipt). The cash flows must occur at regular intervals such as monthly or yearly.
- financeRate is a Number that specifies the interest rate paid as the cost of financing.
- reinvestRate is a Number that specifies the interest rate received on gains from cash reinvestment.

Returns

Number value

Action

MIRR returns a Number specifying the modified internal rate of return for a series of periodic cash flows (payments and receipts).

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose that you run a business that makes equipment investments, which result in a loss in your first and fourth years. Your expected annual returns are: -\$50,000, \$40,000, \$65,000, -\$60,000, \$50,000, \$70,000.

Your losses are financed at 10 percent while you reinvest your earnings in an account at 6 percent. The modified internal rate of return is:

```
Rem Basic syntax  
formula = MIRR(Array(-50000, 40000, 65000, -60000, 50000, 70000), 0.10, 0.06)  
//Crystal syntax  
MIRR([-50000, 40000, 65000, -60000, 50000, 70000], 0.10, 0.06)
```

Returns 0.214 (rounded to 3 decimals) or 21.4 percent.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.29 Nominal (rate, nCompoundingPeriods)

Basic and Crystal syntax.

Arguments

- rate is a number that specifies the annual interest rate.
- nCompoundingPeriods is a positive number that specifies the number of compounding periods per year.

Returns

Number value

Action

Nominal returns the nominal annual interest rate given the effective interest rate and the number of compounding periods.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a new savings account is advertised with a compounding interest rate (or effective interest rate) of 4.02% on semiannual interest payments, which you want to compare with your existing account.

```
Nominal (0.0402, 2)
```

Returns 0.0398 (rounded to four decimal places) which corresponds to an annual interest rate of 3.98%.

Comments

This function is designed to work like the Excel function of the same name.

See Also

- [Effect \(rate, nCompoundingPeriods\) \[page 1624\]](#)

41.5.30 NPer

Basic and Crystal syntax.

Overloads

- NPer (rate, payment, presentValue)
- NPer (rate, payment, presentValue, futureValue)
- NPer (rate, payment, presentValue, futureValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- payment is a Number or Currency that specifies the payment to be made each period. Payments usually contain principal and interest that doesn't change over the life of the annuity.
- presentValue is a Number or Currency that specifies the present value, or value today, of a series of future payments or receipts.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.
- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

NPer returns a Number specifying the number of periods for an annuity based on periodic, fixed payments and a fixed interest rate.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose that you want a \$200,000 loan to buy a house. The interest rate is 7 percent and you can afford to pay \$2500 per month. How long of a mortgage do you need?

```
NPer (0.07/12, -2500, 200000)
```

Returns 108.08 (rounded to 2 decimals) months. That is $108.08 / 12 = 9$ years.

Rather than seeking a loan, suppose that you want to save up \$200,000 and then buy the house. You can get a 7 percent interest rate compounded monthly from your savings plan and want to save \$2500 per month. How long will you need to save to have the money?

```
NPer (0.07/12, -2500, 0, 200000)
```

Returns 65.85 (rounded to 2 decimals) months. That is about 5.5 years.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.31 NPV (rate, values)

Basic and Crystal syntax.

Arguments

- rate is a Number that specifies the discount rate over the length of the period, expressed as a decimal.
- values is a Number or Currency type array that specifies cash flow values. Negative values represent payments and positive values receipts. The cash flows must occur at regular intervals such as monthly or yearly.

Returns

Number value

Action

NPV returns a Number specifying the net present value of an investment based on a series of periodic cash flows (payments and receipts) and a discount rate.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose that someone offers to pay you \$1000 after 1 year, \$2000 after 2 years, \$1500 after 3 years and \$1200 after 4 years. If the discount rate (the time value of money) is 5 percent, the value of this offer to you today is:

```
Rem Basic syntax
formula = NPV (0.05, Array (1000, 2000, 1500, 1200))
//Crystal syntax
NPV (0.05, [1000, 2000, 1500, 1200])
```

The formula returns 5049.44 (rounded to 2 decimals). So this scheme is worth \$5,049.44 to you today. This is less than the sum of the payments, which is \$5700, since you have to wait for this money.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The NPV and [IRR \[page 1631\]](#) functions are related since $\text{NPV}(\text{IRR}(\text{values}), \text{values}) = 0$. That is, the internal rate of return of a sequence of cash flows is the discount rate for which that sequence of cash flows has a net present value of 0.

41.5.32 OddFPrice

Basic and Crystal syntax.

Overloads

- OddFPrice (settlementDate, maturityDate, issueDate, firstCouponDate, rate, yield, redemptionValue, frequency)
- OddFPrice (settlementDate, maturityDate, issueDate, firstCouponDate, rate, yield, redemptionValue, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- firstCouponDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security has its first interest payment. This date must be after the settlement date but before the maturity date. For example, if the security pays quarterly then the first coupon date must be an exact number of quarters before the maturity date.
- rate is a non-negative number specifying the interest rate for the coupons that the security pays.
- yield is a non-negative number specifying the security's yield.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

OddFPrice returns the price of a security that pays interest periodically, but has an odd first period. The first period can be shorter or longer than the other periods.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a bond was issued on October 15, 1999 that had a first coupon date of September 1, 2000 and a maturity of September 1, 2005. The bond's interest rate was 4.5% and yield was 5%. The bond has semiannual interest payments (after the first odd period), a \$100 redemption price (per \$100 of face value) and uses a 30/360 basis. If the bond is purchased on January 22, 2000 then the following formula calculates the bond's price.

```
OddFPrice (DateValue(2000,1,22), DateValue(2005,9,1), DateValue(1999,10,15),  
DateValue (2000,9,1), 0.045, 0.05, 100, 2, 0)
```

Returns \$97.54. Note that this function calculates the price given the yield while the OddFYield function calculates the yield given the price.

Comments

- This function is similar to the Excel function of the same name.
- This function is the inverse of the related OddFYield function.

See Also

- [OddFYield \[page 1641\]](#)
- [OddLPrice \[page 1643\]](#)
- [OddLYield \[page 1645\]](#)
- [Price \[page 1651\]](#)
- [Yield \[page 1672\]](#)

41.5.33 OddFYield

Basic and Crystal syntax.

Overloads

- OddFYield (settlementDate, maturityDate, issueDate, firstCouponDate, rate, price, redemptionValue, frequency)
- OddFYield (settlementDate, maturityDate, issueDate, firstCouponDate, rate, price, redemptionValue, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- firstCouponDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security has its first interest payment. This date must be after the settlement date but before the maturity date. For example, if the security pays quarterly then the first coupon date must be an exact number of quarters before the maturity date.
- rate is a non-negative number specifying the interest rate for the coupons that the security pays.
- price is a non-negative number or currency specifying the security's purchase price per \$100 of face value.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

OddFYield returns the yield of a security that pays interest periodically but has an odd first period. The first period can be shorter or longer than the other periods.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a bond was issued on October 15, 1999, with a coupon date of September 1, 2000 and a maturity of September 1, 2005. The bond's interest rate was 4.5%, and the price was \$97.54. The bond has semiannual interest payments (after the first odd period), a \$100 redemption price (per \$100 of face value) and uses a 30/360 basis. If the bond is purchased on January 22, 2000 then the following formula calculates the bond's yield.

```
OddFYield(DateValue(2000,1,22), DateValue(2005,9,1), DateValue(1999,10,15),  
DateValue(2000,9,1), 0.045, 97.54, 100, 2, 0)
```

Returns 0.0500 (rounded to four decimal places) representing a yield of 5%.

Comments

- This function is similar to the Excel function of the same name.
- This function is the inverse of the related OddFPrice function.
- Since there is no explicit function to calculate the yield, Crystal Reports uses an iterative technique using the formula for OddFPrice. A value for the yield is estimated and the odd first price calculated. This calculated price is compared to the specified price and a new yield is estimated. This process is repeated until the price calculated equals the specified price.

See Also

- [OddFPrice \[page 1640\]](#)
- [OddLPrice \[page 1643\]](#)
- [OddLYield \[page 1645\]](#)
- [Price \[page 1651\]](#)
- [Yield \[page 1672\]](#)

41.5.34 OddLPrice

Basic and Crystal syntax.

Overloads

- OddLPrice (settlementDate, maturityDate, lastInterestDate, rate, yield, redemptionValue, frequency)

- OddLPrice (settlementDate, maturityDate, lastInterestDate, rate, yield, redemptionValue, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- lastInterestDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security has its last payment. This date must be before the settlementDate.
- rate is a non-negative number specifying the interest rate for the coupons that the security pays.
- yield is a non-negative number specifying the security's yield.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

OddLPrice returns the price of a security that pays interest periodically but has an odd last period. The last period can be shorter or longer than the other periods.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a bond matured on June 25, 2000 that had an interest rate of 4.5% and a yield of 4.15%. The last interest date before maturity was September 30, 1999 and the bond was purchased on January 10, 2000. With

quarterly interest payments, a \$100 redemption price and a 30/360 basis, the purchase price can be found using the following formula:

```
OddLPrice(DateValue(2000,1,10), DateValue(2000,6,25), DateValue(1999,9,30),  
0.045, 0.0415, 100, 4, 0)
```

Returns \$100.13 (rounded to the nearest cent).

Comments

- This function is similar to the Excel function of the same name.
- This function is the inverse of the related function OddLYield.

See Also

- [OddFPrice \[page 1640\]](#)
- [OddFYield \[page 1641\]](#)
- [OddLYield \[page 1645\]](#)
- [Price \[page 1651\]](#)
- [Yield \[page 1672\]](#)

41.5.35 OddLYield

Basic and Crystal syntax.

Overloads

- OddLYield (settlementDate, maturityDate, lastInterestDate, rate, price, redemptionValue, frequency)
- OddLYield (settlementDate, maturityDate, lastInterestDate, rate, price, redemptionValue, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.

- lastInterestDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security has its last payment. This date must be before the settlementDate.
- rate is a non-negative number specifying the interest rate for the coupons that the security pays.
- price is a non-negative number or currency specifying the security's purchase price per \$100 of face value.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

OddLYield returns the yield of a security that pays interest periodically but has an odd last period. The last period can be shorter or longer than the other periods.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a bond matured on June 25, 2000 with an interest rate of 4.5% and a price of \$100.13. The last interest date before maturity was September 30, 1999 and the bond was purchased on January 10, 2000. With quarterly interest payments, a \$100 redemption price and a 30/360 basis, the yield can be found using the following formula:

```
OddLYield(DateValue(2000,1,10), DateValue(2000,6,25), DateValue(1999,9,30),
0.045, 100.13, 100, 4, 0)
```

Returns 0.0416 (rounded to four decimal places), representing a yield of 4.16%.

Comments

- This function is similar to the Excel function of the same name.
- This function is the inverse of the related OddLPrice function.
- Since there is no explicit function to calculate the yield, Crystal Reports uses an iterative technique using the formula for OddLPrice. A value for the yield is estimated and the odd last price calculated. This calculated price is compared to the specified price and a new yield is estimated. This process is repeated until the price calculated equals the specified price.

See Also

- [OddFPrice \[page 1640\]](#)
- [OddFYield \[page 1641\]](#)
- [OddLPrice \[page 1643\]](#)
- [Price \[page 1651\]](#)
- [Yield \[page 1672\]](#)

41.5.36 Pmt

Basic and Crystal syntax.

Overloads

- Pmt (rate, nPeriods, presentValue)
- Pmt (rate, nPeriods, presentValue, futureValue)
- Pmt (rate, nPeriods, presentValue, futureValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate and nPeriods must consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate.
- presentValue is a Number or Currency that specifies the present value or principal. That is, the amount that a series of payments in the future is worth now.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.

- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

Pmt returns a Number specifying the payment for an annuity based on periodic, fixed payments and a fixed interest rate.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose that you want to take out a \$250,000 loan payable monthly over 15 years at an annual interest rate of 7 percent. The following formula returns your monthly loan payment. Note that the monthly interest rate is $0.07 / 12$ and the number of months of the loan is $15 * 12$.

```
Pmt (0.07 / 12, 15 * 12, 250000)
```

Returns the Number value -2247.07 (rounded to 2 decimals). The value is negative because it represents a payment out from you whereas the loan amount of \$250,000 is positive because it represents a payment in to you.

Now suppose that the payments are made at the beginning of the month instead of the end (as is the default). Your monthly loan payment is calculated as:

```
Pmt (0.07 / 12, 15 * 12, 250000, 0, 1)
```

Returns -2234.04 (rounded to 2 decimals). Note that your monthly payment is about \$13 less each month than in the previous example where payments are made at the end of the month.

Now suppose that you know that you'll receive \$100,000 in 15 years so there is no need to fully pay off the loan; only to reduce the amount owed to \$100,000 after 15 years. Note that the future value is negative since after 15 years you need to pay out \$100,000 to clear the loan. Your monthly loan payment is calculated as:

```
Pmt (0.07 / 12, 15 * 12, 250000, -100000)
```

Returns -1931.58 (rounded to 2 decimals).

Comments

- This function is designed to work like the Visual Basic function of the same name.
- To find the total amount paid out over the whole loan, multiply the payment per period (the value returned by Pmt) by the total number of periods (nPeriods).

41.5.37 PPmt

Basic and Crystal syntax.

Overloads

- PPmt (rate, period, nPeriods, presentValue)
- PPmt (rate, period, nPeriods, presentValue, futureValue)
- PPmt (rate, period, nPeriods, presentValue, futureValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- period is a Number that specifies the payment period in the range 1 through nPeriods.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate, period and nPeriods must be consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate and period specifies a month.
- presentValue is a Number or Currency that specifies the present value, or value today, of a series of future payments or receipts.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.
- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

PPmt returns a Number specifying the principal payment for a given period of an annuity based on periodic, fixed payments and a fixed interest rate.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose that you want to take out a \$250,000 loan payable monthly over 15 years at an annual interest rate of 7 percent. The following formula returns the amount of principal that you pay in your first loan payment. Note that the monthly interest rate is $0.07 / 12$ and the number of months of the loan is $15 * 12$.

```
PPmt (0.07 / 12, 1, 15 * 12, 250000)
```

Returns the Number value -788.74 (rounded to 2 decimals). The value is negative because it represents a payment out from you whereas the loan amount of \$250,000 is positive because it represents a payment in to you.

The following formula returns the amount of principal that you pay in your 121st payment (after 10 years of payments):

```
PPmt (0.07 / 12, 10*12 + 1, 15 * 12, 250000)
```

Returns -1585.09 (rounded to 2 decimals). You've made progress on the loan and are paying off more of the principal per payment. This is because less interest can accrue each month since more of the loan is paid off so that your fixed monthly payment is applied more to the principal.

The following Basic syntax formula returns the amount of the loan that has been paid off after 10 years by summing up the principal payments made for the first 120 monthly payment:

```
Rem Basic syntax
Dim total, i
total = 0
For i = 1 To 10 * 12
    total = total + PPmt (0.07 / 12, i, 15 * 12, 250000)
Next i
formula = total
```

Returns -136518.45 (rounded to 2 decimals). So you have paid off \$136,518.45 of the loan and still owe \$113,481.55 after the 120th payment.

Another way to calculate the amount you still owe after 10 years is to use 2 financial functions:

```
Rem Basic syntax
Dim payment
payment = Pmt (0.07 / 12, 15 * 12, 250000)
formula = FV (0.07 / 12, 10 * 12, payment, 250000)
```

Returns -113481.55 (rounded to 2 decimals). So you still owe \$113,481.55 after 10 years of payments.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.38 Price

Basic and Crystal syntax.

Overloads

- Price (settlementDate, maturityDate, couponRate, yield, redemptionValue, frequency)
- Price (settlementDate, maturityDate, couponRate, yield, redemptionValue, frequency, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- couponRate is a non-negative number specifying the interest rate for the security.
- yield is a non-negative number specifying the security's yield.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- frequency is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

Price returns the price of a security that pays interest periodically per \$100 of face value. There are three parts to the calculation. The first term calculates the present value of the redemption price, the second term handles the present value of the coupon payments and the last term subtracts the accrued interest payments.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a municipal bond is purchased on March 1, 1999 with a maturity date of January 1, 2005. The interest rate is 6.7% and the yield is 6.5%. At redemption, the face value of the bond is returned for a value of \$100 and the interest is paid out quarterly.

```
Price(DateValue(1999,3,1), DateValue(2005,1,1), 0.067, 0.065, 100, 4, 0)
```

Returns \$100.96 (rounded to the nearest cent) as the bond's price.

Comments

This function is similar to the Excel function of the same name.

See Also

- [PriceDisc \[page 1652\]](#)
- [PriceMat \[page 1654\]](#)
- [Yield \[page 1672\]](#)
- [YieldDisc \[page 1674\]](#)
- [YieldMat \[page 1676\]](#)

41.5.39 PriceDisc

Basic and Crystal syntax.

Overloads

- PriceDisc (settlementDate, maturityDate, discountRate, redemptionValue)

- PriceDisc (settlementDate, maturityDate, discountRate, redemptionValue, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- discountRate is a positive number specifying the security's discount rate.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

PriceDisc returns the price of a discounted security per \$100 of face value. The formula subtracts the discounted amount from the redemption value.

Examples

The following example is applicable to both Basic and Crystal syntax:

U.S. Treasury bills are purchased on a discounted basis and are redeemed at face value. Suppose that a bill is purchased on July 15, 1999, it matures on October 1, 1999 and has a discount rate of 6.3%. Using an actual/360 basis for this discounted security, the price is found using the following formula:

```
PriceDisc(DateValue(1999,7,15), DateValue(1999,10,1), 0.063, 100, 2)
```

Returns \$98.64 (rounded to the nearest cent) as the Treasury bill's price. Since there are 78 days between the settlement and maturity, the discounted amount is $100 \times 0.063 \times (78/360) = 1.365$. This is subtracted from the \$100 redemption value to get the answer.

Comments

This function is similar to the Excel function of the same name.

See Also

- [Price \[page 1651\]](#)
- [PriceMat \[page 1654\]](#)
- [Yield \[page 1672\]](#)
- [YieldDisc \[page 1674\]](#)
- [YieldMat \[page 1676\]](#)

41.5.40 PriceMat

Basic and Crystal syntax.

Overloads

- PriceMat (settlementDate, maturityDate, issueDate, interestRate, yield)
- PriceMat (settlementDate, maturityDate, issueDate, interestRate, yield, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- interestRate is a non-negative number specifying the interest rate for the security.
- yield is a non-negative number specifying the security's yield.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

PriceMat returns the price of a security that pays all the interest at maturity per \$100 of face value. The formula subtracts the interest accrued by the seller, from the present value of the amount received at maturity, based on the yield and the duration of the security.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a \$150,000 Certificate of Deposit pays interest of 4.3% at maturity and has a yield of 4.0%. If the certificate was issued on January 1, 2000, purchased on March 4, 2000 and has a maturity date of July 31, 2000, then the following formula, using the 30/360 basis, can be used to calculate the price.

```
150000/100 * PriceMat(DateValue(2000,3,4), DateValue(2000,7,31),  
DateValue(2000,1,1), 0.043, 0.04)
```

This example returns \$150,162.66 (rounded to the nearest cent) as the certificate's price. The face value of the certificate has to be divided by 100, since the function returns the price per \$100.

Comments

This function is similar to the Excel function of the same name.

See Also

- [Price \[page 1651\]](#)
- [PriceDisc \[page 1652\]](#)
- [Yield \[page 1672\]](#)
- [YieldDisc \[page 1674\]](#)
- [YieldMat \[page 1676\]](#)

41.5.41 PV

Basic and Crystal syntax.

Overloads

- PV (rate, nPeriods, payment)
- PV (rate, nPeriods, payment, futureValue)
- PV (rate, nPeriods, payment, futureValue, type)

Arguments

- rate is a Number that specifies the interest rate per period.
- nPeriods is a positive Number that specifies the total number of payment periods in the annuity. The units used for specifying rate and nPeriods must be consistent. For example, if nPeriods is the number of periods in months, then rate is a monthly interest rate.
- payment is a Number or Currency that specifies payment to be made each period.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.
- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.

Returns

Number value

Action

PV returns a Number specifying the present value of an annuity based on periodic, fixed payments to be paid in the future and a fixed interest rate.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose that you want to buy a condo and can make payments of \$1100 twice a month (24 annual payments). If the mortgage rates are 7 percent, and you want to pay off the condo in 10 years, what is the maximum loan that you can take out?

```
PV (0.07 / 24, 10 * 24, -1100)
```

Returns 189668 (rounded to the nearest dollar). Thus you can afford a loan of about \$190,000. Notice that the payment argument is negative since you are paying out the money each month.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.42 Rate

Basic and Crystal syntax.

Overloads

- Rate (nPeriods, payment, presentValue)
- Rate (nPeriods, payment, presentValue, futureValue)
- Rate (nPeriods, payment, presentValue, futureValue, type)
- Rate (nPeriods, payment, presentValue, futureValue, type, guess)

Arguments

- nPeriods is a positive Number that specifies the total number of payment periods in the annuity.
- payment is a Number or Currency that specifies payment to be made each period.
- presentValue is a Number or Currency that specifies the present value, or value today, of a series of future payments or receipts.
- futureValue is an optional Number or Currency that specifies the future value or cash balance you want after you've made the final payment. If omitted, 0 is assumed.
- type is an optional Number that specifies when payments are due. Specify 0 if payments are due at the end of the payment period, and 1 if payments are due at the beginning of the period. If omitted, 0 is assumed.
- guess is an optional Number value that is estimated to be returned by Rate. If omitted, guess is 0.1 (10 percent).

Returns

Number value

Action

Rate returns a Number specifying the interest rate per period for an annuity. The units of the returned value are consistent with the units of nPeriods. For example, if nPeriods is in months, then the rate returned will be a monthly interest rate.

Examples

The following examples are applicable to both Basic and Crystal syntax:

An electronics store offers to finance a \$1500 television for \$70 per month, over 2 years, with no money down. Is this a good deal? A first step to determine this is to figure out what interest rate the store is charging:

```
Rate (2 * 12, -70, 1500)
```

Returns 0.00927 (rounded to 5 decimals). Notice that nPeriods is 24 months, payment (-70) is negative since you are paying out the monthly payments, and the present value (1500) is positive since at the start of the loan, you effectively received \$1500 (the value of the television). The interest rate returned is a monthly interest rate, since nPeriods was in months.

This next expression calculates the interest rate but expressed as a yearly interest rate and as a percent.

```
Rate (2 * 12, -70, 1500) * 12 * 100
```

Returns 11.1 (rounded to 1 decimal). Thus, the store is charging an effective annual interest rate of 11.1 percent.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- There is no direct formula for the Rate function and so Crystal Reports calculates the value by iteration. The process depends on the initial guess for the rate. If the program reports an error, try changing the value of the guess argument to be closer to what you expect the interest rate should be.

41.5.43 Received

Basic and Crystal syntax.

Overloads

- Received (settlementDate, maturityDate, investment, discountRate)
- Received (settlementDate, maturityDate, investment, discountRate, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- investment is a positive number or currency specifying the amount of the investment.
- discountRate is a positive number specifying the discount rate for the security.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Currency value

Action

Received returns the amount received for a "fully invested" security at maturity. Discounted securities are usually sold at an even par value at maturity. Therefore when purchased, the value of the security is discounted to some non-round number. Since investors commonly want to "fully invest" or invest round dollar amounts (such as \$100,000) special calculations are needed. The Received function calculates a par value such that when the discount rate is applied for the time period specified, the discounted value is equal to the investment amount.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose \$10,000 is invested at a discount rate of 6.5% between March 15, 2000 and the security's maturity date on June 1, 2000 using the American 30/360 basis.

```
Received (DateValue(2000, 3, 15), DateValue(2000, 6, 1), 10000, 0.065, 0)
```

Returns \$10,139.13 as the par value for this security.

Comments

This function is similar to the Excel function of the same name.

See Also

- [IntRate \[page 1627\]](#)

41.5.44 SLN (cost, salvage, life)

Basic and Crystal syntax.

Arguments

- cost is a Number or Currency that specifies the initial cost of the asset.
- salvage is a Number or Currency that specifies the value of the asset at the end of its useful life.
- life is a Number that specifies the length of the useful life of the asset. It must not equal 0.

Returns

Number value

Action

SLN returns a Number specifying the straight-line depreciation of an asset for a single period.

Example

The following example is applicable to both Basic and Crystal syntax:

Suppose a company purchases a fleet of cars for \$500,000. The cars have a lifetime of 10 years and a salvage value of \$60,000. The depreciation per year is:

```
SLN (500000, 60000, 10)
```

Returns 44000. Thus the depreciation per year is \$44,000.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.45 SYD (cost, salvage, life, period)

Basic and Crystal syntax.

Arguments

- cost is a Number or Currency that specifies the initial cost of the asset.
- salvage is a Number or Currency that specifies the value of the asset at the end of its useful life.
- life is a positive Number that specifies the length of the useful life of the asset.
- period is a Number that specifies the period for which asset depreciation is calculated. The value is positive and less than or equal to life. The arguments life and period must have the same units.

Returns

Number value

Action

SYD returns a Number specifying the sum-of-years' digits depreciation of an asset for a single period.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose a company purchases a fleet of cars for \$500,000. The cars have a lifetime of 10 years and a salvage value of \$60,000. They are depreciated as follows:

```
SYD (500000, 60000, 10, 1)
```

Returns 80000. The first year's depreciation is \$80,000.

```
SYD (500000, 60000, 10, 4)
```

Returns 56000. The fourth year's depreciation is \$56,000.

```
SYD (500000, 60000, 10, 10)
```

Returns 8000. The final year's depreciation is \$8,000.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.5.46 TBillEq (settlementDate, maturityDate, discountRate)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the Treasury bill was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the Treasury bill matures.
- discountRate is a positive number specifying the discount rate for the Treasury bill.

Returns

Number value

Action

TBillEq returns the equivalent bond yield equivalent for a T-Bill's discount rate. Treasury bills are a promise to pay a fixed amount on a certain day. They are purchased at a discount from this amount and the return on the investment is the difference between the purchase price and the face value. The discount rate is calculated based on the face value and a 360-day year. In contrast, bond yields are calculated based on the purchase price and a full-length year basis.

Examples

The following example is applicable to both Basic and Crystal syntax:

If a T-Bill was purchased on February 15, 2000 with a maturity of July 1, 2000, at a discount rate of 8.5%, the equivalent yield can be found by the formula:

```
TBillEq(DateValue(2000, 2, 15), DateValue(2000, 7, 1), 0.085)
```

Returns 0.0891 (rounded to four decimal places) corresponding to an interest rate of 8.91%.

Comments

This function is similar to the Excel function of the same name.

See Also

- [TBillYield \(settlementDate, maturityDate, price\) \[page 1665\]](#)
- [TBillPrice \(settlementDate, maturityDate, discountRate\) \[page 1663\]](#)
- [DISC \[page 1618\]](#)

41.5.47 TBillPrice (settlementDate, maturityDate, discountRate)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the Treasury bill was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the Treasury bill matures.
- discountRate is a positive number specifying the discount rate for the Treasury bill.

Returns

Currency value

Action

TBillPrice returns a currency representing the current price per \$100 face value of a Treasury bill given the discount rate. The discount rate is a calculation based on the return on investment and the length of time between settlement and maturity. This function works this relation in reverse, calculating \$100 minus the return on investment given the discount rate. Treasury bills use an actual/360 basis.

Examples

The following example is applicable to both Basic and Crystal syntax:

If a T-Bill was purchased on February 15, 2000 with a maturity of July 1, 2000, at a discount rate of 8.5%, the purchase price is found by:

```
TBillPrice(DateValue(2000, 2, 15), DateValue(2000, 7, 1), 0.085)
```

Returns \$96.77 (rounded to the nearest cent).

Comments

- This function is similar to the Excel function of the same name.
- The functions DISC and TBillPrice are inverse relations. The discount rate DISC using the price returned by TBillPrice, a redemption value of \$100 and using a basis of 2 (actual/360) is the same discount rate entered in the TBillPrice function. That is, $r = \text{DISC}(a, b, \text{TBillPrice}(a, b, r), 100)$ where a and b are dates and r is the discount rate.

See Also

- [TBillEq \(settlementDate, maturityDate, discountRate\) \[page 1662\]](#)
- [TBillYield \(settlementDate, maturityDate, price\) \[page 1665\]](#)
- [DISC \[page 1618\]](#)

41.5.48 TBillYield (settlementDate, maturityDate, price)

Basic and Crystal syntax.

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the Treasury bill was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the Treasury bill matures. Treasury bills typically have maturities of 90 days, 182 days and 52 weeks.
- price is a positive number or currency specifying the purchase price for a Treasury bill per \$100 of par value.

Returns

Number value

Action

TBillYield returns a number representing the yield on a Treasury bill. This is the amount the Treasury bill returns, as a percentage of the purchase price annualized using an actual/360 basis.

Examples

The following example is applicable to both Basic and Crystal syntax:

If a \$10,000 T-Bill was purchased on February 15, 2000, with a maturity date of July 1, 2000, at a purchase price of \$9,850.00, the yield on this investment is:

```
TBillYield(DateValue(2000, 2, 15), DateValue(2000, 7, 1), 98.50)
```

Returns 0.0400 (rounded to four decimal places). The purchase price of \$9,850 had to be divided by (100/10000) to get the price per \$100. The return, in this case, is \$1.50 per \$100 and the investment is held for 137 days of the 360-day year. The value returned is $(1.5/98.50)/(137/360)$.

Comments

This function is similar to the Excel function of the same name.

See Also

- [TBillEq \(settlementDate, maturityDate, discountRate\) \[page 1662\]](#)
- [TBillPrice \(settlementDate, maturityDate, discountRate\) \[page 1663\]](#)
- [DISC \[page 1618\]](#)

41.5.49 VDB

Basic and Crystal syntax.

Overloads

- VDB (cost, salvage, lifetime, startPeriod, endPeriod)
- VDB (cost, salvage, lifetime, startPeriod, endPeriod, depreciationFactor)
- VDB (cost, salvage, lifetime, startPeriod, endPeriod, noSwitch)
- VDB (cost, salvage, lifetime, startPeriod, endPeriod, depreciationFactor, noSwitch)

Arguments

- cost is a number or currency that specifies the initial cost of the asset.
- salvage is a number or currency that specifies the value of the asset at the end of its useful life.
- lifetime is a positive number specifying the number of periods in the lifetime of the asset.
- startPeriod is a positive number specifying the starting point of the calculation for the depreciation total. The startPeriod is not included in the calculation.
- endPeriod is a number specifying the last period that is used in the calculation of the depreciation. This number must be equal to, or greater than the start period, and cannot be larger than the lifetime of the asset.

- depreciationFactor is a non-negative number specifying the rate at which the asset depreciates. The default value is two, based on the double declining balance method.
- noSwitch is a boolean value determining if the function switches to a straight line depreciation if the declining balance method is less than the straight line method. The default FALSE value causes VDB to switch to the linear method while a TRUE value forces the declining balance method to be used.

Returns

Currency value

Action

VDB returns the total depreciation between the start and end periods specified. If noSwitch is FALSE, then the sum of the depreciations over the lifetime of the asset will sum to the difference between the cost and the salvage.

The switch between methods occurs when the remaining depreciation room, divided by the number of periods left in the lifetime, is more than the value calculated using the declining balance method. Depreciation room is the salvage and cumulative depreciation subtracted from the cost of the asset.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a machine is purchased for \$1000, and has a salvage value of \$250 and depreciates using the double-declining method. The lifetime is 10 periods (years).

```
VDB (1000, 250, 10, 0, 5, 2, FALSE)
```

Returns \$672.32 as the depreciation for the first 5 years of the machine's lifetime. The value is the same in this situation for noSwitch=TRUE.

```
VDB (1000, 250, 10, 5, 6, 1, FALSE)
```

Returns \$67.68 (rounded to the nearest cent) as the depreciation in only the sixth year of the machine's lifetime. The value calculated with noSwitch=FALSE is \$59.05 (rounded). This shows that in the first case, VDB has switched to the linearly declining method, which means that all later periods will also have a depreciation of \$67.68 per year.

Comments

This function is similar to the Excel function of the same name.

See Also

- [DB \[page 1614\]](#)
- [DDB \[page 1615\]](#)
- [AmorLINC \[page 1600\]](#)
- [AmorDEGRC \[page 1598\]](#)

41.5.50 XIRR

Basic and Crystal syntax.

Overloads

- XIRR (values, dates)
- XIRR (values, dates, rateGuess)

Arguments

- values is a number or currency type array that specifies cash flow values. The array must contain at least one negative value (a payment) and one positive value (a receipt). These transactions occur on the dates specified by the dates argument.
- dates is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) type array specifying the dates of the cash flows specified in the values argument. The first date represents the start of the schedule. The dates may occur in any order as long as they are after the first date. The array of values must be the same size as the array of dates.
- rateGuess is an optional number value that is estimated to be returned by IRR. If omitted, the value 0.1 (10 percent) is used.

Returns

Number value

Action

XIRR calculates the internal rate of return for a series of cash flows (payments and receipts).

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a venture capital deal has the following conditions. In return for a \$100,000 investment paid on March 1, 1999, the business returned \$60,000 on August 31, 1999, and a \$35,000 installment on November 1, 1999. Another \$50,000 in capital was put into the venture on January 1, 2000. The deal was closed on April 15, 2000 with a \$75,000 payout. A summary of the cash flows is:

March 1, 1999	-\$100,000
August 31, 1999	\$60,000
November 1, 1999	\$35,000
January 1, 2000	-\$50,000
April 15, 2000	\$75,000

```
Rem Basic Syntax
formula = XIRR(Array( -100000, 60000, 35000, -50000, 75000),
Array(DateValue(1999, 3, 1), DateValue(1999, 8, 31), DateValue(1999, 11, 1),
DateValue(2000, 1, 1), DateValue(2000, 4, 15)))
//Crystal Syntax
XIRR([ -100000, 60000, 35000, -50000, 75000],
[DateValue(1999, 3, 1), DateValue(1999, 8, 31), DateValue(1999, 11, 1),
DateValue(2000, 1, 1), DateValue(2000, 4, 15)])
```

Returns 0.2605 for an internal rate of return of 26.05%.

Comments

- This function is designed to work like the Excel function of the same name.
- The XNPV and XIRR functions are related since using the rate of return, calculated by XIRR, in the XNPV function, results in zero being returned by XNPV. In other words, the internal rate of return is the interest for which the net present value is zero.
- There is no direct formula for the XIRR function, so Crystal Reports calculates the value by iteration. The process depends on the initial guess for the internal rate of return. If the program reports an error, try changing the value of the guess argument to be closer to what you expect the internal rate of return should be.

See Also

- [XNPV \(rate, values, dates\) \[page 1670\]](#)
- [IRR \[page 1631\]](#)
- [NPV \(rate, values\) \[page 1638\]](#)

41.5.51 XNPV (rate, values, dates)

Basic and Crystal syntax.

Arguments

- rate is a number specifying the annual discount rate.
- values is a number or currency type array, that specifies cash flow values. The array must contain at least one negative value (a payment) and one positive value (a receipt). These transactions occur at the date specified in dates.
- dates is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) type array specifying the dates of the cash flows specified in the values argument. The first date represents the start of the schedule. The dates may occur in any order as long as they are after the first date.

Returns

Currency value

Action

XNPV returns a number specifying the net present value of an investment based on a series of cash flows (payments and receipts) and a discount rate.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose on April 1, 1999, someone was offered \$1000 immediately, \$2500 on July 1, 1999, \$2500 on October 1, 1999 and \$5000 on December 31, 1999. If the discount rate (the time value of money) is 4.5 percent, the value of the offer is:

```
Rem Basic Syntax
formula = XNPV(0.045,Array(1000,2500,2500,5000),
Array(DateValue(1999,4,1),DateValue(1999,7,31),DateValue(1999,10,1),
DateValue(1999,12,31)))
//Crystal Syntax
XNPV(0.045,[1000,2500,2500,5000],
[DateValue(1999,4,1),DateValue(1999,7,31),DateValue(1999,10,1),
DateValue(1999,12,31)])
```

Returns \$10,746.70, which is less than the sum of the payments since the person has to wait for the money.

Comments

- This function is designed to work like the Excel function of the same name.
- The XNPV and XIRR functions are related since using the rate of return, calculated by XIRR, in the XNPV function, results in zero being returned by XNPV. In other words, the internal rate of return is the interest for which the net present value is zero.

See Also

- [XIRR \[page 1668\]](#)
- [IRR \[page 1631\]](#)
- [NPV \(rate, values\) \[page 1638\]](#)

41.5.52 YearFrac

Basic and Crystal syntax.

Overloads

- YearFrac (startDate, endDate)
- YearFrac (startDate, endDate, basis)

Arguments

- startDate is a Date or DateTime for the start of an interval of time.
- endDate is a Date or DateTime for the end of an interval of time.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

YearFrac returns the fraction of a year spanned by the interval of time between the two dates.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
YearFrac(DateValue(2000,1,1),DateValue(2000,4,1))
```

Returns 0.25 since three months between the start of January and the start of April.

Suppose a retroactive pay increase needs to be calculated for all employees, from their first day of work to January 1, 2000. If the database table contains a field called 'StartDate', containing the date the employee started work, then the following formula can be used to determine how many years and partial years each employee is eligible for.

```
yearFrac({StartDate},DateValue(2000,1,1));
```

Comments

This function is similar to the Excel function of the same name.

41.5.53 Yield

Basic and Crystal syntax.

Overloads

- Yield (settlementDate, maturityDate, couponRate, price, redemptionValue, frequency)
- Yield (settlementDate, maturityDate, couponRate, price, redemptionValue, frequency, basis)

Arguments

- `settlementDate` is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- `maturityDate` is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- `couponRate` is a non-negative number specifying the interest rate for the security.
- `price` is a non-negative number or currency specifying the security's purchase price per \$100 of face value.
- `redemptionValue` is a number or currency specifying the security's value at redemption per \$100 of face value.
- `frequency` is a number specifying the number of coupons per year. The supported values are 1 (annual payments), 2 (semiannual payments), and 4 (quarterly).
- `basis` is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

Yield returns the yield for a security that pays interest periodically. Interest is accrued daily when there is one or fewer payment periods left until maturity after the settlement date.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a municipal bond is purchased on March 1, 1999 with a maturity date of January 1, 2005. The interest rate is 6.7% and the purchase price is \$100.96 per \$100 par value. At redemption, the face value of the bond is returned for a value of \$100 and interest is paid out quarterly and a 30/360 basis is used.

```
Yield(DateValue(1999, 3, 1), DateValue(2005, 1, 1), 0.067, 100.96, 100, 4, 0)
```

Returns 0.0650 (rounded to four decimal places) as the bond's yield. Note that this is the reverse operation of [Price \[page 1651\]](#).

Comments

- This function is similar to the Excel function of the same name.
- When multiple coupon periods occur between the settlement date and maturity, Crystal Reports uses an iterative technique to calculate the price, using the formula for Price. A value for the yield is guessed and the price is calculated. The calculated price is compared to the specified price and a new yield is estimated. This process is repeated until the price calculated equals the specified price.

See Also

- [Price \[page 1651\]](#)
- [PriceDisc \[page 1652\]](#)
- [PriceMat \[page 1654\]](#)
- [YieldDisc \[page 1674\]](#)
- [YieldMat \[page 1676\]](#)

41.5.54 YieldDisc

Basic and Crystal syntax.

Overloads

- YieldDisc (settlementDate, maturityDate, price, redemptionValue)
- YieldDisc (settlementDate, maturityDate, price, redemptionValue, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- price is a non-negative number or currency specifying the security's purchase price per \$100 of face value.
- redemptionValue is a number or currency specifying the security's value at redemption per \$100 of face value.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual

- 2 - actual/360
- 3 - actual/365
- 4 - European 30/360

Returns

Number value

Action

YieldDisc returns the yield on a discounted security. The formula divides the difference between the purchase price and the redemption price by the purchase price. This quotient is converted to an annual rate using the specified basis.

Examples

The following example is applicable to both Basic and Crystal syntax:

U.S. Treasury bills are purchased on a discounted basis and are redeemed at face value. Suppose that a bill is purchased on July 15, 1999, matures on October 1, 1999 and has a discounted price of \$98.64. Using an actual/360 basis for this discounted security, the yield is found using the following formula:

```
YieldDisc (DateValue(1999,7,15),DateValue(1999,10,1),98.64,100,2)
```

Returns 0.0636 (rounded to four decimal places) as the Treasury bill's yield.

Comments

- This function is similar to the Excel function of the same name.
- This function is closely related to the PriceDisc function.

See Also

- [Price \[page 1651\]](#)
- [PriceDisc \[page 1652\]](#)
- [PriceMat \[page 1654\]](#)
- [Yield \[page 1672\]](#)

- [YieldMat \[page 1676\]](#)

41.5.55 YieldMat

Basic and Crystal syntax.

Overloads

- YieldMat (settlementDate, maturityDate, issueDate, interestRate, price)
- YieldMat (settlementDate, maturityDate, issueDate, interestRate, price, basis)

Arguments

- settlementDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was purchased.
- maturityDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) after the settlement date specifying when the security matures.
- issueDate is a [Date \[page 1758\]](#) or [DateTime \[page 1762\]](#) specifying when the security was issued.
- interestRate is a non-negative number specifying the interest rate for the security.
- price is a non-negative number or currency specifying the security's purchase price per \$100 of face value.
- basis is an optional number specifying the day basis system to use. The following types are supported:
 - 0 - American 30/360 (default)
 - 1 - actual/actual
 - 2 - actual/360
 - 3 - actual/365
 - 4 - European 30/360

Returns

Number value

Action

YieldMat returns the yield for a security that pays interest at maturity.

Examples

The following example is applicable to both Basic and Crystal syntax:

Suppose a \$150,000 Certificate of Deposit pays interest of 4.3%, and at maturity it has a price of \$150,162.66. If the certificate was issued January 1, 2000, purchased on March 4, 2000 and had a maturity date of July 31, 2000, then the following formula, using the 30/360 basis, can be used to calculate the yield.

```
YieldMat (DateValue(2000,3,4), DateValue(2000,7,31), DateValue(2000,1,1), 0.043,  
150162.66/150000*100)
```

Returns 0.0400 (rounded to four decimal places) as the certificate's yield. The price has to be specified on a per \$100 of face value basis. Note that this is the reverse operation of PriceMat.

Comments

This function is similar to the Excel function of the same name.

See Also

- [Price \[page 1651\]](#)
- [PriceDisc \[page 1652\]](#)
- [PriceMat \[page 1654\]](#)
- [Yield \[page 1672\]](#)
- [YieldDisc \[page 1674\]](#)

41.6 Grid Value

Grid Value functions are used for a variety of Cross-Tab related formulas.

41.6.1 CurrentColumnIndex

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

CurrentColumnIndex returns the column index number for the cell in which the formula is executed. Column index numbers start at 0.00 and increase from left to right.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
CurrentColumnIndex
```

Returns 0.00 for the first column, 1.00 for the second column, and so forth.

Note

Totals and sub-totals are included when counting index numbers.

41.6.2 CurrentRowIndex

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

CurrentRowIndex returns the row index number for the cell in which the formula is executed. Row index numbers start at 0.00 and increase from top to bottom.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
CurrentRowIndex
```

Returns 0.00 for the first row, 1.00 for the second row, and so forth.

Note

Totals and sub-totals are included when counting index numbers.

41.6.3 CurrentSummaryIndex

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

CurrentSummaryIndex returns the summary index number for the cell in which the formula is executed. Summary index numbers start at 0.00. You can specify the order of summaries in the [Embedded Summaries dialog box](#).

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
CurrentSummaryIndex
```

Returns 0.00 for the first summary, 1.00 for the second summary, and so forth.

41.6.4 GetColumnGroupIndexOf (columnIndex)

Basic and Crystal syntax.

Arguments

- Number

Returns

Number

Action

GetColumnGroupIndexOf returns the group index of a specific column within a Cross-Tab.
The Grand Total returns 0.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GetColumnGroupIndexOf (GetIndexOf("Customer.Contact Title","Mr."))
```

Returns 1 when Customer.Contact Title is the first grouping in a column and Customer.Contact Position is the second grouping in a column.

Returns 2 when Customer.Contact Position is the first grouping in a column and Customer.Contact Title is the second grouping in a column.

Note

If do not specify an argument for this function, the return value is the column Grand Total index.

41.6.5 GetColumnPathIndexOf (columnPath)

Basic and Crystal syntax.

Arguments

- The arguments are dependent on the types of the column grouping fields.

Returns

Number

Action

GetColumnPathIndexOf returns the absolute index of the path or -1 if the path cannot be found.

When the path is not specified GetColumnPathIndexOf returns the index of the Grand Total column.

Typical uses

To use with GridValueAt function in Calculation formulas.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GridValueAt(currentRowIndex, GetColumnPathIndexOf("Owner"), CurrentSummaryIndex)  
+ GridValueAt(CurrentRowIndex, GetColumnPathIndexOf("Sales  
Agent"), CurrentSummaryIndex)
```

Returns the sums of all cells in the "Owner" and "Sales Agent" columns.

41.6.6 GetIndexOf (groupAlias, searchValue, (optional) startingIndex)

Basic and Crystal syntax.

Arguments

- String (group alias)
- String (search value)
- Number (optional starting index)

Returns

Number

Action

GetIndexOf returns the row or column index of the row or column with the same value as searchValue. If startingIndex is present, the rows/columns are searched from that starting point. Otherwise, only the group that contains the cell being evaluated is searched.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GetIndexOf("Customer.Contact Position", "Sales Agent")
```

If Sales Agent is the first row or column displayed for Customer.Contact Position, shows 1.00.

If Sales Agent is the second row or column displayed for Customer.Contact Position, shows 2.00.

If Sales Agent is the third row or column displayed for Customer.Contact Position, shows 3.00.

Note

Totals and sub-totals do not count toward index numbers.

41.6.7 GetNumColumnGroups

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

GetNumColumnGroups returns the total number of column groups in the Cross-Tab.

Typical uses

To format cells or modify calculation formulas based on the number of column groupings in a Cross-Tab.

41.6.8 GetNumColumns

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

GetNumColumns returns the total number of columns in the Cross-Tab.

Typical uses

To format cells or modify calculation formulas based on the number of columns in a Cross-Tab.

41.6.9 GetNumRowGroups

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

GetNumRowGroups returns the total number of row groups in the Cross-Tab.

Typical uses

To format cells or modify calculation formulas based on the number of row groupings in a Cross-Tab.

41.6.10 GetNumRows

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

GetNumRows returns the total number of rows in the Cross-Tab.

Typical uses

To format cells or modify calculation formulas based on the number of rows in a Cross-Tab.

Note

GetNumRows includes the Grand Total row.

41.6.11 GetNumSummaries

Basic and Crystal syntax.

Arguments

- None

Returns

Number

Action

GetNumSummaries returns the total number of summaries in a Cross-Tab.

Typical uses

To format cells or modify calculation formulas based on the number of summaries in a Cross-Tab.

41.6.12 GetRowGroupIndexOf (rowIndex)

Basic and Crystal syntax.

Arguments

- Number

Returns

Number

Action

GetRowGroupIndexOf returns the group index of a specific row within a Cross-Tab.
The grand total returns 0.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GetRowGroupIndexOf (GetIndexOf("Customer.Contact Title","Mr."))
```

Returns 1 when Customer.Contact Title is the first grouping in a row and Customer.Contact Position is the second grouping in a row.

Returns 2 when Customer.Contact Position is the first grouping in a row and Customer.Contact Title is the second grouping in a row.

Note

If do not specify an argument for this function, the return value is the row Grand Total index.

41.6.13 GetRowPathIndexOf (rowPath)

Basic and Crystal syntax.

Argument

- The arguments are dependent on the types of the row grouping fields.

Returns

Number

Action

GetRowPathIndexOf returns the absolute index of the path or -1 if the path cannot be found.

When the path is not specified GetRowPathIndexOf returns the index of the Grand Total row.

Typical uses

To use with GridValueAt function in Calculation formulas.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GridValueAt(GetRowPathIndexOf("Owner"),currentColumnIndex,CurrentSummaryIndex)  
+ GridValueAt(GetRowPathIndexOf("Sales  
Agent"),currentColumnIndex,CurrentSummaryIndex)
```

Returns the sums of all cells in the "Owner" and "Sales Agent" rows.

41.6.14 GetTotalValueFor (groupAlias, (optional) summaryIndex)

Basic and Crystal syntax.

Arguments

- String (group alias)
- Number (optional summary index), if the summary index is not specified the current summary index is used

Returns

Number

Action

GetTotalValueFor returns the total value of the group above the group specified by the groupAlias.

Typical uses

To calculate percentages within a Cross-Tab

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GetTotalValueFor("Customer.Region")
```

If Customer.Last Year's Sales is the only summary field and Customer.Region is grouped under Customer.Country, returns the total value of sales for all regions in the country.

```
GetTotalValueFor("Customer.Region", 2)
```

If Customer.Last Year's Sales is the first summary field, and count of Customer.Region is the second summary field, returns the total count of all regions for the country.

41.6.15 GridLabelAt (groupAlias, row/columnIndex)

Basic and Crystal syntax.

Arguments

- String (groupAlias)
- Number (row/column index)

Returns

The row or column header

Action

GridLabelAt returns the row or column header for the specified group and index.

Typical uses

To format cells or modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GridLabelAt("Customer.Contact Title", 1)
```

If Customer.Contact Title is a column, returns the first header from the left.

If Customer.Contact Title is a row, returns the first header from the top.

```
GridLabelAt("Customer.Contact Title", 2)
```

If Customer.Contact Title is a column, returns the second header from the left.

If Customer.Contact Title is a row, returns the second header from the top.

Note

Totals and sub-totals do not count toward index numbers.

41.6.16 GridRowColumnValue (groupAlias, (optional) row/columnIndex)

Basic and Crystal syntax.

Arguments

- String (group alias)
- Number (optional row or column index)

Returns

String

Action

GridRowColumnValue returns the group value of the row or column specified by the alias.

Typical uses

To edit [Insertion Formulas](#) for [Calculated Members](#).

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GridRowColumnValue("Customer.Contact Position") = "Owner"
```

When used as an [Insertion Formula](#), inserts a [Calculated Member](#) after the Owner field.

41.6.17 GridValueAt (rowIndex, columnIndex, summaryIndex)

Basic and Crystal syntax.

Arguments

- Number (row index)
- Number (column index)
- Number (summary index)

Returns

Depends on the value type of the specified cell.

Action

GridValueAt returns the value of the summary at the specified location.

Typical uses

To modify calculation formulas in a Cross-Tab.

Examples

The following example is applicable to both Crystal and Basic syntax:

```
GridValueAt(currentRowIndex, GetColumnPathIndexOf("Owner"), CurrentSummaryIndex)  
+ GridValueAt(CurrentRowIndex, GetColumnPathIndexOf("Sales  
Agent"), CurrentSummaryIndex)
```

Sums the "Owner" and "Sales Agent" columns.

41.7 Strings

String functions are used for the evaluation, manipulation and conversion of text strings.

41.7.1 Len (str)

Len and [Length \(str\) \[page 1695\]](#) are equivalent functions. However, Len is preferred in Basic syntax whereas Length is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

Whole number

Action

Len returns a Number indicating the length of the given string.

ⓘ Note

Text strings must be enclosed in quotation marks. Any blank spaces are included as part of the character count.

Typical use

Use this function any time you have a manipulation, comparison, or calculation that is dependent on the length of a text string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Len("Account")
```

Returns 7.

```
Len("Text String")
```

Returns 11. The blank space between "Text" and "String" is counted as a character.

```
Len("  Center  ")
```

Returns 10. There are 2 spaces on either side of the word "Center".

```
Len(ToText({orders.ORDER AMOUNT}))
```

Returns 9, where {orders.ORDER AMOUNT} is 14233.08, and ToText ({orders.ORDER AMOUNT}) returns the String "14,233.08". Here, the comma and period have been specified in the Number format for the formula field as the thousands and decimal separators respectively.

```
Len(ToText({orders.ORDER AMOUNT}, 2, "", "."))
```

Returns 8 where {orders.ORDER AMOUNT} = 14233.08, and ToText ({orders.ORDER AMOUNT}, 2, "", ".") returns the String "14233.08". Here, the null character and period have been specified in the Number format for the formula field as the thousands and decimal separators respectively.

```
Len({customer.CUSTOMER NAME})
```

The length of the text string stored as a value in {customer.CUSTOMER NAME}.

The following examples are applicable to Basic syntax:

```
{customer.CUSTOMER NAME} (Len({customer.CUSTOMER NAME})-2)
```

Returns the third character from the right in the Customer Name. This can also be used to extract a substring from a String that always starts at a fixed position from the end of the String.

In the following example, an address line contains the province name and postal code. Full province names are of different lengths; however, a postal code has a fixed number of characters: the first character of the postal code is always the seventh from the last character in the String. The formula determines the length of the address line, subtracts 7 from it, and uses the result to identify the first character of the postal code in the address line.

```
Dim addressLine As String
Dim addressLen As Number
addressLine = "British Columbia V6X 3W2"
addressLen = Len (addressLine)
formula = addressLine (addressLen-7+1 To addressLen)
```

Returns the String "V6X 3W2".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.2 Length (str)

Length and [Len \(str\) \[page 1693\]](#) are equivalent functions. However, Length is preferred in Crystal syntax whereas Len is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

Whole Number

Action

Length returns the number of characters in a text string that you enter into the formula, or in a text string stored as a value in a data field.

Note

Text strings must be enclosed in either double or single quotation marks (" " or ` `). Any blank spaces are included as part of the character count.

Typical uses

Use this function any time you have a manipulation, comparison, or calculation that is dependent on the length of a text string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Length("Account")
```

Returns 7.

```
Length("Text_String")
```

Returns 11.

```
Length("__Center__")
```

Returns 10.

```
Length(ToText({orders.ORDER AMOUNT}))
```

Returns 9, where {orders.ORDER AMOUNT} is 14233.08, and ToText ({orders.ORDER AMOUNT}) returns the String "14,233.08". Here, the comma and period have been specified in the Number format for the formula field as the thousands and decimal separators respectively,

```
Length(ToText({orders.ORDER AMOUNT}, 2, ",", "."))
```

Returns 8 where {orders.ORDER AMOUNT} = 14233.08, and ToText ({orders.ORDER AMOUNT}, 2, "", ".") returns the String "14233.08". Here, the null character and period have been specified in the Number format for the formula field as the thousands and decimal separators respectively.

```
Length("BOB")
```

Returns 3.

```
Length("SMITH")
```

Returns 5.

```
Length("BOB SMITH")
```

Returns 9. (The blank space between BOB and SMITH is counted as a character.)

```
Length({customer.CUSTOMER NAME})
```

The length of the text string stored as a value in {customer.CUSTOMER NAME}.

The following examples are applicable to Crystal syntax.

```
{customer.CUSTOMER NAME}[Length({customer.CUSTOMER NAME})-2]
```

Returns the third character from the right in the Customer Name. This can also be used to extract a substring from a String that always starts at a fixed position from the end of the String.

In the following example, an address line contains the province name and postal code. Full province names are of different lengths, but the first character of the postal code is always the seventh from the last character. The formula determines the length of the address line, subtracts 7 from it, and uses the result to identify the first character of the postal code in the address line.

```
Local StringVar addressLine;  
Local NumberVar addrLength;  
addressLine := "British Columbia V6X 3W2";  
addrLength := Length (addressLine);  
addressLine [addrLength-7+1 to addrLength]
```

41.7.3 Trim (str)

Basic and Crystal syntax.

Arguments

- str is a text string.

Returns

String value

Action

The Trim function removes leading and trailing spaces from string arguments.

Typical uses

Use this function any time there are leading and/or trailing blanks in a text object that may interfere with an alignment of text strings, a character count, or with a calculation (if the string is eventually converted to a Number).

Examples

The following example is applicable to both Basic and Crystal syntax:

```
Trim("  abcde  ")
```

Returns "abcde".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.4 LTrim (str)

LTrim and [TrimLeft \(str\) \[page 1699\]](#) are equivalent functions. However, LTrim is preferred in Basic syntax, whereas TrimLeft is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

String value

Action

LTrim removes all spaces stored to the left of the given string and returns it.

Typical uses

- Use this function any time there are leading blanks in a text object that may interfere with an alignment of text strings, a character count, or with a calculation (if the string is eventually converted to a Number).
- Use whenever you combine (concatenate) justified text strings and want to have the proper spacing between strings.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
LTrim("  A1/4520/B12")
```

Returns "A1/4520/B12".

```
LTrim("  200")
```

Returns "200", where the text string " 200" is right-justified with leading blanks.

You have two right-justified database fields, {file.FOOD1} and {file.FOOD2}. Each field can hold up to 15 characters. {file.FOOD1} contains the word bread and {file.FOOD2} contains the word butter. If you were to print these words, they would appear like this:

```
"      bread"
```

```
"      butter"
```

For each 15-character field the database includes the field value, right-justified plus enough blank spaces to fill up the field. To use these words without the leading spaces (to create the expression bread and butter, for example) use the LTrim function in the following manner:

```
LTrim({file.FOOD1}) + " and " + LTrim({file.FOOD2})
```

Returns "bread and butter".

The spaces enclosed in the quotation marks before and after the word and assure that there is the correct spacing between the three words in the resulting sentence.

Comments

This function is designed to work like the Visual Basic function of the same name.

See Also

- [Trim \(str\) \[page 1697\]](#)
- [RTrim \(str\) \[page 1701\]](#)

41.7.5 TrimLeft (str)

TrimLeft and [LTrim \(str\) \[page 1698\]](#) are equivalent functions. However, TrimLeft is preferred in Crystal syntax, whereas LTrim is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

String value

Action

TrimLeft removes all spaces to the left of a string or data field which is stored as a right-justified string in a database.

Typical uses

- Use this function any time there are leading blanks in a text object that may interfere with an alignment of text strings, a character count, or with a calculation (if the string is eventually converted to a Number).
- Use whenever you combine (concatenate) justified text strings and want to have the proper spacing between strings.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
TrimLeft("    A1/4520/B12")
```

Returns "A1/4520/B12".

```
TrimLeft("    200")
```

Returns "200", where the text string "200" is right-justified with leading blanks.

You have two right-justified database fields, {file.FOOD1} and {file.FOOD2}. Each field can hold up to 15 characters. {file.FOOD1} contains the word bread and {file.FOOD2} contains the word butter. If you were to print these words, they would appear like this:

```
"    bread"
```

```
"    butter"
```

For each 15-character field the database includes the field value, right-justified plus enough blank spaces to fill up the field. To use these words without the leading spaces (to create the expression bread and butter, for example) use the TrimLeft function in the following manner:

```
TrimLeft({file.FOOD1}) + " and " + TrimLeft({file.FOOD2})
```

Returns "bread and butter".

The spaces enclosed in the quotation marks before and after the word and assure that there is the correct spacing between the three words in the resulting sentence.

See Also

- [Trim \(str\) \[page 1697\]](#)
- [TrimRight \(str\) \[page 1702\]](#)

41.7.6 RTrim (str)

RTrim and [TrimRight \(str\) \[page 1702\]](#) are equivalent functions. However, RTrim is preferred in Basic syntax, whereas TrimRight is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

String value

Action

RTrim removes all spaces to the right of the given string and returns it.

Typical uses

- Use this function any time there are trailing blanks in a text object that may interfere with an alignment of text strings, a character count, or with a calculation (if the string is eventually converted to a Number).
- Use whenever you combine (concatenate) justified text strings and want to have the proper spacing between strings.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
RTrim("A1/4520/B12    ")
```

Returns "A1/4520/B12".

```
RTrim({file.REFERENCE})
```

Returns "Bal Fwd." where {file.REFERENCE} contains the left-justified text string "Bal Fwd. ".

Comments

This function is designed to work like the Visual Basic function of the same name.

See Also

- [Trim \(str\) \[page 1697\]](#)
- [LTrim \(str\) \[page 1698\]](#)

41.7.7 TrimRight (str)

TrimRight and [RTrim \(str\) \[page 1701\]](#) are equivalent functions. However, TrimRight is preferred in Crystal syntax, whereas RTrim is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

String value

Action

TrimRight removes all spaces to the right of a string or data field which is stored left-justified in a database.

Typical uses

- Use this function any time there are trailing blanks in a text object that may interfere with an alignment of text strings, a character count, or with a calculation (if the string is eventually converted to a Number).
- Use whenever you combine (concatenate) justified text strings and want to have the proper spacing between strings.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
TrimRight("A1/4520/B12    ")
```

Returns "A1/4520/B12".

```
TrimRight({file.REFERENCE})
```

Returns "Bal Fwd." where the text string "Bal Fwd. " is left-justified in the field.

See Also

- [Trim \(str\) \[page 1697\]](#)
- [TrimLeft \(str\) \[page 1699\]](#)

41.7.8 UCase (str)

UCase and [UpperCase \(str\) \[page 1705\]](#) are equivalent functions. However, UCase is preferred in Basic syntax, whereas UpperCase is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

String value

Action

UCase converts the given string to all upper case and returns it.

Typical use

A good use of this function is when a field contains both uppercase and lowercase letters and you want to convert all letters to uppercase for consistency.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
UCase("Description")
```

Returns "DESCRIPTION".

```
UCase({customer.FIRST NAME})
```

Returns "RONALD", where {customer.FIRST NAME} contains "Ronald".

```
UCase("Ronald Black")
```

Returns "RONALD BLACK".

```
UCase("abc12345")
```

Returns "ABC12345".

```
UCase("BrEaD " + "AND " + "bUtTeR")
```

Returns "BREAD AND BUTTER".

Comments

This function is designed to work like the Visual Basic function of the same name.

See Also

- [LCase \(str\)](#) [page 1706]

41.7.9 UpperCase (str)

UpperCase and [UCase \(str\) \[page 1703\]](#) are equivalent functions. UpperCase is preferred in Crystal syntax, whereas UCase is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

String value

Action

UpperCase returns the text string or text value in the data field in upper case (capital letters).

Typical uses

A good use of this function is when a field contains both uppercase and lowercase letters and you want to convert all letters to uppercase for consistency.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
UpperCase("Description")
```

Returns "DESCRIPTION".

```
UpperCase({customer.FIRST NAME})
```

Returns "RONALD", where {customer.FIRST NAME} contains "Ronald".

```
UpperCase("Ronald Black")
```

Returns "RONALD BLACK".

```
UpperCase("abc12345")
```

Returns "ABC12345".

```
UpperCase("BrEaD " + "AND " + "bUtTeR" )
```

Returns "BREAD AND BUTTER".

See Also

- [LowerCase \(str\) \[page 1707\]](#)

41.7.10 LCase (str)

LCase and [LowerCase \(str\) \[page 1707\]](#) are equivalent functions. However, LCase is preferred in Basic syntax, whereas LowerCase is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

String value

Action

LCase converts the given string to all lower case and returns it.

Typical use

A good use of this function is when a field contains both uppercase and lowercase letters and you want to convert all letters to lowercase for consistency.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
LCase ("Description")
```

Returns "description".

```
LCase ({customer.FIRST NAME})
```

Returns "ronald", where {customer.FIRST NAME} is "Ronald".

```
LCase ("Ronald Black")
```

Returns "ronald black".

```
LCase ("ABC12345")
```

Returns "abc12345".

```
LCase ("BrEaD " + "AND " + "bUtTeR")
```

Returns "bread and butter".

Comments

- This function is designed to work like the Visual Basic function of the same name.
- Numbers that are part of the text are not affected by the LCase function.

See Also

- [UCase \(str\)](#) [page 1703]

41.7.11 LowerCase (str)

LowerCase and [LCase \(str\)](#) [page 1706] are equivalent functions. However, LowerCase is preferred in Crystal syntax, whereas LCase is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

String value

Action

LowerCase returns the given text value in all lower case letters.

Typical uses

A good use of this function is when a field contains both uppercase and lowercase letters and you want to convert all letters to lowercase for consistency.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
LowerCase("Description")
```

Returns "description".

```
LowerCase({customer.FIRST NAME})
```

Returns "ronald", where {customer.FIRST NAME} is "Ronald".

```
LowerCase("Ronald Black")
```

Returns "ronald black".

```
LowerCase("ABC12345")
```

Returns "abc12345".

```
LowerCase("BrEaD " + "AND " + "bUtTeR")
```

Returns "bread and butter".

Comments

Numbers that are part of the text are not affected by the LowerCase function.

See Also

- [UpperCase\(str\) \[page 1705\]](#)

41.7.12 ProperCase(str)

Basic and Crystal syntax.

Argument

- str is a text string.

Returns

String value

Action

ProperCase capitalizes the first letter of each word in the string and converts all other letters to lower case. It also capitalizes any letter that follows a non-alpha character.

→ Tip

This function is particularly useful for capitalizing people's names correctly when they are stored as all capitals in a database.

Examples

```
ProperCase ("LINDA FONG")
```

This example will return "Linda Fong".

```
ProperCase ("123Michael's")
```

This example will return "123Michael'S".

Comments

This function is similar to the "Proper" function in Excel.

41.7.13 StrReverse (str)

Basic and Crystal syntax.

Argument

str is a string whose characters are to be reversed.

Returns

String value.

Action

StrReverse returns a string in which the character order of inputString is reversed. If inputString is a zero-length string (""), a zero-length string is returned.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
StrReverse("abc")
```

Returns "cba".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.14 IsNumeric (str)

IsNumeric and [NumericText \(str\) \[page 1712\]](#) are equivalent functions. However, IsNumeric is preferred in Basic syntax, whereas NumericText is preferred in Crystal syntax.

Argument

- str is a text string.

Returns

Boolean value (True or False)

Action

IsNumeric returns True if the String argument can be converted to a Number and False otherwise.

Typical use

If you store numbers (like weight) in a text String object, use IsNumeric to check the value of each record to make sure it is OK to convert using ToNumber or [CDBl \[page 1832\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IsNumeric( {file.REFERENCE} )
```

Returns FALSE where {file.REFERENCE} = "ABCDEFGF".

```
IsNumeric( {file.IDNUM} )
```

Returns TRUE where {file.IDNUM} = "12345".

```
IsNumeric( {file.IDNUM} )
```

Returns TRUE where {file.IDNUM} = "12345443".

```
IsNumeric( {file.IDNUM} )
```

Returns FALSE where {file.IDNUM} = "12345-443".

```
IsNumeric ({file.IDNUM} (1 to 5))
```

Returns TRUE where {file.IDNUM} = "12345-443".

```
IsNumeric ({file.IDNUM} (6))
```

Returns FALSE where {file.IDNUM} = "12345-443".

```
IsNumeric ({file.IDNUM} (7 to 9))
```

Returns TRUE where {file.IDNUM} = "12345-443".

```
IsNumeric({file.IDNUM})
```

Returns FALSE where {file.IDNUM} = "12345T".

Note

You can use this function in combination with [ToNumber \[page 1714\]](#) or [CStr \[page 1834\]](#) to test for a Number in the Reference field, then return the string as a Number or return 0 if the text string is not a Number.

The following example is applicable to Basic syntax:

```
If IsNumeric({file.IDNUM}) Then  
    formula = ToNumber({file.IDNUM})  
Else  
    formula = 0  
End If
```

41.7.15 NumericText (str)

NumericText and [IsNumeric \(str\) \[page 1711\]](#) are equivalent functions. However, NumericText is preferred in Crystal syntax, whereas IsNumeric is preferred in Basic syntax.

Arguments

- str is a text string.

Returns

Boolean Value

Action

NumericText tests to see if the content of a text object is a Number.

- If the entire content of the field is a Number - or if the characters extracted via the [Subscript for arrays and strings \[page 1481\]](#) are entirely a Number - the function returns a TRUE value.
- If any part of the content of the field - or of the characters extracted - are not a Number, the function returns the value FALSE.

Typical uses

If you store numbers (like weight) in a text object, use NumericText to check the value of each record to make sure it is OK to convert using [ToNumber \[page 1714\]](#).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
NumericText({file.REFERENCE})
```

Returns FALSE where {file.REFERENCE} = "ABCDEFGF".

```
NumericText({file.IDNUM})
```

Returns TRUE where {file.IDNUM} = "12345".

```
NumericText({file.IDNUM})
```

Returns TRUE where {file.IDNUM} = "12345443".

```
NumericText({file.IDNUM})
```

Returns FALSE where {file.IDNUM} = "12345-443".

```
NumericText ({file.IDNUM} [1 to 5])
```

Returns TRUE where {file.IDNUM} = "12345-443".

```
NumericText ({file.IDNUM} [6])
```

Returns FALSE where {file.IDNUM} = "12345-443".

```
NumericText ({file.IDNUM} [7 to 9])
```

Returns TRUE where {file.IDNUM} = "12345-443".

```
NumericText({file.IDNUM})
```

Returns FALSE where {file.IDNUM} = "12345T".

Note

You can use this function in combination with [ToNumber \[page 1714\]](#) to test for a Number in the Reference field, then print the string as a Number or print 0 if the text string is not a Number.

The following example is applicable to Crystal syntax:

```
If NumericText({file.REFERENCE}) Then
    ToNumber({file.REFERENCE})
Else
    0;
```

41.7.16 ToNumber

Basic and Crystal syntax.

ToNumber and [CDBl \[page 1832\]](#) are equivalent functions.

Overloads

- ToNumber (numeric)
- ToNumber (string)
- ToNumber (Boolean)

Arguments

- numeric is a Number or a Currency type value.
- string is a text string that holds numeric text.
- Boolean is a Boolean value that you want to treat as a number.

Returns

Fractional Number

Action

ToNumber converts a Number, Currency, text string, or Boolean value to a Number.

In a database, some numbers are stored in numeric fields, as Numbers, and some are stored in character fields, as text. You make the determination which fields are to be numeric and which are to be text when you set up the database in the first place. Numbers on which you might wish to perform arithmetic (item cost, quantity ordered, and so on) are typically stored in numeric fields; numbers on which you do not expect to perform arithmetic (customer number, telephone number, and so on) are typically stored in text objects.

ToNumber allows you to convert a Number stored as text to a Number on which you can perform arithmetic.

The Boolean argument enables you to treat SQL_BIT data values as if they were numbers.

Typical uses

You might use this function, for example, if your item numbers contain coded product information and you want to use that information in calculations. Typically, you would be using this function to convert a Currency or String value that contains all numeric characters to a Number type value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
ToNumber ( {orders.ORDER AMOUNT} )
```

Returns a Number type value that the Currency field {orders.ORDER AMOUNT} contains.

```
ToNumber ( "123.45" )
```

Returns 123.45.

```
ToNumber ( {file.REFERENCE} )
```

Returns 200.00 where "200" is the text string in the {file.REFERENCE} field.

```
ToNumber ( {file.ACCTNO}/2 )
```

Returns 22144 where {file.ACCTNO} = 44288.

```
ToNumber ( {customer.POSTAL CODE} ) < 33333
```

Returns TRUE where {customer.POSTAL CODE} is "21385".

Comments

When using the ToNumber function, you should first test the value with the NumericText function. NumericText returns a value of TRUE only if the value in the string can be correctly converted to a Number. Otherwise, if you try to convert a value to a Number that is not a Number, the formula will produce an error in your report.

The following example is a common use of the NumericText and ToNumber functions together, in Crystal syntax:

```
If NumericText ({file.FIELD}) Then
    ToNumber ({file.FIELD})
Else
    0
```

41.7.17 ToText

Basic and Crystal syntax.

ToText and [CStr \[page 1834\]](#) are equivalent functions.

Overloads

- ToText (x)
- ToText (x, y)
- ToText (x, y, z)
- ToText (x, y, z, w)
- ToText (x, y, z, w, q)

Arguments

Converting . . .	Description
Boolean values	• x is a Boolean value that is converted to a String, either "True" or "False".

Converting . . .	Description
Number and Currency values	- x is a Number or Currency value to be converted into a text string; it can be a whole or fractional value. - y is a whole number indicating the number of decimal places to carry the value in x to (This argument is optional.). - z is a single character text string indicating the character to be used to separate thousands in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.) - w is a single character text string indicating the character to be used as a decimal separator in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.)
Number and Currency values (formatting)	- x is a Number or Currency value to be converted into a text string; it can be a whole or fractional value. - y is a text string used to indicate the format for displaying the value in x. See Format Strings [page 1720] for information on creating a format string. - z is a whole number indicating the number of decimal places to carry the value in x to. (This argument is optional.) - w is a single character text string indicating the character to be used to separate thousands in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.) - q is a single character text string indicating the character to be used as a decimal separator in x. The default is the character specified in your International or Regional settings control panel. (This argument is optional.)
Date values	- x is a Date value to be converted into a text string. - y is a text string that defines how the value in x is to be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating this format string. (This argument is optional.)
Time values	- x is a Time value to be converted into a text string. - y is a text string that defines how the value in x is to be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating this format string. (This argument is optional.) - z is a text string to be used as a label for A.M. (morning) hours. (This argument is optional.) - w is a text string to be used as a label for P.M. (evening) hours. (This argument is optional.)

Converting . . .	Description
DateTime values	- x is a DateTime value to be converted into a text string. - y is a text string of characters that indicate how the resulting text string will be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating a format string. (This argument is optional.) - z is a text string to be used as a label for A.M. (morning) hours. (This argument is optional.) - w is a text string to be used as a label for P.M. (evening) hours. (This argument is optional.)

Returns

Text String

Action

The ToText function converts Boolean, Numbers, Currency, Date, Time, and DateTime values to text strings.

Typical uses

Use this function to convert a Boolean, Number, Currency, Date, Time, or DateTime value to a text string to appear as text in your report (form letters, comments, and so on).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
ToText ( {orders.SHIPPED} )
```

Returns True where the value in ({orders.SHIPPED}) is True.

```
ToText ( 123.45 )
```

Returns "123.45".

```
ToText ( 12345.6749, 2 )
```

Returns "12345.67".

```
ToText(12345.6750, 2)
```

Returns "12345.68".

```
ToText(12345.4999, 0)
```

Returns "12345".

```
ToText(12345.5000, 0)
```

Returns "12346".

```
ToText({file.AMT} * {file.QUANTITY})
```

Returns 44,890.20 where Amt = 24.45 and Quantity = 1836.

ToText is useful when you want to build a sentence by combining (concatenating) a converted Number or other value with other text strings:

```
"The base price of item # " + {file.ITEM NUMBER} + " is $" + ToText({file.BASE PRICE}) + "."
```

Prints the sentence "The base price of item A1/4520/B12 is \$50.00." where the Item number is A1/4520/B12 and the Base Price is 50.00, converted to text and formatted with two decimal places.

The following examples are applicable to Crystal syntax:

```
ToText(CDate(1996, 11, 1), "yy MMM dd, dddd")
```

Returns 96 Nov 01, Friday.

```
ToText(DateTime(1995, 10, 12, 3, 30, 11), "HH:mm, yy MMMM ddd")
```

Returns 03:30, 95 October Thu.

```
ToText(Time(12, 10, 10), "HH*mm*ss tt", "amStr", "pmStr")
```

Returns 12*10*10 pmStr.

Comments

The overloads of ToText that take only one argument work like the Visual Basic function of the same name.

Converting Boolean values:

- The ToText function, when used with Boolean values, is most useful for combining (concatenating) a Boolean value with other text. Otherwise, a Boolean field can be formatted to appear as True or False in your report simply by changing the format on the Boolean Tab of the Format Editor.

Converting numbers and currency values:

- If the number of decimal places is specified, this function does not truncate the number when converted to text, but rounds it to the number of decimal places specified. See [Round \[page 1528\]](#), for more information on the rounding procedure.

Converting Date, Time, and DateTime values:

- Any character, with the exception of the Date or Time format characters, can be used within the format string. For example, you may wish to use a slash to separate the different elements (month, day, year) of the date, as in "12/30/95", or you may wish to use a colon to separate the different elements (hours, minutes, seconds) of the time, as in "12:30:10".
- If you wish to use any of the above characters in the format string, they must appear in quotes. For example: `ToText(CDateTime(1995,10,12,13,20,22), "MM/dd/yy hh 'h' mm 'min' ss 'sec' tt", 'am', 'pm') = "10/12/95 01 h 20 min 22 sec pm"`

Passing optional arguments:

- Many arguments for the ToText function have been specified as optional. However, you can only leave an argument blank if all the arguments that follow it are left blank as well. In other words, you cannot leave the y and z arguments blank and provide an argument for w. It is possible, however, to leave one, two, or all of the optional arguments blank, as long as no arguments are supplied after the blank arguments. The following combinations are possible when supplying arguments to the ToText function:
 - ToText (x)
 - ToText (x, y)
 - ToText (x, y, z)
 - ToText (x, y, z, w)
 - ToText (x, y, z, w, q)
- Using the "t" or "tt" format characters in a time format string provides default strings for indicating a.m. (morning) and p.m. (evening) hours. "t" produces just a single character, "a" or "p", while "tt" produces the entire string, "am" or "pm". You can pass your own custom strings for indicating am/pm strings. (See the arguments for converting Time and DateTime values with ToText in the Arguments section above.) If you do pass your own am/pm strings, the "t" and "tt" format characters will have the same effect on them (producing single character vs. multiple character strings).

41.7.17.1 Format Strings

The ToText and CStr [\[page 1834\]](#) functions can use format strings to control how the Number, Currency, Date, Time, or DateTime value passed to the x argument will appear when converted to a String. This section describes how to create format strings for the various data types.

Format strings for fractional Numbers and Currency values

The following characters are used to create format strings for fractional Numbers and Currency values:

Character	Description	Comments/Examples
#	Represents numerical digits or spaces. When the original number is converted, # characters are ignored unless the number of # characters in the format string extends beyond the number of significant digits in the original value. Then, for every additional # that appears, a space is added to the string.	If your original value is 125.34: - The format string "#" produces the string "125" (number of significant digits exceeds the number of # characters). - The format string "###.##" produces the string "125.34" (number of significant digits equal to number of # characters). - The format string "####.###" produces the string "125.34" (number of significant digits is less than the number of # characters).
0	Represents numerical digits or zeros (0). When the original number is converted, 0 characters are ignored unless the number of 0 characters in the format string extends beyond the number of significant digits in the original value. Then, for every additional 0 that appears, a zero is added to the string.	If your original value is 125.34: - The format string "0" produces the string "125" (number of significant digits exceeds the number of 0 characters). - The format string "000.00" produces the string "125.34" (number of significant digits equal to number of 0 characters). - The format string "0000.000" produces the string "0125.340" (number of significant digits is less than the number of 0 characters). # and 0 characters can be combined in format strings. If your original value is 125.34: - The format string "####.0000" produces the string "125.3400". - A comma (,) - A decimal point (.) indicates where a decimal separator should appear. The actual character that appears as a decimal separator in the resulting string can be changed.
comma (,)	Indicates where a thousands separator should appear. The actual character that appears as a thousands separator in the resulting string can be changed.	

Character	Description	Comments/Examples
decimal point (.)	Indicates where a decimal separator should appear. The actual character that appears as a decimal separator in the resulting string can be changed.	

How format string can affect values

The following table illustrates how format strings can affect different fractional Numbers and Currency values being converted. In this table, the underscore character (_) represents a space in the resulting string:

Original Value	Format string	Resulting String
16.13	#	16
16.13	0	16
16.13	##	16
16.13	00	16
16.13	###	_16
16.13	000	016
16.13	#.####	16.13_
16.13	0.0000	16.1300
16125.00	###,###.#	_16,125.0
16125.00	000,000.0	016,125.0
16125.00	000000.0	016125.0

41.7.17.2 Format strings for Date, Time, and DateTime values

The following characters are used to create format strings for Date, Time, and DateTime values:

Character	Comments
d	day of month without leading zero for single digits
dd	day of month with leading zero for single digits.
ddd	day of week as a three letter abbreviation
dddd	full name of day of week
M	month without leading zero for single digit
MM	month with leading zero for single digit
MMM	month as three letter abbreviation.
MMMM	full name of month
yy	last two digits of year
yyyy	full four digits of year
h	hours without leading zeros for single digits (12 hour)
hh	hours without leading zeros for single digits (12 hour).
H	hours without leading zeros for single digits (24 hour)
HH	hours with leading zeros for single digits (24 hour)
m	minutes without leading zeros for single digits
mm	minutes with leading zeros for single digits
s	seconds without leading zeros for single digits
ss	seconds with leading zeros for single digits
t, tt	single character or multiple character a.m./p.m. string

The following table illustrates how format strings can affect different Date, Time, and DateTime values being converted by the ToText or [CStr \[page 1834\]](#) function:

Date, Time, or DateTime value	Format String	Resulting String
CDate(1996, 11, 1)	yy MMM dd, dddd	96 Nov 01, Friday
CTime(12, 10, 10)	HH*mm*ss tt	12*10*10
CDateTime(1995, 10, 12, 3, 30, 11)	HH:mm, yy MMMM ddd	03:30, 96 October Mon
CTime(13, 20, 22)	hh 'h' mm 'min' ss 'sec' tt	01 h 20 min 22 sec

Date, Time, or DateTime value	Format String	Resulting String
CDate(1998, 3, 17)	M/dd/yy	3/17/98

41.7.17.3 Converting Boolean values

The ToText function, when used with Boolean values, is most useful for combining (concatenating) a Boolean value with other text. Otherwise, a Boolean field can be formatted to appear as True or False in your report simply by changing the format on the Boolean tab (Format Editor dialog box) of the Format Editor dialog box.

41.7.17.4 Converting fractional Numbers and Currency values

If the number of decimal places is specified, this function does not truncate the number when converted to text, but rounds it to the number of decimal places specified. See [Round \[page 1528\]](#), for more information on the rounding procedure.

41.7.17.5 Converting Date, Time, and DateTime values

Any character, with the exception of the date or time format characters, can be used within the format string. For example, you may wish to use a slash to separate the different elements (month, day, year) of the date, as in "12/30/95", or you may wish to use a colon to separate the different elements (hours, minutes, seconds) of the time, as in "12:30:10".

If you wish to use any of the above characters in the format string, they must appear in quotes. For example:

```
ToText(DateTime(1995,10,12,13,20,22), "MM/dd/yy hh 'h' mm 'min' ss 'sec' ",
'am', 'pm') = "10/12/95 1 h 20 min 22 sec pm"
```

41.7.17.6 Passing optional arguments

- Many arguments for the ToText function have been specified as optional. However, you can only leave an argument blank if all the arguments that follow it are left blank as well. In other words, you cannot leave the y and z arguments blank and provide an argument for w. It is possible, however, to leave one, two, or all of the optional arguments blank, as long as no arguments are supplied after the blank arguments.

The following combinations are possible when supplying arguments to the ToText function:

- ToText(x)
- ToText (x, y)
- ToText (x, y, z)
- ToText (x, y, z, w)

- ToText (x, y, z, w, q)
- Using the "t" or "tt" format characters in a time format string provides default strings for indicating a.m. (morning) and p.m. (evening) hours. "t" produces just a single character, "a" or "p", while "tt" produces the entire string, "am" or "pm". You can pass your own custom strings for indicating am/pm strings. See the arguments for converting time and DateTime values with ToText in the Arguments section of [ToText \[page 1716\]](#).
If you do pass your own am/pm strings, the "t" and "tt" format characters will have the same effect on them (producing single character vs. multiple character strings). However, the "t" and "tt" format characters are optional and only necessary when default am/pm strings are needed.

See Also

- CStr [\[page 1834\]](#)

41.7.18 ToWords

Basic and Crystal syntax.

Overloads

- ToWords (x)
- ToWords (x, #places)
- ToWords (x, #places, formtype)

Arguments

- x is a Number or Currency value to be converted into words.
- #places is a whole number indicating the number of decimal places to be converted. (This argument is optional.)
- formtype specifies what type of form x is. This argument is used only for Asian languages—specifically, Japanese, Korean, Simplified Chinese, and Traditional Chinese. It is ignored for English or any other non-Asian language.

Form Type 0	Classic form, also called the check form
Form Type 1	Daily form

Note

The daily form has the same structure as the classic form, except it uses a different set of characters for the numbers. In Korean, there is only one set of characters for numbers, so the daily form and the classic form are the same for this language. The daily form is also similar to the style used in articles.

Note

The casual form is used for quoting phone numbers, addresses, and so on.

Returns

Text String

Action

You can use this function to convert a Number or Currency field value or the result of a numeric calculation to words so it can be used as text. The ability to adjust the number of decimal places can be useful when the number is the result of a calculation that may produce more decimal places than you want.

Typical uses

You can use this function to spell out the dollar amount for each check if you are using computer checks.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
ToWords(12345)
```

Returns twelve thousand three hundred forty-five and xx/100.

```
ToWords(12345.6749, 2)
```

Returns twelve thousand three hundred forty-five and 67/100.

```
ToWords(12345.4999, 0)
```

Returns twelve thousand three hundred forty-five.

```
ToWords(12345.5000,0)
```

Returns twelve thousand three hundred forty six.

```
ToWords(-12345)
```

Returns negative twelve thousand three hundred forty-five and xx/100.

```
ToWords(12.3499)
```

Returns twelve and 35/100.

```
ToWords({orders.ORDER AMOUNT})
```

Returns two thousand sixteen and 84/100 where Amount = 2016.84.

```
ToWords(((file.QTY1 + file.QTY2 + file.QTY3) * file.PRICE) * 1.075)
```

Returns one hundred two and 13/100.

```
ToWords(((file.QTY1 + file.QTY2 + file.QTY3) * file.PRICE) * 1.075,0)
```

Returns one hundred two where QTY1 = 1, QTY2 = 82, QTY3 = 12, and Price = 1.00 (sums three quantities, multiplies them times the price and adds 7.5% sales tax). In this case the numeric answer is 102.125 which is then rounded to 102.13 (the standard two decimal places) before putting into words.

Comments

- The function treats the number as a whole, rather than as a series of individual numbers. That is, 123 is treated as the number one hundred twenty-three rather than the individual digits one, two, and three.
- Negative numbers begin with the word "negative."
- Currency field values and Number field values are treated in the same way and produce identical results.
- Since the spelled out value will be much longer than the Number value, you will need to increase the size of the field box to accommodate the new field length.

41.7.19 ReplicateString (str, #copies)

Basic and Crystal syntax.

Arguments

- str is the text string to be replicated.

- #copies is a whole number indicating the number of times str is to be replicated.

Returns

Text String

Action

Replicates the string in str the number of times specified by #copies.

Typical uses

You can use this function to insert a line of characters any time they are needed. Some typical uses are:

- To flag critical data on your report.
- To build simple bar graphs.
- To split your report into visible sections.
- To highlight totals, subtotals, and other summary data.

Examples

The following examples are applicable to Crystal syntax:

```
If {file.SALES} < {file.QUOTA} Then
    ReplicateString(" ",10)
Else
    ""
```

Prints the character * ten times as a flag.

```
{file.NAME}+ "  " + ReplicateString(" ", {file.SCORE})
```

Prints a simple bar graph showing test results. The formula prints an asterisk for each point in a test score (the value in the file.SCORE field).

For example, given the following data:

Name	Score
Smith	23

Name	Score
Jones	17
Brown	21
Johnson	13
Reynolds	25

The formula will produce the following results:

Name	@Score
Smith	*****
Jones	*****
Brown	*****
Johnson	*****
Reynolds	*****

41.7.20 Space (integer)

Basic and Crystal syntax.

Arguments

- integer is a whole number indicating the number of spaces.

Returns

Text String (one or more spaces)

Action

The Space function returns a specified number of spaces.

Typical uses

Use the Space function to easily add a specific number of spaces to a text string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Space(2)
```

Returns " ".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.21 InStr

Basic and Crystal syntax.

Overloads

- InStr (str1, str2)
- InStr (start, str1, str2)
- InStr (str1, str2, compare)
- InStr (start, str1, str2, compare)

Arguments

- start is the character in str1 where the search is to begin. This is a 1 based index. (This argument is optional.)
- str1 is the text string to be searched.
- str2 is the text string being sought within str1.
- compare is an optional number value indicating which type of string comparison should be used. 0 indicates case-sensitive comparison, and 1 indicates case-insensitive comparison. If this argument is not used, a case-sensitive comparison is performed.

Returns

Whole Number

Action

The InStr function returns the position of the first occurrence of one string within another. This position is a 1 based index of the characters in str1. If str2 is not found in str1, the InStr function returns 0. The start argument sets the starting position for the search. If the compare argument is not used, the string comparison will be case-sensitive.

Typical uses

Use this function to determine if one string contains another.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
InStr("abcdefg", "bcd")
```

Returns 2.

```
InStr(3, "abcdefg", "cde")
```

Returns 3.

```
InStr("KarenJudith", "karen", 1)
```

Returns 1. Note that because the compare argument equals 1, the comparison was case-insensitive.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- There are two versions of this function, with regards to the start argument. The first does not use the start argument, the second does. If the start argument is not used, InStr searches all of str1 to try to find str2. If the start argument is used, InStr begins searching at the character in str2 indicated by the start argument.

41.7.22 InStrRev

Basic and Crystal syntax.

Overloads

- InStrRev (inputString, findString)
- InStrRev (inputString, findString, startPosition)
- InStrRev (inputString, findString, startPosition, compare)

Arguments

- inputString is the text string to be searched.
- findString is the text string being sought within str1.
- startPosition is an optional numeric expression that sets the starting position for each search. If omitted, -1 is used, which means that the search begins at the last character position.
- compare is an optional numeric value indicating the kind of comparison to use when evaluating substrings:
 - 0 performs a comparison that is case-sensitive
 - 1 performs a comparison that is case-insensitiveIf omitted, a case-sensitive comparison is performed.

Returns

Number value that indicates the position of the matching string in the string to be matched. The first character of inputString (from the beginning) is 1.

Action

InStrRev returns the position of the first occurrence of one string within another. The matching proceeds in the reverse direction from the specified start position.

Examples

The following examples are applicable to Basic and Crystal syntax:

```
InStrRev ("abcdefgbchijk", "bc")
```

Returns 8, where matching starts from the very end of the string in the reverse direction.

```
InStrRev ("abcdefgbchijk", "bc", -1)
```

Returns 8, where matching starts from the very end of the string in the reverse direction.

```
InStrRev ("abcdefgbchijk", "bc", 2)
```

Returns 0, i.e., no match is found. Matching starts from the "b" in "ab" in the reverse direction.

```
InStrRev ("abcdefgbchijk", "bc", 3)
```

Returns 2. Matching starts from the "c" in "abc" in the reverse direction.

```
InStrRev ("abcdefgbchijk", "bc", 10)
```

Returns 8. Matching starts from the 'h' in "abcdefgbch" in the reverse direction, and the first occurrence of "bc" is found at position 8 from the beginning of the input string.

```
InStrRev ("abcdefgbchijk", "BC", -1, 1)
```

Returns 8. Matching starts from the very end of the string in the reverse direction, and the first occurrence of "BC", independent of case since case-insensitive comparison is specified, is found at position 8 from the beginning of the input string.

```
InStrRev ("aBCdefgbchijk", "BC", -1, 0)
```

Returns 2. Matching starts from the very end of the string in the reverse direction, and the first occurrence of "BC" is found at position 2 from the beginning of the input string.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.23 StrCmp

Basic and Crystal syntax.

Overloads

- StrCmp (str1, str2)
- StrCmp (str1, str2, compare)

Arguments

- str1 is the first text string to be compared.
- str2 is the second text string to be compared.
- compare is an optional Number value indicating the kind of string comparison to use:
 - 0 performs a comparison that is case-sensitive
 - 1 performs a comparison that is case-insensitiveIf omitted, a case-sensitive comparison is performed.

Returns

Whole Number where

- -1 = Less Than
- 0 = Equal To
- 1 = Greater Than

Action

The StrCmp function compares two strings.

Typical Uses

Use this function to determine if two string values are identical.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
StrCmp("abcd", "aa")
```

Returns = 1.

```
StrCmp("abcd", "aa", 1)
```

Returns = 1.

```
StrCmp ("abcd", "ac")
```

Returns = -1.

```
StrCmp ("aa", "aa")
```

Returns = 0.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.24 Mid

Basic and Crystal syntax.

Overloads

- Mid (str, start)
- Mid (str, start, length)

Arguments

- str is the text string from which one or more characters is being extracted.
- start is a whole number indicating the position of the first character to extract.
- length is a whole number indicating the number of characters to be extracted from the text string. This argument is optional. If a value is not specified, the rest of the string from the starting position is extracted.

Returns

Text String

Action

The Mid function returns a specified number of characters from a string. The second argument is the character position where the part to be taken begins. The third argument is the length of the string you want to be

taken out. If the third argument is not specified, everything from the start position to the end of the string is extracted.

Typical uses

Use this function anytime you need to extract a set of characters from somewhere in the middle of a text string.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
Mid("abcdef", 3, 2)
```

Returns "cd".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.25 Left (str, length)

Basic and Crystal syntax.

Arguments

- str is a text string.
- length is a whole number indicating the number of characters to be extracted from str.

Returns

String value

Action

Extracts the specified number of characters from the left side of the string.

Typical uses

Use this function to obtain a certain number of characters from the left end of a text string. For instance, you could use the Left function to obtain just the area code from the values in a field containing phone numbers.

Examples

The following examples are applicable to Crystal syntax:

```
Left("abcdefg", 4)
```

Returns "abcd".

```
Left({customer.FAX}, 3)
```

Retrieves the area code of each fax number when the first three characters of the Fax field contains the area code.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.26 Right (str, length)

Basic and Crystal syntax.

Arguments

- str is a text string.
- length is a whole number indicating the number of characters to be extracted from str.

Returns

String value

Action

Extracts the given number of text characters from the right side of the specified string.

Typical uses

Use this function to obtain just the right part of a string. For instance, use it to obtain the last four digits of social security numbers stored in a field.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Right("abcdefg", 3)
```

Returns "efg".

```
Right({table.SSNUM}, 4)
```

Returns the last 4 digits of the social security number as a String.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.27 Val (str)

Basic and Crystal syntax.

Arguments

- str is a text string.

Returns

Fractional Number

Action

The Val (str) function reads a string containing Numbers (example: an address, phone number, or social security number) and converts them to a decimal value. Val stops reading the string when it finds the first character in the string that it finds that it cannot recognize as a number or as a space.

Typical uses

Use this function to extract only the numeric portion of a string that contains both numbers and text. For instance, you can extract only the street number from an address, or only the numeric portion of an ID that contains letters and numbers.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
Val("2234 100th Street")
```

Returns 2234100.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.28 ChrW (x)

Basic and Crystal syntax.

Arguments

- x is a whole number, specifically, it is any Unicode value. x must be between 0 and 65535, or you will get a numeric overflow error.

Returns

Text String (one character long)

Action

The ChrW (x) function returns the single character text string associated with the Unicode value passed in as x.

Typical uses

Use this function any time you need to obtain the character associated with a particular Unicode value.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ChrW(65)
```

Returns "A".

```
ChrW(332)
```

Returns "Ö"

Comments

This function is designed to work like the Visual Basic function of the same name.

See also

- [AscW \(str\) \[page 1741\]](#)

41.7.29 AscW (str)

Basic and Crystal syntax.

Arguments

- str is a text string.

Returns

Number

Action

The AscW (str) function returns the Unicode value of the first character of the string.

Typical uses

Use this function to obtain the Unicode value of a character. This function is most useful for application developers using the Report Engine.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
AscW("Karen")
```

Returns 75, which is the Unicode value of the letter "K".

```
AscW("Ö")
```

Returns 213.

Comments

This function is designed to work like the Visual Basic function of the same name.

See also

- [ChrW \(x\) \[page 1739\]](#)

41.7.30 Filter

Basic and Crystal syntax.

Overloads

- Filter (inputStrings, searchString)
- Filter (inputStrings, searchString, include)
- Filter (inputStrings, searchString, include, compare)

Arguments

- inputStrings is an array of strings to be searched.
- searchString is a string to search for.
- include is an optional Boolean value indicating whether to return substrings that include or exclude searchString. If include is True, Filter returns the subset of the array that contains searchString as a substring. If include is False, Filter returns the subset of the array that does not contain searchString as a substring. If omitted, the value True is assumed.
- compare is an optional Number value indicating the kind of string comparison to use:
 - 0 performs a comparison that is case-sensitive
 - 1 performs a comparison that is case-insensitiveIf omitted, a case-sensitive comparison is performed.

Returns

Array of String values.

Action

Filter searches an array of strings for a specified string, and returns the strings in an array.

Examples

The following examples are applicable to Basic and Crystal syntax.

The examples assume that inputStrings is a String array consisting of the 5 elements: "pineapple", "Apple", "APPLE", "grape", and "orange".

```
Filter (inputStrings, "ap")
```

Returns an array containing the 2 strings "pineapple" and "grape".

```
Filter (inputStrings, "Ap", True)
```

Returns an array containing the 1 string "Apple".

```
Filter (inputStrings, "ap", True, 1)
```

Returns an array containing the 4 strings "pineapple", "Apple", "APPLE" and "grape". Matching to the search string uses case-insensitive comparison.

```
Filter (inputStrings, "ap", False)
```

Returns an array containing the 3 strings "Apple", "APPLE" and "orange".

```
Filter (inputStrings, "cd")
```

Returns an array containing 1 string, the empty string "". The returned array does not contain any matching strings but contains 1 element since the formula language does not support empty arrays.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.7.31 Replace

Basic and Crystal syntax.

Overloads

- Replace (inputString, findString, replaceString)
- Replace (inputString, findString, replaceString, startPosition)
- Replace (inputString, findString, replaceString, startPosition, count)
- Replace (inputString, findString, replaceString, startPosition, count, compare)

Arguments

- inputString is a String containing substring to replace.
- findString is a substring being searched for.
- replaceString is the replacement substring.
- startPosition is an optional Number indicating the position within inputString where substring search is to begin. If omitted, 1 is assumed.
- count is an optional Number of substring substitutions to perform. If omitted, the default value is -1, which means make all possible substitutions.
- compare is an optional Number indicating the kind of comparison to use when evaluating substrings:
 - 0 performs a comparison that is case-sensitive
 - 1 performs a comparison that is case-insensitiveIf omitted, a case-sensitive comparison is performed.

Returns

String value.

Action

Replace returns a String in which a specified substring has been replaced with another substring a specified number of times. As an option, you can also specify where in the String to begin replacing, and return a String starting from that position.

Typical use

To systematically replace in a String one substring with another.

Examples

The following examples are applicable to both Basic and Crystal syntax. They assume that `inputString` is the text String "Monday is my day off. Can we meet next Monday?".

```
Replace (inputString, "Monday", "Wednesday")
```

Returns "Wednesday is my day off. Can we meet next Wednesday?".

```
Replace (inputString, "Monday", "Wednesday", 23)
```

Returns "Can we meet next Wednesday?" The replacement begins at the 23rd character from the beginning of the original `inputString`. The returned string is this portion of the string with the substitutions made.

```
Replace (inputString, "Monday", "Wednesday", 1, 1)
```

Returns "Wednesday is my day off. Can we meet next Monday?" There is only 1 replacement beginning at the 1st character from the beginning of the original `inputString`. The returned string is thus the original string with the single substitution made.

```
Replace (inputString, "monday", "Wednesday", 1, -1, 0)
```

Returns the same `inputString` with no replacements since no match was found for "monday" with a case-sensitive comparison.

```
Replace (inputString, "monday", "Wednesday", 1, -1, 1)
```

Returns the `inputString` with both occurrences of "Monday" replaced by "Wednesday". That is, "Wednesday is my day off. Can we meet next Wednesday?".

Comments

- This function is designed to work like the Visual Basic function of the same name.
- The returned value of the `Replace` function is a String, with the specified substring replacements made, that begins at the position specified by `startPosition` and ends at the end of the `inputString` string. It is not a copy of the original string from start to finish.

41.7.32 Join

Basic and Crystal syntax.

Overloads

- Join (list)
- Join (list, delimiter)

Arguments

- list is a String array containing substrings to be joined.
- delimiter is an optional String used to separate the substrings in the returned string. If omitted, the space character (" ") is used. If delimiter is a zero-length string (""), all items in the list are concatenated with no delimiters.

Returns

String value.

Action

Join returns a String created by joining a number of substrings contained in an array.

Typical use

To convert elements stored in a String array to a single String.

Examples

The following examples are applicable to Basic and Crystal syntax.

The examples assume that list is a String array consisting of the 3 elements: "Chocolate", "Vanilla" and "Strawberry".

```
Join (list)
```

Returns the String "Chocolate Vanilla Strawberry".

```
Join (list, "****")
```

Returns the String "Chocolate****Vanilla****Strawberry".

Comments

This function is designed to work like the Visual Basic function of the same name.

See Also

- [Split \[page 1747\]](#)

41.7.33 Split

Basic and Crystal syntax.

Overloads

- Split (inputString)
- Split (inputString, delimiter)
- Split (inputString, delimiter, count)
- Split (inputString, delimiter, count, compare)

Arguments

- inputString is a String expression containing substrings and delimiters.
- delimiter is an optional String character used to identify substring limits. If omitted, the space character (" ") is assumed to be the delimiter. If delimiter is a zero-length string, a single-element array containing the entire inputString string is returned.
- count is an optional number value of substrings to be returned. The value -1 indicates that all substrings are returned. If omitted, -1 is assumed.
- compare is an optional Number indicating the kind of comparison to use when evaluating the delimiter string:
 - 0 performs a comparison that is case-sensitive
 - 1 performs a comparison that is case-insensitiveIf omitted, a case-sensitive comparison is performed.

Note

Unlike in Visual Basic, in Crystal Reports if you omit an optional argument, you must omit all the following arguments. For example, if you do not specify a delimiter, you cannot specify count nor compare.

Returns

Array of String values.

Action

Split takes a String that contains a number of substrings, breaks it up into a specified number of substrings and returns an array containing the substrings.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Split ("Chocolate Strawberry Pineapple")
```

Returns an array that contains 3 elements, "Chocolate", "Strawberry" and "Pineapple".

```
Split ("Chocolate//Strawberry//Pineapple", "//")
```

Returns an array that contains 3 elements, "Chocolate", "Strawberry" and "Pineapple".

```
Split ("Chocolate//Strawberry//Pineapple", "//", 2)
```

Returns an array that contains 2 elements, "Chocolate" and "Strawberry//Pineapple". The last element in the array is a concatenation of the 2nd substring and the remaining substring.

```
Split ("Chocolate and Strawberry and Pineapple", " And ", -1, 0)
```

Returns an array that contains 1 element, "Chocolate and Strawberry and Pineapple". The delimiter " And " cannot be matched.

```
Split ("Chocolate and Strawberry and Pineapple", " And ", -1, 1)
```

Returns an array that contains 3 elements, "Chocolate", "Strawberry" and "Pineapple". The delimiter " And " is matched with " and " regardless of the case.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- If count, c, is less than the total number of substrings in inputString, then at most c substrings will be returned as elements in the resultant array. The last element in the array being a concatenation of the c-th substring and all the remaining substrings.

See Also

- [Join \[page 1745\]](#)

41.7.34 Roman

Basic and Crystal syntax.

Overloads

- Roman (arabicVal)
- Roman (arabicVal, form)

Arguments

- arabicVal is a number representing the Arabic number you want converted. Valid values are 0 through 3999.
- form is a number specifying the type of Roman numeral you want. The Roman numeral style ranges from Classic to Simplified, becoming more concise as the value of form increases. See the examples below. Since form is an optional argument, if it is omitted then Classic is assumed for the form value.

Form	Type
0 or omitted	Classic
1	More concise. See example.
2	More concise. See example.
3	More concise. See example.
4	Simplified

Returns

A string which is the roman numeral representation of arabicVal.

Action

Converts an Arabic numeral to a Roman numeral, as text.

Examples

You could use this function to print lower case page numbers in Roman numerals. For example:

```
LCase (Roman (PageNumber))
```

Other examples:

```
Roman (499,0)
```

Returns CDXCIX

```
Roman (499,1)
```

Returns LDVLIV.

```
Roman (499,2)
```

Returns XDIX.

```
Roman (499,3)
```

Returns VDIV.

```
Roman (499,4)
```

Returns ID.

Comments

This function is similar to the Excel function "Roman".

41.7.35 URLEncode

Basic and Crystal syntax.

Overloads

- URLEncode (inputString)
- URLEncode (inputString, encodingScheme)
- URLEncode (inputString, encodingScheme, neverEncode)
- URLEncode (inputString, encodingScheme, neverEncode, alwaysEncode)
- URLEncode (inputString, encodingScheme, characterSet)
- URLEncode (inputString, encodingScheme, characterSet, neverEncode)
- URLEncode (inputString, encodingScheme, characterSet, neverEncode, alwaysEncode)

Arguments

- inputString is a string containing the URL that you want to encode.
- encodingScheme is an integer specifying the scheme that you want to use to encode the string:
 - 0 specifies URL encoding—following RFC 1738
 - 1 specifies HTML form encoding—following application/x-www-form-urlencoded
 - 2 specifies user-defined encoding—only alphanumeric characters are left unencoded by default.If encodingScheme is not specified, the default is 0 (URL encoding).
- neverEncode is a string of characters that you want left untouched by the encoding process. This argument defaults to an empty string.

Note

Only the characters that have ASCII values between 0 and 127 inclusive are allowed in the string that makes up this argument.

- alwaysEncode is a string of characters that you always want encoded by the encoding process. This argument defaults to an empty string.

Note

- Only the characters that have ASCII values between 0 and 127 inclusive are allowed in the string that makes up this argument.
- alwaysEncode takes precedence over neverEncode if the same characters occur in both arguments.

- characterSet is an integer specifying the character encoding (codepage) that you want to use:
 - 0 specifies UTF-8
 - 1 specifies ISO-8859
 - 2 specifies Shift-JISIf characterSet is not specified, the default is 0 (UTF-8 encoding).

Returns

A string that contains the URL-encoded version of your input string.

Action

URLEncode takes a string that contains spaces and special characters and returns an encoded version of that string that can be used as a URL for cross-platform reporting.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
URLEncode ("http://your site.com", 0)
```

Returns http://your%20site.com

```
URLEncode ("http://your site.com", 0, " ")
```

Returns http://your site.com

```
URLEncode ("http://your site.com", 0, " ", " :.")
```

Returns http%3A//your site%2Ecom

Comments

- For URL encoding, these rules apply:
 - Characters that are not encoded by default: A...Z a...z 0...9 \$ - _ . + ! * ' () , ; / ? : @ & = #
 - Characters that are encoded by default: everything else.
- For HTML form encoding, these rules apply:
 - Characters that are not encoded by default: A...Z a...z 0...9 . - * _
 - Special characters: " " is encoded as +
 - Characters that are encoded by default: everything else.
- For user-defined encoding, these rules apply:
 - Characters that are not encoded by default: A...Z a...z 0...9
 - Characters that are encoded by default: everything else.
- Multi-character substitutions are not supported (for example, "<" to "<").

See Also

- [URLDecode \[page 1753\]](#)

41.7.36 URLDecode

Basic and Crystal syntax.

Overloads

- `URLDecode (inputString)`
- `URLDecode (inputString, encodingScheme)`
- `URLDecode (inputString, encodingScheme, characterSet)`

Arguments

- `inputString` is a string containing the encoded URL that you want to decode.
- `encodingScheme` is an integer specifying the scheme that you want to use to decode the string:
 - 0 specifies URL encoding—following RFC 1738
 - 1 specifies HTML form encoding—following `application/x-www-form-urlencoded`
 - 2 specifies user-defined encoding—only alphanumeric characters are left unencoded by default.If `encodingScheme` is not specified, the default is 0 (URL encoding).
- `characterSet` is an integer specifying the character encoding (codepage) that you want to use:
 - 0 specifies UTF-8
 - 1 specifies ISO-8859
 - 2 specifies Shift-JISIf `characterSet` is not specified, the default is 0 (UTF-8 encoding).

Returns

A string that contains the URL-decoded version of your input string.

Action

URLDecode takes a string that contains encoding for spaces and special characters and returns a decoded version of that string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
URLDecode ("http://your+site.com", 0)
```

Returns http://your+site.com

```
URLDecode ("http://your+site.com", 1)
```

Returns http://your site.com

Comments

- For URLEncode and URLDecode to be the inverse of each other, URLDecode need only know what character encoding to use and how to treat the + character (that is, whether to leave it alone or turn it into a space). For this reason, the encodingScheme parameter is necessary.

See Also

- [URLEncode \[page 1750\]](#)

41.8 Date and Time

Date functions allow you to convert numbers to dates (which you can then format to display as you wish) and to convert dates to numbers.

Additional Date/Time functions

[DateTo2000 \(Date, Number\) \[page 1900\]](#)

[DTSTo2000 \(DateTimeString, Number\) \[page 1902\]](#)

[DTSToDate \(DateTimeString\) \[page 1904\]](#)

[DTSToSeconds \(DateTimeString\) \[page 1905\]](#)

[DTSToTimeString \(DateTimeString\) \[page 1906\]](#)

41.8.1 CurrentDate

Basic and Crystal syntax.

Returns

Date Value

Action

CurrentDate returns the print date of your report.

Typical Uses

Use CurrentDate any time you want to stamp your report with the print date.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CurrentDate
```

Returns 1998/07/04, if you begin printing your report on July 4, 1998.

```
CurrentDate
```

Returns 1996/12/11, if you begin printing your report on December 11, 1996.

Comments

- CurrentDate and [PrintDate \[page 1876\]](#) are synonymous.
- The date format is determined by the settings in the International or Regional Settings control panel.
- CurrentDate is identical to the Today function, which was included with previous version of the program. Today is provided for compatibility with reports created with earlier versions of the program. New reports should use the CurrentDate function.
- Formulas and filters that use CurrentDate are re-evaluated when a report is loaded. In a report with saved data, the filter or formula is applied to the saved records. To avoid re-evaluation and to preserve the data, DataDate should be used instead of CurrentDate.

41.8.2 CurrentTime

Basic and Crystal syntax.

Returns

Time Value

Action

CurrentTime returns the print time of your report.

Typical Uses

Use CurrentTime any time you want to stamp your report with the print time.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CurrentTime
```

Returns 4:45:18 if you begin printing your report at 4:45:18.

```
CurrentTime
```

Returns 17:14:33 if you begin printing your report at 17:14:33.

Comments

- CurrentTime and [PrintTime \[page 1877\]](#) are synonymous.
- The time format is determined by the settings in the International or Regional Settings control panel.
- The CurrentTime function is identical to the Now function, which was included with previous versions of the program. Now is provided for compatibility with reports created with earlier versions of the program. New reports should use the CurrentTime function.

41.8.3 CurrentDateTime

Basic and Crystal syntax.

Returns

DateTime Value

Action

CurrentDateTime returns the current date and time on a report. The date and time is taken from your computer's internal clock.

Typical Uses

Use CurrentDateTime any time you want to stamp your report with the current date and time.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CurrentDateTime
```

Returns 1998/07/04 4:45:18, if you begin printing your report at 4:45:18 on July 4, 1998.

```
CurrentDateTime
```

Returns 1996/12/11 17:14:33, if you begin printing your report at 17:14:33 on December 11, 1996.

Comments

- The date and time formats are determined by the settings in the International or Regional Settings control panel.
- CurrentDateTime produces the same result as the formula:

```
CDateTime(CurrentDate, CurrentTime)
```

41.8.4 Date

Crystal syntax.

The [CDate \[page 1838\]](#) and [DateValue \[page 1764\]](#) functions are equivalent to Date. However, Date can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- Date (number)
- Date (string)
- Date (dateTime)
- Date (YYYY, MM, DD)

Arguments

Date (number)	• number is a value representing the number of days starting from December 30, 1899. • It can be positive or negative, and is truncated if fractional.
Date (string)	• string is a text string representing a date, example: "September 20, 1999".
Date (dateTime)	• dateTime is a DateTime value.

Date (YYYY, MM, DD)

- YYYY is a whole number representing a year, example: 1996.
 - MM is a whole number representing a month, example: 12 for December.
 - DD is a whole number representing a day of the month, example: 05.
-

Returns

A Date value.

Action

Date (number)	Returns a Date value given a number of days starting from December 30, 1899.
Date (string)	Returns a Date value given a string that represents a date from January 1, 100 through December 31, 9999.
Date (dateTime)	Returns the date portion of a given DateTime value.
Date (YYYY, MM, DD)	Returns a Date value given numeric arguments of the year, month, and day.

Examples

The following examples are applicable to Crystal syntax:

```
Date ("Dec 31, 1999")
```

Returns the Date value for Dec. 31, 1999.

```
Date (50)
```

Returns the Date value for February 18, 1900.

```
Date (#Oct. 20, 1999 12:02pm#)
```

Returns the Date value for October 20, 1999.

```
Date (1930, 7, 30)
```

Returns the Date value for July 30, 1930.

Comments

You can use the `IsDate` function to check if a `String` argument can be converted to a `Date` before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [DateSerial \(year, month, day\) \[page 1771\]](#)

41.8.5 Time

Crystal syntax.

The [CTime \[page 1840\]](#) and [TimeValue \[page 1766\]](#) functions are equivalent to `Time`. However, `Time` can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- `Time (number)`
- `Time (string)`
- `Time (dateTime)`
- `Time (HH, MM, SS)`

Arguments

Time (number)	• Returns a <code>Time</code> value given a number in units of 24 hours • number can be fractional or negative as well
Time (string)	Returns a <code>Time</code> value that represents the time, given a <code>String</code> expression specifying a time from 0:00:00 (12:00:00 A.M.) to 23:59:59 (11:59:59 P.M.), inclusive.
Time (dateTime)	<code>DateTime</code> is a <code>DateTime</code> value.
Time (hour, min, sec)	• Hour is a whole number representing an hour of the day. • Min is a whole number representing a minute. • Sec is a whole number representing a second.

Returns

Time Value

Action

Creates a Time value from the information provided.

Typical uses

Use this function any time you need to work with a Time value that is obtained either from a DateTime value or from the individual elements of the time (i.e. hour, minute, second).

Examples

The following examples are applicable to Crystal syntax:

```
Time(.2) and Time (1.2)
```

Both return 4:48:00 AM

```
Time(-.2)
```

Returns 7:12:00 PM

```
Time("3:05pm")
```

Returns 3:05:00 PM

```
Time(DateTime(1996, 1, 1, 12, 10, 10))
```

Returns 12:10:10.

```
Time(10, 20, 22)
```

Returns 10:20:22.

See Also

- [TimeValue \[page 1766\]](#)

41.8.6 DateTime

Crystal syntax.

The [CDateTime \[page 1842\]](#) and [DateTimeValue \[page 1768\]](#) functions are equivalent to DateTime. However, DateTime can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- DateTime (date)
- DateTime (number)
- DateTime (string)
- DateTime (date, time)
- DateTime (YYYY, MM, DD)
- DateTime (YYYY, MM, DD, HH, MM, SS)

Arguments

DateTime (date)	date is a Date value.
DateTime (date, time)	• date is a Date value. • time is a Time value.
DateTime (string)	string represents a date and time Example: "September 15, 1999 10:45am"
DateTime (number)	number represents a number of days from December 30, 1899. Example: 20 represents 20 days from December 30, 1899, which is January 19, 1900.

DateTime (year, month, day)

DateTime (year, month, day, hour, min, sec)

- year is a whole number representing a calendar year, example: 1996.
- month is a whole number representing a month, example: 1 for January.
- day is a whole number representing a day of the month, example: 10.
- hour is a whole number representing an hour of the day, example: 12.
- min is a whole number representing a minute, example: 59.
- sec is a whole number representing seconds, example: 30.

Returns

DateTime value.

Action

DateTime (date)

Returns a DateTime value given a Date value, assigning 12:00:00 AM for the time portion for the returned DateTime value.

DateTime (date, time)

Returns a DateTime value given a Date and a Time value.

DateTime (number)

Returns a DateTime value given a number that specifies the number of days from December 30, 1899.

Number can be positive or negative, and can be fractional.

DateTime (string)

Returns a DateTime value given a string that specifies a date and time; various formats of the string are supported.

DateTime (YYYY, MM, DD)

Returns a DateTime value given numeric arguments for the year, month and day.

Assigns 12:00:00 AM for the time portion for the returned DateTime value.

DateTime (YYYY, MM, DD, HH, MM, SS)

Returns a DateTime value given numeric arguments for the year, month, day, hour, minute and second.

Examples

The following examples are applicable to Crystal syntax:

```
DateTime ("10/4/1999 10:20am")
```

Returns the DateTime value October 4, 1999 10:20:00 am.

```
DateTime (12.5)
```

Returns the DateTime value January 11, 1900 12:00:00 pm.

```
DateTime (-2.5)
```

Returns the DateTime value December 27, 1899, 12:00:00 pm.

```
DateTime (CDate ("Dec. 25, 1999"))
```

Returns the DateTime value December 25, 1999 12:00:00 am.

```
DateTime (CDate ("November 10, 1999"), CTime("12:20am"))
```

Returns the DateTime value November 10, 1999 12:20:00 am.

```
DateTime (1945, 8, 21, 0, 0, 0)
```

Returns the DateTime value August 21, 1945 12:00:00 am.

```
DateTime (1945, 8, 21, 10, 0, 0)
```

Returns the DateTime value August 21, 1945 10:00:00 am.

Comments

You can use the `IsDateTime` function to check if a String argument can be converted to a DateTime before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [DateTimeValue \[page 1768\]](#)

41.8.7 DateValue

Basic and Crystal syntax.

The [CDate \[page 1838\]](#) and [Date \[page 1758\]](#) functions are equivalent to DateValue. However, Date can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- DateValue (string)
- DateValue (number)
- DateValue (dateTime)
- DateValue (YYYY, MM, DD)

Arguments

DateValue (number)	• number is a value representing the number of days starting from December 30, 1899. • It can be positive or negative, and is truncated if fractional.
DateValue (string)	string is a text string representing a date, example: "September 20, 1999".
DateValue (dateTime)	dateTime is a DateTime value.
DateValue (YY,MM,DD)	• year is a whole number representing a year, example: 1996. • month is a whole number representing a month, example: 12 for December. • day is a whole number representing a day of the month, example: 05.

Returns

Date value.

Action

DateValue (string) returns a Date value given a string that represents a date from January 1, 100 through December 31, 9999.

DateValue (number) returns a Date value given a number of days starting from December 30, 1899. Number can be positive or negative, and truncated if fractional.

DateValue (*dateTime*) returns the date portion of a given DateTime value.

DateValue (YYYY, MM, DD) returns a Date value given numeric arguments of the year, month and day.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateValue ("Dec 31, 1999")
```

Returns the Date value for Dec. 31, 1999.

```
DateValue (50)
```

Returns the Date value for February 18, 1900.

```
DateValue (#Oct. 20, 1999 12:02pm#)
```

Returns the Date value for October 20, 1999.

```
DateValue (1930, 7, 30)
```

Returns the Date value for July 30, 1930.

Comments

You can use the [IsDate \[page 1775\]](#) function to check if a String argument can be converted to a Date before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [DateSerial \(year, month, day\) \[page 1771\]](#)

41.8.8 TimeValue

Basic and Crystal syntax.

The [CTime \[page 1840\]](#) and [Time \[page 1760\]](#) functions are equivalent to TimeValue. However, Time can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- TimeValue (number)
- TimeValue (string)
- TimeValue (dateTime)
- TimeValue (HH, MM, SS)

Arguments

TimeValue (number)	• Returns a Time value given a number in units of 24 hours • number can be fractional or negative as well.
TimeValue (string)	Returns a Time value that represents the time, given a String expression specifying a time from 0:00:00 (12:00:00 A.M.) to 23:59:59 (11:59:59 P.M.), inclusive.
TimeValue (dateTime)	DateTime is a DateTime value.
TimeValue (hour, min, sec)	• Hour is a whole number representing an hour of the day. • Min is a whole number representing a minute. • Sec is a whole number representing a second.

Returns

Time value.

Action

TimeValue (number) returns a Time value given a number in units of 24 hours. The number can be negative or fractional as well.

TimeValue (string) returns a Time value that represents the time, given a String expression specifying a time from 0:00:00 (12:00:00 A.M.) to 23:59:59 (11:59:59 P.M.), inclusive.

TimeValue (dateTime) returns the time portion of a given DateTime value.

TimeValue (HH, MM, SS) returns a Time value given numeric arguments of the hour, minute and seconds.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
TimeValue (.2)
```

```
TimeValue (1.2)
```

Both return the same value 4:48:00 a.m.

```
TimeValue (-.2)
```

Returns 7:12:00 pm

```
TimeValue ("Jan. 6, 1999 2:30pm")
```

Returns 2:30:00 pm

```
TimeValue (#Feb. 24, 1999 2:40pm#)
```

Returns 2:40:00 pm

```
TimeValue (18, 30, 30)
```

Returns 6:30:30 pm

Comments

You can use the [IsTime \[page 1774\]](#) function to check if a string argument can be converted to a Time before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately. See also [TimeSerial \(hour, minute, second\) \[page 1773\]](#).

41.8.9 DateTimeValue

Basic and Crystal syntax.

The [CDateTime \[page 1842\]](#) and [DateTime \[page 1762\]](#) functions are equivalent to DateTimeValue. However, DateTime can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- DateTimeValue (date)
- DateTimeValue (date, time)
- DateTimeValue (number)

- DateTimeValue (string)
- DateTimeValue (YYYY, MM, DD)
- DateTimeValue (YYYY, MM, DD, HH, MM, SS)

Arguments

DateTimeValue (date)	• date is a Date value.
DateTimeValue (date, time)	• date is a Date value. • time is a Time value.
DateTimeValue (number)	• number represents a number of days from December 30, 1899, example: 20 represents 20 days from December 30, 1899, which is January 19, 1900.
DateTimeValue (string)	• string represents a date and time, example: "September 15, 1999 10:45am".
DateTimeValue (year, month, day)	• year is a whole number representing a calendar year, example: 1996.
DateTimeValue (year, month, day, hour, min, sec)	• month is a whole number representing a month, example: 1 for January. • day is a whole number representing a day of the month, example: 10. • hour is a whole number representing an hour of the day, example: 12. • min is a whole number representing a minute, example: 59. • sec is a whole number representing seconds, example: 30.

Returns

DateTime value.

Action

DateTimeValue (date) returns a DateTime value given a Date value, assigning 12:00:00 AM for the time portion for the returned DateTime value.

DateTimeValue (date, time) returns a DateTime value given a Date and a Time value.

DateTimeValue (number) returns a DateTime value given a number that specifies the number of days from December 30, 1899. Number can be positive or negative, and can be fractional.

DateTimeValue (string) returns a DateTime value given a string that specifies a date and time; various formats of the string are supported.

DateTimeValue (YYYY, MM, DD) returns a DateTime value given numeric arguments for the year, month and day. Assigns 12:00:00 AM for the time portion for the returned DateTime value.

DateTimeValue (YYYY, MM, DD, HH, MM, SS) returns a DateTime value given numeric arguments for the year, month, day, hour, minute and second.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateTimeValue ("10/4/1999 10:20am")
```

Returns the DateTime value October 4, 1999 10:20:00 am

```
DateTimeValue (12.5)
```

Returns the DateTime value January 11, 1900 12:00:00 pm

```
DateTimeValue (-2.5)
```

Returns the DateTime value December 27, 1899, 12:00:00 pm

```
DateTimeValue (CDate ("Dec. 25, 1999"))
```

Returns the DateTime value December 25, 1999 12:00:00 am

```
DateTimeValue (CDate ("November 10, 1999"), CTime("12:20am"))
```

Returns the DateTime value November 10, 1999 12:20:00 am

```
DateTimeValue (1945, 8, 21, 0, 0, 0)
```

Returns the DateTime value August 21, 1945 12:00:00 am

```
DateTimeValue (1945, 8, 21, 10, 0, 0)
```

Returns the DateTime value August 21, 1945 10:00:00 am

Comments

You can use the `IsDateTime` function to check if a String argument can be converted to a DateTime before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

41.8.10 DateSerial (year, month, day)

Basic and Crystal syntax.

Arguments

- year is a whole Number or numeric expression representing a year, example: 1996.
- month is a whole Number or numeric expression representing a month, example: 12 for December.
- day is a whole Number or numeric expression representing a day of the month, example: 5.

Returns

A Date value.

Action

DateSerial returns a Date value for the specified year, month and day. It also handles relative Date expressions.

Typical uses

DateSerial can be used instead of [CDate \[page 1838\]](#) or [DateValue \[page 1764\]](#) to create a Date value out of a year, month and day.

A useful feature of DateSerial is that the month argument does not have to be from 1 to 12 and the day argument does not have to be in the valid range for the number of days in the month. Such dates are interpreted as relative dates, and DateSerial will produce a valid Date value. This can be used to perform many computations with dates, without checking for special cases such as the end of the year, leap years and the number of days in a month. See the examples below for some typical applications.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateSerial (1999, 6, 15)
DateSerial (2000, 1 - 7, 15)
DateSerial (1999, 1, 166)
```

All three return the date June 15, 1999. The second says that 7 months before January 1, 2000 is June 15, 1999. The third says that the 166th day of 1999 is June 15, 1999.

```
DateSerial (1996 + 12, 2 + 13, 29 + 14)
```

Returns April 12, 2009. What it means is that 12 years, 13 months and 14 days from February 29, 1996 is April 12, 2009.

Suppose you want to calculate the last day of the month for the DateTime field {Orders.Order Date}. Notice that in the calculation, DateSerial(Year(d), Month(d) + 1, 1) is the first day of the month after {Orders.Order Date} and then subtracting one day gives the desired result:

```
Rem Basic syntax
Dim d
d = {Orders.Order Date}
formula = DateSerial(Year(d), Month(d) + 1, 1 - 1)
//Crystal syntax
Local DateTimeVar d := {Orders.Order Date};
DateSerial(Year(d), Month(d) + 1, 1 - 1)
```

It is often useful to combine DateSerial with other date functions. For example, if you want to calculate the date of the last Friday of the month for the field {Orders.Order Date} you can do the following.

The calculation proceeds by finding the last day of the month and then subtracting off a number of days to get to a Friday. Since some months have 5 Fridays (example: October 1999) and some have 4 Fridays (example: November 1999), this approach is easier than working from the first day of the month.

```
Rem Basic syntax
Dim d1, d2
d1 = {Orders.Order Date}
d2 = DateSerial(Year(d1), Month(d1) + 1, 1 - 1)
formula = d2 - (WeekDay(d2, crFriday) - 1)
//Crystal syntax
Local DateTimeVar d1 := {Orders.Order Date};
Local DateVar d2;
d2 := DateSerial(Year(d1), Month(d1) + 1, 1 - 1);
d2 - (DayOfWeek(d2, crFriday) - 1)
```

Returns the Date value March 27, 1998 if {Orders.Order Date} is March 18, 1998.

Here is an example that returns the date of the second Tuesday of the month 3 months before {Orders.Order Date}:

```
Rem Basic syntax
Dim d1, d2
d1 = {Orders.Order Date}
d2 = DateSerial(Year(d1), Month(d1) - 3, 1)
formula = d2 + (2 * 7 - (WeekDay(d2, crTuesday) - 1))
//Crystal syntax
Local DateTimeVar d1 := {Orders.Order Date};
Local DateVar d2 := DateSerial(Year(d1), Month(d1) - 3, 1);
d2 + (2 * 7 - (DayOfWeek(d2, crTuesday) - 1))
```

Returns the Date value December 9, 1997 if {Orders.Order Date} is March 18, 1998.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.11 TimeSerial (hour, minute, second)

Basic and Crystal syntax.

Arguments

- hour is a Number or numeric expression specifying the hour.
- minute is a Number or numeric expression specifying the number of minutes.
- second is a Number or numeric expression specifying the number of seconds.

Returns

Time value.

Action

TimeSerial returns a Time value specifying the time for a specific hour, minute and second.

Typical uses

TimeSerial can be used instead of [CTime \[page 1840\]](#) or [TimeValue \[page 1766\]](#) to create a Time value out of an hour, minute and second.

A useful feature of TimeSerial is that the hour, minute and second arguments do not need to fall within normally valid ranges. That is, the hour does not need to be from 0 to 24 and the minute and second do not need to be from 0 to 59. Such times are interpreted as relative times, and TimeSerial will produce a valid Time value. This can be used to easily perform many computations with times.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
TimeSerial (18, 45, 0)
TimeSerial (22 - 3, 15 - 30, 0)
```

Both return the same time 6:45 PM. The second has the interpretation that 3 hours and 30 minutes before 10:15 PM is 6:45 PM.

```
TimeSerial (10 + 20, 30 + 55, 0)
```

Returns 7:25 AM. What it means is that 20 hours and 55 minutes from 10:30 AM is 7:25 AM (of the next day).

Comments

This function is similar to the Visual Basic function of the same name. There is a difference when the relative times span more than one day. For example, in Crystal Reports,

```
TimeSerial (1, 0 - 180, 0)
```

returns 10:00 PM. This has the interpretation that 180 minutes before 1:00 AM is 10:00 PM. However, the same function call in Visual Basic returns 2:00 AM. The difference occurs because 180 minutes before 1:00 AM moves the time to 10:00 PM of the previous day and Visual Basic then handles this differently.

41.8.12 IsTime

Basic and Crystal syntax.

Overloads

- IsTime (string)
- IsTime (number)

Arguments

- string is a String value or expression to be tested for being convertible to a Time value. Many forms are accepted.
- number is a Number value or expression to be tested for being convertible to a Time value. It can be positive, negative or fractional. It is interpreted as units of 24 hours. What this means is that 0 is 12 midnight and 0.5 is 12 noon.

Returns

Boolean value (True or False).

Action

IsTime returns True if the given String or Number value can be converted to a valid Time and returns False otherwise.

Typical use

If you want to convert a String value to a Time using the functions [CTime \[page 1840\]](#) or [TimeValue \[page 1766\]](#), use the IsTime function first to check if the conversion will succeed. See the examples section of [IsDate \[page 1775\]](#) for an example of this type.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IsTime ("10:30 pm")
```

Returns True.

```
IsTime ("September 30,1999")
```

Returns True since the given String argument is interpreted as 12:00:00 am.

```
IsTime (3.2)
```

Also returns True since the given Number argument is interpreted as 3.2 units of 24 hours, which is 4:48:00am.

Comments

This function is similar to the Visual Basic function IsDate, except that it works specifically with Crystal Report's Time type which holds Time values only.

41.8.13 IsDate

Basic and Crystal syntax.

Overloads

- IsDate (string)
- IsDate (number)

Arguments

- string is a String value or expression to be tested for being convertible to a Date value. Many forms are accepted.
- number is a Number value or expression to be tested for being convertible to a Date value. It can be positive, negative or fractional. It is interpreted as a number of days since December 30, 1899.

Returns

Boolean value (True or False).

Action

IsDate returns True if the given String or Number value can be converted to a valid Date and returns False otherwise. A valid Date is any date between January 1, 100 through December 31, 9999.

Typical use

If you want to convert a String value to a Date using the functions [CDate \[page 1838\]](#) or [DateValue \[page 1764\]](#), use the IsDate function first to check if the conversion will succeed. See below for an example.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IsDate ("Jan 1, 1999")
```

Returns True.

```
IsDate (100)
```

Also returns True since the number 100 is interpreted as 100 days from Dec. 30, 1899, which is April 9, 1900.

```
IsDate ("Feb 29, 1999")
```

Returns False since 1999 is not a leap year and so the String cannot be converted to a Date.

Suppose an orders report is grouped monthly by order date. Here is a formula that returns the Date value that is extracted from the GroupName String field. If the report is a Top N report, the GroupName field can have a value "Others". This cannot be converted into a Date, and so IsDate is used to prevent an error from being generated in the call to CDate:

```
Rem Basic syntax
Dim s As String
s = GroupName ({Orders.Order Date}, "monthly")
If IsDate(s) Then
    formula = CDate(s)
Else
    formula = CDate(0,0,0)
End If
//Crystal syntax
Local StringVar s := GroupName ({Orders.Order Date}, "monthly");
If IsDate(s) Then
    CDate(s)
Else
    CDate(0,0,0)
```

Returns the Date value May 1, 1998 if the GroupName field value is "May - 1998". Returns the null Date (a non-printing Date value) if the GroupName field value is "Others".

Comments

This function is similar to the Visual Basic function of the same name, except that it works specifically with Crystal Report's Date type which holds date values only.

41.8.14 IsDateTime

Basic and Crystal syntax.

Overloads

- IsDateTime (string)
- IsDateTime (number)

Arguments

- string is a String value or expression to be tested for being convertible to a DateTime value. Many forms are accepted.
- number is a Number value or expression to be tested for being convertible to a DateTime value. It can be positive, negative or fractional. It is interpreted as a number of days since December 30, 1899.

Returns

Boolean value (True or False).

Action

IsDateTime returns True if the given String or Number value can be converted to a valid DateTime and returns False otherwise. A valid DateTime is any date-time between January 1, 100 through December 31, 9999.

Typical use

If you want to convert a String value to a DateTime using the functions CDateTime or DateTimeValue, use the IsDateTime function first to check if the conversion will succeed. See the examples section of [IsDate \[page 1775\]](#) for an example of this type.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IsDateTime ("September 30, 1999")
```

Returns True since the given string "September 30, 1999 is interpreted as the DateTime value September 30, 1999, 12:00:00am.

```
IsDateTime (-100.2)
```

Returns True since the given number argument is interpreted as the DateTime value September 20, 1899, 7:12:00 pm.

Comments

This function is similar to the Visual Basic function IsDate.

41.8.15 Year (x)

Basic and Crystal syntax.

Arguments

- x is a Date value or a DateTime value.

Returns

Whole Number

Action

Year extracts the year from a date and returns it as a Number.

Typical uses

Use this function any time you need to use a year, converted to a Number, in calculations or comparisons.

Examples

The following examples are applicable to Crystal syntax:

```
Year({file.LAST INVOICE ON})
```

Returns 1989, where the last invoice date was in 1989.

```
If Year({file.LAST UPGRADE}) < 1988 Then  
    "P.S. You are missing out on many of the benefits of our new, improved  
    version."  
Else
```

```
""
```

Prints a P.S. for those customers who last upgraded prior to 1988, and prints nothing for those customers who show a more recent upgrade.

```
Year(CDateTime(1996, 3, 3, 10, 33, 20))
```

Returns 1996.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.16 Month (x)

Basic and Crystal syntax.

Arguments

- x is a Date value or a DateTime value.

Returns

Whole Number

Action

Month extracts the month component of a date and converts it to a Number.

Typical uses

Use any time you need only the month component of a date. For example, if you are tracking payments that fall within a given year, you are interested only in the month they arrive; day and year information would be redundant. Also, if you need to use the number of the month in a numeric calculation (Month({file.OCTPMT}) - Month({file.SEPPTMT})), for example) use the Month function to extract the month component of the date and convert it to a Number.

Examples

The following examples are applicable to Crystal syntax:

```
Month({file.INVOICE}) - Month({file.PAYMENT})
```

Calculates the number of months between the Invoice and the Payment.

```
If Month({file.LAST PURCHASE}) < 6 Then
    "You will see some changes since you last shopped our store."
Else
    "As you have certainly noticed, we have been making many changes in our
    store recently.";
```

Prints "You will see some changes since you last shopped our store." for those customers whose last purchase was in May or earlier, and prints "As you have certainly noticed, we have been making many changes in our store recently." for customers who have purchased more recently than May.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.17 Day (x)

Basic and Crystal syntax.

Arguments

- x is a Date value or a DateTime value.

Returns

Whole Number (the day of the month)

Action

Day extracts the day from a Date or DateTime value and returns a whole number.

Typical uses

Use any time you need only the day component of a Date or DateTime value. For example, if you are tracking payments that fall within a given month, you are interested only in the day they arrive; month and year information would be redundant. Also, if you need to use the day of the month in a numeric calculation (example: `Day({file.OCTPMT}) - Day ({file.SEPPMT})`), use the Day function to extract the day of the month and convert it to a Number.

Examples

The following examples are applicable to Crystal syntax:

```
Day({file.LAST INVOICE})
```

Returns 10 where the last invoice date was the 10th day of the month.

```
If Day({file.LAST PAYMENT}) < 15 Then
    "Past Due"
Else
    ""
```

If the day of the last payment was less than 15, print "Past Due", otherwise print nothing.

```
Day(CDateTime(1996, 3, 3, 10, 33, 20))
```

Returns 3.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.18 WeekDay

WeekDay and [DayOfWeek \[page 1784\]](#) are equivalent functions. However, WeekDay is preferred in Basic syntax and DayOfWeek is preferred in Crystal syntax.

Overloads

- WeekDay (date)
- WeekDay (date, firstDayOfWeek)

Arguments

- `date` is a `Date` or `DateTime` value for which a numeric designation for the day of the week is to be returned.
- `firstDayOfWeek` is an optional `Number` indicating the first day of the week. See [DayOfWeek \[page 1784\]](#) for more information. If not specified, Sunday is assumed.

Returns

Number value.

Action

`WeekDay` returns a number `n` which is the `n`th day of the week given a `Date` or `DateTime` value.

Typical use

`WeekDay` is often useful in performing date computations that take into account special days of the week such as weekends. See the [DateSerial \(year, month, day\) \[page 1771\]](#) examples section for some examples.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
WeekDay (#Oct . 4, 1999#)
```

Returns 2 since Oct. 4, 1999 is a Monday, and the first day of the week is assumed to be Sunday.

```
WeekDay (#Oct . 4, 1999#, crMonday)
```

Returns 1 since Oct. 4, 1999 is a Monday, and the first day of the week is specified to be Monday.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.19 DayOfWeek

DayOfWeek and [WeekDay \[page 1782\]](#) are equivalent functions. However, WeekDay is preferred in Basic syntax and DayOfWeek is preferred in Crystal syntax.

Overloads

- DayOfWeek (date)
- DayOfWeek (date, firstDayOfWeek)

Arguments

- date is a Date value or dateTime value.

Returns

Whole Number

Action

DayOfWeek determines the day of the week the given date falls on, and converts the day of the week to a Number (1 to 7). Optionally a numeric value for the first day of the week can be specified. If the first day of the week is not specified, Sunday is assumed.

Typical uses

Use this function any time you need to use the day of the week as a Number.

Examples

The following examples are applicable to Crystal syntax:

```
DayOfWeek(Date(1990,10,1))
```

Returns 2 where October 1, 1990 is a Monday.

```
If DayOfWeek({orders.ORDER DATE}) = 3 Then
    "Sam"
Else
    "Bill"
```

Determines whether Sam or Bill was on duty on September 8, 1990.

```
If DayOfWeek({orders.ORDER DATE}) = 7 Then
    "Saturday"
Else
    ""
```

Returns "Saturday" if the DayOfWeek is 7; otherwise, it returns an empty string.

```
DayOfWeek(#Sept. 24, 1999#, crMonday)
```

Returns 5 where Sept. 24, 1999, is a Friday, and Monday is specified to be the first day of the week.

Comments

If you want to get the day of the week spelled out, in Crystal syntax, use this formula:

```
["Sun", "Mon", "Tues", ...] [DayOfWeek(Date)]
```

Sets up an array (["Sun",...]) and uses the number of the day of the week (Sun = 1, Sat = 7) to select the desired date name from the array.

41.8.20 Hour (x)

Basic and Crystal syntax.

Arguments

- x is a Time value or DateTime value.

Returns

Whole Number

Action

The Hour function extracts the hour portion of a Time value and returns it as a whole number.

Typical uses

Use this function anytime you need to display just the hour portion of a Time value or to perform calculations based on hours.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Hour(CTime(10, 20, 22))
```

Returns 10.

```
Hour(CTime(10, 00, 30)) - Hour(CTime(6, 00, 00))
```

Returns 4.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.21 Minute (x)

Basic and Crystal syntax.

Arguments

- x is a Time value or DateTime value.

Returns

Whole Number

Action

Retrieves just the minutes from a Time value.

Typical uses

Use this function whenever you need to work with the minutes specified in a Time value for performing time calculations.

Examples

The following examples are applicable to Crystal syntax:

```
Minute(CTime(10, 20, 22))
```

Returns 20.

```
TimeVar ShipTime := CTime({orders.SHIP DATE});  
Minute(ShipTime)
```

Returns 30 if {orders.SHIP DATE} is 1998/12/05 8:30:15.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.22 Second (x)

Basic and Crystal syntax.

Arguments

- x is a Time value or DateTime value.

Returns

Whole Number

Action

Extracts the seconds portion of a specified Time value.

Typical uses

Use this function anytime you need to perform calculations with the seconds indicated in a Time value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Second(CTime(10, 20, 22))
```

Returns 22.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.23 MonthName

Basic and Crystal syntax.

Overloads

- MonthName(month)
- MonthName(month, abbreviate)

Arguments

- month is a whole Number representing the month of the year, value between 1 and 12, with 1 being January.
- abbreviate is an optional Boolean value that indicates if the month name is to be abbreviated. If omitted, the default is False.

Returns

String value.

Action

MonthName returns a string name for the specified month.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
MonthName (4)
```

Returns the String value "April".

```
MonthName (4, True)
```

Returns "Apr".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.24 WeekdayName

Basic and Crystal syntax.

Overloads

- WeekdayName (weekday)
- WeekdayName (weekday, abbreviate)
- WeekdayName (weekday, abbreviate, firstDayOfWeek)

Arguments

- weekday is the numeric designation for the day of the week. Numeric value of each day depends on the value of the firstDayOfWeek argument. If firstDayOfWeek is not specified, crSunday is assumed.
- abbreviate is an optional Boolean value that indicates if the weekday name is to be abbreviated. If omitted, the default is False.
- firstDayOfWeek is an optional Number indicating the first day of the week. See [Day of Week constants \[page 1519\]](#).

Returns

String value.

Action

WeekdayName returns a String indicating the name of the specified day of the week.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
WeekdayName (3)
```

Returns "Tuesday" since Sunday is assumed to be the first day of the week.

```
WeekdayName (3, True, crMonday)
```

Returns "Wed" since abbreviation is selected and the first day of the week is specified to be Monday.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.25 Timer

Basic and Crystal syntax.

Returns

Number value.

Action

Timer returns the number of seconds elapsed since midnight.

Example

The following example is applicable to both Basic and Crystal syntax:

```
Timer
```

Returns 57,720 if the current time is 4:02 PM.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.26 DateAdd (intervalType, nIntervals, startDateTime)

Basic and Crystal syntax.

Arguments

- intervalType is a String expression specifying the interval of time to be added. Values can be:

IntervalType value	Description
yyyy	Year
q	Quarter (3-month period)
m	Month
y	Day of year
d	Day
w	Weekday
ww	Week (7-day period)
h	Hour
n	Minute
s	Second

- nIntervals is a Number or numeric expression specifying the number of intervals to be added. It can be positive (to get date-times in the future) or negative (to get date-times in the past).
- startDateTime is the DateTime value to which the intervals are to be added.

Returns

A DateTime value.

Action

DateAdd returns a DateTime value to which a specified number of time intervals have been added.

Typical use

DateAdd is used to add intervals of time to a DateTime. Its main feature is that the DateTime returned will always be valid. For example, DateAdd takes into account such factors as the number of days in a month and leap years. If you want to add or subtract days to a DateTime, you could use the addition and subtraction

operators instead of DateAdd with the "d" parameter. However, DateAdd also handles other types of intervals such as adding months or hours to a DateTime.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateAdd("d", -32, #Sept 28, 1999#)
```

Returns the DateTime value for August 27, 1999.

```
DateAdd ("m", 1, #1/31/1996#)
```

Returns the DateTime value for February 29, 1996. Notice that DateAdd will not return the invalid value February 31, 1996.

```
DateAdd ("q", 17, #Sept 28, 1999#)
```

Returns the DateTime value for December 28, 2003.

```
DateAdd ("h", -400, #Sept 28, 1999#)
```

Returns the DateTime value for September 11, 1999 8:00 AM. In other words, this is the result of subtracting 400 hours from September 28, 1999 12:00 AM.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- To add days to a date-time, you can use any of the interval type parameters "y", "d" or "w". They all have the same effect for DateAdd.
- DateAdd returns a DateTime value and not a Date value. However, you may need to convert this DateTime value to a Date value in certain situations (such as if you wanted to assign the value returned by DateAdd to a Date type variable). To convert to a Date value, use DateAdd in combination with the CDate type conversion function. For example, the following returns the Date value for October 6, 1997:

```
CDate(DateAdd("yyyy", -2, #October 6, 1999#))
```

41.8.27 DateDiff

Basic and Crystal syntax.

Overloads

- `DateDiff (intervalType, startDateTime, endDateTime)`
- `DateDiff (intervalType, startDateTime, endDateTime, firstDayOfWeek)`

Arguments

- `intervalType` is a String expression that is the interval of time you use to calculate the difference between `startDateTime` and `endDateTime`. Possible values can be:

Interval type value	Description
yyyy	Year
q	Quarter
m	Month
y	Day of year
d	Day (both "y" and "d" find the difference in days)
w	Number of weeks between startDateTime and endDateTime
ww	Number of firstDayOfWeek's between startDateTime and endDateTime
h	Hour
n	Minute
s	Second

- `startDateTime` is the first DateTime value used in calculating the difference.
- `endDateTime` is the second DateTime value used in calculating the difference.
- `firstDayOfWeek` is an optional constant specifying the first day of the week. If not specified, `crSunday` is assumed. See [Day of Week constants \[page 1519\]](#).

Returns

A Number value.

Action

DateDiff returns a number of time intervals between two specified dates.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Use DateDiff with the "d" or "y" interval type parameter to find the number of days between two dates:

```
DateDiff ("d", #10/7/1999#, #10/10/1999#)
```

Returns 3.

Use DateDiff with the "yyyy" interval type parameter to find the number of years difference between two dates. This use of DateDiff is the same as finding the difference between the year of endDateTime and the year of startDateTime.

```
DateDiff ("yyyy", #10/7/1999#, #2/10/2005#)
```

Returns 6.

```
DateDiff ("yyyy", #12/31/1999#, #1/1/2000#)
```

Returns 1 (a 1 year difference), even though there is only a 1 day difference between the dates.

```
DateDiff ("yyyy", #1/1/1999#, #12/31/1999#)
```

Returns 0 (a 0 year difference), even though there is a 364 day difference.

Suppose that for the above examples, the first date is the date that you bought a mutual fund, and the second date is the date you sold it. The mutual fund company must send you an annual report for every year in which you owned units in the fund. Thus, you would get 7, 2 and 1 annual reports respectively, in the above cases.

Use DateDiff with the "q" parameter to find the number of quarters (3 month periods) difference between two dates.

```
DateDiff ("q", #10/6/1999#, #5/20/2003#)
```

Returns 14.

```
DateDiff ("q", #3/31/1999#, #4/1/1999#)
```

Returns 1. The two dates are in adjacent quarters.

```
DateDiff ("q", #1/1/1999#, #3/31/1999#)
```

Returns 0. The two dates are in the same quarter.

Suppose the mutual fund company in the "yyyy" example mailed out quarterly reports. Then it would need to mail out 15, 2 and 1 quarterly reports respectively, in the above cases.

Use DateDiff with the "m" parameter to find the number of months difference between two dates.

```
DateDiff ("m", #3/15/1999#, #7/13/1999#)
```

Returns 4.

Use DateDiff with the "w" parameter to calculate the number of weeks between two dates. For example, if startDateTime is on a Tuesday, DateDiff counts the number of Tuesdays between startDateTime and endDateTime not including the initial Tuesday of startDateTime. Note however that it counts endDateTime if endDateTime is on a Tuesday.

```
DateDiff ("w", #10/19/1999#, #10/25/1999#)
```

Returns 0.

```
DateDiff ("w", #10/19/1999#, #10/26/1999#)
```

Returns 1.

Use DateDiff with the "ww" parameter to calculate the number of firstDayOfWeek's occurring between two dates. For the DateDiff function, the "ww" parameter is the only one that makes use of the firstDayOfWeek argument. It is ignored for all the other interval type parameters. For example, if firstDayOfWeek is crWednesday, it counts the number of Wednesday's between startDateTime and endDateTime. It does not count startDateTime even if startDateTime falls on a Wednesday, but it does count endDateTime if endDateTime falls on a Wednesday. For the examples, note that October 6, 13, 20 and 27 all fall on a Wednesday in 1999.

```
DateDiff ("ww", #10/5/1999#, #10/29/1999#, crWednesday)
```

Returns 4.

```
DateDiff ("ww", #10/6/1999#, #10/29/1999#, crWednesday)
```

Returns 3.

```
DateDiff ("ww", #10/5/1999#, #10/27/1999#, crWednesday)
```

Returns 4.

For example, suppose you want to calculate the number of days between the order date and ship date, excluding Saturdays and Sundays:

```
Rem Basic syntax
Dim d1, d2
d1 = {Orders.Order Date}
d2 = {Orders.Ship Date}
formula = DateDiff("d", d1, d2) - _
          DateDiff("ww", d1, d2, crSaturday) - _
          DateDiff("ww", d1, d2, crSunday)
//Crystal syntax
Local DateTimeVar d1 := {Orders.Order Date};
Local DateTimeVar d2 := {Orders.Ship Date};
DateDiff ("d", d1, d2) -
  DateDiff ("ww", d1, d2, crSaturday) -
  DateDiff ("ww", d1, d2, crSunday)
```

Comments

This function is designed to work like the Visual Basic function of the same name.

41.8.28 DatePart

Basic and Crystal syntax.

Overloads

- DatePart (intervalType, inputDateTime)
- DatePart (intervalType, inputDateTime, firstDayOfWeek)
- DatePart (intervalType, inputDateTime, firstDayOfWeek, firstWeekOfYear)

Arguments

- intervalType a String expression that specifies the part of a date to be returned. Possible values can be:

Interval type value	Description
yyyy	Extracts the year
q	Quarter (the result is 1, 2, 3 or 4)
m	Month (the result is from 1 to 12)
y	Day of year (1 to 365 or 366 in a leap year)
d	Day part of the date (1 to 31)
w	Day of week (1 to 7 with the result depending on first-DayOfWeek)
ww	Week of year (1 to 53 with firstDayOfWeek and firstWeekOfYear determining the exact days of the first calendar week of the year)
h	Extracts the hour part of the given DateTime (0 to 23)
n	Minute part (0 to 59)

Interval type value	Description
s	Second part (0 to 59)

- inputDateTime is the DateTime value whose part will be extracted.
- firstDayOfWeek is an optional constant used to specify the first day of the week. If not specified, crSunday is assumed. See [Day of Week constants \[page 1519\]](#) for more information.
- firstWeekOfYear is an optional constant specifying the first week of the year. If not specified, the first week is assumed to be the one in which Jan. 1 occurs (crFirstJan1). See [First Week of Year constants \[page 1520\]](#).

Returns

A Number value.

Action

DatePart returns a Number that specifies a given part of a given date.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DatePart ("d", #August 15, 1999#)
```

Returns 15.

```
DatePart ("m", #August 15, 1999#)
```

Returns 8.

```
DatePart ("n", #10:35 am#)
```

Returns 35.

```
DatePart ("q", #September 29, 1999#)
```

Returns 3 since September 29 is in the third quarter of the year.

```
DatePart ("ww", #September 14, 1997#)
```

Returns 38 since September 14, 1997 is in the 38th week of 1997.

Suppose that a company wants to stamp an id number on each of its shipments for quality control purposes. The id is composed of the order id, followed by a dash, followed by the week the order was shipped, followed by the last two digits of the year. Here is an example formula that produces these ids:

```
Rem Basic syntax
formula = CStr({Orders.Order ID}, 0, "") & "-" & _
          CStr(DatePart("ww", {Orders.Ship Date}), 0) & _
          CStr({Orders.Ship Date}, "yy")
//Crystal syntax
CStr({Orders.Order ID}, 0, "") & "-" &
CStr(DatePart("ww", {Orders.Ship Date}), 0) &
CStr({Orders.Ship Date}, "yy")
```

Returns the string "2082-3897" if {Orders.Order ID} is 2082 and {Orders.Ship Date} is September 14, 1997.

Comments

This function is designed to work like the Visual Basic function of the same name.

The DatePart function with "yyyy" intervalType argument is the same as the Year function. Similarly, the DatePart function with "m", "d", "w", "h", "n" and "s" intervalType argument is the same as the functions Month, Day, Weekday (or DayOfWeek), Hour, Minute and Second respectively. On the other hand, there is no easy alternative to using the DatePart function for the "q", "y" and "ww" intervalType arguments.

The firstDayOfWeek argument affects the DatePart function when the interval type argument is "w" or "ww". For all other intervalType argument values, it is ignored.

The firstWeekOfYear argument affects the DatePart function only when the intervalType argument is "ww". For all other intervalType argument values, it is ignored.

41.8.29 First Day of Week/Year constants

[Day of Week constants \[page 1519\]](#)

[Day of Week enclosure constants \[page 1520\]](#)

[First Week of Year constants \[page 1520\]](#)

41.8.30 ShiftDateTime (inputDateTime, inputTimeZone, newTimeZone)

Basic and Crystal syntax.

Arguments

- `inputDateTime`—a `DateTime` value to be shifted.
- `inputTimeZone`—a `"TimeZoneString"` representing the time zone of the `inputDateTime`.
- `newTimeZone`—a `"TimeZoneString"` representing the time zone to which the `inputDateTime` is shifted

Returns

A `DateTime` value.

Action

`ShiftDateTime` shifts a `DateTime` value from one time zone to another time zone.

Example

The following example is applicable to both Basic and Crystal syntax:

```
ShiftDateTime (DateTime (2000, 1, 1, 12, 0, 0), "EST,300,EDT", "PST,480,PDT")
```

Returns 1/1/2000 9:00:00AM

In this example, noon Eastern Daylight Time is shifted to 9 am Pacific Daylight Time.

Comments

Time zones are represented by `TimeZoneString`, which is a string that follows these formats:

- `"std,offset,dst,[offset];start[/time],end[/time]"`
Example: `"PST,480,PDT,-60;4.1.0/02:00,10.1.0/02:00"`
- `"std,offset,dst,[offset]"` (if the time zone follows the US Daylight Saving Time (DST) rules)
Example `"PST,480,PDT,-60"`
- `"std,offset"` (if there is no DST)
Example `"South Africa,-120"`
- `""` (an empty string indicates the local time zone from the OS settings)

Syntax	Description
std	String name of the standard time zone.
offset	Offset, in minutes, of the standard time zone (west is positive).
dst	String name of the DST time zone.
offset	Optional DST offset from standard (defaults to one hour ahead of standard offset).
start	Date when DST goes into effect. The format is m.w.d—representing the month, week, and day number of the date. w=1 is the first week of the month, and w=5 is the last week of the month.
time	Optional time that DST goes into effect (24-hour clock). The default is 02:00 (local to the given time zone).
end	Date when time zone changed back to standard.

41.9 Date Ranges

Date range functions are preset date ranges that can be used in formulas.

41.9.1 WeekToDateFromSun

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all days from last Sunday to Today (including today).

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in WeekToDateFromSun Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls within the range of last Sunday to Today (including today). The following table identifies the first and last dates included in the date range if today equals 04/13/95.

If today = 04/13/95	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range (previous Saturday)	04/08/95	0
first date included (previous Sunday)	04/09/95	1252.24
	04/11/95	1597.87
	04/12/95	2546.35
last date included	04/13/95	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.2 MonthToDate

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates from the first day of the month to today.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in MonthToDate Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from the first day of the month to today. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	03/31/98	0
first date included	04/01/98	1252.24
	04/05/98	1597.87
	04/09/98	2546.35
last date included	04/10/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.3 YearToDate

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all days from the first day of the calendar year to today.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in YearToDate Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Order Amount if the date falls within the range of the first day of the calendar year to today. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	12/31/97	0
first date included	01/01/98	1252.24
	02/28/98	1597.87
	03/15/98	2546.35
last date included	04/10/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.4 Last7Days

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates from seven days ago to today (including today).

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Last7Days Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from seven days ago to today (including today). The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	04/03/98	0
first date included	04/04/98	1252.24
	04/06/98	1597.87
	04/08/98	2546.35
last date included	04/10/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.5 Last4WeeksToSun

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of dates that includes the four weeks previous to last Sunday. A week begins on a Monday and ends on a Sunday. For example, September, 1996, begins on a Sunday.

- If today is September 22 (Sunday), Last4WeeksToSun begins on August 26 (Monday) and ends on September 22 (today, Sunday).
- If today is September 28 (a Saturday), Last4WeeksToSun begins on August 26 (a Monday) and ends on September 22 (the previous Sunday).

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Last4WeeksToSun Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls within the last 4 weeks to Sunday (See Action section for specifics). The following table identifies the first and last dates included in the date range if today equals 04/10/95 (Monday).

If today = 04/10/95	{orders. ORDER DATE}	{orders. ORDER AMOUNT}
not in range	03/12/95	0
first date included	03/13/95	1252.24
	03/27/95	1597.87
	04/05/95	2546.35
last date included (Sunday before current date)	04/09/95	5462.21

 **Note**

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.6 LastFullWeek

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates from Sunday to Saturday of the previous week.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in LastFullWeek Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from Sunday to Saturday of the last full week. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	03/28/98	0
first date included	03/29/98	1252.24
last date included	04/04/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.7 LastFullMonth

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates from the first to last day of the previous month.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in LastFullMonth Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from the previous month. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	02/28/98	0
first date included	03/01/98	1252.24
	03/15/98	1597.87
	03/27/98	2546.35
last date included	03/31/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.8 AllDatesToToday

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes every day up to and including the present day.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in AllDatesToToday Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from the first date entered to today (including today). The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
first date included (earliest date in field)	12/10/97	1252.24
	01/05/98	1597.87
	03/05/98	2546.35
last date included	04/10/98	5462.21
not in range	04/11/98	0

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.9 AllDatesToYesterday

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes every day up through the previous day. AllDatesToYesterday includes all dates before today, but does not include the present day.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in AllDatesToYesterday Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from the first date entered to yesterday. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
first date included (earliest date in field)	12/10/97	1252.24
	01/05/98	1597.87
	03/05/98	2546.35
last date included	04/09/98	5462.21
not in range	04/10/98	0

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.10 AllDatesFromToday

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes any date from the present day to any future Date value that may appear in a field. AllDatesFromToday includes the present day.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in AllDatesFromToday Then  
    {orders.ORDER AMOUNT}  
Else  
    0
```

Returns the Amount if the date falls on any date from today (including today) to all future dates. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	04/09/98	0
first date included	04/10/98	1252.24
	07/08/98	1597.87
	09/24/98	2546.35
last date included (latest date in field)	10/08/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.11 AllDatesFromTomorrow

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that fall after the present day. AllDatesFromTomorrow does not include the present day, but does include any future date.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in AllDatesFromTomorrow Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date from tomorrow to all future dates. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	04/10/98	0
first date included	04/11/98	1252.24
	07/08/98	1597.87
	09/24/98	2546.35
last date included (latest date in field)	10/08/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.12 Aged0To30Days, Aged31To60Days, Aged61To90Days

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that fall within a certain time period before the present date. If the current date is 12/30/98, Aged0To30Days specifies the period from 12/1/98 to the present date, Aged31To60Days specifies the period from 11/1/98 to 11/30/98, and Aged61To90Days specifies the period 10/2/98 to 10/31/98.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Aged0To30Days Then  
    {orders.ORDER AMOUNT}  
Else  
    0
```

Returns the Amount if the date falls within the previous 30 days from today's date. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	03/10/98	0
first date included	03/11/98	1252.24
	03/15/98	1597.87
	04/05/98	2546.35
last date included	04/10/98	5462.21

If {orders.ORDER DATE} in Aged31To60Days Then {orders.ORDER AMOUNT} Else 0

Returns the Amount if the date falls within the previous 31 to 60 days from today's date. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	02/08/98	0
first date included	02/09/98	1252.24
	02/25/98	1597.87
	03/05/98	2546.35
last date included	03/10/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.13 Over90Days

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all values that are more than 90 days older than the current date.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Over90Days Then  
    {orders.ORDER AMOUNT}  
Else  
    0
```

Returns the Amount if the date falls in the range of over 90 days from today's date. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
first date included (earliest date in field)	12/10/97	1252.24
	12/31/97	1597.87
	01/05/98	2546.35
last date included	01/09/98	5462.21
not in range	01/10/98	0

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.14 Next30Days, Next31To60Days, Next61To90Days, Next91To365Days

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates in the period specified starting from today (including today).

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Next30Days Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls anywhere in the next 30 days starting from today (including today). The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	04/09/98	0
first date included	04/10/98	1252.24
	04/30/98	1597.87
	05/05/98	2546.35
last date included	05/10/98	5462.21

If {orders.ORDER DATE} in Next31To60Days Then {orders.ORDER AMOUNT} Else 0

Returns the Amount if the date falls anywhere in the next 31 to 60 days starting from today. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	05/10/98	0
first date included	05/11/98	1252.24
	05/30/98	1597.87

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
	06/03/98	2546.35
last date included	06/09/98	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.15 Calendar1stQtr, Calendar2ndQtr, Calendar3rdQtr, Calendar4thQtr

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that falls within the 1st, 2nd, 3rd, or 4th quarter of the current calendar year. The first quarter of the calendar year includes all dates from January 1st through March 31st. The second quarter of the calendar year includes all dates from April 1st through June 30th. The third quarter of the calendar year includes all dates from July 1st through September 30th. The fourth quarter of the calendar year includes all dates from October 1st through December 31st.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Calendar1stQtr Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date in the 1st quarter of the calendar year. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	12/10/97	0
	01/01/97	0
first date included	01/01/98	1252.24
	02/05/98	1597.87
	03/15/98	2546.35
last date included	03/31/98	5462.21

41.9.16 Calendar1stHalf, Calendar2ndHalf

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates that fall within the first or second half of the current calendar year, respectively. The first half of the calendar year includes all dates from January 1st through June 30th. The second half of the calendar year includes all dates from July 1st through December 31st.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in Calendar1stHalf Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date in the 1st half of the calendar year. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	12/10/97	0
	01/01/97	0
first date included	01/01/98	1252.24
	03/05/98	1597.87
	05/15/98	2546.35
last date included	06/30/98	5462.21

41.9.17 LastYearMTD

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values in the previous year that matches the current month to date.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in LastYearMTD Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date in the current month last year, up to the current date last year. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	03/31/97	0
first date included	04/01/97	1252.24
	04/05/97	1597.87
	04/08/97	2546.35
last date included	04/10/97	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.9.18 LastYearYTD

Basic and Crystal syntax.

Returns

None

Action

Specifies a range of Date values that includes all dates in the last year, up to the current date last year.

Examples

The following examples are applicable to Crystal syntax:

```
If {orders.ORDER DATE} in LastYearYTD Then
    {orders.ORDER AMOUNT}
Else
    0
```

Returns the Amount if the date falls on any date in the last year, up to the current date last year. The following table identifies the first and last dates included in the date range if today equals 04/10/98.

If today = 04/10/98	{orders.ORDER DATE}	{orders.ORDER AMOUNT}
not in range	12/31/96	0
first date included	01/01/97	1252.24
	02/16/97	1597.87
	03/24/97	2546.35
last date included	04/10/97	5462.21

Note

If you would like to base the date range on a date other than today's date, you must either change the date via the Date/Time option in the Windows Control Panel, or change the report print date via the Set Print Date and Time command in the Report Menu.

41.10 Arrays

Click a link to view one of the topics in this section:

- [Array \(x,...\) \[page 1820\]](#)
- [MakeArray \(x, ...\) \[page 1821\]](#)
- [UBound \(array\) \[page 1822\]](#)
- [Array summary functions \[page 1824\]](#)

41.10.1 Array (x,...)

Basic syntax.

Array and [MakeArray \(x, ...\) \[page 1821\]](#) are equivalent functions. However, Array cannot be used in Crystal syntax since it is a keyword for declaring array variables in that syntax.

Arguments

- The given arguments are array elements with which to initialize the array. The number of array elements can be any number between 1 and 1000. All the arguments must be of the same type. Unlike Visual Basic, you may not have 0 number of elements.

Returns

Array of values.

Action

The Array function creates an array, initializes it with the given array elements and returns it.

Typical use

To create and initialize an array variable in Basic syntax.

In Crystal syntax, you can use the square bracket notation for arrays instead.

Example

The following example is applicable to Basic syntax:

The following sets up an array of days for a week, starting with Monday:

```
Dim week, firstDay, lastDay
week = Array("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", _
             "Saturday", "Sunday")
Rem firstDay contains "Monday"
firstDay = week(1)
Rem lastDay contains "Sunday"
lastDay = week(7)
formula = firstDay
```

Returns "Monday".

Comments

This function is designed to work like the Visual Basic function of the same name.

41.10.2 MakeArray (x, ...)

Basic and Crystal syntax.

[Array \(x,...\) \[page 1820\]](#) and MakeArray are equivalent functions. However, Array cannot be used in Crystal syntax since it is a keyword for declaring array variables in that syntax.

Arguments

- The given arguments are array elements with which to initialize the array. The number of array elements can be any number between 1 and 1000. All the arguments must be of the same type.

Returns

Array of values.

Action

MakeArray creates an array, initializes it with the given array elements and returns it.

Typical use

To create and initialize an array variable, especially in Basic syntax.

In Crystal syntax, you can use the square bracket notation for arrays instead.

Example

The following example is applicable to Crystal syntax:

```
Local NumberVar Array x := MakeArray (1, 1000, 10, 100, 10);
```

Creates and initializes an array x with five Number elements: 1, 1000, 10, 100, 10.

41.10.3 UBound (array)

Basic and Crystal syntax.

Argument

array is an array value, expression or variable.

Returns

Number value.

Action

UBound returns a Number containing the largest available subscript for the given array.

Typical use

Commonly used to obtain the size of an array before using a looping control mechanism to systematically manipulate elements of the array.

Examples

The following examples are for both Basic and Crystal syntax:

```
Rem Basic syntax
Dim simpleArray(10) As Number
formula = UBound(simpleArray)
//Crystal syntax
Local NumberVar Array simpleArray;
Redim simpleArray[10];
UBound(simpleArray)
```

Returns 10.

```
Rem Basic syntax
Dim dateArray () As Date
dateArray = Array (CDate(#12/25/1998#), CDate(#12/24/1999#))
formula = UBound(dateArray)
//Crystal syntax
Local DateVar Array dateArray;
dateArray := [CDate(#12/25/1998#), CDate(#12/24/1999#)];
UBound (dateArray)
```

Returns 2.

Comments

- This function is designed to work like the Visual Basic function of the same name.

Note

Unlike Visual Basic, only 1 dimensional arrays are supported in Crystal Reports. Also, array subscripts start at 1 in Crystal Reports and not 0 as is the default in Visual Basic.

- UBound returns 1 for an uninitialized array variable, however, it causes an error to subscript an uninitialized array variable. The reason for this behavior is that at present Crystal Reports does not support arrays with zero elements. To work around this problem, always initialize your array variables. For example, initialize Global array variables in a formula located in the report header and local array variables within the formula itself.

41.10.4 Array summary functions

The following functions include an overload for an array:

- [Average \[page 1554\]](#)
- [Count \[page 1567\]](#)
- [DistinctCount \[page 1568\]](#)
- [Maximum \[page 1563\]](#)
- [Minimum \[page 1565\]](#)
- [PopulationStdDev \[page 1557\]](#)
- [PopulationVariance \[page 1561\]](#)
- [StdDev \[page 1555\]](#)
- [Sum \[page 1552\]](#)
- [Variance \[page 1559\]](#)

41.11 Ranges

These functions will help you determine what kind of range you are working with. They all accept a single range or range array argument, and return a Boolean result. So, for example, HasLowerBound will return True if the range you're working with has a lower limit, or False if the range does not have a lower limit.

41.11.1 GetLowerBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

x must be a single range.

Returns

Return type depends on type of the input range.

Action

Returns lower bound of the range.

Typical uses

Use when you want to get and process the lower limit value of a range.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
GetLowerBound (2 To_ 5)
```

Returns 2 (Number).

```
GetLowerBound (Is<"A3")
```

Returns empty string.

41.11.2 GetUpperBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

x must be a single range.

Returns

Return type depends on type of the input range.

Action

Returns upper bound of the range.

Typical uses

Use when you want to get and process the upper limit value of a range.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
GetUpperBound (2 To_ 5)
```

Returns 5 (Number).

```
GetUpperBound (Is<"A3")
```

Returns 'A3' (String).

41.11.3 HasLowerBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

- x can be a single range or a range array.

Returns

Boolean value

Action

HasLowerBound returns True if the range is not of the form "Is<a" or "Is<=a". On any array, HasLowerBound returns True if HasLowerBound evaluates to True on every element of the array.

Typical uses

Use when you want to see if a range of data or a range array has a lower limit.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
HasLowerBound (Is>5)
```

Returns True.

```
HasLowerBound (Is<=8)
```

Returns False.

To see an example of what you can do if you don't know the form of the range in advance, see the custom functions `cdFormatNumberRange`, `cdFormatDateRange`, and `cdFormatStringRange`. They can be found in the SAP BusinessObjects Enterprise Repository.

41.11.4 HasUpperBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

- x can be a single range or a range array.

Returns

Boolean value

Action

Returns True if the range is not of the form "Is>a" or "Is>=a". On any array, HasUpperBound returns True if HasUpperBound evaluates to True on every element of the array.

Typical uses

Use when you want to see if a range of data or a range array has an upper limit.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
HasUpperBound (Is<5)
```

Returns True.

```
HasUpperBound (Is>=7)
```

Returns False.

To see an example of what you can do if you don't know the form of the range in advance, see the custom functions `cdFormatNumberRange`, `cdFormatDateRange`, and `cdFormatStringRange`. They can be found in the SAP BusinessObjects Enterprise Repository.

41.11.5 IncludesLowerBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

- `x` can be a single range or a range array.

Returns

Boolean value

Action

Returns True for ranges of the form "a To b", "a To_ b", and "Is>=b". `IncludesLowerBound` returns True on any array if the range that is the Minimum element of the array `IncludesLowerBound`.

Typical uses

Use when you want to see if a range of data or a range array includes the lower limit value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IncludesLowerBound (2 To_ 2)
```

Returns True.

```
IncludesLowerBound (Is<10)
```

Returns False.

To see an example of what you can do if you don't know the form of the range in advance, see the custom functions `cdFormatNumberRange`, `cdFormatDateRange`, and `cdFormatStringRange`. They can be found in the BusinessObjects Enterprise Repository.

41.11.6 IncludesUpperBound (x)

This function can be used in both Basic and Crystal syntax formulas as well as in custom functions.

Arguments

- `x` can be a single range or a range array.

Returns

Boolean value

Action

Returns True for ranges of the form "a To b", "a _To b", and "Is<=a". `IncludesUpperBound` returns True on any array if the range that is the Maximum element of the array `IncludesUpperBound`.

Typical uses

Use when you want to see if a range of data or a range array includes the upper limit value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IncludesUpperBound (2 _To 3)
```

Returns True.

```
IncludesUpperBound (Is>=9)
```

Returns False.

To see an example of what you can do if you don't know the form of the range in advance, see the custom functions `cdFormatNumberRange`, `cdFormatDateRange`, and `cdFormatStringRange`. They can be found in the SAP BusinessObjects Enterprise Repository.

41.12 Type Conversion

These functions allow you to convert from one data type to another.

41.12.1 CBool (number or currency)

Basic and Crystal syntax.

Argument

A single argument that can be either a Number or Currency value or expression.

Returns

Boolean value

Action

CBool returns True if the argument is positive or negative but not 0 and returns False if the argument is 0.

Example

The following example is applicable to Basic and Crystal syntax:

```
CBool({Item.NETSALES})
```

Returns True if there is an increase or decrease in net sales; returns False if net sales is 0.

Comments

This function is designed to work like the Visual Basic function of the same name.

41.12.2 CCur

Basic and Crystal syntax.

Overloads

- CCur (number or currency)
- CCur (string)

Arguments

CCur (number or currency)	number or currency is a Number or Currency value or expression.
CCur (string)	string is a String value or expression.

Returns

Currency value

Action

CCur returns a Currency value created by converting the given Number, Currency or String type argument to a Currency value. If the conversion cannot be done, an error occurs.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CCur (200.20)
```

In Crystal syntax this is equivalent to writing \$200.20. In Basic syntax, CCur (200.20) is the way to directly create a Currency value of 200.20.

```
CCur ("200.20")
```

Returns the Currency value CCur (200.20) given the String value "\$200.20".

```
CCur ("two hundred dollars")
```

Causes an error to occur. The CCur function cannot convert the String "two hundred dollars" to a Currency value.

Comments

- This function is designed to work like the Visual Basic function of the same name.
- You can use the [IsNumeric \(str\) \[page 1711\]](#) or [NumericText \(str\) \[page 1712\]](#) function to check if the String argument can be converted to a Currency before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.
- In Crystal syntax, you can use the [To currency operator \[page 1471\]](#) operator instead of CCur when converting from a Number value to a Currency value.

41.12.3 CDbI

Basic and Crystal syntax.

CDbl and [ToNumber \[page 1714\]](#) are equivalent functions.

Overloads

- CDbI (number or currency)
- CDbI (string)

Arguments

CDBl (number or currency)	number or currency is a Number or Currency value or expression.
CDBl (string)	string is a String value or expression.

Returns

Number value

Action

CDBl returns a Number value created by converting the given Number, Currency or String type argument to a Number value. If the conversion cannot be done, an error occurs.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CDBl ({Orders.Order Amount})
```

Returns the value of the Currency field {Orders.Order Amount} converted to Number type.

```
CDBl ("120.20")
```

Returns the Number value of 120.2.

Comments

This function is designed to work like the Visual Basic function of the same name.

You can use the [IsNumeric \(str\) \[page 1711\]](#) or [NumericText \(str\) \[page 1712\]](#) function to check if the String argument can be converted to a Currency before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

41.12.4 CStr

Basic and Crystal syntax.

CStr and [ToText \[page 1716\]](#) are equivalent functions.

Overloads

- CStr (x)
- CStr (x, y)
- CStr (x, y, z)
- CStr (x, y, z, w)
- CStr (x, y, z, w, q)

Arguments

Converting . . .	Description
Boolean values	• x is a Boolean value that is converted to a String, either "True" or "False".
Number and Currency values	• x is a Number or Currency value to be converted into a text string; it can be a whole or fractional value. • y is a whole number indicating the number of decimal places to carry the value in x to (This argument is optional.). • z is a single character text string indicating the character to be used to separate thousands in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.) • w is a single character text string indicating the character to be used as a decimal separator in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.)

Converting . . .	Description
Number and Currency values (formatting)	- x is a Number or Currency value to be converted into a text string; it can be a whole or fractional value. - y is a text string used to indicate the format for displaying the value in x. See Format Strings [page 1720] for information on creating a format string. - z is a whole number indicating the number of decimal places to carry the value in x to. (This argument is optional.) - w is a single character text string indicating the character to be used to separate thousands in x. Default is the character specified in your International or Regional settings control panel. (This argument is optional.) - q is a single character text string indicating the character to be used as a decimal separator in x. The default is the character specified in your International or Regional settings control panel. (This argument is optional.)
Date values	- x is a Date value to be converted into a text string. - y is a text string that defines how the value in x is to be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating this format string. (This argument is optional.)
Time values	- x is a Time value to be converted into a text string. - y is a text string that defines how the value in x is to be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating this format string. (This argument is optional.) - z is a text string to be used as a label for A.M. (morning) hours. (This argument is optional.) - w is a text string to be used as a label for P.M. (evening) hours. (This argument is optional.)
DateTime values	- x is a DateTime value to be converted into a text string. - y is a text string of characters that indicate how the resulting text string will be formatted. See Format strings for Date, Time, and DateTime values [page 1722] for more information on creating a format string. (This argument is optional.) - z is a text string to be used as a label for A.M. (morning) hours. (This argument is optional.) - w is a text string to be used as a label for P.M. (evening) hours. (This argument is optional.)

Returns

Text String

Action

The CStr function converts Numbers, Currency, Date, Time, and DateTime values to text strings.

Typical uses

Use this function to convert a Number, Currency, Date, Time, or DateTime value to a text string to appear as text in your report (form letters, comments, and so on).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CStr ({Orders.Shipped})
```

Returns "True" if the value of {Orders.Shipped} is True.

```
CStr(123.45)
```

Returns "123.45".

```
CStr(12345.6749, 2)
```

Returns "12345.67".

```
CStr(12345.6750, 2)
```

Returns "12345.68".

```
CStr(12345.4999, 0)
```

Returns "12345".

```
CStr(12345.5000, 0)
```

Returns "12346".

```
CStr({file.AMT} * {file.QUANTITY})
```

Returns "44,890.20" where Amt = 24.45 and Quantity = 1836.

CStr is useful when you want to build a sentence by combining (concatenating) a converted number or other value with other text strings:

```
"The base price of item # " + {file.ITEM NUMBER} + " is $" + CStr({file.BASE PRICE}) + "."
```

Prints the sentence "The base price of item A1/4520/B12 is \$50.00." where the Item number is A1/4520/B12 and the Base Price is 50.00, converted to text and formatted with two decimal places.

```
CStr(CDate(1996, 11, 1), "yy MMM dd, dddd")
```

Returns "96 Nov 01, Friday".

```
CStr(CDateTime(1995,10,12,3,30,11),"HH:mm, yy MMMM ddd")
```

Returns "03:30, 95 October Thu".

```
CStr(CTime(12, 10, 10), "HH*mm*ss tt", "amStr", "pmStr")
```

Returns "12*10*10 pmStr".

Comments

The overloads of CStr that take only one argument work like the Visual Basic function of the same name.

Converting Boolean values:

- The CStr function, when used with Boolean values, is most useful for combining (concatenating) a Boolean value with other text. Otherwise, a Boolean field can be formatted to appear as True or False in your report simply by changing the format on the Boolean Tab of the Format Editor.

Converting numbers and currency values:

- If the number of decimal places is specified, this function does not truncate the number when converted to text, but rounds it to the number of decimal places specified. See [Round \[page 1528\]](#), for more information on the rounding procedure.

Converting Date, Time, and DateTime values:

- Any character, with the exception of the Date or Time format characters, can be used within the format string. For example, you may wish to use a slash to separate the different elements (month, day, year) of the date, as in "12/30/95", or you may wish to use a colon to separate the different elements (hours, minutes, seconds) of the time, as in "12:30:10".
- If you wish to use any of the above characters in the format string, they must appear in quotes. For example: `CStr(CDateTime(1995,10,12,13,20,22), "MM/dd/yy hh 'h' mm 'min' ss 'sec' tt", 'am', 'pm') = "10/12/95 01 h 20 min 22 sec pm"`

Passing optional arguments:

- Many arguments for the CStr function have been specified as optional. However, you can only leave an argument blank if all the arguments that follow it are left blank as well. In other words, you cannot leave the y and z arguments blank and provide an argument for w. It is possible, however, to leave one, two, or all of the optional arguments blank, as long as no arguments are supplied after the blank arguments. The following combinations are possible when supplying arguments to the CStr function:

- CStr (x)
- CStr (x, y)
- CStr (x, y, z)
- CStr (x, y, z, w)
- CStr (x, y, z, w, q)
- Using the "t" or "tt" format characters in a time format string provides default strings for indicating a.m. (morning) and p.m. (evening) hours. "t" produces just a single character, "a" or "p", while "tt" produces the entire string, "am" or "pm". You can pass your own custom strings for indicating am/pm strings. (See the arguments for converting Time and DateTime values with CStr in the Arguments section above.) If you do pass your own am/pm strings, the "t" and "tt" format characters will have the same effect on them (producing single character vs. multiple character strings).

41.12.5 CDate

Basic and Crystal syntax.

The [DateValue \[page 1764\]](#) and [Date \[page 1758\]](#) functions are equivalent to CDate. However, Date can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- CDate (number)
- CDate (string)
- CDate (dateTime)
- CDate (YYYY, MM, DD)

Arguments

CDate (number)	Number is a value representing the number of days starting from December 30, 1899. It can be positive or negative, and is truncated if fractional.
CDate (string)	string is a text string representing a date, example: "September 20, 1999"; many formats are supported for the string
CDate (dateTime)	dateTime is a DateTime value.

CDate (YY,MM,DD)

- year is a whole number representing a year, example: 1996.
 - month is a whole number representing a month, example: 12 for December.
 - day is a whole number representing a day of the month, example: 05.
-

Returns

Date value

Action

CDate (number) converts and returns a Date given a number which is the number of days starting from December 30, 1899.

CDate (string) converts and returns a Date given a string.

CDate (dateTime) converts and returns a Date given a DateTime value.

CDate (YYYY, MM, DD) uses the given arguments to create a Date value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CDate ("Dec. 31, 1999")
```

Returns the Date value for Dec. 31, 1999.

```
CDate (50)
```

Returns the Date value for February 18, 1900.

```
CDate (#Oct. 20, 1999 12:02pm#)
```

Returns the Date value for October 20, 1999.

```
CDate (1930, 7, 30)
```

Returns the Date value for July 30, 1930.

Comments

You can use the [IsDate \[page 1775\]](#) function to check if a String argument can be converted to a Date before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [DateSerial \(year, month, day\) \[page 1771\]](#)

41.12.6 CTime

Basic and Crystal syntax.

The [TimeValue \[page 1766\]](#) and [Time \[page 1760\]](#) functions are equivalent to CTime. However, Time can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- CTime (number)
- CTime (string)
- CTime (dateTime)
- CTime (HH, MM, SS)

Arguments

CTime (number)	• Returns a Time value given a number in units of 24 hours • number can be fractional or negative as well
CTime (string)	Returns a Time value that represents the time, given a String expression specifying a time from 0:00:00 (12:00:00 A.M.) to 23:59:59 (11:59:59 P.M.), inclusive.
CTime (dateTime)	DateTime is a DateTime value.

CTime (hour, min, sec)

- Hour is a whole number representing an hour of the day.
 - Min is a whole number representing a minute.
 - Sec is a whole number representing a second.
-

Returns

A Time value.

Action

CTime (number) converts the given number to a Time value; the given number is in units of 24 hours, can be fractional or negative as well.

CTime (string) converts and returns a time given a string that identifies a Time value.

CTime (dateTime) converts and returns a time given a DateTime value.

CTime (hh, mm, ss) converts and returns a time given the arguments of the hour, minute and seconds.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CTime (.2)
```

```
CTime (1.2)
```

Both return the same value 4:48:00 a.m.

```
CTime (-.2)
```

Returns 7:12:00 pm

```
CTime ("Jan. 6, 1999 2:30pm")
```

Returns 2:30:00 pm

```
CTime (#Feb. 24, 1999 2:40pm#)
```

Returns 2:40:00 pm

```
CTime (18, 30, 30)
```

Returns 6:30:30 pm

Comments

You can use the `IsTime` function to check if a `String` argument can be converted to a `Time` before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [TimeSerial \(hour, minute, second\) \[page 1773\]](#)
- [IsTime \[page 1774\]](#)

41.12.7 CDateTime

Basic and Crystal syntax.

The [DateTimeValue \[page 1768\]](#) and [DateTime \[page 1762\]](#) functions are equivalent to `CDateTime`. However, `DateTime` can only be used in Crystal syntax since it is a type name in Basic syntax.

Overloads

- `CDateTime (string)`
- `CDateTime (number)`
- `CDateTime (date)`
- `CDateTime (date, time)`
- `CDateTime (YYYY, MM, DD)`
- `CDateTime (YYYY, MM, DD, HH, MM, SS)`

Arguments

<code>CDateTime (date)</code>	• <code>date</code> is a <code>Date</code> value.
<code>CDateTime (date, time)</code>	• <code>date</code> is a <code>Date</code> value. • <code>time</code> is a <code>Time</code> value.
<code>CDateTime (number)</code>	• <code>number</code> represents a number of days from December 30, 1899, example: 20 represents 20 days from December 30, 1899, which is January 19, 1900. • <code>number</code> can be positive, negative or fractional.

CDateTime (string)	string represents a date and time, example: "September 15, 1999 10:45am"
CDateTime (year, month, day)	year is a whole number representing a calendar year, example: 1996.
CDateTime (year, month, day, hour, min, sec)	- month is a whole number representing a month, example: 1 for January. - day is a whole number representing a day of the month, example: 10. - hour is a whole number representing an hour of the day, example: 12. - min is a whole number representing a minute, example: 59. - sec is a whole number representing seconds, example: 30.

Returns

A DateTime value.

Action

CDateTime (string) converts the given string to a DateTime value.

CDateTime (number) converts the given number to a DateTime value, where number holds the number of days since Dec. 30, 1899.

CDateTime (date) uses the given date to create the date portion, assigns 12:00:00 AM for the time portion for the returned DateTime value.

CDateTime (date, time) uses the given Date and Time data types to create a combined DateTime value.

CDateTime (YYYY, MM, DD) uses the given arguments to create the date portion, and assigns 12:00:00 AM for the time portion for the returned DateTime value.

CDateTime (YYYY, MM, DD, HH, MM, SS) uses the given arguments to create a combined DateTime value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
CDateTime ("10/4/1999 10:20am")
```

Returns the DateTime value October 4, 1999 10:20:00 am

```
CDatetime (12.5)
```

Returns the DateTime value January 11, 1900 12:00:00 pm

```
CDatetime (-2.5)
```

Returns the DateTime value December 27, 1899 12:00:00 pm

```
CDatetime (CDate ("Dec. 25, 1999"))
```

Returns the DateTime value December 25, 1999 12:00:00 am

```
CDatetime (CDate ("November 10, 1999"), CTime("12:20am"))
```

Returns the DateTime value November 10, 1999 12:20:00 am

```
CDatetime (1945, 8, 21, 0, 0, 0)
```

Returns the DateTime value August 21, 1945 12:00:00 am

```
CDatetime (1945, 8, 21, 10, 0, 0)
```

Returns the DateTime value August 21, 1945 10:00:00 am

Comments

You can use the `IsDateTime` function to check if a String argument can be converted to a DateTime before doing the actual conversion. That way, if the conversion cannot be done, you can handle the situation appropriately.

See Also

- [IsDateTime \[page 1777\]](#)

41.13 Programming Shortcuts

These functions provide shortcut alternatives to using control structures to achieve the same results.

41.13.1 Choose (index, choice1, choice2, ...)

Arguments

- index is a Number or numeric expression that specifies the index of the choice. It should be between 1 and the number of available choices. If it is out of bounds, Choose returns a default value. (The default value returned depends on the type of the choices. For example, if the choices are of Number type, the default value is 0 and if the choices are of String type, the default value is the empty string ("").)
- choice is one of the choices to choose from. All choices must be of the same type. A choice can be any simple type (Number, Currency, String, Boolean, Date, Time or DateTime) or range type (Number Range, Currency Range, String Range, Date Range, Time Range or DateTime Range), but it may not be an array.

Returns

A value from the given list of choices. The type of the returned value is the same as the type of the choices.

Action

Choose returns a value from the list of choices based on the value of index. For example, if index is 1, it returns choice1 and if index is 2 it returns choice2 and so forth.

Typical uses

- Choose can be used instead of the If-Then-Else operators or the Select control structures in some situations. For example, if you are mapping an index value to a String value, it can be simplest to use Choose.
- One situation where Choose may be better than a control structure is when writing record selection formulas so that they can be pushed down to the database server.
See [Switch \(expr1, value1, expr2, value2, ...\) \[page 1848\]](#) function for an example of this.
See "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the Crystal Reports User's Guide for an explanation of the techniques involved.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Choose (3, "Poor", "Fair", "Good", "Excellent")
```

Returns the String "Good".

```
Choose (2, 10 To 20, 100 To 200, 60 To 70)
```

Returns the Number Range value 100 To 200.

If a company has fiscal year starting July 1 (the 3rd calendar quarter) and it wants to print the fiscal quarter of each order date:

```
Rem Basic syntax
formula = Choose (DatePart ("q", {Orders.Order Date}), _
                  "third fiscal quarter", "fourth fiscal quarter", _
                  " first fiscal quarter", "second fiscal quarter") _
//Crystal syntax
Choose (DatePart ("q", {Orders.Order Date}),
        "third fiscal quarter", "fourth fiscal quarter",
        "first fiscal quarter", "second fiscal quarter")
```

If {Orders.Order Date} is February 18, 1996, then the DatePart function call returns 1 since this date is in the first calendar quarter. Thus, the first choice value, namely the String value "third fiscal quarter" is returned by the formula.

Comments

- This function is designed to work like the Visual Basic function of the same name. A difference is that the function does not return a Variant and thus all the choices must have the same type.
- Every argument of the Choose function is evaluated before the choice is returned. Thus, you should watch out for undesirable side effects when using Choose. For example, if one of the choice expressions results in division by zero, an error will occur, even if that choice is not the choice that is returned by the Choose function.

See Also

- [IIF \(expression, truePart, falsePart\) \[page 1846\]](#) function
- [Switch \(expr1, value1, expr2, value2, ...\) \[page 1848\]](#) function

41.13.2 IIF (expression, truePart, falsePart)

Basic and Crystal syntax.

Argument

- expression is a Boolean expression.
- truePart is the value returned if expression is True. It can be any simple type (Number, Currency, String, Boolean, Date, Time or DateTime) or range type (Number Range, Currency Range, String Range, Date Range, Time Range or DateTime Range), but it may not be an array.
- falsePart is the value returned if expression is False. It must be of the same type as truePart.

Returns

truePart if expression is True and falsePart if expression is False. The type of the returned value is the same as the type of truePart and falsePart.

Action

IIF returns one of two parts, depending on the evaluation of the expression.

Typical uses

- IIF can be used as an alternative to the If/Then/Else control structure in some situations. One situation where IIF may be better than a control structure is when writing record selection formulas so that they can be pushed down to the database server. See [Switch \(expr1, value1, expr2, value2, ...\) \[page 1848\]](#) for an example. See "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the Crystal Reports User's Guide for an explanation of the techniques involved.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
IIF ({Orders.Order Amount} > 10000, "large order", "standard order")
```

Returns the String value "large order" if {Orders.Order Amount} is more than \$10,000 and the String value "standard order" otherwise.

You can insert a formula field that represents the order amount if the payment has been made already, and is 0 otherwise:

```
Rem Basic syntax
formula = IIF ({Orders.Payment Received}, {Orders.Order Amount}, 0)
//Crystal syntax
IIF ({Orders.Payment Received}, {Orders.Order Amount}, 0)
```

Comments

- This function is designed to work like the Visual Basic function of the same name. A difference is that the function does not return a Variant and thus truePart and falsePart must have the same type.
- Every argument of the IIF function is evaluated before the result is returned. Thus, you should watch out for undesirable side effects when using IIF. For example, if falsePart results in division by zero, an error will occur, even if expression is True and so truePart is returned.

See Also

- [Choose \(index, choice1, choice2, ...\) \[page 1845\]](#)
- [Switch \(expr1, value1, expr2, value2, ...\) \[page 1848\]](#)

41.13.3 Switch (expr1, value1, expr2, value2, ...)

Basic and Crystal syntax.

Arguments

- expr1, expr2, and so on are Boolean expressions.
- value1, value2, and so on are the possible values that may be returned. All the values must be of the same type. A value can be any simple type (Number, Currency, String, Boolean, Date, Time or DateTime) or range type (Number Range, Currency Range, String Range, Date Range, Time Range or DateTime Range), but it may not be an array.

Returns

One of the values in the value list value1, value2, and so on. The type of the returned value is the same as the type of the values.

Action

The argument list for Switch consists of pairs of expressions and values. Switch evaluates the expressions from left to right, and returns the value associated with the first expression to evaluate to True.

For example, if expression1 is True, the Switch returns value1. If expression1 is False and expression2 is True, then Switch returns value2. If expression1 and expression2 are False and expression3 is True, then Switch returns value3.

If all of the expressions are False, then Switch returns a default value. (The default value returned depends on the type of the values in the value list. For example, if the values are of Number type, the default value is 0 and if the values are of String type, the default value is the empty string ("").)

Typical uses

Switch can be used instead of If-Then-Else operators or Select statements (Basic syntax) and Select expressions (Crystal syntax) in some situations.

One situation where Switch may be better than a control structure is when writing record selection formulas so that they can be pushed down to the database server. See below for an example. See "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the *Crystal Reports User's Guide* for an explanation of the techniques involved.

Examples

The following examples are applicable to both Basic and Crystal syntax:

Suppose that a company wants to classify orders into "large", "medium" or "small" based on the order size:

```
Rem Basic syntax
formula = Switch ({Orders.Order Amount} > 5000, "large", _
                  {Orders.Order Amount} > 1000, "medium", _
                  True, "small")

//Crystal syntax
Switch ({Orders.Order Amount} > 5000, "large",
        {Orders.Order Amount} > 1000, "medium",
        True, "small")
```

If {Orders.Order Amount} is greater than \$5,000 then the formula returns the String value "high". Otherwise, if {Orders.Order Amount} is greater than \$1,000, the formula returns the String "medium". Otherwise, the formula returns "small".

Here is an example showing how to use the Switch function for writing efficient record selection formulas that can be pushed down to the database server. Writing this formula using If-Then-Else operators or Select statements (Basic syntax) and Select expressions (Crystal syntax) would not let it be pushed down.

See "Using enhanced record selection formulas" in the Designing Optimized Web Reports chapter of the *Crystal Reports User's Guide* for an explanation of the techniques involved.

Note

This example is in Crystal syntax since record selection formulas edited with the Formula Workshop must be in Crystal syntax only.

Sample report

A more complete version of the example, with 27 Switch conditions, is provided with the sample report named "Record Selection on DateRange.rpt."

Sample reports can be downloaded from the following link: [Direct download link for samples](#) .

When you preview the report, you supply a reference order date parameter, {?reference date} and a range condition, {?reference condition} to indicate a range of dates around the reference order date. The report is then previewed with only the records whose order dates fall in the specified range.

The entire record selection is performed on the database server. This is because the Switch function and all its arguments can be evaluated before accessing the database.

- In this sample report, the Switch function is returning a Date Range value.
- The indentation style of the code makes the Switch function call look similar to a control structure and makes reading the function easier.
- The last expression is the Boolean value True.
This Switch function call provides for an explicit default value since if none of the other expression are True, the last expression is automatically True, and so the Date Range CDate(1899, 12, 30) To CDate(1899, 12, 30) is returned.

```
//Crystal syntax record selection formula
{Orders.Order Date} in
Switch
(
    {?reference condition} = "Aged 0 to 30 days",
        ({?reference date} - 30) To {?reference date},
    {?reference condition} = "Aged 31 to 60 days",
        ({?reference date} - 60) To ({?reference date} - 31),
    {?reference condition} = "Aged 61 to 90 days",
        ({?reference date} - 90) To ({?reference date} - 61),
    True, // provide default handling and specify a valid range
        CDate(1899, 12, 30) To CDate(1899, 12, 30)
)
```

Comments

- This function is designed to work like the Visual Basic function of the same name. A difference is that the function does not return a Variant and thus all the values must have the same type.
- Every argument of the Switch function is evaluated before the result is returned. Thus, you should watch out for undesirable side effects when using Switch.
For example, if one of the values results in division by zero, an error will occur, even if that value is not the value that is returned by the Switch function.

See Also

- [Choose \(index, choice1, choice2, ...\) \[page 1845\]](#)

- [IIF \(expression, truePart, falsePart\) \[page 1846\]](#)

41.14 Evaluation Time

Evaluation Time functions can control how data is presented by controlling when it is calculated or presented.

41.14.1 BeforeReadingRecords

Basic and Crystal syntax.

Returns

None

Action

Specifies that the formula is to be evaluated before the database records are read.

Typical uses

Formulas are normally evaluated at the following times:

- If no database or group field is included in the formula, the formula is evaluated before the program reads database records.
- If a database field is included in the formula, the formula is evaluated while the program reads database records.
- If a group field, page number field, subtotal, and so on, is included in the formula, the formula is evaluated after database records are read and while the data from the records is being printed in the report.

BeforeReadingRecords forces the formula to be evaluated before the program reads database records. When this function is used in a formula, the Formula Checker returns an error message if you attempt to include elements in the formula (database fields, groups, and so on) that must be evaluated at a later time (while reading or while printing records).

Examples

The following example is applicable to Crystal syntax:

```
BeforeReadingRecords;  
ToNumber ("12345")
```

Forces the formula (which contains no database or group fields), to be evaluated before reading records.

Note

If you try to include a database field or group in this formula, you will get an error message.

41.14.2 WhileReadingRecords

Basic and Crystal syntax.

Returns

None

Action

Forces the program to evaluate the formula while it is reading database record data.

Typical uses

Formulas are normally evaluated at the following times:

- If no database or group field is included in the formula, the formula is evaluated before the program reads database records.
- If a database is included in the formula, the formula is evaluated while the program reads database records.
- If a group field, page # field, subtotal, and so on, is included in the formula, the formula is evaluated after database records are read and while the data from the records is being printed in the report.

WhileReadingRecords forces the formula to be evaluated while the program reads database records. When this function is used in a formula, the Formula Checker returns an error message if you attempt to include elements in the formula (groups, page number fields, and so on) that must be evaluated at a later time (while printing records). This function can also be used to force a formula that includes no database fields and no group fields to process while reading records instead of before reading records.

Examples

The following examples are applicable to Crystal syntax:

```
WhileReadingRecords;  
ToNumber({orders detail.QUANTITY})
```

Forces the formula (which contains a database field) to be evaluated at its normal time (while reading records).

Note

If you try to include a group in this formula, you will get an error message.

```
WhileReadingRecords;  
ToNumber ("12345")
```

Forces the formula (which contains no database fields or groups), to be evaluated later than it would normally be evaluated. In this case, it causes the formula to be evaluated while reading records instead of before reading records.

Note

If you try to include a group in this formula, you will get an error message.

41.14.3 WhilePrintingRecords

Basic and Crystal syntax.

Returns

None

Action

Forces the program to evaluate the formula while it is printing database record data.

Typical uses

Formulas are normally evaluated at the following times:

- If no database or group field is included in the formula, the formula is evaluated before the program reads database records.

- If a database is included in the formula, the formula is evaluated while the program reads database records.
- If a group field, page # field, subtotal, and so on, is included in the formula, the formula is evaluated after database records are read and while the data from the records is being printed in the report.

WhilePrintingRecords forces the formula to be evaluated while the program prints database records.

This function can also force a formula that includes no database fields and no group fields to process while printing records instead of before reading records. It will also force a formula that includes database fields to process while printing records instead of while reading records.

Examples

The following examples are applicable to Crystal syntax:

```
WhilePrintingRecords;
3* Sum ({file.QTY1}, {file.QTY2})
```

Forces the formula (which contains a group) to be evaluated at its normal time (while printing records).

```
WhilePrintingRecords;
ToNumber ("12345")
```

Forces the formula (which contains no database fields or groups), to be evaluated later than it would normally be evaluated. In this case, it causes the formula to be evaluated while printing records instead of before reading records.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.14.4 EvaluateAfter (x)

Basic and Crystal syntax.

Arguments

- x is any valid formula name.

Returns

None

Action

This function can be used to force one formula to be evaluated after another. For example, two separate formulas in your report may have the same evaluation time ([BeforeReadingRecords \[page 1851\]](#), [WhileReadingRecords \[page 1852\]](#), or [WhilePrintingRecords \[page 1853\]](#)), or they appear in the same section of the report and, therefore, are automatically evaluated while records are printing. In such cases, it is often unclear which formula is evaluated first.

In many situations, it may be unimportant which formula is evaluated first. However, some situations may require that one formula be evaluated after another. For instance, one formula may set the value of a variable while the other formula reads the value of that variable. The EvaluateAfter function can be used to force the second formula to be evaluated after the first formula has completed.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
EvaluateAfter({@SetVariable})
```

Forces the formula to wait until the @SetVariable formula has completed calculations.

41.15 Print State

These functions deal with the state of a report being previewed.

41.15.1 CountHierarchicalChildren(groupLevel)

Basic and Crystal Syntax.

Arguments

- groupLevel is a function that determines the level of an existing group.

Returns

Number

Action

Returns the number of direct child hierarchical groups for any given level of the grouping hierarchy. Grandchild (and deeper) groups are excluded from the count.

Typical uses

This function can be used to suppress hierarchical group headers or footers when no child levels exist.

Comments

The function returns -1 if hierarchical grouping is not used at the specified level or the specified group level is invalid.

This function will not count the number of detail records in any group.

41.15.2 CurrentPageOrientation

Basic and Crystal syntax.

Returns

Constant

Action

CurrentPageOrientation returns the current page orientation as a constant in a formula.

Note

CurrentPageOrientation returns crPortrait for portrait orientation and crLandscape for landscape orientation.

Typical uses

You can use this function to add conditional formatting to a report.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
If CurrentPageOrientation = crPortrait then 1440 else 2880
```

In a conditional margin formula, sets margin to one inch if orientation is portrait; sets margin to two inches if orientation is landscape.

41.15.3 DrillDownGroupLevel

Basic and Crystal syntax.

Returns

Number

Action

Returns a number that indicates the group level of the current drill down view, or 0 if it is not a drill down view.

Examples

If a report has groups on Country and Region, then DrillDownGroupLevel returns 0 in the view without drill downs. Within the drill down view for a Country, DrillDownGroupLevel returns 1, and within the drill down view of a Region, DrillDownGroupLevel returns 2.

To check if you are in a drill down view, test to see if DrillDownGroupLevel > 0 returns true.

41.15.4 GroupNumber

Basic and Crystal syntax.

Returns

Whole Number

Action

Returns the current group number.

Typical uses

You can use this function in a group selection formula to print some groups and exclude others.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
GroupNumber
```

Creates a field containing the current group number that you can place on your report. Using this formula is the equivalent of using the GroupNumber field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.15.5 GroupSelection

Basic and Crystal syntax.

Returns

The group selection formula

Action

GroupSelection inserts the group selection formula for the report in a formula that you can place on your report.

Typical uses

You can use this function any time you want to print the group selection formula on the report, or whenever you want to set something as conditional on the group selection formula.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
GroupSelection
```

Creates a field that contains the group selection formula. You can then place this field on your report. Using this formula is the equivalent of using the Group Selection Formula field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.15.6 HasValue (fld)

Basic and Crystal syntax.

Arguments

- Parameter field

Returns

Boolean Value

Action

Evaluates the parameter value passed as the argument and returns TRUE if it has a value, FALSE if it has no value.

Typical uses

HasValue() is typically used to check that an optional parameter has a value before it is evaluated by the report engine. A runtime error is generated when a formula references an optional parameter that does not have a value.

It is applicable to both discrete and range values.

Examples

The following examples are applicable to both Basic and Crystal syntax:

HasValue({sales.SalesPerson}) = produces the following results:

If {sales.SalesPerson} shows this value	HasValue({sales.SalesPerson }) returns
Jim Smith	True
123	True
(null)	False

You must decide which part of the SQL statement to leave out based on HasValue() testing. For example, the following selection formula has the effect of removing {Sales.SalesPerson} = {?SalesPerson} from the WHERE clause depending on whether the user has provided a specific value for {?SalesPerson}

```
(not HasValue({?SalesPerson}) or {Sales.SalesPerson} = {?SalesPerson})  
and {Sales.ProductName} = {?ProductName}
```

41.15.7 HierarchyLevel (GroupingLevel)

Basic and Crystal syntax.

Arguments

- GroupingLevel is a function that determines the level of an existing group. See [GroupingLevel \(fld\) \[page 1888\]](#).

Returns

Number

Action

Returns a number that indicates the hierarchy level of the current group member when you have sorted your group hierarchically.

Typical uses

If you want to use a conditional-x-position formula to determine the indentation of individual members in a hierarchical group sort, use this function to track the levels.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
HierarchyLevel (GroupingLevel ({Employee.EmployeeID}))
```

Returns the level number of each member in your hierarchy assuming that your report is grouped hierarchically on the EmployeeID field.

Comments

This function is evaluated in Pass 2.

41.15.8 InRepeatedGroupHeader

Basic and Crystal syntax.

Returns

Boolean

Action

Returns TRUE when a Group Header section is repeated on a second, third, and so on, page.

Typical uses

You can use this function to add conditional formatting to any group headers that appear when a group has broken onto a second, third, or fourth, and so on, page. For example, you may want to add the text: "cont." to the group header when it appears on subsequent pages after the first.

41.15.9 IsNull (fld)

Basic and Crystal syntax.

Arguments

- fld is any valid database, memo, or BLOB field.

Returns

Boolean Value

Action

Evaluates the field specified in the current record and returns TRUE if the field contains a null value.

Typical uses

You can use this function in a record selection formula to limit the report to records that have something other than a null value in the field specified. You can also use it to have the program take some action whenever it encounters a null value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

`IsNull({orders.ORDER AMOUNT})` = produces the following results:

If{orders.ORDER AMOUNT} shows this value	IsNull({orders.ORDER AMOUNT}) returns
200.00	False
(null)	True
100.00	False
347.12	False

`Not IsNull({orders.ORDER AMOUNT})`

When used as a record selection formula, includes in the report only those records that have something other than a null value in the {orders.ORDER AMOUNT} field.

Comments

Some databases support null data values and others do not. This function will not work if the active database(s) do not support null values. If the database(s) do support null values, the function can be used to reference the null values that get created as the result of a failed lookup while joining.

Note

- This function must be used first in a formula for it to be evaluated properly.
- Zero and blank (empty or " ") are not null values.

41.15.10 Next (fld)

Crystal syntax.

Next and [NextValue \(fld\) \[page 1866\]](#) are equivalent functions. However, you can use Next only with Crystal syntax, and [NextValue \(fld\) \[page 1866\]](#) only with Basic syntax.

Arguments

- fld is any valid database field or formula field.

Returns

A field value of the same type as the argument.

Action

Next(fld) returns the value of the specified field for the next record.

Typical uses

You can use Next to identify the first or last record in a range, to identify the starting point for a new group, or to test for duplicate values.

Examples

The following examples are applicable to Crystal syntax:

```
If Next({file.QTY})<>0 Then
    {file.QTY}/2
Else
    {file.QTY}
```

Tests the next value in the {file.QTY} field to see if it is a zero value. If it is not, it divides the value by two. If it is a zero value, it returns the value itself.

```
If Remainder(Next ({file.SERIALNUM}), 300) = 0 Then
    "End of block"
Else
    ""
```

Divides the next value in the {file.SERIALNUM} field by 300. If there is no remainder, it flags the current value as "End of block". If there is a remainder (Else) it prints no flag. (This formula divides serial numbers into blocks of 300.)

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.11 NextIsNull (fld)

Basic and Crystal syntax.

Arguments

- fld is any valid database, formula, memo or BLOB field in the report.

Returns

Boolean Value

Action

Evaluates the field specified in the next record and returns TRUE if the field contains a null value.

Typical uses

You can use this function to test for the last item in a list and to take some action when you identify that last item.

Examples

The following example is applicable to Crystal syntax:

```
If NextIsNull ({employee.EMPLOYEE ID}) Then
    "Last employee of record"
Else
    ""
```

In an employee database with no null values in the Employee ID field, flags the last employee on the list.

Comments

Some databases support null data values and others do not. This function will not work if the active database(s) do not support null values. If the database(s) do support null values, the function can be used to reference the null values that get created as the result of a failed lookup while joining.

Note

This function must be used first in a formula for it to be evaluated properly.

- Zero and blank (empty or " ") are not null values.
- Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.12 OnFirstRecord

Basic and Crystal syntax.

Returns

Boolean

Action

Returns TRUE when the current record being evaluated is the first record in the report.

41.15.13 NextValue (fld)

Basic syntax.

NextValue and [Next \(fld\) \[page 1863\]](#) are equivalent functions. However, you can use NextValue only with Basic syntax and Next only with Crystal syntax.

Argument

fld is the name of a database field.

Returns

Value of specified field; type depends on type of that field.

Action

NextValue returns the value of the next record of the specified field.

Typical use

You can use NextValue to identify the first or last record in a range, to identify the starting point for a new group, or to test for duplicate values.

Examples

The following examples are applicable to Basic syntax:

```
If NextValue ({Orders Detail.Quantity}) <> 0 Then
    formula = {Orders Detail.Quantity} / 2
Else
    formula = {Orders Detail.Quantity}
End If
```

Tests the next value in the {Orders Detail.Quantity} field to see if it is zero. If it is not, it divides the current value by two. If it is zero, it returns the current value itself.

```
If NextValue ({Orders.Order ID}) Mod 10 = 0 Then
    formula = "End of block"
Else
    formula = "More to come"
End If
```

This formula divides orders into blocks of 10. If the next Order ID is a multiple of 10, it flags the current value as "End of block". Otherwise, it prints "More to come".

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.14 OnLastRecord

Basic and Crystal syntax.

Returns

Boolean

Action

Returns TRUE when the current record being evaluated is the last record in the report.

41.15.15 PageNofM

Basic and Crystal syntax.

Returns

A text string

Action

PageNofM inserts the "Page [current page number] of [total page count]" as a field in a formula.

Typical uses

Use this function any time you want to determine the page number as compared to the total number of pages, at print time, and use that number in calculating formula results. For example, you can use this function when you want to print something on certain pages and not on others.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
PageNofM
```

Creates a field containing the current page number and the total page count, which you can place anywhere on the page. Using this formula is the equivalent of using the PageNofM field from the Insert|Special Field menu.

41.15.16 PageNumber

Basic and Crystal syntax.

Returns

Whole Number

Action

PageNumber inserts the current page number as a field in a formula.

Typical uses

Use this function any time you want to determine the page number, at print time, and use that number in calculating formula results. For example, you can use this function when you want to print something on certain pages and not on others.

Examples

The following examples are applicable to Crystal syntax:

PageNumber

Creates a field containing the current page number that you can place anywhere on the page. Using this formula is the equivalent of using the Page Number field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

```
If PageNumber > 1 Then
    PageNumber
Else
    0
```

Creates a field that prints the page number on every page but the first. By activating the Suppress if Zero check box on the Number tab (Custom Style dialog box) of the Custom Style dialog box in the Format Editor dialog box, the formula will print nothing on the first page instead of a zero.

- You do not have to have a page number field in your report to use the PageNumber function in a report formula.
- Using this function in a formula forces the formula to be evaluated at print time.
For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.15.17 Previous (fld)

Crystal syntax.

[PreviousValue \(fld\) \[page 1872\]](#) and Previous are equivalent functions. However, you can use Previous only with Crystal syntax, and PreviousValue only with Basic syntax.

Arguments

- fld is any valid database or formula field in the report.

Returns

A field value of the same type that was passed as the fld argument.

Action

Previous (fld) returns the value of the specified field in the previous record.

Typical uses

You can use Previous to identify the last record in a previous range or the last record occurring before a new range begins, or to test for duplicate values.

Examples

The following examples are applicable to Crystal syntax:

```
If Previous({file.QTY}) <> 0 Then
    {file.QTY}/2
Else
    {file.QTY}
```

Tests the previous value in the {file.QTY} field to see if it is a zero value. If it is not, it divides the value by two. If it is a zero value, it returns the value itself.

```
If Previous ({customer.CUSTOMER ID}) = {customer.CUSTOMER ID} Then
    "Repeated Value"
Else
```

```
""
```

This flags repeated values in the {customer.CUSTOMER ID} field.

```
If Remainder(Previous ({file.SERIALNUM}), 300) = 0 Then  
    "Beginning, new block"  
Else  
    ""
```

Divides the previous value in the {file.SERIALNUM} field by 300. If there is no remainder, it flags the current value as "Beginning, new block". If there is a remainder (Else) it prints no flag. (This formula divides serial numbers into blocks of 300.)

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.18 PreviousIsNull (fld)

Basic and Crystal syntax.

Arguments

- fld is any numeric, formula, memo, or BLOB field in the report.

Returns

Boolean Value

Action

Evaluates the field specified in the previous record and returns TRUE if the field contains a null value.

Typical uses

You can use this function to test for the first item in a list and to take some action when that first item is identified.

Examples

The following example is applicable to Crystal syntax:

```
If PreviousIsNull ({employee.EMPLOYEE ID}) Then
    "First employee of record"
Else
    ""
```

In an employee database with no null values in the Employee ID field, flags the first employee on the list.

Comments

Some databases support null data values and others do not. This function will not work if the active database(s) do not support null values. If the database(s) do support null values, the function can be used to reference the null values that get created as the result of a failed lookup while joining.

Note

This function must be used first in a formula for it to be evaluated properly.

- Zero and blank (empty or " ") are not null values.
- Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.19 PreviousValue (fld)

Basic syntax

PreviousValue and [Previous \(fld\) \[page 1870\]](#) are equivalent functions. However, you can use PreviousValue only with Basic syntax and Previous only with Crystal syntax

Argument

fld is the name of a database field.

Returns

Value of specified field; type depends on type of that field.

Action

PreviousValue returns the value of the previous record of the specified field.

Typical use

You can use PreviousValue to identify the last record in a previous range or the last record occurring before a new range begins, or to test for duplicate values.

Examples

The following examples are applicable to Basic syntax:

```
If PreviousValue ({Orders Detail.Quantity}) <> 0 Then
    formula = {Orders Detail.Quantity} / 2
Else
    formula = {Orders Detail.Quantity}
End If
```

Tests the previous value in the {Orders Detail.Quantity} field to see if it is a zero value. If it is not, it divides the current value by two. If it is a zero value, it returns the current value itself.

```
If PreviousValue ({Orders.Order ID}) Mod 10 = 0 Then
    formula = "Beginning of new block"
Else
    formula = "Same block"
End If
```

Divides orders into blocks of 10. If the previous Order ID is a multiple of 10, it flags the current value as "Beginning of new block". Otherwise, it prints "Same block".

```
If PreviousValue ({Customer.Customer ID}) = {Customer.Customer ID} Then
    formula = "Customer Repeated"
Else
    formula = "Customer Changed"
End If
```

This flags repeated values in the {Customer.Customer ID} field.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see [Evaluation Time \[page 1851\]](#).

41.15.20 RecordNumber

Basic and Crystal syntax.

Returns

Whole Number

Action

RecordNumber returns the current record number.

Typical uses

You can use this function in creating a record selection formula to print some records and exclude others.

Examples

The following example is applicable to both Basic and Crystal syntax:

RecordNumber

Creates a field containing the current record number that you can place on your report. Using this formula is the equivalent of using the Record Number field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

Note

Using this function in a formula forces the formula to be evaluated at print time. For more information on evaluation time considerations, see the [Evaluation Time \[page 1851\]](#).

41.15.21 RecordSelection

Basic and Crystal syntax.

Returns

The record selection formula

Action

RecordSelection inserts the record selection formula for the report in a formula that you can place on your report.

Typical uses

You can use this function any time you want to print the record selection formula on the report, or whenever you want to set something as conditional on the record selection formula.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
RecordSelection
```

Creates a field that contains the record selection formula. You can then place this field on your report. Using this formula is the equivalent of using the Record Selection Formula field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.15.22 TotalPageCount

Basic and Crystal syntax.

Returns

Whole number

Action

Passes the report and returns the total number of pages.

Typical uses

You might use this function, for example, to place the number of pages into the report header or footer.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
TotalPageCount
```

Returns 2 where your report contains a total of 2 pages.

Comments

This function will add an extra pass to the report.

41.16 Document Properties

These functions return values of attributes pertaining to a document.

41.16.1 PrintDate

Basic and Crystal syntax.

Returns

Date Value

Action

PrintDate inserts the date the report is printed as a field in a formula.

Typical uses

You can use this function any time you want to print the date of printing on the report, or to condition something in the report on the print date.

Examples

The following examples are applicable to Crystal syntax:

PrintDate

Creates a field containing the date the report is printed that you can place on your report. Using this formula is the equivalent of using the PrintDate field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

```
If PrintDate >= CDate(1992,01,01) Then
    "Please excuse the delayed report."
Else
    ""
```

Prints the message apologizing for the delay if the report prints after January 1, 1992, and prints nothing if the report prints before that date.

Comments

- PrintDate and [CurrentDate \[page 1755\]](#) are synonymous.
- The PrintDate function retrieves a value from the setting established in the report via the Set Print Date and Time command on the Report menu. By default, this value matches the settings in the Windows Date/Time control panel unless changed by the user.
- Formulas and filters that use PrintDate are re-evaluated when a report is loaded. In a report with saved data, the filter or formula is applied to the saved records. To avoid re-evaluation and to preserve the data, DataDate should be used instead of PrintDate.

41.16.2 PrintTime

Basic and Crystal syntax.

Returns

Time Value

Action

PrintTime inserts the time the report is printed as a field in a formula.

Typical uses

You can use this function any time you want to print the time of printing on the report, or to condition something in the report on the print time.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
PrintTime
```

Creates a field containing the time the report is printed that you can place on your report. Using this formula is the equivalent of using the Print Time field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

Comments

- PrintTime and [CurrentTime \[page 1756\]](#) are synonymous.

41.16.3 PrintTimeZone

Basic and Crystal syntax.

Returns

String Value

Action

PrintTimeZone inserts the time zone of the Print Date and Print Time that the report is printed in as a TimeZoneString field in a formula.

Note

For more information about TimeZoneString fields, see [ShiftDateTime \(inputDateTime, inputTimeZone, newTimeZone\) \[page 1799\]](#).

Typical uses

You can use this function any time you want to print the time zone of the printing time on the report, or to condition something in the report on the print time zone.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
PrintTimeZone
```

Creates a field containing the time zone the report is printed in that you can place on your report. Using this formula is the equivalent of using the Print Time Zone field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.4 ModificationDate

Basic and Crystal syntax.

Returns

Date Value

Action

ModificationDate inserts the date the report was last modified as a field in a formula.

Typical uses

Use this function any time you want to print the date the report was last modified, or to condition something in the report on the modification date.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ModificationDate
```

Creates a field that you can place on your report which will return the date the report was last modified. Using this formula is the equivalent of using the ModificationDate field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.5 ModificationTime

Basic and Crystal syntax.

Returns

Time Value

Action

ModificationTime inserts the time the report was last modified as a field in a formula.

Typical uses

Use this function any time you want to time the date the report was last modified, or to condition something in the report on the modification time.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ModificationTime
```

Creates a field containing the time of the report was last modified, which you can place on your report. Using this formula is the equivalent of using the ModificationTime field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.6 DataDate

Basic and Crystal syntax.

Returns

Date Value

Action

DataDate returns the date that the report was last refreshed.

Typical uses

Use DataDate any time you want to date stamp your data.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DataDate
```

Returns 1998/05/06 if the report was last refreshed on 1998/05/06.

```
DataDate
```

Returns 1973/12/06 if the report was last refreshed on 1973/12/06.

Comments

The date format is determined by the settings in the international or regional settings control panel.

41.16.7 DataTime

Basic and Crystal syntax.

Returns

Time Value

Action

DateTime returns the time that the report was last refreshed.

Typical uses

Use DateTime any time you want to time stamp your data.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateTime
```

Returns 12:45:23 if the report was last refreshed at 12:45:23.

```
DateTime
```

Returns 17:26:10 if the report was last refreshed at 17:26:10.

41.16.8 DateTimeZone

Basic and Crystal syntax.

Returns

String Value

Action

DateTimeZone inserts the time zone of the Data Date and Data Time that the report is printed in as a TimeZoneString field in a formula.

Note

For more information about TimeZoneString fields, see [ShiftDateTime \(inputDateTime, inputTimeZone, newTimeZone\) \[page 1799\]](#).

Typical uses

You can use this function any time you want to print the time zone of the data time on the report, or to condition something in the report on the data time zone.

Examples

The following example is applicable to both Basic and Crystal syntax:

`DateTimeZone`

Creates a field containing the time zone in which the data was last refreshed; you can place this field on your report. Using this formula is the equivalent of using the Data Time Zone field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.9 ReportTitle

Basic and Crystal syntax.

Returns

String

Action

ReportTitle inserts the title of the report as a field in a formula.

Typical uses

You can use this function any time you want to print the title of the report.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ReportTitle
```

Creates a field containing the title that you can place on your report. Using this formula is the equivalent of using the ReportTitle field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.10 ReportComments

Basic and Crystal syntax.

Returns

String

Action

ReportComments inserts the comments included with the report as a field in a formula.

Typical uses

You can use this function any time you want to print the comments for report entered in the Document Properties dialog box.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ReportComments
```

Creates a field containing the comments that you can place on your report. Using this formula is the equivalent of using the ReportComments field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.11 Filename

Basic and Crystal syntax.

Returns

String

Action

Filename inserts (in a formula) the file name and file path that are included with the report as a field.

Typical uses

You can use this function any time you want to print the file name for the report.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
Filename
```

Creates a field containing the filename. Using this formula is the equivalent of using the File Path and Name field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.12 FileAuthor

Basic and Crystal syntax.

Returns

String

Action

FileAuthor inserts (in a formula) the file author name that is included with the report as a field.

Typical uses

You can use this function any time you want to print the name of the file author (as specified in the Document Properties dialog box).

Examples

The following example is applicable to both Basic and Crystal syntax:

```
FileAuthor
```

Creates a field containing the name of the file author. Using this formula is the equivalent of using the File Author field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.13 FileCreationDate

Basic and Crystal syntax.

Returns

String

Action

FileCreationDate inserts (in a formula) the report creation date that is included with the report as a field.

Typical uses

You can use this function any time you want to print the date when the report was created (as shown in the Document Properties dialog box).

Examples

The following example is applicable to both Basic and Crystal syntax:

```
FileCreationDate
```

Creates a field containing the report creation date. Using this formula is the equivalent of using the File Creation Date field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.14 CurrentCEUserName

Basic and Crystal syntax.

Returns

String

Action

CurrentCEUserName inserts (in a formula) the name that was used to log on to the current session when using the [Log On to SAP BusinessObjects Enterprise](#) dialog box.

Typical uses

You can use this function any time you want to print the name of the user who is currently logged on.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
CurrentCEUserName
```

Creates a field containing the current user name. Using this formula is the equivalent of using the Current CE User Name field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.15 CurrentCEUserID

Basic and Crystal syntax.

Returns

Number

Action

CurrentCEUserID inserts (in a formula) the ID of the user logged on to the current session when using the Log On to BusinessObjects Enterprise dialog box.

Typical uses

You can use this function any time you want to print the ID of the user who is currently logged on.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
CurrentCEUserID
```

Creates a field containing the current user ID. Using this formula is the equivalent of using the Current CE User ID field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.16 GroupingLevel (fld)

Basic and Crystal syntax.

Arguments

- fld is any valid field in the report.

Returns

Number

Action

GroupingLevel determines the level of your group.

Typical uses

Use this function in conjunction with the [HierarchyLevel \(GroupingLevel\) \[page 1860\]](#) function to determine the level of members of a group that you have sorted hierarchically.

41.16.17 CurrentCEUserTimeZone

Basic and Crystal syntax.

Returns

String

Action

CurrentCEUserTimeZone inserts (in a formula) the time zone of the user logged on to the current session when using the [Log On to SAP BusinessObjects Enterprise](#) dialog box.

Typical uses

You can use this function any time you want to print the time zone of the user who is currently logged on.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
CurrentCEUserTimeZone
```

Creates a field containing the current user's time zone. Using this formula is the equivalent of using the Current CE User Time Zone field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

Comments

Time zones are represented by `TimeZoneString`. For information about `TimeZoneString`, see [ShiftDateTime \(inputDateTime, inputTimeZone, newTimeZone\) \[page 1799\]](#).

41.16.18 ContentLocale

Basic and Crystal syntax.

Returns

String

Action

`ContentLocale` inserts the current *Preferred Viewing Locale* (PVL) setting.

Typical uses

You can use this function any time you want to find the current PVL setting.

Note

`ContentLocale` is a special variable that cannot be used in record selection formulas; it can only be used in print-time formulas.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
ContentLocale
```

Creates a field containing the current PVL setting of Crystal Reports. Using this formula is the equivalent of using the Content Locale field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.16.19 SelectionLocale

Basic and Crystal syntax.

Returns

String

Action

SelectionLocale inserts the *Preferred Viewing Locale* (PVL) setting that was set the last time the data was refreshed.

Typical uses

You can use this function any time you want to verify that the current PVL matches the PVL that was used when your data was sorted.

- SelectionLocale is a constant function that you can use in the Crystal Reports formula language (including record and group selection filters). This function is updated only when your report's data is refreshed.
- When you schedule your report through BusinessObjects Enterprise, this function uses the locale of the scheduling user to query the database.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
SelectionLocale
```

Using this formula is the equivalent of using the Selection Locale field in the Special Fields (Field Explorer dialog box) folder in the Field Explorer dialog box.

41.17 Xcelsius

These functions are useful when sending multiple values to an Xcelsius SWF with Flash variables exposed.

41.17.1 ToXMLColumn(x)

Basic and Crystal syntax.

Arguments

- **Boolean Values**
x is a Boolean value that is converted to a String, either "True" or "False".
- **Number and Currency Values**
x is a Number or Currency value that is converted into a text string. It can be a whole number or a fractional value. It will be converted to a string with ten decimal places, a period (.) character as the decimal separator, and no thousands separator. No currency symbol will be added.
- **Date, Time, and DateTime Values**
x is a Date, Time, or DateTime value to be converted to an OLEDateTime value and then converted to a string.
- **String Values**
x is a string value that will not be converted.

Returns

String

Action

ToXMLColumn wraps the converted argument x in the tags <column> and </column>.

Typical uses

Use this formula function for formatting data to be sent to a flash variable defined in an Xcelsius SWF.

Examples

```
ToXMLColumn('string value')
```

Returns "<column>string value</column>".

```
ToXMLColumn(1)
```

Returns "<column>1.0000000000</column>".

```
ToXMLColumn(dateTime(2007,01,01,12,00,00))
```

Returns "<column>39083.5000000000</column>".

These functions are useful when sending multiple values to an Xcelsius SWF with Flash variables exposed. The following example constructs an array of three numbers to send to an Xcelsius SWF:

```
toXMLData( toXMLRow ( toXMLColumn (1) + toXMLColumn (2) + toXMLColumn (3) ) )
```

Returns "<data><row><column>1.0000000000</column><column>2.0000000000</column><column>3.0000000000</column></row></data>".

41.17.2 ToXMLRow(x)

Basic and Crystal syntax.

Arguments

- **Boolean Values**
x is a Boolean value that is converted to a String, either "True" or "False".
- **Number and Currency Values**
x is a Number or Currency value that is converted into a text string. It can be a whole number or a fractional value. It will be converted to a string with ten decimal places, a period (.) character as the decimal separator, and no thousands separator. No currency symbol will be added.
- **Date, Time, and DateTime Values**
x is a Date, Time, or DateTime value to be converted to an OLEDateTime value and then converted to a string.
- **String Values**
x is a string value that will not be converted.

Returns

String

Action

ToXMLRow wraps the converted argument x in the tags <row> and </row>.

Typical uses

Use this formula function for formatting data to be sent to a flash variable defined in an Xcelsius SWF.

Examples

```
ToXMLRow('string value')
```

Returns "<row>string value</row>".

```
ToXMLRow(1)
```

Returns "<row>1.0000000000</row>".

```
ToXMLRow(dateTime(2007,01,01,12,00,00))
```

Returns "<row>39083.5000000000</row>".

These functions are useful when sending multiple values to an Xcelsius SWF with Flash variables exposed. The following example constructs an array of three numbers to send to an Xcelsius SWF:

```
toXMLData( toXMLRow ( toXMLColumn (1) + toXMLColumn (2) + toXMLColumn (3) ) )
```

Returns "<data><row><column>1.0000000000</column><column>2.0000000000</column><column>3.0000000000</column></row></data>".

41.17.3 ToXMLData(x)

Basic and Crystal syntax.

Arguments

- **Boolean Values**
x is a Boolean value that is converted to a String, either “True” or “False”.
- **Number and Currency Values**
x is a Number or Currency value that is converted into a text string. It can be a whole number or a fractional value. It will be converted to a string with ten decimal places, a period (.) character as the decimal separator, and no thousands separator. No currency symbol will be added.
- **Date, Time, and DateTime Values**
x is a Date, Time, or DateTime value to be converted to an OLEDateTime value and then converted to a string.
- **String Values**
x is a string value that will not be converted.

Returns

String

Action

ToXMLData wraps the converted argument x in the tags <data> and </data>.

Typical uses

Use this formula function for formatting data to be sent to a flash variable defined in an Xcelsius SWF.

Examples

```
ToXMLData('string value')
```

Returns “<data>string value</data>”.

```
ToXMLData(1)
```

Returns “<data>1.0000000000</data>”.

```
ToXMLData(dateTime(2007,01,01,12,00,00))
```

Returns “<data>39083.5000000000</data>”.

These functions are useful when sending multiple values to an Xcelsius SWF with Flash variables exposed. The following example constructs an array of three numbers to send to an Xcelsius SWF:

```
toXMLData( toXMLRow ( toXMLColumn (1) + toXMLColumn (2) + toXMLColumn (3) ) )
```

Returns "<data><row><column>1.0000000000</column><column>2.0000000000</column><column>3.0000000000</column></row></data>".

41.18 Alerts

The functions related to Report Alerts allow you to create print time formulas that refer to the results of report alerts. Any formula that references an alert becomes a print time formula.

41.18.1 IsAlertEnabled(alertName)

Basic and Crystal syntax.

Arguments

- alertName is the name of an alert (in quotation marks) that you've already created. Locate the available alerts in the Alert Names list, which appears in the Formula Workshop's Functions tree once you've created an alert.

Returns

Boolean value indicating whether or not the alert is enabled.

→ Tip

To enable or disable an existing alert, click Create Alerts on the Report Menu. Select the desired alert and click Edit. In the Edit Alert dialog box, select or clear the Enabled option.

Examples

Crystal Syntax:

```
If IsAlertEnabled("myAlert") Then  
    AlertMessage("myAlert")
```

```
Else  
    "myAlert is disabled"
```

Basic Syntax:

```
If IsAlertEnabled("myAlert") Then  
    Formula = AlertMessage("myAlert")  
Else  
    Formula = "myAlert is disabled"  
End If
```

Use this formula to display a message that lets report users know whether or not an alert is enabled. For instance, open the sample report called Alerting.rpt. Create the formula, replacing "myAlert" with "Sales" (which is the name of the alert in the sample report). Then drag the formula field into the Report Header section. By default, the Sales alert is enabled, so its alert message will appear in the Report Header. If you disable the alert (in the Edit Alert dialog box), the message displayed in the report will change to the "disabled" message that you specified in the formula.

Comments

Any formula that references an alert becomes a print time formula.

41.18.2 IsAlertTriggered(alertName)

Basic and Crystal syntax.

Arguments

- alertName is the name of an alert (in quotation marks) that you've already created. Locate the available alerts in the Alert Names list, which appears in the Formula Workshop's Functions tree once you've created an alert.

Returns

Boolean value indicating whether or not the alert is "triggered"—that is, whether or not the report data meets the alert condition formula.

Examples

Crystal syntax:

```
If IsAlertTriggered("myAlert") Then
    AlertMessage("myAlert")
Else
    "not triggered"
```

Basic syntax:

```
If IsAlertTriggered("myAlert") Then
    Formula = AlertMessage("myAlert")
Else
    Formula = "not triggered"
End If
```

Use this formula to display messages that let report users know which records have triggered the report alert. For instance, open the sample report called Alerting.rpt. Create the formula, replacing "myAlert" with "Sales" (which is the name of the alert in the sample report). Then drag the formula field into the Details section. (You might have to resize the Details section in order to see the formula field properly.) The formula displays the alert message for records that meet the alert conditions; otherwise, the formula displays your "not triggered" message.

Comments

Any formula that references an alert becomes a print time formula.

41.18.3 AlertMessage(alertName)

Basic and Crystal syntax.

Arguments

- alertName is the name of an alert (in quotation marks) that you've already created. Locate the available alerts in the Alert Names list, which appears in the Formula Workshop's Functions tree once you've created an alert.

Returns

- Returns the alert message string for records that "trigger" the alert (that is, for records that meet the alert condition formula).

- Returns an empty string ("") for records that do not meet the alert condition formula.

Examples

Crystal syntax:

```
If IsAlertEnabled("myAlert") Then
    AlertMessage("myAlert")
Else
    "myAlert is disabled"
```

Basic syntax:

```
If IsAlertEnabled("myAlert") Then
    Formula = AlertMessage("myAlert")
Else
    Formula = "myAlert is disabled"
End If
```

In these examples, `AlertMessage` returns the message that you defined when you created the alert.

Comments

Any formula that references an alert becomes a print time formula.

41.19 Additional Functions

Click a link to view one of the topics in this section:

- [DateTo2000 \(Date, Number\) \[page 1900\]](#)
- [DTSTo2000 \(DateTimeString, Number\) \[page 1902\]](#)
- [DTSToDate \(DateTimeString\) \[page 1904\]](#)
- [DTSToSeconds \(DateTimeString\) \[page 1905\]](#)
- [DTSToTimeString \(DateTimeString\) \[page 1906\]](#)
- [ExchGetId \(address\) \[page 1907\]](#)
- [ExchGetOrganization \(address\) \[page 1908\]](#)
- [ExchGetSite \(address\) \[page 1909\]](#)
- [ExtractString \(orig, startStr, endStr\) \[page 1910\]](#)
- [EventNumber \(eventNum\) \[page 1911\]](#)
- [ExchGetPath \(Path\) \[page 1912\]](#)
- [ByteToText \(byte\) \[page 1913\]](#)
- [FRCCurrentRatio \(CurrentAssets, CurrentLiabilities\) \[page 1914\]](#)

- [FRQuickRatio \(CurrentAssets, Inventories, CurrentLiabilities\)](#) [page 1915]
- [FRDebtEquityRatio \(TotalLiabilities, TotalEquity\)](#) [page 1917]
- [FREquityVsTotalAssets \(TotalEquity, TotalAssets\)](#) [page 1918]
- [FRNetProfitMargin \(NetProfit, Sales\)](#) [page 1920]
- [FRGrossProfitMargin \(GrossProfit, Sales\)](#) [page 1921]
- [FROperatingProfitMargin \(OperatingProfit, Sales\)](#) [page 1923]
- [FRInterestCoverage \(CashFlow, InterestExpenses\)](#) [page 1924]
- [FRCashFlowVsTotalDebt \(CashFlow, TotalDebt\)](#) [page 1925]
- [FRReturnOnEquity \(NetProfit, TotalEquity\)](#) [page 1927]
- [FRReturnOnNetFixedAssets \(NetProfit, NetFixedAssets\)](#) [page 1928]
- [FRReturnOnTotalAssets \(NetProfit, TotalAssets\)](#) [page 1930]
- [FRReturnOnInvestedCapital \(NetProfit, TotalBankDebt, TotalEquity\)](#) [page 1931]
- [FRReturnOnCommonEquity \(NetProfit, PreferredDividend, CommonEquity\)](#) [page 1933]
- [FREarningsPerCommonShare \(NetProfit, PreferredDividend, NumOfCommonShare\)](#) [page 1934]
- [FRAccRecTurnover \(AccountReceivable, Sales, NumOfDays\)](#) [page 1936]
- [FRInventoryTurnover \(Inventory, Sales, NumOfDays\)](#) [page 1938]
- [FRPriceEarningsRatio \(MarketPrice, EarningsPerShare\)](#) [page 1939]
- [FRDividendYield \(Dividend, MarketPrice\)](#) [page 1940]
- [Now](#) [page 1942]
- [Picture \(string, picture\)](#) [page 1943]
- [LooksLike \(string, mask\)](#) [page 1944]
- [Soundex \(string\)](#) [page 1946]
- [DateTimeTo2000 \(dateTime, number\)](#) [page 1947]
- [DTSToDateTime \(DateTimeString\)](#) [page 1949]
- [DTSToTimeField \(DateTimeString\)](#) [page 1950]
- [DateTimeToDate \(dateTime\)](#) [page 1951]
- [DateTimeToTime \(dateTime\)](#) [page 1951]
- [DateTimeToSeconds \(dateTime\)](#) [page 1952]

41.19.1 DateTo2000 (Date, Number)

Basic and Crystal syntax.

Arguments

- **Date:** accepts only valid date fields, with either 2 digit or 4 digit years.
 - A database field defined as a date field. File|Report Options should be set to Convert Date-Time fields to Date.
 - The CDate function (CDate(yyyy, MM, dd)). A four digit year is required by this function. If two digits are entered it will assume 00xx (for example, 98 AD).

- Number: a number between 0 and 99 corresponding with the desired windowing year.

Returns

a date field with a four digit year

Action

- 2 digit years (xx)
If the Year value is greater than the windowing number, 19 is appended before the 2 digits (19xx). If the Year value is less than or equal to the windowing number, 20 is appended (20xx).
- 4 digit years (19xx)
If the last two digits in the Year value are greater than the windowing number, the Year is preserved as found in the date field (19xx). If the last two digits in the Year value are less than or equal to the windowing number, the first two digits are changed to 20 (20xx). If the first two digits in the year field are 20, the Year is preserved as found in the date field (20xx).

Note

If the year is less than or equal to 1899 and greater or equal to 100, then there will be no change to the date.

Note

Whether your database dates are interpreted as two digit years or 4 digit years (19xx) is dependent on the database driver you are using. The result of the formula should be the same in either case.

Examples

The following examples are applicable to Basic and Crystal syntax:

Here the window value is greater than the year and so it will change it to 20XX.

```
DateTo2000(CDate(1993,12,12), 99)
```

should return 2093/12/12

Here the window value is less than the year and so it will not change the year.

```
DateTo2000(CDate(1993,12,12), 92)
```

should return 1993/12/12

Other Examples

This is an example of a date that will not be affected by the function.

```
DateTo2000(CDate(1899,12,12), 99)
```

should return 1899/12/12

This is an example of a date that will not be affected by the function.

```
DateTo2000(CDate(100,12,12), 99)
```

should return 100/12/12

Some databases maintain the year of a date as a two digit field and so this will simulate that date field.

```
DateTo2000(CDate(98,12,12), 99)
```

should return 2098/12/12

```
DateTo2000(CDate(98,12,12), 97)
```

should return 1998/12/12

```
DateTo2000(CDate(9,12,12), 10)
```

should return 2009/12/12

```
DateTo2000(CDate(1,12,12), 0)
```

should return 1901/12/12

41.19.2 DTSTo2000 (DateTimeString, Number)

Basic and Crystal syntax.

Arguments

- DateTimeString: accepts only valid date fields, with either 2 digit or 4 digit years.
 - A database field defined as a string field, but containing a date.
 - A date-time string entered in the format: "yyyy/MM/dd HH:mm:ss.00" or "yy/MM/dd HH:mm:ss.00", example: "1997/04/11 12:12:12.00" or "97/04/11 12:12:12.00".
- Number: a number between 0 and 99 corresponding with the desired windowing year.

Returns

a DateTime string with a four digit year

Action

- 2 digit years (xx)
If the Year value is greater than the windowing number, 19 is appended before the 2 digits (19xx). If the Year value is less than or equal to the windowing number, 20 is appended (20xx).
- 4 digit years (19xx)
If the last two digits in the Year value are greater than the windowing number, the Year is preserved as found in the date field (19xx). If the two digits in the Year value are less than or equal to the windowing number, the first two digits are changed to 20 (20xx). If the first two digits in the year field are 20, the Year is preserved as found in the date field (20xx).

Note

If the year is less than or equal to 1899 and greater than or equal to 100, there will be no change to the date.

Examples

Note

Your computer date setting must be "yy/mm/dd" or "yyyy/mm/dd" in order for the following examples to return the correct results.

The following examples are applicable to Basic and Crystal syntax:

Here the window value is greater than the year and so it will change it to 20XX.

```
DTSTo2000(ToText(CDateTime(1988,12,12,12,12,12)), "yyyy/MM/dd HH:mm:ss"), 90)
```

should return 2088/12/12 12:12:12PM

Here the window value is less than the year and so it will not change the year.

```
DTSTo2000(ToText(CDateTime(1984,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 83)
```

should return 1984/12/12 1:02:03AM

Other Examples

These are examples of a date that will not be affected by the function.

```
DTSTo2000(ToText(CDateTime(1899,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 99)
```

should return 1899/12/12 1:02:03AM

```
DTSTo2000(ToText(CDateTime(1899,12,12)), "yyyy/MM/dd HH:mm:ss"), 99)
```

should return 1899/12/12 12:00:00PM

```
DTSTo2000(ToText(CDateTime(100,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 99)
```

should return 100/12/12 1:02:03AM

Some databases maintain the year of a date as a two digit field and so these samples will simulate that date field.

```
DTSTo2000(ToText(CDateTime(98,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 99)
```

should return 2098/12/12 1:02:03AM

```
DTSTo2000(ToText(CDateTime(98,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 97)
```

should return 1998/12/12 1:02:03AM

```
DTSTo2000(ToText(CDateTime(98,12,12)), "yyyy/MM/dd HH:mm:ss"), 97)
```

should return 1998/12/12 12:00:00PM

```
DTSTo2000(ToText(CDateTime(9,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 10)
```

should return 2009/12/12 1:02:03AM

```
DTSTo2000(ToText(CDateTime(1,12,12,1,2,3)), "yyyy/MM/dd HH:mm:ss"), 0)
```

should return 1901/12/12 1:02:03AM

41.19.3 DTSToDate (DateTimeString)

Basic and Crystal syntax.

Arguments

- DateTimeString is a string including a date and a Time value.

Returns

Date Value

Action

Evaluates the string specified and returns only the date.

Examples

```
DTSToDate ("2000/01/13 11:30:15")
```

Returns the Date value, January 13, 2000. The month must be entered as 01 rather than just 1.

Comments

The Date value returned by DTSToDate is the date at midnight. The Time value is dropped, and midnight is established as the implied time of the date. This is only relevant when performing operations with the resulting date, such as adding a Time value.

Note

For new formulas, use the [CDate \[page 1838\]](#) or [DateValue \[page 1764\]](#) functions instead of DTSToDate. The CDate and DateValue functions are able to convert more types of strings.

41.19.4 DTSToSeconds (DateTimeString)

Basic and Crystal syntax.

Arguments

- DateTimeString is a string including a Date and a Time value.

Returns

Whole Number

Action

Evaluates the string specified and converts the Time value to the number of seconds from 00:00:00 (12:00 midnight) to the specified time.

Examples

The following example is applicable to Crystal syntax:

```
DTSToSeconds("2000/01/13 11:30:15")
```

Returns 41415.

Comments

Instead of using the DTSToSeconds function, you can use:

```
DateDiff ("s", #12:00 am#, CDateTime (CDate(0,0,0), CTime (DateTimeString)))
```

The above expression is able to convert more types of strings.

41.19.5 DTSToTimeString (DateTimeString)

Basic and Crystal syntax.

Arguments

- DateTimeString is a string including a Date and a Time value.

Returns

Time String Value

Action

Evaluates the string specified and returns only the Time value in military format (00:00:00).

Examples

The following example is applicable to Crystal syntax:

```
DTSToTime("1998/10/25 08:35:30")
```

Returns 08:35:30.

41.19.6 ExchGetId (address)

Basic and Crystal syntax.

Argument

address is the address of sender/recipient (string data type).

Return

String

Action

The ExchGetID (address) function will begin by determining whether the address in X500 or X400 format. Once this has been solved, the function will then determine the ID.

- If the field is in X500 format, the function will extract the last instance of the "CN=" code (not case sensitive).
- If the field is in the X400 format, it will extract the SMTP or MS Ids.

Note

The fields must follow the address type standards for the functions to operate them.

Examples

The following examples are applicable to Crystal syntax:

```
Field = /o=Business Objects/ou=CRYSTALNT  
/cn=Configuration/cn=Servers/cn=ESPRESSO/cn=Fredt  
ExchGetID({Field})
```

Returns "Fredt".

```
Field = c=US; a= ;p=Microsoft; o=apps-wga;  
dda:smtp=James@crystal.com  
ExchGetID({Field})
```

Returns "James@crystal.com".

```
Field = c=US; a= ;p=Microsoft; o=apps-wga; dda:ms=com/  
crystal/James  
ExchGetID({Field})
```

Returns "com/crystal/James".

41.19.7 ExchGetOrganization (address)

Basic and Crystal syntax.

Arguments

- Address is the Address of the sender/recipient (string data type).

Returns

String

Action

The function will begin by determining whether the address is in X500 or X400 format. Once this has been solved, the function will then determine the Organization Name.

- If the field is in X500 format, the function will extract the last instance of the "/O=" code (not case sensitive).
- If the field is in the X400 format, it will extract the instance of the "P=" code (not case sensitive).

Note

The fields must follow the address type standards for the functions to operate on them.

Examples

The following examples are applicable to Crystal syntax:

```
Field = /o=Business Objects/ou=CRYSTALNT/  
cn=Configuration/  
cn=Servers/cn=ESPRESSO/cn=Fredt  
ExchGetOrganization({Field})
```

Returns "Business Objects".

```
Field = c=US; a= ; p=Microsoft; o=apps-wga; da:ms=com/  
Rich/James.  
ExchGetOrganization({Field})
```

Returns "Microsoft".

```
Field = c=US; a= ; p=Microsoft; o=apps-wga;  
da:smtp=James@crystal.com.  
ExchGetOrganization({Field})
```

Returns "Microsoft".

41.19.8 ExchGetSite (address)

Basic and Crystal syntax.

Argument(s)

Address is the Address of sender/recipient (string data type).

Return(s)

String

Action

The function will begin by determining whether the address is in X500 or X400 format. Once this has been solved, the function will then determine the Site Name.

- If the field is in X500 format, the function will extract the last instance of the "/OU=" code (not case sensitive).
- If the field is in the X400 format, it will extract the instance of the "O=" code (not case sensitive).

Note

The fields must follow the address type standards for the functions to operate on them.

Examples

The following examples are applicable to Crystal syntax:

```
Field = /o=Business Objects /ou=CRYSTALNT/  
cn=Configuration/cn=Servers/cn=ESPRESSO/cn=Fredt  
ExchGetSite({Field})
```

Returns "CRYSTALNT".

```
c=US;a= ;p=Microsoft;o=apps-wga;dda:smtp=James@Rich.com.  
ExchGetSite({Field})
```

Returns "apps-wga".

41.19.9 ExtractString (orig, startStr, endStr)

Basic and Crystal syntax.

Arguments

- orig: String
- startStr: String
- endStr: String

Returns

String

Action

This function will return the first occurrence (in the origin string) of a string that starts with the startStr and ends with endStr. If the endStr is not found, the string starting with startStr until the end of the string is returned.

Examples

The following examples are applicable to Crystal syntax:

```
StringVar strTest := "James and crystal";  
ExtractString (strTest, "J", "s");
```

Returns "ame".

```
StringVar strTest := "James and crystal";  
ExtractString (strTest, "", "s");
```

Returns "Jame".

41.19.10 EventNumber (eventNum)

Basic and Crystal syntax.

Arguments

- eventNum is a number value that represents an event logged by Exchange.

Returns

String

Action

The function will return the appropriate text description of the event logged, by Exchange, that matches eventNum. If there is no appropriate text description for an eventNum, the message "Unknown Event" will be returned, with the eventNum in brackets after the message.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
EventNumber (1000)
```

Returns "Local Delivery".

```
EventNumber (4)
```

Returns "Message Submission".

```
EventNumber (0)
```

Returns "Message Received".

```
EventNumber (100)
```

Returns "Unknown Event(100)".

41.19.11 ExchGetPath (Path)

Basic and Crystal syntax.

Arguments

- Path is the Address of sender/receiver (string data type).

Returns

String

Action

This function will return the container information in an address field.

- If the address type is X500, the function will return all of the information from the first instance of the "CN=" code until the last instance of the "CN=" code. If there is only one instance of the "CN=" code, the function will return NULL.
- If the address type is X400, the function will return all "OU*=" codes (residing between the "P=" and "O=" codes).

- If the address starts with "DDA:", the function will return all information after the "DDA:" code.
- If the field is blank, "UNKNOWN ADDRESS" will be returned.

Examples

The following examples are applicable to Crystal syntax:

```
Field = /o=Business Objects/ou=CRYSTALNT/  
cn=Configuration/cn=Servers/cn=ESPRESSO/cn=Fredt  
ExchGetPath ({Field})
```

Returns "cn=Configuration/cn=Servers/cn=ESPRESSO".

```
Field - c=US; a= ; p=Microsoft; ou1=James;  
ou2=RichCorp; o=apps-wga;  
dda:smtp=James@Rich.com  
ExchGetPath ({Field})
```

Returns "ou1=James; ou2=Richcorp".

```
Field = DDA:MS=CRYSTAL/JAMES/crystal  
ExchGetpath9{Field})
```

Returns "MS=CRYSTAL/JAMES/crystal".

41.19.12 ByteToText (byte)

Basic and Crystal syntax.

Arguments

- bytes is a number, which represents an amount of storage in bytes.

Returns

String

Action

This function will return a string description of the number passed in.

- If the argument is less than 1024, the result will be in bytes.
- If the argument is between 1024 and 1048576, then the result will be in kilobytes.
- Otherwise, the result will be in megabytes.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
ByteToText(1000)
```

Returns "1000 bytes".

```
ByteToText(345555)
```

Returns "337KB".

41.19.13 FRCurrentRatio (CurrentAssets, CurrentLiabilities)

Basic and Crystal syntax.

Format

```
FRCurrentRatio(CurrentAssets, CurrentLiabilities)
```

Arguments

- CurrentAssets is a Number or formula.
In general, CurrentAssets consist of cash, accounts receivable, inventories, prepaid expenses, and short term marketable investment in a company's balance sheet.
- CurrentLiabilities is a Number or formula.
In general, CurrentLiabilities consist of bank advances, accounts payable, dividends/income taxes payable, and a portion of long term loans due within one year in a company's balance sheet.

Action

FRCurrentRatio returns the ratio between CurrentAssets and CurrentLiabilities (for example, CurrentRatio = CurrentAssets / CurrentLiabilities).

Typical uses

This ratio is used to measure the short term liquidity of a company. It is a measure of how much the value of current assets exceeds the value of current liabilities. The higher the ratio, the higher the liquidity.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRCurrentRatio(5, 2.5)
```

Returns 2.

```
FRCurrentRatio(4.2, 2)
```

Returns 2.1.

```
FRCurrentRatio(1.1, 2.2)
```

Returns 0.5.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRCurrentRatio is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.14 FRQuickRatio (CurrentAssets, Inventories, CurrentLiabilities)

Basic and Crystal syntax.

Format

```
FRQuickRatio(CurrentAssets, Inventories, CurrentLiabilities)
```

Arguments

- CurrentAssets is a Number or formula. In general, CurrentAssets consists of cash, accounts receivable, inventories, prepaid expenses, and short term marketable investments in a company's balance sheet.
- Inventories is a Number or formula. In general, Inventories consists of raw materials, work in progress, and finished goods in a company's balance sheet.
- CurrentLiabilities is a Number or formula. In general, CurrentLiabilities consists of bank advances, accounts payable, dividends/income taxes payable, and a portion of long term loans due within one year in a company's balance sheet.

Action

FRQuickRatio returns the ratio between CurrentAssets (less Inventories) and CurrentLiabilities (for example, $\text{QuickRatio} = (\text{CurrentAssets} - \text{Inventories}) / \text{CurrentLiabilities}$).

Typical uses

Since it takes time to convert raw materials to finished goods, and even finished goods are not really a liquid asset, this ratio removes inventories from current assets in calculating the liquidity of the company. It is a measure of how well current liabilities are covered by cash and ready cash equivalent. The higher the ratio, the higher the liquidity.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRQuickRatio(7.2, 2.2, 2)
```

Returns 2.5.

```
FRQuickRatio(3, 1.5, 3)
```

Returns 0.5.

- None of the arguments can be negative; you may have to check for negative values before passing the arguments.

- The Formula Workshop will automatically check to see if CurrentAssets is smaller than or equal to Inventories. If CurrentAssets is smaller than Inventories, you will receive an error message.

Comments

FRQuickRatio is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.15 FRDebtEquityRatio (TotalLiabilities, TotalEquity)

Basic and Crystal syntax.

Format

```
FRDebtEquityRatio(TotalLiabilities, TotalEquity)
```

Arguments

- TotalLiabilities is a Number or formula. In general, TotalLiabilities consist of all the current and long term liabilities in a company's balance sheet. However, some analysts would only include short and long term banks debts in calculating this ratio.
- TotalEquity is a Number or formula. In general, TotalEquity consist of common stocks, preferred stocks, capital surplus, and retained earnings in a company's balance sheet.

Action

FRDebtEquityRatio returns the ratio between TotalLiabilities and TotalEquity (for example, DebtEquityRatio = TotalLiabilities / TotalEquity).

Typical uses

This ratio is used to measure the capital structure of the company. It can be used to find out the relationship of total liabilities/debt to a company's equity and can be a warning that the company's borrowing is excessive. The higher the ratio, the higher the financial risk.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRDebtEquityRatio(2.6, 1.3)
```

Returns 2.

```
FRDebtEquityRatio(1.5, 2.5)
```

Returns 0.6.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRDebtEquityRatio is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.16 FREquityVsTotalAssets (TotalEquity, TotalAssets)

Basic and Crystal syntax.

Format

```
FREquityVsTotalAssets(TotalEquity, TotalAssets)
```

Arguments

- TotalEquity is a Number or formula. In general, TotalEquity consist of common stocks, preferred stocks, capital surplus, and retained earnings in a company's balance sheet.
- TotalAssets is a Number or formula. In general, TotalAssets consist of total current assets, net fixed assets and all other assets in a company's balance sheet.

Action

FREquityVsTotalAssets returns the ratio between TotalEquity and TotalAssets (for example, $\text{EquityVsTotalAssets} = \text{TotalEquity} / \text{TotalAssets}$).

Typical uses

This ratio is used to measure the financial structure of a company. The higher the ratio, the stronger a company's capital structure and the better it is capitalized.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FREquityVsTotalAssets(2, 4)
```

Returns 0.5.

```
FREquityVsTotalAssets(2.82, 18.8)
```

Returns 0.15.

ⓘ Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

ⓘ Note

TotalEquity must be smaller or equal to TotalAssets, otherwise you will receive an error message.

Comments

FREquityVsTotalAssets is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.17 FRNetProfitMargin (NetProfit, Sales)

Basic and Crystal syntax.

Format

```
FRNetProfitMargin(NetProfit, Sales)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- Sales is a Number or formula. Sales is the total sales during the period that appears in a company's income statement.

Action

FRNetProfitMargin returns the ratio between NetProfit and Sales (for example, $\text{NetProfitMargin} = \text{NetProfit} / \text{Sales}$).

Typical uses

This ratio is used to measure how well the company is doing. The higher the result, the more profitable the company.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRNetProfitMargin(3, 50)
```

Returns 0.06.

```
FRNetProfitMargin(5.6, 100)
```

Returns 0.056.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRNetProfitMargin is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.18 FRGrossProfitMargin (GrossProfit, Sales)

Basic and Crystal syntax.

Format

```
FRGrossProfitMargin(GrossProfit, Sales)
```

Arguments

- GrossProfit is a Number or formula. In general, GrossProfit equals sales minus cost of goods sold in a company's income statement.

- Sales is a Number or formula. Sales is the total sales during the period that appears in a company's income statement.

Action

FRGrossProfitMargin returns the ratio between GrossProfit and Sales (for example, $\text{GrossProfitMargin} = \text{GrossProfitMargin} / \text{Sales}$).

Typical uses

This is a measure of how much return is received from the sales of goods prior to deducting all other expenses. The result varies from industry to industry depending on accounting treatment of cost of production that is included in cost of goods sold.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRGrossProfitMargin(3.4, 50)
```

Returns 0.068.

```
FRGrossProfitMargin(36.15, 120.5)
```

Returns 0.3.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRGrossProfitMargin is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.19 FROperatingProfitMargin (OperatingProfit, Sales)

Basic and Crystal syntax.

Format

```
FROperatingProfitMargin(OperatingProfit, Sales)
```

Arguments

- OperatingProfit is a Number or formula. In general, OperatingProfit equals gross profit less all selling, administrative and general expenses in a company's income statement.
- Sales is a Number or formula. Sales is the total sales during the period that appears in a company's income statement.

Action

FROperatingProfitMargin returns the ratio between OperatingProfit and Sales (for example, $\text{OperatingProfitMargin} = \text{OperatingProfit} / \text{Sales}$).

Typical uses

FROperatingProfitMargin is used to measure the rate of profit on sales after allowing for all the directly related expenses. It is an indication of how profitable a company's operation is. The higher the result, the higher its profitability.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FROperatingProfitMargin(3.3, 30)
```

Returns 0.11.

```
FROperatingProfitMargin(11, 16.5)
```

Returns 0.6875.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FROperatingProfitMargin is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.20 FRInterestCoverage (CashFlow, InterestExpenses)

Basic and Crystal syntax.

Format

```
FRInterestCoverage(CashFlow, InterestExpenses)
```

Arguments

- CashFlow is a Number or formula. In general, CashFlow equals the net profit of the company plus all the expense items not involving cash (such as depreciation, capital losses, and so on) minus all the profit not involving cash (such as capital gains, and so on). This information can be found in a company's income statement and statement of change in financial position.
- InterestExpenses is a Number or formula. InterestExpenses is the total interest expenses that can be found in a company's income statement.

Action

FRInterestCoverage returns the ratios between CashFlow and InterestExpenses (for example, InterestCoverage = CashFlow / InterestExpenses).

Typical uses

This ratio is used to measure the ability of a company to pay the interest charges on its debt with the cash flows generated from its operation. It is usually measured in number of times (for example, 3X, and so on). The higher the result, the higher the ability for the company to meet interest payment even in time of interest rate hike.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRInterestCoverage(120, 50)
```

Returns 2.40.

```
FRInterestCoverage(30, 100)
```

Returns 0.3.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRInterestCoverage is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.21 FRCashFlowVsTotalDebt (CashFlow, TotalDebt)

Basic and Crystal syntax.

Format

```
FRCashFlowVsTotalDebt(CashFlow, TotalDebt)
```

Arguments

- CashFlow is a Number or formula. In general, CashFlow equals the net profit of the company plus all the expense items not involving cash (such as depreciation, capital losses, and so on) and minus all the profit not involving cash (such as capital gains, and so on). This information can be found in a company's income statement and statement of change in financial position.
- TotalDebt is a Number or formula. In general, TotalDebt is the total short term and long term bank borrowings. Some analysts may even include all the short and long term liabilities as part of total debt. All this information can be found in a company's income statement.

Action

FRCashFlowVsTotalDebt returns the ratio between CashFlow and TotalDebt (for example, $\text{CashFlowVsTotalDebt} = \text{CashFlow} / \text{TotalDebt}$).

Typical uses

This ratio is used to measure the ability of a company to pay off its total debts with cash flows generated from operations. It is usually measured in number of times. The higher the result, the higher the ability for the company to pay off its debts.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRCashFlowVsTotalDebt(45, 30)
```

Returns 1.5.

```
FRCashFlowVsTotalDebt(74.4, 93)
```

Returns 0.8.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRCashFlowVstotalDebt is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.22 FRReturnOnEquity (NetProfit, TotalEquity)

Basic and Crystal syntax.

Format

```
FRReturnOnEquity(NetProfit, TotalEquity)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- TotalEquity is a Number or formula. In general, TotalEquity consist of common stocks, preferred stocks, capital surplus, and retained earnings in a company's balance sheet.

Action

FRReturnOnEquity returns the ratio between NetProfit and TotalEquity (for example, ReturnOnEquity = NetProfit / TotalEquity).

Typical uses

This ratio is used to measure the returns on the total capital employed to run the company. The higher the result, the higher the return.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRReturnOnEquity(34.5, 150)
```

Returns 0.23.

```
FRReturnOnEquity(12, 15)
```

Returns 0.8.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRReturnOnEquity is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.23 FRReturnOnNetFixedAssets (NetProfit, NetFixedAssets)

Basic and Crystal syntax.

Format

```
FRReturnOnNetFixedAssets(NetProfit, NetFixedAssets)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- NetFixedAssets is a Number or formula. NetFixedAssets equals total fixed assets minus accumulated depreciation in a company's balance sheet.

Action

FRReturnOnNetFixedAssets returns the ratio between NetProfit and NetFixedAssets (for example, $\text{ReturnOnNetFixedAssets} = \text{NetProfit} / \text{NetFixedAssets}$).

Typical uses

This ratio is used to measure the return on capital assets employed. This ratio is normally used for measuring the efficiency of manufacturing companies or servicing industries that required employment of capital assets to provide its services (airlines, and so on). The higher the ratio the better the return.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRReturnOnNetFixedAssets(34, 50)
```

Returns 0.68.

```
FRReturnOnNetFixedAssets(50.4, 42)
```

Returns 1.2.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRReturnOnNetFixedAssets is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. They can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.24 FRReturnOnTotalAssets (NetProfit, TotalAssets)

Basic and Crystal syntax.

Format

```
FRReturnOnTotalAssets(NetProfit, TotalAssets)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- TotalAssets is a Number or formula. In general, TotalAssets consist of total current assets, net fixed assets and all other assets in a company's balance sheet.

Action

FRReturnOnTotalAssets returns the ratio between NetProfit and TotalAssets (for example, $\text{ReturnOnTotalAssets} = \text{NetProfit} / \text{TotalAssets}$).

Typical uses

This measures the return on total assets employed. It is a measure of the efficiency of the company in employing its total assets. The higher the result the better is its efficiency.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRReturnOnTotalAssets(23, 100)
```

Returns 0.23.

```
FRReturnOnTotalAssets(442, 340)
```

Returns 1.3.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRReturnOnTotalAssets is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. They can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.25 FRReturnOnInvestedCapital (NetProfit, TotalBankDebt, TotalEquity)

Basic and Crystal syntax.

Format

```
FRReturnOnInvestedCapital(NetProfit, TotalBankDebts, TotalEquity)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- TotalBankDebts is a Number or formula. TotalBankDebts includes all the short and long term bank debts of a company. They can all be found in a company's balance sheet.
- TotalEquity is a Number or formula. In general, TotalEquity consists of common stocks, preferred stocks, capital surplus, and retained earnings in a company's balance sheet.

Action

FRReturnOnInvestedCapital returns the ratio between NetProfit and InvestedCapital (which is TotalBankDebts plus TotalEquity) (for example, ReturnOnInvested Capital = $\text{NetProfit} / (\text{TotalBankDebts} + \text{TotalEquity})$).

Typical uses

This ratio is used to measure the return on a company's invested capital, which includes total bank borrowing and the company's total equity. The higher the return the better.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRReturnOnInvestedCapital(23, 17, 33)
```

Returns 0.46.

```
FRReturnOnInvestedCapital(54, 23, 17)
```

Returns 1.35.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRReturnOnInvestedCapital is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. They can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.26 FRReturnOnCommonEquity (NetProfit, PreferredDividend, CommonEquity)

Basic and Crystal syntax.

Format

```
FRReturnOnCommonEquity(NetProfit, PreferredDividend, CommonEquity)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- PreferredDividend is a Number or formula. PreferredDividend is the total dividend paid out to the preferred shareholders during the accounting period being studied. This information can be found in a company's income statement and statement of change in financial position. It can also be derived in a company's balance sheet.
- CommonEquity is a Number or formula. In general, CommonEquity is the sum of all common shares, contributed surplus, and retained earnings. All of these can be found in a company's balance sheet.

Action

FRReturnOnCommonEquity returns the ratio between the net profit distributable to common shareholders (NetProfit-PreferredDividend) and CommonEquity (for example, ReturnOnCommonEquity = (NetProfit - PreferredDividend) / CommonEquity).

Typical uses

This ratio is used to measure the return on common equity employed. Note that preferred dividend is deducted prior to doing the calculation. The higher the result the higher the return.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRReturnOnCommonEquity(340, 40, 600)
```

Returns 0.5.

```
FRReturnOnCommonEquity(245, 50, 150)
```

Returns 1.3.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Note

The Formula Workshop will automatically check to see if NetProfit is smaller than PreferredDividend. If NetProfit is smaller than PreferredDividend, you will receive an error message.

Comments

FRReturnOnCommonEquity is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.27 FREarningsPerCommonShare (NetProfit, PreferredDividend, NumOfCommonShare)

Basic and Crystal syntax.

Format

```
FREarningsPerCommonShare(NetProfit, PreferredDividend, NumOfCommonShare)
```

Arguments

- NetProfit is a Number or formula. NetProfit is the amount of earnings a company earns during the year from the sale of its products or services minus all the expense items. It is the bottom line figure in a company's income statement. Some analysts may want to use the net profit before extraordinary items and/or income taxes for calculating this ratio.
- PreferredDividend is a Number or formula. PreferredDividend is the total dividend paid out to the preferred shareholders during the accounting period being studied. This information can be found in a company's income statement and statement of change in financial position. It can also be derived in a company's balance sheet.
- NumOfCommonShare is a Number or formula. In general, NumOfCommonShare is the number of common shares issued and outstanding. This information can be found in a company's balance sheet.

Action

FREarningsPerCommonShare returns the ratio between net profit distributable to common shareholders (NetProfit-PreferredDividend) and NumOfCommonShare (for example, $\text{EarningsPerCommonShare} = (\text{NetProfit} - \text{PreferredDividend}) / \text{NumOfCommonShare}$).

Typical uses

This ratio is used to calculate how much profit that a common share is entitled to. This ratio is used side by side with dividend per common share to determine the dividend pay out ratio. Some analysts may want to perform a fully diluted earnings per share. In this case, they have to take into account of all the common shares that will be converted from all the convertible preferred shares.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FREarningsPerCommonShare(550000, 50000, 100000)
```

Returns 5.

```
FREarningsPerCommonShare(450000, 10000, 2000000)
```

Returns 0.22.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Note

NetProfit cannot be smaller than PreferredDividend, otherwise you will receive an error message.

Comments

FREarningsPerCommonShare is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.28 FRAccRecTurnover (AccountReceivable, Sales, NumOfDays)

Basic and Crystal syntax.

Format

```
FRAccRecTurnover(AccountReceivable, Sales, NumOfDays)
```

Arguments

- AccountReceivable is a Number or formula. AccountReceivable is the total accounts receivable in a company's balance sheet.
- Sales is a Number or formula. Sales is the total sales during the period that appears in a company's income statement.
- NumOfDays is a Number or formula. In general, it is the number of days in a year. Some analysts use 360 and some use 365. The default is 360 for this function.

Action

FRAccRecTurnover returns the average turnover of AccountReceivable in days (for example, AccRecTurnover = (AccountReceivable / Sales) * 360).

Typical uses

This ratio measures the turnover of accounts receivable in days. A smaller ratio represents higher turnover which means that accounts receivable is paid shortly after they are incurred. A higher turnover is generally preferable over a lower turnover which might be an indication of doubtful accounts.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRAccRecTurnover(100, 600, 360)
```

Returns 60.

```
FRAccRecTurnover(200, 300, 360)
```

Returns 240

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRAccRecTurnover is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.29 FRInventoryTurnover (Inventory, Sales, NumOfDays)

Basic and Crystal syntax.

Format

```
FRInventoryTurnover(Inventory, Sales, NumOfDays)
```

Arguments

- Inventory is a Number or formula. Inventory is the total inventories in a company's balance sheet. In general, it includes all raw material, work in progress and finished goods.
- Sales is a Number or formula. Sales is the total sales during the period that appears in a company's income statement.
- NumOfDays is a Number or formula. In general, NumOfDays is the number of days in a year. Some analysts use 360 and some use 365. The default is 360 for this function.

Action

FRInventoryTurnover returns the average turnover of inventory in days (for example, $\text{InventoryTurnover} = (\text{Inventory}/\text{Sales}) * 360$).

Typical uses

This ratio measures the turnover of inventories in days. A smaller ratio represents higher turnover, which means that inventories are sold quickly and are not stuck in the warehouses. A high turnover is generally preferred over a low turnover, which might be an indication of stale or obsolete inventory.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRInventoryTurnover(40, 400, 360)
```

Returns 36.

```
FRInventoryTurnover(36, 90, 360)
```


Returns 144.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRInventoryTurnover is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.30 FRPriceEarningsRatio (MarketPrice, EarningsPerShare)

Basic and Crystal syntax.

Format

```
FRPriceEarningsRatio(MarketPrice, EarningsPerShare)
```

Arguments

- MarketPrice is a Number or formula. MarketPrice is the current market price of a company's common share. This information is publicly available if the company's shares are traded in a stock exchange.
- EarningsPerShare is a Number or formula. EarningsPerShare is the amount of earnings that is attributable to each common share. This information is available in a company's financial reports.

Action

FRPriceEarningsRatio returns the ratio between MarketPrice and EarningsPerShare (for example, $\text{PriceEarningsRatio} = \text{MarketPrice} / \text{EarningsPerShare (12 months)}$).

Typical uses

This ratio is used as a comparison between the share's market price and its earning ability. It is a convenient figure and it can be used easily to compare one company's earning ability with another.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRPriceEarningsRatio(3.5, 2)
```

Returns 1.75.

```
FRPriceEarningsRatio(27, 1.5)
```

Returns 18.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRPriceEarningsRatio is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.31 FRDividendYield (Dividend, MarketPrice)

Basic and Crystal syntax.

Format

```
FRDividendYield(Dividend, MarketPrice)
```

Arguments

- Dividend is a Number or formula. Dividend is the indicated annual dividend per common share.
- MarketPrice is a Number or formula. MarketPrice is the current market price of a company's common share. This information is publicly available if a company's shares are traded in a stock exchange.

Action

FRDividendYield returns the ratio between Dividend and MarketPrice (for example, $\text{DividendYield} = \text{Dividend} / \text{MarketPrice}$).

Typical uses

This ratio is used as an indication of the return on investment for a common share investor. It is a superficial comparison between shares of different companies. Some analysts may also want to use the same function for calculating yield for preferred share.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
FRDividendYield(3, 25)
```

Returns 0.12.

```
FRDividendYield(5, 80)
```

Returns 0.0625.

Note

None of the arguments can be negative; you may have to check for negative values before passing the arguments.

Comments

FRDividendYield is one of the many financial analysis tools used in interpreting the financial position of a company. As in the case of all ratios, it has to be used with caution. It can just be used as a clue and not solid proof for forming a judgment on the financial status of a company.

Neither one of the ratios can be used alone for doing financial analysis and there are no fixed rules on the results of the ratios. Apart from differences between type of industries, result varies among different companies within the same industry and within different account periods. The results should only be used for relative comparison and trend analysis, rather than treating them as something absolute.

41.19.32 Now

Basic and Crystal syntax.

Returns

Text String

Action

Now prints the current time on a report. The time is taken from your computer's internal clock.

Typical Uses

Use Now any time you want to time-stamp your report.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Now
```

Returns 4:45:18 if you begin printing your report at 4:45:18.

```
Now
```

Returns 17:14:33 if you begin printing your report at 17:14:33.

Comments

- The time format is determined by the settings in the International section of the Windows Control Panel.
- The [CurrentTime \[page 1756\]](#) function is identical to the Now function, which was included with previous version of the program. Now is provided for compatibility with reports created with earlier versions of the program. New reports should use the CurrentTime function.

41.19.33 Picture (string, picture)

Basic and Crystal syntax.

Arguments

- String is a text string to be formatted according to the picture format.
- Picture is a text string representing the way you want the characters in the string to be printed.

Return

string

Action

Picture (string, picture) prints a string or values in a text string in a predetermined format.

Typical Uses

Picture can be used to print a string "2065555555" in a telephone number format "(206) 555-5555", or to print other strings in fixed formats. If you are using a database field as the string, it will only work if the field is of a fixed and known length.

Note

You cannot use an asterisk as a wildcard in the picture argument. If you put an asterisk in the argument, it will show up as an asterisk in the resulting string.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
Picture ({customer.FAX}, "(xxx) xxx-xxxx" )
```

Prints the values in the Fax field in the format (805) 555-5555, (313) 555-2555, and so on.

```
Picture ({customer.REGION}, "[xx]")
```

Prints the values in the Region field in the format [AK], [CA], and so on.

The size of the string you specify in the picture parameter, that is the number of x's you use, must be greater than or equal to the largest value of the string. So, for example, if you want to print city names in square brackets, you would do the following:

```
Picture ({customer.CITY}, "[xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx]")
```

This would print [Calgary],[Hamilton],[Toronto], and so on. The picture function trims to the size of the data in each row.

📘 Note

The maximum number of x's that can be entered is 252.

Comments

- Picture must be enclosed in quotes.
- Use the character "x" in the mask to designate characters from the string.
- If the database field includes brackets or other formatting, for example (xxx) xxx-xxxx, you will not be able to convert it to xxxxxx. The parentheses and hyphen will still appear.

41.19.34 LooksLike (string, mask)

Basic and Crystal syntax.

Arguments

- string is the text string or field containing text string values that are being compared to the mask.
- mask is a text string that provides a mask for comparing the value in the string argument.

Returns

Boolean Value

Action

Enables you to locate field values using a standard DOS wildcard (? = wildcard for single character, * = wildcard for any number of characters). It does this by comparing a string to a mask that contains one or more wildcards. The function returns True if the string matches the mask and False if the string does not match the mask.

Typical uses

Use this function whenever you want to locate records based on two or more field values that are spelled differently yet look alike (have an essentially similar structure).

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
LooksLike ("Snyder", "Sn?der")
```

Returns TRUE.

```
LooksLike ("Schneider", "S?der")
```

Returns FALSE.

```
LooksLike ("Schneider", "S?????der")
```

Returns TRUE.

The following example is applicable to Crystal syntax:

```
If LooksLike ({customer.CONTACT FIRST NAME}, "?ris" Then  
    {customer.CONTACT FIRST NAME}  
Else  
    ""
```

Returns Cris, Kris, and Iris, but not Chris (assuming all of these values are in the {customer.CONTACT FIRST NAME} database).

41.19.35 Soundex (string)

Basic and Crystal syntax.

Arguments

- string is one of two or more strings that sound alike.

Returns

Text String

Action

Soundex (string) evaluates a text string and returns a four character value that symbolizes the way string sounds.

Typical uses

Use this function whenever you want to locate records based on two or more field values that are spelled differently yet sound alike.

This function can also be used to find customer names that have been misspelled.

Examples

The following examples are applicable to Crystal syntax:

```
If Soundex{customer.LAST NAME} = Soundex ("Snider") Then
    {customer.LAST NAME}
Else
    ""
```

Runs a Soundex on each value in the Last Name field and prints only those values that have a Soundex value equal to the Soundex value of the name Snider. This formula returns the names Sneider, Schneider, Snyder, and Snider if they are all in the database.

Comments

- The value string must be in quotes.
- Soundex works only with values that begin with the same letter: For example, Soundex will return the same value for Chris and Cris (C620) but not for Kris (K620)
- Soundex creates a code based on the first character in the string plus three characters based on the following:
 - Non-alpha characters (i.e. numbers and punctuation) become -1.
 - The characters a, e, i, o, u, y, h, and w are ignored (except when they are the first character in the original string).
 - The characters b, f, p, and v become 1.
 - The characters c, g, j, k, q, s, x, and z become 2.
 - The characters d and t become 3.
 - The character l becomes 4.
 - The characters m and n become 5.
 - The character r becomes 6.
- If the resulting code is only 2 or 3 characters long, Soundex uses zeros to fill out the code to four characters. For example, in the name Lauren, only the L, r, and n are translated (Lrn), so the resulting Soundex code becomes L650.
- If the resulting code is more than four characters long, all characters after the fourth character are ignored. For example, in the name Patrick, the P, t, r, c, and k can be translated (Ptrck), but the resulting Soundex code will only be four characters: P362.

41.19.36 DateTimeTo2000 (dateTime, number)

Basic and Crystal syntax.

Arguments

- **dateTime:** accepts only valid date-time fields, with either 2 digit or 4 digit years.
 - The CDateTime function (CDateTime(yyyy, MM, dd, hh, mm, ss)). A four digit year is required by this function. If two digits are entered it will assume 00xx, example: 98 AD.
- **Number:** a number between 0 and 99 corresponding with the desired windowing year.

Returns

a date-time field with a four digit year

Action

- 2 digit years (xx)
If the Year value is greater than the windowing number, 19 is appended before the 2 digits (19xx). If the Year value is less than or equal to the windowing number, 20 is appended (20xx).
- 4 digit years (19xx)
If the last two digits in the Year value are greater than the windowing number, the Year is preserved as found in the date field (19xx). If the two digits in the Year value are less than or equal to the windowing number, the first two digits are changed to 20 (20xx). If the first two digits in the year field are 20, the Year is preserved as found in the date field (20xx).

Note

If the year is less than or equal to 1899 and greater or equal to 100, then there will be no change to the date.

Examples

The following examples are applicable to Basic and Crystal syntax:

Here the window value is greater than the year and so it will change it to 20XX.

```
DateTimeTo2000(CDateTime(1998,12,12,3,2,1), 99)
```

should return 2098/12/12 3:02:01

Here the window value is less than the year and so it will not change the year.

```
DateTimeTo2000(CDateTime(1995,1,2,3,2,1), 94)
```

should return 1995/01/02 3:02:01

Other Examples

These are examples of a date that will not be affected by the function.

```
DateTimeTo2000(CDateTime(1899,12,12,5,6,7), 99)
```

should return 1899/12/12 5:06:07

```
DateTimeTo2000(CDateTime(999,12,12,5,6,7), 99)
```

should return 999/12/12 5:06:07

Some databases maintain the year of a date as a two digit field and so these samples will simulate that date field.

```
DateTimeTo2000(CDateTime (93,12,12,5,6,7), 96)
```

should return 2093/12/12 5:06:07

```
DateTimeTo2000(CDateTime (98,12,12,5,6,7), 50)
```

should return 1998/12/12 5:06:07

```
DateTimeTo2000(CDateTime (9,12,12,5,6,7), 10)
```

should return 2009/12/12 5:06:07

```
DateTimeTo2000(CDateTime (2,12,12,5,6,7), 1)
```

should return 1902/12/12 5:06:07

41.19.37 DTSToDateTime (DateTimeString)

Basic and Crystal syntax.

Arguments

- DateTimeString is a string including a date and a Time value.

Returns

DateTime value

Action

Evaluates the string specified and returns a DateTime data type.

Examples

```
DTSToDateTime ("2000/01/13 11:30:15")
```

Returns the Date value, January 13, 2000 11:30:15 am. The month must be entered as 01 rather than just 1.

Comments

For new formulas, use the [CDateTime \[page 1842\]](#) or [DateTimeValue \[page 1768\]](#) functions instead of DTSToDateTime. The CDateTime and DateTimeValue functions are able to convert more types of strings.

41.19.38 DTSToTimeField (DateTimeString)

Basic and Crystal syntax.

Arguments

- DateTimeString is a string including a Date and a Time value.

Returns

Time value

Action

Evaluates the string specified and returns a Time data type.

Examples

```
DTSToTimeField ("2000/01/13 11:30:15")
```

Returns the Time value, 11:30:15 am. The month must be entered as 01 rather than just 1.

Comments

For new formulas, use the [CTime \[page 1840\]](#) or [TimeValue \[page 1766\]](#) functions instead of DTSToTimeField. The CTime and TimeValue functions are able to convert more types of strings.

41.19.39 DateTimeToDate (dateTime)

Basic and Crystal syntax.

Arguments

- dateTime is a DateTime value.

Returns

Date value

Action

Evaluates the DateTime value specified and converts it to a Date value.

Examples

The following example is applicable to both Basic and Crystal syntax:

```
DateTimeToDate(#11/01/1998 11:15:00 AM#)
```

Returns the Date value 11/01/1998.

Comments

For new formulas, you can use the [CDate \[page 1838\]](#) or [DateValue \[page 1764\]](#) functions instead of DateTimeToDate when converting a DateTime value to a Date value. The CDate and DateValue functions are more efficient since they are not UFLs.

41.19.40 DateTimeToTime (dateTime)

Basic and Crystal syntax.

Arguments

- dateTime is a DateTime value.

Returns

Time value

Action

Evaluates the DateTime value specified and converts it to a Time value.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateTimeToTime(#11/01/1998 11:15:00 AM#)
```

Returns 11:15:00 AM.

Comments

For new formulas, you can use the [CTime \[page 1840\]](#) or [TimeValue \[page 1766\]](#) functions instead of DateTimeToTime when converting a DateTime value to a Time value. The CTime and TimeValue functions are more efficient since they are not UFLs.

41.19.41 DateTimeToSeconds (dateTime)

Basic and Crystal syntax.

Arguments

- dateTime is a DateTime value.

Returns

Time value (in seconds)

Action

Evaluates the DateTime value and returns the number of seconds that have passed between 00:00:00 (12:00 midnight) and the specified time.

Examples

The following examples are applicable to both Basic and Crystal syntax:

```
DateTimeToSeconds(#11/01/1998 11:30:15 AM#)
```

Returns 41415.

Comments

Instead of using DateTimeToSeconds (dt), you can use:

```
DateDiff ("s", #12:00 am#, CDateTime (CDate(0,0,0), CTime (dt)))
```

The above expression does not use UFLs and can be more efficient.

41.20 Functions to duplicate group fields

Use these functions whenever you want to duplicate a group field in a formula.

Note

In order to use these functions, you must have already entered a group field in your report with identical parameters to those you plan to reproduce in the formula. For example, the same field, same sort-and-group-by field, same date/Boolean condition (if applicable), or same action (average, count).

Sort and group by field = character, number, or dollar value

Average (fld, condFld)

See [Average \[page 1554\]](#)

Count (fld, condFld)

See [Count \[page 1567\]](#)

DistinctCount (fld, condFld)

See [DistinctCount \[page 1568\]](#)

Maximum (fld, condFld)

See [Maximum \[page 1563\]](#)

Minimum (fld, condFld)

See [Minimum \[page 1565\]](#)

PopulationStdDev (fld, condFld)

See [PopulationStdDev \[page 1557\]](#)

PopulationVariance (fld, condFld)

See [PopulationVariance \[page 1561\]](#)

StdDev (fld, condFld)

See [StdDev \[page 1555\]](#)

Sum (fld, condFld)

See [Sum \[page 1552\]](#)

Variance (fld, condFld)

See [Variance \[page 1559\]](#)

Sort and group by field = date or Boolean

Average (fld, condFld, cond)

See [Average \[page 1554\]](#)

Count (fld, condFld, cond)

See [Count \[page 1567\]](#)

DistinctCount (fld, condFld, cond)

See [DistinctCount \[page 1568\]](#)

Maximum (fld, condFld, cond)

See [Maximum \[page 1563\]](#)

Minimum (fld, condFld, cond)

See [Minimum \[page 1565\]](#)

PopulationStdDev (fld, condFld, cond)

See [PopulationStdDev \[page 1557\]](#)

PopulationVariance (fld, condFld, cond)

See [PopulationVariance \[page 1561\]](#)

StdDev (fld, condFld, cond)

See [StdDev \[page 1555\]](#)

Sum (fld, condFld, cond)

See [Sum \[page 1552\]](#)

Variance (fld, condFld, cond)

See [Variance \[page 1559\]](#)

41.21 Grand total functions

Use these functions whenever you want to evaluate all the values in a given field for the entire report and return a calculated value. For example, a "grand total" average or a "grand total" count.

Average (fld)

See [Average \[page 1554\]](#)

Count (fld)

See [Count \[page 1567\]](#)

DistinctCount (fld)

See [DistinctCount \[page 1568\]](#)

Maximum (fld)

See [Maximum \[page 1563\]](#)

Minimum (fld)

See [Minimum \[page 1565\]](#)

PopulationStdDev (fld)

See [PopulationStdDev \[page 1557\]](#)

PopulationVariance (fld)

See [PopulationVariance \[page 1561\]](#)

StdDev (fld)

See [StdDev \[page 1555\]](#)

Sum (fld)

See [Sum \[page 1552\]](#)

Variance (fld)

See [Variance \[page 1559\]](#)

41.22 Conditional formatting functions

Click a link to view one of the topics in this section:

- [GridRowColumnValue \(name\) \[page 1956\]](#)
- [CurrentFieldValue \[page 1958\]](#)
- [DefaultAttribute \[page 1959\]](#)
- [RGB \(red, green, blue\) \[page 1960\]](#)
- [Color \(red, green, blue\) \[page 1961\]](#)

41.22.1 GridRowColumnValue (name)

Basic and Crystal syntax.

Arguments

- name is a String value which is an Alias for Formulas. The Alias for Formulas is a name you have specified for a level of dimension in an OLAP grid or a Cross-Tab.

Returns

Value of a level of dimension in an OLAP grid or Cross-Tab, that has been specified by the argument, name. The type of the value thus depends on the type of the specified dimension.

Action

GridRowColumnValue returns the value of a specified level of dimension in an OLAP grid or Cross-Tab.

Typical uses

To format cells under a level of dimension in an OLAP grid or cross-tab, based on the value of that level of dimension. For instance, you can format the font color for cells in a level of dimension based on the value of that level of dimension. The attributes of a cell that you can format conditionally using a formula include: font, border, object positioning, and so on. They are accessible through the Format Editor.

Examples

The following example is applicable to both Crystal and Basic syntax:

Assume an OLAP grid with Products in the rows, and Regions in the columns.

'All reps'		Year 1995						
		World						
				Europe	North America			
				Austria	Canada			
'All products'			2,179,462,00	346,764,23	17,941,85	1,774,331,60	150,024,51	
	'Bicycle'			2,121,030,45	336,674,30	17,483,72	1,727,259,81	145,923,62
		Competition	1,458,914,39	218,885,20	8,919,51	1,189,996,52	96,485,17	
		Hybrid	217,815,98	39,001,70	1,538,55	175,929,24	10,258,80	
	'Accessory'			58,431,55	10,089,93	458,13	47,071,79	4,100,89
		Gloves	8,863,13	1,408,09	173,36	7,099,78	502,66	
Helmets		37,996,62	6,760,10	214,01	30,562,84	2,521,95		

July 22, 1999

Level 1 of the Regions dimension

Level 2 of the Products dimension

Columns under Level 1 of the Regions dimension

Suppose you would like to use GridRowColumnValue to conditionally format cells under Level 1 of the Regions dimension and Level 2 of the Products dimension. In particular, you want to format the font color of the total sales of individual products for all of Europe to be red, and that under North America to be blue.

1. Set up an Alias for Formulas for Level 1 of the Regions dimension to be "Continent". (You can do so using the OLAP Report Expert or Format OLAP Grid Object command.
After setting up the necessary Alias for Formulas, you can then express the formatting condition as: if GridRowColumnValue("Continent") returns "Europe", then you would like the font color of the cells for each Level 2 input of the Products dimension to be red, otherwise, since there are only two continents in this scenario, the font color should be blue.
2. Click in the OLAP grid on one of the cells representing total sales for Europe for a particular (Level 2) product.
The cells for all Level 2 inputs for Products combined with all Level 1 inputs for Regions are also selected. Formatting done to one of these cells will be carried over to all other selected cells.
3. Use the Format Field command to enter a conditional formatting formula for the selected cells.
In Crystal syntax, you can enter a formula like:

```
If GridRowColumnValue("Continent") = "Europe" Then crRed Else crBlue
```

In Basic syntax, the formula will be:

```
If GridRowColumnValue("Continent") = "Europe" Then formula = crRed Else  
formula = crBlue
```

You will see the total sales of individual products for Europe highlighted in red, and the total sales of individual products for North America highlighted in blue.

		World			
		Europe		North America	
			Austria		Canada
'All products'		2,179,462,00	346,764,23	17,941,85	1,774,331,60
	'Bicycle'	2,121,030,45	336,674,30	17,483,72	1,727,259,81
		Competition	1,458,914,39	218,885,20	8,919,51
		Hybrid	217,815,98	39,001,70	1,538,55
	'Accessory'	58,431,55	10,089,93	458,13	47,071,79
		Gloves	8,863,13	1,408,09	173,36
		Helmets	37,996,62	6,760,10	214,01
				30,562,84	2,521,95

41.22.2 CurrentFieldValue

Basic and Crystal syntax.

Returns

Can return any field type

Action

CurrentFieldValue returns the current value of the field that is about to be printed.

Typical Uses

You can use CurrentFieldValue to help do conditional formatting on a field element, field object or cross-tab summary or label.

Examples

In Crystal syntax, you can enter a formula like:

```
if CurrentFieldValue = 0 then crRed
```

In Basic syntax, the formula will be:

```
If CurrentFieldValue = 0 Then formula = crRed
```

Formats the font color of the object to print red when the value of the field is zero.

Comments

- CurrentFieldValue is only available when entering field formatting formulas.
- The return type depends on the context of the formula. For example, if you are formatting a string field, CurrentFieldValue returns a string type, if you are formatting a date field, CurrentFieldValue returns a date type.

41.22.3 DefaultAttribute

Basic and Crystal syntax.

Returns

Can return any field type

Action

The value returned by DefaultAttribute depends upon the value selected for the attribute being formatted in the attribute value interface.

Examples

DefaultAttribute can be used in conjunction with CurrentFieldValue to conditionally format fields based upon their values and default condition.

The following examples are applicable to Crystal syntax:

```
if CurrentFieldValue < 0 then DefaultAttribute else crBlack
```

In Basic syntax, the formula will be:

```
If CurrentFieldValue < 0 Then  
    formula = DefaultAttribute  
Else  
    formula = crBlack  
End If
```

Formats the font color of the object to print in the default color when the value of the field is less than zero and to print black when the value of the field is greater than zero.

Comments

DefaultAttribute is only available when entering formatting formulas.

41.22.4 RGB (red, green, blue)

RGB and [Color \(red, green, blue\) \[page 1961\]](#) are equivalent functions. However, RGB is preferred in Basic syntax and Color is preferred in Crystal syntax.

Arguments

- red is a Number value representing the red component of the resultant color, value is between 0 and 255.
- green is a Number value representing the green component of the resultant color, value is between 0 and 255.
- blue is a Number value representing the blue component of the resultant color, value is between 0 and 255.

Returns

Font color attribute.

Action

RGB returns the color attribute as specified by the red, green and blue components.

Typical use

To specify a custom color when writing a conditional formatting formula for the font or background color of a report field.

Example

The following example is applicable to Basic syntax:

```
If CurrentFieldValue > 10000 Then
    formula = RGB (150,230,150)
Else
    formula = RGB (10,19,200)
End If
```

This formula formats a Number field to display in the color as specified by RGB (150,230,150) if its current value is greater than 10000, otherwise to display it in RGB (10,19,200).

See Also

- [Color \(red, green, blue\) \[page 1961\]](#)

41.22.5 Color (red, green, blue)

Color and [RGB \(red, green, blue\) \[page 1960\]](#) equivalent functions. However, RGB is preferred in Basic syntax and Color is preferred in Crystal syntax.

Arguments

- red is a Number value representing the red component of the resultant color, value is between 0 and 255.
- green is a Number value representing the green component of the resultant color, value is between 0 and 255.
- blue is a Number value representing the blue component of the resultant color, value is between 0 and 255.

Returns

Font color attribute.

Action

Color returns the color attribute as specified by the red, green and blue components.

Typical use

To specify a custom color when writing a conditional formatting formula for the font or background color of a report field.

Example

The following example is applicable to Crystal syntax:

```
if CurrentFieldValue > 10000 then
    Color (150,230,150)
else
    Color (10,19,200)
```

This formula formats a Number field to display in the color as specified by Color (150,230,150) if its current value is greater than 10000, otherwise to display it in Color (10,19,200).

See Also

- [RGB \(red, green, blue\) \[page 1960\]](#)

41.23 SQL Expression Builder additional functions

The SQL Expression Builder uses a standard set of ODBC-compliant functions. The following lists of functions are SQL Extension server functions that are specific to Teradata.

For detailed information on how to use the functions, refer to the Teradata SQL Extension documentation.

41.23.1 Attribute functions

Details on how to use the following attribute functions can be found in the Teradata reference documentation:

- BYTES
- CHARACTER_LENGTH

- FORMAT
- OCTET_LENGTH
- TITLE
- TYPE

41.23.2 CASE Expressions

Details on how to use the following CASE expressions can be found in the Teradata reference documentation:

- Valued CASE expressions
- Searched CASE expressions
- COALESCE
- NULLIF

41.23.3 Date/Time functions

Details on how to use the following date/time functions can be found in the Teradata reference documentation:

- ADD_MONTHS
- EXTRACT

41.23.4 Hash-related functions

Details on how to use the following hash-related functions can be found in the Teradata reference documentation:

- HASHAMP
- HASHBAKAMP
- HASHBUCKET
- HASHROW

41.23.5 Numeric functions

Details on how to use the following numeric functions can be found in the Teradata reference documentation:

- ACOS
- ACOSH
- ASIN
- ASINH

- ATAN2
- ATANH
- CASE_N
- COSH
- LN
- NULLIFZERO
- RANDOM
- RANGE_N
- SINH
- TANH
- WIDTH_BUCKET
- ZEROIFNULL

41.23.6 Logical Predicates

Details on how to use the following logical predicates can be found in the Teradata reference documentation:

- ALL
- ANY
- BETWEEN/NOT BETWEEN
- EXISTS/NOT EXISTS
- IN/NOT IN
- IS NULL/IS NOT NULL
- LIKE
- SOME
- OVERLAPS

41.23.7 String functions

Details on how to use the following string functions can be found in the Teradata reference documentation:

- CHAR2HEXINT
- LOWER
- POSITION
- SOUNDEX
- SUBSTRING/SUBSTR
- TRANSLATE
- TRANSLATE_CHK
- TRIM
- UPPER
- VARGRAPHIC

41.23.8 Built-in Functions

Details on how to use the following built-in functions can be found in the Teradata reference documentation:

- ACCOUNT
- CURRENT_DATE
- CURRENT_TIME
- CURRENT_TIMESTAMP
- DATABASE
- PROFILE
- ROLE
- SESSION
- USER

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.

About the icons:

- Links with the icon  : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
 - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
 - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon  : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

© 2025 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see <https://www.sap.com/about/legal/trademark.html> for additional trademark information and notices.

